非均匀有理 B 样条

非均匀有理 B 样条

Edited By

Les Piegl & Wayne Tiller

US

L&W

1996

PUBLISHER

直到最近，B 样条曲线和曲面 (NURBS) 主要引起计算机辅助设计社区的关注，在该领域它们已成为曲线和曲面描述的标准。如今，我们看到 NURBS 在视觉艺术对象建模中的应用不断扩大，包括电影和娱乐行业、艺术和雕塑。NURBS 现在也被用于虚拟现实应用的场景建模。这些应用预计将会增加。因此，《NURBS 书籍》成为视觉传播系列专著的一部分是非常合适的。

B 样条曲线和曲面在我的职业生涯中一直是一个持久的元素。1972 年出版的《计算机图形学的数学元素》第一版是第一本包含 B 样条材料的计算机辅助设计/交互式计算机图形教材。这些材料是通过比尔 • 戈登和路易 • 纳普在锡拉丘兹大学时的好意获得的。我在 1977 年夏季于海军建筑师和海洋工程师协会会议上展示的一篇论文，可以说是第一篇研究 B 样条曲线在船舶设计中应用的论文。

对许多人来说，B 样条、理性 B 样条和 NURBS 一直有些神秘。因此，在过去的几年里，迫切需要一本关于 B 样条和理性 B 样条的全面、详细、清晰易懂的书籍。因此，我怀着相当大的期待等待 Les Piegl 和 Wayne Tiller 的书籍。我没有失望: 他们通过《NURBS 书》优雅而充分地满足了这一需求。在为书籍开发材料时，他们利用自己在 NURBS 方面的丰富学术和工业经验，以直接的方式呈现这一相对复杂的主题: 他们的呈现风格清晰且详细。必要的数学内容以相当关注细节和足够的严谨性呈现。算法 (其中许多是类似 C 的伪代码) 经过深思熟虑，准备得非常细致。为了准确起见，书中的每一幅插图都是计算机生成的——这是一项巨大的任务。他们创造了一本具有持久价值的书籍。

B 样条曲线和曲面源于皮埃尔 • 贝济耶在 1970 年代初的开创性工作。也许可以将 B 样条曲线和曲面视为贝济耶曲线和曲面的“孩子”，而非均匀理性 B 样条 (NURBS) 则是“孙子”。时机恰到好处；它们无疑已经成熟。

最后，适当承认我与 Les Piegl 和 Wayne Tiller 合作使这个项目得以实现的愉悦。

大卫 • F • 罗杰斯

非均匀有理 B 样条，通常称为 NURBS，已成为计算机处理几何信息的表示、设计和数据交换的事实行业标准。许多国家和国际标准，如 IGES、STEP 和 PHIGS，承认 NURBS 是几何设计的强大工具。NURBS 取得巨大成功的原因主要在于

- NURBS 为表示解析形状 (如圆锥曲线和二次曲面) 以及自由形状实体 (如汽车车身和船体) 提供了统一的数学基础；
- 使用 NURBS 进行设计是直观的；几乎每个工具和算法都有易于理解的几何解释；
- NURBS 算法快速且数值稳定；
- NURBS 曲线和曲面在常见几何变换下是不变的，例如平移、旋转、平行投影和透视投影；
- NURBS 是非有理 B 样条和有理及非有理贝塞尔曲线和曲面的推广。

优秀的数学和算法特性，加上成功的工业应用，促进了 NURBS 的巨大普及。NURBS 在 CAD/CAM/CAE 领域的作用类似于英语在科学和商业中的地位: “想谈生意？学会说 NURBS”。

本书的目的基本上有两个: 填补自七十年代初以来文献中存在的巨大空白，并提供关于 NURBS 各个方面的全面参考。关于 NURBS 的文献稀少且分散，现有论文主要涉及样条的数学，这相当复杂，需要对样条理论有详细的理解。本书旨在面向具有扎实基础的普通工程师，他们具备初级大学数学和计算的背景。理解这些概念和实现书中介绍的数百个算法并不需要博士学位。

在撰写本书的四年中，我们

- 调查了现有文献并呈现了重要结果；
- 继续我们的 NURBS 研究，并纳入了最新的发展；实际上，本书大约一半的内容是过去几年开发的新材料；x 前言
- 开发了一个全面的 NURBS 库，称为 Nlib V1.0、V2.0。该库是超过 20 人年 NURBS 研究和开发经验的结果，结合了我们在之前设计的系统中应用的新软件实践和经过验证的软件实践；
- 测试了每一个公式和算法，并呈现了使用 Nlib 例程精确计算的图形插图。本书不包含任何手绘图形；每个图形都是精确计算的，因此是准确的。

我们很高兴向读者展示所有的成就:(1) 本书作为全面的参考，(2)Nlib 源代码 (请参见本卷第 639 页以订购)，以及 (3) 向采用本书教授 NURBS 课程的教师提供的插图。为了让读者理解这本参考书所付出的巨大工作量，我们提供了一些数据。为了生成图形插图并构建 Nlib，我们编写了确切的 (不计算数百个测试程序)

- 1,524 programs, that required
- 15,001,600 字节的存储，这大致相当于
- 350,000 行代码。

这可不是轻松的工作！

几年前，一些研究人员开玩笑说 NURBS 这个缩写实际上代表的是“没有人理解有理 B 样条”。我们承认我们的同事是对的。在过去的四年里，我们在很大程度上受到这种解释的影响，并试图以直观的方式呈现书中的材料。我们希望这有助于将缩写 NURBS 改为 EURBS，即“每个人都理解有理 B 样条”。我们欢迎读者对我们的工作提出意见和可能的改进建议。

我们很高兴能感谢许多人和组织的帮助与支持。首先，我们要感谢我们的配偶，Karen Piegl 和 LaVella Tiller，感谢他们的耐心、支持和爱。我们特别感谢 NAR Associates 的 Nancy Rogers 提供的精美排版工作，以及 David Rogers 在编辑和技术讨论中所做的贡献，这些讨论促成了手稿的许多改进。我们还感谢 Jim Oliver 和 Tim Strotman 提出的许多建议和技术交流，这些帮助使本书形成了现在的样子。Tiller 还感谢许多过去和现在在行业中的同事，他们多年来提供了鼓舞人心的讨论、宝贵的见解、支持和同事间的友谊: 他们知道自己是谁。Piegl 的研究部分得到了国家科学基金会的资助，资助编号 CCR-9217768，授予南佛罗里达大学，以及来自佛罗里达高科技和工业委员会的各种资助。

Les Piegl
Wayne Tiller
1996 年 7 月

古人之观于天地、山川、草木、虫鱼、鸟兽，往往有得，以其求思之深而无不在也。夫夷以近，则游者众；险以远，则至者少。而世之奇伟、瑰怪，非常之观，常在于险远，而人之所罕至焉，故非有志者不能至也。有志矣，不随以止也，然力不足者，亦不能至也。有志与力，而又不随以怠，至于幽暗昏惑而无物以相之，亦不能至也。然力足以至焉，于人为可讥，而在己为有悔；尽吾志也而不能至者，可以无悔矣。

王安石

北宋

人最宝贵的是生命。生命对于每个人来说只有一次。人的一生应当这样度过：当他回忆往事的时候，不会因为虚度年华而悔恨，也不会因为碌碌无为而羞愧。这样，在临死的时候，他就可以说：“我的整个生命和全部精力，都已经献给了世界上最壮丽的事业—为人类的解放而斗争。”

奥斯特洛夫斯基

前苏联

目录

目录 **3**

第一章 曲线和曲面基础 **5**

1 隐式和参数形式 . . . 5
2 曲线的幂基形式 . . . 6
3 Bézier 曲线 . . . 8
4 有理 Bézier 曲线 . . . 19
5 张量积曲面 . . . 24

第二章 B 样条基函数距离空间 **31**

1 引言 . . . 31
2 B 样条基函数的定义和性质 . . . 32
3 B 样条基函数的导数 . . . 36
4 基函数的进一步性质 . . . 38
5 计算算法 . . . 40

第三章 B 样条曲线和曲面 **47**

1 引言 . . . 47
2 B 样条曲线的定义和性质 . . . 47
3 B 样条曲线的导数 . . . 53
4 B 样条曲面的定义与性质 . . . 60
5 B 样条曲面的导数 . . . 66

第四章 有理 B 样条曲线与曲面 **70**

1 引言 . . . 70
2 NURBS 曲线的定义与性质 . . . 70
3 NURBS 曲线的导数 . . . 75
4 NURBS 曲面的定义与性质 . . . 76
5 NURBS 曲面的导数 . . . 81

第五章 基本几何算法 **83**

1 引言 . . . 83
2 节点插入 . . . 83
3 节点细化 . . . 96
4 节点移除 . . . 108
5 次数提升 . . . 112
6 次数降低 . . . 131

第六章 高级几何算法 **144**

1 曲线和曲面的点反演与投影 . . . 144
2 曲面切向量反演 . . . 146
3 曲线和曲面的变换与投影 . . . 147
4 NURBS 曲线和曲面的重参数化 . . . 149
5 曲线和曲面的反转 . . . 161
6 B-spline 和分段幂基形式之间的转换 . . . 162

第七章 圆锥曲线和圆 **172**

1 引言 . . . 172
2 表示圆锥曲线的各种形式 . . . 172
3 二次有理贝塞尔弧 . . . 177
4 无限控制点 . . . 179
5 圆的构造 . . . 180
6 圆锥曲线的构造 . . . 188
7 锥形分类与形式转换 . . . 193
8 高阶圆 . . . 196

第八章 常见曲面的构造 **199**

1 引言 . . . 200
2 双线性曲面 . . . 200
3 一般圆柱体 . . . 200
4 规则曲面 . . . 203
5 旋转曲面 . . . 206
6 曲面的非均匀缩放 . . . 212
7 三边球面 . . . 214

第九章 曲线和曲面拟合 **220**

1 引言 . . . 220
2 全局插值 . . . 220
2.1 针对点数据的全局曲线插值 . . . 220
2.2 指定端导数的全局曲线插值 . . . 223
2.3 三次样条曲线插值 . . . 223
2.4 指定一阶导数的全局曲线插值 . . . 225
2.5 全局曲面插值 . . . 226
3 局部插值 . . . 230
3.1 局部曲线插值初步 . . . 230
3.2 局部抛物线曲线插值 . . . 236
3.3 局部有理二次曲线插值 . . . 238
3.4 局部三次曲线插值 . . . 241
3.5 局部双三次曲面插值 . . . 243
4 全局逼近 . . . 246
4.1 最小二乘曲线逼近 . . . 250
4.2 加权和约束最小二乘曲线拟合 . . . 251
4.3 最小二乘曲面逼近 . . . 254
4.4 在指定精度内的逼近 . . . 257
5 局部逼近 . . . 266
5.1 局部有理二次曲线逼近 . . . 266
5.2 局部非有理三次曲线逼近 . . . 268

第十章 高级表面构造技术 **276**

1 引言 . . . 277
2 摆动表面 . . . 277
3 皮肤表面 . . . 278
4 扫掠表面 . . . 289
5 双向曲线网络的插值 . . . 299
6 库恩斯表面 . . . 307

第十一章 形状修改工具 **317**

1 引言 . . . 317
2 控制点重新定位 . . . 317
3 权重修改 . . . 321
3.1 单曲线权重的修改 . . . 322
3.2 两个相邻曲线权重的修改 . . . 324
3.3 单表面权重的修改 . . . 327
4 形状操作符 . . . 328
4.1 变形 . . . 328
4.2 扁平化 . . . 334
4.3 弯曲 . . . 338
5 基于约束的曲线和表面造型 . . . 343
5.1 基于约束的曲线修改 . . . 343
5.2 基于约束的表面修改 . . . 348

第十二章　标准与数据交换 **352**
1　引言 . . . 352
2　结点向量 . . . 352
3　标准中的 NURBS . . . 358
3.1　IGES . . . 358
3.2　STEP . . . 360
3.3　PHIGS . . . 361
4　NURBS 系统的数据交换 . . . 362

第十三章　B-样条编程概念 **364**
1　引言 . . . 364
2　数据类型与可移植性 . . . 364
3　数据结构 . . . 364
4　内存分配 . . . 366
5　错误控制 . . . 369
6　实用程序 . . . 372
7　算术例程 . . . 373
8　示例程序 . . . 373
9　额外结构 . . . 375
10　系统结构 . . . 376

第十四章　参考文献 **377**

第十五章　索引 **379**

第一章　曲线和曲面基础

1　隐式和参数形式

在几何建模中，表示曲线和曲面的两种最常见的方法是隐式方程和参数函数。

位于 xy 平面的曲线的隐式方程具有形式 $f(x,y)=0$ 。该方程描述了曲线上点的 x 和 y 坐标之间的隐式关系。对于给定的曲线，该方程在乘法常数的情况下是唯一的。一个例子是单位半径的圆，其中心位于原点，由方程 $f(x,y)=x^2+y^2-1=0$ 指定 (图 1.1)。

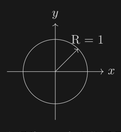


图 1.1: 半径为 1，中心位于原点的圆

在参数形式中，曲线上点的每个坐标都作为独立参数的显式函数单独表示。

$$\mathbf{C}(u)=(x(u),y(u))\ \ a\le u\le b$$

因此，$\mathbf{C}(u)$ 是独立变量 u 的向量值函数。尽管区间 $[a,b]$ 是任意的，但通常归一化为 $[0,1]$ 。图 1.1 中所示的圆的第一象限由参数函数定义。

$$x(u)=\cos(u)$$

$$y(u)=\sin(u)\ \ 0\le u\le\frac{\pi}{2} \tag{1.1}$$

设置 $t=\tan(u/2)$ ，可以推导出替代表示。

$$x(t)=\frac{1-t^2}{1+t^2}$$

$$y(t)=\frac{2t}{1+t^2}\ \ 0\le t\le 1 \tag{1.2}$$

因此，曲线的参数表示并不是唯一的。

将 $\mathbf{C}(u)=(x(u),y(u))$ 理解为粒子随时间变化所描绘的路径是很有启发性的：u 是时间变量，$[a,b]$ 是时间间隔。$\mathbf{C}(u)$ 的一阶和二阶导数分别是粒子的速度和加速度。对方程 (1.1) 和 (1.2) 进行一次求导得到速度函数。

$$\mathbf{C}'(u)=(x'(u),y'(u))=(-\sin(u),\cos(u))$$

$$\mathbf{C}'(t)=(x'(t),y'(t))=\left(\frac{-4t}{(1+t^2)^2},\frac{2\left(1-t^2\right)}{(1+t^2)^2}\right)$$

注意，速度向量 $\mathbf{C}'(u)$ 的大小是一个常数。

$$|\mathbf{C}'(u)|=\sqrt{\sin^2(u)+\cos^2(u)}=1$$

即，粒子的方向随时间变化，但其速度保持不变。这被称为均匀参数化。将 $t=0$ 和 $t=1$ 代入 $\mathbf{C}'(t)$ 得到 $\mathbf{C}'(0)=(0,2)$ 和 $\mathbf{C}'(1)=(-1,0)$ ，即，粒子的起始速度是其结束速度的两倍 (图 1.2)。

表面由形式为 $f(x,y,z)=0$ 的隐式方程定义。一个例子是以原点为中心的单位半径球体，如图 1.3 所示，并由方程 $x^2+y^2+z^2-1=0$ 指定。同一球体的参数化表示 (不唯一) 由 $\mathbf{S}(u,v)=(x(u,v),y(u,v),z(u,v))$ 给出，其中

$$x(u,v)=\sin(u)\cos(v)$$

$$y(u,v)=\sin(u)\sin(v)$$

$$z(u,v)=\cos(u)\ \ 0\le u\le\pi,0\le v\le 2\pi \tag{1.3}$$

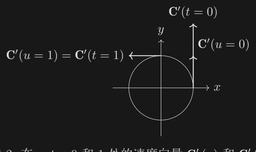


图 1.2: 在 $u,t=0$ 和 1 处的速度向量 $\mathbf{C}'(u)$ 和 $\mathbf{C}'(t)$ 。

请注意，定义一个表面需要两个参数。保持 u 固定并改变 v 生成球体的纬线；保持 v 固定并改变 u 生成经线。

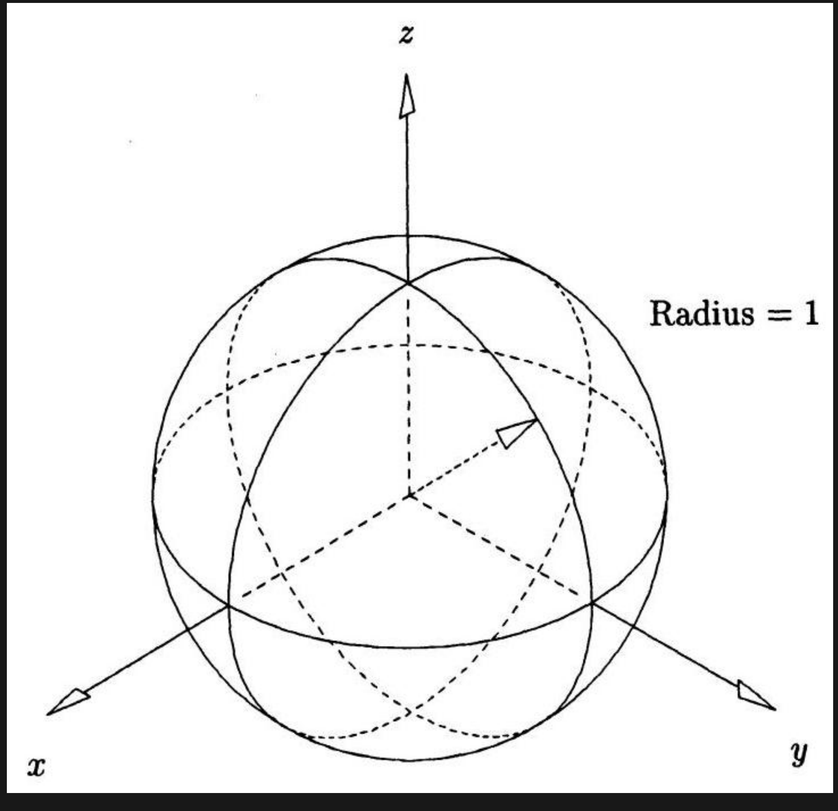


图 1.3. 半径为 1 的球体，位于原点。

记 $\mathbf{S}(u,v)$ 的偏导数为 $\mathbf{S}_u(u,v)=(x_u(u,v),y_u(u,v),z_u(u,v))$ 和 $\mathbf{S}_v(u,v)=(x_v(u,v),y_v(u,v),z_v(u,v))$ ，即沿纬线和经线的速度。在表面上的任何一点，如果向量叉积 $\mathbf{S}_u\times\mathbf{S}_v$ 不为零，则单位法向量 $\mathbf{N}$ 由 (图 1.4) 给出。

$$\mathbf{N}=\frac{\mathbf{S}_u\times\mathbf{S}_v}{|\mathbf{S}_u\times\mathbf{S}_v|} \tag{1.4}$$

在某一点存在法向量及相应的切平面是表面的几何属性，与参数化无关。不同的参数化给出不同的偏导数，但方程 (1.4) 始终产生 $\mathbf{N}$ ，前提是分母不为零。从方程 (1.3) 可以看出，对于所有 $v,0\le v\le 2\pi,\mathbf{S}_v(0,v)=\mathbf{S}_v(\pi,v)=0$ ，即 $\mathbf{S}_v$ 在球体的南北极消失。显然，在两个极点确实存在法向量，但在这种参数化下，方程 (1.4) 不能用于计算它们。

在隐式和参数化形式中，很难坚持认为其中一种形式总是比另一种更合适。两者各有优缺点。成功的几何建模使用了这两种技术。以下是两种方法的比较.

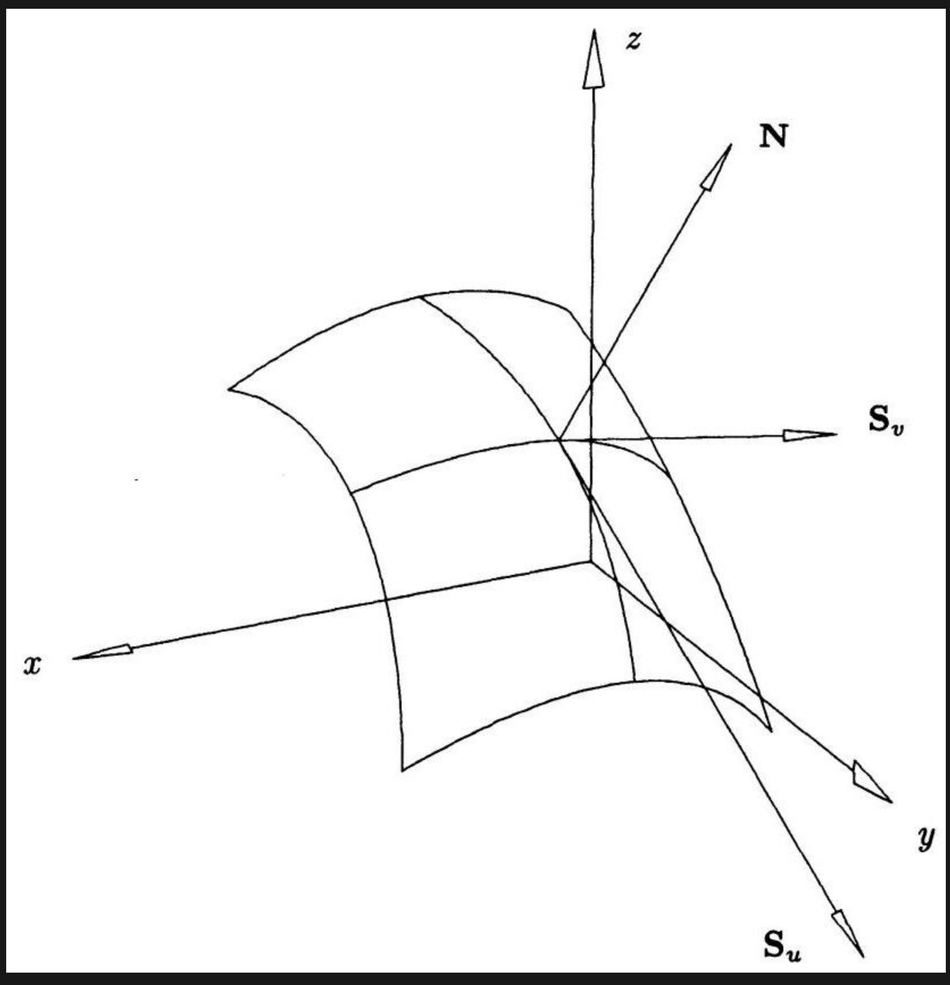


图 1.4. $\mathbf{S}(u,v)$ 的偏导数和单位法向量。

- 通过添加一个 z 坐标，参数化方法可以轻松扩展以表示三维空间中的任意曲线，$\mathbf{C}(u)=(x(u),y(u),z(u))$；隐式形式仅指定 xy (或 xz 或 yz) 平面上的曲线；
- 使用隐式形式表示有界曲线段 (或表面片段) 是繁琐的。然而，有界性通过参数区间的界限内置于参数化形式中。另一方面，使用参数化几何实现无界几何 (例如，由 $f(x,y)=ax+by+c=0$) 给出的简单直线) 是困难的；
- 参数曲线具有自然的遍历方向 (如果 $a\le u\le b$，则从 $\mathbf{C}(a)$ 到 $\mathbf{C}(b)$)；隐式曲线则没有。因此，沿着参数曲线生成有序点序列是容易的。类似的说法也适用于在表面上生成点网格；
- 参数形式在计算机中设计和表示形状时更为自然。许多参数函数的系数，例如，Bézier 和 B-spline，具有相当大的几何意义。这转化为直观的设计方法和具有明显几何特征的数值稳定算法；
- 许多几何操作和处理的复杂性在很大程度上取决于表示方法。两个经典的例子是:
- 计算曲线或表面上的一个点 - 在隐式形式中困难；- 给定一个点，确定它是否在曲线或表面上 - 在参数形式中困难；
- 在参数形式中，有时必须处理与真实几何无关的参数异常。一个例子是单位球体 (见 Eq.[1.3])。极点是参数临界点，算法上难以处理，但在几何上，极点与球体上的任何其他点没有区别。

本书其余部分几乎完全关注参数形式。关于隐式和参数形式的更多细节可以在标准文献中找到 ([Faux81; Mort85; Hoff89; Beac91])。

2　曲线的幂基形式

显然，通过允许坐标函数 $x(u),y(u)$ 和 $z(u)$ 为任意，我们可以获得各种各样的曲线。然而，在实现几何建模系统时存在权衡。理想的情况是将自己限制在一类函数中，这些函数

- 能够精确表示系统用户所需的所有曲线；
- 在计算机中易于、高效且准确地处理，特别是:
- 曲线上的点和导数的计算是高效的；
- 函数的数值处理对浮点舍入误差相对不敏感；
- 这些函数对存储的内存要求很少；
- 简单且在数学上被很好地理解。

一类广泛使用的函数是多项式。尽管它们满足此列表中的最后两个标准，但仍有许多重要的曲线 (和表面) 类型无法通过多项式精确表示；这些曲线必须在使用多项式的系统中进行近似。在本节及下一节中，我们研究表达多项式函数的两种常见方法: 幂基和贝塞尔。尽管在数学上是等价的，但我们将看到贝塞尔方法更适合在计算机中表示和操作形状。

一个 n 次幂基曲线由以下公式给出

$$\mathbf{C}(u)=(x(u),y(u),z(u))=\sum_{i=0}^{n}\mathbf{a}_i u^i \quad 0\le u\le 1 \tag{1.5}$$

$\mathbf{a}_i=(x_i,y_i,z_i)$ 是向量，因此

$$x(u)=\sum_{i=0}^{n}x_i u^i \; y(u)=\sum_{i=0}^{n}y_i u^i \; z(u)=\sum_{i=0}^{n}z_i u^i$$

在矩阵形式下，方程 (1.5) 为

$$\mathbf{C}(u)=\begin{bmatrix}\mathbf{a}_0 & \mathbf{a}_1 & \cdots & \mathbf{a}_n\end{bmatrix}\begin{bmatrix}1\\u\\\vdots\\u^n\end{bmatrix}=\left[\mathbf{a}_i\right]^T\left[u^i\right] \tag{1.6}$$

(我们将行向量写作列向量的转置。)

对方程 (1.5) 进行微分得到

$$\mathbf{a}_i=\frac{\mathbf{C}^{(i)}(u)\big|_{u=0}}{i!}$$

其中 $\mathbf{C}^{(i)}(u)\big|_{u=0}$ 是 i 在 $u=0$ 处的 $\mathbf{C}(u)$ 次导数。$n+1$ 函数 $\left\{u^i\right\}$ 被称为基函数 (或混合函数)，而 $\{\mathbf{a}_i\}$ 是幂基表示的系数。

给定 u_0 ，在幂基曲线上的点 $\mathbf{C}(u_0)$ 最有效的计算方法是使用霍纳法

- 对于 $=1:\mathbf{C}(u_0)=\mathbf{a}_1u_0+\mathbf{a}_0$ 次
- 学位 $=2:\mathbf{C}(u_0)=(\mathbf{a}_2u_0+\mathbf{a}_1)u_0+\mathbf{a}_0$

. :

$$\text{degree }=\text{n}:\mathbf{C}(u_0)=((\cdots(\mathbf{a}_nu_0+\mathbf{a}_{n-1})u_0+\mathbf{a}_{n-2})u_0+\cdots+\mathbf{a}_0$$

一般算法是

ALGORITHM A1.1

```
Horner1(a, n, u0, C)
    { /* Compute point on power basis curve. */
        /* Input: a, n, u0 */
        /* Output: C */
      C = a[n];
      for (i=n-1; i>=0; i--)
          C = C*u0 + a[i];
    }
```

示例

Ex1.1 $n=1$ 。$\mathbf{C}(u)=\mathbf{a}_0+\mathbf{a}_1 u, 0 \leq u \leq 1$ 是连接点 $\mathbf{a}_0$ 和 $\mathbf{a}_0+\mathbf{a}_1$ 的直线段 (图 1.5)。常数 $\mathbf{C}'(u)=\mathbf{a}_1$ 给出了线的方向。

Ex1.2 $n=2$ 。一般来说，$\mathbf{C}(u)=\mathbf{a}_0+\mathbf{a}_1 u+\mathbf{a}_2 u^2, 0 \leq u \leq 1$ 是连接点 $\mathbf{a}_0$ 和 $\mathbf{a}_0+\mathbf{a}_1+\mathbf{a}_2$ 的抛物线弧 (图 1.6)。这可以通过以下方式显示:

1. 将 $\mathbf{C}(u)$ 变换到 xy 平面 ($\mathbf{C}(u)$ 确实位于一个唯一的平面内)；

2. 设定 $x=x_0+x_1 u+x_2 u^2$ 和 $y=y_0+y_1 u+y_2 u^2$ ，然后从这些方程中消去 u 和 u^2 ，以获得关于 x 和 y 的二次隐式方程；

3. 观察隐式方程的形式是抛物线的形式。

注意，加速度向量 $\mathbf{C}''(u)=2\mathbf{a}_2$ 是一个常量。有两个特殊的 (退化的) 情况值得关注，均发生在向量 $\mathbf{a}_2$ 与初始切向量 $\mathbf{a}_1$ 平行时 (当 $x_1 y_2=x_2 y_1$ 时)。在这种情况下，切向量不转动，即我们得到一条直线。向量 a_2 可以指向与 a_1 相同的方向 (图 1.7a)，或指向相反的方向 (图 1.7b)。在图 1.7 b, $\mathrm{a}_1+2 \mathrm{a}_2 u_0=0$ 中，对于某些 $0 \leq u_0 \leq 1$ (速度趋于零，粒子停止)，线段的一部分在相反方向上被重新追踪。

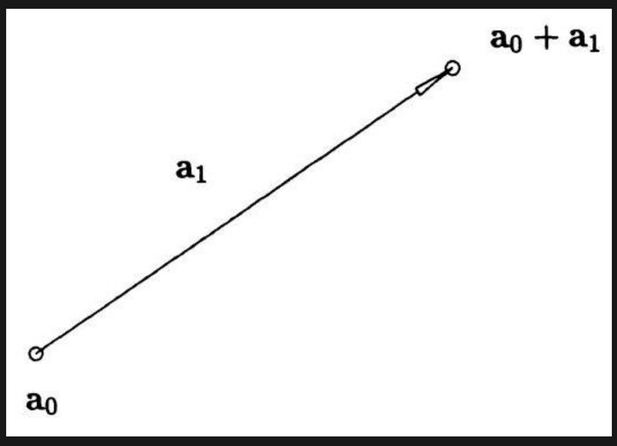


图 1.5. 直线段，$\mathbf{C}(u)=\mathbf{a}_0+\mathbf{a}_1 u$

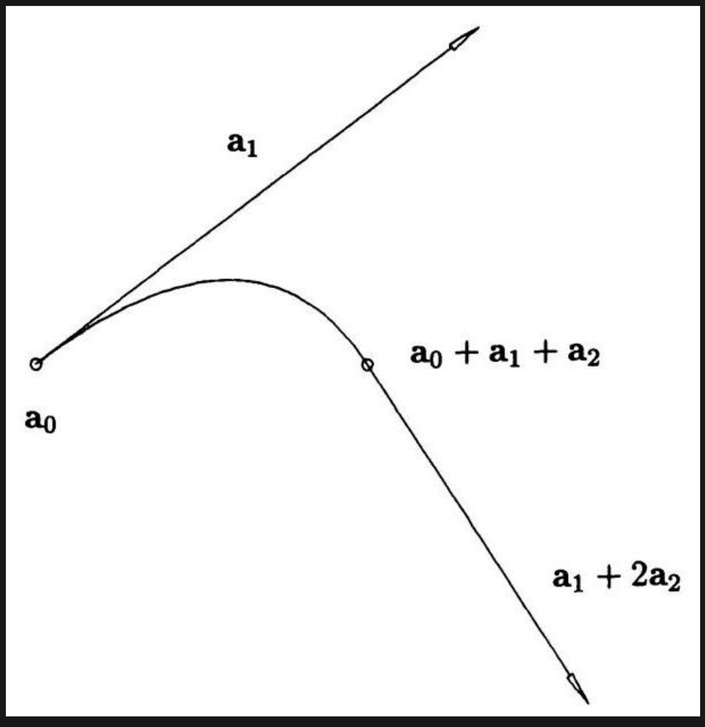


图 1.6. 抛物线弧，$\mathbf{C}(u)=\mathbf{a}_0+\mathbf{a}_1 u+\mathbf{a}_2 u^2$ 。

Ex1.3 $n=3$ 。三次曲线 $\mathbf{C}(u)=\mathbf{a}_0+\mathbf{a}_1 u+\mathbf{a}_2 u^2+\mathbf{a}_3 u^3$ 是一种非常一般的曲线；它可以是一个真正扭曲的三维曲线，不在同一平面内 (图 1.8a)；它可以有一个拐点 (图 1.8b)；一个尖点 (图 1.8c)；或一个环 (图 1.8d)。如果 $\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ 不在唯一平面内，则会产生扭曲曲线。平面曲线上的拐点被定义为曲线光滑 (没有尖点) 且该点的切线通过曲线的点。这意味着曲线的转向方向发生了变化。在拐点处，要么 $\mathbf{C}''(u)=0$ ，要么 $\mathbf{C}'(u) \ \mathbf{C}''(u)$ 。在 $u=u_0$ 处形成尖点的必要 (但不充分) 条件是 $\mathbf{C}'(u_0)=0$ (速度为零)。环出现的条件也已知 (见 [Ferg66, 67, 69, 93; Forr70, 80; Wang81; Ston89; Su89])。

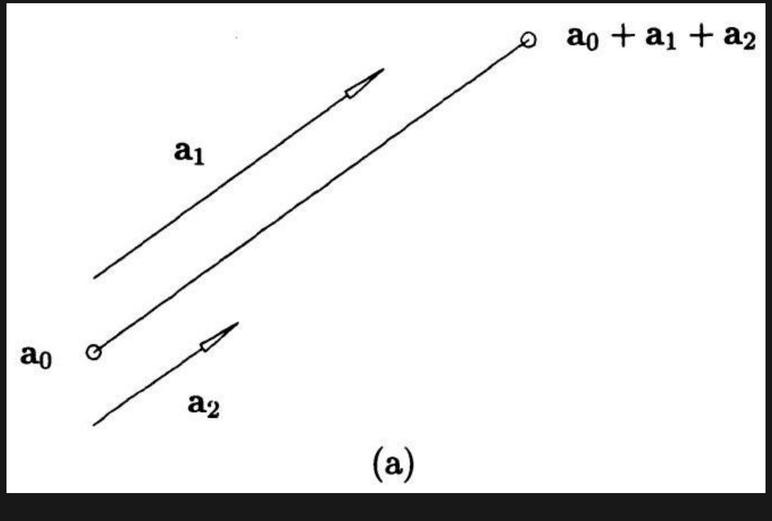


图 1.7. $\mathbf{a}_1$ 和 $\mathbf{a}_2$ 平行。(a) 同一方向；(b) 相反方向。

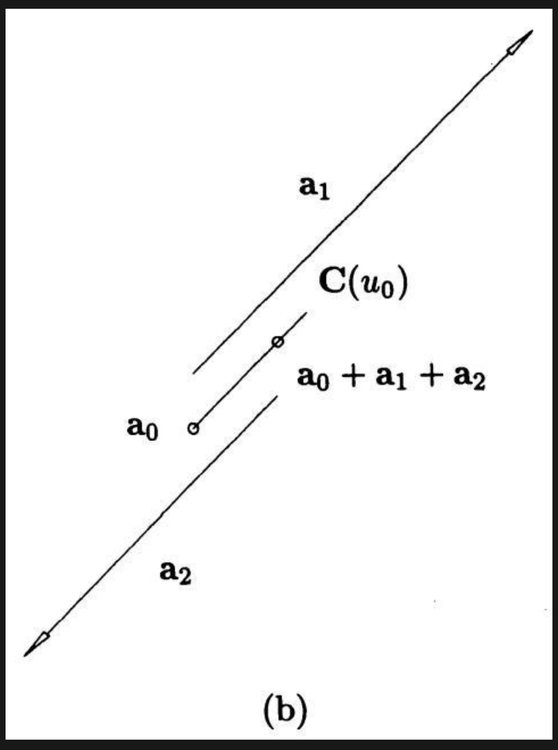


图 1.7. (续。)

3 Bézier 曲线

接下来我们研究另一种参数多项式曲线，贝塞尔曲线。由于它们都使用多项式作为坐标函数，幂基和贝塞尔形式在数学上是等价的；即，任何可以用一种形式表示的曲线也可以用另一种形式表示。然而，贝塞尔方法在几何建模方面优于幂基形式。我们对贝塞尔曲线的介绍相对非正式；对于更严格和完整的处理，读者应参考其他文献 [Forr72; Bezi72, 86; Gord74a; Chan81; Fari93; Yama88; Hosc93; Roge90]。

幂基形式有以下缺点:

- 对于交互式形状设计来说不自然；系数 $\{\mathbf{a}_i\}$ 对曲线形状的几何洞察力非常有限。此外，设计师通常希望在曲线的两端指定边界条件，而不仅仅是在起始点；
- 处理幂基多项式的算法具有代数而非几何的特征 (例如，霍纳法则)；
- 在数值上，它是一种相对较差的形式；例如，当系数的大小差异很大时，霍纳法则容易产生舍入误差 (参见 [Faro87, 88; Dani89])。

贝塞尔方法弥补了这些缺点。

一个 n 次贝塞尔曲线定义为

$$\mathbf{C}(u) = \sum_{i=0}^{n} B_{i,n}(u)\,\mathbf{P}_i \quad 0 \le u \le 1 \tag{1.7}$$

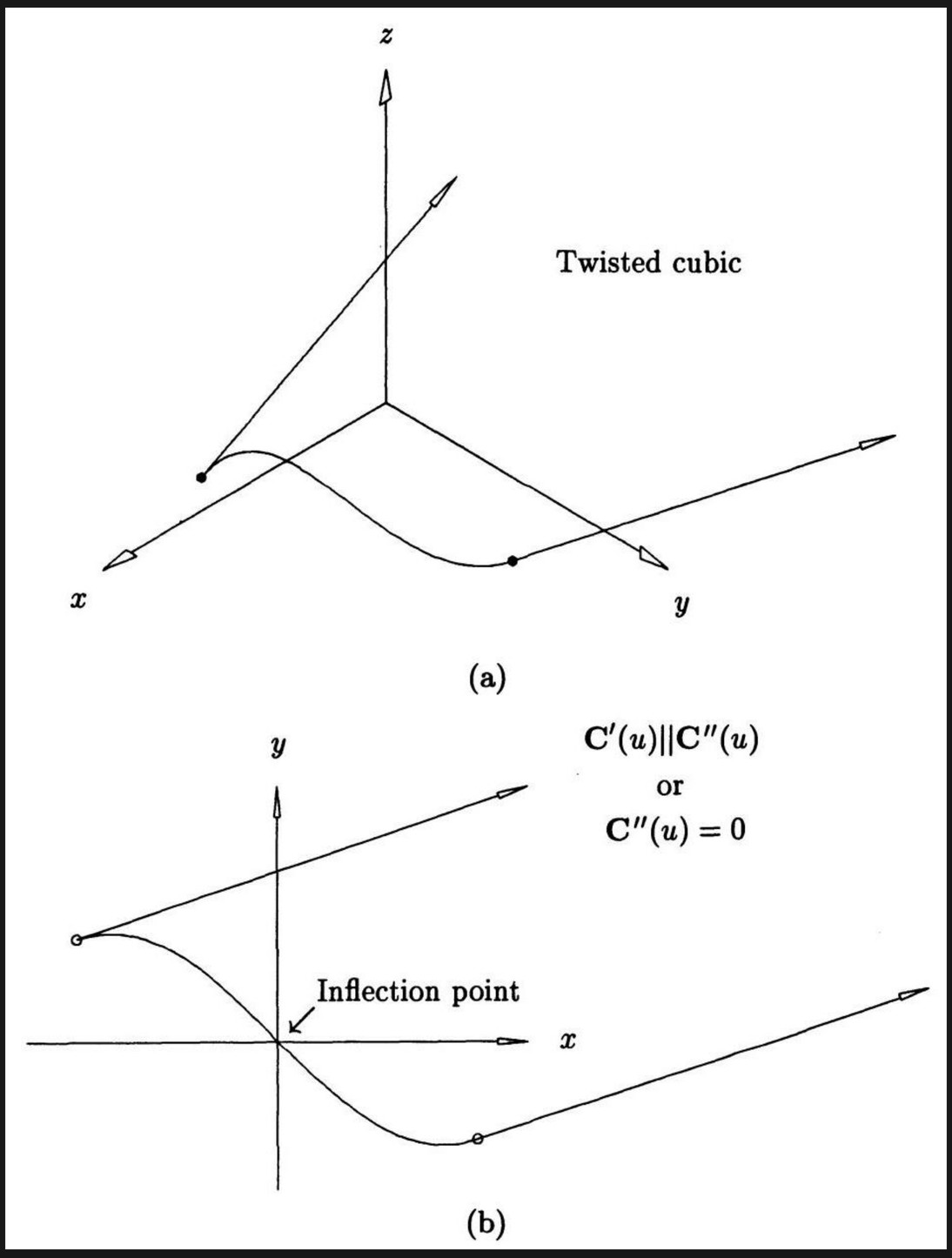


图 1.8. 三次曲线。(a) 三维扭曲；(b) 拐点；(c) 尖点；(d) 环。

基函数 (混合函数) $\{B_{i,n}(u)\}$ 是经典的 n 次伯恩斯坦多项式 ([Bern12; Lore86])，其表达式为

$$B_{i,n}(u) = \frac{n!}{i!\,(n-i)!} u^i (1-u)^{n-i} \tag{1.8}$$

这种形式的几何系数 $\{\mathbf{P}_i\}$ 被称为控制点。注意，定义 Eq. (1.7) 要求 $u \in [0,1]$ 。

示例

Ex1.4 $n = 1$ 。根据公式 (1.8)，我们有 $B_{0,1}(u) = 1 - u$ 和 $B_{1,1}(u) = u$ ；而公式 (1.7) 的形式为 $\mathbf{C}(u) = (1-u)\mathbf{P}_0 + u\mathbf{P}_1$ 。这是从 $\mathbf{P}_0$ 到 $\mathbf{P}_1$ 的一条直线段 (见图 1.9)。

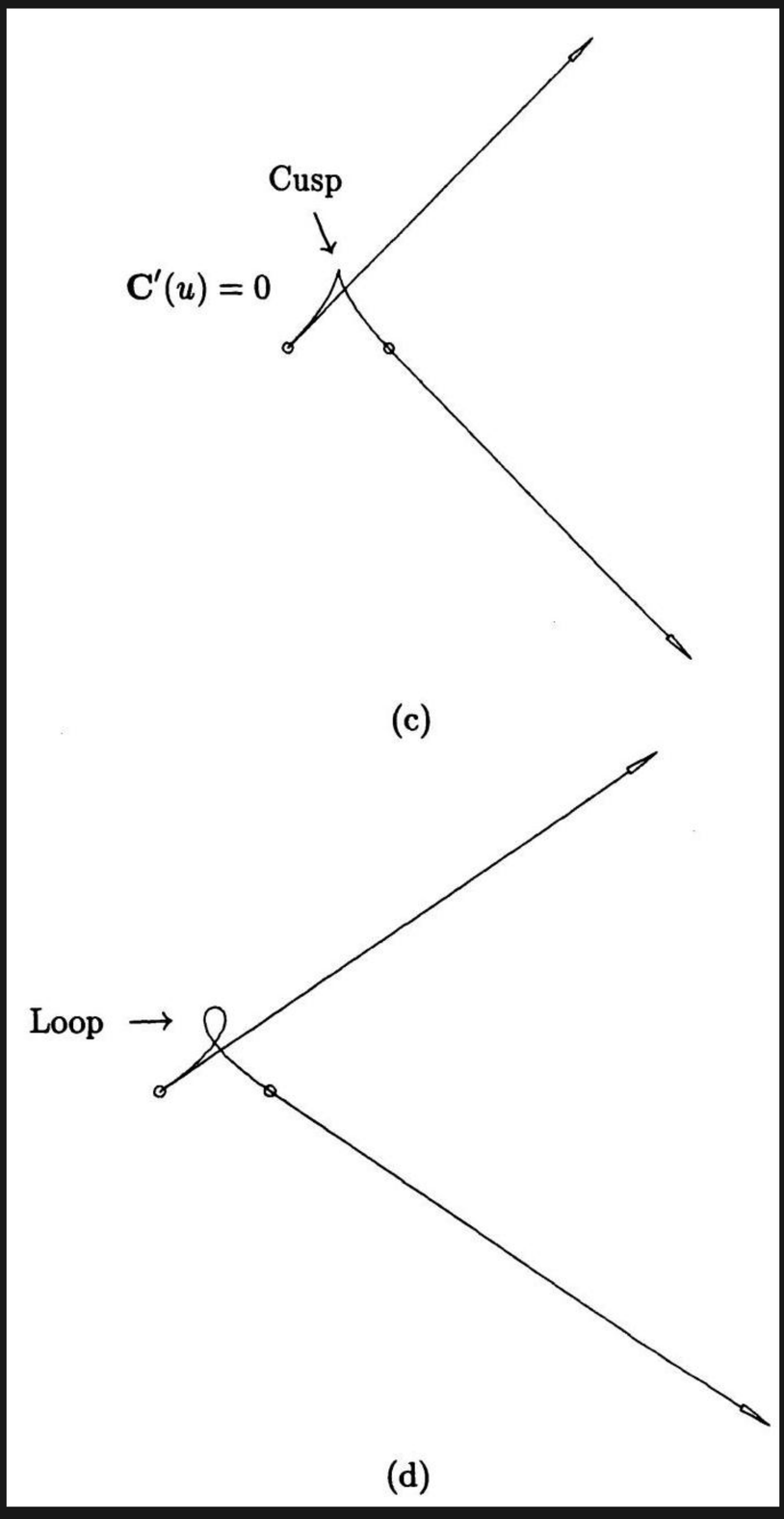


图 1.8. (续)

Ex1.5 $n = 2$ 。根据公式 (1.7) 和 (1.8)，我们有 $\mathbf{C}(u) = (1-u)^2\mathbf{P}_0 + 2u(1-u)\mathbf{P}_1 + u^2\mathbf{P}_2$ 。这是从 $\mathbf{P}_0$ 到 $\mathbf{P}_2$ 的一条抛物线弧 (见图 1.10)。注意到

- 由 $\{\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2\}$ 形成的多边形，称为控制多边形，较好地近似了曲线的形状；
- $\mathbf{P}_0 = \mathbf{C}(0)$ 和 $\mathbf{P}_2 = \mathbf{C}(1)$ ；
- 曲线在其端点的切线方向与 $\mathbf{P}_1 - \mathbf{P}_0$ 和 $\mathbf{P}_2 - \mathbf{P}_1$ 平行 (这一点将在后面推导)；
- 曲线包含在由 $\mathbf{P}_0\mathbf{P}_1\mathbf{P}_2$ 形成的三角形内。

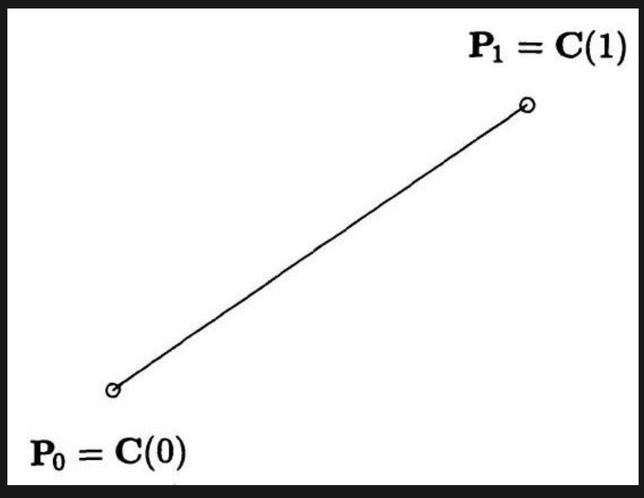


图 1.9. 一阶 Bézier 曲线。

Ex 1.6$n = 3$ 。我们有 $\mathbf{C}(u) = (1-u)^3\mathbf{P}_0 + 3u(1-u)^2\mathbf{P}_1 + 3u^2(1-u)\mathbf{P}_2 + u^3\mathbf{P}_3$ 。立方 Bézier 曲线的例子见图 1.11a 至 1.11f。注意到

- 控制多边形近似了曲线的形状；
- $\mathbf{P}_0 = \mathbf{C}(0)$ 和 $\mathbf{P}_3 = \mathbf{C}(1)$ ；
- 端点切线方向与 $\mathbf{P}_1 - \mathbf{P}_0$ 和 $\mathbf{P}_3 - \mathbf{P}_2$ 平行；
- 凸包性质: 曲线包含在其定义控制点的凸包内 (图 1.11c)；
- 变差减少性质: 没有直线与曲线相交的次数超过它与曲线的控制多边形相交的次数 (对于三维 Bézier 曲线，将“直线”替换为

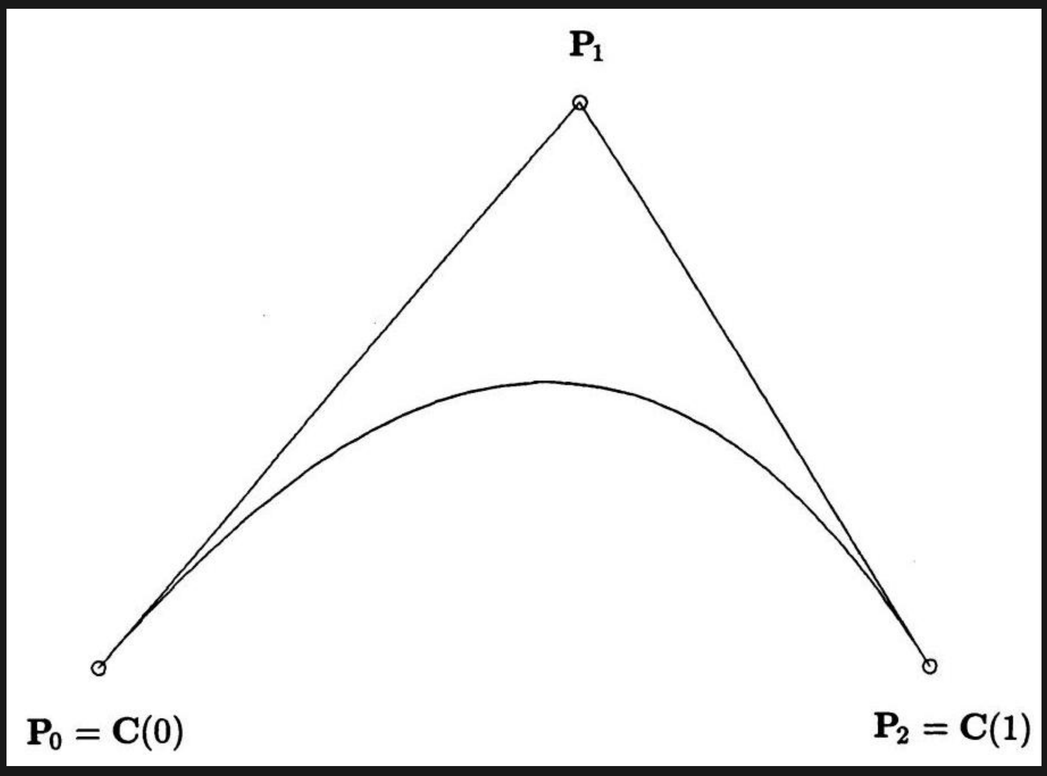


图 1.10. 二次贝塞尔曲线。

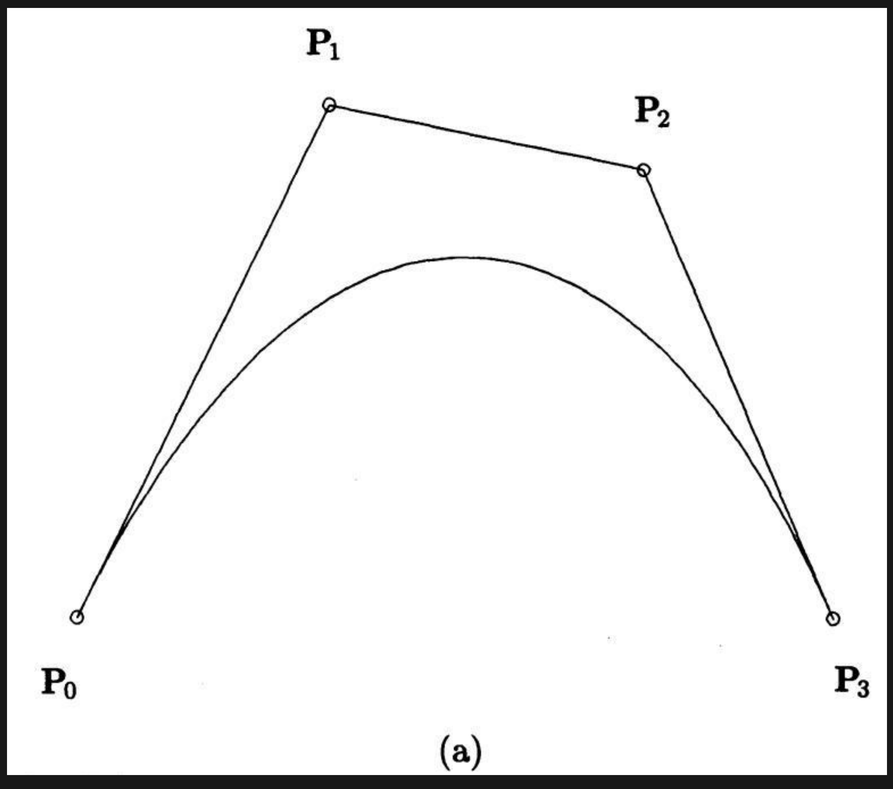


图 1.11. 三次贝塞尔曲线。

单词“平面”。这表达了贝塞尔曲线紧密跟随其控制多边形的特性，并且不会比其控制多边形更大幅度地摆动 (图 1.11f)：

- 最初 (在 $u = 0$ 时) 曲线的转向与 $\mathbf{P}_0\mathbf{P}_1\mathbf{P}_2$ 相同。在 $u = 1$ 时，它的转向为 $\mathbf{P}_1\mathbf{P}_2\mathbf{P}_3$ ；
- 控制多边形中的环可能暗示曲线中的环，也可能不暗示。从图 1.11e 到图 1.11f 的过渡是一个具有尖点的曲线。

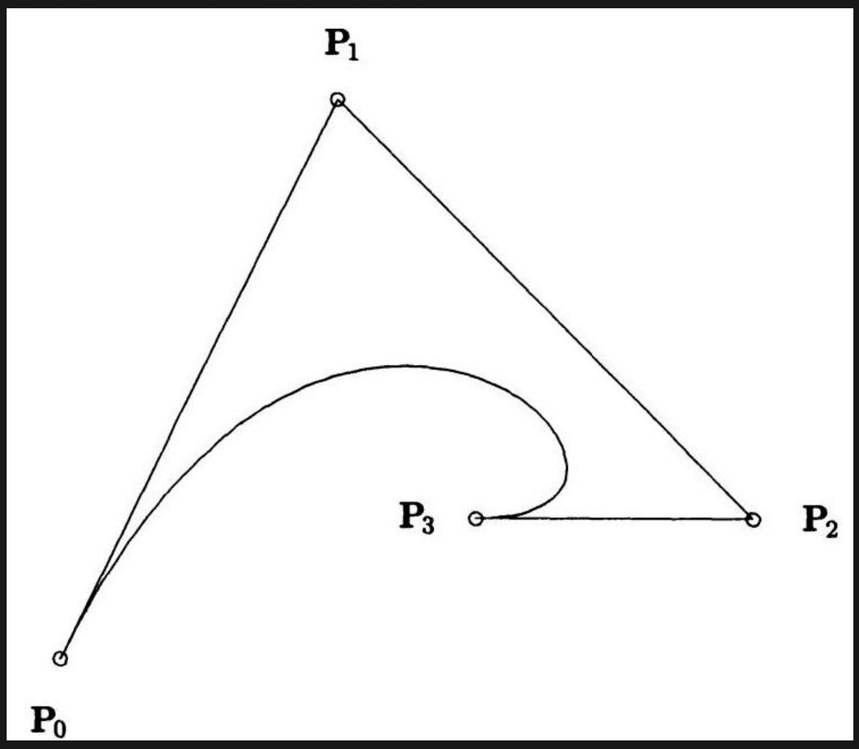
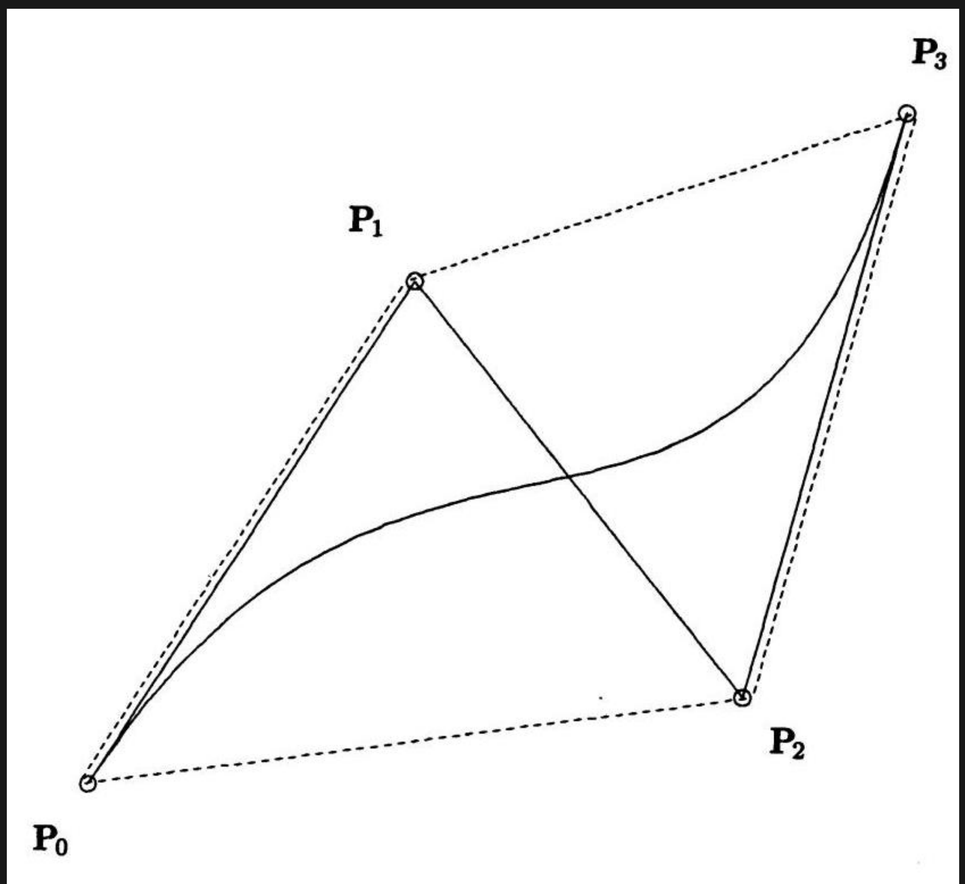
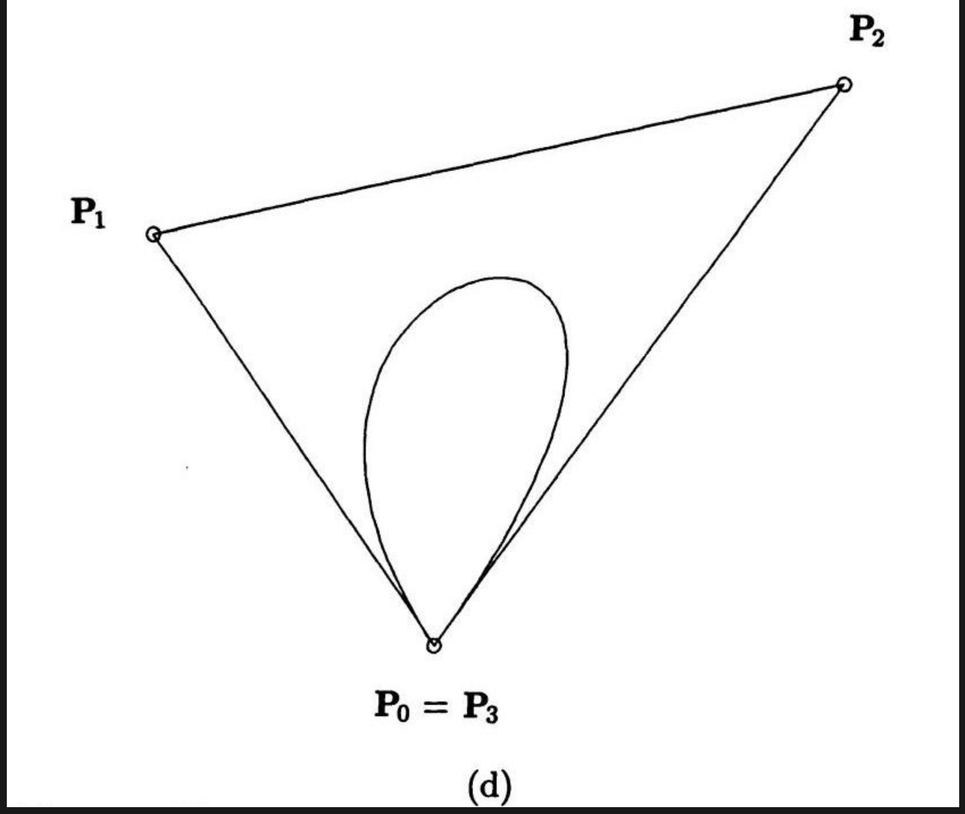


图 1.11. (续)



(c)

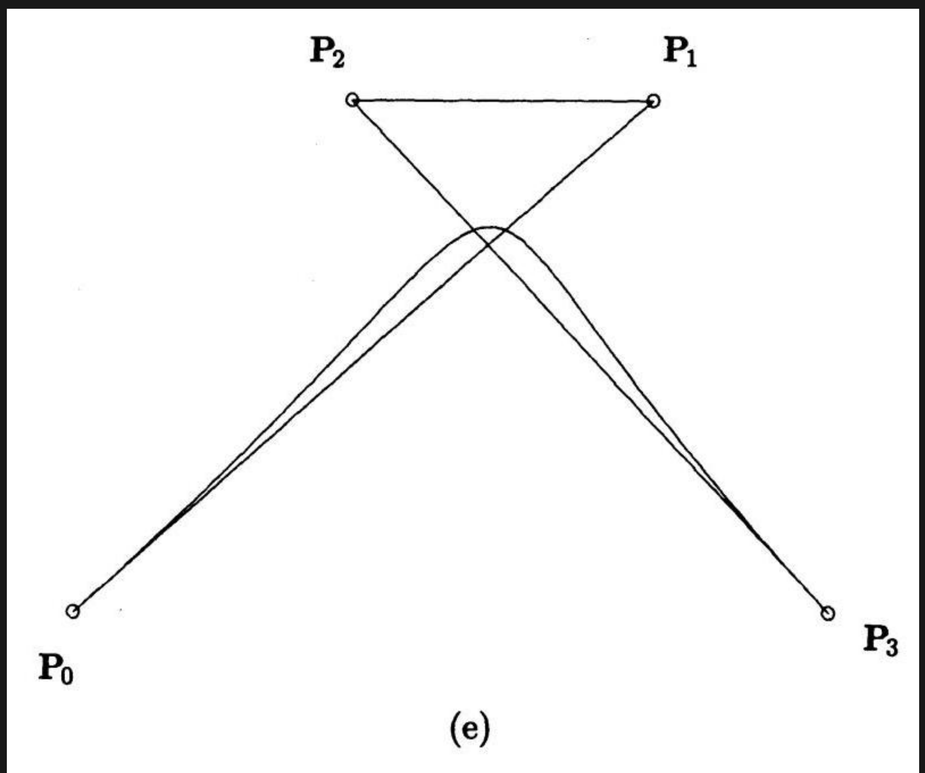


(d)

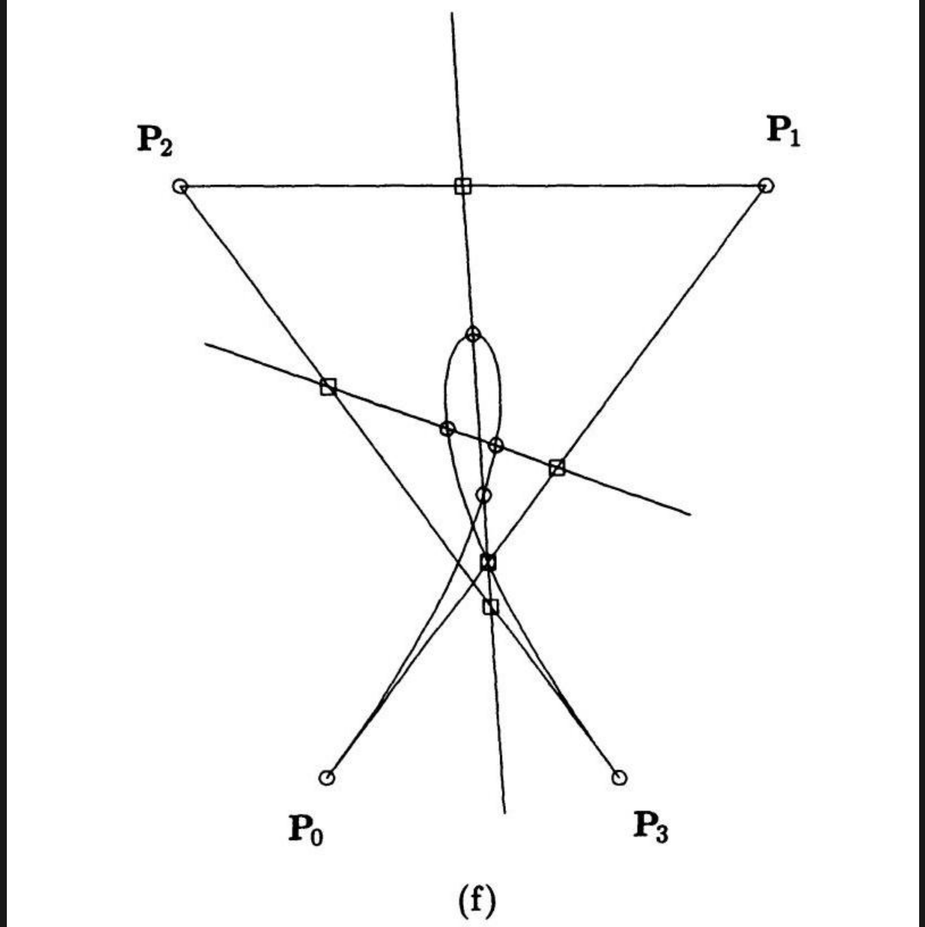
图 1.11. (续)

Ex1.7 $n = 6$ 。图 1.12 显示了一个六次闭合的 Bézier 曲线。由于 $\mathbf{P}_1 - \mathbf{P}_0$ 与 $\mathbf{P}_6 - \mathbf{P}_5$ 平行，该曲线在 $\mathbf{C}(0)\,(= \mathbf{C}(1))$ 处是光滑的。我们所说的光滑是指在 $u = 0$ 和 $u = 1$ 处的切向量具有相同的方向。

除了前面提到的性质外，Bézier 曲线在旋转、平移和缩放等常见变换下是不变的；也就是说，通过对控制多边形应用变换来对曲线应用变换。我们将在第 3 章中更严格地介绍这个概念，针对 B-spline 曲线 (Bézier 曲线是其特例)。



(e)



(f)

图 1.11. (续。)

在任何曲线 (或曲面) 表示方案中，基函数的选择决定了该方案的几何特性。图 1.13a – d 显示了 $\{B_{i,n}(u)\}$ 的基函数 $n = 1, 2, 3, 9$ 。这些函数具有以下性质:

P1.1 非负性: 对于所有 i, n 和 $0 \leq u \leq 1$ ，有 $B_{i,n}(u) \geq 0$;

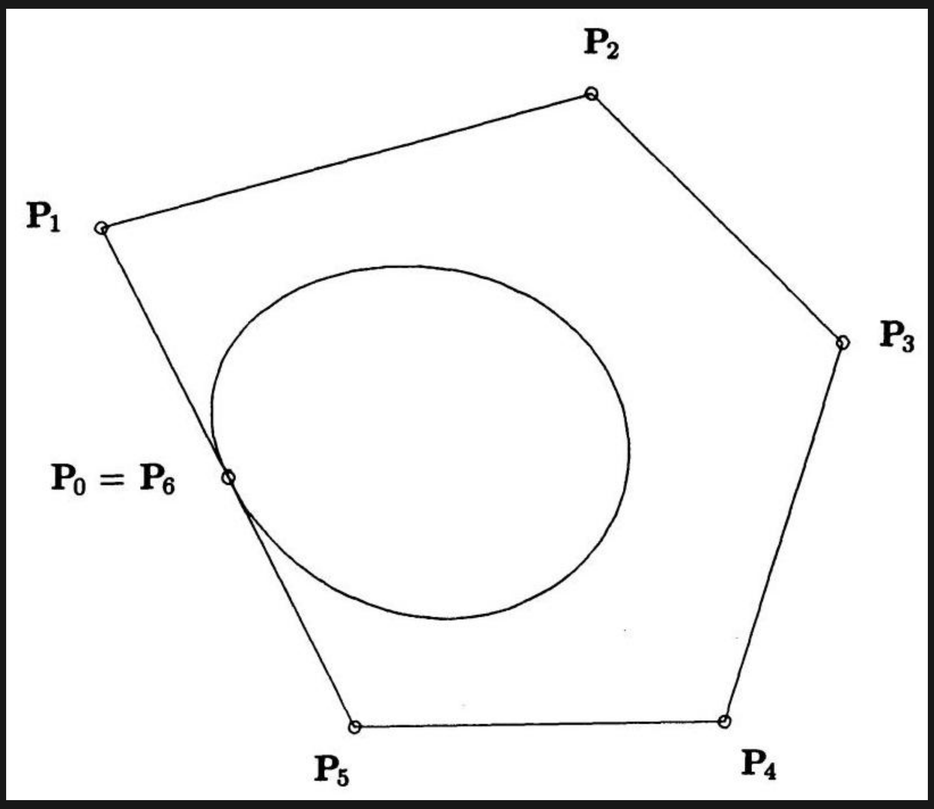


图 1.12. 一个光滑的闭合六次 Bézier 曲线。

P1.2 单位分割: 对于所有 $0 \leq u \leq 1$ ，有 $\sum_{i=0}^{n} B_{i,n}(u) = 1$ ；

P1. $3 B_{0,n}(0) = B_{n,n}(1) = 1$ ；

P1.4 $B_{i,n}(u)$ 在区间 $[0,1]$ 上恰好达到一个最大值，即在 $u = i/n$ 处。

P1.5 对称性: 对于任何 n ，多项式集合 $\{B_{i,n}(u)\}$ 关于 $u = 1/2$ 是对称的；

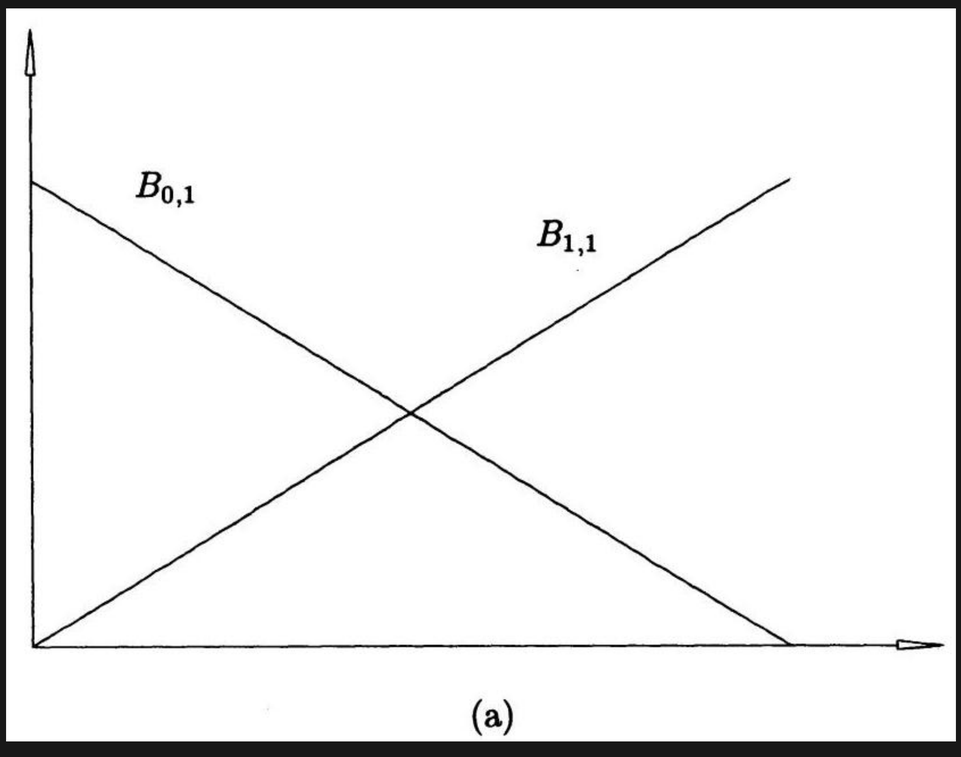


图 1.13. 伯恩斯坦多项式 (a) $n = 1$; (b) n = 2; (c) $n = 3$; (d) n = 9 。

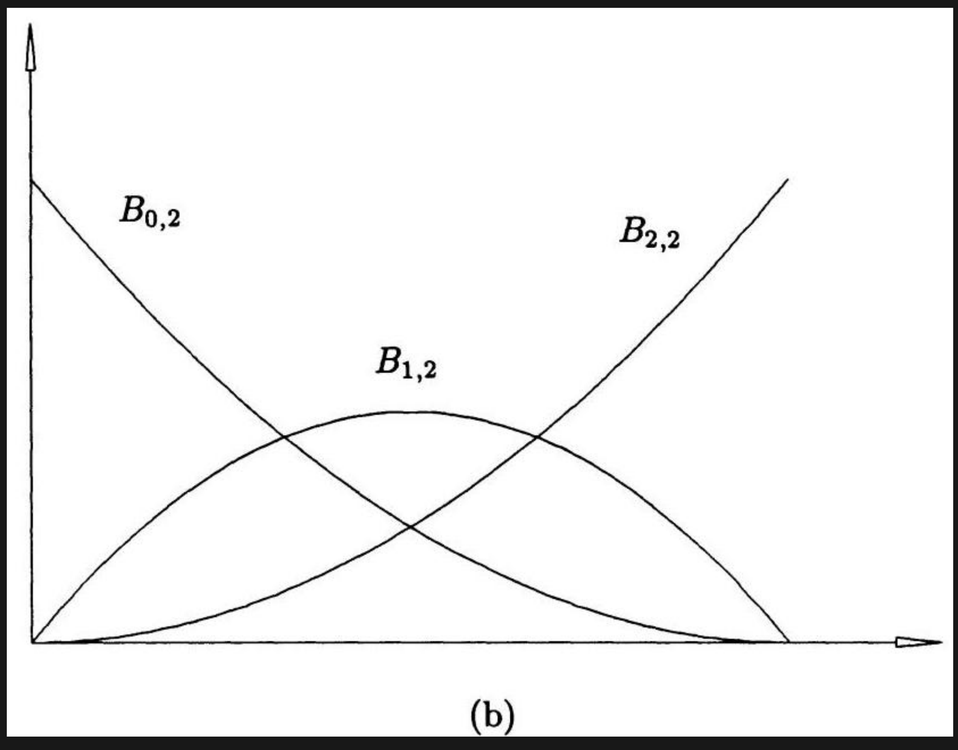


图 1.13. (续)

P1.6 递归定义: $B_{i,n}(u) = (1-u)\,B_{i,n-1}(u) + uB_{i-1,n-1}(u)$ (见图 1.14)；我们定义 $B_{i,n}(u) \equiv 0$ 如果 $i < 0$ 或 $i > n$ ；

P1.7 导数:

$$B'_{i,n}(u) = \frac{dB_{i,n}(u)}{du} = n\left(B_{i-1,n-1}(u) - B_{i,n-1}(u)\right)$$

与

$$B_{-1,n-1}(u) \equiv B_{n,n-1}(u) \equiv 0$$

图 1.15a 显示了 $B'_{2,5}$ 的定义，而图 1.15 b 则说明了所有的三次导数函数。18 曲线与曲面基础

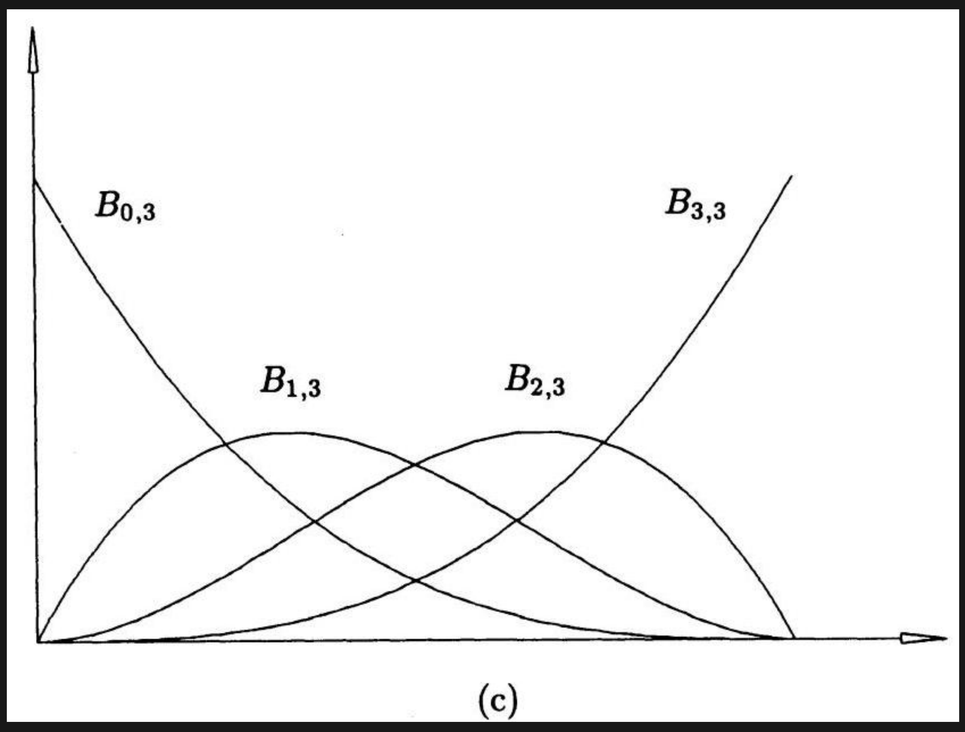


图 1.13. (续)

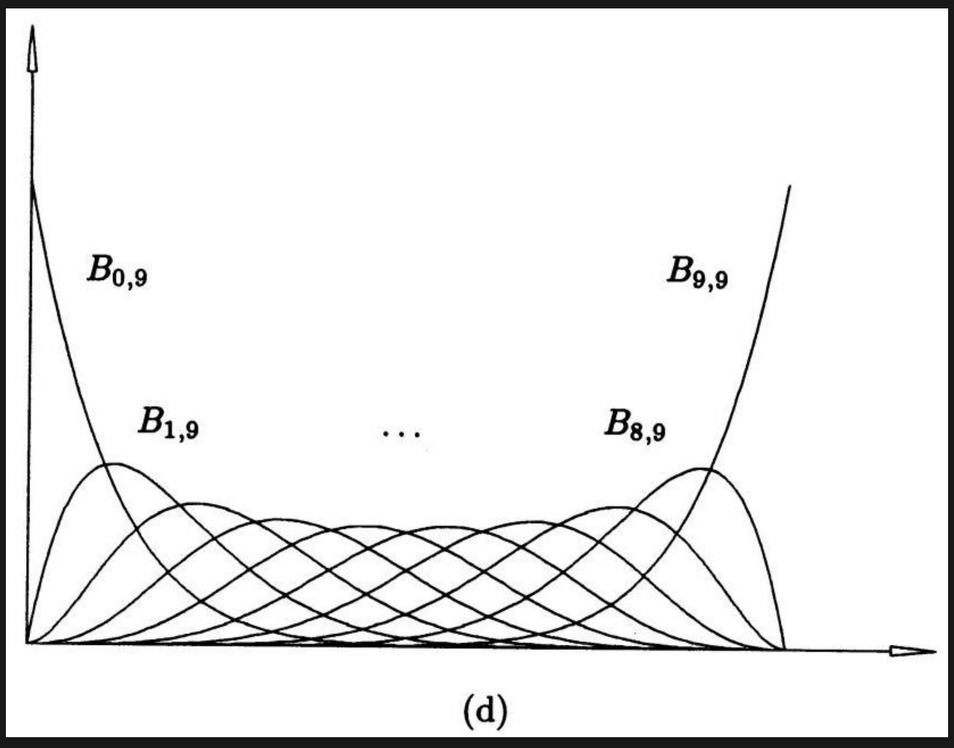


图 1.13. (续)

从方程 (1.8) 我们得到 $B_{0,0}(u) = 1$ 。利用性质 P1.6，线性和二次 Bernstein 多项式为

$$B_{0,1}(u) = (1-u)\,B_{0,0}(u) + uB_{-1,0}(u) = 1-u$$

$$B_{1,1}(u) = (1-u)\,B_{1,0}(u) + uB_{0,0}(u) = u$$

$$B_{0,2}(u) = (1-u)\,B_{0,1}(u) + uB_{-1,1}(u) = (1-u)^2$$

$$B_{1,2}(u) = (1-u)\,B_{1,1}(u) + uB_{0,1}(u) = (1-u)\,u + u(1-u) = 2u(1-u)$$

$$B_{2,2}(u) = (1-u)\,B_{2,1}(u) + uB_{1,1}(u) = u^2$$

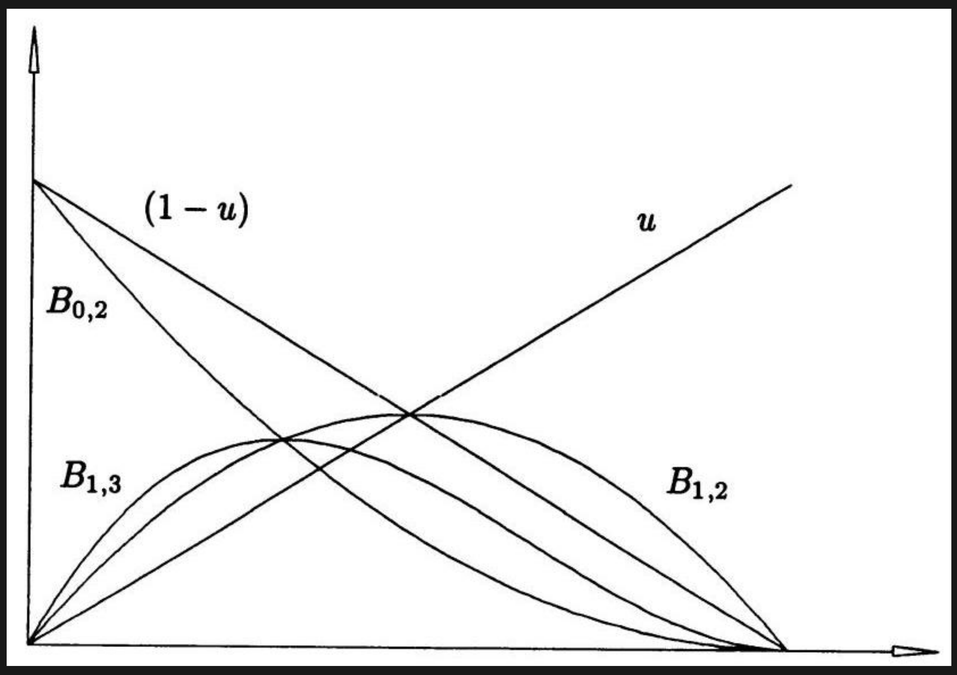


图 1.14. Bernstein 多项式的递归定义，$B_{1,3}(u)$ 。

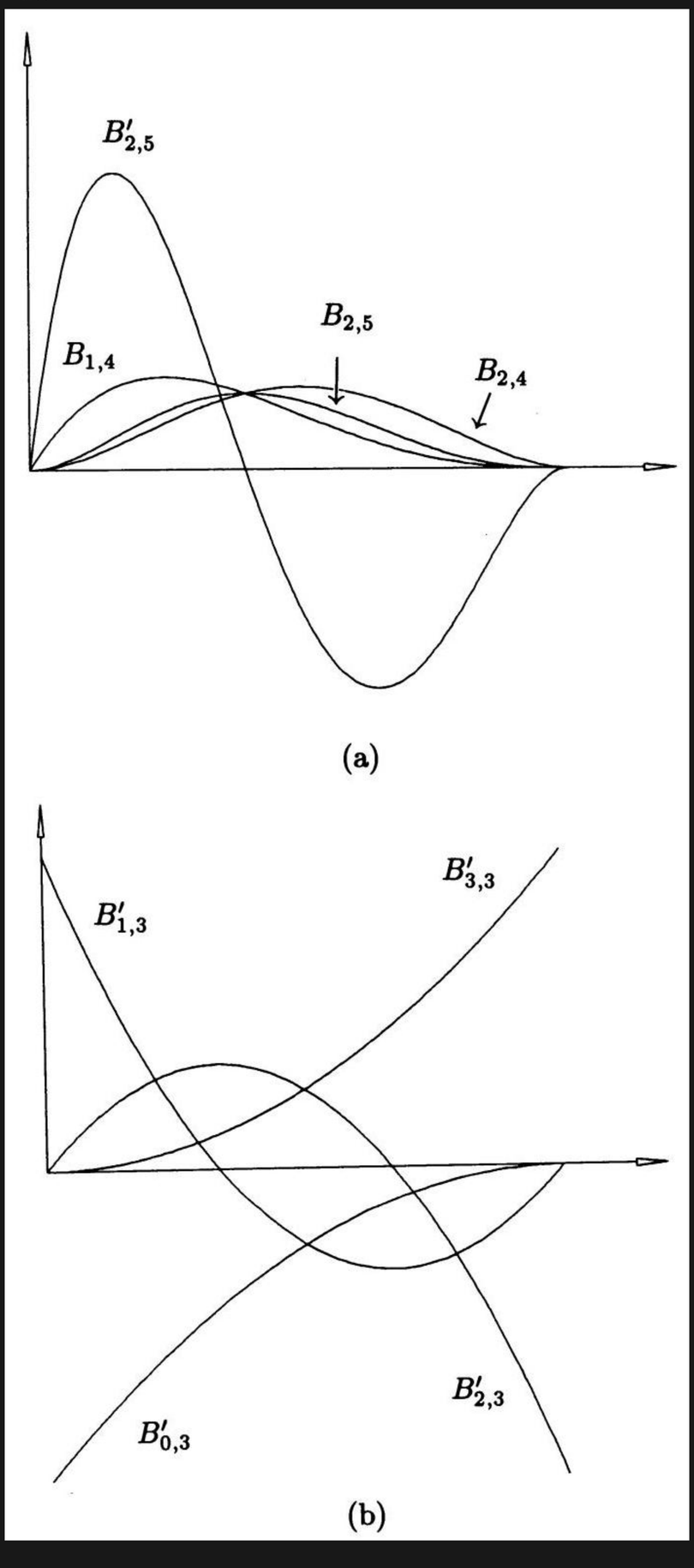


图 1.15. 导数。(a) 关于 $B_{1,4}(u)$ 和 $B_{2,4}(u)$ 的导数 $B'_{2,5}(u)$ ； (b) 四个三次 Bernstein 多项式的导数，$B'_{0,3}(u)$; $B'_{1,3}(u)$; $B'_{2,3}(u)$ ； $B'_{3,3}(u)$ 。

性质 P1.6 产生简单的算法来计算在固定值 u 下 Bernstein 多项式的值。算法 A1.2 计算固定 u 下的值 $B_{i,n}(u)$ 。$B_{1,3}$ 的计算如表 1.1 所示。

ALGORITHM A1.2

```
Bernstein(i, n, u, B)
    { /* Compute the value of a Bernstein polynomial. */
        /* Input: i, n, u */
        /* Output: B */
    for (j = 0; j <= n; j + +) /* compute the columns * /
        temp[j] = 0.0 ; /* of Table 1.1 */
    temp [n − i] = 1.0; /* in a temporary array * /
    u1 = 1.0-u;
    for (k=1; k<=n; k++)
        for (j=n; j>=k; j–)
            temp[j] = u1*temp[j] + u*temp[j-1];
    B = temp[n];
    }
```

算法 A1.3 计算在固定 u 处非零的 $n+1n$ 次伯恩斯坦多项式。它避免了不必要的零项计算。该算法在表 1.2 中展示了立方体的情况。

算法 A1.3

AllBernstein (n, u, B)

Table 1.1. The computation of $B_{1,3}$.

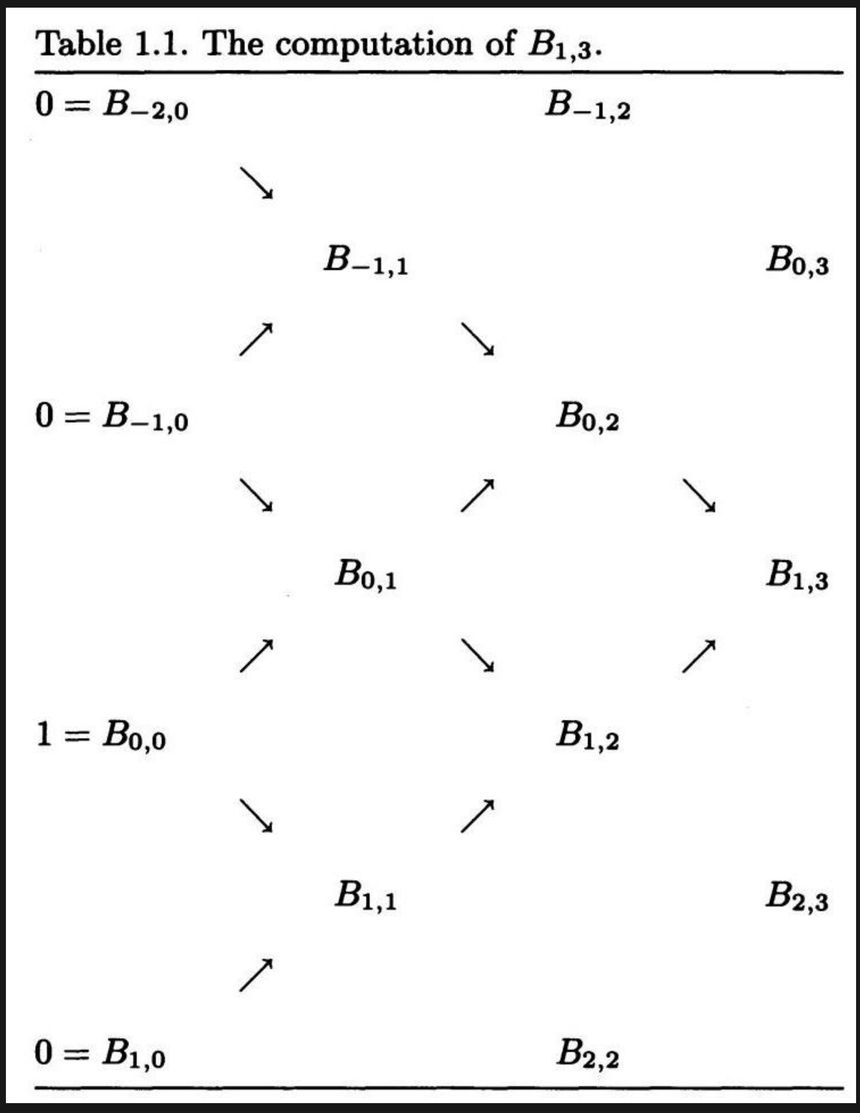
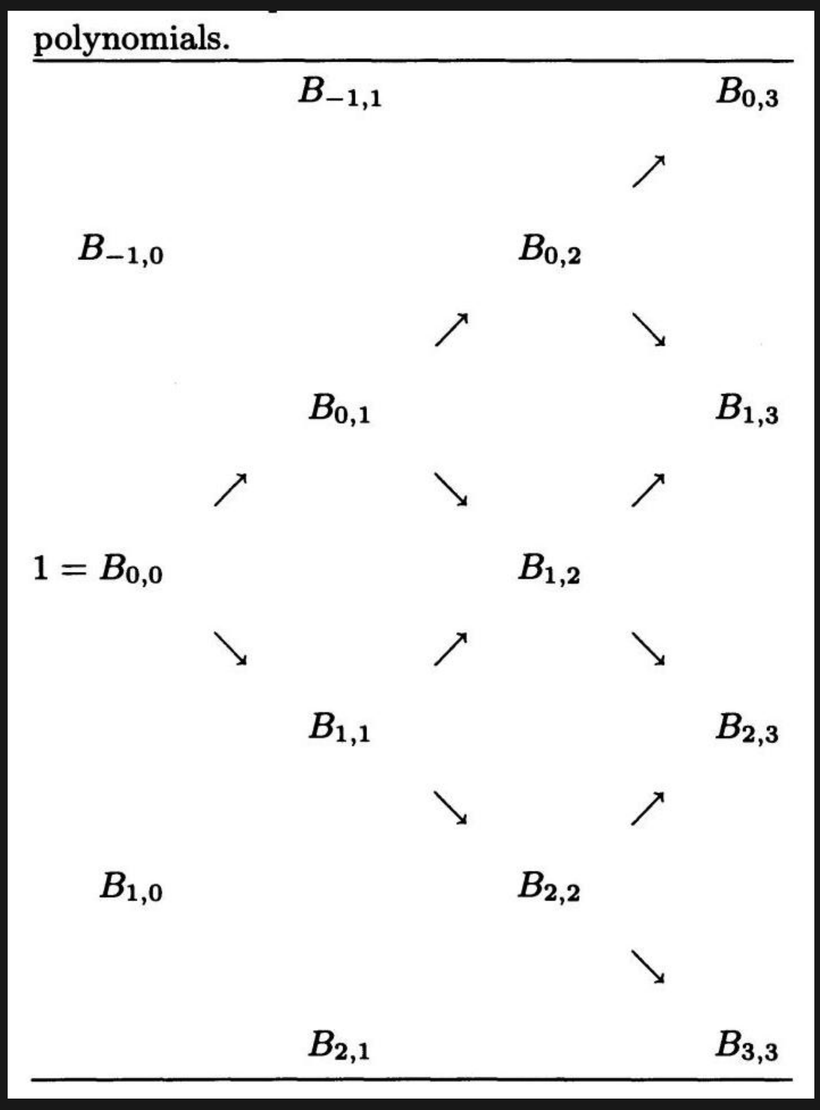


表 1.2. 所有立方伯恩斯坦的计算




```
{ /* Compute all nth-degree Bernstein polynomials. */
      /* Input: n, u */
      /* Output: B (an array, B[0],..., B[n]) */
B[0] = 1.0;
u1 = 1.0-u;
for (j=1; j<=n; j++)
      {
      saved = 0.0 ;
      for (k = 0; k < j; k++)
          {
          temp = B[k];
          B[k] = saved+u1*temp;
          saved = u*temp;
          }
      B[j] = saved;
      }
}
```

Algorithm A1.4 combines A1.3 and Eq. (1.7) to compute the point on an n th-degree Bézier curve at a fixed u value.

算法 A1.4

PointOnBezierCurve(P, n, u, C)

{ /* 计算贝塞尔曲线上的点。*/

/* 输入: P, n, u */

/* 输出: C (一个点) */

AllBernstein (n, u, B) / * B 是一个局部数组 * /

C = 0.0 ;

对于 $(k = 0; k <= n; k++)$ $C = C + B[k] * P[k]$;

}

使用性质 P1.7，可以轻松推导出 Bézier 曲线导数的一般表达式。

$$\mathbf{C}'(u) = \frac{d\left(\sum_{i=0}^{n} B_{i,n}(u)\,\mathbf{P}_i\right)}{du} = \sum_{i=0}^{n} B'_{i,n}(u)\,\mathbf{P}_i$$

$$= \sum_{i=0}^{n} n\left(B_{i-1,n-1}(u) - B_{i,n-1}(u)\right)\mathbf{P}_i$$

$$= n\sum_{i=0}^{n-1} B_{i,n-1}(u)\left(\mathbf{P}_{i+1} - \mathbf{P}_i\right) \tag{1.9}$$

从公式 (1.9) 我们可以轻松获得 Bézier 曲线的端点导数公式，例如。

$$\mathbf{C}'(0) = n(\mathbf{P}_1 - \mathbf{P}_0)\ \ \mathbf{C}''(0) = n(n-1)(\mathbf{P}_0 - 2\mathbf{P}_1 + \mathbf{P}_2)$$

$$\mathbf{C}'(1) = n(\mathbf{P}_n - \mathbf{P}_{n-1})\ \ \mathbf{C}''(1) = n(n-1)(\mathbf{P}_n - 2\mathbf{P}_{n-1} + \mathbf{P}_{n-2}) \tag{1.10}$$

从公式 (1.9) 和 (1.10) 注意到:

- 一个 n 次 Bézier 曲线的导数是一个 (n - 1) 次 Bézier 曲线；
- 在 $u = 0$ 和 $u = 1$ 处的端点导数表达式是对称的 (当然，这是由于基函数的对称性)；
- 在一个端点的 k 次导数仅依赖于该端点的 $k + 1$ 控制点 (以几何上非常直观的方式)。

设 $n = 2$ 和 $\mathbf{C}(u) = \sum_{i=0}^{2} B_{i,2}(u)\,\mathbf{P}_i$ 。那么

$$\mathbf{C}(u) = (1-u)^2\mathbf{P}_0 + 2u(1-u)\,\mathbf{P}_1 + u^2\mathbf{P}_2$$

$$= (1-u)\left(\underbrace{(1-u)\,\mathbf{P}_0 + u\mathbf{P}_1}_{\text{linear}}\right) + u\left(\underbrace{(1-u)\,\mathbf{P}_1 + u\mathbf{P}_2}_{\text{linear}}\right)$$

因此，$\mathbf{C}(u)$ 是两个一阶 Bézier 曲线的线性插值；特别地，$\mathbf{C}(u)$ 上的任何点都是通过三次线性插值得到的。

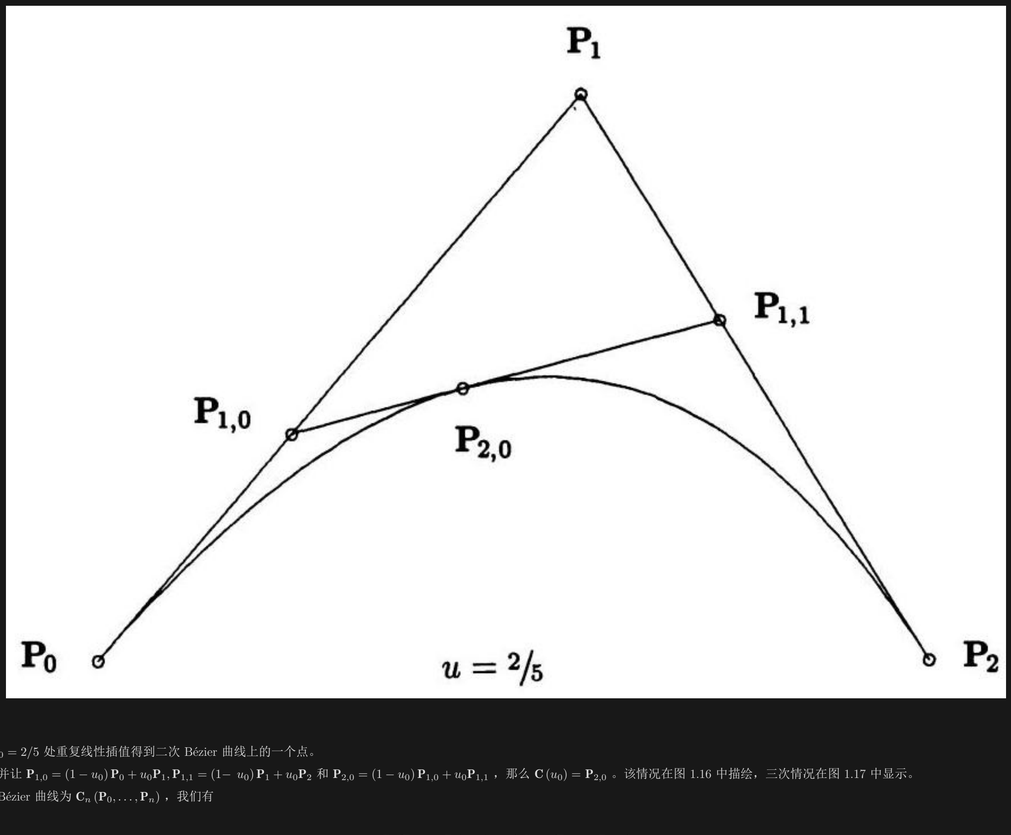


图 1.16. 通过在 $u_0 = 2/5$ 处重复线性插值得到二次 Bézier 曲线上的一个点。

假设固定 $u = u_0$ 并让 $\mathbf{P}_{1,0} = (1-u_0)\,\mathbf{P}_0 + u_0\mathbf{P}_1, \mathbf{P}_{1,1} = (1-u_0)\,\mathbf{P}_1 + u_0\mathbf{P}_2$ 和 $\mathbf{P}_{2,0} = (1-u_0)\,\mathbf{P}_{1,0} + u_0\mathbf{P}_{1,1}$，那么 $\mathbf{C}(u_0) = \mathbf{P}_{2,0}$。该情况在图 1.16 中描绘，三次情况在图 1.17 中显示。

表示一般的 n 阶 Bézier 曲线为 $\mathbf{C}_n(\mathbf{P}_0, \ldots, \mathbf{P}_n)$，我们有

$$\mathbf{C}_n(\mathbf{P}_0, \ldots, \mathbf{P}_n) = (1-u)\,\mathbf{C}_{n-1}(\mathbf{P}_0, \ldots, \mathbf{P}_{n-1})$$

$$+u\mathbf{C}_{n-1}(\mathbf{P}_1, \ldots, \mathbf{P}_n) \tag{1.11}$$

这源于基函数的递归定义 (见 P1.6)。固定 $u = u_0$ 并将 $\mathbf{P}_i$ 表示为 $\mathbf{P}_{0,i}$，方程 (1.11) 产生了计算 $\mathbf{C}(u_0) = \mathbf{P}_{n,0}(u_0)$ 在 n 阶 Bézier 曲线上的递归算法，即

$$\mathbf{P}_{k,i}(u_0) = (1-u_0)\,\mathbf{P}_{k-1,i}(u_0) + u_0\mathbf{P}_{k-1,i+1}(u_0) \quad \text{for} \quad \begin{cases} k = 1, \ldots, n \\ i = 0, \ldots, n-k \end{cases} \tag{1.12}$$

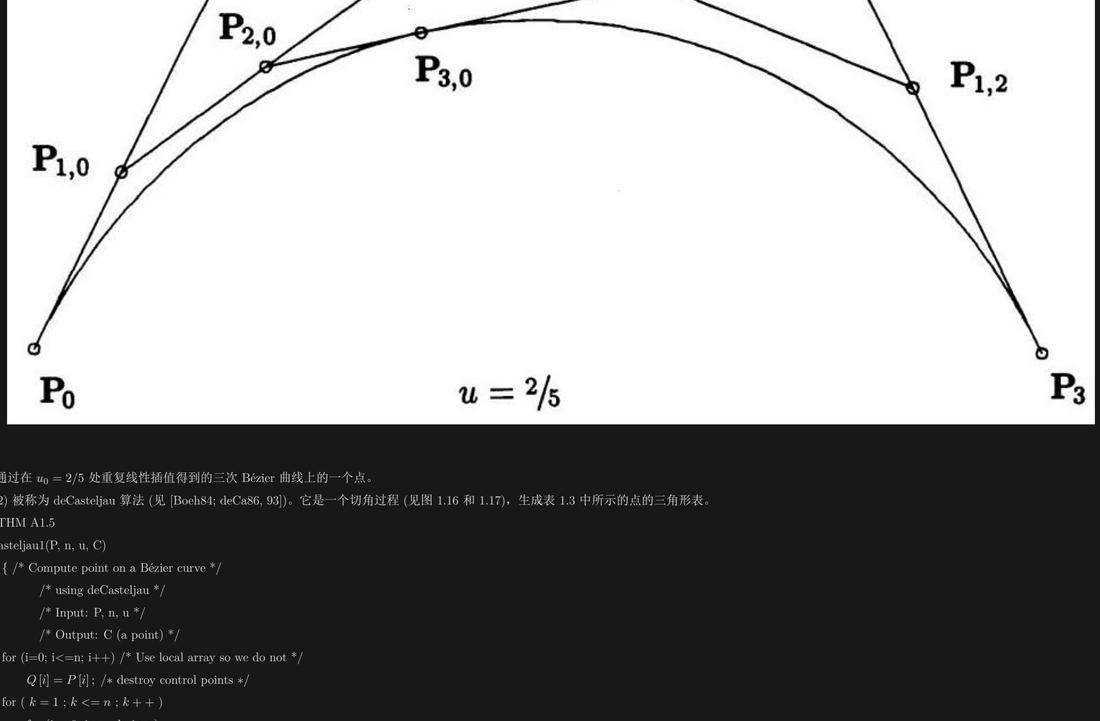


图 1.17. 通过在 $u_0 = 2/5$ 处重复线性插值得到的三次 Bézier 曲线上的一个点。

方程 (1.12) 被称为 deCasteljau 算法 (见 [Boeh84; deCa86, 93])。它是一个切角过程 (见图 1.16 和 1.17)，生成表 1.3 中所示的点的三角形表。

ALGORITHM A1.5

```
deCasteljau1(P, n, u, C)
    { /* Compute point on a Bézier curve */
        /* using deCasteljau */
        /* Input: P, n, u */
        /* Output: C (a point) */
    for (i=0; i<=n; i++) /* Use local array so we do not */
        Q[i] = P[i]; /* destroy control points */
    for ( k = 1 ; k <= n ; k + + )
        for (i = 0; i<=n-k; i++)
            Q[i] = (1.0 − u) ∗ Q[i] + u ∗ Q[i + 1];
    C = Q[0];
    }
```

我们以 Bézier 方法和幂基方法的比较结束本节。显然，Bézier 形式在两者中更具几何性。方程 (1.10) 结合凸包和变差减小特性，使得

表 1.3. 由 deCasteljau 算法生成的点。

$\mathbf{P}_0$

$\mathbf{P}_{1,0}$

$\mathbf{P}_1$ $\mathbf{P}_{2,0}$

$\mathbf{P}_{1,1}$

$\mathbf{P}_2$

$\mathbf{P}_{n-1,0}$

$\vdots\ \vdots\ \cdots\ \mathbf{P}_{n,0} = \mathbf{C}(u_0)$

$\mathbf{P}_{n-1,1}$

$\mathbf{P}_{n-2}$

$\mathbf{P}_{1,n-2}$

$\mathbf{P}_{n-1}$ $\mathbf{P}_{2,n-2}$

$\mathbf{P}_{1,n-1}$

$\mathbf{P}_n$

Bézier 曲线更适合于交互式曲线设计。控制点为设计师提供了比幂基系数更直观的曲线形状把握。此外，deCasteljau 算法比霍纳算法更不容易受到舍入误差的影响。当考虑到 deCasteljau 算法仅仅是在曲线附近的点之间进行重复线性插值时，这一点是直观清晰的。Bézier 形式的唯一缺点是点评估效率较低 (见算法 A1.1、A1.4 和 A1.5，以及本章后面的练习 1.13)。

4 有理 Bézier 曲线

接下来，我们介绍有理曲线和齐次坐标的概念。为了说明这些概念，我们简要介绍有理贝塞尔曲线。这些曲线是有理 B 样条曲线的特例，因此在后续章节中将更全面和严格地处理。

尽管多项式提供了许多优势，但存在一些重要的曲线和曲面类型无法用多项式精确表示，例如圆、椭圆、双曲线、圆柱、圆锥、球体等。作为一个例子，我们证明单位圆在 xy 平面中，以原点为中心，无法用多项式坐标函数表示。相反，假设

$$x(u) = a_0 + a_1u + \cdots + a_nu^n$$

$$y(u)=b_0+b_1u+\cdots+b_nu^n$$

然后 $x^2+y^2-1=0$ 意味着

$$0=\left(a_0+a_1u+\cdots+a_nu^n\right)^2+\left(b_0+b_1u+\cdots+b_nu^n\right)^2-1$$

$$=\left(a_0^2+b_0^2-1\right)+2\left(a_0a_1+b_0b_1\right)u+\left(a_1^2+2a_0a_2+b_1^2+2b_0b_2\right)u^2$$

$$+\cdots+\left(a_{n-1}^2+2a_{n-2}a_n+b_{n-1}^2+2b_{n-2}b_n\right)u^{2n-2}$$

$$+2\left(a_na_{n-1}+b_nb_{n-1}\right)u^{2n-1}+\left(a_n^2+b_n^2\right)u^{2n}$$

这个方程必须对所有 u 成立，这意味着所有系数都是零。从最高次开始逐步推导，我们在 n 步骤中证明所有 $a_i=0$ 和 $b_i=0$ 对于 $1\leq i\leq n$ 。

步骤

1. $a_n^2+b_n^2=0$ 意味着 $a_n=b_n=0$ 。

2. $a_{n-1}^2+2a_{n-2}a_n+b_{n-1}^2+2b_{n-2}b_n=0$ 和步骤 1 意味着 $a_{n-1}^2+b_{n-1}^2=0$ ，这意味着 $a_{n-1}=b_{n-1}=0$ 。

n. $a_1^2+2a_0a_2+b_1^2+2b_0b_2=0$ 和步骤 $n-1$ 意味着 $a_1^2+b_1^2=0$ ，这意味着 $a_1=b_1=0$ 。

因此，$x(u)=a_0$ 和 $y(u)=b_0$ ，这是一个明显的矛盾。

从经典数学中已知，所有的圆锥曲线，包括圆，都可以用有理函数表示，有理函数被定义为两个多项式的比率。实际上，它们是用有理函数表示的形式为

$$x(u)=\frac{X(u)}{W(u)}\,y(u)=\frac{Y(u)}{W(u)}\tag{1.13}$$

其中 $X(u),Y(u)$ 和 $W(u)$ 是多项式，也就是说，每个坐标函数具有相同的分母。

示例

Ex1.8 半径为 1 的圆，中心位于原点

$$x(u)=\frac{1-u^2}{1+u^2}\,y(u)=\frac{2u}{1+u^2}$$

Ex1.9 椭圆，中心位于原点；y 轴是主轴，x 轴是次轴，主半径和次半径分别为 2 和 1

$$x(u)=\frac{1-u^2}{1+u^2}\,y(u)=\frac{4u}{1+u^2}$$

Ex1.10 双曲线，中心在 $\mathbf{P}=(0,4/3)$ ； y 轴是横轴

$$x(u)=\frac{-1+2u}{1+2u-2u^2}\,y(u)=\frac{4u(1-u)}{1+2u-2u^2}$$

下支 (顶点在 $\mathbf{P}=(0,2/3)$) 的轨迹为

$$u\in\left(\frac{1-\sqrt{3}}{2},\frac{1+\sqrt{3}}{2}\right)$$

Ex1.11 抛物线，顶点在原点；y 轴是对称轴

$$x(u)=u\,y(u)=u^2$$

注意，抛物线不需要有理函数。读者应绘制这些函数。对于圆的方程，很容易看出，对于任何 $u,(x(u),y(u))$ 都位于以原点为中心的单位圆上

$$(x(u))^2+(y(u))^2=\left(\frac{1-u^2}{1+u^2}\right)^2+\left(\frac{2u}{1+u^2}\right)^2$$

$$=\frac{1-2u^2+u^4+4u^2}{\left(1+u^2\right)^2}=\frac{\left(1+u^2\right)^2}{\left(1+u^2\right)^2}=1$$

定义一个 n 次有理贝塞尔曲线 (见 [Forr68; Pieg86; Fari83,89])

$$\mathbf{C}(u)=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i\mathbf{P}_i}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}\;0\leq u\leq1\tag{1.14}$$

$\mathbf{P}_i=(x_i,y_i,z_i)$ 和 $B_{i,n}(u)$ 与之前相同；w_i 是标量，称为权重。因此，$W(u)=\sum_{i=0}^{n}B_{i,n}(u)\,w_i$ 是公共分母函数。除非另有明确说明，否则我们假设对于所有 i 有 $w_i>0$ 。这确保了对于所有 $u\in[0,1]$ 有 $W(u)>0$ 。我们写

$$\mathbf{C}(u)=\sum_{i=0}^{n}R_{i,n}(u)\,\mathbf{P}_i\;0\leq u\leq1\tag{1.15}$$

其中

$$R_{i,n}(u)=\frac{B_{i,n}(u)\,w_i}{\sum_{j=0}^{n}B_{j,n}(u)\,w_j}$$

$R_{i,n}(u)$ 是该曲线形式的有理基函数。图 1.18a 显示了三次基函数的一个示例，图 1.18b 显示了相应的三次有理 Bézier 曲线。

$R_{i,n}(u)$ 具有可以很容易从方程 (1.15) 及相应的 $B_{i,n}(u)$ 的性质推导出的特性:

P1.8 非负性: 对所有 i,n 和 $0\leq u\leq1$ ，都有 $R_{i,n}(u)\geq0$ ；

P1.9 统一划分: 对所有 $0\leq u\leq1$ ，都有 $\sum_{i=0}^{n}R_{i,n}(u)=1$ ；

P1. $10R_{0,n}(0)=R_{n,n}(1)=1$ ；

P1.11 $R_{i,n}(u)$ 在区间 $[0,1]$ 上恰好达到一个最大值；

P1.12 如果对所有 i 都有 $w_i=1$ ，那么对所有 i 都有 $R_{i,n}(u)=B_{i,n}(u)$ ；即，$B_{i,n}(u)$ 是 $R_{i,n}(u)$ 的一个特例。

这些结果得出了有理 Bézier 曲线的以下几何特性:

P1.13 凸包性质: 曲线包含在其定义控制点 ($\mathbf{P}_i$) 的凸包内；

P1.14 变换不变性: 通过对控制点应用旋转、平移和缩放来对曲线进行变换；

P1.15 变差减小性质: 与多项式 Bézier 曲线相同 (见前一节)；

P1.16 端点插值: $\mathbf{C}(0)=\mathbf{P}_0$ 和 $\mathbf{C}(1)=\mathbf{P}_n$ ；

P1.17 在 $u=0\,(u=1)$ 处的 k 阶导数依赖于第一个 (最后一个) $k+1$ 控制点和权重；特别地，$\mathbf{C}'(0)$ 和 $\mathbf{C}'(1)$ 分别与 $\mathbf{P}_1-\mathbf{P}_0$ 和 $\mathbf{P}_n-\mathbf{P}_{n-1}$ 平行；

P1.18 多项式贝塞尔曲线是有理贝塞尔曲线的一个特例。

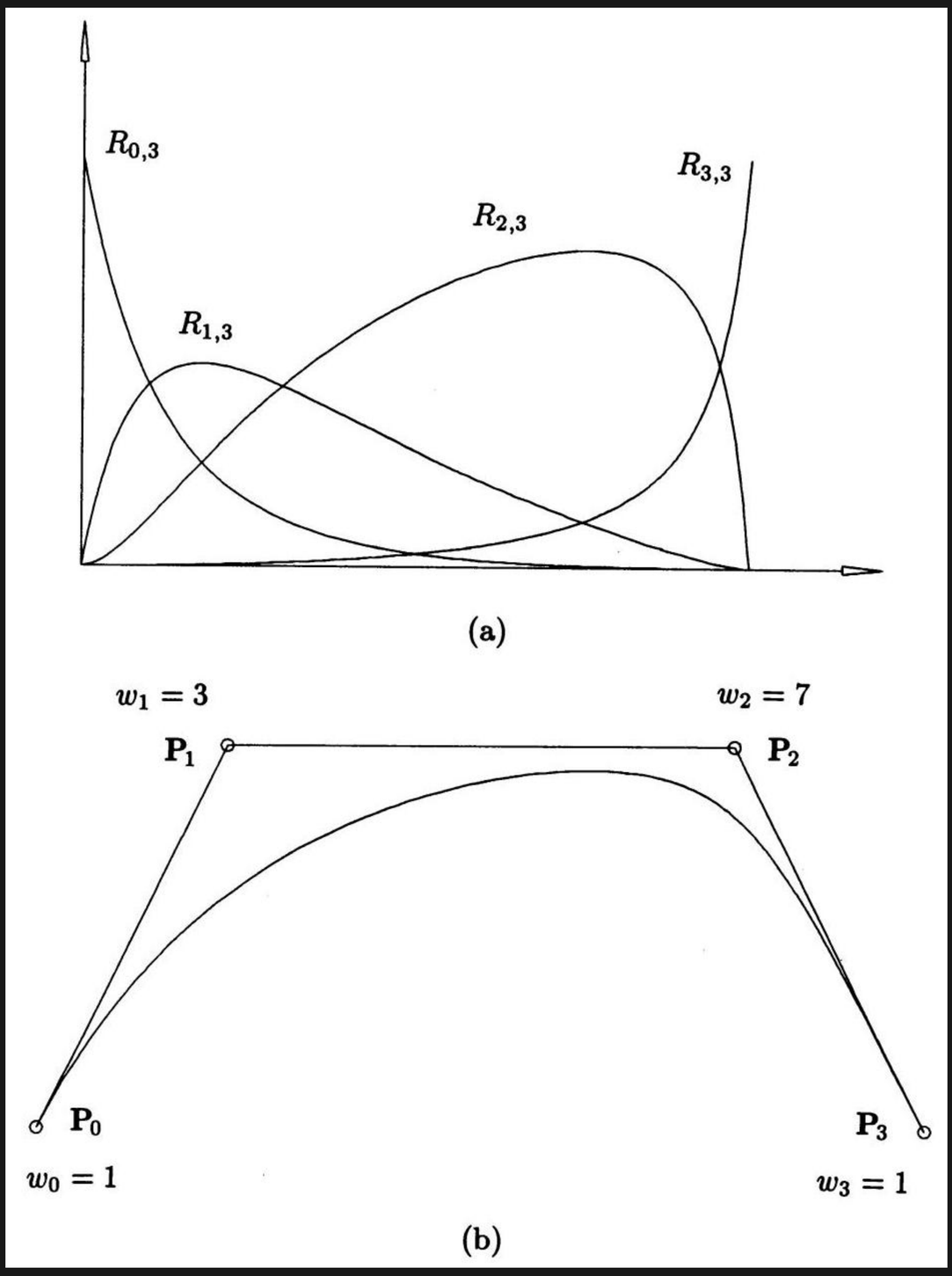


图 1.18. 有理三次曲线。(a) 基函数；(b) 贝塞尔曲线。

示例

Ex1.12 让我们考虑有理贝塞尔圆弧。

$$\mathbf{C}(u) = (x(u), y(u)) = \left(\frac{1-u^2}{1+u^2}, \frac{2u}{1+u^2}\right) \quad 0 \le u \le 1$$

表示单位圆的一个象限，如图 1.19a 所示。我们现在推导这个圆弧的二次有理贝塞尔表示。显然，从 P1.16 和 P1.17 可得 $\mathbf{P}_0 = (1,0), \mathbf{P}_1 = (1,1)$ 和 $\mathbf{P}_2 = (0,1)$ 。对于权重，我们有

$$W(u) = 1 + u^2 = \sum_{i=0}^{2} B_{i,2}(u)\, w_i = (1-u)^2 w_0 + 2u(1-u)\, w_1 + u^2 w_2$$

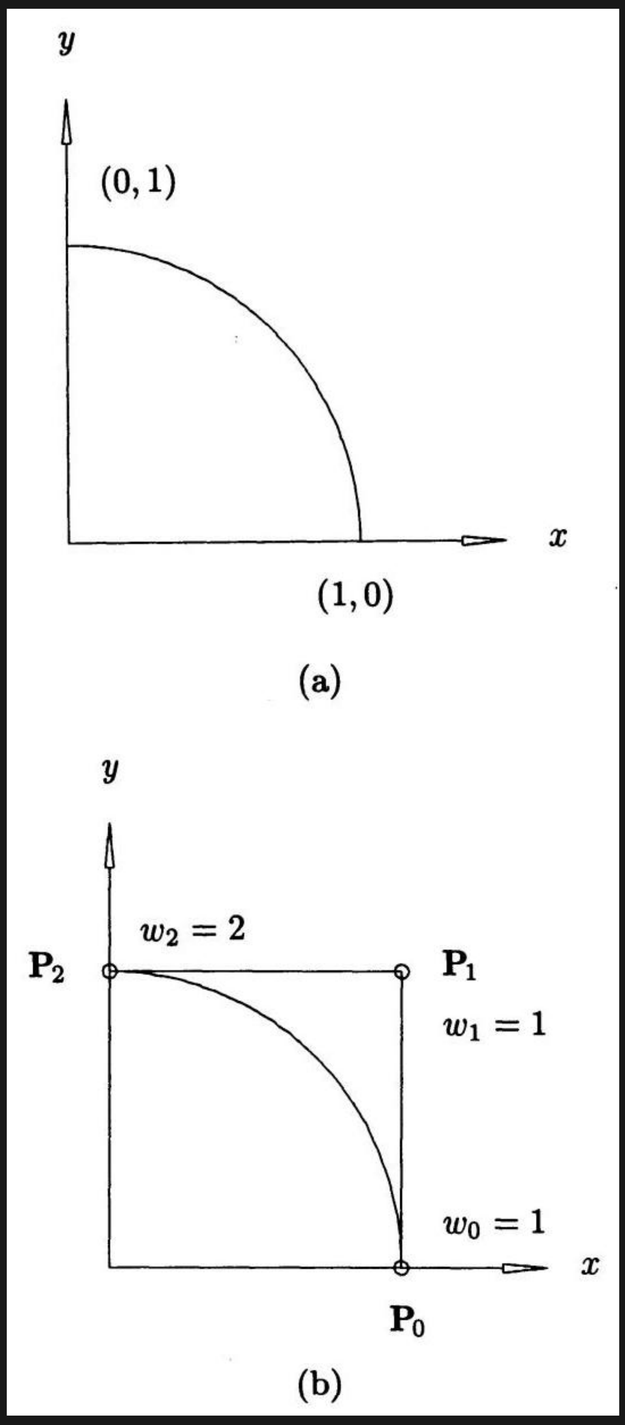


图 1.19. 单位圆的表示。(a) 一个象限的 $x(u) = (1-u^2)/(1+u^2)$ 和 $y(u) = (2u)/(1+u^2)$；(b) 与图 1.19a $(w_0 = 1, w_1 = 1, w_2 = 2)$ 对应的贝塞尔表示。

代入 $u = 0$ 得到 $w_0 = 1$，代入 $u = 1$ 得到 $w_2 = 2$。最后，代入 $u = 1/2$ 得到 $5/4 = 1/4w_0 + 1/2w_1 + 1/4w_2$，并使用 $w_0 = 1$ 和 $w_2 = 2$ 得到 $w_1 = 1$ (见图 1.19b)。

具有形式为 Eq. (1.13)(一个共同分母) 的坐标函数的有理曲线具有优雅的几何解释，这种解释能够实现高效处理和紧凑的数据存储。其思想是使用齐次坐标将有理曲线表示为 n 维空间中的多项式曲线 (见 [Robe65; Ries81; Patt85])。我们从三维欧几里得空间中的一个点 $\mathbf{P} = (x, y, z)$ 开始。然后 $\mathbf{P}$ 在四维空间 $w \neq 0$ 中写作 $\mathbf{P}^w = (wx, wy, wz, w) = (X, Y, Z, W)$。现在

$\mathbf{P}$ 是通过将所有坐标除以第四个坐标 W 得到的，即通过将 $\mathbf{P}^w$ 从原点映射到超平面 $W = 1$ (见图 1.20 的二维情况，$\mathbf{P} = (x, y)$))。该映射，记作 H，是一个以原点为中心的透视映射。

$$\mathbf{P} = H\{\mathbf{P}^w\} = H\{(X, Y, Z, W)\} = \begin{cases} \left(\frac{X}{W}, \frac{Y}{W}, \frac{Z}{W}\right) & \text{if } W \neq 0 \\ \text{direction } (X, Y, Z) & \text{if } W = 0 \end{cases} \tag{1.16}$$

注意，对于任意的 x, y, z, w_1, w_2，其中 $w_1 \neq w_2$

$$H\{\mathbf{P}^{w_1}\} = H\{(w_1 x, w_1 y, w_1 z, w_1)\} = (x, y, z)$$

$$= H\{(w_2 x, w_2 y, w_2 z, w_2)\} = H\{\mathbf{P}^{w_2}\}$$

现在，对于给定的一组控制点 $\{\mathbf{P}_i\}$ 和权重 $\{w_i\}$，构造加权控制点 $\mathbf{P}_i^w = (w_i x_i, w_i y_i, w_i z_i, w_i)$。然后在四维空间中定义非有理 (多项式)Bézier 曲线。

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} B_{i,n}(u)\,\mathbf{P}_i^w \tag{1.17}$$

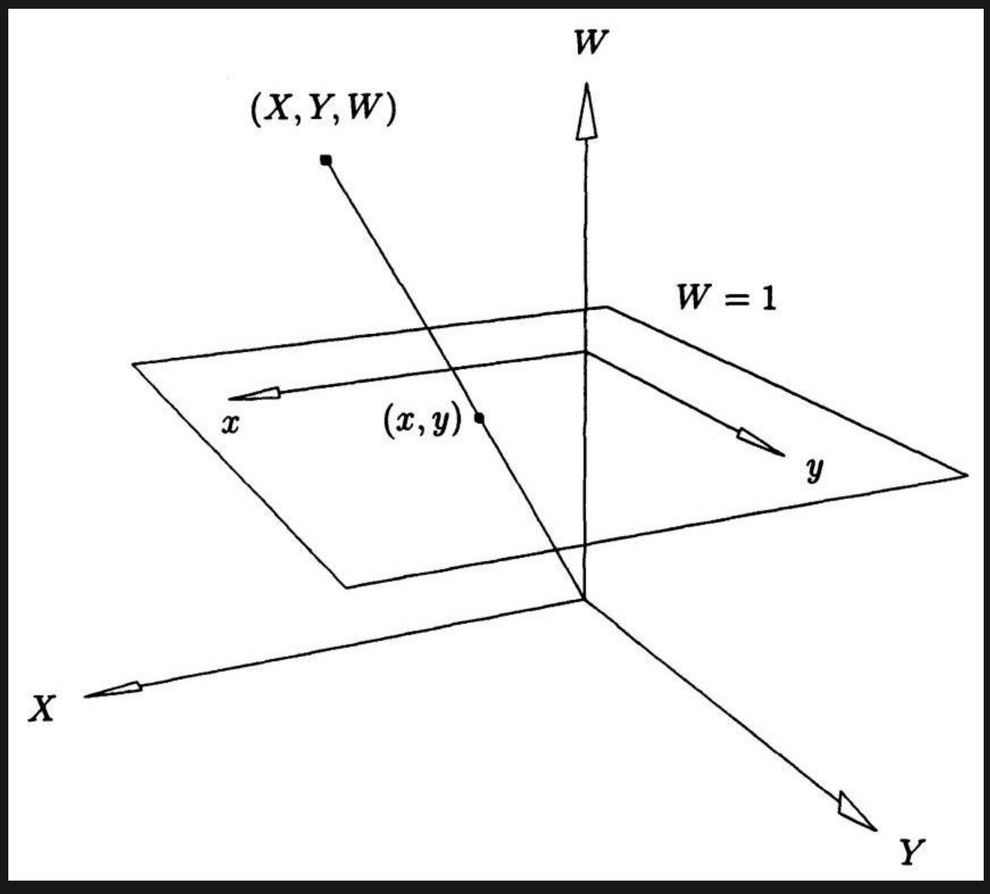


图 1.20. 齐次形式下的欧几里得点的表示。

然后，将透视映射 H 应用于 $\mathbf{C}^w(u)$ ，得到 Eq. (1.14) 的相应有理 Bézier 曲线 (见图 1.21)，即写出 Eq. (1.17) 的坐标函数，我们得到

$$X(u)=\sum_{i=0}^{n}B_{i,n}(u)\,w_i x_i \quad Y(u)=\sum_{i=0}^{n}B_{i,n}(u)\,w_i y_i$$

$$Z(u)=\sum_{i=0}^{n}B_{i,n}(u)\,w_i z_i \quad W(u)=\sum_{i=0}^{n}B_{i,n}(u)\,w_i$$

在三维空间中定位曲线得到

$$x(u)=\frac{X(u)}{W(u)}=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i x_i}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}$$

$$y(u)=\frac{Y(u)}{W(u)}=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i y_i}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}$$

$$z(u)=\frac{Z(u)}{W(u)}=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i z_i}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}$$

使用向量表示法，我们得到

$$\begin{aligned}\mathbf{C}(u)=(x(u),y(u),z(u))&=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i\,(x_i,y_i,z_i)}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}\\&=\frac{\sum_{i=0}^{n}B_{i,n}(u)\,w_i\mathbf{P}_i}{\sum_{i=0}^{n}B_{i,n}(u)\,w_i}\end{aligned}\tag{1.18}$$

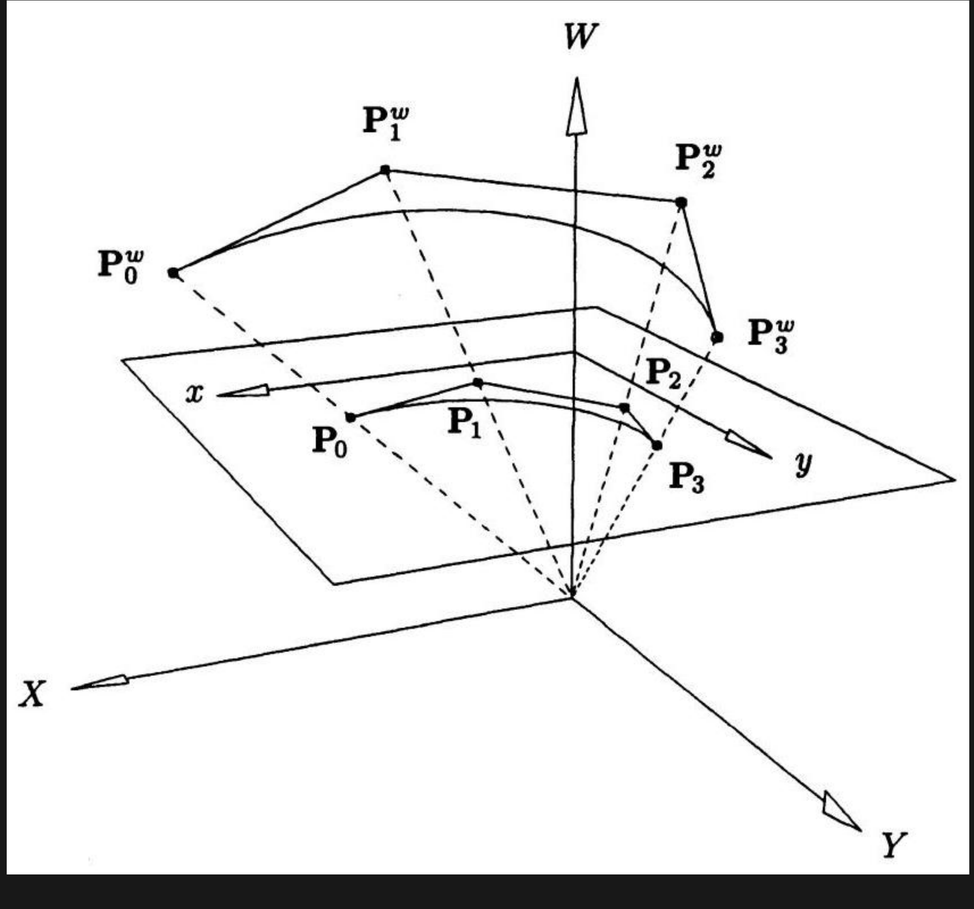


图 1.21. 有理 Bézier 曲线的几何构造。

本书中的算法主要使用由方程 (1.17) 给出的形式，以及有理 B 样条曲线的类似形式。因此，无理形式在四维空间中处理，结果通过映射 H 定位于三维空间中。我们可以交替地将 $\mathbf{C}^w(u)$ 或 $\mathbf{C}(u)$ 称为有理 Bézier(或 B 样条) 曲线，尽管严格来说，$\mathbf{C}^w(u)$ 不是一条有理曲线。

示例

示例 1.13 让我们回到图 1.19b 中的圆弧。我们有 $\mathbf{P}_0 = (1,0)$ 、$\mathbf{P}_1 = (1,1), \mathbf{P}_2 = (0,1)$ 和 $w_0 = 1, w_1 = 1, w_2 = 2$ 。因此，对于方程 (1.17)，三维控制点为 $\mathbf{P}_0^w = (1,0,1)$， $\mathbf{P}_1^w = (1,1,1)$ 和 $\mathbf{P}_2^w = (0,2,2)$ 。然后 $\mathbf{C}^w(u) = (1-u)^2\mathbf{P}_0^w + 2u(1-u)\mathbf{P}_1^w + u^2\mathbf{P}_2^w$ 是一条抛物线弧 (无理)，它在 $W = 1$ 平面上投影为一条圆弧 (见图 1.22)。

设 u_0 为固定值。由于 $\mathbf{C}^w(u)$ 是一条多项式 Bézier 曲线，我们使用 deCasteljau 算法计算 $\mathbf{C}^w(u_0)$ ；随后，得到 $\mathbf{C}(u_0) = H\{\mathbf{C}^w(u_0)\}$ 。因此，我们将方程 (1.12) 应用于 $\mathbf{P}_i^w$ 。

$$\mathbf{P}_{k,i}^w(u_0) = (1-u_0)\mathbf{P}_{k-1,i}^w + u_0\mathbf{P}_{k-1,i+1}^w \text{ for } \begin{cases} k = 1,\ldots,n \\ i = 0,\ldots,n-k \end{cases} \tag{1.19}$$

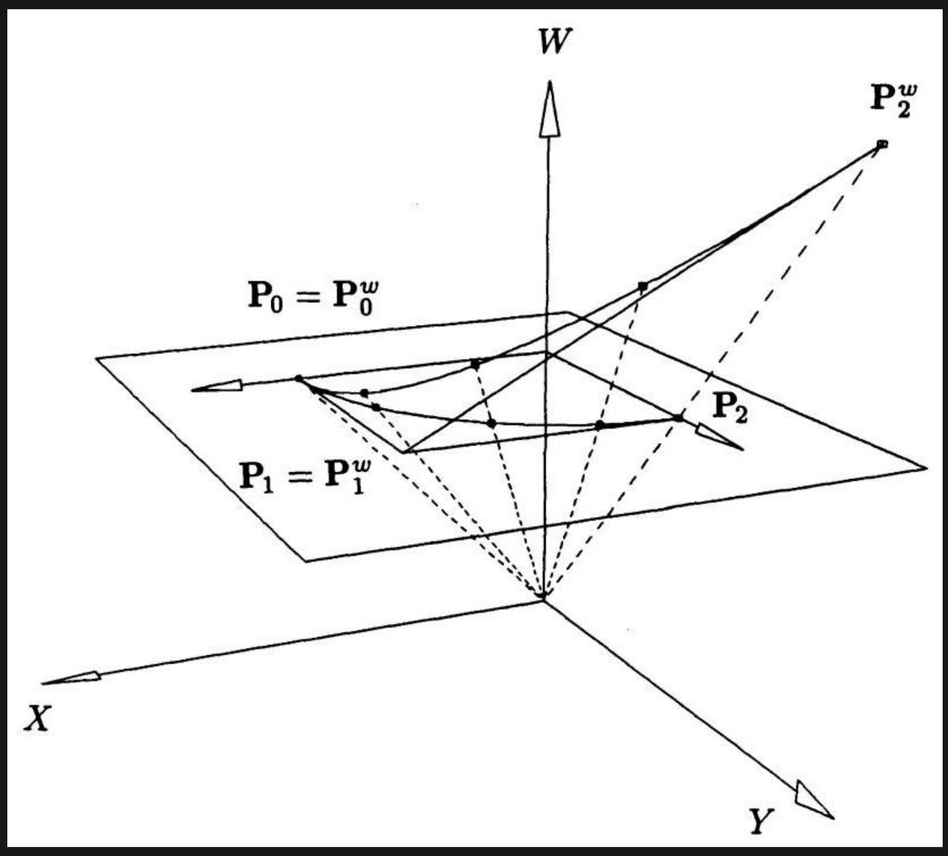


图 1.22. 圆弧的齐次表示。

示例 1.14 让我们应用方程 (1.19) 来计算示例 1.13 中有理 Bézier 圆弧在 $u = 1/2$ 处的点。该弧由 $\mathbf{C}^w(u) = (1-u)^2\mathbf{P}_0^w + 2u(1-u)\mathbf{P}_1^w + u^2\mathbf{P}_2^w$ 给出，其中 $\mathbf{P}_0^w = (1,0,1), \mathbf{P}_1^w = (1,1,1), \mathbf{P}_2^w = (0,2,2)$ 。生成的点的三角形集合如表 1.4 所示。然后 $\mathbf{C}(1/2) = H\{\mathbf{C}^w(1/2)\} = H\{(3/4,1,5/4)\} = (3/5,4/5)$ 。

现在让我们使用我们开发的其他表示法来计算该点。设

$$\mathbf{C}(u) = \left(\frac{(1-u^2)}{(1+u^2)}, \frac{(2u)}{(1+u^2)}\right)$$

表 1.4. 在圆弧上生成点 $\mathbf{C}^w(1/2)$ 。

(1,0,1)	
	$\left(1, \frac{1}{2}, 1\right)$
$(1,1,1)$ $\left(\frac{3}{4}, 1, \frac{5}{4}\right) = \mathbf{C}^w\left(\frac{1}{2}\right)$	
	$\left(\frac{1}{2}, \frac{3}{2}, \frac{3}{2}\right)$
(0,2,2)	

然后

$$\mathbf{C}\left(\frac{1}{2}\right) = \left(\frac{1-\left(\frac{1}{2}\right)^2}{1+\left(\frac{1}{2}\right)^2}, \frac{2\left(\frac{1}{2}\right)}{1+\left(\frac{1}{2}\right)^2}\right) = \left(\frac{3}{5}; \frac{4}{5}\right)$$

使用公式 (1.17)

$$\mathbf{C}^w\left(\frac{1}{2}\right) = \sum_{i=0}^{2} B_{i,2}\left(\frac{1}{2}\right)\mathbf{P}_i^w =$$

$$= \left(1-\left(\frac{1}{2}\right)\right)^2 (1,0,1) + 2\left(\frac{1}{2}\right)\left(1-\left(\frac{1}{2}\right)\right)(1,1,1)$$

$$+\left(\frac{1}{2}\right)^2 (0,2,2)$$

$$= \frac{1}{4}(1,0,1) + \frac{1}{2}(1,1,1) + \frac{1}{4}(0,2,2) = \left(\frac{3}{4}, 1, \frac{5}{4}\right)$$

投影得到 $(3/5, 4/5)$ 。公式 (1.18) 和 (1.15) 得到相同的结果。

最后，我们注意到 $\mathbf{C}(1/2) = (3/5, 4/5)$ 不是第一象限圆弧的中点；即，参数化不是均匀的 (见第 1.1 节)。点 $(3/5, 4/5)$ 离起始点的弧长超过一半。这在直观上是正确的，因为通过对 $\mathbf{C}(u)$ 进行微分，可以看出起始速度是结束速度的两倍。

5 张量积曲面

曲线 $\mathbf{C}(u)$ 是一个单参数的向量值函数。它是将一段直线段映射 (变形) 到欧几里得三维空间中的一种方式。表面是一个双参数的向量值函数 u 和 v ，表示将 uv 平面上的一个区域 $\mathcal{R}$ 映射到欧几里得三维空间中的一种方式。因此，它具有形式 $\mathbf{S}(u,v) = (x(u,v), y(u,v), z(u,v)), (u,v) \in \mathcal{R}$ 。表示表面的方法有很多种 (见 [Hosc93; Pieg89a, 93] 及 [Pieg89a] 中引用的众多参考文献)。它们在使用的坐标函数和区域类型 $\mathcal{R}$ 上有所不同。可能最简单的方法，也是几何建模应用中最广泛使用的方法，是张量积方案。这是我们在本书其余部分中使用的方法。

张量积方法基本上是一种双向曲线方案。它使用基函数和几何系数。基函数是 u 和 v 的双变量函数，构造为单变量基函数的乘积。

几何系数在一个双向的 $n \times m$ 网中 (拓扑上) 排列。因此，张量积曲面具有以下形式

$$\mathbf{S}(u,v) = (x(u,v), y(u,v), z(u,v)) = \sum_{i=0}^{n}\sum_{j=0}^{m} f_i(u) g_j(v) \mathbf{b}_{i,j} \tag{1.20}$$

其中

$$\begin{cases} \mathbf{b}_{i,j} = (x_{i,j}, y_{i,j}, z_{i,j}) \\ 0 \le u, v \le 1 \end{cases}$$

注意，这个映射的 (u, v) 域是一个正方形 (一般来说是一个矩形)。还要注意 $\mathbf{S}(u,v)$ 具有矩阵形式

$$\mathbf{S}(u,v) = [f_i(u)]^T [\mathbf{b}_{i,j}] [g_j(v)]$$

其中 $[f_i(u)]^T$ 是一个 $(1) \times (n+1)$ 行向量，$[g_j(v)]$ 是一个 $(m+1) \times (1)$ 列向量，而 $[\mathbf{b}_{i,j}]$ 是一个三维点的 $(n+1) \times (m+1)$ 矩阵。

作为一个例子，我们考虑幂基曲面

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} \mathbf{a}_{i,j} u^i v^j = [u]^T [\mathbf{a}_{i,j}] [v^j] \quad \begin{cases} \mathbf{a}_{i,j} = (x_{i,j}, y_{i,j}, z_{i,j}) \\ 0 \le u, v \le 1 \end{cases} \tag{1.21}$$

我们有 $f_i(u) = u^i$ 和 $g_j(v) = v^j$ ，基函数是乘积 $\{u^i v^j\}$ 。如果我们固定 $u = u_0$ ，那么

$$\mathbf{C}_{u_0}(v) = \mathbf{S}(u_0, v) = \sum_{j=0}^{m}\left(\sum_{i=0}^{n} \mathbf{a}_{i,j} u_0^i\right) v^j = \sum_{j=0}^{m} \mathbf{b}_j(u_0) v^j \tag{1.22}$$

其中

$$\mathbf{b}_j(u_0) = \sum_{i=0}^{n} \mathbf{a}_{i,j} u_0^i$$

是一个位于曲面 $\mathbf{S}(u,v)$ 上的幂基曲线。$\mathbf{C}_{v_0}(u)$ 是一个位于 $\mathbf{S}(u,v)$ 上的幂基曲线；曲线 $\mathbf{C}_{u_0}(v)$ 和 $\mathbf{C}_{v_0}(u)$ 在曲面点 $\mathbf{S}(u_0, v_0)$ 处相交。这些曲线称为等参数曲线 (或等曲线)。$\mathbf{C}_{u_0}(v)$ 被称为 v 曲线，$\mathbf{C}_{v_0}(u)$ 是 u 曲线 (见图 1.23)。

方程 (1.21) 可以写成

$$\begin{aligned}\mathbf{S}(u,v) = &\left\{\underbrace{\mathbf{a}_{0,0}+\mathbf{a}_{0,1}v+\mathbf{a}_{0,2}v^2+\cdots+\mathbf{a}_{0,m}v^m}_{\mathbf{b}_0}\right\}\\ &+u\left\{\underbrace{\mathbf{a}_{1,0}+\mathbf{a}_{1,1}v+\mathbf{a}_{1,2}v^2+\cdots+\mathbf{a}_{1,m}v^m}_{\mathbf{b}_1}\right\}\\ &+u^2\left\{\underbrace{\mathbf{a}_{2,0}+\mathbf{a}_{2,1}v+\mathbf{a}_{2,2}v^2+\cdots+\mathbf{a}_{2,m}v^m}_{\mathbf{b}_2}\right\}\\ &\vdots\\ &+u^n\left\{\underbrace{\mathbf{a}_{n,0}+\mathbf{a}_{n,1}v+\mathbf{a}_{n,2}v^2+\cdots+\mathbf{a}_{n,m}v^m}_{\mathbf{b}_n}\right\}\\ &=\mathbf{b}_0+\mathbf{b}_1u+\mathbf{b}_2u^2+\cdots+\mathbf{b}_nu^n\end{aligned}$$

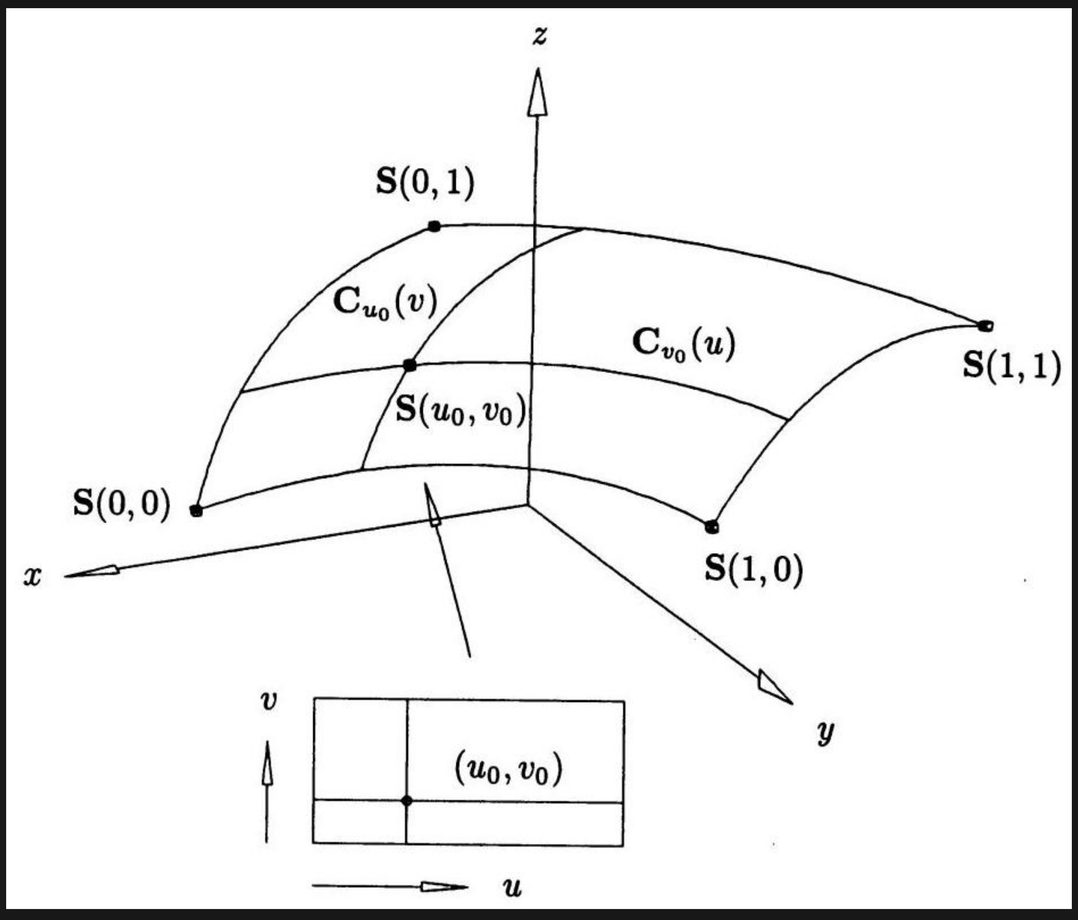


图 1.23. 显示等参数曲线的张量积曲面。

大括号中的项是可以通过霍纳算法 (A1.1) 评估的简单多项式，得到 $\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_n$ 。使用基数并重新应用该算法，我们获得曲面上的点。因此，我们有算法 A1.6。

ALGORITHM A1.6

```
Horner2(a, n, u0, v0, S)
    { /* Compute point on a power basis surface. */
        /* Input: a, n, m, u0, v0 */
        /* Output: S */
      for (i = 0; i<=n; i++)
          Horner1(a[i][], m, v0, b[i]); /* a[i][] is the ith row */
      Horner1(b, n, u0, S);
    }
```

算法 A1.6 是张量积曲面的典型算法。它们通常可以通过从曲线算法扩展而来，通常通过在一个方向上处理 n (或 m) 系数的行 (作为曲线)，然后在另一个方向上处理一行或多行。

对方程 (1.21) 进行求导，我们得到

$$\mathbf{S}_u(u,v) = \sum_{i=1}^{n}\sum_{j=0}^{m} i\mathbf{a}_{i,j}u^{i-1}v^j \quad \mathbf{S}_v(u,v) = \sum_{i=0}^{n}\sum_{j=1}^{m} j\mathbf{a}_{i,j}u^iv^{j-1}$$

注意，对于固定的 (u_0, v_0) , $\mathbf{S}_u(u_0, v_0) = \mathbf{C}'_{v_0}(u_0)$ 和 $\mathbf{S}_v(u_0, v_0) = \mathbf{C}'_{u_0}(v_0)$ 。法向量 $\mathbf{N}$ 是使用方程 (1.4) 计算的。

非理性贝塞尔曲面是通过取控制点的双向网格和单变量伯恩斯坦多项式的乘积获得的。

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u)\, B_{j,m}(v)\, \mathbf{P}_{i,j} \quad 0 \le u, v \le 1 \tag{1.23}$$

基函数 $B_{0,2}(u)\, B_{1,3}(v)$ 如图 1.24a 所示，图 1.24b 显示了一个二次 × 三次贝塞尔曲面。

对于固定的 $u = u_0$

$$\begin{aligned}\mathbf{C}_{u_0}(v) = \mathbf{S}(u_0,v) &= \sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u_0)\, B_{j,m}(v)\, \mathbf{P}_{i,j}\\ &= \sum_{j=0}^{m} B_{j,m}(v)\left(\sum_{i=0}^{n} B_{i,n}(u_0)\, \mathbf{P}_{i,j}\right)\\ &= \sum_{j=0}^{m} B_{j,m}(v)\, \mathbf{Q}_j(u_0)\end{aligned} \tag{1.24}$$

其中 $\mathbf{Q}_j(u_0) = \sum_{i=0}^{n} B_{i,n}(u_0)\, \mathbf{P}_{i,j}\ j = 0, \ldots, m$

是一个位于表面上的贝塞尔曲线。类似地，$\mathbf{C}_{v_0}(u) = \sum_{i=0}^{n} B_{i,n}(u)\, \mathbf{Q}_i(v_0)$ 是一个位于表面上的贝塞尔 u 等值线。

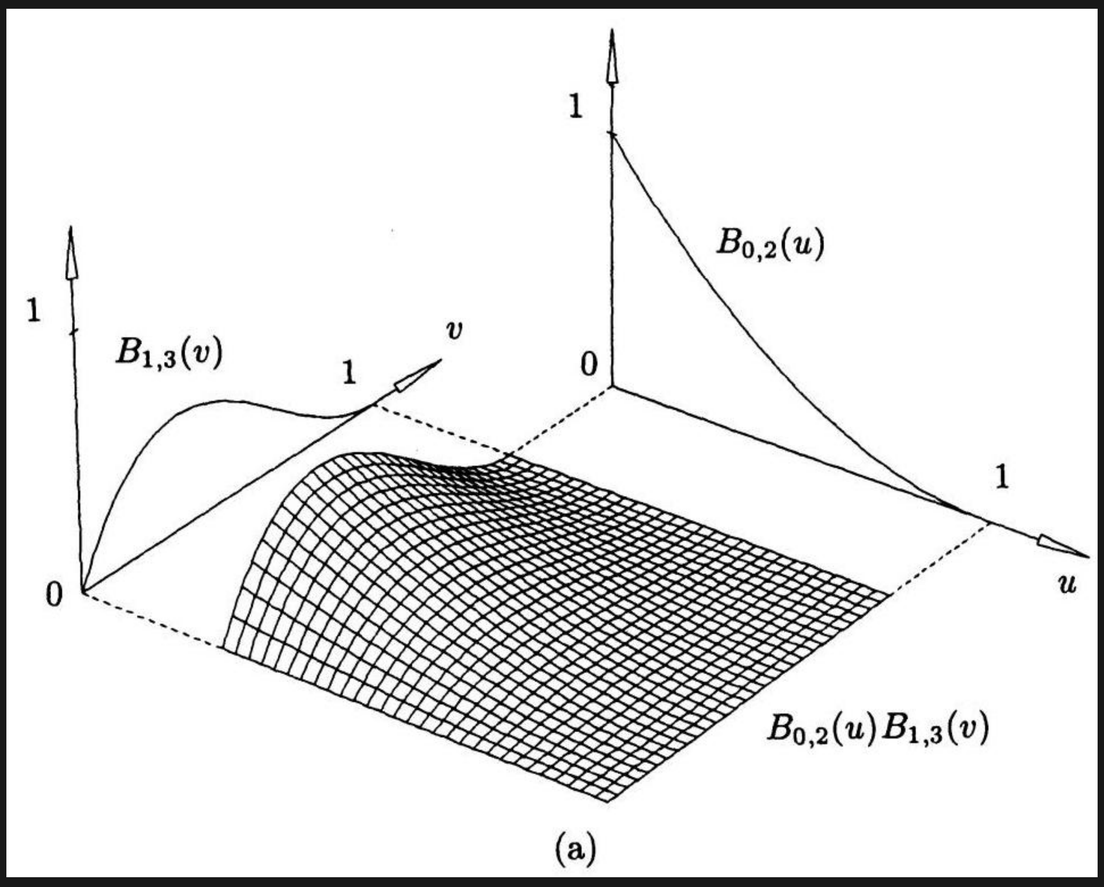


图 1.24. (a) 贝塞尔张量积基函数，$B_{0,2}(u)B_{1,3}(v)$；(b) 二次 × 三次贝塞尔曲面。

与曲线的情况一样，由于其优良的特性，贝塞尔曲面比幂基曲面更适合几何建模应用。特别是，

- 非负性: $B_{i,n}(u)B_{j,m}(v) \geq 0$ 对所有 i, j, u, v 成立；
- 单位分割: $\sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u)B_{j,m}(v) = 1$ 对所有 u 和 v 适用；
- $\mathbf{S}(u, v)$ 包含在其控制点的凸包内；
- 变换不变性；
- 表面插值四个角控制点；
- 当进行三角剖分时，控制网形成表面的平面多面体近似。

有趣的是，目前尚无已知的贝塞尔曲面变差性质 (见 [Prau92])。

deCasteljau 算法 (A1.5) 也可以轻松扩展以计算贝塞尔曲面上的点。参见方程 (1.24) 和图 1.25。设 (u_0, v_0) 为固定值。对于固定的 $j_0, \mathbf{Q}_{j_0}(u_0) = \sum_{i=0}^{n} B_{i,n}(u_0)\mathbf{P}_{i,j_0}$，通过将 deCasteljau 算法应用于 j_0 行的控制点，即 $\{\mathbf{P}_{i,j_0}\}, i = 0, \ldots, n$，可以得到点。因此，将 deCasteljau 算法应用 $(m+1)$ 次得到 $\mathbf{C}_{u_0}(v)$；再将其应用一次于 $\mathbf{C}_{u_0}(v)$ 在 $v = v_0$ 处得到 $\mathbf{C}_{u_0}(v_0) = \mathbf{S}(u_0, v_0)$。这个过程需要

$$\frac{n(n+1)(m+1)}{2} + \frac{m(m+1)}{2} \tag{1.25}$$

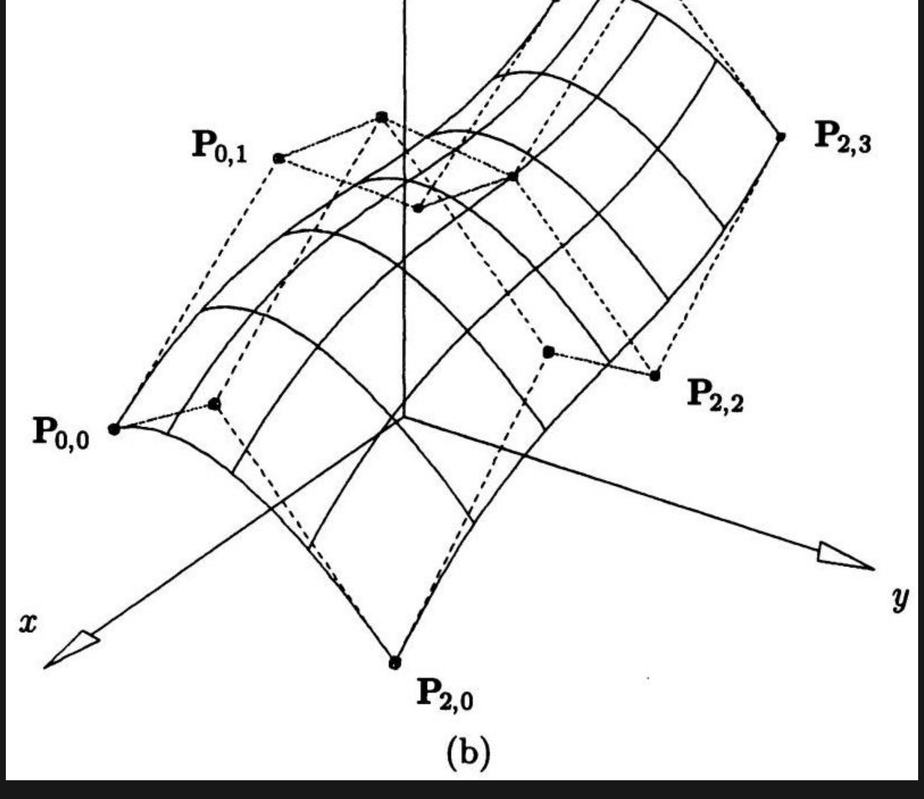


图 1.24. (续)

线性插值 (见练习 1.21)。通过对称性，我们可以先计算 $\mathbf{C}_{v_0}(u)$ ($n+1$ 次 deCasteljau 应用)，然后计算 $\mathbf{C}_{v_0}(u_0) = \mathbf{S}(u_0, v_0)$。

这需要

$$\frac{m(m+1)(n+1)}{2} + \frac{n(n+1)}{2} \tag{1.26}$$

线性插值。因此，如果 $n > m$ 先计算 $\mathbf{C}_{v_0}(u)$，则 $\mathbf{C}_{v_0}(u_0)$；否则，先计算 $\mathbf{C}_{u_0}(v)$，然后 $\mathbf{C}_{u_0}(v_0)$。

ALGORITHM A1.7

```
deCasteljau2(P, n, m, u0, v0, S)
  { /* Compute a point on a Bézier surface */
      /* by the deCasteljau. */
      /* Input: P, n, m, u0, v0 */
```

```
                /* Output: S
        if (n <= m)
            {
            for (j=0; j<=m; j++) /* P[j][] is jth row */
                deCasteljau1(P[j][], n, u0, Q[j]);
            deCasteljau1(Q, m, v0, S);
            }
          else
            {
            for (i = 0; i<=n; i++)
```

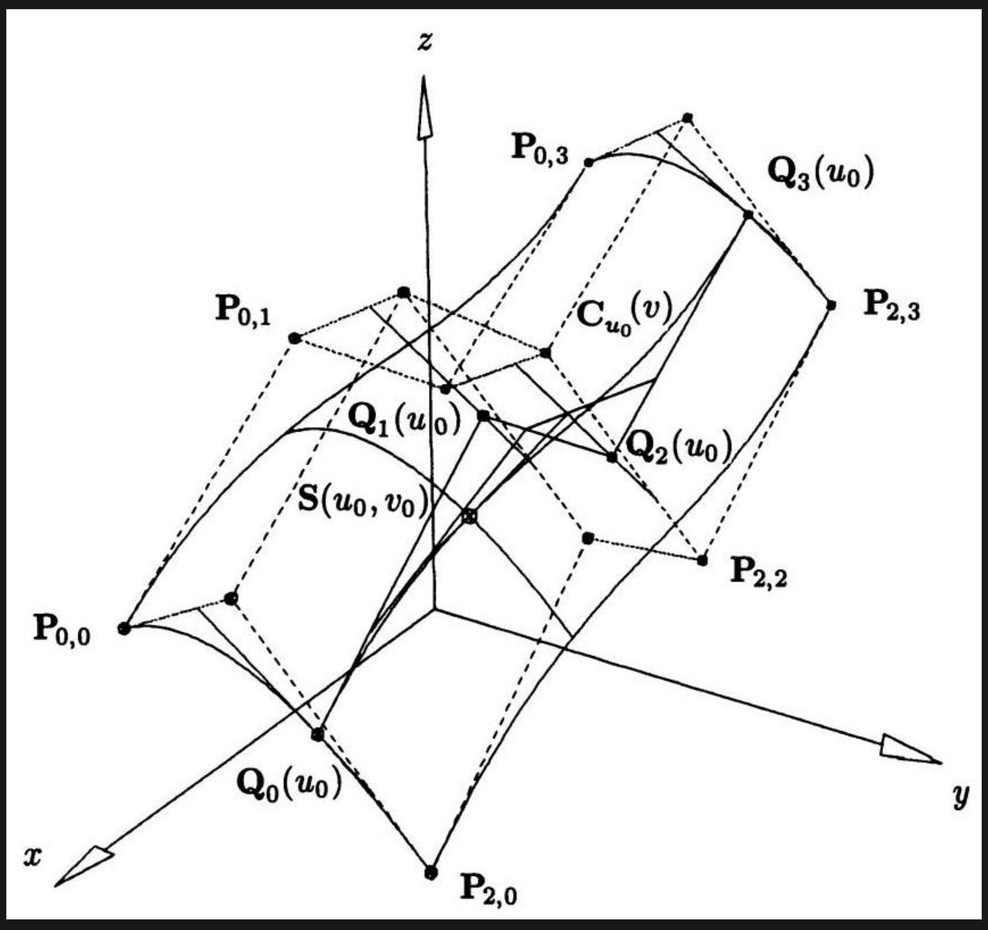


图 1.25. deCasteljau 算法用于 Bézier 曲面。

```
        deCasteljau1(P[][i], m, v0, Q[i]);
    deCasteljau1(Q, n, u0, S);
    }
}
```

我们定义有理 Bézier 曲面为四维多项式 Bézier 曲面的透视投影 (见 [Pieg86; Fari89])。

$$\mathbf{S}^w(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u)\, B_{j,m}(v)\, \mathbf{P}_{i,j}^w \tag{1.27}$$

和

$$\mathbf{S}(u,v) = H\left\{\mathbf{S}^w(u,v)\right\} = \frac{\sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u)\, B_{j,m}(v)\, w_{i,j}\mathbf{P}_{i,j}}{\sum_{i=0}^{n}\sum_{j=0}^{m} B_{i,n}(u)\, B_{j,m}(v)\, w_{i,j}}$$

$$= \sum_{i=0}^{n}\sum_{j=0}^{m} R_{i,j}(u,v)\, \mathbf{P}_{i,j} \tag{1.28}$$

where

$$R_{i,j}(u,v) = \frac{B_{i,n}(u)\, B_{j,m}(v)\, w_{i,j}}{\sum_{r=0}^{n}\sum_{s=0}^{m} B_{r,n}(u)\, B_{s,m}(v)\, w_{r,s}}$$

注意到 $R_{i,j}(u,v)$ 是有理函数，但它们不是其他基函数的乘积。因此，$\mathbf{S}(u,v)$ 不是张量积表面，而 $\mathbf{S}^w(u,v)$ 是。与曲线一样，我们通常使用方程 (1.27) 并对结果进行投影。图 1.26a 显示了一个有理基函数，图 1.26b 描绘了一个二次 × 三次有理 Bézier 表面。将这些图与图 1.24a 和 1.24b 进行比较。

假设 $w_{i,j} > 0$ 对所有 i 和 j 成立，之前列出的非有理 Bézier 表面的性质 (以及乘积函数 $B_{i,n}(u)\, B_{j,m}(v)$) 自然扩展到有理 Bézier 表面。此外，如果 $w_{i,j} = 1$ 对所有 i 和 j 成立，则 $R_{i,j}(u,v) = B_{i,n}(u)\, B_{j,m}(v)$，相应的表面是非有理的。

示例

Ex1.15 让我们构造一个圆柱面片。从第 1.4 节我们知道

该

$$\mathbf{C}^w(u) = \sum_{i=0}^{2} B_{i,2}(u)\, \mathbf{P}_i^w$$

对于 $\{\mathbf{P}_i^w\} = \{(0,1,0,1), (0,1,1,1), (0,0,2,2)\}$，是在 yz 平面中的一个圆弧。使用平移 (P1.14，第 1.4 节)

$$\mathbf{C}_0^w(u) = \sum_{i=0}^{2} B_{i,2}(u)\, \mathbf{P}_{i,0}^w \quad \text{and} \quad \mathbf{C}_1^w(u) = \sum_{i=0}^{2} B_{i,2}(u)\, \mathbf{P}_{i,1}^w$$

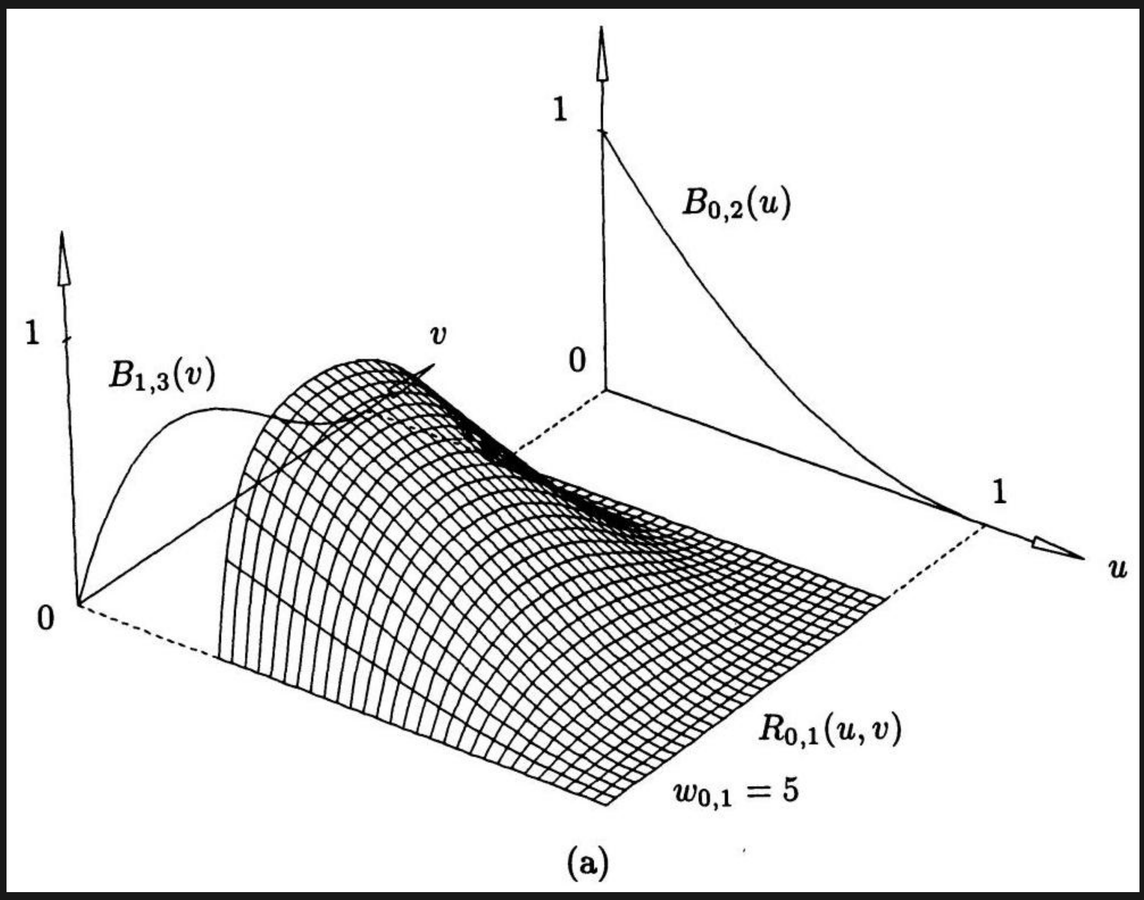


图 1.26. (a) 有理基函数 $R_{0,1}(u,v)$ (其中 $w_{0,1}=5$ 和所有其他权重均为零)；(b) 二次 × 三次有理 Bézier 曲面。

在哪里

$$\left\{\mathbf{P}_{i,0}^{w}\right\}=\{(1,1,0,1),(1,1,1,1),(2,0,2,2)\}$$

和

$$\left\{\mathbf{P}_{i,1}^{w}\right\}=\{(-1,1,0,1),(-1,1,1,1),(-2,0,2,2)\}$$

是分别位于 $x=1$ 和 $x=-1$ 平面中的圆弧（见图 1.27）。在 $\mathbf{C}_{0}^{w}$ 和 $\mathbf{C}_{1}^{w}$ 之间进行线性插值会产生一个圆柱面，即

$$\mathbf{S}^{w}(u,v)=\sum_{i=0}^{2}\sum_{j=0}^{1}B_{i,2}(u)\,B_{j,1}(v)\,\mathbf{P}_{i,j}^{w}$$

对于固定的 $u=u_0,\mathbf{C}_{u_0}^{w}(v)=\sum_{j=0}^{1}B_{j,1}(v)\,\mathbf{Q}_{j}^{w}(u_0)$ ，是从 $\mathbf{C}_{0}^{w}(u_0)$ 到 $\mathbf{C}_{1}^{w}(u_0)$ 的一条平行于 x 轴的直线段。对于固定的 $v=v_0$ ，$\mathbf{C}_{v_0}^{w}=\mathbf{S}^{w}(u,v_0)=\sum_{i=0}^{2}B_{i,2}(u)\,\mathbf{Q}_{i}^{w}(v_0)$ 是平面 $x=(1-v_0)(1)+v_0(-1)=1-2v_0$ 中的一个圆弧。现在让我们使用算法 A1.7 计算点 $\mathbf{S}(1/2,1/2)$ 。注意 $n>m$ 。首先获得 $C_{v_0=1/2}^{w}(u)$

$$(1,1,0,1)$$

$$(0,1,0,1)=\mathbf{Q}_{0}^{w}(v_0)$$

(-1,1,0,1) (1,1,1,1)

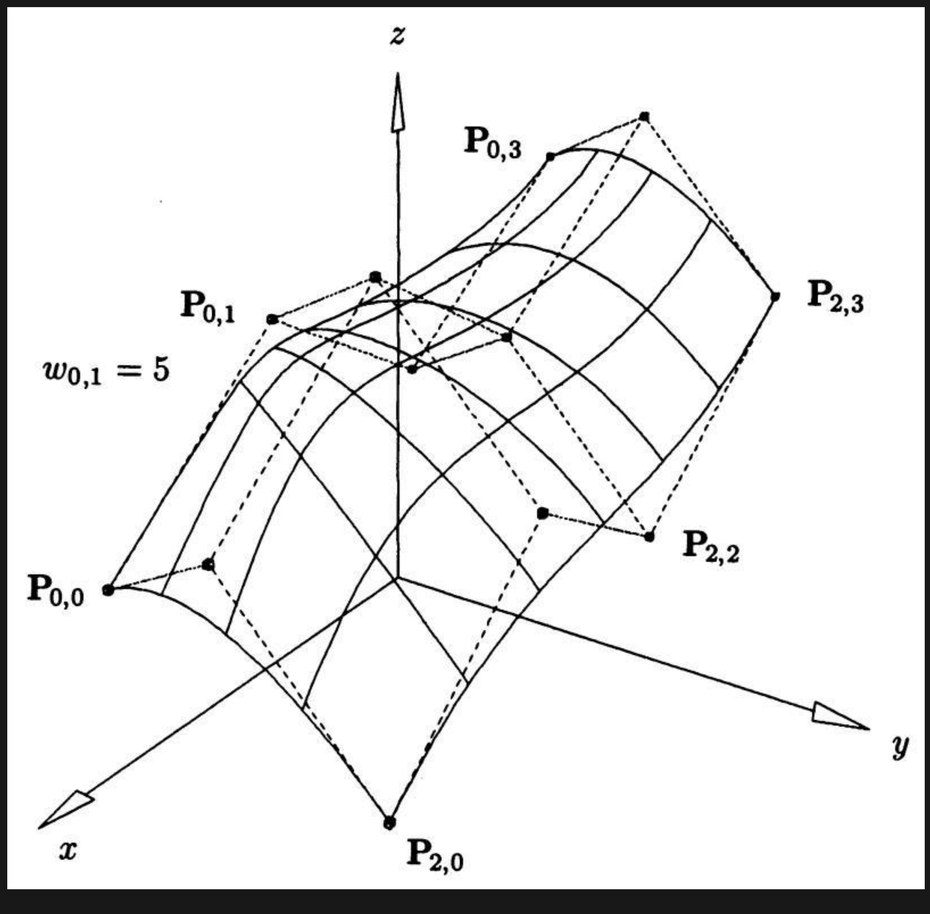
(-1,1,1,1)

$$(0,1,1,1)=\mathbf{Q}_{1}^{w}(v_0)$$

(2,0,2,2)

$$(-2,0,2,2)$$

$$(0,0,2,2)=\mathbf{Q}_{2}^{w}(v_0)$$



(b)

图 1.26. (b) (续)

现在 $\mathbf{C}^w_{v_0=1/2}(u)=\sum_{i=0}^{2}B_{i,2}(u)\,\mathbf{Q}^w_i(v_0)$ 是 yz 平面中的圆弧
平面。然后
(0,1,0,1)

$$\left(0,1,\frac{1}{2},1\right)$$

$$\left(0,\frac{3}{4},1,\frac{5}{4}\right)=\mathbf{S}^w\left(\frac{1}{2},\frac{1}{2}\right)$$

$$\left(0,\frac{1}{2},\frac{3}{2},\frac{3}{2}\right)$$

$$(0,0,2,2)$$

投影得到

$$\mathbf{S}\left(\frac{1}{2},\frac{1}{2}\right)=H\left\{\left(0,\frac{3}{4},1,\frac{5}{4}\right)\right\}=\left(0,\frac{3}{5},\frac{4}{5}\right)$$

练习 43

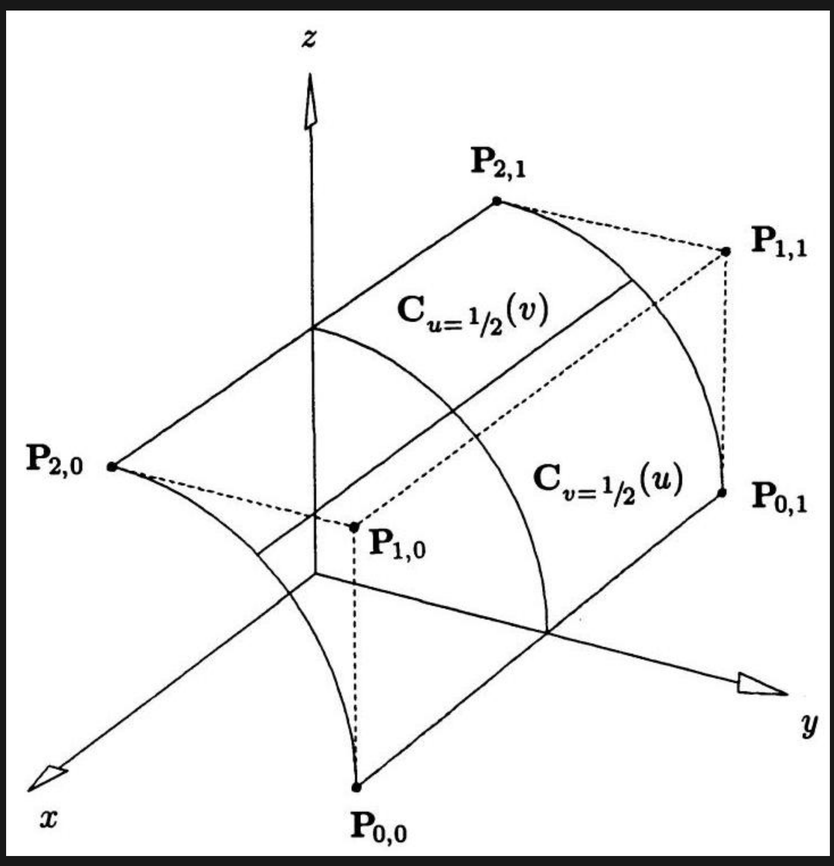


图 1.27. 作为有理 Bézier 曲面的圆柱面片。

练习

1.1. 考虑由方程 (1.1) 和 (1.2) 给出的圆弧的两个参数表示。使用方程 (1.1) 计算曲线在 $u=\pi/4$ 的点，并使用方程 (1.2) 计算在 $t=1/2$ 的点。解释结果。

1.2. 计算方程 (1.1) 的加速度向量 $\mathbf{C}''(u)$ 。解释结果。

1.3. 使用三角函数，给出有界表面的参数定义

- 一个以原点为顶点，轴对称沿 z 轴的直圆锥；
- 该圆锥向上打开，并在 $z=2$ 处被半径为 $=1$ 的圆界定。

修改方程 (1.2) 以获得同一圆锥的另一表示。计算三角表示的第一个偏导数 $\mathbf{S}_u$ 和 $\mathbf{S}_v$ 。这些导数在圆锥顶点处的值是多少？

1.4. 考虑抛物线弧 $\mathbf{C}(u)=(x(u),y(u))=\left(-1-u+2u^2,-2u+u^2\right)$ ，$0<u<1$ 。绘制此曲线。通过对函数 $x(u)$ 和 $y(u)$ 应用变换来旋转和平移该曲线。应用这两个变换
(1) 90° 关于原点的旋转。旋转矩阵 (从左侧应用) 为

$$\begin{bmatrix}0&-1\\1&0\end{bmatrix}$$

(2) 使用向量 (-1, -1) 进行平移。
底层抛物线的隐式方程为 $x^2-4xy+4y^2-4x-y-5=0$ 。绘制此曲线。将先前的旋转和平移应用于此方程。提示: 让 $\bar{x},\bar{y}$ 为变换后的坐标。找到表达式 $x=f(\bar{x},\bar{y})$ 和 $y=g(\bar{x},\bar{y})$ ，并将这些代入隐式方程以获得变换后抛物线的隐式方程。

1.5. 确定计算一个 n 次三维幂基曲线上的点所需的加法和乘法的公式。

1.6. 构造一个带有环的三次幂基曲线。提示: 考虑需要什么样的端点和端导数，$\mathbf{C}'(0)$ 和 $\mathbf{C}'(1)$ 。

1.7. 构造一个带有尖点的三次幂基曲线。提示: 考虑 $\mathbf{C}'(u)$ 和 $\mathbf{C}''(u)$ 。草图展示 $x'(u),y'(u),x''(u)$ 和 $y''(u)$ 作为 u 的函数应呈现的样子。确定一个合适的 $\mathbf{C}''(u)$ ，然后进行积分以获得 $\mathbf{C}'(u)$ 和 $\mathbf{C}(u)$ 。

1.8. 构造一个带有拐点的三次幂基曲线。

1.9. 设 $\mathbf{C}(u)=(x(u),y(u))=\left(1+u-2u^2+u^3,1-2u+u^3\right),-1\le u\le 1$ 。设 $u=2v-1$ 。通过在 $\mathbf{C}(u)$ 中将 $2v-1$ 替换为 u 来推导曲线 $\mathbf{C}(v)$ 。曲线 $\mathbf{C}(v)$ 的度数是多少？计算 $\mathbf{C}(u)$ 对于 $u=-1,0,1$ 。计算 $\mathbf{C}(v)$ 对于 $v=0,1/2,1$ 。你对曲线 $\mathbf{C}(u)$ 和 $\mathbf{C}(v)$ 有什么看法？$\mathbf{C}(v)$ 被称为 $\mathbf{C}(u)$ 的重新参数化。

1.10. 检查伯恩斯坦多项式在 $n=2$ 和 $n=3$ 情况下的性质 P1.7。

1.11. 有时需要反转一条曲线，即，给定 $\mathbf{C}_1(u),0\le u\le 1$ ，生成 $\mathbf{C}_2(v),0\le v\le 1$ ，使得这两条曲线在几何上是相同的，但 $\mathbf{C}_1(0)=\mathbf{C}_2(1)$ 和 $\mathbf{C}_1(1)=\mathbf{C}_2(0)$ 不同。你将如何使用贝塞尔形式来实现这一点？使用幂基形式呢？

1.12. 考虑由控制点 $\mathbf{P}_0=(0,6)$, $\mathbf{P}_1=(3,6),\mathbf{P}_2=(6,3),\mathbf{P}_3=(6,0)$ 给出的 xy 平面三次 Bézier 曲线。使用 deCasteljau 算法计算点 $\mathbf{C}(1/3)$ 。直接使用公式 (1.7) 和 (1.8) 计算相同的点，即在 $u=1/3$ 处评估基函数并乘以相应的控制点。

1.13. 确定使用 de Casteljau 算法、A1.5 和算法 A1.4 计算 n 阶三维 Bézier 曲线上的一个点所需的加法和乘法的公式。将结果与练习 1.5(霍纳算法) 进行比较。

1.14. 对于给定的 n ，行向量 $[B_{0,n}(u),\ldots,B_{n,n}(u)]$ 可以写成 $[1,u,\ldots,u^n]M$ ，其中 M 是一个 $(n+1)\times(n+1)$ 矩阵。因此，Bézier 曲线可以用矩阵形式表示为 $\mathbf{C}(u)=\left[u^i\right]^T M\left[\mathbf{P}_i\right]$ 。计算矩阵 M 对于 $n=1,2,3$ 。注意，设置 $[\mathbf{a}_i]=M\left[\mathbf{P}_i\right]$ 会将 Bézier 曲线转换为幂基形式。假设 $0\le u\le 1,[\mathbf{P}_i]=M^{-1}[\mathbf{a}_i]$ 给出了从幂基到 Bézier 形式的转换。

1.15. 将此与练习 1.9 进行比较。也可以在参数区间 $[0,1]$ 之外定义 Bézier 曲线。这相当于对 Bézier 曲线进行重新参数化。
曲线。设

$$\mathbf{C}(u)=\sum_{i=0}^{n}B_{i,n}(u)\,\mathbf{P}_i\ u\in[0,1]$$

设定 $v\in[a,b]$ 。然后 $u=(v-a)/(b-a)$ 。将这个方程代入方程 (1.8) 中，并推导出重新参数化曲线的表达式

$$\mathbf{C}(v)=\frac{1}{(b-a)^n}\sum_{i=0}^{n}\frac{n!}{i!\,(n-i)!}(v-a)^i(b-v)^{n-i}\mathbf{P}_i$$

有趣的是，控制点并没有改变，只有基函数发生了变化。幂基形式的重新参数化改变了几何系数，但没有改变基函数。

1.16. 考虑圆形

$$\mathbf{C}(u)=\left(\frac{1-u^2}{1+u^2},\frac{2u}{1+u^2}\right)$$

确定参数 u 的哪些范围产生圆的哪些象限。这些方程是否产生整个圆？你对参数化有什么看法？

1.17. 考虑以下在 xy 平面上的有理三次 Bézier 曲线: $\mathbf{P}_0=(0,6)$, $\mathbf{P}_1=(3,6),\mathbf{P}_2=(6,3),\mathbf{P}_3=(6,0),w_0=4,w_1=1,w_2=1,w_3=4$ 。通过扩展 de Casteljau 表计算点 $\mathbf{C}(2/3)$ 。

1.18. 我们使用的有理函数有什么特征使我们能够使用齐次坐标表示？这种表示有什么优势？

1.19. 找到第二象限中圆弧的有理 Bézier 表示，即确定 $\mathbf{P}_i$ 和 w_i 。提示: 使用对称性，并通过显示 $(x(u))^2+(y(u))^2=1$ 对所有 $u\in[0,1]$ 进行检查来验证你的结果。

1.20. 第一象限中的圆弧也由以下方程给出

$$\mathbf{C}(u)=\left(\frac{1+\left(\sqrt{2}-2\right)u+\left(1-\sqrt{2}\right)u^2}{1+\left(\sqrt{2}-2\right)u+\left(2-\sqrt{2}\right)u^2},\frac{\frac{\sqrt{2}}{2}u\left(\left(\sqrt{2}-2\right)u+2\right)}{1+\left(\sqrt{2}-2\right)u+\left(2-\sqrt{2}\right)u^2}\right)$$

确定与这些方程相对应的有理 Bézier 表示。提示: $\mathbf{P}_i$ 必须与之前的 $-(1,0),(1,1),(0,1)$ 相同；为什么？通过使多项式相等并替换 $u=0,1/2,1$ 来计算权重 w_i ，如之前所做。使用任何方法计算点 $\mathbf{C}(1/2)$ 。$\mathbf{C}(1/2)$ 有什么有趣之处？

1.21. 推导方程 (1.25) 和 (1.26)。提示: 使用公式 $1+2+\cdots+n=n(n+1)/2$ 。

1.22. 对于圆柱面示例 (Ex1.15)，计算等曲线 $\mathbf{C}^w_{u_0=1/3}(v)$ 的控制点 $\mathbf{Q}^w_j(u_0)$ 。

1.23. 设 $n=3$ 和 $m=2$ 。考虑由控制网定义的非有理 Bézier 曲面

控制网

$$\{\mathbf{P}_{i,0}\}=\{(0,0,0),(3,0,3),(6,0,3),(9,0,0)\}$$

$$\{\mathbf{P}_{i,1}\}=\{(0,2,2),(3,2,5),(6,2,5),(9,2,2)\}$$

$$\{\mathbf{P}_{i,2}\}=\{(0,4,0),(3,4,3),(6,4,3),(9,4,0)\}$$

a. 草绘该曲面；

b. 使用 deCasteljau 算法计算表面点 $\mathbf{S}(1/3,1/2)$ ；

c. 固定 $u_0=1/2$ 并提取曲线 $\mathbf{C}_{u_0=1/2}(v)$ 的贝塞尔表示 (控制点)。

1.24. 设

$$\mathbf{S}(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{m}B_{i,n}(u)\,B_{j,m}(v)\,\mathbf{P}_{i,j}$$

并假设 $\mathbf{P}_{0,0}=\mathbf{P}_{1,0}=\cdots=\mathbf{P}_{n,0}$ 。这如何影响 $\mathbf{S}(u,v)$ 、导数 $\mathbf{S}_u(u,v)$ 和 $\mathbf{S}_v(u,v)$ 以及曲线 $\mathbf{C}_{v_0}(u)$ ？ 假设 $\mathbf{P}_{i,0}=(1,0,0)$ 对于示例 Ex1.15，其中 $w_{0,0}=1,w_{1,0}=1$ 和 $w_{2,0}=2$ 。你得到什么类型的表面？

1.25. 这个问题的前提是练习 1.14。有理贝塞尔曲面 (方程 [1.27]) 具有矩阵形式

$$\mathbf{S}^w(u,v)=\left[B_{i,n}(u)\right]^T\left[\mathbf{P}^w_{i,j}\right]\left[B_{j,m}(v)\right]=\left[u^i\right]^T M_n\left[\mathbf{P}^w_{i,j}\right]M_m^T\left[v^j\right]$$

其中 $\left[u^i\right]^T$ 和 $\left[v^j\right]$ 是向量，M_n 是一个 $(n+1)\times(n+1)$ 矩阵，M_m^T 是一个 $(m+1)\times(m+1)$ 矩阵，而 $\left[\mathbf{P}^w_{i,j}\right]$ 是一个四维点的 $(n+1)\times(m+1)$ 矩阵。将这个形式明确写出用于圆柱面示例 Ex1.15。使用这个矩阵形式，计算点 $\mathbf{S}^w(1/2,1/2)$ ，然后投影以获得 $\mathbf{S}(1/2,1/2)$ 。对于 $\mathbf{S}(u,v)$ 没有直接的矩阵形式；为什么？

第二章　B 样条基函数距离空间

1　引言

仅由一个多项式或有理段组成的曲线通常是不够的。它们的缺点是:

- 为了满足大量约束，需要高阶；例如，(n - 1) 阶需要通过 n 数据点的多项式贝塞尔曲线。然而，高阶曲线处理效率低且数值不稳定；
- 为了准确拟合一些复杂形状，需要高阶；
- 单段曲线 (曲面) 不适合交互式形状设计；尽管贝塞尔曲线可以通过其控制点 (和权重) 进行塑形，但控制并不够局部。

解决方案是使用分段多项式或分段有理曲线 (曲面)。图 2.1 显示了一条曲线 $\mathbf{C}(u)$ ，由 $m(=3)$ n 次多项式段组成。$\mathbf{C}(u)$ 定义在 $u \in [0,1]$ 上。参数值 $u_0 = 0 < u_1 < u_2 < u_3 = 1$ 被称为断点。它们映射到三个多项式段的端点。我们用 $\mathbf{C}_i(u), 1 \le i \le m$ 表示这些段。段的构造方式使得它们在某种程度上具有连续性 (不一定在每个断点处相同)。设 $\mathbf{C}_i^{(j)}$ 表示 j 次导数，如果对于所有 $0 \le j \le k$ ，在断点 u_i 处是 C^k 连续的。

任何标准多项式形式都可以用来表示 $\mathbf{C}_i(u)$ 。图 2.2 显示了图 2.1 的曲线，其中三个段以三次贝塞尔形式表示。$\mathbf{P}_i^j$ 表示 j 段的 i 次控制点。

如果度数等于三且断点 $U = \{u_0, u_1, u_2, u_3\}$ 保持固定，并且如果我们允许十二个控制点 $\mathbf{P}_i^j$ 任意变化，我们得到的向量空间 $\mathcal{V}$ 由所有在 U $\mathcal{V}$ 上的分段三次多项式曲线组成，其维度为十二，并且 $\mathcal{V}$ 中的曲线可能在 u_1 或 u_2 处不连续。现在假设我们指定 (如图 2.2 所示) $\mathbf{P}_3^1 = \mathbf{P}_0^2$ 和 $\mathbf{P}_3^2 = \mathbf{P}_0^3$ 。这导致了 $\mathcal{V}^0$ ，即所有在 U 上至少在 C^0 处处连续的分段三次多项式曲线的向量空间。$\mathcal{V}^0$ 的维度为十，并且 $\mathcal{V}^0 \subset \mathcal{V}$ 。

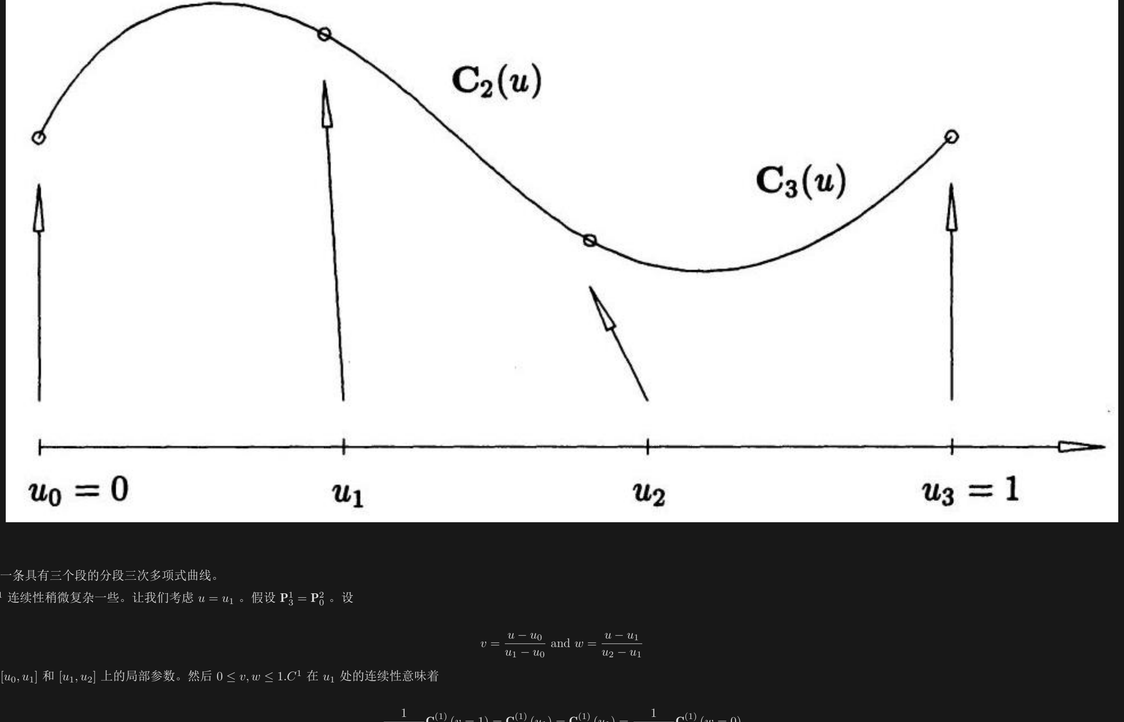


图 2.1. 一条具有三个段的分段三次多项式曲线。

强加 C^1 连续性稍微复杂一些。让我们考虑 $u = u_1$ 。假设 $\mathbf{P}_3^1 = \mathbf{P}_0^2$ 。设

$$v = \frac{u - u_0}{u_1 - u_0} \text{ and } w = \frac{u - u_1}{u_2 - u_1}$$

为区间 $[u_0, u_1]$ 和 $[u_1, u_2]$ 上的局部参数。然后 $0 \le v, w \le 1$.C^1 在 u_1 处的连续性意味着

$$\frac{1}{u_1 - u_0}\mathbf{C}_1^{(1)}(v=1) = \mathbf{C}_1^{(1)}(u_1) = \mathbf{C}_2^{(1)}(u_1) = \frac{1}{u_2 - u_1}\mathbf{C}_2^{(1)}(w=0)$$

从方程 (1.10) 可以得出

$$\frac{3}{u_1 - u_0}\left(\mathbf{P}_3^1 - \mathbf{P}_2^1\right) = \frac{3}{u_2 - u_1}\left(\mathbf{P}_1^2 - \mathbf{P}_0^2\right)$$

因此

$$\mathbf{P}_3^1 = \frac{(u_2 - u_1)\,\mathbf{P}_2^1 + (u_1 - u_0)\,\mathbf{P}_1^2}{u_2 - u_0} \tag{2.1}$$

方程 (2.1) 表示 $\mathbf{P}_3^1$ 和 $\mathbf{P}_3^2$ 可以分别用 $\mathbf{P}_2^1, \mathbf{P}_1^2$ 和 $\mathbf{P}_2^2, \mathbf{P}_1^3$ 来表示。因此，$\mathcal{V}^1$ ，即所有 C^1 连续的分段三次多项式曲线在 U 上的向量空间，维度为八，并且 $\mathcal{V}^1 \subset \mathcal{V}^0 \subset \mathcal{V}$ 。

这清楚地表明，存储和操作分段多项式曲线的各个多项式段并不是处理此类曲线的理想方法。首先，必须存储冗余数据: 十二个系数，而对于 C^1 连续立方曲线，仅需要八个系数，对于 C^2 连续立方曲线，仅需要六个系数。其次，对于贝塞尔形式，连续性 $\mathbf{C}(u)$ 取决于控制点的位置，因此在保持连续性的同时，控制点的定位灵活性较小。如果设计师希望获得 C^1 连续性，并对段落 $\mathbf{C}_1(u)$ 和 $\mathbf{C}_3(u)$ 感到满意，但希望修改 $\mathbf{C}_2(u)$ 的形状，他就没有运气了: $\mathbf{C}_2(u)$ 的控制点无法修改。第三，确定曲线的连续性需要计算 (例如公式 [2.1])。

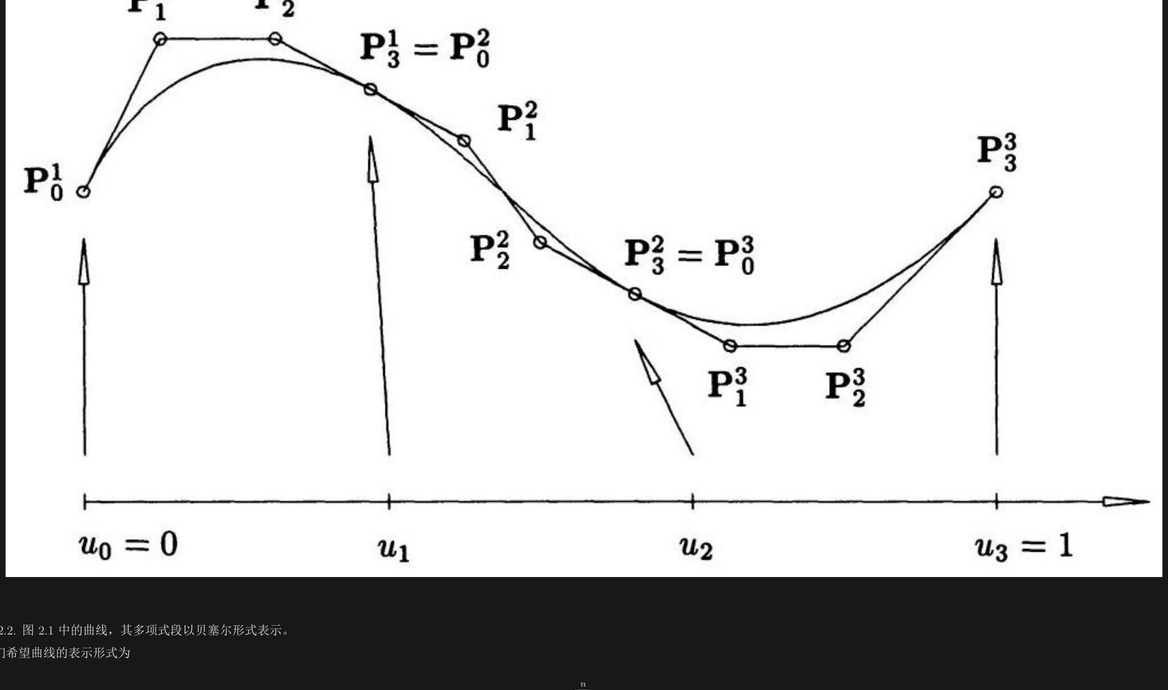


图 2.2. 图 2.1 中的曲线，其多项式段以贝塞尔形式表示。

我们希望曲线的表示形式为

$$\mathbf{C}(u) = \sum_{i=0}^{n} f_i(u)\,\mathbf{P}_i \tag{2.2}$$

在这里，$\mathbf{P}_i$ 是控制点，而 $\{f_i(u), i = 0, \ldots, n\}$ 是分段多项式函数，构成所需度数和连续性 (对于固定的断点序列) 的所有分段多项式函数的向量空间的基。注意，连续性由基函数决定，因此控制点可以在不改变曲线连续性的情况下进行修改。此外，$\{f_i\}$ 应具有“通常”的良好解析性质，例如在第 1.3 节中列出的那些。这确保了由方程 (2.2) 定义的曲线具有类似于 Bézier 曲线的良好几何性质，例如，凸包、变差减小、变换不变性。我们在基函数中寻求的另一个重要性质是局部支持；这意味着每个 $f_i(u)$ 仅在有限数量的子区间上非零，而不是整个域。$[u_0, u_m]$ 。由于 $\mathbf{P}_i$ 乘以 $f_i(u)$ ，移动 $\mathbf{P}_i$ 仅在 $f_i(u)$ 非零的子区间内影响曲线形状。

最后，给定适当的分段多项式基函数，我们可以构造分段有理曲线。

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} f_i(u)\,\mathbf{P}_i^w \tag{2.3}$$

以及非有理和有理的张量积曲面。

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} f_i(u)\,g_j(v)\,\mathbf{P}_{i,j}$$

$$\mathbf{S}^w(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{m}f_i(u)\,g_j(v)\,\mathbf{P}_{i,j}^w \tag{2.4}$$

在本章的其余部分，我们研究所谓的 B-spline 基函数。在第 3 章和第 4 章中，我们将这些函数与三维和四维控制点结合，以分别获得非有理和有理曲线和曲面。

2 B 样条基函数的定义和性质

有多种方法可以定义 B-spline 基函数并证明其重要性质，例如，通过截断幂函数的差商 [Curr47; Scho46]、通过开花法 [Rams87]，以及通过 deBoor、Cox 和 Mansfield 提出的递归公式 [Cox72; DeBo72, 78]。我们使用递归公式，因为它对于计算机实现最为有用。

设 $U=\{u_0,\ldots,u_m\}$ 为一个非递减的实数序列，即 $u_i\le u_{i+1}, i=0,\ldots,m-1$ 。u_i 被称为节点，U 是节点向量。第 i 个 B-spline 基函数，具有 p 度 (阶数 $p+1$)，记作 $N_{i,p}(u)$ ，定义为

$$N_{i,0}(u)=\begin{cases}1 & \text{if } u_i\le u<u_{i+1}\\ 0 & \text{otherwise}\end{cases}$$

$$N_{i,p}(u)=\frac{u-u_i}{u_{i+p}-u_i}N_{i,p-1}(u)+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}(u) \tag{2.5}$$

注意

- $N_{i,0}(u)$ 是一个阶跃函数，除了在半开区间 $u\in[u_i,u_{i+1})$ 上为零外，处处为零；
- 对于 $p>0, N_{i,p}(u)$ 是两个 (p - 1) 度基函数的线性组合 (图 2.3)；

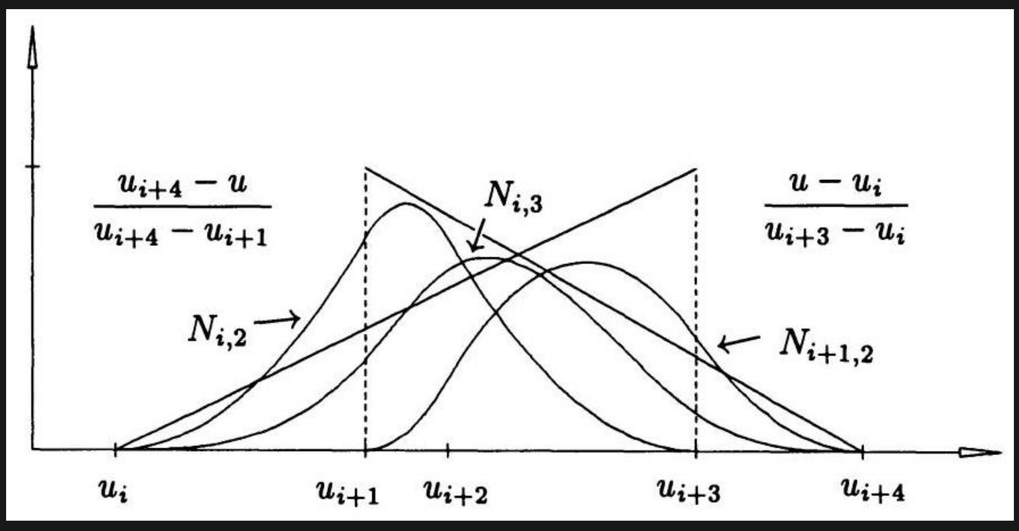


图 2.3. B-spline 基函数的递归定义。

- 计算一组基函数需要指定一个节点向量 U 和度 p ；
- 方程 (2.5) 可以产生商 0/0 (见后面的例子)；我们定义这个商为零；
- $N_{i,p}(u)$ 是分段多项式，定义在整个实数线上；通常只有区间 $[u_0,u_m]$ 是感兴趣的；
- 半开区间 $[u_i,u_{i+1})$ 被称为第 i 个节点跨度；它可以具有零长度，因为节点不必是不同的；
- 计算 p 度的函数会生成一个截断的三角形表

$$N_{0,0}$$

$$N_{0,1}$$

$$N_{1,0}\ N_{0,2}$$

$$N_{1,1}\ N_{0,3}$$

$$N_{2,0}\ N_{1,2}$$

$$N_{2,1}\ N_{1,3}$$

$$N_{3,0}\ N_{2,2}$$

$$N_{3,1}$$

$$N_{4,0}\ :$$

$$\vdots$$

为了简洁，我们通常写 $N_{i,p}$ 而不是 $N_{i,p}(u)$ 。

关于术语的一点说明。在第 2.1 节中，我们使用了术语断点，并要求 $u_i<u_{i+1}$ 对所有 i 成立。在本书的其余部分，我们使用术语结点，并假设 $u_i\le u_{i+1}$ 。断点对应于一组不同的结点值，而非零长度的结点跨度定义了各个多项式段。因此，我们使用“结点”一词有两种不同的含义: 一是在集合 U 中的一个不同值 (断点)，二是集合 U 的一个元素 (在 U 中可以存在具有相同值的额外结点)。从上下文中应该可以清楚地看出所指的含义。

示例

Ex2.1 设 $U=\{u_0=0,u_1=0,u_2=0,u_3=1,u_4=1,u_5=1\}$ 和 $p=2$ 。我们现在计算 0、1 和 2 阶的 B 样条基函数。

$$N_{0,0}=N_{1,0}=0\ -\infty<u<\infty$$

$$N_{2,0}=\begin{cases}1 & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{3,0}=N_{4,0}=0\ -\infty<u<\infty$$

$$N_{0,1}=\frac{u-0}{0-0}N_{0,0}+\frac{0-u}{0-0}N_{1,0}=0\ -\infty<u<\infty$$

$$N_{1,1}=\frac{u-0}{0-0}N_{1,0}+\frac{1-u}{1-0}N_{2,0}=\begin{cases}1-u\\ 0\end{cases}$$

$$N_{2,1}=\frac{u-0}{1-0}N_{2,0}+\frac{1-u}{1-1}N_{3,0}=\begin{cases}u & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{3,1}=\frac{u-1}{1-1}N_{3,0}+\frac{1-u}{1-1}N_{4,0}=0\ -\infty<u<\infty$$

$$N_{0,2}=\frac{u-0}{0-0}N_{0,1}+\frac{1-u}{1-0}N_{1,1}=\begin{cases}(1-u)^2 & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{1,2}=\frac{u-0}{1-0}N_{1,1}+\frac{1-u}{1-0}N_{2,1}=\begin{cases}2u(1-u) & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{2,2}=\frac{u-0}{1-0}N_{2,1}+\frac{1-u}{1-1}N_{3,1}=\begin{cases}u^2 & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

注意，限制在区间 $u\in[0,1]$ 上的 $N_{i,2}$ 是二次 Bernstein 多项式 (第 1.3 节和图 1.13b)。因此，具有以下形式的结点向量的 B 样条表示是 Bézier 表示的推广。

$$U=\{\underbrace{0,\ldots,0}_{p+1},\underbrace{1,\ldots,1}_{p+1}\}$$

是 Bézier 表示的推广。

Ex2.2 设 $U=\{u_0=0,u_1=0,u_2=0,u_3=1,u_4=2,u_5=3,u_6=4,u_7=\ 4,u_8=5,u_9=5,u_{10}=5\}$ 和 $p=2$ 。这里计算了零阶、第一阶和第二阶基函数。那些不恒为零的分别显示在图 2.4、2.5 和 2.6 中。

$$N_{0,0}=N_{1,0}=0\ \text{ for }\ -\infty<u<\infty$$

$$N_{2,0}=\begin{cases}1 & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{3,0}=\begin{cases}1 & 1\le u<2\\ 0 & \text{otherwise}\end{cases}$$

$$N_{4,0}=\begin{cases}1 & 2\le u<3\\ 0 & \text{otherwise}\end{cases}$$

$$N_{5,0}=\begin{cases}1 & 3\le u<4\\ 0 & \text{otherwise}\end{cases}$$

$$N_{6,0}=0\ \text{for}\ -\infty<u<\infty$$

$$N_{7,0}=\begin{cases}1 & 4\le u<5\\ 0 & \text{otherwise}\end{cases}$$

$$N_{8,0}=N_{9,0}=0\ \text{ for }\ -\infty<u<\infty$$

$$N_{0,1}=\frac{u-0}{0-0}N_{0,0}+\frac{0-u}{0-0}N_{1,0}=0\ -\infty<u<\infty$$

$$N_{1,1}=\frac{u-0}{0-0}N_{1,0}+\frac{1-u}{1-0}N_{2,0}=\begin{cases}1-u & 0\le u<1\\ 0 & \text{otherwise}\end{cases}$$

$$N_{2,1}=\frac{u-0}{1-0}N_{2,0}+\frac{2-u}{2-1}N_{3,0}=\begin{cases}u & 0\le u<1\\ 2-u & 1\le u<2\\ 0 & \text{otherwise}\end{cases}$$

$$N_{3,1}=\frac{u-1}{2-1}N_{3,0}+\frac{3-u}{3-2}N_{4,0}=\begin{cases}u-1 & 1\le u<2\\ 3-u & 2\le u<3\\ 0 & \text{otherwise}\end{cases}$$

$$N_{4,1}=\frac{u-2}{3-2}N_{4,0}+\frac{4-u}{4-3}N_{5,0}=\begin{cases}u-2 & 2\le u<3\\ 4-u & 3\le u<4\\ 0 & \text{otherwise}\end{cases}$$

$$N_{5,1}=\frac{u-3}{4-3}N_{5,0}+\frac{4-u}{4-4}N_{6,0}=\begin{cases}u-3 & 3\le u<4\\ 0 & \text{otherwise}\end{cases}$$

$$N_{6,1}=\frac{u-4}{4-4}N_{6,0}+\frac{5-u}{5-4}N_{7,0}=\begin{cases}5-u & 4\le u<5\\ 0 & \text{otherwise}\end{cases}$$

$$N_{7,1}=\frac{u-4}{5-4}N_{7,0}+\frac{5-u}{5-5}N_{8,0}=\begin{cases}u-4 & 4\le u<5\\ 0 & \text{otherwise}\end{cases}$$

$$N_{8,1}=\frac{u-5}{5-5}N_{8,0}+\frac{5-u}{5-5}N_{9,0}=0\ -\infty<u<\infty$$

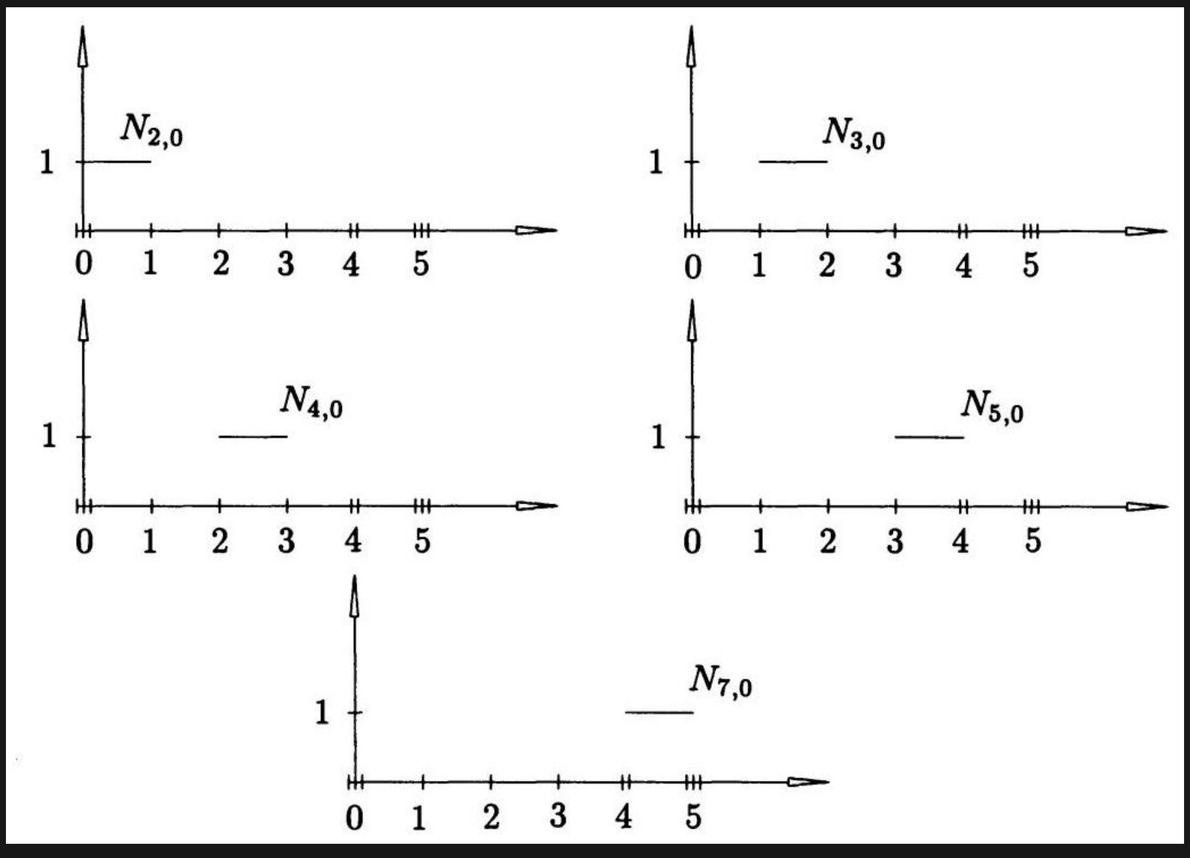


图 2.4。非零的零阶基函数，$U=\{0,0,0,1,2,3,4,4,5,5,5\}$ 。

以下所有 $N_{i,2}$ 在指定区间以外处处为零，即

$$N_{0,2}=\frac{u-0}{0-0}N_{0,1}+\frac{1-u}{1-0}N_{1,1}=(1-u)^2\ 0\le u<1$$

$$N_{1,2}=\frac{u-0}{1-0}N_{1,1}+\frac{2-u}{2-0}N_{2,1}=\begin{cases}2u-3/2u^2 & 0\le u<1\\ 1/2(2-u)^2 & 1\le u<2\end{cases}$$

$$N_{2,2}=\frac{u-0}{2-0}N_{2,1}+\frac{3-u}{3-1}N_{3,1}=\begin{cases}1/2u^2 & 0\le u<1\\ -3/2+3u-u^2 & 1\le u<2\\ 1/2(3-u)^2 & 2\le u<3\end{cases}$$

$$N_{3,2}=\frac{u-1}{3-1}N_{3,1}+\frac{4-u}{4-2}N_{4,1}=\begin{cases}1/2(u-1)^2 & 1\le u<2\\ -11/2+5u-u^2 & 2\le u<3\\ 1/2(4-u)^2 & 3\le u<4\end{cases}$$

$$N_{4,2}=\frac{u-2}{4-2}N_{4,1}+\frac{4-u}{4-3}N_{5,1}=\begin{cases}1/2(u-2)^2 & 2\le u<3\\ -16+10u-3/2u^2 & 3\le u<4\end{cases}$$

$$N_{5,2}=\frac{u-3}{4-3}N_{5,1}+\frac{5-u}{5-4}N_{6,1}=\begin{cases}(u-3)^2 & 3\le u<4\\ (5-u)^2 & 4\le u<5\end{cases}$$

$$N_{6,2}=\frac{u-4}{5-4}N_{6,1}+\frac{5-u}{5-4}N_{7,1}=2\,(u-4)\,(5-u)\ 4\le u<5$$

$$N_{7,2} = \frac{u-4}{5-4} N_{7,1} + \frac{5-u}{5-5} N_{8,1} = (u-4)^2$$

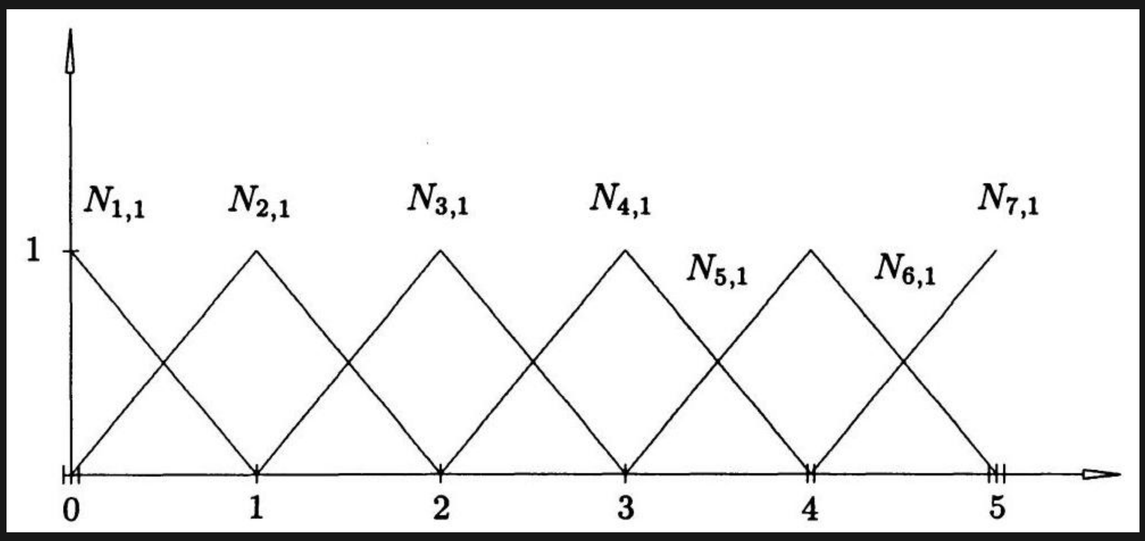


图 2.5. 非零的一阶基函数，$U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$ 。

现在我们列出 B-spline 基函数的一些重要属性。正如我们在下一章中看到的，这些属性决定了 B-spline 曲线和曲面的许多理想几何特性。假设度数为 p ，并且有一个节点向量 $U = \{u_0, \ldots, u_m\}$ 。

P2. 1 $N_{i,p}(u) = 0$ 如果 u 在区间 $[u_i, u_{i+p+1})$ 之外 (局部支持属性)。这由这里所示的三角形方案说明。请注意

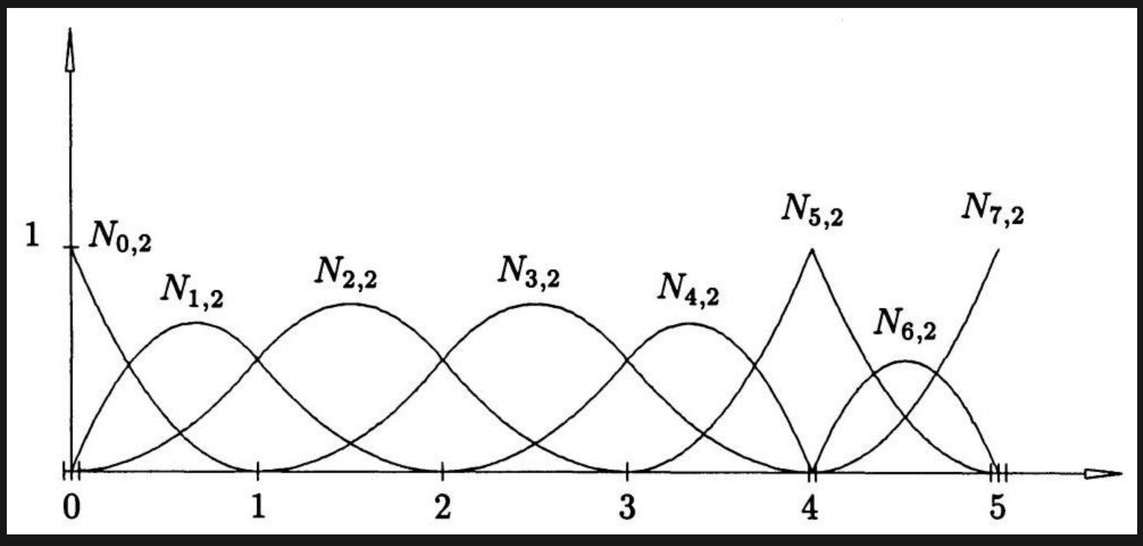
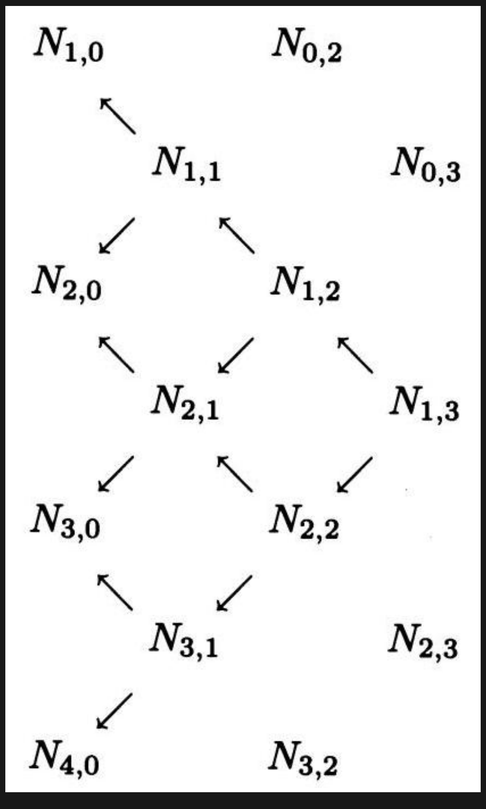
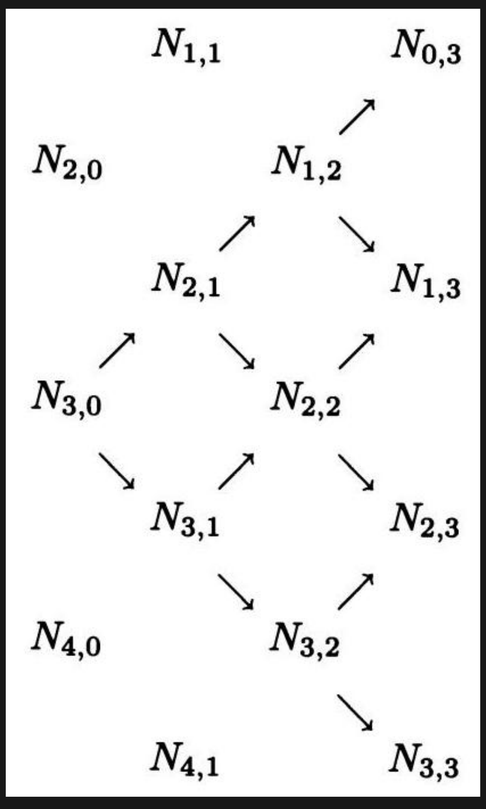


图 2.6. 非零的二阶基函数，$U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$ 。

$N_{1,3}$ 是 $N_{1,0}, N_{2,0}, N_{3,0}$ 和 $N_{4,0}$ 的组合。因此，$N_{1,3}$ 仅在 $u \in [u_1, u_5)$ 时非零。



P2.2 在任何给定的节点跨度 $[u_j, u_{j+1})$ 中，至多有 $p+1$ 个 $N_{i,p}$ 是非零的，即函数 $N_{j-p,p}, \ldots, N_{j,p}$ 。在 $[u_3, u_4)$ 上，唯一的非零零阶函数是 $N_{3,0}$ 。因此，在 $[u_3, u_4)$ 上唯一不为零的三次函数是 $N_{0,3}, \ldots, N_{3,3}$ 。这个属性在这里得到了说明。



P2.3 对于所有 i, p 和 u (非负性)。这通过对 p 的归纳证明。对于 $p = 0$ 显然是正确的；假设它对 $p-1, p \geq 0$ 成立，且 i 和 u 是任意的。根据定义

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \tag{2.6}$$

根据 P2.1，$N_{i,p-1}(u) = 0$ 如果 $u \notin [u_i, u_{i+p})$。但 $u \in [u_i, u_{i+p})$ 意味着

$$\frac{u - u_i}{u_{i+p} - u_i}$$

是非负的。根据假设，$N_{i,p-1}(u)$ 是非负的，因此方程 (2.6) 的第一项是非负的。第二项也是如此，因此 $N_{i,p}(u)$ 是非负的；

P2.4 对于任意的结跨度，$[u_i, u_{i+1})$，$\sum_{j=i-p}^{i} N_{j,p}(u) = 1$ 对于所有 $u \in [u_i, u_{i+1})$ (统一划分)。为了证明这一点，考虑

$$\sum_{j=i-p}^{i} N_{j,p}(u) = \sum_{j=i-p}^{i} \frac{u - u_j}{u_{j+p} - u_j} N_{j,p-1}(u)$$

$$+ \sum_{j=i-p}^{i} \frac{u_{j+p+1} - u}{u_{j+p+1} - u_{j+1}} N_{j+1,p-1}(u)$$

将第二个求和中的变量从 $i - p$ 改为 $i - p + 1$，并考虑 $N_{i-p,p-1}(u) = N_{i+1,p-1}(u) = 0$，我们得到

$$\sum_{j=i-p}^{i} N_{j,p}(u) = \sum_{j=i-p+1}^{i} \left[\frac{u - u_j}{u_{j+p} - u_j} + \frac{u_{j+p} - u}{u_{j+p} - u_j} \right] N_{j,p-1}(u)$$

$$= \sum_{j=i-p+1}^{i} N_{j,p-1}(u)$$

递归应用相同的概念得到

$$\sum_{j=i-p}^{i} N_{j,p}(u) = \sum_{j=i-p+1}^{i} N_{j,p-1}(u) = \sum_{j=i-p+2}^{i} N_{j,p-2}(u)$$

$$= \cdots = \sum_{j=i}^{i} N_{j,0}(u) = 1$$

P2.5 所有 $N_{i,p}(u)$ 的导数在结跨度的内部存在 (在此处它是一个多项式，见图 2.7)。在一个结 $N_{i,p}(u)$ 是 $p - k$ 次连续可微的，其中 k 是结的重数。因此，增加次数会增加连续性，而增加结的重数会降低连续性；

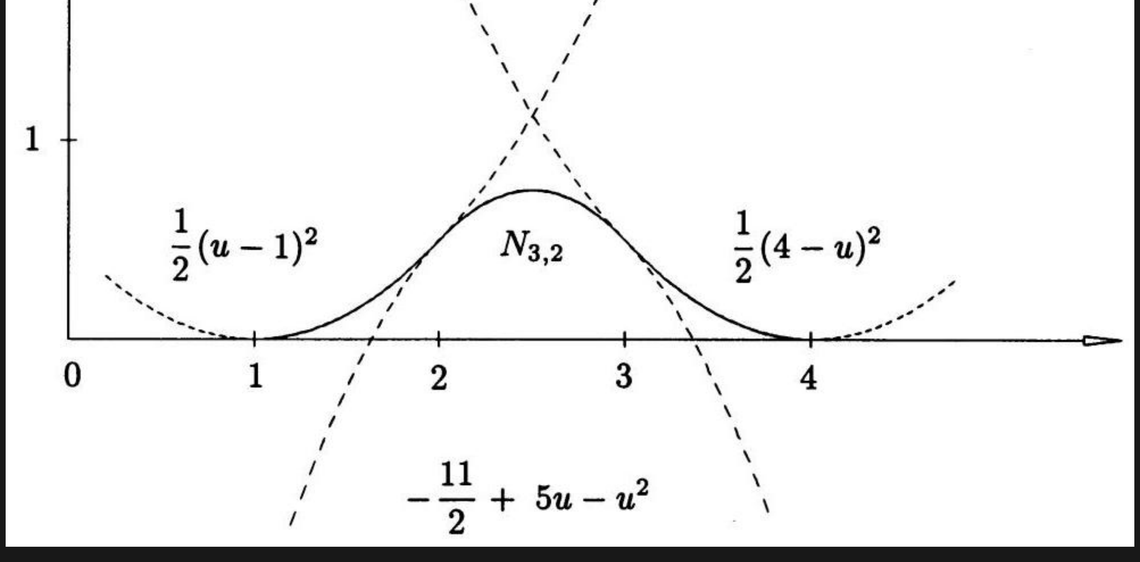


图 2.7. $N_{3,2}$ 被分解为其多项式片段 (抛物线)。

P2.6 除非 $p = 0, N_{i,p}(u)$ 恰好达到一个最大值的情况。

理解多个结的影响是重要的。考虑图 2.6 中的函数 $N_{0,2}, N_{1,2}, N_{2,2}, N_{5,2}$ 和 $N_{6,2}$。回忆 $U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$，根据方程 (2.5) 和 P2.1，我们看到这些函数在以下结跨度上计算，并且在这些跨度之外为零。

$$N_{0,2} : \{0, 0, 0, 1\}$$

$$N_{1,2} : \{0, 0, 1, 2\}$$

$$N_{2,2} : \{0, 1, 2, 3\}$$

$$N_{5,2} : \{3, 4, 4, 5\}$$

$$N_{6,2} : \{4, 4, 5, 5\}$$

现在“重数”这个词有两种不同的理解:

- 结在结向量中的重数；
- 相对于特定基函数的结的重数。

例如，$u=0$ 在之前的节点向量 U 中具有三重性。但就函数 $N_{0,2}, N_{1,2}, N_{2,2}$ 和 $N_{5,2}$，$u=0$ 而言，分别是三重、二重、一重和零重的节点。从 P2.5 开始，这些函数在 $u=0$ 的连续性是 $N_{0,2}$ 不连续的；$N_{1,2}C^0$ 连续的；$N_{2,2}C^1$ 连续的；以及 $N_{5,2}$ 完全不受影响（ $N_{5,2}$ 及其所有导数在 $u=0$ 处从两侧均为零）。$N_{1,2}$ 将 $u=0$ 视为双重节点，因此它是 C^0 连续的。$N_{2,2}$ 将其所有节点视为一重，因此它在任何地方都是 C^1 连续的。显然，多重节点的另一个效果（如函数所见）是减少函数非零的“明显”区间的数量；例如，$N_{6,2}$ 仅在 $u\in[4,5)$ 上非零，并且它仅在 $u=4$ 和 $u=5$ 处是 C^0 连续的。

3 B 样条基函数的导数

基函数的导数由以下公式给出

$$N'_{i,p}=\frac{p}{u_{i+p}-u_i}N_{i,p-1}(u)-\frac{p}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}(u) \tag{2.7}$$

（见图 2.8 以获取图形说明。）我们通过对 p 的归纳法证明这一点。对于 $p=1, N_{i,p-1}$ 和 $N_{i+1,p-1}$ 来说，它们要么是 0，要么是 1，因此 $N'_{i,p}$ 要么是

$$\frac{1}{u_{i+1}-u_i} \text{ or } -\frac{1}{u_{i+2}-u_{i+1}}$$

（见图 2.5）。现在假设方程 (2.7) 对 $p-1, p>1$ 成立。使用乘积法则，$(fg)'=f'g+fg'$ 来对基函数进行求导

$$N_{i,p}(u)=\frac{u-u_i}{u_{i+p}-u_i}N_{i,p-1}(u)+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}(u)$$

得到 $N'_{i,p}=\frac{1}{u_{i+p}-u_i}N_{i,p-1}+\frac{u-u_i}{u_{i+p}-u_i}N'_{i,p-1}$ (2.8)

$$-\frac{1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}N'_{i+1,p-1}$$

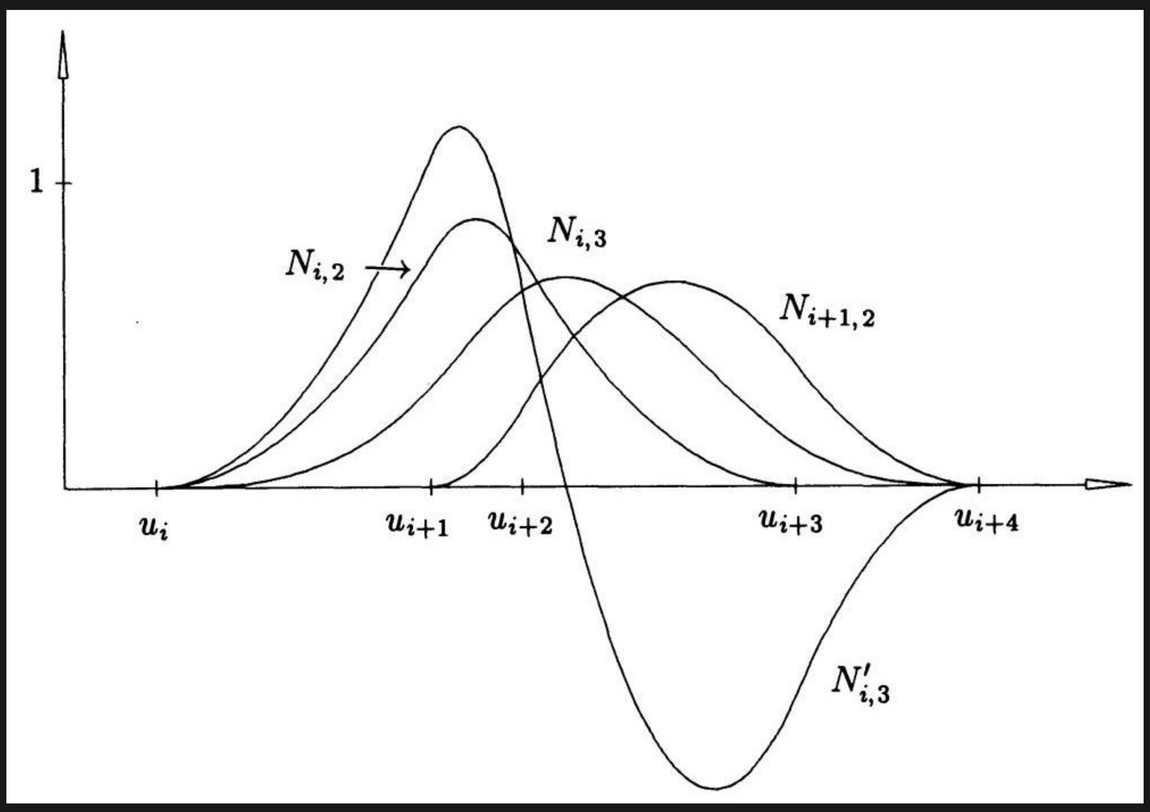


图 2.8. B 样条导数的递归定义。

将方程 (2.7) 代入方程 (2.8) 得到 $N'_{i,p-1}$ 和 $N'_{i+1,p-1}$ 的结果

$$N'_{i,p}=\frac{1}{u_{i+p}-u_i}N_{i,p-1}-\frac{1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

$$+\frac{u-u_i}{u_{i+p}-u_i}\left(\frac{p-1}{u_{i+p-1}-u_i}N_{i,p-2}-\frac{p-1}{u_{i+p}-u_{i+1}}N_{i+1,p-2}\right)$$

$$+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}\left(\frac{p-1}{u_{i+p}-u_{i+1}}N_{i+1,p-2}-\frac{p-1}{u_{i+p+1}-u_{i+2}}N_{i+2,p-2}\right)$$

$$=\frac{1}{u_{i+p}-u_i}N_{i,p-1}-\frac{1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

$$+\frac{p-1}{u_{i+p}-u_i}\frac{u-u_i}{u_{i+p-1}-u_i}N_{i,p-2}$$

$$+\frac{p-1}{u_{i+p}-u_{i+1}}\left(\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}-\frac{u-u_i}{u_{i+p}-u_i}\right)N_{i+1,p-2}$$

$$-\frac{p-1}{u_{i+p+1}-u_{i+1}}\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+2}}N_{i+2,p-2}$$

注意到

$$\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}-\frac{u-u_i}{u_{i+p}-u_i}=-1+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}+1-\frac{u-u_i}{u_{i+p}-u_i}$$

$$=-\frac{u_{i+p+1}-u_{i+1}}{u_{i+p+1}-u_{i+1}}+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}$$

$$+\frac{u_{i+p}-u_i}{u_{i+p}-u_i}-\frac{u-u_i}{u_{i+p}-u_i}$$

$$=\frac{u_{i+p}-u}{u_{i+p}-u_i}-\frac{u-u_{i+1}}{u_{i+p+1}-u_{i+1}}$$

我们得到

$$N'_{i,p}=\frac{1}{u_{i+p}-u_i}N_{i,p-1}-\frac{1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

$$+\frac{p-1}{u_{i+p}-u_i}\left(\frac{u-u_i}{u_{i+p-1}-u_i}N_{i,p-2}+\frac{u_{i+p}-u}{u_{i+p}-u_{i+1}}N_{i+1,p-2}\right)$$

$$-\frac{p-1}{u_{i+p+1}-u_{i+1}}\left(\frac{u-u_{i+1}}{u_{i+p}-u_{i+1}}N_{i+1,p-2}+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+2}}N_{i+2,p-2}\right)$$

根据 Cox-deBoor 公式（方程 [2.5]），括号中的表达式可以分别用 $N_{i,p-1}$ 和 $N_{i+1,p-1}$ 替换。因此

$$N'_{i,p}=\frac{1}{u_{i+p}-u_i}N_{i,p-1}-\frac{1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

$$+\frac{p-1}{u_{i+p}-u_i}N_{i,p-1}-\frac{p-1}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

$$=\frac{p}{u_{i+p}-u_i}N_{i,p-1}-\frac{p}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}$$

这完成了证明。

现在让 $N^{(k)}_{i,p}$ 表示 k 阶导数 $N_{i,p}(u)$ 。对方程 (2.7) 进行重复求导得到一般公式

$$N^{(k)}_{i,p}(u)=p\left(\frac{N^{(k-1)}_{i,p-1}}{u_{i+p}-u_i}-\frac{N^{(k-1)}_{i+1,p-1}}{u_{i+p+1}-u_{i+1}}\right) \tag{2.9}$$

方程 (2.10) 是方程 (2.7) 的另一个推广。它以函数 $N_{i,p-k},\ldots,N_{i+k,p-k}$ 的形式计算 $N_{i,p}(u)$ 的 k 阶导数

$$N_{i,p}^{(k)}=\frac{p!}{(p-k)!}\sum_{j=0}^{k}a_{k,j}N_{i+j,p-k} \tag{2.10}$$

其中

$$a_{0,0}=1$$

$$a_{k,0}=\frac{a_{k-1,0}}{u_{i+p-k+1}-u_i}$$

$$a_{k,j}=\frac{a_{k-1,j}-a_{k-1,j-1}}{u_{i+p+j-k+1}-u_{i+j}}\quad j=1,\ldots,k-1$$

$$a_{k,k}=\frac{-a_{k-1,k-1}}{u_{i+p+1}-u_{i+k}}$$

关于方程 (2.10) 的备注:

- k 不应超过 p (所有更高阶导数为零);
- 涉及节点差异的分母可能会变为零；在这种情况下，商被定义为零 (见下一节的示例和算法)。

我们省略了公式 (2.10) 的证明，但验证它在 $k=1,2$ 下成立。根据定义

$$a_{1,0}=\frac{1}{u_{i+p}-u_i}\;a_{1,1}=-\frac{1}{u_{i+p+1}-u_{i+1}}$$

和

$$N_{i,p}^{(1)}=2\left(a_{1,0}N_{i,p-1}+a_{1,1}N_{i+1,p-1}\right)$$

将此与公式 (2.7) 进行比较证明了 $k=1$ 的情况；现在设 $k=2$ 。对公式 (2.7) 进行微分得到

$$\begin{aligned}N_{i,p}^{(2)}&=\frac{p}{u_{i+p}-u_i}N_{i,p-1}^{(1)}-\frac{p}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}^{(1)}\\&=\frac{p}{u_{i+p}-u_i}\left(\frac{p-1}{u_{i+p-1}-u_i}N_{i,p-2}-\frac{p-1}{u_{i+p}-u_{i+1}}N_{i+1,p-2}\right)\\&-\frac{p}{u_{i+p+1}-u_{i+1}}\left(\frac{p-1}{u_{i+p}-u_{i+1}}N_{i+1,p-2}-\frac{p-1}{u_{i+p+1}-u_{i+2}}N_{i+2,p-2}\right)\\&=p(p-1)\left[\frac{a_{1,0}}{u_{i+p-1}-u_i}N_{i,p-2}\right.\\&-\frac{1}{u_{i+p}-u_{i+1}}\left(\frac{1}{u_{i+p}-u_i}+\frac{1}{u_{i+p+1}-u_{i+1}}\right)N_{i+1,p-2}\\&\left.+\frac{a_{1,1}}{u_{i+p+1}-u_{i+2}}N_{i+2,p-2}\right]\\&=p(p-1)\left(a_{2,0}N_{i,p-2}+\frac{a_{1,1}-a_{1,0}}{u_{i+p}-u_{i+1}}N_{i+1,p-2}+a_{2,2}N_{i+2,p-2}\right)\end{aligned}$$

注意到 $k=2$ 和

$$a_{2,1}=\frac{a_{1,1}-a_{1,0}}{u_{i+p}-u_{i+1}}$$

由此可得

$$N_{i,p}^{(2)}(u)=2\sum_{j=0}^{2}a_{2,j}N_{i+j,p-2}(u)$$

为了完整性，我们给出一个额外的公式来计算 B 样条基函数的导数 (见 [Butt76])

$$N_{i,p}^{(k)}=\frac{p}{p-k}\left(\frac{u-u_i}{u_{i+p}-u_i}N_{i,p-1}^{(k)}+\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}^{(k)}\right)$$

$$k=0,\ldots,p-1 \tag{2.11}$$

公式 (2.11) 给出了 $N_{i,p}(u)$ 的 k 阶导数，以 $N_{i,p-1}$ 的 k 阶导数和 $N_{i+1,p-1}$ 表示。
图 2.9b 和 2.10b 显示了与图 2.9a 和 2.10a 中的基函数对应的导数。图 2.11 显示了 $N_{i,3}$ 的所有非零导数。注意图 2.10b 中多个节点的影响；$N'_{6,3}$ 在三重节点处有一个跳跃。

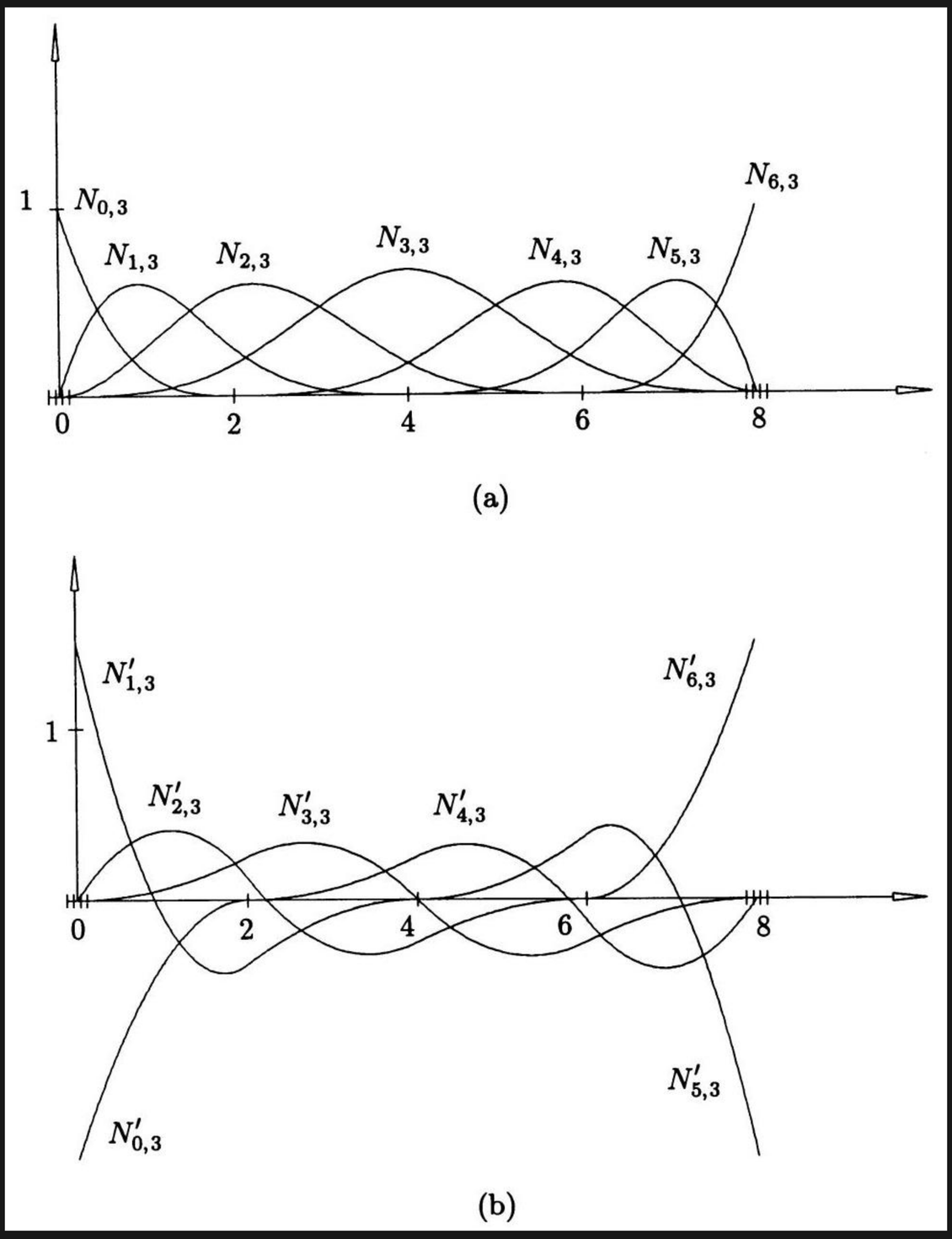


图 2.9. (a) 三次基函数；(b) 与图 2.9a 中的基函数对应的导数。

4　基函数的进一步性质

设 $\{u_j\}, 0 \leq j \leq k$ 为一个严格递增的断点集合。在 $\{u_j\}$ 上，所有在 $u = u_j$ 处 C^{r_j} 连续的 p 次分段多项式函数的集合形成一个向量空间 $\mathcal{V}(-1 \leq r_j \leq p)$。如果对所有 j) 不施加连续性约束 ($r_j = -1$，则 $\mathcal{V}($ 的维度 $\dim(\mathcal{V})$) 等于 $k(p+1)$。每个连续性约束使维度减少一个，因此

$$\dim(\mathcal{V}) = k(p+1) - \sum_{j=0}^{k}(r_j + 1) \tag{2.12}$$

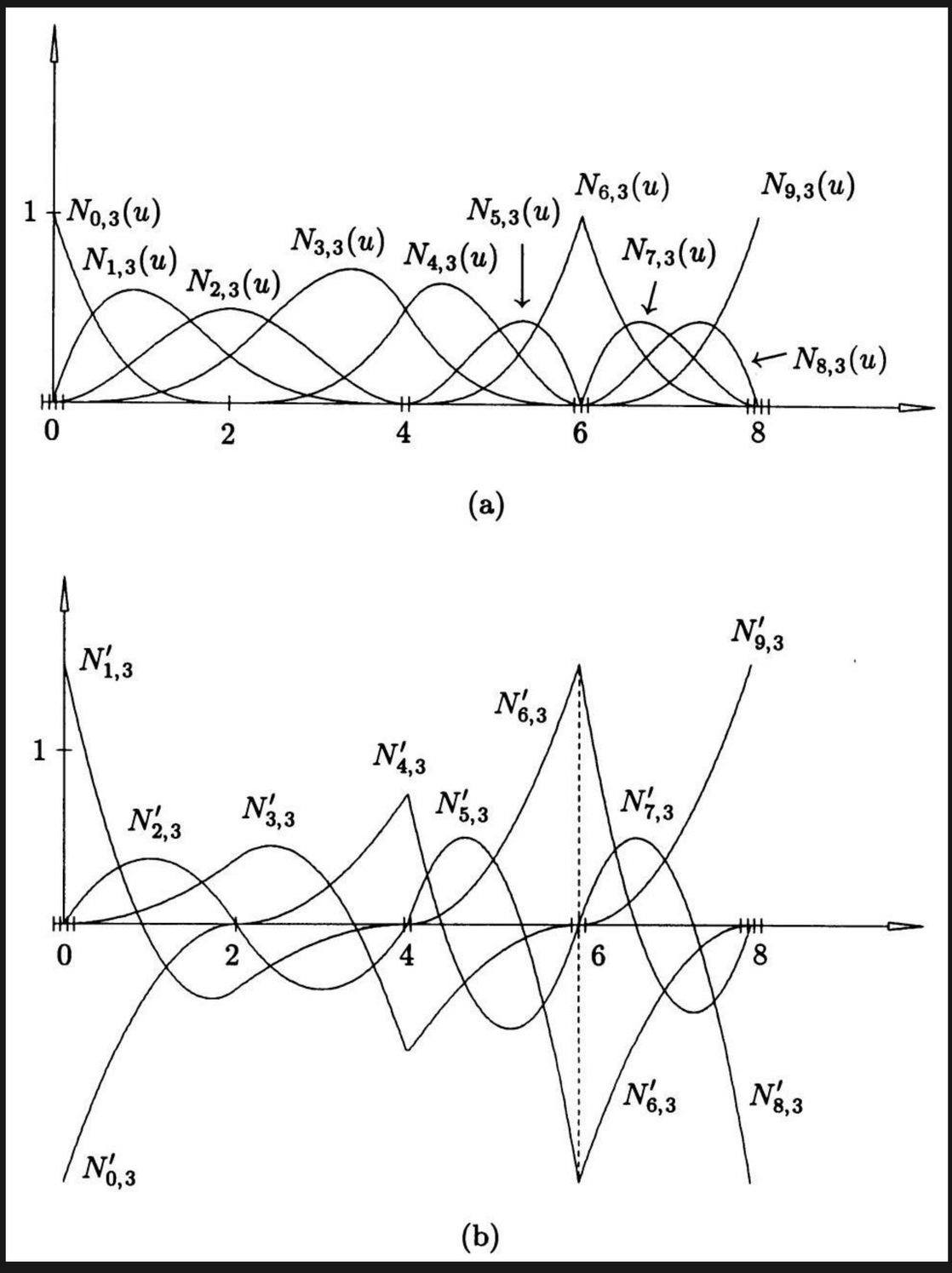


图 2.10. (a) 立方基函数显示单个、双重和三重节点；(b) 图 2.10a 中函数的导数。

根据性质 P2.5，我们通过设置适当的节点重数 s_j 来获得具有 $\{u_j\}$ 处节点的 p 次 B-spline 基函数，并具有所需的连续性 $s_j = p - r_j$ 。因此，我们使用形式为

$$U = \{\underbrace{u_0, \ldots, u_0}_{s_0}, \underbrace{u_1, \ldots, u_1}_{s_1}, \ldots, \underbrace{u_k, \ldots, u_k}_{s_k}\}$$

现在设置

$$m = \left(\sum_{j=0}^{k} s_j\right) - 1$$

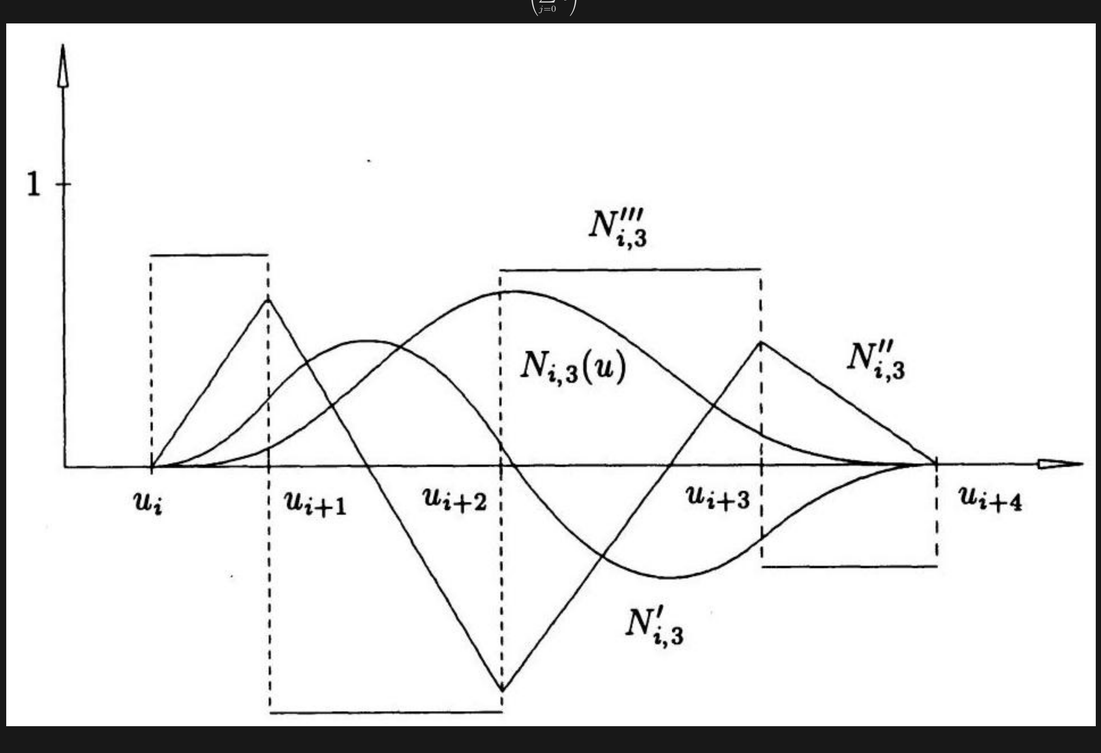


图 2.11. $N_{i,3}$ 及其所有非零导数。

因此，显然有 m 个零次函数，$N_{i,0}, m-1$ 个一次函数，$N_{i,1}$ ，以及一般的 $m-pp$ 次函数 $N_{i,p}$ ，它们具有所需的连续性 $r_j = p - s_j$ 。因此，$N_{i,p}$ 包含在 $\mathcal{V}$ 中。将 $s_j = p - r_j$ 代入方程 (2.12) 得到

$$\dim(\mathcal{V}) = k(p+1) - \sum_{j=0}^{k}(p - s_j + 1)$$

$$= k(p+1) - (k+1)p + \sum_{j=0}^{k} s_j - (k+1)$$

$$= -p - 1 + \sum_{j=0}^{k} s_j$$

$$= m - p$$

因此，在 U 上的 p 次 B-spline 基函数的数量等于 $\dim(\mathcal{V})$。我们现在通过证明 $N_{i,p}$ 是线性无关的来证明“基”函数的术语，即它们构成向量空间 $\mathcal{V}$ 的基。证明是通过对 p 的归纳。显然，零次函数是线性无关的。假设 (p - 1) 次函数在 $p > 0$ 上是线性无关的。设 $n = m - p - 1$，并假设

$$\sum_{i=0}^{n} \alpha_i N_{i,p}(u) = 0 \quad \text{for all } u$$

使用方程 (2.7) 我们得到

$$0 = \left(\sum_{i=0}^{n} \alpha_i N_{i,p} \right)' = \sum_{i=0}^{n} \alpha_i N'_{i,p}$$

$$= p \sum_{i=0}^{n} \alpha_i \left(\frac{N_{i,p-1}}{u_{i+p} - u_i} - \frac{N_{i+1,p-1}}{u_{i+p+1} - u_{i+1}} \right)$$

这意味着

$$0 = \sum_{i=0}^{n} \alpha_i \frac{N_{i,p-1}}{u_{i+p} - u_i} - \sum_{i=0}^{n} \alpha_i \frac{N_{i+1,p-1}}{u_{i+p+1} - u_{i+1}}$$

现在注意到 $N_{0,p-1} = N_{n+1,p-1} = 0$，并在第二项中更改求和变量，我们有

$$0 = \sum_{i=1}^{n} \frac{\alpha_i - \alpha_{i-1}}{u_{i+p} - u_i} N_{i,p-1}$$

这意味着对于所有 i (根据假设) $\alpha_i - \alpha_{i-1} = 0$，这反过来意味着对于所有 i $\alpha_i = 0$。这完成了证明。

现在我们将注意力转向节点向量。显然，一旦度数固定，节点向量就完全决定了函数 $N_{i,p}(u)$。节点向量有几种类型，不幸的是，文献中的术语有所不同。在本书中，我们仅考虑非周期性 (或夹紧或开放) 节点向量，其形式为

$$U = \{\underbrace{a, \ldots, a}_{p+1}, u_{p+1}, \ldots, u_{m-p-1}, \underbrace{b, \ldots, b}_{p+1}\} \tag{2.13}$$

即，第一个和最后一个节点的重数为 $p+1$。对于非周期性节点向量，我们有两个额外的基函数性质:

P2.7 形式的节点向量

$$U = \{\underbrace{0, \ldots, 0}_{p+1}, \underbrace{1, \ldots, 1}_{p+1}\}$$

产生度数为 p 的伯恩斯坦多项式 (见第 2.2 节的示例 Ex2.1)；

P2.8 设 $m+1$ 为节点的数量。则有 $n+1$ 个基函数，其中 $n = m - p - 1$; $N_{0,p}(a) = 1$ 和 $N_{n,p}(b) = 1$。例如，$N_{0,p}(a) = 1$ 由 $N_{0,0}, \ldots, N_{p-1,0} = 0$ 的事实得出，因为这意味着 $N_{0,p}(a) = N_{p,0}(a) = 1$。从 P2.4 可以得出，对于 $i \neq 0$，$N_{i,p}(a) =$ 为 0，对于 $i \neq n$，$N_{i,p}(b) = 0$ 为。

在本书的其余部分，所有节点向量都被理解为非周期性。我们定义一个节点向量 $U = \{u_0, \ldots, u_m\}$ 为均匀的，如果所有内部节点均匀分布，即，如果存在一个实数 d，使得对于所有 $p \leq i \leq m - p - 1$，都有 $d = u_{i+1} - u_i$；否则它是非均匀的。示例的节点向量

Ex2.2，第 2.2 节是非均匀的，因为在 $u = 4$ 处有双重节点。图 2.9a 显示了一组均匀的三次基函数，图 2.10a 和 2.12 显示了非均匀的三次基函数。

5 计算算法

在本节中，我们开发算法来计算基函数及其导数的值。设 $U = \{u_0, \ldots, u_m\}$ 为形式如公式 (2.13) 的节点向量，并假设我们对度数为 p 的基函数感兴趣。此外，假设 u 是固定的，并且 $u \in [u_i, u_{i+1})$。我们开发了五个算法来计算:

- 节点跨度索引 i；
- $N_{i-p,p}(u), \ldots, N_{i,p}(u)$ (基于公式 [2.5])；
- $N^{(k)}_{i-p,p}(u), \ldots, N^{(k)}_{i,p}(u)$ 对于 $k = 0, \ldots, p$；对于 $k > p$，导数为零 (该算法基于公式 [2.10])；
- 单个基函数 $N_{j,p}(u)$，其中 $0 \leq j \leq m - p - 1$；
- 单个基函数 $N^{(k)}_{j,p}(u)$ 的导数，其中 $0 \leq j \leq m - p - 1$ 和 $k = 0, \ldots, p$ (基于公式 [2.9])。

我们首先介绍计算 $p+1$ 函数的两个算法，然后是仅计算一个的两个算法，因为它们是最重要的，并且实际上相对简单。

从 P2.2 和假设 $u \in [u_i, u_{i+1})$ 可得，我们可以将注意力集中在函数 $N_{i-p,p}, \ldots, N_{i,p}$ 及其导数上；所有其他函数均为零，实际计算它们是浪费。因此，评估的第一步是确定 u 所在的节点跨度。可以使用线性或二分搜索节点向量；我们在此介绍二分搜索。由于我们使用的区间形式为 $u \in [u_i, u_{i+1})$，在基函数评估中存在一个微妙的问题，即特殊情况 $u = u_m$。最好在最低级别处理此问题，将跨度索引设置为 $n\,(= m - p - 1)$。因此，在这种情况下 $u \in (u_{m-p-1}, u_{m-p}]$。FindSpan 是一个整数函数，返回跨度索引。

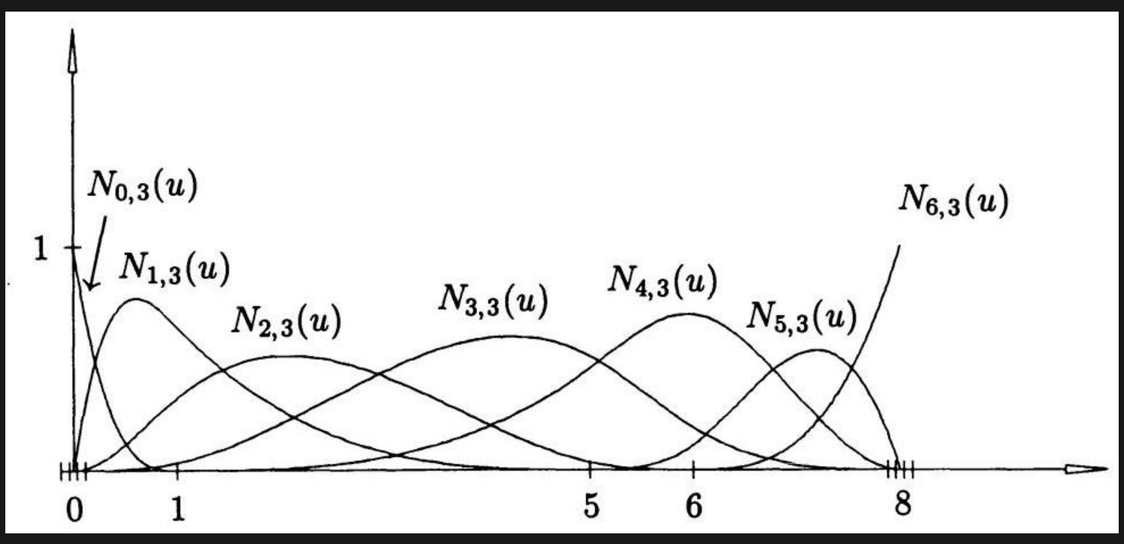


图 2.12. 定义在 $U = \{0, 0, 0, 0, 1, 5, 6, 8, 8, 8, 8\}$ 上的非均匀立方基函数。

ALGORITHM A2.1

```
int FindSpan(n, p, u, U)
    { /* Determine the knot span index */
      /* Input: n, p, u, U */
      /* Return: the knot span index */
    if (u == U[n+1]) return(n); /* Special case */
  low = p;  high = n+1;  /* Do binary search */
  mid = (low+high) / 2 ;
  while (u < U[mid] || u >= U[mid+1])
      {
      if (u < U[mid]) high = mid;
        else low = mid;
      mid = (low+high)/2;
      {
  return(mid);
  }
```

现在我们来解决第二个算法。假设 u 在 i 的跨度中，非零函数的计算结果是一个反三角形方案。

$$N_{i-p,p}$$

$$N_{i-1,1}$$

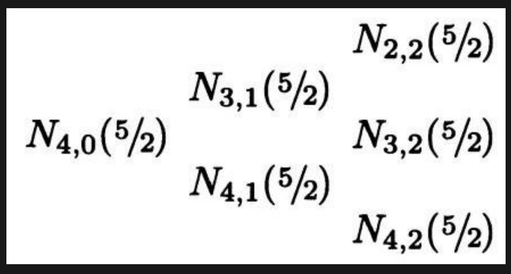
$$N_{i,0} \cdots \vdots$$

$$N_{i,1}$$

$$N_{i,p}$$

示例

Ex2.3 设 $p=2, U=\{0,0,0,1,2,3,4,4,5,5,5\}$ 和 $u=5/2$ (见图 2.6)。那么 $i=4$ ，因为 $u \in [u_4, u_5)$ 。因此，我们计算



将 $u=5/2$ 代入方程 (2.5)(读者应自行完成此操作) 得到

$$N_{4,0}\left(\frac{5}{2}\right)=1$$

$$N_{3,1}\left(\frac{5}{2}\right)=\frac{1}{2}N_{4,1}\left(\frac{5}{2}\right)=\frac{1}{2}$$

$$N_{2,2}\left(\frac{5}{2}\right)=\frac{1}{8}N_{3,2}\left(\frac{5}{2}\right)=\frac{6}{8}N_{4,2}\left(\frac{5}{2}\right)=\frac{1}{8}$$

注意，对于固定的度数，函数的和为 1(P2.4)。

对于在本例中进行替换的读者来说，很明显方程 (2.5) 中存在大量冗余计算。例如，写出一般形式的二次函数，我们有

$$N_{i-2,2}(u)=\frac{u-u_{i-2}}{u_i-u_{i-2}}N_{i-2,1}(u)+\frac{u_{i+1}-u}{u_{i+1}-u_{i-1}}N_{i-1,1}(u) \tag{2.14}$$

$$N_{i-1,2}(u)=\frac{u-u_{i-1}}{u_{i+1}-u_{i-1}}N_{i-1,1}(u)+\frac{u_{i+2}-u}{u_{i+2}-u_i}N_{i,1}(u) \tag{2.15}$$

$$N_{i,2}(u)=\frac{u-u_i}{u_{i+2}-u_i}N_{i,1}(u)+\frac{u_{i+3}-u}{u_{i+3}-u_{i+1}}N_{i+1,1}(u) \tag{2.16}$$

注意

- 方程 (2.14) 的第一项和方程 (2.16) 的最后一项未被计算，因为 $N_{i-2,1}(u)=N_{i+1,1}(u)=0$ ；
- 表达式

$$\frac{N_{i-1,1}(u)}{u_{i+1}-u_{i-1}}$$

出现在方程 (2.14) 的第二项中的表达式出现在方程 (2.15) 的第一项中；方程 (2.15) 的第二项和方程 (2.16) 的第一项也有类似的情况。

我们引入符号

$$\text{left}\ [j]=u-u_{i+1-j}\ \text{right}\ [j]=u_{i+j}-u$$

方程 (2.14)-(2.16) 然后是

$$N_{i-2,2}(u)=\frac{\text{left}\ [3]}{\text{right}\ [0]+\ \text{left}\ [3]}N_{i-2,1}(u)+\frac{\text{right}\ [1]}{\text{right}\ [1]+\ \text{left}\ [2]}N_{i-1,1}(u)$$

$$N_{i-1,2}(u)=\frac{\text{left}\ [2]}{\text{right}\ [1]+\ \text{left}\ [2]}N_{i-1,1}(u)+\frac{\text{right}\ [2]}{\text{right}\ [2]+\ \text{left}\ [1]}N_{i,1}(u)$$

$$N_{i,2}(u)=\frac{\text{left}\ [1]}{\text{right}\ [2]+\ \text{left}\ [1]}N_{i,1}(u)+\frac{\text{right}\ [3]}{\text{right}\ [3]+\ \text{left}\ [0]}N_{i+1,1}(u)$$

基于这些观察，算法 A2.2 计算所有非消失基函数并将其存储在数组 $\mathrm{N}[0], \ldots, \mathrm{N}[\mathrm{p}]$ 中。

ALGORITHM A2.2

```
BasisFuns(i, u, p, U, N)
    { /* Compute the nonvanishing basis functions */
      /* Input: i, u, p, U */
      /* Output: N */
    N[0] = 1.0 ;
    for (j=1; j<=p; j++)
        {
        left[j] = u-U[i+1-j];
        right[j] = U[i+j]-u;
        saved = 0.0 ;
        for (r=0; r<j; r++)
          {
          temp = N[r] / (right[r + 1] + left[j − r]) ;
          N[r] = saved+right[r+1]*temp;
          saved = left[j-r]*temp;
          }
        N[j] = saved;
        }
    }
```

我们指出，算法 A2.2 不仅高效，而且还保证不会出现除以零的情况，这在直接应用方程 (2.5) 时可能发生。

现在谈到第三个算法：特别是，我们想要计算所有 $N_{r,p}^{(k)}(u)$ ，对于 $i-p \le r \le i$ 和 $0 \le k \le n$ ，其中 $n \le p$ 。检查方程 (2.10) 显示基本成分为:

- 在算法 A2.2 中计算的非零基函数的倒三角形；
- 节点的差异 (和: 右 $[r+1]+$ 左 $[j-r]$)，同样在算法 A2.2 中计算；
- $a_{k,j}$ 的差异；注意 $a_{k,j}$ 依赖于 $a_{k-1,j}$ 但不依赖于 $a_{s,j}$ ，对于 $s<k-1$ 。

视为维度为 $(p+1)\times(p+1)$ 的二维数组，基函数适合于上三角形 (包括对角线)，而节点差异适合于下三角形，即

$$\text{max width}=\begin{array}{|c|c|c|}\hline N_{i,0}(u) & N_{i-1,1}(u) & N_{i-2,2}(u)\\ \hline u_{i+1}-u_i & N_{i,1}(u) & N_{i-1,2}(u)\\ \hline u_{i+1}-u_{i-1} & u_{i+2}-u_i & N_{i,2}(u)\\ \hline\end{array}$$

示例

Ex2.4 设 $p=2, U=\{0,0,0,1,2,3,4,4,5,5,5\}$ ，和 $u=5/2$ 。则 $u \in [u_4, u_5)$ ，数组变为

$$\text{max width}=\begin{array}{|c|c|c|}\hline N_{4,0}\left(\frac{5}{2}\right)=1 & N_{3,1}\left(\frac{5}{2}\right)=\frac{1}{2} & N_{2,2}\left(\frac{5}{2}\right)=\frac{1}{8}\\ \hline u_5-u_4=1 & N_{4,1}\left(\frac{5}{2}\right)=\frac{1}{2} & N_{3,2}\left(\frac{5}{2}\right)=\frac{6}{8}\\ \hline u_5-u_3=2 & u_6-u_4=2 & N_{4,2}\left(\frac{5}{2}\right)=\frac{1}{8}\\ \hline\end{array}$$

现在计算 $N_{4,2}^{(1)}(5/2)$ 和 $N_{4,2}^{(2)}(5/2)$; 结合方程 (2.10) 中的 $i=4$ ，我们得到

$$a_{1,0}=\frac{1}{u_6-u_4}=\frac{1}{2}$$

$$a_{1,1}=-\frac{1}{u_7-u_5}=-1$$

$$a_{2,0}=\frac{a_{1,0}}{u_5-u_4}=\frac{1}{2}$$

$$a_{2,1}=\frac{a_{1,1}-a_{1,0}}{u_6-u_5}=\frac{-1-\frac{1}{2}}{4-3}=-\frac{3}{2}$$

$$a_{2,2}=-\frac{a_{1,1}}{u_7-u_6}=\frac{1}{4-4}=\frac{1}{0}$$

和

$$N_{4,2}^{(1)}=2\left[a_{1,0}N_{4,1}\left(\frac{5}{2}\right)+a_{1,1}N_{5,1}\left(\frac{5}{2}\right)\right]$$

$$N_{4,2}^{(2)}=2\left[a_{2,0}N_{4,0}\left(\frac{5}{2}\right)+a_{2,1}N_{5,0}\left(\frac{5}{2}\right)+a_{2,2}N_{6,0}\left(\frac{5}{2}\right)\right]$$

现在 $a_{1,1}, a_{2,1}$ 和 $a_{2,2}$ 都使用不在数组中的节点差异，但它们分别乘以 $N_{5,1}(5/2), N_{5,0}(5/2)$ ，并且 $N_{6,0}(5/2)$ ，同样不在数组中。这些项被定义为零，剩下的就是

$$N_{4,2}^{(1)} = 2a_{1,0}N_{4,1}\left(\frac{5}{2}\right) = \frac{1}{2}$$

$$N_{4,2}^{(2)} = 2a_{2,0}N_{4,0}\left(\frac{5}{2}\right) = 1$$

为了检查这些值，请回忆第 2.2 节中的 $N_{4,2}(u) = 1/2(u-2)^2$ 在 $u \in [2,3)$ 上。计算 $N_{3,2}^{(1)}(5/2), N_{3,2}^{(2)}(5/2), N_{2,2}^{(1)}(5/2)$ 和 $N_{2,2}^{(2)}(5/2)$ 是类似的。

基于这些观察 (以及 Ex2.4)，开发算法 A2.3 并不困难，该算法计算非零基函数及其导数，直到并包括 n 阶导数 $(n \le p)$ 。输出在二维数组 ders 中。ders [k] [j] 是函数 $N_{i-p+j,p}$ 的 k 阶导数，其中 $0 \le k \le n$ 和 $0 \le j \le p$ 。使用了两个局部数组:

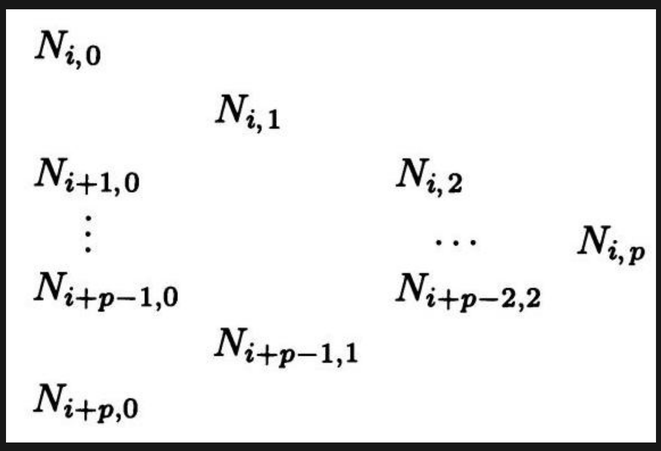
- ndu $[p+1][p+1]$ ，用于存储基函数和节点差异；
- a [2] [p+1]，以交替方式存储最近计算的两行 $a_{k,j}$ 和 $a_{k-1,j}$ 。

该算法避免了除以零和/或使用不在数组 ndu[] [] 中的项。

ALGORITHM A2.3

```
DersBasisFuns(i, u, p, n, U, ders)
    { /* Compute nonzero basis functions and their */
      /* derivatives. First section is A2.2 modified */
        /* to store functions and knot differences. */
        /* Input: i, u, p, n, U */
        /* Output: ders */
    ndu[0][0]=1.0;
    for (j=1; j<=p; j++)
        {
        left[j] = u-U[i+1-j];
        right[j] = U[i+j]-u;
        saved = 0.0 ;
        for (r=0; r<j; r++)
            { /* Lower triangle */
            ndu[j][r] = right[r+1]+left[j-r];
            temp = ndu[r][j-1]/ndu[j][r];
                                                    /* Upper triangle */
            ndu[r][j] = saved+right[r+1]*temp;
            saved = left[j-r]*temp;
          }
        ndu[j][j] = saved;
   }
for (j = 0; j <= p; j + +)  /* Load the basis functions * /
    ders[0][j] = ndu[j][p];
/* This section computes the derivatives (Eq. [2.9]) */
for (r = 0; r <= p; r + +)  /* Loop over function index * /
    {
    s1=0; s2=1; /* Alternate rows in array a */
    a [0] [0] = 1.0;
    /* Loop to compute kth derivative */
    for (k=1; k<=n; k++)
      {
      d = 0.0 ;
      rk = r − k; pk = p − k;
      if (r >= k)
        {
          a[s2][0] = a[s1][0]/ndu[pk+1][rk];
        d = a[s2][0]*ndu[rk][pk];
        }
      if (rk >= −1)  j1 = 1 ;
        else j1 = -rk;
      if (r − 1 <= pk)  j2 = k − 1 ;
        else j2 = p-r;
      for (j=j1; j<=j2; j++)
        {
        a[s2][j] = (a[s1][j]-a[s1][j-1])/ndu[pk+1][rk+j];
        d += a[s2][j]*ndu[rk+j][pk];
        }
      if (r <= pk)
        {
          a [s2] [k] = −a [s1] [k − 1] /ndu [pk + 1] [r] ;
        d += a[s2][k]*ndu[r][pk];
        }
      ders [k] [r] = d ;
      j=s1; s1=s2; s2=j; /* Switch rows */
      }
    }
/* Multiply through by the correct factors */
/* (Eq. [2.9]) */
r = p;
for (k=1; k<=n; k++)
    {
    for (j=0; j<=p; j++) ders[k][j] *= r;
    r *= (p-k);
    }
}
```

现在我们将注意力转向最后两个算法，即计算单个基函数 $N_{i,p}(u)$ 或单个基函数的导数 $N_{i,p}^{(k)}(u)$ 。这些问题的解决方案产生了如下形式的三角表:



示例

Ex2.5 设 $p = 2, U = \{0,0,0,1,2,3,4,4,5,5,5\}$ ，并且 $u = 5/2$ 。计算 $N_{3,2}(5/2)$ 得到

$$N_{3,0}(5/2) = 0$$

$$N_{3,1}(5/2) = \frac{1}{2}$$

$$N_{4,1}(5/2) = \frac{1}{2}$$

$$N_{5,0}(5/2) = 0$$

$N_{4,2}(5/2)$ 是从中获得的

$$N_{4,0}(5/2) = 1$$

$$N_{4,1}(5/2) = \frac{1}{2}$$

$$N_{5,0}(5/2) = 0\ N_{4,2}(5/2) = \frac{1}{8}$$

$$N_{5,1}(5/2) = 0$$

$$N_{6,0}(5/2) = 0$$

请注意，表中非零条目的位置和相对数量依赖于 p 以及第一列中 1 的位置。算法 A2.4 仅计算非零条目。值 $N_{i,p}(u)$ 在 Nip 中返回；m 是 U ($m+1$ knots) 的高索引。该算法在使用变量 temp 和 saved 时与算法 A2.2 类似。

ALGORITHM A2.4

```
OneBasisFun(p, m, U, i, u, Nip)
    { /* Compute the basis function Nip */
      /* Input: p, m, U, i, u */
      /* Output: Nip */
      if ((i == 0&u == U [0]) ||  / * Special */ )
            (i == m-p-1 && u == U[m])) /* cases */
        {
        Nip = 1.0; return;
        }
      if (u < U [i] || u >= U [i + p + 1])  /* Local property * /
          {
        Nip = 0.0; return;
        }
      for (j = 0; j <= p; j + +) /* Initialize zeroth-degree functs * /
          if (u >= U [i + j]&&u < U [i + j + 1]) N [j] = 1.0 ;
            else N[j] = 0.0 ;
      for (k = 1; k <= p; k + +) /* Compute triangular table */
          {
          if (N [0] == 0.0) saved = 0.0 ;
            else saved = ((u − U [i]) * N [0]) / (U [i + k] − U [i]) ;
          for (j = 0; j < p − k + 1; j + +)
            {
            Uleft = U[i+j+1];
            Uright = U[i+j+k+1];
            if (N [j + 1] == 0.0)
              {
              N [j] = saved; saved = 0.0 ;
              }
              else
              {
              temp = N [j + 1] / ( Uright-Uleft ) ;
              N[j] = saved+(Uright-u)*temp;
              saved = (u-Uleft)*temp;
              }
            }
          }
      Nip = N[0];
      }
```

Now for fixed i , the computation of the derivatives, $N_{i,p}^{(k)}(u)$, for $k = 0, \ldots, n$, $n \le p$, uses Eq. (2.9). For example, if $p = 3$ and $n = 3$, then

$$N_{i,3}^{(1)} = 3\left(\frac{N_{i,2}}{u_{i+3} - u_i} - \frac{N_{i+1,2}}{u_{i+4} - u_{i+1}}\right)$$

$$N_{i,3}^{(2)} = 3\left(\frac{N_{i,2}^{(1)}}{u_{i+3} - u_i} - \frac{N_{i+1,2}^{(1)}}{u_{i+4} - u_{i+1}}\right)$$

$$N_{i,3}^{(3)} = 3\left(\frac{N_{i,2}^{(2)}}{u_{i+3} - u_i} - \frac{N_{i+1,2}^{(2)}}{u_{i+4} - u_{i+1}}\right)$$

76 B-样条基函数

使用三角表，我们必须计算

$$k = 0 :$$

$$N_{i,0}$$

$$N_{i,1}$$

$$N_{i+1,0}\ N_{i,2}$$

$$N_{i+2,0}\ N_{i+1,2}$$

$$N_{i+2,1}$$

$$N_{i+3,0}$$

$$k = 1 :$$

$$k = 2 :$$

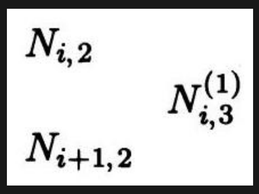
$$N_{i,1}$$

$$N_{i,2}^{(1)}$$

$$N_{i+1,1}\ N_{i,3}^{(2)}$$

$$N_{i+1,2}^{(1)}$$

$$N_{i+2,1}$$



$k=3$:

$N_{i,0}$

$N_{i,1}^{(1)}$

$N_{i+1,0}$ $N_{i,2}^{(2)}$

$N_{i+1,1}^{(1)}$ $N_{i,3}^{(3)}$

$N_{i+2,0}$ $N_{i+1,2}^{(2)}$

$N_{i+2,1}^{(1)}$

$N_{i+3,0}$

用语言描述，该算法为:

1. 计算并存储与 $k=0$ 对应的整个三角表；

2. 要获得 k 阶导数，加载包含 $p-k$ 次的函数的表列，并计算三角形的其余部分。

算法 A2.5 计算 $N_{i,p}^{(k)}(u)$ 对于 $k=0,\ldots,n, n \leq p$ 。k 阶导数在 ders [k] 中返回。

算法 A2.5

DersOneBasisFun(p, m, U, i, u, n, ders)

{ /* 计算基函数 Nip 的导数 */

/* Input: p, m, U, i, u, n */

/* Output: ders */

if $(u<U[i]\ ||u>=U[i+p+1])$ /* Local property */

{

for $(k=0;k<=n;k++)$ ders $[k]=0.0$;

return;

}

for $(j=0;j<=p;j++)$ /* Initialize zeroth-degree functs */

if $(u)=U[i+j]\&\&u<U[i+j+1]$ $N[j][0]=1.0$;

else N[j][0] = 0.0 ;

for $(k=1;k<=p;k++)$ /* Compute full triangular table */

$$
\begin{cases}
u_{0} = 0, \\
u_{1} = 0, \\
u_{2} = 0, \\
u_{3} = 0, \\
u_{4} = 0, \\
u_{5} = 0, \\
u_{6} = 0, \\
u_{7} = 0, \\
u_{8} = 0, \\
u_{9} = 0, \\
u_{10} = 0, \\
u_{11} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{14} = 0, \\
u_{15} = 0, \\
u_{16} = 0, \\
u_{21} = 0, \\
u_{22} = 0, \\
u_{22} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{22} = 0, \\
u_{33} = 0, \\
u_{44} = 0, \\
u_{44} = 0, \\
u_{44

```
            {
            Uleft = U[i+j+1];
            Uright = U[i+j+k+1];
            if (N[j + 1][k − 1] == 0.0)
               {
               N[j][k] = saved; saved = 0.0 ;
               }
               else
               {
               temp = N[j + 1][k − 1] / ( Uright-Uleft ) ;
               N[j][k] = saved+(Uright-u)*temp;
               saved = (u-Uleft)*temp;
               }
            }
      }
   ders[0] = N[0][p]; /* The function value */
   for (k = 1; k <= n; k + +) /* Compute the derivatives */
      {
      for (j = 0; j <= k; j + +) /* Load appropriate column */
            ND[j] = N[j][p-k];
      for (jj = 1; jj <= k; jj + +) /* Compute table of width k */
            {
            if (ND[0] == 0.0) saved = 0.0 ;
               else saved = ND[0] / (U[i + p − k + jj] − U[i]) ;
            for (j=0; j<k-jj+1; j++)
               {
               Uleft = U[i+j+1];
               Uright = U[i+j+p+jj+1];
               if (ND[j+1] == 0.0)
                  {
                  ND[j] = (p-k+jj)*saved; saved = 0.0;
                  }
                  else
                  {
                  temp = ND[j + 1] / ( Uright-Uleft ) ;
                  ND[j] = (p-k+jj)*(saved-temp);
                  saved = temp;
                  }
            }
         }
      ders[k] = ND[0]; /* kth derivative */
      }
   }
```

最后，请注意算法 A2.3 和 A2.5 在 u 是节点时从右侧计算导数。然而，本章中的方程 (2.5)、(2.9)、(2.10) 以及其他方程可以使用形式为 $u \in (u_i, u_{i+1}]$ 的区间来定义。这不会改变算法 A2.2 到 A2.5。换句话说，从左侧获得导数只需让跨度查找算法使用形式为 $(u_i, u_{i+1}]$ 的区间，而不是 $[u_i, u_{i+1})$ 。在前面的例子中，若 $p = 2$ 和 $U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$ ，如果 $u = 2$ ，则跨度 $i = 3$ 产生来自左侧的导数，而 $i = 4$ 产生来自右侧的导数。

练习

2.1. 考虑之前计算的线性和二次函数，并在图 2.5 和 2.6 中显示。将 $u = 5/2$ 代入多项式方程以获得 $N_{3,1}(5/2)$ 、$N_{4,1}(5/2)$, $N_{2,2}(5/2)$, $N_{3,2}(5/2)$ 和 $N_{4,2}(5/2)$ 。你注意到两个线性函数的和以及三个二次函数的和有什么不同吗？

2.2. 考虑图 2.6 中的二次函数。使用 $N_{3,2}(u)$ 的多项式表达式，在 $u = 2$ 处从左侧和右侧评估该函数及其一阶和二阶导数。观察连续性。性质 P2.5 是否成立？在 $u = 4$ 处对 $N_{4,2}(u)$ 进行相同的操作。

2.3. 设 $U = \{0, 0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5, 5\}$ 。这如何改变图 2.4-2.6 中的 0、1 和 2 次函数？计算并绘制与 U 相关的九个三次基函数。

2.4. 考虑图 2.5, $N_{2,2}(u) = 1/2u^2$ 中的函数 $N_{2,2}(u)$ 在 $[0, 1)$, $-3/2 + 3u - u^2$ 上的表现，以及在 $[1, 2)$ 和 $1/2(3 - u)^2$ 上的表现。使用公式 (2.10) 来获得 $N_{2,2}(u)$ 的一阶和二阶导数的表达式。

2.5. 再次考虑图 2.5 中的 $N_{2,2}(u)$ 。通过直接对多项式表达式进行求导，获得 $N_{2,1}$ 和 $N_{3,1}$ 的一阶导数。然后结合公式 (2.11) 来获得 $N'_{2,2}$ 。

2.6. 再次设定 $p = 2, u = 5/2$ 和 $U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$ 。手动跟踪算法 A2.2，以找到三个非零基函数的值。跟踪算法 A2.3，以找到基函数的一阶和二阶导数。

2.7. 使用与练习 2.6 中相同的 p 和 U ，设定 $u = 2$ 。使用 $n = 1$ 跟踪算法 A2.3，一次使用 $i = 3$ ，一次使用 $i = 4$ 。然后对第 2.2 节中给出的 $N_{j,2}$ 的适当多项式表达式进行求导，并在 $u = 2$ 处从左侧和右侧评估导数。将结果与从算法 A2.3 获得的结果进行比较。

2.8. 使用与练习 2.6 中相同的 p 和 U ，设定 $u = 4$ 。跟踪算法 A2.2 和 A2.3，以确保没有双重节点的问题。

2.9. 使用与练习 2.6 中相同的 p 和 U ，设定 $u = 5/2$ 。跟踪算法 A2.5 并计算 $k = 0, 1, 2$ 的导数 $N^{(k)}_{4,2}(5/2)$ 。

第三章　B 样条曲线和曲面

1　引言

在本章中，我们定义了非有理 B 样条曲线和曲面，研究它们的性质，并推导出它们的导数表达式。为简洁起见，本章其余部分将省略“非有理”一词。主要目标是获得对 B 样条曲线和曲面的直观理解，为此，读者应仔细研究本章中给出的许多示例和图形。我们还提供了计算 B 样条曲线和曲面上的点及其导数的算法。Gordon 和 Riesenfeld 首次提出使用 B 样条定义计算机辅助几何设计的曲线和曲面 [Gord74b; Ries73]。目前，许多关于曲线和曲面的书籍中都涵盖了 B 样条技术 - 参见 [DeBo78; Mort85; Bart87; Fari93; Yama88; Hosc93; Su89; Roge90; Beac91]。

2　B 样条曲线的定义和性质

p 次 B 样条曲线定义为

$$\mathbf{C}(u)=\sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i \quad a\le u\le b \tag{3.1}$$

其中 $\{\mathbf{P}_i\}$ 是控制点，而 $\{N_{i,p}(u)\}$ 是在非周期 (且非均匀) 节点向量上定义的 p 次 B 样条基函数 (公式 [2.5])。

$$U=\left\{\underbrace{a,\ldots,a}_{p+1},u_{p+1},\ldots,u_{m-p-1},\underbrace{b,\ldots,b}_{p+1}\right\}$$

($m+1$ knots) 。除非另有说明，我们假设 $a=0$ 和 $b=1$ 。由 $\{\mathbf{P}_i\}$ 形成的多边形称为控制多边形。B 样条曲线的示例 (在某些情况下与其相应的基函数一起) 显示在图 3.1-3.14 中。

计算固定 u 值下的 B 样条曲线上的点需要三个步骤:

1. 找到 u 所在的节点区间 (算法 A2.1)；
2. 计算非零基函数 (算法 A2.2)；
3. 将非零基函数的值与相应的控制点相乘。

考虑第 2.5 节的例子 Ex2.3，具有 $U=\{0,0,0,1,2,3,4,4,5,5,5\}$ 、$u=5/2$ 和 $p=2$ 。那么 $u\in[u_4,u_5)$ ，并且

$$N_{2,2}\left(\frac{5}{2}\right)=\frac{1}{8}\quad N_{3,2}\left(\frac{5}{2}\right)=\frac{6}{8}\quad N_{4,2}\left(\frac{5}{2}\right)=\frac{1}{8}$$

与控制点相乘得到

$$\mathbf{C}\left(\frac{5}{2}\right)=\frac{1}{8}\mathbf{P}_2+\frac{6}{8}\mathbf{P}_3+\frac{1}{8}\mathbf{P}_4$$

算法如下。

ALGORITHM A3.1:

```
CurvePoint(n, p, U, P, u, C)
  { /* Compute curve point */
    /* Input: n, p, U, P, u */
    /* Output: C */
  span = FindSpan(n, p, u, U) ;
  BasisFuns(span, u, p, U, N);
  C = 0.0 ;
  for (i = 0; i<=p; i++)
    C = C + N[i]*P[span-p+i];
  }
```

现在我们列出 B 样条曲线的一些性质。这些性质源于第 2 章中给出的关于函数 $N_{i,p}(u)$ 的性质。令 $\mathbf{C}(u)$ 由方程 (3.1) 定义。

P3.1 如果 $n=p$ 和 $U=\{0,\ldots,0,1,\ldots,1\}$ ，那么 $\mathbf{C}(u)$ 是一条贝塞尔曲线 (图 3.1)；

P3.2 $\mathbf{C}(u)$ 是一条分段多项式曲线 (因为 $N_{i,p}(u)$ 是分段多项式)；度数 p 、控制点数量 $n+1$ 和节点数量 $m+1$ 之间的关系为

$$m=n+p+1 \tag{3.2}$$

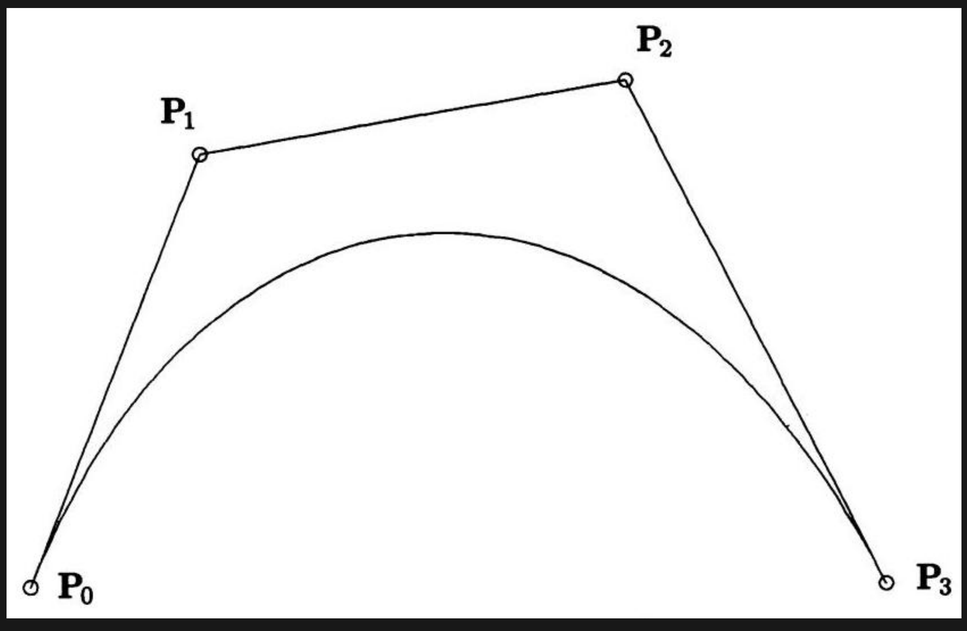


图 3.1. 在 $U=\{0,0,0,0,1,1,1,1\}$ 上的三次 B 样条曲线，即三次贝塞尔曲线。

(见第 2.4 节)。图 3.2 和 3.3 显示了与各个节点区间对应的 B 样条曲线的基函数和截面；在这两幅图中，交替的实线/虚线段对应于定义曲线的不同多项式 (节点区间)。

93.3 端点插值: $\mathbf{C}(0)=\mathbf{P}_0$ 和 $\mathbf{C}(1)=\mathbf{P}_n$ ；

3.4 仿射不变性: 通过对控制点应用仿射变换来对曲线应用仿射变换。令 $\mathbf{r}$ 为 $\mathcal{E}^3$ 中的一个点 (三维欧几里得空间)。仿射变换，记作 Φ ，将 $\mathcal{E}^3$ 映射到 $\mathcal{E}^3$ ，其形式为

$$\Phi(\mathbf{r})=A\mathbf{r}+\mathbf{v}$$

其中 A 是一个 3×3 矩阵，而 $\mathbf{v}$ 是一个向量。仿射变换包括平移、旋转、缩放和剪切。B 样条曲线的仿射不变性属性源于 $N_{i,p}(u)$ 的统一性划分属性。因此，设 $\mathbf{r}=\sum\alpha_i\mathbf{p}_i$ ，其中 $\mathbf{p}_i\in\mathcal{E}^3$ 和 $\sum\alpha_i=1$ 。然后

$$\Phi(\mathbf{r})=\Phi\left(\sum\alpha_i\mathbf{p}_i\right)=A\left(\sum\alpha_i\mathbf{p}_i\right)+\mathbf{v}=\sum\alpha_iA\mathbf{p}_i+\sum\alpha_i\mathbf{v}$$

$$=\sum\alpha_i\left(A\mathbf{p}_i+\mathbf{v}\right)=\sum\alpha_i\Phi(\mathbf{p}_i)$$

P3.5 强凸包属性: 曲线包含在其控制多边形的凸包中；事实上，如果 $u\in[u_i,u_{i+1}),p\le i<m-p-1$ ，则 $\mathbf{C}(u)$ 在控制点 $\mathbf{P}_{i-p},\ldots,\mathbf{P}_i$ 的凸包中 (图 3.4、3.5 和 3.6)。这源于 $N_{i,p}(u)$ 的非负性和统一性划分属性 (属性 P2.3 和 P2.4)，以及当 $u\in[u_i,u_{i+1})$ 时 $N_{j,p}(u)=0$ 对于 $j<i-p$ 和 $j>i$ 的属性 (属性 P2.2)。图 3.6 显示了如何构造一个包含直线段的二次曲线。由于 $\mathbf{P}_2,\mathbf{P}_3$ 和 $\mathbf{P}_4$ 共线，强

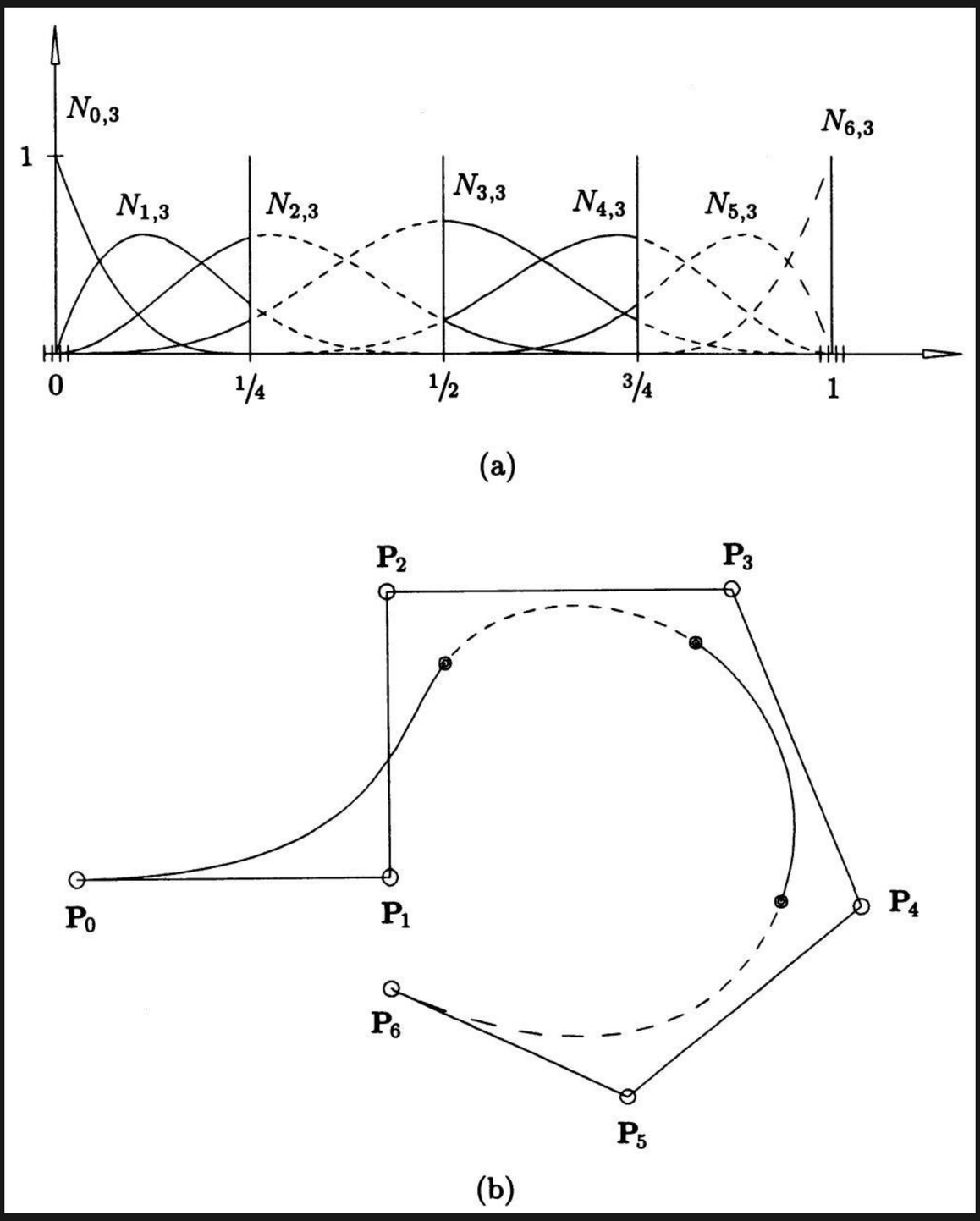


图 3.2. (a) 三次基函数 $U = \{0, 0, 0, 0, 1/4, 1/2, 3/4, 1, 1, 1, 1\}$; (b) 使用图 3.2a 的基函数构造的三次曲线。

凸包属性强制曲线从 $\mathbf{C}(2/5)$ 到 $\mathbf{C}(3/5)$ 是一条直线段；

P3.6 局部修改方案: 移动 $\mathbf{P}_i$ 仅在区间 $[u_i, u_{i+p+1})$ 内改变 $\mathbf{C}(u)$ (图 3.7)。这源于 $u \notin [u_i, u_{i+p+1})$ 的 $N_{i,p}(u) = 0$ (属性 P2.1)。

P3.7 控制多边形表示曲线的分段线性近似；通过节点插入或次数提升可以改善这种近似 (见第 5 章)。一般来说，次数越低，B 样条曲线越接近其控制多边形 (见图 3.8 和 3.9)。图 3.9 中的曲线使用相同的六个控制点定义，并且节点向量

$$p = 1 : U = \left\{0, 0, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, 1, 1\right\}$$

$$p = 2 : U = \left\{0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1, 1\right\}$$

$$p = 3 : U = \left\{0, 0, 0, 0, \frac{1}{3}, \frac{2}{3}, 1, 1, 1, 1\right\}$$

$$p = 4 : U = \left\{0, 0, 0, 0, 0, \frac{1}{2}, 1, 1, 1, 1, 1\right\}$$

$$p = 5 : U = \{0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1\}$$

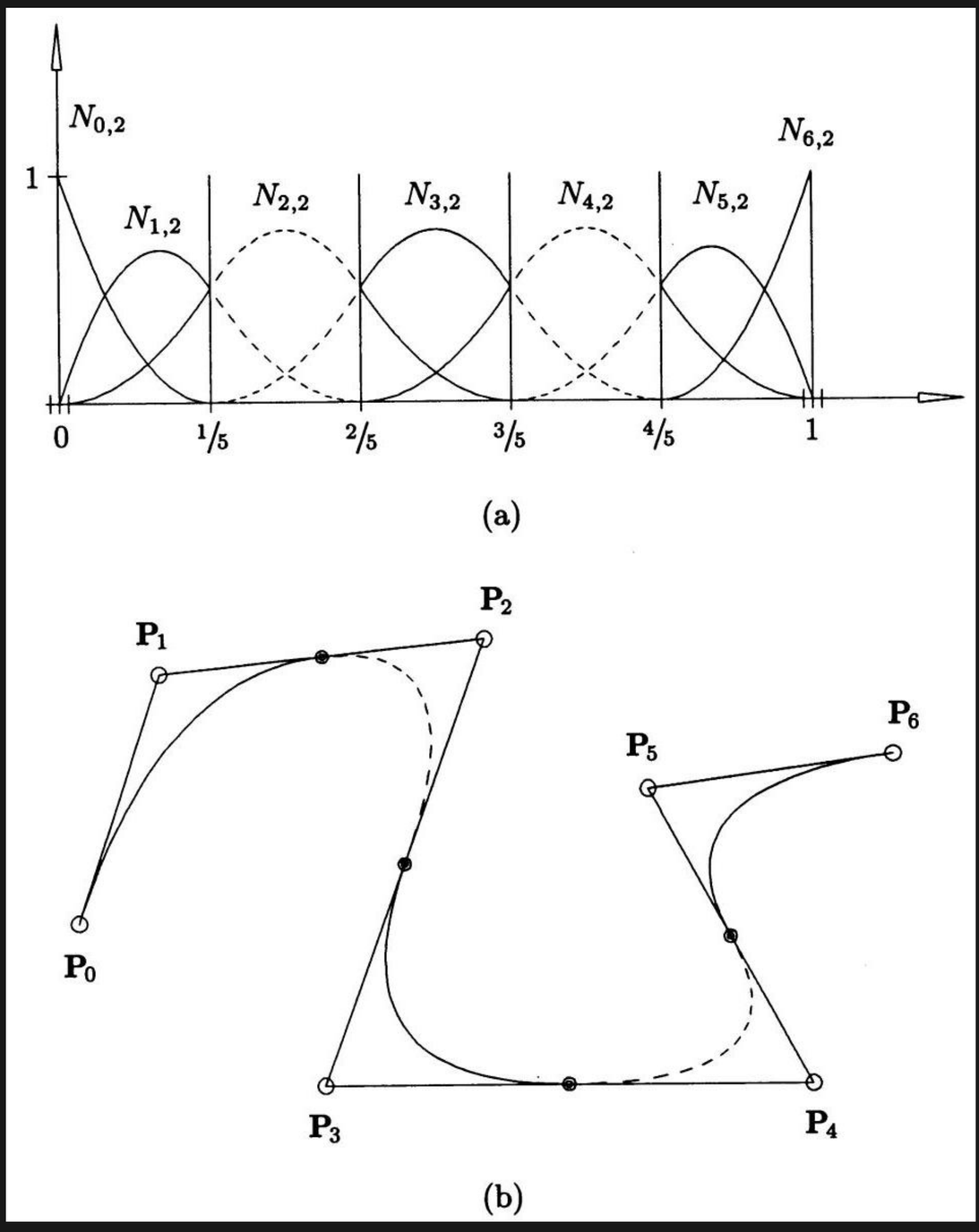


图 3.3. (a) 在 $U = \{0, 0, 0, 1/5, 2/5, 3/5, 4/5, 1, 1, 1\}$ 上的二次基函数；(b) 使用图 3.3a 的基函数绘制的二次曲线。
这一现象的原因是直观的: 次数越低，参与计算的控制点越少

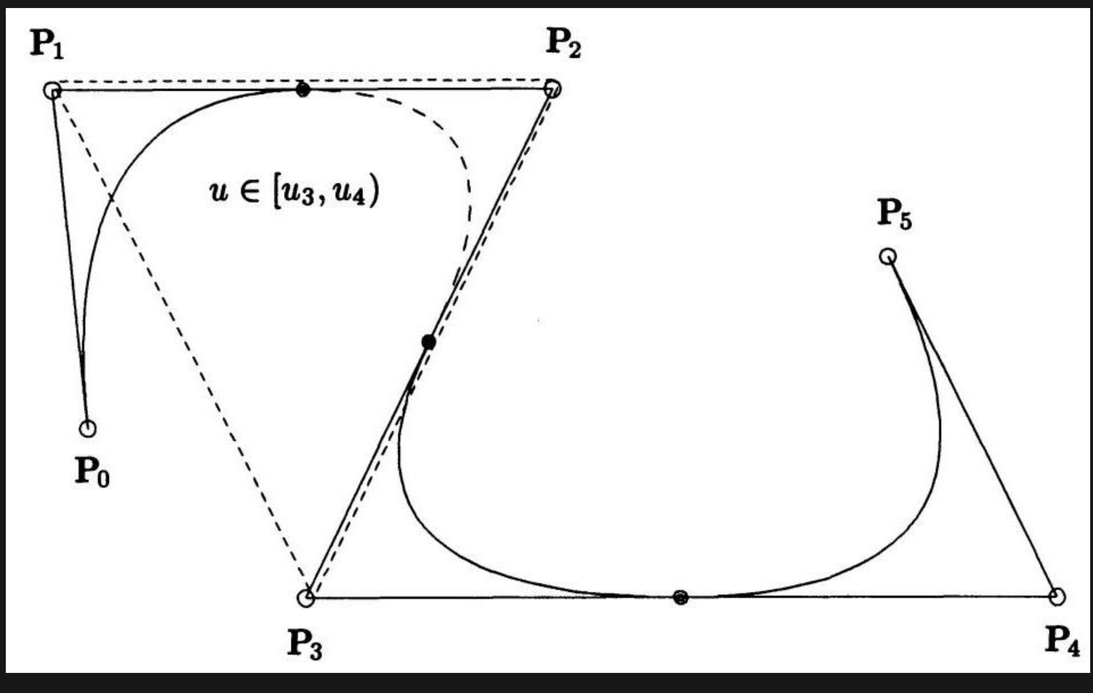


图 3.4. 二次 B 样条曲线的强凸包性质：对于 $u \in [u_i, u_{i+1})$，$\mathbf{C}(u)$ 位于三角形 $\mathbf{P}_{i-2}\mathbf{P}_{i-1}\mathbf{P}_i$ 内。
$\mathbf{C}(u_0)$ 对于任何给定的 u_0 。极端情况是 $p = 1$ ，在这种情况下，每个点 $\mathbf{C}(u)$ 只是两个控制点之间的线性插值。在这种情况下，曲线就是控制多边形；
P3.8 沿着曲线从 $u = 0$ 移动到 $u = 1$ ，$N_{i,p}(u)$ 函数像开关一样起作用：当 u 经过一个节点时，一个 $N_{i,p}(u)$ (以及相应的 $\mathbf{P}_i$) 关闭，下一个则开启 (图 3.2 和 3.3)。

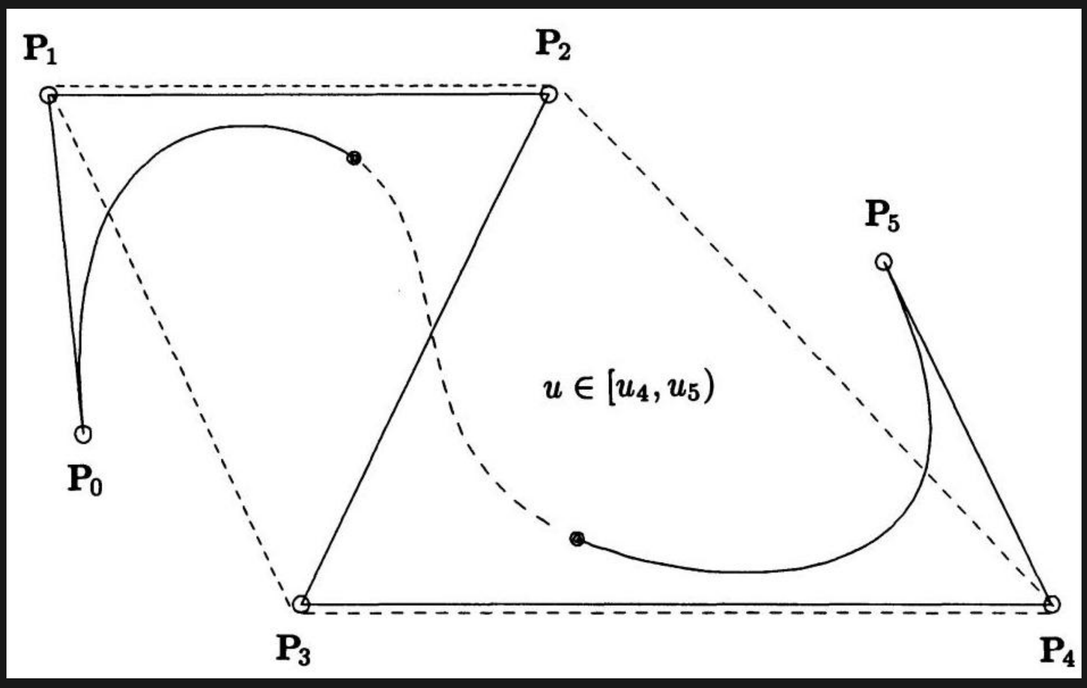


图 3.5. 三次 B 样条曲线的强凸包性质：对于 $u \in [u_i, u_{i+1})$，$\mathbf{C}(u)$ 位于四边形 $\mathbf{P}_{i-3}\mathbf{P}_{i-2}\mathbf{P}_{i-1}\mathbf{P}_i$ 内。

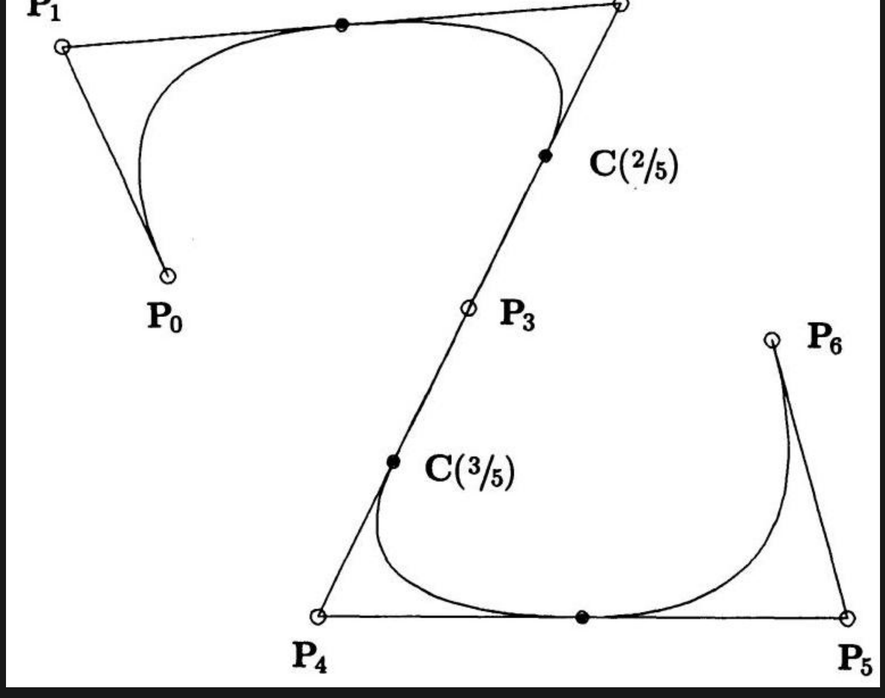


图 3.6. 在 $U = \{0, 0, 0, 1/5, 2/5, 3/5, 4/5, 1, 1, 1\}$ 上的二次 B 样条曲线。该曲线是 $\mathbf{C}(2/5)$ 和 $\mathbf{C}(3/5)$ 之间的直线。

P3.9 变异减少性质: 没有平面与曲线的交点比与控制多边形的交点更多 (对于二维曲线，将“平面”替换为“直线”) - 见 [Lane83] 的证明；

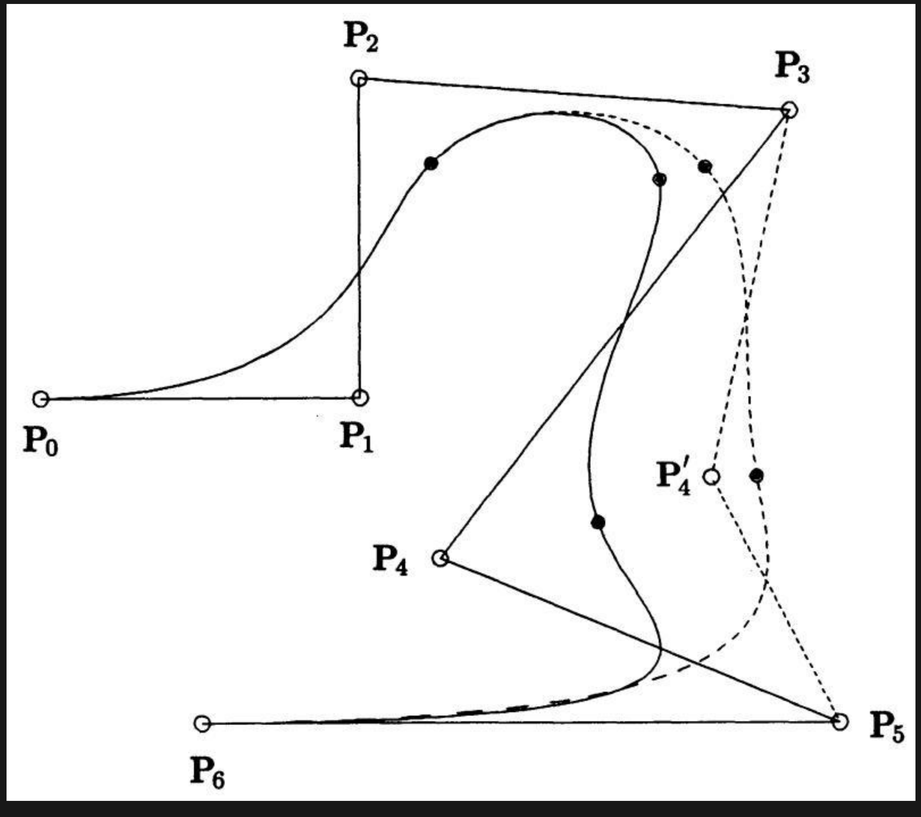


图 3.7. 在 $U = \{0, 0, 0, 0, 1/4, 1/2, 3/4, 1, 1, 1, 1\}$ 上的三次曲线：移动 $\mathbf{P}_4$ (到 $\mathbf{P}'_4$) 会改变区间 $[1/4, 1)$ 中的曲线。

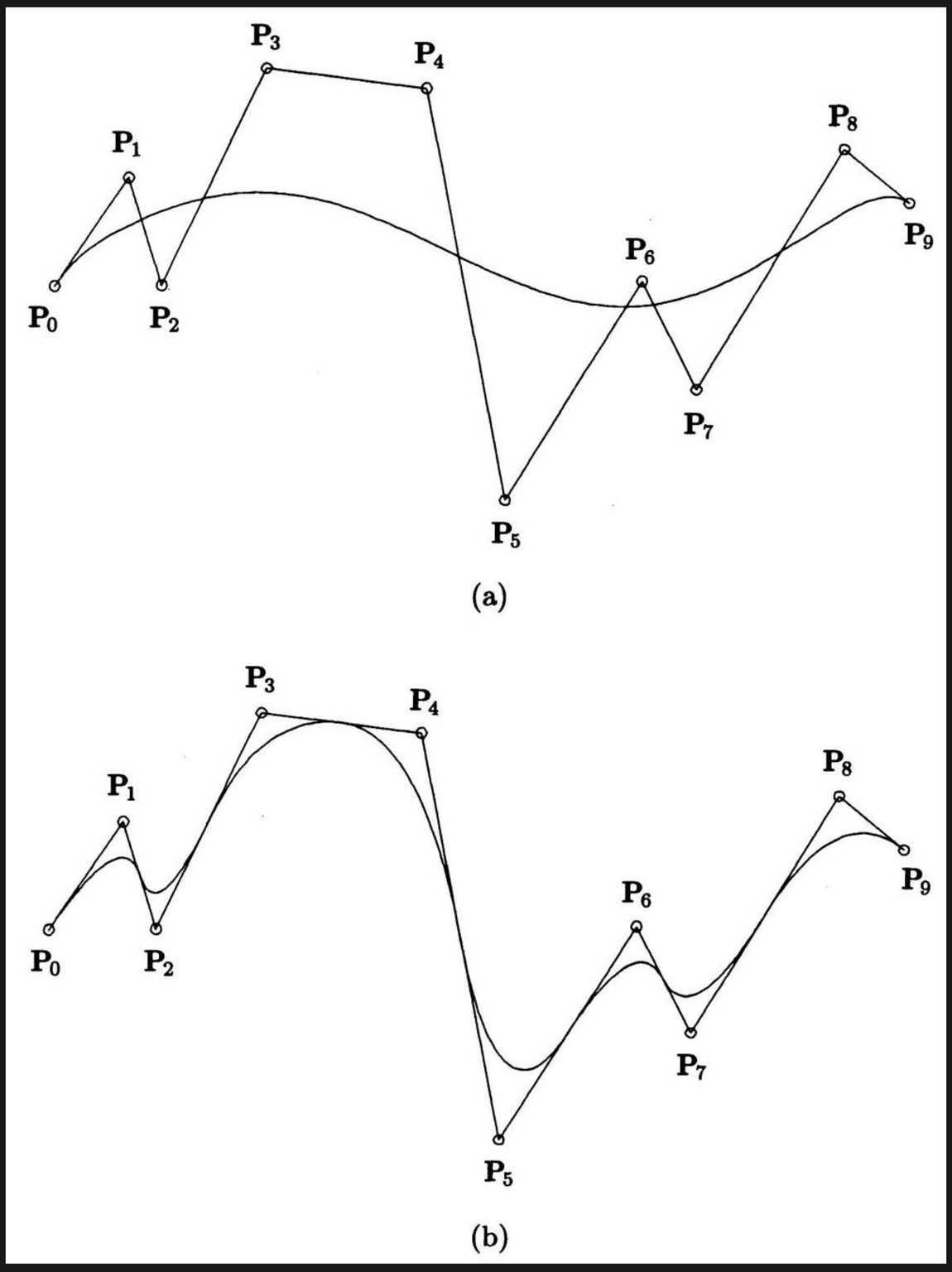


图 3.8. B-spline 曲线。(a) 在节点向量 $U = \{0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1\}$ 上的九次 Bézier 曲线；(b) 使用在 $U = \{0, 0, 0, 1/8, 2/8, 3/8, 4/8, 5/8, 6/8, 7/8, 1, 1, 1\}$ 上定义的相同控制多边形的二次曲线。

P3.10 $\mathbf{C}(u)$ 的连续性和可微性源于 $N_{i,p}(u)$ 的连续性和可微性 (因为 $\mathbf{C}(u)$ 只是 $N_{i,p}(u)$ 的线性组合)。因此，$\mathbf{C}(u)$ 在结点区间的内部是无限可微的，并且在重数为 k 的结点处至少是 $p-k$ 次连续可微的。图 3.10 显示了一条二次曲线 ($p = 2$) 。该曲线在重数为 1 的所有内部结点处是 $\mathbf{C}^1$ 连续的 (第一导数是连续的，但第二导数不是)。在双重结点处，$u = 4/5$, $\mathbf{C}(u)$ 仅是 $\mathbf{C}^0$ 连续的，因此存在一个尖点 (视觉不连续性)。图 3.11 显示了一条在同一结点向量上定义的二次曲线。因此，这两条曲线使用相同的基函数 $N_{i,p}(u)$ 进行定义。但图 3.11 的曲线在 $u = 4/5$ 处是 $\mathbf{C}^1$ 连续的；这并不明显，但可以通过第 3.3 节中给出的导数表达式来观察到。这只是因为不连续函数有时可以以某种方式组合，从而使结果是连续的。注意 $\mathbf{P}_4$ 、$\mathbf{P}_5$ 和 $\mathbf{P}_6$ 是共线的，长度为 $(\mathbf{P}_4\mathbf{P}_5) =$ 长度 $(\mathbf{P}_5\mathbf{P}_6)$ 。图 3.12 显示了一条在 $u = 1/4$ 和 $u = 1/2$ 处是 $\mathbf{C}^2$ 连续的三次曲线，但在双重结点 $u = 3/4$ 处仅是 $\mathbf{C}^1$ 连续的。眼睛能够检测到第二导数的不连续性，但可能无法检测到第三及更高阶导数的不连续性。因此，三次曲线通常适用于视觉目的。

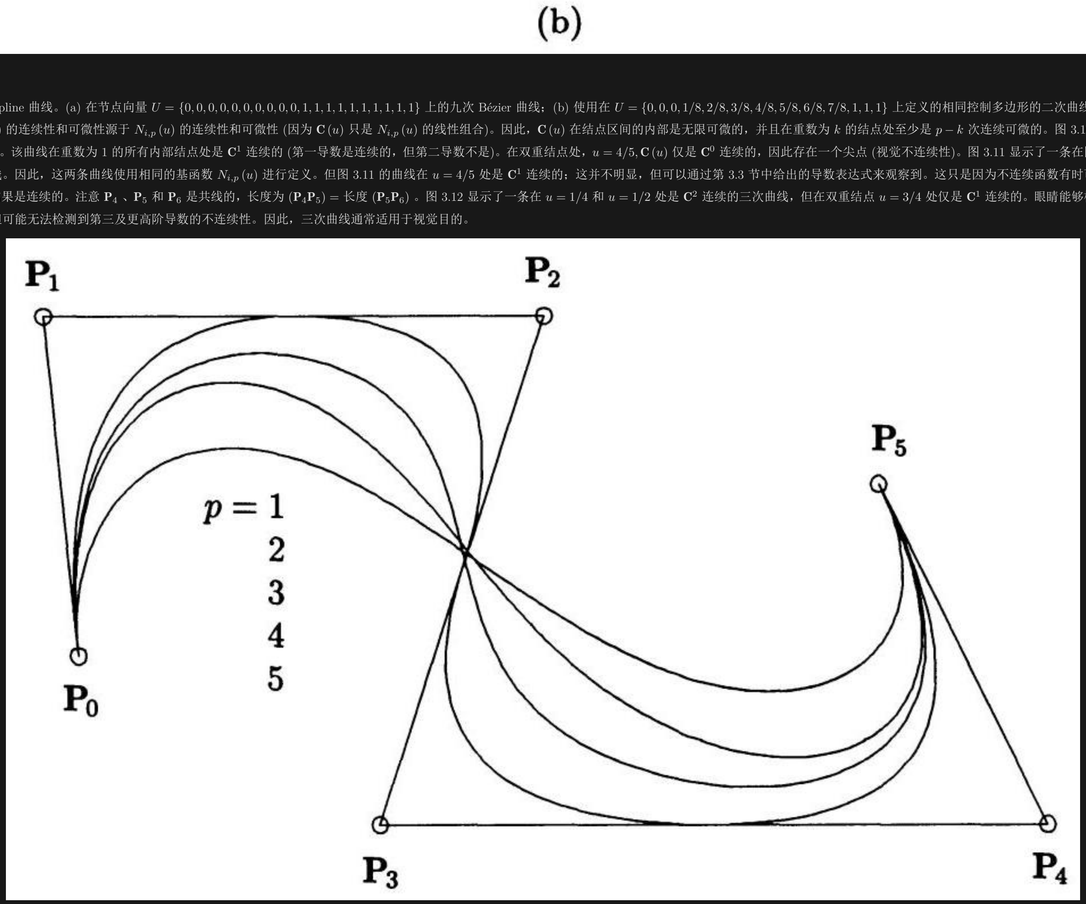


图 3.9. 不同阶数的 B-spline 曲线，使用相同的控制多边形。

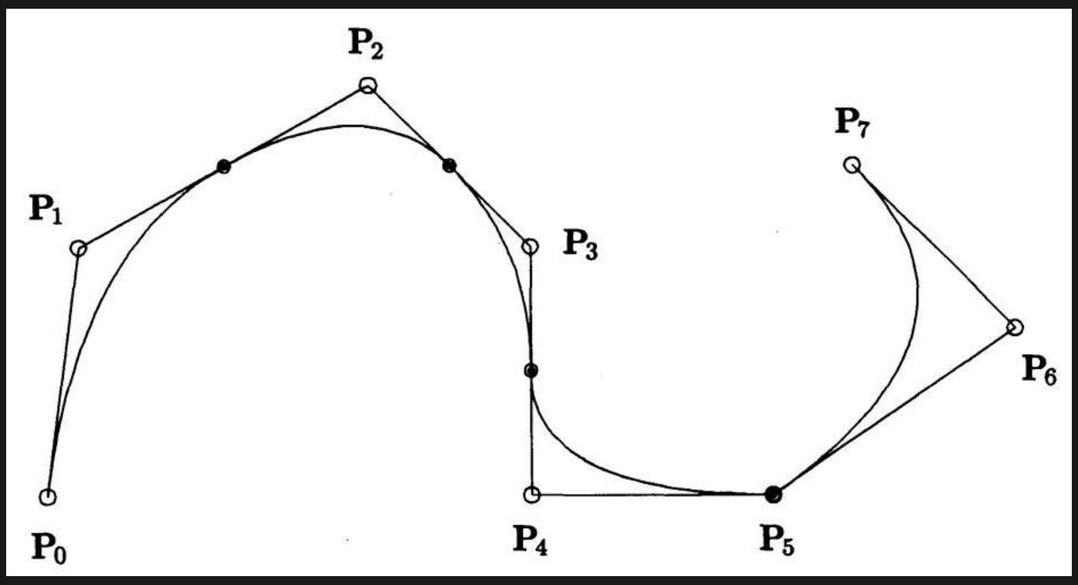


图 3.10. 在 $U = \{0, 0, 0, 1/5, 2/5, 3/5, 4/5, 4/5, 1, 1, 1\}$ 上的二次曲线，在 $u = 4/5$ 处有一个尖点。

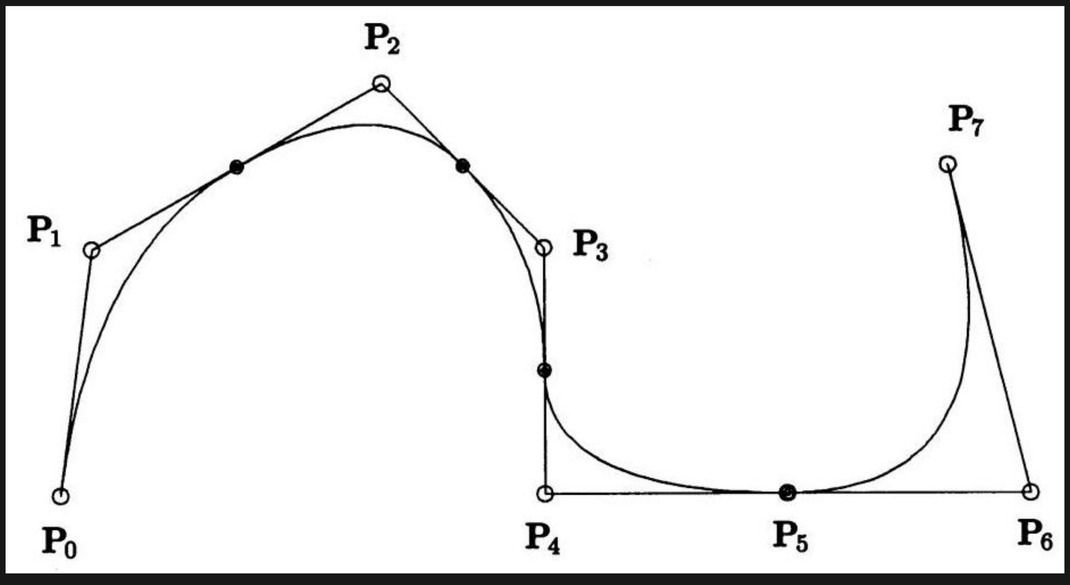


图 3.11. 在 $U = \{0, 0, 0, 1/5, 2/5, 3/5, 4/5, 4/5, 1, 1, 1\}$ 上的二次曲线；在 $u = 4/5$ 处一阶导数是连续的。

P3.11 使用多个 (重合的) 控制点是可能的 (有时也是有用的)。图 3.13 显示了一个具有双重控制点的二次曲线，$\mathbf{P}_2 = \mathbf{P}_3$ 。该曲线的有趣部分位于 $\mathbf{C}(5/2)$ 和 $\mathbf{C}(3/4)$ 之间。实际上，$\mathbf{C}(1/2) = \mathbf{P}_2 = \mathbf{P}_3$ ，并且在 $\mathbf{C}(1/4)$ 和 $\mathbf{C}(1/2)$ 之间，以及 $\mathbf{C}(1/2)$ 和 $\mathbf{C}(3/4)$ 之间的曲线段都是直线。这是由于性质 P3.5 的结果，例如，$\mathbf{C}(u)$ 在 $\mathbf{P}_1\mathbf{P}_2\mathbf{P}_3$ 的凸包 (即一条线) 内，如果 $u \in [1/4, 1/2)$ 。此外，由于节点 $u = 1/2$ 的重数为 $= 1$ ，因此曲线在该处必须是 $\mathbf{C}^1$ 连续的，尽管它有一个尖点 (视觉不连续性)。这是因为在 $u = 1/2$ 处一阶导数向量的大小连续趋向于零的结果。在下一节中，我们将看到在 $u = 1/2$ 处的导数与差值 $\mathbf{P}_3 - \mathbf{P}_2$ 成正比。图 3.14a 和 3.14 b 是使用相同控制多边形的三次示例，包括双重控制点 $\mathbf{P}_2 = \mathbf{P}_3$ ，但具有不同的节点向量。

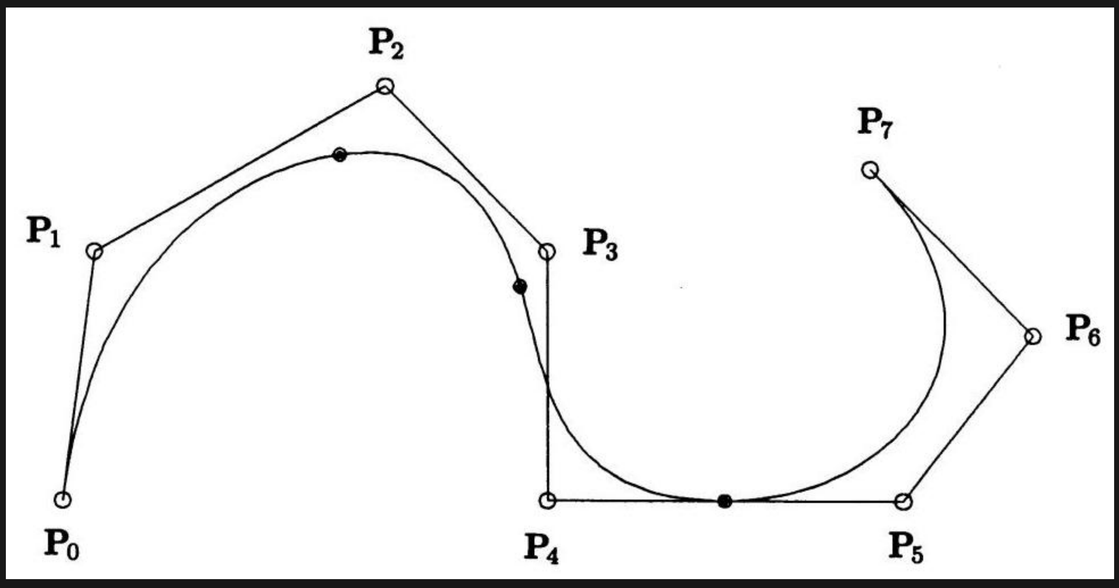


图 3.12. 在 $U = \{0, 0, 0, 0, 1/4, 1/2, 3/4, 3/4, 1, 1, 1, 1\}$, $\mathbf{C}^2$ 上的三次曲线在 $u = 1/4$ 和 $u = 1/2$ 处是连续的，并且在 $\mathbf{C}^1$ 处在 $u = 3/4$ 是连续的。

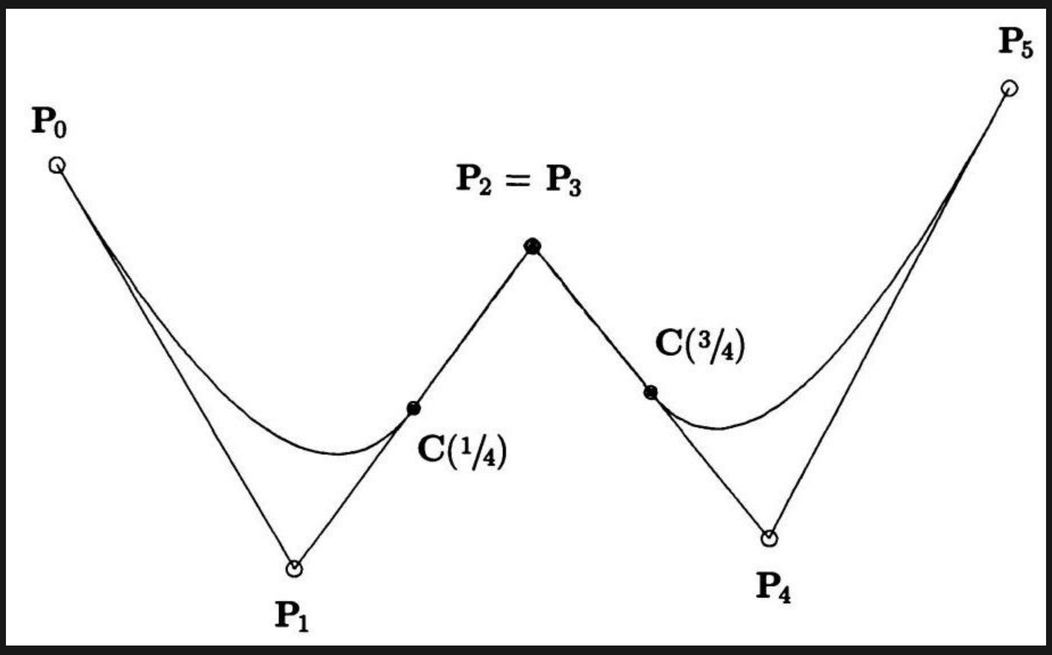


图 3.13. 在 $U = \{0, 0, 0, 1/4, 1/2, 3/4, 1, 1, 1\}$; $\mathbf{P}_2 = \mathbf{P}_3$ 上的二次曲线是一个双重控制点。
控制多边形，包括双重控制点 $\mathbf{P}_2 = \mathbf{P}_3$ ，但具有不同的节点向量。

3 B 样条曲线的导数

令 $\mathbf{C}^{(k)}(u)$ 表示 k 阶导数的 $\mathbf{C}(u)$ 。如果 u 固定，我们可以通过计算基函数的 k 阶导数来获得 $\mathbf{C}^{(k)}(u)$ (见方程 [2.7]、[2.9] 和 [2.10] 以及算法 A2.3)。特别地

$$\mathbf{C}^{(k)}(u) = \sum_{i=0}^{n} N_{i,p}^{(k)}(u)\,\mathbf{P}_i \tag{3.3}$$

考虑第 2.5 节的例子，设 $p = 2, U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$ 和 $u = 5/2$ 。从方程 (2.7) 我们得到

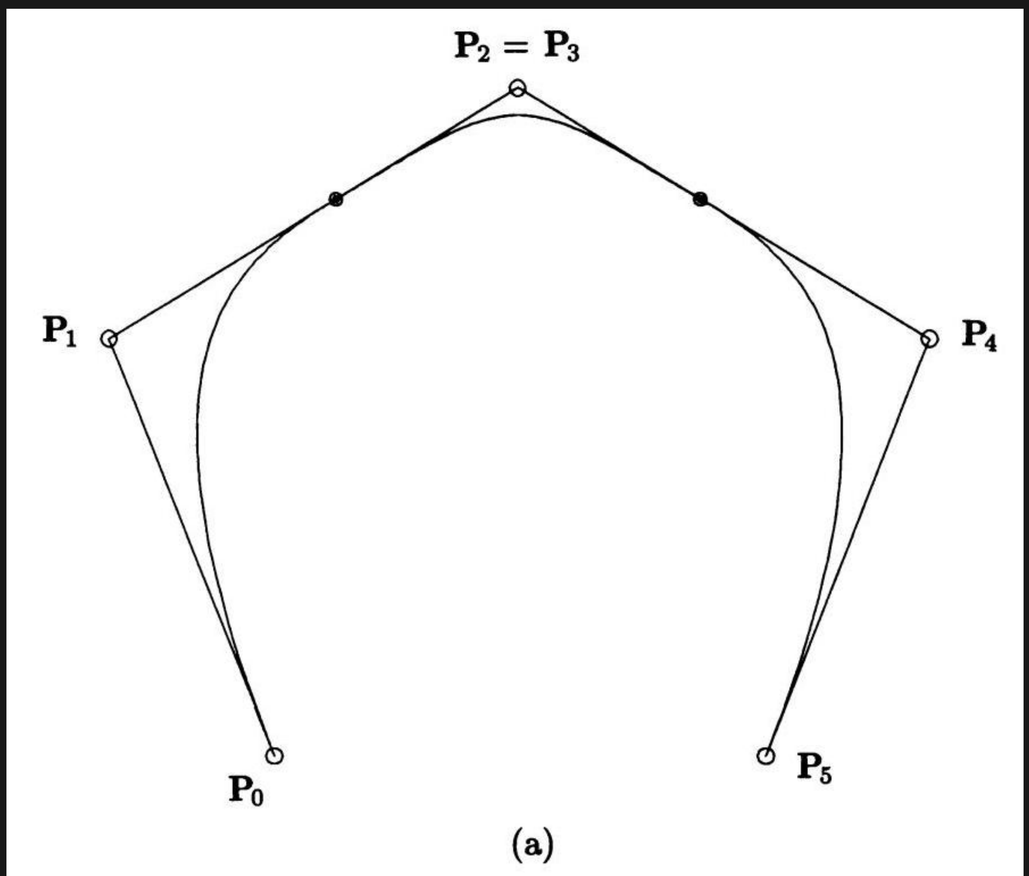
$$N'_{2,2}\left(\frac{5}{2}\right) = 0 - \frac{2}{3-1}\frac{1}{2} = -\frac{1}{2}$$

$$N'_{3,2}\left(\frac{5}{2}\right) = \frac{2}{3-1}\frac{1}{2} - \frac{2}{4-2}\frac{1}{2} = 0$$

$$N'_{4,2}\left(\frac{5}{2}\right) = \frac{2}{4-2}\frac{1}{2} - 0 = \frac{1}{2}$$

由此可得

$$\mathbf{C}'\left(\frac{5}{2}\right) = -\frac{1}{2}\mathbf{P}_2 + \frac{1}{2}\mathbf{P}_4$$



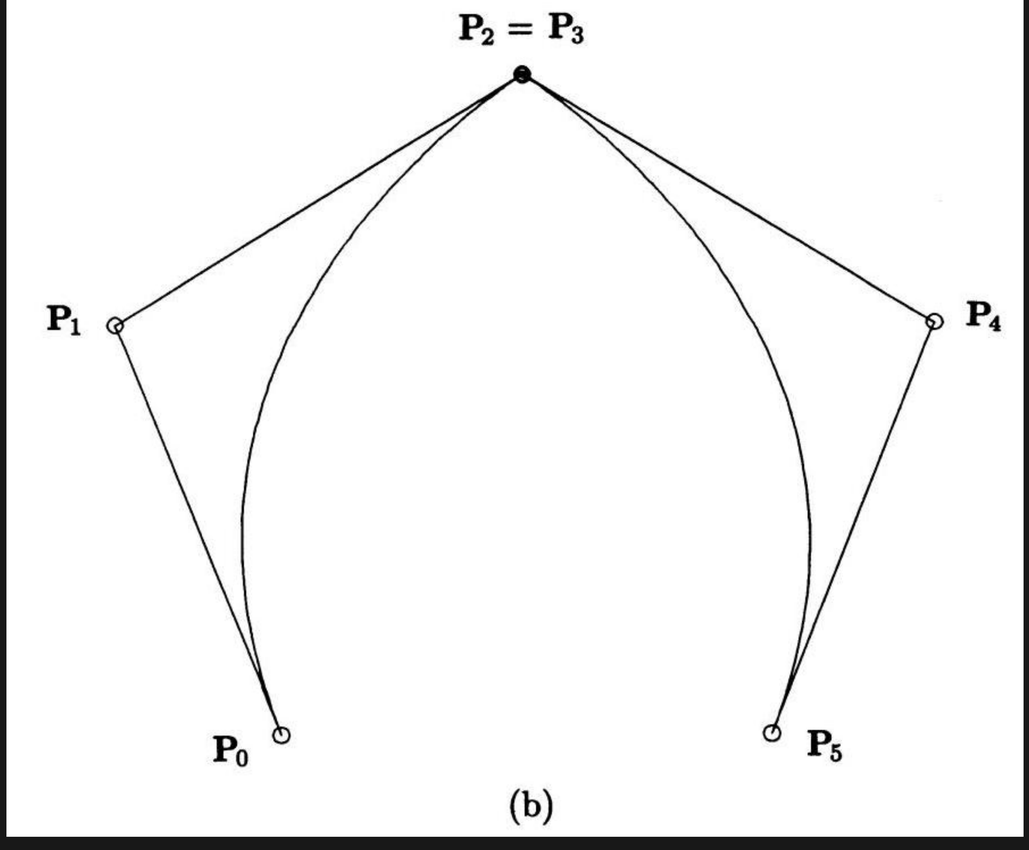


图 3.14. 具有双控制点的三次曲线 $\mathbf{P}_2 = \mathbf{P}_3$ 。(a) $U = \{0, 0, 0, 0, 1/4, 3/4\ ,\ 1, 1, 1, 1\}$; (b) $U = \{0, 0, 0, 0, 1/2, 1/2, 1, 1, 1, 1\}$ 。

一个算法用于计算 B 样条曲线上的点及其所有导数，直到并包括在固定的 u 值下的 d 阶导数。我们允许 $d > p$ ，尽管在这种情况下导数为 0(对于非有理曲线)；这些导数对于有理曲线是必要的。算法的输入为 u, d ，以及 B 样条曲线，通过本书其余部分定义为

n：控制点的数量为 $n + 1$;

p：曲线的阶数；

U：节点；

P：控制点。

输出为数组 CK [] ，其中 CK [k] 是 k 阶导数 $0 \le k \le d$ 。我们使用算法 A2.1 和 A2.3。一个局部数组 nders[][] 用于存储基函数的导数。

ALGORITHM A3.2:

```
CurveDerivsAlg1(n, p, U, P, u, d, CK)
    { /* Compute curve derivatives */
      /* Input: n, p, U, P, u, d */
      /* Output: CK */
    du = min(d, p);
    for (k=p+1; k<=d; k++) CK[k] = 0.0;
    span = FindSpan(n, p, u, U);
    DersBasisFuns(span, u, p, du, U, nders);
    for ( k = 0 ; k <= du ; k + + )
      {
      CK[k] = 0.0 ;
      for (j=0; j<=p; j++)
        CK[k] = CK[k] + nders[k][j]*P[span-p+j];
      }
    }
```

现在，我们想要正式区分 p 阶 B-样条曲线，而不是修正 u 。

样条曲线，

$$\mathbf{C}(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i$$

定义在节点向量上

$$U = \{\underbrace{0, \ldots, 0}_{p+1}, u_{p+1}, \ldots, u_{m-p-1}, \underbrace{1, \ldots, 1}_{p+1}\}$$

从方程 (3.3) 和 (2.7) 我们得到

$$\mathbf{C}'(u)=\sum_{i=0}^{n}N'_{i,p}(u)\,\mathbf{P}_i$$

$$=\sum_{i=0}^{n}\left(\frac{p}{u_{i+p}-u_i}N_{i,p-1}(u)-\frac{p}{u_{i+p+1}-u_{i+1}}N_{i+1,p-1}(u)\right)\mathbf{P}_i$$

$$=\left(p\sum_{i=-1}^{n-1}N_{i+1,p-1}(u)\frac{\mathbf{P}_{i+1}}{u_{i+p+1}-u_{i+1}}\right)$$

$$-\left(p\sum_{i=0}^{n}N_{i+1,p-1}(u)\frac{\mathbf{P}_i}{u_{i+p+1}-u_{i+1}}\right)$$

$$=p\frac{N_{0,p-1}(u)\,\mathbf{P}_0}{u_p-u_0}+p\sum_{i=0}^{n-1}N_{i+1,p-1}(u)\frac{(\mathbf{P}_{i+1}-\mathbf{P}_i)}{u_{i+p+1}-u_{i+1}}-p\frac{N_{n+1,p-1}(u)\,\mathbf{P}_n}{u_{n+p+1}-u_{n+1}}$$

第一个和最后一个项的值为 0/0，根据定义为 0。因此

$$\mathbf{C}'(u)=p\sum_{i=0}^{n-1}N_{i+1,p-1}(u)\frac{(\mathbf{P}_{i+1}-\mathbf{P}_i)}{u_{i+p+1}-u_{i+1}}=\sum_{i=0}^{n-1}N_{i+1,p-1}(u)\,\mathbf{Q}_i$$

其中

$$\mathbf{Q}_i=p\frac{\mathbf{P}_{i+1}-\mathbf{P}_i}{u_{i+p+1}-u_{i+1}}\tag{3.4}$$

现在设 U' 为通过从 U 中去掉第一个和最后一个节点得到的节点向量，即

$$U'=\{\underbrace{0,\ldots,0}_{p},u_{p+1},\ldots,u_{m-p-1},\underbrace{1,\ldots,1}_{p}\}\tag{3.5}$$

(U' 有 $m-1$ 个节点)。然后很容易检查在 U 上计算的函数 $N_{i+1,p-1}(u)$ 等于在 U' 上计算的 $N_{i,p-1}(u)$。因此

$$\mathbf{C}'(u)=\sum_{i=0}^{n-1}N_{i,p-1}(u)\,\mathbf{Q}_i\tag{3.6}$$

其中 $\mathbf{Q}_i$ 由方程 (3.4) 定义，而 $N_{i,p-1}(u)$ 在 U' 上计算。因此，$\mathbf{C}'(u)$ 是一个 (p - 1) 阶 B-样条曲线。

示例

Ex3.1 设 $\mathbf{C}(u)=\sum_{i=0}^{4}N_{i,2}(u)\,\mathbf{P}_i$ 为定义在

$$U=\{0,0,0,2/5,3/5,1,1,1\}$$

然后 $U'=\{0,0,2/5,3/5,1,1\}$ 和 $\mathbf{C}'(u)=\sum_{i=0}^{3}N_{i,1}(u)\,\mathbf{Q}_i$，其中

$$\mathbf{Q}_0=\frac{2(\mathbf{P}_1-\mathbf{P}_0)}{\frac{1}{3}-0}=6(\mathbf{P}_1-\mathbf{P}_0)$$

$$\mathbf{Q}_1=\frac{2(\mathbf{P}_2-\mathbf{P}_1)}{\frac{3}{4}-0}=\frac{8}{3}(\mathbf{P}_2-\mathbf{P}_1)$$

$$\mathbf{Q}_2=\frac{2(\mathbf{P}_3-\mathbf{P}_2)}{1-\frac{1}{3}}=3(\mathbf{P}_3-\mathbf{P}_2)$$

$$\mathbf{Q}_3=\frac{2(\mathbf{P}_4-\mathbf{P}_3)}{1-\frac{3}{4}}=8(\mathbf{P}_4-\mathbf{P}_3)$$

$\mathbf{C}(u)$ 和 $\mathbf{C}'(u)$ 分别如图 3.15a 和 3.15 b 所示。

Ex3.2 设 $\mathbf{C}(u)=\sum_{i=0}^{6}N_{i,3}(u)\,\mathbf{P}_i$ 为定义在

$$U=\{0,0,0,0,2/5,3/5,3/5,1,1,1,1\}$$

然后 $U'=\{0,0,0,2/5,3/5,3/5,1,1,1\}$ 和 $\mathbf{C}'(u)=\sum_{i=0}^{5}N_{i,2}(u)\,\mathbf{Q}_i$，

其中

$$\mathbf{Q}_0=\frac{3(\mathbf{P}_1-\mathbf{P}_0)}{\frac{1}{3}-0}=9(\mathbf{P}_1-\mathbf{P}_0)$$

$$\mathbf{Q}_1=\frac{3(\mathbf{P}_2-\mathbf{P}_1)}{\frac{2}{3}-0}=\frac{9}{2}(\mathbf{P}_2-\mathbf{P}_1)$$

$$\mathbf{Q}_2=\frac{3(\mathbf{P}_3-\mathbf{P}_2)}{\frac{2}{3}-0}=\frac{9}{2}(\mathbf{P}_3-\mathbf{P}_2)$$

$$\mathbf{Q}_3=\frac{3(\mathbf{P}_4-\mathbf{P}_3)}{1-\frac{1}{3}}=\frac{9}{2}(\mathbf{P}_4-\mathbf{P}_3)$$

$$\mathbf{Q}_4=\frac{3(\mathbf{P}_5-\mathbf{P}_4)}{1-\frac{2}{3}}=9(\mathbf{P}_5-\mathbf{P}_4)$$

$$\mathbf{Q}_5=\frac{3(\mathbf{P}_6-\mathbf{P}_5)}{1-\frac{2}{3}}=9(\mathbf{P}_6-\mathbf{P}_5)$$

$\mathbf{C}(u)$ 和 $\mathbf{C}'(u)$ 分别如图 3.16a 和 3.16 b 所示。注意 $\mathbf{C}'(u)$ 是一个在双结点 $u=3/5$ 处具有尖点的二次曲线。

Ex3.3 回想一下，p 阶 Bézier 曲线是 $U=\{0,\ldots,0,1,\ldots,1\}$ 上的 B-spline 曲线 (没有内部结点)，方程 (3.4) 简化为 $\mathbf{Q}_i=p(\mathbf{P}_{i+1}-\mathbf{P}_i)$ 对于 $0\le i\le n-1$。由于 $n=p$ 和 $N_{i,p-1}(u)=B_{i,n-1}(u)$，伯恩斯坦多项式，方程 (3.6) 等价于方程 (1.9)。

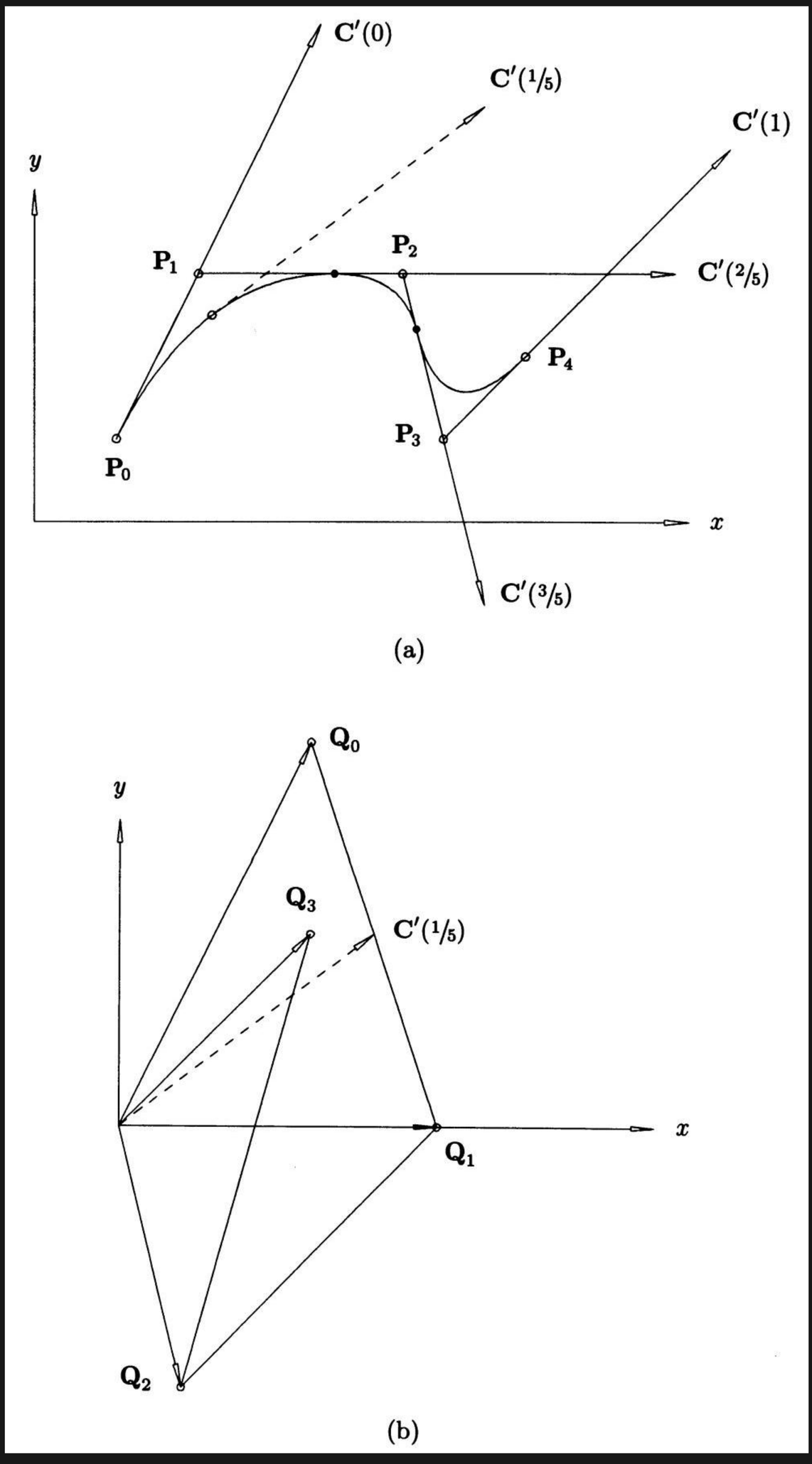


图 3.15. (a) 在 $U = \{0, 0, 0, 2/5, 3/5, 1, 1, 1\}$ 上的二次曲线；(b) 曲线的导数是 $U' = \{0, 0, 2/5, 3/5, 1, 1\}$ 上的一次 B-spline 曲线。

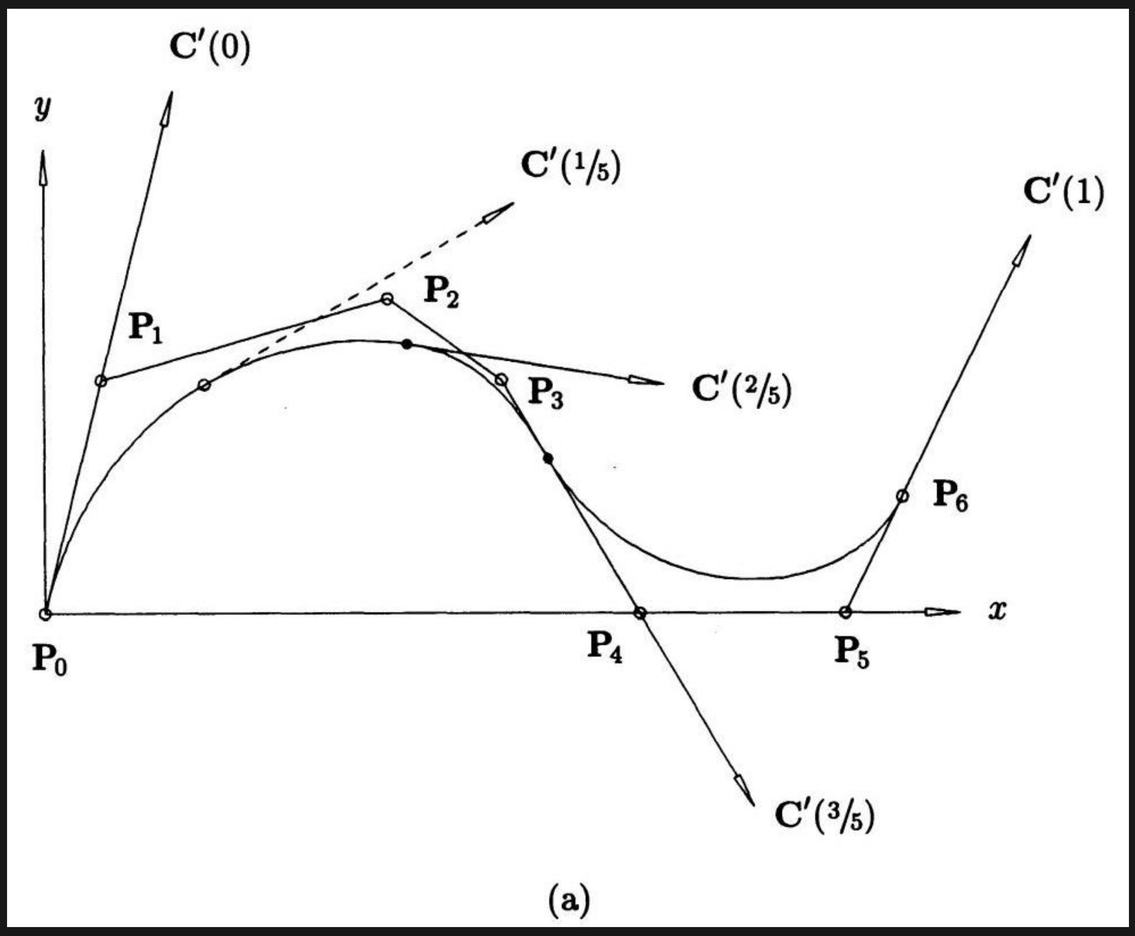


图 3.16. (a) 在 $U = \{0, 0, 0, 0, 2/5, 3/5, 3/5, 1, 1, 1, 1\}$ 上的三次曲线；(b) 在 $U' = \{0, 0, 0, 2/5, 3/5, 3/5, 1, 1, 1\}$ 上的二次导数曲线。

B 样条曲线在端点的第一导数由以下公式给出

$$\mathbf{C}'(0) = \mathbf{Q}_0 = \frac{p}{u_{p+1}}(\mathbf{P}_1 - \mathbf{P}_0)$$

$$\mathbf{C}'(1) = \mathbf{Q}_{n-1} = \frac{p}{1 - u_{m-p-1}}(\mathbf{P}_n - \mathbf{P}_{n-1}) \tag{3.7}$$

(见示例 Ex3.1 和 Ex3.2，以及图 3.15(a) 和 (b) 以及图 3.16(a) 和 (b))。注意，在图 3.15b 和 3.16 b 中，导数向量和控制点差异被缩小以便于更好的可视化，分别由 1/2 和 1/3 表示。

由于 $\mathbf{C}'(u)$ 是 B 样条曲线，我们递归地应用方程 (3.4) 至 (3.6) 以获得更高阶的导数。设 $\mathbf{P}_i^{(0)} = \mathbf{P}_i$ ，我们写成

$$\mathbf{C}(u) = \mathbf{C}^{(0)}(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^{(0)}$$

然后

$$\mathbf{C}^{(k)}(u) = \sum_{i=0}^{n-k} N_{i,p-k}(u)\,\mathbf{P}_i^{(k)} \tag{3.8}$$

具有

$$\mathbf{P}_i^{(k)} = \begin{cases} \mathbf{P}_i & k = 0 \\ \frac{p-k+1}{u_{i+p+1} - u_{i+k}}\left(\mathbf{P}_{i+1}^{(k-1)} - \mathbf{P}_i^{(k-1)}\right) & k > 0 \end{cases}$$

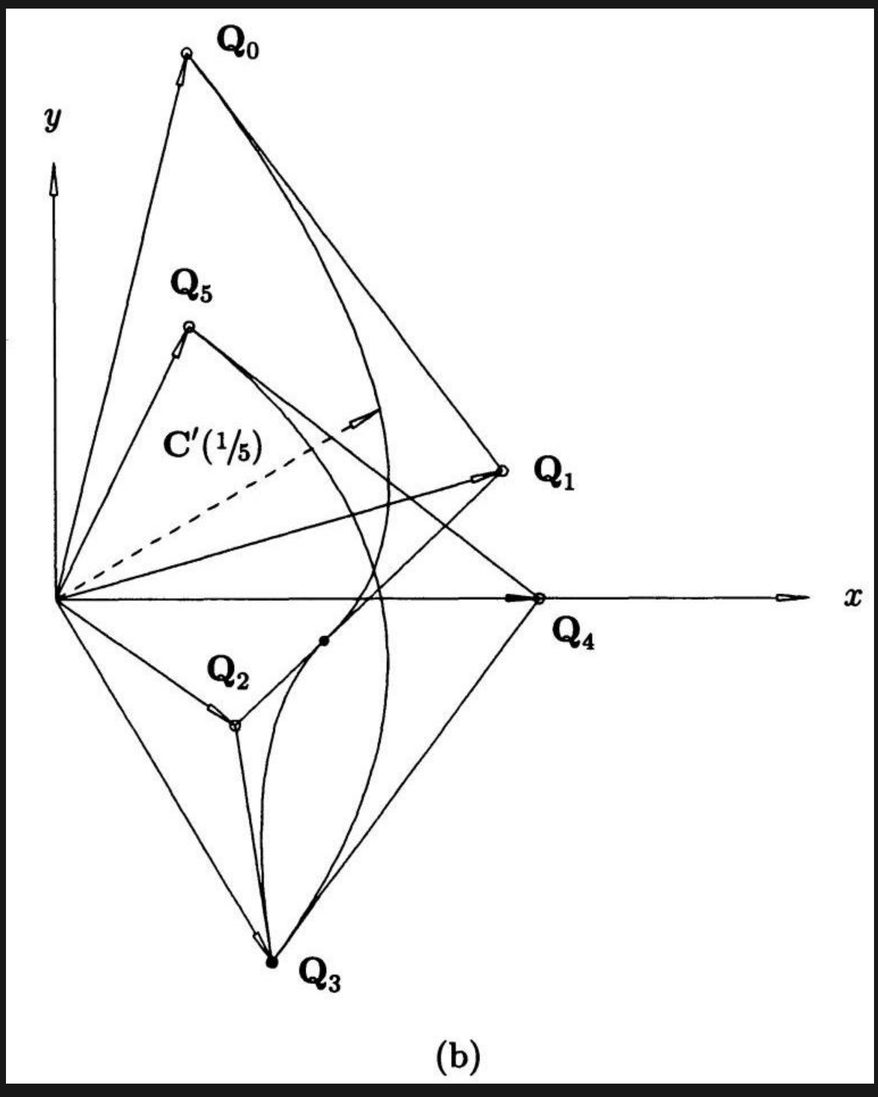


图 3.16。(续。)

和

$$U^{(k)} = \left\{ \underbrace{0, \ldots, 0}_{p-k+1}, u_{p+1}, \ldots, u_{m-p-1}, \underbrace{1, \ldots, 1}_{p-k+1} \right\}$$

算法 A3.3 是方程 (3.8) 的非递归实现。它计算所有导数曲线的控制点，直到并包括 d 阶导数 ($d \le p$) 。输出时，PK[k][i] 是 k 阶导数曲线的 i 阶控制点，其中 $0 \le k \le d$ 和 $r_1 \le i \le r_2 - k$ 。如果 $r_1 = 0$ 和 $r_2 = n$ ，则计算所有控制点。

ALGORITHM A3.3

```
CurveDerivCpts(n, p, U, P, d, r1, r2, PK)
  { /* Compute control points of curve derivatives */
    /* Input: n, p, U, P, d, r1, r2 */
    /* Output: PK */
  r = r2-r1;
  for (i = 0; i<=r; i++)
    PK[0][i] = P[r1+i];
  for (k=1; k<=d; k++)
    {
    tmp = p-k+1;
  for (i = 0; i<=r-k; i++)
    PK[k][i] = tmp*(PK[k-1][i+1]-PK[k-1][i])/
        (U[r1+i+p+1] - U[r1+i+k]);
  }
}
```

使用方程 (3.8)，我们计算 B 样条曲线 ($p > 1$) 在端点 $u = 0$ 处的第二导数

$$\mathbf{C}^{(2)}(0) = \mathbf{P}_0^{(2)} = \frac{p-2+1}{u_{p+1} - u_2} \left(\mathbf{P}_1^{(1)} - \mathbf{P}_0^{(1)} \right)$$

$$= \frac{p-1}{u_{p+1} - u_2} \left[\frac{p}{u_{p+2} - u_2} \left(\mathbf{P}_2^{(0)} - \mathbf{P}_1^{(0)} \right) - \frac{p}{u_{p+1} - u_1} \left(\mathbf{P}_1^{(0)} - \mathbf{P}_0^{(0)} \right) \right]$$

从 $u_1 = u_2 = 0$ 可以得出

$$\mathbf{C}^{(2)}(0) = \frac{p(p-1)}{u_{p+1}} \left[\frac{\mathbf{P}_0}{u_{p+1}} - \frac{(u_{p+1} + u_{p+2})\mathbf{P}_1}{u_{p+1} u_{p+2}} + \frac{\mathbf{P}_2}{u_{p+2}} \right] \tag{3.9}$$

类似地，

$$\mathbf{C}^{(2)}(1) = \frac{p(p-1)}{1 - u_{m-p-1}} \times$$

$$\left[\frac{\mathbf{P}_n}{1 - u_{m-p-1}} - \frac{(2 - u_{m-p-1} - u_{m-p-2})\mathbf{P}_{n-1}}{(1 - u_{m-p-1})(1 - u_{m-p-2})} + \frac{\mathbf{P}_{n-2}}{1 - u_{m-p-2}} \right] \tag{3.10}$$

注意，对于贝塞尔曲线，这些方程简化为方程 (1.10) 的相应表达式。图 3.17 显示了图 3.15a 的二次曲线及其向量 $\mathbf{C}^{(2)}(0)$ 和 $\mathbf{C}^{(2)}(1)$ 。$\mathbf{C}^{(2)}(u)$ 是一条分段的零度曲线，即在三个区间 $[0, 2/5)$ 、$[2/5, 3/5)$ 和 $[3/5, 1]$ 上，它是一个常量 (但不同) 的向量。

我们用另一个算法结束本节，该算法用于计算 B 样条曲线上的一点及其所有导数，直到并包括在固定的 u 值下的 d 阶导数 (与算法 A3.2 比较)。该算法基于方程 (3.8) 和算法 A3.3。我们假设有一个例程 AllBasisFuns，这是对 BasisFuns(算法 A2.2) 的简单修改，用于返回从 0 到 p 的所有非零基函数。特别地，N[j][i] 是 i 阶基函数 $N_{\text{span}-i+j,i}(u)$ 的值，其中 $0 \le i \le p$ 和 $0 \le j \le i$ 。

ALGORITHM A3.4

```
CurveDerivsAlg2(n, p, U, P, u, d, CK)
    { /* Compute curve derivatives */
      /* Input: n, p, U, P, u, d */
      /* Output: CK */
    du = min(d, p);
```

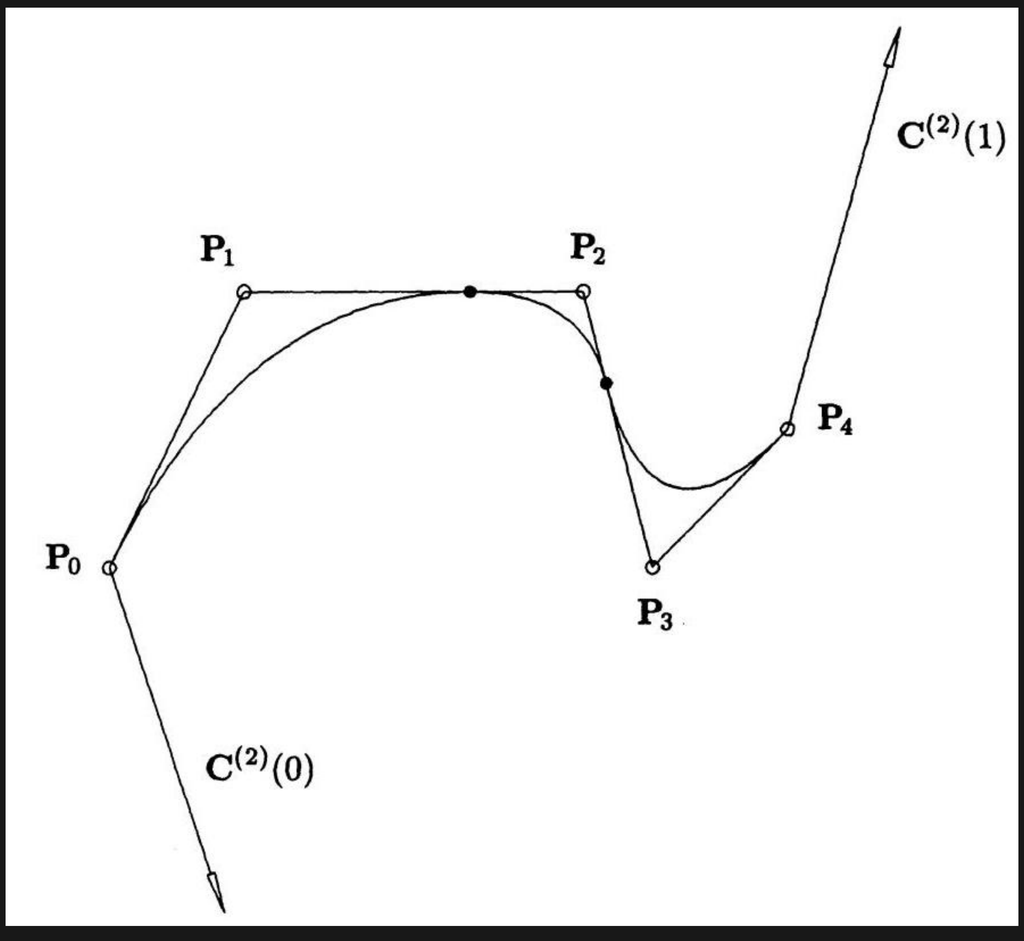


图 3.17。图 3.15a 曲线端点的二阶导数。

```
for (k = p+1; k <= d; k++) CK[k] = 0.0;
span = FindSpan(n, p, u, U);
AllBasisFuns(span, u, p, U, N);
CurveDerivCpts(n, p, U, P, du, span-p, span, PK);
for (k = 0; k <= du; k++)
```

$$
\begin{cases}
u_0 = 0, \\
u_1 = 0, \\
u_2 = 0, \\
u_3 = 0, \\
u_4 = 0, \\
u_5 = 0, \\
u_6 = 0, \\
u_7 = 0, \\
u_8 = 0, \\
u_9 = 0, \\
u_{10} = 0, \\
u_{11} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{14} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{14} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{34} = 0, \\
u_{34} = 0, \\
u_{34

}
}
图 3.18 显示了一条在 $u = 2/5$ 处计算的一阶、二阶和三阶导数的三次曲线。(导数被缩放至 2/5 。)

4 B 样条曲面的定义与性质

B 样条曲面是通过取一个双向控制点网、两个节点向量以及单变量 B 样条函数的乘积获得的。

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u)\, N_{j,q}(v)\, \mathbf{P}_{i,j} \tag{3.11}$$

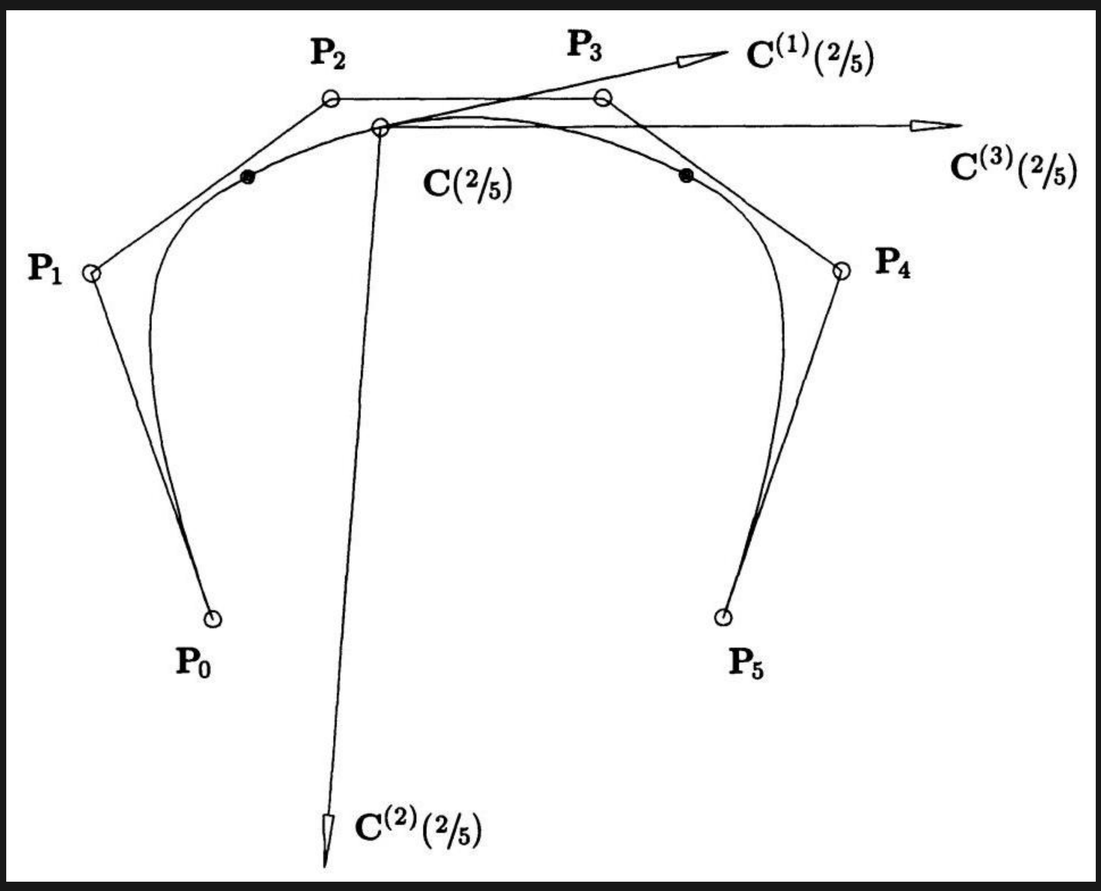


图 3.18。在 $U = \{0,0,0,0,1/4,3/4,1,1,1,1\}$ 上的一条三次曲线，其一阶、二阶和三阶导数在 $u = 2/5$ 处计算。
具有

$$U = \{\underbrace{0,\ldots,0}_{p+1}, u_{p+1},\ldots,u_{r-p-1}, \underbrace{1,\ldots,1}_{p+1}\}$$

$$V = \left\{\underbrace{0,\ldots,0}_{q+1}, v_{q+1},\ldots,v_{s-q-1}, \underbrace{1,\ldots,1}_{q+1}\right\}$$

U 具有 $r+1$ 个节点，且 V 具有 $s+1$ 个节点。方程 (3.2) 的形式为

$$r = n + p + 1 \text{ and } s = m + q + 1 \tag{3.12}$$

设 U 和 $\{N_{i,3}(u)\}$ 为图 3.2a 中的节点向量和三次基函数，且 $\{N_{j,2}(v)\}$ 为定义在 $V = \{0,0,0,1/5,2/5,3/5$ 和 $^3/_5, 4/_5, 1, 1, 1\}$ 上的二次基函数。图 3.19a 和 3.19b 分别显示了张量积基函数 $N_{4,3}(u)\, N_{4,2}(v)$ 和 $N_{4,3}(u)\, N_{2,2}(v)$。图 3.20 – 3.25 显示了 B-spline 曲面的示例。

计算固定 (u, v) 参数值下 B-spline 曲面上的一个点需要五个步骤:

1. 找到 u 所在的节点区间，记为 $u \in [u_i, u_{i+1})$ (算法 A2.1)；
2. 计算非零基函数 $N_{i-p,p}(u),\ldots,N_{i,p}(u)$ (A2.2)；
3. 找到 v 所在的节点区间，记为 $v \in [v_j, v_{j+1})$ (A2.1)；
4. 计算非零基函数 $N_{j-q,q}(v),\ldots,N_{j,q}(v)$ (A2.2)；

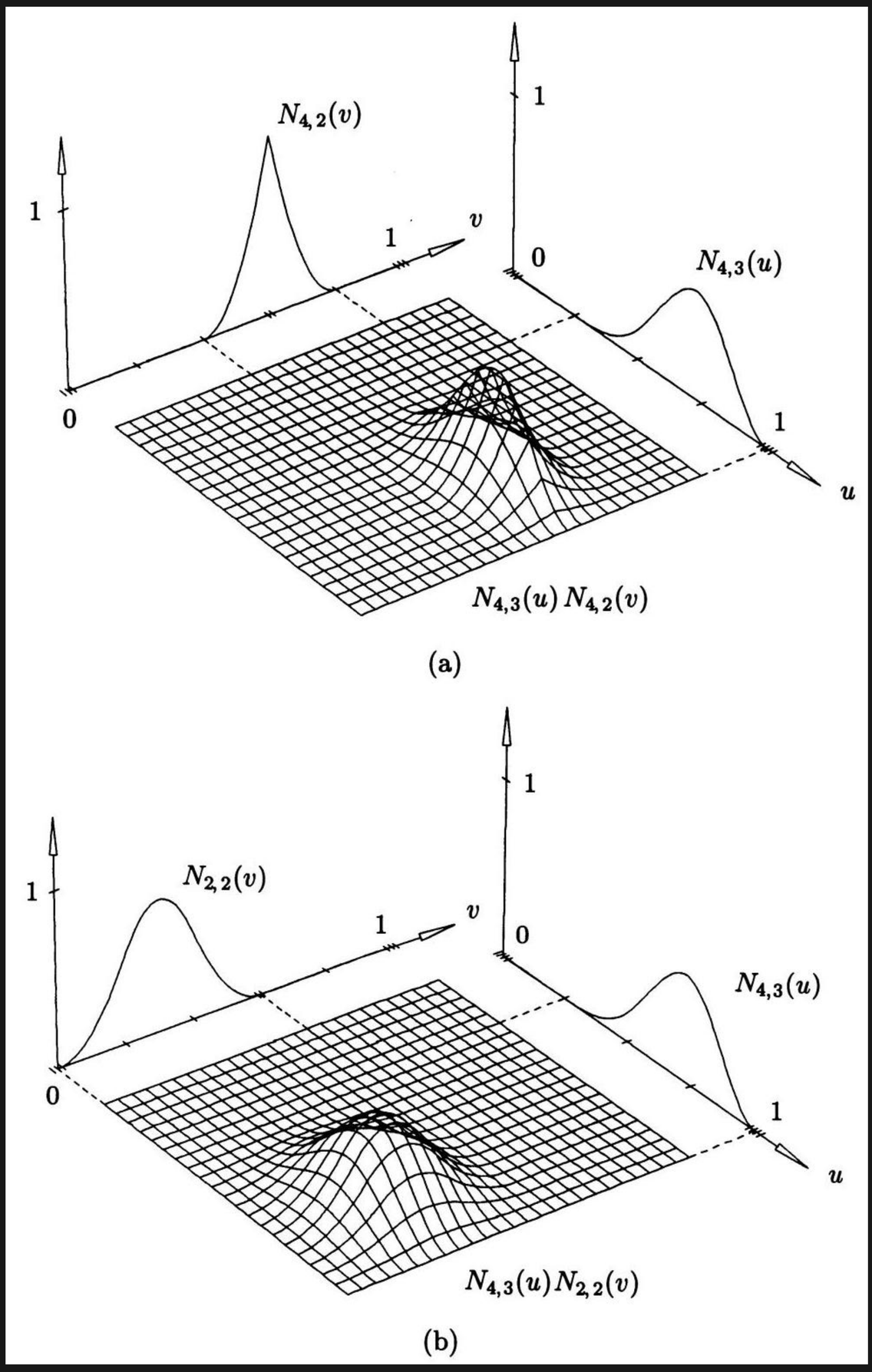


图 3.19. 三次 × 二次基函数。(a) $N_{4,3}(u)\,N_{4,2}(v)$；(b) $N_{4,3}(u)\,N_{2,2}(v)$；$U=\{0,0,0,0,1/4,1/2,3/4,1,1,1,1\}$ 和 $V=\{0,0,0,1/5,2/5,3/5,3/5,4/5,1,1,1\}$。

5. 将非零基函数的值与相应的控制点相乘。

最后一步的形式为

$$\mathbf{S}(u,v)=[N_{k,p}(u)]^T\,[\mathbf{P}_{k,l}]\,[N_{l,q}(v)]\;\; i-p\le k\le i,\; j-q\le l\le j \tag{3.13}$$

注意 $[N_{k,p}(u)]^T$ 是一个 $1\times(p+1)$ 行的标量向量，$[\mathbf{P}_{k,l}]$ 是一个 $(p+1)\times(q+1)$ 的控制点矩阵，而 $[N_{l,q}(v)]$ 是一个 $(q+1)\times 1$ 列的标量向量。

示例

Ex3.4 设 $p=q=2$ 和 $\sum_{i=0}^{4}\sum_{j=0}^{5}N_{i,2}(u)\,N_{j,2}(v)\,\mathbf{P}_{i,j}$，并且

$$U=\{0,0,0,2/5,3/5,1,1,1\}$$

$$V=\{0,0,0,1/5,1/2,4/5,1,1,1\}$$

计算 $\mathbf{S}(1/5,3/5)$。然后 $1/5\in[u_2,u_3)$ 和 $3/5\in[v_4,v_5)$，并且

$$\mathbf{S}\left(\frac{1}{5},\frac{3}{5}\right)=\begin{bmatrix} N_{0,2}\left(\frac{1}{5}\right) & N_{1,2}\left(\frac{1}{5}\right) & N_{2,2}\left(\frac{1}{5}\right)\end{bmatrix}$$

$$\begin{bmatrix}\mathbf{P}_{0,2} & \mathbf{P}_{0,3} & \mathbf{P}_{0,4}\\ \mathbf{P}_{1,2} & \mathbf{P}_{1,3} & \mathbf{P}_{1,4}\\ \mathbf{P}_{2,2} & \mathbf{P}_{2,3} & \mathbf{P}_{2,4}\end{bmatrix}\begin{bmatrix}N_{2,2}\left(\frac{3}{5}\right)\\ N_{3,2}\left(\frac{3}{5}\right)\\ N_{4,2}\left(\frac{3}{5}\right)\end{bmatrix}$$

算法 A3.5 计算在固定 (u, v) 值下的 B 样条曲面上的点。为了提高效率，它使用一个局部数组 temp[] 来存储向量/矩阵乘积 $[N_{k,p}(u)]^T\,[\mathbf{P}_{k,l}]$。然后，将生成的点向量 (在 temp[] 中) 进行乘法运算与向量 $[N_{l,q}(v)]$。

ALGORITHM A3.5

```
SurfacePoint(n, p, U, m, q, V, P, u, v, S)
    { /* Compute surface point */
      /* Input: n, p, U, m, q, V, P, u, v */
      /* Output: S */
    uspan = FindSpan(n, p, u, U);
    BasisFuns(uspan, u, p, U, Nu);
    vspan = FindSpan(m, q, v, V);
```

```
BasisFuns(vspan, v, q, V, Nv);
for (l=0; l<=q; l++)
   {
   temp[l] = 0.0 ;
   for (k = 0; k <= p; k + +)
         temp[l] = temp[l] + Nu[k]*P[uspan-p+k][vspan-q+l];
   }
s = 0.0 ;
for (l=0; l<=q; l++)
   S = S + Nv[l]*temp[l];
}
```

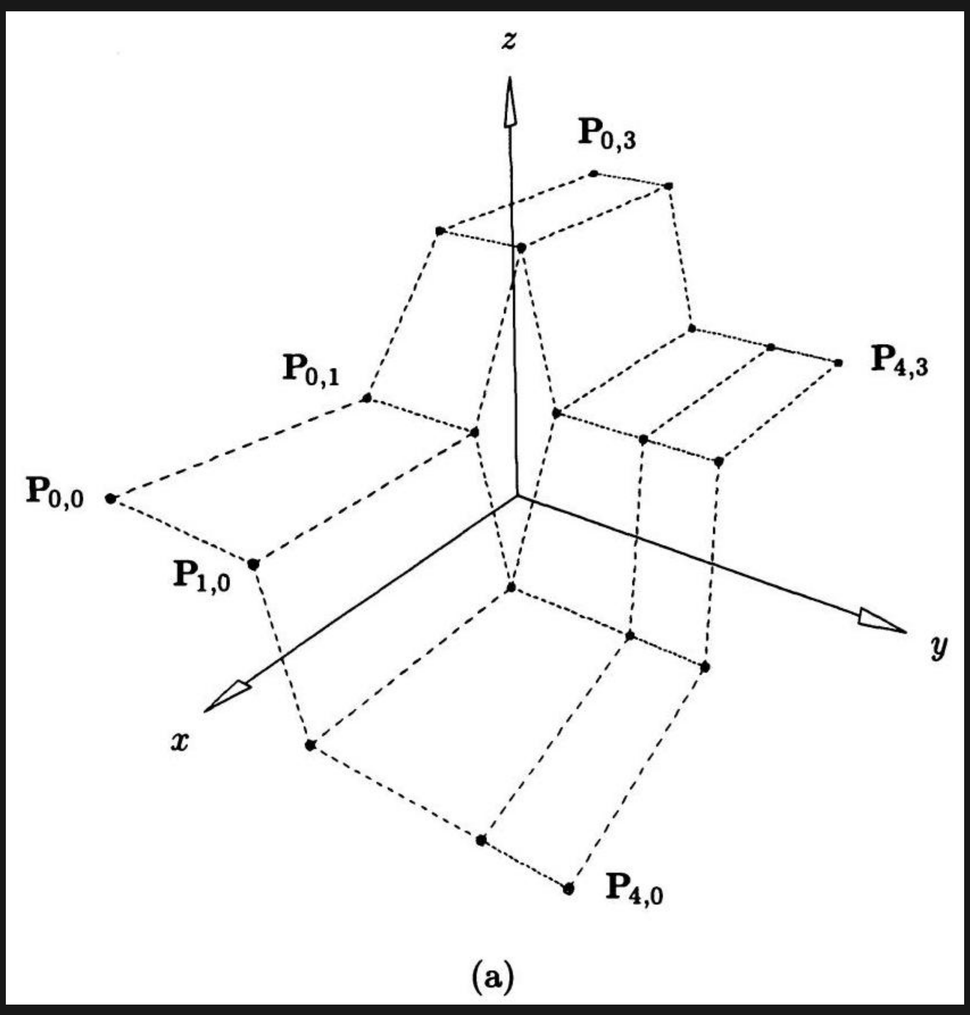


图 3.20. B 样条曲面。(a) 控制网；(b) 曲面。

张量积基函数的性质源于第 2 章中列出的单变量基函数的相应性质。

P3.12 非负性: $N_{i,p}(u)\,N_{j,q}(v) \geq 0$ 对所有 i, j, p, q, u, v ；

P3.13 单位划分: $\sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u)\,N_{j,q}(v) = 1$ 对所有 $(u, v) \in [0,1] \times [0,1]$ ；

P3.14 如果 $n = p, m = q, U = \{0, \ldots, 0, 1, \ldots, 1\}$ ，并且 $V = \{0, \ldots, 0, 1, \ldots, 1\}$ ，则对于所有 i, j ，有 $N_{i,p}(u)\,N_{j,q}(v) = B_{i,n}(u)\,B_{j,m}(v)$ ；即，B 样条函数的乘积退化为伯恩斯坦多项式的乘积；

P3.15 $N_{i,p}(u)\,N_{j,q}(v) = 0$ 如果 (u, v) 在矩形 $[u_i, u_{i+p+1}) \times [v_j, v_{j+q+1})$ 之外 (见图 3.19a 和 3.19 b)；

P3.16 在任何给定的矩形中，$[u_{i_0}, u_{i_0+1}) \times [v_{j_0}, v_{j_0+1})$ ，至多有 $(p+1)(q+1)$ 个基函数非零，特别是 $N_{i,p}(u)\,N_{j,q}(v)$ 对于 $i_0 - p \leq i \leq i_0$ 和 $j_0 - q \leq j \leq j_0$ ；

P3.17 如果 $p > 0$ 和 $q > 0$ ，那么 $N_{i,p}(u)\,N_{j,q}(v)$ 恰好达到一个最大值 (见图 3.19a 和 3.19b)；

在由 u 和 v 节点线形成的矩形内部，函数是一个双变量多项式，所有 $N_{i,p}(u)\,N_{j,q}(v)$ 的偏导数存在；在 u 节点 (v 节点) 处，它在 $u\,(v)$ 方向上是 $p - k\,(q - k)$ 次可微的，其中 k 是节点的重数。在

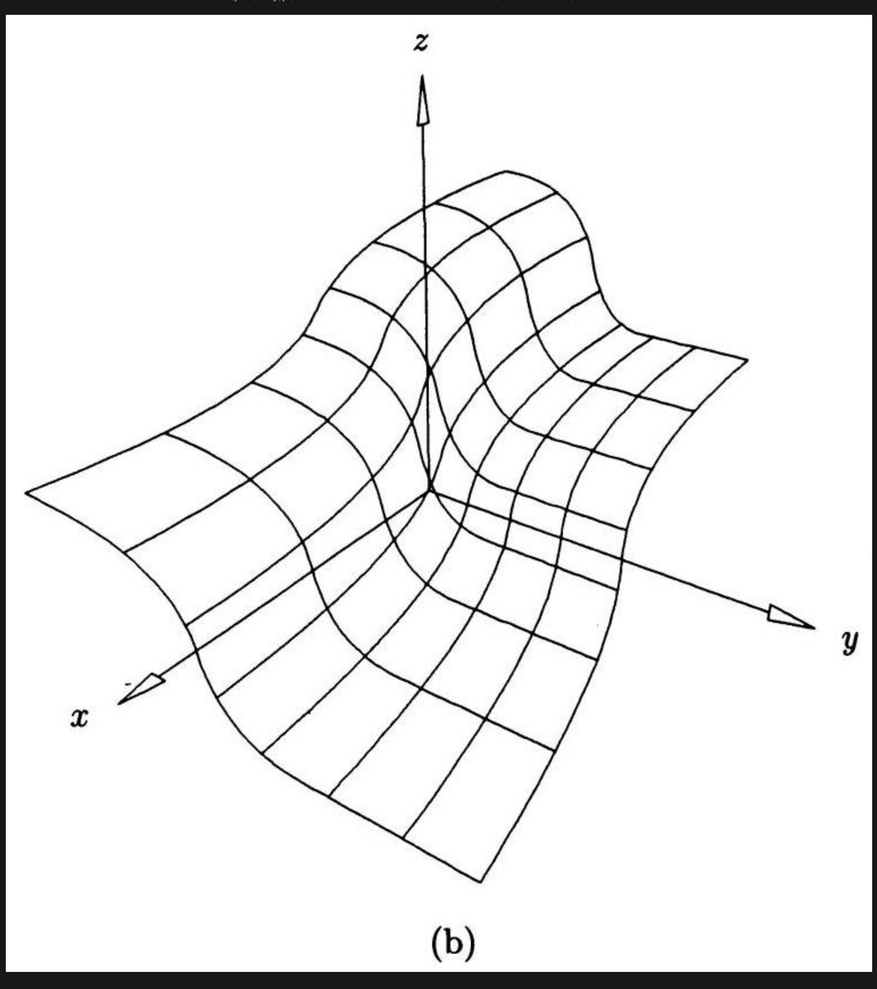


图 3.20.(续)

在图 3.19a 中，$N_{4,3}(u)\,N_{4,2}(v)$ 关于 v 的第一个偏导数在节点线 $v = 3/5$ 上是不连续的，那里 $N_{4,2}(v)$ 有一个尖点。关于 u 的第二个偏导数处处连续，因为 $N_{4,3}(u)$ 是 $\mathbf{C}^2$ 连续的。

B-spline 曲面具有以下特性:

P3.19 如果 $n=p, m=q, U=\{0,\ldots,0,1,\ldots,1\}$ ， 并且 $V=\{0,\ldots,0,1,\ldots,1\}$ ， 那么 $\mathbf{S}(u,v)$ 是一个 Bézier 曲面；这来自 P3.14；

P3.20 该曲面插值四个角控制点: $\mathbf{S}(0,0)=\mathbf{P}_{0,0}$ ， $\mathbf{S}(1,0)=\mathbf{P}_{n,0}, \mathbf{S}(0,1)=\mathbf{P}_{0,m}$ 和 $\mathbf{S}(1,1)=\mathbf{P}_{n,m}$ (见图 3.20 至 3.25)；这来自 P3.13 和该恒等式

$$N_{0,p}(0)\,N_{0,q}(0)=N_{n,p}(1)\,N_{0,q}(0)=N_{0,p}(0)\,N_{m,q}(1)$$

$$=N_{n,p}(1)\,N_{m,q}(1)=1$$

P3.21 仿射不变性: 通过对控制点应用仿射变换来对曲面应用该变换；这来自 P3.13；

P3.22 强凸包性质: 如果 $(u,v)\in[u_{i_0},u_{i_0+1})\times[v_{j_0},v_{j_0+1})$ ， 则 $\mathbf{S}(u,v)$ 在控制点 $\mathbf{P}_{i,j}, i_0-p\le i\le i_0$ 和 $j_0-q\le j\le j_0$ 的凸包内 (见图 3.21)；这由 P3.12、P3.13 和 P3.16 得出；

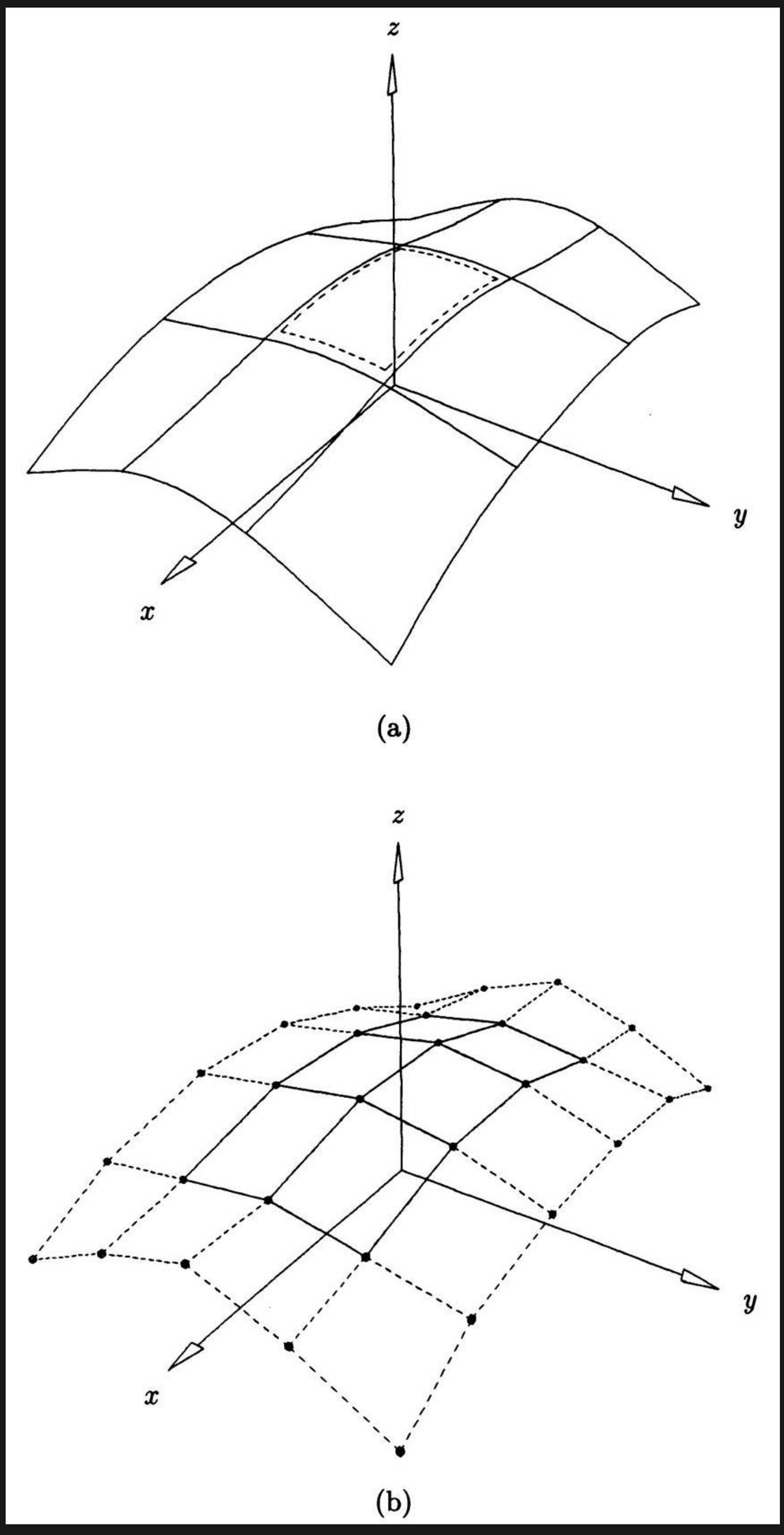


图 3.21. (a) 一个三次 × 二次 B-spline 曲面；(b) 强凸包性质。

P3.23 如果进行三角剖分，控制网形成对曲面的分段平面近似；与曲线的情况一样，度数越低，近似效果越好 (见图 3.22a 和 3.22b)；

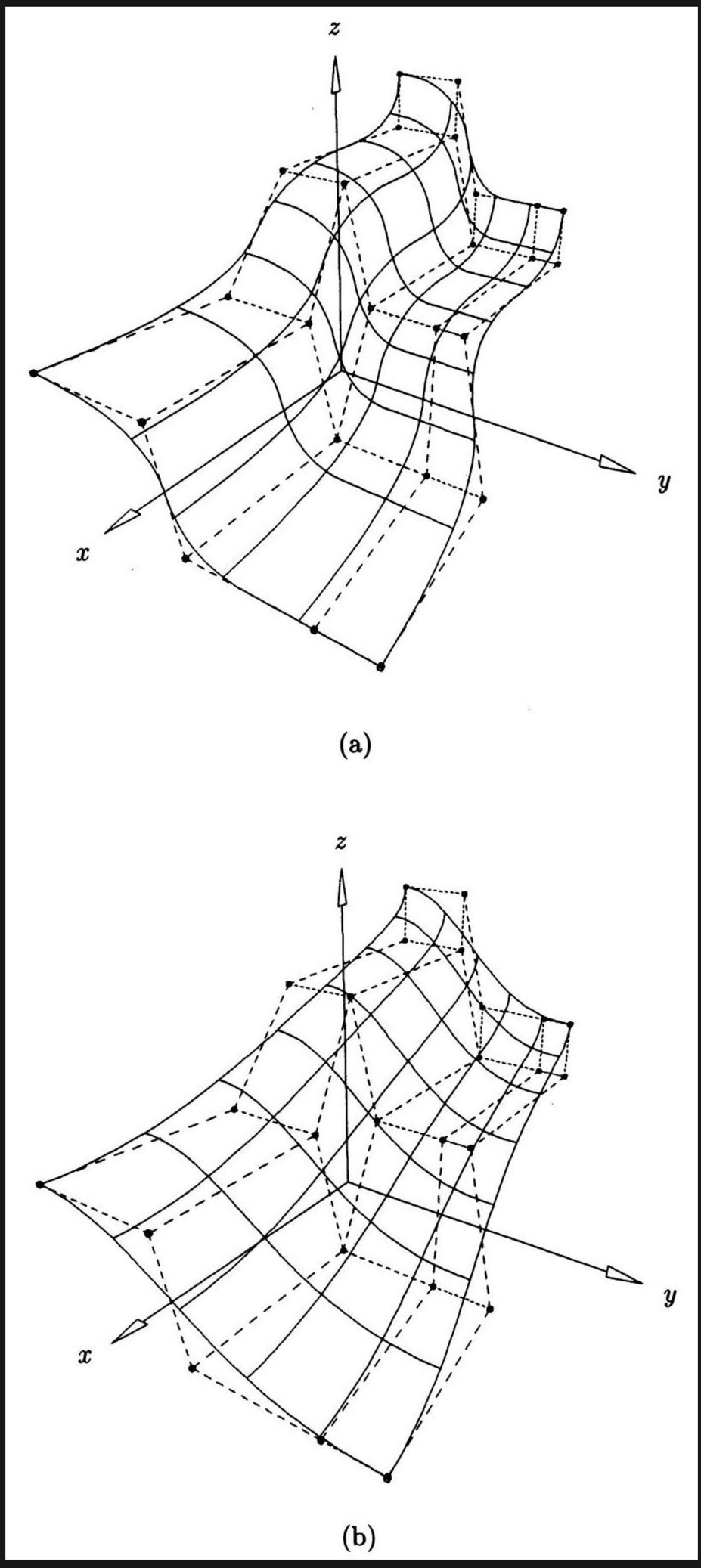


图 3.22. (a) 一个双二次曲面；(b) 一个双四次曲面 ($p = q = 4$) 使用与图 3.22a 相同的控制点。

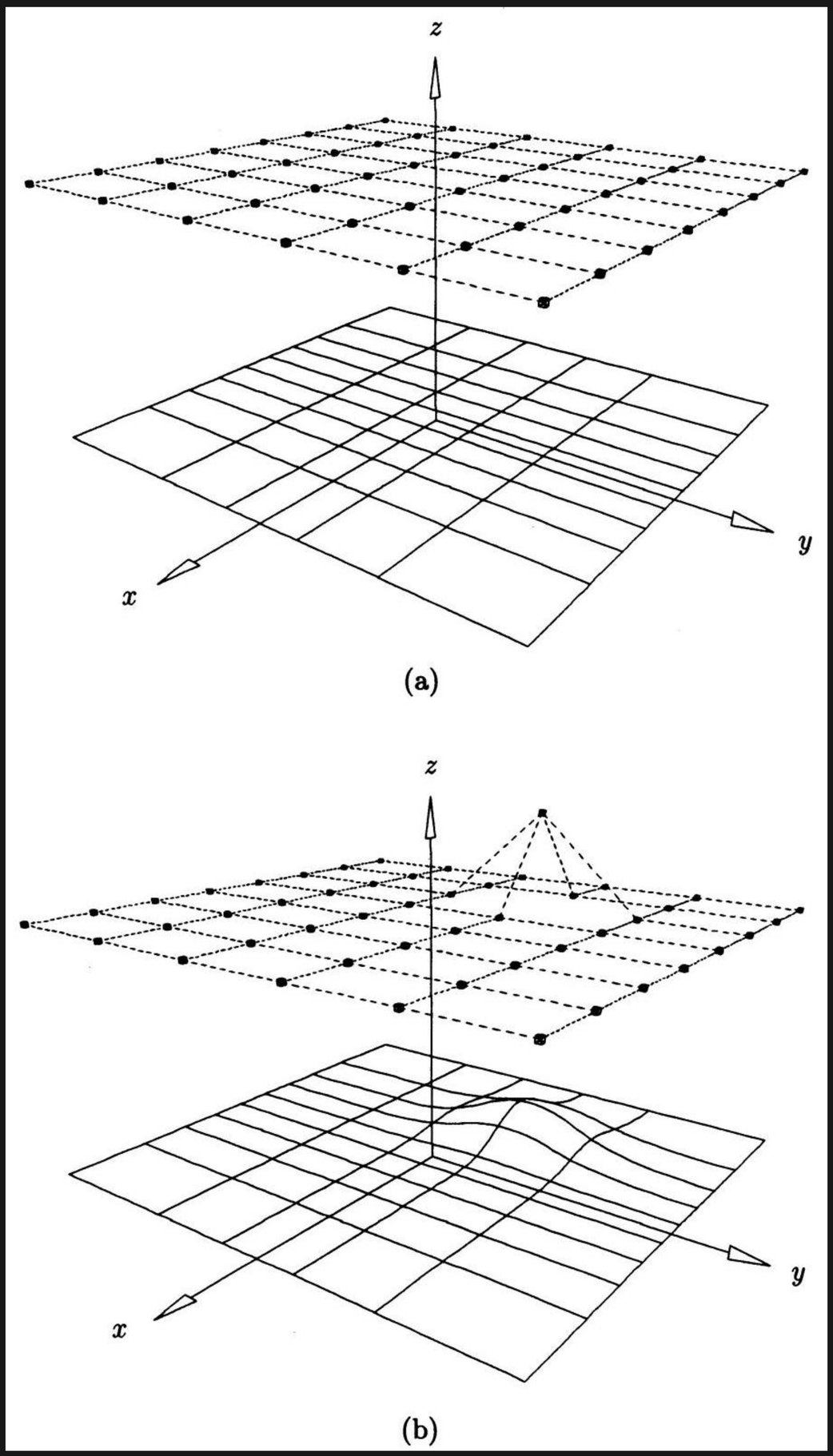


图 3.23. (a) 一个平面二次 × 三次曲面，$U = \{0,0,0,1/4,1/2,3/4,1,1,1\}$ 和 $V = \{0,0,0,0,1/5,2/5,3/5,4/5,1,1,1,1\}$；(b) $\mathbf{P}_{3,5}$ 被移动，仅在矩形 $[1/4,1) \times [2/5,1)$ 内影响曲面形状。

P3.24 局部修改方案: 如果 $\mathbf{P}_{i,j}$ 被移动，它仅在矩形 $[u_i, u_{i+p+1}) \times [v_j, v_{j+q+1})$ 内影响曲面；这由 P3.15 得出。现在考虑图 3.23a 和 3.23b: 初始曲面是平坦的，因为所有控制点都位于同一平面内 (P3.22)；控制网偏离曲面以便于可视化。当 $\mathbf{P}_{3,5}$ 被移动时，它仅在矩形 $[1/4,1) \times [2/5,1)$ 内影响曲面形状；

P3.25 $\mathbf{S}(u,v)$ 的连续性和可微性源于基函数的连续性。特别地，$\mathbf{S}(u,v)$ 在重数为 k 的 $u(v)$ 节点的 $u(v)$ 方向上是 $p-k(q-k)$ 次可微的。图 3.24 显示了在节点向量 $U = \{0,0,0,1/2,1/2,1,1,1\}$ 和 $V = \{0,0,0,0,1/2,1,1,1,1\}$ 上定义的二次 × 三次曲面。注意表面中的折痕，对应于节点线 $u = 1/2$。当然，正如曲线的情况一样，可以以某种方式定位控制点，使其抵消基函数中的不连续性。通过使用重合的控制点，可以在基函数中没有相应不连续性的地方创建视觉不连续性；图 3.25 显示了这样一个表面，它是三次的且没有重节点。因此，二阶偏导数处处连续。折痕是由于多个控制点造成的。

我们在此指出，目前尚无已知的 B-spline 曲面变差减小性质 (见 [Prau92])。

$\mathbf{S}(u,v)$ 上的等参数曲线是以类似于 Bézier 曲面的方式获得的。固定 $u = u_0$

$$\mathbf{C}_{u_0}(v) = \mathbf{S}(u_0,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u_0)\, N_{j,q}(v)\, \mathbf{P}_{i,j}$$

$$= \sum_{j=0}^{m} N_{j,q}(v) \left(\sum_{i=0}^{n} N_{i,p}(u_0)\, \mathbf{P}_{i,j} \right) = \sum_{j=0}^{m} N_{j,q}(v)\, \mathbf{Q}_j(u_0) \tag{3.14}$$

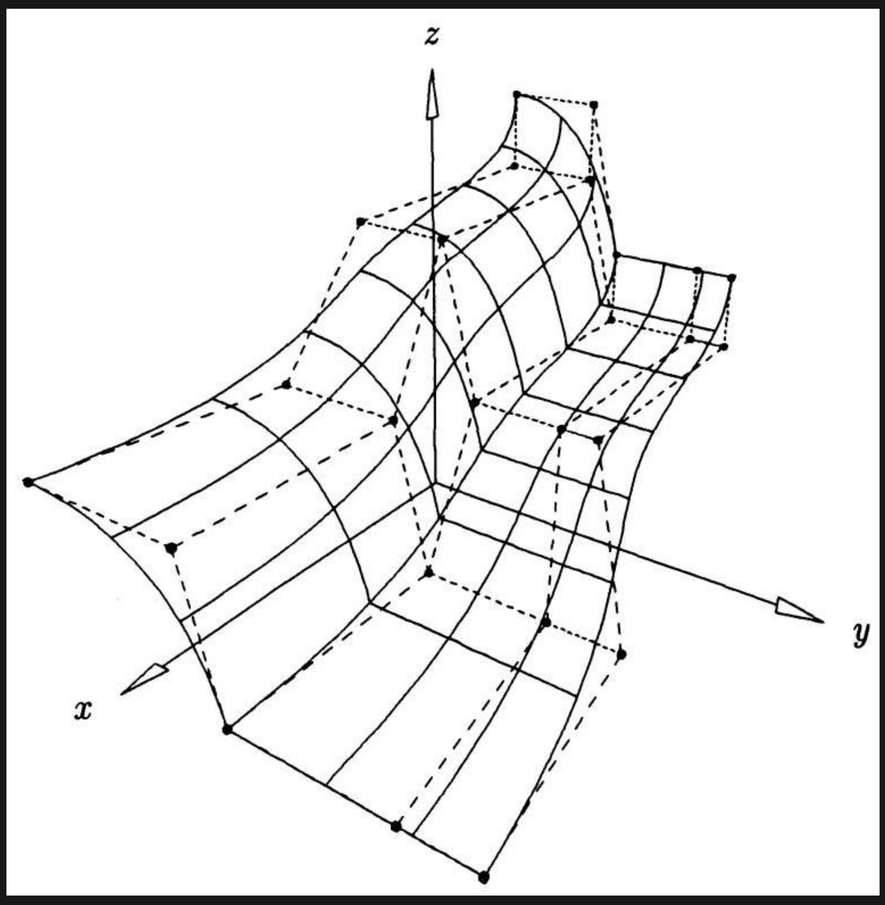


图 3.24. 带折痕的二次 × 三次曲面，$U = \{0, 0, 0, 1/2, 1/2, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 0, 0, 1/2, 1, 1, 1\}$.

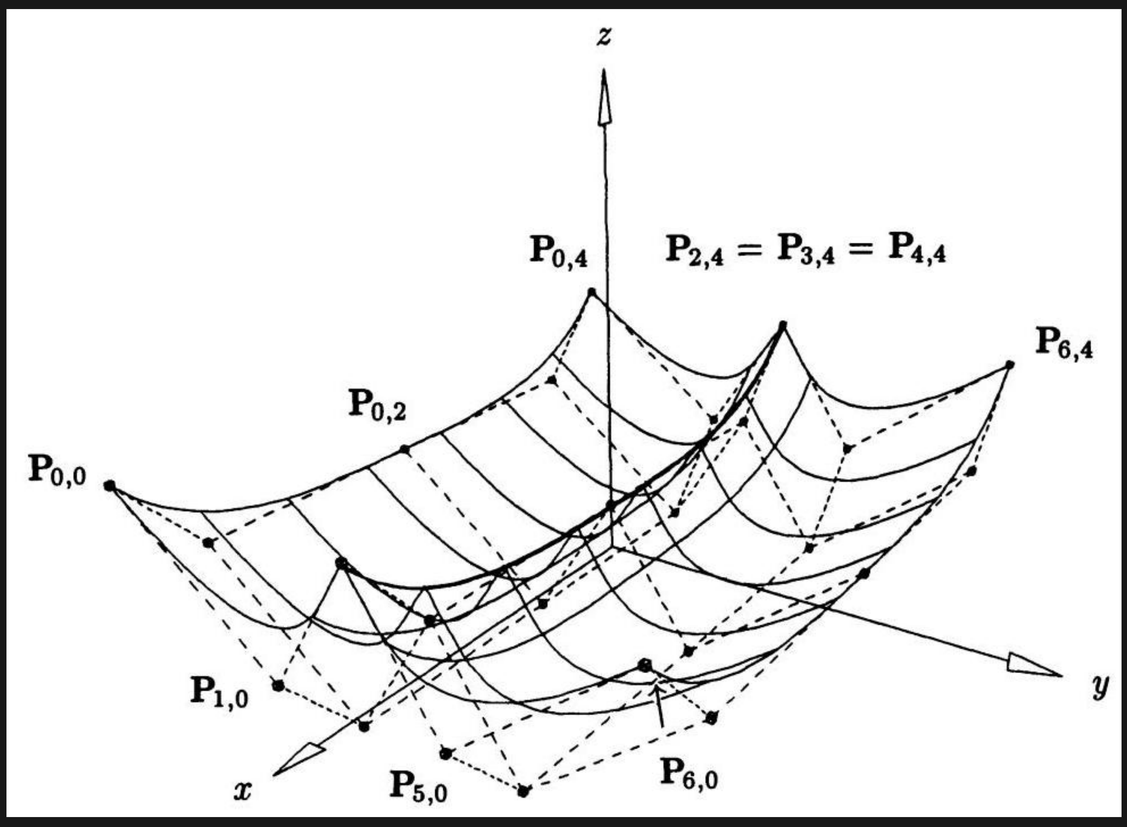


图 3.25. 带折痕的三次曲面，$U = \{0, 0, 0, 0, 1/4, 1/2, 3/4, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 0, 1/2, 1, 1, 1\}$; $\mathbf{P}_{2,j} = \mathbf{P}_{3,j} = \mathbf{P}_{4,j}$ 对于 $0 \le j \le 4$ 。

其中

$$\mathbf{Q}_j(u_0) = \sum_{i=0}^{n} N_{i,p}(u_0)\,\mathbf{P}_{i,j}$$

类似地

$$\mathbf{C}_{v_0}(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{Q}_i(v_0)$$

其中

$$\mathbf{Q}_i(v_0) = \sum_{j=0}^{m} N_{j,q}(v_0)\,\mathbf{P}_{i,j} \tag{3.15}$$

是一个 u 等值线在 $\mathbf{S}(u,v)$.$\mathbf{C}_{u_0}(v)$ 上，是一个 q 次 B 样条曲线在 V 上，而 $\mathbf{C}_{v_0}(u)$ 是一个 p 次 B 样条曲线在 U 上。点 $\mathbf{S}(u_0, v_0)$ 位于 $\mathbf{C}_{u_0}(v)$ 和 $\mathbf{C}_{v_0}(u)$ 的交点上。图 3.20-3.25 中显示的所有线条都是等高线。

5　B 样条曲面的导数

设 (u, v) 为固定值。一般来说，人们感兴趣的是计算 $\mathbf{S}(u,v)$ 的所有偏导数，直到包括 d 阶，即

$$\frac{\partial^{k+l}}{\partial^k u \partial^l v}\mathbf{S}(u,v) \quad 0 \le k + l \le d \tag{3.16}$$

对于曲线，我们通过计算基函数的导数来获得这些导数 (见方程 [2.9] 和 [2.10] 以及算法 A2.3)。特别地，

$$\frac{\partial^{k+l}}{\partial^k u \partial^l v}\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}^{(k)} N_{j,q}^{(l)} \mathbf{P}_{i,j} \tag{3.17}$$

算法 A3.6 计算 B 样条曲面上的点及所有偏导数，直到包括 d 阶 (允许 $d > p, q$)。与算法 A3.5 类似，这是一个五步过程，最后一步是形式为向量/矩阵/向量的乘法。

$$\frac{\partial^{k+l}}{\partial^k u \partial^l v}\mathbf{S}(u,v) = \left[N_{r,p}^{(k)}(u)\right]^T \left[\mathbf{P}_{r,s}\right] \left[N_{s,q}^{(l)}(v)\right]$$

$$0 \le k + l \le d \quad \text{uspan} \quad - p \le r \le u \text{ span}$$

$$v \text{ span} \quad - q \le s \le v \text{ span} \tag{3.18}$$

输出是数组 SKL[] []，其中 SKL[k] [1] 是 $\mathbf{S}(u,v)$ 关于 uk 和 vl 的导数。对于固定的 $k, 0 \le k \le d$ ，局部数组 temp [] 存储向量/矩阵乘积 $\left[N_{r,p}{}^{(k)}(u)\right]^T [\mathbf{P}_{r,s}]$ ，同时它与 $\left[N_{s,q}{}^{(l)}(v)\right]$ 相乘，针对 $0 \le l \le d-k$ 。数组 Nu [] [] 和 Nv [] [] 用于存储基函数的导数。

ALGORITHM A3.6

```
SurfaceDerivsAlg1(n, p, U, m, q, V, P, u, v, d, SKL)
    { /* Compute surface derivatives */
      /* Input: n, p, U, m, q, V, P, u, v, d */
      /* Output: SKL */
    du = min(d, p);
    for (k = p + 1; k <= d; k++)
        for (l = 0; l <= d − k; l++) SKL[k] [l] = 0.0 ;
    dv = min(d, q);
    for (l=q+1; l<=d; l++)
        for (k = 0; k <= d − l; k++) SKL[k] [l] = 0.0 ;
    uspan = FindSpan(n, p, u, U);
    DersBasisFuns(uspan, u, p, du, U, Nu);
    vspan = FindSpan(m, q, v, V);
    DersBasisFuns(vspan, v, q, dv, V, Nv);
    for ( k = 0 ; k <= du ; k++ )
        {
        for (s = 0 ; s<=q ; s++)
          {
          temp[s] = 0.0 ;
          for (r=0; r<=p; r++)
            temp[s] = temp[s] + Nu[k][r]*P[uspan-p+r][vspan-q+s];
          }
112 B 样条曲线和曲面
    dd = min(d-k, dv);
    for (l=0; l<=dd; l++)
        {
        SKL[k][l] = 0.0 ;
    for (s = 0; s <= q; s++)
        SKL[k][l] = SKL[k][l] + Nv[l][s]*temp[s];
    }
  }
}
```

图 3.26 显示了一个双立方曲面及其一阶和二阶偏导数。请注意，为了更好地可视化，导数被缩小了 1/2 。

让我们正式区分 $\mathbf{S}(u,v)$ 。关于 u 我们有

$$\mathbf{S}_u(u,v) = \frac{\partial}{\partial u}\mathbf{S}(u,v) = \sum_{j=0}^{m} N_{j,q}(v)\left(\frac{\partial}{\partial u}\sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_{i,j}\right) \tag{3.19}$$

$$= \sum_{j=0}^{m} N_{j,q}(v)\left(\frac{\partial}{\partial u}\mathbf{C}_j(u)\right)$$

其中

$$\mathbf{C}_j(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_{i,j} \quad j = 0, \ldots, m$$

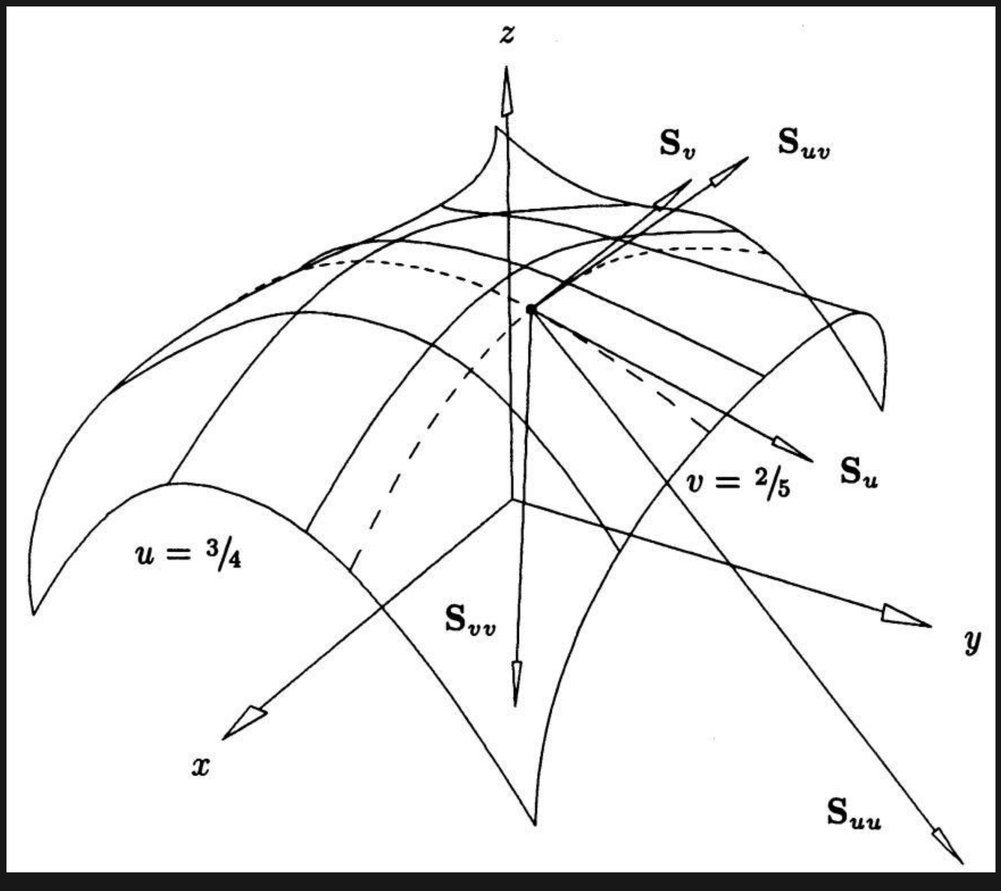


图 3.26. 在 $U = V = \{0,0,0,0,1/2,1,1,1,1\}$ 上定义的双三次曲面及其在 $u = 3/4$ 和 $v = 2/5$ 处计算的第一和第二偏导数。

是 B 样条曲线。将公式 (3.6) 应用到每个 $\mathbf{C}_j(u)$ 并进行替换

代入公式 (3.19)，我们得到

$$\mathbf{S}_u(u,v) = \sum_{i=0}^{n-1}\sum_{j=0}^{m} N_{i,p-1}(u)\,N_{j,q}(v)\,\mathbf{P}_{i,j}^{(1,0)} \tag{3.20}$$

其中

$$\mathbf{P}_{i,j}^{(1,0)} = p\frac{\mathbf{P}_{i+1,j} - \mathbf{P}_{i,j}}{u_{i+p+1} - u_{i+1}} \text{ (see Eq. [3.4])}$$

$$U^{(1)} = \left\{\underbrace{0,\ldots,0}_{p}, u_{p+1}, \ldots, u_{r-p-1}, \underbrace{1,\ldots,1}_{p}\right\}$$

$$V^{(0)} = V$$

类似地 $\mathbf{S}_v(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m-1} N_{i,p}(u)\,N_{j,q-1}(v)\,\mathbf{P}_{i,j}^{(0,1)}$ (3.21)

其中

$$\mathbf{P}_{i,j}^{(0,1)} = q\frac{\mathbf{P}_{i,j+1} - \mathbf{P}_{i,j}}{v_{j+q+1} - v_{j+1}}$$

$$U^{(0)} = U$$

$$V^{(1)} = \left\{\underbrace{0,\ldots,0}_{q}, v_{q+1}, \ldots, v_{s-q-1}, \underbrace{1,\ldots,1}_{q}\right\}$$

首先应用公式 (3.20)，然后应用公式 (3.21) 得到

$$\mathbf{S}_{uv}(u,v)=\sum_{i=0}^{n-1}\sum_{j=0}^{m-1}N_{i,p-1}(u)\,N_{j,q-1}(v)\,\mathbf{P}_{i,j}^{(1,1)} \tag{3.22}$$

其中

$$\mathbf{P}_{i,j}^{(1,1)}=q\frac{\mathbf{P}_{i,j+1}^{(1,0)}-\mathbf{P}_{i,j}^{(1,0)}}{v_{j+q+1}-v_{j+1}}$$

并且 $U^{(1)}$ 和 $V^{(1)}$ 的定义如前所述。

一般来说

$$\frac{\partial^{k+l}}{\partial^k u\partial^l v}\mathbf{S}(u,v)=\sum_{i=0}^{n-k}\sum_{j=0}^{m-l}N_{i,p-k}(u)\,N_{j,q-l}(v)\,\mathbf{P}_{i,j}^{(k,l)} \tag{3.23}$$

在哪里

$$\mathbf{P}_{i,j}^{(k,l)}=(q-l+1)\frac{\mathbf{P}_{i,j+1}^{(k,l-1)}-\mathbf{P}_{i,j}^{(k,l-1)}}{v_{j+q+1}-v_{j+l}}$$

使用公式 (3.20)-(3.23)，我们推导出角导数的有用公式。例如，在角点 $(u,v)=(0,0)$ 处，我们有

$$\mathbf{S}_u(0,0)=\mathbf{P}_{0,0}^{(1,0)}=\frac{p}{u_{p+1}}(\mathbf{P}_{1,0}-\mathbf{P}_{0,0})$$

$$\mathbf{S}_v(0,0)=\mathbf{P}_{0,0}^{(0,1)}=\frac{q}{v_{q+1}}(\mathbf{P}_{0,1}-\mathbf{P}_{0,0}) \tag{3.24}$$

$$\mathbf{S}_{uv}(0,0)=\mathbf{P}_{0,0}^{(1,1)}=\frac{q}{v_{q+1}}\left(\mathbf{P}_{0,1}^{(1,0)}-\mathbf{P}_{0,0}^{(1,0)}\right)$$

$$=\frac{pq}{u_{p+1}v_{q+1}}(\mathbf{P}_{1,1}-\mathbf{P}_{0,1}-\mathbf{P}_{1,0}+\mathbf{P}_{0,0})$$

现在设 $u_0=0$ 和 $v_0=0$。根据基函数的性质，很容易看出等曲线 $\mathbf{C}_{u_0}(v)$ 和 $\mathbf{C}_{v_0}(u)$ 是由以下公式给出的

$$\mathbf{C}_{u_0}(v)=\sum_{j=0}^{m}N_{j,q}(v)\,\mathbf{P}_{0,j}\;\mathbf{C}_{v_0}(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_{i,0}$$

从公式 (3.7) 可以得出

$$\mathbf{S}_u(0,0)=\mathbf{C}'_{v_0}(0)\;\mathbf{S}_v(0,0)=\mathbf{C}'_{u_0}(0)$$

算法 A3.7 计算所有 (或可选的一些) 控制点 $\mathbf{P}_{i,j}^{(k,l)}$，导数曲面最高到 $d\,(0\le k+l\le d)$ 阶。该算法基于公式 (3.23) 和算法 A3.3。输出为数组 PKL[][][][]，其中 PKL [k] [1] [i] [j] 是曲面的第 i,j 个控制点，相对于 u 导数求导 k 次，相对于 l 导数求导 v 次。

ALGORITHM A3.7

```
SurfaceDerivCpts (n, p, U, m, q, V, P, d, r1, r2, s1, s2, PKL)
{ /* Compute control points of derivative surfaces */
    /* Input: n, p, U, m, q, V, P, d, r1, r2, s1, s2 */
    /* Output: PKL */
  du = min (d, p) ; dv = min (d, q) ;
  r = r2-r1; s = s2-s1;
  for (j=s1; j<=s2; j++)
      {
      CurveDerivCpts(n, p, U,&P[][j], du, r1, r2, temp);
      for (k=0; k<=du; k++)
        for (i = 0; i<=r-k; i++)
          PKL[k][0][i][j-s1] = temp[k][i];
      }
  for (k = 0; k < du; k + +)
      for (i = 0; i<=r-k; i++)
{
dd = min(d-k, dv);
CurveDerivCpts(m, q,&V[s1],&PKL[k][0][i][], dd,0, s, temp);
for (l=1; l<=dd; l++)
  for (j=0; j<=s-1; j++)
    PKL[k][l][i][j] = temp[l][j];
}
}
```

算法 A3.8 计算 B 样条曲面上的点及所有偏导数，最高到并包括 d 阶，固定参数 (u, v)(与算法 A3.6 比较)。允许使用 $d>p,q$。输出时，SKL[k][1] 是相对于 u 导数求导 $\mathbf{S}(u,v)$ k 次，相对于 v 导数求导 l 次。

ALGORITHM A3.8:

```
SurfaceDerivsAlg2(n, p, U, m, q, V, P, u, v, d, SKL)
{ /* Compute surface derivatives */
  /* Input: n, p, U, m, q, V, P, u, v, d */
  /* Output: SKL */
  du = min(d, p);
  for (k=p+1; k<=d; k++)
    for (l=0; l<=d-k; l++) SKL[k][l] = 0.0;
  dv = min(d, q);
  for (l=q+1; l<=d; l++)
    for (k = 0; k <= d − l; k + +) SKL [k] [1] = 0.0 ;
  uspan = FindSpan(n, p, u, U);
  AllBasisFuns(uspan, u, p, U, Nu);
  vspan = FindSpan(m, q, v, V);
  AllBasisFuns(vspan, v, q, V, Nv);
  SurfaceDerivCpts (n, p, U, m, q, V, P, d, uspan − p, uspan,
                                          vspan-q, vspan, PKL);
  for ( k = 0 ; k <= du ; k + + )
      {
      dd = min(d-k, dv);
    for (l=0; l<=dd; l++)
        {
        SKL[k][l] = 0.0 ;
        for (i = 0; i<=q-l; i++)
          {
          tmp = 0.0 ;
          for (j=0; j<=p-k; j++)
            tmp = tmp + Nu[j][p-k]*PKL[k][l][j][i];
          SKL[k][l] = SKL[k][l] + Nv[i][q-l]*tmp;
          }
        }
    }
  }
```

练习

3.1. 为什么二次曲线在节点处接触其控制多边形？

3.2. 如果一条二次曲线有一个拐点，它必须位于一个节点上 (见图)。为什么？

3.3. 构造一个具有尖点的 $\mathbf{C}^2$ 连续三次曲线。

3.4. 设三次曲线由 $\mathbf{C}(u)=\sum_{i=0}^{7}N_{i,3}(u)\,\mathbf{P}_i$ 和节点向量 $U=\{0,0,0,0,1/4,1/4,2/3,3/4,1,1,1,1\}$. 定义。

a. 假设 $\mathbf{P}_i$ 的一些任意位置，并绘制曲线。

b. 点 $\mathbf{C}(1/4)$ 在哪里？

c. 如果 $\mathbf{P}_2$ 被移动，$\mathbf{C}(u)$ 在 $[0,1]$ 的哪个子区间受到影响？如果 $\mathbf{P}_5$ 被移动，哪个子区间受到影响？

d. 哪些控制点在区间 $u\in[1/4,2/3)$ 上影响曲线形状？在区间 $u\in[2/3,3/4)$ 上呢？

3.5. 设 $\mathbf{C}(u)=\sum_{i=0}^{3}N_{i,2}(u)\,\mathbf{P}_i$，其中 $U=\{0,0,0,1/2,1,1,1\}$ 和 $\mathbf{P}_0=(-1,0)$，$\mathbf{P}_1=(-1,1),\mathbf{P}_2=(1,1)$ 和 $\mathbf{P}_3=(1,0)$。绘制 $\mathbf{C}(u)$。计算 $\mathbf{C}'(u)$，即其控制点和节点向量。绘制 $\mathbf{C}'(u)$。

3.6. 对于图 3.16a 的三次曲线，假设控制点为 $\{\mathbf{P}_i\}=\{(0,0),(1,2),(3,4),(5,2),(5,0),(8,0),(9,3)\}$。绘制曲线。使用公式 (3.8) 计算二阶导数曲线 $\mathbf{C}^{(2)}(u)$。绘制 $\mathbf{C}^{(2)}(u)$。设 $\mathbf{P}_0^{(2)}$ 和 $\mathbf{P}_4^{(2)}$ 为 $\mathbf{C}^{(2)}(u)$ 的第一个和最后一个控制点，即 $\mathbf{P}_0^{(2)}=\mathbf{C}^{(2)}(0)$ 和 $\mathbf{P}_4^{(2)}=\mathbf{C}^{(2)}(1)$。使用公式 (3.9) 和 (3.10) 计算 $\mathbf{C}^{(2)}(0)$ 和 $\mathbf{C}^{(2)}(1)$。

3.7. 可以使用多个节点或多个控制点在表面上创建折痕 (见图 3.24 和 3.25)。如果您想要一个折痕长度小于表面全长，您会使用哪种方法？构造并绘制这样的示例。

3.8. 考虑 B-spline 表面 $\mathbf{S}(u,v)=\sum_{i=0}^{3}\sum_{j=0}^{2}N_{i,2}(u)\,N_{j,2}(v)\,\mathbf{P}_{i,j}$

其中

$$U = \{0, 0, 0, 1/2, 1, 1, 1\}$$

$$V = \{0, 0, 0, 1, 1, 1\}$$

和

$$\mathbf{P}_{0,0} = (0, 0, 0) \ \mathbf{P}_{1,0} = (3, 0, 3) \ \mathbf{P}_{2,0} = (6, 0, 3) \ \mathbf{P}_{3,0} = (9, 0, 0)$$

$$\mathbf{P}_{0,1} = (0, 2, 2) \ \mathbf{P}_{1,1} = (3, 2, 5) \ \mathbf{P}_{2,1} = (6, 2, 5) \ \mathbf{P}_{3,1} = (9, 2, 2)$$

$$\mathbf{P}_{0,2} = (0, 4, 0) \ \mathbf{P}_{1,2} = (3, 4, 3) \ \mathbf{P}_{2,2} = (6, 4, 3) \ \mathbf{P}_{3,2} = (9, 4, 0)$$

通过评估非零 $\mathbf{B}$ -样条基函数并将其与适当的控制点相乘来计算 $\mathbf{S}(3/10, 6/10)$ 。

3.9. 推导 $\mathbf{S}_u(u, v)$, $\mathbf{S}_v(u, v)$ 和 $\mathbf{S}_{uv}(u, v)$ 在三个角 $(u, v) = (0, 1), (1, 0)$ 和 $(1, 1)$ 的表达式 (见公式 [3.24])。

3.10. 设 $\mathbf{S}(u, v)$ 如习题 3.8 所示。绘制该曲面。使用公式 (3.20) 和 (3.21)，计算曲面 $\mathbf{S}_u(u, v)$ 和 $\mathbf{S}_v(u, v)$ 。绘制这两个曲面。使用公式 (3.22) 和在习题 3.9 中推导的表达式，计算曲面四个角的混合偏导数 $\mathbf{S}_{uv}(u, v)$ 。这四个向量的几何意义是什么？

第四章　有理 B 样条曲线与曲面

1　引言

在本章中，我们结合了第 1 章第 1.4 节和 1.5 节的概念以及第 3 章的内容，以获得非均匀有理 B-样条 (NURBS) 曲线和曲面。我们给出定义和一般性质，并推导出 NURBS 曲线和曲面的导数的公式和算法，这些导数以其非有理对应物为基础。关于 NURBS 的最早出版作品是 [Vers75; Till83]。更近期的综述可以在 [Pieg91a] 中找到。

2　NURBS 曲线的定义与性质

一个 p 次 NURBS 曲线定义为

$$\mathbf{C}(u)=\frac{\sum_{i=0}^{n} N_{i,p}(u)\, w_i \mathbf{P}_i}{\sum_{i=0}^{n} N_{i,p}(u)\, w_i} \quad a \le u \le b \tag{4.1}$$

其中 $\{\mathbf{P}_i\}$ 是控制点 (形成控制多边形)，$\{w_i\}$ 是权重，$\{N_{i,p}(u)\}$ 是在非周期 (且非均匀) 节点向量上定义的 p 次 B-样条基函数。

$$U=\left\{\underbrace{a,\ldots,a}_{p+1}, u_{p+1},\ldots,u_{m-p-1},\underbrace{b,\ldots,b}_{p+1}\right\}$$

除非另有说明，我们假设 $a=0, b=1$ 和 $w_i>0$ 对所有 i 成立。设定

$$R_{i,p}(u)=\frac{N_{i,p}(u)\, w_i}{\sum_{j=0}^{n} N_{j,p}(u)\, w_j} \tag{4.2}$$

允许我们将方程 (4.1) 重写为以下形式

$$\mathbf{C}(u)=\sum_{i=0}^{n} R_{i,p}(u)\mathbf{P}_i \tag{4.3}$$

$\{R_{i,p}(u)\}$ 是有理基函数；它们是在 $u\in[0,1]$ 上的分段有理函数。

$R_{i,p}(u)$ 具有以下从方程 (4.2) 及其对应的 $N_{i,p}(u)$ 性质推导出的特性:

P4.1 非负性: 对于所有 i,p 和 $u\in[0,1]$，有 $R_{i,p}(u)\ge 0$；

P4.2 单位分割: 对于所有 $u\in[0,1]$，有 $\sum_{i=0}^{n} R_{i,p}(u)=1$；

P4. $3R_{0,p}(0)=R_{n,p}(1)=1$；

P4.4 对于 $p>0$，所有 $R_{i,p}(u)$ 在区间 $u\in[0,1]$ 上恰好达到一个最大值。

P4.5 局部支持: 对于 $u\notin[u_i,u_{i+p+1})$，有 $R_{i,p}(u)=0$。此外，在任何给定的节点区间内，最多有 $p+1$ 个 $R_{i,p}(u)$ 是非零的 (一般情况下，$R_{i-p,p}(u),\ldots,R_{i,p}(u)$ are nonzero in $[u_i,u_{i+1})$)；

P4.6 所有 $R_{i,p}(u)$ 的导数在节点区间的内部存在，在那里它是一个具有非零分母的有理函数。在节点处，$R_{i,p}(u)$ 是 $p-k$ 次连续可微的，其中 k 是节点的重数；

P4.7 如果对于所有 i 有 $w_i=1$，那么对于所有 i 有 $R_{i,p}(u)=N_{i,p}(u)$；即，$N_{i,p}(u)$ 是 $R_{i,p}(u)$ 的特例。实际上，对于任何 $a\ne 0$，如果对于所有 i 有 $w_i=a$，那么对于所有 i 有 $R_{i,p}(u)=N_{i,p}(u)$。

性质 P4.1-P4.7 产生了 NURBS 曲线的以下重要几何特征:

P4.8 $\mathbf{C}(0)=\mathbf{P}_0$ 和 $\mathbf{C}(1)=\mathbf{P}_n$；这由 P4.3 推导而来；

P4.9 仿射不变性: 通过对控制点应用仿射变换来对曲线施加仿射变换 (见 P3.4，第 3.1 节)；NURBS 曲线在透视投影下也是不变的 ([Lee87; Pieg91a])，这一事实在计算机图形学中非常重要；

P4.10 强凸包性质: 如果 $u\in[u_i,u_{i+1})$，则 $\mathbf{C}(u)$ 位于控制点 $\mathbf{P}_{i-p},\ldots,\mathbf{P}_i$ 的凸包内 (见图 4.1，其中 $\mathbf{C}(u)$ 对于 $u\in[1/4,1/2)$ (虚线段) 包含在凸包内)；

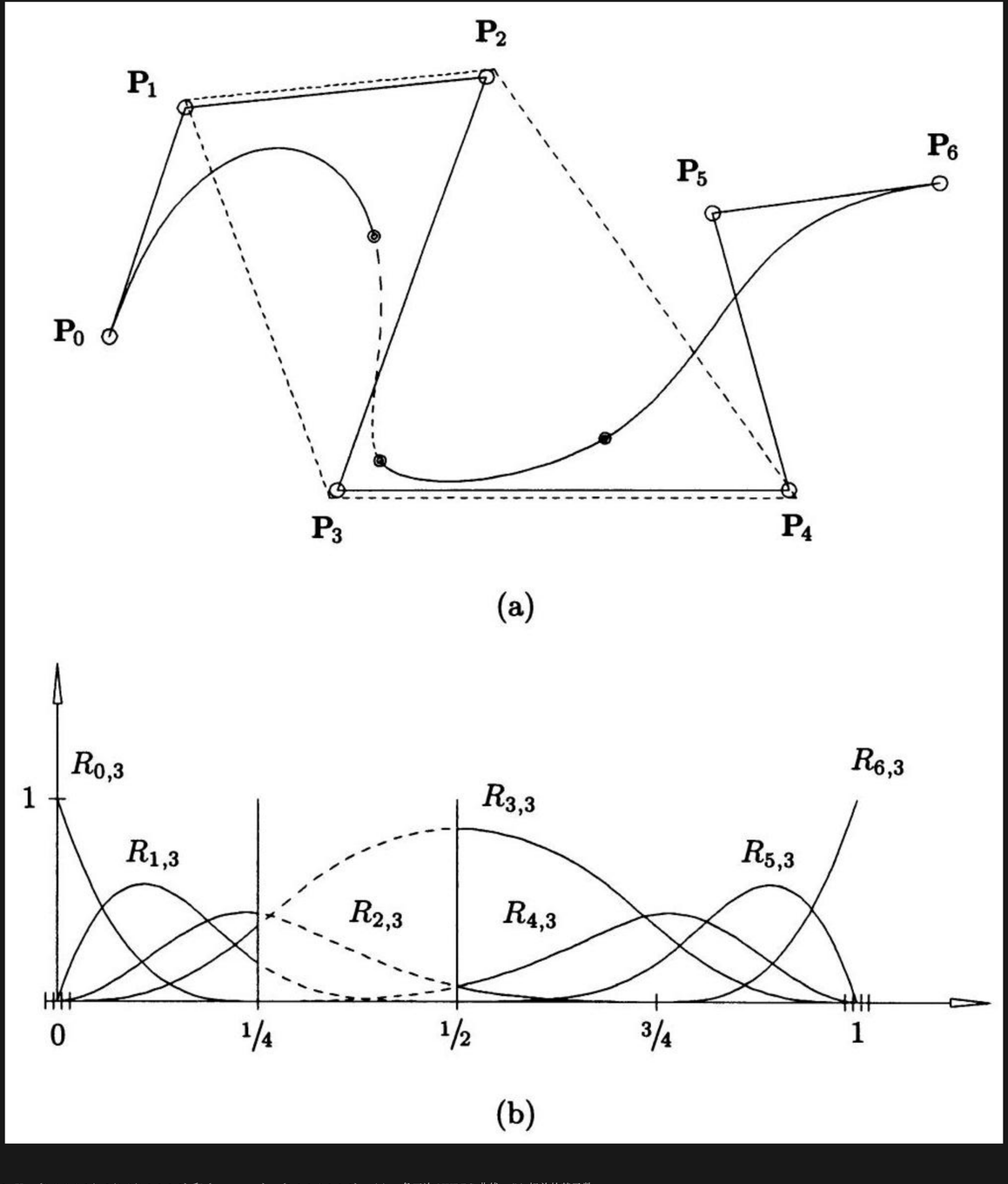


图 4.1. $U=\{0,0,0,0,1/4,1/2,3/4,1,1,1,1\}$ 和 $\{w_0,\ldots,w_6\}=\{1,1,1,3,1,1,1\}$。(a) 一条三次 NURBS 曲线；(b) 相关的基函数。

$\{\mathbf{P}_1,\mathbf{P}_2,\mathbf{P}_3,\mathbf{P}_4\}$ 的虚线区域)；这由 P4.1、P4.2 和 P4.5 推导而来；

P4.11 $\mathbf{C}(u)$ 在节点跨度的内部是无限可微的，并且在重数为 k 的节点处是 $p-k$ 次可微的；

P4.12 变化减小性质: 没有平面与曲线的交点比与控制多边形的交点更多 (对于二维曲线，将“平面”一词替换为“直线”)；

P4.13 没有内部节点的 NURBS 曲线是有理 Bézier 曲线，因为 $N_{i,p}(u)$ 简化为 $B_{i,n}(u)$；将方程 (4.2) 和 (4.3) 与方程 (1.15) 进行比较。这与 P4.7 一起，意味着 NURBS 曲线包含非有理 B-样条和有理及非有理 Bézier 曲线作为特例；

P4.14 局部近似: 如果控制点 $\mathbf{P}_i$ 被移动，或者权重 w_i 被改变，它只影响区间 $u\in[u_i,u_{i+p+1})$ 上曲线的那一部分；这遵循于 P4.5。

性质 P4.14 对于交互式形状设计非常重要。使用 NURBS 曲线，我们可以利用控制点移动和权重修改来实现局部形状控制。图 4.2-4.6 显示了修改单个权重的效果。定性效果是: 假设 $u\in[u_i,u_{i+p+1})$；那么如果 w_i 增加 (减少)，点 $\mathbf{C}(u)$ 会更靠近 (远离) $\mathbf{P}_i$，因此曲线被拉向 (推离) $\mathbf{P}_i$。此外，对于固定的 u，$\mathbf{C}(u)$ 的移动沿着一条直线 (图 4.6)。在图 4.6 中，u 固定而 w_3 变化。让

$$\mathbf{B} = \mathbf{C}(u; w_3 = 0) \tag{4.4}$$

$$\mathbf{N} = \mathbf{C}(u; w_3 = 1)$$

然后由 $\mathbf{B}$ 和 $\mathbf{N}$ 定义的直线通过 $\mathbf{P}_3$ ，对于任意的 $0 < w_3 < \infty, \mathbf{B}_3 = \mathbf{C}(u; w_3)$ ，它位于 $\mathbf{B}$ 和 $\mathbf{P}_3$ 之间的这条线段上。我们将在后面的章节中回到这个主题。

与有理 Bézier 曲线一样，同质坐标提供了一种有效的方法来表示 NURBS 曲线。设 H 为由方程 (1.16) 给出的透视映射。对于给定的一组控制点 $\{\mathbf{P}_i\}$ 和权重 $\{w_i\}$ ，构造加权控制点 $\mathbf{P}_i^w = (w_i x_i, w_i y_i, w_i z_i, w_i)$ 。然后在四维空间中定义非有理 (分段多项式)B-spline 曲线为

空间为

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,p}(u)\, \mathbf{P}_i^w \tag{4.5}$$

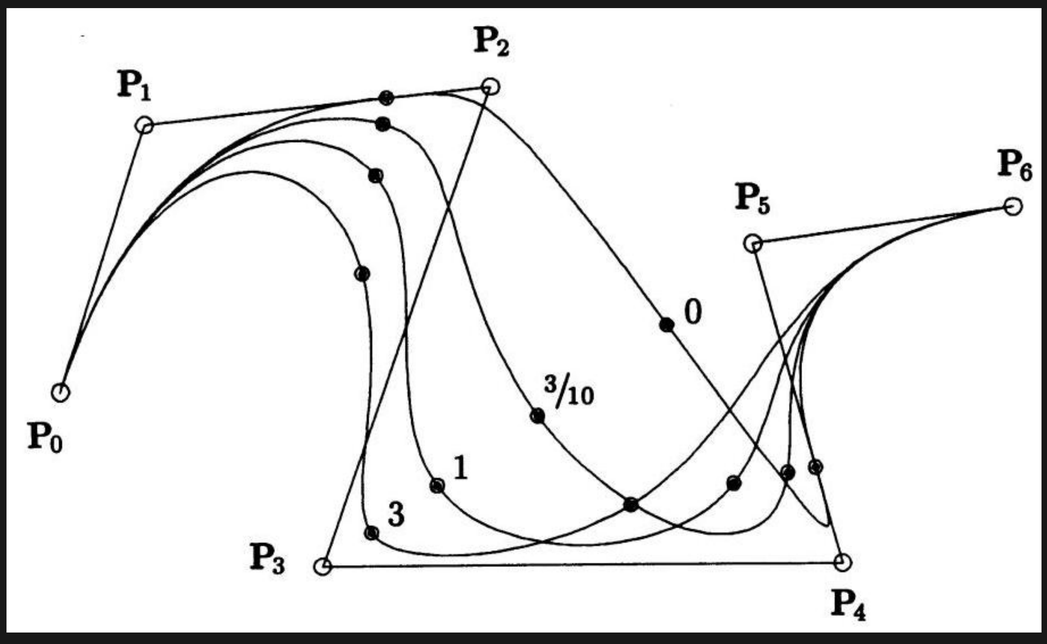


图 4.2. 有理三次 B-spline 曲线，随着 w_3 的变化。

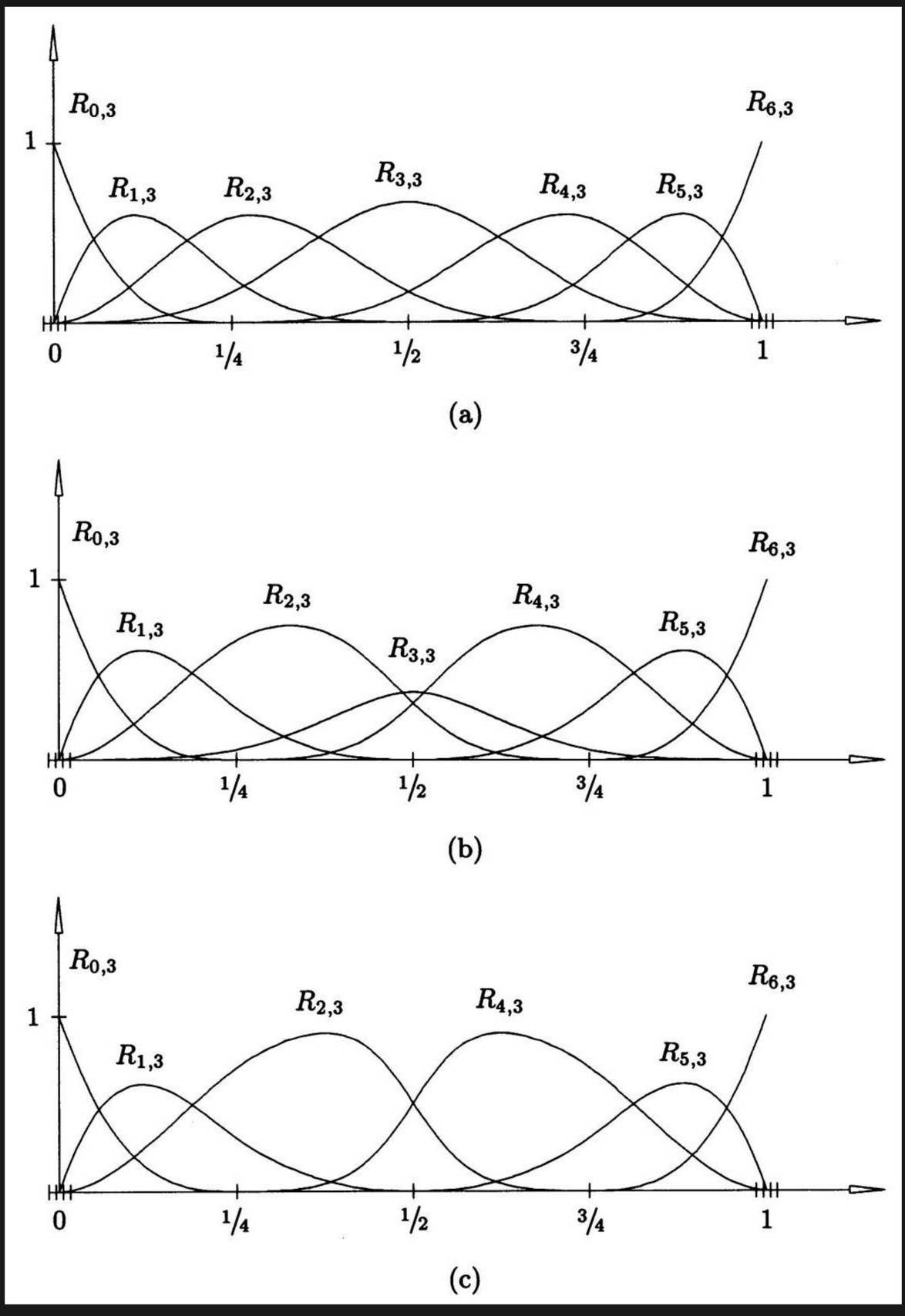


图 4.3. 图 4.2 曲线的立方基函数。(a) $w_3 = 1$; (b) $w_3 = 3/10$; (c) $w_3 = 0$.

应用透视映射 H 到 $\mathbf{C}^w(u)$ 产生相应的有理 B 样条曲线 (在三维空间中分段有理)

$$\mathbf{C}(u) = H\{\mathbf{C}^w(u)\} = H\left\{\sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w\right\}$$

$$= \frac{\sum_{i=0}^{n} N_{i,p}(u)\,w_i\mathbf{P}_i}{\sum_{i=0}^{n} N_{i,p}(u)\,w_i} = \sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i$$

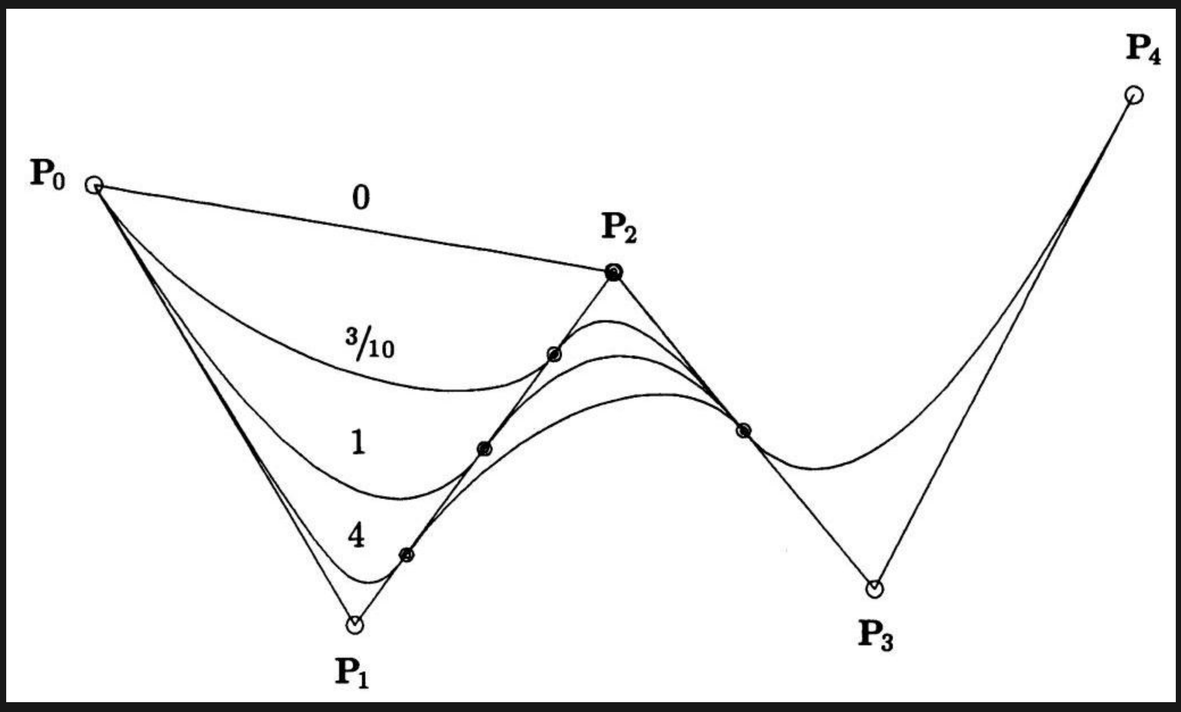


图 4.4. 有理二次曲线，具有变化的 w_1 。

我们可以交替地将 $\mathbf{C}^w(u)$ 或 $\mathbf{C}(u)$ 称为 NURBS 曲线，尽管严格来说，$\mathbf{C}^w(u)$ 不是一条有理曲线。

示例

Ex4.1 设 $U=\{0,0,0,1,2,3,3,3\},\{w_0,\ldots,w_4\}=\{1,4,1,1,1\},\{\mathbf{P}_0,\ldots,\mathbf{P}_4\}=\{(0,0),(1,1),(3,2),(4,1),(5,-1)\}$ 。我们计算在 $u=1$ 处的有理 B 样条曲线上的点。现在 u 在节点跨度 $[u_3,u_4)$ 中，并且

$$N_{3,0}(1)=1$$

$$N_{2,1}(1)=\frac{2-1}{2-1}N_{3,0}(1)=1$$

$$N_{3,1}(1)=\frac{1-1}{2-1}N_{3,0}(1)=0$$

$$N_{1,2}(1)=\frac{2-1}{2-0}N_{2,1}(1)=\frac{1}{2}$$

$$N_{2,2}(1)=\frac{1-0}{2-0}N_{2,1}(1)=\frac{1}{2}$$

$$N_{3,2}(1)=0$$

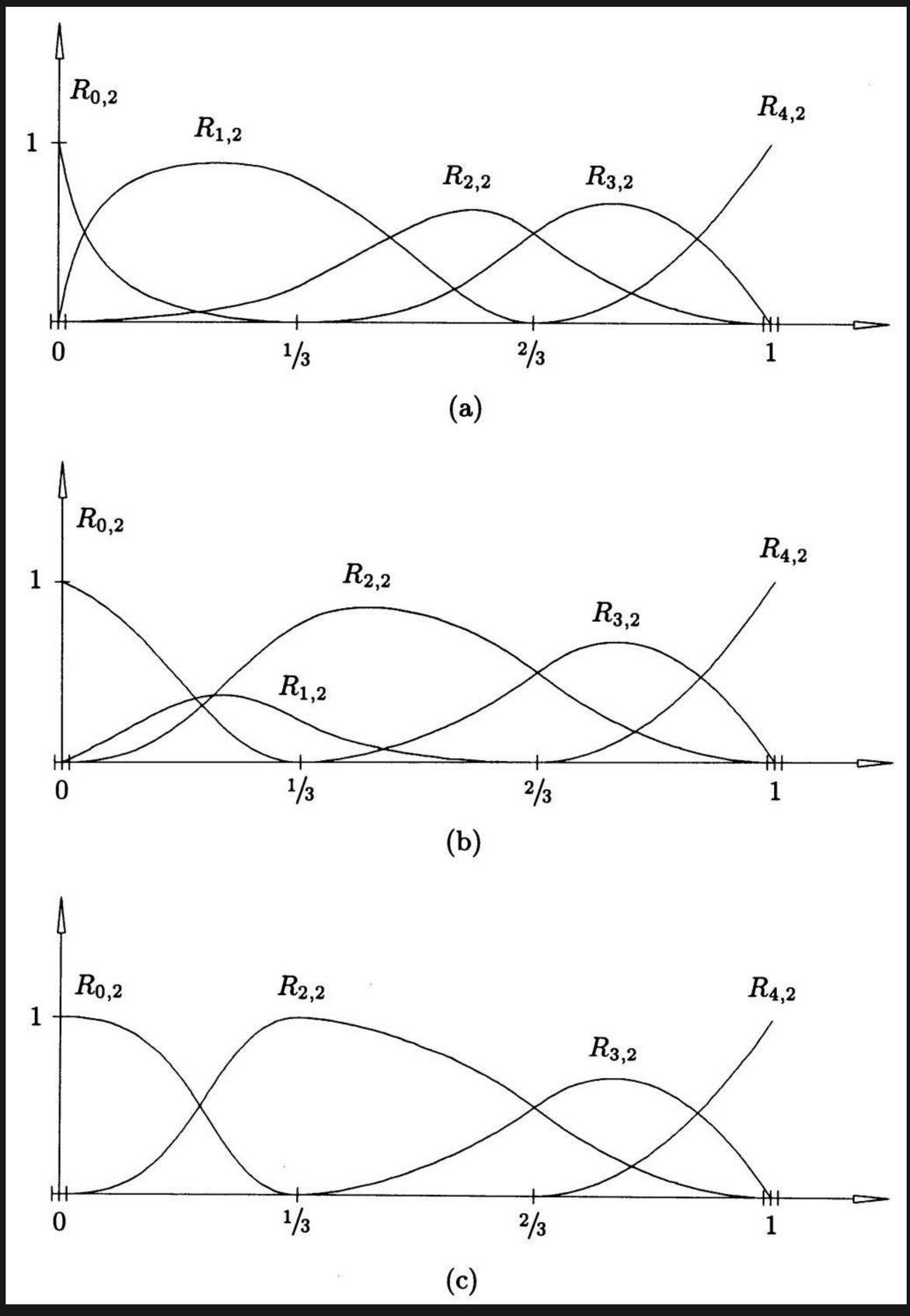


图 4.5. 图 4.4 曲线的二次基函数。(a) $w_1 = 4$; (b) $w_1 = 3/10$; (c) $w_1 = 0$ 。

因此

$$\mathbf{C}^w(1) = \frac{1}{2}\mathbf{P}_1^w + \frac{1}{2}\mathbf{P}_2^w = \frac{1}{2}(4,4,4) + \frac{1}{2}(3,2,1) = \left(\frac{7}{2}, 3, \frac{5}{2}\right)$$

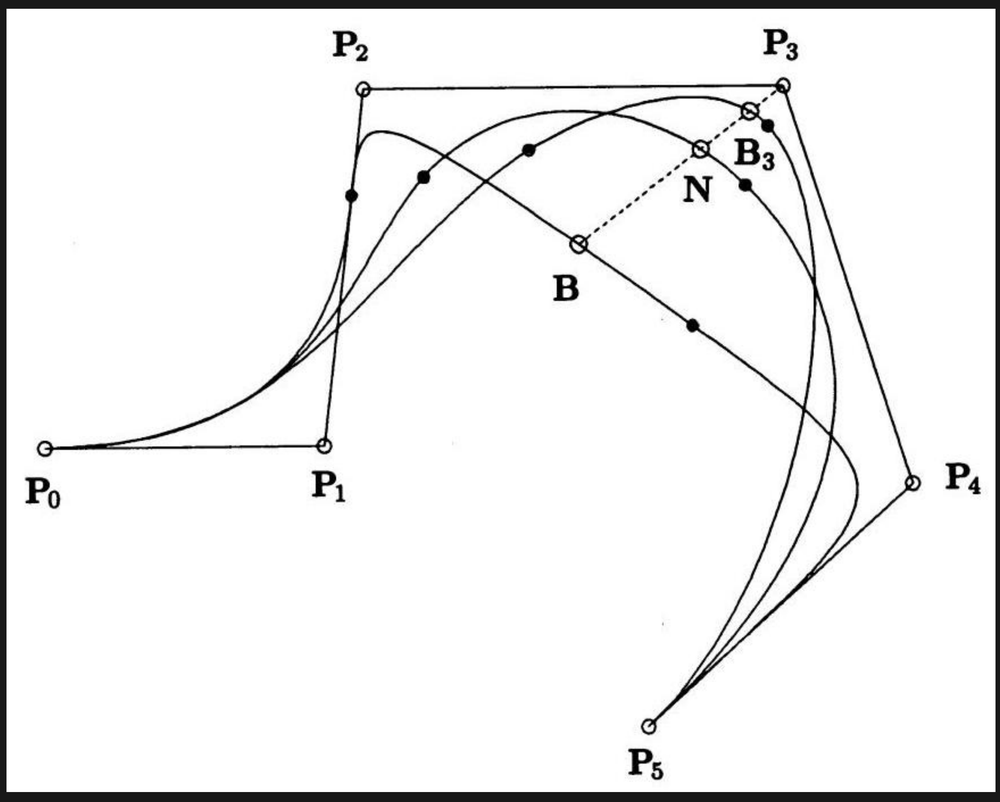


图 4.6. 权重 w_3 的修改。

投影产生

$$\mathbf{C}(1)=\left(\frac{7}{5},\frac{6}{5}\right)$$

算法 A4.1 计算在固定 u 值下的有理 B-样条曲线上的点。它基于公式 (4.5)，即假设数组 Pw 中的加权控制点 (本书其余部分的所有算法均如此，除非另有说明)。因此，Pw[i] 包含 $\mathbf{P}_i=(w_ix_i,w_iy_i,w_iz_i,w_i)$。Cw 表示 $\mathbf{C}^w(u)$ 上的四维点，而 $\mathbf{C}$ 表示 $\mathbf{C}(u)$ 上的三维点 (输出)。在本书的其余部分，我们使用符号 $C=Cw/w$ 来表示投影。

ALGORITHM A4.1

```
CurvePoint(n, p, U, Pw, u, C)
{ /* Compute point on rational B-spline curve */
  /* Input: n, p, U, Pw, u */
  /* Output: C */
span = FindSpan(n, p, u, U);
BasisFuns(span, u, p, U, N);
Cw = 0.0;
for (j=0; j<=p; j++)
  Cw = Cw + N[j] * Pw[span - p + j];
C = Cw/w; /* Divide by weight */
}
```

3 NURBS 曲线的导数

有理函数的导数是复杂的，涉及高次幂的分母。在第 3.3 节中，我们开发了计算非有理 B-样条曲线导数的公式和算法。这些公式和算法当然适用于 $\mathbf{C}^w(u)$，因为它是四维空间中的非有理曲线。在本节中，我们开发了公式，以导数 $\mathbf{C}(u)$ 表达为 $\mathbf{C}^w(u)$ 的导数。

设

$$\mathbf{C}(u)=\frac{w(u)\,\mathbf{C}(u)}{w(u)}=\frac{\mathbf{A}(u)}{w(u)}$$

其中 $\mathbf{A}(u)$ 是向量值函数，其坐标是 $\mathbf{C}^w(u)$ ($\mathbf{A}(u)$ is the numerator of Eq. [4.1]). Then) 的前三个坐标

$$\mathbf{C}'(u)=\frac{w(u)\,\mathbf{A}'(u)-w'(u)\,\mathbf{A}(u)}{w(u)^2}$$

$$=\frac{w(u)\,\mathbf{A}'(u)-w'(u)\,w(u)\,\mathbf{C}(u)}{w(u)^2}=\frac{\mathbf{A}'(u)-w'(u)\,\mathbf{C}(u)}{w(u)} \tag{4.7}$$

由于 $\mathbf{A}(u)$ 和 $w(u)$ 表示 $\mathbf{C}^w(u)$ 的坐标，我们使用公式 (3.4)-(3.6) 获得它们的第一导数。我们通过使用莱布尼茨法则对 $\mathbf{A}(u)$ 进行求导来计算高阶导数

$$\mathbf{A}^{(k)}(u)=\left(w(u)\,\mathbf{C}(u)\right)^{(k)}=\sum_{i=0}^{k}\binom{k}{i}w^{(i)}(u)\,\mathbf{C}^{(k-i)}(u)$$

$$=w(u)\,\mathbf{C}^{(k)}(u)+\sum_{i=1}^{k}\binom{k}{i}w^{(i)}(u)\,\mathbf{C}^{(k-i)}(u)$$

从中我们得到

$$\mathbf{C}^{(k)}(u)=\frac{\mathbf{A}^{(k)}(u)-\sum_{i=1}^{k}\binom{k}{i}w^{(i)}(u)\,\mathbf{C}^{(k-i)}(u)}{w(u)} \tag{4.8}$$

方程 (4.8) 给出了 k 的 $\mathbf{C}(u)$ 次导数，以 k 的 $\mathbf{A}(u)$ 次导数以及 $\mathbf{C}(u)$ 和 $w(u)$ 的第一到 (k - 1) 次导数为基础。导数 $\mathbf{A}^{(k)}(u)$ 和 $w^{(i)}(u)$ 是通过使用方程 (3.3) 和算法 A3.2 或方程 (3.8) 和算法 A3.4 获得的。

让我们推导 NURBS 曲线在其端点 ($u=0, u=1$) 的第一导数的表达式。从方程 (3.7) 我们得到

$$\mathbf{A}'(0)=\frac{p}{u_{p+1}}\left(w_1\mathbf{P}_1-w_0\mathbf{P}_0\right)\ w'(0)=\frac{p}{u_{p+1}}\left(w_1-w_0\right)$$

并且从方程 (4.7)

$$\mathbf{C}'(0)=\frac{\frac{p}{u_{p+1}}\left(w_1\mathbf{P}_1-w_0\mathbf{P}_0\right)-\frac{p}{u_{p+1}}\left(w_1-w_0\right)\mathbf{P}_0}{w_0}$$

从中可以得出

$$\mathbf{C}'(0)=\frac{p}{u_{p+1}}\frac{w_1}{w_0}\left(\mathbf{P}_1-\mathbf{P}_0\right) \tag{4.9}$$

类似地

$$\mathbf{C}'(1)=\frac{p}{1-u_{m-p-1}}\frac{w_{n-1}}{w_n}\left(\mathbf{P}_n-\mathbf{P}_{n-1}\right) \tag{4.10}$$

示例

Ex4.2 考虑在第 1.4 节中给出的二次有理贝塞尔圆弧 (图 1.19b)。这是一个在节点向量 $U=\{0,0,0\ ,\ 1,1,1\}$ 上的 NURBS 曲线，具有 $\{\mathbf{P}_i\}=\{(1,0),(1,1),(0,1)\}$ 和 $\{w_i\}=\{1,1,2\}$。从方程 (4.9) 和 (4.10) 我们得到

$$\mathbf{C}'(0)=\frac{2}{1}\frac{1}{1}\left(\mathbf{P}_1-\mathbf{P}_0\right)=(0,2)$$

$$\mathbf{C}'(1)=\frac{2}{1-0}\frac{1}{2}\left(\mathbf{P}_2-\mathbf{P}_1\right)=(-1,0)$$

从方程 (4.8)

$$\mathbf{C}''(0)=\frac{\mathbf{A}''(0)-2w'(0)\,\mathbf{C}'(0)-w''(0)\,\mathbf{C}(0)}{w_0}$$

从方程 (3.9) 或方程 (1.10)

$$\mathbf{A}''(0)=2\left(w_0\mathbf{P}_0-2w_1\mathbf{P}_1+w_2\mathbf{P}_2\right)$$

并且

$$w''(0) = 2(w_0 - 2w_1 + w_2)$$

从 $w'(0) = 2(w_1 - w_0)$ 可以得出

$$\mathbf{C}''(0) = \frac{2}{w_0}[w_0\mathbf{P}_0 - 2w_1\mathbf{P}_1 + w_2\mathbf{P}_2 - 4(w_1 - w_0)(\mathbf{P}_1 - \mathbf{P}_0)$$

$$-(w_0 - 2w_1 + w_2)\mathbf{P}_0]$$

$$= 2(\mathbf{P}_0 - 2\mathbf{P}_1 + 2\mathbf{P}_2 - \mathbf{P}_0) = 4(\mathbf{P}_2 - \mathbf{P}_1) = (-4, 0)$$

$\mathbf{C}''(1)$ 的计算留作练习。

现在假设 u 是固定的，并且 $\mathbf{A}(u)$ 和 $w(u)$ 的零阶到 d 阶导数已经计算并分别加载到数组 Aders 和 wders 中，即 $\mathbf{C}^w(u)$ 已经被求导，其坐标分离到 Aders 和 wders 中。算法 A4.2 计算点 $\mathbf{C}(u)$ 和导数 $\mathbf{C}^{(k)}(u), 1 \le k \le d$ 。曲线点返回在 CK [0] 中，第 k 阶导数返回在 CK[k] 中。数组 Bin[][] 包含预计算的二项式系数 (Bin [k] [i] 是 $\binom{k}{i}$)。

ALGORITHM A4.2

```
RatCurveDerivs (Aders, wders, d, CK)
    { /* Compute C(u) derivatives from Cw(u) derivatives */
      /* Input: Aders, wders, d */
      /* Output: CK */
    for (k=0; k<=d; k++)
      {
      v = Aders[k];
      for (i=1; i<=k; i++)
        v = v - Bin[k][i]*wders[i]*CK[k-i];
      CK[k] = v/wders[0];
      }
    }
```

图 4.7 显示了三次 NURBS 曲线的一阶、二阶和三阶导数。导数向量分别缩放为 0.4、0.08 和 0.03。

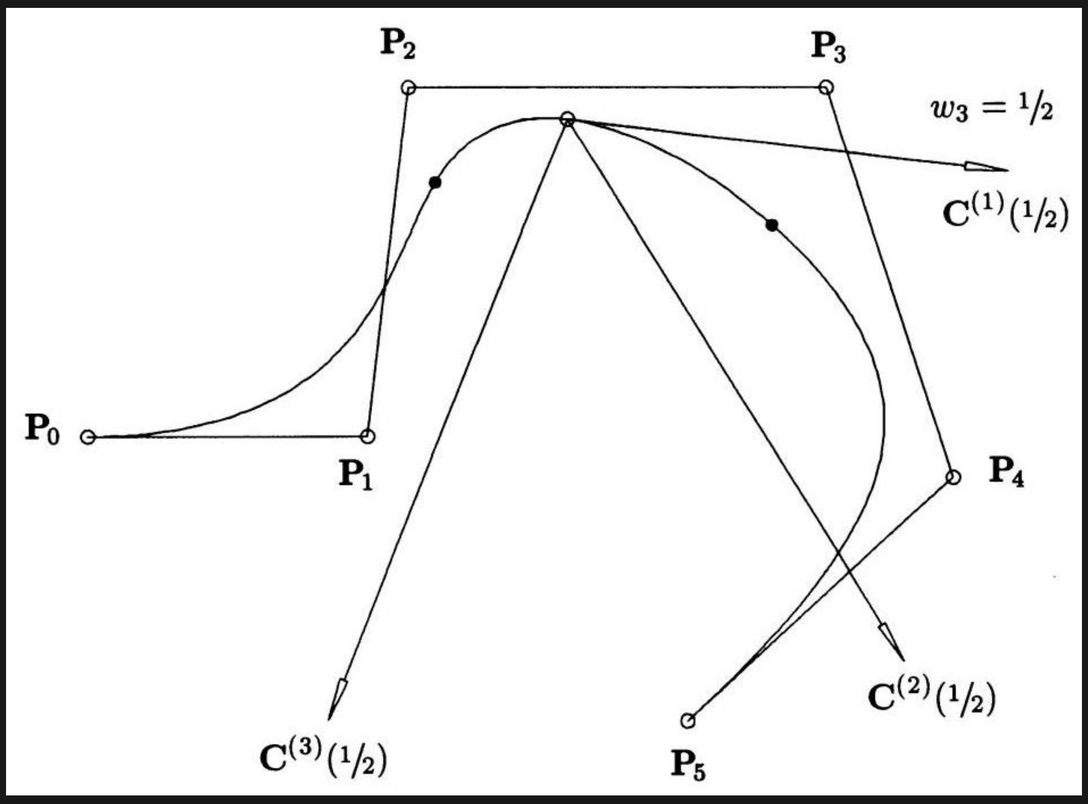


图 4.7. 在 $u = 1/2$ 处计算的三次 NURBS 曲线的一阶、二阶和三阶导数，使用 $w_3 = 1/2$ 和 $w_i = 1, i \neq 3$ 。

4 NURBS 曲面的定义与性质

在 u 方向上为 p 阶，在 v 方向上为 q 阶的 NURBS 曲面是一个双变量向量值分段有理函数，形式为

$$\mathbf{S}(u,v) = \frac{\sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u) N_{j,q}(v) w_{i,j}\mathbf{P}_{i,j}}{\sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u) N_{j,q}(v) w_{i,j}} \quad 0 \le u, v \le 1 \tag{4.11}$$

$\{\mathbf{P}_{i,j}\}$ 形成一个双向控制网，$\{w_{i,j}\}$ 是权重，$\{N_{i,p}(u)\}$ 和 $\{N_{j,q}(v)\}$ 是定义在节点向量上的非有理 B 样条基函数。

$$U = \left\{\underbrace{0,\ldots,0}_{p+1}, u_{p+1}, \ldots, u_{r-p-1}, \underbrace{1,\ldots,1}_{p+1}\right\}$$

$$V = \{\underbrace{0,\ldots,0}_{q+1}, v_{q+1}, \ldots, v_{s-q-1}, \underbrace{1,\ldots,1}_{q+1}\}$$

其中 $r = n + p + 1$ 和 $s = m + q + 1$ 。

引入分段有理基函数

$$R_{i,j}(u,v) = \frac{N_{i,p}(u) N_{j,q}(v) w_{i,j}}{\sum_{k=0}^{n}\sum_{l=0}^{m} N_{k,p}(u) N_{l,q}(v) w_{k,l}} \tag{4.12}$$

表面方程 (4.11) 可以写成

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} R_{i,j}(u,v)\mathbf{P}_{i,j} \tag{4.13}$$

图 4.8 和 4.9 显示了 NURBS 曲面的示例。

函数的主要属性 $R_{i,j}(u,v)$ 大致与第 3.4 节中给出的非理性基函数的属性相同 $N_{i,p}(u) N_{j,q}(v)$ 。我们在此总结它们。

P4.15 非负性: $R_{i,j}(u,v) \ge 0$ 对所有 i, j, u 和 v 成立；

P4.16 单位划分: $\sum_{i=0}^{n}\sum_{j=0}^{m} R_{i,j}(u,v) = 1$ 对所有 $(u,v) \in [0,1] \times [0,1]$ 成立；

P4.17 局部支持: $R_{i,j}(u,v) = 0$ 如果 (u, v) 在由 $[u_i, u_{i+p+1}) \times [v_j, v_{j+q+1})$ 给出的矩形之外；

P4.18 在任何给定的形式为 $[u_{i_0}, u_{i_0+1}) \times [v_{j_0}, v_{j_0+1})$ 的矩形中，最多有 $(p+1)(q+1)$ 个基函数非零，特别是 $R_{i,j}(u,v)$ 对于 $i_0 - p \le i \le i_0$ 和 $j_0 - q \le j \le j_0$ 是非零的；

P4.19 极值: 如果 $p > 0$ 和 $q > 0$ ，则 $R_{i,j}(u,v)$ 恰好达到一个最大值；

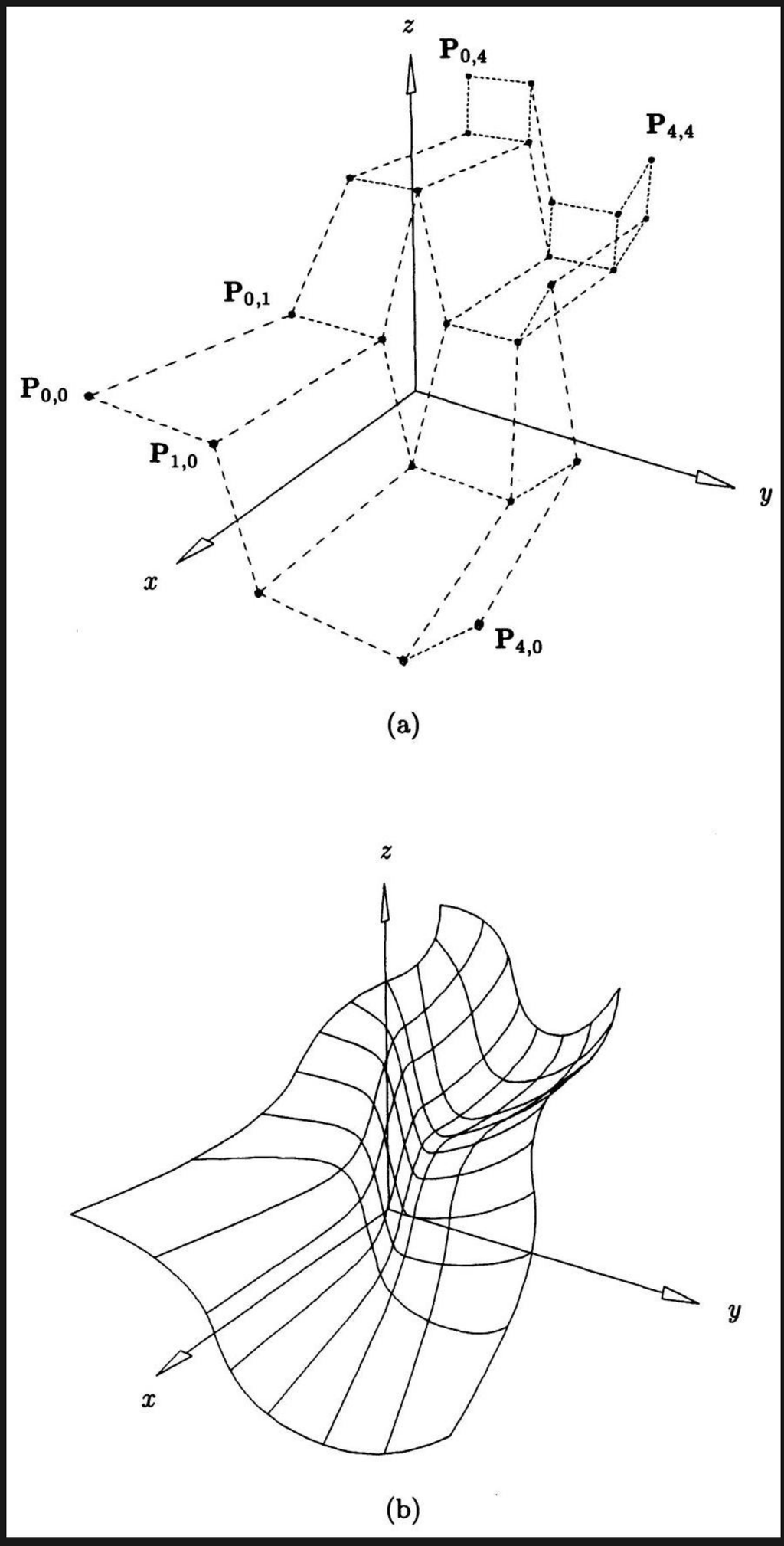


图 4.8. 控制网和二次 NURBS 曲面，$w_{1,1} = w_{1,2} = w_{2,1} = w_{2,2} = 10$，其他权重为 1.$U = V = \{0, 0, 0, 1/3, 2/3, 1, 1, 1\}$ 。(a) 控制网；(b) 二次 NURBS 曲面。

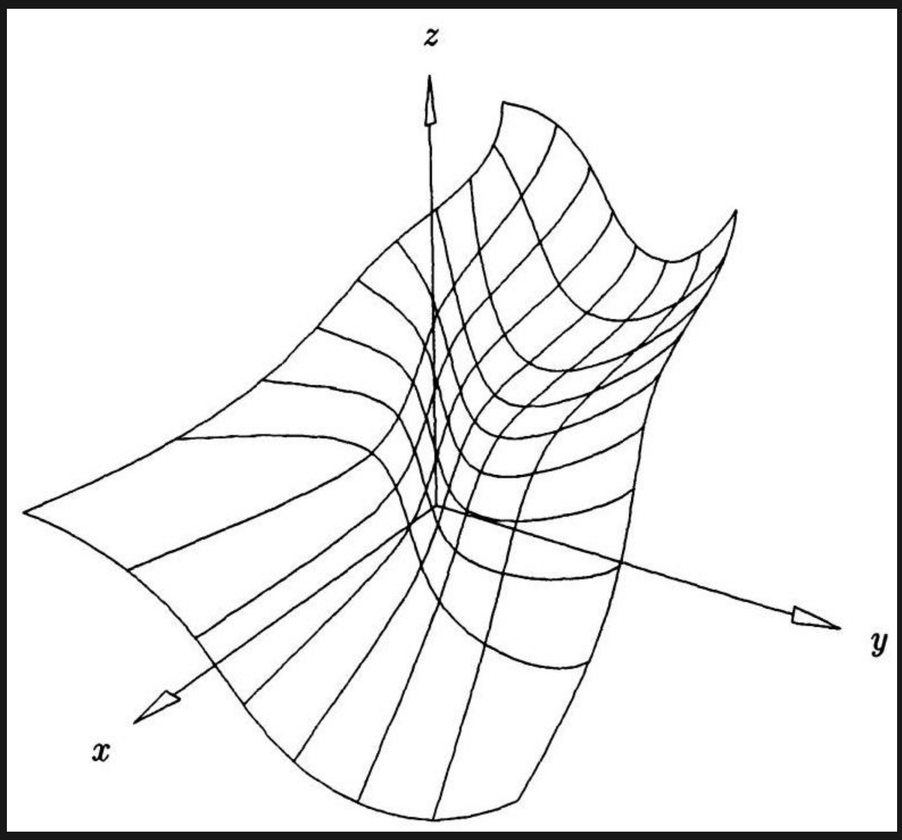


图 4.9. 由图 4.8a 中的控制网定义的三次 NURBS 曲面，权重与图 4.8 中相同。

P4. $20 R_{0,0}(0,0)=R_{n,0}(1,0)=R_{0,m}(0,1)=R_{n,m}(1,1)=1$;

P4.21 可微性: 在由 u 和 v 节点线形成的矩形内部，所有 $R_{i,j}(u,v)$ 的偏导数存在。在 u 节点 (vknot) 处，它在 $u(v)$ 方向上是 $p-k(q-k)$ 次可微的，其中 k 是节点的重数;

P4.22 如果所有 $w_{i,j}=a$ 对于 $0 \leq i \leq n, 0 \leq j \leq m$ 和 $a \neq 0$ 成立，则 $R_{i,j}(u,v)=N_{i,p}(u) N_{j,q}(v)$ 对所有 i, j 成立。

性质 P4.15-P4.22 产生了 NURBS 曲面的以下重要几何性质:

P4.23 角点插值: $\mathbf{S}(0,0)=\mathbf{P}_{0,0}, \mathbf{S}(1,0)=\mathbf{P}_{n,0}, \mathbf{S}(0,1)=\mathbf{P}_{0,m}$ ， 和 $\mathbf{S}(1,1)=\mathbf{P}_{n,m}$;

P4.24 仿射不变性: 通过对控制点应用仿射变换来对曲面进行仿射变换;

P4.25 强凸包性质: 假设 $w_{i,j} \geq 0$ 对所有 i, j 成立。如果 $(u,v) \in [u_{i_0}, u_{i_0+1}) \times [v_{j_0}, v_{j_0+1})$ ，则 $\mathbf{S}(u,v)$ 在控制点 $\mathbf{P}_{i,j}, i_0-p \leq i \leq i_0$ 和 $j_0-q \leq j \leq j_0$ 的凸包内;

P4.26 局部修改: 如果 $\mathbf{P}_{i,j}$ 被移动，或者 $w_{i,j}$ 被改变，它只会影响矩形 $[u_i, u_{i+p+1}) \times [v_j, v_{j+q+1})$ 内的曲面形状;

P4.27 非理性 B-spline 和 Bézier 以及理性 Bézier 曲面是 NURBS 曲面的特例;

P4.28 可微性: $\mathbf{S}(u,v)$ 在一个重数为 k 的 u 节点 (vknot) 处对 $u(v)$ 是 $p-k(q-k)$ 次可微的。

我们注意到，目前尚无已知的 NURBS 曲面的变差减小性质 (见 [Prau92])。

我们可以使用控制点移动和权重修改来局部改变 NURBS 曲面的形状。图 4.10 和 4.11 显示了当单个权重 $w_{i,j}$ 被修改时，对基函数 $R_{i,j}(u,v)$ 和曲面形状的影响。将这些图与图 3.19b 和 3.20a, b 进行比较。从定性上看，对曲面的影响是: 假设 $(u,v) \in [u_i, u_{i+p+1}) \times [v_j, v_{j+q+1})$ ；那么如果 $w_{i,j}$ 增加 (减少)，点 $\mathbf{S}(u,v)$ 将更靠近 (远离) $\mathbf{P}_{i,j}$ ，因此曲面被拉向 (推离) $\mathbf{P}_{i,j}$ 。与曲线的情况一样，$\mathbf{S}(u,v)$ 的移动是沿着一条直线。在图 4.12 (u,v) 中，$w_{2,2}$ 是变化的。

$$\mathbf{S}=\mathbf{S}(u,v; w_{2,2}=0)$$

$$\mathbf{M}=\mathbf{S}(u,v; w_{2,2}=1) \tag{4.14}$$

然后由 $\mathbf{S}$ 和 $\mathbf{M}$ 定义的直线通过 $\mathbf{P}_{2,2}$ ，对于任意的 $w_{2,2}, 0<w_{2,2}<\infty, \mathbf{S}_{2,2}=\mathbf{S}(u,v; w_{2,2})$ ，它位于这条线段 $\mathbf{S}$ 和 $\mathbf{P}_{2,2}$ 之间。

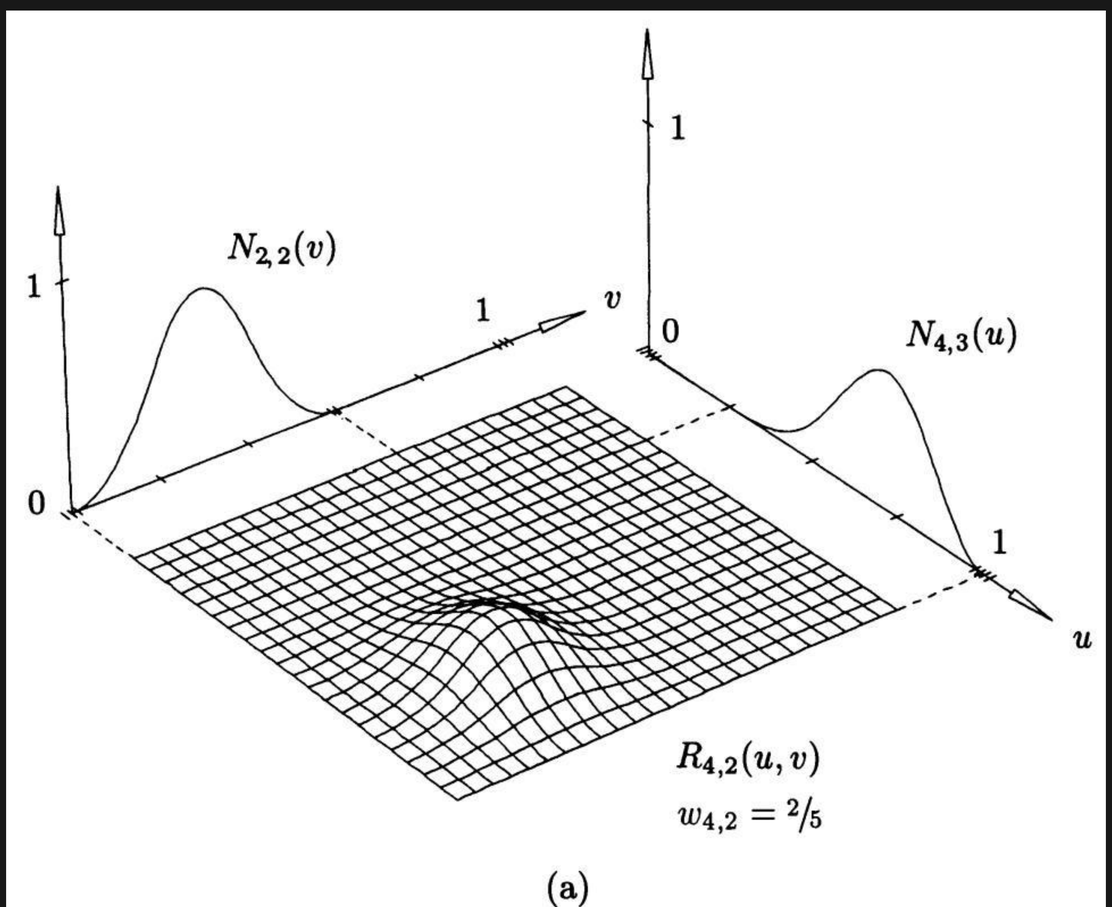


图 4.10. 基函数 $R_{4,2,(u,v)}$ ， 对于所有 $(i,j) \neq (4,2)$. $(a)\, w_{4,2}=2/5$; 的 $U=\{0,0,0,0,1/4,1/2,3/4,1,1,1\}$ 和 $V=\{0,0,0,1/5,2/5,3/5,3/5,4/5,1,1/1\}$.$w_{i,j}=1$ (b) $w_{4,2}=6$ 。

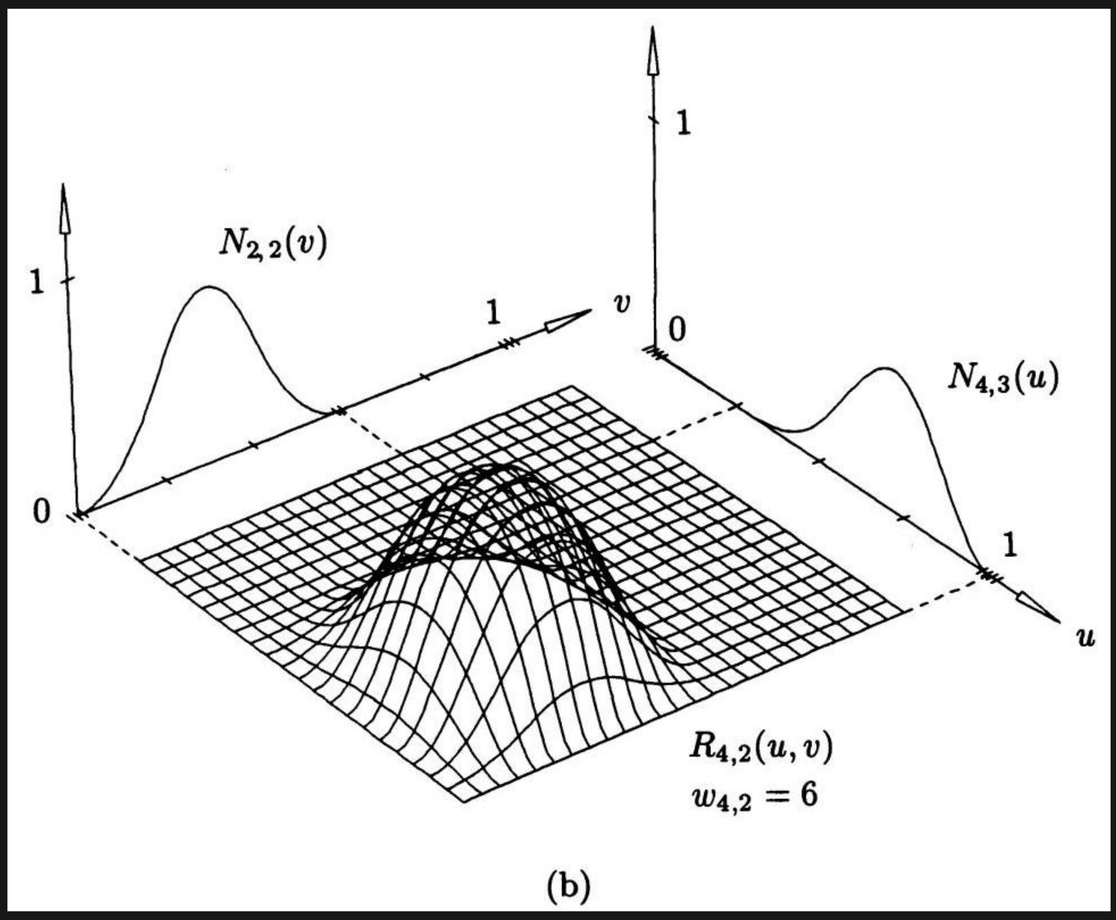


图 4.10. (续)

使用齐次坐标表示 NURBS 曲面是方便的，

即

$$\mathbf{S}^w(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{m}N_{i,p}(u)\,N_{j,q}(v)\,\mathbf{P}_{i,j}^w \tag{4.15}$$

其中 $\mathbf{P}_{i,j}^w=(w_{i,j}x_{i,j},w_{i,j}y_{i,j},w_{i,j}z_{i,j},w_{i,j})$ 。然后 $\mathbf{S}(u,v)=H\{\mathbf{S}^w(u,v)\}$ 。我们可以交替地将 $\mathbf{S}^w(u,v)$ 或 $\mathbf{S}(u,v)$ 称为 NURBS 曲面。严格来说，$\mathbf{S}^w(u,v)$ 是一个张量积的分段多项式曲面，位于四维空间中。$\mathbf{S}(u,v)$ 是一个分段有理曲面，位于三维空间中；它不是一个张量积曲面，因为 $R_{i,j}(u,v)$ 不是单变量基函数的乘积。

示例

Ex4.3 设 $\mathbf{S}^w(u,v)=\sum_{i=0}^{7}\sum_{j=0}^{4}N_{i,2}(u)\,N_{j,2}(v)$ ，具有

$$U=\{0,0,0,1,2,3,4,4,5,5,5\}$$

和

$$V=\{0,0,0,1,2,3,3,3\}$$

让我们在 $(u,v)=(5/2,1)$ 处评估表面。然后 $u\in[u_4,u_5)$ 和 $v\in[v_3,v_4)$ ，根据第 3.2 节和第 4.2 节，我们知道

$$N_{2,2}\left(\frac{5}{2}\right)=\frac{1}{8}\,N_{3,2}\left(\frac{5}{2}\right)=\frac{6}{8}\,N_{4,2}\left(\frac{5}{2}\right)=\frac{1}{8}$$

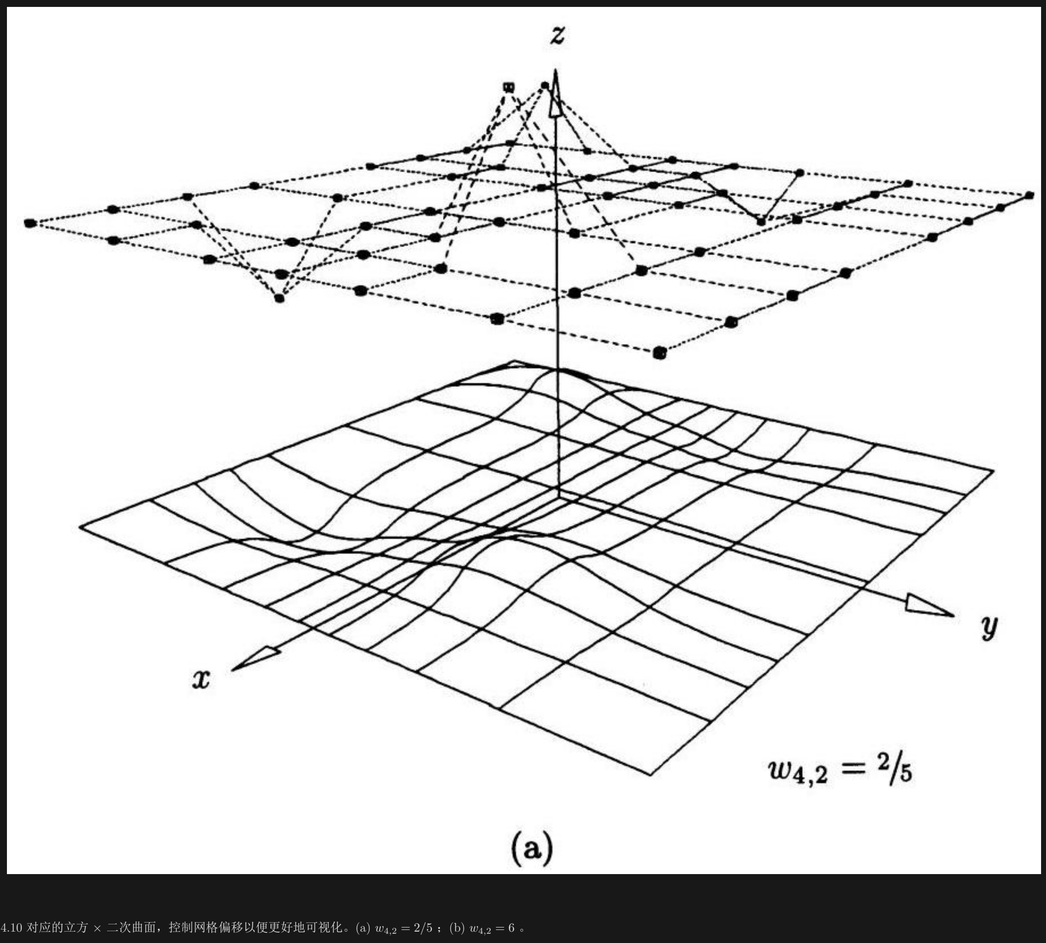


图 4.11。与图 4.10 对应的立方 × 二次曲面，控制网格偏移以便更好地可视化。(a) $w_{4,2}=2/5$ ；(b) $w_{4,2}=6$ 。

和

$$N_{1,2}(1)=\frac{1}{2}\,N_{2,2}(1)=\frac{1}{2}\,N_{3,2}(1)=0$$

现在假设

$$[\mathbf{P}_{i,j}^w]=\begin{bmatrix}(0,2,4,1) & (0,6,4,2) & (0,2,0,1)\\ (4,6,8,2) & (12,24,12,6) & (4,6,0,2)\\ (4,2,4,1) & (8,6,4,2) & (4,2,0,1)\end{bmatrix}$$

$$i = 2, 3, 4;\ j = 1, 2, 3$$

然后

$$\mathbf{S}^w\left(\frac{5}{2}, 1\right) = \left[\begin{array}{ccc} \frac{1}{8} & \frac{6}{8} & \frac{1}{8} \end{array}\right] \times$$

$$\begin{bmatrix} (0,2,4,1) & (0,6,4,2) & (0,2,0,1) \\ (4,6,8,2) & (12,24,12,6) & (4,6,0,2) \\ (4,2,4,1) & (8,6,4,2) & (4,2,0,1) \end{bmatrix} \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 0 \end{bmatrix}$$

$$= \left(\frac{54}{8}, \frac{98}{8}, \frac{68}{8}, \frac{27}{8}\right)$$

投影得到

$$\mathbf{S}\left(\frac{5}{2}, 1\right) = \left(2, \frac{98}{27}, \frac{68}{27}\right)$$

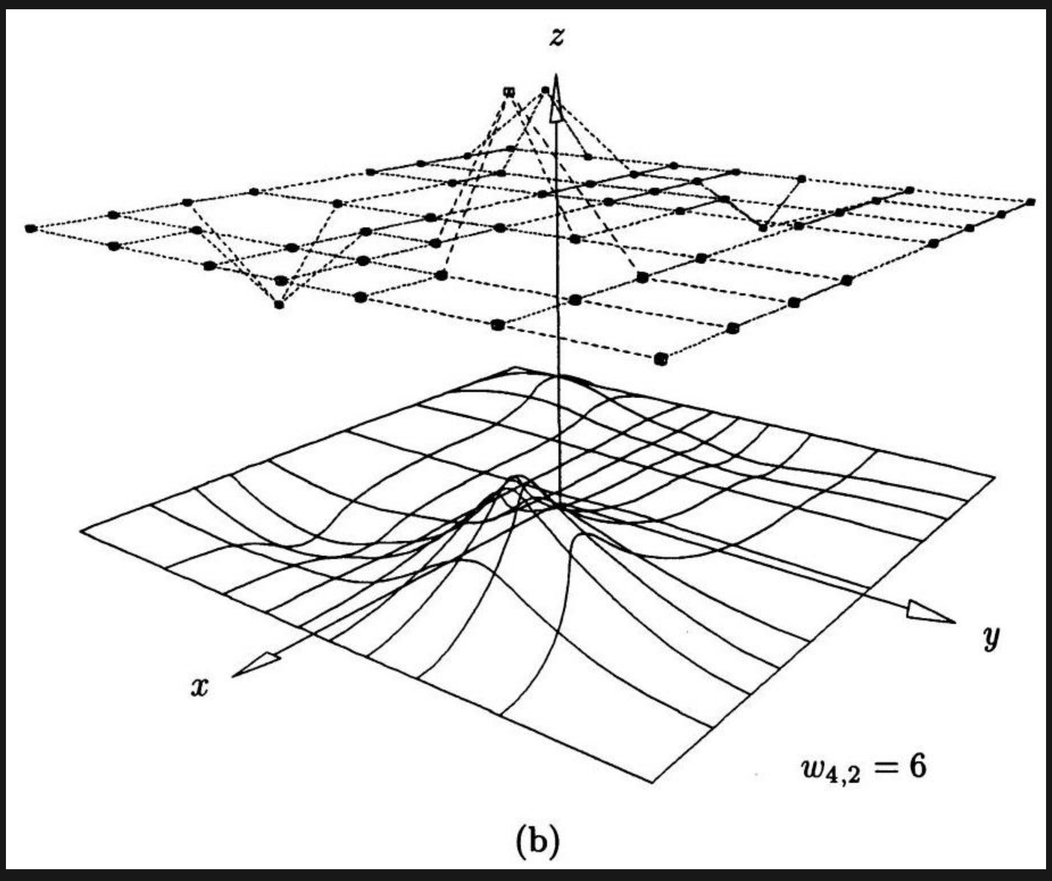


图 4.11。(续)

算法 A3.5 可以通过简单地允许数组 P 包含加权控制点 (使用 Pw)，累积四维表面点在 Sw 中，并在算法末尾插入行 $S = Sw/w$ 来计算有理 B 样条曲面上的一个点。

ALGORITHM A4.3

```
SurfacePoint(n, p, U, m, q, V, Pw, u, v, S)
    { /* Compute point on rational B-spline surface */
      /* Input: n, p, U, m, q, V, Pw, u, v */
      /* Output: S */
    uspan = FindSpan(n, p, u, U);
    BasisFuns(uspan, u, p, U, Nu);
    vspan = FindSpan(m, q, v, V);
    BasisFuns(vspan, v, q, V, Nv);
    for (l=0; l<=q; l++)
      {
      temp[l] = 0.0 ;
      for (k = 0; k <= p; k + +)
        temp[l] = temp[l] + Nu[k]*Pw[uspan-p+k][vspan-q+l];
      }
    Sw = 0.0 ;
    for (l=0; l<=q; l++)
      Sw = Sw + Nv[l]*temp[l];
    S = Sw/w
    }
```

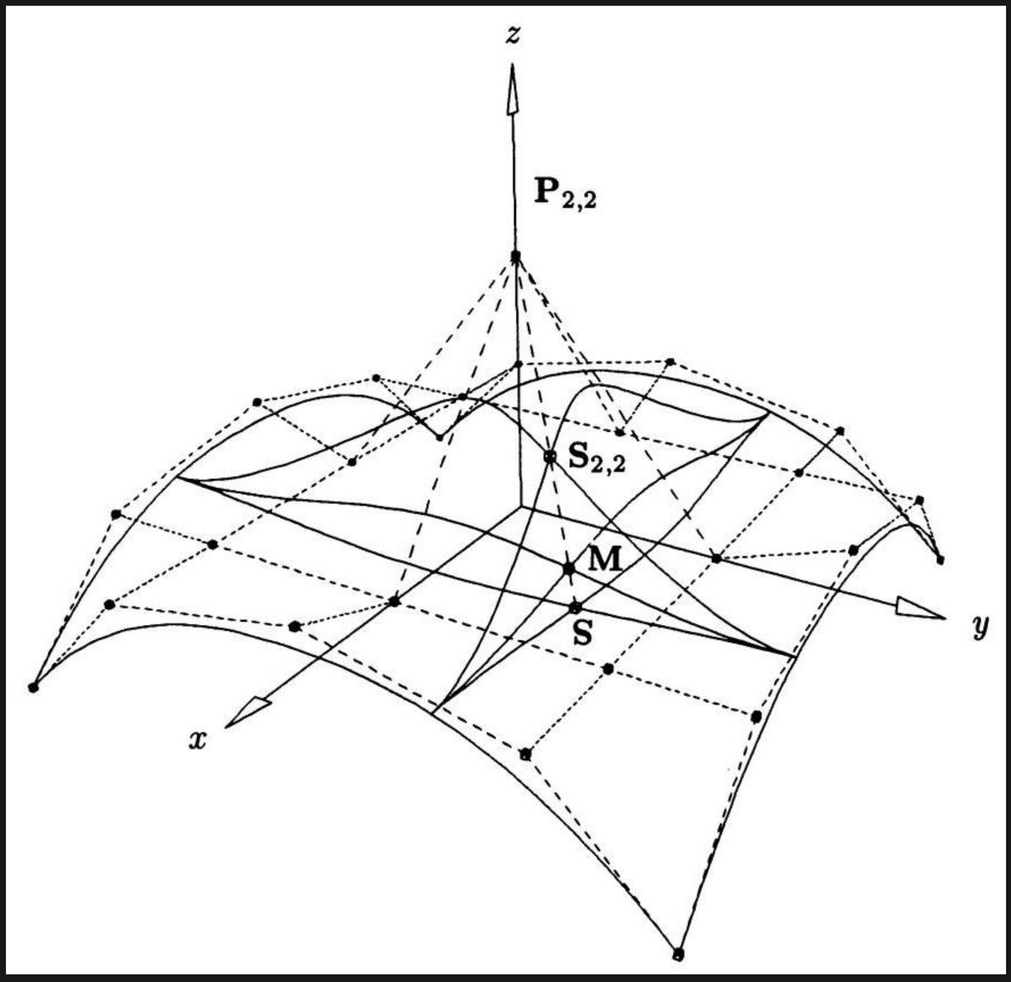


图 4.12。权重 $w_{2,2}$ 的修改。

通过应用方程 (3.14) 和 (3.15)，我们在 NURBS 表面上获得等参数曲线。首先，固定 $u = u_0$

$$\begin{aligned}\mathbf{C}^w_{u_0}(v) &= \mathbf{S}^w(u_0, v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u_0) N_{j,q}(v) \mathbf{P}^w_{i,j} \\ &= \sum_{j=0}^{m} N_{j,q}(v) \left(\sum_{i=0}^{n} N_{i,p}(u_0) \mathbf{P}^w_{i,j}\right) \\ &= \sum_{j=0}^{m} N_{j,q}(v) \mathbf{Q}^w_j(u_0)\end{aligned} \tag{4.16}$$

其中

$$\mathbf{Q}^w_j(u_0) = \sum_{i=0}^{n} N_{i,p}(u_0) \mathbf{P}^w_{i,j}$$

类似地

$$\mathbf{C}^w_{v_0}(u) = \sum_{i=0}^{n} N_{i,p}(u) \mathbf{Q}^w_i(v_0) \tag{4.17}$$

在哪里

$$\mathbf{Q}^w_i(v_0) = \sum_{j=0}^{m} N_{j,q}(v_0) \mathbf{P}^w_{i,j}$$

$\mathbf{C}^w_{u_0}(v)$ $(\mathbf{C}^w_{v_0}(u))$ 是节点向量 V (U) 上的 q th-(pth)-度 NURBS 曲线。点 $\mathbf{S}^w(u_0, v_0)$ 位于 $\mathbf{C}^w_{u_0}(v)$ 和 $\mathbf{C}^w_{v_0}(u)$ 的交点上。投影结果为

$$\mathbf{C}_{u_0}(v) = H\left\{\mathbf{C}^w_{u_0}(v)\right\} = H\left\{\mathbf{S}^w(u_0, v)\right\} = \mathbf{S}(u_0, v)$$

$$\mathbf{C}_{v_0}(u) = H\left\{\mathbf{C}^w_{v_0}(u)\right\} = H\left\{\mathbf{S}^w(u, v_0)\right\} = \mathbf{S}(u, v_0) \tag{4.18}$$

5 NURBS 曲面的导数

$\mathbf{S}^w(u, v)$ 的导数通过公式 (3.17)-(3.24) 计算。我们现在推导 $\mathbf{S}(u, v)$ 的导数公式，基于 $\mathbf{S}^w(u, v)$ 的导数。设

$$\mathbf{S}(u, v) = \frac{w(u, v)\mathbf{S}(u, v)}{w(u, v)} = \frac{\mathbf{A}(u, v)}{w(u, v)}$$

其中 $\mathbf{A}(u, v)$ 是 $\mathbf{S}(u, v)$ 的分子 (公式 [4.11])。那么

$$\mathbf{S}_\alpha(u, v) = \frac{\mathbf{A}_\alpha(u, v) - w_\alpha(u, v)\mathbf{S}(u, v)}{w(u, v)} \tag{4.19}$$

其中 α 表示 u 或 v 。

一般来说

$$\begin{aligned}\mathbf{A}^{(k,l)} &= \left[(w\mathbf{S})^k\right]^l = \left(\sum_{i=0}^{k}\binom{k}{i} w^{(i,0)}\mathbf{S}^{(k-i,0)}\right)^l \\ &= \sum_{i=0}^{k}\binom{k}{i}\sum_{j=0}^{l}\binom{l}{j} w^{(i,j)}\mathbf{S}^{(k-i,l-j)} \\ &= w^{(0,0)}\mathbf{S}^{(k,l)} + \sum_{i=1}^{k}\binom{k}{i} w^{(i,0)}\mathbf{S}^{(k-i,l)} + \sum_{j=1}^{l}\binom{l}{j} w^{(0,j)}\mathbf{S}^{(k,l-j)} \\ &\quad + \sum_{i=1}^{k}\binom{k}{i}\sum_{j=1}^{l}\binom{l}{j} w^{(i,j)}\mathbf{S}^{(k-i,l-j)}\end{aligned}$$

因此

$$\begin{aligned}\mathbf{S}^{(k,l)} = \frac{1}{w}\Bigg(&\mathbf{A}^{(k,l)} - \sum_{i=1}^{k}\binom{k}{i} w^{(i,0)}\mathbf{S}^{(k-i,l)} \\ &- \sum_{j=1}^{l}\binom{l}{j} w^{(0,j)}\mathbf{S}^{(k,l-j)} - \sum_{i=1}^{k}\binom{k}{i}\sum_{j=1}^{l}\binom{l}{j} w^{(i,j)}\mathbf{S}^{(k-i,l-j)}\Bigg)\end{aligned} \tag{4.20}$$

从公式 (4.20) 我们得到

$$\mathbf{S}_{uv} = \frac{\mathbf{A}_{uv} - w_{uv}\mathbf{S} - w_u\mathbf{S}_v - w_v\mathbf{S}_u}{w} \tag{4.21}$$

$$\mathbf{S}_{uu} = \frac{\mathbf{A}_{uu} - 2w_u\mathbf{S}_u - w_{uu}\mathbf{S}}{w} \tag{4.22}$$

$$\mathbf{S}_{vv} = \frac{\mathbf{A}_{vv} - 2w_v\mathbf{S}_v - w_{vv}\mathbf{S}}{w} \tag{4.23}$$

从公式 (3.24)、(4.19) 和 (4.20)

$$\mathbf{S}_u(0, 0) = \frac{p}{u_{p+1}}\frac{w_{1,0}}{w_{0,0}}(\mathbf{P}_{1,0} - \mathbf{P}_{0,0}) \tag{4.24}$$

$$\mathbf{S}_v(0,0) = \frac{q}{v_{q+1}} \frac{w_{0,1}}{w_{0,0}} (\mathbf{P}_{0,1} - \mathbf{P}_{0,0}) \tag{4.25}$$

$$\mathbf{S}_{uv}(0,0) = \frac{pq}{w_{0,0} u_{p+1} v_{q+1}} \left(w_{1,1}\mathbf{P}_{1,1} - \frac{w_{1,0}w_{0,1}}{w_{0,0}} (\mathbf{P}_{1,0} + \mathbf{P}_{0,1}) + \left(\frac{2w_{1,0}w_{0,1}}{w_{0,0}} - w_{1,1} \right) \mathbf{P}_{0,0} \right) \tag{4.26}$$

图 4.13 显示了 NURBS 曲面的一级和二级偏导数。一级偏导数缩小了 1/2，而二级偏导数缩小了 1/3。

现在假设 (u, v) 是固定的，并且所有导数 $\mathbf{A}^{(k,l)}, w^{(k,l)}$ 对于 $k, l \geq 0$ 和 $0 \leq k + l \leq d$ 已经计算并加载到数组 Aders 和 wders 中。算法 A4.4 计算点 $\mathbf{S}(u,v)$ 和导数 $\mathbf{S}^{(k,l)}(u,v), 0 \leq k + l \leq d$。Bin [] [] 包含预计算的二项式系数。

ALGORITHM A4.4

```
RatSurfaceDerivs (Aders, wders, d, SKL)
    { /* Compute S(u,v) derivatives */
        /* from Sw(u,v) derivatives */
        /* Input: Aders, wders, d */
        /* Output: SKL */
    for (k=0; k<=d; k++)
        for (l = 0; l <= d − k; l + +)
            {
            v = Aders[k][l];
            for (j=1; j<=1; j++)
                v = v - Bin[l][j]*wders[0][j]*SKL[k][l-j];
```

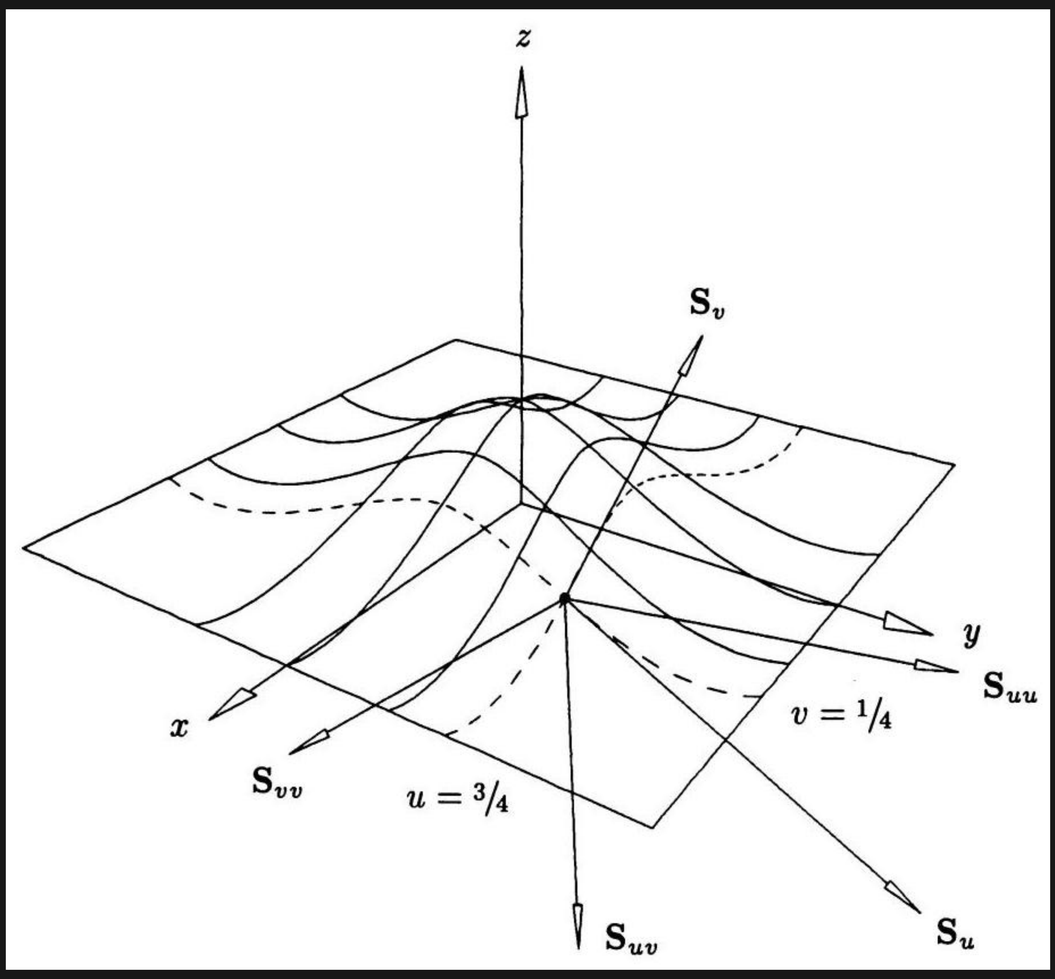



图 4.13. 在 $u = 3/4$ 和 $v = 1/4$ 处计算的双三次 NURBS 曲面的一级和二级偏导数。

```
    for (i = 1; i<=k; i++)
        {
        v = v - Bin[k][i]*wders[i][0]*SKL[k-i][l];
        v2 = 0.0 ;
        for (j=1; j<=1; j++)
          v2 = v2 + Bin[l][j]*wders[i][j]*SKL[k-i][l-j];
      v = v - Bin[k][i]*v2;
      }
    SKL[k][l] = v/wders[0][0];
    }
}
```

练习

4.1. 设 $U = \{0,0,0,1/3,2/3,1,1,1\}$ 和 $\{w_0, \ldots, w_4\} = \{1,4,1,1,1\}$。使用 Cox-deBoor 递归公式 (公式 [2.5]) 和公式 (4.2)，计算五个二次有理函数 $R_{i,2}(u), 0 \leq i \leq 4$。这些函数的图形如图 4.5a 所示。假设 $\{\mathbf{P}_0, \ldots, \mathbf{P}_4\} = \{(0,0), (1,1), (3,2), (4,1), (5,-1)\}$ 是 xy 平面中的控制点。计算有理坐标函数 $x(u)$ 和 $y(u)$，表示在区间 $u \in [1/3, 2/3)$ 内的 $\mathbf{C}(u)$。

4.2. 参考示例 Ex4.2 计算二次有理 Bézier 圆弧; 计算 $\mathbf{C}''(1)$。

4.3. 设 $\mathbf{C}^w(u) = \sum_{i=0}^{1} N_{i,1}(u)\mathbf{P}_i^w$ 为 xy 平面中的一条线段，其中 $\mathbf{P}_0 = (0,1), \mathbf{P}_1 = (2,0), w_0 = 1, w_1 = 3$，且 $U = \{0,0,1,1\}$。推导出表示该线段的 x 和 y 坐标的有理函数，即 $x(u)$ 和 $y(u)$，其中 $\mathbf{C}(u) = (x(u), y(u))$。使用公式 (4.9) 计算 $\mathbf{C}'(0)$，并使用公式 (4.8) 和 (3.9) 计算 $\mathbf{C}''(0)$。然后设定 $w_1 = 1$ 并重新计算 $x(u), y(u), \mathbf{C}'(0)$ 和 $\mathbf{C}''(0)$。

4.4. 设 $\mathbf{S}^w(u,v) = \sum_{i=0}^{1}\sum_{j=0}^{1} N_{i,1}(u) N_{j,1}(v) \mathbf{P}_{i,j}^w$，其中 $\{\mathbf{P}_{0,0}, \mathbf{P}_{1,0}, \mathbf{P}_{0,1}, \mathbf{P}_{1,1}\} = \{(0,0,1), (0,1,3), (2,1,1), (2,0,3)\}$, $\{w_{0,0}, w_{1,0}, w_{0,1}, w_{1,1}\} = \{2,1,1,1\}$，且 $U = V = \{0,0,1,1\}$。推导出四个有理基函数 $R_{i,j}(u,v), 0 \leq i,j \leq 1$，以及曲面 $\mathbf{S}(u,v)$ 的有理坐标函数 $x(u,v), y(u,v)$ 和 $z(u,v)$。

4.5. 从练习 4.4 中的 $\mathbf{S}^w(u,v)$ 推导出两个有理等参数曲线 $\mathbf{C}_{u_0}^w(v)$ 和 $\mathbf{C}_{v_0}^w(u)$，对于 $u_0 = 1/3$ 和 $v_0 = 1/2$。然后分别在 $v = 1/2$ 和 $u = 1/3$ 处评估曲线 $\mathbf{C}_{u_0}(v)$ 和 $\mathbf{C}_{v_0}(u)$。通过将 $(u,v) = (1/3, 1/2)$ 代入在练习 4.4 中获得的有理坐标函数来检查结果。

4.6. 设 $\mathbf{S}^w(u,v) = \sum_{i=0}^{1}\sum_{j=0}^{1} N_{i,1}(u) N_{j,1}(v) \mathbf{P}_{i,j}^w$ 为练习 4.4 中给出的曲面。由于 $N_{0,1}(1/2) = N_{1,1}(1/2) = 1/2$，因此得出

$$\mathbf{S}^w\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{1}{4}(\mathbf{P}_{0,0}^w + \mathbf{P}_{0,1}^w + \mathbf{P}_{1,0}^w + \mathbf{P}_{1,1}^w) = \left(1, \frac{1}{2}, \frac{9}{4}, \frac{5}{4}\right)$$

计算 $\mathbf{S}_u(1/2,1/2), \mathbf{S}_v(1/2,1/2), \mathbf{S}_{uv}(1/2,1/2), \mathbf{S}_{uu}(1/2,1/2)$。

第五章　基本几何算法

1　引言

在本章中，我们介绍五种在实现 B 样条曲线和曲面中至关重要的工具；这些工具包括节点插入、节点细化、节点删除、度数提升和度数降低。我们为每个主题专门设立一个部分，每个部分的布局大致如下:

- 问题陈述 (对于曲线)；
- 应用列表；
- 问题和解决方法的澄清 (曲线)；
- 可以找到更严格的推导和证明的参考文献列表；
- 解决方案公式 (曲线)；
- 具体示例 (曲线)；
- 计算机算法；
- 应用示例；
- 表面的解决方案，以及表面算法的概述。

2　节点插入

设 $\mathbf{C}^w(u)=\sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w$ 为定义在 $U=\{u_0,\ldots,u_m\}$ 上的 NURBS 曲线。设 $\bar{u}\in[u_k,u_{k+1})$ ，并将 $\bar{u}$ 插入 U 以形成新的结点向量 $\bar{U}=\{\bar{u}_0=u_0,\ldots,\bar{u}_k=u_k,\bar{u}_{k+1}=\bar{u},\bar{u}_{k+2}=u_{k+1},\ldots,\bar{u}_{n+1}=u_m\}$ 。如果 $\mathcal{V}_U$ 和 $\mathcal{V}_{\bar{U}}$ 分别表示定义在 U 和 $\bar{U}$ 上的曲线的向量空间，则显然 $\mathcal{V}_U\subset\mathcal{V}_{\bar{U}}\,(\text{and}\dim(\mathcal{V}_U)=\dim(\mathcal{V}_{\bar{U}})+1)$ ；因此 $\mathbf{C}^w(u)$ 在 $\bar{U}$ 上的表示形式为

$$\mathbf{C}^w(u)=\sum_{i=0}^{n+1}\bar{N}_{i,p}(u)\,\mathbf{Q}_i^w \tag{5.1}$$

其中 $\left\{\bar{N}_{i,p}(u)\right\}$ 是 p 阶的基函数，定义在 $\bar{U}$ 上。术语 *knot* 插入通常指的是确定 Eq. (5.1) 中的 $\{\mathbf{Q}_i^w\}$ 的过程。需要注意的是，节点插入实际上只是向量空间基的变化；曲线在几何上或参数上并没有改变。
尽管并不明显，节点插入是所有 B-spline 算法中最重要的之一。它的一些用途包括:

- 在曲线和曲面上评估点和导数；
- 对曲线和曲面进行细分；
- 添加控制点以增加形状控制的灵活性 (交互设计)。

现在可以通过建立并求解线性方程组来获得 Eq. (5.1) 中的 $\{\mathbf{Q}_i^w\}$ 。如果我们设定

$$\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_i^w=\sum_{i=0}^{n+1}\bar{N}_{i,p}(u)\,\mathbf{Q}_i^w \tag{5.2}$$

那么通过将 u 的适当值代入 Eq. (5.2)，我们得到一个非奇异的带状 $n+2$ 线性方程组，未知数为 $n+2$ ，即 $\mathbf{Q}_i^w$ 。然而，还有一种更有效的解决方案。性质 P2.2 和 $\bar{u}\in[u_k,u_{k+1})$ 意味着

$$\sum_{i=k-p}^{k}N_{i,p}(u)\,\mathbf{P}_i^w=\sum_{i=k-p}^{k+1}\bar{N}_{i,p}(u)\,\mathbf{Q}_i^w \tag{5.3}$$

对于所有 $u\in[u_k,u_{k+1})$ ，以及

$$N_{i,p}(u)=\bar{N}_{i,p}(u)\;\; i=0,\ldots,k-p-1$$

$$N_{i,p}(u)=\bar{N}_{i+1,p}(u)\;\; i=k+1,\ldots,n \tag{5.4}$$

方程 (5.3) 和 (5.4)，结合基函数的线性独立性 (第 2.4 节)，意味着

$$\mathbf{P}_i^w=\mathbf{Q}_i^w\;\; i=0,\ldots,k-p-1$$

$$\mathbf{P}_i^w=\mathbf{Q}_{i+1}^w\;\; i=k+1,\ldots,n \tag{5.5}$$

现在考虑 $N_{i,p}(u)$ 对于 $i=k-p,\ldots,k$ 。当 $i=k-p,\ldots,k+1$ 时，它们可以用 $\bar{N}_{i,p}(u)$ 表示，通过

$$N_{i,p}(u)=\frac{\bar{u}-\bar{u}_i}{\bar{u}_{i+p+1}-\bar{u}_i}\bar{N}_{i,p}(u)+\frac{\bar{u}_{i+p+2}-\bar{u}}{\bar{u}_{i+p+2}-\bar{u}_{i+1}}\bar{N}_{i+1,p}(u) \tag{5.6}$$

方程 (5.6) 通过对 p 的归纳法证明 (并使用 Eq. [2.5])，但我们在这里省略证明，因为它相当繁琐。使用分割差的证明见于 [DeBo78; Boeh80; Lee83]。
为了简洁，我们现在将 $\bar{N}_i$ 写作 $\bar{N}_{i,p}(u)$ 。将方程 (5.6) 代入方程 (5.3) 得到

$$\begin{aligned}
&\left(\frac{\bar{u}-\bar{u}_{k-p}}{\bar{u}_{k+1}-\bar{u}_{k-p}}\bar{N}_{k-p}+\frac{\bar{u}_{k+2}-\bar{u}}{\bar{u}_{k+2}-\bar{u}_{k-p+1}}\bar{N}_{k-p+1}^w\right)\mathbf{P}_{k-p}^w\\
&+\left(\frac{\bar{u}-\bar{u}_{k-p+1}}{\bar{u}_{k+2}-\bar{u}_{k-p+1}}\bar{N}_{k-p+1}+\frac{\bar{u}_{k+3}-\bar{u}}{\bar{u}_{k+3}-\bar{u}_{k-p+2}}\bar{N}_{k-p+2}^w\right)\mathbf{P}_{k-p+1}^w\\
&\vdots\\
&+\left(\frac{\bar{u}-\bar{u}_k}{\bar{u}_{k+p+1}-\bar{u}_k}\bar{N}_k+\frac{\bar{u}_{k+p+2}-\bar{u}}{\bar{u}_{k+p+2}-\bar{u}_{k+1}}\bar{N}_{k+1}\right)\mathbf{P}_k^w\\
&=\bar{N}_{k-p}\mathbf{Q}_{k-p}^w+\cdots+\bar{N}_{k+1}\mathbf{Q}_{k+1}^w
\end{aligned}$$

通过等同系数并使用结点向量 U 替代 $\bar{U}$ ，我们得到

$$\begin{aligned}
&0=\bar{N}_{k-p}\left(\mathbf{Q}_{k-p}^w-\mathbf{P}_{k-p}^w\right)\\
&+\bar{N}_{k-p+1}\left(\mathbf{Q}_{k-p+1}^w-\frac{\bar{u}-u_{k-p+1}}{u_{k+1}-u_{k-p+1}}\mathbf{P}_{k-p+1}^w-\frac{u_{k+1}-\bar{u}}{u_{k+1}-u_{k-p+1}}\mathbf{P}_{k-p}^w\right)\\
&+\bar{N}_k\left(\mathbf{Q}_k^w-\frac{\bar{u}-u_k}{u_{k+p}-u_k}\mathbf{P}_k^w-\frac{u_{k+p}-\bar{u}}{u_{k+p}-u_k}\mathbf{P}_{k-1}^w\right)+\bar{N}_{k+1}\left(\mathbf{Q}_{k+1}^w-\mathbf{P}_k^w\right)
\end{aligned} \tag{5.7}$$

对于 $i=k-p+1,\ldots,k$ ，我们设定

$$\alpha_i=\frac{\bar{u}-u_i}{u_{i+p}-u_i} \tag{5.8}$$

并注意到

$$1-\alpha_i=\frac{u_{i+p}-\bar{u}}{u_{i+p}-u_i} \tag{5.9}$$

利用基函数的线性独立性，并将方程 (5.8) 和 (5.9) 代入方程 (5.7)，得到

$$\mathbf{Q}_{k-p}^w=\mathbf{P}_{k-p}^w$$

$$\mathbf{Q}_i^w=\alpha_i\mathbf{P}_i^w+(1-\alpha_i)\,\mathbf{P}_{i-1}^w\;\; k-p+1\le i\le k$$

$$\mathbf{Q}_{k+1}^w=\mathbf{P}_k^w \tag{5.10}$$

最后，通过结合方程 (5.5) 和 (5.10)，我们得到了计算方程 (5.1) 中所有新控制点 $\mathbf{Q}_i^w$ 的公式，即

$$\mathbf{Q}_i^w=\alpha_i\mathbf{P}_i^w+(1-\alpha_i)\,\mathbf{P}_{i-1}^w \tag{5.11}$$

其中

$$\alpha_i=\begin{cases}1 & i\le k-p\\ \frac{\bar{u}-u_i}{u_{i+p}-u_i} & k-p+1\le i\le k\\ 0 & i\ge k+1\end{cases}$$

方程 (5.11) 表示仅需计算 p 个新控制点。为了简洁，我们在示例 Ex5.1-Ex5.4 中使用 $\mathbf{P}$ 代替 $\mathbf{P}^w$ 。

示例

Ex5.1 设 $p=3$ 和 $U=\{0,0,0,0,1,2,3,4,5,5,5,5\}$ 。控制点为 $\mathbf{P}_0,\ldots,\mathbf{P}_7$ 。我们插入 $\bar{u}=5/2$ 。然后 $\bar{u}\in[u_5,u_6)$ 和 $k=5$ 。因此，$\mathbf{Q}_0=\mathbf{P}_0,\ldots,\mathbf{Q}_2=\mathbf{P}_2$ 和 $\mathbf{Q}_6=\mathbf{P}_5,\ldots,\mathbf{Q}_8=\mathbf{P}_7$ 。应用方程 (5.11)，我们发现

$$\alpha_3=\frac{\frac{5}{2}-0}{3-0}=\frac{5}{6}\Rightarrow\mathbf{Q}_3=\frac{5}{6}\mathbf{P}_3+\frac{1}{6}\mathbf{P}_2$$

$$\alpha_4=\frac{\frac{5}{2}-1}{4-1}=\frac{1}{2}\Rightarrow\mathbf{Q}_4=\frac{1}{2}\mathbf{P}_4+\frac{1}{2}\mathbf{P}_3$$

$$\alpha_5=\frac{\frac{5}{2}-2}{5-2}=\frac{1}{6}\Rightarrow\mathbf{Q}_5=\frac{1}{6}\mathbf{P}_5+\frac{5}{6}\mathbf{P}_4$$

图 5.1a 显示了插入前后的控制多边形，图 5.1b 显示了插入前后的基函数。图 5.1a 的底部显示了用于细分多边形边的比例。

Ex5.2 使用与 Ex5.1 中相同的曲线，即 $p=3,U=\{0,0,0,0,1,2,3,4,5\ 、\ 5,5,5\}$ 和 $\mathbf{P}_0,\ldots,\mathbf{P}_7$ 。这次我们插入一个已经在节点向量中的节点，即 $\bar{u}=2$ 。然后 $\bar{u}\in[u_5,u_6),k=5$ ，我们再次计算 $\mathbf{Q}_3,\mathbf{Q}_4$ 和 $\mathbf{Q}_5$ 。

$$\alpha_3=\frac{2-0}{3-0}=\frac{2}{3}\Rightarrow\mathbf{Q}_3=\frac{2}{3}\mathbf{P}_3+\frac{1}{3}\mathbf{P}_2$$

$$\alpha_4=\frac{2-1}{4-1}=\frac{1}{3}\Rightarrow\mathbf{Q}_4=\frac{1}{3}\mathbf{P}_4+\frac{2}{3}\mathbf{P}_3$$

$$\alpha_5=\frac{2-2}{5-2}=0\Rightarrow\mathbf{Q}_5=\mathbf{P}_4$$

控制多边形和基函数在插入节点之前和之后如图 5.2a 和 5.2b 所示。从公式 (5.6) 可以验证 $N_5=\bar{N}_6$ 。如我们之前所做，观察通过映射图 5.2a 底部所示的适当比例获得的腿的细分。

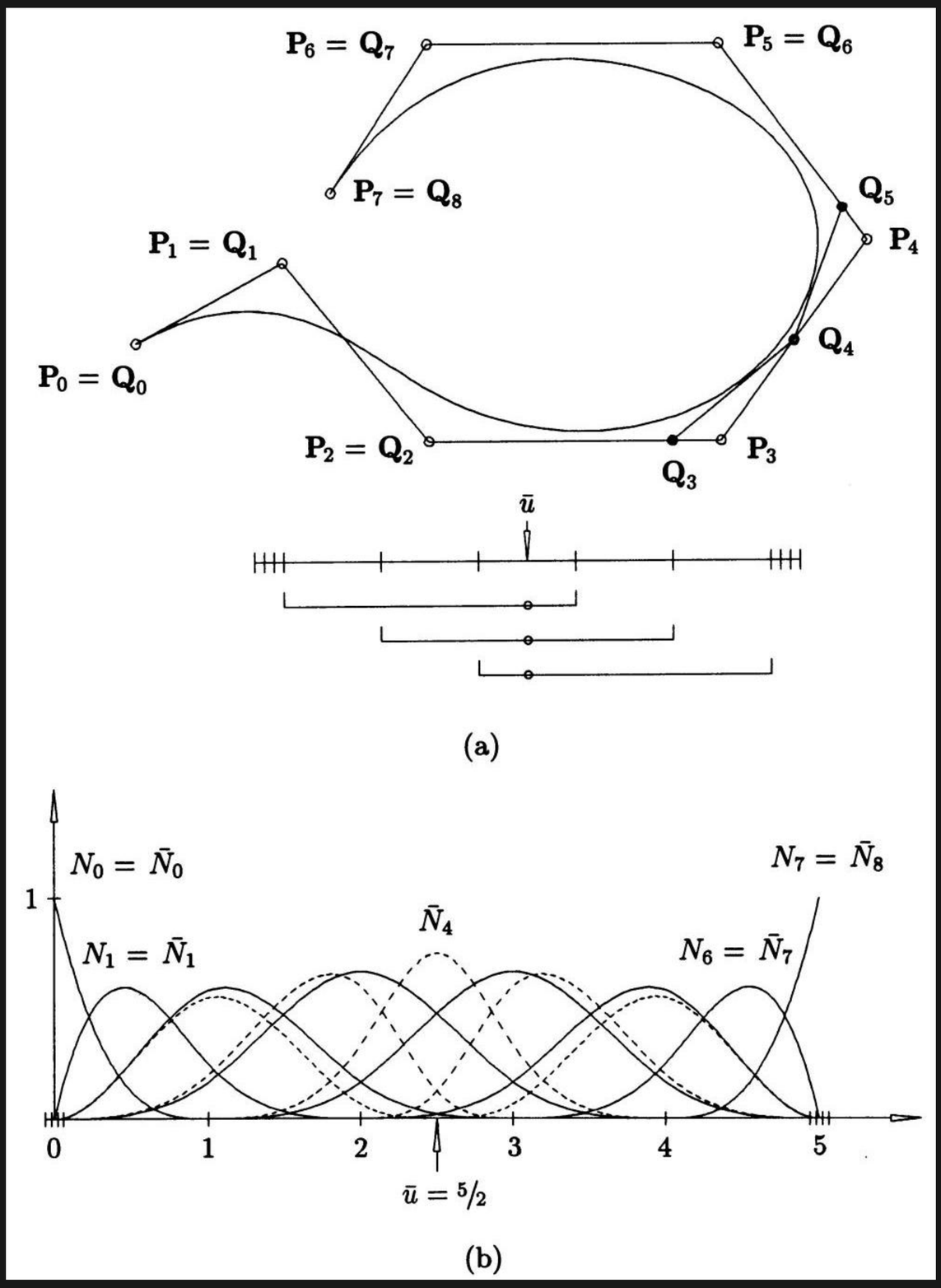


图 5.1. 将节点插入到三次曲线中。(a) 在节点向量 $\{0,0,0,0,1,2,3,4,5,5,5,5\}$ 中插入 $u=5/2$ 后的控制多边形；(b) 插入节点前后的原始 (实线) 和新的 (虚线) 基函数。

显然，节点插入类似于 Bézier 曲线的 deCasteljau 算法，实际上对于没有内部节点的曲线，即 Bézier 曲线，它们是等价的。然而，查看示例 Ex5.1 和图 5.1a 可以看出，控制多边形的每个 p 腿上的线性插值并不相同。实际上，在该示例中，$\alpha_3=5/6,\alpha_4=1/2$ 和 $\alpha_5=1/6$ 。基于之前的

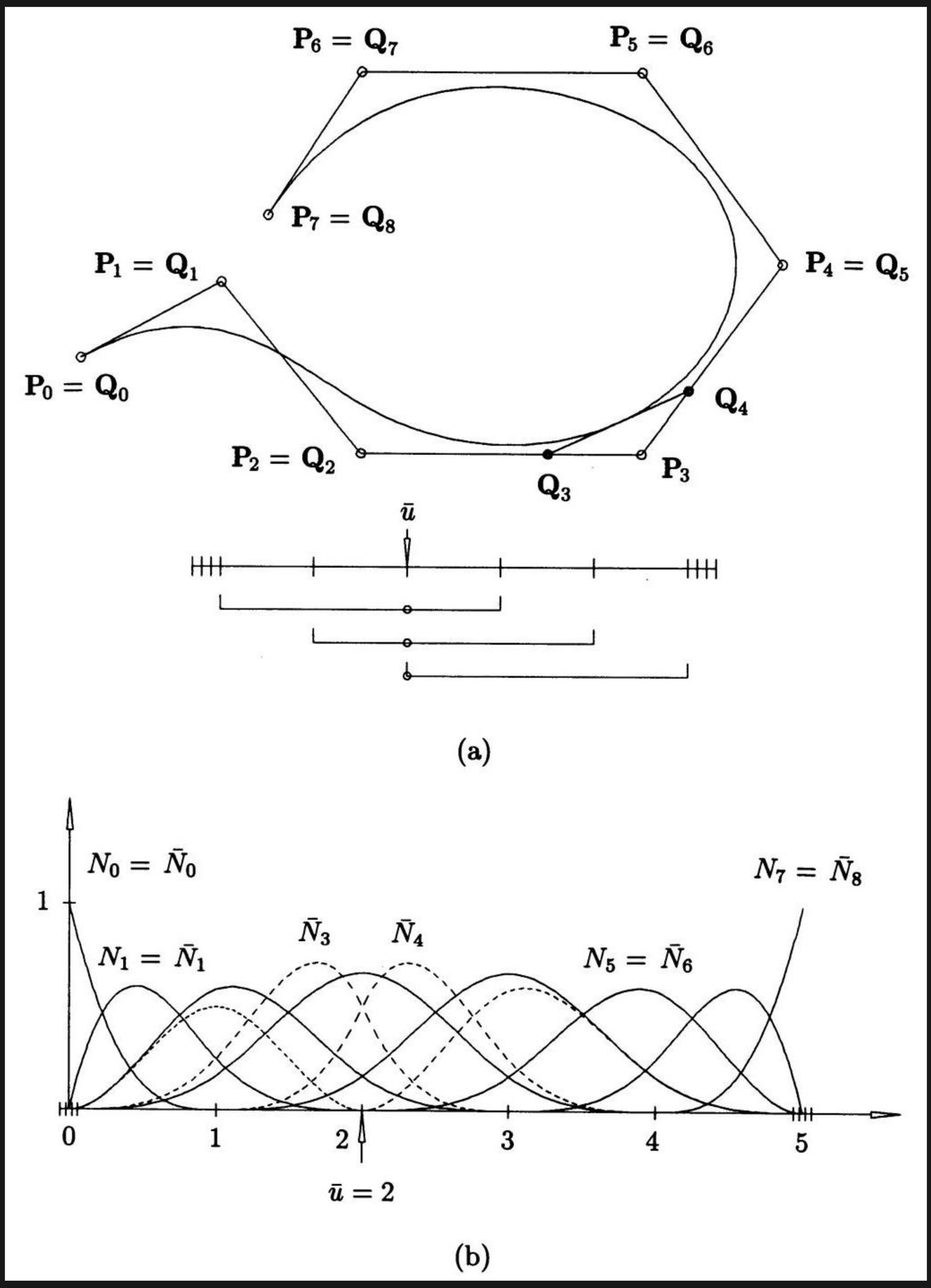


图 5.2. 将现有节点插入到三次曲线中。(a) 在 $\{0,0,0,0,1,2,3,4,5,5,5,5\}$ 中插入 $u=2$ 后的控制多边形；(b) 插入节点前后的原始 (实线) 和新的 (虚线) 基函数。

示例，我们得出一般结果: 对于 $i=1,\ldots,n$ ，令 L_i 表示控制多边形的 i 腿。

$$L_i = \mathbf{P}_{i-1}^w \mathbf{P}_i^w \text{ and } d_i = \text{ length } (L_i) = \left|\mathbf{P}_i^w - \mathbf{P}_{i-1}^w\right|$$

我们强调 d_i 是在四维 (均匀) 空间中测量的，即

$$d_i = \sqrt{(w_i x_i)^2 + (w_i y_i)^2 + (w_i z_i)^2 + (w_i)^2}$$

对于每个 $\operatorname{leg} L_i$ ，我们关联结点 $u_{i+j}, j=0,\ldots,p$ 。参见图 5.3a 和 5.3 b 以及图 5.1a 和 5.2a。对于固定的 i ，设 $\lambda_i^j, j=0,\ldots,p-1$ ，表示相对于 $[u_i, u_{i+p})$ 的结点跨度 $[u_{i+j}, u_{i+j+1})$ 的长度

$$\lambda_i^j = \frac{u_{i+j+1} - u_{i+j}}{u_{i+p} - u_i} \; j=0,\ldots,p-1 \tag{5.12}$$

然后 L_i 被划分为段 L_i^j ，其长度为

$$d_i^j = \lambda_i^j d_i \tag{5.13}$$

分别。注意到 λ_i^j ，因此 d_i^j 可以为零。例如，示例 Ex5.1 中的 $d_1^0 = d_1^1 = 0$ 和 $d_1^2 = d_1$ ；因此，在第一条腿上只有一个段。

现在图 5.3a 和 5.3b 以及公式 (5.13) 显示了多边形腿是如何细分的，图 5.1a 和 5.2a 显示了在结点插入过程中多边形角落是如何被切割的。特别地，如果 $\bar{u} \in [u_k, u_{k+1})$ 是重数为 s 的结点 $0 \le s \le p-1$ ，则带有控制点 $\mathbf{P}_{k-p+1}, \ldots, \mathbf{P}_{k-s-1}$ 的角落被切割，并在指示的段上生成以下 $p-s$ 个新的控制点

$$\mathbf{Q}_{k-j}^w \in L_{k-j}^j \; j=p-1,\ldots,s \tag{5.14}$$

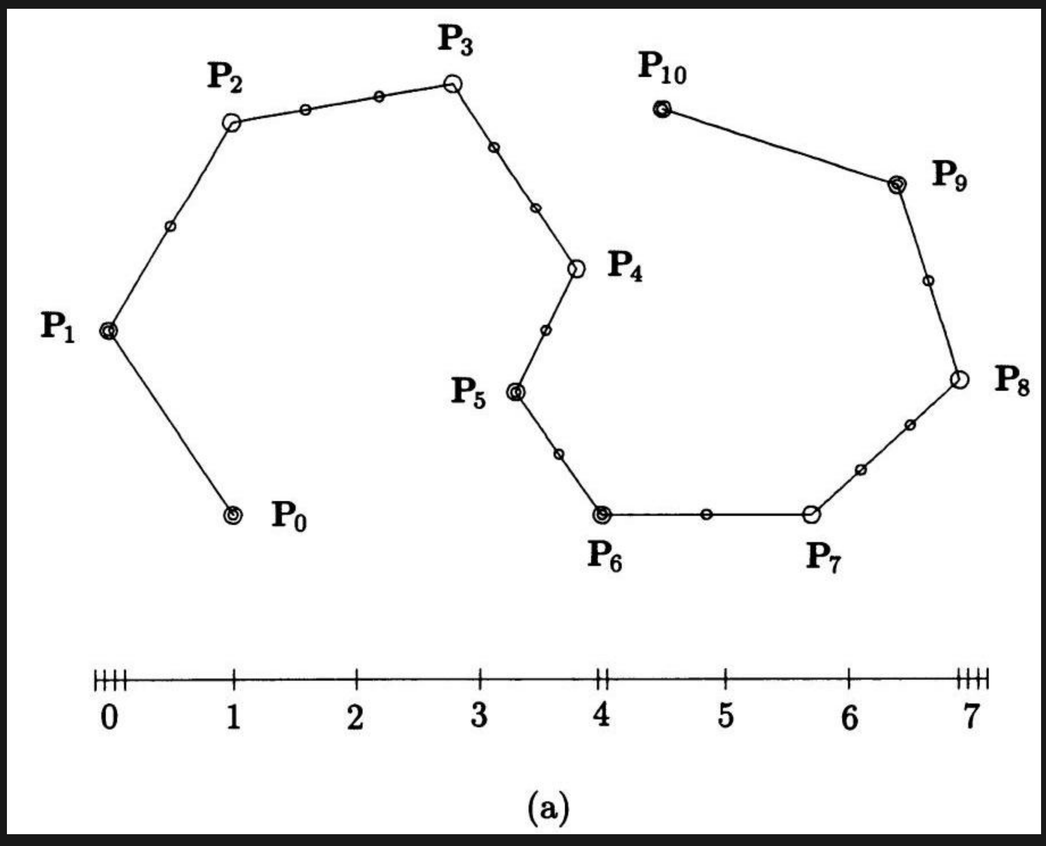


图 5.3. 控制多边形及其通过结点向量的划分。(a) 通过结点向量 $\{0,0,0,0,1,2,3,4,4,5,6,7,7,7,7\}$ 划分整个多边形；(b) 通过结点跨度 $[u_3,u_6]$ 划分多边形边 $\mathbf{P}_2\mathbf{P}_3$ 。

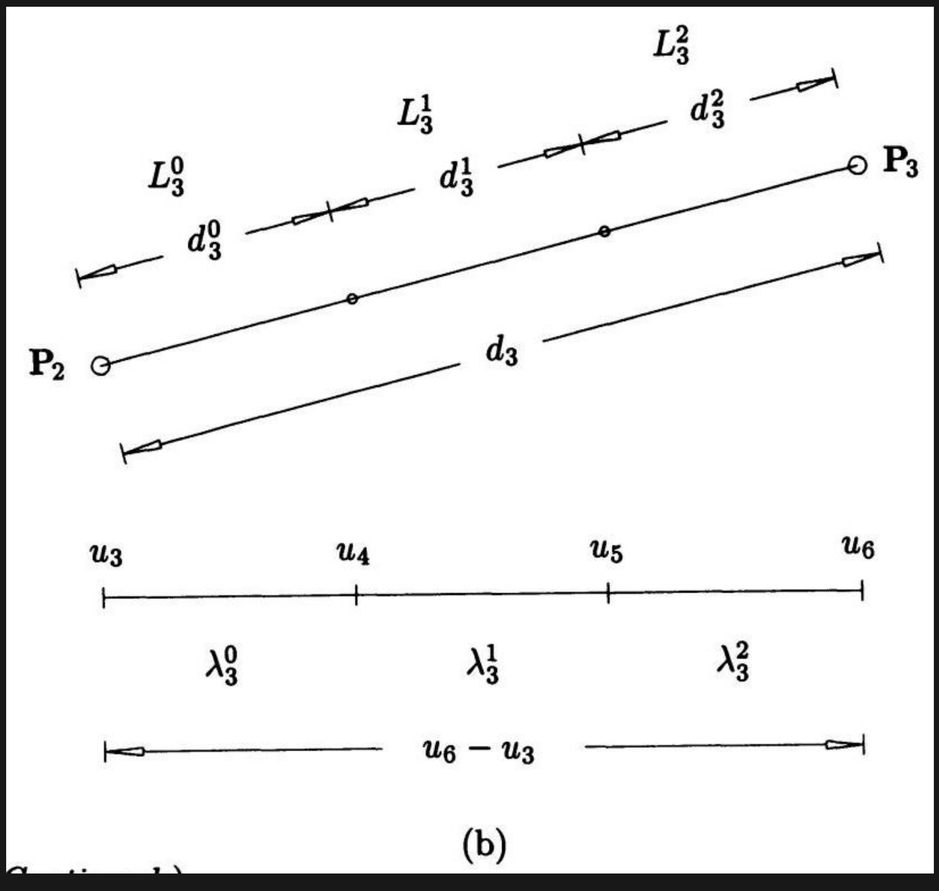


图 5.3.(续)

示例

Ex5.3 让我们检查示例 Ex5.1 中的公式 (5.12)-(5.14)。我们有 $k=5$ 和 $s=0$ 。根据公式 (5.14)

$$\mathbf{Q}_3 \in L_5^2\, \mathbf{Q}_4 \in L_4^1\, \mathbf{Q}_5 \in L_3^0$$

此外

$$\lambda_3^2=\frac{u_6-u_5}{u_6-u_3}=\frac{3-2}{3-0}=\frac{1}{3}$$

$$\lambda_4^1=\frac{u_6-u_5}{u_7-u_4}=\frac{3-2}{4-1}=\frac{1}{3}$$

$$\lambda_5^0=\frac{u_6-u_5}{u_8-u_5}=\frac{3-2}{5-2}=\frac{1}{3}$$

(这并不令人惊讶，因为节点是均匀分布的)。回想一下 $\alpha_3=5/6,\alpha_4=1/2$ 和 $\alpha_5=1/6$ ，我们看到 $\mathbf{Q}_3,\mathbf{Q}_4$ 和 $\mathbf{Q}_5$ 位于它们段的中点 L_3^2,L_4^1 和 L_5^0 。这源于 $\bar{u}=5/2$ 位于 $u_5=2$ 和 $u_6=3$ 之间的中点。

Ex5.4 例如 Ex5.2，$k=5$ 和 $s=1$ 。根据公式 (5.14)

$$\mathbf{Q}_3 \in L_3^2\, \mathbf{Q}_4 \in L_4^1$$

我们仍然有 $\lambda_3^2=\lambda_4^1=1/3$ 。现在 $\alpha_3=2/3$ 和 $\alpha_4=1/3$ ，这意味着 $\mathbf{Q}_3$ 和 $\mathbf{Q}_4$ 分别位于它们段的起始点 L_3^2 和 L_4^1 。这源于 $\bar{u}=2$ 位于节点跨度的起始位置 $[u_5,u_6)$ 。

图 5.4 显示了与图 5.3a 相同控制多边形的划分，但使用了非均匀节点向量 $U=\{0,0,0,0,1,1.5,2.3,3.9,4.3,5,6.5$ 和 $7,7,7,7\}$ 。

正如我们稍后将看到的，通常需要多次插入一个节点。方程 (5.11) 可以推广以处理这种情况。假设 $\bar{u}\in[u_k,u_{k+1})$ 最初具有重数 s ，并假设它将被插入 r 次，其中 $r+s\le p$ (通常没有实际意义使内部节点重数大于 p)。将第 i 个新控制点在第 r 次插入步骤中表示为 $\mathbf{Q}_{i,r}^w$ (与 $\mathbf{Q}_{i,0}^w=\mathbf{P}_i^w$))。那么 $\mathbf{Q}_{i,r}^w$ 是

$$\mathbf{Q}_{i,r}^w=\alpha_{i,r}\mathbf{Q}_{i,r-1}^w+(1-\alpha_{i,r})\,\mathbf{Q}_{i-1,r-1}^w \tag{5.15}$$

其中

$$\alpha_{i,r}=\begin{cases}1 & i\le k-p+r-1\\ \frac{\bar{u}-u_i}{u_{i+p-r+1}-u_i} & k-p+r\le i\le k-s\\ 0 & i\ge k-s+1\end{cases}$$

如果 $s=0$ 和 $r=p$ ，则公式 (5.15) 生成控制点的三角形表 (见表 5.1，见图 5.5 与 $p=3$)。外部控制点

$$\mathbf{Q}^w_{k-p+1,1}, \mathbf{Q}^w_{k-p+2,2}, \ldots, \mathbf{Q}^w_{k,p}, \ldots, \mathbf{Q}^w_{k,2}, \mathbf{Q}^w_{k,1}$$

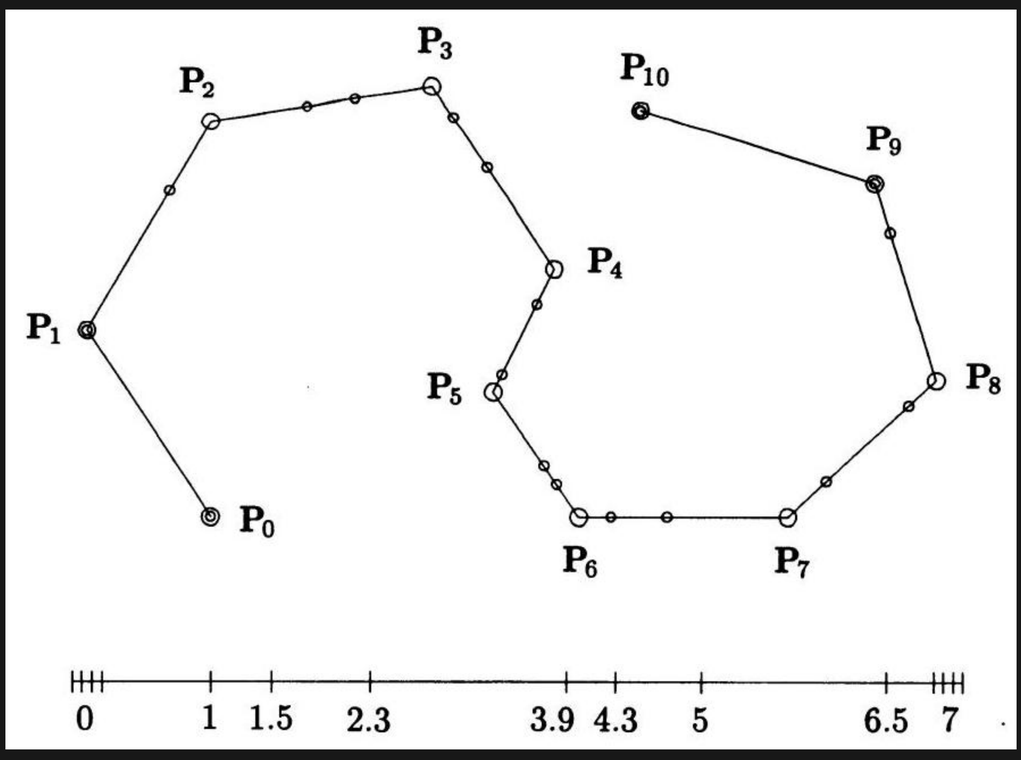


图 5.4. 由节点向量 $\{0, 0, 0, 0, 1, 1.5, 2.3, 3.9, 4.3, 5, 6.5, 7, 7, 7, 7\}$ 定义的图 5.3a 中控制多边形的非均匀划分。

保留内层的控制点，外层的被丢弃。一般来说，如果 $s > 0$ 和/或 $r + s < p$ ，表格的填充程度会低于满格。但在任何情况下，最终控制点是通过顺时针遍历生成的表格获得的，从第一列的顶部开始。最后一列的新控制点数量为 $p - (s + r) + 1$ 。在所有其他列中，保留两个新点 (顶部和底部)；因此，总的新 (保留) 控制点数量为

$$p - s + r - 1 \tag{5.16}$$

这些新控制点替代

$$p - s - 1 \tag{5.17}$$

旧控制点，从索引开始

$$k - p + 1 \tag{5.18}$$

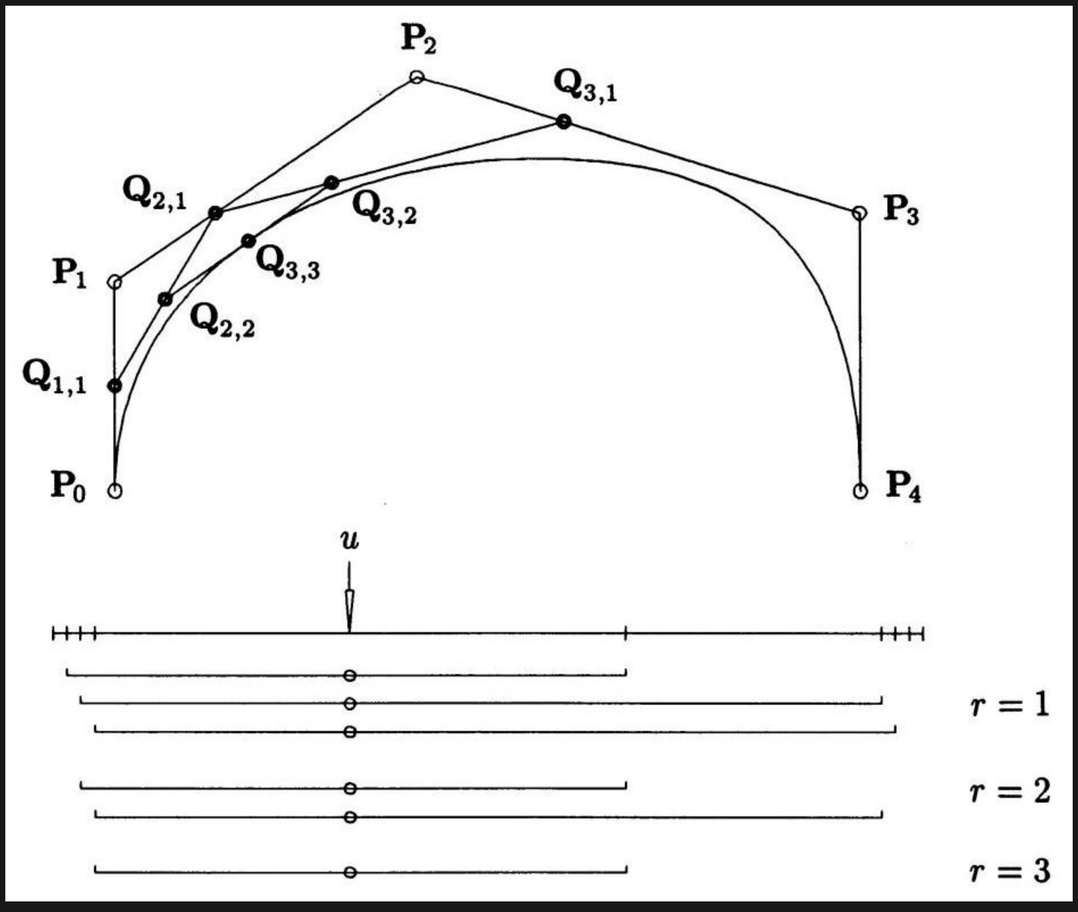


图 5.5. 三次将节点插入到三次曲线中。该曲线在 $\{0, 0, 0, 0, 2, 3, 3, 3, 3\}$ 上定义。

算法 A5.1 实现了公式 (5.15)。它计算与将 $\bar{u}$ 插入 $[u_k, u_{k+1})$ r 次相对应的新曲线，假设 $r + s \leq p$ 。Rw[] 是一个长度为 $p + 1$ 的局部数组。UP[] 和 UQ[] 分别是插入节点前后的节点向量。

算法 A5.1

```
CurveKnotIns(np, p, UP, Pw, u, k, s, r, nq, UQ, Qw)
    { /* Compute new curve from knot insertion */
            /* Input: np, p, UP, Pw, u, k, s, r */
            /* Output: nq, UQ, Qw */
    mp = np+p+1;
    nq = np + r ;
            /* Load new knot vector */
    for(i=0; i<=k; i++) UQ[i] = UP[i];
    for(i=1; i<=r; i++) UQ[k+i] = u;
    for(i=k+1; i<=mp; i++) UQ[i+r] = UP[i];
            /* Save unaltered control points * /
    for(i=0; i<=k-p; i++) Qw[i] = Pw[i];
    for(i=k; i<=np; i++) Qw[i+r] = Pw[i];
    for(i=0; i<=p-s; i++) Rw[i] = Pw[k-p+i];
    for (j = 1; j <= r; j + +) /* Insert the knot r times * /
            {
            L = k − p + j ;
            for(i=0; i<=p-j-s; i++)
```

```
            {
            alpha = (u-UP[L+i])/(UP[i+k+1]-UP[L+i]);
            Rw[i] = alpha*Rw[i+1] + (1.0-alpha)*Rw[i];
            }
        Qw[L] = Rw[0];
        Qw[k + r − j] = Rw[p − j] ;
      }
    for(i=L+1; i<k; i++) /* Load remaining control points */
        Qw[i] = Rw[i-L];
  }
```

现在我们考虑结点插入的三个应用: 曲线分割 (或细分)、点和导数的评估，以及插入以获得额外的控制点用于交互式形状设计。请参见图 5.5。原始曲线 $\mathbf{C}(u)$ 是由 $U = \{0,0,0,0,2,3,3,3,3\}$ 定义的三次曲线，而 $\mathbf{P}_0,\ldots,\mathbf{P}_4, \bar{u} = 1$ 现在插入了三次。图 5.6a 和 5.6b 显示了插入前后的基函数。该过程将曲线分割。控制点集 $\{\mathbf{P}_0,\mathbf{Q}_{1,1},\mathbf{Q}_{2,2},\mathbf{Q}_{3,3}\}$ 和 $\{\mathbf{Q}_{3,3},\mathbf{Q}_{3,2},\mathbf{Q}_{3,1},\mathbf{P}_3,\mathbf{P}_4\}$ 定义了在结点向量 $U_l = \{0,0,0,0,1,1\ 、\ 1,1\}$ 和 $U_r = \{1,1,1,1,2,3,3,3,3\}$ 上的三次 B 样条曲线 $\mathbf{C}_l(u)$ 和 $\mathbf{C}_r(u)$ 。此外，$\mathbf{C}_l(u)$ 是在域 $u \in [0,1]$ 上限制的 $\mathbf{C}(u)$ ，而 $\mathbf{C}_r(u)$ 是在 $u \in [1,3]$ 上限制的 $\mathbf{C}(u)$ 。曲线分割与凸包性质 (P4.10) 共同构成了使用 NURBS 的“分而治之”算法的基础。图 5.7 显示了递归曲线分割的示例: 原始曲线 $\mathbf{C}$ 在 $u = 1/2$ 处被分割，得到 $\mathbf{C}_1$ 和 $\mathbf{C}_2$ ，然后分别在 1/8 和 3/8 处被分割，产生 $\mathbf{C}_{1,1}$ 和 $\mathbf{C}_{1,2}$ ，以及 $\mathbf{C}_{2,1}$ 和 $\mathbf{C}_{2,2}$ 。

图 5.5 和 5.6 b 还显示了结点插入可以用于评估曲线上的点和导数。显然，$\mathbf{C}(1) = \mathbf{Q}_{3,3}$ ；此外，可以使用起始点和终点导数公式 (方程 (3.7)、(3.9)、(3.10)、(4.9) 和 (4.10)) 轻松计算左右两侧的导数。

算法 A5.2 通过节点插入 (“角落切割”) 计算曲线上的一个点。它假设一个例程 FindSpanMult，该例程找到节点跨度 k 和重数 s 。

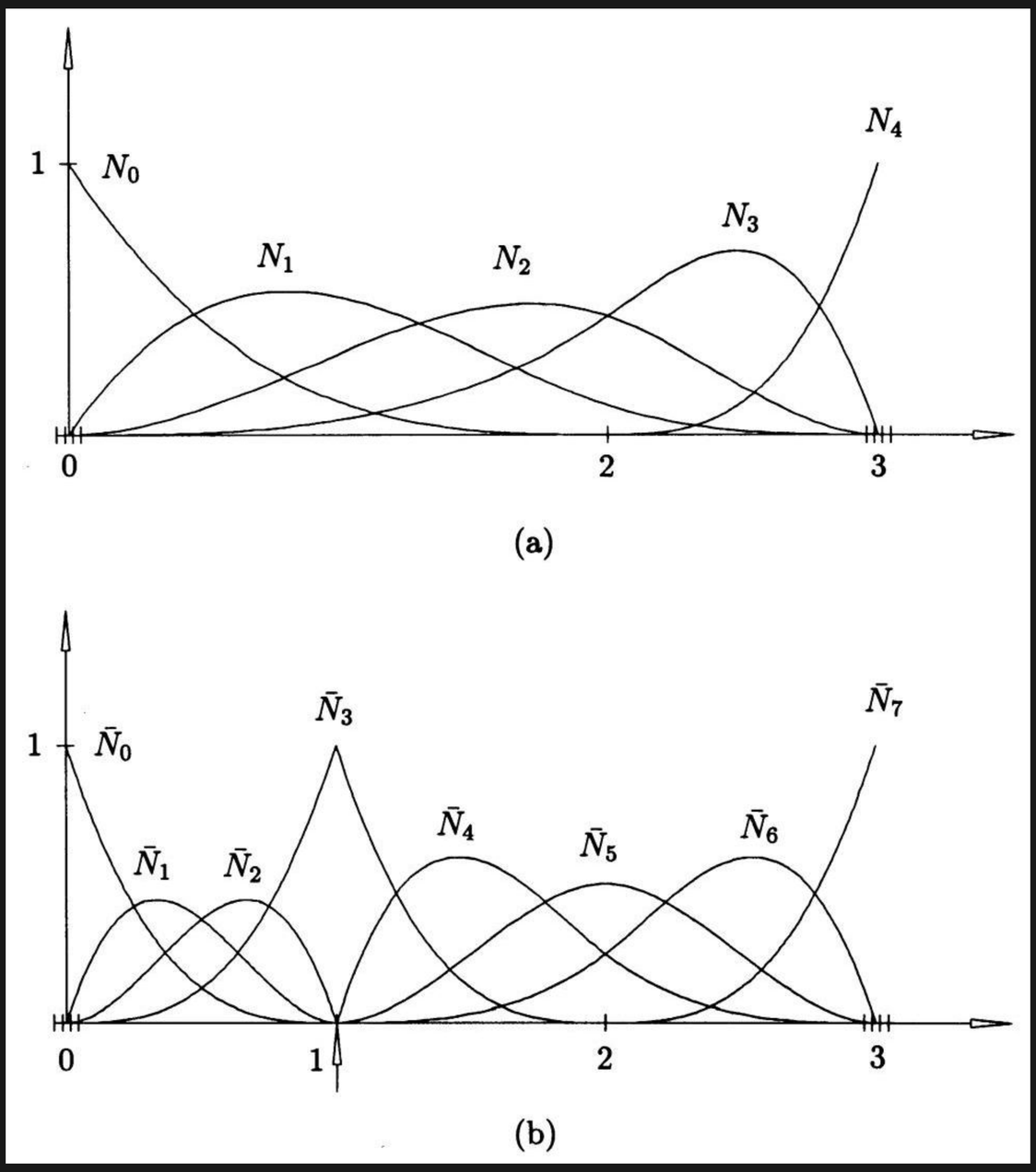


图 5.6. 对应于图 5.5 的基函数。(a) 在节点插入之前；(b) 在节点插入之后。

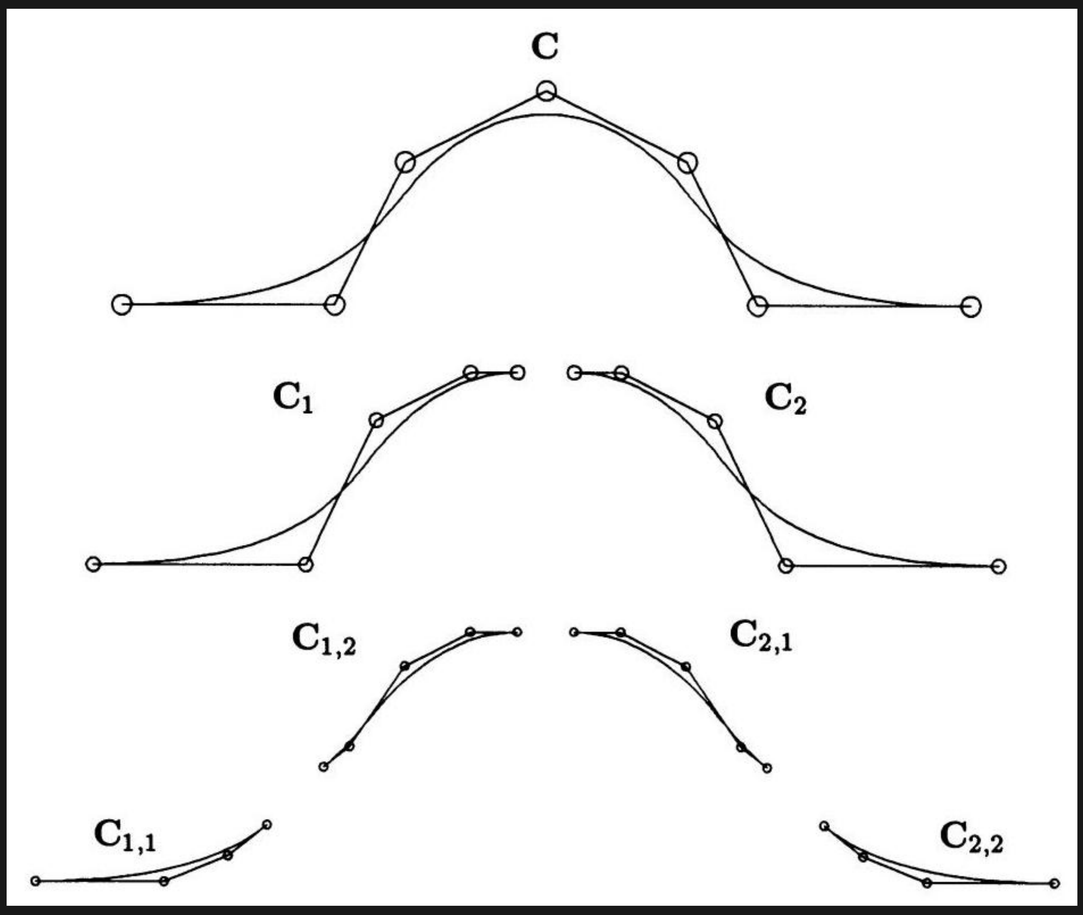


图 5.7. 在 $\{0,0,0,0,1/4,1/2,3/4,1,1,1,1\}$ 上定义的三次曲线的分割。

ALGORITHM A5.2

```
CurvePntByCornerCut(n, p, U, Pw, u, C)
    { /* Compute point on rational B-spline curve */
        /* Input: n, p, U, Pw, u */
        /* Output: C */
      if (u == U[0]) /* Endpoints are special cases */
        {
        C = Pw[0]/w; return;
        }
      if (u == U[n+p+1])
        {
        C = Pw[n]/w; return;
        }
      FindSpanMult (n, p, u, U, &k, &s); /* General case */
      r = p-s;
      for (i=0; i<=r; i++) Rw[i] = Pw[k-p+i];
      for (j=1; j<=r; j++)
          for (i = 0; i<=r-j; i++)
            {
            alfa = (u-U[k-p+j+i])/(U[i+k+1]-U[k-p+j+i]);
            Rw[i] = alfa*Rw[i+1] + (1.0-alfa)*Rw[i];

C = Rw[0]/w;

\
```

第三个应用是交互式形状设计。正如我们在第 4 章中看到的，曲线和表面可以通过移动控制点和/或修改权重来塑造。为了修改的目的，设计师可能希望在当前不存在的控制多边形上的某个地方添加一个控制点。例如，假设他在三维多边形 $\text{leg}, \mathbf{P}_{i-1}\mathbf{P}_i$ 上选择点 $\mathbf{Q}$ (见图 5.8)。将对应的四维点 $L_i = \mathbf{P}_{i-1}^w\mathbf{P}_i^w$ 表示为 $\mathbf{Q}^w$ 。然后存在一个 $\bar{u} \in [u_i, u_{i+p})$ ，使得单次插入 $\bar{u}$ 使 $\mathbf{Q}^w$ 成为一个新的控制点，而 $\mathbf{Q}$ 为其投影。确定 $\bar{u}$ 的过程称为逆节点插入 [Pieg89c]。现在存在一个值 $s, 0 \le s \le 1$ ，使得

$$\mathbf{Q}^w = (1-s)\,\mathbf{P}_{i-1}^w + s\mathbf{P}_i^w$$

投影后得到

$$\mathbf{Q} = \frac{(1-s)\,w_{i-1}\mathbf{P}_{i-1} + sw_i\mathbf{P}_i}{(1-s)\,w_{i-1} + sw_i} \tag{5.19}$$

因此

$$s = \frac{w_{i-1}\,|\mathbf{Q}-\mathbf{P}_{i-1}|}{w_{i-1}\,|\mathbf{Q}-\mathbf{P}_{i-1}| + w_i\,|\mathbf{P}_i-\mathbf{Q}|} \tag{5.20}$$

因此

$$\bar{u} = u_i + s\,(u_{i+p} - u_i) \tag{5.21}$$

通过简单地将先前的公式和算法应用于控制点的行和/或列，可以将节点插入到曲面中。特别地，设 $\mathbf{P}_{i,j}^w$, $0 \le i \le n, 0 \le j \le m$ 为 NURBS 曲面的控制点；将 i 称为行索引，将 j 称为列索引。然后，通过对每个 $m+1$ 控制点列进行 $\bar{u}$ 节点插入，将 $\bar{u}$ 添加到节点向量 U 中。得到的控制网为 $\mathbf{Q}_{i,j}^w, 0 \le i \le n+1, 0 \le j \le m$ 。类似地，必须在每个 $n+1$ 控制点行上插入 $\bar{v}$ 。得到的控制网为 $\mathbf{Q}_{i,j}^w$, $0 \le i \le n, 0 \le j \le m+1$ 。图 5.9 显示了在 $U = \{0,0,0,0,1,1,1,1\}$ and $V = \{0,0,0,1/2,1,1,1\}$. Figures 5.10*a* and 5.10*b* 上的立方 × 二次曲面，分别展示了节点 $\bar{u} = 4/10$ 和 $\bar{v} = 7/10$ 的插入，图 5.10c 显示了两个节点的插入。我们指出，曲面节点插入算法不应仅仅由一个循环组成，在该循环中调用算法 A5.1 $m+1$ (或 $n+1$) 次。阿尔法 (A5.1 中的 alpha) 的计算不依赖于控制点。因此，它们应仅计算一次并存储在本地数组中，然后再进入执行 $m+1$ 或 $n+1$ 节点插入的循环。算法 A5.3 就是这样一个算法。

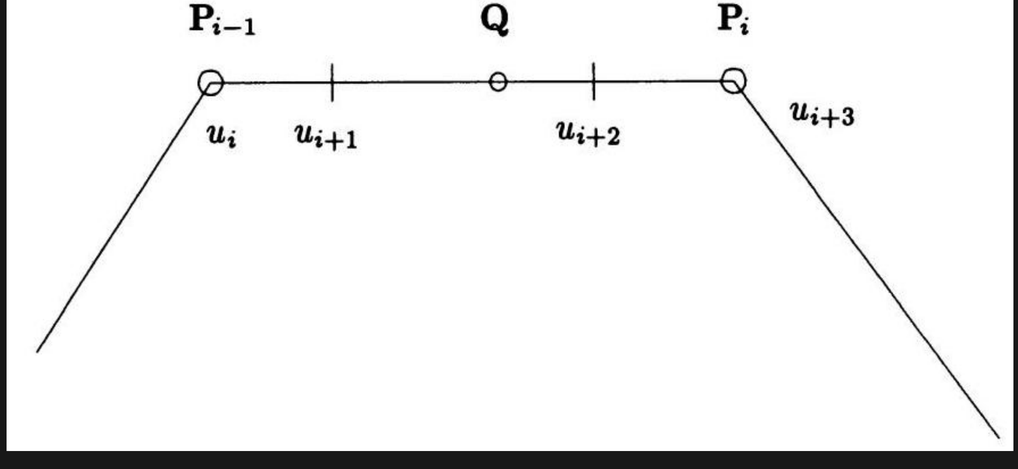


图 5.8. 立方曲线的逆节点插入：点 $\mathbf{Q}$ 位于由节点 u_i, u_{i+1}, u_{i+2} 和 u_{i+3} 划分的腿 $\mathbf{P}_{i-1}\mathbf{P}_i$ 上。

算法 A5.3

```
SurfaceKnotIns(np, p, UP, mp, q, VP, Pw, dir, uv, k, s, r, nq, UQ, mq, VQ, Qw)
    { /* Surface knot insertion */
      /* Input: np, p, UP, mp, q, VP, Pw, dir, uv, k, s, r */
      /* Output: nq, UQ, mq, VQ, Qw */
    if (dir == U_DIRECTION)
```

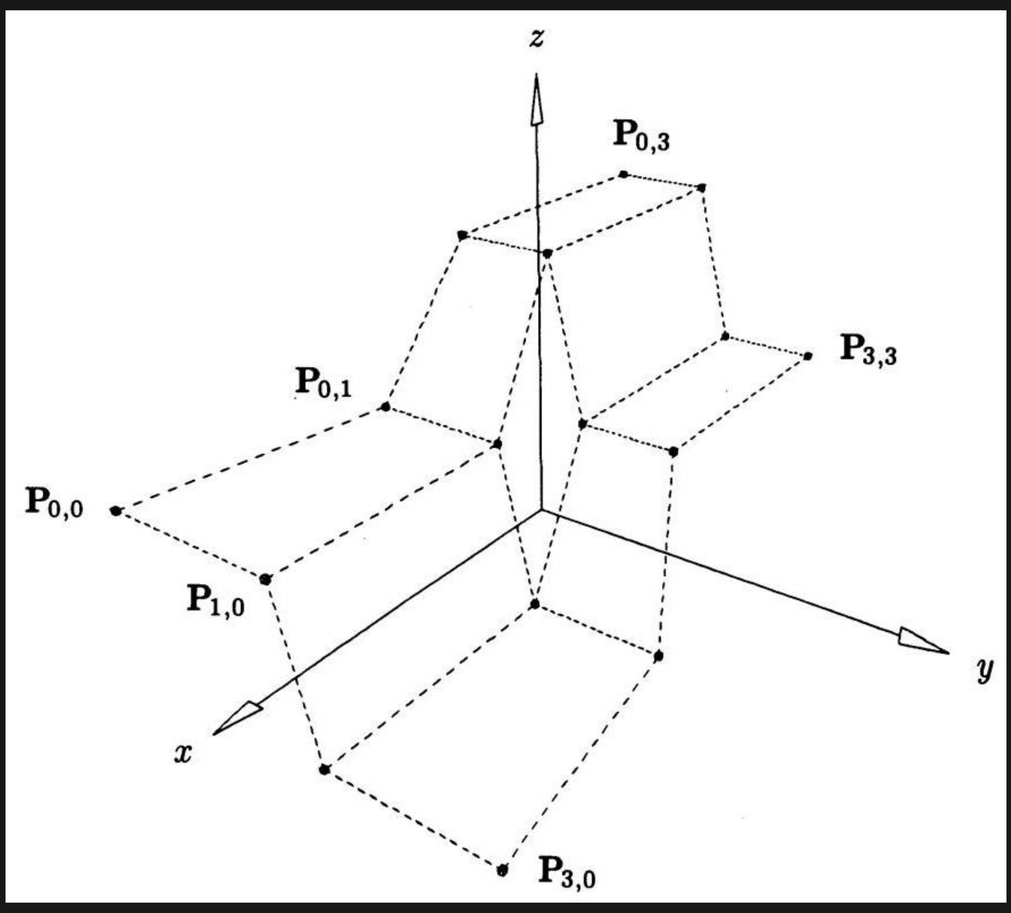
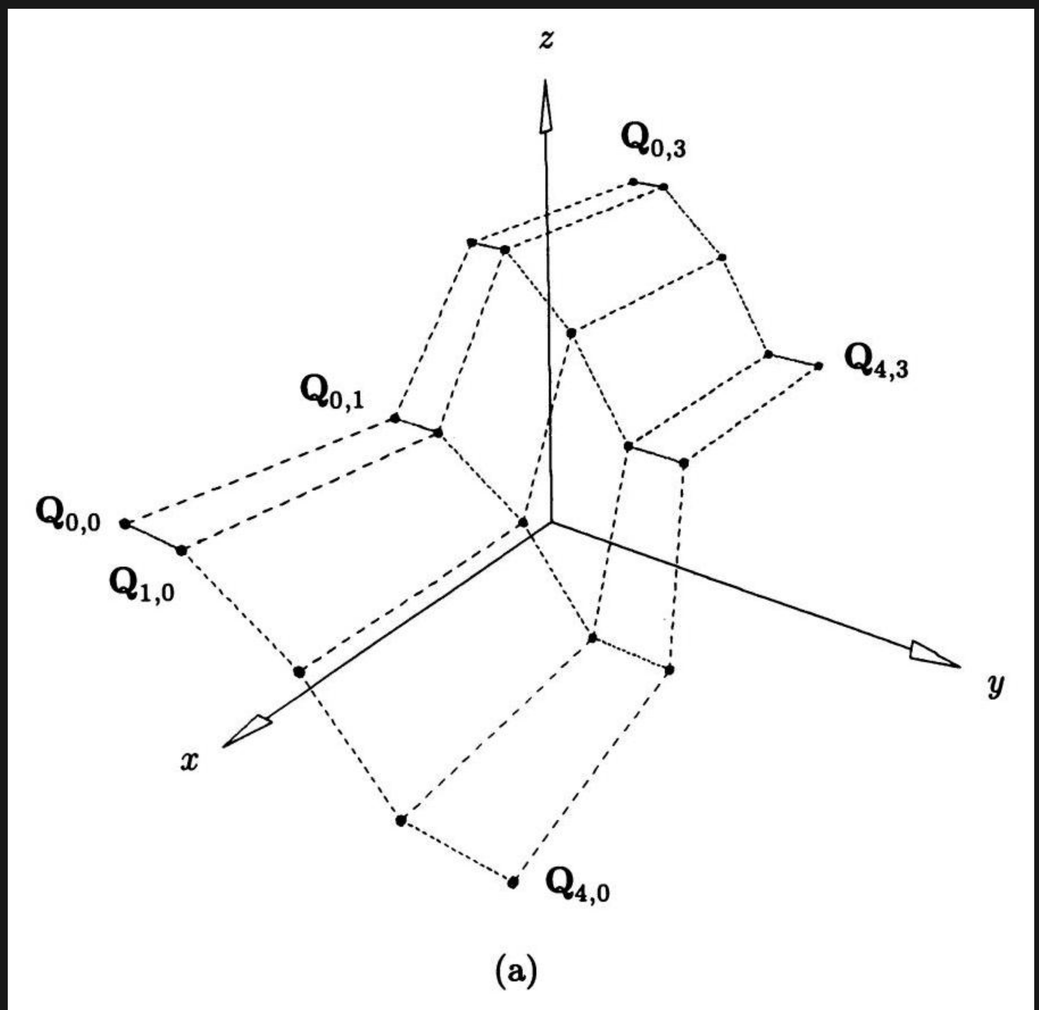


图 5.9. 定义在节点向量 $U = \{0, 0, 0, 0, 1, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 1/2, 1, 1, 1\}$ 上的 (立方 × 二次) 曲面的控制网。



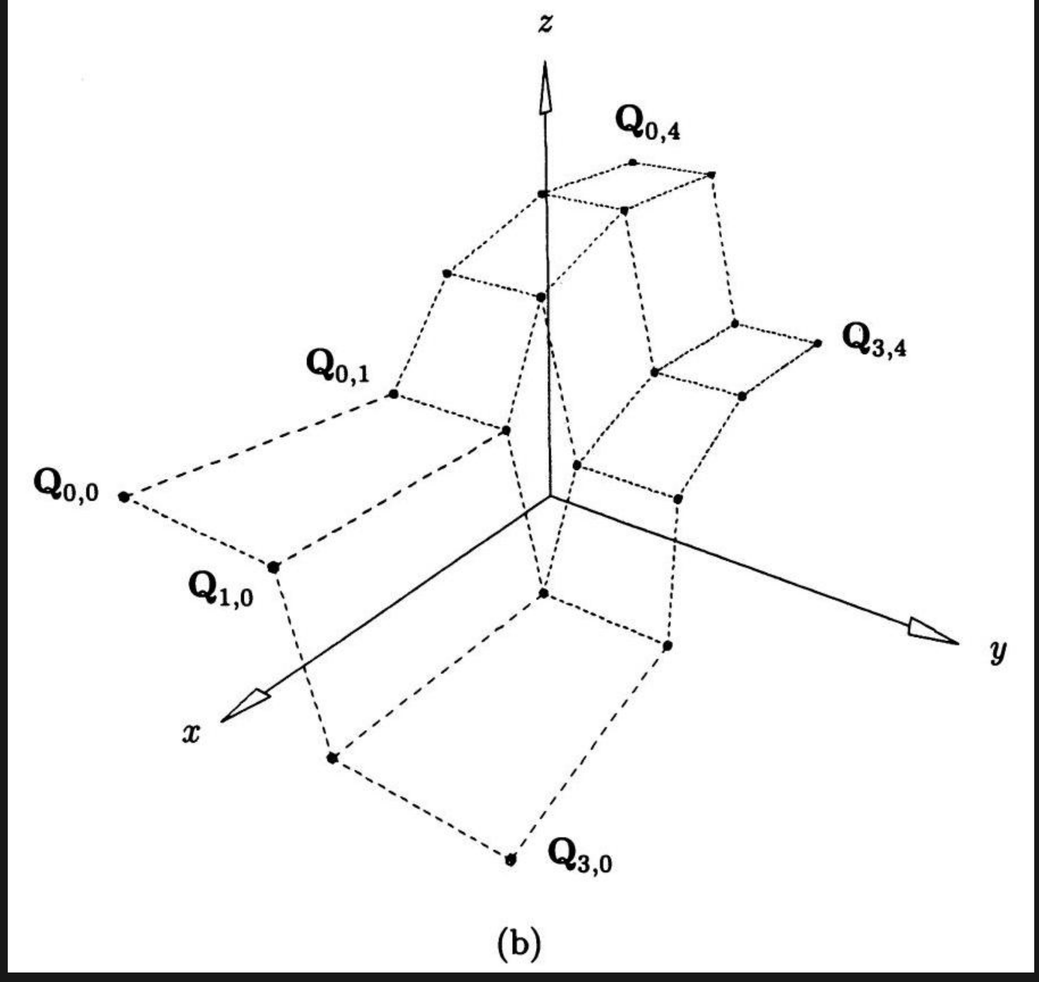


图 5.10. 将节点插入到图 5.9 的表面中。(a) 在 u 方向上插入 $u = 2/5$ 一次；(b) 在 v 方向上插入 $v = 7/10$ 一次；(c) 同时插入两个节点。

(c)

Figure 5.10. (Continued.)

```
{
load u-vector as in A5.1
copy v-vector into VQ
        /* Save the alphas */
for (j=1; j<=r; j++)
  {
  L = k – p + j ;
  for (i=0; i<=p-j-s; i++)
       alpha[i][j] = (uv-UP[L+i])/(UP[i+k+1]-UP[L+i]);
  }
for (row=0; row<=mp; row++) /* For each row do */
  {
            /* Save unaltered control points * /
  for (i=0; i<=k-p; i++) Qw[i][row] = Pw[i][row];
  for (i=k; i<=np; i++) Qw[i+r][row] = Pw[i][row];
            /* Load auxiliary control points. */
  for (i=0; i<=p-s; i++) Rw[i] = Pw[k-p+i][row];
  for (j = 1; j <= r; j + +) /* Insert the knot r times * /
```

```
                                    {
                                    L = k − p + j ;
                                    for (i=0; i<=p-j-s; i++)
                Rw[i] = alpha[i][j]*Rw[i+1] + (1.0-alpha[i][j])*Rw[i];
            Qw[L][row] = Rw[0];
            Qw [k + r − j] [row] = Rw [p − j] ;
            }
                    /* Load the remaining control points * /
          for (i=L+1; i<k; i++) Qw[i][row] = Rw[i-L];
        }
      }
if (dir == V_DIRECTION)
      {
      /* Similar code as above with u and v directional
            parameters switched */
      }
}
```

我们以对表面节点插入的评论结束本节。

- 表面细分: 设 $\bar{u}$ 和 $\bar{v}$ 为内部参数值。插入 ing $\bar{u}p$ 次和 $\bar{v}q$ 次将表面细分为四个 B 样条 (子) 表面。这个事实，加上凸包性质，意味着递归细分适用于 B 样条表面问题。图 5.11 显示了一个待细分的表面。在图 5.12-5.14 中，分别展示了在 u 方向、v 方向以及两个方向上的细分；

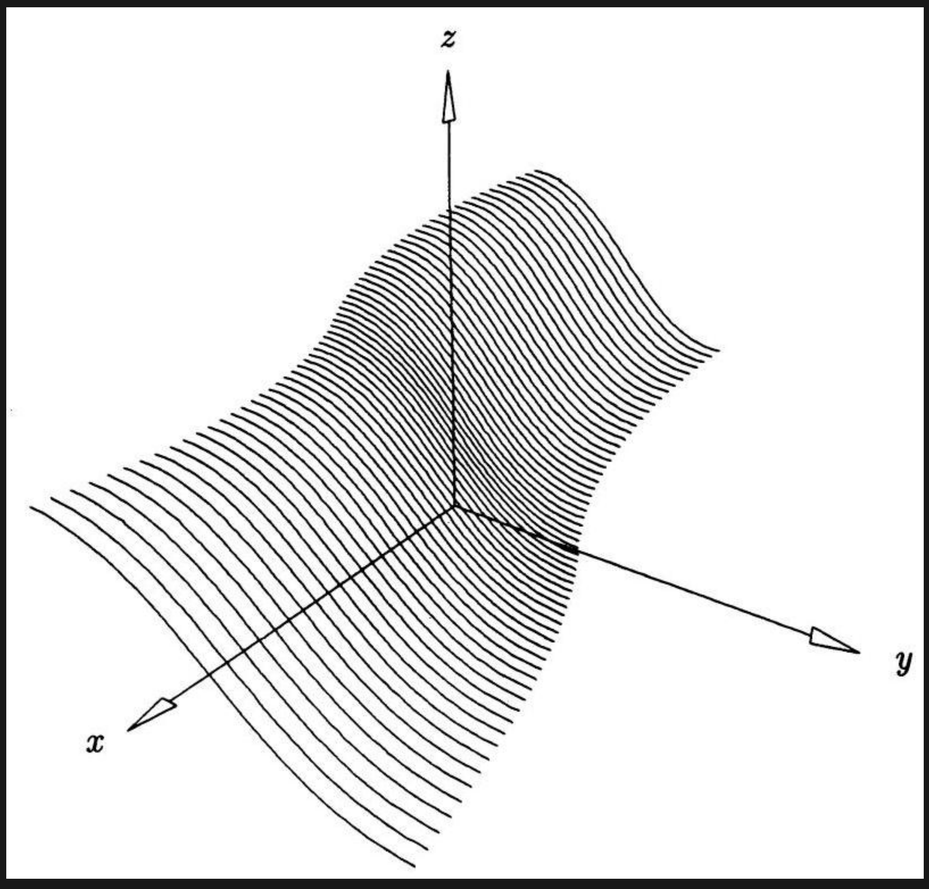


图 5.11. 如图 5.9 所示定义的 (立方 × 二次) 表面。

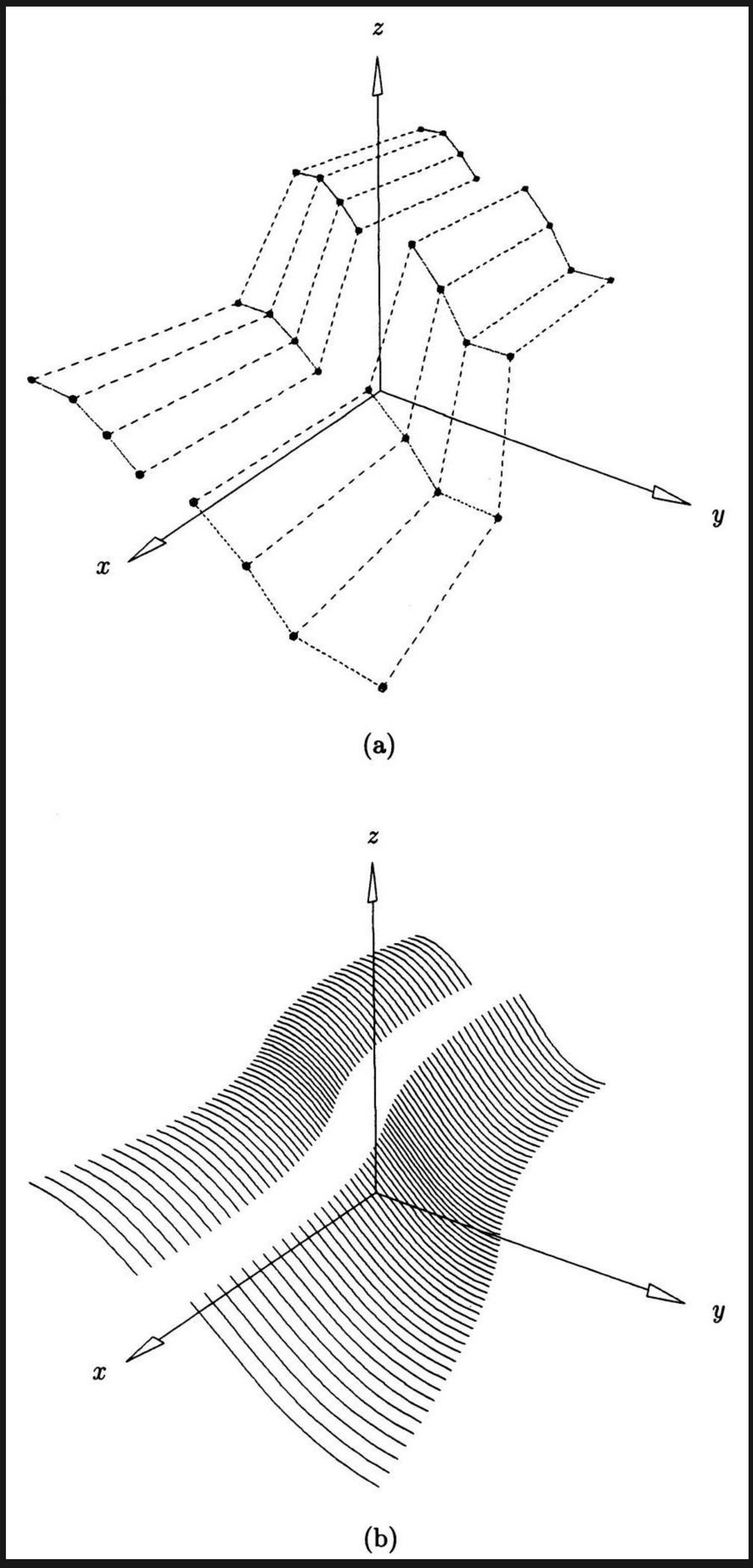


图 5.12. 在 $u = 2/5$ 处沿 v 方向细分图 5.11 的表面。(a) 被细分表面的控制网；(b) 被细分的表面。

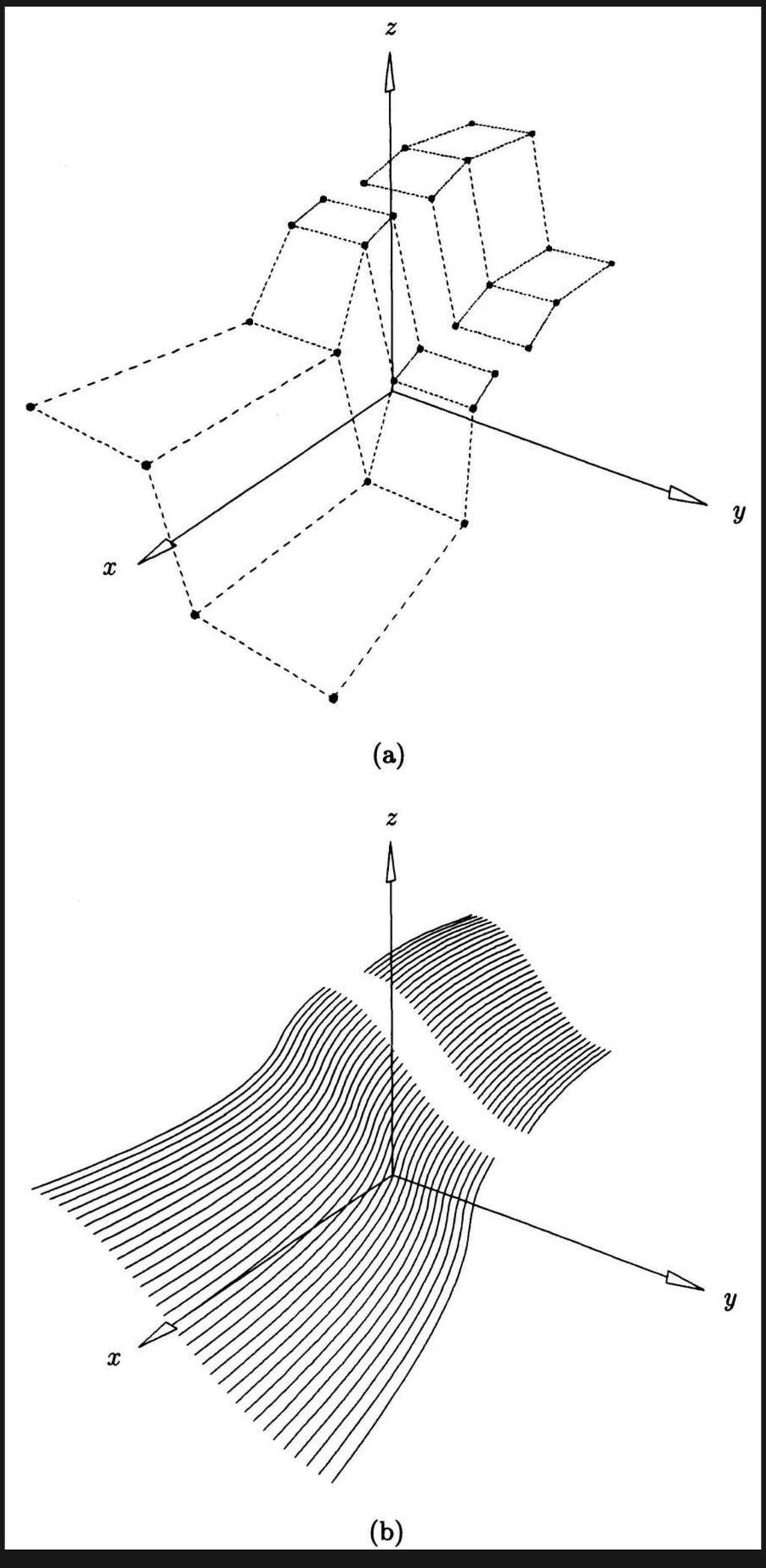


图 5.13. 在 $v = 7/10$ 处沿 u 方向细分图 5.11 的表面。(a) 被细分表面的控制网；(b) 被细分的表面。

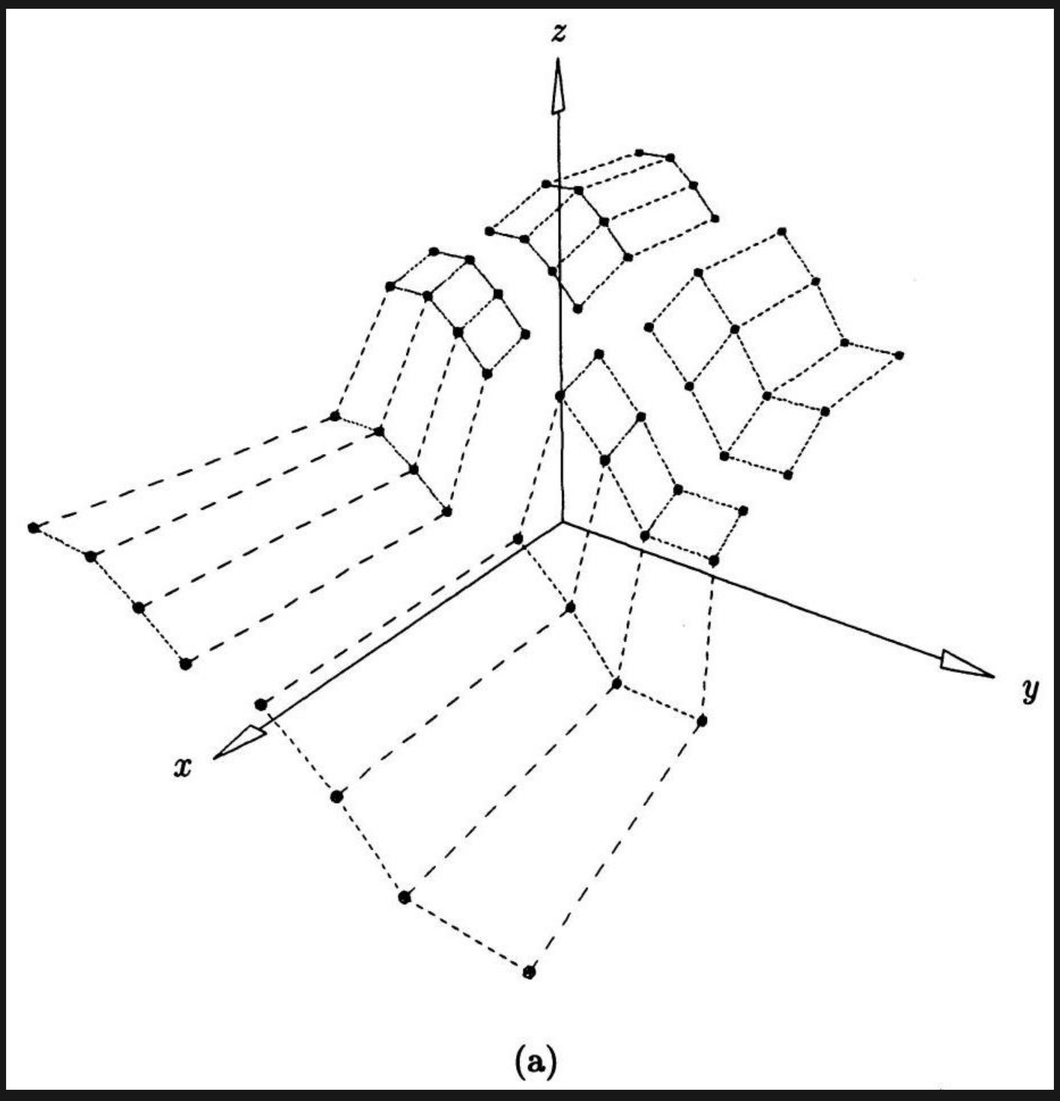


图 5.14. 在 $u = 2/5$ 和 $v = 7/10$ 处沿 u 和 v 两个方向细分图 5.11 的表面。(a) 被细分表面的控制网；(b) 被细分的表面。

- 曲线提取：要提取等参数曲线 $\mathbf{C}_{\bar{u}}^{w}(v)$，我们必须在每个控制点列 $m+1$ 上进行 $p\bar{u}$ 次节点插入，即总共进行 $p(m+1)$ 次节点插入；这将产生公式 (4.16) 的 $\left\{\mathbf{Q}_{j}^{w}(\bar{u})\right\}$。提取曲线 $\mathbf{C}_{\bar{v}}^{w}(u)$ 需要 $q(n+1)$ 次节点插入；图 5.15 显示了一个例子；
- 曲面评估：插入 $\bar{u}p$ 次和 $\bar{v}q$ 次会导致曲面点 $\mathbf{S}(\bar{u},\bar{v})$ 成为生成的四个子曲面的角控制点。可以开发类似于算法 A5.2 的角切割算法。可以先插入 $\bar{u}$ 或 $\bar{v}$，但正如使用 deCasteljau 算法进行 Bézier 曲面评估时的情况一样，计算复杂度是顺序相关的，如果 $p \neq q$（见公式 [1.25] 和 [1.26]）。因此，我们可以用以下任一方法计算 $\mathbf{S}(\bar{u},\bar{v})$：
- $p(q+1)$ $\bar{u}$ 次节点插入，加上 $q\bar{v}$ 次节点插入；
- $q(p+1)$ $\bar{v}$ 次节点插入，加上 $p\bar{u}$ 次节点插入。

使用角点导数公式（例如，公式 [3.24]、[4.24]、[4.25] 和 [4.26]），从四个方向（左、右，分别针对 u 和 v）获得偏导数和法向量。

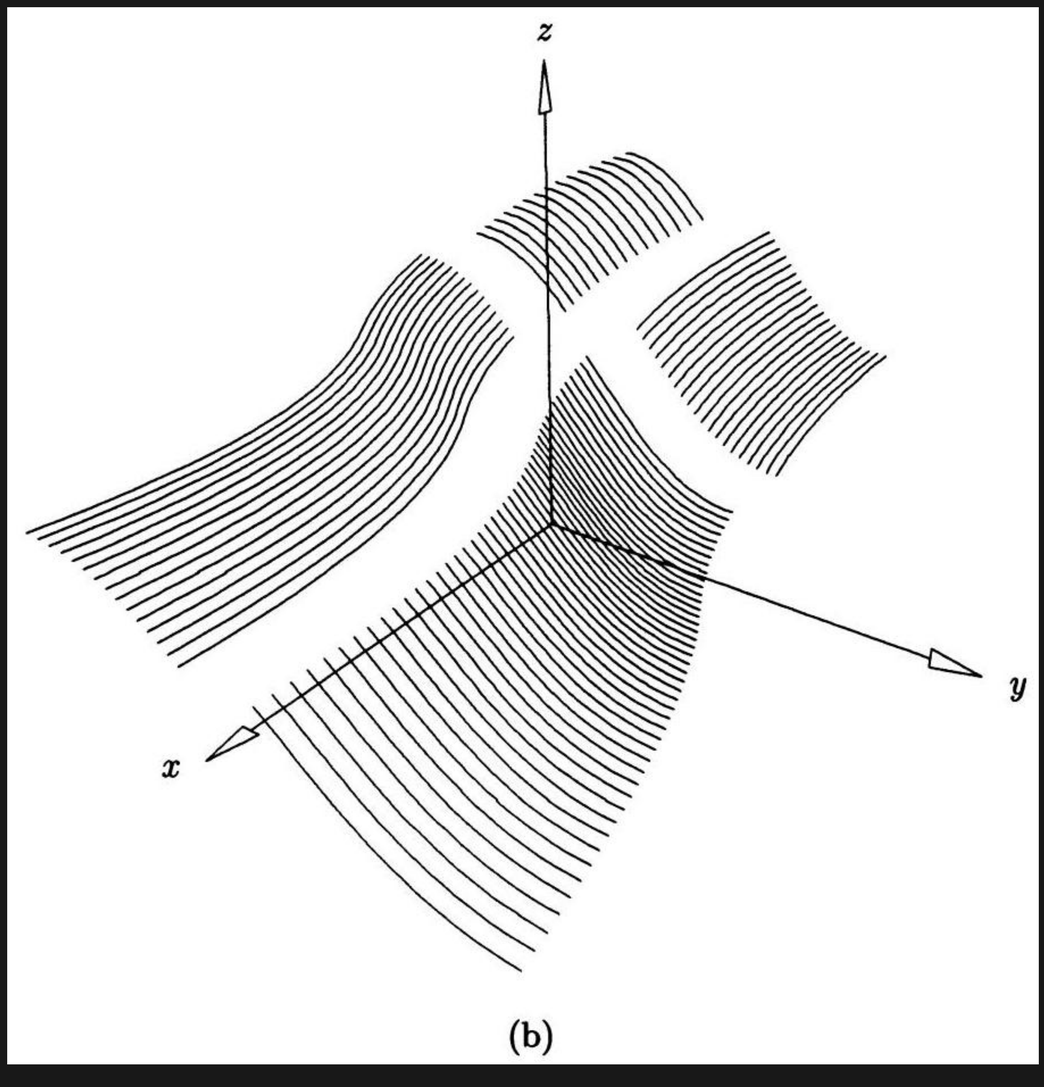


图 5.14.(续)

3 节点细化

节点插入关注的是插入一个单一的节点，可能会多次插入。通常需要一次插入多个节点；这称为节点细化。为了解释这个问题，设 $\mathbf{C}^w(u)=\sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w$ 定义在节点向量 $U=\{u_0,\ldots,u_m\}$ 上，并且设 $X=\{x_0,\ldots,x_r\}$ 满足 $x_i\le x_{i+1}$ 和 $u_0<x_i<u_m$ 对所有 i。将 X 的元素插入到 U 中，并计算相应的新控制点集 $\{\mathbf{Q}_i^w\},i=0,\ldots,n+r+1$。新的节点 x_i 应在 X 中按其重数重复；例如，如果 x 和 $y\,(x<y)$ 分别以重数 2 和 3 插入，则 $X=\{x,x,y,y,y\}$。显然，节点细化可以通过多次应用节点插入来完成。然而，我们在这里区分这两个问题，因为存在更高效的节点细化算法 (见 [Cohe80; Boeh85a, 85b; Lych85])。

节点细化的应用包括:

- 将 B-spline 曲线和曲面分解为其组成的 (Bézier) 多项式片段 - 我们将在后面详细阐述这一点；
- 合并两个或多个节点向量，以获得在一个共同节点向量上定义的一组曲线；正如我们在后续章节中看到的，这在构建某些类型的曲面时非常重要；

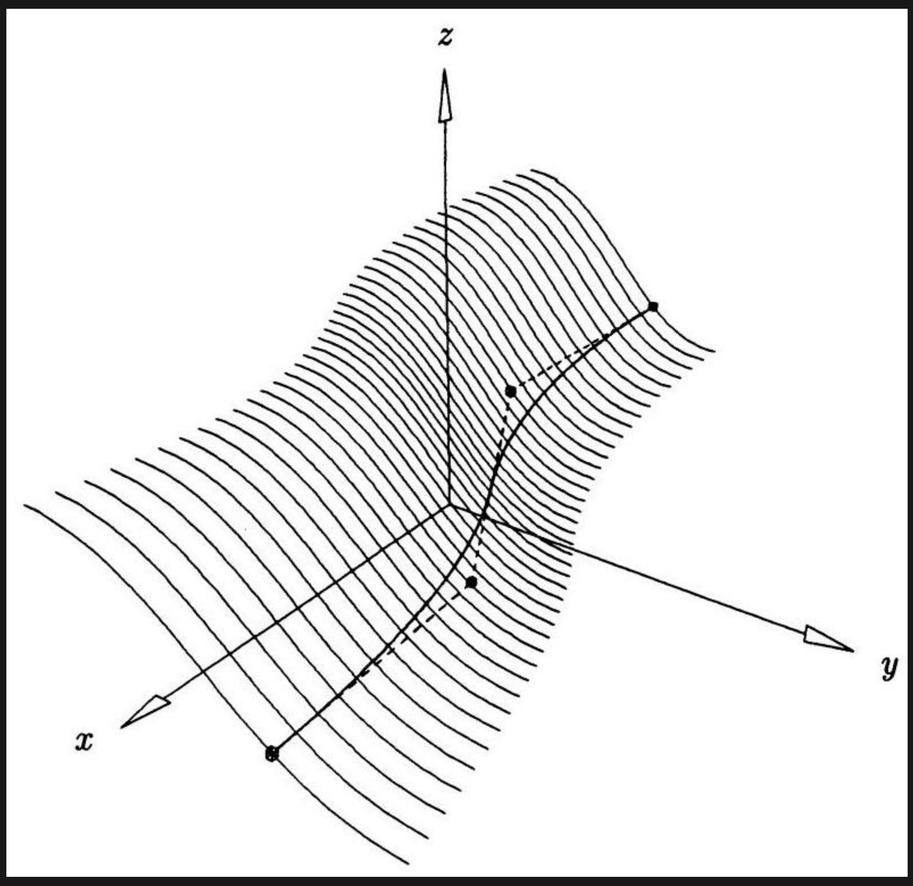


图 5.15. 通过节点插入提取曲面等参数曲线。

- 获取曲线 (表面) 的多边形 (多面体) 近似；细化结点向量使控制多边形 (网) 更接近曲线 (表面)，在极限情况下，多边形 (网) 收敛于曲线 (表面)。这一点的证明见于 [Lane80; DeBo87]。图 5.16b 和 5.16c 显示了对图 5.16a 中立方曲线应用的一次和两次中点结点细化 (在每个结点跨度的中点插入一个新结点)。图 5.17a 到 5.17c 显示了对图 5.9 的表面进行的三次中点结点细化，分别在 $u-,v-$ 和两个方向上。

我们不需要额外的数学知识来开发结点细化算法；这实际上只是一个软件问题。解决步骤如下:

1. 找到索引 a 和 b，使得对于所有 i 有 $u_a\le x_i<u_b$；
2. 从方程 (5.14) 和 (5.18) 可以得出控制点 $\mathbf{P}_0^w,\ldots,\mathbf{P}_{a-v}^w$ 和 $\mathbf{P}_{b-1}^w\ldots,\mathbf{P}_n^w$ 不变；因此，将这些复制到适当的 $\mathbf{Q}_i^w$ 位置，为新的控制点 $r+p+b-a-1$ 留出空间；
3. 用 $\bar{U}$ (U merged with X) 表示新的结点向量；复制两端不变的结点；
4. 进入循环并

4.1. 计算新的控制点；

4.2. 将 U 和 X 的元素合并到 $\bar{U}$ 中；

循环可以向前 (从 $\mathbf{Q}_{a-p+1}^w$ 开始) 或向后 (从 $\mathbf{Q}_{b+r-1}^w$) 开始) 进行。

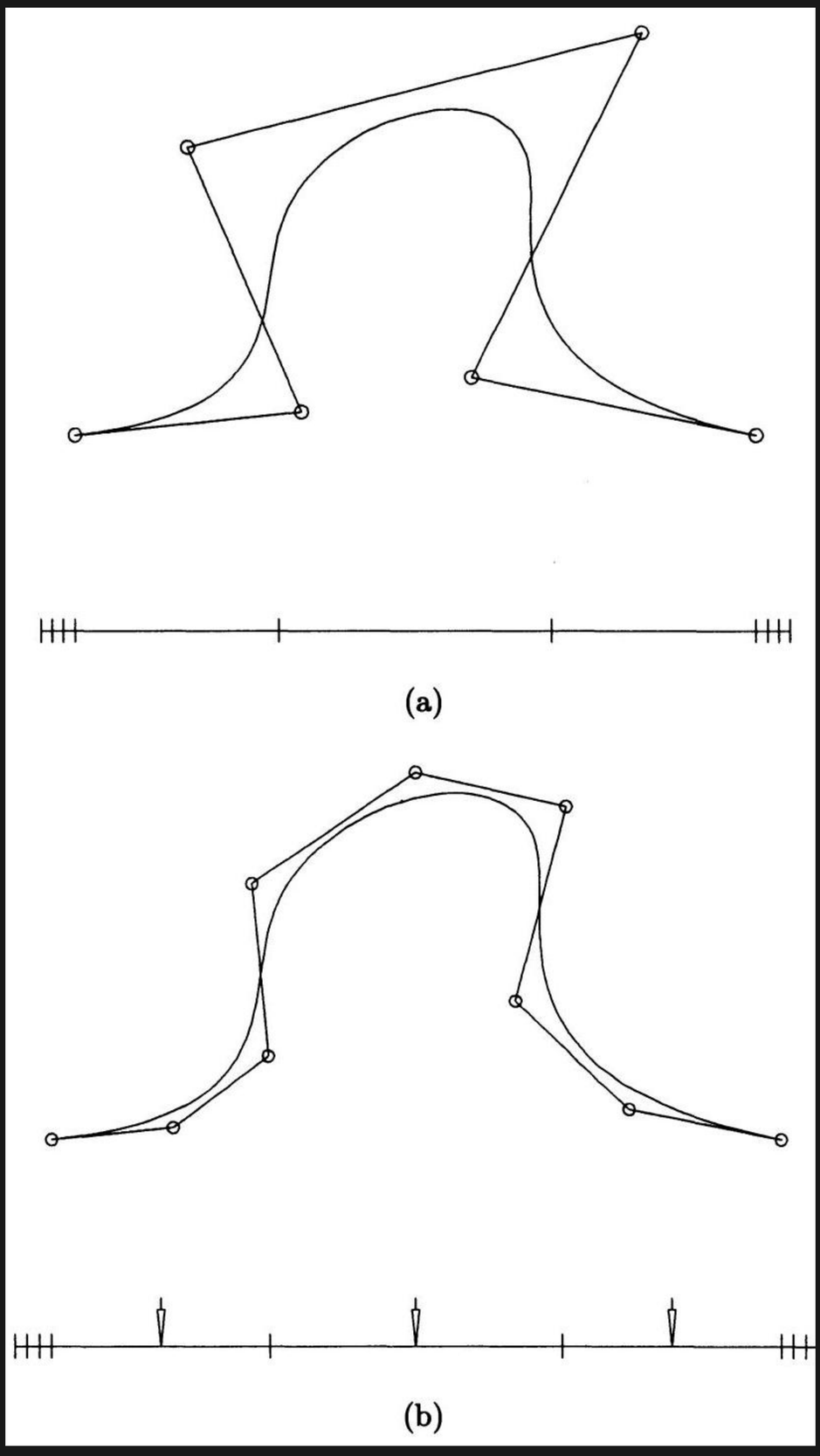


图 5.16. 曲线细化。(a) 定义在 $\{0,0,0,0,{}^{3}/_{10},1,1,1,1\}$ 上的立方曲线；(b) 第一次中点结点细化；(c) 第二次中点结点细化。

算法 A5.4 由 Boehm 和 Prautzsch 提出 [Boeh85a]。它向后工作，并在插入节点时覆盖中间控制点。Ubar 是新的节点向量，$\bar{U}$ 。

ALGORITHM A5.4

```
RefineKnotVectCurve(n, p, U, Pw, X, r, Ubar, Qw)
  { /* Refine curve knot vector */
      /* Input: n, p, U, Pw, X, r */
```

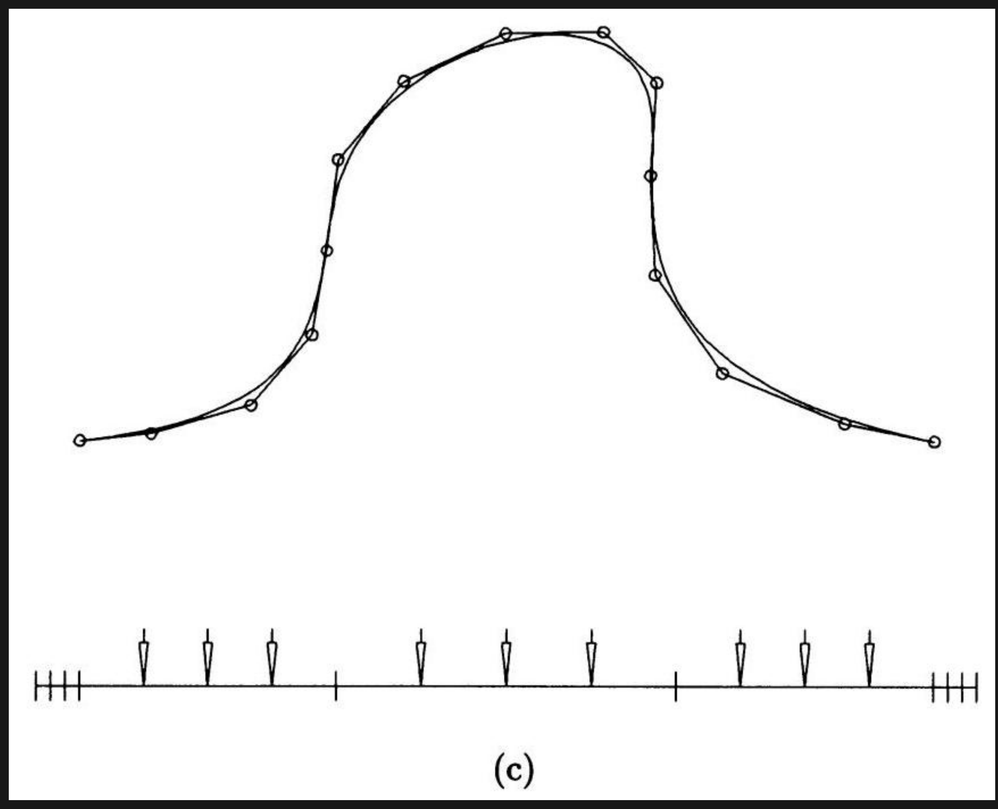


图 5.16. (续)

```
        /* Output: Ubar, Qw */
m = n+p+1;
a = FindSpan(n, p, X[0], U); /* Algorithm A2.1 */
b = FindSpan(n, p, X[r], U);
b = b+1;
for (j=0; j<=a-p; j++) Qw[j] = Pw[j];
for (j = b − 1; j <= n; j + +) Qw [j + r + 1] = Pw [j] ;
for (j = 0; j <= a; j + +)  Ubar [j] = U [j] ;
for (j=b+p; j<=m; j++) Ubar[j+r+1] = U[j];
i = b+p-1; k = b+p+r;
for (j=r; j>=0; j–)
    {
    while ( X [j] <= U [i] &&i > a )
        {
        Qw [k − p − 1] = Pw [i − p − 1] ;
        Ubar[k] = U[i];
      k = k − 1; i = i − 1 ;
      }
    Qw [k − p − 1] = Qw [k − p] ;
    for (l=1; l<=p; l++)
        {
        ind = k-p+l;
        alfa = Ubar[k+l]-X[j];
        if (abs(alfa) == 0.0)
            Qw[ind-1] = Qw[ind];
        else
```

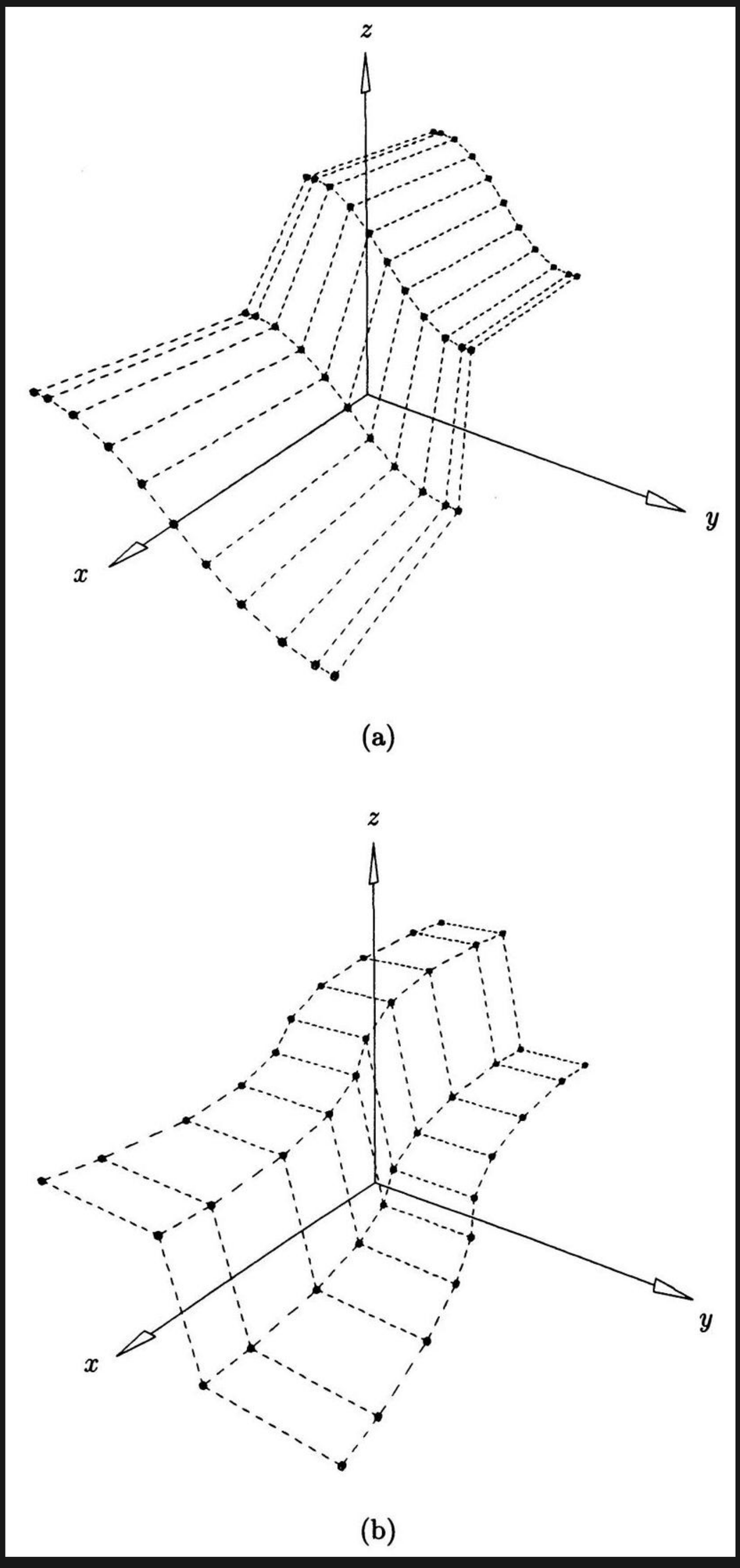


图 5.17. 对图 5.9 中的表面进行细化。(a) 在 u 方向上的第三次中点细化；(b) 在 v 方向上的第二次中点细化；(c) 在 u 方向上的第三次细化和在 v 方向上的第二次细化。

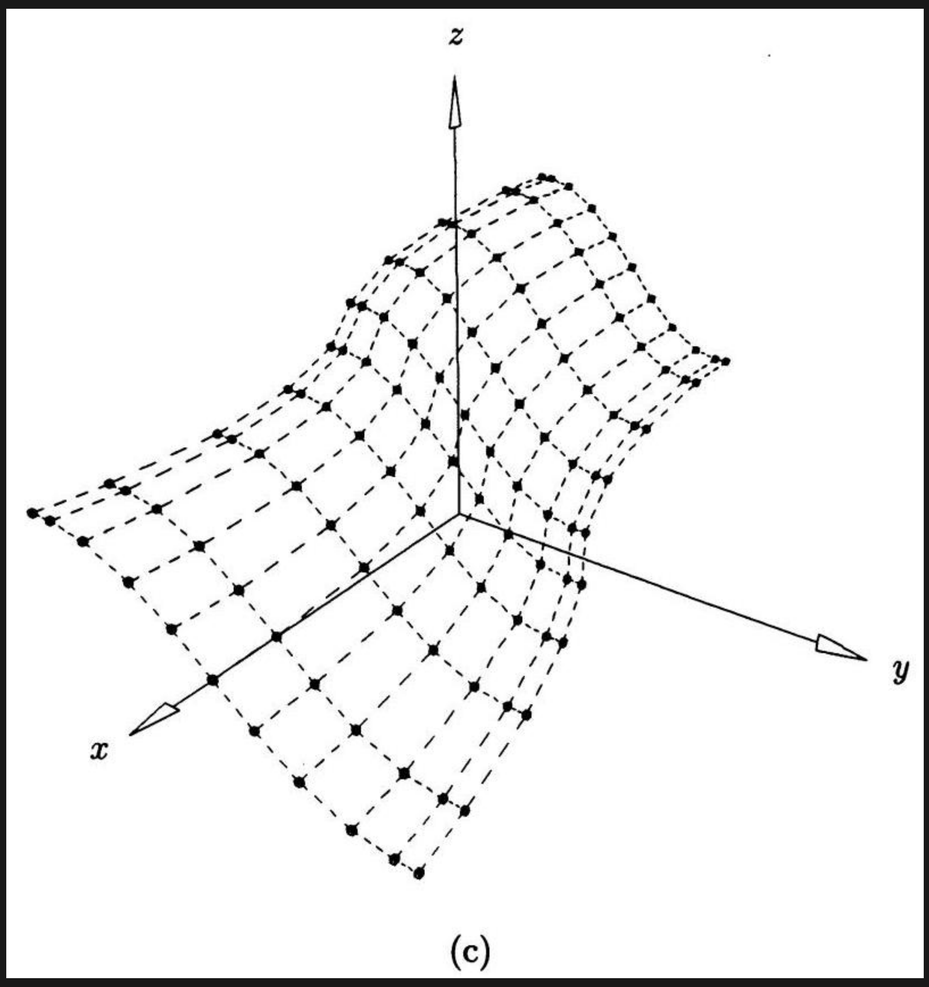


图 5.17. (续)

```
        {
        alfa = alfa/(Ubar[k+l]-U[i-p+l]);
        Qw[ind-1] = alfa*Qw[ind-1] + (1.0-alfa)*Qw[ind];
        }
      }
    Ubar[k] = X[j];
    k = k − 1 ;
    }
}
```

设 $\mathbf{S}^w(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u)\,N_{j,q}(v)\,\mathbf{P}^w_{i,j}$ 为 U 和 V 上的 NURBS 曲面。通过简单地将算法 A5.4 应用于 $m+1$ 列控制点来完成 U 节点向量细化。进行 V 细化需要 $n+1$ 次算法 A5.3 的应用。算法可以组织得如此，以消除冗余操作 (例如，最后一个 for 循环中的 alfas 值对于每个 $m+1$ 列控制点都是相同的)。算法的草图是:

ALGORITHM A5.5

```
RefineKnotVectSurface(n, p, U, m, q, V, Pw, X, r, dir, Ubar, Vbar, Qw)
  { /* Refine surface knot vector */
    /* Input: n, p, U, m, q, V, Pw, X, r, dir */
    /* Output: Ubar, Vbar, Qw */
    if (dir == U_DIRECTION)
      {
      find indexes a and b ;
      initialize Ubar;
      copy V into Vbar;
                          /* Save unaltered ctrl pts */
      for (row=0; row<=m; row++)
          {
          for (k = 0; k <= a − p; k++) Qw[k][row] = Pw[k][row] ;
          for (k = b − 1; k <= n; k++) Qw[k + r + 1][row] = Pw[k][row] ;
          }
      for (j=r; j>=0; j−)
          {
          while ( X[j] <= U[i] && i>a)
            {
            compute Ubar;
            for ( row ... ) Qw[k-p-1][row] = Pw[i-p-1][row];
            k = k − 1; i = i − 1;
            }
          for ( row ... ) Qw[k-p-1][row] = Qw[k-p][row];
          for (l=1; l<=p; l++)
            {
            ind = k-p+l;
            compute alfa;
            if (abs(alfa) == 0.0)
                for ( row ... ) Qw[ind-1][row] = Qw[ind][row];
            else
                {
                compute alfa;
                for ( row ... )
                  Qw[ind-1][row] =
                      alfa*Qw[ind-1][row]+(1.0-alfa)*Qw[ind][row];
                }
            }
          Ubar[k] = X[j]; k = k-1;
        }
      }
    if (dir == V_DIRECTION)
      {
      /* Similar code as above with u and v directional
          parameters switched */
      }
  }
```

An important application of knot refinement is the problem of decomposing a

将 NURBS 曲线分解为其组成的 (四维) 多项式段。这在将 NURBS 曲线转换为另一种样条形式时是必需的，例如，转换为 IGES 参数样条曲线，实体类型 112 [IGE93]。在这样的转换中，第一步是将曲线分解为分段的 Bézier 形式。通过插入每个内部节点，直到其重数为 p，可以获得段的 Bézier 控制点。这分两步完成:

1. 通过 U 并构建细化向量 X；

2. 调用算法 A5.4。

图 5.18a 和 5.18b 显示了一个立方曲线及其对应的基函数；图 5.19a 和 5.19b 显示了同一曲线及其基函数在分解后的情况。图 5.19a 中的控制点是曲线段的贝塞尔控制点。图 5.20a、5.20b、5.21a 和 5.21b 显示了一个表面的分解。

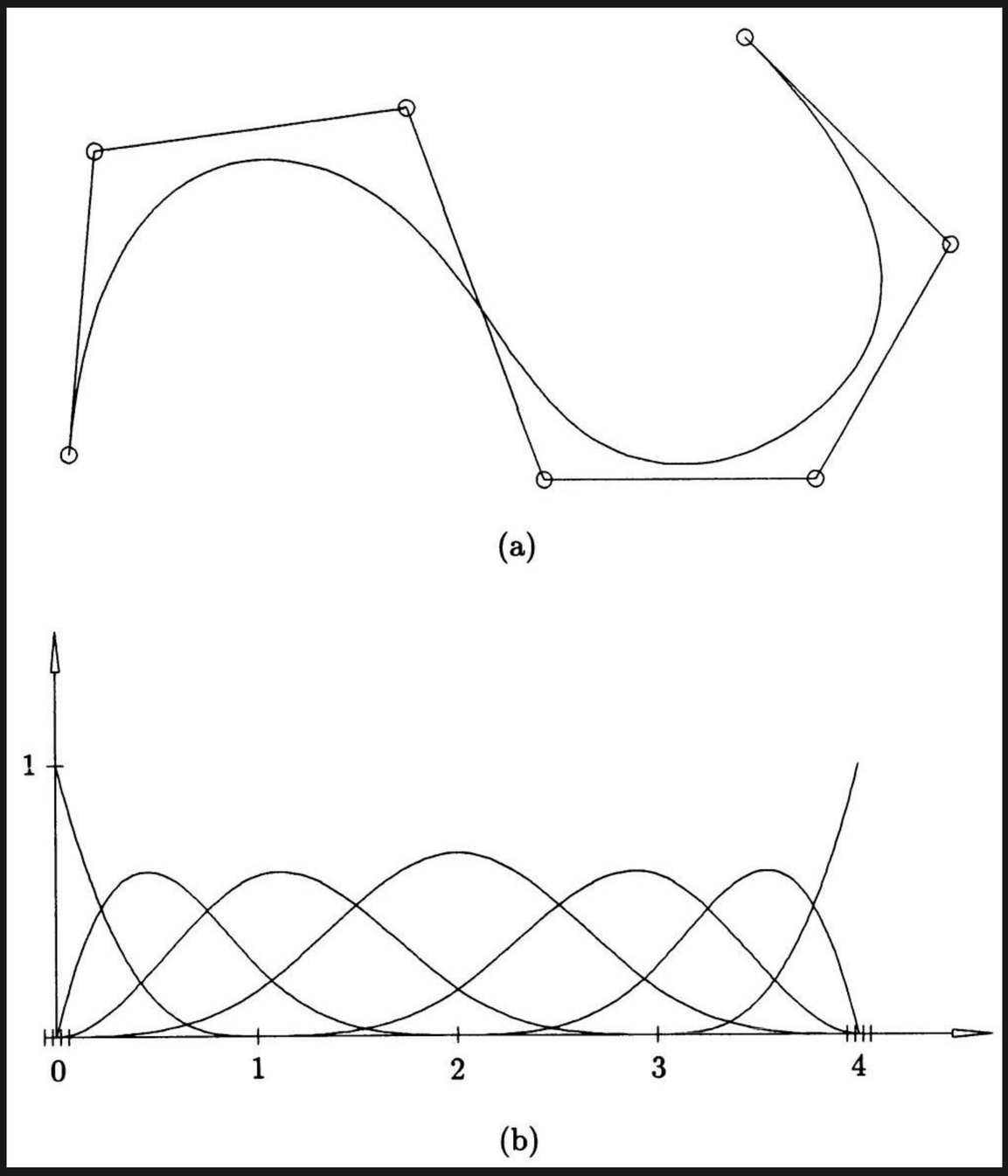


图 5.18. 一个立方曲线。(a) 曲线及其控制多边形；(b) 定义在 $\{0, 0, 0, 0, 1, 2, 3, 4, 4, 4, 4\}$ 上的基函数。

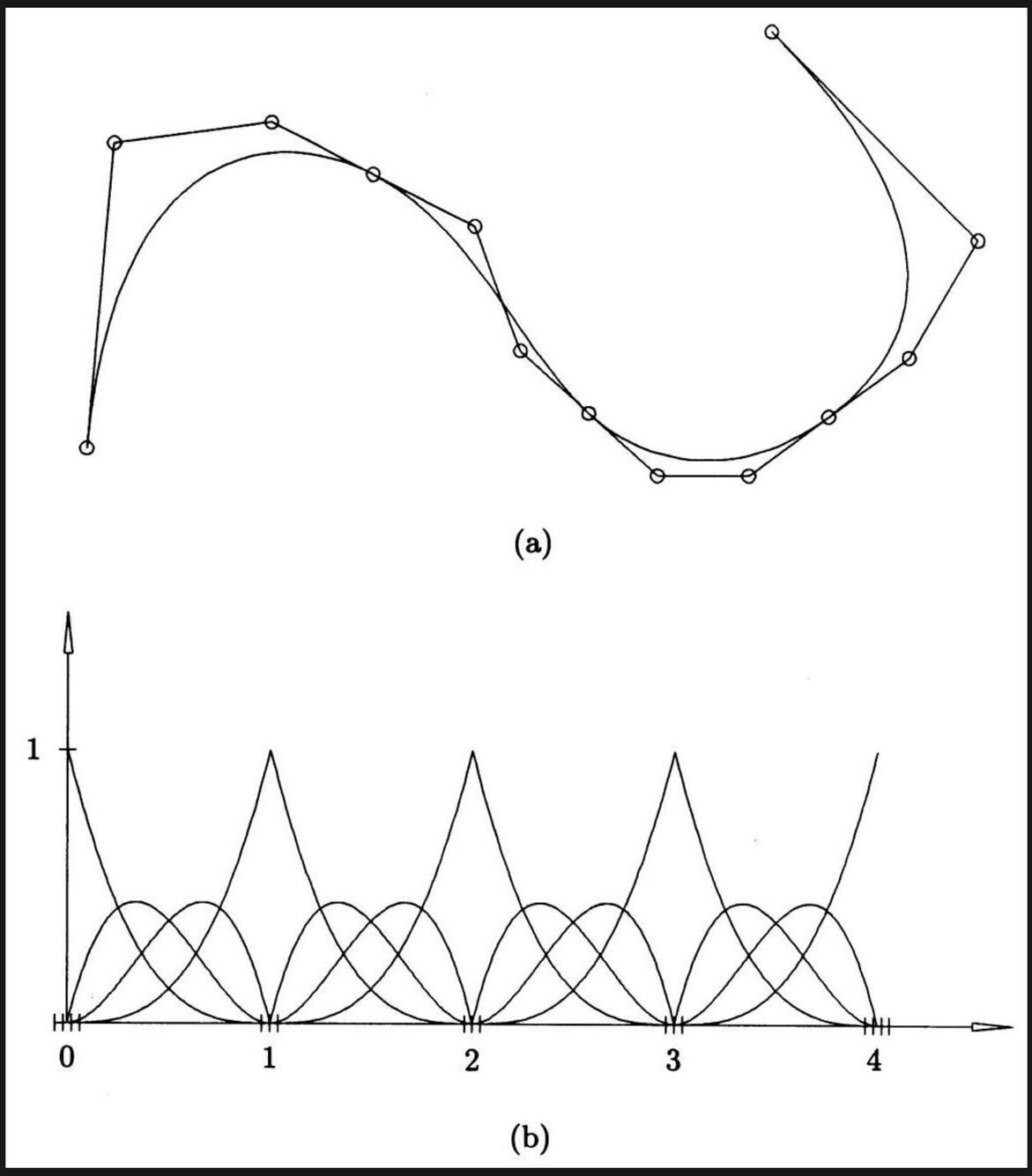


图 5.19. 将图 5.18 中的曲线通过节点细化分解为分段贝塞尔曲线。(a) 分解后的曲线及其控制多边形；(b) 在插入节点 1、2 和 3 各两次后的基函数。

现在我们给出一个算法，该算法以高效的方式从左到右提取曲线的所有贝塞尔段。为方便起见，我们省略控制点上的 w 上标。图 5.22a 到 5.22 d 最好地解释了该算法。设 $\mathbf{C}(u)=\sum_{i=0}^{5} N_{i,3}(u)\,\mathbf{P}_i$ 为一个具有两个不同内部节点的立方曲线，设 $\mathbf{Q}_k^j$ 为第 k 个贝塞尔段的第 j 个控制点，$k=0,\ldots,p$ 和 $j=0,1,2$ 。注意在图 5.22 b 和 5.22c 中，当计算第零段的最右侧贝塞尔控制点 $\mathbf{Q}_0^2$ 和 $\mathbf{Q}_0^3$ 时 (通过节点插入)，第一段的最左侧贝塞尔点 $\mathbf{Q}_1^0$ 和 $\mathbf{Q}_1^1$ 也正在计算并存储。

与使用一般结精细化相比，通过检查结插入　的计算 (公式 [5.15])，实现了另一个主要的计算减少。假设 $[u_a,u_b]$ 是当前正在处理的段，其中 a 和 b 是结 u_a 和 u_b 的最右侧出现的索引。

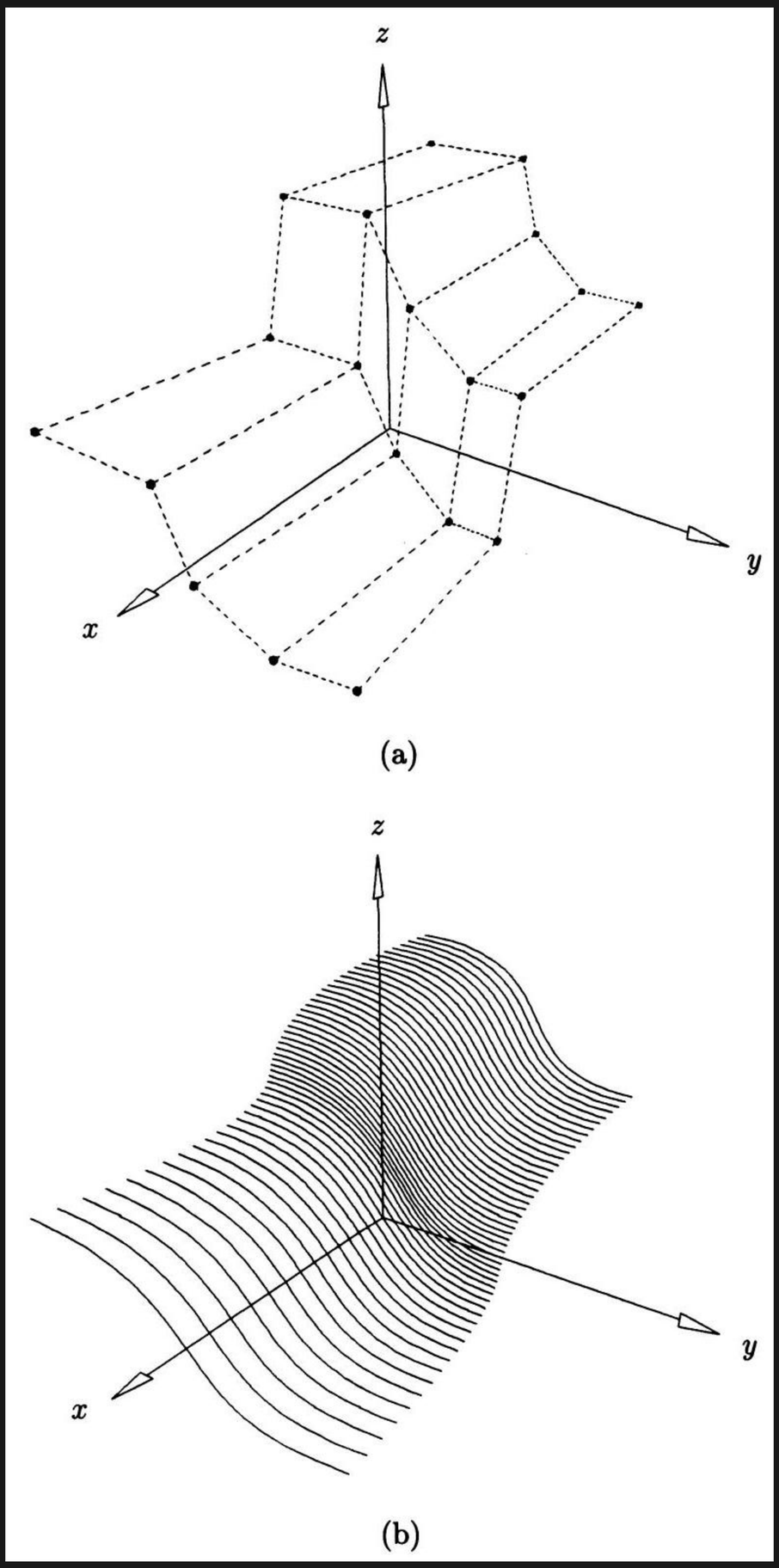


图 5.20. 要分解的 (立方 × 二次) 曲面。(a) 控制网；(b) 在 $U = \{0, 0, 0, 0, 3/5, 1, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 2/5, 1, 1, 1\}$ 上定义的曲面。

然后，当我们开始插入 u_b 时，结向量的形式为 (局部)

$$\ldots, \underbrace{u_{a-p+1} = \cdots = u_a}_{p}, \underbrace{u_{b-s+1} = \cdots = u_b}_{s} \tag{5.22}$$

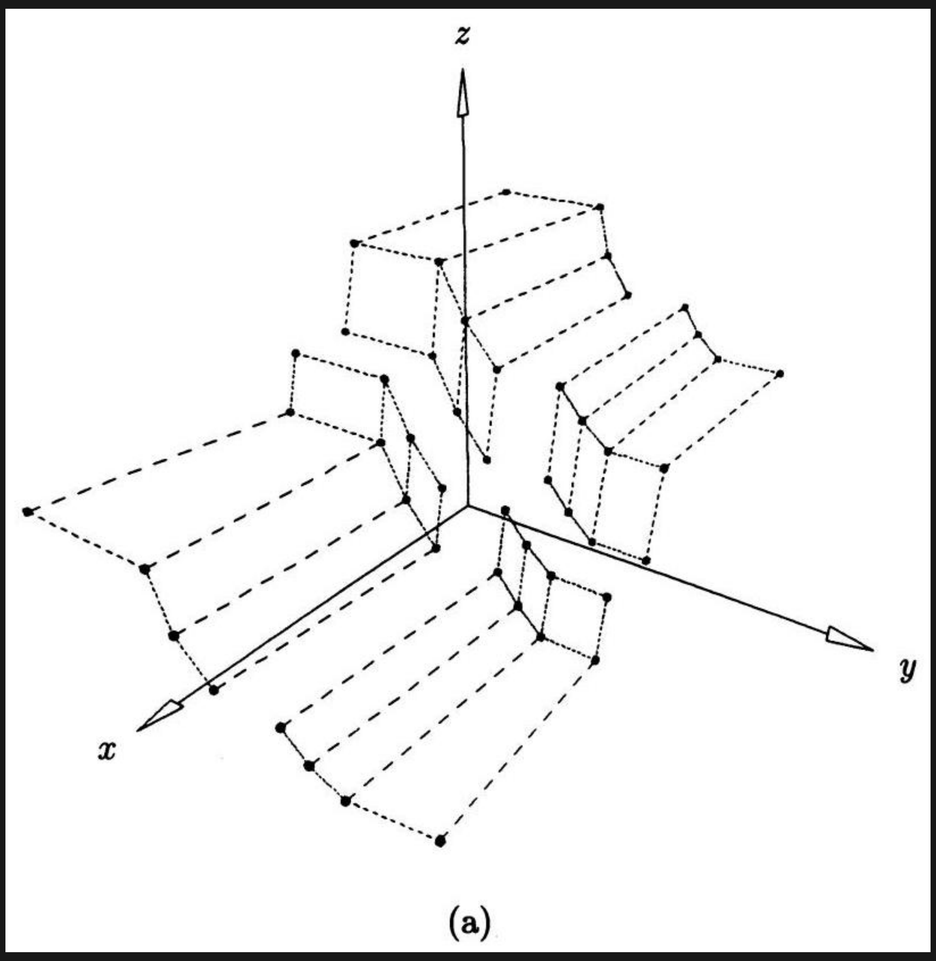


图 5.21. 通过结精细化进行的分段贝塞尔补丁分解。(a) 控制网；(b) 贝塞尔曲面补丁。
其中 s 是 u_b 的原始重数。根据公式 (5.15)，插入 $u_b p-s$ 次数更多会生成三角形表。

$$\alpha_{b-p+1,1}$$

$$\alpha_{b-p+2,2}$$

$$\alpha_{b-p+2,1}$$

$$\vdots$$

$$\cdots \alpha_{b-s,p-s}$$

$$\alpha_{b-s-1,1}$$

$$\alpha_{b-s,2}$$

$$\alpha_{b-s,1}$$

检查公式 (5.15) 和 (5.22) 显示了两个事实:

- $\alpha_{b-p+i,1}=\alpha_{b-p+i+1,2}=\cdots=\alpha_{b-s,p-s-i+1}$ 对于 $i=1,\ldots,p-s$ ；αs 在指向东南的对角线上是相等的；
- 公式 (5.15) 中的分子在整个 $\alpha_{i,j}$ 集合中保持不变，即 u_b-u_a 。

为了说明这一点，考虑图 5.23 中所示的例子。在提取第一个段后，结向量的形式为

$$U=\{0,0,0,0,0,1/5,1/5,1/5,1/5,2/5,3/5,4/5,1,1,1,1,1\}$$

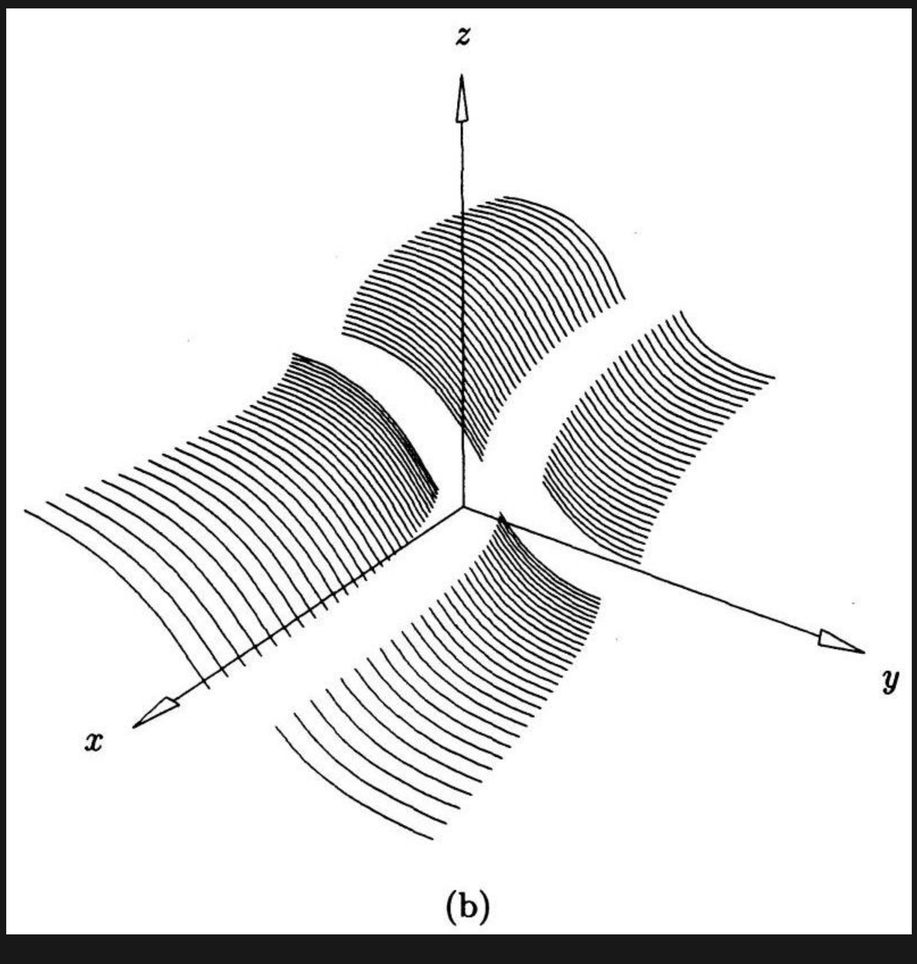


图 5.21. (续。)
当计算第二个段时，生成 $a = 8$ 和 $b = 9$ ，以及以下 αs 。

$$r = 1$$

$$r = 2$$

$$r = 3$$

$$\alpha_{6,1} = \frac{u_9 - u_6}{u_{10} - u_6} = \frac{0.2}{0.4}$$

$$\alpha_{7,2} = \frac{u_9 - u_7}{u_{10} - u_7} = \frac{0.2}{0.4}$$

$$\alpha_{7,1} = \frac{u_9 - u_7}{u_{11} - u_7} = \frac{0.2}{0.6}$$

$$\alpha_{8,3} = \frac{u_9 - u_8}{u_{10} - u_8} = \frac{0.2}{0.4}$$

$$\alpha_{8,2} = \frac{u_9 - u_8}{u_{11} - u_8} = \frac{0.2}{0.6}$$

$$\alpha_{8,1} = \frac{u_9 - u_8}{u_{12} - u_8} = \frac{0.2}{0.8}$$

注意，沿东南对角线的 α 分别为 $0.2/0.4, 0.2/0.6$ 和 $0.2/0.8$ ，而分子保持为 0.2。
算法 A5.6 将 NURBS 曲线分解并返回 nb 个 Bézier 段。Qw [j] [k] 是第 k 个控制点，属于第 j 个段。局部数组 alphas [] 包含 alphas，其索引偏移以从 0 开始。
ALGORITHM A5.6

```
DecomposeCurve(n, p, U, Pw, nb, Qw)
    { /* Decompose curve into Bézier segments */
```

174 基本几何算法插入的节点；(c) 插入第二个节点；(d) 为第二个段做准备。

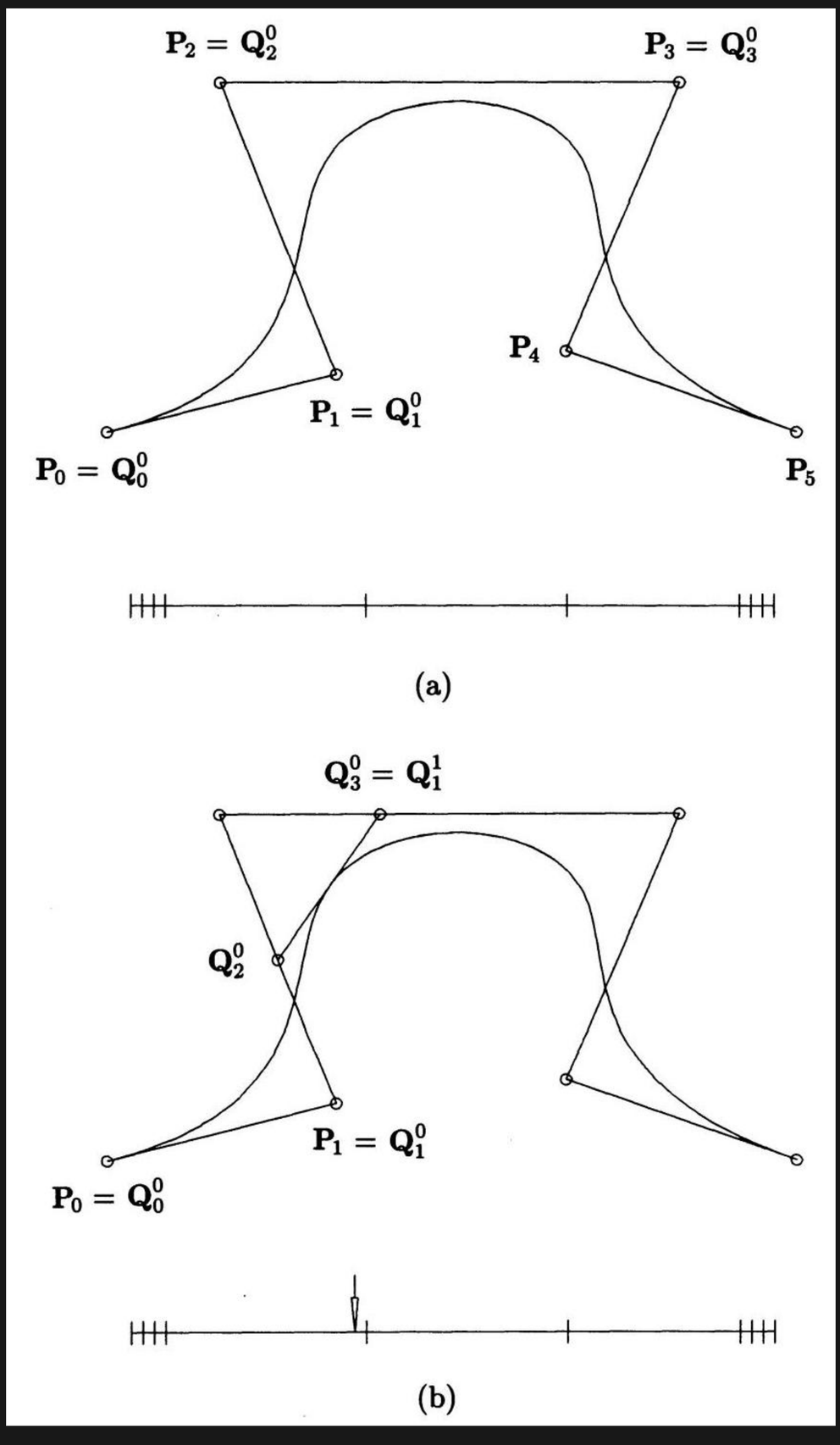


图 5.22. 动态曲线分解。(a) 原始三次曲线；(b) 第一个

```
    /* Input: n, p, U, Pw */
    /* Output: nb, Qw */
m = n+p+1;
a = p;
```

```
b = p+1;
nb = 0 ;
for (i=0; i<=p; i++) Qw[nb][i] = Pw[i];
```

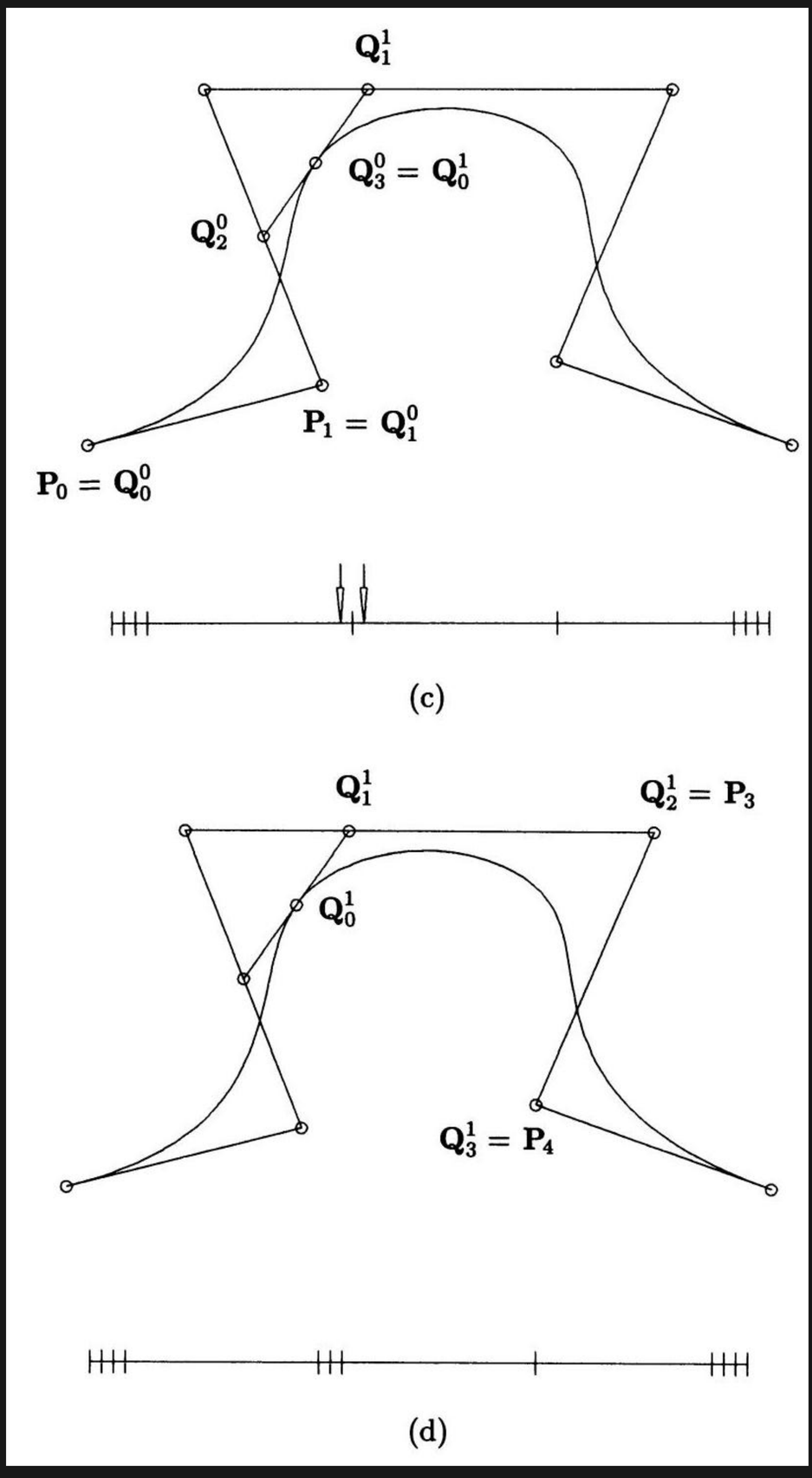


图 5.22. (续。)

```
while (b < m)
   {
   i = b;
   while (b < m && U[b+1] == U[b]) b++;
   mult = b-i+1;
   if (mult < p)
      {
      numer = U [b] – U [a] ; /* Numerator of alpha * /
              /* Compute and store alphas */
         for (j=p; j>mult; j–)
            alphas[j-mult-1] = numer/(U[a+j]-U[a]);
         r = p-mult; /* Insert knot r times */
         for (j=1; j<=r j++)
            {
            save = r – j ;
            s = mult + j; /* This many new points */
            for (k=p; k>=s; k–)
               {
               alpha = alphas[k-s];
               Qw[nb][k] = alpha*Qw[nb][k] + (1.0-alpha)*Qw[nb][k-1];
               }
            if (b < m)  /* Control point of * /
               Qw[nb+1][save] = Qw[nb][p]; /* next segment */
            }
         }
   nb = nb + 1; /* Bézier segment completed * /
   if (b < m)
         { /* Initialize for next segment */
         for (i=p-mult; i<=p; i++) Qw[nb][i] = Pw[b-p+i];
         a = b;
         b = b+1;
         }
```

```
    }
}
```

算法 A5.6 是 NURBS 系统与显示 Bézier 曲线的系统 (硬件或软件) 之间的理想接口。经过修改，算法 A5.6 动态计算 Bézier 段，将每个段传递以供显示，并用下一个段覆盖每个段。没有返回任何内容。这些修改包括:

- 从参数列表中移除 nb 和 Qw ；
- 使用局部数组 Qw[] (长度 $p+1$) 和 NextQw[] (长度 $p-1$)；NextQw 用于存储下一个段的最左控制点；
- 在计算完一个段后 (参见代码中的注释，/* Bézier 段已完成 */)，将其传递以供显示；
- 在显示一个段后，用 NextQw[] 和 Pw[] 中的点覆盖它。

示例见图 5.24a-5.24c；更多细节请参见 [Pieg91b]。

算法 A5.7 显示了表面分解的组织。该例程计算一个 Bézier 条带，即在一个方向上是 Bézier 而在另一个方向上是 B-spline 的 NURBS 表面。该例程必须被调用两次，第一次在 u 方向上获取 Bézier 条带，然后条带必须在 v 方向上输入到例程中以获取 Bézier 补丁。

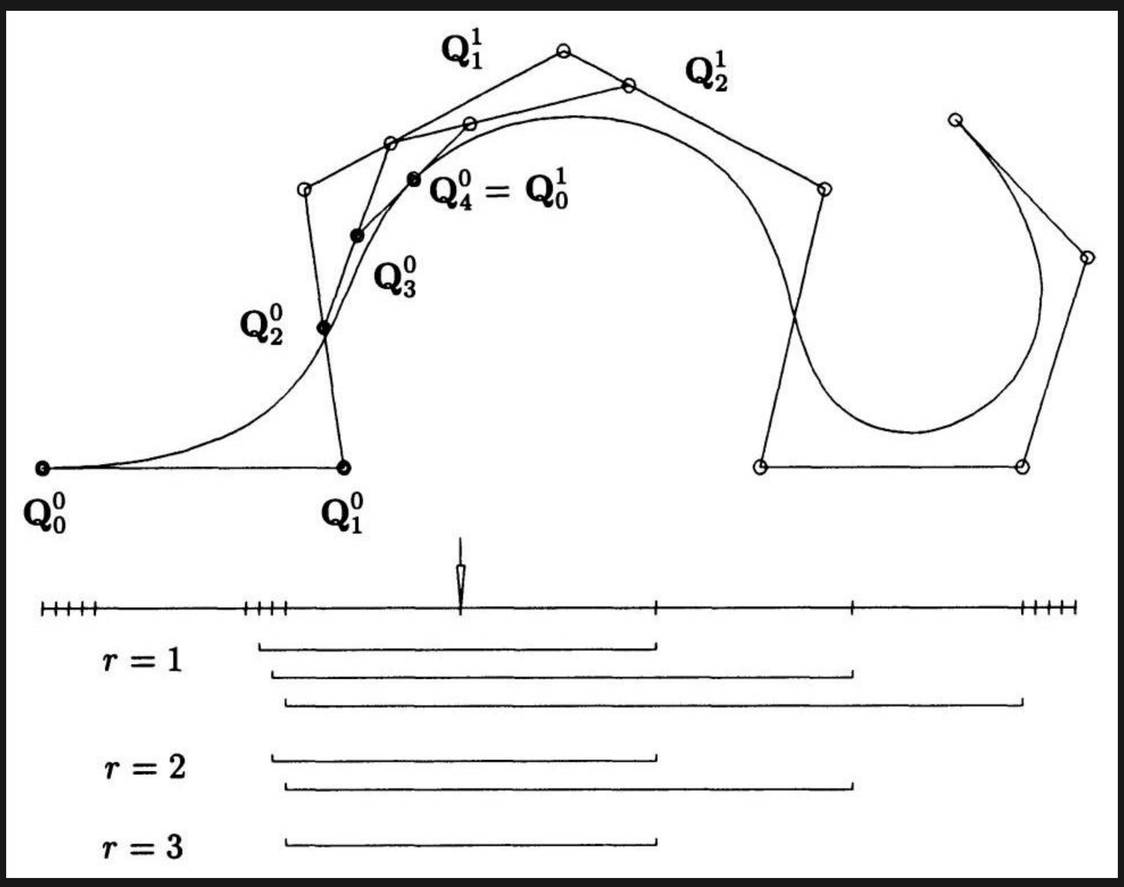


图 5.23. 用于计算四次曲线第二个 Bézier 段的 alpha 比例。

ALGORITHM A5.7

```
DecomposeSurface (n, p, U, m, q, V, Pw, dir, nb, Qw)
  { /* Decompose surface into Bézier patches */
    /* Input: n, p, U, m, q, V, Pw, dir */
    /* Output: nb, Qw */
    if (dir == U_DIRECTION)
```

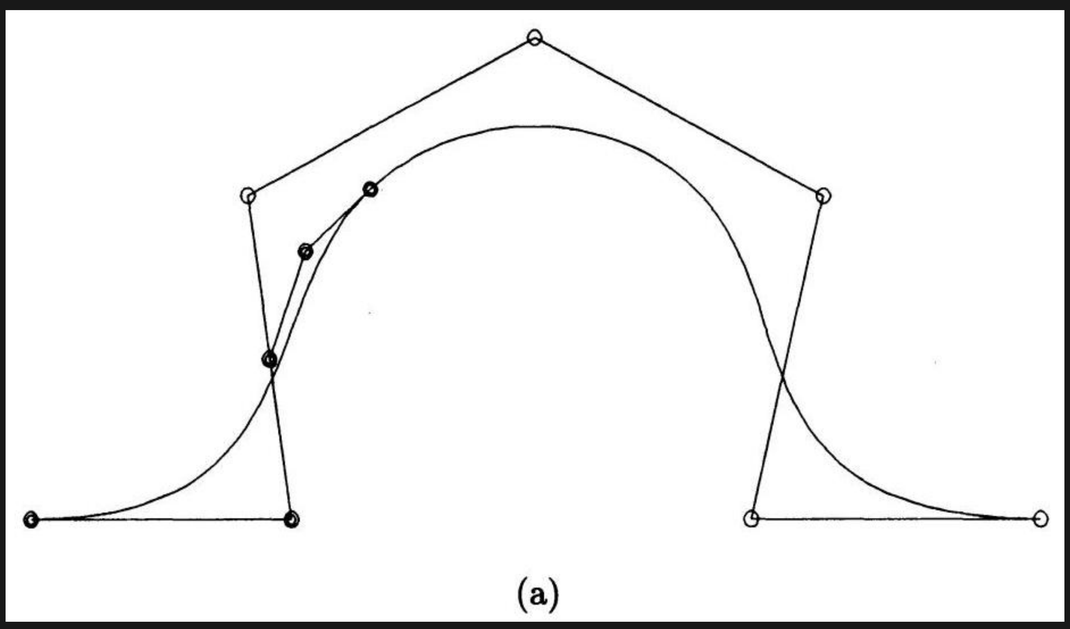


图 5.24. 四次曲线的即时分解。(a) 第一个 Bézier 段；(b) 第二个 Bézier 段；(c) 第三个 Bézier 段。

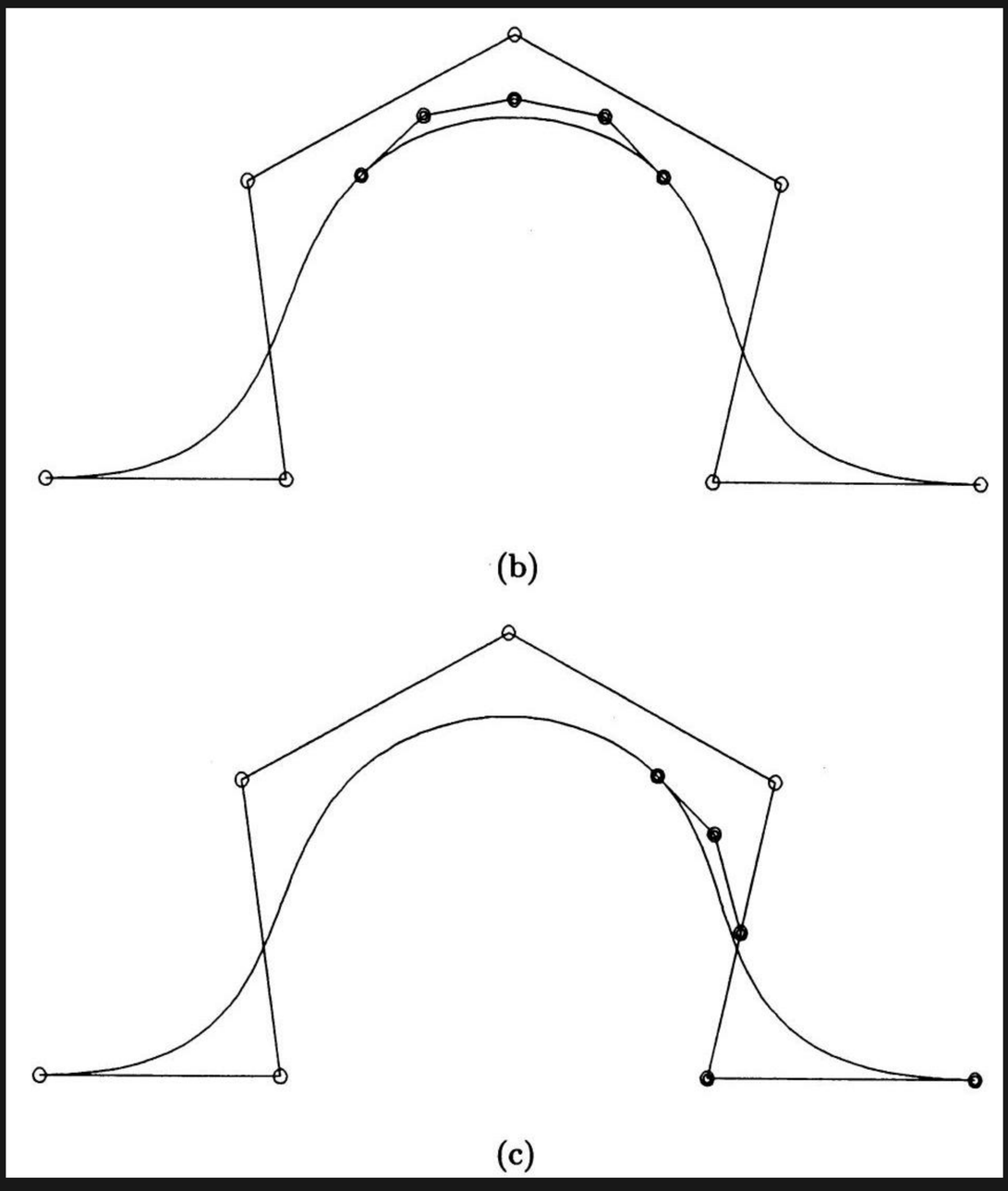


图 5.24.(续。)

```
{
a = p; b = p + 1;
nb=0;
for (i = 0; i<=p; i++)
    for (row = 0; row <=m; row ++)
        Qw[nb][i][row] = Pw[i][row];
while (b < m)
    {
    get mult;
    if ( mult<p )
      {
      get the numerator and the alfas;
      for (j=1; j<=p-mult; j++)
        {
        save= ...;
        s = ...;
                        for(k=p; k>=s; k--)
                            {
                            get alfa;
                            for ( row ... )
                              Qw[nb][k][row] = alfa*Qw[nb][k][row]

                                                                  +(1.0-alfa)*Qw[nb][k-1][row];

                            }
                        if ( b<m )
                            for ( row ... )
                              Qw[nb+1][save][row] = Qw[nb][p][row];
                        }
                    }
                nb=nb+1;
                if ( b<m )
                  {
                  for (i=p-mult; i<=p; i++)
                    for ( row ... )
                        Qw[nb][i][row] = Pw[b-p+i][row];
                  a = b; b = b + 1;
                  }
              }
          }
        if (dir == V_DIRECTION)
          {
          /* Similar code as above with u - and v -directional
            parameters switched */
          }
        }
```

An example is shown in Figure 5.25.

4　节点移除

节点移除是节点插入的反向过程。设

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w \tag{5.23}$$

在 U 上定义，并且设 u_r 是 U 中重数为 s 的内部节点；端节点不被移除。设 U_t 表示通过从 U $(1 \le t \le s)$ 中移除 u_r t 次得到的节点向量。我们说 u_r 是 t 次可移除的，如果 $\mathbf{C}^w(u)$ 具有如下形式的精确表示

$$\mathbf{C}^w(u) = \sum_{i=0}^{n-t} \bar{N}_{i,p}(u)\,\mathbf{Q}_i^w \tag{5.24}$$

其中 $\bar{N}_{i,p}(u)$ 是 U_t 上的基函数，即，方程 (5.23) 和 (5.24) 在几何上和参数上表示相同的曲线。

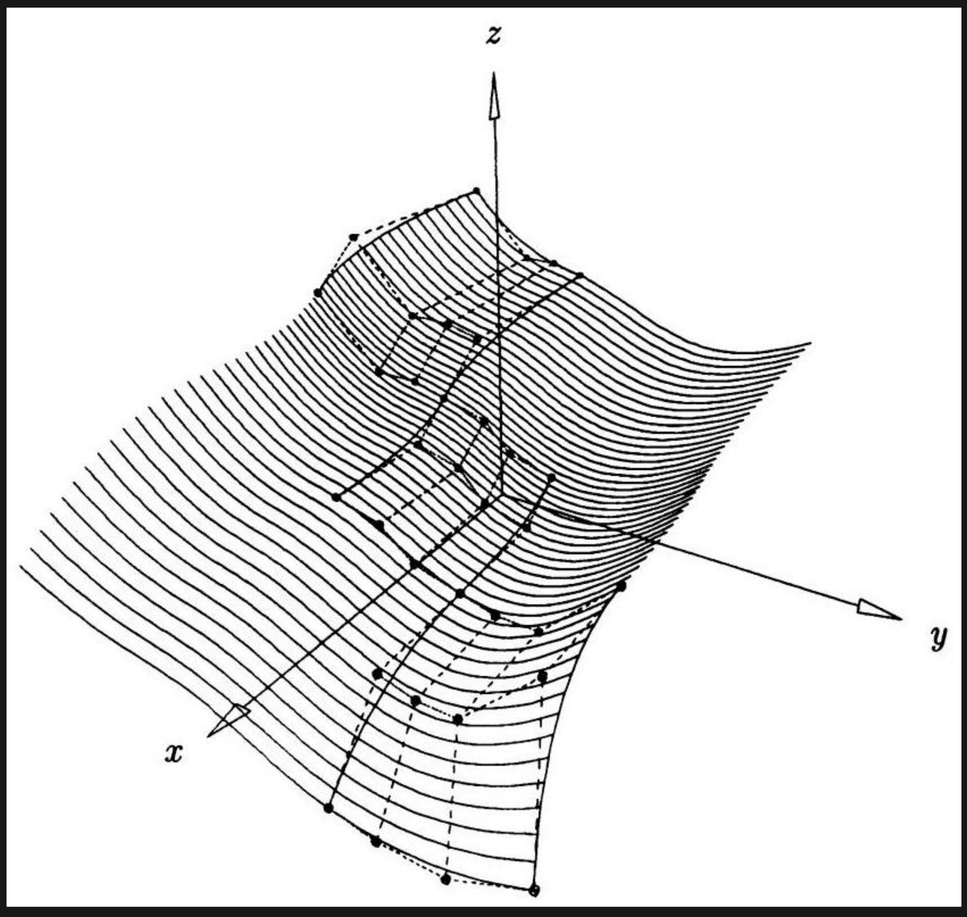


图 5.25. 即时表面分解：显示了三个 Bézier 补丁。

从前面的章节我们知道，基函数 $N_{i,p}(u)$ 在 u_r 处非零的只有 C^{p-s} 是连续的。此外，尽管一般不期望曲线有超过 C^{p-s} 的连续性，但控制点 $\mathbf{P}_i^w$ 可以位于某些位置，使得 $(p-s+1)$ 次 (或更高) 导数是连续的。因此，节点 u_r 是 t 次可去除的，当且仅当曲线 $\mathbf{C}^w(u)$ 在 $u=u_r$ 处是 C^{p-s+t} 连续的。重要的是要注意，连续性必须是相对于 $\mathbf{C}^w(u)$ 的，而不是其投影 $\mathbf{C}(u)$，后者即使 $\mathbf{C}^w(u)$ 不连续也可以是连续的。

节点去除算法必须做两件事:

- 确定一个节点是否可去除以及可去除多少次；
- 计算新的控制点 $\mathbf{Q}_i^w$ 。

详细信息可以在 [Till92] 中找到。

节点去除在多个应用中是一个重要的工具:

- 将以幂基形式表示的样条曲线或曲面转换为 B 样条形式的标准方法是:
- 将段 (补丁) 转换为贝塞尔形式；
- 通过将贝塞尔段拼接在一起并使用所有内部节点的重数等于次数的节点向量来获得 B 样条表示；
- 去除尽可能多的节点 (因此控制点)，以允许样条曲线 (曲面) 的连续性；
- 在交互式塑造 B 样条曲线和曲面时，有时会添加节点以增加可修改的控制点数量。当控制点被移动时，节点处的连续性水平可能会改变 (增加或减少)；因此，在修改完成后，可以调用节点去除以获得曲线或曲面的最紧凑表示；
- 有时将 B-spline 曲线连接在一起以形成复合曲线是有用的。第一步是使曲线兼容，即具有相同的阶数，并且第 i 条曲线的结束参数值等于第 $(i+1)$ 条曲线的起始参数值。一旦完成这一点，组合就通过使用内部结点，其重数等于曲线的共同阶数来实现。然后可以调用结点移除，以去除不必要的结点。

我们通过一个例子描述结点移除过程。像往常一样，算法在控制点的四个坐标上操作，但为了符号的方便，我们省略 w 上标。考虑图 5.26 中的三次曲线。假设原始曲线由 $\{\mathbf{P}_0^0,\ldots,\mathbf{P}_6^0\}$ 和 $U=\{u_0,\ldots,u_{10}\}$ 定义，其中控制点上的上标表示结点移除过程中的步骤编号，结点为 $u_0=\cdots=u_3=0$ 、$u_4=u_5=u_6=1$ 和 $u_7=\cdots=u_{10}=2$ 。考虑移除 $u=1\,(=u_6)$ 。

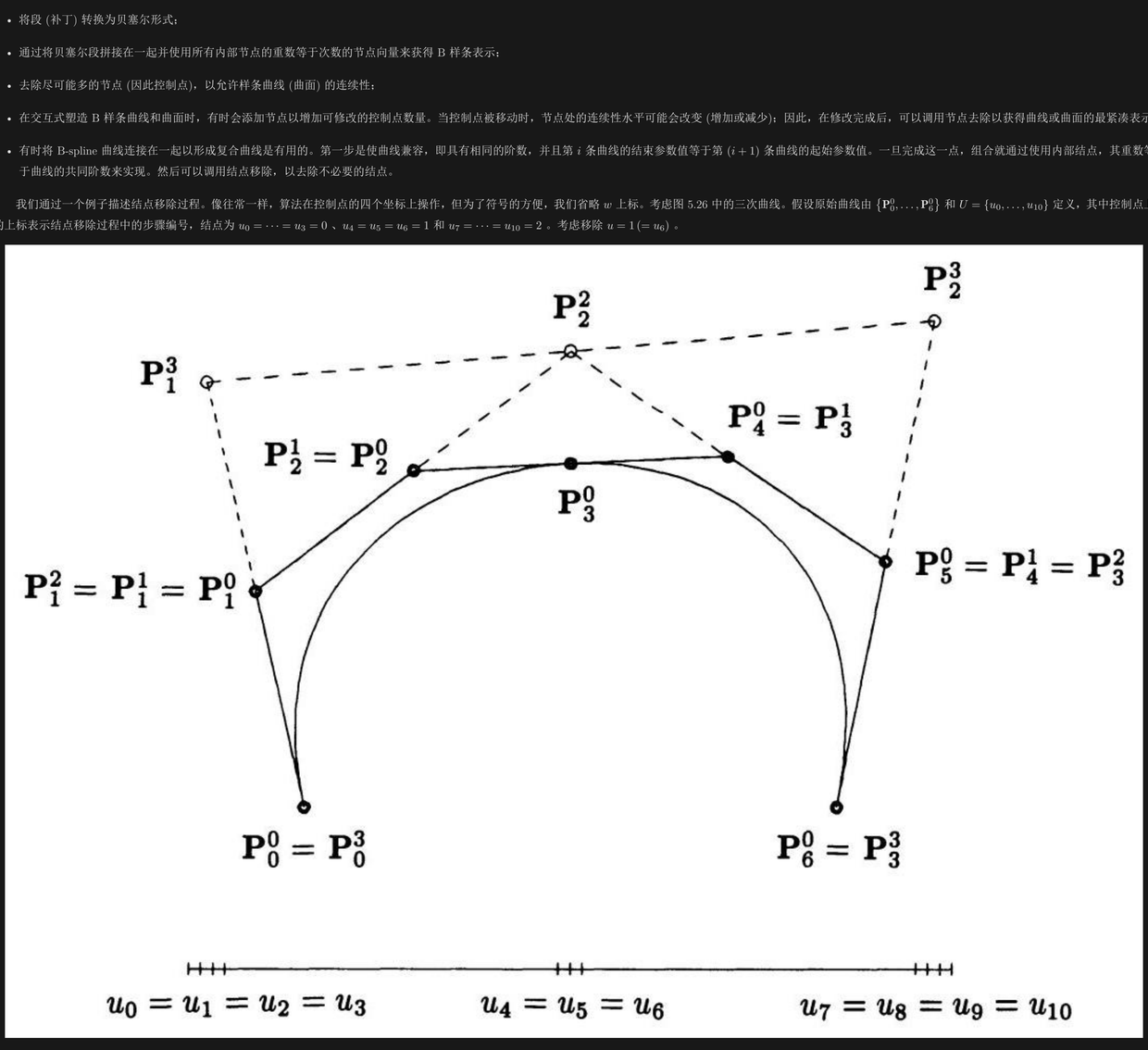


图 5.26. 从具有三重结点的三次曲线中移除结点。

当且仅当一阶导数连续时

$$\frac{p}{u_6-u_3}\left(\mathbf{P}_3^0-\mathbf{P}_2^0\right)=\frac{p}{u_7-u_4}\left(\mathbf{P}_4^0-\mathbf{P}_3^0\right)$$

设置 $u = u_4 = u_6$ 并重新排列项得到

$$\mathbf{P}_3^0 = \frac{u - u_3}{u_7 - u_3}\mathbf{P}_4^0 + \frac{u_7 - u}{u_7 - u_3}\mathbf{P}_2^0$$

使用 $\mathbf{P}_2^0 = \mathbf{P}_2^1$ 和 $\mathbf{P}_4^0 = \mathbf{P}_3^1$ 得到

$$\mathbf{P}_3^0 = \alpha_3\mathbf{P}_3^1 + (1-\alpha_3)\,\mathbf{P}_2^1\;\alpha_3 = \frac{u - u_3}{u_7 - u_3} \tag{5.25}$$

方程 (5.25) 只是将 $u = 1$ 插入具有 1 作为双重结点的结点向量的方程 (在该结点向量中 u_7 是 u_6)。此外，当且仅当结点 $u = 1$ 可以第二次移除时，二阶导数在 $u = 1$ 处是连续的。它可以在

$$\mathbf{P}_2^1 = \alpha_2\mathbf{P}_2^2 + (1-\alpha_2)\,\mathbf{P}_1^2$$

$$\mathbf{P}_3^1 = \alpha_3\mathbf{P}_3^2 + (1-\alpha_3)\,\mathbf{P}_2^2 \tag{5.26}$$

与

$$\alpha_i = \frac{u - u_i}{u_{i+p+2} - u_i}\;i = 2, 3$$

最后，当且仅当 $u = 1$ 可以第三次移除时，三阶导数是连续的，这意味着

$$\mathbf{P}_1^2 = \alpha_1\mathbf{P}_1^3 + (1-\alpha_1)\,\mathbf{P}_0^3$$

$$\mathbf{P}_2^2 = \alpha_2\mathbf{P}_2^3 + (1-\alpha_2)\,\mathbf{P}_1^3$$

$$\mathbf{P}_3^2 = \alpha_3\mathbf{P}_3^3 + (1-\alpha_3)\,\mathbf{P}_2^3 \tag{5.27}$$

与

$$\alpha_i = \frac{u - u_i}{u_{i+p+3} - u_i}\;i = 1, 2, 3$$

设 $n = p - s + 1$ ，其中 s 是要移除的结点的重数。然后注意到

- 在方程 (5.25) 中没有未知数，$\mathbf{P}_2^2$ 是方程 (5.26) 中唯一的未知数，而 $\mathbf{P}_1^3$ 和 $\mathbf{P}_2^3$ 是方程 (5.27) 中的未知数。去结产生了 n 个方程和 $n-1$ 个未知数；
- 去结破坏了 n 控制点，并用 $n-1$ 个新控制点替换它们。

让我们求解方程 (5.25) 到 (5.27)。对于方程 (5.25)，我们计算单个方程的右侧，并检查它是否等于 $\mathbf{P}_3^0$ (在一个有意义的容差范围内)。如果不相等，则无法去结；如果相等，则去掉结并移除 $\mathbf{P}_3^0$ 。
从方程 (5.26) 中的两个方程我们得到

$$\mathbf{P}_2^2 = \frac{\mathbf{P}_2^1 - (1-\alpha_2)\,\mathbf{P}_1^2}{\alpha_2}\;\mathbf{P}_2^2 = \frac{\mathbf{P}_3^1 - \alpha_3\mathbf{P}_3^2}{1-\alpha_3}$$

如果这两个 $\mathbf{P}_2^2$ 的值在容差范围内，则可以去结，并将 $\mathbf{P}_2^1$ 和 $\mathbf{P}_3^1$ 替换为 $\mathbf{P}_2^2$ 。
求解方程 (5.27) 的第一和第三个方程，我们得到

$$\mathbf{P}_1^3 = \frac{\mathbf{P}_1^2 - (1-\alpha_1)\,\mathbf{P}_0^3}{\alpha_1}\;\mathbf{P}_2^3 = \frac{\mathbf{P}_3^2 - \alpha_3\mathbf{P}_3^3}{1-\alpha_3}$$

然后将 $\mathbf{P}_1^3$ 和 $\mathbf{P}_2^3$ 代入方程 (5.27) 的第二个方程的右侧，并检查它是否在 $\mathbf{P}_2^2$ 的容差范围内。如果是，则去掉结并用 $\mathbf{P}_1^3$ 和 $\mathbf{P}_2^3$ 替换 $\mathbf{P}_1^2, \mathbf{P}_2^2$ 和 $\mathbf{P}_3^2$ 。
一般来说，解决方程组的方法是从第一个和最后一个方程开始，向内推进，计算新的控制点。如果方程的数量是偶数 (如方程 [5.26])，则最终的新控制点会被计算两次。如果控制点的两个值在容差范围内，则可以移除节点。如果方程的数量是奇数 (如方程 [5.25] 和 [5.27])，则所有新的控制点只计算一次，最后计算的两个控制点被代入中间方程。如果结果在中间方程左侧但控制点的容差范围内，则该节点是可移除的。
现在我们给出一般公式。设 $u = u_r \neq u_{r+1}$ 为重数为 s 的节点，其中 $1 \le s \le p$ 。计算移除 u 的新控制点的方程为

$$\mathbf{P}_i^1 = \frac{\mathbf{P}_i^0 - (1-\alpha_i)\,\mathbf{P}_{i-1}^1}{\alpha_i}\;r - p \le i \le \frac{1}{2}(2r - p - s - 1)$$

$$\mathbf{P}_j^1 = \frac{\mathbf{P}_j^0 - \alpha_j\mathbf{P}_{j+1}^1}{(1-\alpha_j)}\;\frac{1}{2}(2r - p - s + 2) \le j \le r - s \tag{5.28}$$

其中

$$\alpha_k = \frac{u - u_k}{u_{k+p+1} - u_k}\;k = i, j$$

编程方程 (5.28) 的最简单方法是

$$i = r - p; j = r - s;$$

$$\text{while } (j - i > 0)$$

$$\alpha_i = \cdots$$

$$\alpha_j = \cdots$$

$$\mathbf{P}_i^1 = \cdots$$

$$\mathbf{P}_j^1 = \cdots$$

$$i = i + 1; j = j - 1;$$

现在假设我们想要多次移除 $u = u_r$ 。每次移除节点时，s 和 r 会在方程 (5.28) 中递减，控制点的上标会递增。因此，移除 $u = u_r$ 第 t 次的方程为

$$\mathbf{P}_i^t = \frac{\mathbf{P}_i^{t-1} - (1-\alpha_i)\,\mathbf{P}_{i-1}^t}{\alpha_i}\;r - p - t + 1 \le i \le \frac{1}{2}(2r - p - s - t)$$

$$\mathbf{P}_j^t = \frac{\mathbf{P}_j^{t-1} - \alpha_j\mathbf{P}_{j+1}^t}{(1-\alpha_j)}\;\frac{1}{2}(2r - p - s + t + 1) \le j \le r - s + t - 1$$

(5.29)
其中

$$\alpha_i = \frac{u - u_i}{u_{i+p+t} - u_i}\;\alpha_j = \frac{u - u_{j-t+1}}{u_{j+p+1} - u_{j-t+1}}$$

假设 u 要被移除 k 次。方程 (5.29) 可以编程为

$$\text{first } = r - p + 1;\ \text{last } = r - s - 1\ ;$$

$$\text{for } t = 1, \ldots, k$$

$$\text{first } = \text{ first -1 ; last } = \text{ last +1 ;}$$

$$i = \text{ first; } j = \text{ last;}$$

$$\text{while } (j - i > t - 1)$$

$$\alpha_i = \cdots$$

$$\alpha_j = \cdots$$

$$\mathbf{P}_i^t = \cdots$$

$$\mathbf{P}_j^t = \cdots$$

$$i = i + 1; j = j - 1;$$

使结去除实现复杂化的因素在于，通常无法提前知道一个结是否可以去除，以及如果可以去除，能够去除多少次。必须在临时存储中计算 (潜在的) 新的控制点，并且无法提前知道输出中将有多少个结和控制点。算法 A5.8 尝试去除结 $u = u_r$ num 次，其中 $1 \le \text{num} \le s$ 。它返回 t ，即实际去除结的次数，如果 $t > 0$ ，则返回新的结向量和控制点。它在原地计算新的控制点，覆盖旧的控制点。每一步只需要一个大小为 $2p + 1$ 的局部数组 (temp) 来计算新的控制点。如果在步骤 t 的去除失败，则步骤 $t - 1$ 的控制点仍然完好无损。最后，结和控制点向下移动以填补去除留下的空隙。进行最少次数的移动。为了检查重合点，使用一个值 TOL 和一个函数 Distance4D( )，该函数计算点之间的距离。如果曲线是有理的，我们假设采用齐次表示；在这种情况下，Distance4D( ) 计算四维距离。权重被视为普通坐标。TOL 的含义 [Till92]

- 如果曲线是非有理的 (所有 $w_i = 1$)，那么一次结去除会导致曲线在整个参数域内与原始曲线的偏差小于 TOL；

- 如果曲线是有理的，那么偏差在任何地方都小于

$$\frac{\text{TOL}\left(1+|\mathbf{P}|_{\max}\right)}{w_{\min}}$$

在这里 $w_{\min}$ 是原始曲线上的最小权重，而 $|\mathbf{P}|_{\max}$ 是原始曲线上的任何三维点到原点的最大距离。B 样条的凸包性质用于计算 $w_{\min}$ 和 $|\mathbf{P}|_{\max}$ 的界限。如果期望的偏差界限是 d ，那么 TOL 应设置为

$$\text{TOL} = \frac{dw_{\min}}{1+|\mathbf{P}|_{\max}} \tag{5.30}$$

ALGORITHM A5.8

```
RemoveCurveKnot (n, p, U, Pw, u, r, s, num, t)
    { /* Remove knot u (index r) num times. */
        /* Input: n, p, U, Pw, u, r, s, num */
        /* Output: t, new knots & ctrl pts in U & Pw */
  m = n + p + 1 ;
  ord = p+1;
  fout = (2 * r - s - p) /2; /* First control point out */
  last = r-s;
  first = r-p;
  for (t = 0; t < num; t++)
      { /* This loop is Eq. (5.28) */
      off = first-1; /* Diff in index between temp and P */
      temp[0] = Pw[off]; temp[last+1-off] = Pw[last+1];
      i = first; j=last;
      ii = 1; j = last-off;
      remflag = 0 ;
      while (j - i > t)
          { /* Compute new control points for one removal step */
          alfi = (u-U[i])/(U[i+ord+t]-U[i]);
          alfj = (u-U[j-t])/(U[j+ord]-U[j-t]);
          temp[ii] = (Pw[i]-(1.0-alfi)*temp[ii-1])/alfi;
          temp[jj] = (Pw[j]-alfj*temp[jj+1])/(1.0-alfj);
          i = i+1; ii = ii+1;
          j = j-1; jj = jj-1;
          } /* End of while-loop */
      if (j - i < t) /* Check if knot removable */
          {
          if (Distance4D(temp[ii-1], temp[jj+1]) <= TOL)
              remflag = 1 ;
    }
    else
      {
      alfi = (u-U[i])/(U[i+ord+t]-U[i]);
      if (Distance4D(Pw[i], alfi*temp[ii+t+1]
                                                  +(1.0-alfi)*temp[ii-1]) <= TOL)
        remflag = 1 ;
      }
    if (remflag == 0 ) /* Cannot remove any more knots */
      break; /* Get out of for-loop */
    else
      { /* Successful removal. Save new cont.pts. */
      i = first; j = last;
      while (j - i > t)
        {
        Pw[i] = temp[i-off]; Pw[j] = temp[j-off];
        i = i+1; j = j-1;
        }
      }
  first = first-1; last = last+1;
  } /* End of for-loop */
if (t == 0) return;
for (k=r+1; k<=m; k++) U[k-t] = U[k]; /* Shift knots */
j = fout;  i = j; /* Pj thru Pi will be overwritten */
for (k=1; k<t; k++)
  if (Mod (k, 2) == 1)  /* k modulo 2 */
      i = i+1; else j = j-1;
for (k=i+1; k<=n; k++) /* Shift */
    { Pw [j] = Pw [k] ; j = j + 1; }
return;
}
```

表 5.2 显示了图 5.26 中给出的三次曲线示例的控制点数组如何变化。第一行显示了进入 for 循环 ($0 \le t <$ num) 之前的内容。行 2 – 4 显示了 for 循环底部的数组 (仅变化) 对于 $t = 0, t = 1$ 和 $t = 2$ 。' X ' 表示未使用的数组元素。

我们注意到算法 A5.8 可以创建负权重或零权重。从理论上讲这是正确的，但从软件的角度来看可能是不理想的。可以通过在距离计算后、设置 remflag 为 1 之前插入一个检查来避免这种情况。

设 $\mathbf{S}^w(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u)\, N_{j,q}(v)\, \mathbf{P}_{i,j}^w$ 为 NURBS 曲面。通过将节点移除算法 (算法 A5.8) 应用于控制点的 $m+1$ 列 ($n+1$ rows) ，可以从 $\mathbf{S}^w(u,v)$ 中移除一个 u 节点 (v 节点)。但只有在所有 $m+1$ 列 ($n+1$ 行) 成功移除的情况下，节点才能被移除。

在 [Till92] 中给出了两个额外的节点移除算法。这些是:

- 给定一条曲线，尽可能移除更多的节点；

表 5.2. 图 5.15 的三次曲线的控制点数组。

max width=

$\mathbf{P}_0^0$	$\mathbf{P}_1^0$	$\mathbf{P}_2^0$	$\mathbf{P}_3^0$	$\mathbf{P}_4^0$	$\mathbf{P}_5^0$	$\mathbf{P}_6^0$	循环之前
			X				$t=0$
		$\mathbf{P}_2^2$	X	$\mathbf{P}_2^2$			$t=1$
	$\mathbf{P}_1^3$	X	X	X	$\mathbf{P}_2^3$		$t=2$

- 给定一个曲面，尽可能移除更多的 u 节点 (v 节点是类似的)。

我们通过各种示例说明曲线和曲面结点移除的效果，而不是详细介绍这些相当复杂的算法。图 5.27a 显示了一个具有单个、双重和三重结点的曲线。在图 5.27 b 中，单个结点 $u = 3/10$ 被移除，而在图 5.27c – 5.27e 中，分别移除了一个、两个和三个三重结点 $u = 1/2$ 。

曲面结点移除的示例显示在图 5.28a – 5.28 d 中。图 5.28a 显示了原始曲面。在图 5.28 b 中，三重结点 $u = 3/10$ 被移除三次。在图 5.28c 中，单个结点 $v = 1/4$ 被移除，而在图 5.28d 中，两个结点同时被移除。在图 5.27b-5.27d 以及图 5.28 b – 5.28 d 中，原始曲线/曲面以虚线表示，而移除结点的曲线/曲面以实线表示。

图 5.29a 中的曲线移除了所有可移除的结点。图 5.29 b – 5.29f 中显示的曲线分别使用容差值 0.007、0.025、0.07、0.6 和 1.2 计算得出。这些图示说明了有多少个结点是可移除的 (被移除的结点用圆圈标出，原始多边形和曲线以虚线表示)；如何计算新的控制点。

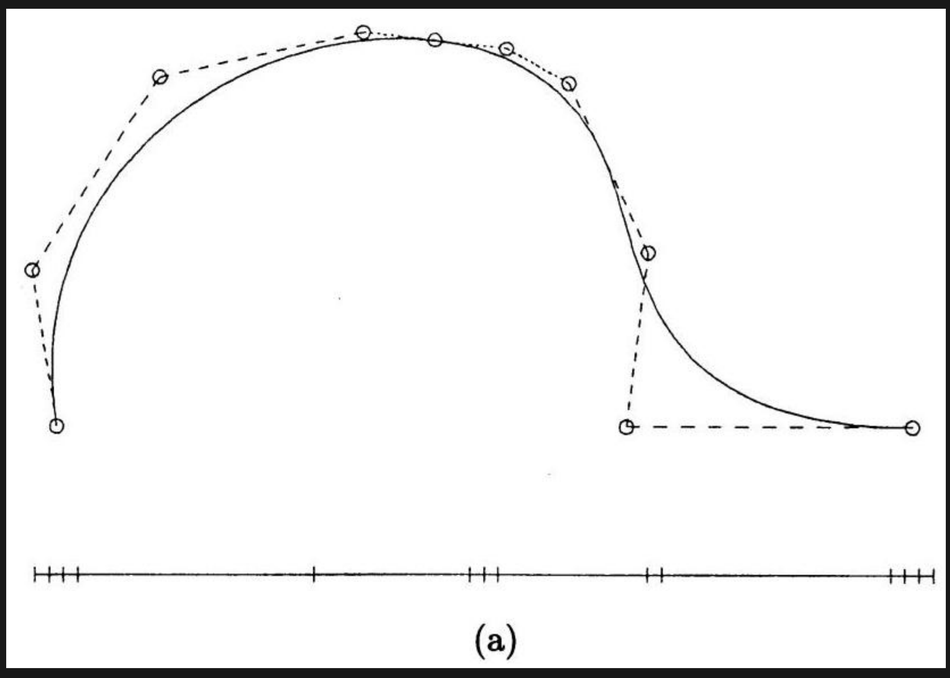


图 5.27. 曲线结点移除示例。(a) 定义在结点向量 $\{0,0,0,0,3/10,1/2,1/2,1/2,7/10,7/10,1,1,1,1\}$ 上的原始三次曲线；(b) 移除 3/10，一次；(c) 移除 1/2 一次；(d) 移除 1/2 两次；(e) 移除 1/2 三次。

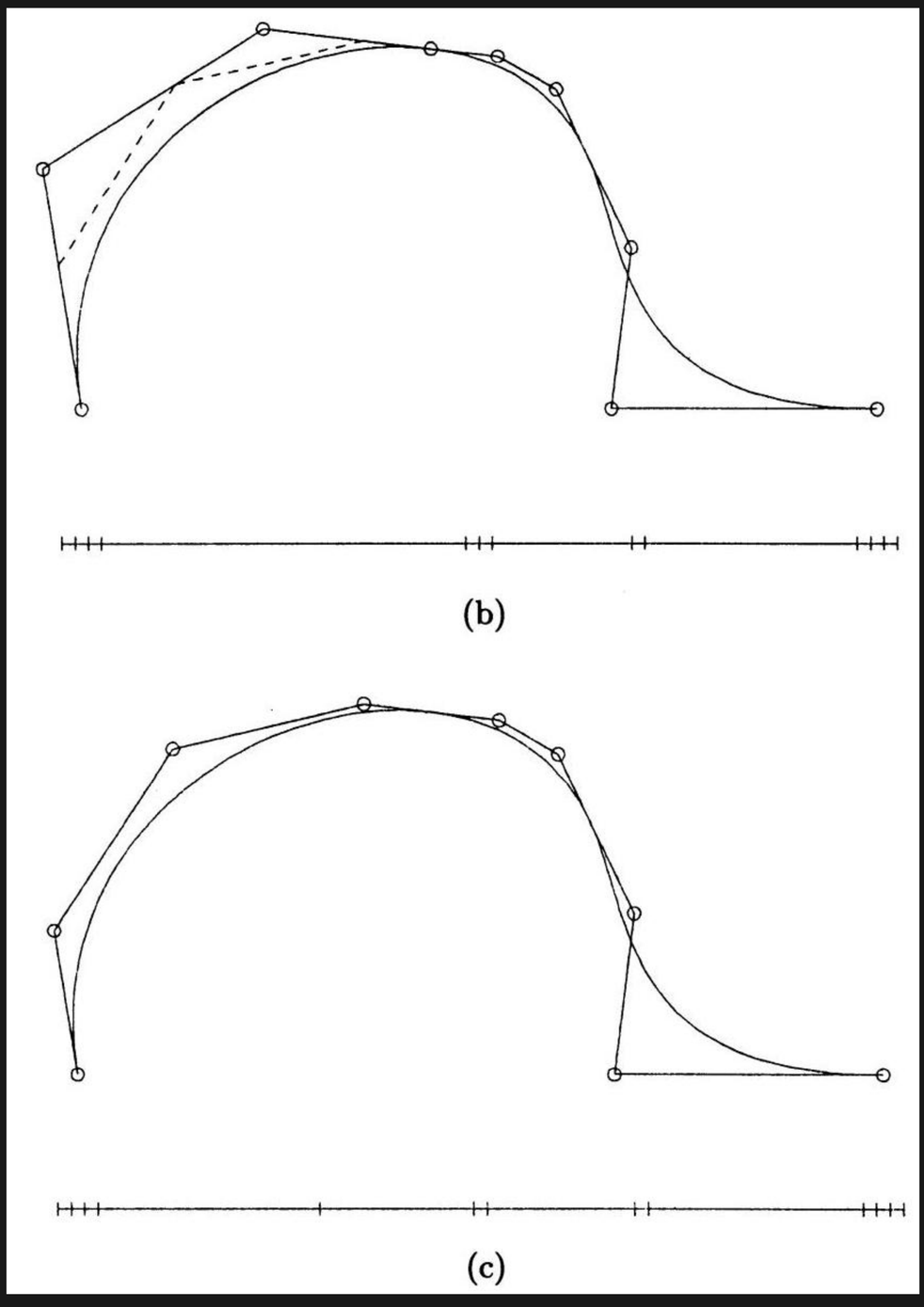


图 5.27. (续。)

点；以及原始曲线与去结点曲线之间的偏差。图 5.30a 和 5.30b 至图 5.34a 和 5.34b 展示了类似的示例。原始曲面的控制网如图 5.30a 所示。原始曲面如图 5.30b 所示。去结点曲面在图 5.31a 和 5.31 b 至图 5.34a 和 5.34b 中进行了说明，使用的公差分别为 0.05、0.1、0.3 和 0.5。

5 次数提升

设 $\mathbf{C}_n(u)=\sum_{i=0}^{n}\mathbf{a}_i u^i$ 为一个 n 次多项式曲线。显然，我们可以设定 $\mathbf{a}_{n+1}=0$ 并将 $\mathbf{C}_n(u)$ 写成一个 $(n+1)$ 次曲线。

$$\mathbf{C}_n(u)=\mathbf{C}_{n+1}(u)=\sum_{i=0}^{n+1}\mathbf{a}_i u^i$$

从向量空间的角度来看，$\mathbf{C}_{n+1}(u)$ 只是 $\mathbf{C}_n(u)$ 嵌入在一个更高维的空间中。

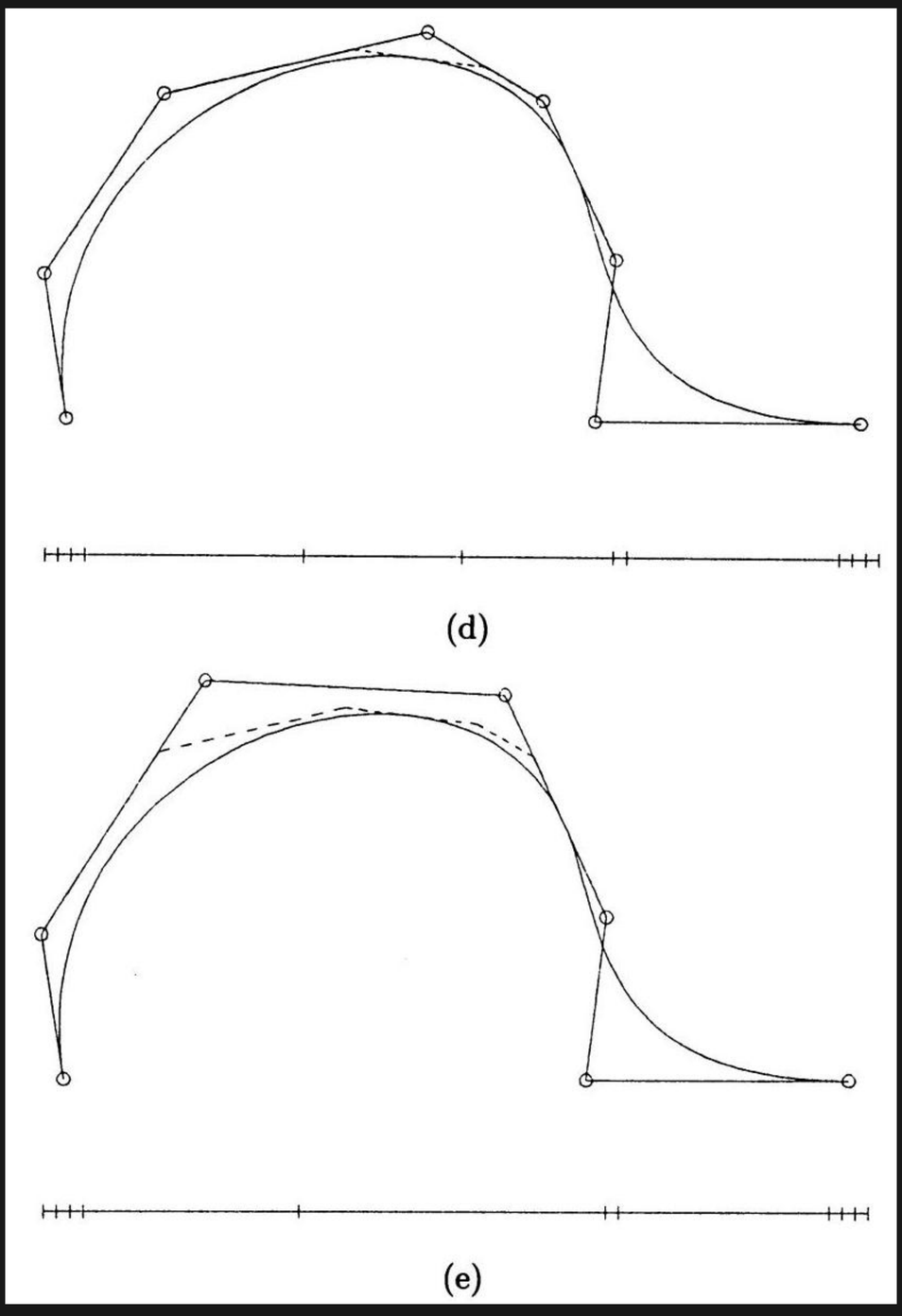


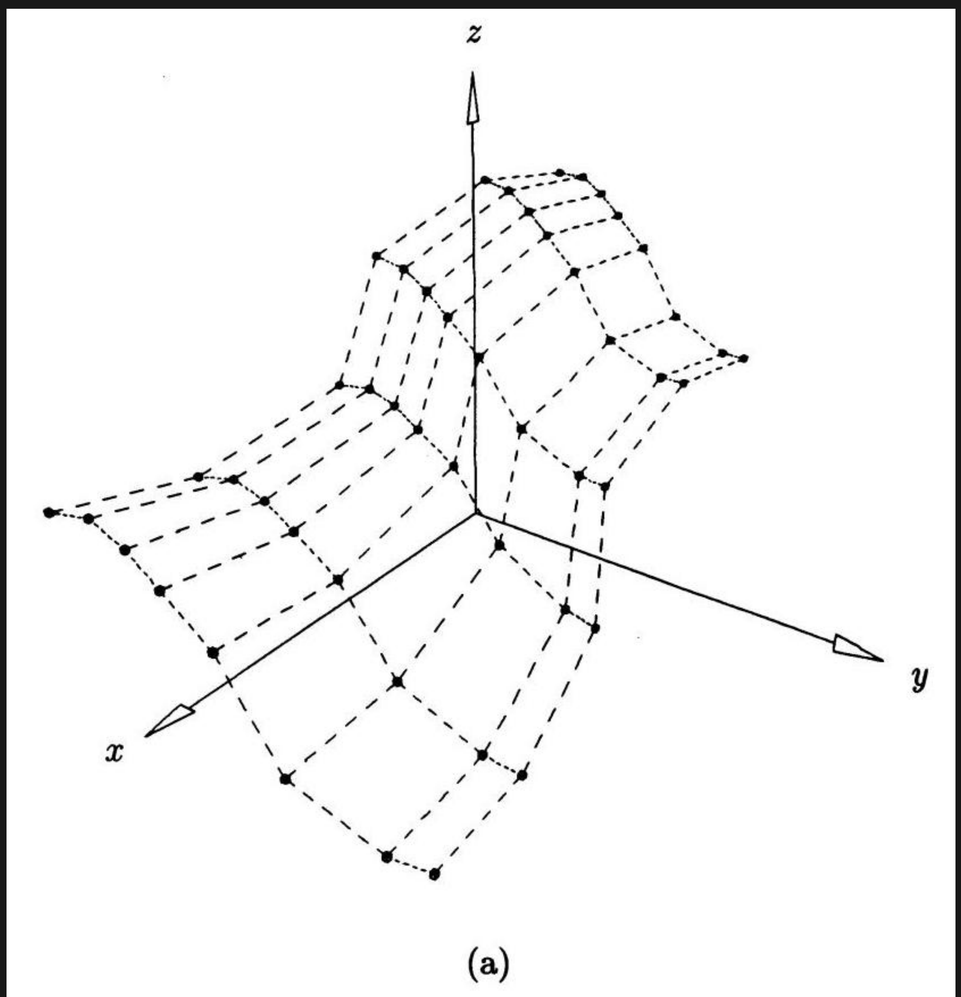
图 5.27.(续)

现在设 $\mathbf{C}_p^w(u)=\sum_{i=0}^{n} N_{i,\,p}(u)\,\mathbf{P}_i^w$ 为结点向量 U 上的一个 p 次 NURBS 曲线。由于 $\mathbf{C}_p^w(u)$ 是一个分段多项式曲线，因此应该可以将其次数提升到 $p+1$，也就是说，必须存在控制点 $\mathbf{Q}_i^w$ 和一个结点向量 $\widehat{U}$，使得

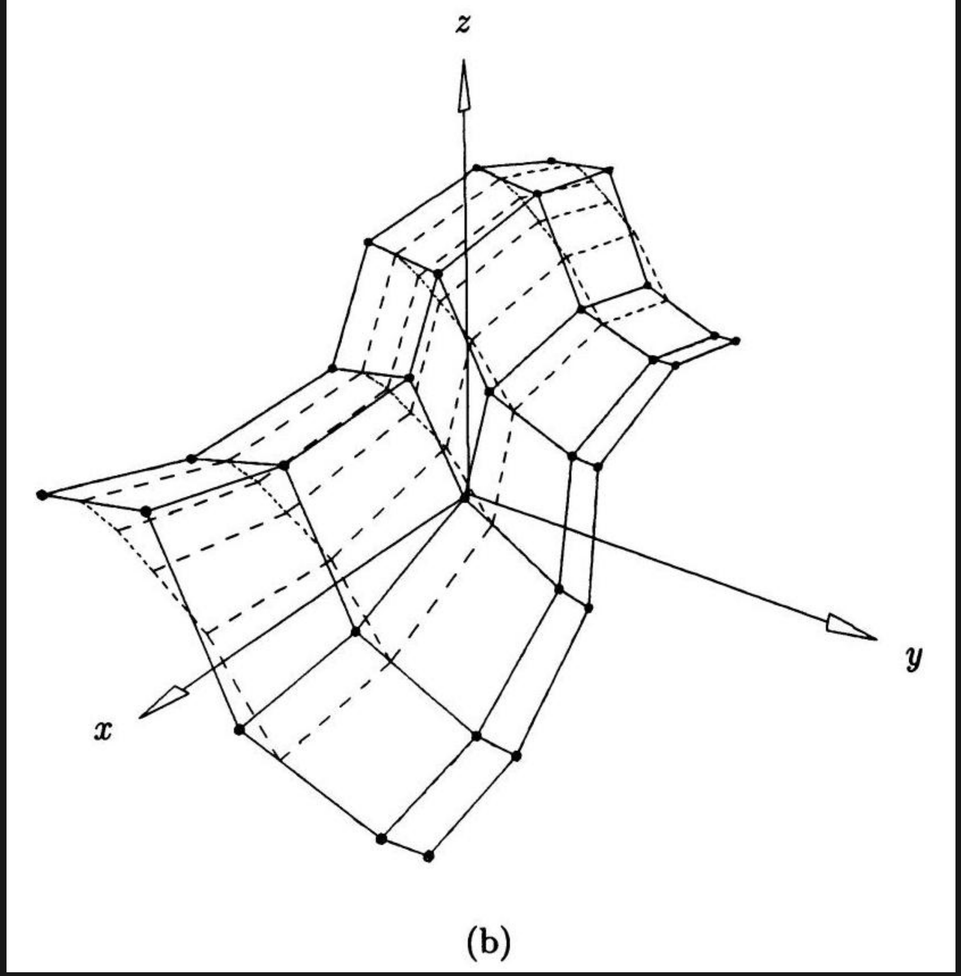
$$\mathbf{C}_p^w(u)=\mathbf{C}_{p+1}^w(u)=\sum_{i=0}^{\widehat{n}} N_{i,p+1}(u)\,\mathbf{Q}_i^w \tag{5.31}$$

$\mathbf{C}_{p+1}^w(u)$ 和 $\mathbf{C}_p^w(u)$ 在几何和参数上是相同的曲线。$\mathbf{C}_{p+1}^w(u)$ 只是 $\mathbf{C}_p^w(u)$ 嵌入在一个更高维的空间中。次数提升是指计算未知的 $\mathbf{Q}_i^w$ 和 $\widehat{U}$ 的过程 (算法)。

次数提升的应用包括:190 基本几何算法

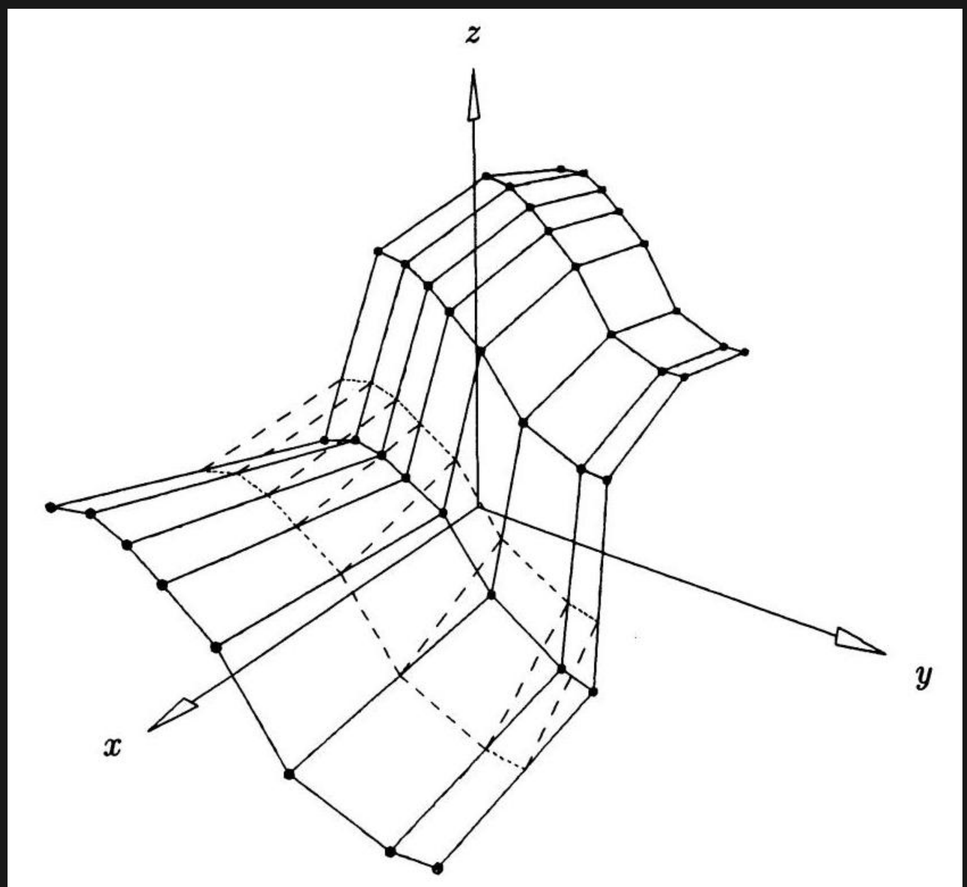


(a)

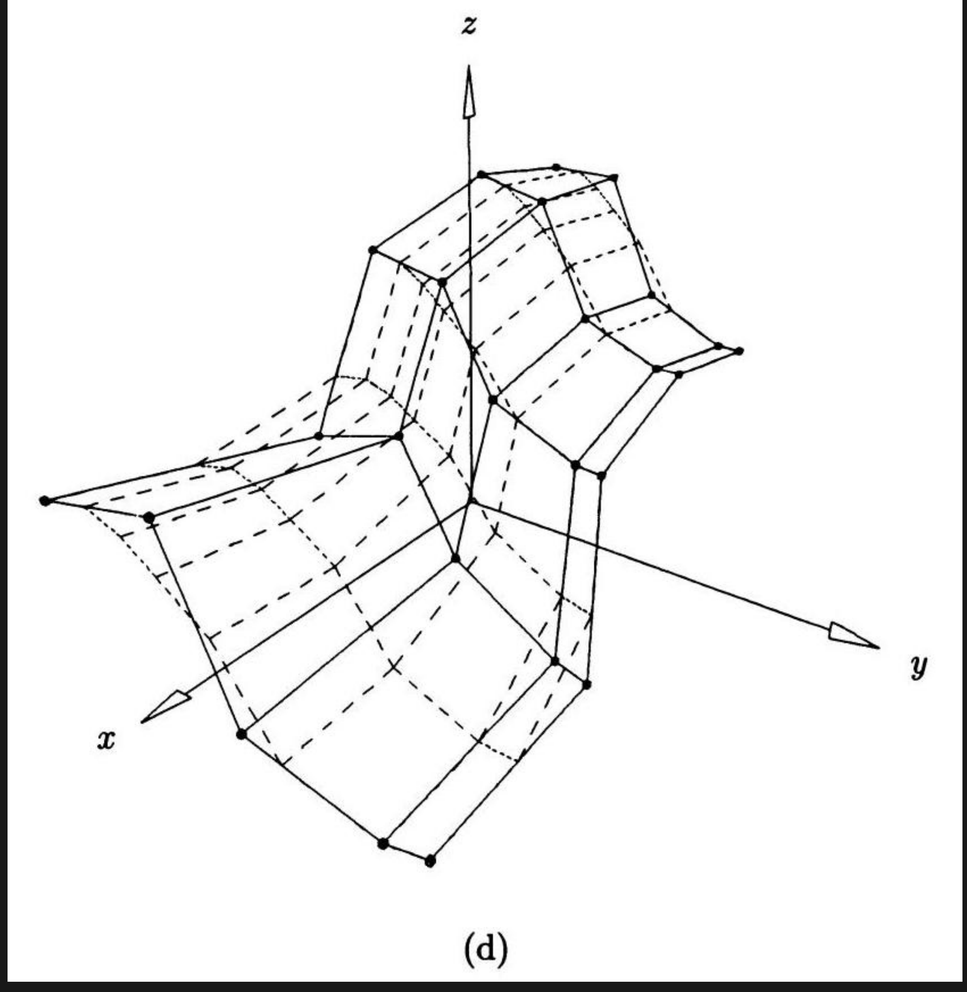


(b)

图 5.28. 表面结点移除示例。(a) 定义在 $U = \{0, 0, 0, 0, 3/10, 3/10, 3/10, 7/10, 1, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 1/4, 1/2, 3/4\ , 1, 1, 1\}$ 上的原始 (三次 $\times$ 二次) 表面；(b) 在 u 方向上移除 $u = 3/10$ 三次。



(c)



(d)

图 5.28. 续。(c) 在 v 方向上移除 $v = 1/4$ 一次；(d) 在两个方向上移除结点。

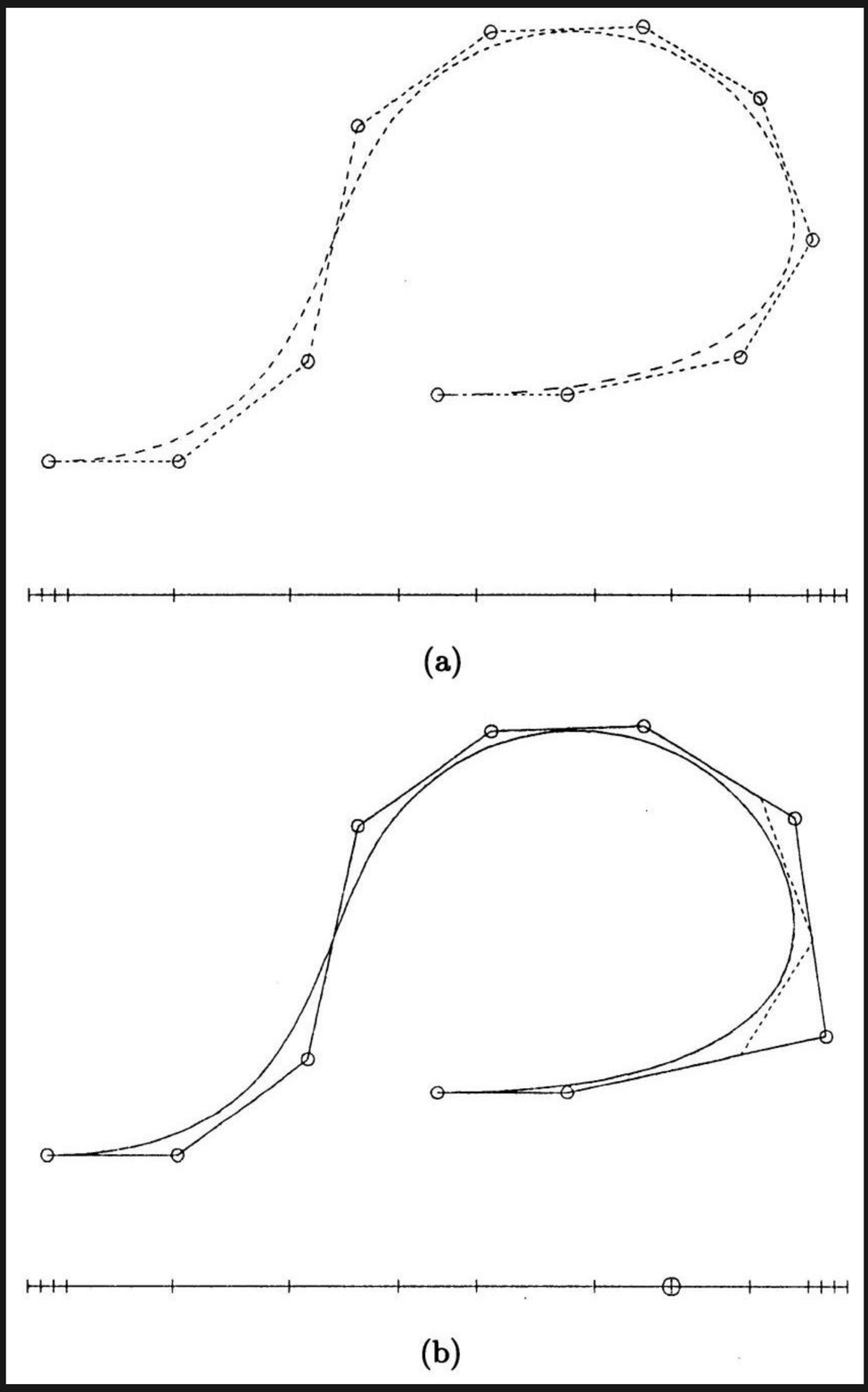


图 5.29. 从三次曲线中移除所有可移除的结点。(a) 定义在 $\{0,0,0,0,0.16,0.31,0.45,0.5501,0.702,0.8,0.901,1,1,1,1\}$ 上的原始曲线；(b) 使用容差 0.007 移除结点后的曲线；(c) 容差为 0.025；(d) 容差为 0.07；(e) 容差为 0.6；(f) 容差为 1.2。

- 在后续章节中，我们从一组曲线 $\mathbf{C}_1,\ldots,\mathbf{C}_n,(n\geq 2)$ 构造某些类型的表面。使用张量积表面要求这些曲线具有共同的度数，因此某些曲线的度数可能需要提升；
- 设 $\mathbf{C}_1,\ldots,\mathbf{C}_n,(n\geq 2)$ 为一系列具有以下性质的 NURBS 曲线: 曲线 $\mathbf{C}_i$ 的端点与曲线 $\mathbf{C}_{i+1}$ 的起点重合；则这些曲线可以组合成一条单一的 NURBS 曲线。此过程中的一步是将曲线提升到共同的度数。

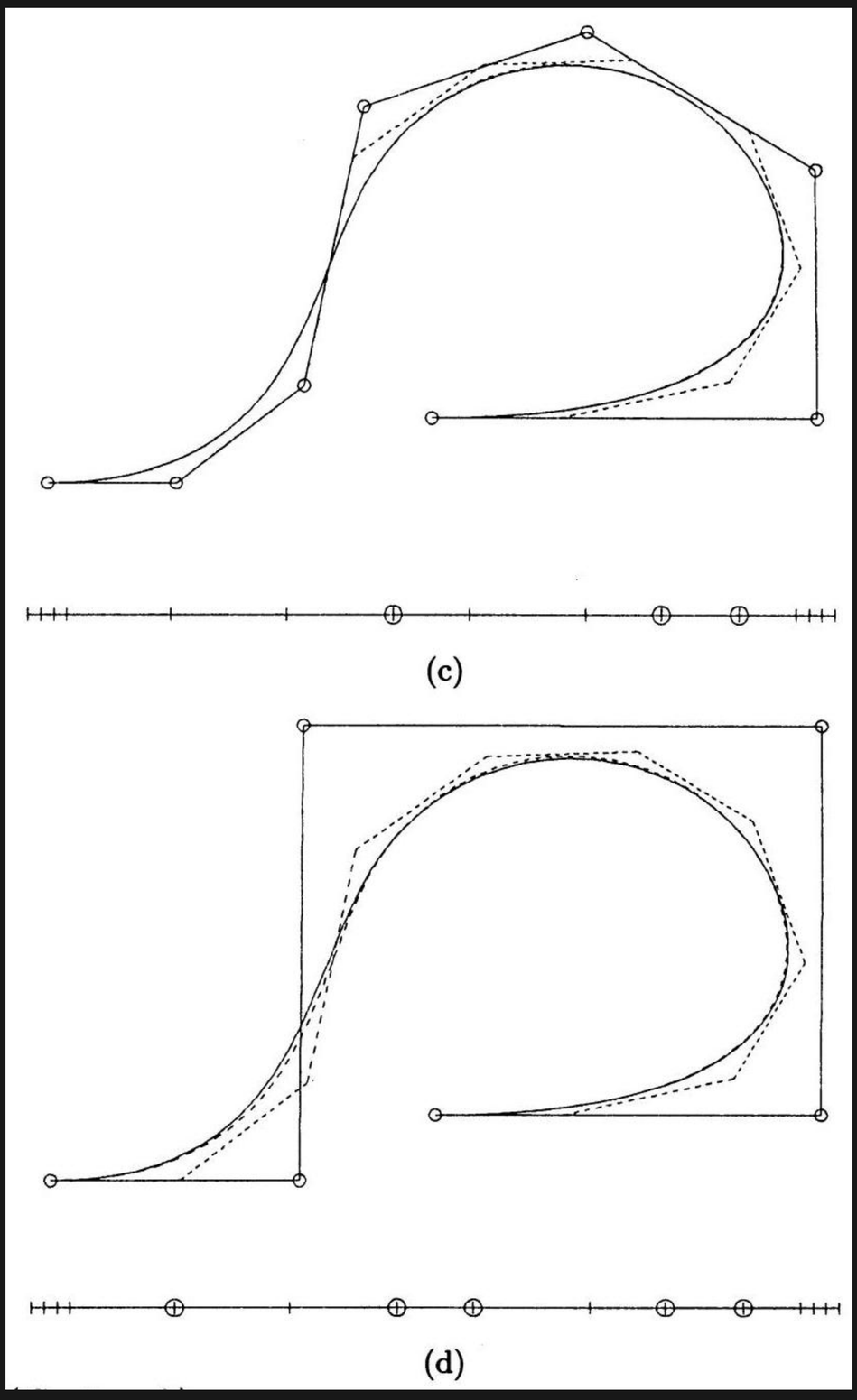


图 5.29. (续。)

像往常一样，度数提升算法应用于四维空间中的 $\mathbf{C}_p^w(u)$，而不是 $\mathbf{C}_p(u)$，并且我们在本节的其余部分中再次省略 w。

在方程 (5.31) 中有三个未知数，$\widehat{n}, \overline{U}$ 和 $\{\mathbf{Q}_i\}$。为了确定 $\widehat{n}$ 和 $\overline{U}$，假设 U 具有以下形式

$$U = \{u_0, \ldots, u_m\} = \left\{ \underbrace{a, \ldots, a}_{p+1}, \underbrace{u_1, \ldots, u_1}_{m_1}, \ldots, \underbrace{u_s, \ldots, u_s}_{m_s}, \underbrace{b, \ldots, b}_{p+1} \right\}$$

其中 $m_1, \ldots, m_s$ 表示内部节点的重数。现在 $\mathbf{C}_p(u)$ 是每个非退化节点跨度上的多项式曲线，因此其度数可以在每个这样的节点跨度上提升到 $p+1$。在重数为 m_i, $\mathbf{C}_p(u)$ 的节点上是 C^{p-m_i} 连续的。由于提升度数的曲线 $\mathbf{C}_{p+1}(u)$ 必须具有相同的连续性，因此可以得出相同的节点必须对 $\mathbf{C}_p(u)$ 具有重数 $m_1, \ldots, m_s$。这导致

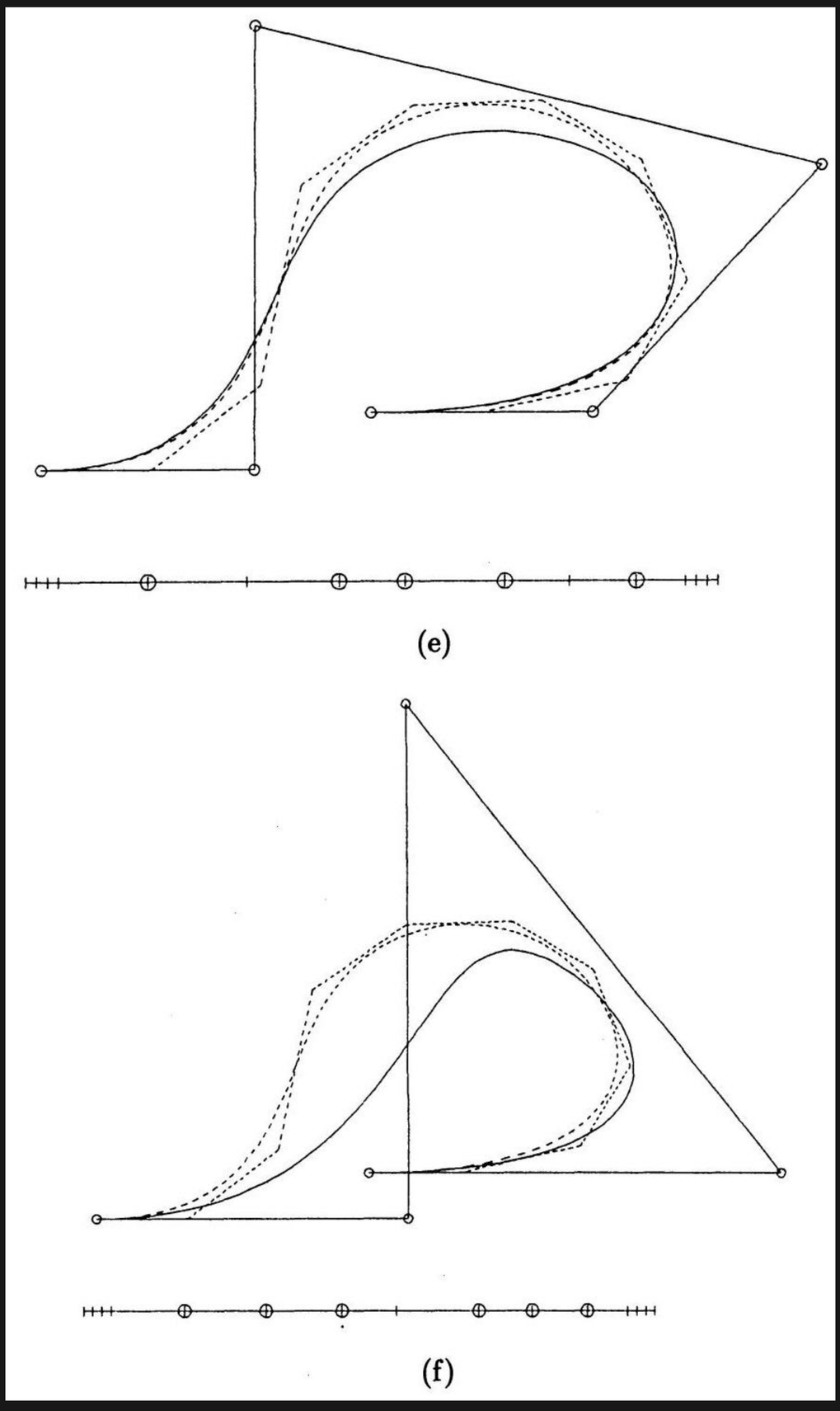


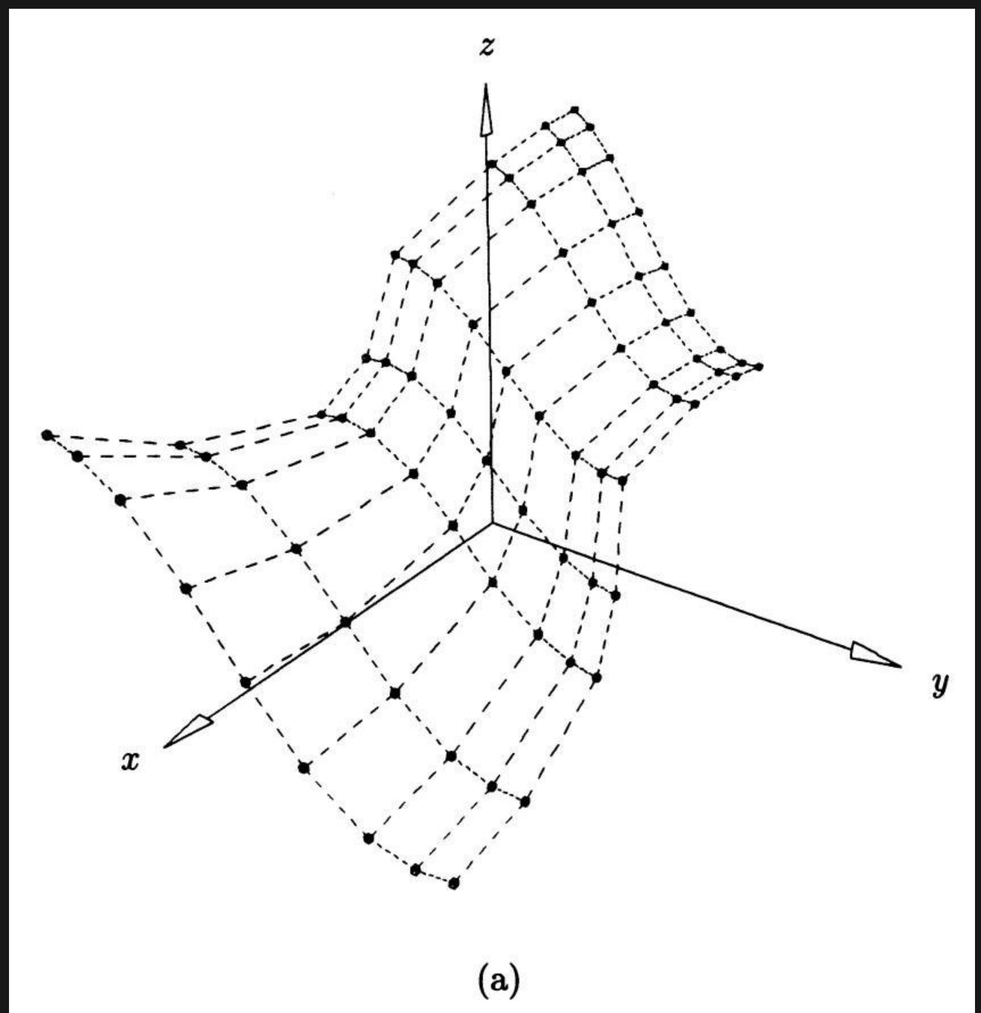
图 5.29. (续。)

相同的连续性，因此可以得出相同的节点必须对 $\mathbf{C}_{p+1}(u)$ 具有重数 m_i+1 。这导致

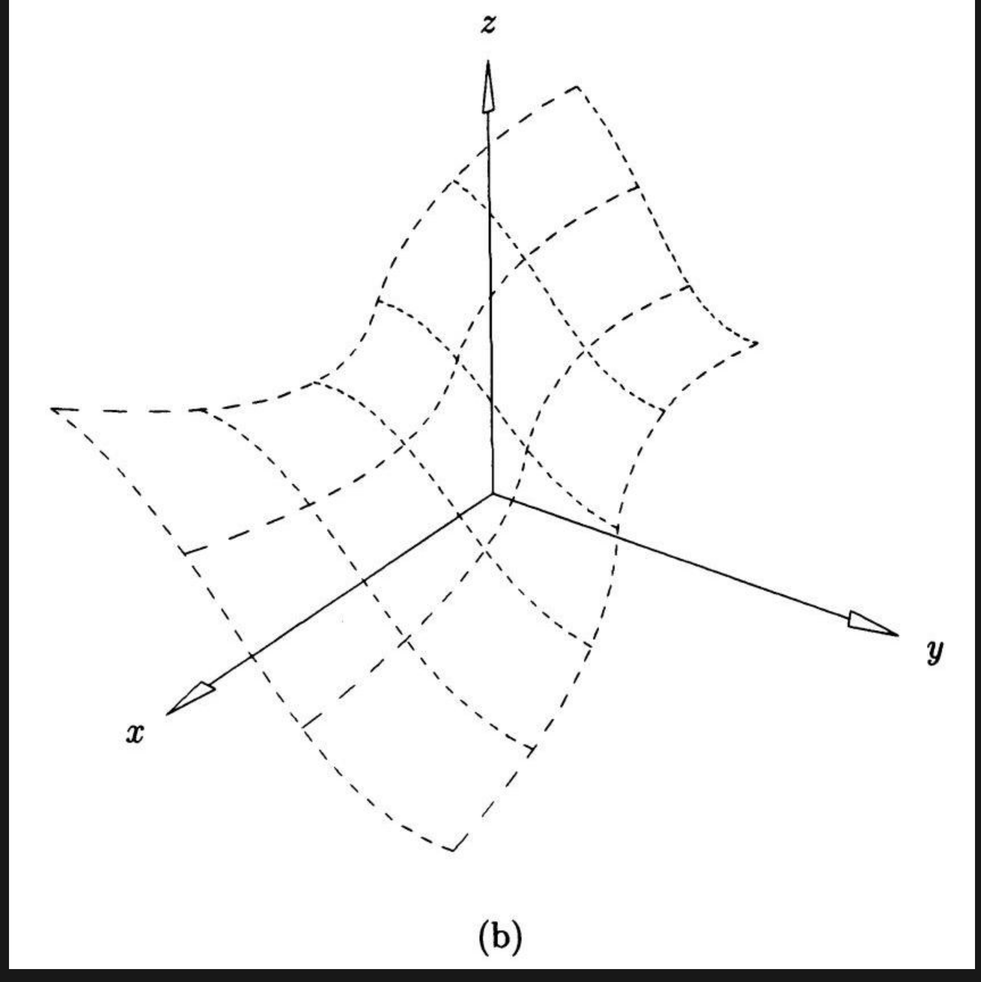
$$\widehat{n} = n + s + 1 \tag{5.32}$$

$$\text{and } \widehat{U} = \{u_0, \ldots, u_{\widehat{m}}\} = \left\{ \underbrace{a, \ldots, a}_{p+2}, \underbrace{u_1, \ldots, u_1}_{m_1+1}, \ldots, \underbrace{u_s, \ldots, u_s}_{m_{s+1}}, \underbrace{b, \ldots, b}_{p+2} \right\} \tag{5.33}$$

其中 $\widehat{m} = m + s + 2$ 。



(a)



(b)

图 5.30. 用于去除节点的 (立方 × 二次) 曲面。(a) 控制网；(b) 在 $U = \{0, 0, 0, 0, 0.22, 0.3, 0.52, 0.7, 0.79, 1, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 0.2, 0.42, 0.5, 0.81, 0.9, 1, 1, 1\}$ 上定义的曲面。

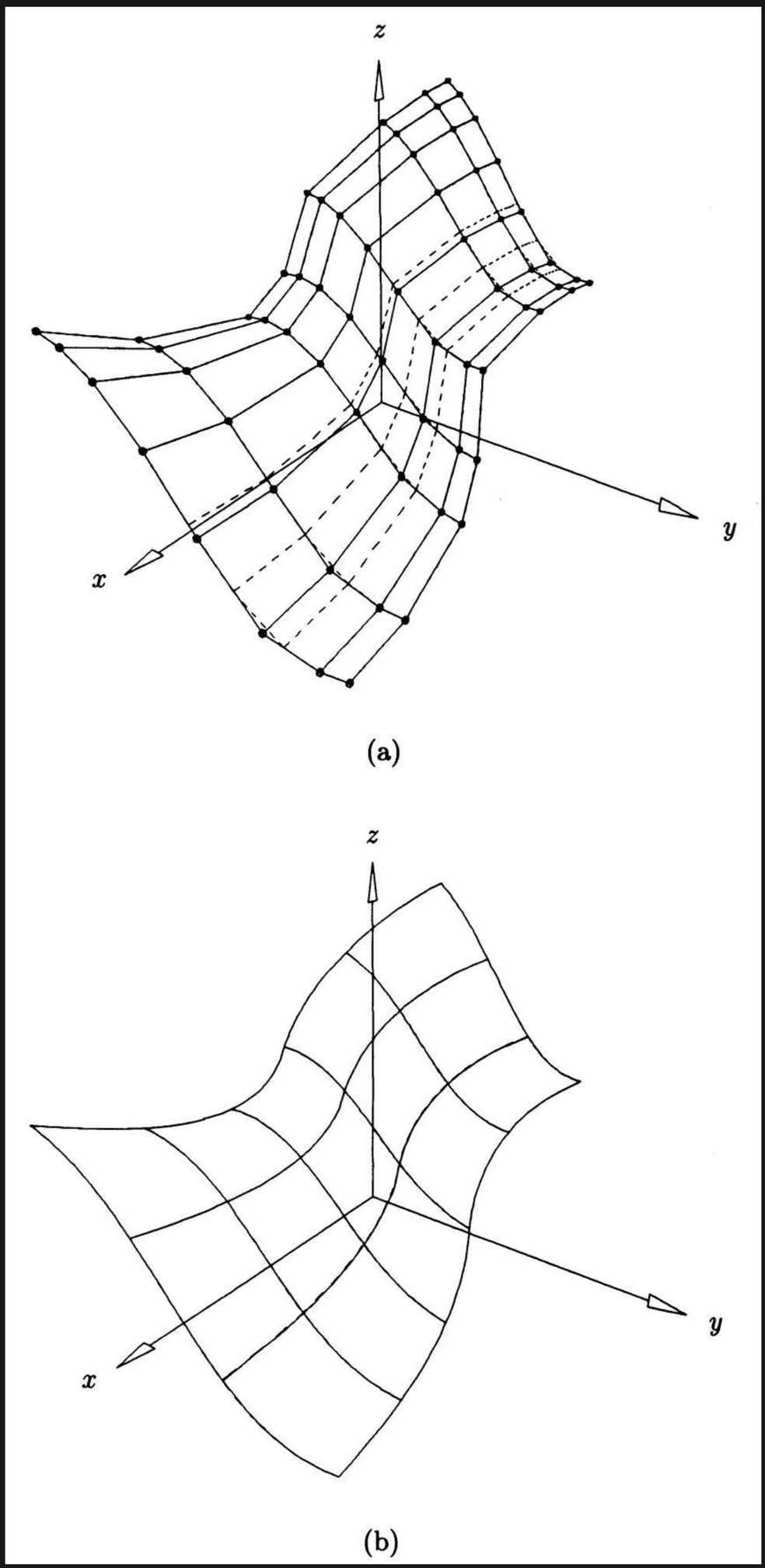


图 5.31. 在两个方向上使用容差 0.05 去除所有可去除的节点。(a) 原始 (虚线) 和去除节点的曲面 (实线) 的控制网；

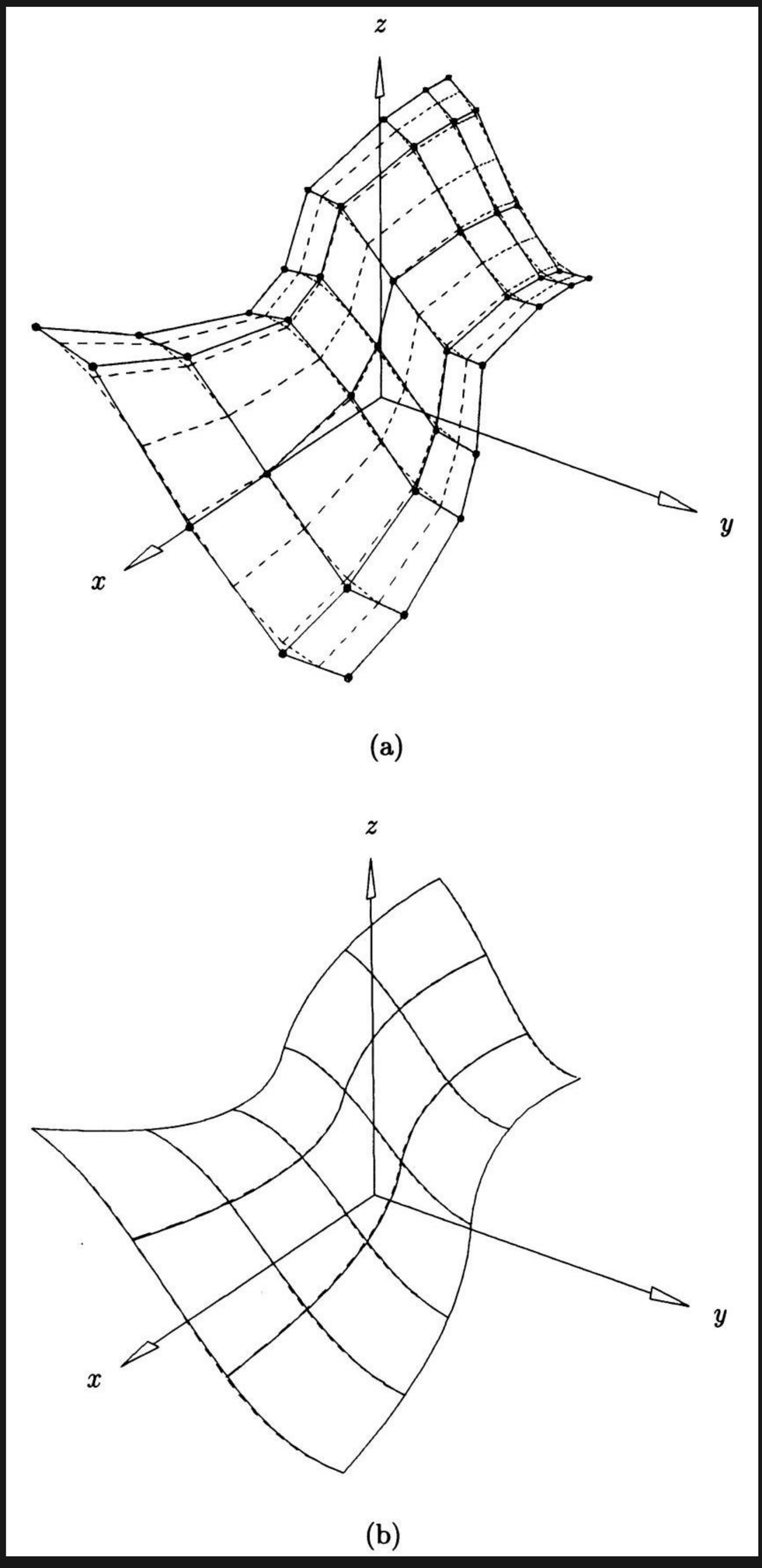


图 5.32. 在两个方向上使用容差 0.1 去除所有可去除的节点。(a) 原始 (虚线) 和去除节点的曲面 (实线) 的控制网；(b) 两个曲面的等参数线。

198 基本几何算法 (b) 两个曲面的等参数线。(b) 两个曲面的等参数线。

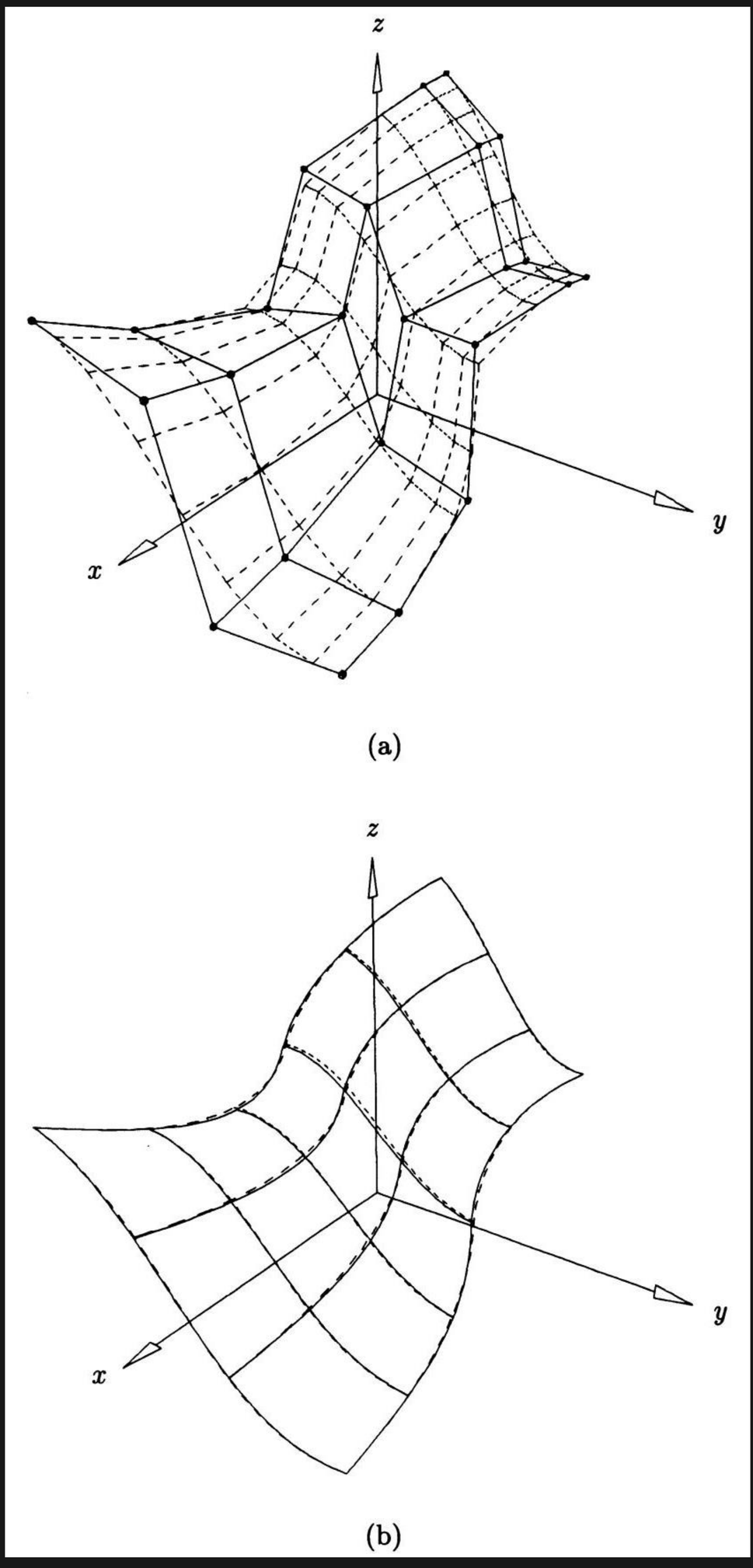


图 5.33. 在两个方向上使用容差 0.3 去除所有可去除的节点。(a) 原始 (虚线) 和去除节点的曲面 (实线) 的控制网；

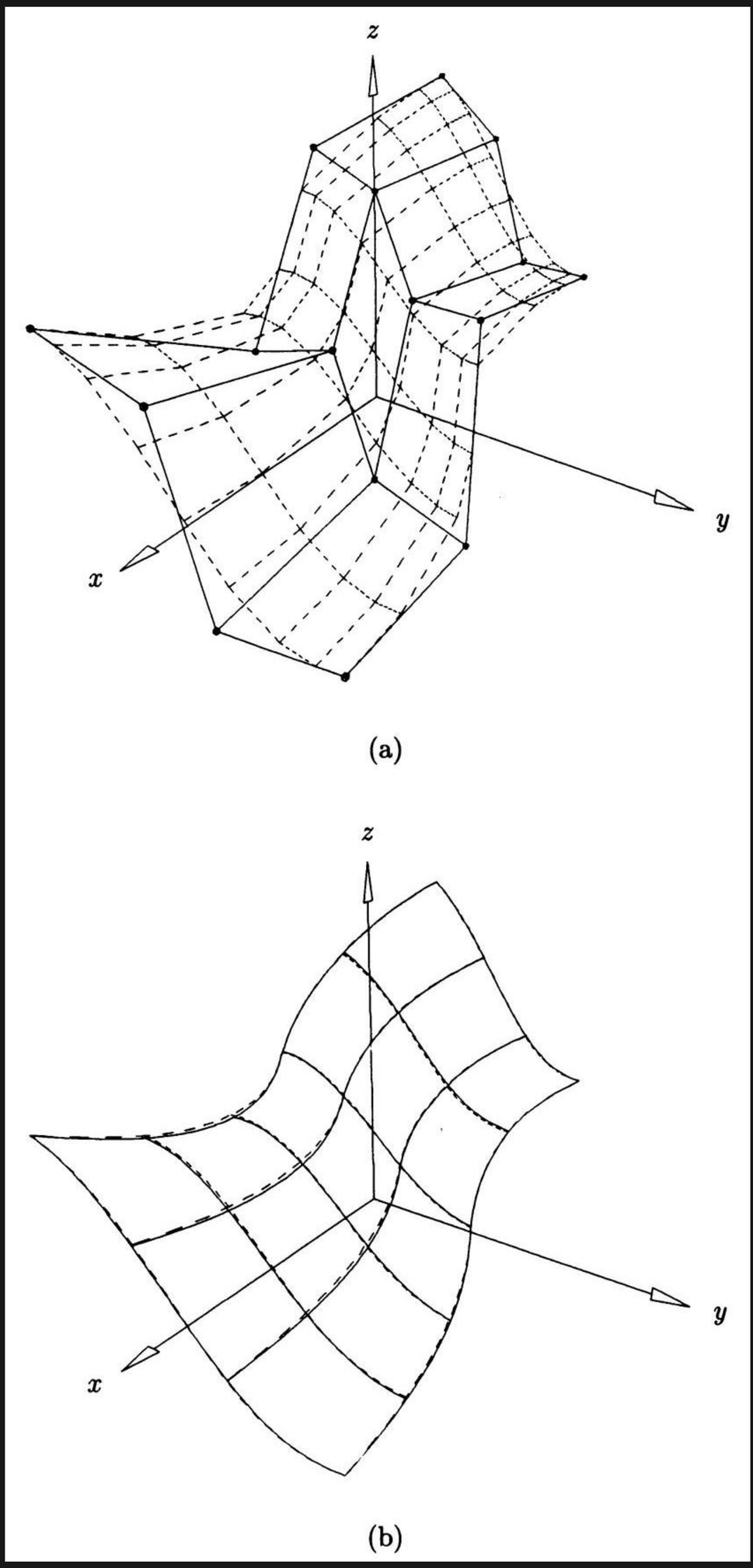


图 5.34. 在两个方向上使用容差 0.5 去除所有可去除的节点。(a) 原始 (虚线) 和去除节点的曲面 (实线) 的控制网；

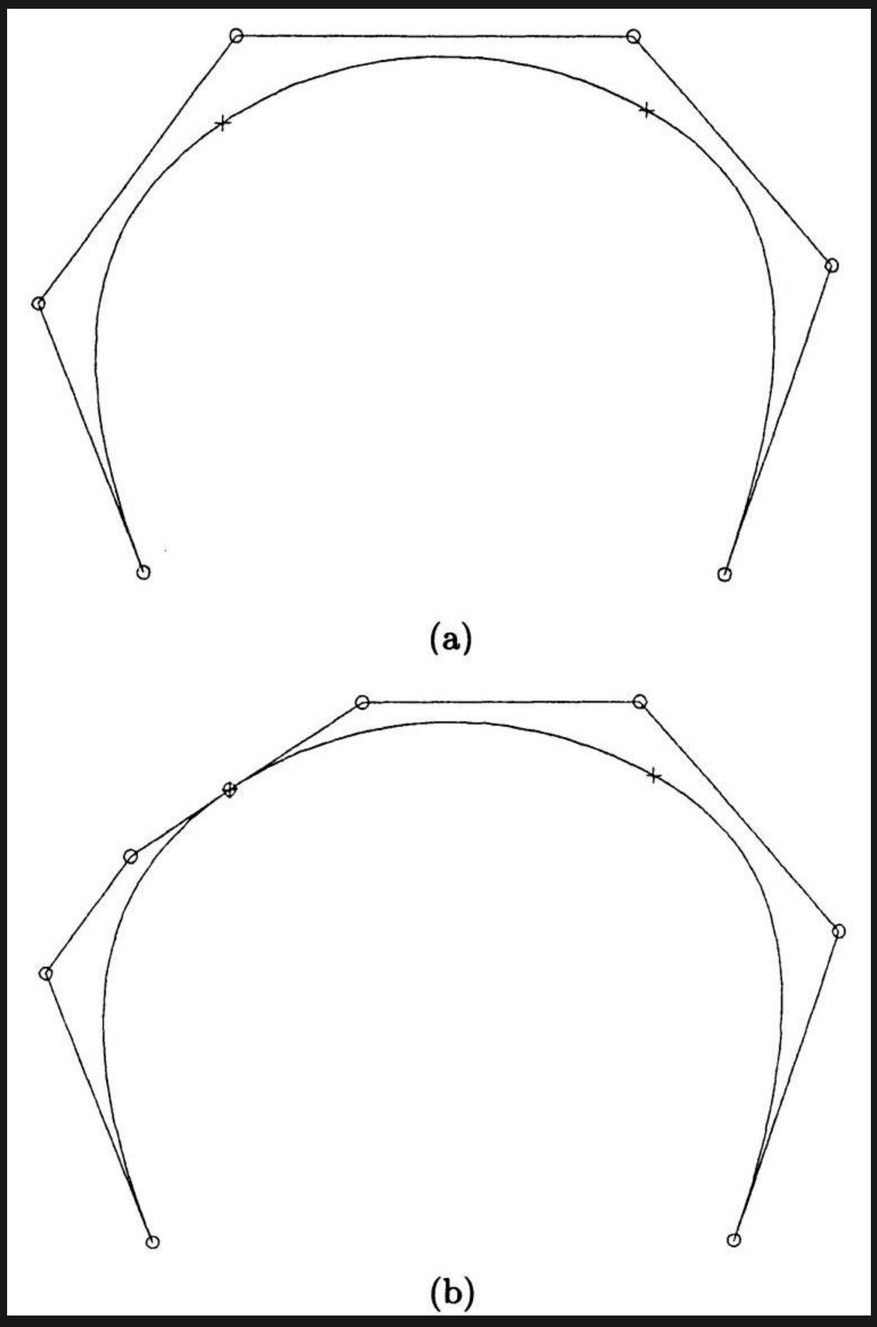


图 5.35. 定义在 $\{0,0,0,0,3/10,7/10,1,1,1,1\}$ 上的三次曲线的度数提升。(a) 原始曲线；(b) 通过节点插入获得的第一个 Bézier 段。

唯一剩下的问题是计算 $\{\mathbf{Q}_i\}$。一个显而易见但非常低效的方法是求解一个线性方程组。设定

$$\sum_{i=0}^{\hat{n}} N_{i,p+1}(u)\,\mathbf{Q}_i = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i$$

并在适当的 $\hat{n}+1$ 值下评估 $N_{i,p}(u)$ 和 $N_{i,p+1}(u)$，得到一个带状的 $\hat{n}+1$ 线性方程组，未知数为 $\mathbf{Q}_i$。

更高效但在数学上更复杂的方法由 Prautzsch [Prau84]、Cohen 等 [Cohe85] 和 Prautzsch 与 Piper [Prau91] 提供。Prautzsch 和 Piper 提出的算法在一般情况下是最有效的，但 Cohen 等人也为低度特殊情况 (例如线性到二次，二次到三次) 提供了简单而高效的算法。所有这些算法都将度数提升了 $1\,(p \to p+1)$。

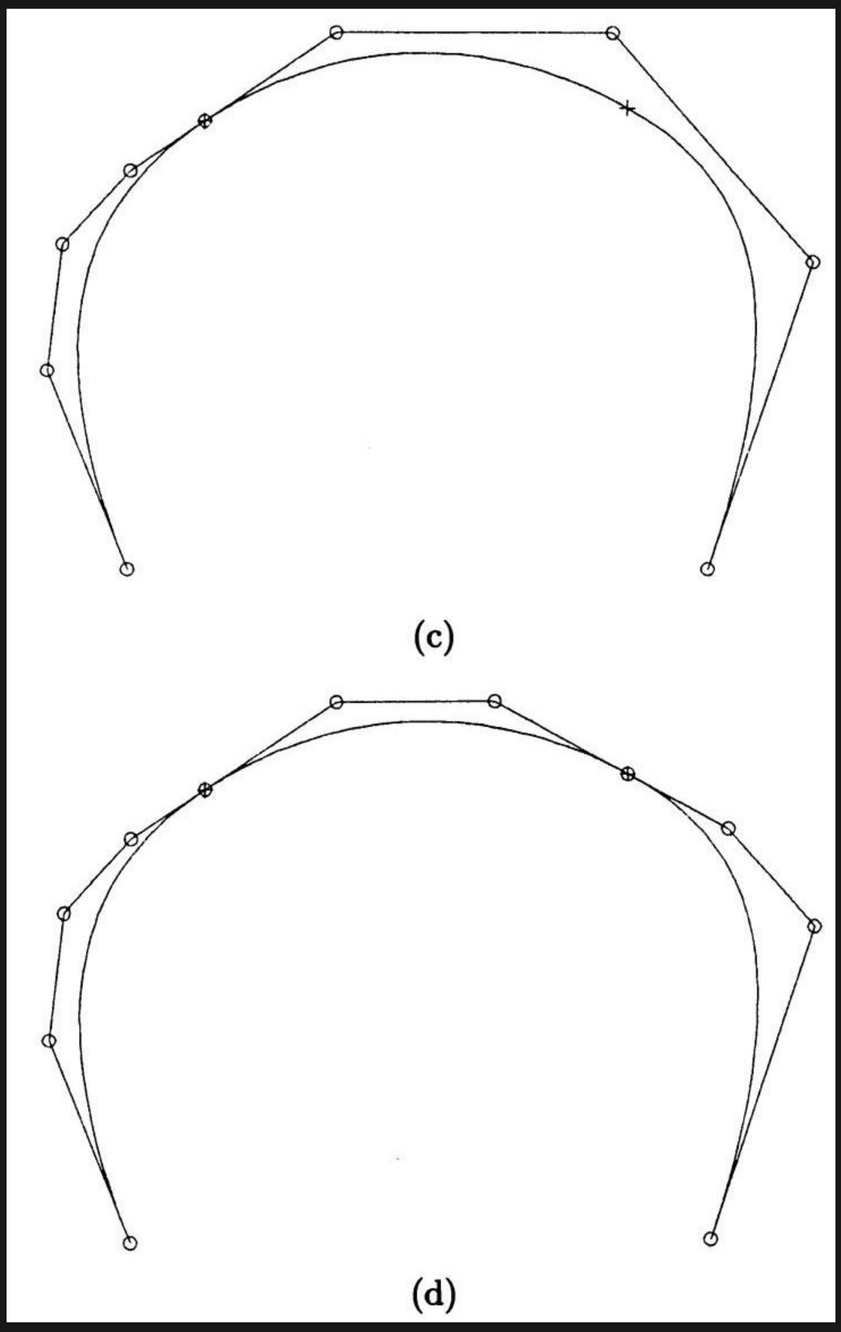


图 5.35. 续。(c) 第一个 Bézier 段被提升度数，(d) 计算第二个 Bézier 段。

我们在这里提出另一种算法，该算法在数学上更简单，并且与 [Prau91] 中给出的算法具有竞争力，特别是在度数需要提升超过 1 的情况下 (见 [Pieg94])。该解决方案包括在遍历节点向量时实时应用三个步骤。这些步骤是:

1. 使用算法 A5.6 的逻辑，从曲线中提取第 i 个 Bézier 段；
2. 提升第 i 个 Bézier 段的度数；
3. 移除分隔 (i - 1) 个和第 i 个段的多余节点。

图 5.35a – 5.35 h 显示了算法的进展。原始曲线是一个具有节点向量 $U = \{0, 0, 0, 0, u_a, u_b, 1, 1, 1, 1\}$ 的三次曲线。它被提升到四次。图的解释如下:

a. 原始曲线；

b. u_a 被插入两次，使其重数达到 3；

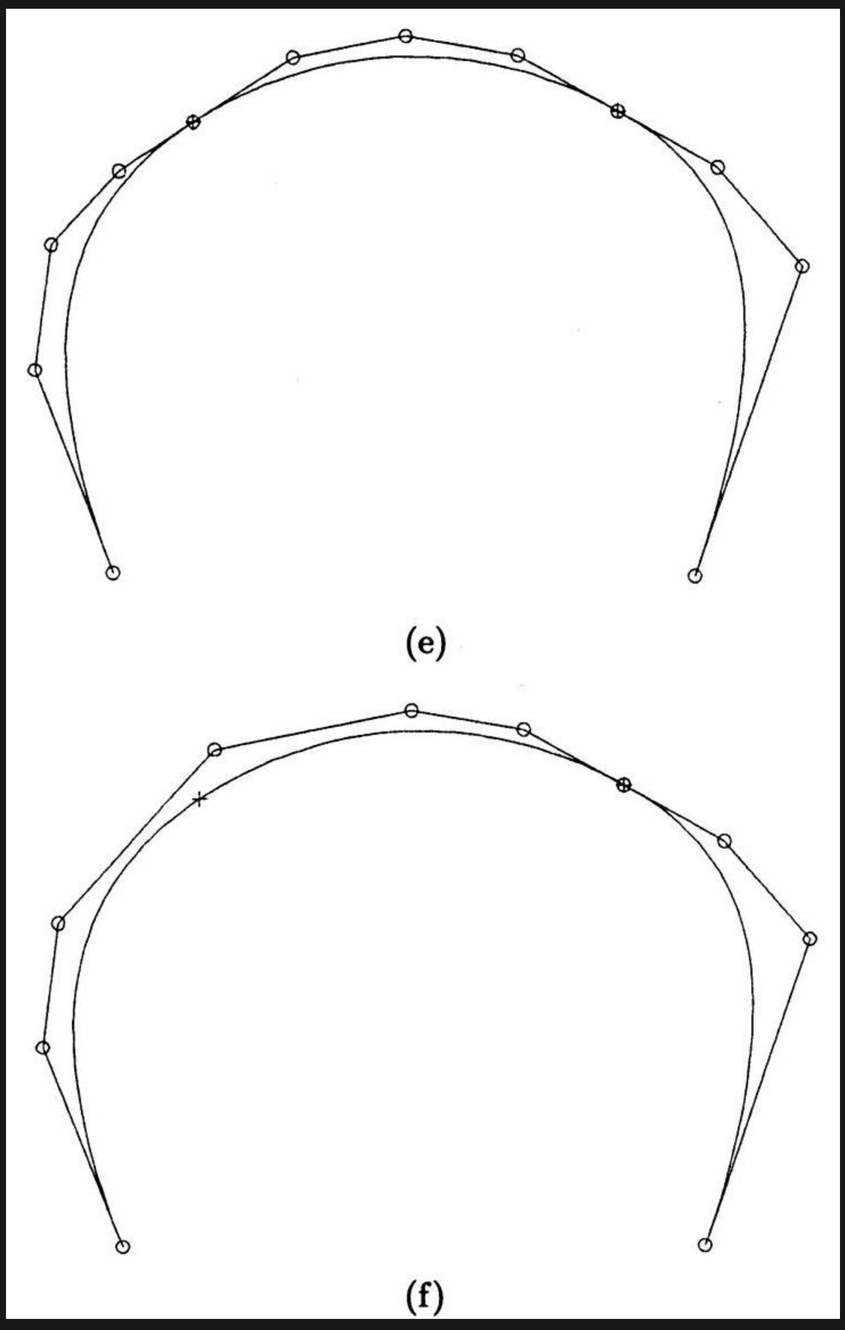


图 5.35. 继续。(e) 第二个贝塞尔段的次数被提升；(f) 移除了两个节点 $u = 3/10$ 的出现。

c. 在 $[0, u_a]$ 上的贝塞尔段的次数被提升；这用五个新控制点替换了前四个控制点 (见后续文本)；

d. u_b 被插入两次以获得在 $[u_a, u_b]$ 上的贝塞尔段；

e. 在 $[u_a, u_b]$ 上的贝塞尔段的次数被提升；

f. 移除了两个节点 u_a 的出现；

g. 最后一个段 $[u_b, 1]$ 的次数被提升；

h. 移除了两个 u_b 的出现，得到了最终的四次 C^2 连续曲线。

第一步是算法 A5.6 的直接应用。它被修改为不返回 Bézier 段，而是实时处理它们。

第二步是提升 Bézier 段的度数。我们提出了两个公式来实现这一点。第一个是众所周知的公式 (由于 Forrest [Forr72]) 用于将度数从 p 提升到 $p + 1$ 。设

$$\mathbf{C}_p(u) = \sum_{i=0}^{p} B_{i,p}(u)\,\mathbf{P}_i$$

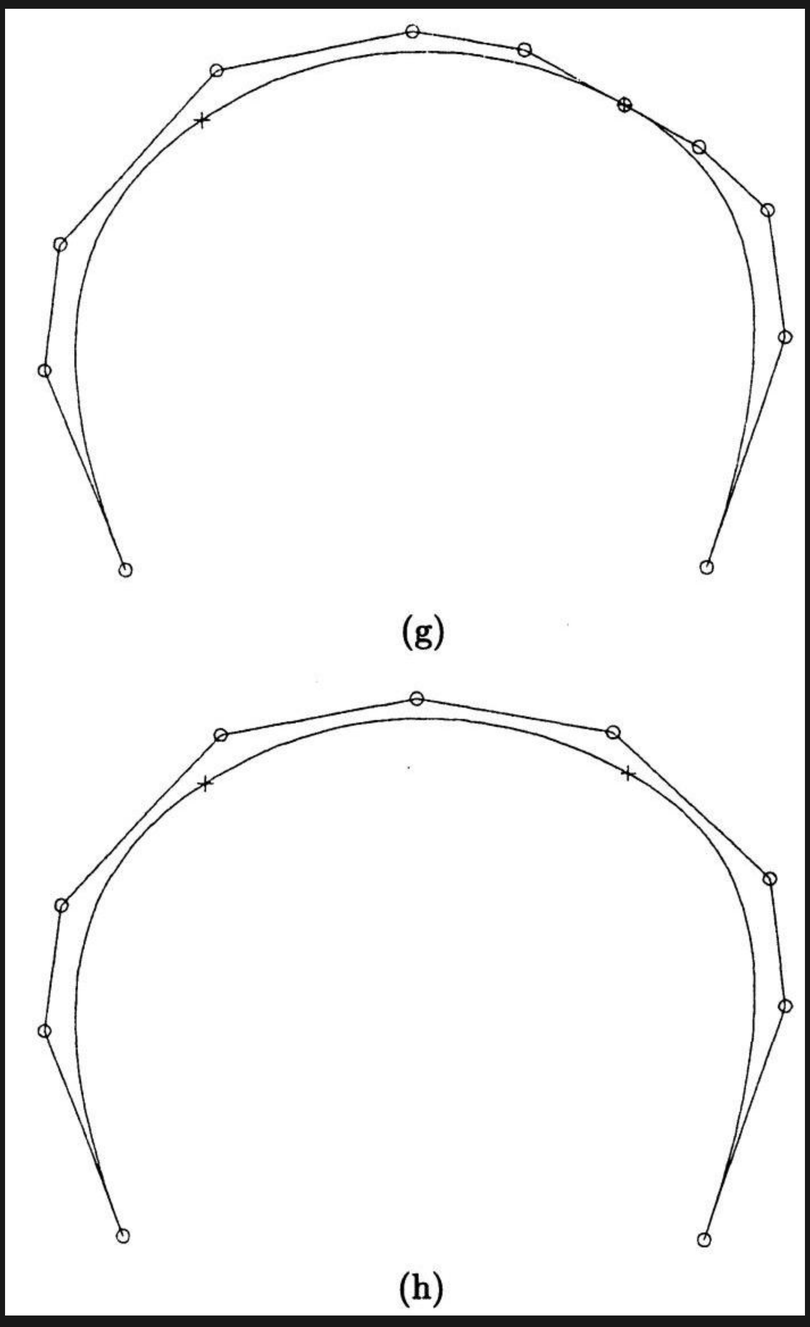


图 5.35. 继续。(g) 第三个 Bézier 段的度数被提升；(h) 删除了两个结点 $u = 7/10$ 的出现。

设为一个 p 次 Bézier 曲线。它作为 $(p+1)$ 次曲线的表示为

$$\mathbf{C}_{p+1}(u) = \sum_{i=0}^{p+1} B_{i,p+1}(u)\,\mathbf{Q}_i \tag{5.34}$$

将这些相等并将 $\mathbf{C}_p(u)$ 乘以 $(u+(1-u))(=1)$ 得到

$$\sum_{i=0}^{p+1} B_{i,p+1}\mathbf{Q}_i = (u+(1-u))\sum_{i=0}^{p} B_{i,p}\mathbf{P}_i$$

$$= \sum_{i=0}^{p} \left((1-u)\,B_{i,p} + uB_{i,p}\right)\mathbf{P}_i$$

使用符号

$$\frac{p!}{i!\,(p-i)!} = \binom{p}{i}$$

并应用公式 (1.8)，我们得到

$$\sum_{i=0}^{p+1} \binom{p+1}{i} u^i(1-u)^{p+1-i}\mathbf{Q}_i$$

$$= \sum_{i=0}^{p} \binom{p}{i} \left(u^i(1-u)^{p+1-i} + u^{i+1}(1-u)^{p-i}\right)\mathbf{P}_i$$

$$= \sum_{i=0}^{p} \binom{p}{i} u^i(1-u)^{p+1-i}\mathbf{P}_i + \sum_{i=1}^{p+1} \binom{p}{i-1} u^i(1-u)^{p+1-i}\mathbf{P}_{i-1}$$

将 $u^i(1-u)^{p+1-i}$ 的系数相等得出

$$\binom{p+1}{i}\mathbf{Q}_i = \binom{p}{i}\mathbf{P}_i + \binom{p}{i-1}\mathbf{P}_{i-1}$$

从

$$\binom{p}{i} \Big/ \binom{p+1}{i} = \frac{p!i!\,(p+1-i)!}{i!\,(p-i)!\,(p+1)!} = \frac{p+1-i}{p+1} = 1 - \frac{i}{p+1}$$

和

$$\binom{p}{i-1} \Big/ \binom{p+1}{i} = \frac{p!i!\,(p+1-i)!}{(i-1)!\,(p+1-i)!\,(p+1)!} = \frac{i}{p+1}$$

可以得出

$$\mathbf{Q}_i = (1-\alpha_i)\,\mathbf{P}_i + \alpha_i\mathbf{P}_{i-1} \tag{5.35}$$

where

$$\alpha_i = \frac{i}{p+1}\; i = 0, \ldots, p+1$$

注意，方程 (5.35) 表示一个角落切割过程 (见图 5.35c 、5.35e 和 5.35g)。关于方程 (5.35):

- 它可以递归应用以提升度 t 次 $(t \geq 1)$ ；
- 它的增长率为 $\mathrm{O}\left(t^2\right)$ ；如果 $t > 1$ ，方程 (5.35) 的递归应用涉及一些冗余计算；
- α_i 仅依赖于度，而不依赖于应用的特定贝塞尔段，因此可以在处理段之前计算一次并存储在本地数组中；
- 这是一种凸组合方案。

可以在一步中将度从 p 提升到 $p+t$ 。写出方程 (5.35) 的多个递归应用，并巧妙地重新排列系数，得到

$$\mathbf{P}_i^t = \sum_{j=\max(0,i-t)}^{\min(p,i)} \frac{\binom{p}{j}\binom{t}{i-j}}{\binom{p+t}{i}} \mathbf{P}_j \quad i = 0, \ldots, p+t \tag{5.36}$$

其中 $\mathbf{P}_i^t$ 表示在 t 次度提升后的控制点。作为一个例子，考虑 $p = 2$ 和 $t = 4$ 的情况，即

$$\mathbf{P}_0^4 = \mathbf{P}_0$$

$$\mathbf{P}_1^4 = \frac{4}{6}\mathbf{P}_0 + \frac{2}{6}\mathbf{P}_1$$

$$\mathbf{P}_2^4 = \frac{6}{15}\mathbf{P}_0 + \frac{8}{15}\mathbf{P}_1 + \frac{1}{15}\mathbf{P}_2$$

$$\mathbf{P}_3^4 = \frac{4}{20}\mathbf{P}_0 + \frac{12}{20}\mathbf{P}_1 + \frac{4}{20}\mathbf{P}_2$$

$$\mathbf{P}_4^4 = \frac{1}{15}\mathbf{P}_0 + \frac{8}{15}\mathbf{P}_1 + \frac{6}{15}\mathbf{P}_2$$

$$\mathbf{P}_5^4 = \frac{2}{6}\mathbf{P}_1 + \frac{4}{6}\mathbf{P}_2$$

$$\mathbf{P}_6^4 = \mathbf{P}_2$$

关于方程 (5.36):

- 它的增长率为 $\mathrm{O}(t)$，因为方案的宽度是固定的 (由 $p+1$) 限制)；
- 每个 $\mathbf{P}_i^t$ 的系数之和为一 (见示例)，因此它是一种凸组合方案；
- 没有冗余计算；
- 系数矩阵相对于行 $^{p+t}/_2$ 是对称的；
- 系数同时依赖于 p 和 t，但不依赖于它们所应用的特定 Bézier 段；因此，它们可以在处理段之前计算一次并存储在本地数组中。

第 3 步的节点移除比前一节中描述的一般节点移除便宜得多。这有两个原因:

- 我们知道可以移除多少个节点 - 插入的节点数量；
- 节点向量在被移除的节点邻域具有特定结构。

作为一个例子，图 5.36 显示了一条曲线，其度数从 4 提高到 $5\,(p = 4)$。设 u_c, u_d, u_e 为定义所示两个段的三个参数值。假设原始重数 u_d 为 1，则需要插入三个节点以形成两个 Bézier 段 (注意 $\mathbf{C}(u_d) = \mathbf{P}_k^0$)，现在需要三个移除。然而，我们可以跳过前两个，直接计算 $\mathbf{P}_{k-1}^3$ 和 $\mathbf{P}_k^3$。这只需要计算两个点，形式为 Eq. (5.29)，且不需要检查点的相等性。这与一般情况下需要的六个点计算和三个点相等性检查相比。此外，Eq. (5.29) 的 α_j 不会改变，无论度数如何 (但 α_i s 一般可以改变)。这是因为算法已经插入了 u_e 以具有重数 $p+1$。因此，在局部上，节点向量具有以下形式:

$$\ldots, u_c, u_c, \underbrace{u_d, \ldots, u_d}_{p+1}, \underbrace{u_e, \ldots, u_e}_{p+1}, \ldots$$

其中 u_e 的重数至少为 2。

算法 A5.9 通过计算 $\widehat{n}$、$\widehat{U}$ 和新的 $\mathbf{Q}_i$ (nh, Uh 和 Qw) 将度数从 p 提高到 $p+t, t \geq 1$；它使用公式 (5.36) 来提升 Bézier 段的度数。Min( ) 和 Max( ) 分别计算两个整数的最小值和最大值，而 $\mathrm{Bin}\,(i,j)$ 计算 $\binom{i}{j}$，即二项式系数，可以预先计算并存储以提高效率。设 p 为原始曲线的度数。所需的局部数组为:

bezalfs [$p+t+1$] [$p+1$] : coefficients for degree elevating the Bézier segments;

bpts [$p+1$] : p th-degree Bézier control points of the current segment;

ebpts [p+t+1] : $(p+t)$ th-degree Bézier control points of the current segment;

Nextbpts [p-1] : leftmost control points of the next Bézier segment;

alphas [$p-1$] : knot insertion αs .

```
## ALGORITHM A5.9
DegreeElevateCurve(n, p, U, Pw, t, nh, Uh, Qw)
    { /* Degree elevate a curve t times. */
        /* Input: n, p, U, Pw, t */
        /* Output: nh, Uh, Qw */
    m = n + p + 1; ph = p + t;
      /* Compute Bézier degree elevation coefficients */
    ph2 = ph/2;
    bezalfs[0][0] = bezalfs[ph][p] = 1.0;
    for (i =1 ; i<=ph2 ; i++)
```

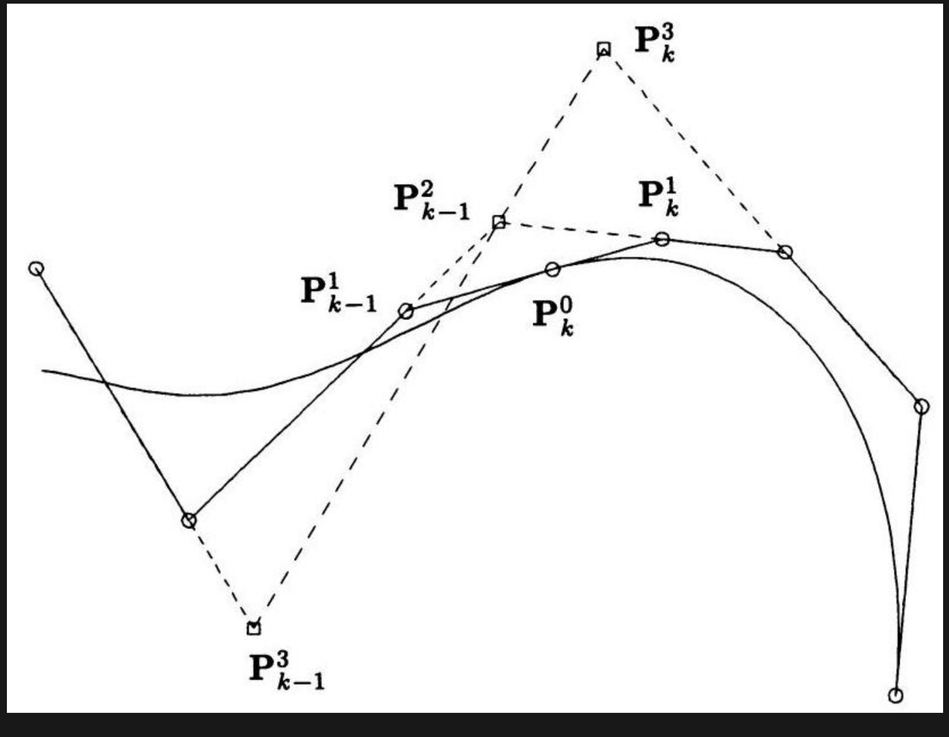


图 5.36. 从四次曲线中移除三个节点。

```
    {
    inv = 1.0/Bin(ph, i);
    mpi = Min(p, i);
    for (j=Max(0, i-t); j<=mpi; j++)
        bezalfs[i][j] = inv*Bin(p, j)*Bin(t, i-j);
    }
for (i=ph2+1; i<=ph-1; i++)
    {
    mpi = Min(p, i);
    for (j=Max(0, i-t); j<=mpi; j++)
        bezalfs[i][j] = bezalfs[ph-i][p-j];
    }
mh = ph;
```

```
    ua = U[0];
    for (i = 0; i<=ph; i++) Uh[i] = ua;
    kind = ph+1;
    Qw[0] = Pw[0]; cind = 1;
    lbz = 1; r = -1; /* Initialize first Bézier seg */
    for (i=0; i<=p; i++) bpts[i] = Pw[i];
    a = p; b = p + 1;
    while (b < m) /* Big loop thru knot vector */
        {
        i = b;
        while (b < m && U [b] == U [b + 1]) b = b + 1 ;
        mul = b-i+1; mh = mh+mul+t;
        ub = U[b];
        oldr = r;
    r = p-mul; /* Insert knot u(b) r times */
    if (oldr > 0 ) lbz = (oldr+2)/2 ;
            else 1 bz = 1 ;
    if (r > 0) rbz = ph − (r + 1) /2 ;
            else rbz = ph;
    if (r > 0 )
            { /* Insert knot to get Bézier segment */
            numer = ub-ua;
            for (k=p; k>mul; k–)
              alphas [k − mul − 1] = numer/(U[a+k]-ua);
            for (j=1; j<=r; j++)
              {
              save = r-j; s = mul+j;
              for (k=p; k>=s; k–)
                    {
                    alf = alphas[k-s];
                    bpts[k] = alf*bpts[k] + (1.0-alf)*bpts[k-1];
                  }
              Nextbpts[save] = bpts[p];
              }
        } /* End of ”insert knot” */
    if (r > 0) RBZ = rbz + 1; /∗ Degree elevate Bézier ∗ /
            else RBZ = rbz;
    for (i=lbz; i<=RBZ; i++)
            { /∗ Only points lbz, . . . , RBZ are used below ∗ /
            ebpts[i] = 0.0 ;
            mpi = Min(p, i);
            for (j=Max(0, i-t); j<=mpi; j++)
              ebpts[i] = ebpts[i] + bezalfs[i][j]*bpts[j];
        } /* End of degree elevating Bezier */
    if (oldr > 1 )
            { /* Must remove knot u=U[a] oldr times */
            if (oldr > 2 )
              { /* Alphas on the right do not change */
              alfj = (ua-Uh[kind-1])/(ub-Uh[kind-1]);
              }
          first = kind-2; last = kind;
          for (tr=1; tr<oldr; tr++)
              { /* Knot removal loop */
              i = first; j = last;
              kj = j-kind+1;
              while (j-i > tr) /* Loop and compute the new */
                { /* control points for one removal step */
                if (i < cind)
                      {
                                  alf = (ua-Uh[i])/(ub-Uh[i]);
                                  Qw[i] = (Qw[i]-(1.0-alf)*Qw[i-1])/alf;
                                  }
                            if ( kj >= 1bz )
                                  ebpts[kj] = ebpts[kj]-alfj*ebpts[kj+1]/(1.0-alfj);
                            i = i+1; j = j-1; kj = kj-1;
                            }
                      first = first-1; last = last+1;
                      }
              } /* End of removing knot, u = U [a] ∗ /
            if (a! = p) /∗ Load the knot ua ∗/
                for (i=0; i<ph-oldr; i++)
                      { Uh[kind] = ua; kind = kind+1; }
            for (j=lbz; j<=rbz; j++) /* Load ctrl pts into Qw */
                { Qw[cind] = ebpts[j]; cind = cind+1; }
            if (b < m)
                { /* Set up for next pass thru loop */
                lbz = 1;
                for (j=0; j<r; j++) bpts[j] = Nextbpts[j];
                for (j=r; j<=p; j++) bpts[j] = Pw[b-p+j];
                a = b; b = b + 1; ua = ub;
                }
                else
                      /* End knot */
                      for (i=0; i<=ph; i++) Uh[kind+i] = ub;
            } /* End of while-loop (b < m) */
    nh = mh-ph-1;
    }
```

图 5.37a − 5.37 d 显示了几个曲线度数提升的例子。原始的三次曲线 (图 5.37a) 在图 5.37 b, 5.37c 、5.37 d 中分别提升到四次、五次和七次。注意，控制多边形在度数提升时会收敛到曲线上。

设 $\mathbf{S}^w(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(u)\, N_{j,q}(v)\, \mathbf{P}_{i,j}^w$ 为 NURBS 曲面。通过将其应用于控制点的行/列来实现曲面的度数提升。特别地，我们通过将算法 A5.9 应用于每个 $m+1$ 列的控制点来提升 p (u 方向) 的度数。通过将算法 A5.9 应用于每个 $n+1$ 行的控制点来提升 v 方向的度数 q 。一个高效的曲面度数提升算法组织需要存储 Bézier 条带的算法是 A5.10。

ALGORITHM A5.10

```
DegreeElevateSurface(n, p, U, m, q, V, Pw, dir, t, nh, Uh, mh, Vh, Qw)
    { /* Degree elevate a surface t times. */
        /* Input: n, p, U, m, q, V, Pw, dir, t */
        /* Output: nh, Uh, mh, Vh, Qw */
```

210 基本几何算法 $\{0,0,0,0,4/10,7/10,1,1,1,1\}$ ；(b) 度数提升了一次；(c) 度数提升了两次；(d) 度数提升了四次。

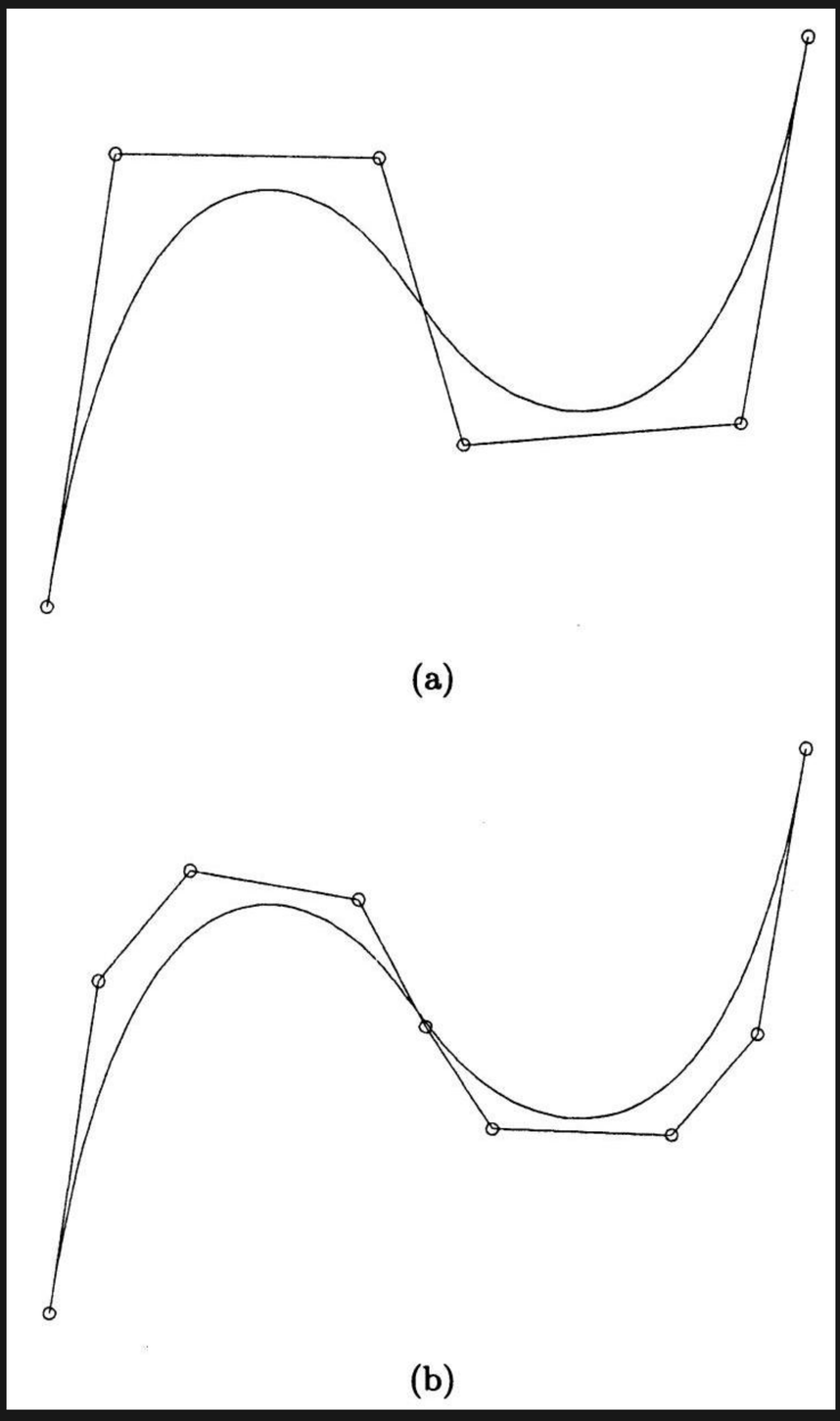


图 5.37. 曲线度数提升示例。(a) 定义在

```
if (dir == U_DIRECTION)
    {
    allocate memory for Bezier and NextBezier strips;
    initialize knot vectors and first row of control points;
    initialize Bezier strip;
    set variables;
    while(b<m)
      {
      get multiplicity;
      get ub, r, oldr, etc;
      save alfas
```

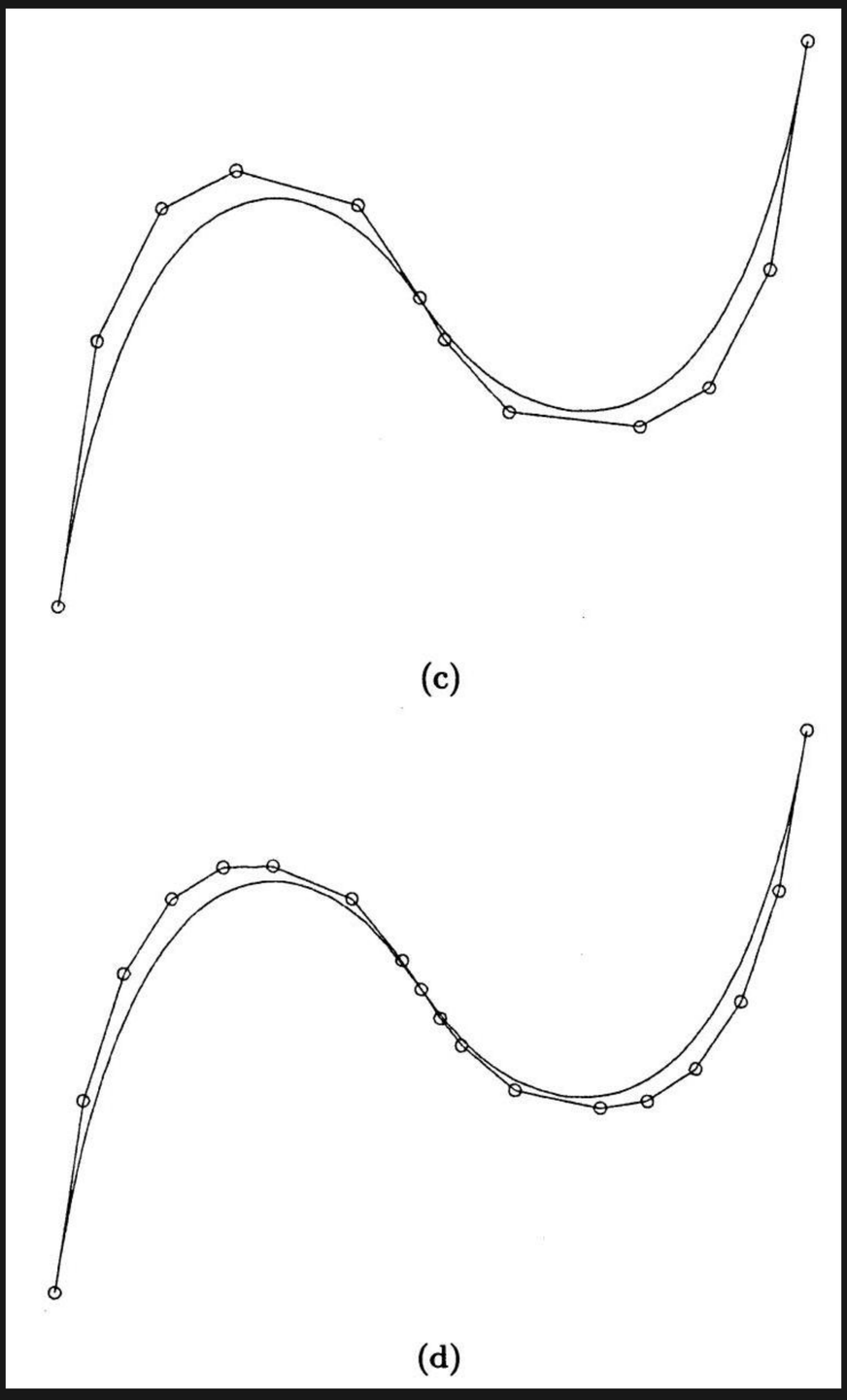


图 5.37. (续。)

```
对于 (j=0; j<=m; j++)
{
插入节点;
提升 Bezier 行 [j] 的度数;
移除节点;
保存新的控制点;
为下一次遍历初始化;
}
更新节点向量;
更新变量;
获取端点节点;
}
}
if (dir == V_DIRECTION)
    {
    /* Similar code as above with u - and v -directional
       parameters switched */
    }
}
```

表面度数提升的例子如图 5.38a-5.38e 所示。原始的 (3,2) 次表面 (图 5.38a) 提升至 $(4,3),(5,4),(6,6)$，而图 5.38 b − 5.38e 中分别为 (9,9) 次；原始网以虚线表示，提升后的网以实线绘制。

6 次数降低

设 $\mathbf{C}(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{Q}_i$ 为节点向量上的一个 p 阶 B-spline 曲线

$$U=\{\underbrace{a,\ldots,a}_{p+1},\underbrace{u_1,\ldots,u_1}_{m_1},\ldots,\underbrace{u_s,\ldots,u_s}_{m_s},\underbrace{b,\ldots,b}_{p+1}\}$$

像往常一样，$\mathbf{C}(u)$ 可以是有理或无理的；在本节的其余部分，我们省略 w 上标。$\mathbf{C}(u)$ 是度可约的，如果它具有如下精确表示形式

$$\mathbf{C}(u)=\widehat{\mathbf{C}}(u)=\sum_{i=0}^{\widehat{n}}N_{i,p-1}(u)\,\mathbf{P}_i \tag{5.37}$$

在节点向量上

$$\widehat{U}=\{\underbrace{a,\ldots,a}_{p},\underbrace{u_1,\ldots,u_1}_{m_1-1},\ldots,\underbrace{u_s,\ldots,u_s}_{m_s-1},\underbrace{b,\ldots,b}_{p}\} \tag{5.38}$$

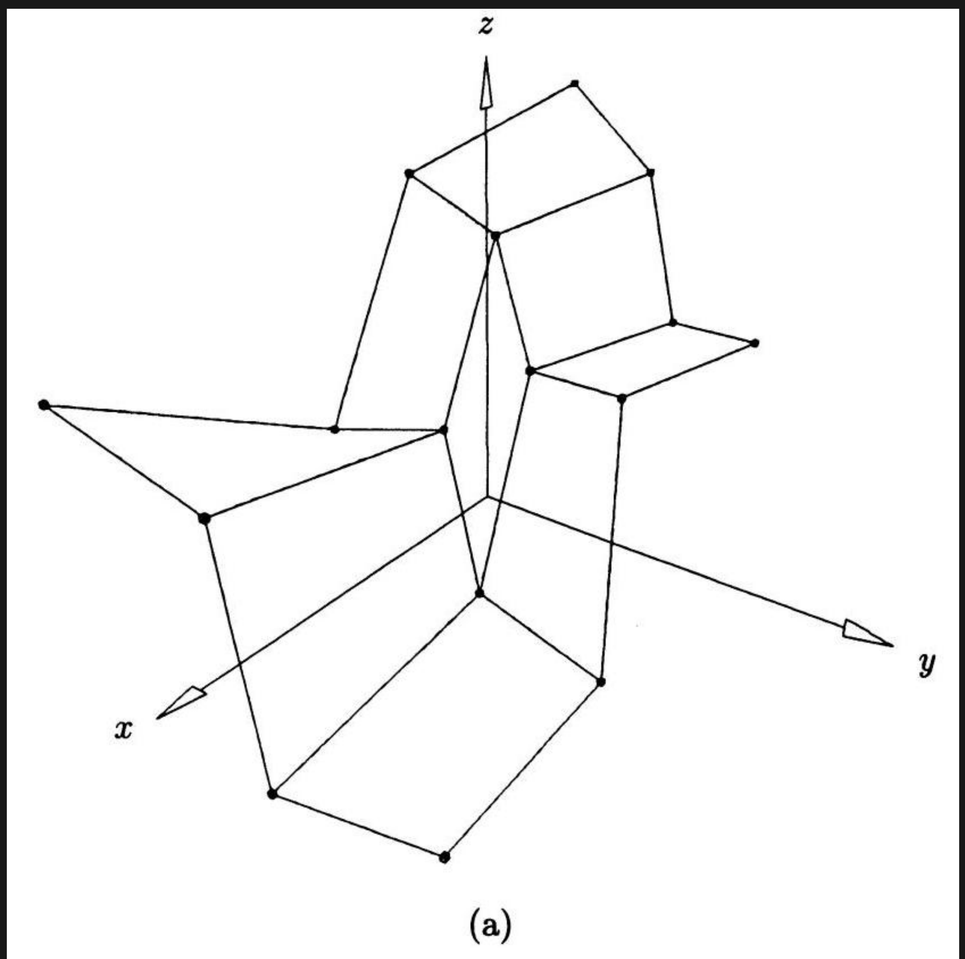
清晰地

$$\widehat{n}=n-s-1 \tag{5.39}$$

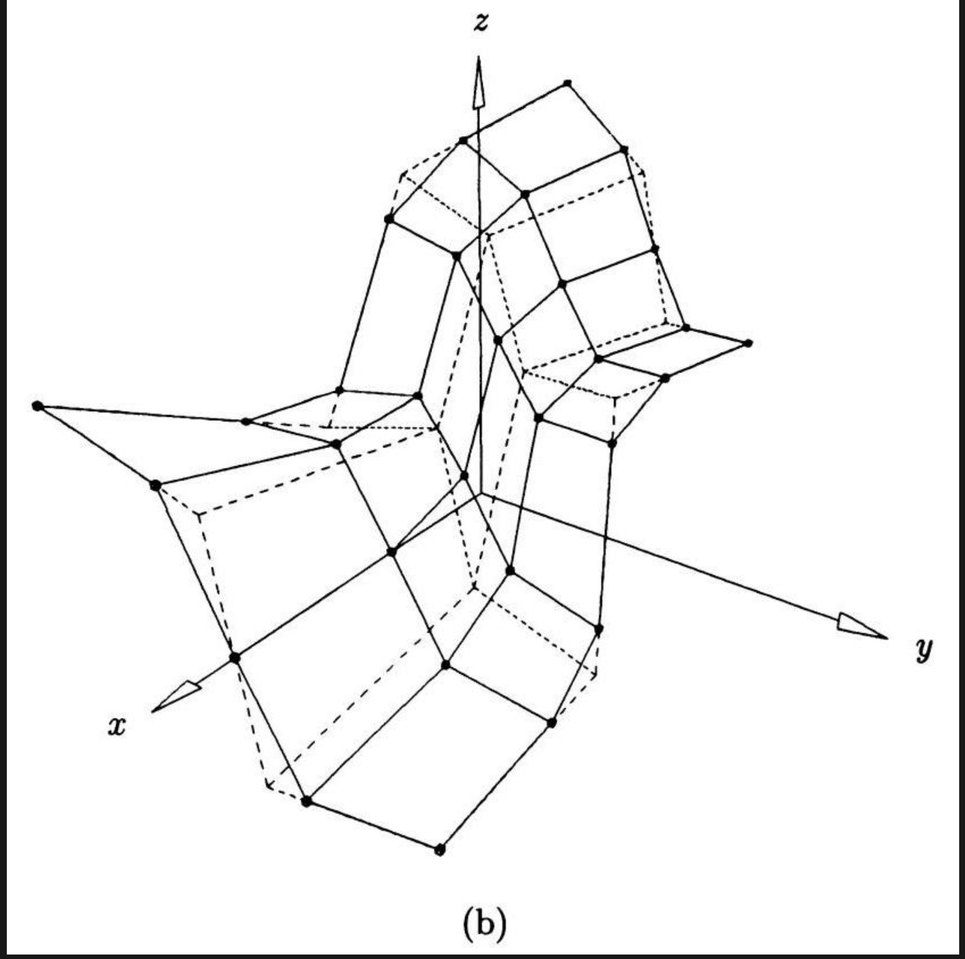
注意到 m_i 可以是 1，这意味着结 u_i 不存在于 $\widehat{U}$ 中。这意味着 u_i 可以从 $\mathbf{C}(u)$ 中精确移除。虽然总是可以提升曲线的次数，但显然曲线可能并不具有可约性。这种情况与结的移除类似，因为该问题是过度确定的，即，任何用于次数减少的算法必须产生比未知数 $\mathbf{P}_i$ 更多的方程。由于浮点数舍入误差，永远无法期望 $\widehat{\mathbf{C}}(u)$ 与 $\mathbf{C}(u)$ 精确重合，因此我们的算法必须测量误差 $E(u)=\left|\mathbf{C}(u)-\widehat{\mathbf{C}}(u)\right|$，并仅在某些用户指定的容差 TOL 下声明 $\mathbf{C}(u)$ 为可约的。

$$\max_u E(u)\le \mathrm{TOL}$$

在本节的大部分内容中，我们仅考虑精确的次数减少，即，我们假设 TOL 非常小。主要应用是逆转次数提升过程。例如，在分解复合体时，应对每个组成部分应用次数减少。



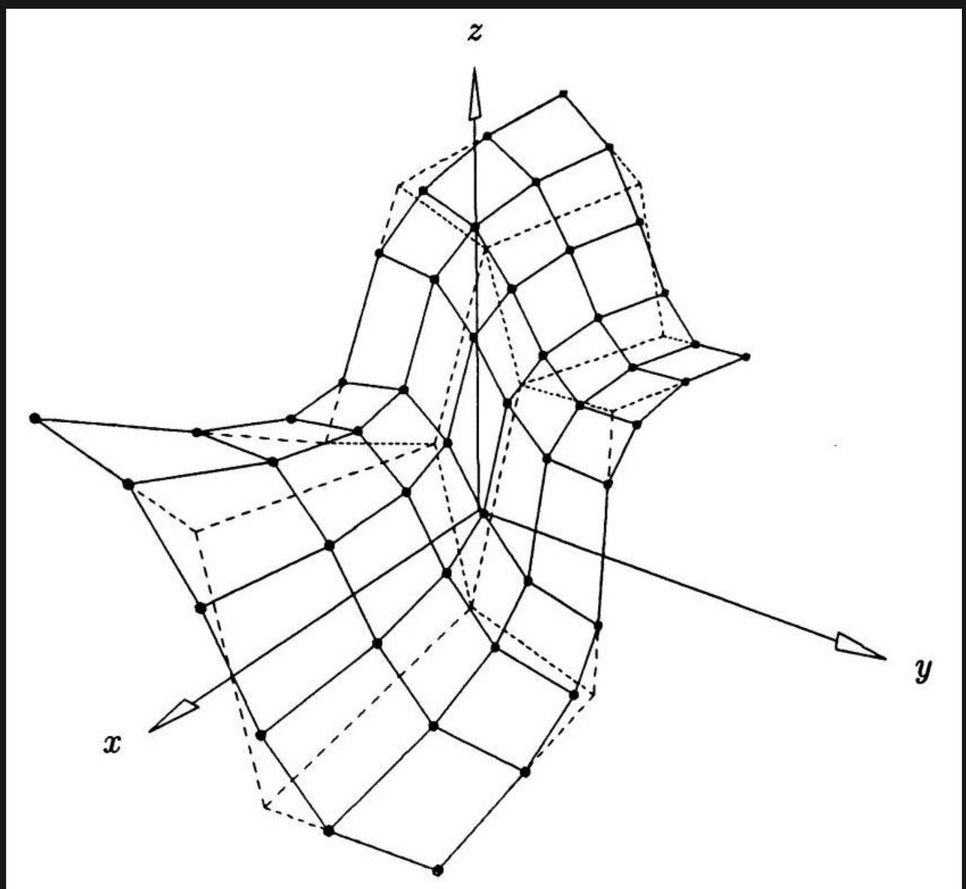
(a)



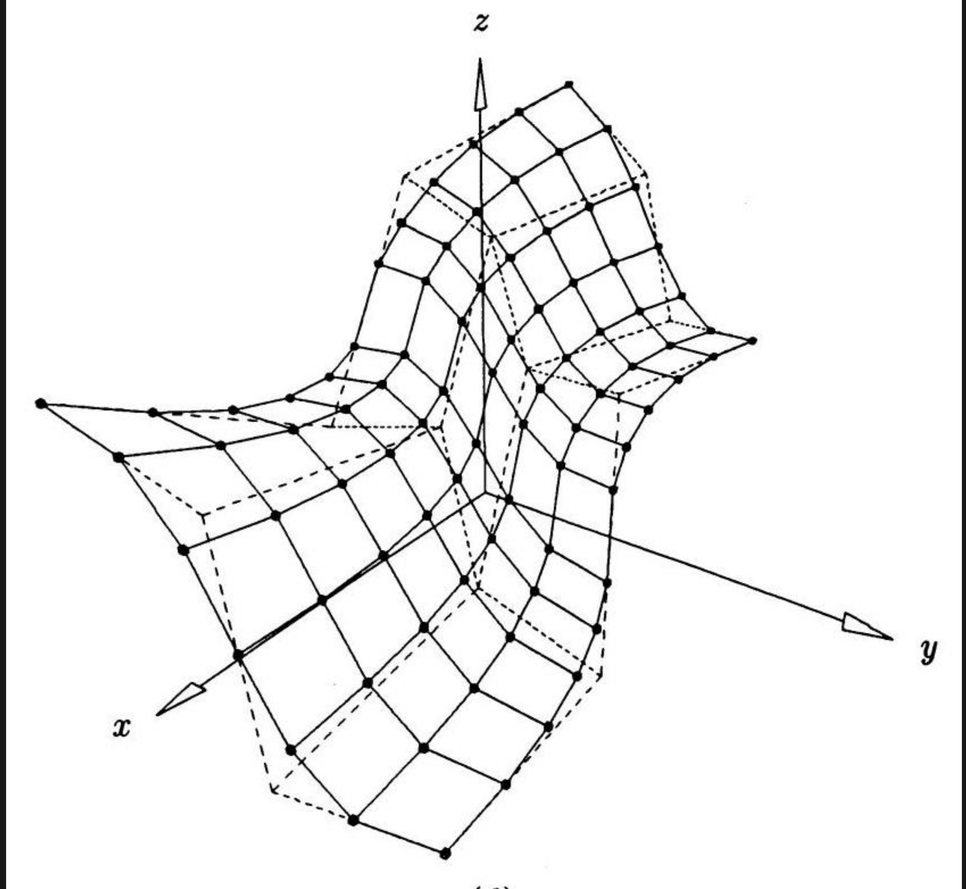
(b)

图 5.38. 表面度数提升示例。(a) 原始的 (立方 × 二次) 表面网，定义在 $U = \{0, 0, 0, 0, 1, 1, 1, 1\}$ 和 $V = \{0, 0, 0, 1/2, 1, 1, 1\}$ 上；(b) 在两个方向上度数提升一个；(c) 在两个方向上度数提升两个；(d) 在 u 方向上度数提升三个，在 v 方向上度数提升四个；(e) 在 u 方向上度数提升六个，在 v 方向上度数提升七个。214 基本几何算法

图 5.38. (续)



(c)



(d)

(*Continued.*)

曲线，因为在组合过程中可能需要进行度数提升。本节的材料取自 [Pieg95]。

贝塞尔曲线的降阶相对比较容易理解，现有多种算法（例如，参见参考文献 [Forr72; Dann85; Lach88; Watk88;

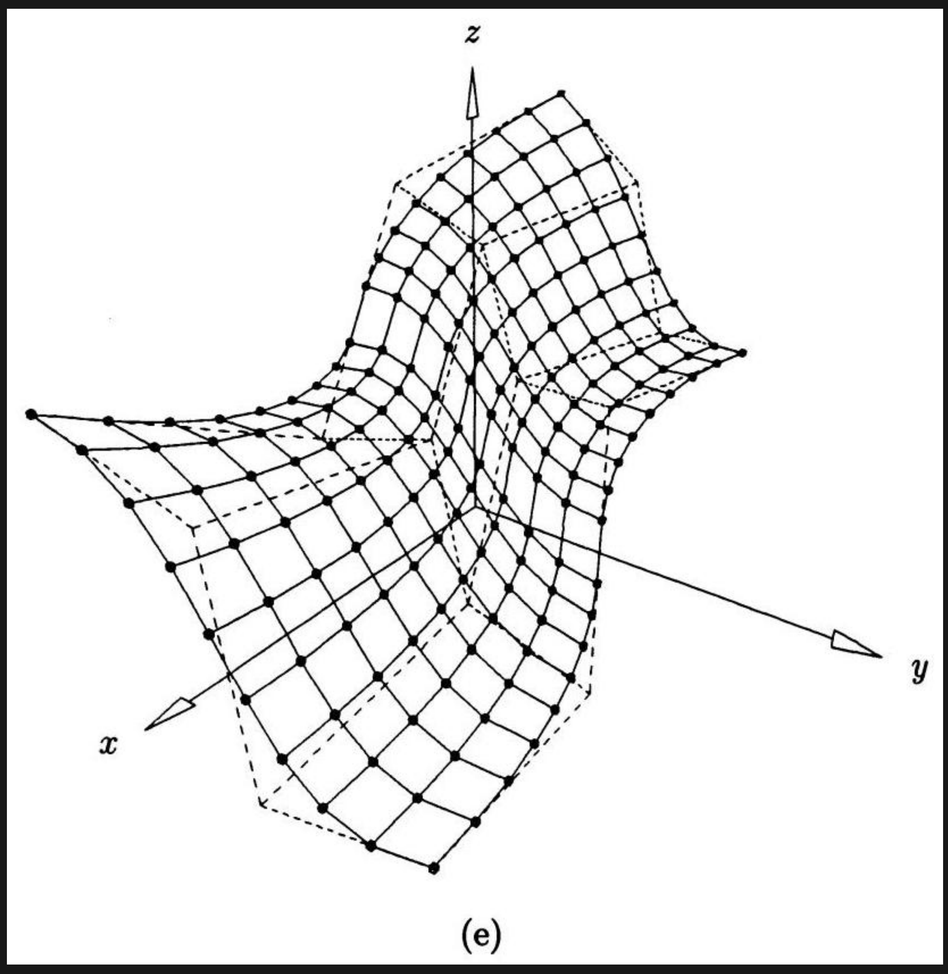


图 5.38. (续。)

Wein92; Eck93])。遵循上一节中为升阶开发的策略，我们提出了一个用于 B 样条曲线降阶的三步算法:

repeat

extract the i th Bézier segment from the B-spline curve,

degree reduce the i th Bézier piece,

remove unnecessary knots between the(i - 1)th and the

ith segments,

until done

在贝塞尔降阶和节点移除步骤中都会产生误差，我们的算法会累积这两种误差，并在任何节点区间超过 TOL 时立即退出。图 5.39a – 5.39 h 展示了算法工作原理的演示示例。每个步骤的详细信息如下:

a. 定义在节点向量上的原始四次 B 样条曲线

$$U = \{0, 0, 0, 0, 0, u_1, u_1, u_2, u_2, 1, 1, 1, 1, 1\}$$

坐标系统 (u, E) 用于绘制每个节点区间的误差，作为 u 的函数，随着算法遍历整个节点向量；

b. 节点 u_1 被插入两次，使总重数达到四；获得第一个 Bézier 片段；

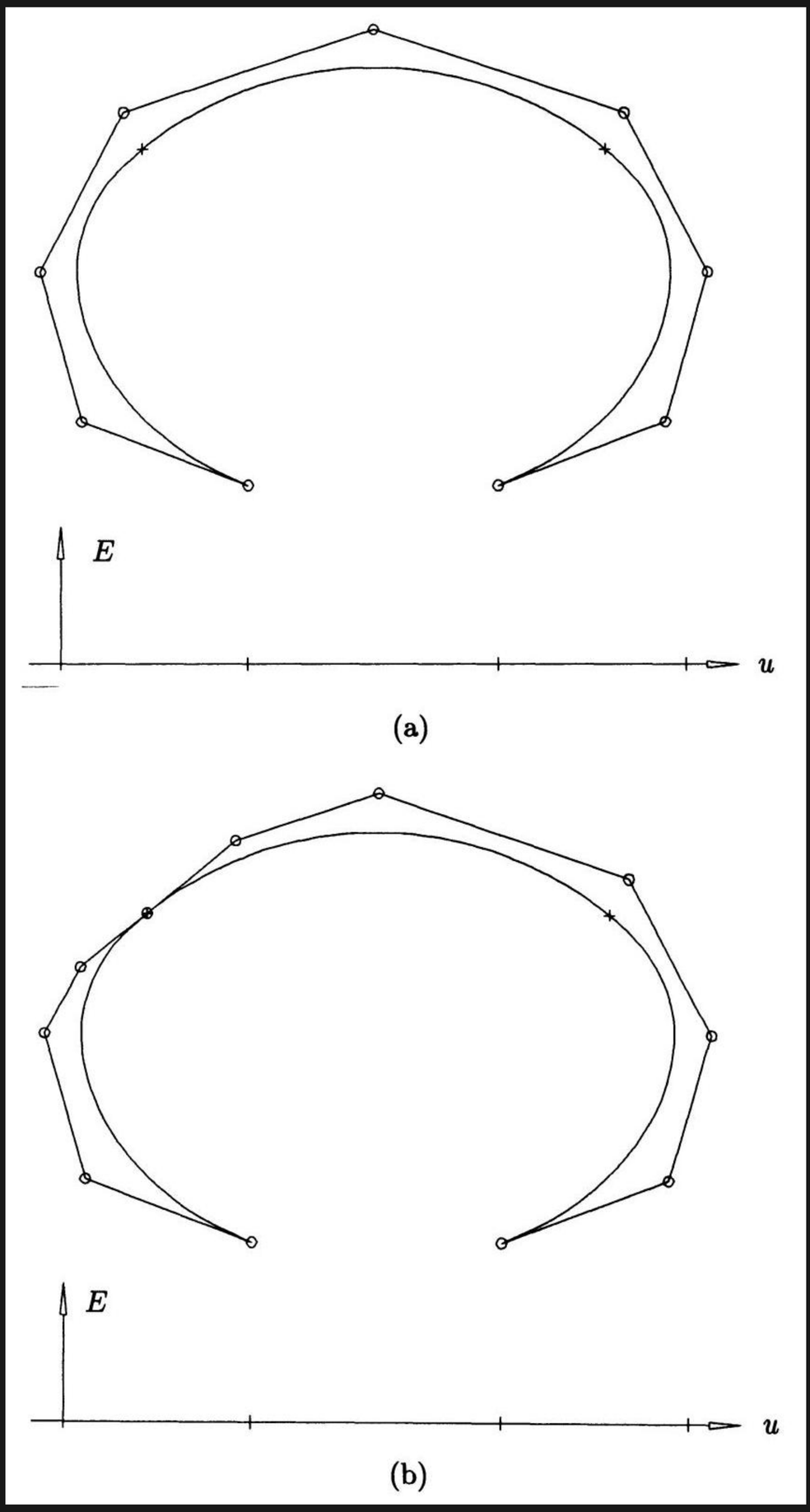
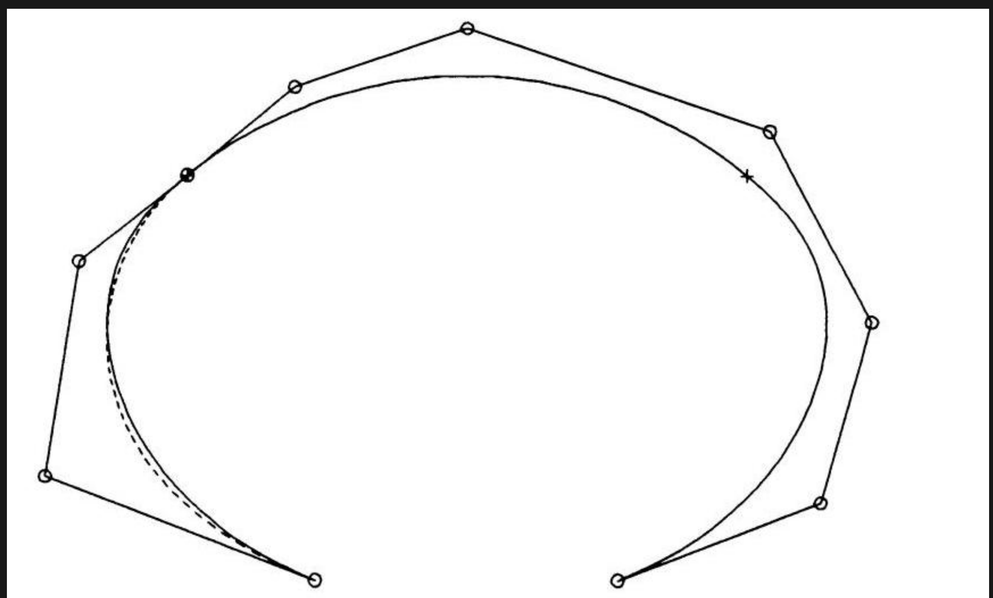
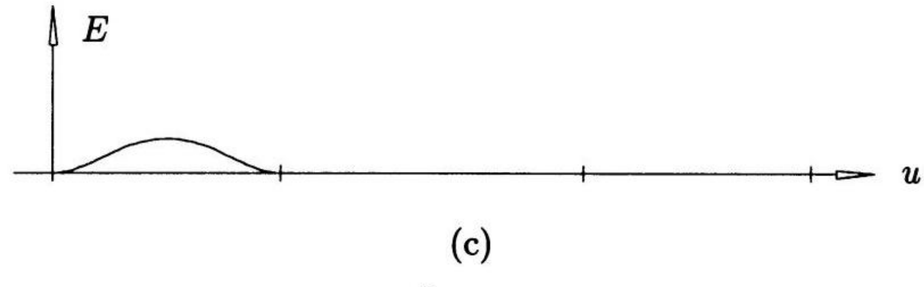


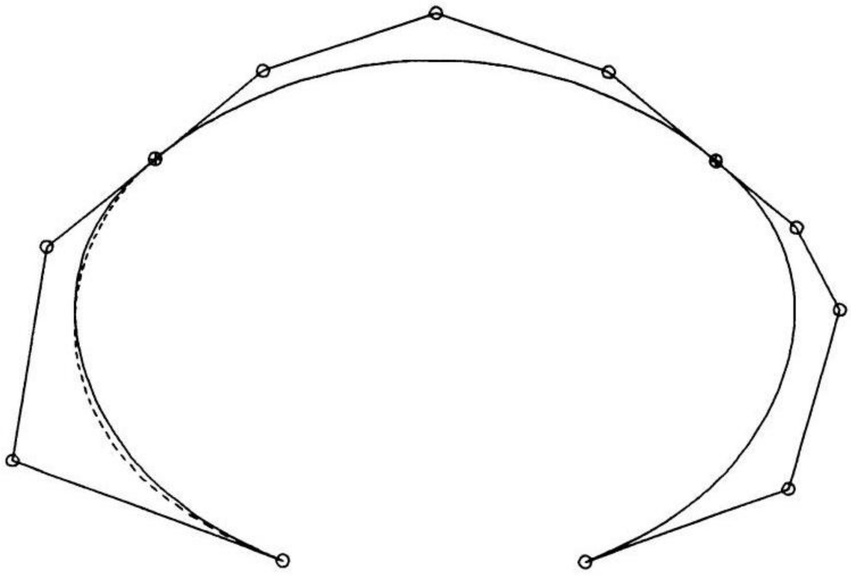
图 5.39. 将四次 B 样条曲线降阶到三次的过程。(a) 要降阶的四次 B 样条曲线；(b) 提取第一个 Bézier 段。

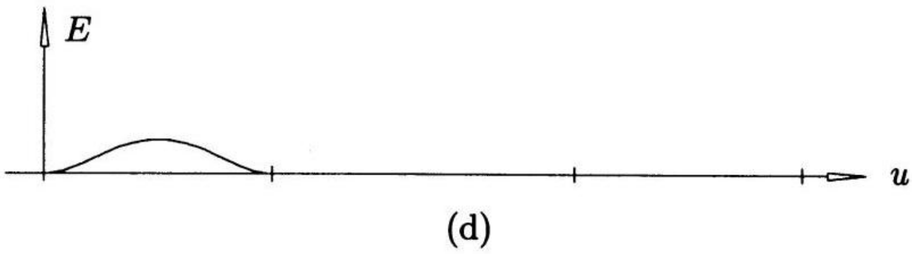
c. 第一个 Bézier 段被降阶，用四个新控制点替代前五个控制点；这是引入误差的第一次，如 (u, E) 系统中所示。实线曲线是原始曲线，而虚线曲线是近似曲线；





(c)





(d)

图 5.39. 续。(c) 第一个 Bézier 段被降阶；(d) 提取第二个 Bézier 段。
d. 节点 u_2 被插入两次以获得第二个 Bézier 段；
e. 第二个 Bézier 段被降阶。现在在两个段上存在误差；这两个都是 Bézier 降阶误差；
f. 删除两个节点 u_1 ；注意节点的移除引入了额外的误差，影响了多个节点区间；

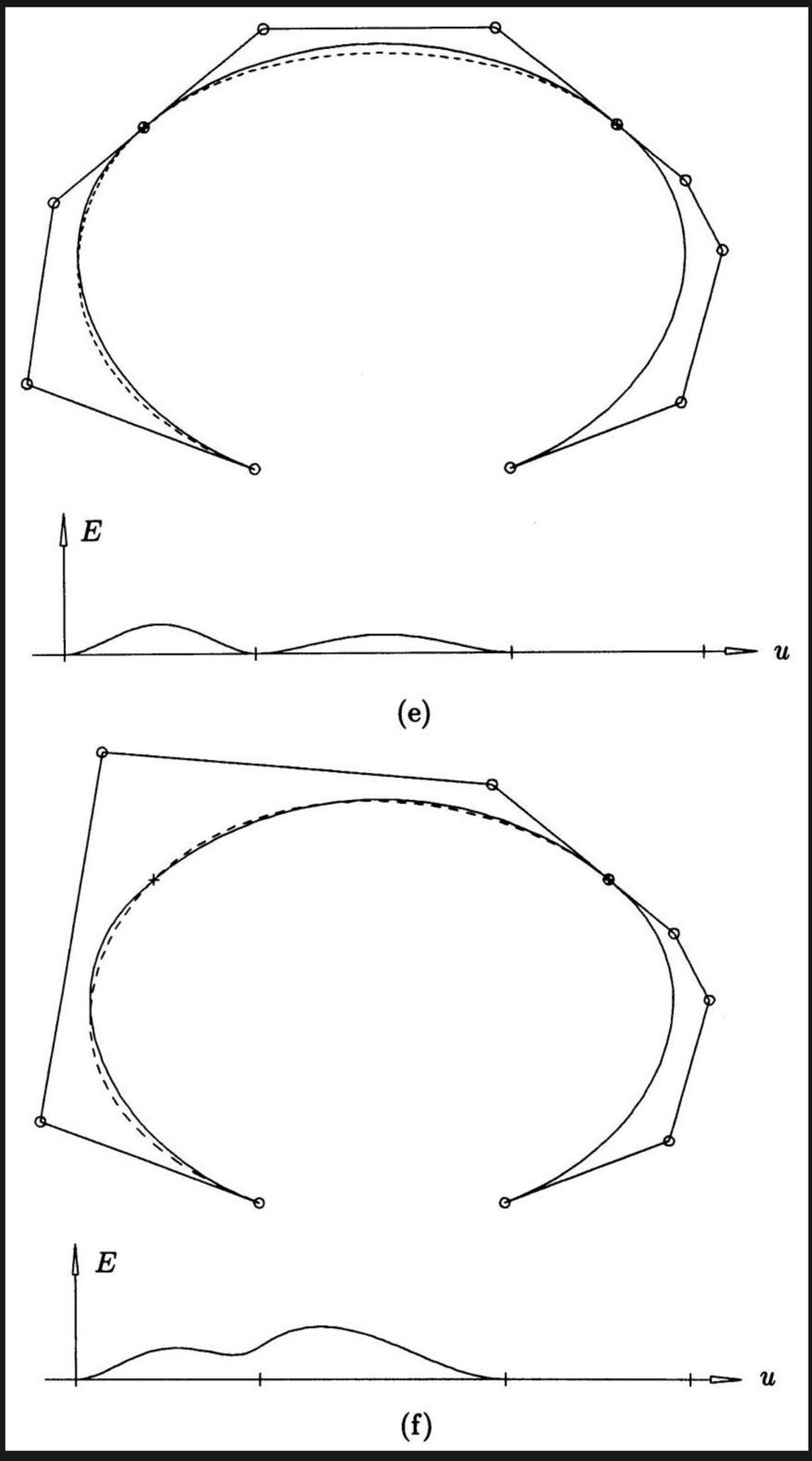


图 5.39. 续。(e) 第二个 Bézier 段被降阶；(f) 第一个内部多个节点被移除两次。
g. 最后一个 Bézier 段被降阶；
h. 删除两个 u_2 的出现，得到最终曲线。
曲线分解、节点移除以及节点移除误差的界定在第 5.3 和 5.4 节中进行了讨论。

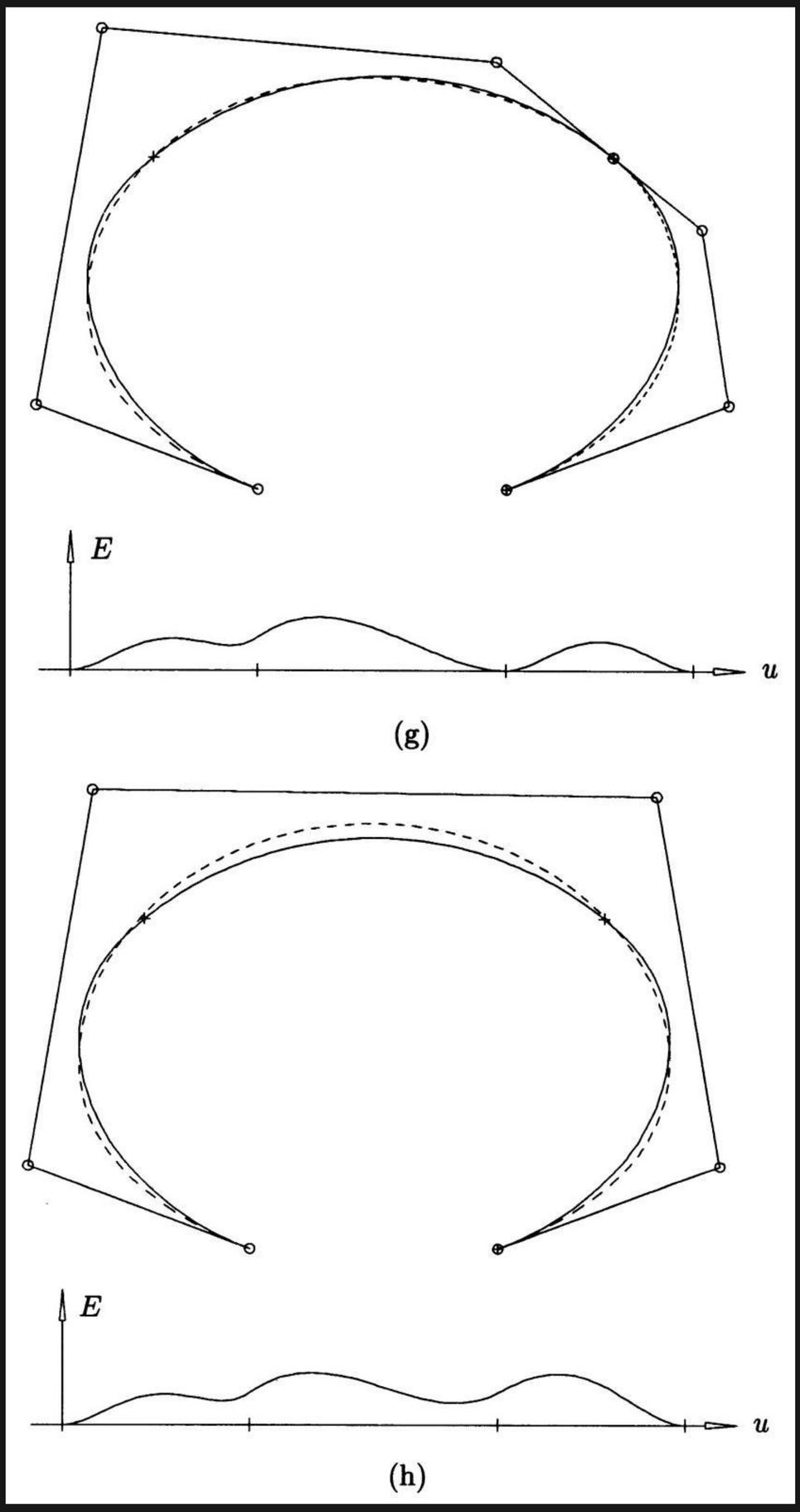


图 5.39. 继续。(g) 第三个贝塞尔段的度数降低；(h) 第二个内部多重节点被移除两次。

现在我们关注贝塞尔度数降低。设

$$\mathbf{C}(u)=\sum_{i=0}^{p}B_{i,p}(u)\,\mathbf{Q}_i$$

为一个 p 阶贝塞尔曲线，并用其度数降低的对应曲线表示为

$$\widehat{\mathbf{C}}(u)=\sum_{i=0}^{p-1}B_{i,p-1}(u)\,\mathbf{P}_i$$

我们通过反转贝塞尔度数提升公式来计算未知的 $\mathbf{P}_i$，公式为 (5.34) 和 (5.35)；设

$$r=\frac{p-1}{2} \tag{5.40}$$

(整数除法) 并用 α_i 表示度数提升系数，即

$$\alpha_i=\frac{i}{p}$$

(我们需要从 $p-1$ 到 p 的度数提升系数)。需要考虑两种情况，p 为偶数，且 p 为奇数。首先假设 p 为偶数。通过求解公式 (5.35) 得到 $\mathbf{P}_i$，我们得到

$$\mathbf{P}_0=\mathbf{Q}_0$$

$$\mathbf{P}_i=\frac{\mathbf{Q}_i-\alpha_i\mathbf{P}_{i-1}}{1-\alpha_i}\;i=1,\ldots,r$$

$$\mathbf{P}_i=\frac{\mathbf{Q}_{i+1}-(1-\alpha_{i+1})\,\mathbf{P}_{i+1}}{\alpha_{i+1}}\;i=p-2,\ldots,r+1$$

$$\mathbf{P}_{p-1}=\mathbf{Q}_p \tag{5.41}$$

现在假设 p 为奇数；则 $\mathbf{P}_i$ 的计算为

$$\mathbf{P}_0=\mathbf{Q}_0$$

$$\mathbf{P}_i = \frac{\mathbf{Q}_i - \alpha_i \mathbf{P}_{i-1}}{1-\alpha_i} \; i = 1, \ldots, r-1$$

$$\mathbf{P}_i = \frac{\mathbf{Q}_{i+1} - (1-\alpha_{i+1})\,\mathbf{P}_{i+1}}{\alpha_{i+1}} \; i = p-2, \ldots, r+1$$

$$\mathbf{P}_r = \frac{1}{2}\left(\mathbf{P}_r^L + \mathbf{P}_r^R\right)$$

$$\mathbf{P}_{p-1} = \mathbf{Q}_p \tag{5.42}$$

其中

$$\mathbf{P}_r^L = \frac{\mathbf{Q}_r - \alpha_r \mathbf{P}_{r-1}}{1-\alpha_r}$$

$$\mathbf{P}_r^R = \frac{\mathbf{Q}_{r+1} - (1-\alpha_{r+1})\,\mathbf{P}_{r+1}}{\alpha_{r+1}}$$

请注意，奇数情况与偶数情况的不同之处在于，出于对称原因，中间控制点 $\mathbf{P}_r$ 被计算为从左侧和右侧分别计算的两个分量的平均值。一般来说，得到的 (p - 1) 阶曲线 $\widehat{\mathbf{C}}(u)$ 是对 $\mathbf{C}(u)$ 的近似；它们仅在 $\mathbf{C}(u)$ 精确可降阶时重合。Piegl 和 Tiller [Pieg95] 推导了近似的误差界限

p 偶数:

$$\left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| = \left|B_{r+1,p}(u)\left[\mathbf{Q}_{r+1} - \frac{1}{2}\left(\mathbf{P}_r + \mathbf{P}_{r+1}\right)\right]\right| \tag{5.43}$$

p 奇数:

$$\left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| = \frac{1}{2}(1-\alpha_r)\left|\left(B_{r,p}(u) - B_{r+1,p}(u)\right)\left(\mathbf{P}_r^L - \mathbf{P}_r^R\right)\right| \tag{5.44}$$

图 5.40a 和 5.40b 显示了将贝塞尔曲线的度数从七降至六，以及从六降至五的示例。误差曲线在图的底部绘制。我们注意到:

- 方程 (5.43) 和 (5.44) 表达了参数误差 (在对应参数值处点之间的距离)；几何误差和参数误差的最大值不一定在相同的 u 值处；
- 即使 $\mathbf{C}(u)$ 不能降度，方程 (5.41) 和 (5.42) 从左侧产生 $p/2$ 精确控制点，从右侧产生 $p/2$ 精确控制点，这意味着如果 $\widehat{\mathbf{C}}(u)$ 的度数提升以产生 $\overline{\mathbf{C}}(u)$，那么 $\mathbf{C}(u)$ 和 $\overline{\mathbf{C}}(u)$ 在首尾的 $p/2$ 控制点是相同的。因此，$\mathbf{C}(u)$ 和 $\widehat{\mathbf{C}}(u)$ 在两端的导数至多为 $p/2-1$ 阶相同。这是 B 样条降度的重要性质，因为这意味着在贝塞尔降度步骤中保持至多 $C^{p/2-1}$ 的连续性，这反过来意味着前 $p/2-1$ 个节点移除步骤不会产生误差；
- 对于 p，即使最大误差发生在 $u = 1/2$ (见图 5.40b)；对于 p 奇数，误差在 $u = 1/2$ 处为零，并且在 $u = 1/2$ 的左右各有两个峰值 (图 5.40a)。奇数情况在其他地方由 Piegl 和 Tiller [Pieg95] 进行了更详细的分析。

我们现在提出一种算法，用于在最大误差容忍度 TOL 的限制下降低 B 样条曲线的阶数。如果曲线是有理的，则应根据公式 (5.30) 调整 TOL。我们维护一个误差向量 $e_i, i = 0, \ldots, m-1$，用于累积每个节点区间的误差。e_i 初始化为零。当第 i 个区间进行 Bézier 阶数降低时，产生的误差会被添加到 e_i 中。为简化起见，我们从公式 (5.43) 和 (5.44) 中省略了标量函数 (其值被限制在 1 之内)，并使用最大误差界限。

$$\left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| \leq \left|\mathbf{Q}_{r+1} - \frac{1}{2}\left(\mathbf{P}_r + \mathbf{P}_{r+1}\right)\right| \tag{5.45}$$

和

$$\left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| \leq \left|\mathbf{P}_r^L - \mathbf{P}_r^R\right| \tag{5.46}$$

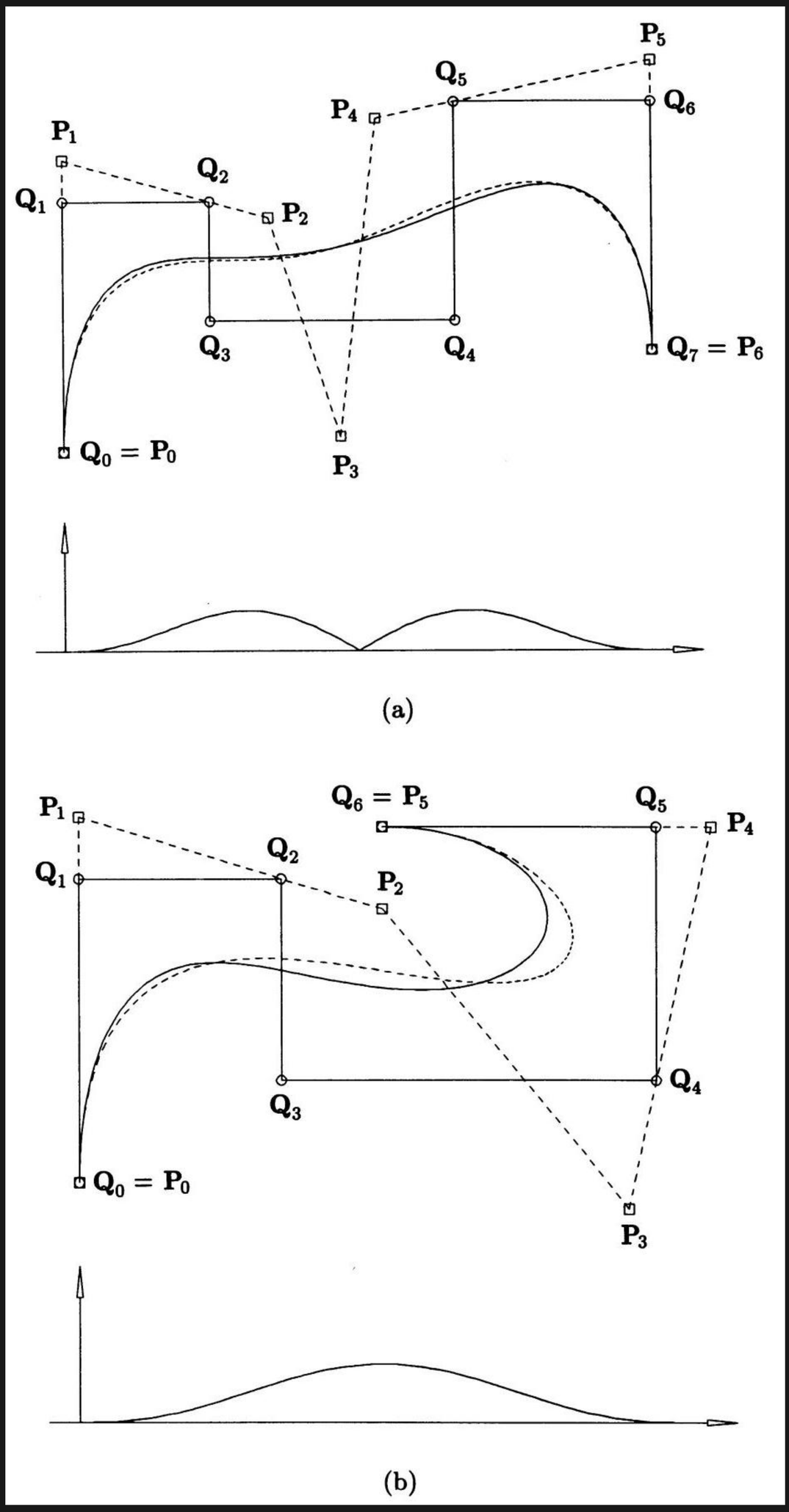


图 5.40. Bézier 阶数降低。(a) 从七阶到六阶；(b) 从六阶到五阶。

[Pieg95] 中的算法为 Bézier 阶数降低和节点移除步骤计算了更严格的误差界限。当移除一个节点时，最大节点移除误差会被添加到每个受移除影响的 e_i 。使用的局部数组为:

bpts [p+1] 2+1] : Bézier control points of the current segment;

Nextbpts [p-1] : leftmost control points of the next Bézier segment;

rbpts[p] : degree reduced Bézier control points;

alphas [p-1] : knot insertion alphas;

e[m] : error vector.

一个支持例程 BezDegreeReduce(bpts, rbpts, MaxErr)，它实现了 Bézier 阶数降低和最大误差的计算。该例程使用公式 (5.41)、(5.42)、(5.45) 和 (5.46)。返回代码 1 表示曲线无法降低阶数；如果曲线可降低阶数，则计算 $\overline{n}, \widehat{U}$ 和 $\mathbf{P}_i$ (nh, Uh, Pw)，并返回代码 0。

ALGORITHM A5.11

```
DegreeReduceCurve(n, p, U, Qw, nh, Uh, Pw)
    { /* Degree reduce a curve from p to p-1. */
        /* Input: n, p, U, Qw */
        /* Output: nh, Uh, Pw */
    ph = p − 1; mh = ph; /* Initialize some variables */
    kind = ph + 1; r = −1; a = p ;
    b = p+1; cind = 1; mult = p;
    m = n+p+1; Pw[0] = Qw[0];
    for (i=0; i<=ph; i++) /* Compute left end of knot vector */
        Uh[i] = U[0];
```

```
        for (i=0; i<=p; i++) /* Initialize first Bézier segment */
              bpts[i] = Qw[i];
        for (i=0; i<m; i++) /* Initialize error vector */
              e[i] = 0.0 ;
                    /* Loop through the knot vector */
        while (b < m)
              { /* First compute knot multiplicity */
              i = b;
              while (b < m&&U[b] == U[b + 1]) b = b + 1 ;
              mult = b-i+1; mh = mh+mult-1;
              oldr = r; r = p-mult;
              if ( oldr  > 0) lbz = ( oldr  + 2) /2 ; else lbz = 1 ;
                          /* Insert knot U[b] r times */
              if (r > 0 )
                    {
                    numer = U[b]-U[a];
                    for (k=p; k>=mult; k--)
            alphas[k-mult-1] = numer/(U[a+k]-U[a]);
      for (j=1; j<=r; j++)
            {
            save = r-j; s = mult+j;
            for (k=p; k>=s; k--)
              bpts[k] = alphas[k-s]*bpts[k]
                                                  + (1.0-alphas[k-s])*bpts[k-1];
            Nextbpts[save] = bpts[p];
            }
      }
            /* Degree reduce Bézier segment */
BezDegreeReduce(bpts, rbpts, MaxErr);
e[a] = e[a]+MaxErr;
if (e[a] > TOL)
      return(1); /* Curve not degree reducible */
            /* Remove knot U[a] oldr times */
if (oldr > 0 )
      {
      first = kind; last = kind;
      for (k=0; k<oldr; k++)
            {
            i = first; j = last; kj = j-kind;
            while (j − i > k)
              {
              alfa = (U[a]-Uh[i-1])/(U[b]-Uh[i-1]);
              beta = (U[a]-Uh[j-k-1])/(U[b]-Uh[j-k-1]);
              Pw[i-1] = (Pw[i-1]-(1.0-alfa)*Pw[i-2])/alfa;
              rbpts[kj] = (rbpts[kj]-beta*rbpts[kj+1])/(1.0-beta);
              i = i+1; j = j-1; kj = kj-1;
              }
              /* Compute knot removal error bounds (Br) */
            if (j − i < k) Br = Distance4D(Pw[i − 2], rbpts[kj + 1]) ;
              else
              {
              delta = (U[a]-Uh[i-1])/(U[b]-Uh[i-1]);
              A = delta*rbpts[kj+1] + (1.0-delta)*Pw[i-2];
              Br = Distance4D(Pw[i-1], A);
              }
              /* Update the error vector */
            K = a + oldr − k; q = (2 ∗ p − k + 1) /2;
            L = K − q;
              for (ii=L; ii<=a; ii++)
                { /* These knot spans were affected */
                e[ii] = e[ii] + Br;
                if (e[ii] > TOL)
                                    return (1); /* Curve not degree reducible */
                                  }
                              first = first-1; last = last+1;
                            } /* End for (k=0; k<oldr; k++) loop */
                        cind = i-1;
                        } /* End if (oldr > 0) */
                            /* Load knot vector and control points ∗ /
                    if (a != p)
                        for (i=0; i<ph-oldr; i++)
                              { Uh[kind] = U[a]; kind = kind+1; }
                    for (i=lbz; i<=ph; i++)
                        { Pw[cind] = rbpts[i]; cind = cind+1; }
                              /* Set up for next pass through */
                    if (b < m)
                        {
                        for (i=0; i<r; i++) bpts[i] = Nextbpts[i];
                        for (i=r; i<=p; i++) bpts[i] = Qw[b-p+i];
                        a = b; b = b + 1;
                        }
                    else
                        for (i=0; i<=ph; i++) Uh[kind+i] = U[b];
                    } /* End of while (b < m) loop */
                  nh = mh-ph-1;
                  return(0);
              }
```

Figure 5.41 shows an example of reducing the degree from five to four. The original knot vector is

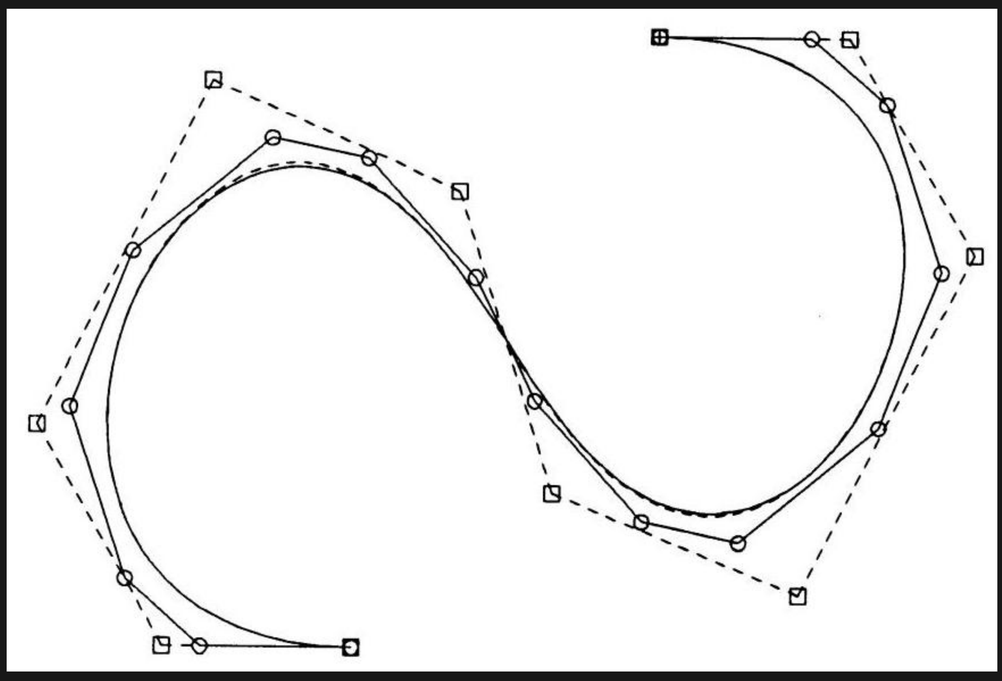


图 5.41. 将 B 样条曲线从五阶降低到四阶。

$$U = \{0, 0, 0, 0, 0, 0, 0.15, 0.15, 0.3, 0.3, 0.5, 0.5,$$

$$0.7, 0.7, 0.85, 0.85, 1, 1, 1, 1, 1, 1\}$$

实线曲线为原始曲线，而虚线曲线为降低阶数后的曲线。图 5.42a 显示了在节点向量上定义的三次曲线。

$$U = \left\{0, 0, 0, 0, \frac{3}{10}, \frac{3}{10}, \frac{1}{2}, \frac{1}{2}, \frac{7}{10}, \frac{7}{10}, 1, 1, 1, 1\right\}$$

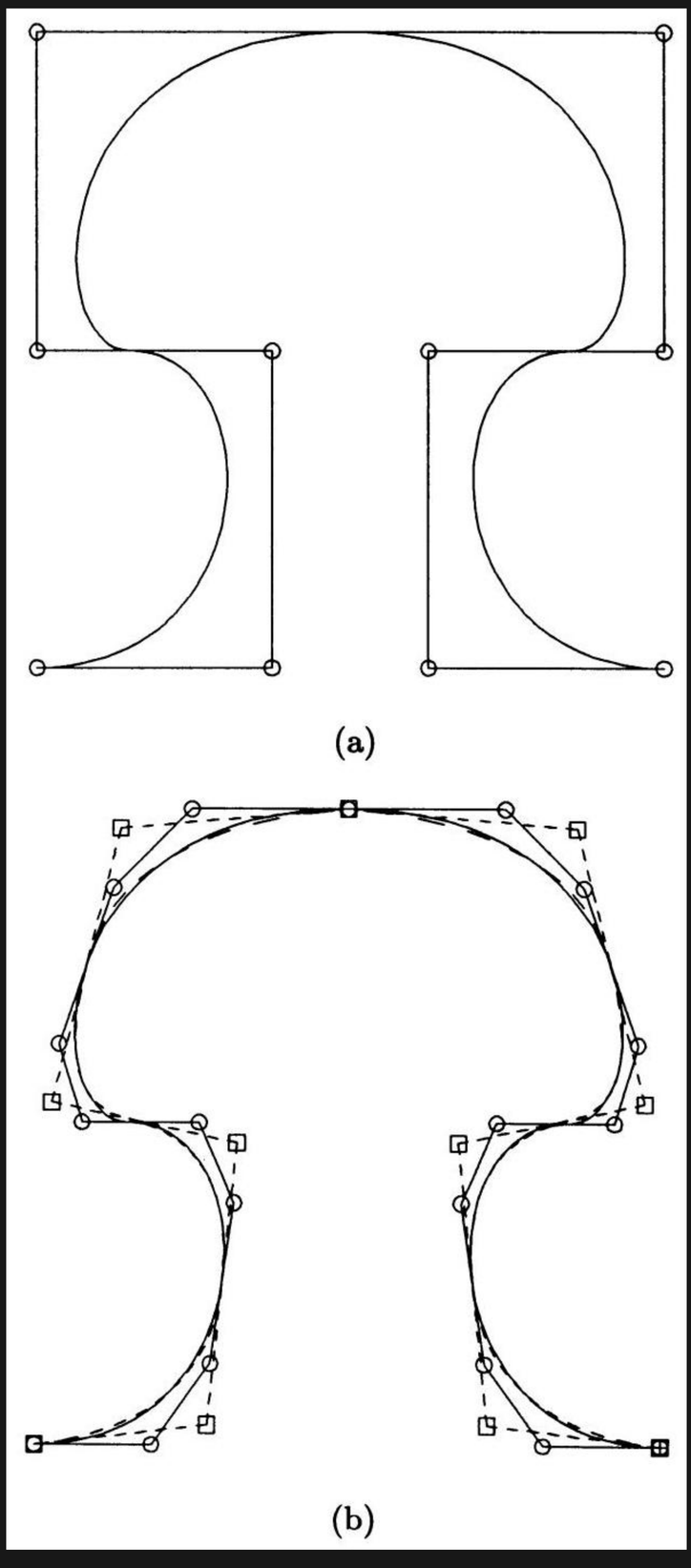


图 5.42. 一条三次曲线。(a) 三次曲线不是可降阶的；(b) 在节点细化后，该曲线是可降阶的。

不可能用二次 B 样条曲线在合理的容差范围内逼近该曲线。在细化节点向量后

$$U^{(r)} = \{0, 0, 0, 0, 0.15, 0.15, 0.3, 0.3, 0.4, 0.4, 0.5, 0.5, 0.5,$$

$$0.6, 0.6, 0.7, 0.7, 0.85, 0.85, 1, 1, 1, 1\}$$

可以得到合理的近似，如图 5.42b 所示。注意 $u = 1/2$ 处的间断。这是由于原始曲线中的三重节点。细化突出了一个有趣的观点: 通过引入至少重数为二的额外节点，可以改善近似的质量。因此，结合节点细化，本节的工具可以用于开发算法，以用低阶曲线逼近高阶曲线。

最后，我们指出，通过将这些曲线技术应用于控制点的行或列，可以降低曲面的 u 或 v 阶。

练习

5.1. 设 $p = 2$ 和 $U = \{0, 0, 0, 1, 2, 3, 4, 4, 5, 5, 5\}$.$\mathbf{C}(u) = \sum_{i=0}^{7} N_{i,2}(u)\,\mathbf{P}_i$ ，一条 B 样条曲线，在 U 上由控制点 $\{(-6, -1), (-5, 2), (-3, 3), (-1, 2)$ 和 $(0, 0), (3, 1), (3, 3), (1, 5)\}$ 定义。使用重复的节点插入，计算 $\mathbf{C}(5/4)$ 和 $\mathbf{C}'(5/4)$ 。检查方程 (5.12) - (5.14) 和 (5.16) - (5.18) 在此示例中是否成立。绘制曲线及其控制多边形。

5.2. 使用与练习 5.1 中相同的曲线。必须插入什么值的 u 作为节点，以使点 $(-3/4, 3/2)$ 成为控制点？

5.3. 考虑 B 样条曲面

$$\mathbf{S}(u, v) = \sum_{i=0}^{7}\sum_{j=0}^{8} N_{i,2}(u)\,N_{j,3}(v)\,\mathbf{P}_{i,j}$$

其中

$$U = \left\{0, 0, 0, \frac{1}{5}, \frac{3}{10}, \frac{3}{5}, \frac{4}{5}, \frac{9}{10}, 1, 1, 1\right\}$$

和

$$V = \left\{0, 0, 0, 0, \frac{1}{10}, \frac{2}{5}, \frac{1}{2}, \frac{7}{10}, \frac{4}{5}, 1, 1, 1, 1\right\}$$

假设您想在由 $3/10 < u < 6/10$ 和 $1/2 < v < 7/10$ 给出的参数空间的矩形区域内稍微修改表面形状。您希望通过添加节点 (控制点) 来实现这一点，直到您至少有一个可以自由移动的控制点，而不改变表面相对于 u 和 v 的连续性，并且不改变该矩形区域外的表面形状。您必须向 U 和 V 添加哪些节点？请明确说明数量和数值。

5.4. 考虑 B-spline 表面

$$\mathbf{S}(u, v) = \sum_{i=0}^{3}\sum_{j=0}^{2} N_{i,2}(u)\,N_{j,2}(v)\,\mathbf{P}_{i,j}$$

其中

$$U = \left\{0, 0, 0, \frac{1}{2}, 1, 1, 1\right\}$$

和

$$V = \{0, 0, 0, 1, 1, 1\}$$

和

$$\mathbf{P}_{0,0} = (0, 0, 0)\ \ \mathbf{P}_{1,0} = (3, 0, 3)\ \ \mathbf{P}_{2,0} = (6, 0, 3)\ \ \mathbf{P}_{3,0} = (9, 0, 0)$$

$$\mathbf{P}_{0,1} = (0, 2, 2)\ \ \mathbf{P}_{1,1} = (3, 2, 5)\ \ \mathbf{P}_{2,1} = (6, 2, 5)\ \ \mathbf{P}_{3,1} = (9, 2, 2)$$

$$\mathbf{P}_{0,2} = (0, 4, 0)\ \ \mathbf{P}_{1,2} = (3, 4, 3)\ \ \mathbf{P}_{2,2} = (6, 4, 3)\ \ \mathbf{P}_{3,2} = (9, 4, 0)$$

使用节点插入计算 $\mathbf{S}(3/10, 6/10)$ 。将此与练习 3.8 进行比较。

5.5. 设 $p = 3$ 和 $U = \{0, 0, 0, 0, 1, 1, 1, 2, 2, 2, 2\}$ 如图 5.26 的示例。假设控制点为 $\mathbf{P}_i^0 = \{(-1, 0), (-1, 1), (-1/2, 3/2), (1/4, 3/2), (1, 3/2), (2, 1)$ ，$(2, 0)\}$ 。确定 $u = 1$ 可以移除多少次，并计算新的控制点。您可以通过使用公式 (5.29) 或通过追踪算法 A5.8 来完成此操作。

第六章　高级几何算法

1　曲线和曲面的点反演与投影

在本章中，我们将讨论在实现 NURBS 几何时相当基础的各种主题。

给定一个点 $\mathbf{P}=(x,y,z)$，假设它位于 NURBS 曲线 $\mathbf{C}(u)$ 上，该曲线的度数为 p，点反演是寻找相应参数 $\bar{u}$ 的问题，使得 $\mathbf{C}(\bar{u})=\mathbf{P}$。理论上，如果满足 $p\leq 4$，点反演可以以封闭形式解决。步骤如下:

1. 使用强凸包性质 (P4.10)，确定 $\mathbf{C}(u)$ 的哪些跨度可能包含 $\mathbf{P}$；
2. 使用节点插入或细化，提取候选跨度并将其转换为幂基形式；
3. 每个跨度导致一个未知数的三个多项式方程；如果这三个方程有一个共同解，则 $\mathbf{P}$ 位于曲线的该段上。

作为步骤 3 的一个例子，假设 $p=2$。然后，跨度 $\mathbf{r}(u)$ 由向量函数给出 (在四维空间中):

$$\mathbf{r}(u)=\mathbf{a}_0^w+\mathbf{a}_1^w u+\mathbf{a}_2^w u^2 \tag{6.1}$$

设 $\mathbf{a}_i^w=(w_i x_i, w_i y_i, w_i z_i, w_i)$。将方程 (6.1) 投影到三维空间并设其等于 $\mathbf{P}=(x,y,z)$，我们得到

$$\frac{w_2 x_2 u^2+w_1 x_1 u+w_0 x_0}{w_2 u^2+w_1 u+w_0}=x$$

$$\frac{w_2 y_2 u^2+w_1 y_1 u+w_0 y_0}{w_2 u^2+w_1 u+w_0}=y$$

$$\frac{w_2 z_2 u^2+w_1 z_1 u+w_0 z_0}{w_2 u^2+w_1 u+w_0}=z$$

这导致 $w_2(x_2-x)u^2+w_1(x_1-x)u+w_0(x_0-x)=0$

$$w_2(y_2-y)u^2+w_1(y_1-y)u+w_0(y_0-y)=0$$

$$w_2(z_2-z)u^2+w_1(z_1-z)u+w_0(z_0-z)=0 \tag{6.2}$$

该方法有其缺点，例如:

- 方程 (6.2) 对于 $p>4$ 不能以封闭形式求解，并且 $p=3$ 和 $p=4$ 的解在计算机上并非没有问题；
- 存在数值容差问题，例如，不能期望方程 (6.2) 的三个解完全相等；但何时应将它们视为相等？当点 $\mathbf{P}$ 并不精确位于曲线上时 (这种情况很常见)，问题会加剧；
- 之前解决方案的软件实现相当复杂。

一种更简单且完全足够的方法是使用牛顿迭代来最小化 $\mathbf{P}$ 和 $\mathbf{C}(u)$ 之间的距离 (见图 6.1a 和 6.1b)。如果最小距离小于指定的容差 ϵ_1，则认为 $\mathbf{P}$ 在曲线上。这有效地解决了点投影到曲线的更一般问题。为了获得牛顿迭代的起始值 u_0：

- 如果已知该点位于曲线上 (在容差范围内)，则使用强凸包性质来确定候选跨度；如果这是一个更一般的点投影问题，则选择所有跨度作为候选；
- 在每个候选跨度上以均匀间隔的参数值评估曲线点 n，并计算每个点与 $\mathbf{P}$ 的距离。选择 u_0 作为使点最接近 $\mathbf{P}$ 的值；数字 n 通常通过某种启发式方法选择。我们强调，良好的起始值对于实现可靠的收敛性至关重要。

现在假设我们有起始值 u_0，并形成点积函数

$$f(u)=\mathbf{C}'(u)\cdot(\mathbf{C}(u)-\mathbf{P})$$

当 $f(u)=0$ 时，从 $\mathbf{P}$ 到 $\mathbf{C}(u)$ 的距离最小，无论 $\mathbf{P}$ 是否在曲线上 (图 6.1a 和 6.1b)。用 u_i 表示在第 i 次牛顿迭代中获得的参数。然后

$$u_{i+1}=u_i-\frac{f(u_i)}{f'(u_i)}=u_i-\frac{\mathbf{C}'(u_i)\cdot(\mathbf{C}(u_i)-\mathbf{P})}{\mathbf{C}''(u_i)\cdot(\mathbf{C}(u_i)-\mathbf{P})+|\mathbf{C}'(u_i)|^2} \tag{6.3}$$

可以使用两个零容差来指示收敛:

ϵ_1：欧几里得距离的度量；

ϵ_2：零余弦度量。

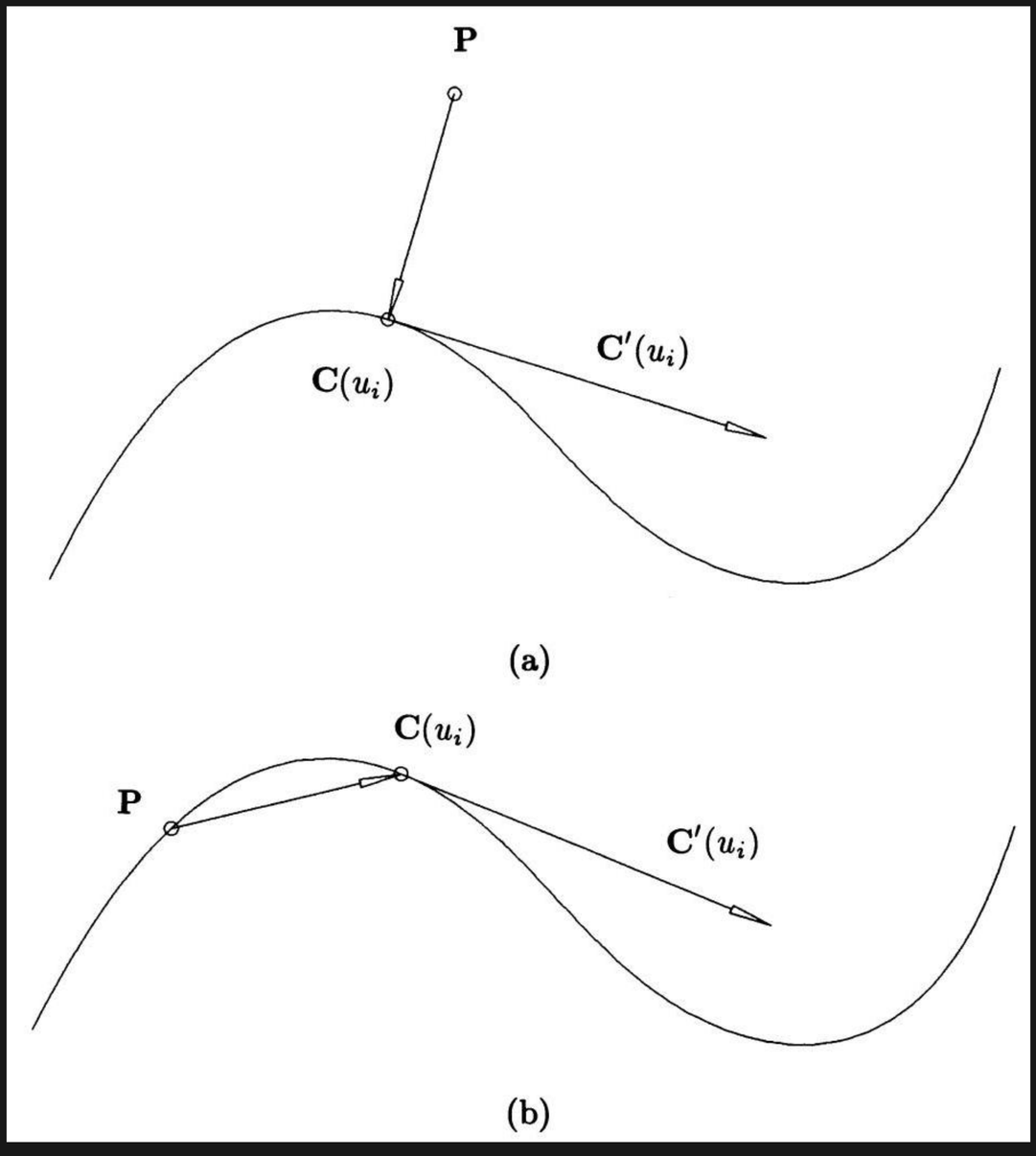


图 6.1. (a) 点投影；(b) 点反演。

收敛标准由以下给出

$$|(u_{i+1}-u_i)\,\mathbf{C}'(u_i)|\leq\epsilon_1$$

$$|\mathbf{C}(u_i) - \mathbf{P}| \le \epsilon_1$$

$$\frac{|\mathbf{C}'(u_i) \cdot (\mathbf{C}(u_i) - \mathbf{P})|}{|\mathbf{C}'(u_i)||\mathbf{C}(u_i) - \mathbf{P}|} \le \epsilon_2 \tag{6.4}$$

标准按以下顺序检查:

1. 点重合:

$$|\mathbf{C}(u_i) - \mathbf{P}| \le \epsilon_1$$

2. 零余弦:

$$\frac{|\mathbf{C}'(u_i) \cdot (\mathbf{C}(u_i) - \mathbf{P})|}{|\mathbf{C}'(u_i)||\mathbf{C}(u_i) - \mathbf{P}|} \le \epsilon_2$$

如果至少有一个条件不满足，则使用公式 (6.3) 计算一个新值 u_{i+1} 。然后检查另外两个条件:

3. 确保参数保持在范围 $(u_{i+1} \in [a, b])$ 内

如果曲线不是闭合的:

$$\text{if } (u_{i+1} < a) \ u_{i+1} = a$$

$$\text{if } (u_{i+1} > b) \ u_{i+1} = b$$

如果曲线是闭合的:

$$\text{if } (u_{i+1} < a) \ u_{i+1} = b - (a - u_{i+1})$$

$$\text{if } (u_{i+1} > b) \ u_{i+1} = a + (u_{i+1} - b)$$

4. 参数没有显著变化，例如，点位于曲线的末端之外

$$|(u_{i+1} - u_i)\mathbf{C}'(u_i)| \le \epsilon_1$$

如果条件 (1)、(2) 或 (4) 中的任何一个满足，则停止迭代。图 6.2 显示了一组点投影到 NURBS 曲线上的情况。

点反演和表面的投影是类似的。形成向量函数

$$\mathbf{r}(u, v) = \mathbf{S}(u, v) - \mathbf{P}$$

和两个标量方程

$$f(u, v) = \mathbf{r}(u, v) \cdot \mathbf{S}_u(u, v) = 0$$

$$g(u, v) = \mathbf{r}(u, v) \cdot \mathbf{S}_v(u, v) = 0 \tag{6.5}$$

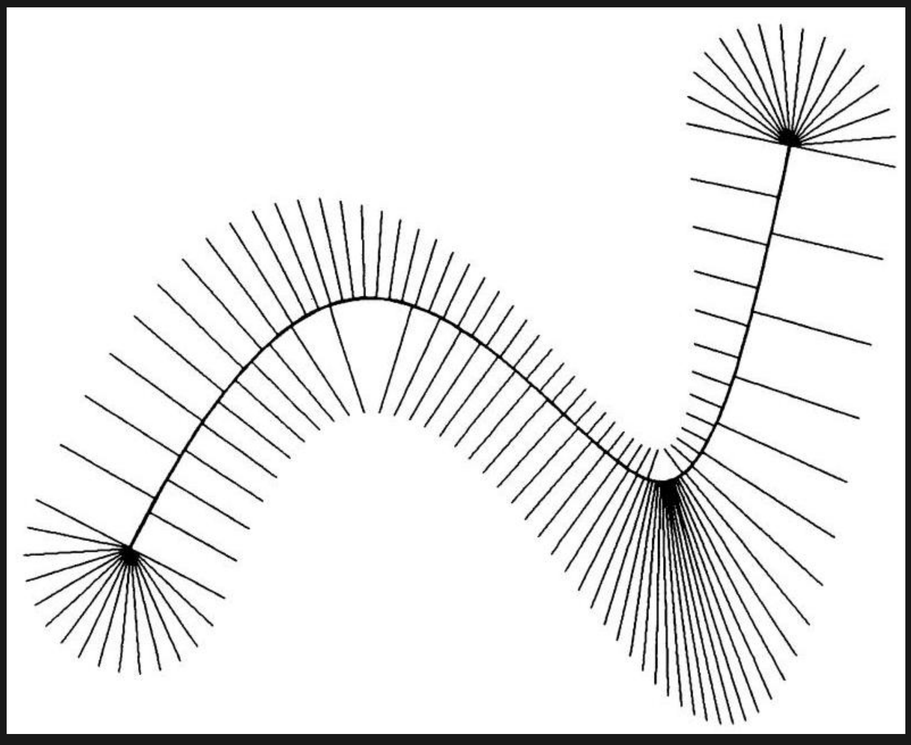


图 6.2. 将一组点投影到 NURBS 曲线。

我们必须求解方程 (6.5)。设

$$\delta_i = \begin{bmatrix} \Delta u \\ \Delta v \end{bmatrix} = \begin{bmatrix} u_{i+1} - u_i \\ v_{i+1} - v_i \end{bmatrix}$$

$$J_i = \begin{bmatrix} f_u & f_v \\ g_u & g_v \end{bmatrix} = \begin{bmatrix} |\mathbf{S}_u|^2 + \mathbf{r} \cdot \mathbf{S}_{uu} & \mathbf{S}_u \cdot \mathbf{S}_v + \mathbf{r} \cdot \mathbf{S}_{uv} \\ \mathbf{S}_u \cdot \mathbf{S}_v + \mathbf{r} \cdot \mathbf{S}_{vu} & |\mathbf{S}_v|^2 + \mathbf{r} \cdot \mathbf{S}_{vv} \end{bmatrix}$$

$$\kappa_i = -\begin{bmatrix} f(u_i, v_i) \\ g(u_i, v_i) \end{bmatrix}$$

矩阵 J_i 中的所有函数在 (u_i, v_i) 处被评估。在第 i 次迭代中，我们必须求解未知数 δ_i 的 2×2 线性方程组，给出如下:

$$J_i \delta_i = \kappa_i \tag{6.6}$$

从 δ_i 我们得到

$$u_{i+1} = \Delta u + u_i$$

$$v_{i+1} = \Delta v + v_i \tag{6.7}$$

收敛标准由以下给出:

$$|(u_{i+1} - u_i)\mathbf{S}_u(u_i, v_i) + (v_{i+1} - v_i)\mathbf{S}_v(u_i, v_i)| \le \epsilon_1$$

$$|\mathbf{S}(u_i, v_i) - \mathbf{P}| \le \epsilon_1$$

$$\frac{|\mathbf{S}_u(u_i, v_i) \cdot (\mathbf{S}(u_i, v_i) - \mathbf{P})|}{|\mathbf{S}_u(u_i, v_i)|} \le \epsilon_2 \quad \frac{|\mathbf{S}_v(u_i, v_i) \cdot (\mathbf{S}(u_i, v_i) - \mathbf{P})|}{|\mathbf{S}_v(u_i, v_i)||\mathbf{S}(u_i, v_i)|\,|\mathbf{S}(u_i, v_i) - \mathbf{P}|} \le \epsilon_2$$

(6.8)

再次，通过以下方式检查条件:

1. 点重合:

$$|\mathbf{S}(u_i, v_i) - \mathbf{P}| \le \epsilon_1$$

2. 零余弦:

$$\frac{|\mathbf{S}_u(u_i, v_i) \cdot (\mathbf{S}(u_i, v_i) - \mathbf{P})|}{|\mathbf{S}_u(u_i, v_i)|} \le \epsilon_2 \quad \frac{|\mathbf{S}_v(u_i, v_i) \cdot (\mathbf{S}(u_i, v_i) - \mathbf{P})|}{|\mathbf{S}_v(u_i, v_i)||\mathbf{S}(u_i, v_i)|\,|\mathbf{S}(u_i, v_i) - \mathbf{P}|} \le \epsilon_2$$

如果这些条件不满足，则使用方程 (6.7) 计算新的值 (u_{i+1}, v_{i+1}) 。然后检查两个条件:

3. 确保参数保持在范围 $(u_{i+1} \in [a,b]$ 和 $v_{i+1} \in [c,d])$ 内:

如果表面在 u 方向上不闭合:

$$\text{if } (u_{i+1} < a) \ u_{i+1} = a$$

$$\text{if } (u_{i+1} > b) \ u_{i+1} = b$$

如果表面在 v 方向上不闭合:

$$\text{if } (v_{i+1} < c) \ v_{i+1} = c$$

$$\text{if } (v_{i+1} > d) \ v_{i+1} = d$$

如果表面在 u 方向上是封闭的:

$$\text{if } (u_{i+1} < a) \ u_{i+1} = b - (a - u_{i+1})$$

$$\text{if } (u_{i+1} > b) \ u_{i+1} = a + (u_{i+1} - b)$$

如果表面在 v 方向上是封闭的:

$$\text{if } (v_{i+1} < c) \ v_{i+1} = d - (c - v_{i+1})$$

$$\text{if } (v_{i+1} > d) \ v_{i+1} = c + (v_{i+1} - d)$$

4. 参数变化不显著，即

$$|(u_{i+1} - u_i)\,\mathbf{S}_u\,(u_i, v_i) + (v_{i+1} - v_i)\,\mathbf{S}_v\,(u_i, v_i)| \leq \epsilon_1$$

如果满足条件 (1)、(2) 或 (4) 中的任何一个，则停止迭代。图 6.3 显示了控制网的一组点投影到 NURBS 表面上的情况。

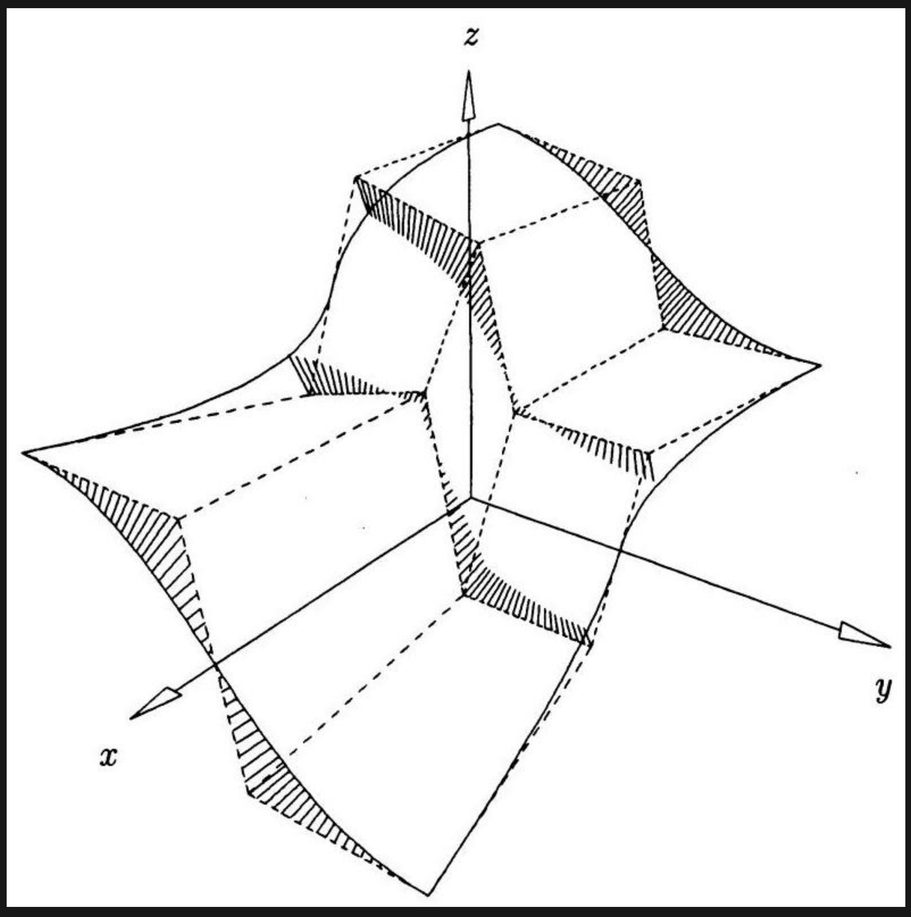


图 6.3. 将控制网的点投影到 NURBS 表面上。

2　曲面切向量反演

设 $(\bar{u}, \bar{v})$ 为一个固定参数点，$\mathbf{P} = \mathbf{S}\,(\bar{u}, \bar{v})$ 为其在表面 $\mathbf{S}\,(u, v)$ 上的像 (见图 6.4)。设 $\mathbf{T} = (dx, dy, dz)$ 为一个从 $\mathbf{P}$ 开始并位于 $\mathbf{S}\,(u, v)$ 在 $\mathbf{P}$ 处的切平面上的向量。用 $\mathbf{S}_u$ 和 $\mathbf{S}_v$ 表示 $\mathbf{S}\,(u, v)$ 在 $(\bar{u}, \bar{v})$ 处的第一偏导数。如果 $\mathbf{S}_u \times \mathbf{S}_v \neq 0$，根据基础微分几何 [DoCa76]，存在一个向量 $\mathbf{W} = (du, dv)$ 在 uv 平面中，使得

$$\mathbf{T} = \mathbf{S}_u du + \mathbf{S}_v dv \tag{6.9}$$

切向量反演是确定向量 $\mathbf{W}$ 的过程。方程 (6.9) 展开为两个未知数的三个方程，即

$$\begin{bmatrix} x_u & x_v \\ y_u & y_v \\ z_u & z_v \end{bmatrix} \begin{bmatrix} du \\ dv \end{bmatrix} = \begin{bmatrix} dx \\ dy \\ dz \end{bmatrix} \tag{6.10}$$

我们将其写为

$$M\mathbf{W} = \mathbf{T}$$

方程 (6.10) 有一个唯一且精确的解 (通常这样的系统没有)，我们通过乘以 M^T (M 的转置) 来获得该解。

$$\left(M^T M\right)\mathbf{W} = M^T\mathbf{T}$$

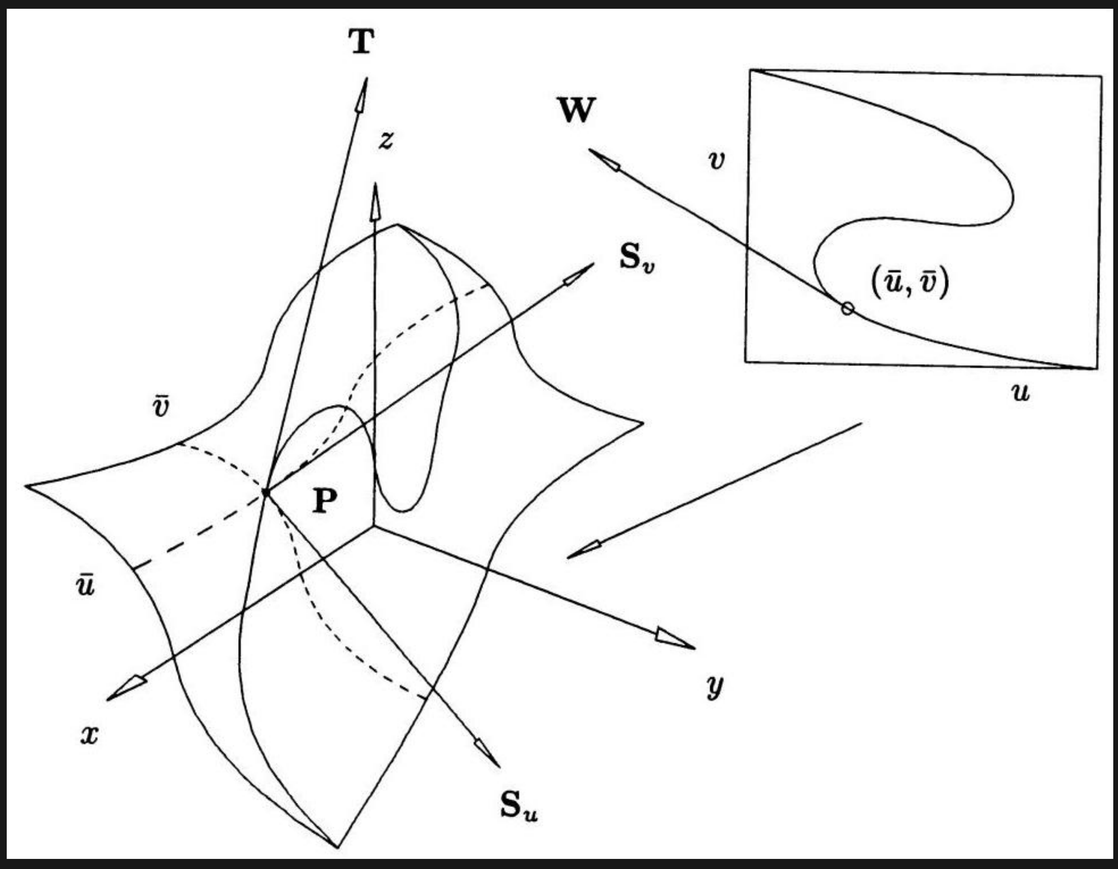


图 6.4. 表面切向量反演。

或

$$\begin{bmatrix} |\mathbf{S}_u|^2 & \mathbf{S}_u \cdot \mathbf{S}_v \\ \mathbf{S}_u \cdot \mathbf{S}_v & |\mathbf{S}_v|^2 \end{bmatrix} \begin{bmatrix} du \\ dv \end{bmatrix} = \begin{bmatrix} \mathbf{S}_u \cdot \mathbf{T} \\ \mathbf{S}_v \cdot \mathbf{T} \end{bmatrix} \tag{6.11}$$

解这个 2×2 方程组可以得到未知数 du 和 dv 。

3 曲线和曲面的变换与投影

曲线和曲面的变换与投影在几何建模和计算机图形学中是常见的。变换包括平移、旋转、缩放、剪切和反射。平行投影和透视投影在视频屏幕或其他二维输出设备上查看三维几何体时是必不可少的。文献中对变换和投影有许多优秀的讨论，例如 [Faux81; Fole90; Roge90]。在本节中，我们假设读者熟悉一般概念，我们仅详细阐述这些概念在 NURBS 曲线和曲面上的应用。

所有变换和投影都是通过对控制点和权重应用操作来对 NURBS 曲线和曲面进行的；因此，讨论可以限制在曲线上。方程 (4.1) 和 (4.5) 表示了思考 NURBS 曲线的两种不同方式，即作为欧几里得空间中的分段有理曲线，或作为齐次空间中的非有理 B 样条曲线。第一种方法使用 B 样条基函数来混合加权的三维控制点；第二种方法使用函数来混合四维控制点。因此，存在两种执行变换和投影的方法:

- 将操作应用于 $\mathbf{C}(u)$ 的三维控制点；对于透视投影，还必须计算新的权重；
- 将 4×4 矩阵应用于 $\mathbf{C}^w(u)$ 的四维控制点。

设

$$\mathbf{C}(u) = \frac{\sum_{i=0}^{n} N_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^{n} N_{i,p}(u) w_i} = \sum_{i=0}^{n} R_{i,p}(u) \mathbf{P}_i \tag{6.12}$$

除了透视，所有对 $\mathbf{C}(u)$ 的变换和投影都是通过对三维控制点 $\mathbf{P}_i$ 应用操作来执行的；权重 w_i 不会改变。这遵循仿射不变性属性 P4.9(该属性源于方程 [6.12] 的形式)。让我们考虑投影。图 6.5 显示了控制点 $\mathbf{P}_i$ 的一般平行投影到由参考点 $\mathbf{Q}$ 和单位长度法向量 $\mathbf{N}$.$\overline{\mathbf{P}}_i$ 给定的投影平面上，表示 $\mathbf{P}_i$ 的投影。如果投影的方向由向量 $\mathbf{W}$ 给出，那么

$$\overline{\mathbf{P}}_i = \mathbf{P}_i + \alpha \mathbf{W} \tag{6.13}$$

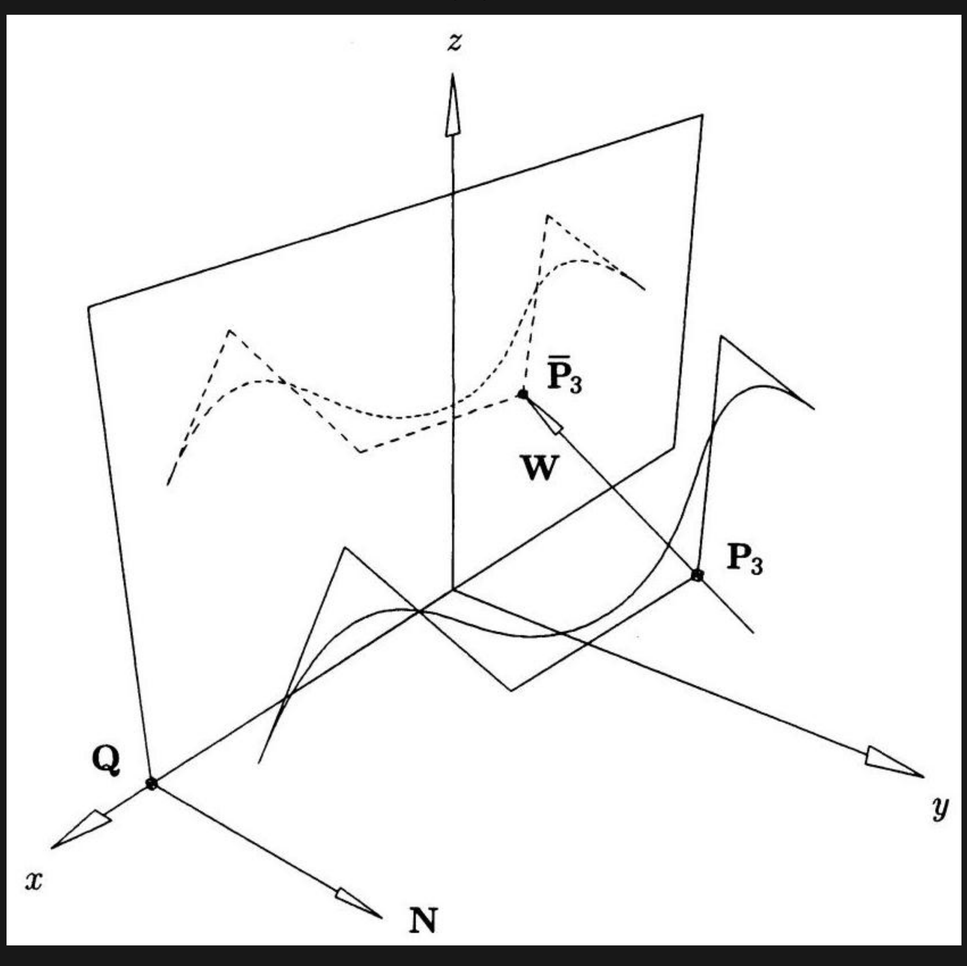


图 6.5. NURBS 曲线的平行投影。

从方程 (6.13) 我们得到

$$\alpha \mathbf{W} = \overline{\mathbf{P}}_i - \mathbf{P}_i - \mathbf{Q} + \mathbf{Q}$$

和

$$\alpha\left(\mathbf{N}\cdot\mathbf{W}\right)=\mathbf{N}\cdot\left(\overline{\mathbf{P}}_i-\mathbf{Q}\right)+\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)$$

由于 $\overline{\mathbf{P}}_i$ 位于投影平面 $\mathbf{N}\cdot\left(\overline{\mathbf{P}}_i-\mathbf{Q}\right)=0$ 上，因此得出

$$\alpha=\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\mathbf{W}}$$

最后

$$\overline{\mathbf{P}}_i=\mathbf{P}_i+\left(\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\mathbf{W}}\right)\mathbf{W} \tag{6.14}$$

对于曲面的相应公式是

$$\overline{\mathbf{P}}_{i,j}=\mathbf{P}_{i,j}+\left(\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_{i,j}\right)}{\mathbf{N}\cdot\mathbf{W}}\right)\mathbf{W} \tag{6.15}$$

图 6.6 显示了透视投影。与之前一样，设投影平面由 $\mathbf{Q}$ 和 $\mathbf{N}$ 定义。将眼睛位置表示为 $\mathbf{E}$ 。那么

$$\overline{\mathbf{P}}_i=\left(1-\alpha\right)\mathbf{P}_i+\alpha\mathbf{E} \tag{6.16}$$

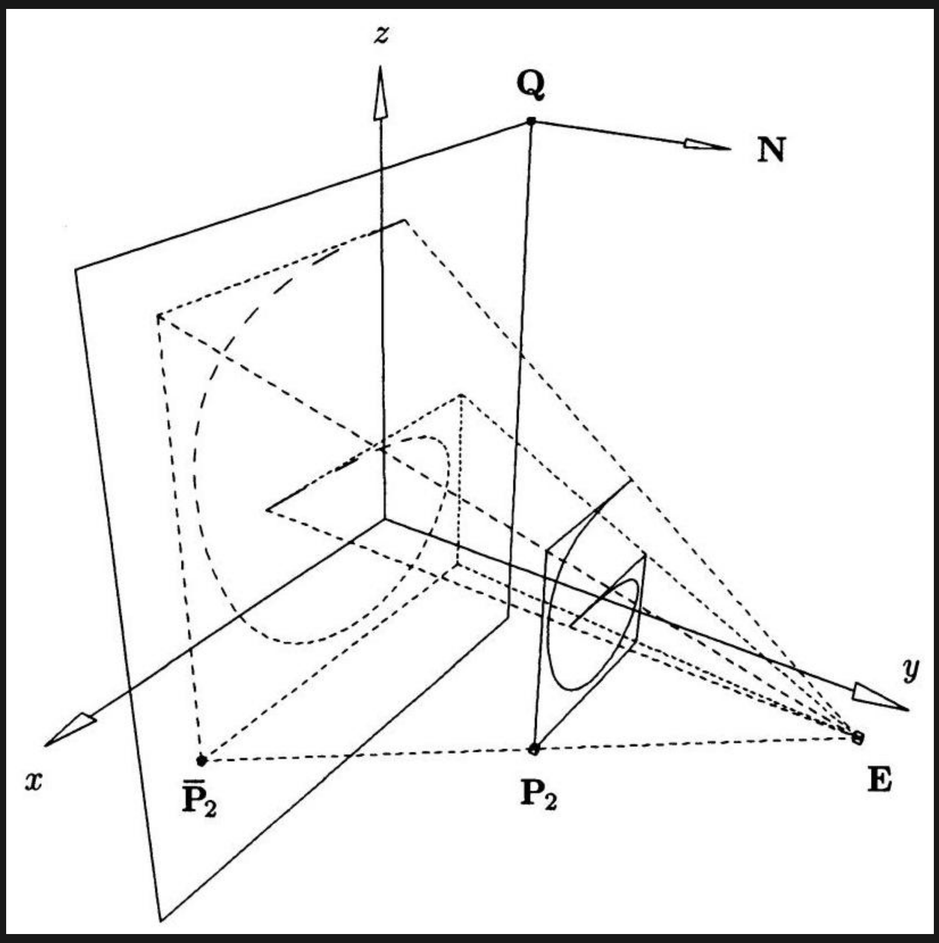


图 6.6. NURBS 曲线的透视投影。

因此得出

$$\alpha\left(\mathbf{E}-\mathbf{P}_i\right)=\overline{\mathbf{P}}_i-\mathbf{P}_i-\mathbf{Q}+\mathbf{Q}$$

而 $\mathbf{N}\cdot\left(\overline{\mathbf{P}}_i-\mathbf{Q}\right)=0$ 意味着

$$\alpha=\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}$$

从

$$1-\alpha=1-\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}=\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i-\mathbf{Q}+\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}=\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}$$

我们得到

$$\overline{\mathbf{P}}_i=\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\mathbf{P}_i+\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\mathbf{E} \tag{6.17}$$

方程 (6.17) 是计算投影曲线的新控制点的公式。

现在我们推导新权重的公式。设

$$\mathbf{P}=\mathbf{C}\left(u\right)=\sum_{i=0}^{n}R_{i,p}\left(u\right)\mathbf{P}_i$$

为 $\mathbf{C}\left(u\right)$ 上的一个任意点，记 $\overline{\mathbf{P}}$ 为 $\mathbf{P}$ 的透视投影

到由 $\mathbf{Q}$ 和 $\mathbf{N}$ 给出的平面上。回顾一下

$$\sum_{i=0}^{n}R_{i,p}\left(u\right)=1$$

对于所有 u ，我们得到

$$\overline{\mathbf{P}}=\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}\right)}\mathbf{P}+\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}\right)}\mathbf{E}$$

$$=\frac{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)\mathbf{P}_i\right)+\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)\right)\mathbf{E}}{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right)}$$

$$=\frac{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right)\left(\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\right)\mathbf{P}_i}{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right)}$$

$$+\frac{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right)\left(\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\right)\mathbf{E}}{\sum R_{i,p}\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right)}$$

设置

$$\bar{w}_i=w_i\left(\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)\right) \tag{6.18}$$

并回顾一下

$$R_{i,p}=\frac{N_{i,p}w_i}{\sum N_{i,p}w_i}$$

我们得到

$$\overline{\mathbf{P}}=\frac{\sum N_{i,p}\bar{w}_i\left(\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\mathbf{P}_i+\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_i\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_i\right)}\mathbf{E}\right)}{\sum N_{i,p}\bar{w}_i}$$

$$=\frac{\sum N_{i,p}\left(u\right)\bar{w}_i\overline{\mathbf{P}}_i}{\sum N_{i,p}\left(u\right)\bar{w}_i} \tag{6.19}$$

方程 (6.19) 显示由方程 (6.17) 和 (6.18) 给出的控制点和权重是投影曲线的控制点和权重。表面公式为

$$\overline{\mathbf{P}}_{i,j}=\frac{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{Q}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_{i,j}\right)}\mathbf{P}_{i,j}+\frac{\mathbf{N}\cdot\left(\mathbf{Q}-\mathbf{P}_{i,j}\right)}{\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_{i,j}\right)}\mathbf{E} \tag{6.20}$$

和

$$\bar{w}_{i,j} = w_{i,j}\left[\mathbf{N}\cdot\left(\mathbf{E}-\mathbf{P}_{i,j}\right)\right] \tag{6.21}$$

图 6.7 显示了 NURBS 曲面的平行投影。

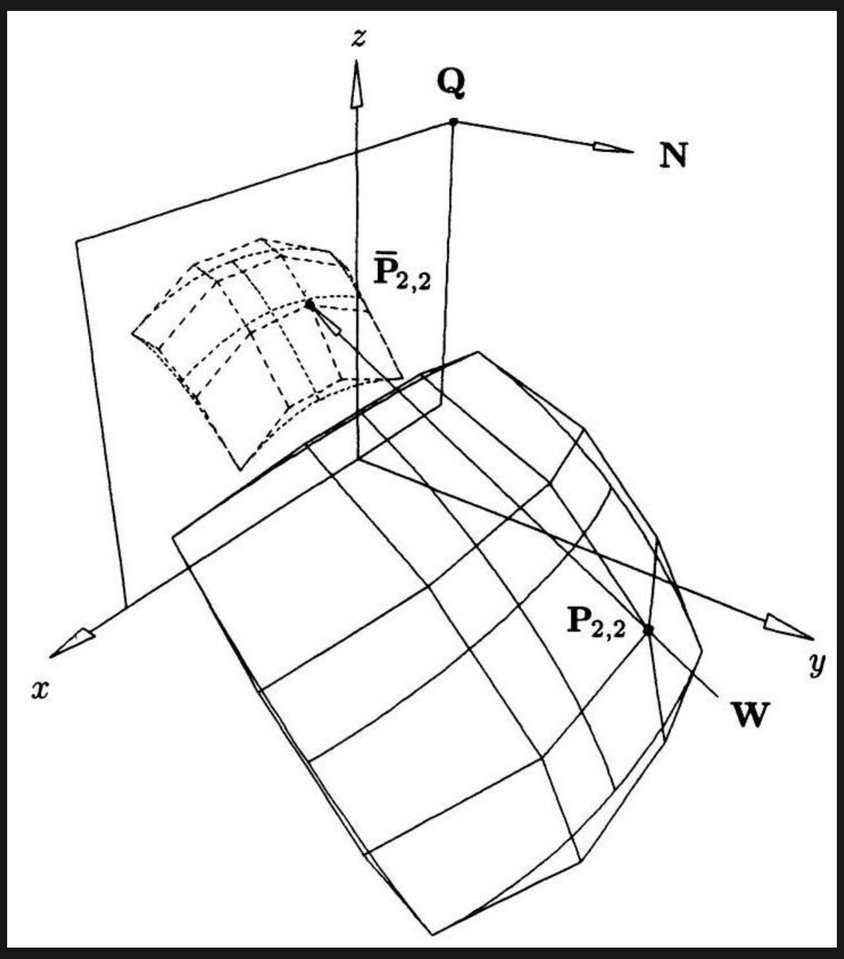


图 6.7. NURBS 曲面的平行投影。
现在设 $\mathbf{C}^w(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_i^w$ 。所有变换和投影可以被打包成一个 4×4 形式的矩阵。

$$A=\begin{bmatrix} B & T \\ P & a_{3,3}\end{bmatrix}=\begin{bmatrix} b_{0,0} & b_{0,1} & b_{0,2} & t_0 \\ b_{1,0} & b_{1,1} & b_{1,2} & t_1 \\ b_{2,0} & b_{2,1} & b_{2,2} & t_2 \\ p_0 & p_1 & p_2 & a_{3,3}\end{bmatrix} \tag{6.22}$$

其中向量 T 和 P 分别表示平移和透视，而 3×3 矩阵 B 包含旋转、缩放、剪切、通过坐标平面的反射以及投影到坐标平面。矩阵 A 将应用于所有控制点 $\mathbf{P}_i^w$ 的 $\mathbf{C}^w(u)$ 。

示例

设 $\mathbf{P}_i^w=(w_ix_i,w_iy_i,w_iz_i,w_i)$ ，并用 $\bar{\mathbf{P}}_i^w=A\mathbf{P}_i^w$ 表示一个变换后的齐次控制点。
Ex6.1 将 $\mathbf{C}^w(u)$ 按向量 $\mathbf{V}=(x,y,z)$ 进行平移。然后

$$\bar{\mathbf{P}}_i^w=\begin{bmatrix}1&0&0&x\\0&1&0&y\\0&0&1&z\\0&0&0&1\end{bmatrix}\begin{bmatrix}w_ix_i\\w_iy_i\\w_iz_i\\w_i\end{bmatrix}=\begin{bmatrix}w_i(x_i+x)\\w_i(y_i+y)\\w_i(z_i+z)\\w_i\end{bmatrix}$$

Ex6.2 将旋转矩阵 R 应用到 $\mathbf{C}^w(u)$ 。

$$\bar{\mathbf{P}}_i^w=\begin{bmatrix}r_{0,0}&r_{0,1}&r_{0,2}&0\\r_{1,0}&r_{1,1}&r_{1,2}&0\\r_{2,0}&r_{2,1}&r_{2,2}&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}w_ix_i\\w_iy_i\\w_iz_i\\w_i\end{bmatrix}=\begin{bmatrix}w_i(r_{0,0}x_i+r_{0,1}y_i+r_{0,2}z_i)\\w_i(r_{1,0}x_i+r_{1,1}y_i+r_{1,2}z_i)\\w_i(r_{2,0}x_i+r_{2,1}y_i+r_{2,2}z_i)\\w_i\end{bmatrix}$$

Ex6.3 透视投影: 设 $\mathbf{E}=(0,0,d)$ 为位于 z 轴上的眼睛位置，距离原点 d 。设 xy 平面为投影平面。然后

$$\bar{\mathbf{P}}_i^w=\begin{bmatrix}d&0&0&0\\0&d&0&0\\0&0&0&0\\0&0&-1&d\end{bmatrix}\begin{bmatrix}w_ix_i\\w_iy_i\\w_iz_i\\w_i\end{bmatrix}=\begin{bmatrix}w_idx_i\\w_idy_i\\0\\w_i(d-z_i)\end{bmatrix} \tag{6.23}$$

4 NURBS 曲线和曲面的重参数化

设 $\mathbf{C}(u)=(x(u),y(u),z(u))$ 为 $u\in[a,b]$ 上的任意参数曲线，并假设 $u=f(s)$ 是在 $s\in[c,d]$ 上的标量值函数，满足:

- 对于所有 $s\in[c,d]$ $(f(s)$ is strictly increasing $)$ 的 $f'(s)>0$;
- $a=f(c)$ 和 $b=f(d)$ $(f(s)$ maps $[c,d]$ onto $[a,b])$ 。

  $\mathbf{C}(u)$ 和 $f(s)$ 的组合，由

$$\mathbf{C}(s)=\mathbf{C}(f(s))=(x(f(s)),y(f(s)),z(f(s)))$$

被称为 $\mathbf{C}(u)$ 的重新参数化 (见图 6.8)。
$\mathbf{C}(s)$ 在几何上与 $\mathbf{C}(u)$ 是相同的曲线，但在参数上它们是不同的。重新参数化的应用包括:

- 内部点映射: 给定参数值 $u_0,\ldots,u_n$ 和 $s_0,\ldots,s_n$ ，其中 $a<u_0<\cdots<u_n<b$ 和 $c<s_0<\cdots<s_n<d$ ，确定一个函数 $u=f(s)$ 使得 $u_i=f(s_i),i=0,\ldots,n$ 。然后形成 $\mathbf{C}(f(s))$ 。这强制曲线点 $\mathbf{C}(u_i)$ 在新的参数值 s_i 下被假定；
- 末端导数的修改: 重新参数化改变曲线导数；这遵循链式法则，例如。

$$\mathbf{C}'(s)=\mathbf{C}'(u)\,f'(s)\ \ \mathbf{C}''(s)=\mathbf{C}'(u)\,f''(s)+\mathbf{C}''(u)\,(f'(s))^2$$

注意，只有末端一阶导数的大小发生变化，但末端二阶及更高阶导数的大小和方向都会改变；

- 末端权重的修改: 如果 $\mathbf{C}^w(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_i^w$ 是 NURBS

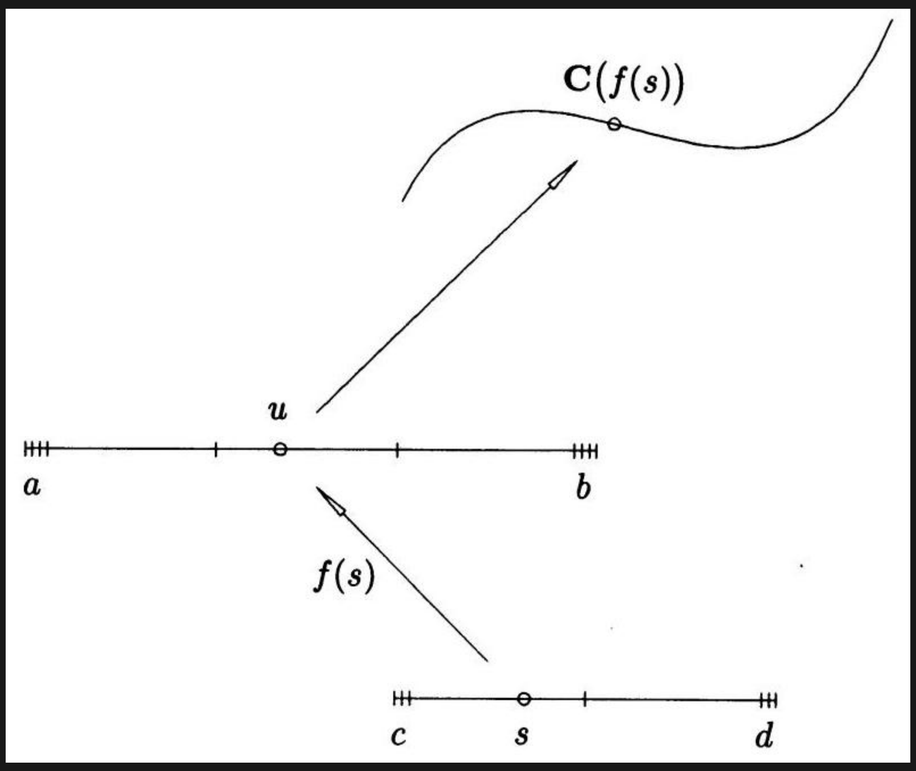


图 6.8. 曲线重新参数化。

在曲线中，有时修改端点权重 w_0 和 w_n 以获得特定值而不改变曲线几何形状是有用的。我们将看到这实际上是可能的，并且简单等同于 $\mathbf{C}(u)$ 的重新参数化。

示例

Ex6.4 设

$$\mathbf{C}(u) = (x(u), y(u)) = \left(u, -2u^2 + 2u\right) \tag{6.24}$$

在 $u \in [0,1]$ 上，假设我们希望一个参数 $s \in [0,1]$ 使得 $u = 1/2$ 对应于 $s = 6/10$，即

$$\mathbf{C}\left(u = \frac{1}{2}\right) = \mathbf{C}\left(s = \frac{6}{10}\right) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

然后 $u = f(s)$ 必须满足三个约束条件

$$0 = f(0)\ \frac{1}{2} = f\left(\frac{6}{10}\right)\ 1 = f(1)$$

我们选择 $f(s)$ 为二次多项式

$$u = f(s) = as^2 + bs + c$$

第一个约束意味着 $c = 0$，最后两个产生线性方程

$$\frac{1}{2} = \frac{9}{25}a + \frac{6}{10}b$$

$$1 = a + b \tag{6.25}$$

解方程 (6.25) 得到

$$a = \frac{5}{12}\ b = \frac{7}{12}$$

因此

$$u = f(s) = \frac{5}{12}s^2 + \frac{7}{12}s \tag{6.26}$$

将方程 (6.26) 代入方程 (6.24)，我们得到重新参数化的曲线

$$\mathbf{C}(s) = \left(\frac{5}{12}s^2 + \frac{7}{12}s, -\frac{25}{72}s^4 - \frac{35}{36}s^3 + \frac{11}{72}s^2 + \frac{7}{6}s\right) \tag{6.27}$$

读者应验证

$$\mathbf{C}(s = 0) = \mathbf{C}(u = 0) = (0, 0)$$

$$\mathbf{C}\left(s = \frac{6}{10}\right) = \mathbf{C}\left(u = \frac{1}{2}\right) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

$$\mathbf{C}(s = 1) = \mathbf{C}(u = 1) = (0, 0)$$

注意，导数也发生了变化。例如，直接对方程 (6.24) 和 (6.27) 进行求导得到

$$\mathbf{C}'(u = 0) = (1, 2)\ \mathbf{C}'(s = 0) = \left(\frac{7}{12}, \frac{7}{6}\right)$$

图 6.9 显示了曲线及其相对于 u 和 s 在 $u = s = 0$ 处的第一导数。

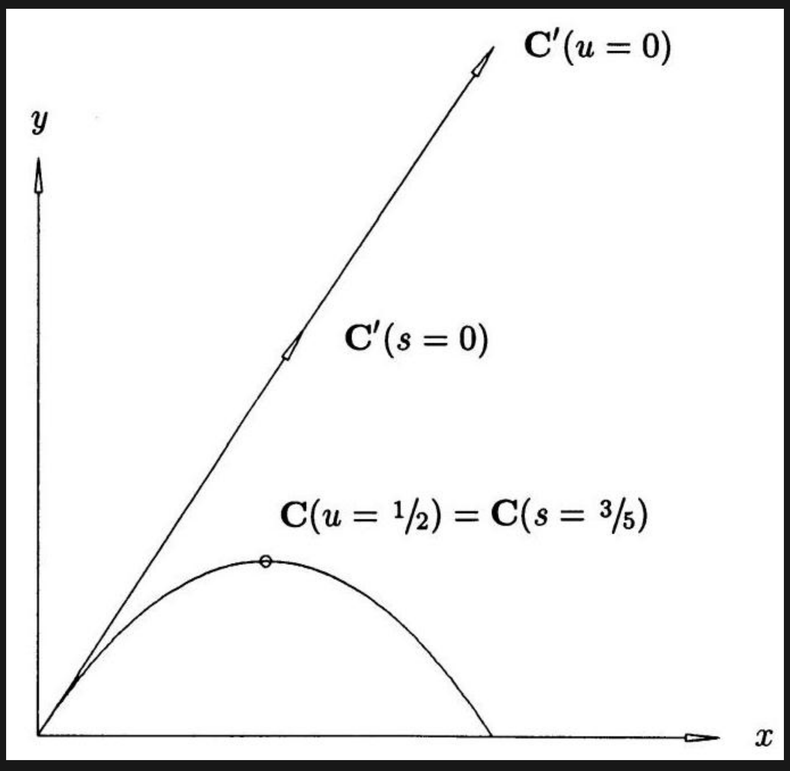


图 6.9. 示例 Ex6.4 的重新参数化曲线。

Ex6.5 考虑由以下公式给出的圆弧

$$\mathbf{C}(u)=\left(\frac{1-u^2}{1+u^2},\frac{2u}{1+u^2}\right) \tag{6.28}$$

在 $u\in[0,1]$ 上。该弧以原点为中心，半径为 1，扫过一个角度 90° (见第 1 章，方程 [1.2]，图 1.2，示例 1.8，以及图 1.19a)。很容易检查到

$$\mathbf{C}\left(u=\frac{1}{2}\right)=\left(\frac{6}{10},\frac{8}{10}\right)$$

和

$$\mathbf{C}'(u=0)=(0,2)$$

$$\mathbf{C}'(u=1)=(-1,0) \tag{6.29}$$

我们希望对 $\mathbf{C}(u)$ 进行一个更对称的重新参数化 $\mathbf{C}(s)$ ，即 $\mathbf{C}(s)$ 应满足

$$|\mathbf{C}'(s=0)|=|\mathbf{C}'(s=1)| \tag{6.30}$$

我们选择形式为的线性有理函数

$$u=f(s)=\frac{as+b}{cs+d}\ s\in[0,1] \tag{6.31}$$

显然，我们可以假设 $d=1$ ，条件 $f(0)=0$ 和 $f(1)=1$ 意味着 $b=0$ 和 $c=a-1$ 。因此， $f(s)$ 的形式为

$$u=\frac{as}{(a-1)s+1} \tag{6.32}$$

方程 (6.30) 给出的条件确定了 a 。从方程 (6.29) 和 $\mathbf{C}'(s)=\mathbf{C}'(u)f'(s)$ 中，我们要求

$$2f'(s=0)=f'(s=1) \tag{6.33}$$

对方程 (6.32) 求导得到

$$f'(s)=\frac{a}{((a-1)s+1)^2}$$

从方程 (6.33) 我们得到 $2a=1/\mathrm{a}$ 。因此 $2a^2-1=0$ ，因此 $a=\pm\sqrt{2}/2$ 。选择 $a=-\sqrt{2}/2$ 会导致方程 (6.32) 在区间 $s\in[0,1]$ 的分母为零，因此我们选择 $a=\sqrt{2}/2$ 。因此，我们得到

$$u=f(s)=\frac{\sqrt{2}s}{(\sqrt{2}-2)s+2} \tag{6.34}$$

将方程 (6.34) 代入方程 (6.28) 得到

$$\mathbf{C}(s)=\left(\frac{(1-\sqrt{2})s^2+(\sqrt{2}-2)s+1}{(2-\sqrt{2})s^2+(\sqrt{2}-2)s+1},\frac{(1-\sqrt{2})s^2+\sqrt{2}s}{(2-\sqrt{2})s^2+(\sqrt{2}-2)s+1}\right)$$

(6.35)

读者可以验证 $\mathbf{C}(s)$ 满足方程 (6.30) 以及条件

$$\mathbf{C}\left(s=\frac{1}{2}\right)=\left(\frac{\sqrt{2}}{2},\frac{\sqrt{2}}{2}\right) \tag{6.36}$$

即 $s=1/2$ 映射到圆弧的中点。

注意:

- 如果 $\mathbf{C}(u)$ 是一个 p 次多项式 (有理) 曲线，而 $f(s)$ 是一个 q 次多项式 (有理) 函数，则 $\mathbf{C}(s)$ 是一个 pq 次多项式 (有理) 曲线。特别地，形式为 $u=as+b$ 或 $u=(\mathrm{as}+\mathrm{b})/(\mathrm{cs}+\mathrm{d})$ 的线性重新参数化函数不会提高曲线的次数；
- 形式为 $u=(as+b)/(cs+d)$ 的线性有理函数将多项式曲线转换为相同次数的有理曲线。

现在我们讨论用多项式函数重新参数化 NURBS 曲线的问题。设 $\mathbf{C}^w(u)=\sum_{i=0}^{n}N_{i,p}(u)\mathbf{P}_i^w$ 为 p 次 NURBS 曲线，定义在 $u\in[a,b]$ 上，且 $u=f(s)$ 为 q 次多项式，定义在 $s\in[c,d]$ 上。现在假设 $\mathbf{C}^w(u)$ 没有内部节点，因此 U 具有以下形式

$$U=\{\underbrace{a,\ldots,a}_{p+1},\underbrace{b,\ldots,b}_{p+1}\}$$

重新参数化的曲线 $\mathbf{C}^w(s)$ 的次数为 pq ，并定义在节点向量上

$$S=\{\underbrace{c,\ldots,c}_{pq+1},\underbrace{d,\ldots,d}_{pq+1}\} \tag{6.37}$$

新的控制点 $pq+1$ 可以通过链式法则的重复应用和 B 样条曲线的端导数公式计算得出。我们通过示例来说明这一点。

示例

Ex6.6 设 $\mathbf{C}(u)=\sum_{i=0}^{2}N_{i,2}(u)\mathbf{P}_i, U=\{0,0,0,1,1,1\}$ 为示例 Ex6.4 中抛物弧的 B-spline 表示 (见图 6.9 和 6.10)。

控制点为 $\{(0,0),(1/2,1),(1,0)\}$ ，重参数化函数为

$$u=f(s)=\frac{5}{12}s^2+\frac{7}{12}s\ \text{ on }\ s\in[0,1]$$

重参数化的曲线，四次度数，表示为

$$\mathbf{C}(s)=\sum_{i=0}^{4}N_{i,4}(s)\mathbf{Q}_i,\ S=\{0,0,0,0,0,1,1,1,1,1\} \tag{6.38}$$

显然 (见图 6.10)

$$\mathbf{Q}_0=\mathbf{P}_0\ \mathbf{Q}_4=\mathbf{P}_2$$

从 $\mathbf{C}'(s)=\mathbf{C}'(u)f'(s)$ 和方程 (3.7) 我们得到

$$\mathbf{C}'(s=0)=\frac{4}{1-0}(\mathbf{Q}_1-\mathbf{Q}_0)=\mathbf{C}'(u=0)f'(s=0)$$

$$=\frac{2}{1-0}(\mathbf{P}_1-\mathbf{P}_0)\left(\frac{7}{12}\right)$$

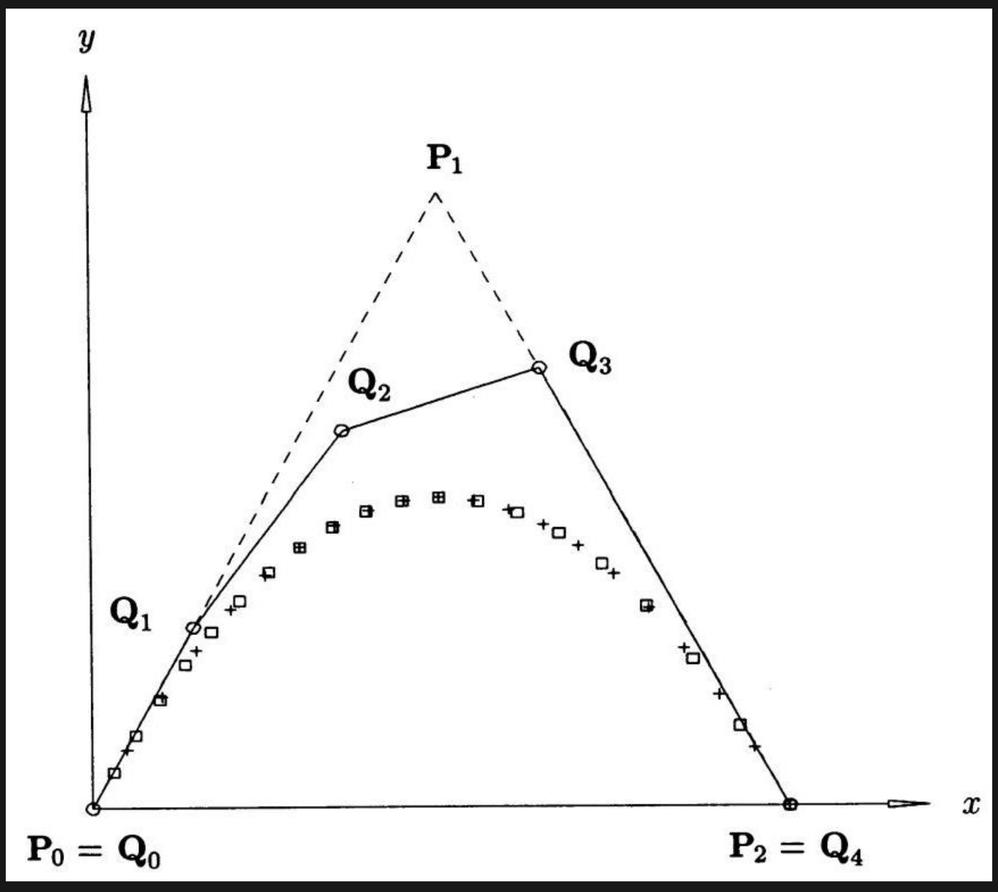


图 6.10. 示例 Ex6.6 的重参数化 B-spline 曲线；+ 显示了原始参数化，而 □ 则说明了新的参数化。

从中得出

$$\mathbf{Q}_1=\frac{7}{24}(\mathbf{P}_1-\mathbf{P}_0)+\mathbf{Q}_0=\left(\frac{7}{48},\frac{7}{24}\right)$$

类似地，$4(\mathbf{Q}_4-\mathbf{Q}_3)=2(\mathbf{P}_2-\mathbf{P}_1)(17/12)$，和

$$\mathbf{Q}_3=\mathbf{Q}_4-\frac{17}{24}(\mathbf{P}_2-\mathbf{P}_1)=\left(\frac{31}{48},\frac{17}{24}\right)$$

从 $\mathbf{C}''(s)=\mathbf{C}'(u)f''(s)+\mathbf{C}''(u)(f'(s))^2$ 和方程 (3.9) 得出，并注意到

$$\mathbf{C}'(u=0)=(1,2)\ \mathbf{C}''(u=0)=(0,-4)$$

$$f'(s=0)=\frac{7}{12}\ f''(s=0)=\frac{5}{6}$$

我们得到

$$\mathbf{C}''(s=0)=12(\mathbf{Q}_0-2\mathbf{Q}_1+\mathbf{Q}_2)=\frac{5}{6}(1,2)+\left(\frac{7}{12}\right)^2(0,-4)$$

从中得出

$$\mathbf{Q}_2=\frac{5}{72}(1,2)+\frac{49}{1728}(0,-4)+2\left(\frac{7}{48},\frac{7}{24}\right)=\left(\frac{156}{432},\frac{263}{432}\right)$$

因此，重新参数化一个没有内部节点的 p 阶曲线 $\mathbf{C}^w(u)$ 为一个 q 阶多项式 $f(s)$ 的一般过程是:

1. 设置 $\mathbf{Q}_0=\mathbf{P}_0$ 和 $\mathbf{Q}_{pq}=\mathbf{P}_p$；
2. 令 $\mathbf{C}^{w(i)}$ 表示 $\mathbf{C}^w(u)$ 的 i 阶导数，并令 $m_\ell=(\mathrm{pq}+1)/2$ (整数运算) 和 $m_r=pq-\mathrm{pq}/2-1$。计算 $\mathbf{C}^{w(i)}(u=a)$ 的导数 $i=1,\ldots,m_\ell$，以及 $\mathbf{C}^{w(i)}(u=b)$ 的导数 $i=1,\ldots,m_r$；注意，微分是在齐次空间中进行的；
3. 令 $f^{(i)}$ 表示 $f(s)$ 的 i 阶导数；计算 $f^{(i)}(s=$ 的导数 c) 和 $i=1,\ldots,m_\ell$ 的导数 $f^{(i)}(s=d)$，以及 $i=1,\ldots,m_r$ 的导数；
4. 使用链式法则的重复应用 (该法则将 $\mathbf{C}^{w(i)}(s)$ 表示为 $\mathbf{C}^{w(i)}(u)$ 和 $f^{(i)}(s)$) 的函数，并结合方程 (6.41) 和 (6.42)，计算新的控制点 $\mathbf{Q}_1,\ldots,\mathbf{Q}_{m_\ell}$ 和 $\mathbf{Q}_{pq-1},\ldots,\mathbf{Q}_{pq-m_r}$。

应用链式法则四次得到

$$\mathbf{C}^{w(1)}(s)=\mathbf{C}^{w(1)}(u)f^{(1)}(s)$$

$$\mathbf{C}^{w(2)}(s)=\mathbf{C}^{w(1)}(u)f^{(2)}(s)+\mathbf{C}^{w(2)}(u)\left(f^{(1)}(s)\right)^2$$

$$\mathbf{C}^{w(3)}(s)=\mathbf{C}^{w(1)}(u)f^{(3)}(s)+3\mathbf{C}^{w(2)}(u)f^{(1)}(s)f^{(2)}(s)+\mathbf{C}^{w(3)}(u)\left(f^{(1)}(s)\right)^3$$

$$\mathbf{C}^{w(4)}(s)=\mathbf{C}^{w(1)}(u)f^{(4)}(s)+\mathbf{C}^{w(2)}(u)\left[4f^{(1)}(s)f^{(3)}(s)+3\left(f^{(2)}(s)\right)^2\right]$$

$$+6\mathbf{C}^{w(3)}(u)\left(f^{(1)}(s)\right)^2f^{(2)}(s)+\mathbf{C}^{w(4)}(u)\left(f^{(1)}(s)\right)^4 \tag{6.39}$$

一般公式

$$\mathbf{C}^{w(n)}(s)=\sum_{j=0}^{n}\sum_{\substack{k_1+k_2+\cdots+k_n=j\\k_1+2k_2+\cdots+k_n=j\\k_1+2k_2+\cdots+k_n\geq 0}}\frac{n!}{k_1!(1!)^{k_1}\cdots k_n!(n!)^{k_n}}\left(f^{(1)}(s)\right)^{k_1}\cdots\left(f^{(n)}(s)\right)^{k_n} \tag{6.40}$$

是由 Faa di Bruno 提出的 (见 [Knut73]，第 50 页)。尽管这个公式优雅，但它的实现并不简单。困难在于获得满足条件的 n -元组

$$k_1+k_2+\cdots+k_n=j$$

$$k_1+2k_2+\cdots+nk_n=n\ k_1,k_2,\ldots,k_n\geq 0$$

为了说明这一点，我们使用方程 (6.40) 推导出方程 (6.39) 中给出的前四个导数:

n=1: $k_1=1$ 必须成立:

$$j=0:k_1=0\rightarrow\ \text{no solution}$$

$$j=1:k_1=1\rightarrow(1)$$

这导致

$$\mathbf{C}^{w(1)}(s)=\mathbf{C}^{w(1)}(u)f^{(1)}(s)$$

$\mathrm{n}=2:k_1+2k_2=2$ 必须成立:

$$j=0:k_1+k_2=0\rightarrow\ \text{no solution}$$

$$j=1:k_1+k_2=1\rightarrow(0,1)$$

$$j=2: k_1+k_2=2 \rightarrow (2,0)$$

产生 $\mathbf{C}^{w(2)}(s)=\mathbf{C}^{w(1)}(u) f^{(2)}(s)+\mathbf{C}^{w(2)}(u)\left(f^{(1)}(s)\right)^2$

$\mathrm{n}=3: k_1+2k_2+3k_3=3$ 必须成立:

$$j=0: k_1+k_2+k_3=0 \rightarrow \text{ no solution}$$

$$j=1: k_1+k_2+k_3=1 \rightarrow (0,0,1)$$

$$j=2: k_1+k_2+k_3=2 \rightarrow (1,1,0)$$

$$j=3: k_1+k_2+k_3=3 \rightarrow (3,0,0)$$

从中我们得到

$$\mathbf{C}^{w(3)}(s)=\mathbf{C}^{w(1)}(u) f^{(3)}(s)+3\mathbf{C}^{w(2)}(u) f^{(1)}(s) f^{(2)}(s)+\mathbf{C}^{w(3)}(u)\left(f^{(1)}(s)\right)^3$$

$\mathrm{n}=4: k_1+2k_2+3k_3+4k_4=4$ must hold:

$$j=0: k_1+k_2+k_3+k_4=0 \rightarrow \text{ no solution}$$

$$j=1: k_1+k_2+k_3+k_4=1 \rightarrow (0,0,0,1)$$

$$j=2: k_1+k_2+k_3+k_4=2 \rightarrow (1,0,1,0) \text{ and } (0,2,0,0)$$

$$j=3: k_1+k_2+k_3+k_4=3 \rightarrow (2,1,0,0)$$

$$j=4: k_1+k_2+k_3+k_4=4 \rightarrow (4,0,0,0)$$

从中我们得到

$$\mathbf{C}^{w(4)}(s)=\mathbf{C}^{w(1)}(u) f^{(4)}(s)+\mathbf{C}^{w(2)}(u)\left[4 f^{(1)}(s) f^{(3)}(s)+3\left(f^{(2)}(s)\right)^2\right]$$

$$+6\mathbf{C}^{w(3)}(u)\left(f^{(1)}(s)\right)^2 f^{(2)}(s)+\mathbf{C}^{w(4)}(u)\left(f^{(1)}(s)\right)^4$$

考虑到 $\mathbf{C}^w(s)$ 关于 $\mathbf{C}^w(u)$ 和 $f(s)$ 的导数，$\mathbf{Q}_i^w$ 的一般公式为

$$\mathbf{Q}_i^w=\frac{(pq-i)!\Delta s^i}{(pq)!}\mathbf{C}^{w(i)}(s=c)+\sum_{j=0}^{i-1}(-1)^{i+j-1}\left(\begin{array}{c} i \\ j \end{array}\right)\mathbf{Q}_j^w \tag{6.41}$$

其中 $\Delta s^i=(d-c)^i, i=1,\ldots,m_\ell$ ，和

$$\mathbf{Q}_{pq-i}^w=(-1)^i\frac{(pq-i)!\Delta s^i}{(pq)!}\mathbf{C}^{w(i)}(s=d)+\sum_{j=0}^{i-1}(-1)^{i+j-1}\left(\begin{array}{c} i \\ j \end{array}\right)\mathbf{Q}_{pq-j}^w \tag{6.42}$$

对于 $i=1,\ldots,m_r$ 。例如，左侧的前五个控制点是

$$\mathbf{Q}_0^w=\mathbf{P}_0^w$$

$$\mathbf{Q}_1^w=\frac{\Delta s}{pq}\mathbf{C}^{w(1)}(s=c)+\mathbf{Q}_0^w$$

$$\mathbf{Q}_2^w=\frac{\Delta s^2}{pq(pq-1)}\mathbf{C}^{w(2)}(s=c)-\mathbf{Q}_0^w+2\mathbf{Q}_1^w$$

$$\mathbf{Q}_3^w=\frac{\Delta s^3}{pq(pq-1)(pq-2)}\mathbf{C}^{w(3)}(s=c)+\mathbf{Q}_0^w-3\mathbf{Q}_1^w+3\mathbf{Q}_2^w$$

$$\mathbf{Q}_4^w=\frac{\Delta s^4}{pq(pq-1)(pq-2)(pq-3)}\mathbf{C}^{w(4)}(s=c)-\mathbf{Q}_0^w+4\mathbf{Q}_1^w-6\mathbf{Q}_2^w+4\mathbf{Q}_3^w$$

(6.43)

右侧的控制点是

$$\mathbf{Q}_{pq}^w=\mathbf{P}_n^w$$

$$\mathbf{Q}_{pq-1}^w=-\frac{\Delta s}{pq}\mathbf{C}^{w(1)}(s=d)+\mathbf{Q}_{pq}^w$$

$$\mathbf{Q}_{pq-2}^w=\frac{\Delta s^2}{pq(pq-1)}\mathbf{C}^{w(2)}(s=d)-\mathbf{Q}_{pq}^w+2\mathbf{Q}_{pq-1}^w$$

$$\mathbf{Q}_{pq-3}^w=-\frac{\Delta s^3}{pq(pq-1)(pq-2)}\mathbf{C}^{w(3)}(s=d)+\mathbf{Q}_{pq}^w-3\mathbf{Q}_{pq-1}^w+3\mathbf{Q}_{pq-2}^w$$

$$\mathbf{Q}_{pq-4}^w=\frac{\Delta s^4}{pq(pq-1)(pq-2)(pq-3)}\mathbf{C}^{w(4)}(s=d)-\mathbf{Q}_{pq}^w$$

$$+4\mathbf{Q}_{pq-1}^w-6\mathbf{Q}_{pq-2}^w+4\mathbf{Q}_{pq-3}^w \tag{6.44}$$

线性重新参数化函数

$$u=\alpha s+\beta \; s\in[c,d] \tag{6.45}$$

是一个有趣的特例。检查方程 (6.39)，注意到

$$\mathbf{C}^{w(i)}(s)=\mathbf{C}^{w(i)}(u)\left(f^{(1)}(s)\right)^i=\alpha^i\mathbf{C}^{w(i)}(u) \tag{6.46}$$

然后

$$\mathbf{Q}_1^w=\frac{\Delta s}{pq}\mathbf{C}^{w(1)}(s)+\mathbf{Q}_0^w=\frac{\alpha\Delta s}{\Delta u}\left(\mathbf{P}_1^w-\mathbf{P}_0^w\right)+\mathbf{P}_0^w \tag{6.47}$$

从 $a=\alpha c+\beta$ 和 $b=\alpha d+\beta$ 我们得到

$$\alpha=\frac{b-a}{d-c}=\frac{\Delta u}{\Delta s} \tag{6.48}$$

将方程 (6.48) 代入方程 (6.47) 得到

$$\mathbf{Q}_1^w=\mathbf{P}_1^w$$

使用方程 (6.41)、(6.46) 和 (6.48)，可以通过归纳法证明

$$\mathbf{Q}_i^w=\mathbf{P}_i^w \text{ for all } i \tag{6.49}$$

因此，既不改变次数也不改变 (齐次) 控制点；方程 (6.45) 的线性函数仅仅将参数范围从 $[a,b]$ 改变为 $[c,d]$ 。

现在让 $\mathbf{C}^w(u)$ 是在 $u\in[a,b]$ 上的任意 p 次 NURBS 曲线，其节点向量为 U ，并且让 $u=f(s)$ 是在 $s\in[c,d]$ 上的 q 次分段多项式重新参数化函数。现在 $f(s)$ 可以是任何形式，但为了方便术语，我们假设它是 B 样条形式，即

$$u=f(s)=\sum_{i=0}^{n}N_{i,q}(s) f_i \tag{6.50}$$

其中 $\{f_i\}$ 是标量。用 S 表示节点向量；那么 $\mathbf{C}^w(f(s))$ 是在 $s\in[c,d]$ 上的 pq 次 NURBS 曲线，其节点和控制点可以如下计算:

1. 设 $\{s_i\}$ 表示 $f(s)$ 的一组不同的内部节点，设 $\{u_i\}=\{f(s_i)\}$ 表示它们的映像；使用节点细化插入所有 u_i 和 U 的所有原始内部节点，直到它们都具有重数 p.$\mathbf{C}^w(u)$ ，然后在分段 Bézier 形式中；用 U' 表示细化后的节点向量；

2. 设

$$s=g(u)=f^{-1}(s) \tag{6.51}$$

是 $f(s)$ 的反函数；然后形成新的节点向量 S' ，其不同的节点是 $s_i=g(u_i)$ 的不同节点的像 U' 。所有内部节点在 S' 中出现的重数为 pq ；

3. 使用方程 (6.39)-(6.44) 和在步骤 2 中从新的 s 节点获得的 Δs_i 来计算 $\mathbf{C}^w(s)$ 的新控制点，该控制点也以分段贝塞尔形式表示；

4. 对 $\mathbf{C}^w(s)$ 和 S' 应用节点移除，以获得 $\mathbf{C}^w(s)$ 的最小表示。已知 $\mathbf{C}^w(s)$ 的连续性及其节点的重数，因此不需要一般的节点移除 (如算法 A5.5 中所示)。设 s_i 为 S' 中的一个节点，设 $u_i=f(s_i)$ ；用 m_i^u 和 m_i^s 表示原始节点向量 U 和 S 中 u_i 和 s_i 的重数，分别为 $(m_i^u,m_i^s\geq 0)$ 。那么 s_i 在 S' 中的重数 m_i 为

$$m_i = pq - p + m_i^u \text{ if } m_i^s = 0$$

$$m_i = pq - q + m_i^s \text{ if } m_i^u = 0$$

$$m_i = \max\left(pq - p + m_i^u, pq - q + m_i^s\right) \text{ if } m_i^u \neq 0, m_i^s \neq 0 \tag{6.52}$$

如果 $u = f(s) = \alpha s + \beta$，那么 $s = g(u) = (u - \beta)/\alpha$。(齐次) 控制点不变，新节点是通过从原始节点获得的

$$s_i = g(u_i) \tag{6.53}$$

例如，节点向量 $U = \{0, 0, 0, 1/3, 1, 1, 1\}$ 和 $S' = \{1, 1, 1, 3, 7, 7, 7\}$ 是等价的 (定义相同的曲线)，因为 $s = 6u + 1$ 将 U 映射到 S'。

图 6.11a-6.11d 显示了重新参数化的示例。曲线是定义在 $U = \{0, 0, 0, 0, 1/2, 1, 1, 1, 1\}$ 上的三次 B 样条曲线。在图 6.11a 中，重新参数化函数 (显示在右上角) 由标量 $F = \{0, 2/5, 1\}$ 定义，位于节点向量 $S = \{0, 0, 0, 1, 1, 1\}$ 上。图 6.11b 显示了由 $F = \{0, 2/5, 9/10, 1\}$ 和 $S = \{0, 0, 0, 1/2, 1, 1, 1\}$ 定义的重新参数化。在图 6.11c 中，该函数由 $F = \{0, 1/2, 7/10, 9/10, 1\}$ 和 $S = \{0, 0, 0$ 、$1/2, 1/2, 1, 1, 1\}$ 给出。图 6.11d 描绘了使用线性 B 样条函数进行的重新参数化，该函数由 $F = \{0, 1\}$ 和 $S = \{0, 0, 1, 1\}$ 给出。

现在我们转向使用有理 (或分段有理) 函数进行重新参数化。

形式为

$$u = f(s) = \frac{g(s)}{h(s)} \tag{6.54}$$

(a) $F = \{0, 2/5, 1\}, S = \{0, 0, 0, 1, 1, 1\}$；(b) $F = \{0, 2/5, 9/10, 1\}, S = \{0, 0, 0, 1/2, 1, 1, 1\}$。

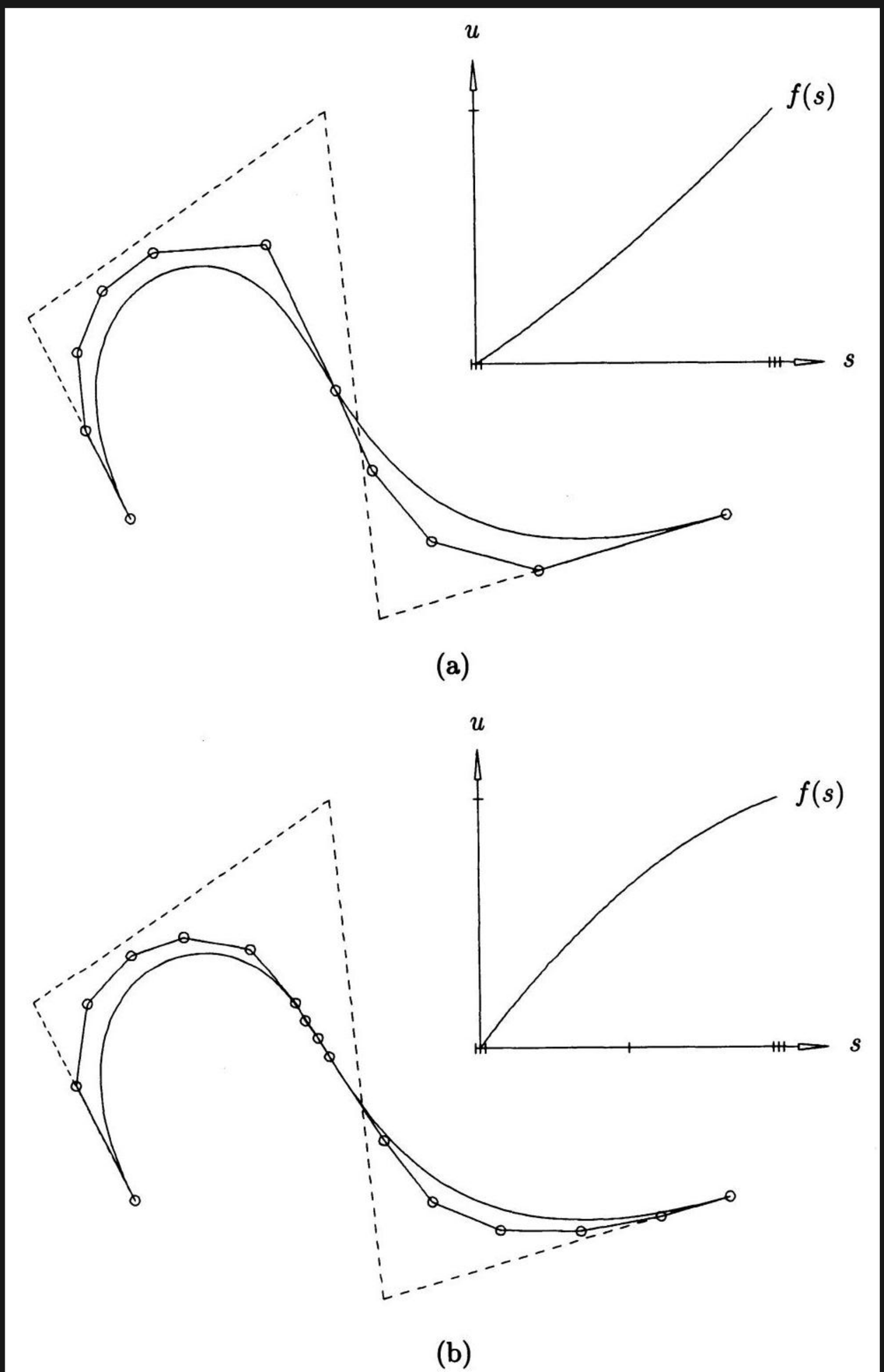
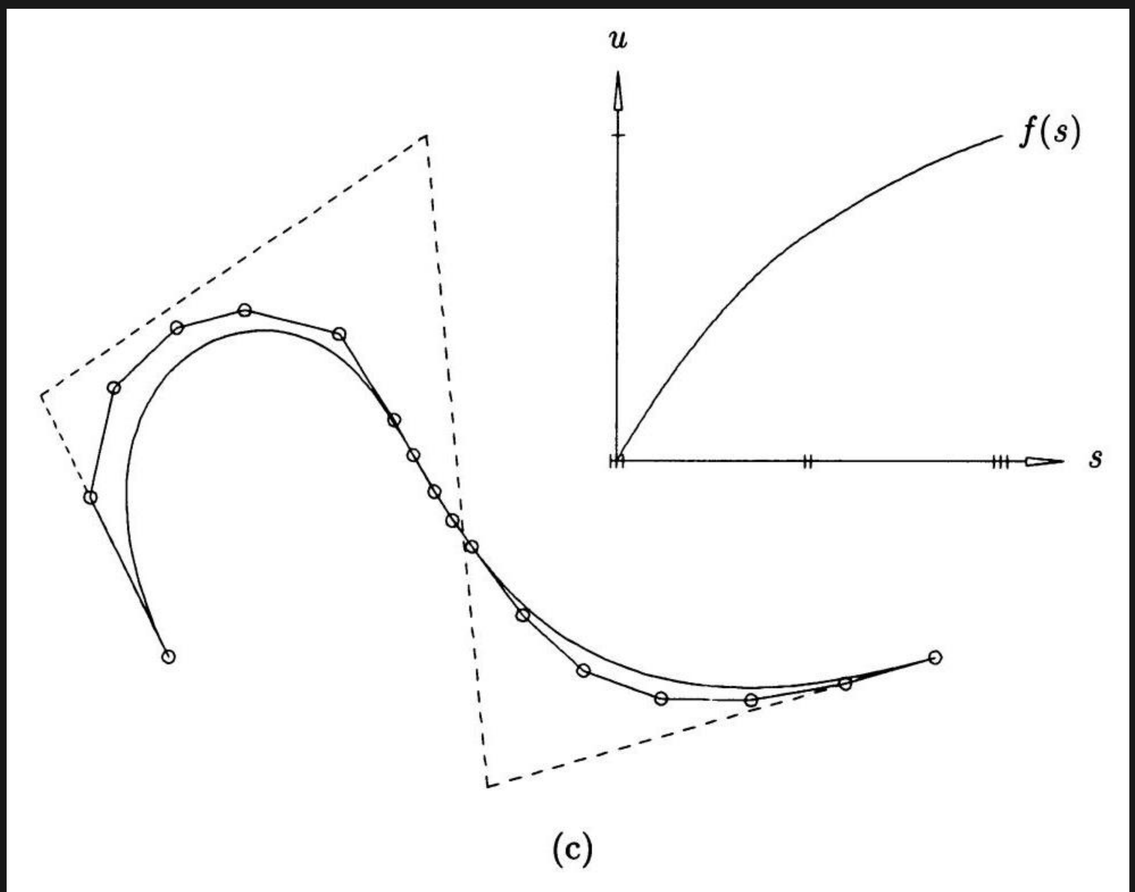


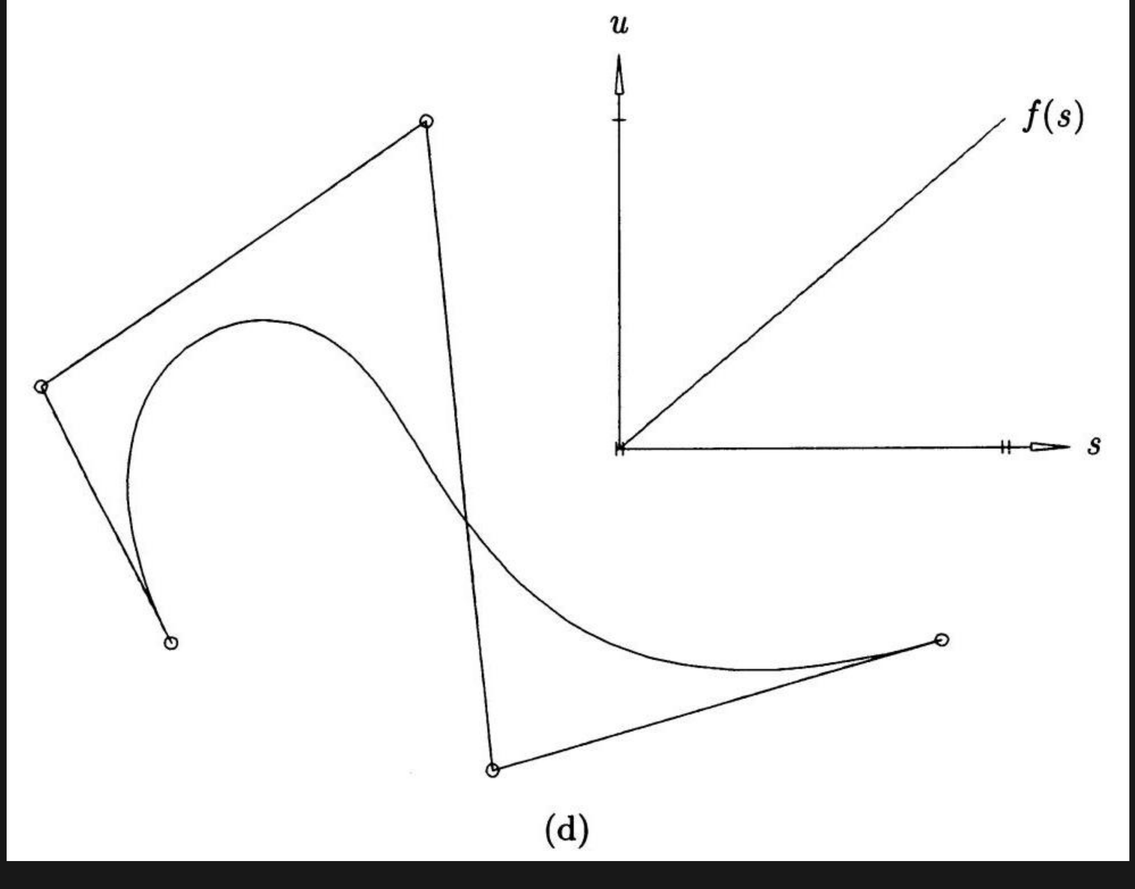
图 6.11. 使用二次 B 样条函数重新参数化的 B 样条曲线。

如前所述，如果 $\mathbf{C}^w(u)$ 或 $f(s)$ 有内部节点，我们可以分三步进行重新参数化:

1. 插入节点以获得 $\mathbf{C}^w(u)$ 作为分段贝塞尔曲线；
2. 重新参数化贝塞尔段；
3. 移除不必要的节点。



(c)



(d)

图 6.11. (续)(c) $F=\{0,1/2,7/10,9/10,1\}, S=\{0,0,0,1/2,1/2,1,1,1\}$；(d) $F=\{0,1\}, S=\{0,0,1,1\}$ 。

因此，我们可以假设 $\mathbf{C}^w(u)$ 和 $f(s)$ 没有内部节点。从理论上讲，可以使用类似于多项式函数的技术通过有理函数进行重新参数化。设 $r(u)=\sum_{i=0}^{p} a_i u^i$ 为一个 p 次多项式。函数。那么

$$r(s)=r\left(\frac{g(s)}{h(s)}\right)=\sum_{i=0}^{p} a_i\left(\frac{g(s)}{h(s)}\right)^i$$

和

$$\bar{r}(s)=(h(s))^p r(s)=\sum_{i=0}^{p} a_i(g(s))^i(h(s))^{p-i}$$

如果 $g(s)$ 和 $h(s)$ 是 q 次多项式，那么 $\bar{r}(s)$ 是一个 pq 次多项式。现在设

$$\mathbf{C}^w(u)=(wx(u), wy(u), wz(u), w(u))$$

为一个 p 次多项式曲线在齐次空间中。那么

$$\mathbf{C}^w(s)=\mathbf{C}^w\left(\frac{g(s)}{h(s)}\right)=\left[wx\left(\frac{g(s)}{h(s)}\right),\ldots,w\left(\frac{g(s)}{h(s)}\right)\right]$$

和

$$\overline{\mathbf{C}}^w(s)=(h(s))^p\mathbf{C}^w(s)=\left[(h(s))^p wx\left(\frac{g(s)}{h(s)}\right),\ldots,(h(s))^p w\left(\frac{g(s)}{h(s)}\right)\right]$$

关于 $\overline{\mathbf{C}}^w(s)$ ，注意到

- $\overline{\mathbf{C}}^w(s)$ 是一个 pq 次多项式曲线在齐次空间中；
- $\overline{\mathbf{C}}^w(s)$ 和 $\mathbf{C}^w(s)$ 是齐次空间中的不同曲线，但它们在欧几里得空间中投影为同一曲线，即 $\overline{\mathbf{C}}(s)=\mathbf{C}(s)$ ；
- 使用 $\overline{\mathbf{C}}^w(s)=(h(s))^p\mathbf{C}^w(g(s)/h(s))$ 、链式法则和末端导数公式，可以计算 $\overline{\mathbf{C}}^w(s)$ 的新齐次控制点。

使用有理函数与多项式函数进行重新参数化的根本区别是:

- 如果 $f(s)$ 是多项式，则 $\mathbf{C}^w(u)$ (因此 $\mathbf{C}(u)$) 被重新参数化，但 $\mathbf{C}^w(u)$ 在几何上没有改变；此外，$\mathbf{C}(u)$ 的末端权重不变；
- 如果 $f(s)$ 是有理的，则生成不同的曲线 $\overline{\mathbf{C}}^w(s)$ ，它投影到相同的 (但重新参数化的) 曲线 $\mathbf{C}(u)$ 。一般来说，$\mathbf{C}(u)$ 的末端权重会改变。

示例

Ex6.7 由方程 (6.35) 给出的圆弧具有贝塞尔表示。显然，欧几里得控制点 $\{\mathbf{P}_i\} = \{(1,0),(1,1),(0,1)\}$ 保持不变。权重通过将方程 (6.35) 的分母函数的幂基和贝塞尔形式相等来确定。

$$\left(2-\sqrt{2}\right)s^2 + \left(\sqrt{2}-2\right)s + 1 = (1-s)^2 w_0 + 2s(1-s)w_1 + s^2 w_2$$

代入 $s = 0, 1, 1/2$ ，并求解 w_i 得到

$$\{w_i\} = \left\{1, \frac{\sqrt{2}}{2}, 1\right\} \tag{6.55}$$

图 6.12 显示了新曲线 $\bar{\mathbf{C}}^w(s)$ 及其投影 $\mathbf{C}(s)$ 。将此图与图 1.22 进行比较。

在实践中，通常不需要使用高于一阶的有理重新参数化函数。Lee 和 Lucian [Lee91] 对使用线性有理函数的重新参数化进行了出色的描述。我们在此总结他们的结果。设

$$\mathbf{C}(u) = \frac{\sum_{i=0}^{n} N_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^{n} N_{i,p}(u) w_i} \quad u \in [a,b]$$

$$s = g(u) = \frac{\alpha u + \beta}{\gamma u + \delta}$$

$$u = f(s) = \frac{-\delta s + \beta}{\gamma s - \alpha} \quad s \in [c,d]$$

($f(s)$ is the inverse of $g(u)$) 。设

$$\mu(u) = \gamma u + \delta \quad \lambda(s) = \gamma s - \alpha$$

为确保 $g(u)$ 和 $f(s)$ 表现良好，我们假设

$$\alpha\delta - \gamma\beta > 0$$

$\mu(u) \neq 0$ 对所有 $u \in [a,b]$

$\lambda(s) \neq 0$ 对所有 $s \in [c,d]$ 重新参数化的曲线 $\mathbf{C}(s)$ 如下获得:

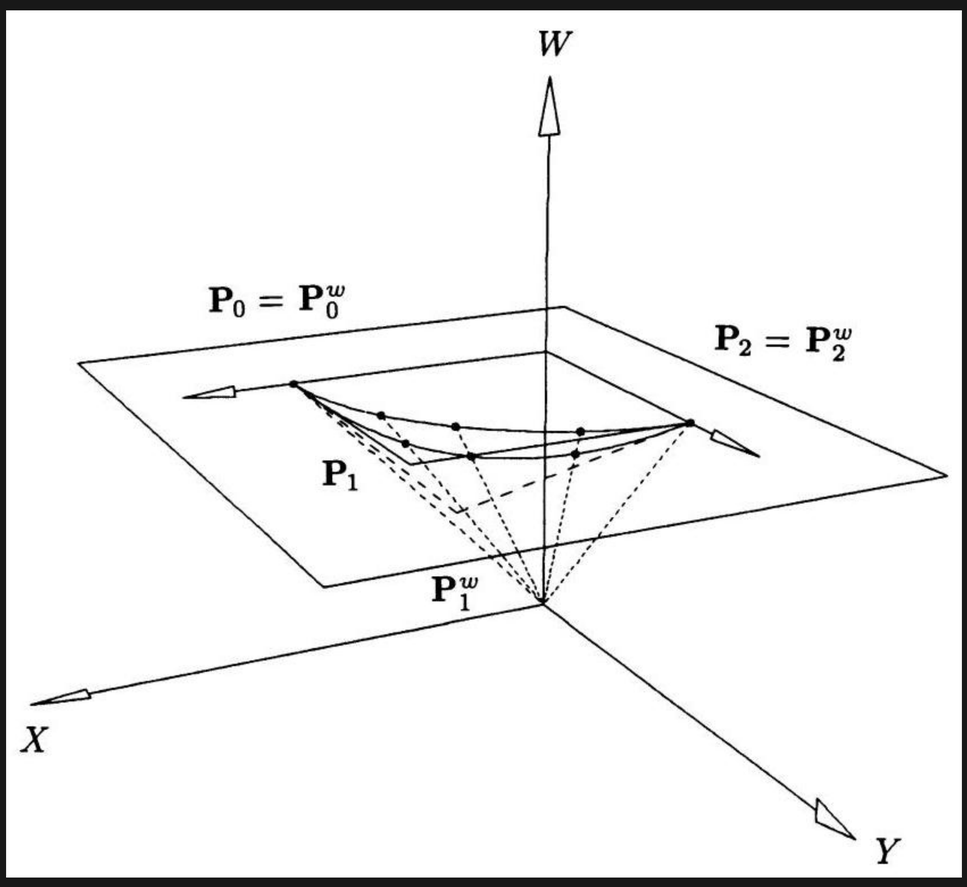


图 6.12. 使用权重 $\{1, \sqrt{2}/2, 1\}$ 的圆弧的均匀表示。

1. 控制点 $\{\mathbf{P}_i\}$ 不变；

2. 新的节点是原始节点在 $g(u)$ 下的映像，$s_i = g(u_i)$ ；

3. 新的权重 $\{\bar{w}_i\}$ (模一个共同的非零因子) 是通过

通过

$$\bar{w}_i = w_i \prod_{j=1}^{p} \lambda(s_{i+j}) \tag{6.56}$$

或

$$\bar{w}_i = \frac{w_i}{\prod_{j=1}^{p} \mu(u_{i+j})} \tag{6.57}$$

s_{i+j} 和 u_{i+j} 分别是新的和旧的节点。

图 6.13a 到 6.13c 显示了重新参数化的例子。三次 NURBS 曲线最初在节点向量 $U = \{0,0,0,0,{}^3/_{10},7/10,1,1,1,1\}$ 上定义。在图 6.13a 和 6.13 b 中，所有权重最初都等于一。在图 6.13c 中，原始权重为 $W = \{1,1/2,2,3,1/2,1\}$ 。在图 6.13a 中，合理函数是

$$g(u) = \frac{2u+1}{3u+2}$$

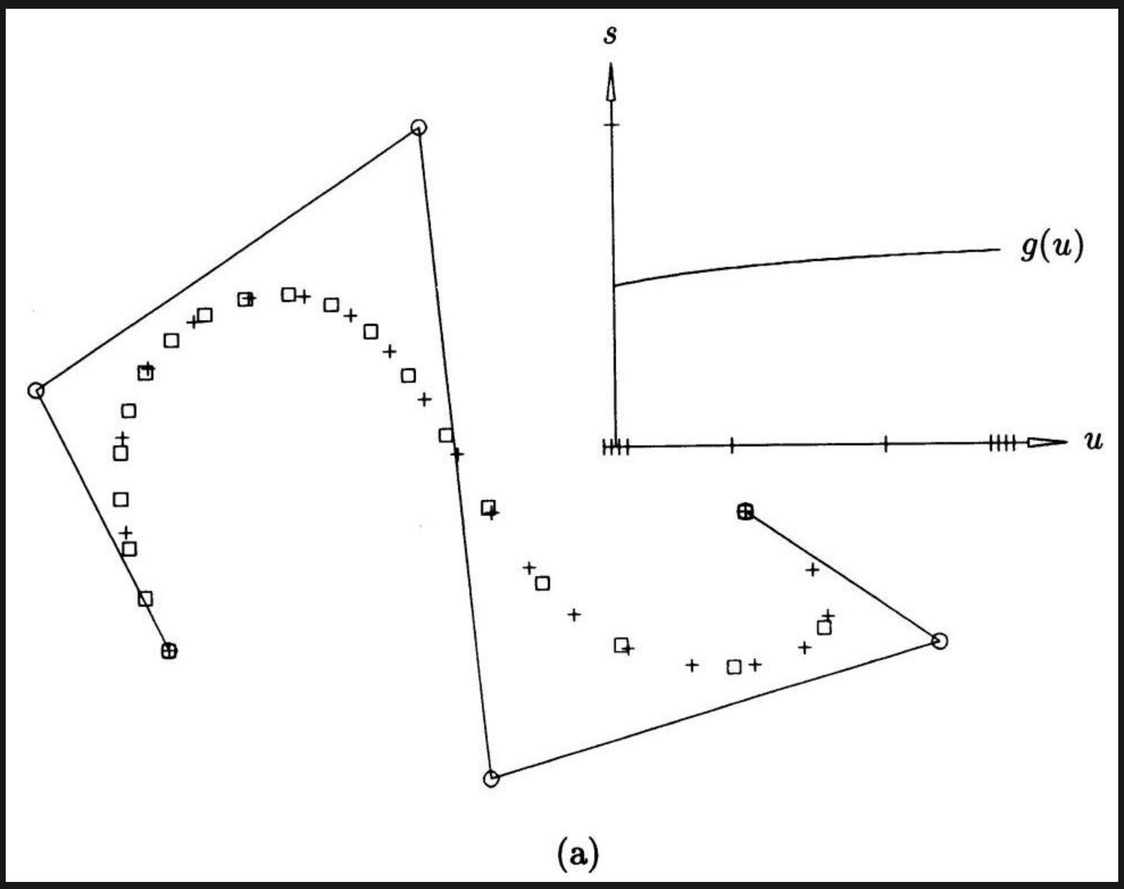
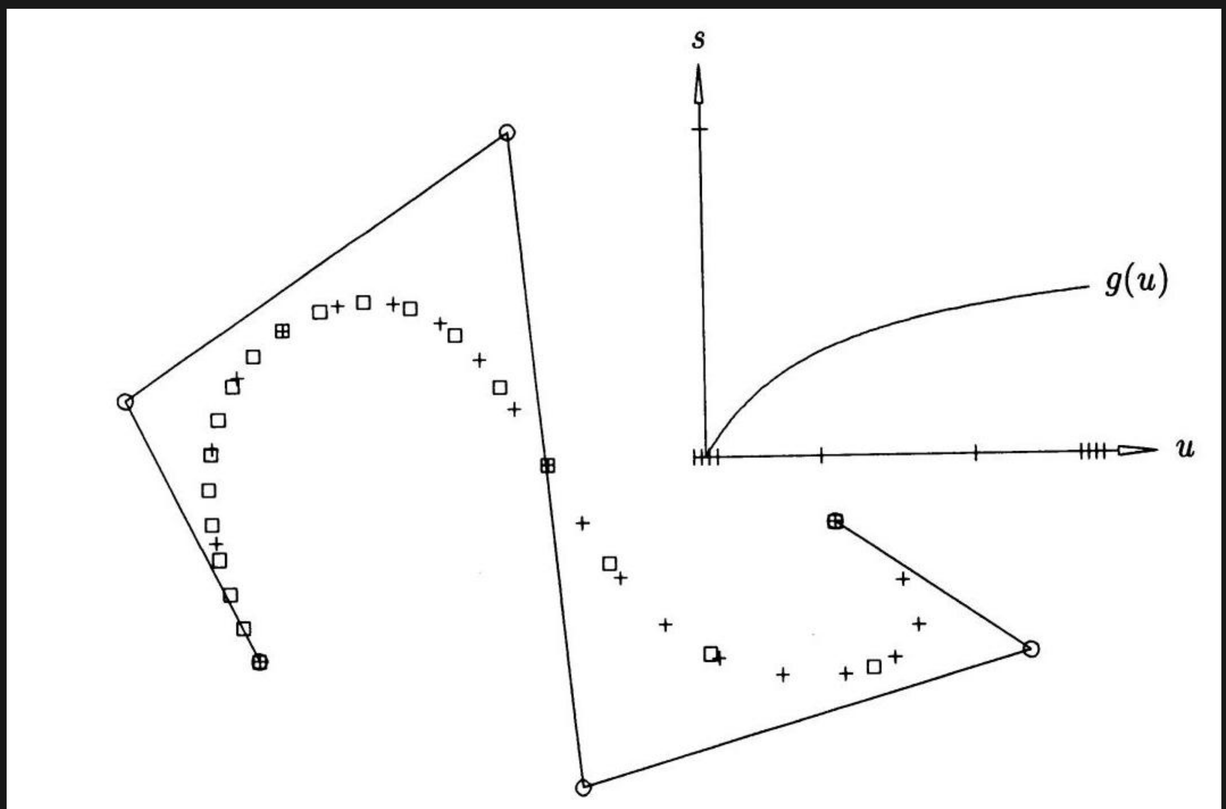
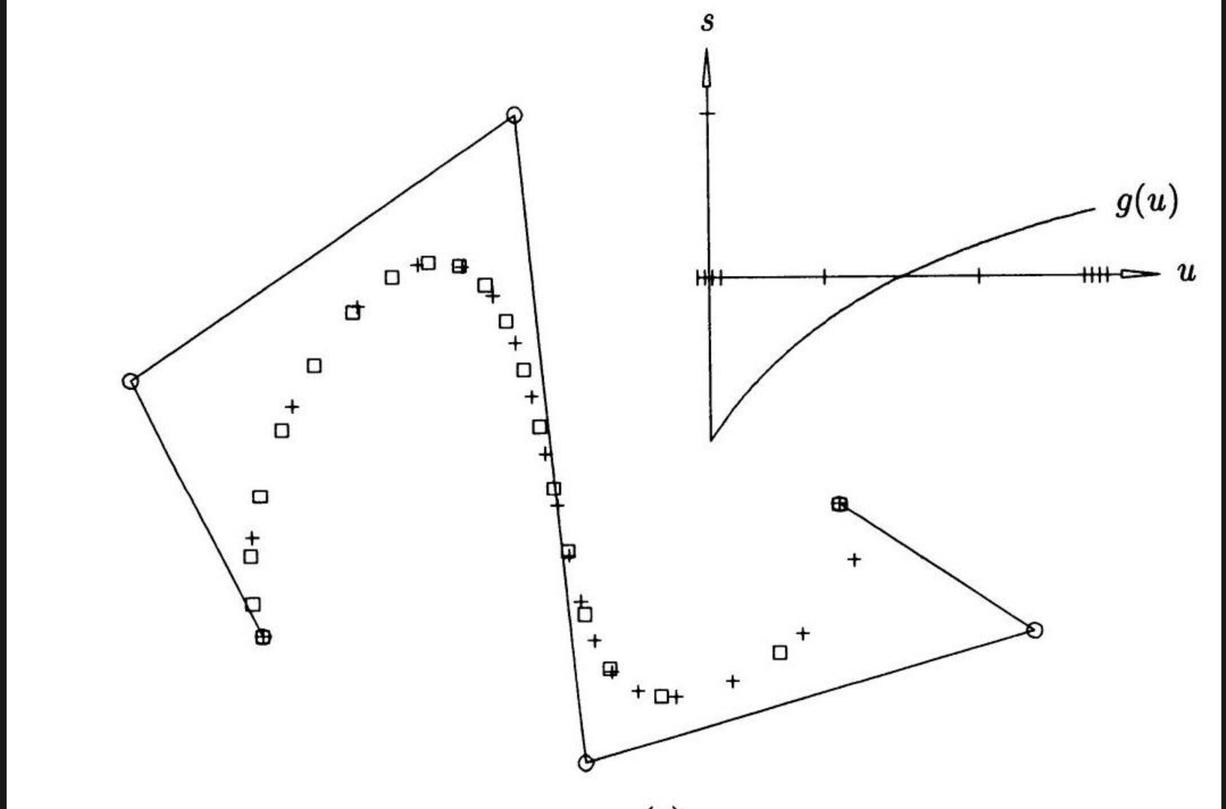


图 6.13. NURBS 曲线的合理重新参数化；+ 显示了原始参数化，而 □ 说明了新的参数化。(a) $g(u) = (2u+1)/(3u+2)$；(b) $g(u) = {}^{2u}/_{(3u+1)}$；(c) $g(u) = {}^{(2u-1)}/_{(3u+2)}$。

图 6.13.(续)。



(b)



(c)

igure 6.13 (*Continued*.)

导致新的节点向量

$$U = \{0.5, 0.5, 0.5, 0.5, 0.55, 0.58, 0.6, 0.6, 0.6, 0.6\}$$

和权重

$$W = \{0.125, 0.08, 0.04, 0.01, 0.009, 0.008\}$$

+ 显示了原始参数化，而 □ 标记了新的参数化。图 6.13b
应用

$$g\left(u\right) = \frac{2u}{3u+1}$$

产生

$$U = \{0, 0, 0, 0, 0.31, 0.45, 0.5, 0.5, 0.5, 0.5\}$$

和

$$W = \{1.0, 0.52, 0.16, 0.04, 0.02, 0.01\}$$

再次，+ 显示了原始参数化，而 □ 显示了新的参数化。最后，在
图 6.13c 中，该函数

$$g\left(u\right) = \frac{2u-1}{3u+2}$$

产生 $U = \{-0.5, -0.5, -0.5, -0.5, -0.13, 0.09, 0.2, 0.2, 0.2, 0.2\}$
和

$$W = \{0.125, 0.04, 0.08, 0.05, 0.004, 0.008\}$$

表面公式是相似的。例如，方程 (6.56) 概括为

$$\bar{w}_{i,j} = w_{i,j} \prod_{k=1}^{p} \lambda\left(s_{i+k}\right) \;\; j = 0, \ldots, m$$

在 u 方向上，并且

$$\bar{w}_{i,j} = w_{i,j} \prod_{\ell=1}^{q} \lambda\left(t_{j+\ell}\right) \ i = 0, \ldots, n$$

在 v 方向上，其中

$$s = g\left(u\right) = \frac{\alpha_u u + \beta_u}{\gamma_u u + \delta_u} \ t = h\left(v\right) = \frac{\alpha_v v + \beta_v}{\gamma_v v + \delta_v}$$

和

$$\lambda\left(s\right) = \gamma_u s - \alpha_u \ \lambda\left(t\right) = \gamma_v t - \alpha_v$$

方程 (6.57) 以类似的方式进行了推广。

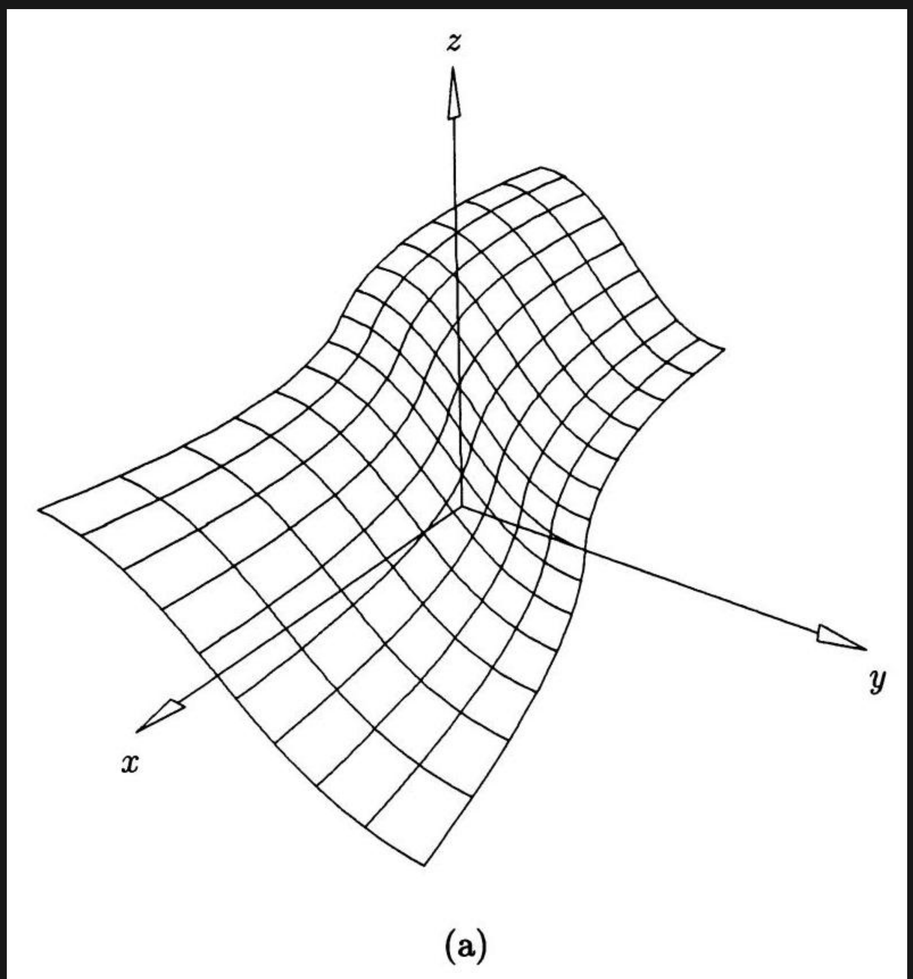
图 6.14a – 6.14 d 显示了表面重新参数化的示例。测试表面是一个 (3×2) 度的表面，定义在

$$U = \{0, 0, 0, 0, 1, 1, 1, 1\} \ V = \{0, 0, 0, \frac{1}{2}, 1, 1, 1\}$$

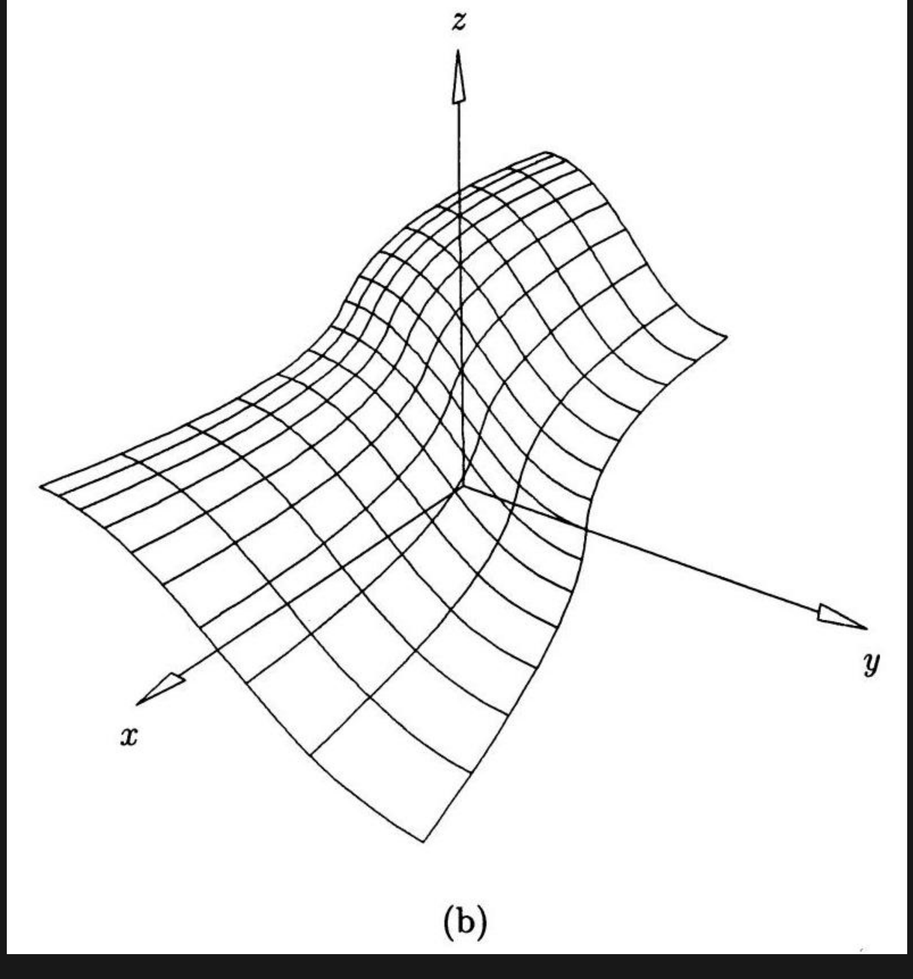
权重均等于一。重新参数化函数为

$$g\left(u\right) = \frac{2u+1}{3u+2} \ h\left(v\right) = \frac{2v+1}{3v+2}$$

图 6.14a 显示了原始表面；注意参数线的均匀分布。在图 6.14b 中，表面在 u 方向上进行了重新参数化，



(a)



(b)

图 6.14. 使用 $g\left(u\right) = {}^{(2u+1)}/_{(3u+2)}$ 的 NURBS 表面的有理重新参数化。(a) 原始表面；(b) 在 u 方向上的重新参数化；(c) 在 v 方向上的重新参数化；(d) 在两个方向上的重新参数化。

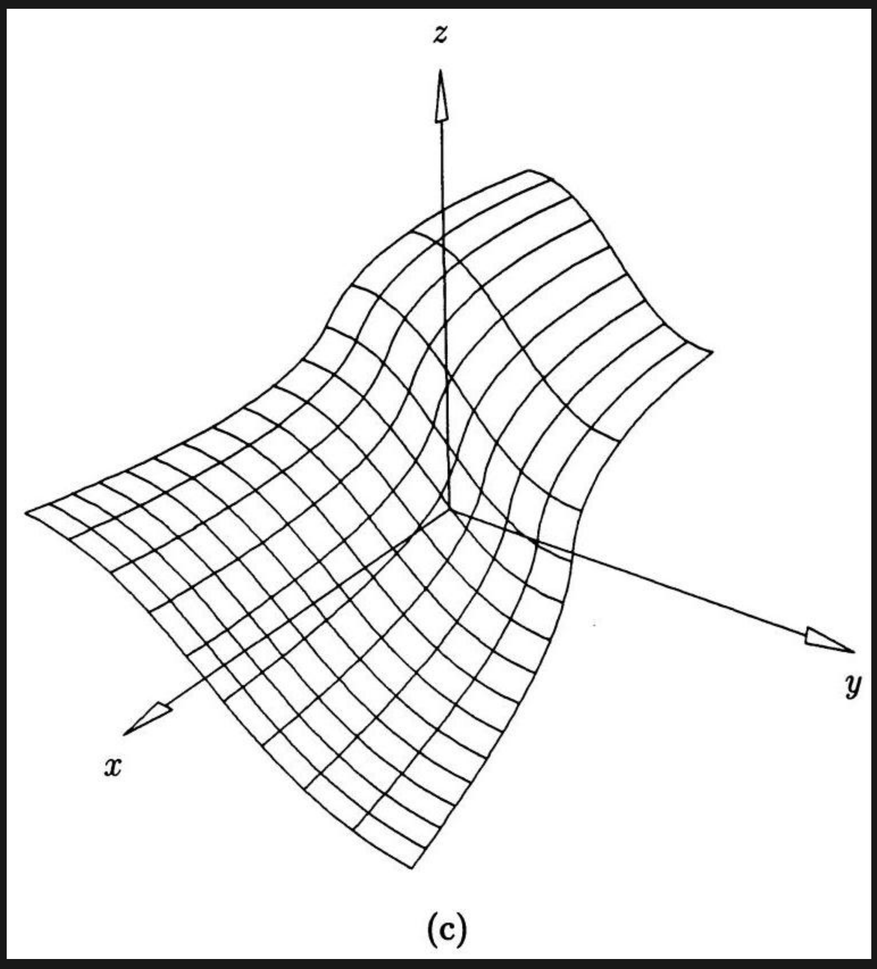


图 6.14. (续)
图 6.14c 在 v 方向上，图 6.14d 在两个方向上。注意图 6.14 b – 6.14 d 中参数线的分布不均匀。
现在假设 $\mathbf{C}(u)$ 的末端权重不相等，即 $w_0 \neq w_n$ 。由于线性有理函数 $s = g(u)$ 有三个自由度，因此应该能够确定满足三个条件的 $g(u)$ 。

$$c = g(a) \;\; d = g(b) \;\; \bar{w}_0 = \bar{w}_n$$

Lee 和 Lucian [Lee91] 推导出满足这些条件的函数。

$$s = g(u) = \frac{\sqrt[d]{w_0}(b-u)\,c + \sqrt[d]{w_n}(u-a)\,d}{\sqrt[d]{w_0}(b-u) + \sqrt[d]{w_n}(u-a)} \tag{6.58}$$

图 6.15 显示了一个重新参数化的例子，以使端权重相等。原始的三次 NURBS 曲线由以下定义

$$U = \left\{0, 0, 0, 0, \frac{3}{10}, \frac{7}{10}, 1, 1, 1, 1\right\} \;\; W = \{2, 8, 3, 4, 2, 5\}$$

选择的新区间为 $[c, d] = [1/10, 4/5]$ 。由公式 (6.58) 计算的重新参数化函数为

$$g(u) = \frac{1.242u + 0.126}{0.45u + 1.26}$$

产生新的节点和权重

$$U' = \{0.1, 0.1, 0.1, 0.1, 0.36, 0.63, 0.8, 0.8, 0.8, 0.8\}$$

$$W' = \{1, 3.61, 1.08, 1.06, 0.43, 1\}$$

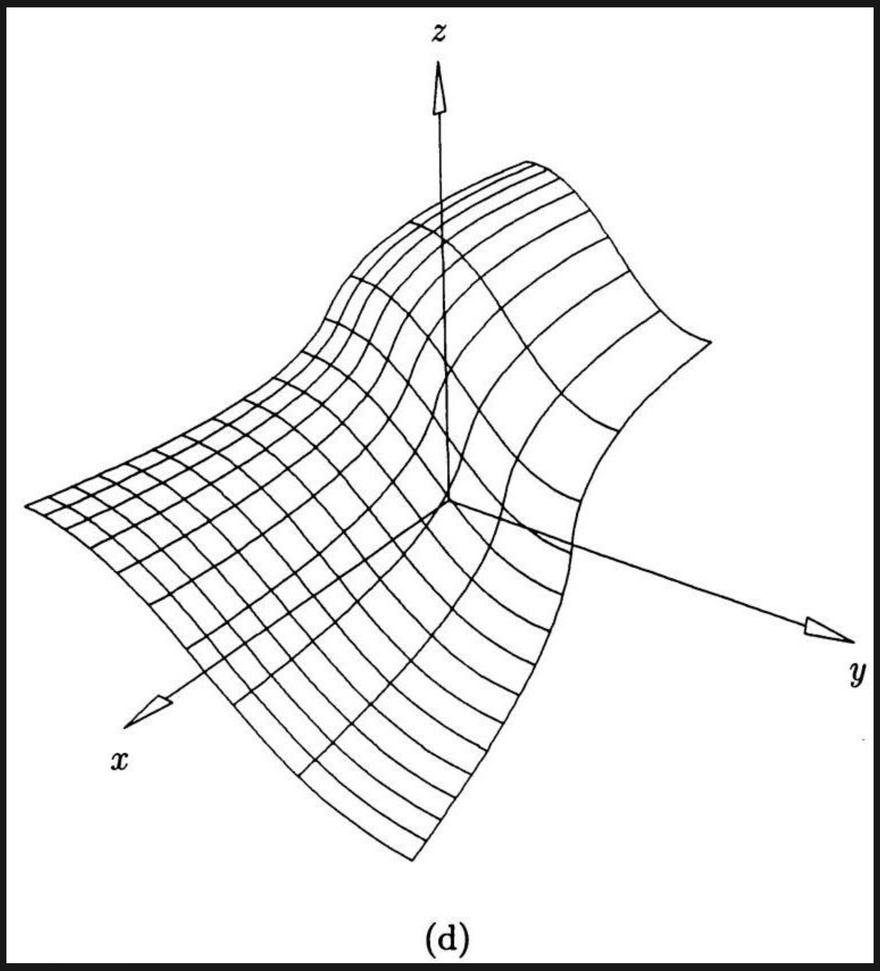


图 6.14. (续)
其中端权重被选择为 1。同样，+ 标记原始参数化，而 □ 标记新的参数化。

我们在第 4 章中给出了例子，表明权重的任意变化通常会改变有理 NURBS 曲线的形状 (例如，见图 4.2 和 4.4)。在本节中，我们在不修改曲线形状的情况下改变了权重，唯一的变化是曲线的参数化。这表明，对于给定的曲线，权重之间必定存在一种关系来决定曲线形状，即保持这种关系的权重变化会导致曲线的重新参数化，而不会改变形状。这些关系被称为形状不变量或形状因子。对于有理 Bézier 曲线，它们仅依赖于次数；对于 NURBS 曲线，它们依赖于次数和节点。一般来说，这些形状不变量的推导相当复杂 (见 [Forr68; Vers75; Patt85; Lee87; Pieg91a])。我们在此提供一些供参考；c_i 表示常数。

- 二次 Bézier 曲线 (圆锥曲线):

$$\frac{w_0 w_2}{w_1^2} = c_1 \tag{6.59}$$

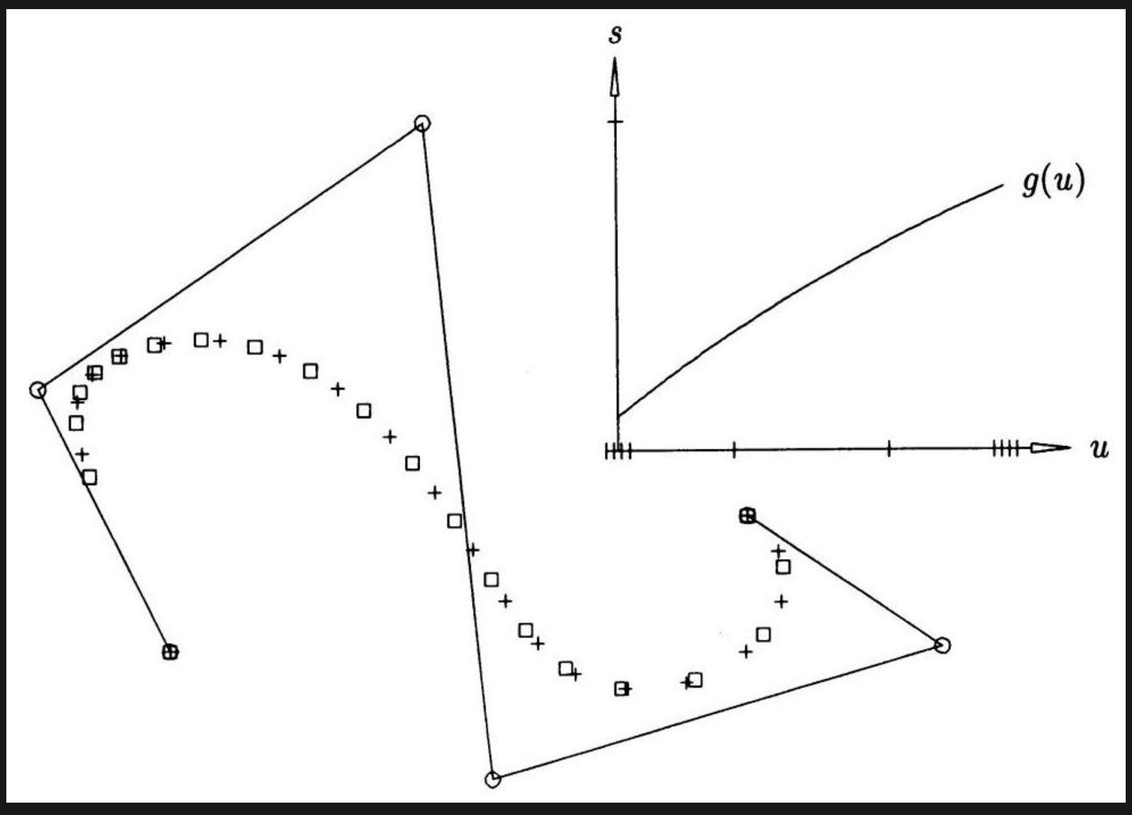


图 6.15. 曲线重新参数化以使端权重相等。

考虑圆弧，由 $\{w_i\} = \{1, 1, 2\}$ (图 1.19b) 或 $\{w_i\} = \left\{1, \sqrt{2}/2, 1\right\}$ (公式 [6.55]) 定义

$$2 = \frac{(1)(2)}{1^2} = \frac{(1)(1)}{\left(\frac{\sqrt{2}}{2}\right)^2} = 2$$

假设控制点固定，任何三个权重代入公式 (6.59) 得到值 2 将产生圆弧；

- 三次 Bézier 曲线:

$$\frac{w_0 w_2}{w_1^2} = c_1 \frac{w_1 w_3}{w_2^2} = c_2 \tag{6.60}$$

- 一个 p 次贝塞尔曲线:

$$\frac{w_0 w_2}{w_1^2} = c_1, \ldots, \frac{w_{p-2} w_p}{w_{p-1}^2} = c_{p-1} \tag{6.61}$$

- 二次 NURBS 曲线 $\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,2}(u)\,\mathbf{P}_i^w$ 在 $U = \{u_0, \ldots, u_m\}$ 上:

$$\frac{(u_{i+2} - u_{i+1})\,w_{i-1} + (u_{i+1} - u_i)\,w_i}{(u_{i+3} - u_{i+1})\,(u_{i+2} - u_i)\,w_i^2} \times$$

$$\frac{(u_{i+3} - u_{i+2})\,w_i + (u_{i+2} - u_{i+1})\,w_{i+1}}{(u_{i+3} - u_{i+1})\,(u_{i+2} - u_i)\,w_i^2} = c_i \tag{6.62}$$

对于 $i = 1, \ldots, n-1$ 。

我们以提及另一种有趣的技术来结束曲线的重新参数化。Fuhr 和 Kallay [Fuhr92] 开发了一种使用 C^1 连续分段线性有理函数插值一组函数值的技术。他们的方法可以用来获得 C^1 连续重新参数化，这不会提高次数，但可以映射任意多个内部点，$u_i = f(s_i)$ 。

NURBS 曲面 $\mathbf{S}^w(u, v) = \sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(u)\,N_{j,q}(v)\,\mathbf{P}_{i,j}^w$ 可以在 u 、v 或两个方向上重新参数化。例如，它可以通过将 u 节点映射到 s 节点 (方程 [6.51] 和 [6.52]) 在 u 方向上用多项式函数重新参数化，并将方程 (6.39) 到 (6.44) 应用于控制点的 $m+1$ 列。

5 曲线和曲面的反转

在本节中，我们使用重新参数化来反转 NURBS 曲线的方向和曲面法向量的方向。

设 $\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w$ 是在 $u \in [a, b]$ 上的 NURBS 曲线，且 $u = f(s)$ 是在 $s \in [c, d]$ 上的重新参数化函数。在第 6.4 节中，我们假设 $f'(s) > 0$ 在 $s \in [c, d]$ 上。这在示例 Ex6.6 和方程 (6.41) 到 (6.44) 以及方程 (6.49) 的推导中隐含使用。

如果我们假设 $f'(s) < 0$ 在 $s \in [c, d]$ 上，6.4 节的概念仍然适用。特别地，设

$$u = f(s) = -s + a + b \; s \in [a, b] \tag{6.63}$$

注意到

$$b = f(a) \;\; a = f(b) \tag{6.64}$$

并且 $f'(s) = -1$ 在 $s \in [a, b]$ 上。$f(s)$ 的逆是

$$s = g(u) = -u + a + b \tag{6.65}$$

从 Eq. (6.53) 开始，$\mathbf{C}^w(s)$ 的节点通过计算得出

$$s_{m-p-i} = -u_{p+i} + a + b \; i = 1, \ldots, m-2p-1 \tag{6.66}$$

其中 u_i 是 $\mathbf{C}^w(u), 0 \le i \le m$ 的节点。例如，如果 $U = \{0, 0, 0, 1, 3, 6\ ,\ 6, 8, 8, 8\}$ ，那么 $S = \{0, 0, 0, 2, 2, 5, 7, 8, 8, 8\}$ 。

$\mathbf{C}^w(s)$ 的控制点 $\mathbf{Q}_i^w$ 被计算为

$$\mathbf{Q}_0^w = \mathbf{C}^w(f(s = a)) = \mathbf{C}^w(u = b) = \mathbf{P}_n^w$$

$$\mathbf{Q}_1^w = \frac{s_{p+1} - a}{p} \mathbf{C}^{w(1)}(s = a) + \mathbf{Q}_0^w$$

$$= \frac{b - u_{m-p-1}}{p} \mathbf{C}^{w(1)}(u = b)\, f'(s = a) + \mathbf{P}_n^w$$

$$= \frac{b - u_{m-p-1}}{p} \frac{p}{b - u_{m-p-1}} \left(\mathbf{P}_n^w - \mathbf{P}_{n-1}^w\right)(-1) + \mathbf{P}_n^w$$

$$= \mathbf{P}_{n-1}^w$$

一般来说

$$\mathbf{Q}_i^w = \mathbf{P}_{n-i}^w \tag{6.67}$$

方程 (6.63)-(6.67) 的效果是反转曲线的方向。这在图 6.16 中显示；注意参数化没有改变。

设 $\mathbf{S}^w(u, v) = \sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(u)\,N_{j,q}(v)\,\mathbf{P}_{i,j}^w$ 为具有节点向量 $U = \{u_0, \ldots, u_r\}$ 和 $V = \{v_0, \ldots, v_s\}$ 的 NURBS 曲面。曲面的参数化也可以被反转。更具体地说，u 的反转产生

$$\mathbf{S}^w(s, v) = \sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(s)\,N_{j,q}(v)\,\mathbf{Q}_{i,j}^w \tag{6.68}$$

在 S 和 V 上，其中

$$s_{r-p-i} = -u_{p+i} + u_0 + u_r \; i = 1, \ldots, r - 2p - 1 \tag{6.69}$$

和

$$\mathbf{Q}^w_{i,j} = \mathbf{P}^w_{n-i,j} \; i = 0, \ldots, n \; j = 0, \ldots, m \tag{6.70}$$

类似地，v 的反转产生

$$\mathbf{S}^w(u,t) = \sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(u) \, N_{j,q}(t) \, \mathbf{Q}^w_{i,j} \tag{6.71}$$

在 U 和 T 上，其中

$$t_{s-q-j} = -v_{q+j} + v_0 + v_s \; j = 1, \ldots, s - 2q - 1 \tag{6.72}$$

和

$$\mathbf{Q}^w_{i,j} = \mathbf{P}^w_{i,m-j} \; i = 0, \ldots, n \; j = 0, \ldots, m \tag{6.73}$$

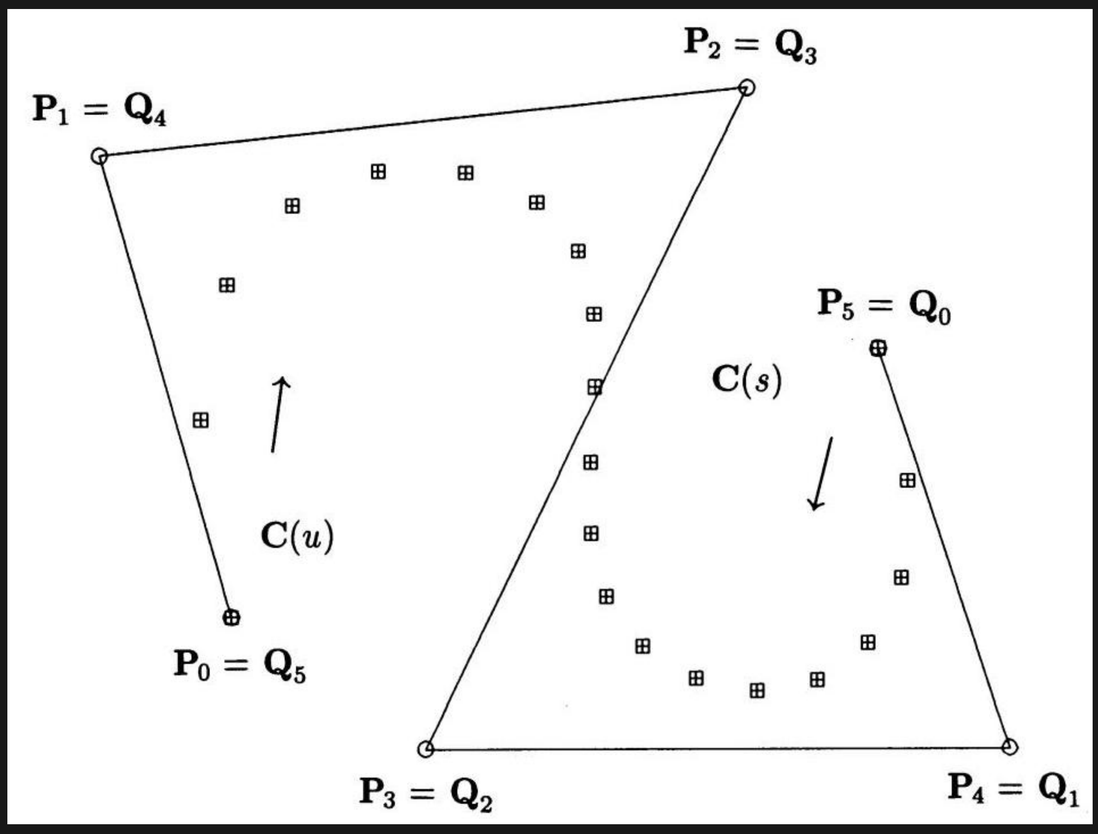


图 6.16. 曲线反转；没有参数化变化发生。

在任意参数点 (u, v) 处，表面法向量由以下公式给出

$$\mathbf{N}(u,v) = \mathbf{S}_u(u,v) \times \mathbf{S}_v(u,v) \tag{6.74}$$

显然，通过反转 $\mathbf{S}_u(u,v)$ 或 $\mathbf{S}_v(u,v)$ (但不能同时反转) 可以反转 $\mathbf{N}(u,v)$ 的方向。这是通过分别应用方程 (6.68)-(6.70) 或方程 (6.71)-(6.73) 来实现的。

图 6.17a – 6.17c 显示了表面反转的示例。在图 6.17a 中，显示了原始度 (3×2) 表面，以及 $\mathbf{S}_u, \mathbf{S}_v$ 和 $\mathbf{N}$ 。在图 6.17 b 和 6.17c 中，参数化在 u 和 v 方向上被反转。注意，在每种情况下法向量都被反转。

6 B-spline 和分段幂基形式之间的转换

将 B-spline 曲线或表面转换为分段幂基形式的步骤如下:

1. 使用节点细化，例如算法 A5.4-A5.7，将曲线或表面分解为分段 Bézier 形式；
2. 通过应用矩阵乘法 (基变换) 将每个 Bézier 曲线段 (或表面片) 转换为幂基形式；
3. 应用矩阵乘法对步骤 2 中获得的幂基形式进行重新参数化。

像往常一样，所有方程都应用于四维控制点和幂基系数；为方便起见，我们省略所有 w 上标。步骤 1 无需进一步阐述。

假设第 j 条曲线段 $\mathbf{C}_j(u)$ 定义在 $u \in [u_j, u_{j+1}]$ 上。现在，在步骤 1 中为 $\mathbf{C}_j(u)$ 获得的 Bézier 控制点 $\mathbf{P}_i, i = 0, \ldots, p$ 是有效的，但我们对 Bézier 曲线的定义要求有 $[0,1]$ 参数化 (方程 [1.7] 和 [1.8])。设

$$u = (u_{j+1} - u_j)\, s + u_j \tag{6.75}$$

然后 $0 \le s \le 1$ ，第 j 段具有 Bézier 形式

$$\mathbf{C}_j(s) = \sum_{i=0}^{p} B_{i,p}(s) \, \mathbf{P}_i = [B_{i,p}(s)]^T [\mathbf{P}_i] = \left[s^i\right]^T M_p [\mathbf{P}_i] \tag{6.76}$$

其中 $[B_{i,p}(s)]^T$ 和 $\left[s^i\right]^T$ 是 $1 \times (p+1)$ 行向量的标量值，M_p 是标量的 $(p+1) \times (p+1)$ 矩阵，$[\mathbf{P}_i]$ 是四维点的 $(p+1) \times 1$ 列向量。设定

$$[\mathbf{a}_i] = M_p [\mathbf{P}_i] \tag{6.77}$$

我们得到

$$\mathbf{C}_j(s) = \left[s^i\right]^T [\mathbf{a}_i] \tag{6.78}$$

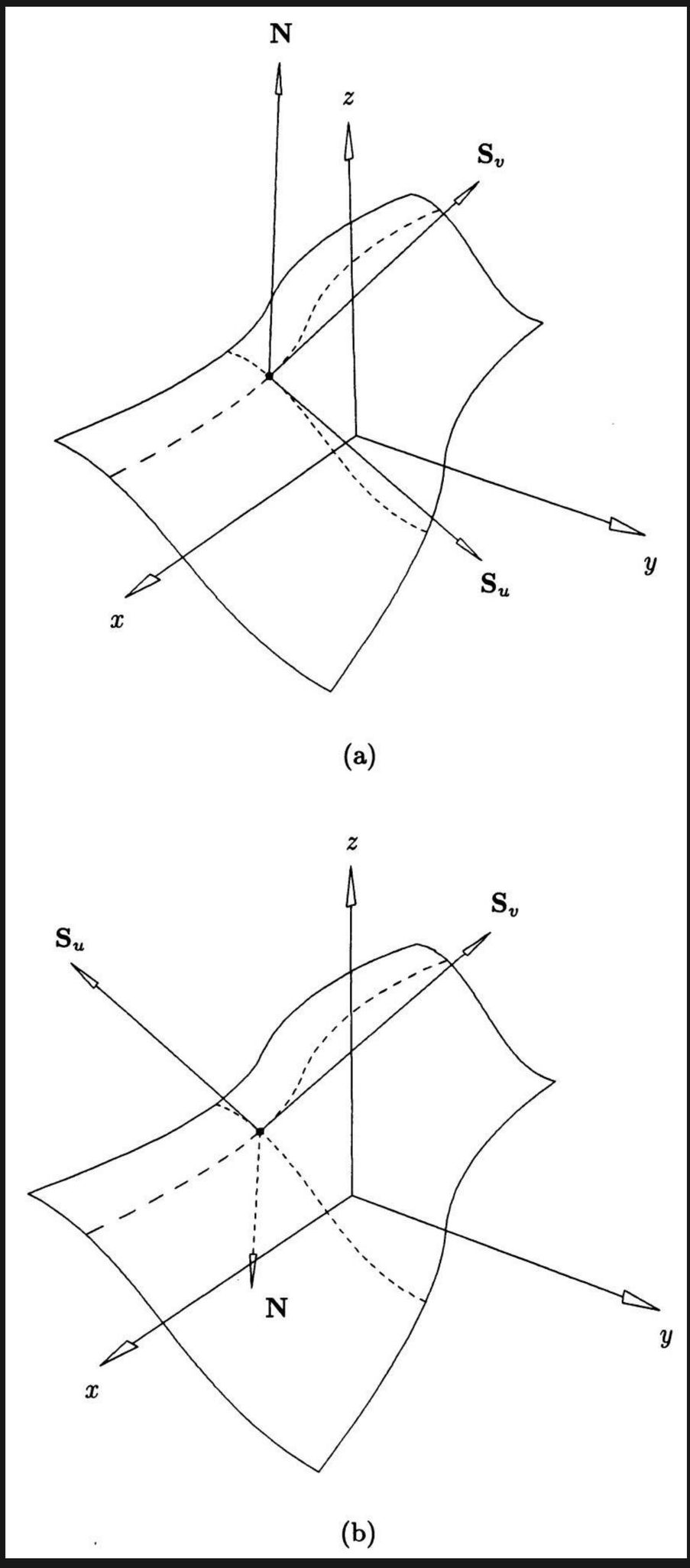


图 6.17. 表面反转。(a) 原始表面显示偏导数和法向量；(b) 在 u 方向上反转参数化；(c) 在 v 方向上反转参数化。

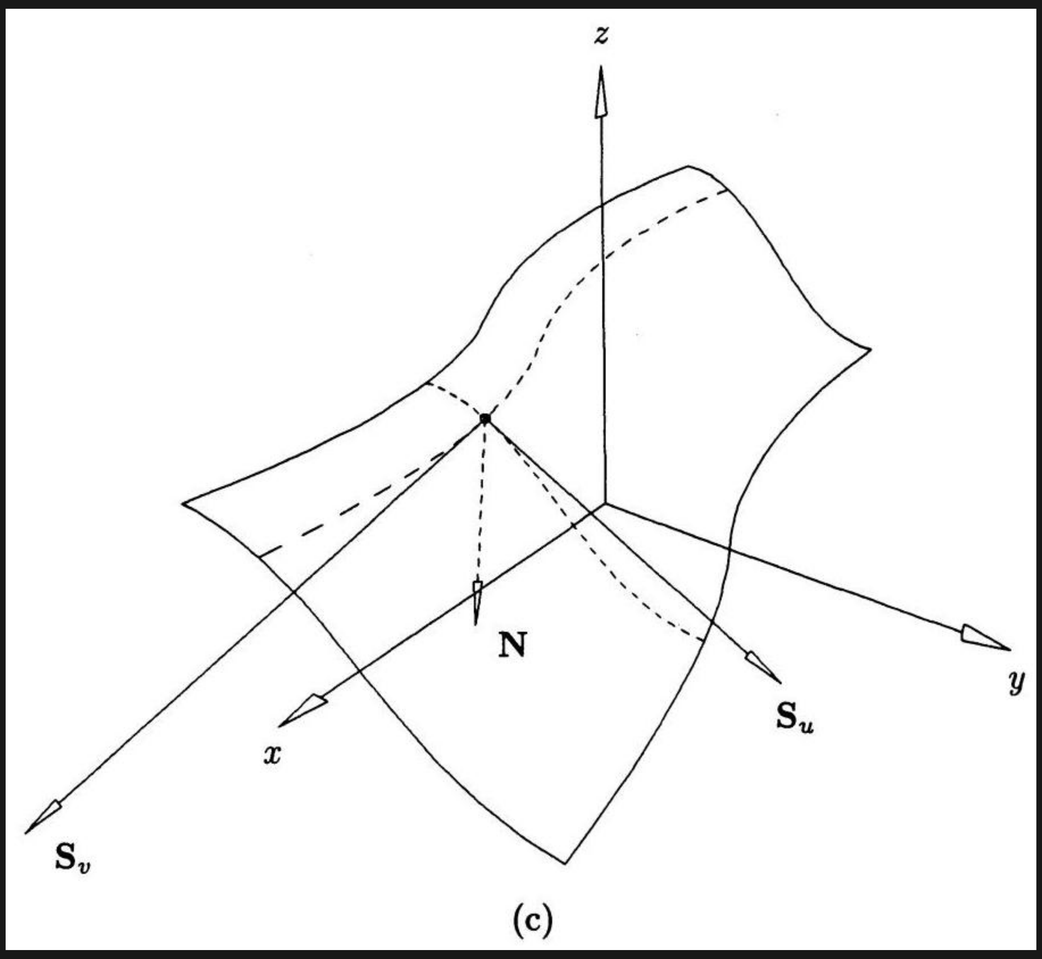


图 6.17.(续)
这是参数 s 中第 j 段的幂基形式。
现在我们推导任意 p 的矩阵 M_p 。作为热身和参考，我们计算前三个矩阵。从方程 (6.76) 可以得出

$$[B_{i,p}(s)]^T = [s^i]^T M_p \tag{6.79}$$

对于 $p = 1, 2, 3$ ，从方程 (1.8) 中，我们得到

$$\begin{bmatrix} 1-s & s \end{bmatrix} = \begin{bmatrix} 1 & s \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} (1-s)^2 & 2s(1-s) & s^2 \end{bmatrix} = \begin{bmatrix} 1 & s & s^2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -2 & 2 & 0 \\ 1 & -2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} (1-s)^3 & 3s(1-s)^2 & 3s^2(1-s) & s^3 \end{bmatrix} = \begin{bmatrix} 1 & s & s^2 & s^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

(6.80)
现在我们推导一般公式。设 $M_p(i,j)$ 表示 M_p 中第 i 行和第 j 列的元素。从方程 (1.8) 中我们有

$$(1-s)^p = \binom{p}{0} s^0 (1-s)^p = B_{0,p}(s) = [s^i]^T [M_p(i,0)]$$

$$= \begin{bmatrix} 1 & s & \cdots & s^p \end{bmatrix} \begin{bmatrix} M_p(0,0) \\ \vdots \\ M_p(p,0) \end{bmatrix}$$

从初等代数来看，展开 $(1-s)^p$ 产生带有交替符号的二项式系数；因此

$$M_p(i,0) = (-1)^i \binom{p}{i} \quad i = 0, \ldots, p \tag{6.81}$$

一般来说，k 列的 M_p 定义为

$$\binom{p}{k} s^k (1-s)^{p-k} = B_{k,p}(s) = [s^i]^T [M_p(i,k)]$$

检查 $\binom{p}{k} s^k (1-s)^{p-k}$ 显示

- 展开项 $(1-s)^{p-k}$ 产生二项式系数

$$(-1)^j \binom{p-k}{j} \quad j = 0, \ldots, p-k$$

- 乘以 s^k 会导致 $M_p(j,k) = 0$ 对于 $j = 0, \ldots, k-1$ ，并且上面的值向下移动了 k 行；
- k 列的所有元素都乘以 $\binom{p}{k}$ 。

总之，对于 $k = 0, \ldots, p$ ， k 列的 M_p 由以下给出

$$M_p(j,k) = \begin{cases} 0 & j = 0, \ldots, k- \\ (-1)^{j-k} \binom{p}{k} \binom{p-k}{j-k} & j = k, \ldots, p \end{cases} \tag{6.82}$$

我们将其留作练习以证明

$$M_p(j,k) = M_p(p-k, p-j) \tag{6.83}$$

注意矩阵 M_p 的附加属性:

- 对角元素是度数为 p 的二项式系数；
- 最后一行和第一列中的元素也是度数为 p 的二项式系数，但符号交替；
- 每行和每列的元素之和为一。方程 (6.83) 表示 M_p 关于从左下角元素到右上角元素的对角线是对称的。

算法 A6.1 高效地计算 M_p 。一个数组 bin[i][j]，它包含
the precomputed binomial coefficients, is used.

```
ALGORITHM A6.1
BezierToPowerMatrix(p, M)
{ /* Compute pth degree Bézier matrix */
    /* Input: p */
    /* Output: M */
for (i=0; i<p; i++) /* Set upper triangle to zero */
    for (j = i + 1; j <= p; j++) M[i][j] = 0.0 ;
```

```
M [0] [0] = M [p] [p] = 1.0; /* Set corner elements */
if (p mod 2) M[p][0] = -1.0;
    else M[p][0] = 1.0 ;
sign = -1.0 ;
for (i=1; i<p; i++) /* Compute first column, last row,*/
    /* and the diagonal */
    M[i][i] = bin[p][i];
    M [i] [0] = M [p] [p − i] = sign ∗M [i] [i] ;
    sign = -sign;
    }
/* Compute remaining elements */
k1 = (p + 1) /2; pk = p − 1;
for (k=1; k<k1; k++)
    {
    sign = -1.0 ;
    for (j=k+1; j<=pk; j++)
        {
        M [j] [k] = M [pk] [p − j] = sign ∗ bin [p] [k] ∗ bin [p − k] [j − k] ;
        sign = -sign;
        }
    pk = pk − 1 ;
    }
}
```

现在我们进入将 B-spline 曲线转换为分段幂基的步骤 3，即重新参数化方程 (6.78) 以获得

$$\mathbf{C}_j(u) = \left[u^1\right]^T [\mathbf{b}_i] \tag{6.84}$$

原参数中的幂基段 u 。我们将详细讨论 $p = 2$ ，然后给出一般公式但不提供证明。解方程 (6.75) 得到 s 结果为

$$s = \frac{1}{u_{j+1} - u_j} u - \frac{u_j}{u_{j+1} - u_j} = cu + d \tag{6.85}$$

对于 $p = 2$ ，将 $s = cu + d$ 代入方程 (6.78) 并将结果与方程 (6.84) 相等，得到

$$\left[s^1\right]^T [\mathbf{a}_i] = \left[(cu+d)^1\right]^T [\mathbf{a}_i] = \begin{bmatrix} 1 & cu+d & c^2u^2 + 2cdu + d^2 \end{bmatrix}^T [\mathbf{a}_i]$$

$$= \begin{bmatrix} 1 & u & u^2 \end{bmatrix} R [\mathbf{a}_i] = \begin{bmatrix} 1 & u & u^2 \end{bmatrix} [\mathbf{b}_i] \tag{6.86}$$

其中

$$R = \begin{bmatrix} 1 & d & d^2 \\ 0 & c & 2cd \\ 0 & 0 & c^2 \end{bmatrix}$$

从方程 (6.86) 可以得出

$$R [\mathbf{a}_i] = [\mathbf{b}_i] \tag{6.87}$$

$[\mathbf{b}_i]$ 是所需的幂基系数。

一般来说，对于 $s = cu + d$ ，第 p 次重参数化矩阵为

$$R_p = [r_{i,j}]$$

其中

$$r_{i,j} = \begin{cases} 0 & j < i \\ \binom{j}{i} c^i d^{j-i} & i \le j \; i, j = 0, \ldots, p \end{cases} \tag{6.88}$$

结合方程 (6.77) 和 (6.87)，我们得到

$$[\mathbf{b}_i] = R_p M_p [\mathbf{P}_i] \tag{6.89}$$

关于以 Bézier 控制点为基础的幂基系数。图 6.18 显示了 B-spline 到幂基转换的一个示例。一个具有三个段的三次 NURBS 曲线被转换为分段幂基形式。图 6.18 说明了曲线并标记了其第二段。请注意，该段由中间的四个控制点定义。中间曲线段也被显示，以及向量系数 $\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2$ 和 $\mathbf{b}_3$ ，它们经过 1/10 的缩放以适应图形。请注意，向量系数与曲线没有几何关系。多项式片段定义在 $[3/10, 7/10]$ 上。

我们以两个最后的备注完成 B-spline 到幂基转换的讨论:

- IGES 参数样条 (实体 112 和 114)[IGE93] 假设每个段的参数化从零开始；这意味着在方程 (6.85) 中 $d = 0$ ，因此 R_p 简化为具有元素的对角矩阵。

$$r_{i,j} = \begin{cases} 0 & i \neq j \\ c^i & i = j \end{cases} \quad i = 0, \ldots, p \tag{6.90}$$

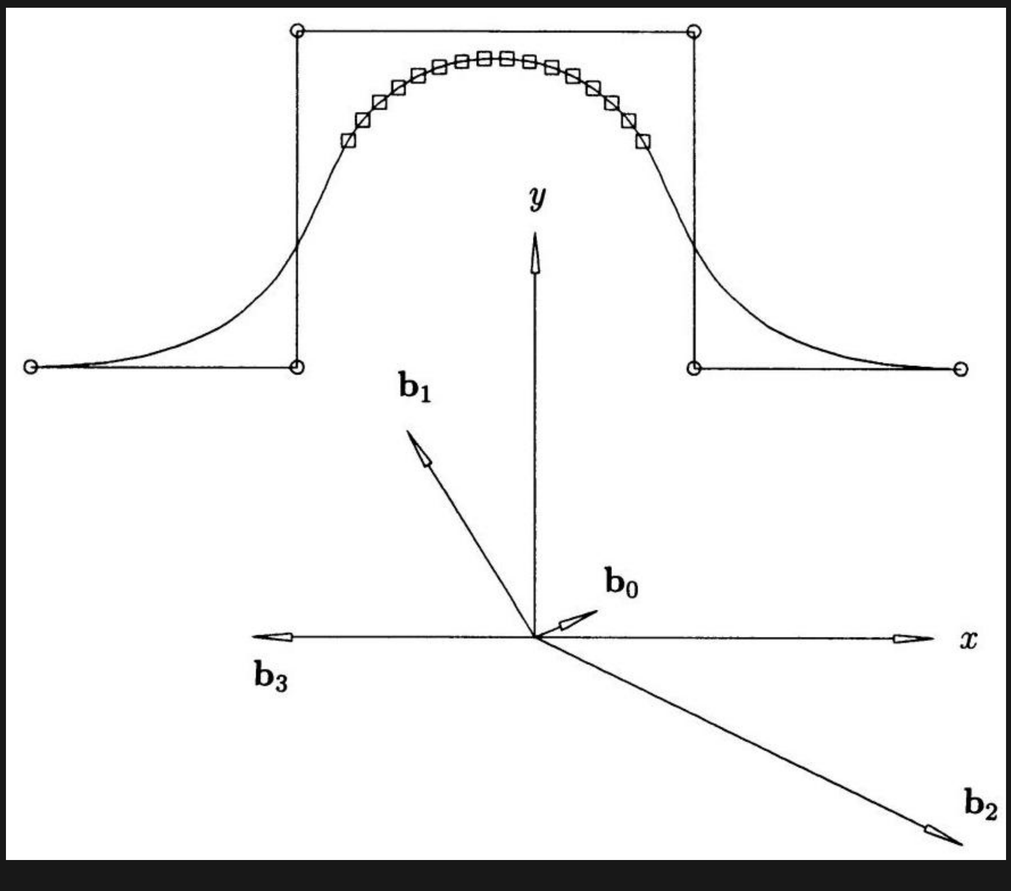


图 6.18. 将 NURBS 曲线转换为以幂基形式表示的分段多项式曲线。

- 对于曲面，方程 (6.89) 变为

$$[\mathbf{b}_{i,j}] = R_p M_p [\mathbf{P}_{i,j}] M_q^T R_q^T \tag{6.91}$$

其中 R_p 和 R_q^T 是重新参数化矩阵，将 u 转换为 s ，将 v 转换为 t ，分别 $(0 \le s, t \le 1)$ 。

图 6.19a 和 6.19 b 显示了 B-spline 曲面到幂基转换。测试曲面是一个定义在的度 (3×2) 曲面。

$$U=\left\{0,0,0,0,\frac{3}{10},\frac{7}{10},1,1,1,1\right\}\ V=\left\{0,0,0,\frac{3}{10},\frac{3}{5},1,1,1\right\}$$

图 6.19a 中显示了三个多项式补丁: 在 $\left[0,\ {}^{3}/_{10}\right]\times\left[{}^{3}/_{5},1\right]$ 上定义的 patch(0,2)；在 $\left[{}^{3}/_{10},\ {}^{7}/_{10}\right]\times\left[{}^{3}/_{10},\ {}^{3}/_{5}\right]$ 上定义的 patch(1,1)；以及在 $[7/10,1]\times[0,3/10]$ 上定义的 patch(2,0)。图 6.19b 展示了一些在幂基形式下定义 patch(1,1) 的向量系数。这些向量经过 1/2 缩放以便更好地可视化。请注意补丁几何形状与向量系数之间缺乏任何几何关系。将此图与第 5 章的图 5.25 进行比较。

现在我们讨论将分段幂基曲线或曲面转换为 B 样条形式的过程。步骤如下:

1. 应用矩阵乘法将幂基段 (补丁) 重新参数化到区间 $[0,1]$ ；

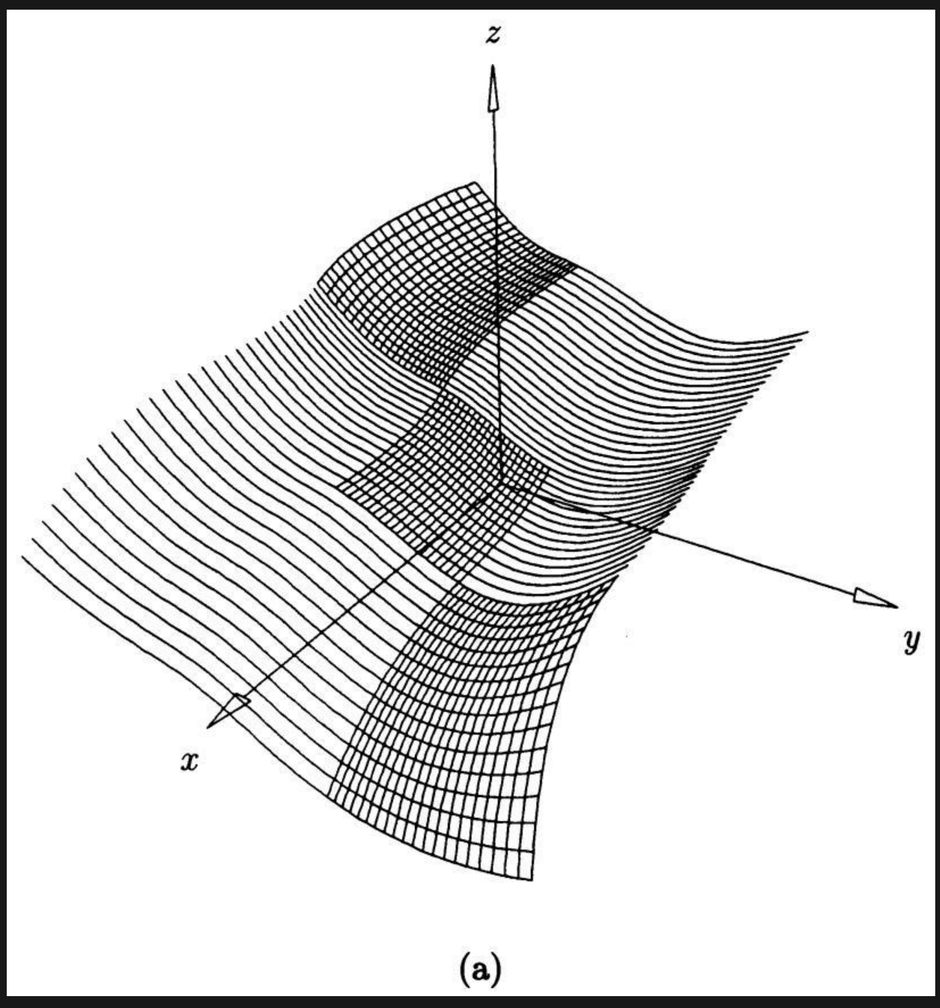


图 6.19. 将 NURBS 曲面转换为以幂基形式表示的多项式补丁集合。(a) 显示了索引为 $(0,2),(1,1)$ 和 $(2,0)$ 的补丁；(b) 定义 patch(1,1) 的一些向量系数。

2. 应用矩阵乘法将每个幂基段 (补丁) 转换为 Bézier 形式；
3. 将各段 (补丁) 拼接在一起形成 B 样条曲线或曲面，所有内部节点的重数等于度数 (如果不能假设 C^0 连续性，则为度数加一)；节点的值由原始幂基段 (补丁) 的参数边界给出；
4. 使用节点移除以获得 B 样条曲线或曲面的最小表示 (曲线的算法为 A5.8，或参见 [Till92] 的曲面算法)。

第 3 步和第 4 步无需进一步阐述。首先考虑曲线。设

$$\mathbf{C}_j(u)=\left[u^i\right]^T\left[\mathbf{a}_i\right] \tag{6.92}$$

是 j 次曲线段的幂基形式，具有 $u\in[u_j,u_{j+1}]$ 。我们需要该段的相应 Bézier 形式

$$\mathbf{C}_j(s)=\left[B_{i,p}(s)\right]^T\left[\mathbf{P}_i\right]=\left[s^i\right]^T M_p\left[\mathbf{P}_i\right]$$

$$=\left[\begin{array}{c}u^i\end{array}\right]^T R_p M_p\left[\mathbf{P}_i\right]=\left[\begin{array}{c}u^i\end{array}\right]^T\left[\mathbf{a}_i\right]=\mathbf{C}_j(u)$$

其中 $s\in[0,1]$ 。因此

$$\left[\mathbf{a}_i\right]=R_p M_p\left[\mathbf{P}_i\right]$$

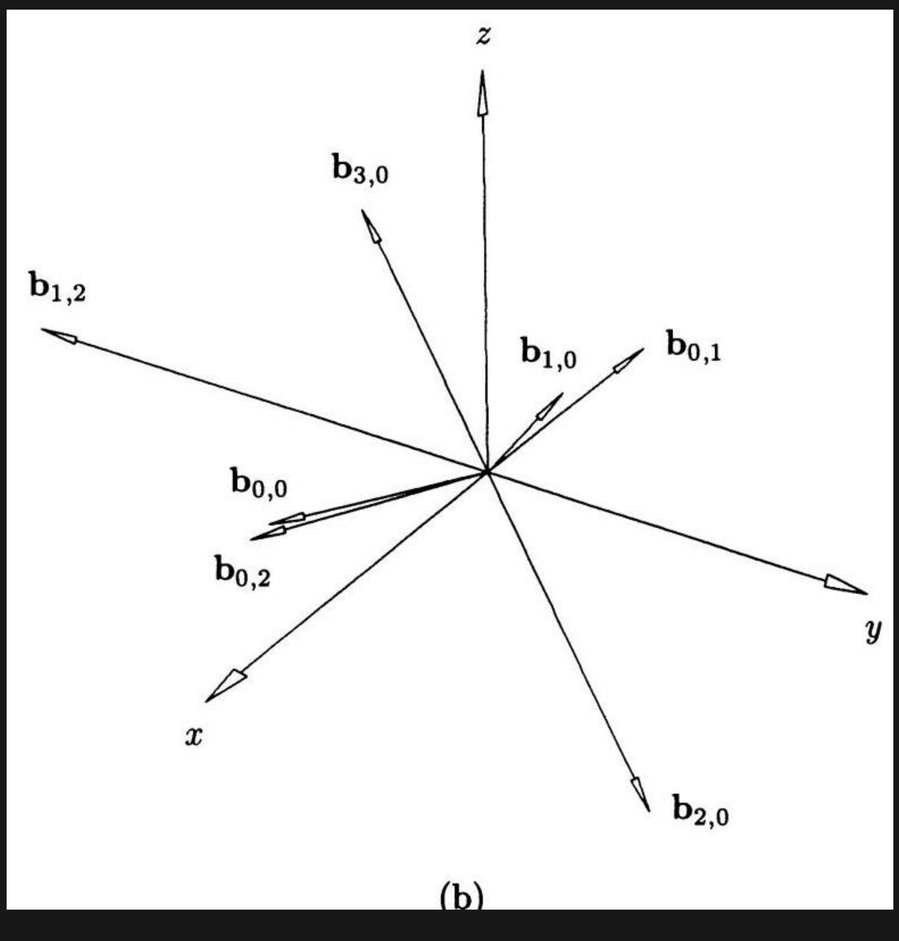


图 6.19. (续)

和

$$[\mathbf{P}_i] = M_p^{-1} R_p^{-1} [\mathbf{a}_i] \tag{6.93}$$

图 6.20a 到 6.20c 显示了幂基到 NURBS 转换的示例。图 6.20a 说明了在 $s_0 < s_1 < s_2 < s_3$ 上定义的分段三次幂基曲线。在图 6.20 b 中，每一段都被转换为 Bézier 形式，即它是具有三重内部节点的 B-spline 曲线。图 6.20c 显示了去除节点后的 B-spline 曲线。注意，原始曲线在 s_1 处是 C^2 连续的，但在 s_2 处仅是 C^1 连续的；因此，s_2 是最终节点向量中的双重节点。很容易反转矩阵 M_1, M_2 和 M_3，如公式 (6.80) 所示，以获得

$$M_1^{-1} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & \frac{1}{2} & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$M_3^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & \frac{1}{3} & 0 & 0 \\ 1 & \frac{2}{3} & \frac{1}{3} & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \tag{6.94}$$

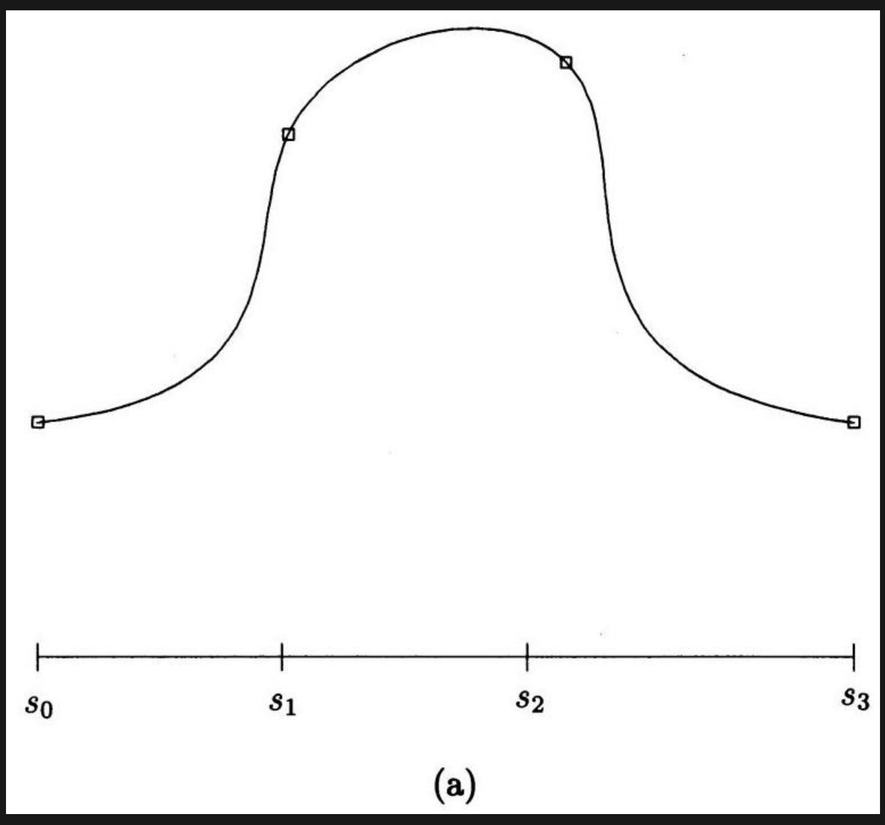


图 6.20. 将分段多项式曲线转换为 NURBS 形式。(a) 在 $s_0 < s_1 < s_2 < s_3$ 上定义的分段多项式；(b) 将每一段转换为 Bézier 形式；(c) 去除节点后的 NURBS 表示。

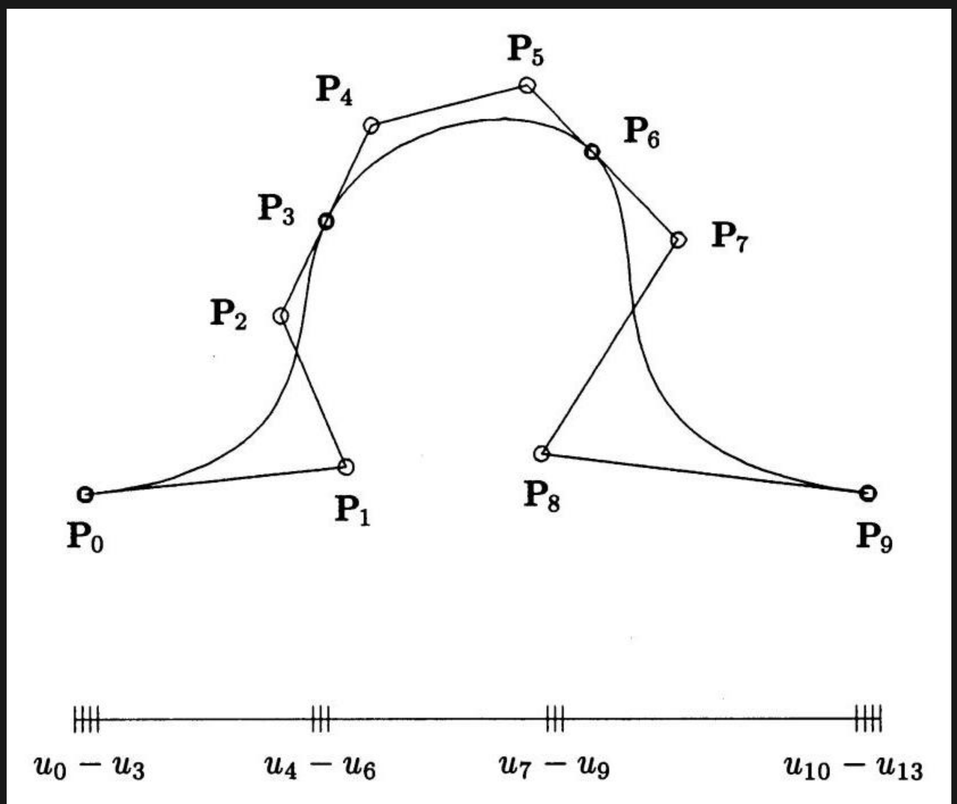
给定 M_p，M_p^{-1} 的 k 列的公式为

$$M_p^{-1}(j,k) = \begin{cases} 0 & j = 0, \ldots, k-1 \\ \frac{1}{M_p(k,k)} & j = k \\ -\sum_{i=k}^{j-1} M_p(j,i)\, M_p^{-1}(i,k) & \\ \frac{M_p(j,j)}{M_p(j,j)} & j = k+1, \ldots, p \end{cases} \tag{6.95}$$

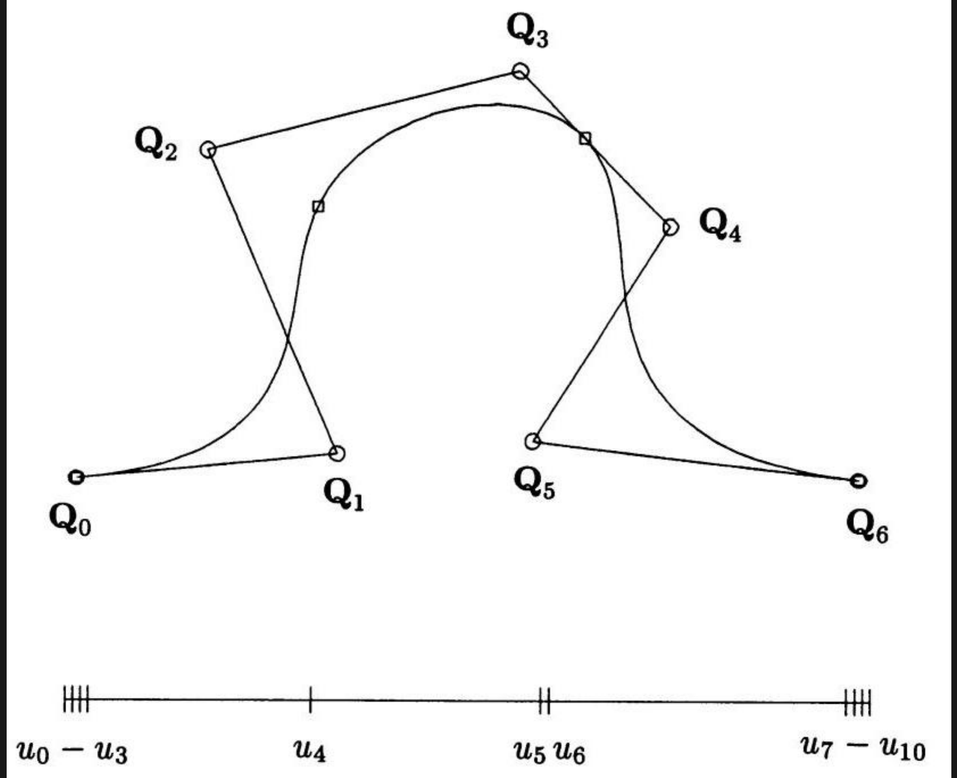
如 M_p, M_p^{-1} 所示，关于从左下角元素到右上角元素的对角线是对称的，因此

$$M_p^{-1}(j,k) = M_p^{-1}(p-k, p-j) \tag{6.96}$$

算法 A6.2 计算 M_p^{-1} (记作 MI)。



(b)



(c)

图 6.20. (续)

算法 A6.2

```
PowerToBezierMatrix(p, M, MI)
{ /* 计算 p 次贝塞尔矩阵的逆 */
/* 输入: p, M */
/* 输出: MI */
for (i=0; i<p; i++) /* Set upper triangle to zero */
    for (j=i+1; j<=p; j++) MI[i][j] = 0.0;
/* Set first col, last row, and diagonal */
for (i = 0; i<=p; i++)
    {
    MI[i][0] = MI[p][i] = 1.0;
    MI[i][i] = 1.0/M[i][i];
    }
/* Compute remaining elements * /
k1 = (p + 1) /2; pk = p − 1;
for (k=1; k<k1; k++)
    {
    for (j=k+1; j<=pk; j++)
        {
        d = 0.0 ;
        for (i=k; i<j; i++) d = d-M[j][i]*MI[i][k];
        MI[j][k] = d/M[j][j];
        MI[pk][p-j] = MI[j][k];
        }
    pk = pk − 1;
    }
}
```

回顾公式 (6.85) 和 (6.86)，我们有 $s = cu + d$ 和

$$\left[s^{\dagger}\right]^{T} = \left[(cu+d)^{\dagger}\right]^{T} = \left[u^{\dagger}\right]^{T} R_p \tag{6.97}$$

反转公式 (6.85) 得到

$$u = \frac{1}{c}s - \frac{d}{c} = (u_{j+1} - u_j)\, s + u_j = \gamma s + \delta$$

从公式 (6.97) 我们得到

$$\left[u^i\right]^T = \left[(\gamma s + \delta)^i\right]^T = \left[s^i\right]^T R_p^{-1}$$

这意味着 R_p^{-1} 只是 R_p，其中 c 被 γ 替换，d 被 δ 替换。总结来说

$$R_p^{-1} = [r_{i,j}]$$

与

$$r_{i,j} = \begin{cases} 0 & j < i \\ \binom{j}{i} \gamma^i \delta^{j-i} & i \le j \; i,j = 0,\ldots,p \end{cases} \tag{6.98}$$

其中

$$\gamma = u_{j+1} - u_j \; \delta = u_j$$

我们总结了两点:

- 对于 IGES 参数样条，$\delta = 0$ 和 R_p^{-1} 简化为对角矩阵 [IGE93]，其元素为

$$r_{i,j} = \begin{cases} 0 & i \neq j \\ \gamma^i & i = j \; i = 0,\ldots,p \end{cases} \tag{6.99}$$

- 对于表面，方程 (6.93) 变为

$$[\mathbf{P}_{i,j}] = M_p^{-1} R_p^{-1} \left[\mathbf{a}_{i,j}\right] \left(R_q^{-1}\right)^T \left(M_q^{-1}\right)^T$$

其中 R_p^{-1} 和 $\left(R_q^{-1}\right)^T$ 是重新参数化矩阵，将 s 转换为 u，将 t 转换为 v，分别 $(0 \le s,t \le 1)$。
图 6.21a – 6.21c 说明了从幂基到 NURBS 表面的转换。测试表面是九个度 (3×2) 多项式片段的集合，如图 6.21a 所示。图 6.21b 说明了分段 Bézier 表示。去结点后产生的表面如图 6.21c 所示。

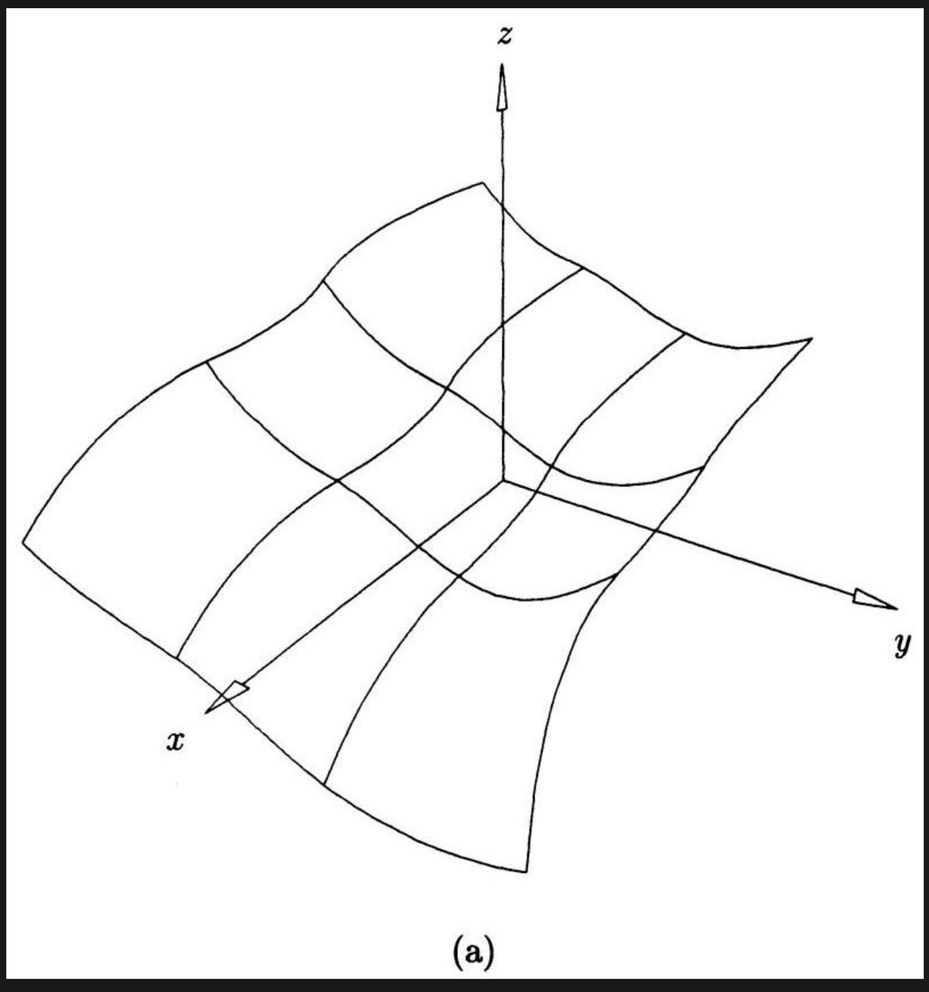


图 6.21. 将分段多项式表面转换为 NURBS 形式。(a) 分段多项式表面；(b) 将每个片段转换为 Bézier 形式；(c) 去结点后的 NURBS 表示。

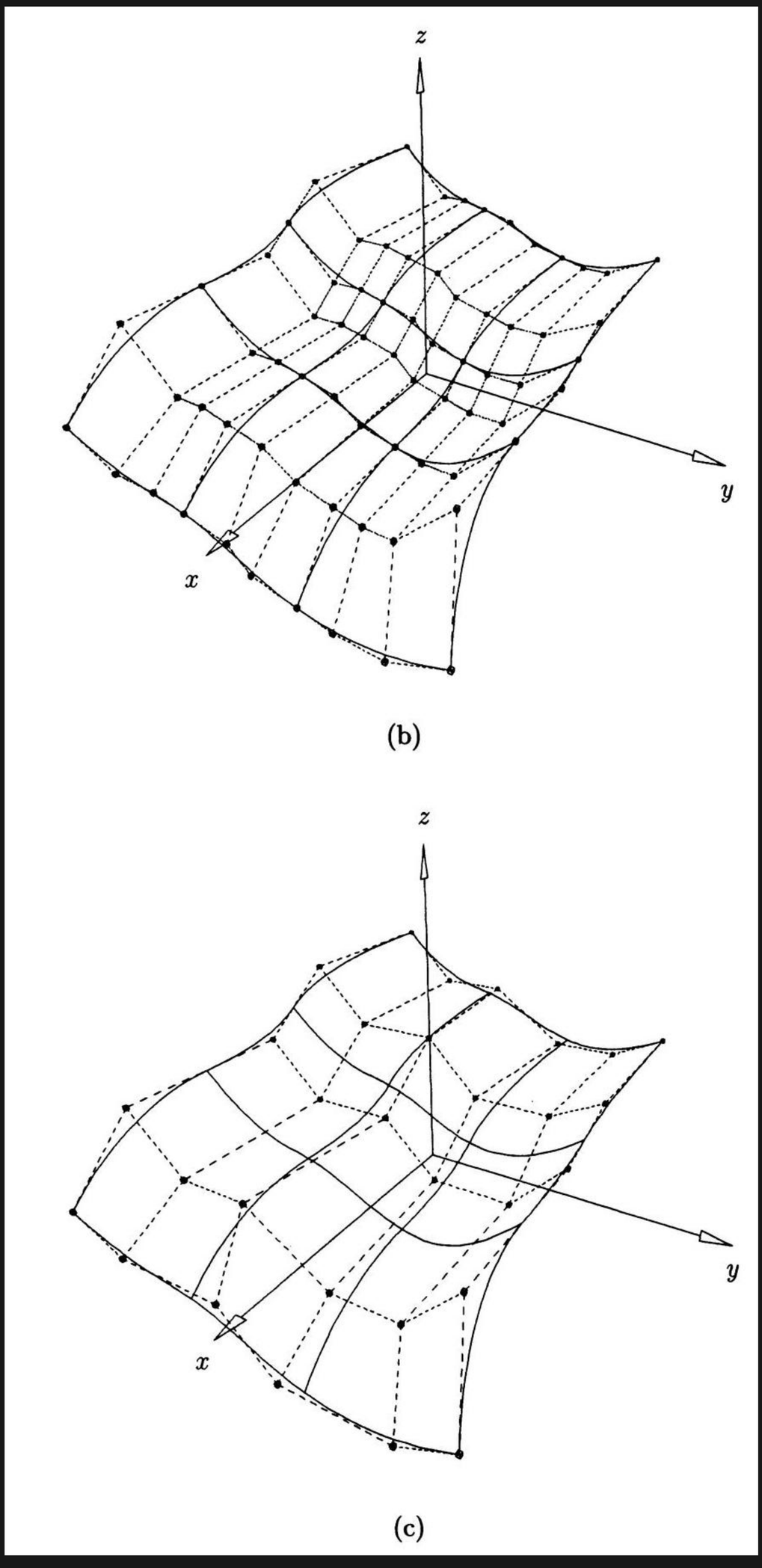


图 6.21.(续)

练习

6.1. 考虑示例 Ex6.3 的透视投影。$\overline{\mathbf{P}}_i^w$ 的欧几里得坐标为

$$\overline{\mathbf{P}}_i = \left(\frac{dx_i}{d - z_i}, \frac{dy_i}{d - z_i}, 0 \right)$$

使用方程 (6.17) 和 (6.18) 进行投影；比较结果。

6.2. 使用二次函数 $u = f(s) = as^2 + bs + c$ 重新参数化由 $\{\mathbf{P}_i\} = \{(1,0), (1,1), (0,1)\}$ 和 $\{w_i\} = \{1,1,2\}$ 给出的 Bézier 圆弧 $\mathbf{C}(u)$，使得

$$\mathbf{P} = \mathbf{C}\left(s = \frac{1}{2}\right) = \left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2} \right)$$

需要三个步骤:

- 确定 $\mathbf{P}$ 达到的 u 值。提示: 使用公式 (6.28)；
- 计算 $f(s)$ 的系数 a, b, c；
- 使用公式 (6.38) 确定新的齐次控制点。

末端权重是否发生了变化？

6.3. 在 $U=\{0,0,0,0,1,1,3,5,7,7,7,7\}$ 上，使用非有理二次 NURBS 函数 $f(s)$ 对立方曲线 $\mathbf{C}^w(u)$ 进行重新参数化 $S=\{0,0,0,2,3,6,6,7,7,7\}$ 。重新参数化曲线 $\mathbf{C}^w(s)$ 的节点向量将是什么样的？在各个节点处 $\mathbf{C}^w(s)$ 的连续性如何？

6.4. 检查公式 (6.34) 与公式 (6.55)-(6.58) 之间的关系，即:

- 使用公式 (6.34)，公式 (6.56) 和公式 (6.57) 是否得出公式 (6.55)；
- 公式 (6.58) 是否产生公式 (6.34)？

请记住，所有权重都可以乘以一个共同因子。

6.5. 推导方程 (6.83)。

6.6. 设立方曲线 $\mathbf{C}(u)$ 由 $U=\{0,0,0,0,1,2,2,2,2\}$ 定义，控制点为 $\{\mathbf{P}_i\}=\{(0,0),(1,1),(3,2),(5,1),(4,-1)\}$ 。使用节点插入 (手动) 将曲线分解为两个贝塞尔段，然后将这两个段转换为幂基形式。通过从幂基段和原始 B-样条曲线计算一些端点和导数值来检查结果。

6.7. 设 $s=cu+d$ 。形成 R_3 和 R_3^{-1} ，并验证 $R_3R_3^{-1}=I$(I 是单位矩阵。

第七章　圆锥曲线和圆

1　引言

圆锥曲线和圆在 CADCAM 应用中发挥着基础性作用。NURBS 的一个最大优势无疑是它们能够精确表示圆锥曲线和圆，以及自由形状的曲线和曲面。我们假设读者具备圆锥曲线和圆的知识；本章的目的是在 NURBS 曲线的表示框架内研究它们。在第 7.2 节中，我们回顾了后续章节所需的圆锥曲线的各种形式和性质。第 7.3 节涵盖了圆锥和圆弧的二次有理贝塞尔表示；第 7.4 节介绍了无限控制点。在第 7.5 和 7.6 节中，我们分别提出了构建任意圆和圆锥的 NURBS 表示的算法，包括完整的圆和椭圆。第 7.7 节涵盖了各种表示形式之间的转换，第 7.8 节给出了高阶圆表示的示例。

2　表示圆锥曲线的各种形式

有许多方法可以定义和表示圆锥曲线。我们首先给出一个几何定义，然后利用它推导出 xy 平面中圆锥曲线的一般隐式 (代数) 方程 (见 [Lawr72])。圆锥曲线是一个点的轨迹，该点移动的方式使得它与固定点 (焦点) 的距离与它到固定直线 (准线) 的距离成比例，即 (见图 7.1)

$$\text{Conic} = \left\{ \mathbf{P} \;\middle|\; \frac{\mathbf{PF}}{\mathbf{PD}} = e \right\}$$

其中 e 是一个称为离心率的比例常数。离心率

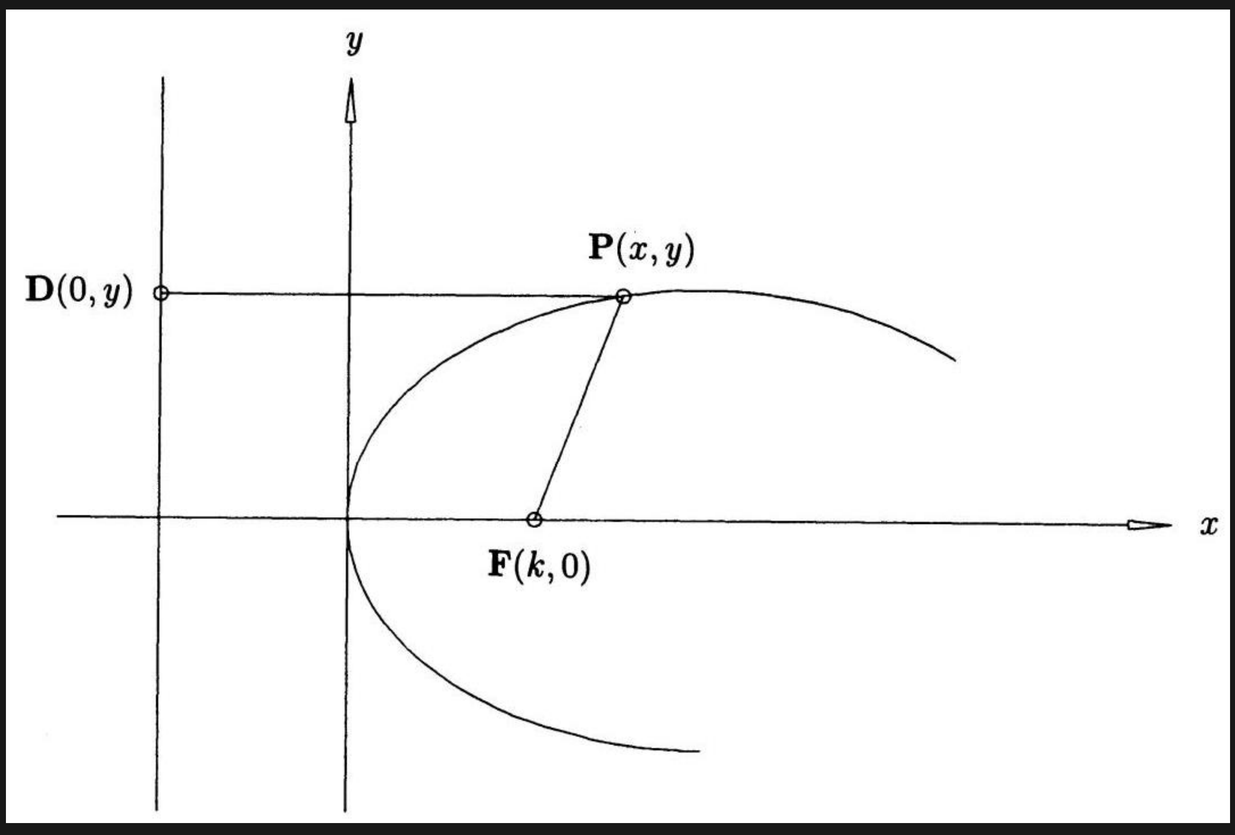


图 7.1. 通过焦点和准线定义圆锥曲线。

决定了圆锥曲线的类型，即

$$e = \begin{cases} = 1 & \text{parabola} \\ < 1 & \text{ellipse} \\ > 1 & \text{hyperbola} \end{cases}$$

如果选择准线为直线 $x = 0$ ，焦点为 $\mathbf{F} = (k, 0)$ ，那么

$$e = \frac{\sqrt{(x-k)^2 + y^2}}{|x|} \tag{7.1}$$

其中 $|x| = \mathbf{PD}$ 和 $\mathbf{D} = (0, y)$ 。对方程 (7.1) 进行平方和重新排列得到

$$\left(1 - e^2\right) x^2 - 2kx + y^2 + k^2 = 0 \tag{7.2}$$

即，位于 xy 平面的圆锥曲线由一个二次代数方程表示。反之，可以证明任何二次代数方程表示一个圆锥曲线。设这个方程写为

$$ax^2 + by^2 + 2hxy + 2fx + 2gy + c = 0 \tag{7.3}$$

设 $\alpha = ab - h^2$ ，并且设

$$D = \begin{vmatrix} a & h & f \\ h & b & g \\ f & g & c \end{vmatrix}$$

为由方程 (7.3) 的系数形成的 3×3 矩阵的行列式。使用 α 和 D ，可以对圆锥曲线进行完整分类

$$\alpha = 0 \; D \neq 0$$

抛物线
$D = 0 \; b \neq 0 \; g^2 - bc > 0$ 2 条平行的实线
$g^2 - bc = 0$ 2 平行重合线
$g^2 - bc < 0$ 2 平行虚线

$$b = h = 0 \; f^2 - ac > 0 \; 2 \text{ parallel real lines}$$

$f^2 - ac = 0$ 2 平行重合线
$f^2 - ac < 0$ 2 平行虚线

$$\alpha > 0 \; D = 0$$

点椭圆

$$D \neq 0$$

$-bD > 0$ 实椭圆
$-bD < 0$ 虚椭圆
$\alpha < 0 \; D \neq 0$ 双曲线

$$D = 0$$

2 条相交线
通过简单的变换，方程 (7.2) 可以重新表述为标准形式之一。对于 $e \neq 1$ 我们使用变换

$$x' = x - \frac{k}{1 - e^2} \; y' = y$$

代入方程 (7.2) 并去掉撇号得到

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1 \tag{7.4}$$

在这里

$$a = \frac{k\epsilon}{1-e^2} \; b^2 = a^2\left(1-e^2\right)$$

有两种情况:

- 椭圆 (见图 7.2): $e<1$ 意味着 $a, b>0$ ，并且方程 (7.4) 变为

$$\frac{x^2}{a^2}+\frac{y^2}{b^2}=1 \; a>b>0 \tag{7.5}$$

在这个位置，原点是椭圆的中心。x 轴是 ma 主轴，而 y 轴是次轴。点 $(-a,0),(a,0),(0,-b)$ 和 (0, b) 是顶点。距离 a 和 b 分别是主半径和次半径。如果 $a=b$ 方程 (7.5) 表示一个圆 (这是极限情况)，$e \to 0, k \to \infty$) 。

- 双曲线 (图 7.3): $e>1$ 意味着 $a<0$ 和 $b^2<0$ (b 是虚数)。设定 $b=\mid b \mid$ ，方程 (7.4) 得到

$$\frac{x^2}{a^2}-\frac{y^2}{b^2}=1 \; a, b>0 \tag{7.6}$$

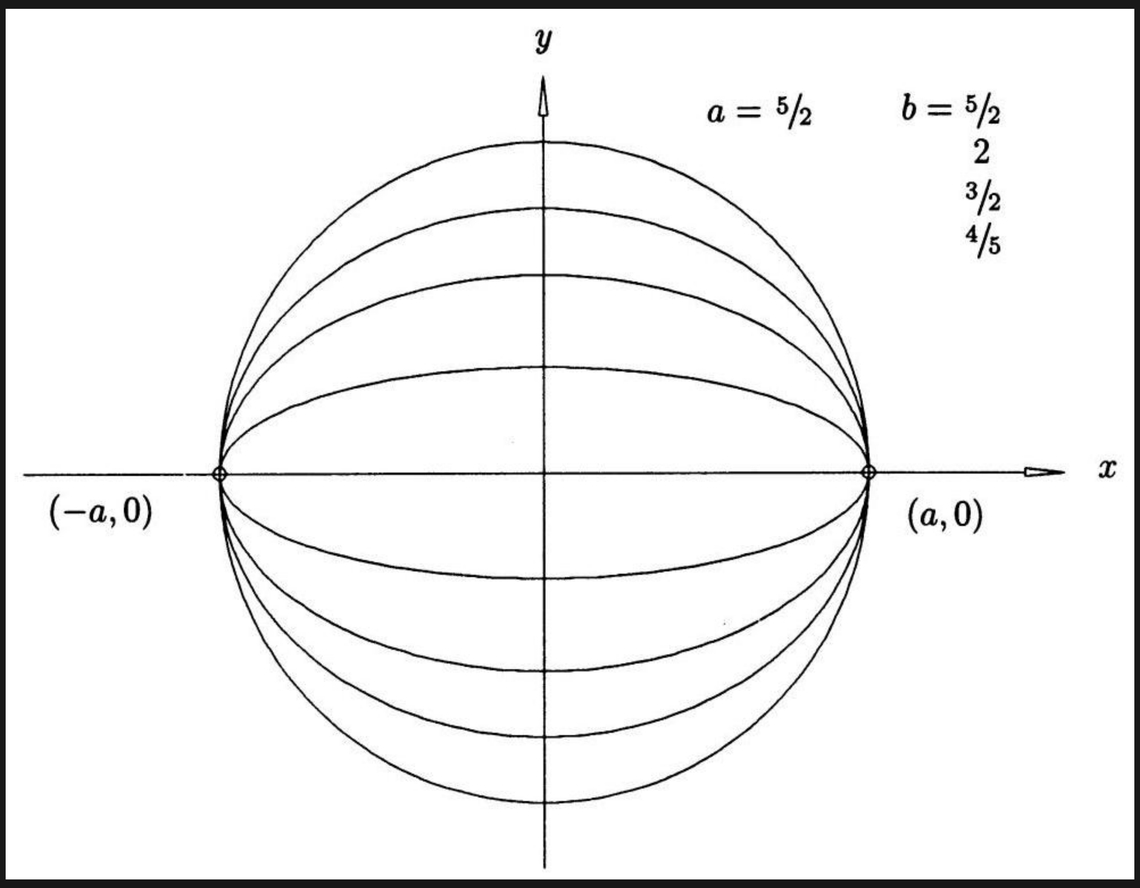


图 7.2. 由不同参数定义的椭圆 ($a=5/2, b=\{5/2,2,3/2,4/5\}$) 。

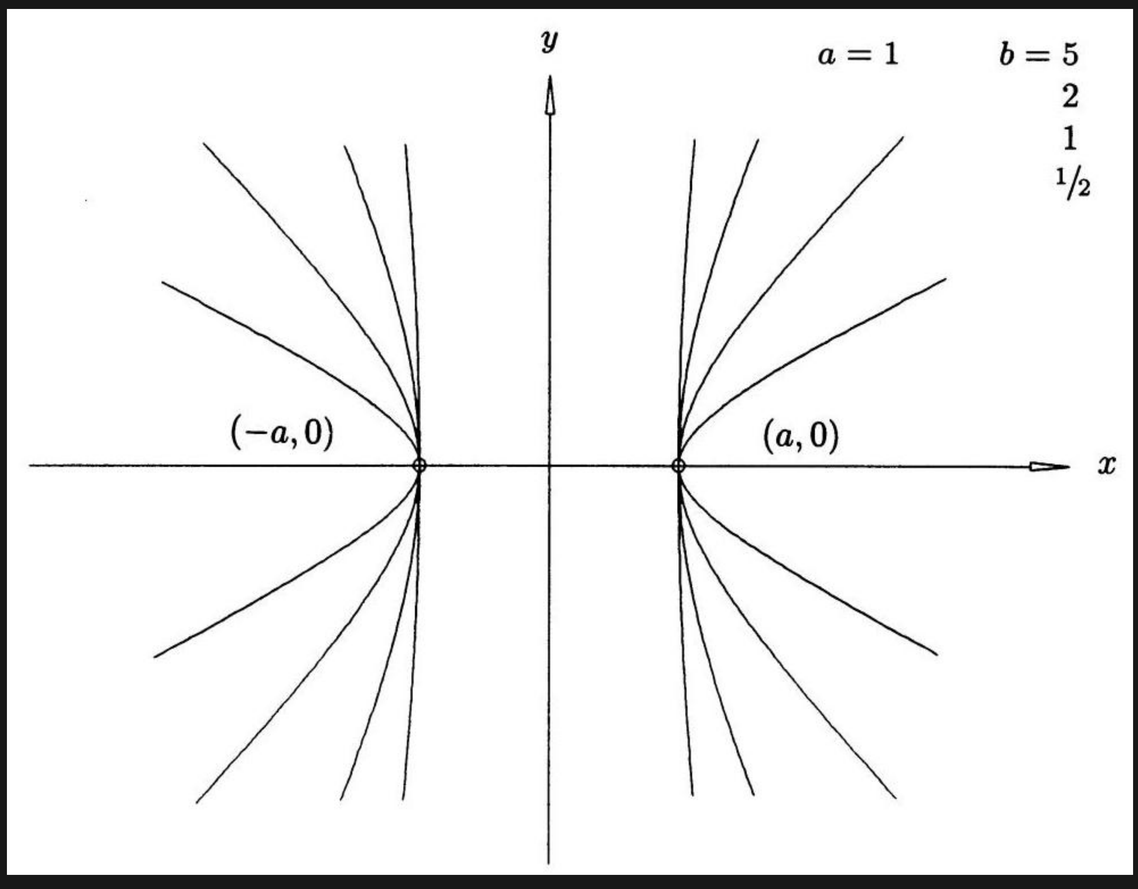


图 7.3. 由不同参数定义的双曲线 ($a=1, b=\{5,2,1,1/2\}$) 。

在这个位置，原点是双曲线的中心。x 轴是横轴，而 y 轴称为半共轭轴或虚轴。点 (-a,0) 和 (a,0) 是顶点。距离 a 和 b 分别称为主半径和次 (或虚) 半径。请注意，双曲线有两个分支，由虚轴分开。

类似地，$e=1$ 和变换

$$x' = x - \frac{1}{2}k \; y' = y$$

得到抛物线

$$y^2 = 4ax \; a = \frac{1}{2}k > 0 \tag{7.7}$$

焦点为 $\mathbf{F}=(a,0)$ ，准线为 $x+a=0$ (见图 7.4)。抛物线没有中心。在标准位置下，其轴是 x 轴，原点是其顶点，而 a 是其焦距。

两种圆锥曲线的参数化表示在 CADCAM 应用中非常重要: 有理形式和最大内接面积形式。我们首先讨论有理形式。方程

$$x(u) = a\frac{1-u^2}{1+u^2}$$

$$y(u) = b\frac{2u}{1+u^2} \; -\infty < u < \infty \tag{7.8}$$

表示标准位置下的椭圆 (图 7.5 显示了线段上的点 $0 \le u \le 1$)。通过将方程 (7.8) 代入方程 (7.5)，可以看出这一点。

$$\frac{a^2\left(\frac{1-u^2}{1+u^2}\right)^2}{a^2}+\frac{b^2\left(\frac{2u}{1+u^2}\right)^2}{b^2}=\frac{1-2u^2+u^4+4u^2}{1+2u^2+u^4}=1$$

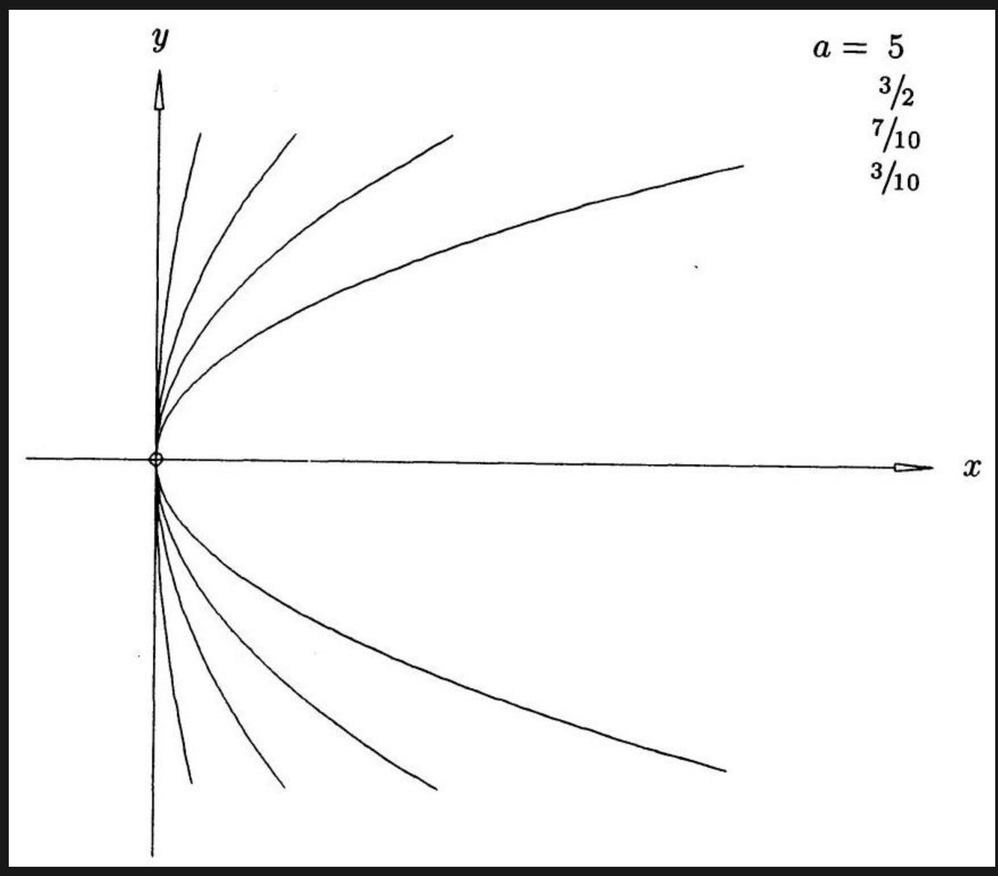


图 7.4. 由不同参数定义的抛物线 ($a=\{5,3/2,7/10,3/10\}$) 。

注意 $(x(0),y(0))=(a,0),(x(1),y(1))=(0,b)$ ，并且顶点 (-a,0) 在极限情况下接近 $u\to-\infty$ 或 $u\to\infty$ 。这些方程

$$x(u)=a\frac{1+u^2}{1-u^2}$$

$$y(u)=b\frac{2u}{1-u^2}\ -\infty<u<\infty \tag{7.9}$$

表示标准位置下的双曲线 (图 7.6)。区间 $u\in(-1,1)$ 对应于右支；左支由 $u\in(-\infty,-1)$ 和 $u\in(1,\infty)$.$\mathbf{C}(-1)$ 描述，而 $\mathbf{C}(1)$ 表示无穷远点。最后

$$x(u)=au^2$$

$$y(u)=2au\ -\infty<u<\infty \tag{7.10}$$

参数化标准位置下的抛物线 (见图 7.7)。设 $\mathbf{C}(u)=(x(u),y(u),0)$ 为嵌入在三维欧几里得空间中的标准位置下的圆锥曲线。通过应用任意的 3×3 旋转矩阵和任意的

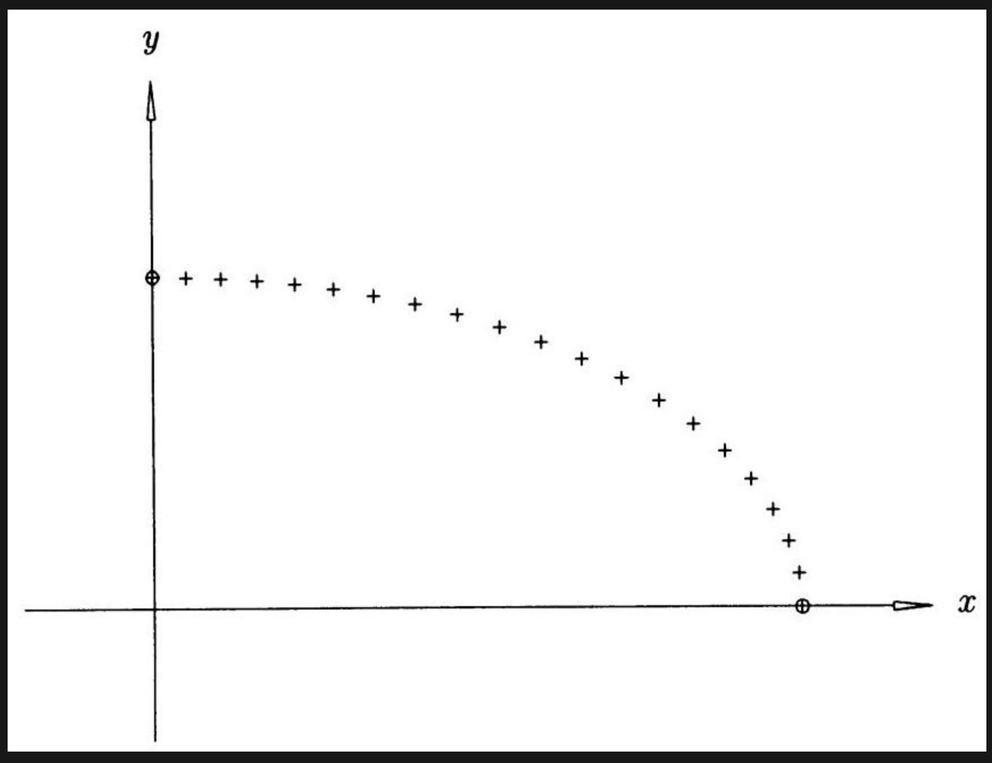


图 7.5. 椭圆弧的有理参数化 (方程 [7.8])。

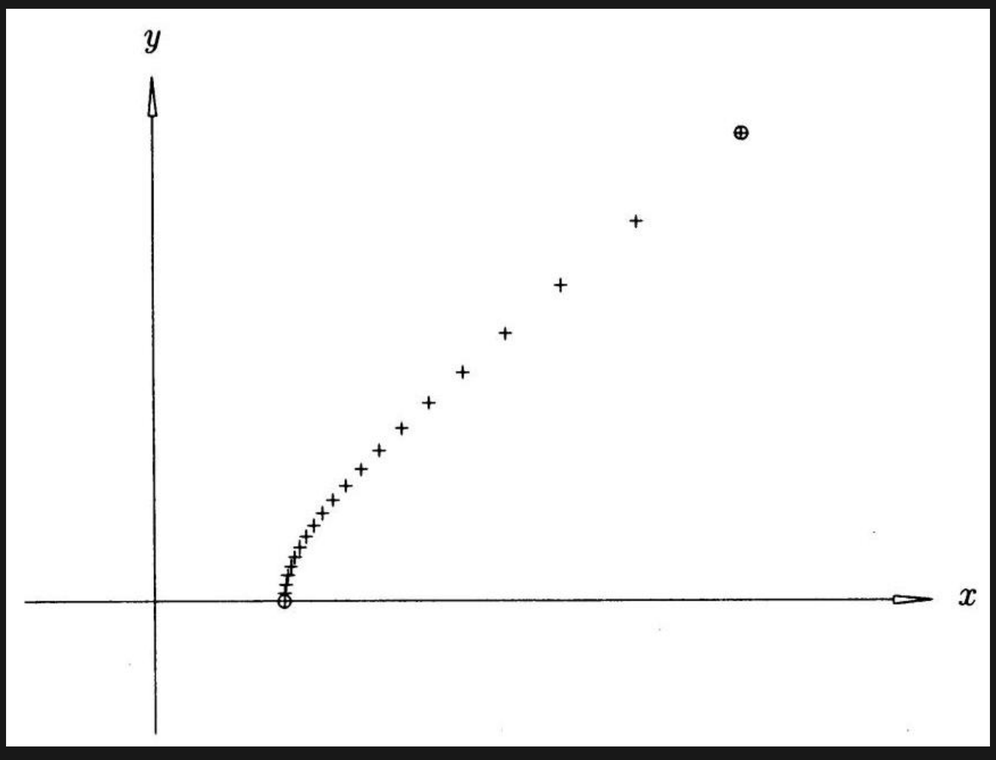


图 7.6. 双曲弧的有理参数化 (方程 [7.9])。

变换向量，很容易看出三维空间中的一般圆锥曲线

具有以下形式

$$x(u)=\frac{a_0+a_1u+a_2u^2}{w_0+w_1u+w_2u^2}=\frac{\sum_{i=0}^{2}a_iu^i}{\sum_{i=0}^{2}w_iu^i}$$

$$y(u)=\frac{b_0+b_1u+b_2u^2}{w_0+w_1u+w_2u^2}=\frac{\sum_{i=0}^{2}b_iu^i}{\sum_{i=0}^{2}w_iu^i}$$

$$z(u)=\frac{c_0+c_1u+c_2u^2}{w_0+w_1u+w_2u^2}=\frac{\sum_{i=0}^{2}c_iu^i}{\sum_{i=0}^{2}w_iu^i}\quad -\infty<u<\infty \tag{7.11}$$

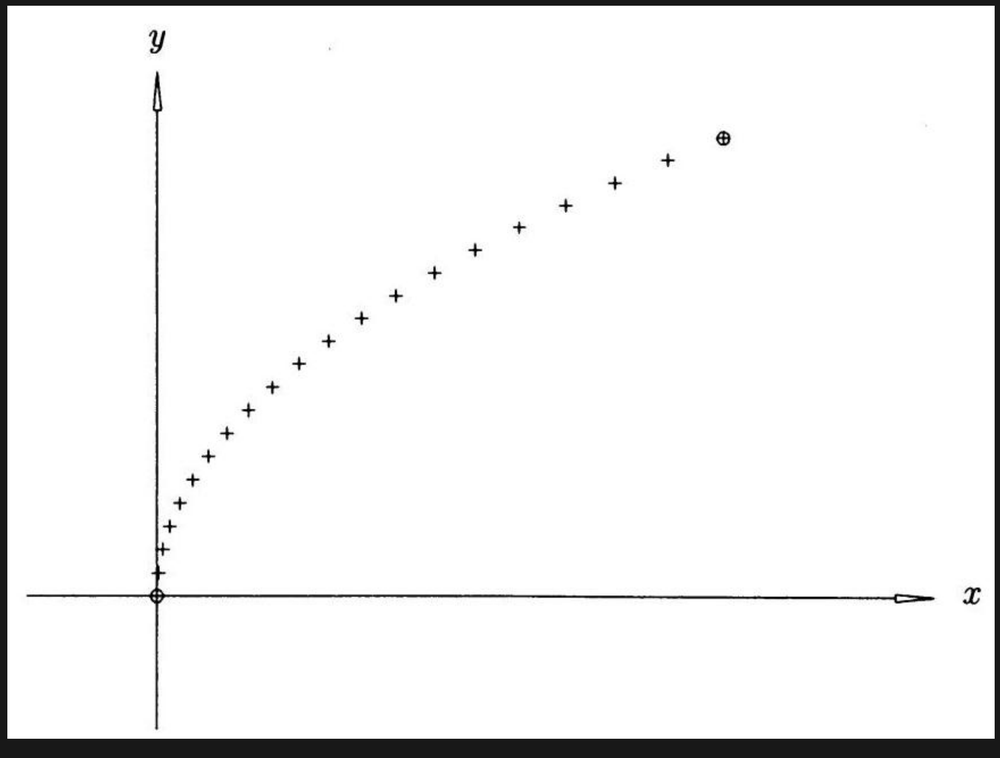


图 7.7. 抛物线弧的参数化 (方程 [7.10])。

设置

$$x_i=\frac{a_i}{w_i}$$

$$y_i=\frac{b_i}{w_i}$$

$$z_i=\frac{c_i}{w_i}$$

和

$$\mathbf{a}_i=(x_i,y_i,z_i)$$

我们得到

$$\mathbf{C}(u)=\frac{w_0\mathbf{a}_0+w_1\mathbf{a}_1u+w_2\mathbf{a}_2u^2}{w_0+w_1u+w_2u^2}=\frac{\sum_{i=0}^{2}w_i\mathbf{a}_iu^i}{\sum_{i=0}^{2}w_iu^i} \tag{7.12}$$

圆锥的有理幂基形式。设置

$$\mathbf{a}_i^w=(w_ix_i,w_iy_i,w_iz_i,w_i) \tag{7.13}$$

产生

$$\mathbf{C}^w(u)=\sum_{i=0}^{2}\mathbf{a}_i^wu^i \tag{7.14}$$

有理幂基圆锥的齐次形式。此外，任何形式为方程 (7.11) 的方程都是一个圆锥。这一事实将从第 7.3 节和第 7.7 节得出。

有理形式可能表示圆锥的较差参数化，因为均匀间隔的参数值可以映射到曲线上的非常不均匀间隔的点。图 7.5 到 7.7 显示了由方程 (7.8)-(7.10) 给出的圆锥的部分上的点。这些点是均匀间隔的 u 值的图像。从第 6 章我们知道，有理曲线的参数化可以通过线性有理函数的重新参数化进行更改 (并可能得到改善)。

现在假设 $\mathbf{C}(u) = (x(u), y(u))$ 是标准位置下圆锥的参数表示。对于每种类型的圆锥，我们现在给出函数 $x(u), y(u)$，它们在以下意义上提供了良好的参数化：如果对于任意整数 n 和参数范围 a 和 b，我们计算 n 个均匀间隔的参数值

$$a = u_1, \ldots, u_n = b \; u_{i+1} - u_i = \text{ constant}, \; i = 1, \ldots, n-1$$

那么点序列 $\mathbf{C}(u_1), \ldots, \mathbf{C}(u_n)$ 形成在 $\mathbf{C}(u)$ 上的 (n - 1) 边形，其闭合具有最大内切面积。椭圆由

$$x(u) = a \cos u$$

$$y(u) = b \sin u \; 0 \leq u \leq 2\pi \tag{7.15}$$

(见图 7.8)。如果 $a = b$ 方程 (7.15) 表示一个圆，并且参数化是均匀的。双曲线使用双曲函数

$$\sinh u = \frac{e^u - e^{-u}}{2}$$

$$\cosh u = \frac{e^u + e^{-u}}{2}$$

其方程为 (图 7.9)

$$x(u) = a \cosh u$$

$$y(u) = b \sinh u \; -\infty < u < \infty \tag{7.16}$$

方程 (7.16) 描绘了右支；左支由以下方程描绘

$$x(u) = -a \cosh u$$

$$y(u) = -b \sinh u \; -\infty < u < \infty$$

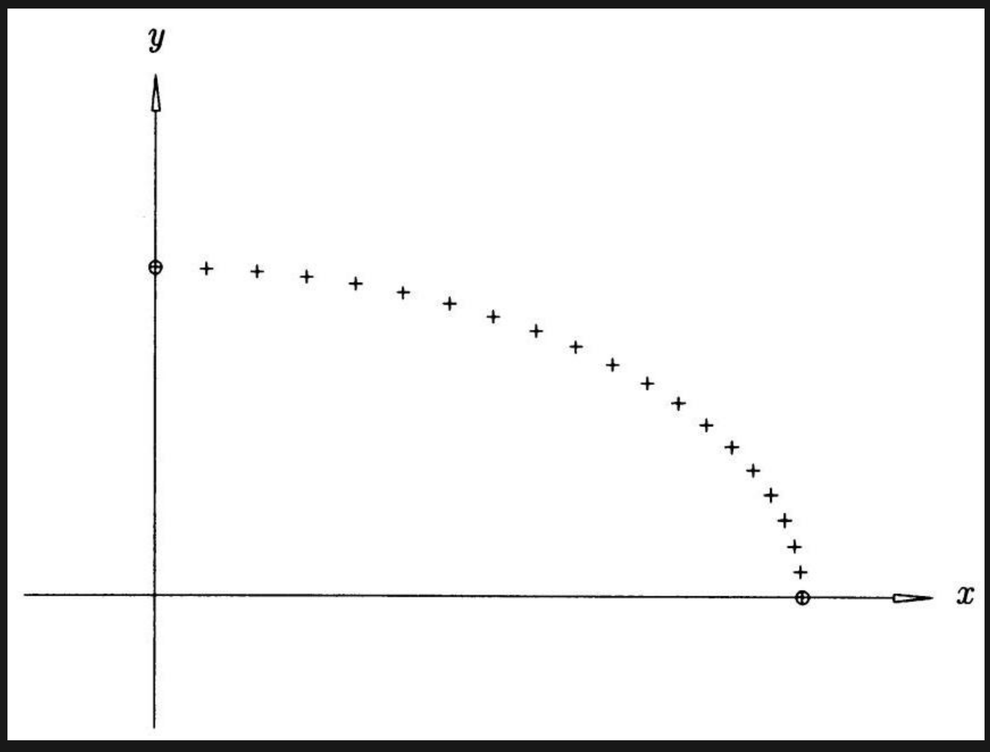


图 7.8. 椭圆的最大面积参数化 (方程 [7.15])。

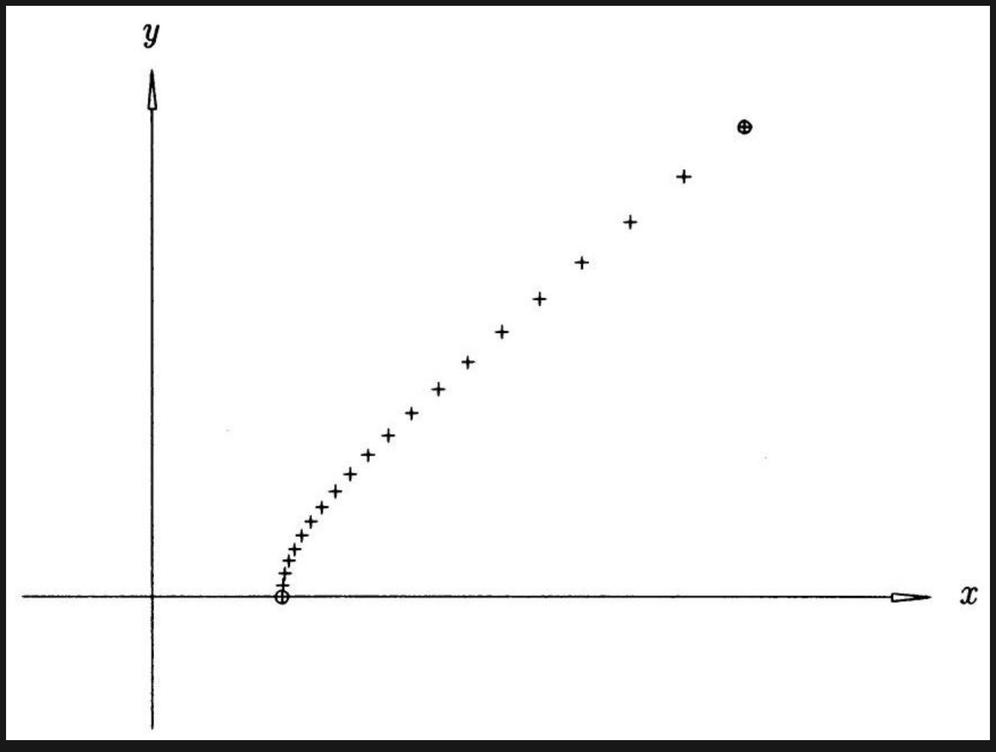


图 7.9. 双曲线的最大面积参数化 (方程 [7.16])。

参数化抛物线由方程给出 (图 7.7)

$$x(u) = au^2$$

$$y(u) = 2au \; -\infty < u < \infty \tag{7.17}$$

这与方程 (7.10) 相同。

方程 (7.15)-(7.17) 可以扩展以表示三维空间中的圆锥曲线。设 $\{\mathbf{O}, \mathbf{X}, \mathbf{Y}\}$ 为三维空间中圆锥曲线的局部坐标系（$\mathbf{O}$ 是一个点，$\mathbf{X}$ 和 $\mathbf{Y}$ 是正交单位长度向量）。圆锥曲线定义为 (图 7.10)

- 椭圆：以 $\mathbf{O}$ 为中心，$\mathbf{X}$ 为长轴，$\mathbf{Y}$ 为短轴，椭圆的方程为

$$\mathbf{C}(u) = \mathbf{O} + a \cos u \mathbf{X} + b \sin u \mathbf{Y} \tag{7.18}$$

- 双曲线：以 $\mathbf{O}$ 为中心，$\mathbf{X}$ 为横轴，$\mathbf{Y}$ 为虚轴，双曲线的方程为

$$\mathbf{C}(u)=\mathbf{O}+a\cosh u\mathbf{X}+b\sinh u\mathbf{Y} \tag{7.19}$$

- 抛物线: 以 $\mathbf{O}$ 为顶点，$\mathbf{X}$ 为轴，$\mathbf{Y}$ 给出了抛物线在其顶点的切线方向。抛物线的方程为

$$\mathbf{C}(u)=\mathbf{O}+au^2\mathbf{X}+2au\mathbf{Y} \tag{7.20}$$

方程 (7.18)-(7.20) 是新标准《产品模型数据交换标准》(STEP)[STEP94] 中规定的圆锥曲线形式。

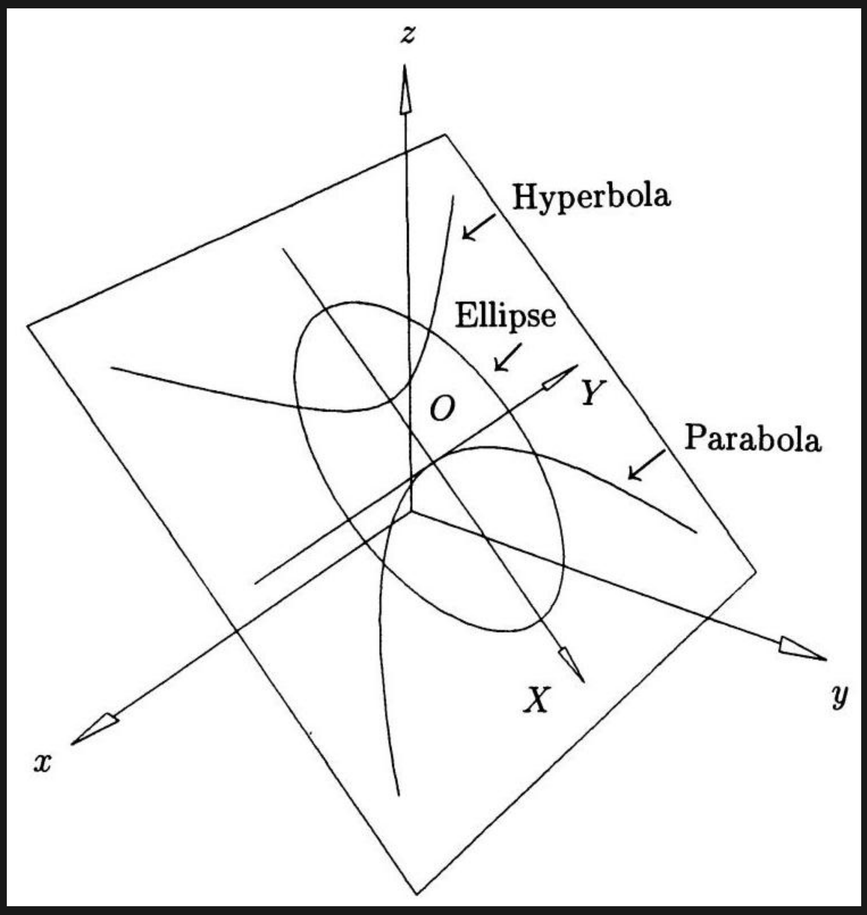


图 7.10. 三维空间中的椭圆、抛物线和双曲线。

几乎所有关于解析几何的书籍都详细介绍了圆锥曲线的隐式方程形式，包括圆锥类型的分类、退化情况以及标准位置的变换 (方程 [7.5]-[7.7])，例如参见 [Salm79; Coxe80; Ilyi84]。一些现代 CADCAM 几何书籍对圆锥曲线进行了详细讨论，特别是有理和最大内接面积的参数形式，例如参见 Rogers [Roge90] 和 Beach [Beac91]。方程 (7.15)-(7.17) 的最大内接面积性质最早由 [Smit71] 提出。Liming 的两本书 ([Limi44, 79]) 包含了关于圆锥构造和圆锥在工程设计中应用的大量信息。

3　二次有理贝塞尔弧

二次有理贝塞尔弧的形式为

$$\mathbf{C}(u)=\frac{(1-u)^2w_0\mathbf{P}_0+2u(1-u)w_1\mathbf{P}_1+u^2w_2\mathbf{P}_2}{(1-u)^2w_0+2u(1-u)w_1+u^2w_2}$$

$$=R_{0,2}(u)\mathbf{P}_0+R_{1,2}(u)\mathbf{P}_1+R_{2,2}(u)\mathbf{P}_2 \tag{7.21}$$

其中

$$R_{i,2}(u)=\frac{B_{i,2}(u)w_i}{\sum_{j=0}^{2}B_{j,2}(u)w_j}\quad u\in[0,1]$$

为了证明方程 (7.21) 是一个圆锥曲线，引入一个局部的斜坐标系

$$\{\mathbf{P}_1,\mathbf{S},\mathbf{T}\} \tag{7.22}$$

其中

$$\mathbf{S}=\mathbf{P}_0-\mathbf{P}_1\ \mathbf{T}=\mathbf{P}_2-\mathbf{P}_1$$

(见图 7.11)。对于任意的 $u\in[0,1]$，点 $\mathbf{C}(u)$ 位于三角形 $\mathbf{P}_0\mathbf{P}_1\mathbf{P}_2$ 内，因此可以写为

$$\mathbf{C}(u)=\mathbf{P}_1+\alpha(u)\mathbf{S}+\beta(u)\mathbf{T}=\alpha(u)\mathbf{P}_0+(1-\alpha(u)-\beta(u))\mathbf{P}_1+\beta(u)\mathbf{P}_2 \tag{7.23}$$

将此与方程 (7.21) 进行比较，我们发现

$$\alpha(u)=R_{0,2}(u)\ \ \beta(u)=R_{2,2}(u) \tag{7.24}$$

使用恒等式

$$B_{0,2}(u)B_{2,2}(u)=\left(\frac{B_{1,2}(u)}{2}\right)^2$$

和方程 (7.23)，并将方程 (7.21) 中的分母函数记为 $w(u)$，我们得到

$$\alpha(u)\beta(u)=R_{0,2}(u)R_{2,2}(u)=\frac{w_0w_2B_{0,2}(u)B_{2,2}(u)}{(w(u))^2}$$

$$=w_0w_2\frac{(B_{1,2}(u))^2}{4(w(u))^2}=\frac{w_0w_2}{w_1^2}\frac{(w_1B_{1,2}(u))^2}{4(w(u))^2}$$

$$=\frac{w_0w_2}{w_1^2}\frac{1}{4}(R_{1,2}(u))^2$$

设置

$$k=\frac{w_0w_2}{w_1^2} \tag{7.25}$$

并使用公式 (7.23) 产生

$$\alpha(u)\beta(u)=\frac{1}{4}k(1-\alpha(u)-\beta(u))^2 \tag{7.26}$$

方程 (7.26) 是斜坐标系中的隐式 (二次) 方程 $\{\mathbf{P}_1,\mathbf{S},\mathbf{T}\}$。注意，公式 (7.25) 中的常数 k 是第六章中给出的圆锥形状因子，公式 (6.59)。方程 (7.26) 表示 k 决定了圆锥；如果三个权重以某种方式改变，使得 k 不变，则只有参数化发生变化，而曲线不变。

圆锥的类型可以通过查看公式 (7.21) 的分母 $w(u)$ 来确定，该分母可以写为

$$w(u)=(1-u)^2w_0+2u(1-u)w_1+u^2w_2$$

$$=(w_0-2w_1+w_2)u^2+2(w_1-w_0)u+w_0 \tag{7.27}$$

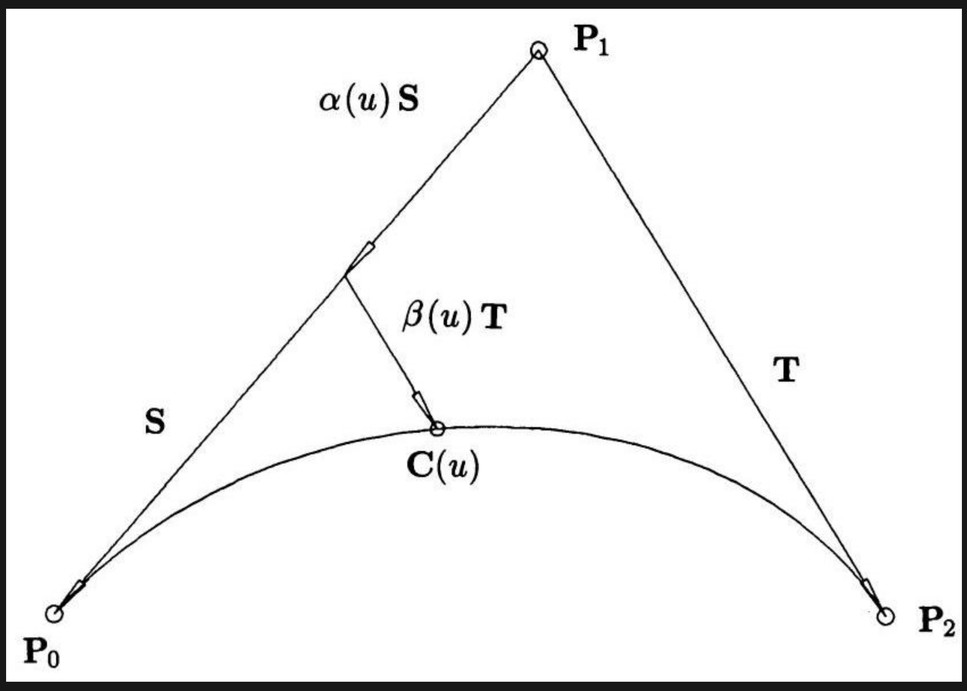


图 7.11. 有理二次贝塞尔曲线的局部坐标系。

方程 (7.27) 的根为

$$u_{1,2} = \frac{w_0 - w_1 \pm w_1\sqrt{1-k}}{w_0 - 2w_1 + w_2} \tag{7.28}$$

其中 k 是圆锥形状因子。方程 (7.25) 表示可以任意选择任意两个权重；然后圆锥由第三个权重决定。通常选择 $w_0 = w_2 = 1$ ；这称为标准参数化。然后如果 $w_1 = 1$ ，则公式 (7.21) 是抛物线。假设 $w_1 \neq 1$ ，则公式 (7.28) 意味着

- 如果 $k > 1$ ，则方程 (7.27) 没有实数解；曲线上没有无穷远点，因此它是一个椭圆；
- 如果 $k = 1$ ，方程 (7.27) 有一个实数解；曲线上有一个无穷远点，曲线是一个抛物线；
- 如果 $k < 1$ ，方程 (7.27) 有两个根；曲线上有两个无穷远点，它是一个双曲线。

将这些条件用 w_1 表达，我们得到:

- ${w_1}^2 < 1 \,(-1 < w_1 < 1) \Rightarrow$ 椭圆;
- ${w_1}^2 = 1 \,(w_1 = 1 \text{ or } -1) \Rightarrow$ 抛物线;
- ${w_1}^2 > 1 \,(w_1 > 1 \text{ 或 } w_1 < -1) \Rightarrow$ 双曲线。

(见图 7.12)。

注意到 w_1 可以为零或负值。$w_1 = 0$ 从 $\mathbf{P}_0$ 到 $\mathbf{P}_2$ 产生一条直线段，而 $w_1 < 0$ 产生互补弧，按相反顺序遍历 (见 [Lee87] 的简单证明)。还要注意，如果 $w_1 < 0$ ，则凸包性质不成立。

改变 w_1 会产生一系列的圆锥弧，其端点为 $\mathbf{P}_0$ 和 $\mathbf{P}_2$ ，并且端切线与 $\mathbf{P}_0\mathbf{P}_1$ 和 $\mathbf{P}_1\mathbf{P}_2$ 平行。然而，指定权重并不是一个方便的设计工具。从这个系列中选择一个圆锥的更方便的方法是指定圆锥上的第三个点，该点在某个参数值下获得，设为 $u = 1/2$ 。这个点称为圆锥的肩点，$\mathbf{S} = \mathbf{C}\,(1/2)$ (见图 7.12)。将 $u = 1/2$ 代入方程 (7.21) 得到

$$\mathbf{S} = \frac{1}{1+w_1}\mathbf{M} + \frac{w_1}{1+w_1}\mathbf{P}_1 \tag{7.29}$$

其中 $\mathbf{M}$ 是弦 $\mathbf{P}_0\mathbf{P}_2$ 的中点。由于我们选择了 $w_0 = w_2 = 1$ ，因此可以得出在 $\mathbf{S}$ 处圆锥的切线与 $\mathbf{P}_0\mathbf{P}_2$ 平行，即圆锥在 $\mathbf{S} = \mathbf{C}\,(1/2)$ 处达到与 $\mathbf{P}_0\mathbf{P}_2$ 的最大距离。设 s 为一个新参数，它在 $\mathbf{M}$ 和 $\mathbf{P}_1$ 之间进行线性插值。那么对于某个 s 的值，我们有

$$\mathbf{S} = (1-s)\,\mathbf{M} + s\mathbf{P}_1 \tag{7.30}$$

从方程 (7.29) 和 (7.30) 可以得出

$$s = \frac{w_1}{1+w_1} \quad w_1 = \frac{s}{1-s} \tag{7.31}$$

参数 s 是一个良好的形状设计工具。设计师可以将他的肩点 (决定曲线的丰满度) 从 $\mathbf{M}$ 线性移动到 $\mathbf{P}_1$.$s = 0$ ，产生一条线段，$0 < s < 1/2$ 产生一个椭圆，$s = 1/2$ 产生一个抛物线，而 $1/2 < s < 1$ 产生一个双曲线。

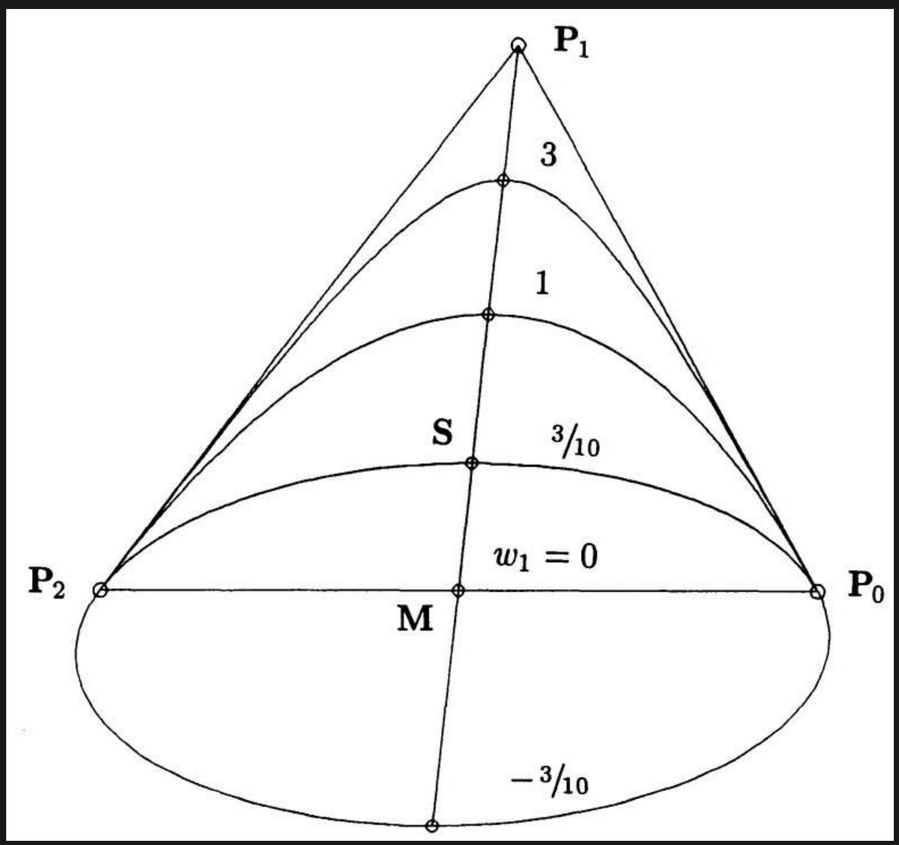


图 7.12. 由 $w_1 = \{3, 1, 3/10, 0, -3/10\}$ 定义的各种圆锥弧。

扫掠角小于 180° 的圆弧也由公式 (7.21) 表示。为了对称，$\mathbf{P}_0\mathbf{P}_1\mathbf{P}_2$ 必须是一个等腰三角形，具有 $\mathbf{P}_0\mathbf{P}_1 = \mathbf{P}_1\mathbf{P}_2$ 。因为圆是椭圆的特例，我们期望 $0 < w_1 < 1$ 。考虑图 7.13。从公式 (7.31) 可以得出

$$w_1 = \frac{s}{1-s} = \frac{\mathbf{MS}}{\mathbf{SP}_1} \tag{7.32}$$

设 $\theta = \angle\mathbf{P}_1\mathbf{P}_2\mathbf{M}$ 。由于对称，弧 $\mathbf{P}_2\mathbf{S}$ 与 $\mathbf{SP}_0$ 相同，因此角 $\angle\mathbf{SP}_2\mathbf{M}$ 平分 θ 。根据平分线的性质可以得出

$$w_1 = \frac{\mathbf{MS}}{\mathbf{SP}_1} = \frac{\mathbf{MP}_2}{\mathbf{P}_1\mathbf{P}_2} = \frac{e}{f} = \cos\theta \tag{7.33}$$

本节中的许多材料可以追溯到 Coons 和 Forrest 的工作 [Coon67; Ahuj68; Forr68]。它也可以在 [Lee87; Pieg87a] 中找到。

4　无限控制点

在本节中，我们引入无限控制点，这是一个用于构造扫掠 180° 的圆形和椭圆弧的概念。Versprille [Vers75] 简要提到无限控制点，而 Piegl 则使用它们构造了许多不同的曲线和曲面 [Pieg87c, 88a, 88b]。

在射影几何中，无穷远点的概念是常见的 [Ahuj68; Coxe74; Ries81]。在四维空间中，点 $\mathbf{P}^w=(x,y,z,0)$ (并且 z 不是全零) 被称为无穷远点。它在三维空间中通过从原点到三维点 (x, y, z) 的方向向量来表示。因此，$\mathbf{P}=H\{\mathbf{P}^w\}$ 是一个方向向量 (见公式 [1.16])。为了强调这一点，我们使用符号 $\vec{\mathbf{P}}$ 。

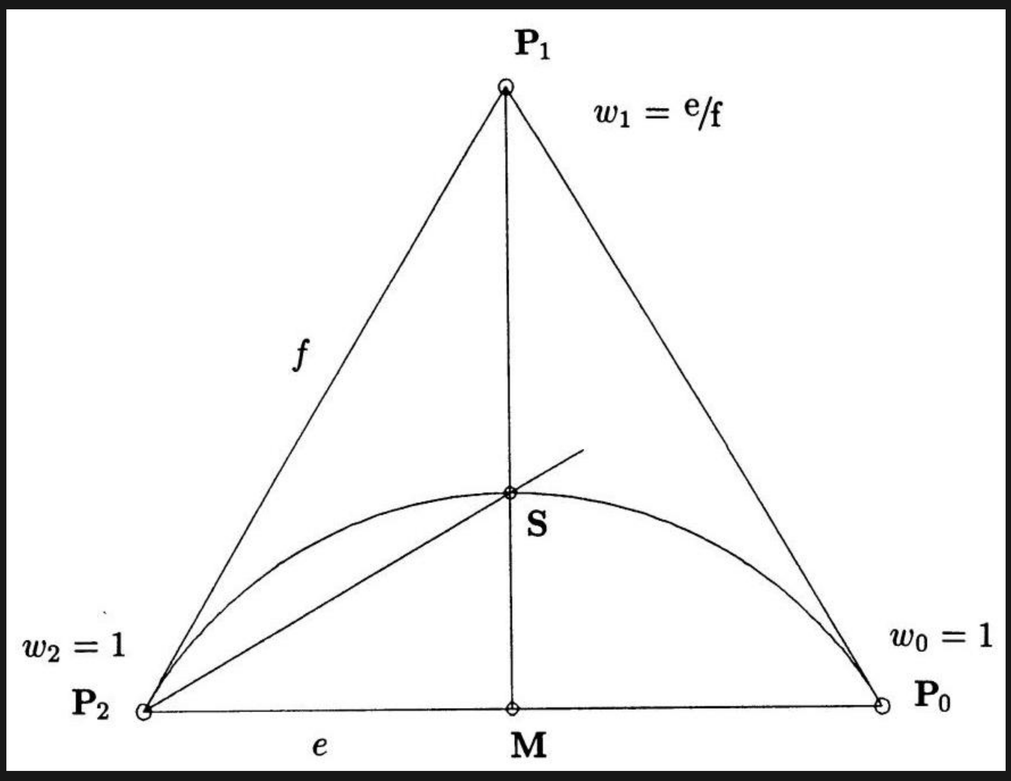


图 7.13。小于 180° 的圆弧的二次有理贝塞尔表示。

假设 $\mathbf{P}_j^w=(x_j,y_j,z_j,0)$ 是曲线 $\mathbf{C}(u)$ 的一个无穷控制点。那么

$$\mathbf{C}^w(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_i^w$$

可以写成

$$\mathbf{C}(u)=\frac{\left(\sum\limits_{j\neq i=0}^{n}N_{i,p}(u)\,w_i\mathbf{P}_i\right)+N_{j,p}(u)\,\vec{\mathbf{P}}_j}{\sum\limits_{j\neq i=0}^{n}N_{i,p}(u)\,w_i}$$

$$=\frac{\sum\limits_{j\neq i}N_{i,p}(u)\,w_i\mathbf{P}_i}{w(u)}+\frac{N_{j,p}(u)\,\vec{\mathbf{P}}_j}{w(u)}$$

$$=\overline{\mathbf{C}}(u)+f(u)\,\vec{\mathbf{P}}_j \tag{7.34}$$

因此，对于固定的 $u_0,\mathbf{C}(u_0)$ ，是 $\overline{\mathbf{C}}(u_0)$ ，它位于控制点 $\mathbf{P}_0,\ldots,\mathbf{P}_{j-1},\mathbf{P}_{j+1},\ldots,\mathbf{P}_n$ 的凸包内，加上向量 $\vec{\mathbf{P}}_j$ 的非负缩放因子 (见图 7.14)。增加 $\vec{\mathbf{P}}_j$ 的大小会将曲线拉向其方向。注意在图 7.14 中，曲线 $\overline{\mathbf{C}}(u)$ 在段 $\mathbf{P}_1\mathbf{P}_3$ 和 $\mathbf{P}_3\mathbf{P}_4$ $\mathbf{C}(u)$ 上接触控制多边形，并且是具有两个不同内部结点 (重数为 1) 的三次曲线。曲线 $\overline{\mathbf{C}}(u)$ 接触控制多边形的点对应于在这些内部结点值下评估的 $\overline{\mathbf{C}}(u)$ 。

我们在此警告读者，从射影几何的角度来看，我们在声明 $\mathbf{P}^w=(x,y,z,0)$ 是一个无穷远点时并不严格正确。在射影几何术语中，点 (x, y, z, w) 和 $(\alpha x,\alpha y,\alpha z,\alpha w)$ 、$\alpha\neq 0$ 是相同的，也就是说，射影空间中的一个点是数学家所称的等价类。这意味着 $\vec{\mathbf{P}}_j=(x_j,y_j,z_j,0)$ 和 $\vec{\mathbf{P}}_j^*=(\alpha x_j,\alpha y_j,\alpha z_j,0)$ 是射影空间中同一点的两种表示。然而，将 $\vec{\mathbf{P}}_j$ 和 $\vec{\mathbf{P}}_j^*$ 代入方程 (7.34) 显然会导致两条不同的曲线。在本书中，我们不深入探讨射影几何，因此我们选择牺牲数学的严谨性以求清晰。严格来说，我们的无穷控制点只是一个射影点的代表，其最后一个坐标恰好为零。

示例

Ex7.1 我们用一个无穷控制点表示以原点为中心、半径为 r 的半圆 (见图 7.15)。设 $\mathbf{P}_0^w=(r,0,1),\mathbf{P}_1^w=(0,\mathrm{r},0)$，和 $\mathbf{P}_2^w=(-r,0,1)$ 。那么

$$\mathbf{C}(u)=\frac{(1-u)^2w_0\mathbf{P}_0+u^2w_2\mathbf{P}_2}{(1-u)^2w_0+u^2w_2}+\frac{2u(1-u)}{(1-u)^2w_0+u^2w_2}\vec{\mathbf{P}}_1$$

$$=\frac{(1-u)^2\mathbf{P}_0+u^2\mathbf{P}_2}{1-2u+2u^2}+\frac{2u(1-u)}{1-2u+2u^2}\vec{\mathbf{P}}_1$$

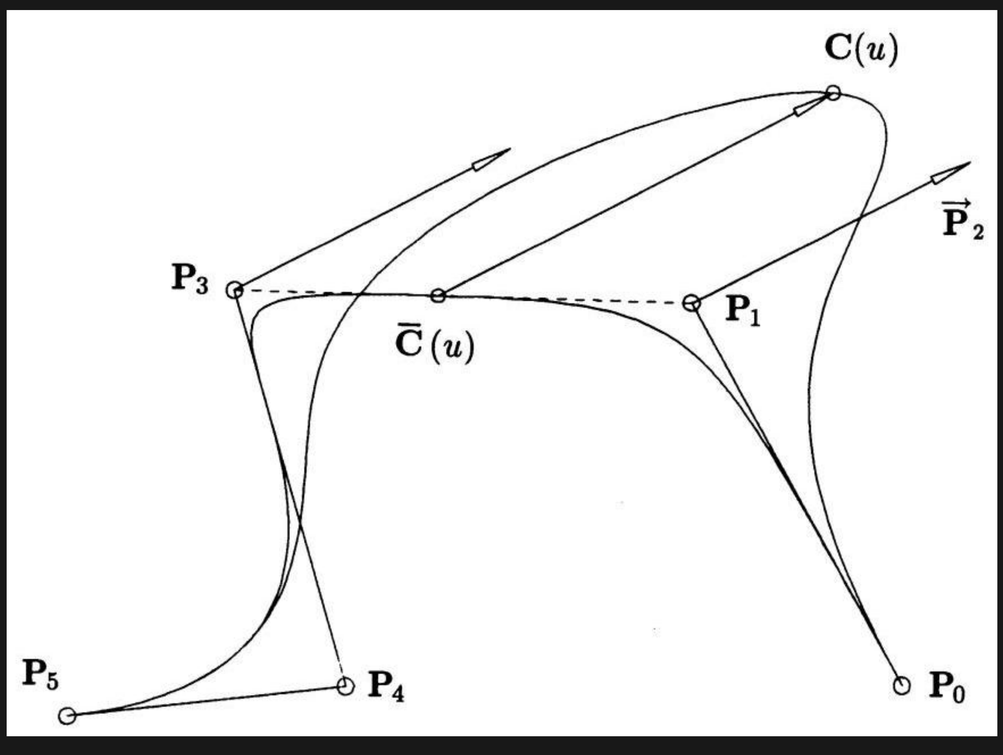


图 7.14. 带有无穷控制点 $\vec{P}_2$ 的 NURBS 曲线；注意更强的凸包性质。

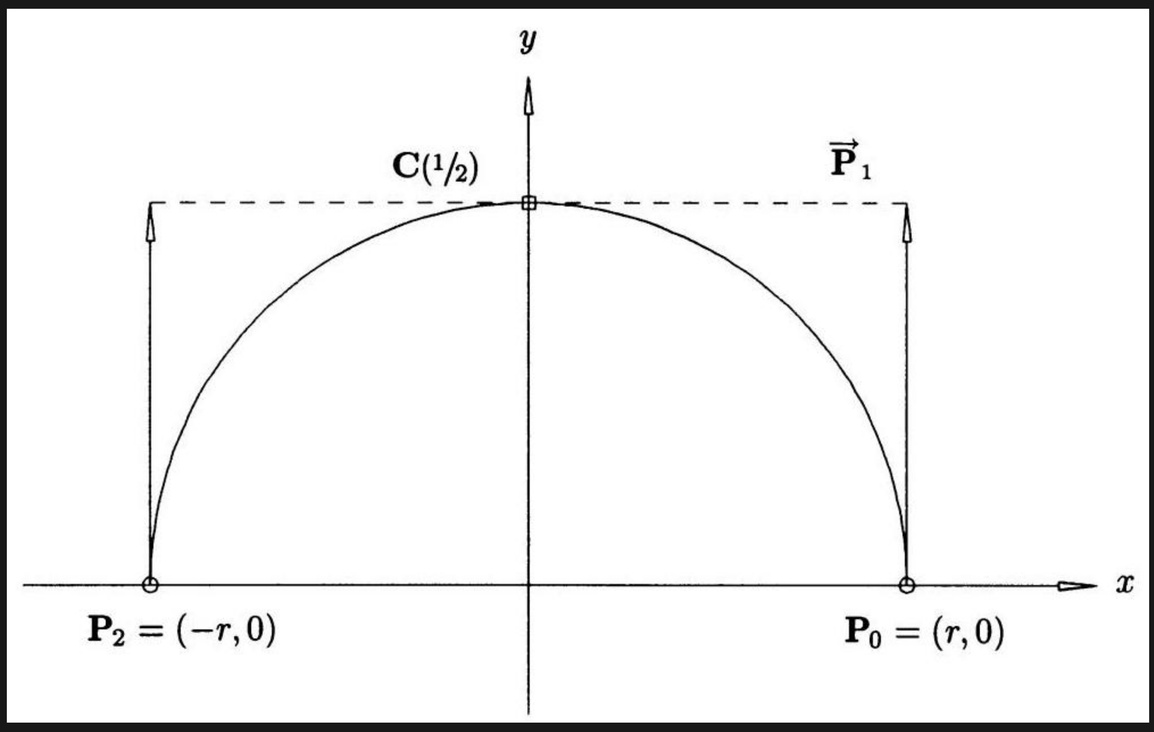


图 7.15. 通过使用无穷控制点定义的半圆。

因此

$$x(u) = \frac{r(1-2u)}{1-2u+2u^2} \quad y(u) = \frac{2ru(1-u)}{1-2u+2u^2}$$

读者可以验证对于所有 u ，$(x(u))^2 + (y(u))^2 = r^2$ 是正确的。注意到

$$\mathbf{C}\left(\frac{1}{2}\right) = (0, r)$$

是弧上距离弦 $\mathbf{P}_0\mathbf{P}_2$ 最远的点，而此点的导数 $\mathbf{C}'(1/2)$ 与 $\mathbf{P}_0\mathbf{P}_2$ 平行，即第 7.3 节的肩点概念在 $\mathbf{P}_1^w$ 是无穷控制点时也成立。

请注意，无限控制点和零权重之间是有区别的。图 7.12 显示 $w_1 = 0$ 从 $\mathbf{P}_0$ 到 $\mathbf{P}_2$ 产生一条直线。将 $w_1 = 0$ 代入方程 (7.21) 并假设正常参数化得到

$$\mathbf{C}(u) = R_{0,2}(u)\,\mathbf{P}_0 + R_{2,2}(u)\,\mathbf{P}_2 = \frac{(1-u)^2\mathbf{P}_0 + u^2\mathbf{P}_2}{(1-u)^2 + u^2}$$

即 $R_{1,2}(u) \equiv 0$ ，并且 $\mathbf{C}(u)$ 是从 $\mathbf{P}_0$ 到 $\mathbf{P}_2$ 的一条直线；另见图 4.2、4.3c、4.4、4.5d 和 4.6。如果我们将 NURBS 曲线视为齐次控制点

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_i^w \tag{7.35}$$

那么我们用 $\mathbf{P}_j^w = (x_j, y_j, z_j, 0)$ 表示一个无限控制点，并通过设置 $\mathbf{P}_j^w = (0,0,0,0)$ 将 w_j 设置为零。尽管在处理这些点时必须小心，但它们通常不会在从方程 (7.35) 推导出的 B 样条算法中造成问题。例如，前几章的点评估和节点插入算法 (算法 A4.1 和 A5.1) 在处理零权重或无限控制点时没有问题 (当然，假设并非所有权重都是零)。

5 圆的构造

在这一部分，我们开发了构造任意扫掠角的圆弧的算法，包括完整的圆。构造一般的 NURBS 圆弧比最初预期的要复杂，并且有许多方法可以实现。此部分的大部分材料取自 [Pieg89b]，这是对 NURBS 圆的详细研究。

本节仅讨论二次表示。当需要的弧大于或等于 180° 时，高阶表示是有用的，且不需要内部节点，也不使用负权重或无限控制点。这种表示可以通过提高阶数获得 (见第 7.8 节)。

从第 7.3 节我们知道如何构造小于 180° 的弧，显然我们可以通过简单地使用多个节点拼接较小的弧来构造任意扫掠角的弧。在决定如何做到这一点时，有四个权衡需要考虑: 连续性、参数化、凸包和所需控制点的数量。最后三个是相关的，因为通过增加控制点的数量可以改善参数化和凸包的特性。我们通过一些例子来阐明这些权衡。为简便起见，所有示例均位于 xy 平面，中心在原点，半径为 1。

示例

Ex7.2 使用九点方形控制多边形绘制完整圆: 示例 Ex6.7 和图 6.12(以及第 7.3 节) 显示第一象限的 90° 弧是通过使用 $\{\mathbf{P}_i\} = \{(1,0),(1,1),(0,1)\}$ 和 $\{w_i\} = \{1, \sqrt{2}/2, 1\}$. 获得的。通过使用双节点将这四个弧拼接在一起，我们得到了图 7.16a 和 7.16b 中的完整圆。节点、权重和控制

点是

$$U = \left\{0,0,0,\frac{1}{4},\frac{1}{4},\frac{1}{2},\frac{1}{2},\frac{3}{4},\frac{3}{4},1,1,1\right\}$$

$$\{w_i\} = \left\{1,\frac{\sqrt{2}}{2},1,\frac{\sqrt{2}}{2},1,\frac{\sqrt{2}}{2},1,\frac{\sqrt{2}}{2},1\right\}$$

$$\{\mathbf{P}_i\} =$$

$$\{(1,0),(1,1),(0,1),(-1,1),(-1,0),(-1,-1),(0,-1),(1,-1),(1,0)\}$$

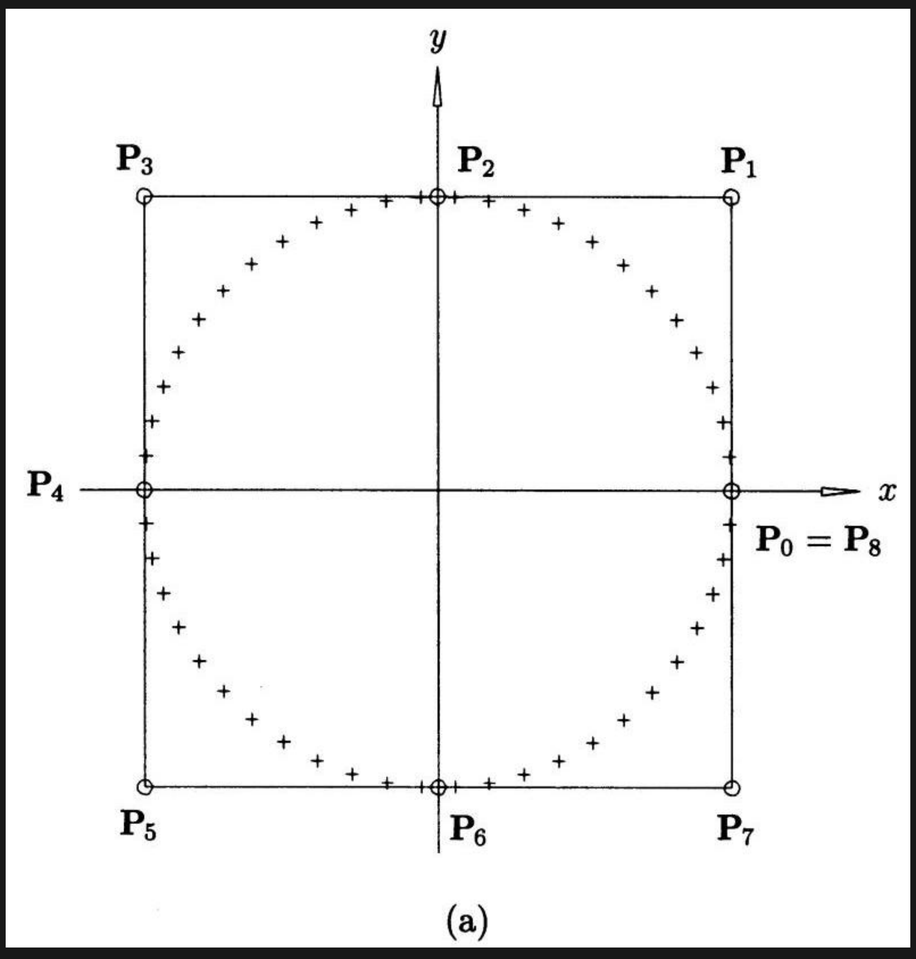


图 7.16. 基于九点方形的 NURBS 圆。(a) 控制多边形和参数化；(b) 有理基函数。

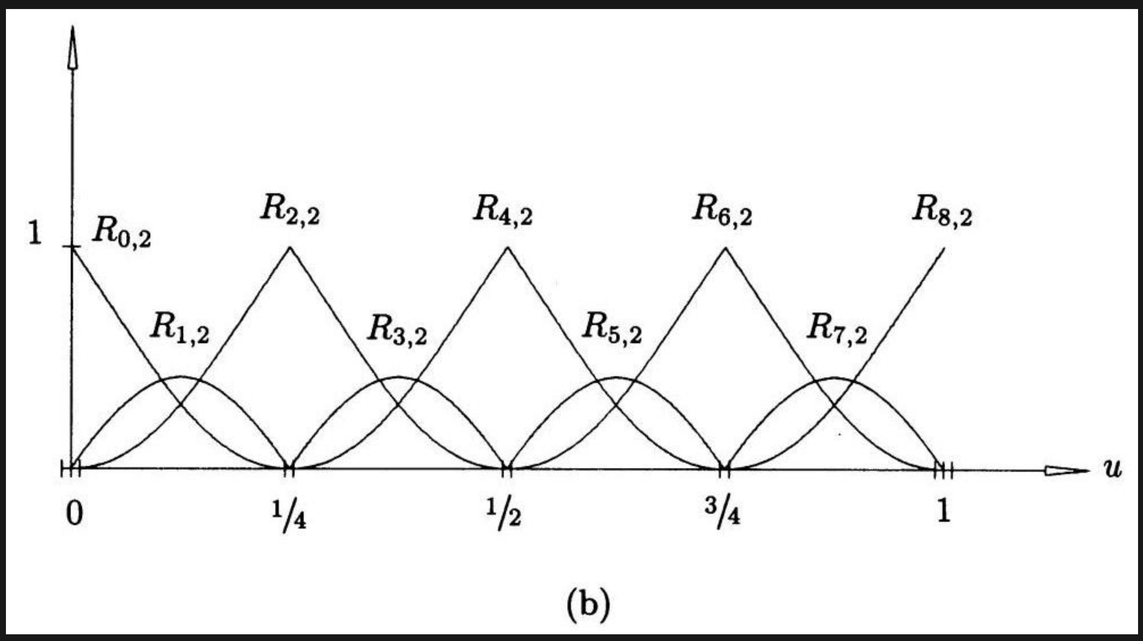


图 7.16.(续。)

为了指示参数化，圆上的点被标记；它们是在均匀间隔的参数值下计算得出的。外接正方形形成了相当紧凑的凸包。现在考虑在 $u = 1/4\,(u = 1/2$ 和 $u = 3/4$ 处的连续性是类似的。我们可以写出任一

$$\mathbf{C}^w(u) = \sum_{i=0}^{n} N_{i,2}(u)\,\mathbf{P}_i^w \quad \text{or} \quad \mathbf{C}(u) = \sum_{i=0}^{n} R_{i,2}(u)\,\mathbf{P}_i$$

显然，基函数 $N_{2,2}(u)$ 和 $R_{2,2}(u)$ 仅在 $u = 1/4$ 处是 C^0 连续的 (见图 7.16b)。$\mathbf{C}^w(u)$ 也仅是 C^0 连续的，即四个抛物弧在齐次空间中围绕圆锥连接在一起，每对相遇的地方都有一个尖点。从分析上看，可以通过从左侧和右侧计算 $\mathbf{C}^w(u)$ 在 $u = 1/4$ 处的第一导数来观察这一点。将方程 (3.7) 应用到仅 w 坐标上得到

$$w'\left(\frac{1}{4}\right)_{\text{left}} = \frac{2}{\frac{1}{4}-0}(w_2 - w_1) = 8\left(1 - \frac{\sqrt{2}}{2}\right)$$

和

$$w'\left(\frac{1}{4}\right)_{\text{right}} = \frac{2}{\frac{1}{2}-\frac{1}{4}}(w_3 - w_2) = 8\left(\frac{\sqrt{2}}{2} - 1\right)$$

显然，这两个导数不相等。令人惊讶的是，$\mathbf{C}(u)$ 是 C^1 连续的。使用方程 (4.9) 和 (4.10) 我们得到

$$\mathbf{C}'\left(\frac{1}{4}\right)_{\text{left}} = \frac{2}{\frac{1}{4}-0}\frac{\sqrt{2}/2}{1}(\mathbf{P}_2 - \mathbf{P}_1) = \left(-4\sqrt{2}, 0\right)$$

$$\mathbf{C}'\left(\frac{1}{4}\right)_{\text{right}} = \frac{2}{\frac{1}{2}-\frac{1}{4}}\frac{\sqrt{2}/2}{1}(\mathbf{P}_3 - \mathbf{P}_2) = \left(-4\sqrt{2}, 0\right)$$

关于前三个标准，这对完整的圆的表示相当不错。

Ex7.3 使用七点三角控制多边形的完整圆: 从基础几何可以很容易看出，一个 120° 的弧需要一个控制三角形，其底角 $\angle\mathbf{P}_1\mathbf{P}_0\mathbf{P}_2$ 等于 60° 。根据方程 (7.33)，$w_1 = \cos 60° = 1/2$ 。通过拼接三个这样的弧，我们得到图 7.17a 和 7.17 b 的完整圆，其中

$$U = \left\{0, 0, 0, \frac{1}{3}, \frac{1}{3}, \frac{2}{3}, \frac{2}{3}, 1, 1, 1\right\}$$

$$\{w_i\} = \left\{1, \frac{1}{2}, 1, \frac{1}{2}, 1, \frac{1}{2}, 1\right\}$$

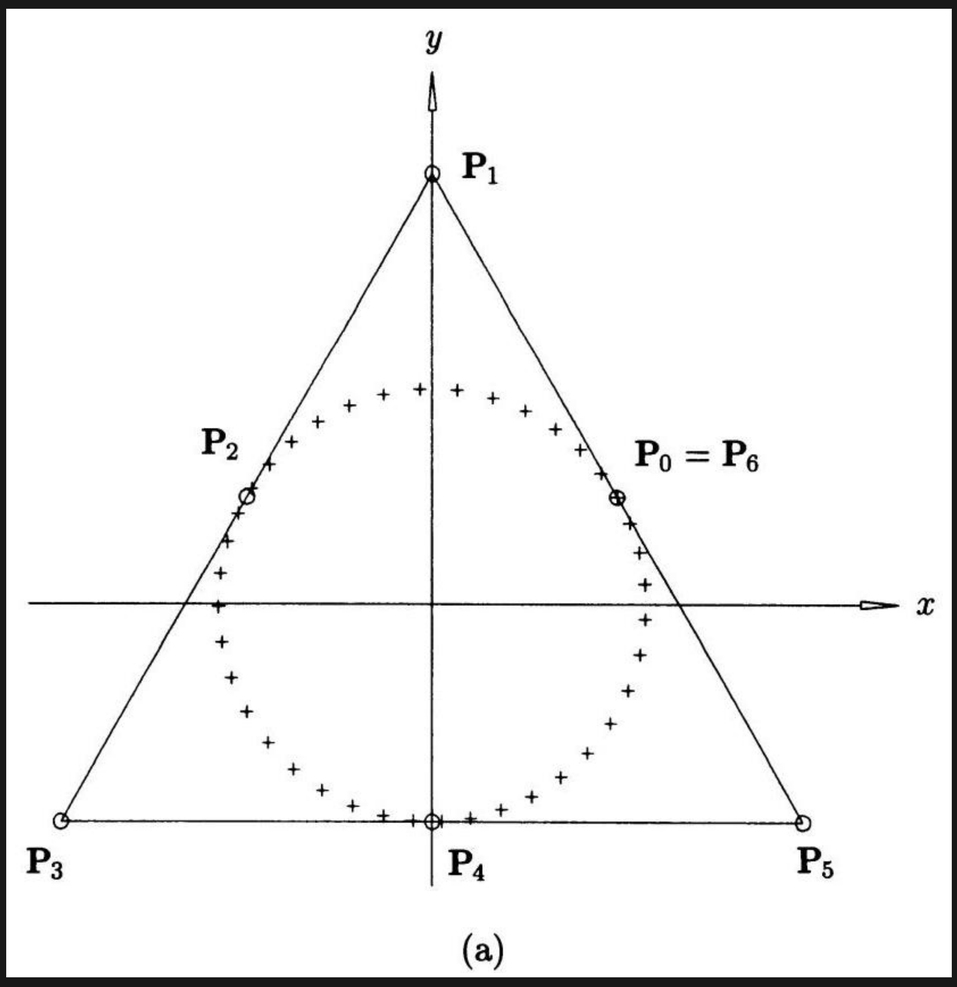


图 7.17. 基于七点三角形的 NURBS 圆。(a) 控制多边形和参数化；(b) 有理基函数。

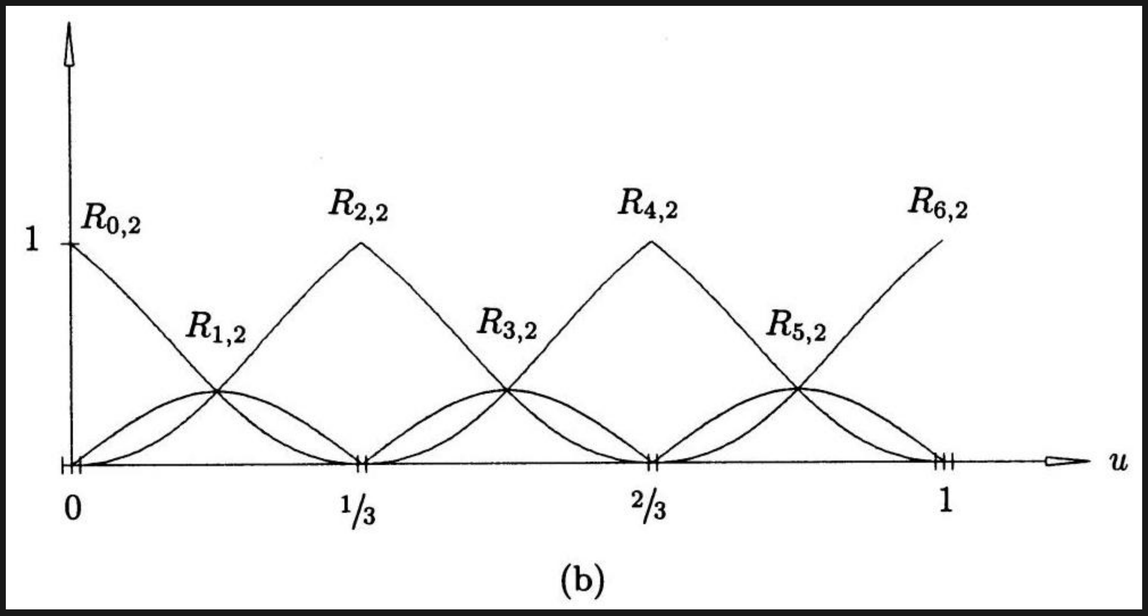


图 7.17.(续)

$\{\mathbf{P}_i\} =$

$$\left\{\left(a,\frac{1}{2}\right),(0,2),\left(-a,\frac{1}{2}\right),(-2a,-1),(0,-1),(2a,-1),\left(a,\frac{1}{2}\right)\right\}$$

在 $a=\cos 30^\circ$ 处。圆上的点被标记以显示参数化，这并不如图 7.16a 中的效果好。凸包也更松散。读者可以验证 $\mathbf{C}(u)$ 在 C^1 和 $u=1/3$ 处是 $u=2/3$ 连续的，尽管基函数在那里是 C^0 连续的 (见图 7.17b)。

Ex7.4 一个 240° 的弧: 设置 w_1 为负值会产生互补弧。因此，我们使用 240°arc 得到图 7.18a 和 7.18 b 。

$$U=\{0,0,0,1,1,1\}$$

$$\{w_i\}=\left\{1,-\frac{1}{2},1\right\}$$

$$\{\mathbf{P}_i\}=\left\{\left(a,\frac{1}{2}\right),(0,2),\left(-a,\frac{1}{2}\right)\right\}$$

负权重通常是不受欢迎的，因为我们失去了凸包属性，并且在计算曲线上的点时可能会出现除以零的情况。然而，重要的是要注意，对于圆和椭圆，$w(u)>0$ 对所有 $u\in[0,1]$ 都成立。这是由于 $w_0=w_2=1$ 、$|w_1|<1$ 和 $w(u)=(1-u)^2+2u(1-u)w_1+u^2$ (从几何上看，抛物线弧 $\mathbf{C}^w(u)$ 位于 $W=0$ 平面上的锥体之上)。因此，在使用算法 A4.1 评估该弧上的点时不会出现问题。让我们在 $u=1/2$ 处插入一个节点。使用公式 (5.11)

$$\alpha_1=\frac{\frac{1}{2}-0}{1-0}=\frac{1}{2}\ \alpha_2=\frac{\frac{1}{2}-0}{1-0}=\frac{1}{2}$$

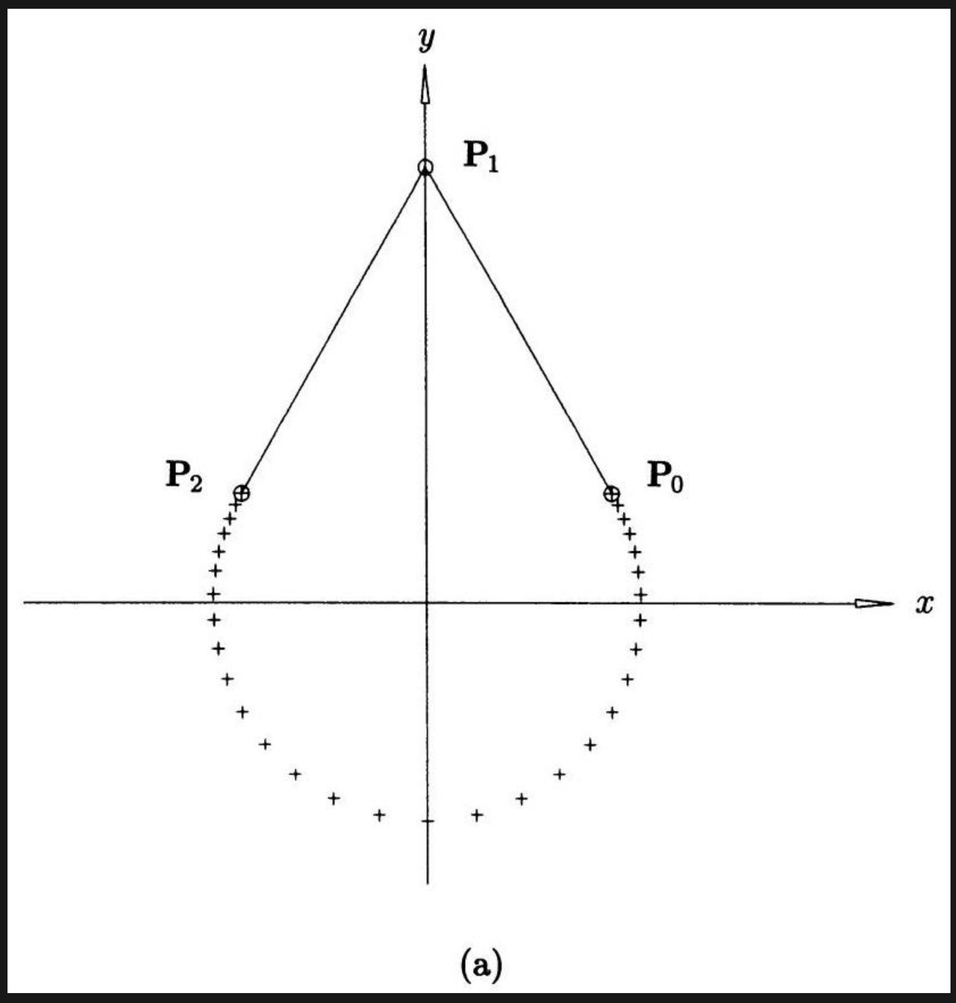


图 7.18。使用负权重的 240° 的弧。(a) 控制多边形和参数化；(b) 有理基函数。

新的控制点 $\mathbf{Q}_i^w$ 是

$$\mathbf{Q}_0^w = \mathbf{P}_0^w \ \mathbf{Q}_3^w = \mathbf{P}_2^w$$

$$\mathbf{Q}_1^w = \frac{1}{2}\mathbf{P}_1^w + \frac{1}{2}\mathbf{P}_0^w = \left(\frac{a}{2}, -\frac{1}{4}, \frac{1}{4}\right)$$

$$\mathbf{Q}_2^w = \frac{1}{2}\mathbf{P}_2^w + \frac{1}{2}\mathbf{P}_1^w = \left(-\frac{a}{2}, -\frac{1}{4}, \frac{1}{4}\right)$$

因此 (见图 7.19a 和 7.19b)

$$U = \left\{0, 0, 0, \frac{1}{2}, 1, 1, 1\right\}$$

$$\{w_i\} = \left\{1, \frac{1}{4}, \frac{1}{4}, 1\right\}$$

$$\{\mathbf{Q}_i\} = \left\{\left(a, \frac{1}{2}\right), (2a, -1), (-2a, -1), \left(-a, \frac{1}{2}\right)\right\}$$

是没有负权重的 240°arc 的表示。参数化效果并不是特别好。$\mathbf{C}^w(u)$ 和 $\mathbf{C}(u)$ 在 C^2 上是 $u \in [0, 1]$ 连续的；插入节点并没有改变这一点。

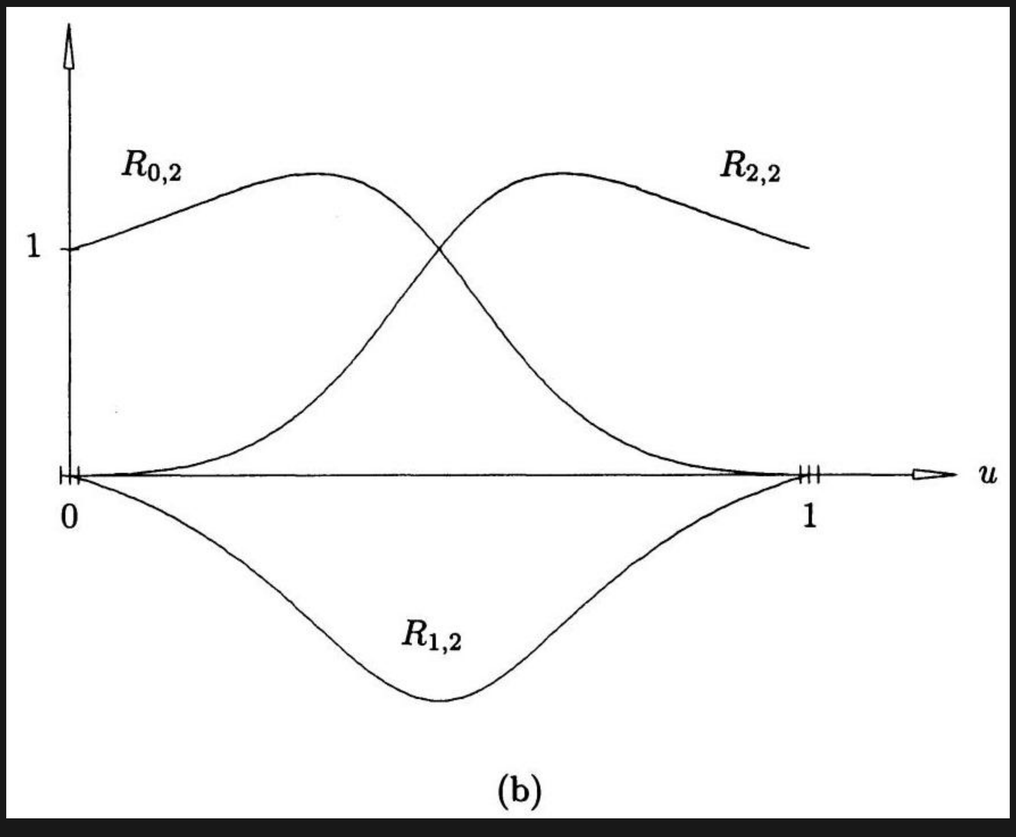


图 7.18。(续)

Ex7.5 使用四个控制点的半圆: 考虑示例 Ex7.1 和图 7.15 中的半圆，具有 $r = 1$ 。插入节点 $u = 1/2$ 一次，我们得到 (如示例 Ex7.4 所示)

$$\alpha_1 = \alpha_2 = \frac{1}{2}$$

$$\mathbf{Q}_0^w = \mathbf{P}_0^w \ \mathbf{Q}_3^w = \mathbf{P}_2^w$$

$$\mathbf{Q}_1^w = \frac{1}{2}\mathbf{P}_1^w + \frac{1}{2}\mathbf{P}_0^w = \left(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$$

$$\mathbf{Q}_2^w = \frac{1}{2}\mathbf{P}_2^w + \frac{1}{2}\mathbf{P}_1^w = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$$

因此 (见图 7.20a 和 7.20b)

$$U = \left\{0, 0, 0, \frac{1}{2}, 1, 1, 1\right\}$$

$$\{w_i\} = \left\{1, \frac{1}{2}, \frac{1}{2}, 1\right\}$$

$$\{\mathbf{Q}_i\} = \{(1, 0), (1, 1), (-1, 1), (-1, 0)\}$$

Ex7.6 使用七点方形控制多边形的完整圆: 两个半圆，如示例 Ex7.5 中所示，可以拼接在一起形成一个七点方形圆 (见图 7.21a 和 7.21b)

$$U = \left\{0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{1}{2}, \frac{3}{4}, 1, 1, 1\right\}$$

$$\{w_i\} = \left\{1, \frac{1}{2}, \frac{1}{2}, 1, \frac{1}{2}, \frac{1}{2}, 1\right\}$$

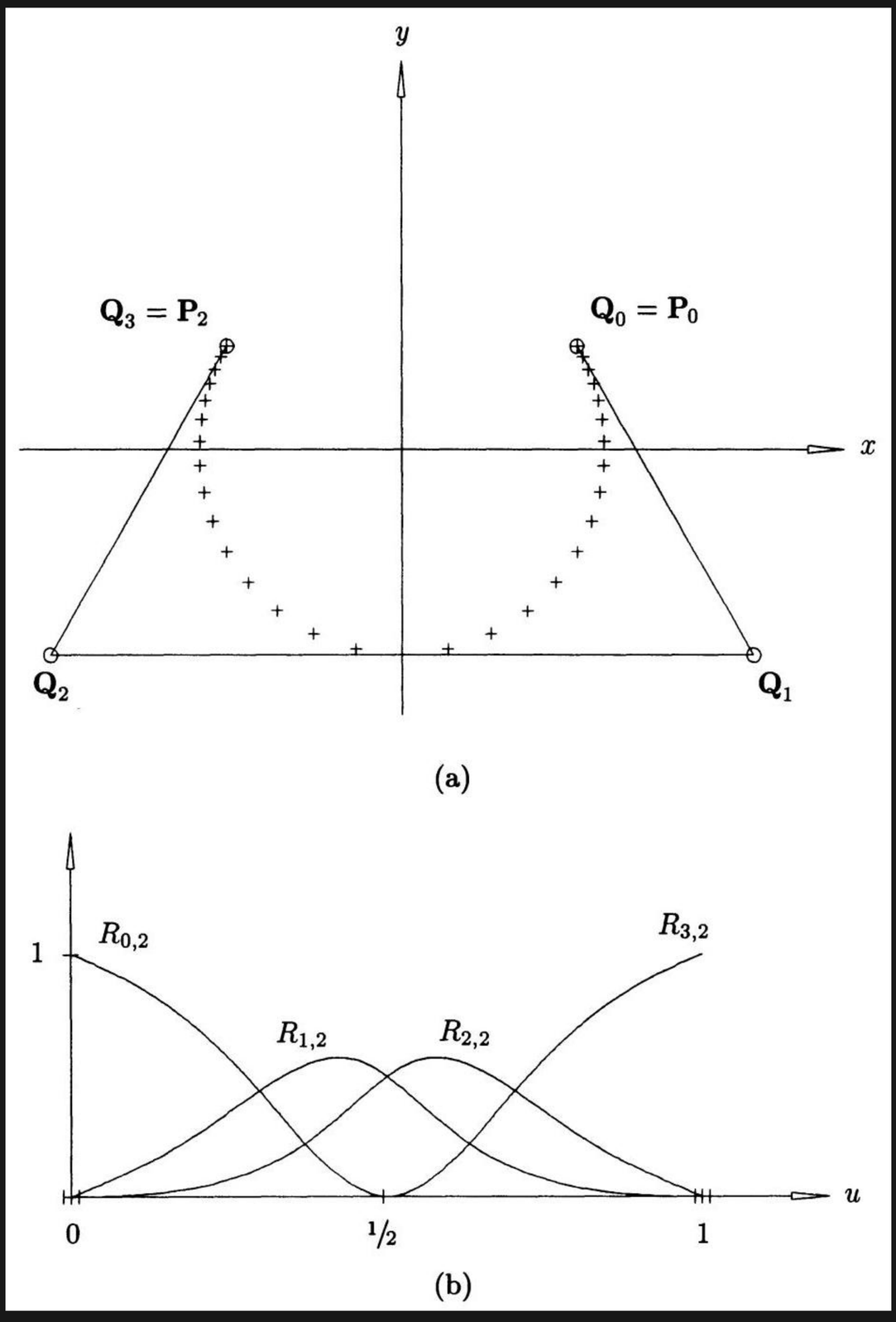


图 7.19. 在插入 $u = 1/2$ 一次后的图 7.18 的弧线。(a) 控制多边形和参数化；(b) 有理基函数。

$\mathbf{C}(u)$ 在 $u = 1/2$ 处是 C^1 连续的，并且在其他地方是 C^2 连续的。该参数化不如示例 Ex7.2 和图 7.16a 的效果好。

算法 A7.1 在三维空间中构建任意扫掠角 $\theta\,(0° < \theta \leq 360°)$ 的 NURBS 圆弧。它拼接了扫掠角 $d\theta, \min(\theta, 45°) < d\theta \leq 90°$ 的等弧，使用双节点，并通过方程 (7.33) 计算权重。得到的角度 θ 的弧是 C^1 连续的，具有紧凑的凸包，并且具有良好的参数化。

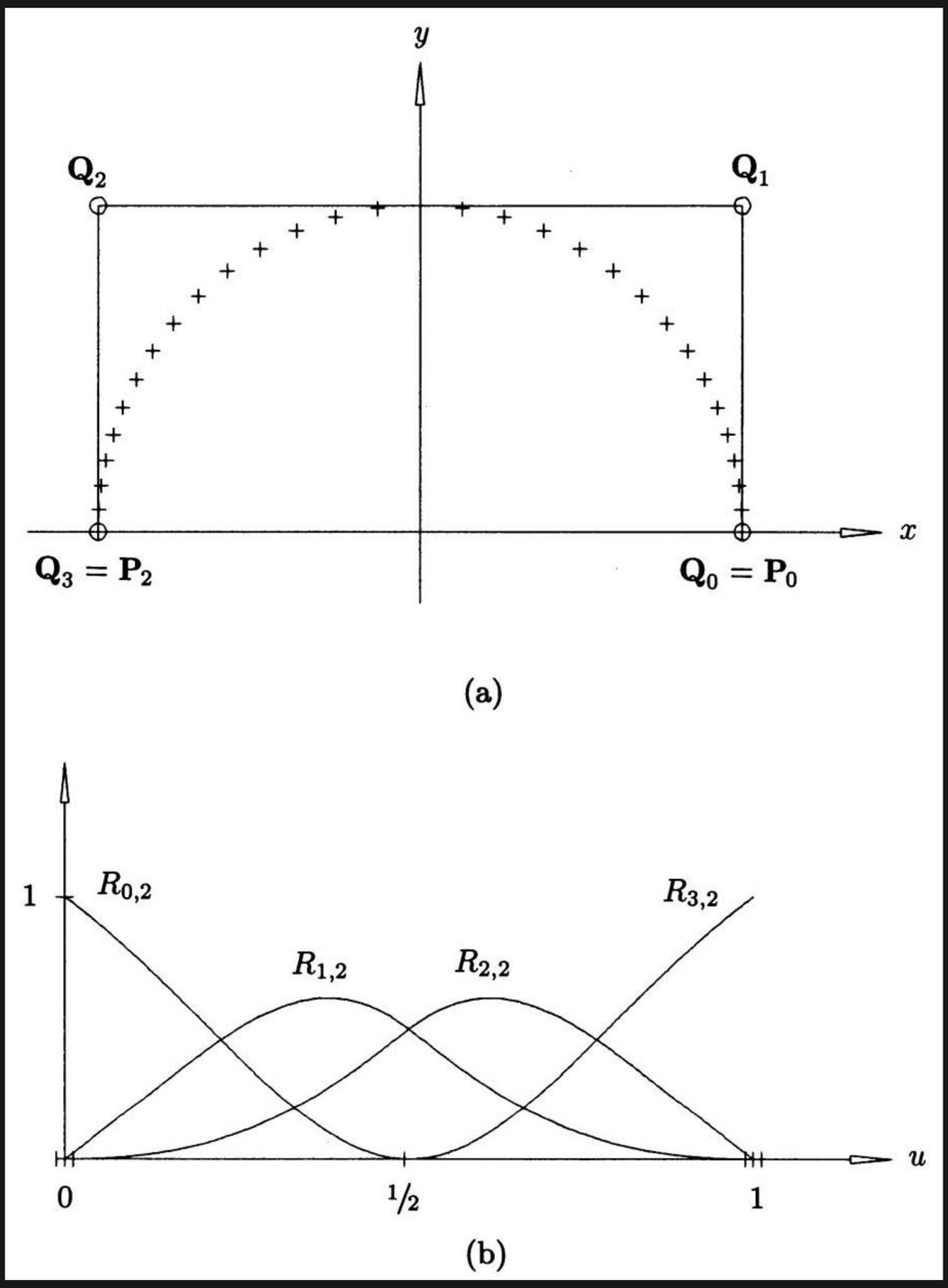


图 7.20. 在插入 $u = 1/2$ 一次后的图 7.15 的半圆。(a) 控制多边形和参数化；(b) 有理基函数。

设计师可以使用许多不同的接口来指定圆弧，但无论接口是什么，以下数据都可以轻松生成:

O：圆心 (局部坐标系的原点)；

X：位于圆的定义平面上的单位长度向量；

Y：单位长度向量，在圆的定义平面内，并且与 **X** 正交；

r：半径。

θ_s, θ_e：起始和结束角度，相对于 **X** 测量。

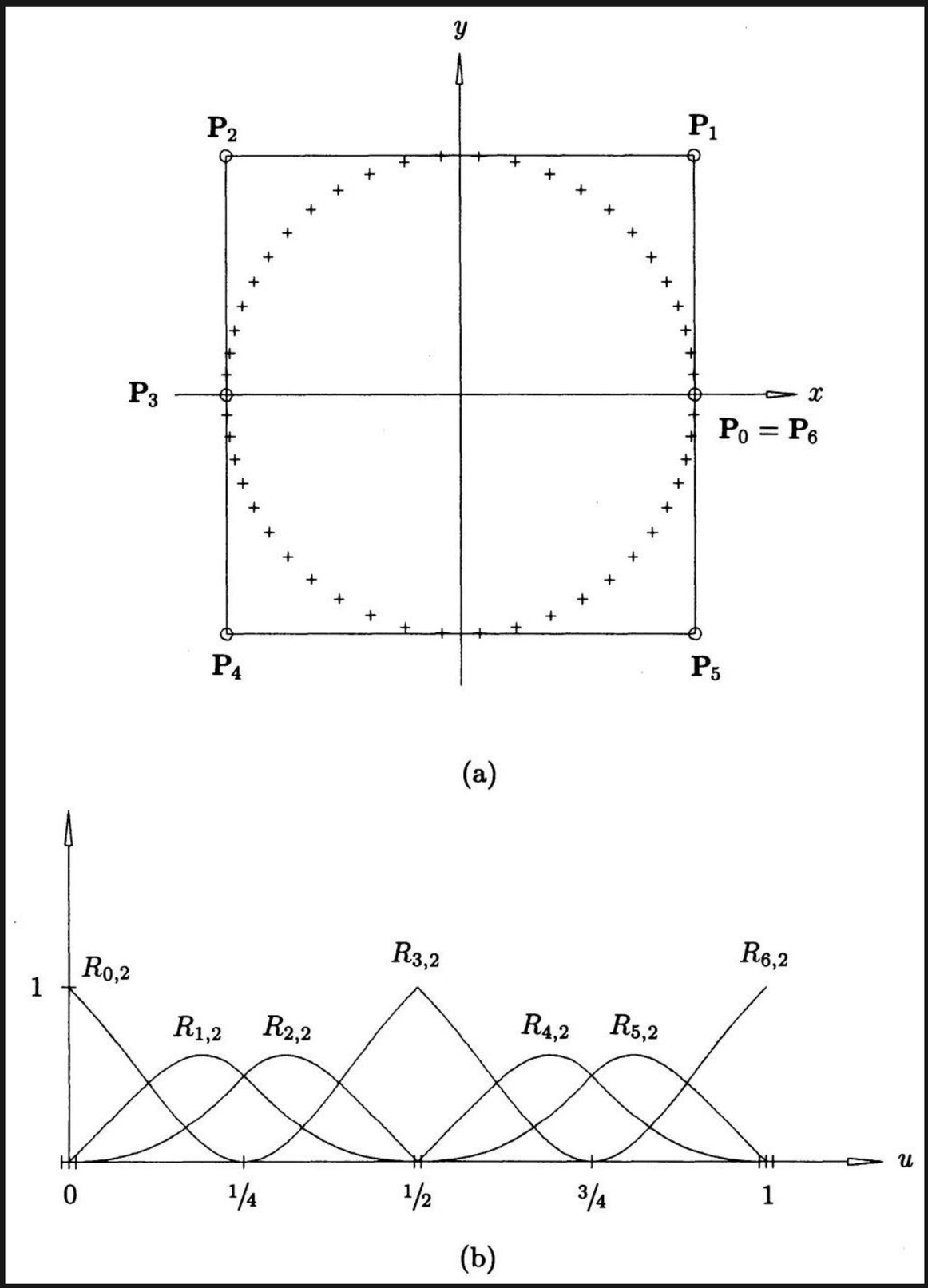


图 7.21. 一个七点方形基础的 NURBS 圆。(a) 控制多边形和参数化；(b) 有理基函数。

弧线表示为

$$\mathbf{C}(u) = \mathbf{O} + r\cos u\mathbf{X} + r\sin u\mathbf{Y}\ \theta_s \le u \le \theta_e \tag{7.36}$$

结果弧线在由 $\mathbf{O}, \mathbf{X}$ 和 $\mathbf{Y}$ 定义的局部坐标系中逆时针定向。我们假设一个实用程序 Intersect3DLines (P0, T0 , P2, T2 , alf0, alf 2, P1 ，它计算由点/切线对 $[\mathbf{P}_0, \mathbf{T}_0]$ 和 $[\mathbf{P}_2, \mathbf{T}_2]$ 给出的三维空间中两条线的交点 ($\mathbf{P}_1$) 。如果两条线平行，则返回整数值 1，否则返回 0。α_0 和 α_2 分别是 $\mathbf{P}_1$ 在 $[\mathbf{P}_0, \mathbf{T}_0]$ 和 $[\mathbf{P}_2, \mathbf{T}_2]$ 上的参数位置。α_0, α_2 和整数返回值在算法 A7.1 中未使用，但在第 7.6 节中使用。Pw=w*P 表示一个欧几里得点与权重 (Pw = P if $w = 1$) 的乘法。该算法计算 NURBS 圆弧的节点和加权控制点。

ALGORITHM A7.1

```
MakeNurbsCircle(0, X, Y, r, ths, the, n, U, Pw)
{ /* Create arbitrary NURBS circular arc */
      /* Input: 0, X, Y, r, ths, the */
    /* Output: n, U, Pw */
  if (the < ths) the = 360.0 + the;
  theta = the-ths;
  if (theta <= 90.0 ) narcs = 1; /* get number of arcs */
     else
        if (theta <= 180.0) narcs = 2 ;
          else
             if (theta <= 270.0) narcs = 3 ;
                else
                   narcs = 4 ;
  dtheta = theta/narcs;
  n = 2* narcs;  /* n + 1 control points */
  w1 = cos(dtheta/2.0); /* dtheta/2 is base angle */
  PO = 0 + r * cos (ths) * X + r * sin (ths) * Y;
  TO = -sin(ths)*X + cos(ths)*Y; /* Initialize start values */
  Pw [0] = P0;
  index = 0; angle = ths;
  for (i=1; i<=narcs; i++) /* create narcs segments */
     {
     angle = angle + dtheta;
     P2 = 0 + r*cos(angle)*X + r*sin(angle)*Y;
     Pw[index+2] = P2;
     T2 = -sin(angle)*X + cos(angle)*Y;
     Intersect3DLines(P0, T0, P2, T2, dummy, dummy, P1);
     Pw[index+1] = w1*P1;
     index = index + 2 ;
     if (i < narcs) { P0 = P2 ; T0 = T2 ; }
    }
  j = 2* narcs +1;  /* load the knot vector */
```

```
            for (i=0; i<3; i++)
                { U[i] = 0.0; U[i+j] = 1.0; }
            switch (narcs)
                {
                case 1: break;
                case 2: U [3] = U [4] = 0.5 ;
                                    break;
    case 3: U [3] = U [4] = 1.0/3.0 ;
                U[5] = U[6] = 2.0/3.0;
                break;
    case 4: U [3] = U [4] = 0.25 ;
                U[5] = U[6] = 0.5 ;
                U[7] = U[8] = 0.75 ;
                break;
    }
}
```

图 7.22a – 7.22 d 显示了使用一个、两个、三个和四个段的圆弧表示。

我们以一些关于 NURBS 圆的杂项备注结束这一部分。Piegl 和 Tiller [Pieg89b] 提出了另一种算法，用于构造任意扫掠角的圆弧。该算法基于使用无限控制点和负权重，然后通过节点插入来去除它们 (如示例 Ex7.4-Ex7.6 所示)。与算法 A7.1 相比，得到的圆弧可能需要更少的控制点和双重节点，但参数化通常不如前者好。还证明了，使用正权重的二次 NURBS 无法在没有双重内部节点的情况下表示完整的圆 [Pieg89b]。

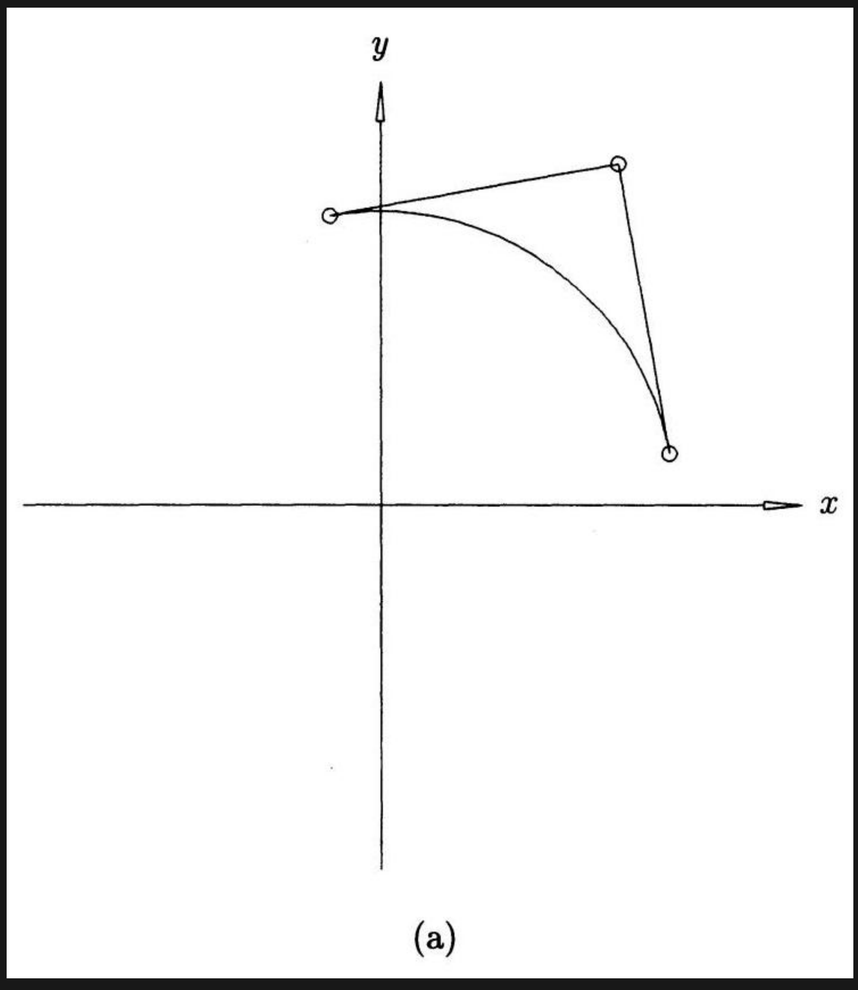


图 7.22. 不同扫掠角的圆弧。(a) $\theta_s = 10°, \theta_e = 100°$; (b) $\theta_s = 30°, \theta_e = 170°$; (c) $\theta_s = 20°, \theta_e = 250°$; (d) $\theta_s = 40°, \theta_e = 330°$.

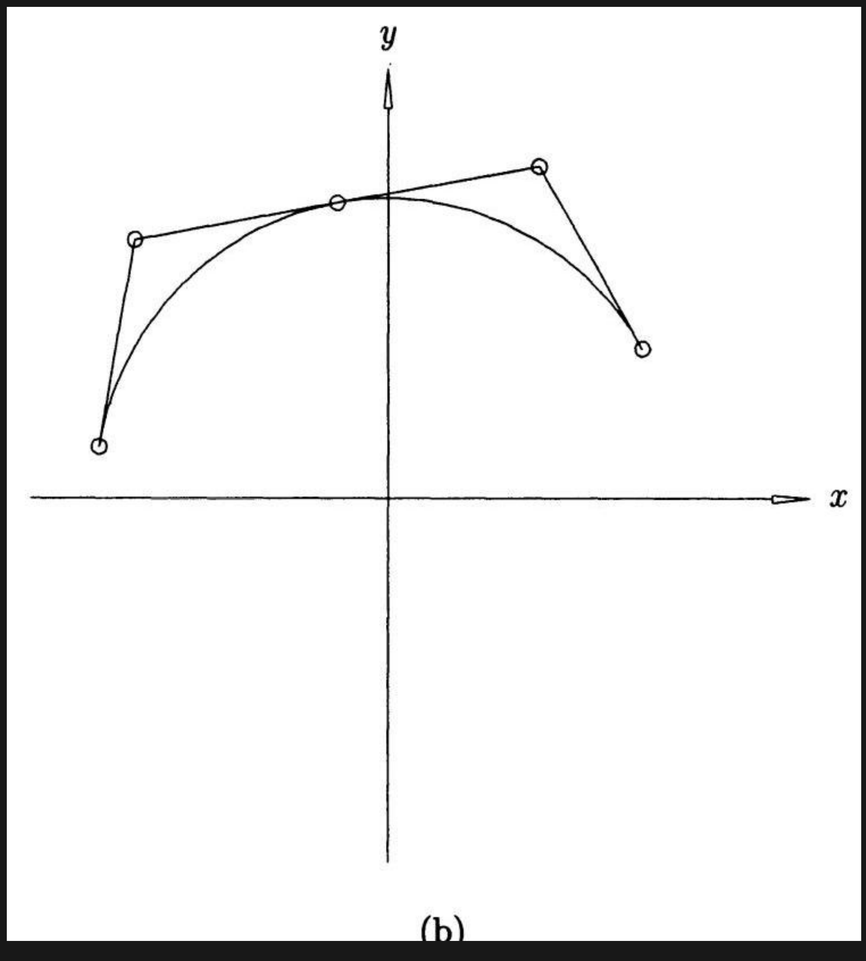


图 7.22. (续。)

6 圆锥曲线的构造

在这一部分中，我们开发了构造圆锥曲线的算法，仅讨论二次表示。抛物线和双曲线弧总是可以用一条有理 Bézier 曲线 (无内部节点) 和正权重来表示。与圆的情况一样，我们可能需要用内部节点将多个段拼接在一起，以仅使用正权重获得任意椭圆弧。在此过程中，问题再次涉及连续性、参数化、凸包和控制点的数量。

我们首先考虑三维空间中的任意开放圆锥弧。有许多方法可以指定圆锥弧。在 CADCAM 应用中，最常见的两种方法是:

- 定义几何参数，如半径、轴、焦距等，以及起点和终点的指定；这有效地导致通过方程 (7.15)-(7.17) 的表示；
- 起始点和终止点的规格，$\mathbf{P}_0$ 和 $\mathbf{P}_2$ ，以及这两个点的切线方向，$\mathbf{T}_0$ 和 $\mathbf{T}_2$ ，加上弧上的一个额外点 $\mathbf{P}$ (见图 7.23)。

如果数据以项目 1 中给出的形式可用，那么推导 $\mathbf{P}_0, \mathbf{T}_0$ 、$\mathbf{P}_2, \mathbf{T}_2$ 和 $\mathbf{P}$ 是很容易的；因此我们在本节中假设数据是按照项目 2 中指定的形式给出的。

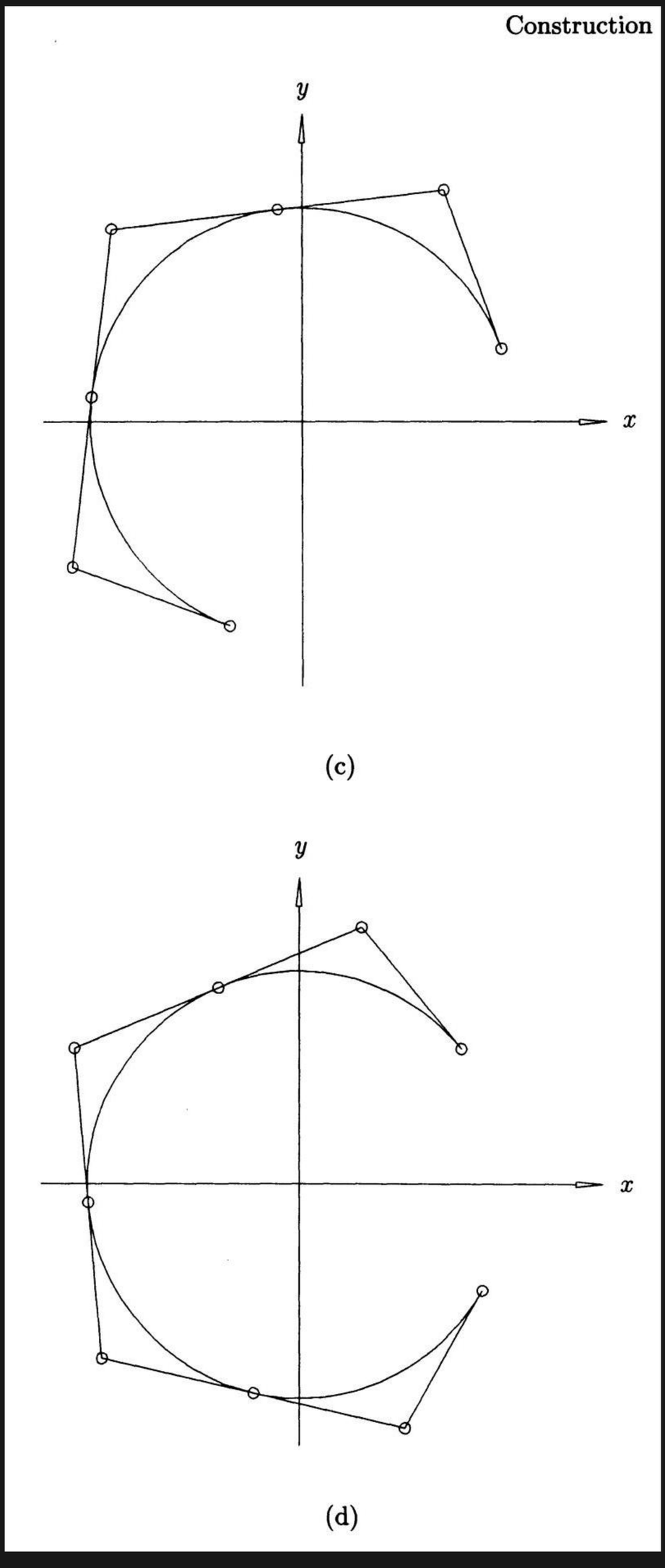


图 7.22.(续。)

$\mathbf{P}_1$ (图 7.23) 可以通过交叉线 $[\mathbf{P}_0, \mathbf{T}_0]$ 和 $[\mathbf{P}_2, \mathbf{T}_2]$ 容易获得。设置 $w_0 = w_2 = 1$ ，唯一缺失的项是 w_1 。允许 w_1 为负，或 $\mathbf{P}_1$ 为无穷大 (暂时)，我们可以用一个有理 Bézier 曲线获得任何圆锥弧。如果需要，我们可以在合适的位置将这个 Bézier 弧拆分，以获得正权重和多个具有良好参数化和凸包特性的段。

附加点 $\mathbf{P}$ 确定了圆锥曲线，因此 w_1 。将 $\mathbf{P} = \mathbf{C}(u)$ 代入方程 (7.21) 的左侧会产生两个未知数 u 和 w_1 的三个方程 (可以求解)。但是，几何论证提供了一种更高效的算法 (见 Piegl [Pieg87b])。首先假设 $\mathbf{P}_1$ 是有限的。我们所寻求的圆锥曲线可以视为由 $\mathbf{P}_0, \mathbf{P}_1$ 和 $\mathbf{P}_2$ 确定的抛物线的透视图，其中 $\mathbf{P}_1$ 是透视的中心。位于该三角形内的任何一对圆锥段都可以相互映射，包括线 $[\mathbf{P}_0, \mathbf{P}_2]$ (退化圆锥曲线) 映射到我们所寻求的圆锥曲线。因此，$\mathbf{P}$ 和 $\mathbf{Q}$ (图 7.23) 在此变换下是对应点。现在，通过设定 $w_1 = 0$ (不使用无限控制点) 来获得线 $\mathbf{L}(u) = [\mathbf{P}_0, \mathbf{P}_2]$ ，得出

$$\mathbf{L}(u)=\frac{(1-u)^2\mathbf{P}_0+u^2\mathbf{P}_2}{(1-u)^2+u^2} \tag{7.37}$$

$\mathbf{L}(u)$ 是 $\mathbf{P}_0$ 和 $\mathbf{P}_2$ 的凸组合，因此距离 $|\mathbf{P}_0\mathbf{Q}|$ 到 $|\mathbf{Q}\mathbf{P}_2|$ 的比率为 $u^2:(1-u)^2$ ，由此可得

$$u=\frac{a}{1+a}\ a=\sqrt{\frac{|\mathbf{P}_0\mathbf{Q}|}{|\mathbf{Q}\mathbf{P}_2|}} \tag{7.38}$$

(将图 7.23 和此论证与图 4.6 及方程 [4.4] 进行比较)。将 u 和 $\mathbf{P}$ 代入方程 (7.21)，我们可以轻松得到 w_1 ，即

$$w_1=\frac{(1-u)^2(\mathbf{P}-\mathbf{P}_0)\cdot(\mathbf{P}_1-\mathbf{P})+u^2(\mathbf{P}-\mathbf{P}_2)\cdot(\mathbf{P}_1-\mathbf{P})}{2u(1-u)|\mathbf{P}_1-\mathbf{P}|^2} \tag{7.39}$$

Piegl [Pieg87b] 使用射影几何的技术推导出 w_1 。

现在假设 $\mathbf{P}_1\left(\vec{\mathbf{P}}_1\right)$ 是无限的 ($\mathbf{T}_0$ 和 $\mathbf{T}_2$ 是平行的)。在这种情况下，$w_1=0$ 和 $\vec{\mathbf{P}}_1$ 与 $\mathbf{T}_0$ 是平行的。只有 $\vec{\mathbf{P}}_1$ 的大小是未知的。考虑图 7.24。透视中心现在在无穷远处，且 u 的获得方式与之前完全相同: $[\mathbf{P},\mathbf{T}_0]$ (即 $\left[\mathbf{P},\vec{\mathbf{P}}_1\right]$) 与 $[\mathbf{P}_0,\mathbf{P}_2]$ 相交，得到 $\mathbf{Q}$ ，然后从方程 (7.38) 计算 u 。对于具有 $w_0=w_2=1$ 的有理二次 Bézier 曲线，方程 (7.34) 简化为

$$\mathbf{C}(u)=\frac{(1-u)^2\mathbf{P}_0+u^2\mathbf{P}_2}{(1-u)^2+u^2}+\frac{2u(1-u)}{(1-u)^2+u^2}\vec{\mathbf{P}}_1=\overline{\mathbf{C}}(u)+f(u)\vec{\mathbf{P}}_1 \tag{7.40}$$

令 u_0 为参数，得到 $\mathbf{Q}$ 。然后

$$\mathbf{P}=\mathbf{Q}+f(u_0)\vec{\mathbf{P}}_1$$

和

$$\vec{\mathbf{P}}_1=\frac{1}{f(u_0)}(\mathbf{P}-\mathbf{Q})\ \ f(u_0)=\frac{2u_0(1-u_0)}{(1-u_0)^2+u_0^2} \tag{7.41}$$

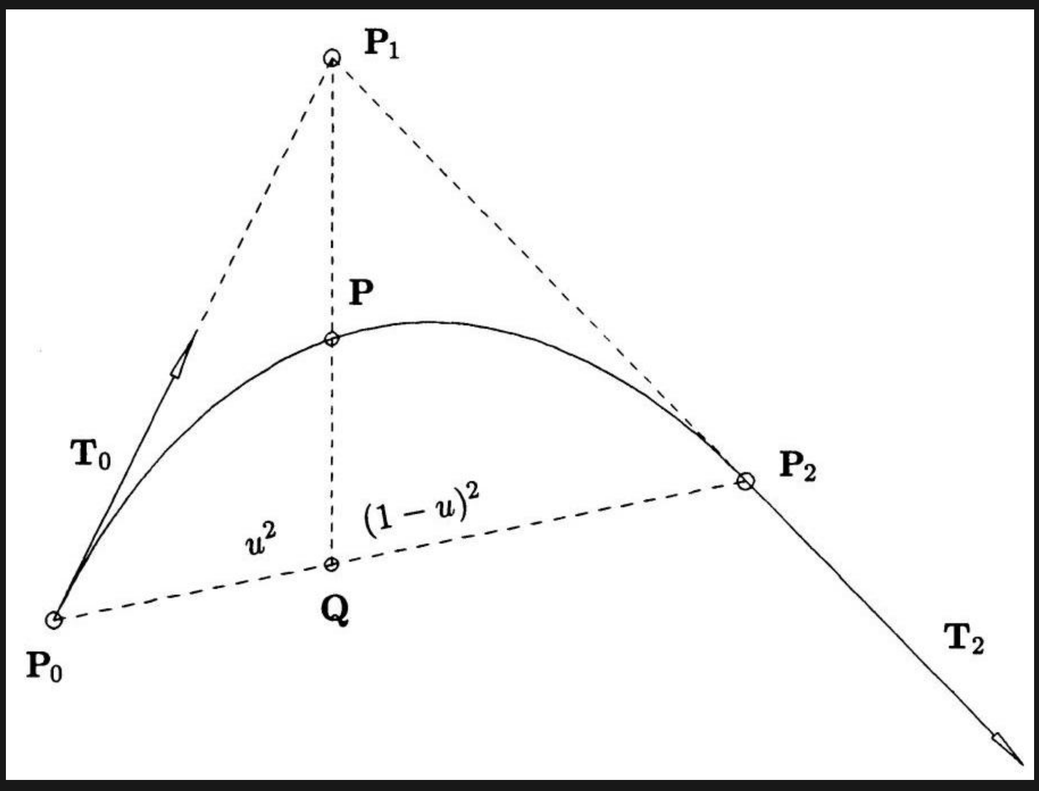


图 7.23。由端点、端切线和一个额外点定义的一般圆锥段，该点位于曲线上。

基于方程 (7.38)、(7.39) 和 (7.41)，我们现在给出构造一个有理 Bézier 圆锥弧的算法。由于 w_1 可以是负值或 $\mathbf{P}_1$ 无穷大，因此该算法处理任何圆锥弧，除了完整的椭圆。我们利用

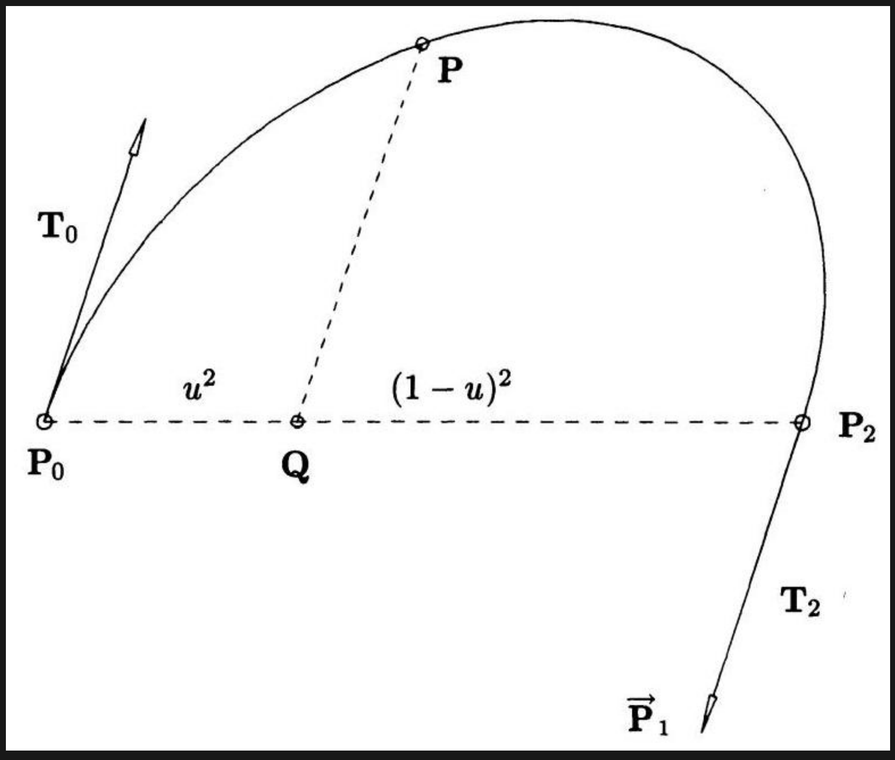


图 7.24。由端点、端切线和一个额外点定义的 180° 椭圆弧。

在 Intersect3DLines 中计算的 α_0,α_2 值。我们进一步假设一个函数 Dot (V1, V2)，它返回向量 $\mathbf{V}_1$ 和 $\mathbf{V}_2$ 的点积。

ALGORITHM A7.2

```
MakeOneArc(P0, T0, P2, T2, P, P1, w1)
    { /* Create one Bézier conic arc */
            /* Input: P0, T0, P2, T2, P */
            /* Output: P1, w1 */
    V02 = P2-P0;
    i = Intersect3DLines(P0, T0, P2, T2, dummy, dummy, P1);
    if (i == 0)
        { /* finite control point */
        V1P = P-P1;
        Intersect3DLines(P1, V1P, P0, V02, alf0, alf2, dummy);
      a = sqrt(alf2/(1.0-alf2));
      u = a/(1.0+a);
      num = (1.0-u)*(1.0-u)*Dot(P-P0, P1-P) + u*u*Dot(P-P2, P1-P);
      den = 2.0 * u * (1.0 − u) * Dot (P1 − P, P1 − P) ;
```

```
        w1 = num/den;
        return;
        }
          else
        { /* infinite control point, 180 degree arc */
        w1 = 0.0 ;
        Intersect3DLines(P, TO, PO, VO2, alf0, alf2, dummy);
        a = sqrt(alf2/(1.0-alf2));
        u = a/(1.0+a);
        b = 2.0*u*(1.0-u);
        b = -alf0*(1.0-b)/b;
        P1 = b * T0 ;
        return;
        }
    }
```

算法 A7.2 适用于抛物线和双曲线弧，以及椭圆弧，其中 $w_1 > 0$ 且其扫掠角度不太大。将椭圆分割成段并不像圆的情况那样简单。我们不知道长轴和短轴及其半径，并且没有可用的形式为 Eq. (7.36) 的方程。一个方便的分割点是肩点 $\mathbf{S}$ 。为了分割弧 $\mathbf{P}_0\mathbf{P}_1\mathbf{P}_2$ ，我们使用有理 de-Casteljau 算法，Eq. (1.19)。分割椭圆有两个步骤 (见图 7.25a-7.25c):

1. 在 $u = 1/2$ 处分割。使用 deCasteljau 算法获得

$$\mathbf{Q}_1^w = \frac{1}{2}\mathbf{P}_0^w + \frac{1}{2}\mathbf{P}_1^w \; \mathbf{R}_1^w = \frac{1}{2}\mathbf{P}_1^w + \frac{1}{2}\mathbf{P}_2^w$$

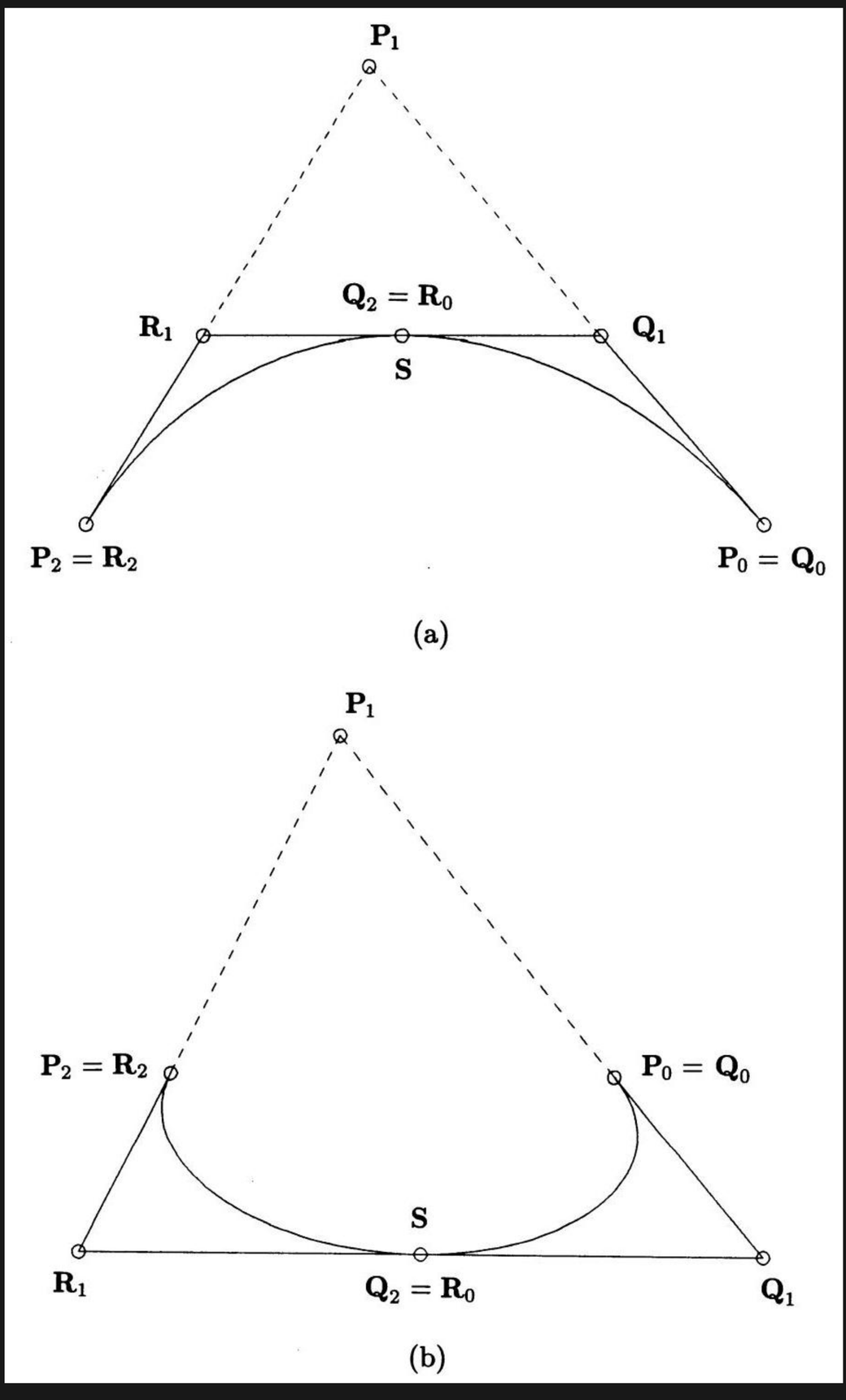


图 7.25. 分割锥曲线。(a) 扫掠小于 180° 的弧；(b) 扫掠大于 180° 的弧；(c) 扫掠 180° 的弧。

并且回忆起 $w_0 = w_2 = 1$ ，因此得出

$$\mathbf{Q}_1 = \frac{\mathbf{P}_0 + w_1\mathbf{P}_1}{1 + w_1} \; \mathbf{R}_1 = \frac{w_1\mathbf{P}_1 + \mathbf{P}_2}{1 + w_1} \tag{7.42}$$

和

$$w_q = w_r = \frac{1}{2}(1 + w_1) \tag{7.43}$$

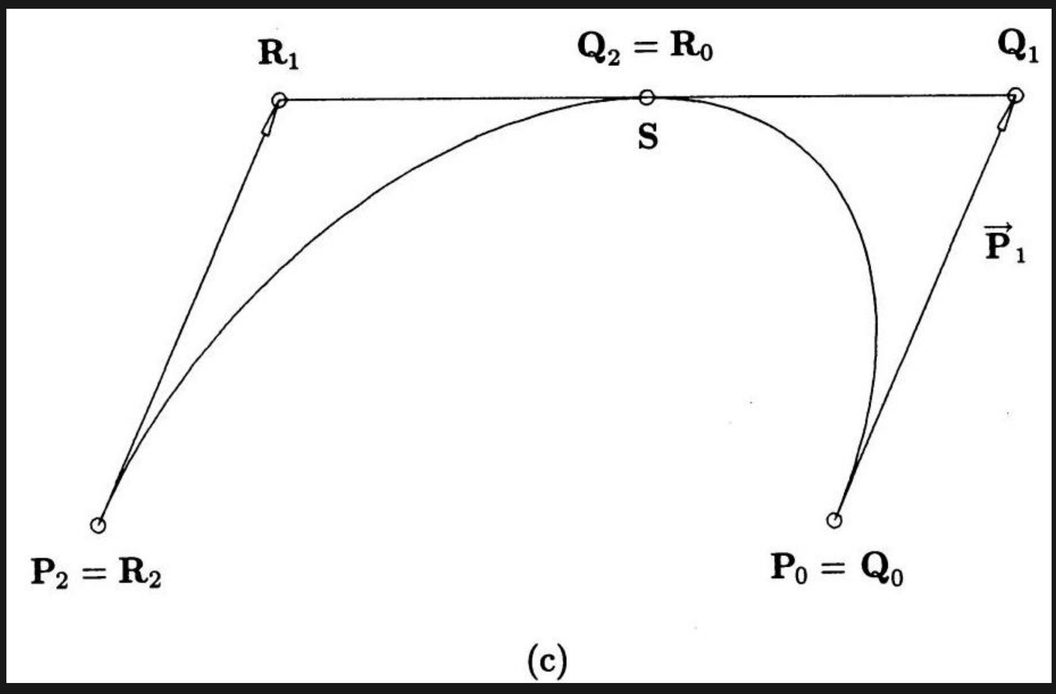


图 7.25. (续)。

其中 w_q 和 w_r 分别是在 $\mathbf{Q}_1$ 和 $\mathbf{R}_1$ 的权重。第二次应用 deCasteljau 算法得到

$$\mathbf{R}_0 = \mathbf{Q}_2 = \mathbf{S} = \frac{1}{2}(\mathbf{Q}_1 + \mathbf{R}_1) \tag{7.44}$$

和

$$w_s = \frac{1}{2}(1 + w_1) \tag{7.45}$$

2. 重新参数化，使得两个新段的末端权重均为 1。分割后，第一个段的权重为

$$w_0 = 1 \; w_q = \frac{1}{2}(1 + w_1) \; w_s = \frac{1}{2}(1 + w_1)$$

我们希望权重为

$$w_0 = 1 \; w_{q_1} \; w_{q_2} = 1$$

其中 w_{q_1} 需要确定。使用锥形形状因子 (Eq. [6.59])

$$\frac{w_0 w_s}{w_q^2} = \frac{w_0 w_{q_2}}{w_{q_1}^2}$$

这意味着

$$w_{q_1} = \sqrt{\frac{1 + w_1}{2}} \tag{7.46}$$

由于对称性，我们也有

$$w_{r_1} = \sqrt{\frac{1 + w_1}{2}} \tag{7.47}$$

图 7.25a 和 7.25 b 说明了带有正负权重的椭圆分裂。前述过程同样适用于无限控制点 (见图 7.25c)；然而，得到的公式

$$\mathbf{Q}_1 = \mathbf{P}_0 + \vec{\mathbf{P}}_1 \; \mathbf{R}_1 = \mathbf{P}_2 + \vec{\mathbf{P}}_1 \tag{7.48}$$

$$\mathbf{R}_0 = \mathbf{Q}_2 = \mathbf{S} = \frac{1}{2}(\mathbf{Q}_1 + \mathbf{R}_1) \tag{7.49}$$

$$w_{q_1} = w_{r_1} = \frac{\sqrt{2}}{2} \tag{7.50}$$

更加简单。方程 (7.50) 不应令人惊讶，因为图 7.25c 的半椭圆是通过对半圆应用仿射变换 (具体来说，是非均匀缩放) 获得的。这种变换不会改变权重。我们留给读者一个练习，编写一个例程 SplitArc(P0, P1, w1, P2, Q1, S, R1, wqr)，该例程实现方程 (7.42)-(7.50)。

利用算法 A7.2 和 SplitArc( )，我们现在提出一个算法，该算法在三维空间中构造任意开放的圆锥弧。生成的 NURBS 曲线由一个、两个或四个段组成，这些段以 C^1 连续性连接。输出为节点 (U)、控制点数减去 1 (n) 和以齐次形式表示的控制点 (Pw)。我们假设有一个工具函数 Angle(P, Q, R)，它返回角度 $\angle PQR$ 。

算法 A7.3

```
MakeOpenConic(PO, TO, P2, T2, P, n, U, Pw)
        { /* Construct open conic arc in 3D */
            /* Input: P0, T0, P2, T2, P */
            /* Output: n, U, Pw */
    MakeOneArc(PO, TO, P2, T2, P, P1, w1);
    if (w1 <= -1.0) /* parabola or hyperbola */
            return(error); /* outside convex hull */
    if (w1 >= 1.0) /* classify type &number of segments */
            nsegs = 1; /* hyperbola or parabola, one segment */
            else
            { /* ellipse, determine number of segments */
            if (w1 > 0.0&& Angle (P0, P1, P2) > 60.0) nsegs = 1 ;
            else
            if (w1 < 0.0&& angle (P0, P1, P2) > 90.0) nsegs = 4 ;
                else nsegs = 2 ;
          }
    n = 2*nsegs;
    j = 2*nsegs+1;
    for (i=0; i<3; i++) /* load end knots */
            { U[i] = 0.0; U[i+j] = 1.0; }
    Pw[0] = P0; Pw[n] = P2; /* load end ctrl pts */
    if (nsegs == 1)
  {
    Pw [1] = w1*P1 ;
    return;
    }
  SplitArc(PO, P1, w1, P2, Q1, S, R1, wqr);
  if (nsegs == 2)
    {
    Pw [2] = S ;
    Pw[1] = wqr*Q1; Pw[3] = wqr*R1;
    U[3] = U[4] = 0.5 ;
    return;
    }
    /* nsegs == 4 */
  Pw [4] = S ;
  w1 = wqr;
  SplitArc(PO, Q1, w1, S, HQ1, HS, HR1, wqr);
  Pw [2] = HS ;
  Pw[1] = wqr*HQ1; Pw[3] = wqr*HR1;
  SplitArc(S, R1, w1, P2, HQ1, HS, HR1, wqr);
  Pw[6] = HS;
```

Pw[5] = wqr*HQ1; Pw[7] = wqr*HR1;
for ($i = 0; i < 2; i++$) /* load the remaining knots */
{
U [i + 3] = 0.25; U [i + 5] = 0.5; U [i + 7] = 0.75 ;
}
return;
}

图 7.26a 和 7.26b 分别展示了使用两个和四个段的示例。

算法 A7.3 生成具有良好参数化的椭圆。对于参数化不被认为重要的应用，可以使用一个更简单的算法，该算法生成 C^2 连续曲线，并具有同样良好的凸包 (见 [Pieg90]):

1. 调用 MakeOneArc(算法 A7.2)；
2. 在适当的位置插入节点，以切割角落并消除负权重或无限控制点 (例如，在 $u = 1/2$ 、$u = 1/3$ 和 $u = 2/3$) 处)。

为了进行比较，图 7.26c 显示了一个椭圆弧，该弧是使用

- 算法 A7.3(由 □ 标记的点)；
- MakeOneArc，随后插入 $u = 1/2$ 一次 (由 + 标记的点)。

仍然存在完整椭圆的问题。如果已知长轴和短轴及其半径，则适合使用矩形控制多边形。权重和

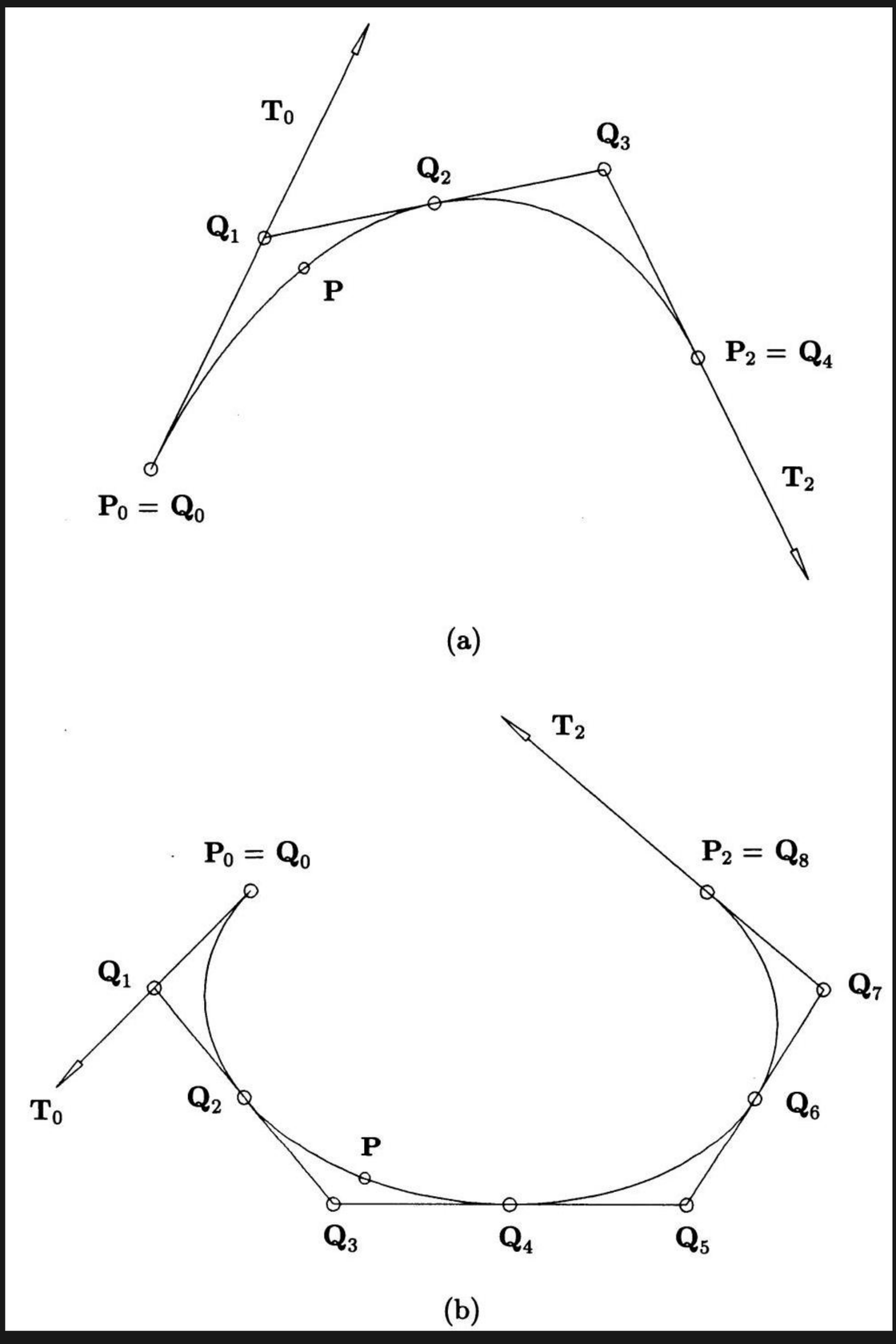


图 7.26. NURBS 表示的锥形段。(a) 弧的扫掠小于 180° ；(b) 弧的扫掠大于 180° ；(c) 通过曲线分割 (□) 和节点插入 (+) 获得的表示之间的比较。

节点与示例 Ex7.2(图 7.16a) 中的圆的节点相同，或者可以使用与示例 Ex7.6(图 7.21a) 类似的配置。

如果输入数据为 $\mathbf{P}_0, \mathbf{T}_0, \mathbf{P}_2, \mathbf{T}_2, \mathbf{P}$ ，并且知道锥形是完整的椭圆 (起始和结束于 $\mathbf{P}_0$)，那么适当的构造为:

1. 调用 MakeOneArc 获取椭圆一个段的有理 Bézier 表示 (权重为正的 w_1)；

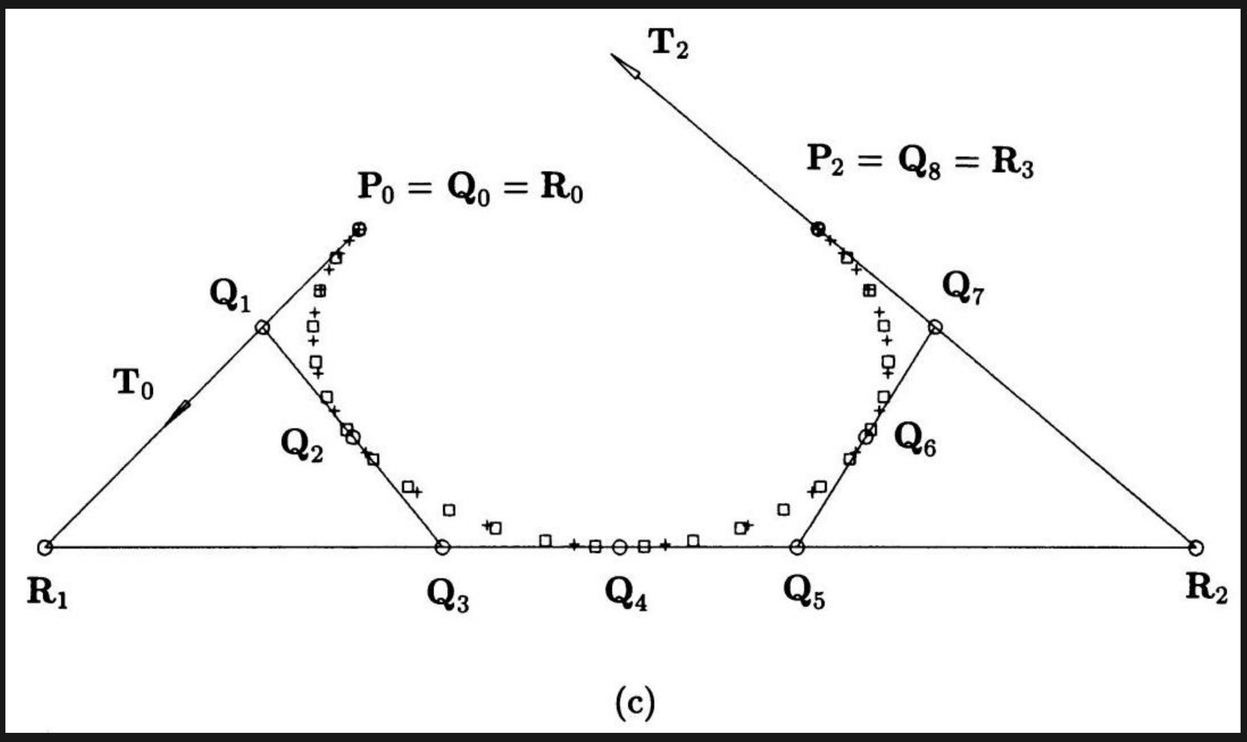


图 7.26.(续)

2. 计算椭圆的中心 $\mathbf{C}$；

3. 计算长轴和短轴，(单位) 向量 (U, V)，以及长半径和短半径 (r_1, r_2)；

4. 计算控制点

$$\mathbf{Q}_0 = \mathbf{C} + r_1\mathbf{U} \ (= \mathbf{Q}_8)$$

$$\mathbf{Q}_1 = \mathbf{Q}_0 + r_2\mathbf{V}$$

$$\mathbf{Q}_2 = \mathbf{C} + r_2\mathbf{V}$$

$$\mathbf{Q}_3 = \mathbf{Q}_2 - r_1\mathbf{U}$$

$$\mathbf{Q}_4 = \mathbf{C} - r_1\mathbf{U}$$

$$\mathbf{Q}_5 = \mathbf{Q}_4 - r_2\mathbf{V}$$

$$\mathbf{Q}_6 = \mathbf{C} - r_2\mathbf{V}$$

$$\mathbf{Q}_7 = \mathbf{Q}_6 + r_1\mathbf{U}$$

步骤 2 和 3 的详细信息在下一节中给出，结果在图 7.28 中进行了说明。

7　锥形分类与形式转换

在前面的章节中，我们介绍了五种表示锥曲线的形式:

- 隐式方程 (方程 [7.3] 和 [7.5]-[7.7])；
- 最大内切面积形式 (方程 [7.15]-[7.17])；
- 二次有理幂基 (方程 [7.8]-[7.14])；
- 二次有理贝塞尔 (方程 [7.21])；
- 二次 NURBS 形式。

在本节中，我们讨论锥曲线类型分类与形式转换，即给定一种形式的曲线，确定它是什么类型的锥曲线——椭圆、抛物线或双曲线——并将其转换为另一种形式。当我们提到隐式形式时，我们假设锥曲线位于 xy 平面；否则，锥曲线可以位于三维空间中。

考虑前两种形式。如果锥曲线处于标准位置，并且可以用方程 (7.5)-(7.7) 之一来描述，则其类型显而易见。如果它不在标准位置 (方程 [7.3])，则其类型由 α 和 D 的值 (行列式) 决定，如第 7.2 节所述，并且可以很容易地转换为标准位置。对于最大内切面积形式，锥曲线类型在定义中是固有的。请注意，最大面积形式与标准位置隐式形式非常相似，因为它们的定义方程是通过锥曲线的所谓几何特征 (即轴、顶点、中心、半径等) 给出的。因此，这两种形式之间的转换是微不足道的。

二次有理幂基 (公式 [7.14]) 与贝塞尔形式之间的转换由第六章的方法给出，特别是公式 (6.89) 和 (6.93)。它们的有理对应物是

$$[\mathbf{a}_i^w] = R_2 M_2 [\mathbf{P}_i^w] \tag{7.51}$$

和

$$[\mathbf{P}_i^w] = M_2^{-1} R_2^{-1} [\mathbf{a}_i^w] \tag{7.52}$$

其中 M_2 和 R_2 分别是二次 (3×3) 贝塞尔和重新参数化矩阵。贝塞尔圆锥的类型可以从圆锥形状因子中轻松获得，见公式 (7.25)。幂基圆锥的类型由公式 (7.52) 和 (7.25) 计算。为简单起见，假设我们的幂基段定义在 $0 \le u \le 1$ 上，且 R_2 为单位矩阵。用 w_i^a 表示幂基权重 (公式 [7.14])，用 w_i^P 表示贝塞尔权重。然后从公式 (6.94) 和 (7.52) 中我们发现

$$\left[w_i^P\right] = M_2^{-1} \left[w_i^a\right] = \begin{bmatrix} 1 & 0 & 0 \\ 1 & \frac{1}{2} & 0 \\ 1 & 1 & 1 \end{bmatrix} \left[w_i^a\right]$$

从中可以得出

$$w_0^P = w_0^a$$

$$w_1^P = w_0^a + \frac{1}{2} w_1^a$$

$$w_2^P = w_0^a + w_1^a + w_2^a \tag{7.53}$$

当代入公式 (7.25) 时，这些 w_i^P 产生了圆锥类型。

带有内部节点的二次 NURBS 曲线可能表示一个唯一的圆锥 (它在每个跨度上是一个圆锥、直线或点)。二次 NURBS 曲线的分类是一个三步过程:

1. 使用公式 (6.62)，计算每个非退化段的圆锥形状因子；
2. 如果所有形状因子指示一个共同类型 (所有 $c_i < 1, = 1$ 或 > 1)，则将曲线分解为贝塞尔段；
3. 计算每个段的几何特征并进行比较。

注意，所有形状因子可以指示一种共同类型，实际上它们甚至可能相等，但曲线仍然可能不是唯一的圆锥曲线。例如，回想一下，圆弧的权重仅依赖于扫角 (方程 [7.33])；因此，可以拼接不同半径的圆弧，但它们具有相等的扫角和形状因子。

基于上述讨论，出于分类和转换的目的，这五种形式可以分为两组:

1. 隐式和最大内接面积形式；
2. 二次有理幂基、贝塞尔和 NURBS 形式。

如果有某些数据，圆锥表示在一个组内是容易获得的，即

- 几何特征 (组 1)；
- 起始点和结束点，以及它们的切线方向和一个附加点 $\mathbf{P}_0, \mathbf{T}_0, \mathbf{P}_2, \mathbf{T}_2, \mathbf{P}$ (组 2)。

使用算法 A7.1-A7.3，从组 1 的形式转换到组 2 的形式是简单的，因为它仅涉及从方程 (7.3) 和 (7.5)-(7.7) 或 (7.15)-(7.17) 中计算点和切线。从组 2 转换到组 1 则需要更多的工作。Beach [Beac91] 推导了从有理幂基转换到隐式形式的公式 (方程 [7.3])。我们在这里展示如何从有理贝塞尔形式转换到方程 (7.3)，以及如何从其有理贝塞尔形式计算圆锥的几何特征。

设 $\mathbf{C}(u)$ 为由方程 (7.21) 给出的有理贝塞尔圆锥。回顾方程 (7.22) 到 (7.26) 和图 7.11

$$\alpha(u)\,\beta(u) = \frac{1}{4}k(1-\alpha(u)-\beta(u))^2 \tag{7.54}$$

是由 $\{\mathbf{P}_1, \mathbf{S}, \mathbf{T}\}$ 给出的斜坐标系中圆锥的隐式方程，其中

$$k = \frac{w_0 w_2}{{w_1}^2}\ \alpha(u) = R_{0,2}(u)\ \beta(u) = R_{2,2}(u)$$

为了从方程 (7.54) 得到方程 (7.3)，我们需要将 α 和 β 表示为笛卡尔坐标 x 和 y 的函数。方程 (7.20) 表示 $\alpha(u)$ 和 $\beta(u)$ 是三角形 β 中点 α 的重心坐标。

固定一个任意的 u_0 ，设 $\alpha = \alpha(u_0), \beta = \beta(u_0)$ 和 $\mathbf{P} = \mathbf{C}(u_0) = (x, y)$ 。然后

$$\alpha = \frac{\text{area}(\mathbf{P}, \mathbf{P}_1, \mathbf{P}_2)}{\text{area}(\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2)}\ \beta = \frac{\text{area}(\mathbf{P}, \mathbf{P}_0, \mathbf{P}_1)}{\text{area}(\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2)}$$

从中可以得出

$$\alpha = \frac{(y-y_1)(x_2-x_1)-(x-x_1)(y_2-y_1)}{(y_0-y_1)(x_2-x_1)-(x_0-x_1)(y_2-y_1)}$$

$$\beta = \frac{(y-y_1)(x_0-x_1)-(x-x_1)(y_0-y_1)}{(y_2-y_1)(x_0-x_1)-(x_2-x_1)(y_0-y_1)} \tag{7.55}$$

现在设

$$h_0 = x_0 - x_1\ h_1 = x_2 - x_1$$

$$h_2 = y_0 - y_1\ h_3 = y_2 - y_1$$

和

$$g_1 = h_2 h_1 - h_0 h_3$$

$$\text{Then } \alpha = \frac{h_1(y-y_1) - h_3(x-x_1)}{g_1}\ \beta = \frac{h_0(y-y_1) - h_2(x-x_1)}{-g_1} \tag{7.56}$$

最后，设定 $s_1 = h_0 - h_1$ 和 $s_2 = h_3 - h_2$ ，并将方程 (7.56) 代入方程 (7.54)，得到方程 (7.3) 的六个系数，即。

$$a = s_2^2 + \frac{4}{k}h_2 h_3$$

$$b = s_1^2 + \frac{4}{k}h_0 h_1$$

$$h = s_1 s_2 - \frac{2}{k}(h_0 h_3 + h_1 h_2)$$

$$f = -hy_1 - ax_1 + g_1 s_2$$

$$g = -hx_1 - by_1 + g_1 s_1$$

$$c = by_1^2 + 2hx_1 y_1 + ax_1^2 - 2g_1(y_1 s_1 + x_1 s_2) + g_1^2 \tag{7.57}$$

现在我们转向从有理贝塞尔形式计算几何特征。我们在此提供公式而不作证明: 虽然不难，但推导过程较为复杂。有关证明，我们建议读者参考 Lee 的优雅文章 [Lee87]，该文是本材料的来源。

我们给定一个由方程 (7.21) 定义的二次有理贝塞尔圆锥，且所有 $w_i > 0$ 。我们假设该圆锥不是退化的。我们定义符号

$$\mathbf{S} = \mathbf{P}_0 - \mathbf{P}_1\ \mathbf{T} = \mathbf{P}_2 - \mathbf{P}_1$$

$$k = \frac{w_0 w_2}{w_1^2}\ \epsilon = \frac{k}{2(k-1)}$$

$$\alpha = |\mathbf{S}|^2\ \beta = \mathbf{S}\cdot\mathbf{T}\ \gamma = |\mathbf{T}|^2$$

$$\delta = \alpha\gamma - \beta^2 = |\mathbf{S}\times\mathbf{T}|^2$$

$$\zeta = \alpha + \gamma + 2\beta = |\mathbf{S}+\mathbf{T}|^2$$

$$\eta = \alpha + \gamma - 2\beta = |\mathbf{S}-\mathbf{T}|^2$$

对于抛物线，我们有以下公式

$$\text{Axis}: \frac{1}{\sqrt{\zeta}}(\mathbf{S}+\mathbf{T}) \tag{7.58}$$

$$\text{Focus}: \mathbf{P}_1 + \frac{\gamma\mathbf{S} + \alpha\mathbf{T}}{\zeta} \tag{7.59}$$

$$\text{Vertex}: \mathbf{P}_1 + \left(\frac{\gamma+\beta}{\zeta}\right)^2\mathbf{S} + \left(\frac{\alpha+\beta}{\zeta}\right)^2\mathbf{T} \tag{7.60}$$

椭圆或双曲线的中心由以下公式给出

$$\text{Center}: \mathbf{P}_1 + \epsilon(\mathbf{S}+\mathbf{T}) \tag{7.61}$$

设 $\lambda_1 \le \lambda_2$ 为二次方程的解

$$2\delta\lambda^2 - (k\eta + 4\beta)\lambda + 2(k-1) = 0$$

该方程的根是实数。对于椭圆，$\lambda_2 \ge \lambda_1 > 0$ 和 $\epsilon > 0$ 。对于双曲线，$\lambda_1 < 0 < \lambda_2$ 和 $\epsilon < 0$ 。长半轴和短半轴为

$$\text{Ellipse}: r_1 = \sqrt{\frac{\epsilon}{\lambda_1}}\ r_2 = \sqrt{\frac{\epsilon}{\lambda_2}} \tag{7.62}$$

$$\text{Hyperbola}: r_1 = \sqrt{\frac{\epsilon}{\lambda_1}}\ r_2 = \sqrt{\frac{-\epsilon}{\lambda_2}} \tag{7.63}$$

定义如下

$$\text{if } \left|\frac{k}{2} - \gamma\lambda_1\right| > \left|\frac{k}{2} - \alpha\lambda_1\right|$$

$$\bar{x} = \frac{k}{2} - \gamma\lambda_1\ \bar{y} = \beta\lambda_1 - \frac{k}{2} + 1$$

$$\text{else } \bar{x} = \beta\lambda_1 - \frac{k}{2} + 1\ \bar{y} = \frac{k}{2} - \alpha\lambda_1$$

和

$$\rho = \alpha\bar{x}^2 + 2\beta\bar{x}\bar{y} + \gamma\bar{y}^2$$

$$x_0 = \frac{\bar{x}}{\rho}\ y_0 = \frac{\bar{y}}{\rho}$$

那么点 $\mathbf{Q}_1$ 和 $\mathbf{Q}_2$

$$\mathbf{Q}_1 = \mathbf{P}_1 + (\epsilon + r_1 x_0)\mathbf{S} + (\epsilon + r_1 y_0)\mathbf{T}$$

$$\mathbf{Q}_2 = \mathbf{P}_1 + (\epsilon - r_1 x_0)\mathbf{S} + (\epsilon - r_1 y_0)\mathbf{T} \tag{7.64}$$

位于椭圆的长轴上或双曲线的横轴上。另一个正交轴可以很容易地获得，椭圆或双曲线的顶点是根据中心、半径和轴的方向计算得出的。

注意，对于圆，它是椭圆的特例，方程 (7.62) 产生 $r_1 = r_2$ 。算法应检查此情况，因为将方程 (7.64) 应用于圆是没有意义的。

图 7.27a – 7.27c 说明了这些公式。图 7.27a 显示了一个抛物线弧及其顶点 (V)、焦点 (F) 和轴向量 (A)。在图 7.27 b 中描绘了一个椭圆弧，包含中心 (C)、负长轴 (-U) 和短轴 (V)。图 7.27c 显示了双曲线的情况。同样，中心 (C)、长轴 (U) 和短轴 (V) 被计算出来。该图还展示了如何计算弧的渐近线 $\mathcal{A}_1$ 和 $\mathcal{A}_2$ ，即在无穷远处与曲线相切的两条线。

最后，图 7.28 显示了如何计算给定 Bézier 形式弧的完整椭圆的 NURBS 表示 (参见上一节末尾给出的四个步骤)。

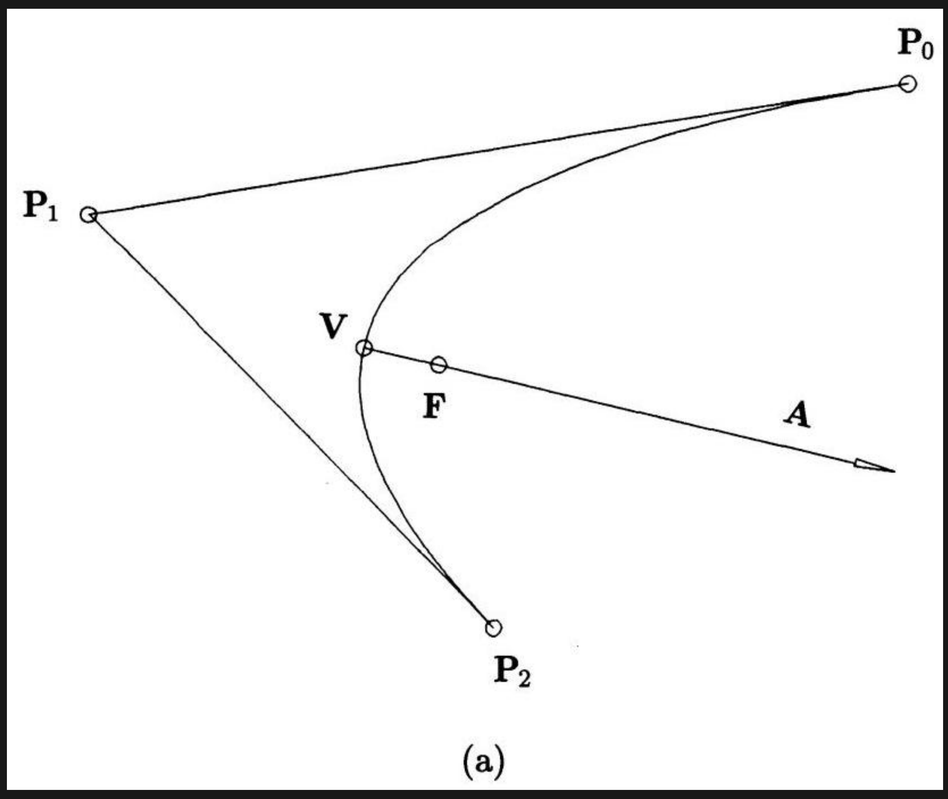


图 7.27. 从贝塞尔表示法计算的几何数据。(a) 一个顶点 (V)、焦点 (F) 和轴 (A) 的抛物线；(b) 一个中心 (C)、负主轴 (-U) 和副轴 (V) 的椭圆；(c) 一个中心 (C)、主轴 (U)、副轴 (V) 和两个渐近线 ($\mathcal{A}_1$ 和 $\mathcal{A}_2$) 的双曲线。

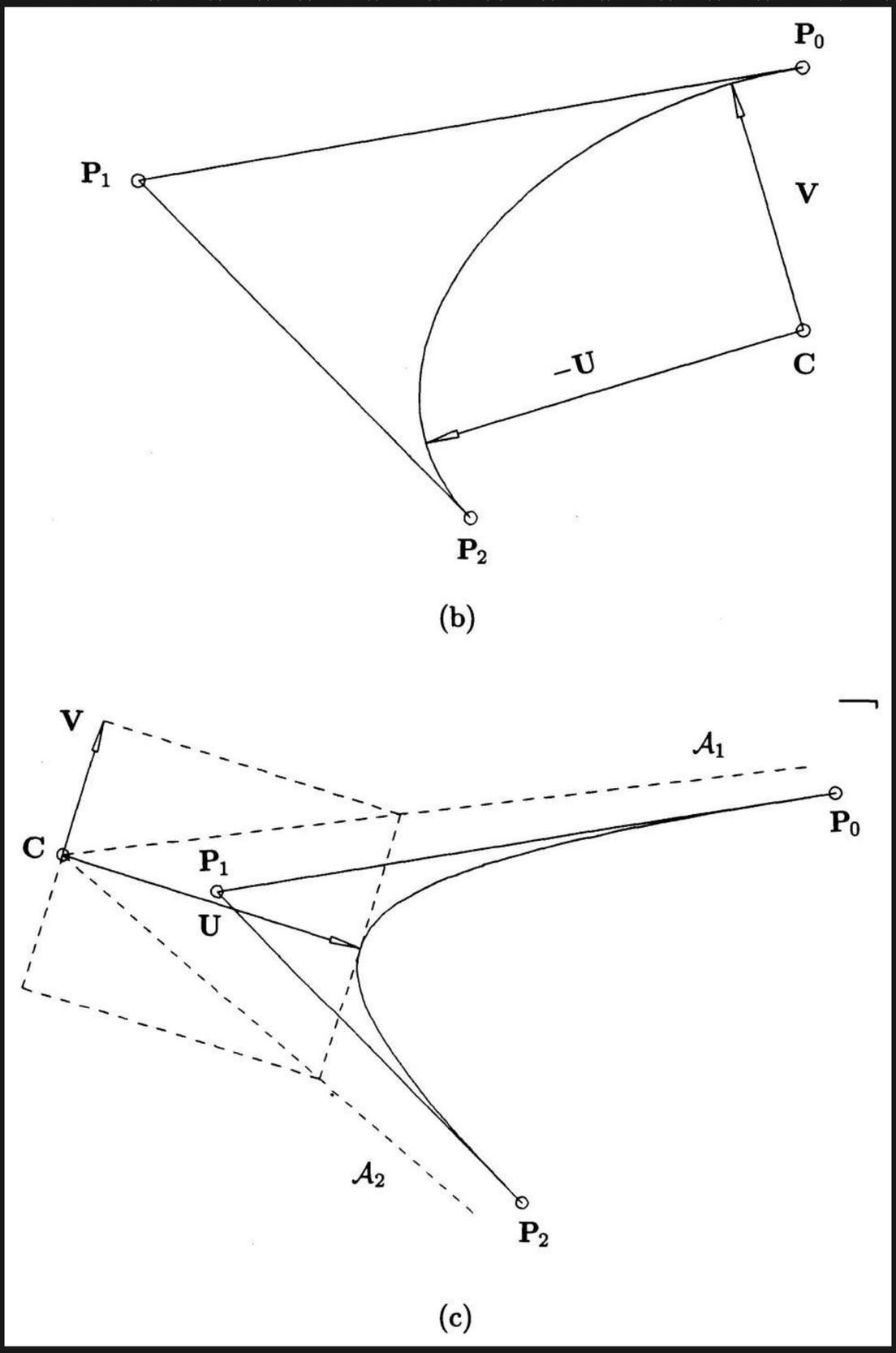


图 7.27.(续)

8 高阶圆

在某些应用中，使用一个有理贝塞尔段 (没有内部节点)、没有无限控制点和没有负权重来表示完整的圆或扫角大于 180° 的圆弧是有用的。在前面的部分中，我们通过使用节点插入消除了无限控制点和负权重。度提升也能达到这个目的。以下是几个例子:

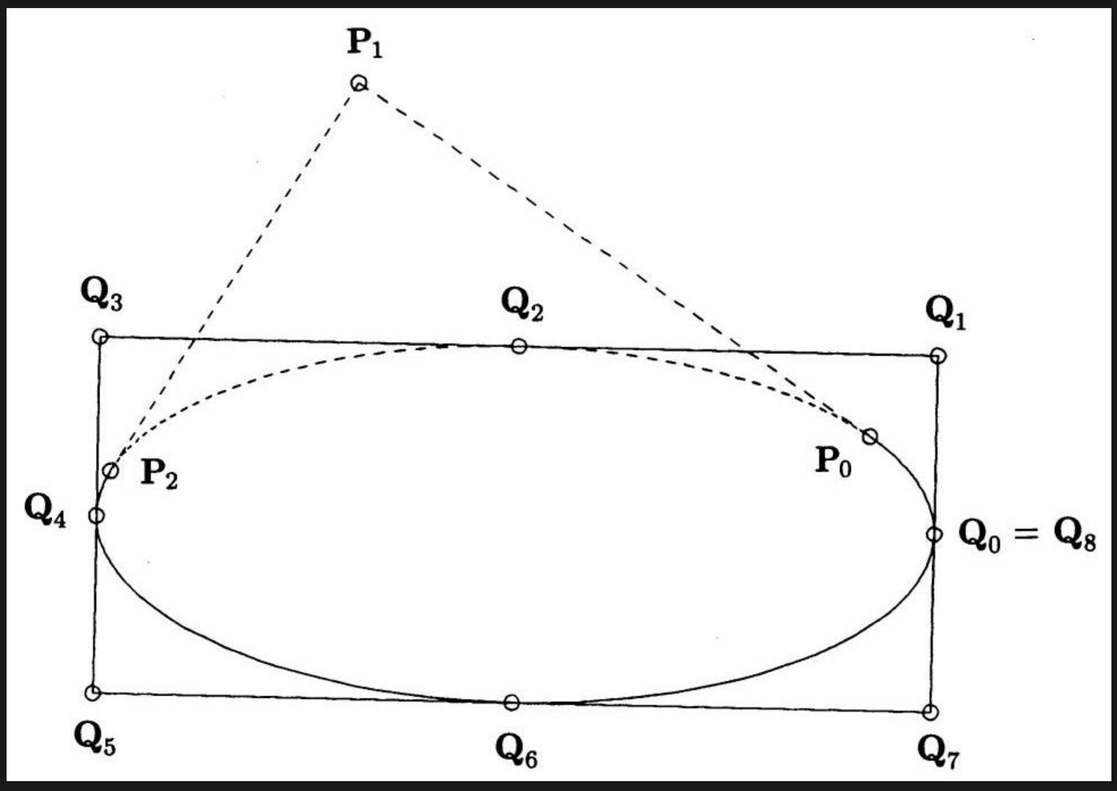


图 7.28. 以贝塞尔形式给出的完整椭圆的 NURBS 表示 (虚线)。

示例

示例 Ex7.7 示例 Ex7.1 的半圆可以进行度提升以得到

$$\mathbf{Q}_0^w = \mathbf{P}_0^w \; \mathbf{Q}_3^w = \mathbf{P}_2^w$$

$$\mathbf{Q}_1^w = \frac{1}{3}\mathbf{P}_0^w + \frac{2}{3}\mathbf{P}_1^w = \left(\frac{1}{3}, \frac{2}{3}, \frac{1}{3}\right)$$

$$\mathbf{Q}_2^w = \frac{2}{3}\mathbf{P}_1^w + \frac{1}{3}\mathbf{P}_2^w = \left(-\frac{1}{3}, \frac{2}{3}, \frac{1}{3}\right)$$

因此 (见图 7.29)

$$U = \{0, 0, 0, 0, 1, 1, 1, 1\}$$

$$\{w_i\} = \left\{1, \frac{1}{3}, \frac{1}{3}, 1\right\}$$

$$\{\mathbf{Q}_i\} = \{(1, 0), (1, 2), (-1, 2), (-1, 0)\}$$

示例 Ex7.8 示例 Ex7.4 中给出的弧 240° 经过度提升以得到 (见图 7.30):

$$U = \{0, 0, 0, 0, 1, 1, 1, 1\}$$

$$\{w_i\} = \left\{1, \frac{1}{6}, \frac{1}{6}, 1\right\}$$

$$\{\mathbf{Q}_i\} = \left\{\left(a, \frac{1}{2}\right), (2a, -3), (-2a, -3), \left(-a, \frac{1}{2}\right)\right\}$$

与 $a = \sqrt{3}/2$. 328 圆锥曲线和圆
以下的四阶和五阶完整圆构造源于 Chou [Chou95] 和 Strotman [Stro91]。
Ex7.9 我们在复平面上写出半圆 (半径为 1) 为

$$z = e^{i\pi u} = f(u) = g(u) + h(u)\, i \; 0 \le u \le 1 \tag{7.65}$$

对方程 (7.65) 进行平方，我们得到完整的圆

$$e^{i2\pi u} = \left(g^2 - h^2\right) + (2gh)\, i \; 0 \le u \le 1 \tag{7.66}$$

从例子 Ex7.1 我们知道

$$g(u) = \frac{1 - 2u}{1 - 2u + 2u^2} \; h(u) = \frac{2u(1 - u)}{1 - 2u + 2u^2} \tag{7.67}$$

将方程 (7.67) 代入方程 (7.66) 并返回到实 xy 平面，我们得到

$$x(u) = \frac{1 - 4u + 8u^3 - 4u^4}{1 - 4u + 8u^2 - 8u^3 + 4u^4}$$

$$y(u) = \frac{4u\left(1 - 3u + 2u^2\right)}{1 - 4u + 8u^2 - 8u^3 + 4u^4} \tag{7.68}$$

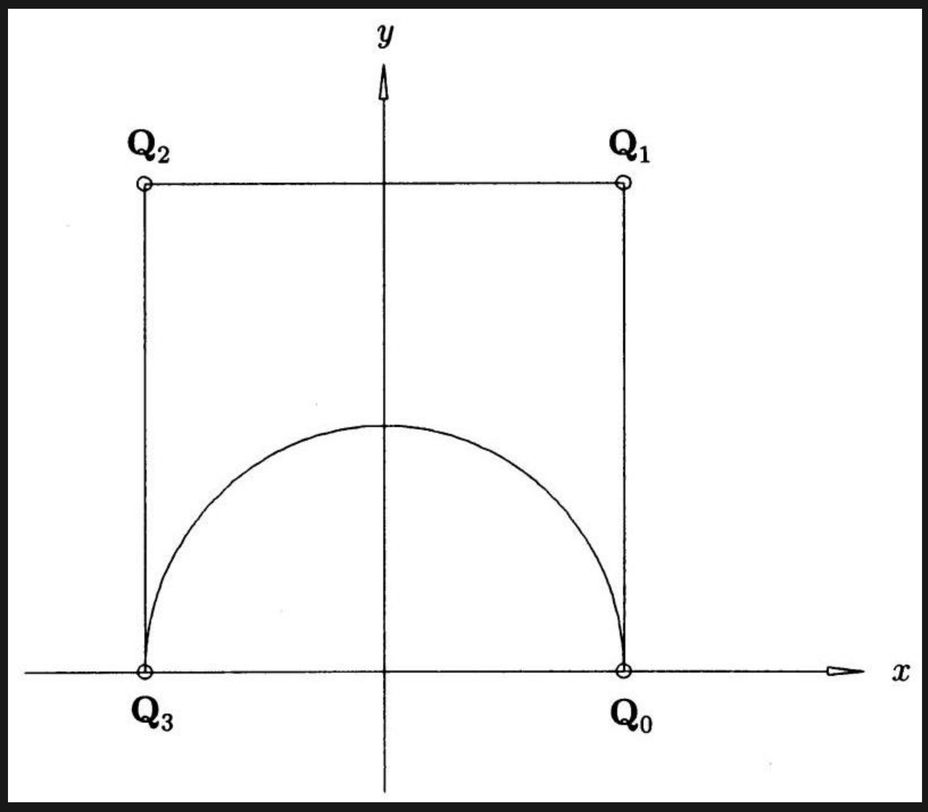


图 7.29. 半圆的有理三次贝塞尔表示。

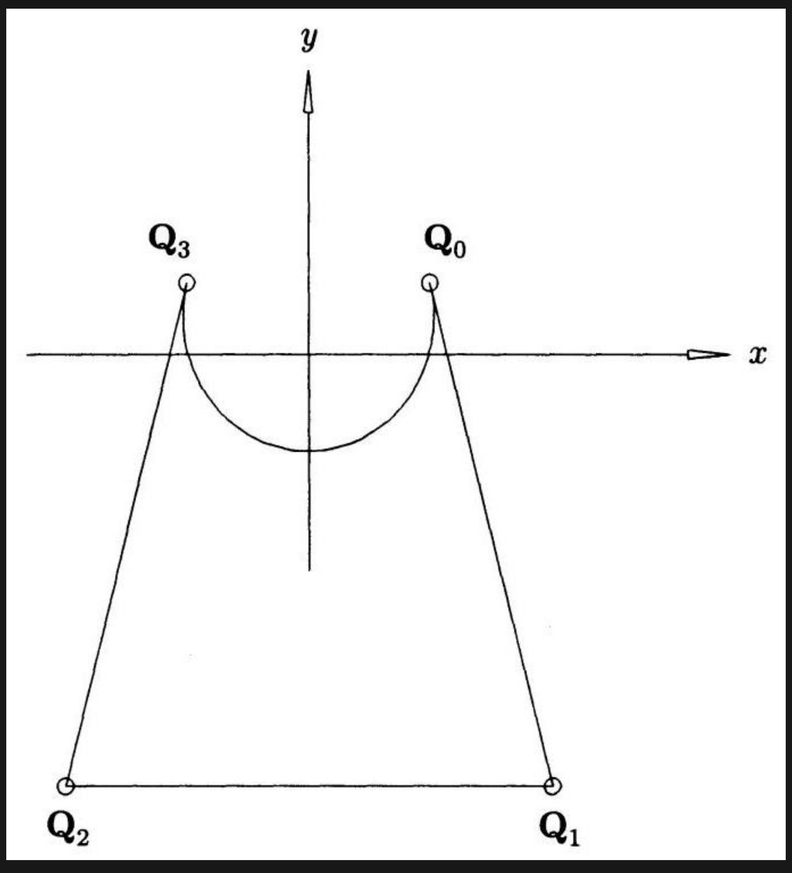


图 7.30. 扫描弧 240° 的有理三次贝塞尔表示。

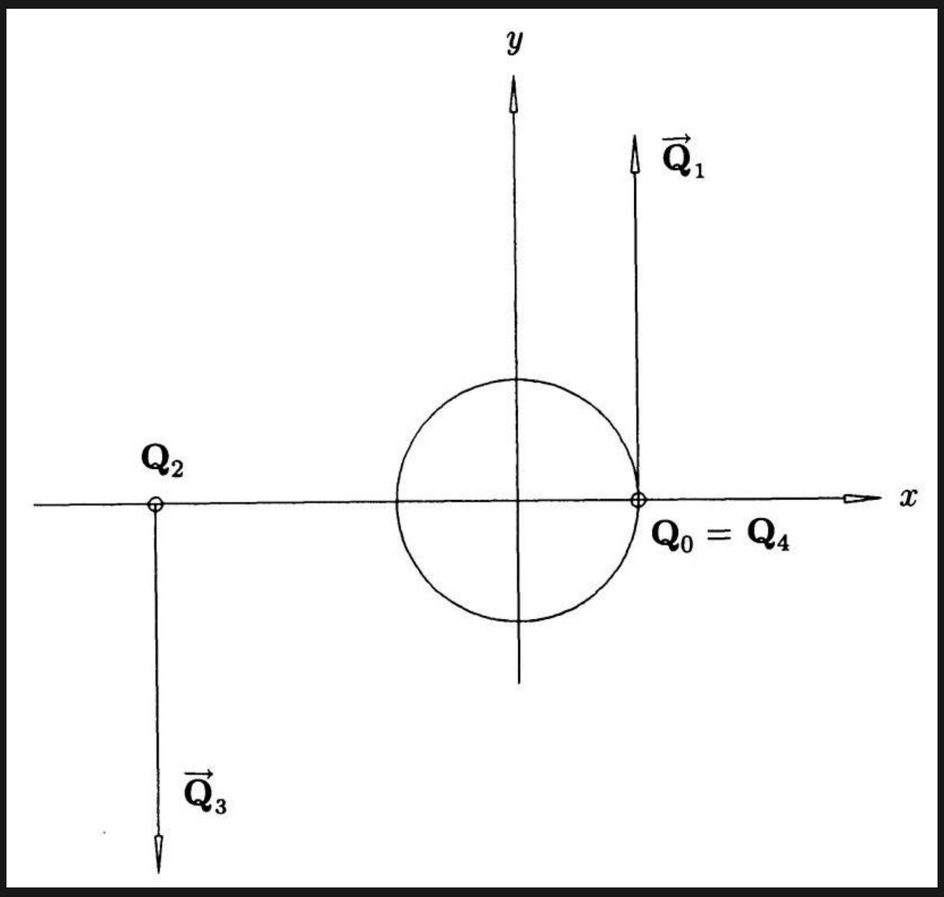


图 7.31. 使用两个无限控制点的完整圆的有理四次表示。

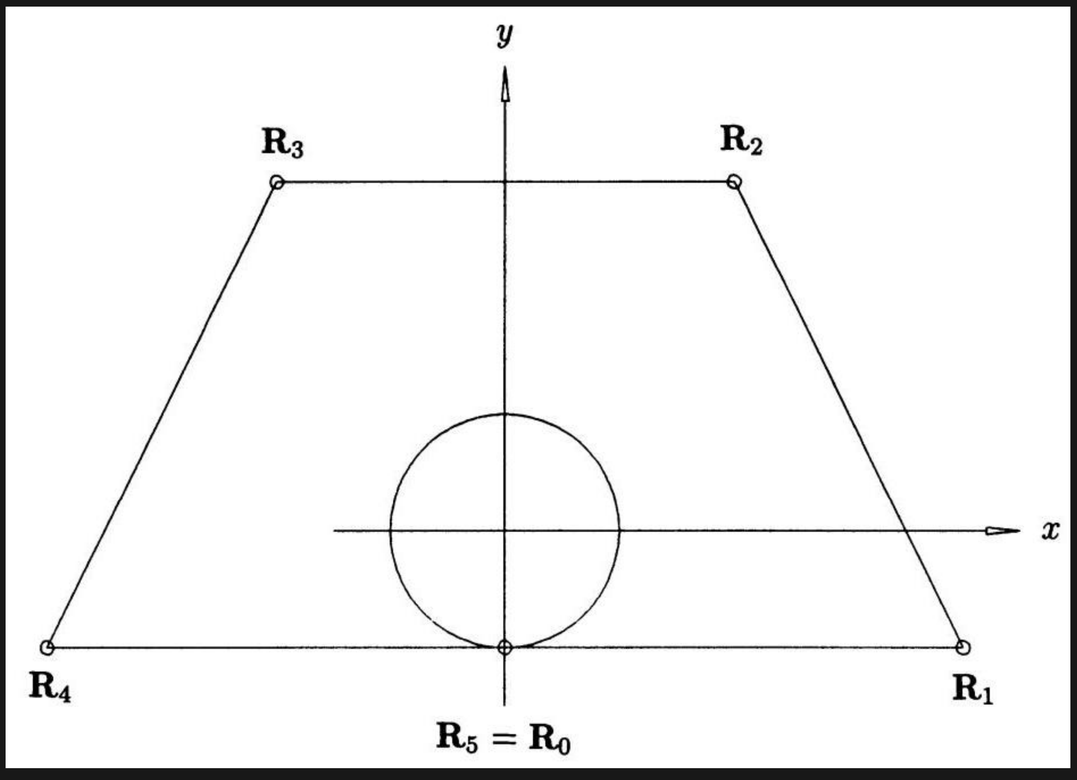


图 7.32. 仅使用有限控制点的完整圆的有理五次表示。

转换为贝塞尔形式得到

$$\{\mathbf{Q}_i^w\} = \{(3,0,3), (0,3,0), (-3,0,1), (0,-3,0), (3,0,3)\}$$

这是一个具有两个无限控制点的四次完整圆 (见图 7.31)。

Ex7.10 例子 Ex7.9 的四次圆的简单度提升得到

$$\mathbf{R}_0^w = (0,-5,5)\ \ \mathbf{R}_1^w = (4,-1,1)\ \ \mathbf{R}_2^w = (2,3,1)$$

$$\mathbf{R}_3^w = (-2,3,1)\ \ \mathbf{R}_4^w = (-4,-1,1)\ \ \mathbf{R}_5^w = (0,-5,5)$$

这是一个使用有限控制点和正权重的五次贝塞尔全圆 (见图 7.32)。

Chou [Chou95] 证明了仅使用正权重和有限控制点无法获得四次贝塞尔全圆。

最后，我们提醒读者，圆只是位于直圆锥上的曲线的透视投影。因此，任何这样的曲线，包括非平面曲线，都是可以的。可能性是无穷无尽的。

习题

7.1. 证明第 7.3 节中所做的陈述: “由于我们选择了 $w_0 = w_2 = 1$ ，因此在 $\mathbf{S} = \mathbf{C}(1/2)$ 处的圆锥的切线与 $\mathbf{P}_0\mathbf{P}_2$ 平行。” 提示: 使用方程 (7.25) 和 (7.26) 以及相似三角形的性质。

7.2. 考虑图 7.13 和方程 (7.33)。它们暗示 w_1 仅依赖于扫掠角 (而不是半径)。考虑在两种极限情况下 w_1 会发生什么:

- 固定 $\mathbf{P}_0$ 和 $\mathbf{P}_2$ ，让 $\mathbf{P}_1$ 趋向于无穷大;
- 固定一个圆，让 $\mathbf{P}_0$ 和 $\mathbf{P}_2$ 结合在一起。

7.3. “将权重设为零” 和 “无限系数” 这两个概念在有理幂基表示中也存在。这两个概念中是否有一个被用来获得方程 (7.8) 和 (7.9)? 如果是，哪一个?

7.4. 通过使用九点方形控制多边形和示例 Ex7.2 的节点，但权重为 $\{w_i\} = \{1,1,2,1,1,1\ ,\ 2,1,1\}$ ，可以获得完整圆的另一种表示。研究在 $u = 1/2$ 处的连续性。使用第 5 章中的公式尽可能多次去除节点 $u = 1/2$ 。将此与示例 Ex7.2 进行比较并解释结果。可视化两个位于同质空间锥体上的圆。

7.5. 使用方程 (7.53) 和 (7.25) 验证方程 (7.8) 和 (7.9) 分别表示椭圆和双曲线。

7.6. NURBS 曲线定义为

$$U = \left\{0,0,0,\frac{1}{2},1,1,1\right\}$$

$$\mathbf{P}_i = \{(1,0), (1,1), (-2,1), (-2,-1)\}$$

$$w_i = \left\{1,\frac{1}{2},\frac{1}{2},1\right\}$$

这条曲线是唯一的圆锥曲线吗?

第八章　常见曲面的构造

1 引言

本章的第 2 到 5 节涵盖了四种表面类型的 NURBS 表示: 双线性表面、一般圆柱体 (挤压表面)、规则表面和旋转表面。正如读者现在肯定意识到的，这些表示并不是唯一的。参数化、凸包、连续性、软件设计、应用领域、数据交换甚至个人品味等因素都可能影响表示方法的选择。我们的展示紧密跟随 Piegl 和 Tiller [Pieg87a] 提出的构造。第 6 节使用非均匀缩放来获得额外的表面，例如椭球体和椭圆抛物面。第 7 节介绍了一种在球面上构造三边补丁的方法，其边界曲线是半径等于球体半径的圆。这种补丁在作为角落圆角表面时非常有用。

为了清楚地指示本章中的度数，我们使用符号 $R_{i,p;j,q}(u,v)$ 表示度数为 $p\times q$ 的第 i,j 个有理基函数 (与方程 [4.12] 进行比较)。

2 双线性曲面

设 $\mathbf{P}_{0,0},\mathbf{P}_{1,0},\mathbf{P}_{0,1},\mathbf{P}_{1,1}$ 为三维空间中的四个点。我们希望构造通过在四条线段之间进行双线性插值而获得的曲面的 NURBS 表示，$\mathbf{P}_{0,0}\mathbf{P}_{1,0},\mathbf{P}_{0,1}\mathbf{P}_{1,1},\mathbf{P}_{0,0}\mathbf{P}_{0,1}$ 和 $\mathbf{P}_{1,0}\mathbf{P}_{1,1}$. 显然，所需的 (非有理) 曲面由以下公式给出

$$\mathbf{S}(u,v)=\sum_{i=0}^{1}\sum_{j=0}^{1}N_{i,1}(u)\,N_{j,1}(v)\,\mathbf{P}_{i,j} \tag{8.1}$$

其中

$$U=V=\{0,0,1,1\}$$

方程 (8.1) 表示在两个方向上相对边界线之间的简单线性插值。在图 8.1a 中，四个点位于同一平面内，因此曲面是由四条线段所界定的平面的一部分。图 8.1b 显示了通过在立方体的非平行对角线之间进行线性插值得到的双曲抛物面。

3 一般圆柱体

设 $\mathbf{W}$ 为单位长度的向量，$\mathbf{C}(u)=\sum_{i=0}^{n}R_{i,p}(u)\,\mathbf{P}_i$ 为节点向量 U 上的 p 次 NURBS 曲线，权重为 w_i 。我们希望得到通过沿 $\mathbf{W}$ 方向以距离 d 进行扫掠而获得的一般圆柱体 $\mathbf{S}(u,v)$ 的表示。用 $v,0\le v\le 1$ 表示扫掠方向的参数，显然 $\mathbf{S}(u,v)$ 必须满足两个条件

- 对于固定的 $\bar{u},\mathbf{S}(\bar{u},v)$ ，是从 $\mathbf{C}(\bar{u})$ 到 $\mathbf{C}(\bar{u})+d\mathbf{W}$ 的一条直线段；
- 对于固定的 $\bar{v}$

$$\mathbf{S}(u,\bar{v})=\mathbf{C}(u)+\bar{v}d\mathbf{W}=\sum_{i=0}^{n}R_{i,p}(\mathbf{P}_i+\bar{v}d\mathbf{W})$$

(平移不变性属性)。

所需的表示为

$$\mathbf{S}(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{1}R_{i,p;j,1}(u,v)\,\mathbf{P}_{i,j} \tag{8.2}$$

在节点向量 U 和 V 上，其中 $V=\{0,0,1,1\}$ 和 U 是 $\mathbf{C}(u)$ 的节点向量。控制点由 $\mathbf{P}_{i,0}=\mathbf{P}_i$ 和 $\mathbf{P}_{i,1}=\mathbf{P}_i+d\mathbf{W}$ 给出，权重为 $w_{i,0}=w_{i,1}=w_i$ 。图 8.2 显示了一般圆柱体。

右圆柱体是通过将 NURBS 圆沿与圆平面法向的向量平移距离 d 得到的。第 7 章中给出的任何圆的表示方式都可以使用。图 8.3 显示了一个右圆柱体，其底层圆是示例 Ex7.2 和图 7.16a 的二次九点方形控制多边形表示。该圆柱体由以下公式定义

$$\mathbf{S}(u,v)=\sum_{i=0}^{8}\sum_{j=0}^{1}R_{i,2;j,1}(u,v)\,\mathbf{P}_{i,j} \tag{8.3}$$

其中 $V=\{0,0,1,1\}$ 和 U ，权重 $(w_{i,0}$ 和 $w_{i,1})$ 是示例 Ex7.2 中给出的圆的权重。注意，控制网形成了圆柱体的外接框。

通过平移由算法 A7.1 和 A7.3 计算出的控制点，可以获得横截面为圆形和锥形且扫掠角小于 360° 的圆柱体。图 8.4 显示了一个椭圆柱体。

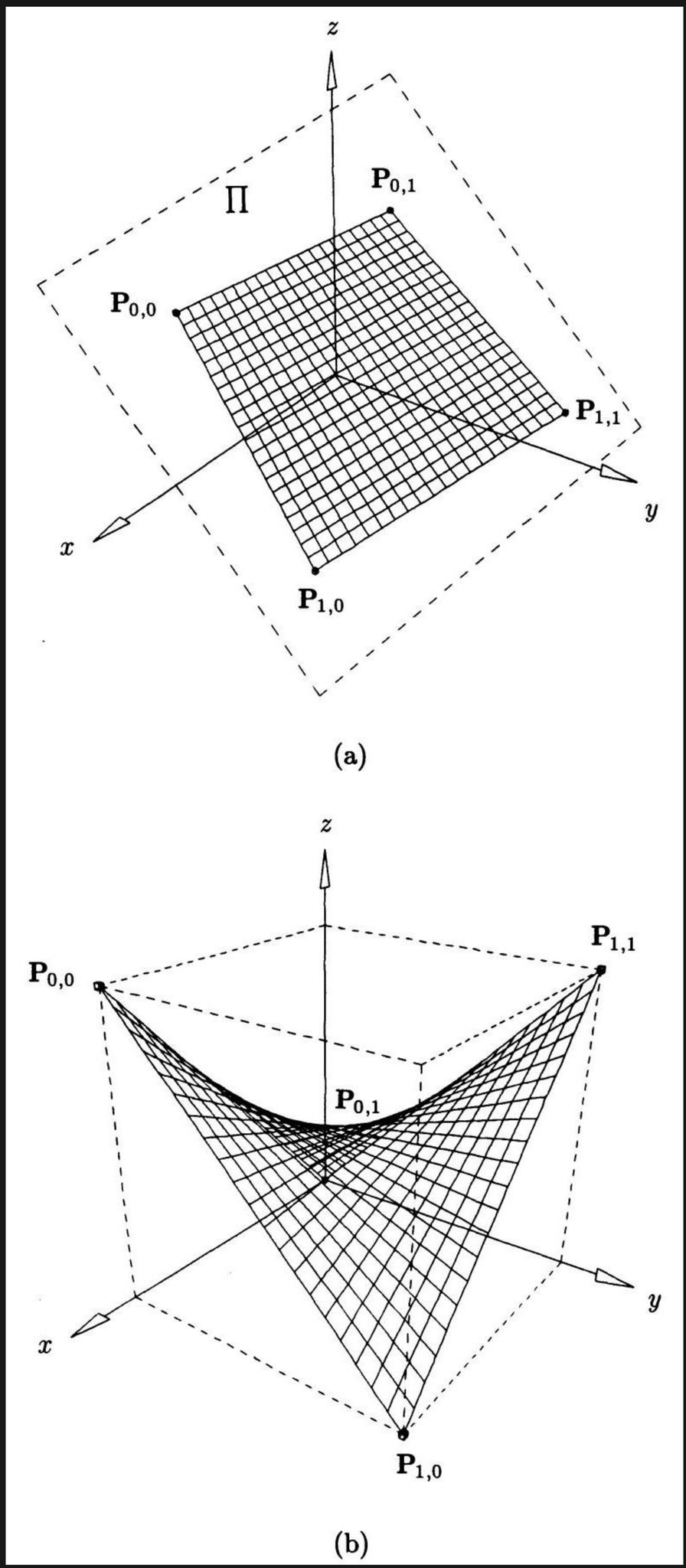


图 8.1. 双线性曲面。(a) 位于平面 Π 的直线边界；(b) 非平面直线边界 (由立方体的对角线定义) 形成双曲抛物面。

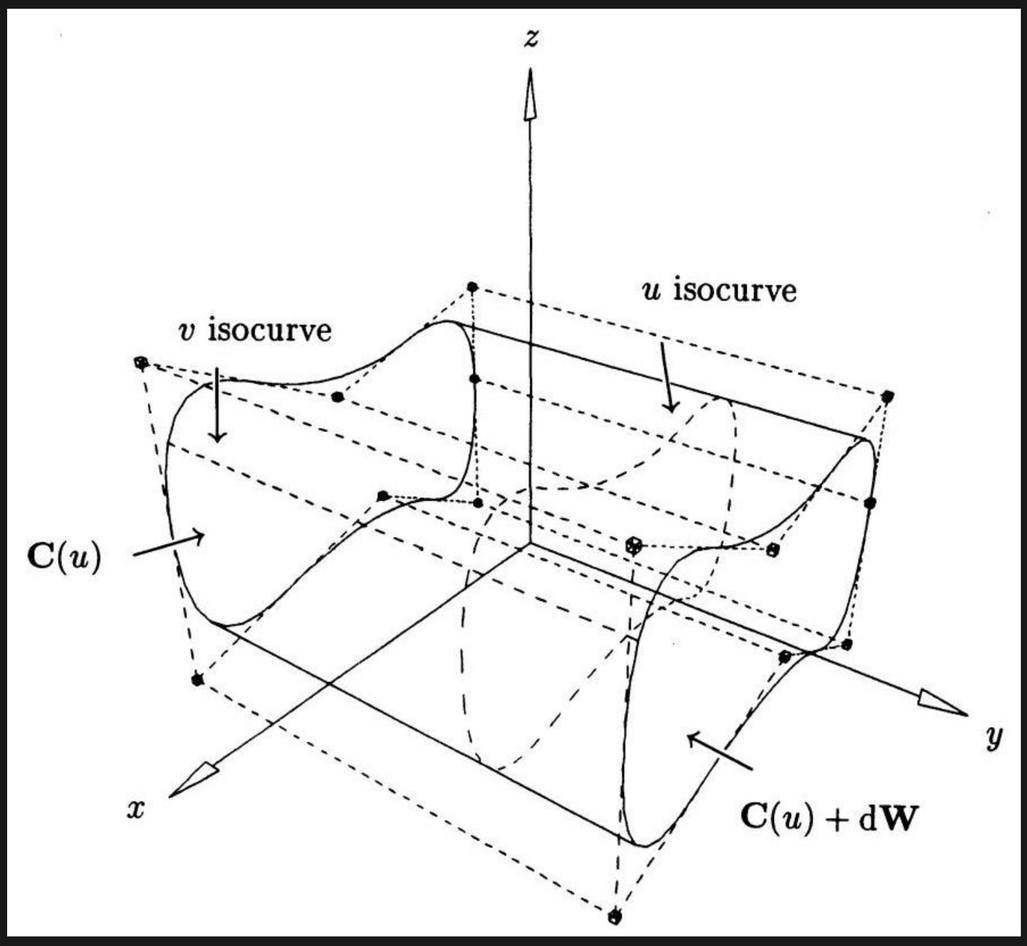


图 8.2. 通过沿 **W** 平移距离 d 得到的一般圆柱体。

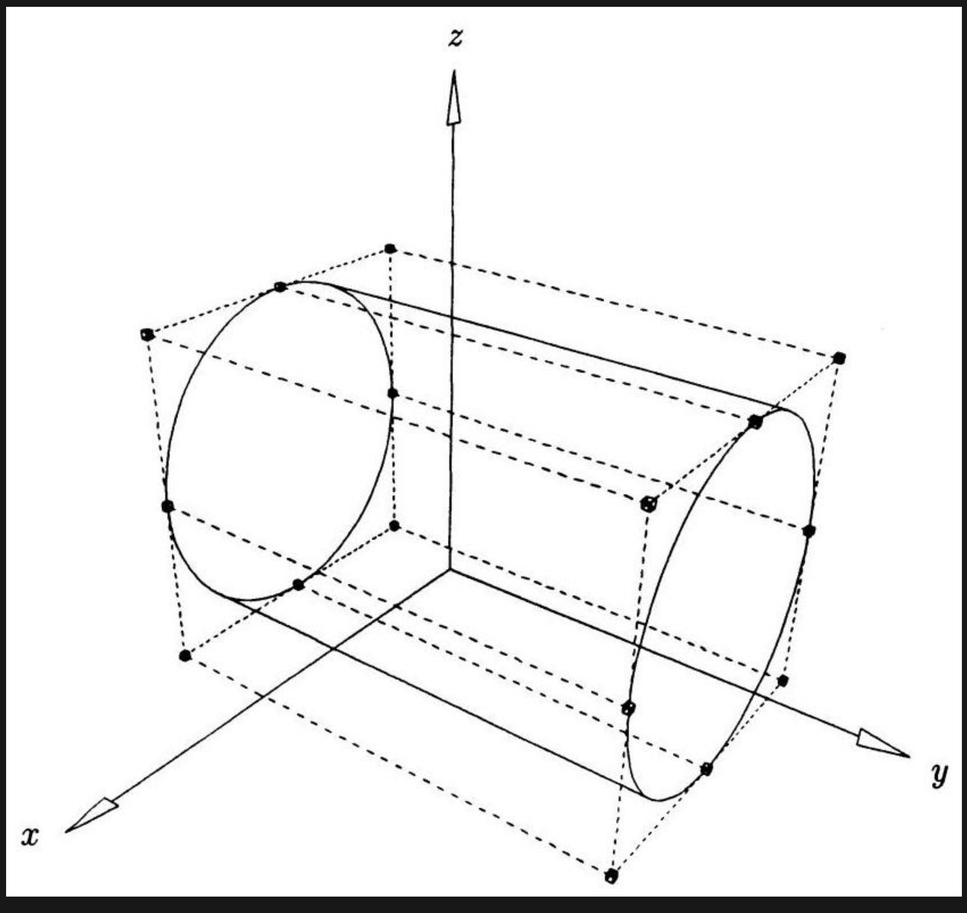


图 8.3. 右圆柱体。

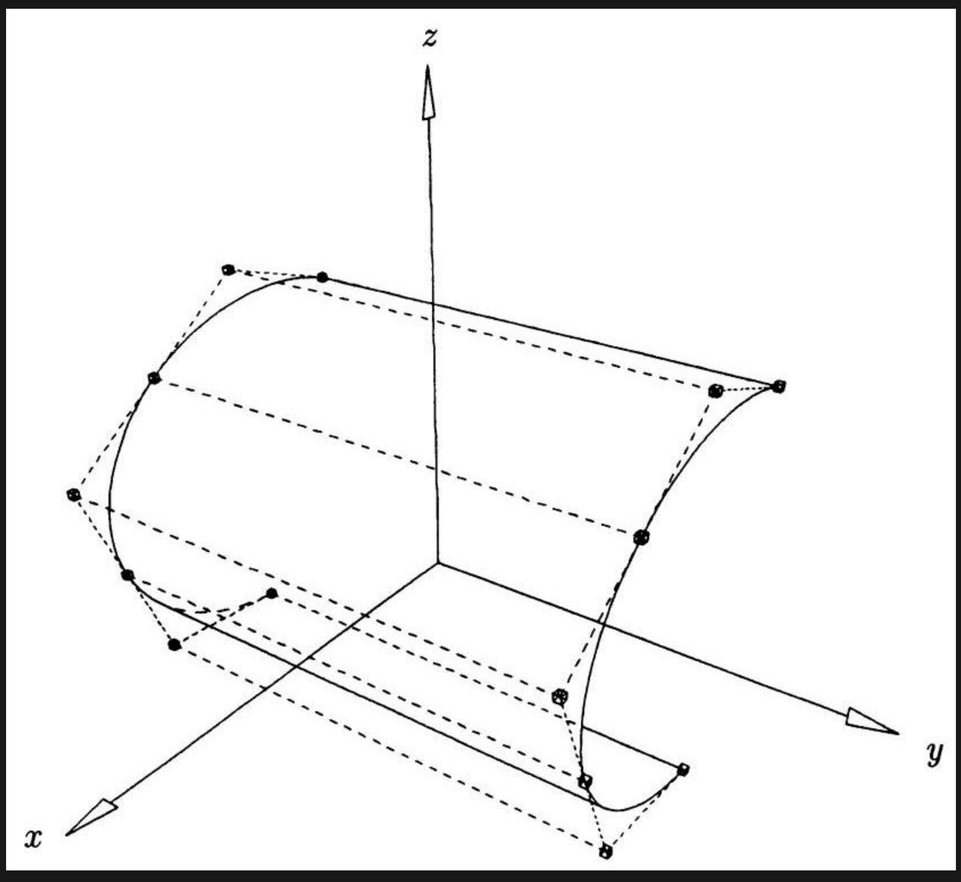


图 8.4. 椭圆柱面片。

4 规则曲面

假设我们有两个 NURBS 曲线

$$\mathbf{C}_k(u)=\sum_{i=0}^{n_k} R_{i,p_k}(u)\,\mathbf{P}_i^k \quad k=1,2$$

定义在结点向量 $U^k=\left\{u_0^k,\ldots,u_{m_k}^k\right\}$ 上。我们希望得到一个在 v 方向上被规则化的曲面，即它是 $\mathbf{C}_1(u)$ 和 $\mathbf{C}_2(u)$ 之间的线性插值。此外，我们要求插值在相同参数值的点之间进行，即，对于固定的 $\bar{u},\mathbf{S}(\bar{u},v)$，它是连接点 $\mathbf{C}_1(\bar{u})$ 和 $\mathbf{C}_2(\bar{u})$ 的直线段。所需的曲面形式是

$$\mathbf{S}(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{1} R_{i,p;j,1}(u,v)\,\mathbf{P}_{i,j} \tag{8.4}$$

其中 $V=\{0,0,1,1\}.n,U,p,w_{i,j}$，并且 $\mathbf{P}_{i,j}$ 必须全部确定。

由于曲面的张量积特性，两个边界曲线 $\mathbf{C}_k(u)$ 必须具有相同的度，并且定义在相同的结点向量上。因此，确定 $n,U,p,w_{i,j}$ 和 $\mathbf{P}_{i,j}$ 的算法是

1. 确保两个曲线定义在相同的参数范围内，即，

$$u_0^1=u_0^2 \text{ and } u_{m_1}^1=u_{m_2}^2$$

2. 如果 $p_1=p_2$，则设置 $p=p_1$；否则，将 p 设置为 p_1,p_2 中的较大值，并将较低度曲线的度提升到 p（算法 A5.9）；

3. 如果结点向量 U^1 和 U^2 不相同，则合并它们以获得结点向量 U；更准确地说，如果 u_j 在 U^1 或 U^2 中，则它在 U 中。u_j 在 U^1 或 U^2 中的最大重数也会转移到 U。例如，如果 $U^1=\{0,0,0,1,2,2,4,4,4\}$ 和 $U^2=\{0,0,0,1,2,3,4,4,4\}$，则 $U=\{0,0,0,1,2,2,3,4,4,4\}$；

4. 使用 U，对两个曲线 $\mathbf{C}_k(u)$ 应用结点细化（算法 A5.4）；此步骤产生 $n,w_{i,j}$ 和 $\mathbf{P}_{i,j}$ 的最终值。

图 8.5a – 8.5e 显示了在二次曲线和三次曲线之间构造规则曲面的过程。

一般锥体也是一个有规则的曲面。设 $\mathbf{P}$ 为锥体的顶点，

并设

$$\mathbf{C}_1(u)=\sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i^1$$

为其基曲线。定义曲线

$$\mathbf{C}_2(u)=\sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i^2$$

以 $\mathbf{P}_i^2=\mathbf{P}$ 和 $w_i^2=w_i^1$ 为所有 i（一个退化曲线）。在 $\mathbf{C}_1(u)$ 和 $\mathbf{C}_2(u)$ 之间的有规则曲面就是所需的锥体。图 8.6 显示了一个直圆锥，图 8.7a-8.7c 显示了一些由直线和圆定义的奇异有规则曲面。

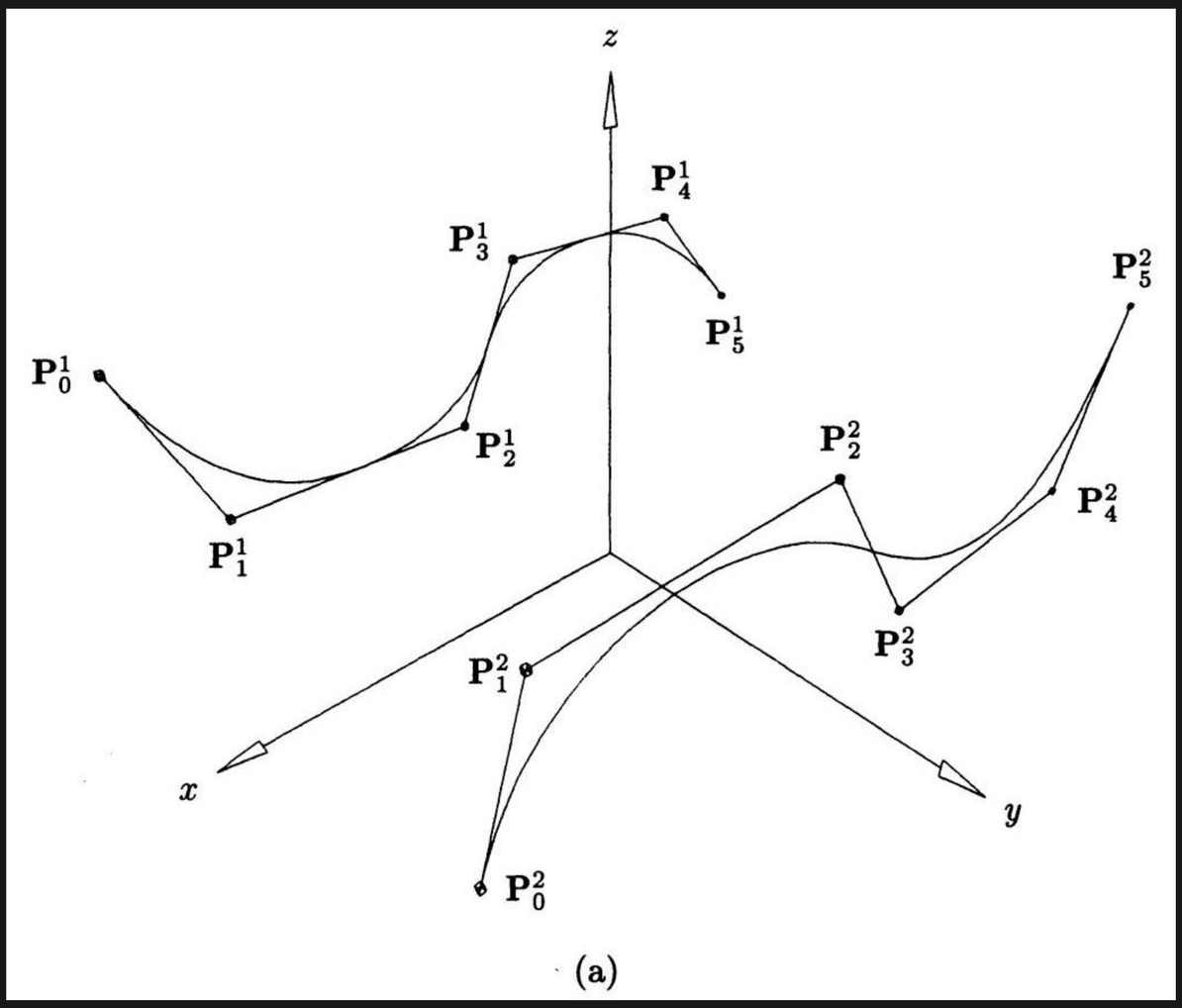
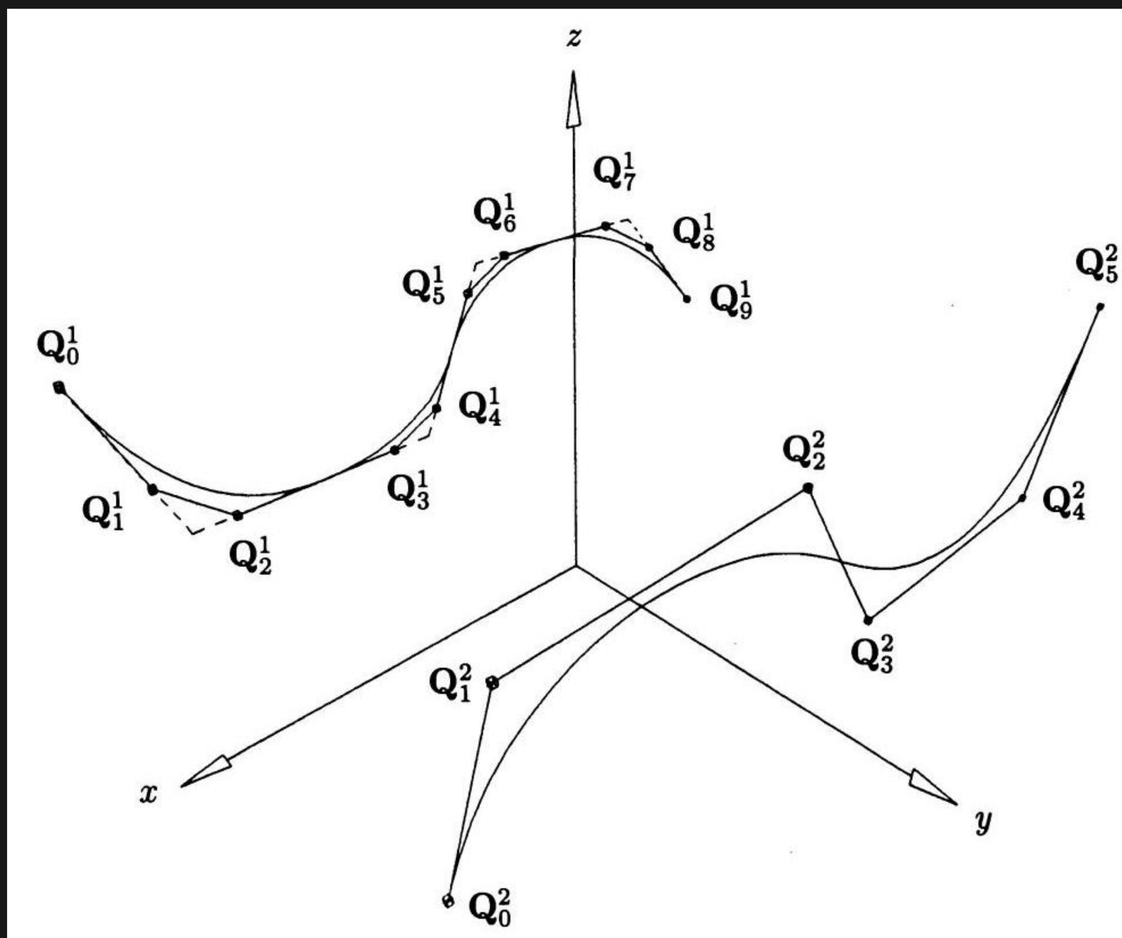
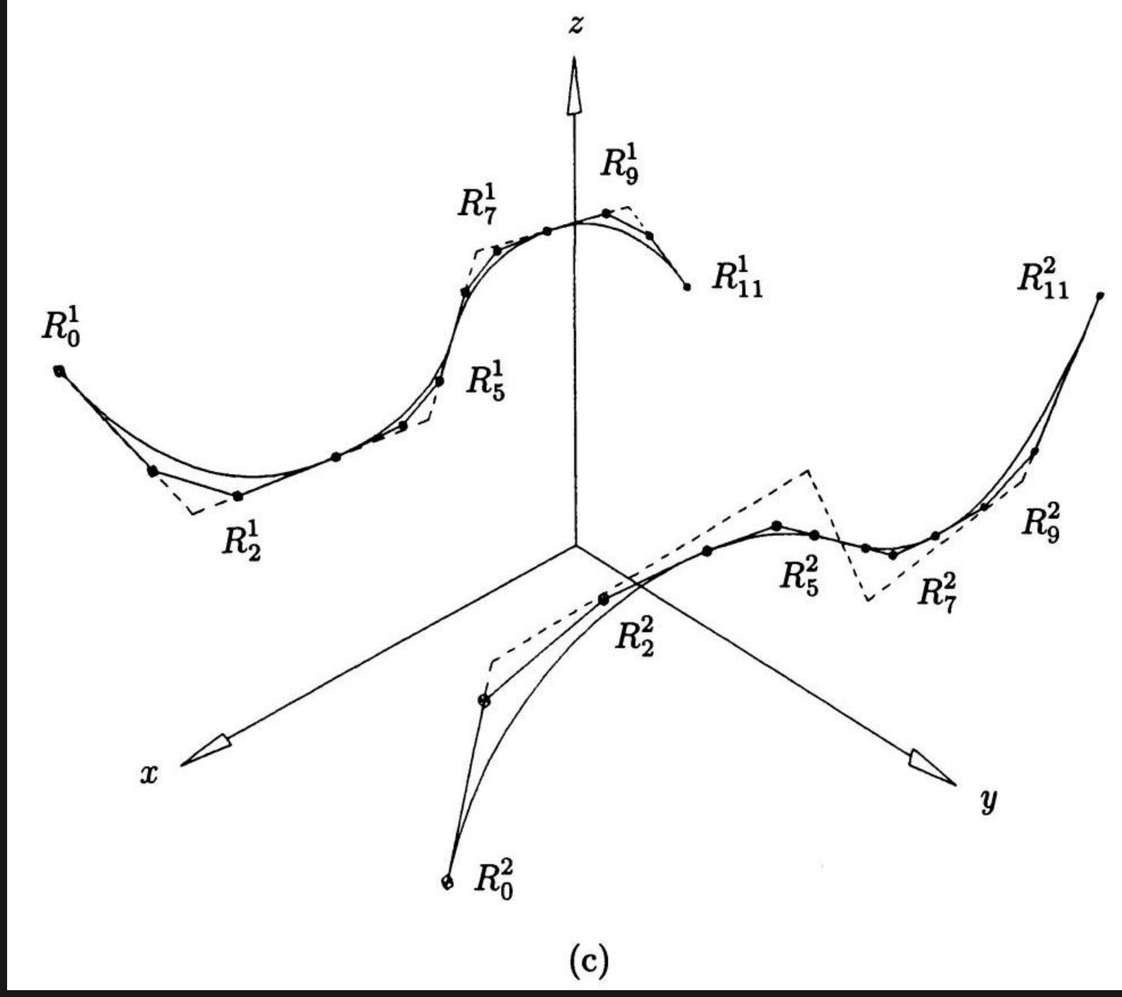


图 8.5. 在两个 NURBS 曲线之间构造有规则曲面。(a) 输入的二次和三次曲线，$U^1 = \{0, 0, 0, 1/4, 1/2, 3/4, 1, 1, 1\}$ 和 $U^2 = \{0, 0, 0, 0, 0, 3/10, 7/10, 1, 1, 1\}$ 。



(b)



(c)

图 8.5.(续。)(b) 二次曲线的次数提升；(c) 合并节点向量后的节点精细化曲线；曲面节点向量为 $U = \{0, 0, 0, 0, 1/4, 1/4, 3/10, 1/2, 1/2, 7/10, 3/4, 3/4, 1, 1, 1, 1\}$ 。

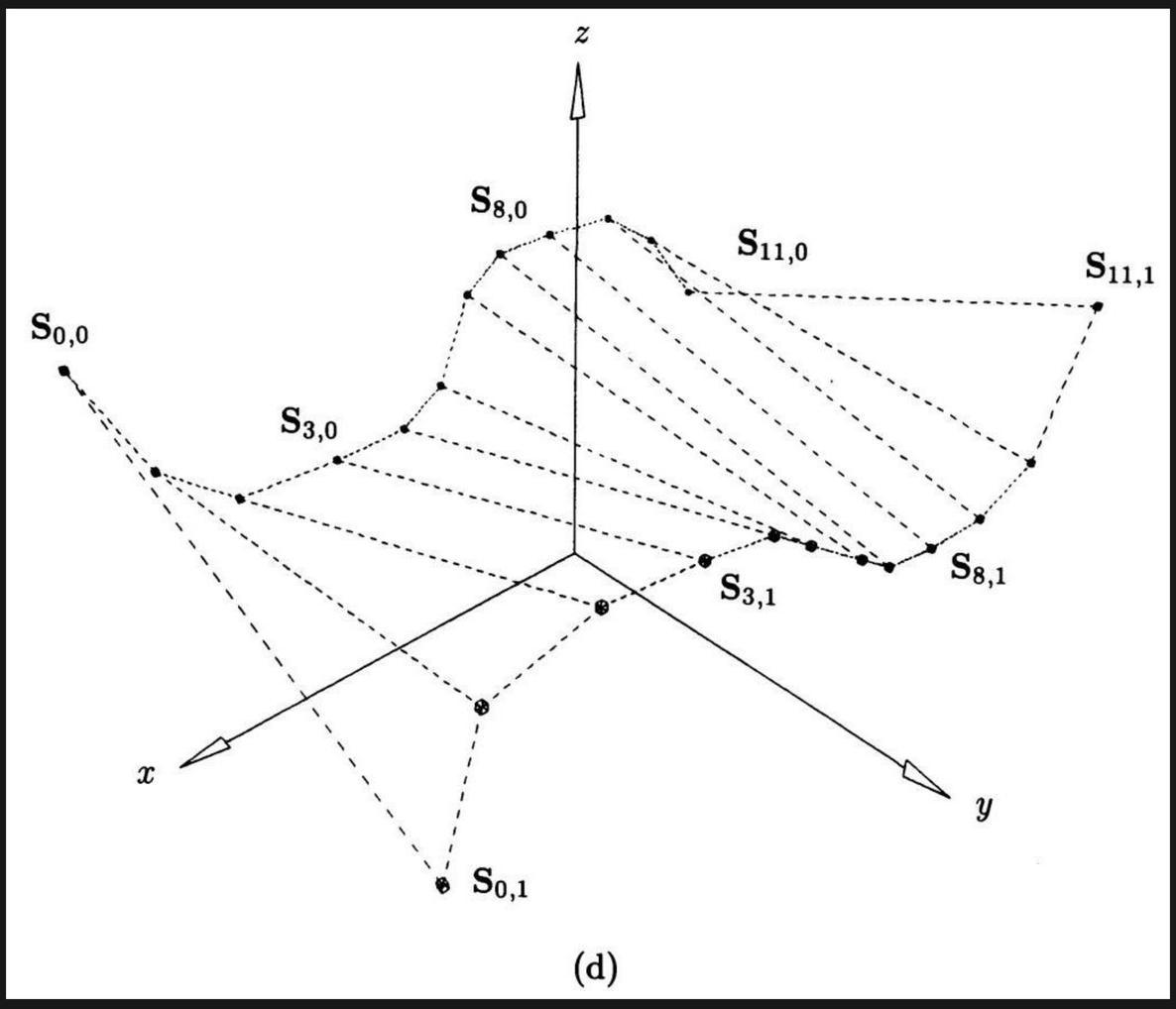


图 8.5.(续。)(d) 有规则曲面的控制网。

我们再次强调，NURBS 曲面上的有规则线段 (直线段) 是在参数值相等的点之间，而不是在相对弧长相等的点之间。一般来说，按照相对弧长进行有规则化会产生几何上不同的曲面，而这无法通过 NURBS 实现。因此，NURBS 与 IGES 有规则曲面 (类型 118)，形式 0 [IGE93] 之间的数学精确转换是不可能的。

5 旋转曲面

设 $\mathbf{C}(v)=\sum_{j=0}^{m} R_{j,q}(v)\,\mathbf{P}_j$ 为在节点向量 V 上的 q 阶 NURBS 曲线。$\mathbf{C}(v)$ 被称为生成线，它将围绕一个轴旋转。在本节的最后，我们将介绍一个算法，该算法可以使 $\mathbf{C}(v)$ 绕任意轴旋转任意角度。然而，为了简单起见，我们现在假设 $\mathbf{C}(v)$ 位于 xz 平面内，并且我们将 $\mathbf{C}(v)$ 绕 z 轴旋转一个完整的 360° 。所需的曲面 $\mathbf{S}(u,v)$ 具有如下特征 (见图 8.8):342 常见曲面的构造

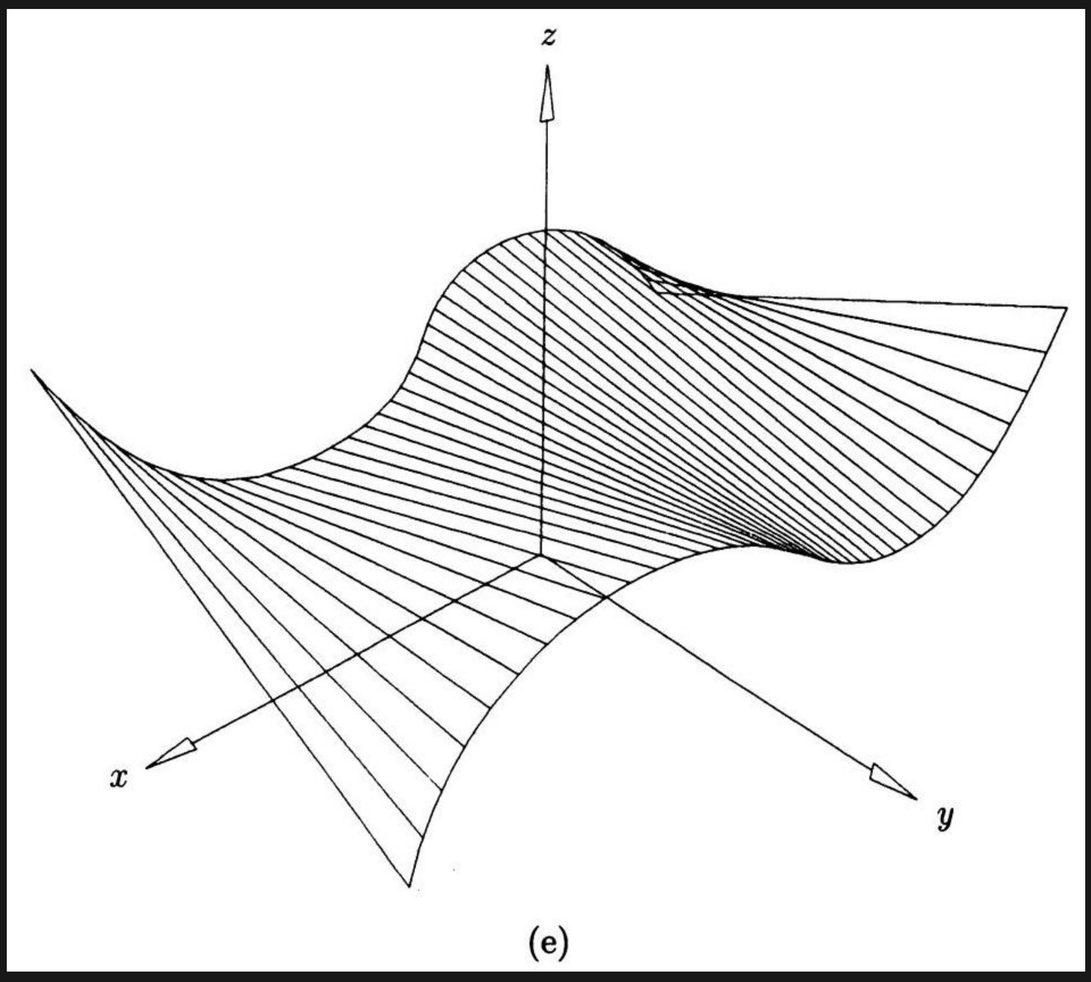


图 8.5. (续) (e) 规则曲面。

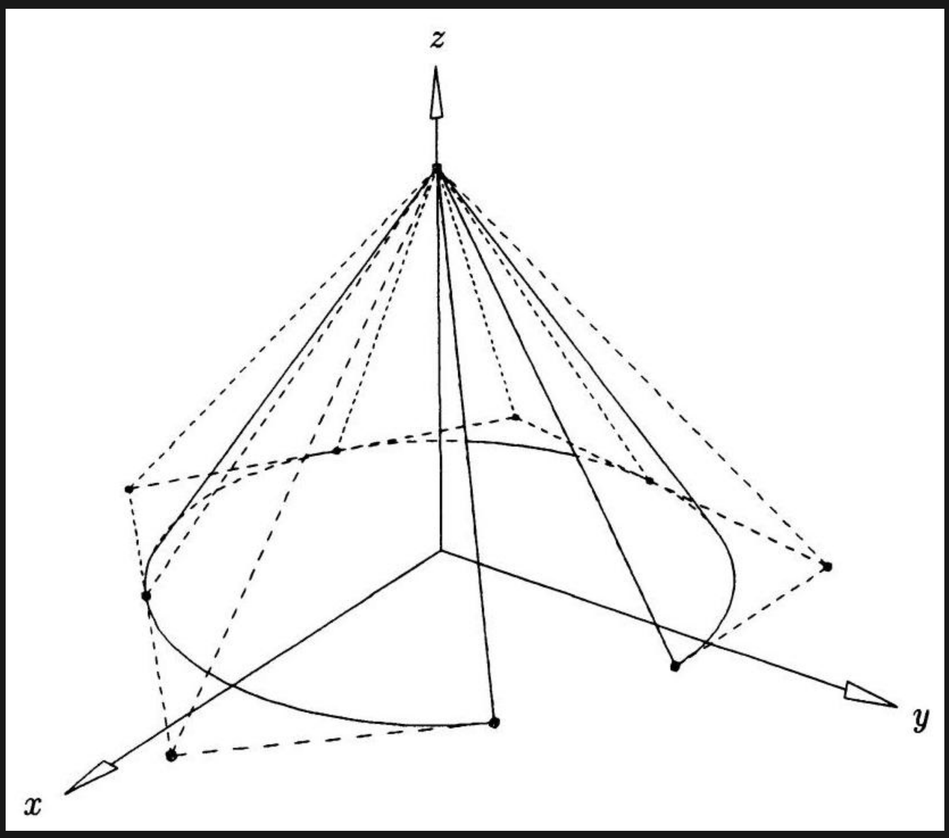


图 8.6. 一个右圆锥面片。

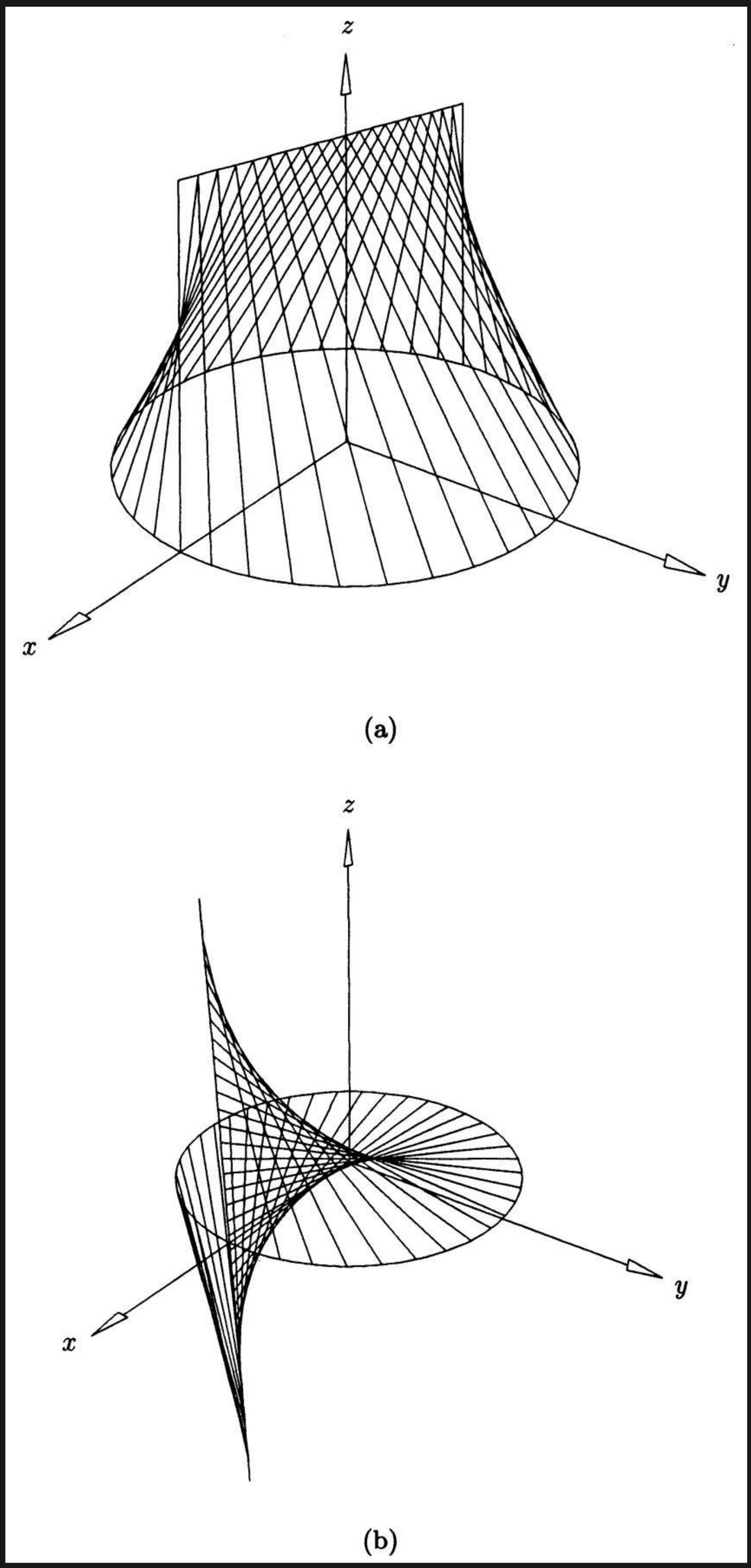


图 8.7. 由圆和直线定义的规则曲面。(a) 二阶规则曲面 (圆锥面)；(b) 三阶规则曲面。

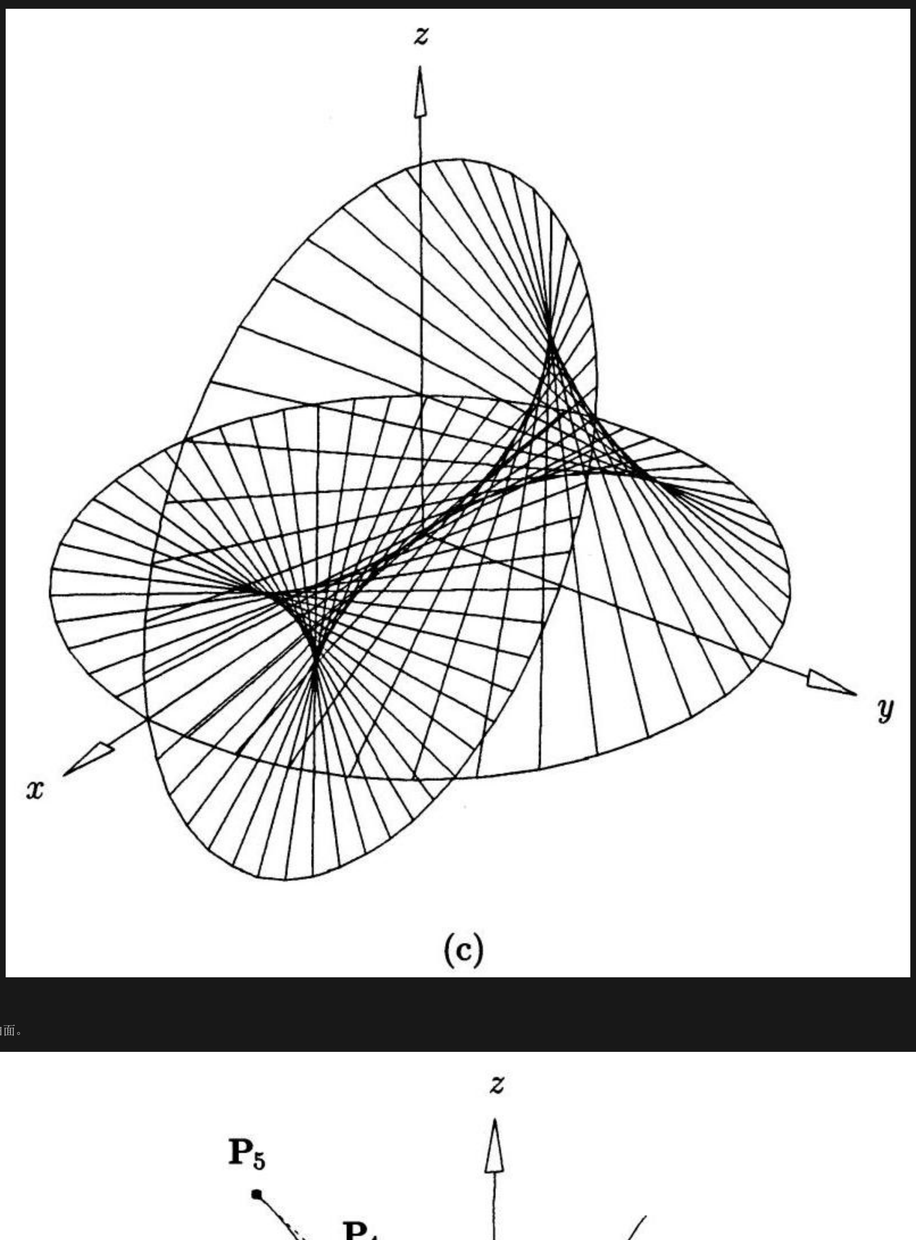


图 8.7. (续) (c) 四阶规则曲面。

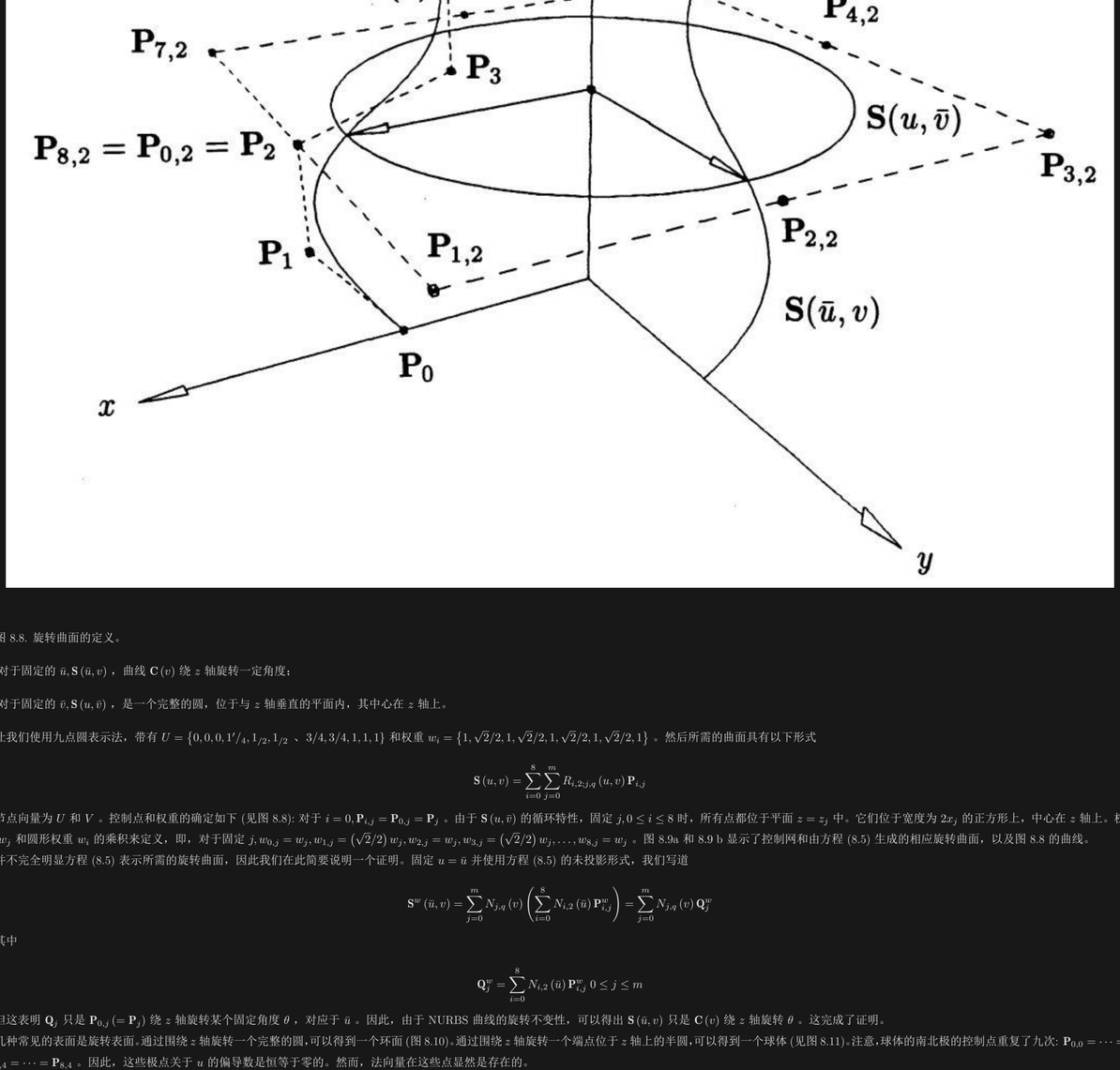


图 8.8. 旋转曲面的定义。

- 对于固定的 $\bar{u}, \mathbf{S}(\bar{u}, v)$ ，曲线 $\mathbf{C}(v)$ 绕 z 轴旋转一定角度；
- 对于固定的 $\bar{v}, \mathbf{S}(u, \bar{v})$ ，是一个完整的圆，位于与 z 轴垂直的平面内，其中心在 z 轴上。

让我们使用九点圆表示法，带有 $U = \{0, 0, 0, 1'/_4, 1/_2, 1/_2 \text{ 、 } 3/4, 3/4, 1, 1, 1\}$ 和权重 $w_i = \{1, \sqrt{2}/2, 1, \sqrt{2}/2, 1, \sqrt{2}/2, 1, \sqrt{2}/2, 1\}$ 。然后所需的曲面具有以下形式

$$\mathbf{S}(u, v) = \sum_{i=0}^{8} \sum_{j=0}^{m} R_{i,2;j,q}(u, v) \mathbf{P}_{i,j} \tag{8.5}$$

节点向量为 U 和 V 。控制点和权重的确定如下 (见图 8.8): 对于 $i = 0, \mathbf{P}_{i,j} = \mathbf{P}_{0,j} = \mathbf{P}_j$ 。由于 $\mathbf{S}(u, \bar{v})$ 的循环特性，固定 $j, 0 \leq i \leq 8$ 时，所有点都位于平面 $z = z_j$ 中。它们位于宽度为 $2x_j$ 的正方形上，中心在 z 轴上。权重通过取 w_j 和圆形权重 w_i 的乘积来定义，即，对于固定 $j, w_{0,j} = w_j, w_{1,j} = (\sqrt{2}/2) w_j, w_{2,j} = w_j, w_{3,j} = (\sqrt{2}/2) w_j, \ldots, w_{8,j} = w_j$ 。图 8.9a 和 8.9 b 显示了控制网和由方程 (8.5) 生成的相应旋转曲面，以及图 8.8 的曲线。

并不完全明显方程 (8.5) 表示所需的旋转曲面，因此我们在此简要说明一个证明。固定 $u = \bar{u}$ 并使用方程 (8.5) 的未投影形式，我们写道

$$\mathbf{S}^w(\bar{u}, v) = \sum_{j=0}^{m} N_{j,q}(v) \left(\sum_{i=0}^{8} N_{i,2}(\bar{u}) \mathbf{P}_{i,j}^w \right) = \sum_{j=0}^{m} N_{j,q}(v) \mathbf{Q}_j^w$$

其中

$$\mathbf{Q}_j^w = \sum_{i=0}^{8} N_{i,2}(\bar{u}) \mathbf{P}_{i,j}^w \; 0 \leq j \leq m$$

但这表明 $\mathbf{Q}_j$ 只是 $\mathbf{P}_{0,j} (= \mathbf{P}_j)$ 绕 z 轴旋转某个固定角度 θ ，对应于 $\bar{u}$ 。因此，由于 NURBS 曲线的旋转不变性，可以得出 $\mathbf{S}(\bar{u}, v)$ 只是 $\mathbf{C}(v)$ 绕 z 轴旋转 θ 。这完成了证明。

几种常见的表面是旋转表面。通过围绕 z 轴旋转一个完整的圆，可以得到一个环面 (图 8.10)。通过围绕 z 轴旋转一个端点位于 z 轴上的半圆，可以得到一个球体 (见图 8.11)。注意，球体的南北极的控制点重复了九次: $\mathbf{P}_{0,0} = \cdots = \mathbf{P}_{8,0}$ 和 $\mathbf{P}_{0,4} = \cdots = \mathbf{P}_{8,4}$ 。因此，这些极点关于 u 的偏导数是恒等于零的。然而，法向量在这些点显然是存在的。

我们现在提出一个算法，该算法通过任意角度 theta 绕任意轴构造 NURBS 旋转曲面 (见图 8.12)。该轴由一个点 S 和一个单位长度向量 T 指定。为了方便，我们在这个算法中将权重和三维控制点分开：m, Pj[], wj[] 定义生成曲线，而 n, U, Pij[][] 被计算 (见公式 [8.5])。我们假设有四个实用函数:

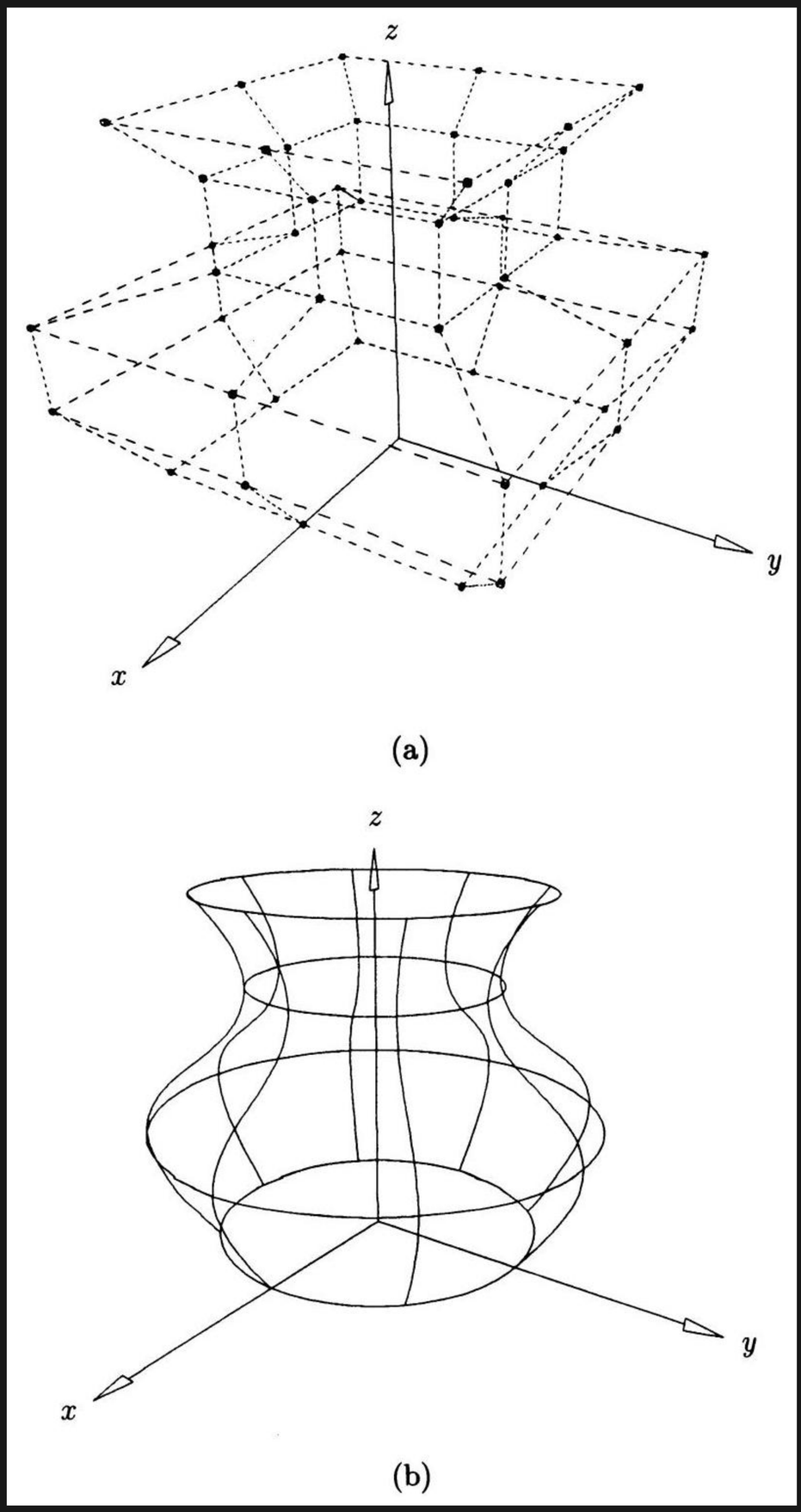


图 8.9. (a.) 一个旋转表面的控制网；(b) 一个旋转表面。

PointToLine( )：将一个点投影到一条线上；

VecNormalize( )：归一化一个向量并返回其大小；

VecCrossProd( )：计算向量的叉积；

Intersect3DLines ( )：交叉两条三维线。

注意这个算法与算法 A7.1 之间的相似性。346 常见表面的构造算法 A8.1

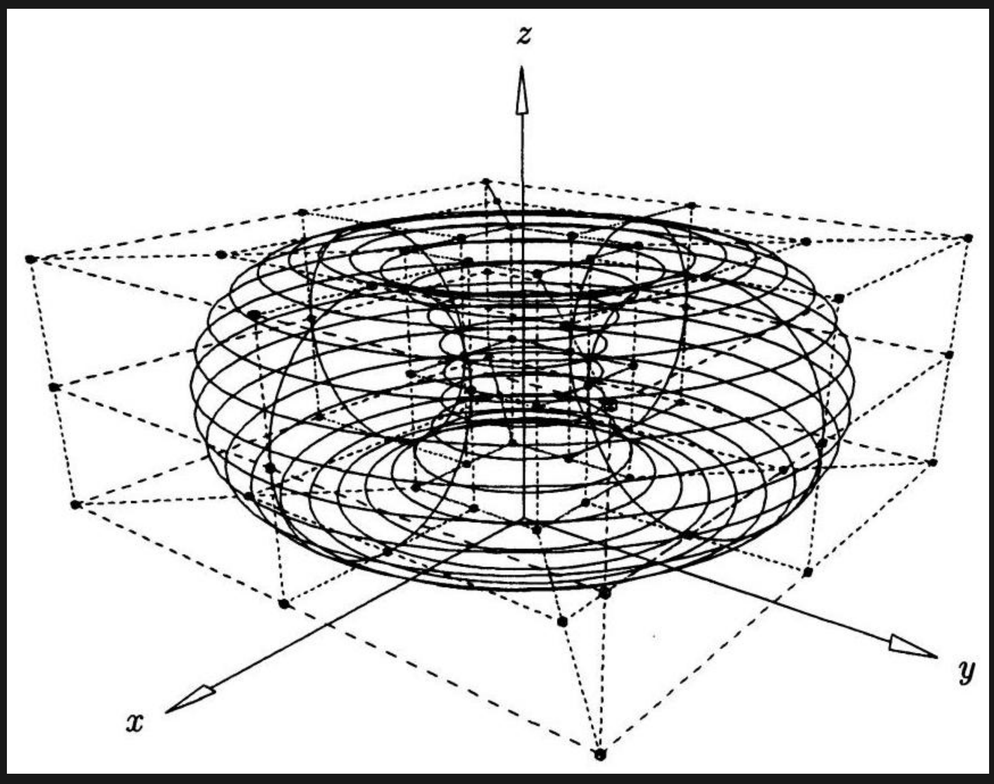


图 8.10. 作为旋转表面的环面。

```
MakeRevolvedSurf (S, T, theta, m, Pj, wj, n, U, Pij, wij)
        { /* Create NURBS surface of revolution */
            /* Input: S, T, theta, m, Pj, wj */
            /* Output: n, U, Pij, wij */
    if (theta <= 90.0) narcs = 1 ;
            else
                if (theta <= 180.0)
                    { narcs = 2; U[3] = U[4] = 0.5; }
                    else
                        if (theta <= 270.0)
                            {
                            narcs = 3 ;
                            U [3] = U [4] = 1.0/3.0;
                            U[5] = U[6] = 2.0/3.0;
                            }
                            else
                            {
                            narcs = 4 ;
                            U[3] = U[4] = 0.25 ;
                            U[5] = U[6] = 0.5 ;
                            U[7] = U[8] = 0.75 ;
                            }
    dtheta = theta/narcs;
    j = 3 + 2*(narcs-1); /* load end knots */
    for (i=0; i<3; j++, i++)
```

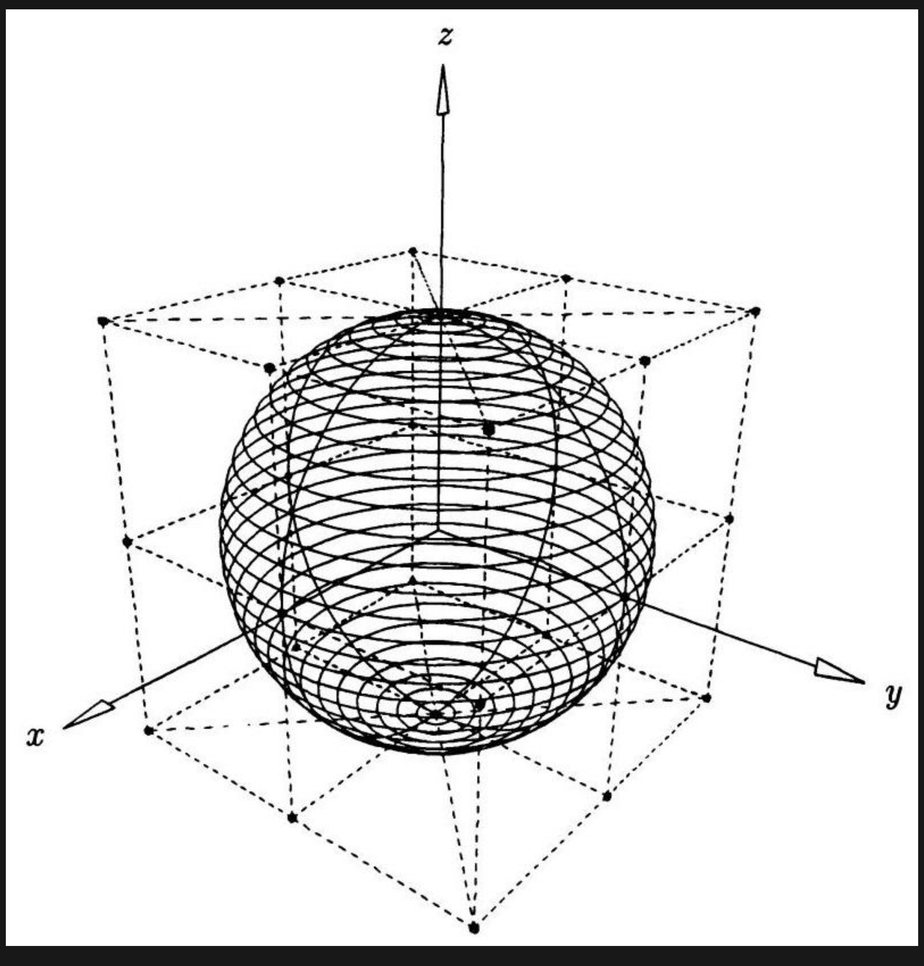


图 8.11. 作为旋转表面的球体。

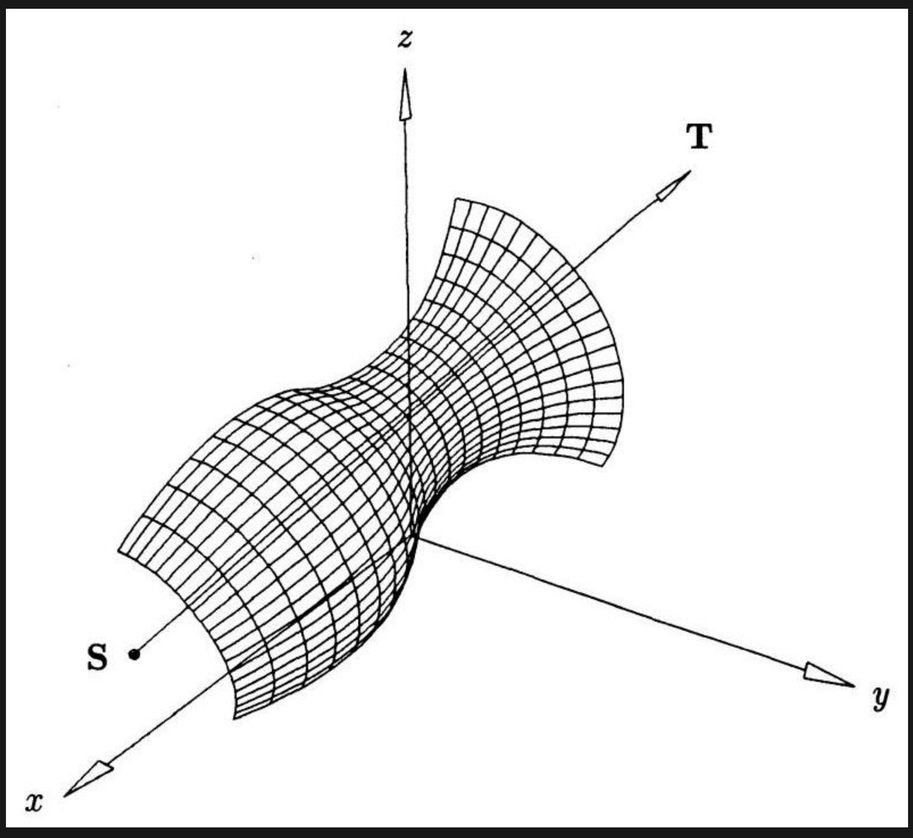


图 8.12. 绕任意轴通过任意角度的通用旋转表面。

```
    { U [i] = 0.0; U [j] = 1.0; }
n = 2*narcs;
wm = cos(dtheta/2.0); /* dtheta/2 is base angle */
angle = 0.0; /* Compute sines and cosines only once */
for (i=1; i<=narcs; i++)
        {
    angle = angle + dtheta;
      cosines[i] = cos(angle);
      sines[i] = sin(angle);
    }
for (j=0; j<=m; j++)
      /* Loop and compute each u row of ctrl pts and weights */
    {
    PointToLine(S, T, Pj[j],0);
    X = Pj [j] − 0 ;
    r = VecNormalize(X); VecCrossProd(T, X, Y);
    Pij[0][j] = P0 = Pj[j]; /* Initialize first */
    wij[0][j] = wj[j]; /* ctrl pt and weight */
      TO = Y; index = 0; angle = 0.0 ;
      for (i=1; i<=narcs; i++) /* compute u row */
        {
        P2 = 0 + r*cosines[i]*X + r*sines[i]*Y;
        Pij[index+2][j] = P2;
        wij[index+2][j] = wj[j];
        T2 = -sines[i]*X + cosines[i]*Y;
        Intersect3DLines(PO, TO, P2, T2, Pij[index+1][j]);
        wij[index+1][j] = wm*wj[j];
        index = index + 2 ;
        if (i < narcs) { P0 = P2 ; T0 = T2 ; }
        }
    }
}
```

图 8.13 显示了一个一般的环形补丁，作为一个旋转曲面，用于圆滑尖锐的角落。

6 曲面的非均匀缩放

设 $\mathbf{P} = (p_i)\,, i = 1, 2, 3$ 为三维空间中的一个点，设 $F = (f_i)$ 、$i = 1, 2, 3$ 为三个实数 (缩放因子)，分别对应每个坐标。关于原点按因子 F 进行缩放 $\mathbf{P}$ 得到的点为

$$\overline{\mathbf{P}} = (\bar{p}_i)\ \bar{p}_i = f_i p_i, i = 1, 2, 3 \tag{8.6}$$

如果 $\mathbf{C} = (c_i)$ 是一个任意点，则 $\mathbf{P}$ 通过方程在 $\mathbf{C}$ 周围进行缩放。

$$\overline{\mathbf{P}} = (\bar{p}_i)\ \bar{p}_i = f_i p_i + (1 - f_i)\, c_i, i = 1, 2, 3 \tag{8.7}$$

如果三个 f_i 相等，则称缩放为均匀；否则，它是非均匀的。

通过对旋转曲面应用非均匀缩放，可以获得几个有趣的曲面。图 8.14 显示了一个以原点为中心的椭球，其隐式方程为

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1 \tag{8.8}$$

三个坐标平面将曲面切割成三个椭圆。椭球的构造是首先生成一个以原点为中心的单位半径球体，然后按因子 $F = (a, b, c)$ 在原点周围进行缩放。

图 8.15b 显示了一个椭圆抛物面。xz 和 yz 平面将曲面切割成抛物线；xy 平面将其切割成一个椭圆。它的隐式方程为

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = cz \tag{8.9}$$

图 8.15 b 的椭圆抛物面是通过对图 8.15a 中显示的旋转抛物面在原点周围按因子 $F = (\mathrm{a,\ b}, 1)$ 进行缩放得到的。

通过将缩放因子应用于三维控制点，对 NURBS 曲面进行非均匀缩放。如果控制点存储在 350 常见曲面的构造中

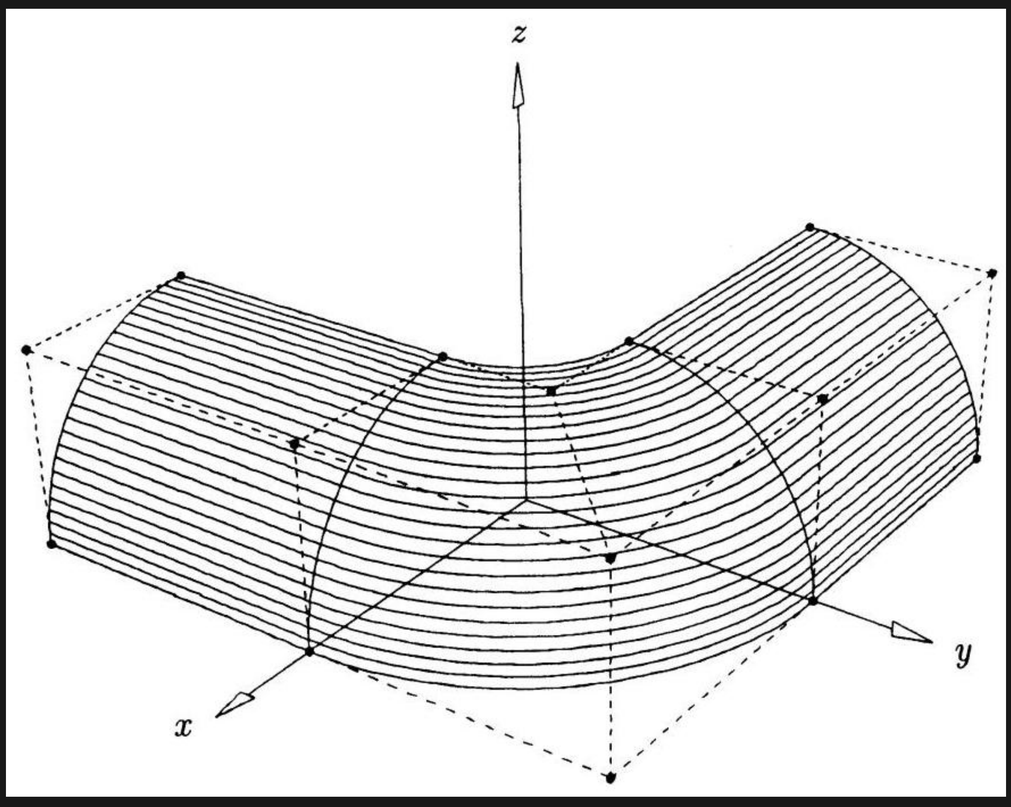


图 8.13. 一个环形补丁作为旋转曲面，用于圆滑尖锐的角落。

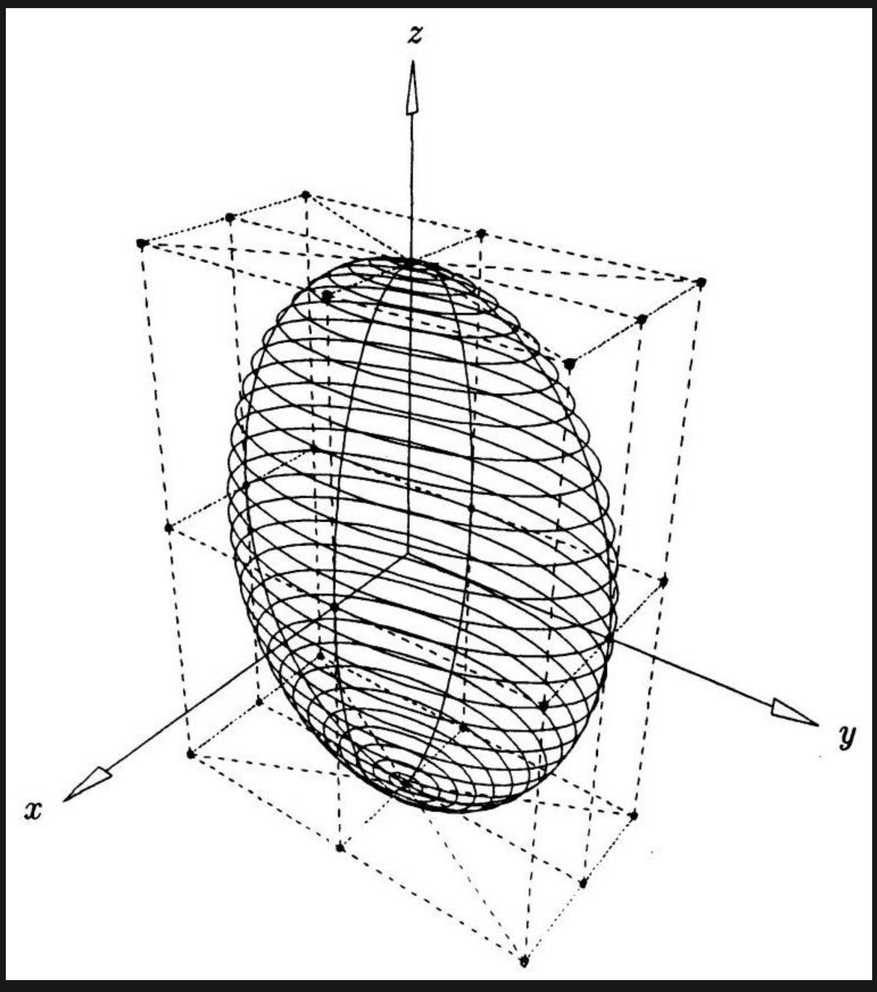


图 8.14. 通过缩放一个球体得到的椭球体。

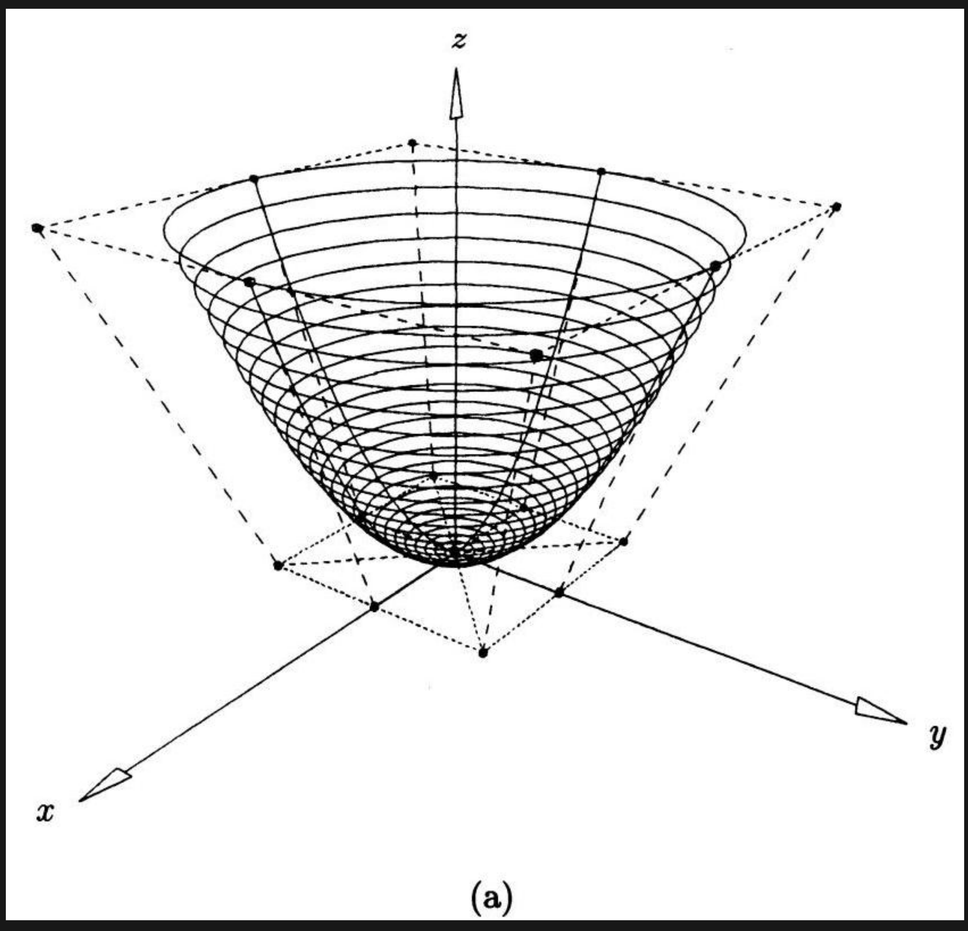


图 8.15. 抛物面。(a) 旋转抛物面；(b) 椭圆抛物面。

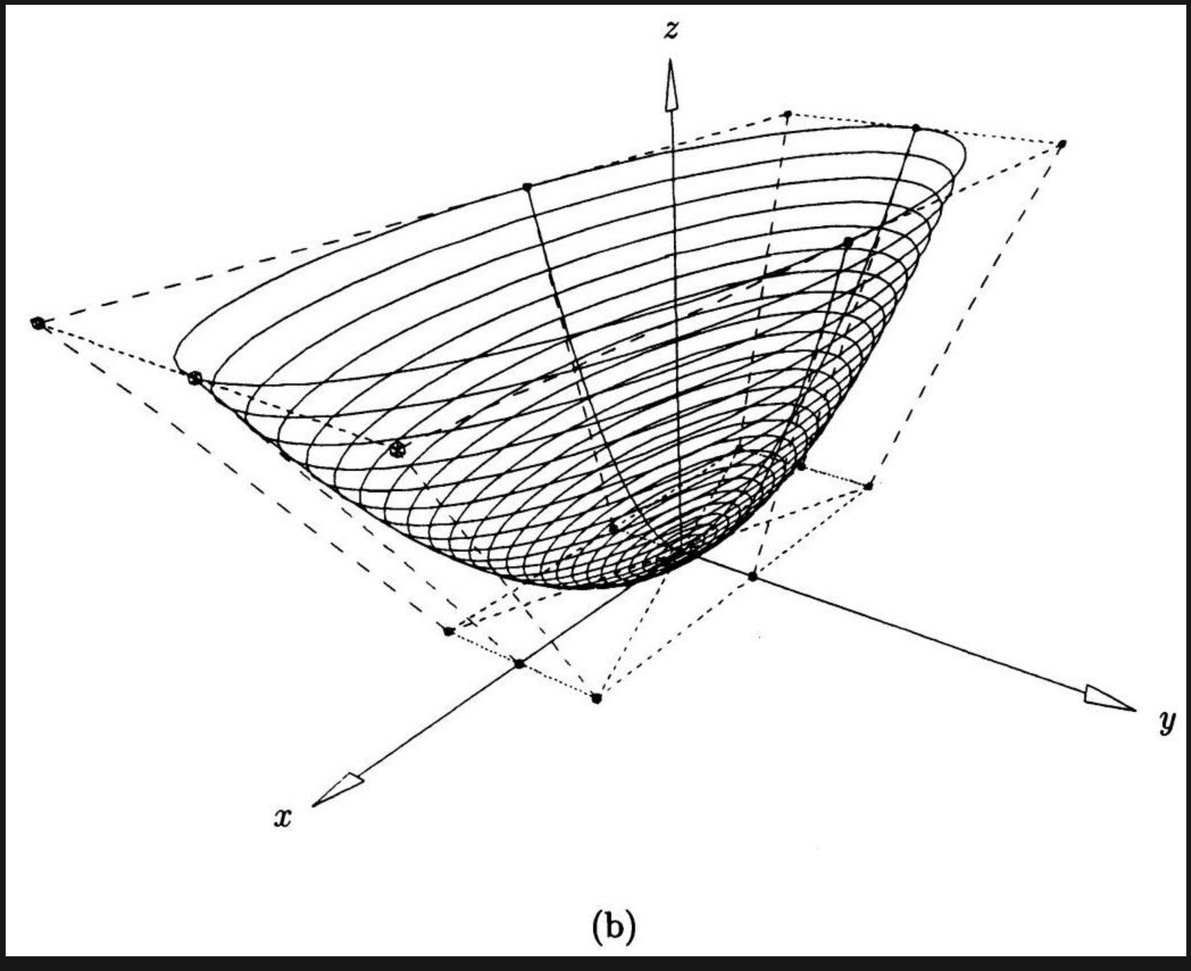


图 8.15. (续。)

在三维 (无权重) 形式下，则可以直接应用公式 (8.6) 或公式 (8.7)。权重不变。如果控制点以四维形式存储

$$\mathbf{P}^w = (p_1^w, p_2^w, p_3^w, w) = (p_i^w, w) \;\; i = 1, 2, 3$$

则只有前三个坐标会发生变化。公式 (8.6) 变为

$$\overline{\mathbf{P}}^w = (\overline{p}_i^w, w) \;\; \overline{p}_i^w = f_i p_i^w, i = 1, 2, 3 \tag{8.10}$$

公式 (8.7) 变为

$$\overline{\mathbf{P}}^w = (\overline{p}_i^w, w) \;\; \overline{p}_i^w = f_i p_i^w + (1 - f_i)\, w c_i, i = 1, 2, 3 \tag{8.11}$$

7　三边球面

当三个表面相交时，结果是三条边缘曲线在一个角点相遇 (见图 8.16a)。由于在制造零件中，尖锐的边缘和角通常是不受欢迎的，因此需要一些“平滑”或“圆滑”尖锐边缘和角的表面。这些被称为圆角表面。获得圆角表面的一个常见概念方法是滚动弹珠模型。如果一个半径为 R 的弹珠沿着所有边缘曲线滚动并进入角落，则会得到图 8.16 b 的表面 $\mathbf{S}_4 - \mathbf{S}_7$ 。$\mathbf{S}_4 - \mathbf{S}_6$ 是边缘圆角，而 $\mathbf{S}_7$ 是一个角落 352 常见表面的圆角。与各自边缘曲线方向垂直的 $\mathbf{S}_4 - \mathbf{S}_6$ 的横截面是半径为 R 的圆弧，其弧长随着表面沿边缘的角度变化而变化。一般来说，这些表面相当复杂，无法精确表示为 NURBS 形式。使用 NURBS 时，通常在用户指定的容差范围内进行近似。角落圆角 $\mathbf{S}_7$ 是一个三边补丁，位于半径为 R 的球体上。我们使用“补丁”一词来表示某个更大表面的子表面。三个边界曲线是半径为 R 的圆弧，其中心与球体的中心重合。此外，假设表面不相切，弧的扫角小于 180° ，因此每个弧都可以用一个所有权重为正的 Bézier 段表示。角落圆角可以精确表示为一个 (4×2) 次 NURBS 表面，具有一个退化边界。

除了获得角落圆角表面外，这里提出的方法是构造二次曲面上张量积补丁的更一般技术的一个简单特例。二次曲面是任何具有以下形式的二阶隐式方程的表面

$$S(x, y, z) = ax^2 + by^2 + cz^2 + dxy + exz + fyz + gx + hy + iz + j = 0 \tag{8.12}$$

球体、椭球体、双曲面、抛物面，以及圆柱体和圆锥体是二次曲面的例子。所有这些，以及许多其他重要的曲面，如环面和生成曲线为二次的旋转曲面，都是由两组最大度数为 2 的曲线生成的。例如，球体是由两组圆 (纬线和经线) 生成的，而右圆柱体则由一组直线和一组圆生成。如果补丁边界曲线是由这两组生成曲线组成的曲线的部分，则可以使用第 8.2-8.6 节中的技术在这些曲面上构建补丁。生成的曲面补丁是双二次的。如果补丁边界不遵循生成曲线 (如一般角落圆角的情况)，那么情况会更复杂，补丁甚至可能无法表示为 NURBS 曲面。对于底层曲面为二次且补丁边界为圆锥截面的情况，已经获得了一些结果和算法，例如，见 [Geis90; Boeh91; Hosc92a]。

现在我们展示如何构建角落圆角。尽管细节相当复杂，但该方法在概念上相当简单，我们只是组合两个映射。为了不失一般性，我们假设底层球体的半径为 R，并且位于 (0,0, R)（见图 8.17）。众所周知的立体投影 *projection (see [Coxe67; D*
球体 $\mathbf{S}_1(u,v) = (x(u,v), y(u,v), z(u,v))$，其中

$$x(u,v) = \frac{4R^2 u}{u^2 + v^2 + 4R^2}$$

$$y(u,v) = \frac{4R^2 v}{u^2 + v^2 + 4R^2}$$

$$z(u,v) = \frac{2R\left(u^2 + v^2\right)}{u^2 + v^2 + 4R^2} \tag{8.13}$$

即，a(u, v) 点映射到 (x, y, z) 点，该点位于球体与从 (u, v,0) 到 (0,0,2R) 的线段的交点（见图 8.17）。在此投影下，我们有以下对应关系：

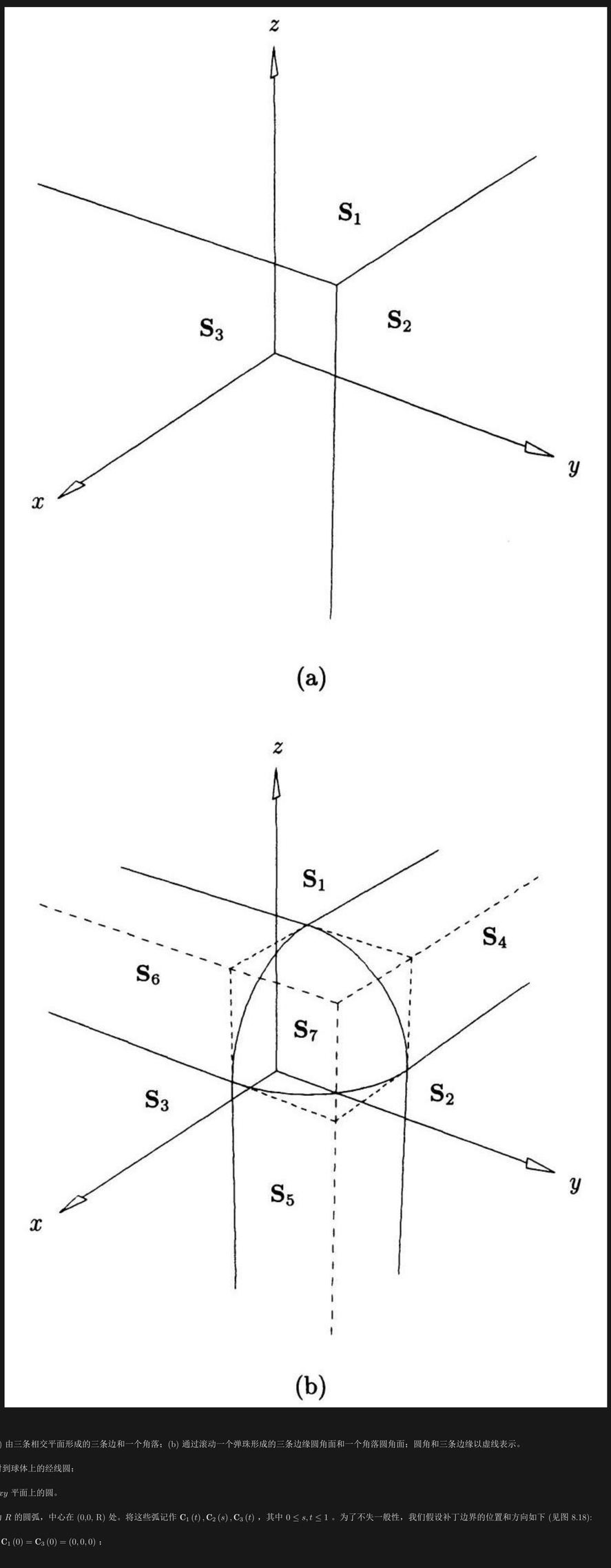


图 8.16. 圆角和一个角落。(a) 由三条相交平面形成的三条边和一个角落；(b) 通过滚动一个弹珠形成的三条边缘圆角面和一个角落圆角面；圆角和三条边缘以虚线表示。

- 通过 xy 平面原点的直线映射到球体上的经线圆；
- 球体上的所有其他圆对应于 xy 平面上的圆。

圆角表面的三个边界是半径为 R 的圆弧，中心在 (0,0, R) 处。将这些弧记作 $\mathbf{C}_1(t), \mathbf{C}_2(s), \mathbf{C}_3(t)$，其中 $0 \le s, t \le 1$。为了不失一般性，我们假设补丁边界的位置和方向如下（见图 8.18）：

- $\mathbf{C}_1(t)$ 和 $\mathbf{C}_3(t)$ 从原点开始：$\mathbf{C}_1(0) = \mathbf{C}_3(0) = (0,0,0)$；
- $\mathbf{C}_1(t)$ 位于 xz 平面内；
- $\mathbf{C}_1(1) = \mathbf{C}_2(0)$ 和 $\mathbf{C}_2(1) = \mathbf{C}_3(1)$。

然后，圆角补丁的投影产生一个平面，$\mathbf{S}_2(s,t)=(u(s,t)，v(s,t),0)$ 在 xy 平面内。$s=0$ 和 $s=1$ 的边界是直线，$t=0$ 边界是退化的，而 $t=1$ 边界是一个圆弧。因此，$\mathbf{S}_2(s,t)$ 的度数为 (2,1)，而复合映射 $\mathbf{S}(s,t)=\mathbf{S}_1\circ\mathbf{S}_2$ 产生了一个 (4×2) 度的有理 Bézier 表示。

显然，可以推导出计算 $\mathbf{S}(s,t)$ 的控制点和权重的一般公式。然而，由于大多数项本身为零，我们特别为每个坐标和权重分别推导公式。计算过程为:

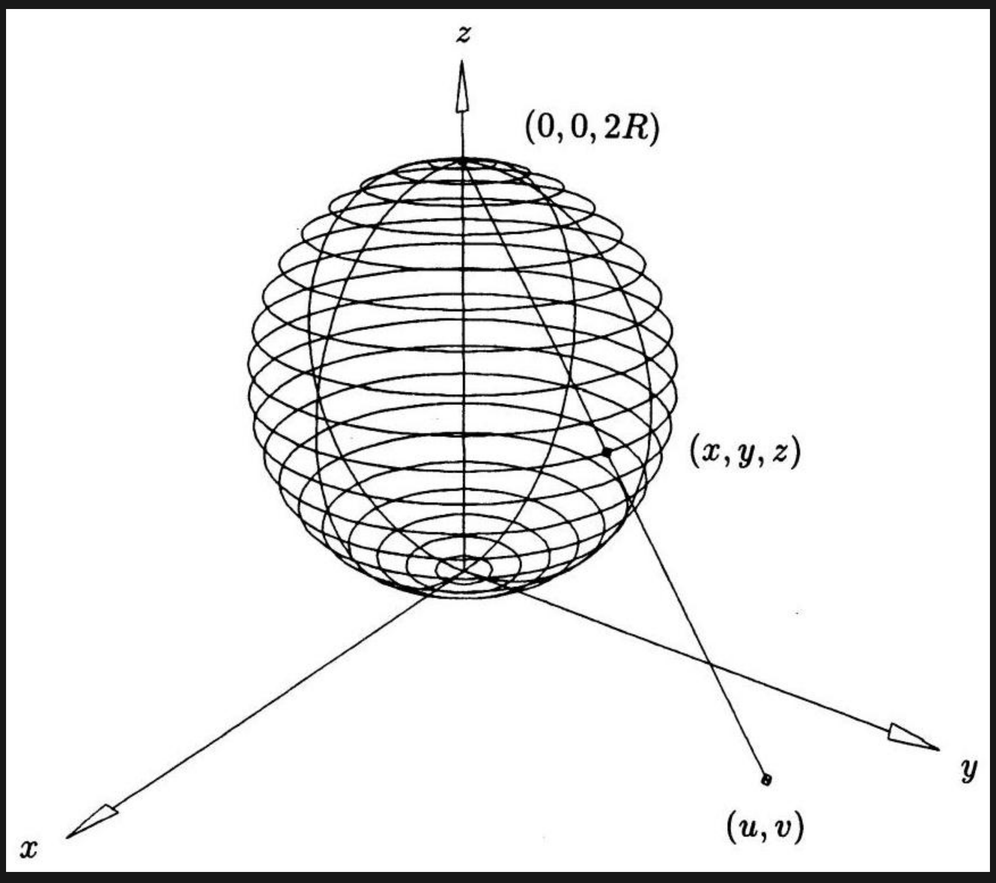


图 8.17. 立体投影。

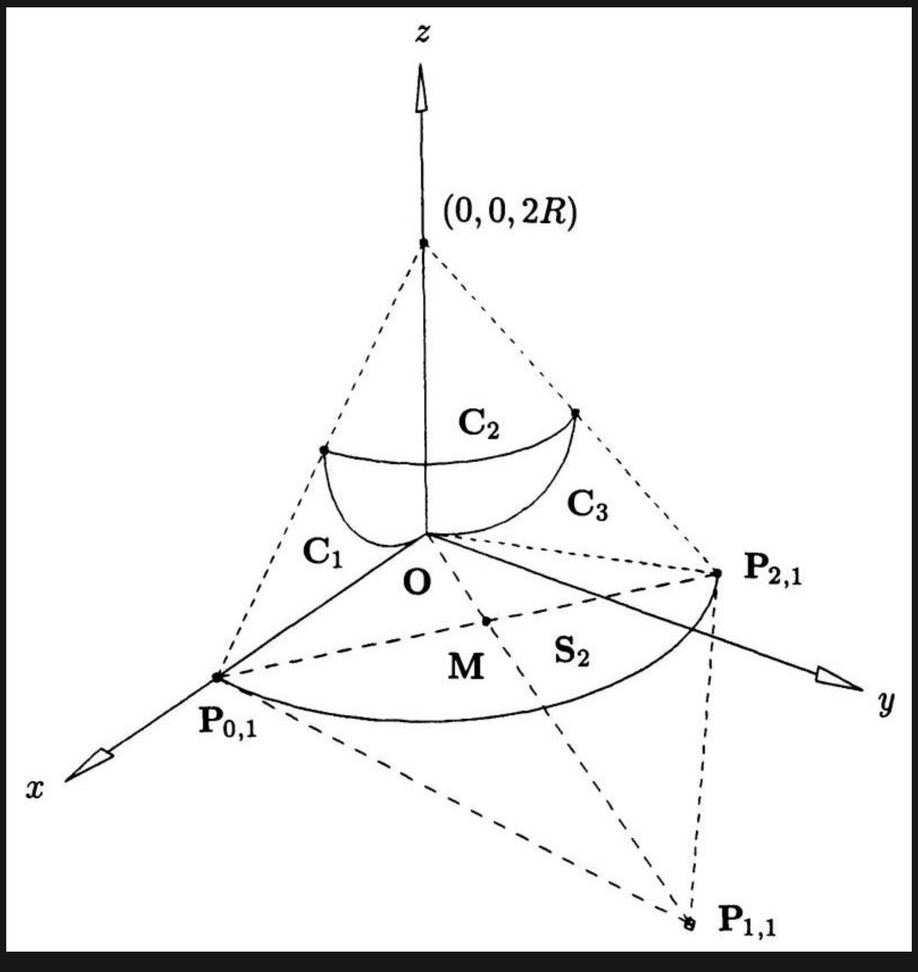


图 8.18. 通过将圆角表面投影到 xy 平面上获得的平面 $\mathbf{S}_2$ 。

1. 获取 $\mathbf{S}_2(s,t)$ 的贝塞尔表示；
2. 将 $\mathbf{S}_2(s,t)$ 转换为幂基形式；
3. 将 $\mathbf{S}_2(s,t)$ 的幂基系数代入 $\mathbf{S}_1(u,v)$ (方程 [8.13]) 以获得圆角补丁的幂基形式 $\mathbf{S}(s,t)$ ；
4. 将 $\mathbf{S}(s,t)$ 转换为贝塞尔形式。

$\mathbf{S}_2(s,t)$ 的贝塞尔表示需要六个四维控制点 $\mathbf{P}^w_{i,j},0\le i\le2,0\le j\le1$ 。显然，$\mathbf{P}_{0,0}=\mathbf{P}_{1,0}=\mathbf{P}_{2,0}=(0,0,0)$ ，其中 $w_{0,0}=w_{2,0}=1.w_{1,0}$ 被设定为等于 $w_{1,1}$ ，这仍需稍后计算。$\mathbf{P}^w_{i,1},0\le i\le2$ 是圆弧的控制点，它是 $\mathbf{C}_2(s)$ 的投影。它们的获取方法如下:

1. 将三点 $\mathbf{C}_2(0),\mathbf{C}_2(1/2)$ 和 $\mathbf{C}_2(1)$ 投影到 xy 平面中的圆弧上。设 (x, y, z) 为这些点之一的坐标；则投影的 uv 坐标通过公式计算得出。

$$\alpha=\frac{-2R}{z-2R}\ u=\alpha x,v=\alpha y \tag{8.14}$$

这得到了 $\mathbf{P}^w_{0,1}$ 和 $\mathbf{P}^w_{2,1}(w_{0,1}=w_{2,1}=1)$ 。

2. 计算投影弧的中心 (图 8.18 中的 O) 和半径 r ； 例如，通过交叉三点 $\mathbf{C}_2(0)$ 、$\mathbf{C}_2(1/2)$ 和 $\mathbf{C}_2(1)$ 投影形成的两条弦的垂直平分线找到 $\mathbf{O}$ ；
3. 然后通过 (见图 8.18) 计算 $\mathbf{P}^w_{1,1}$:

- 计算 $\mathbf{M}=1/2(\mathbf{P}_{0,1}+\mathbf{P}_{2,1})$ ；
- $\mathbf{P}_{1,1}$ 是关于投影圆弧的 $\mathbf{M}$ 的逆 (见 [Coxe67])，因此 $\mathbf{OM}\cdot\mathbf{OP}_{1,1}=r^2$ ；从中我们得到

$$\mathbf{P}_{1,1}=(1-\alpha)\,\mathbf{O}+\alpha\mathbf{M}$$

其中

$$\alpha=\frac{r^2}{|\mathbf{OM}|^2}$$

- 从方程 (7.33) 可得 $w_{1,1}=\cos\angle\mathbf{P}_{1,1}\mathbf{P}_{2,1}\mathbf{P}_{0,1}$.

设 $\left[\mathbf{a}_{i,j}^{w}\right]$ 表示 $\mathbf{S}_{2}^{w}(s,t)$ 的幂基系数。从方程 (6.91) 可以得出

$$\left[\mathbf{a}_{i,j}^{w}\right]=M_{2}\left[\mathbf{P}_{i,j}^{w}\right]M_{1}^{T}$$

其中 M_2 和 M_1 分别是二阶和一阶贝塞尔矩阵 (方程 [6.80])。应用矩阵乘法得到

$$\mathbf{a}_{0,0}^{w}=\mathbf{P}_{0,0}^{w}$$

$$\mathbf{a}_{0,1}^{w}=\mathbf{P}_{0,1}^{w}-\mathbf{P}_{0,0}^{w}$$

$$\mathbf{a}_{1,0}^{w}=2\left(\mathbf{P}_{1,0}^{w}-\mathbf{P}_{0,0}^{w}\right)$$

$$\mathbf{a}_{1,1}^{w}=2\left(\mathbf{P}_{1,1}^{w}+\mathbf{P}_{0,0}^{w}-\mathbf{P}_{0,1}^{w}-\mathbf{P}_{1,0}^{w}\right)$$

$$\mathbf{a}_{2,0}^{w}=\mathbf{P}_{0,0}^{w}-2\mathbf{P}_{1,0}^{w}+\mathbf{P}_{2,0}^{w}$$

$$\mathbf{a}_{2,1}^{w}=\mathbf{P}_{0,1}^{w}+\mathbf{P}_{2,1}^{w}-\mathbf{P}_{0,0}^{w}-\mathbf{P}_{2,0}^{w}+2\left(\mathbf{P}_{1,0}^{w}-\mathbf{P}_{1,1}^{w}\right)\tag{8.15}$$

现在设

$$\mathbf{a}_{i,j}^{w}=\left(u_{i,j},v_{i,j},h_{i,j}\right)\tag{8.16}$$

(注意 $u_{i,j}$ 和 $v_{i,j}$ 是加权的)。z 坐标为零，为了避免后续的冲突，我们使用 h 代替 w 作为权重。将圆弧的控制点表示为 $\mathbf{P}_{i,1}^{w}=\left(\alpha_{i},\beta_{i},\gamma_{i}\right),0\leq i\leq 2$ (α_i 和 β_i) 加权) 。假设角权重为 1，并考虑到 $\mathbf{C}_1(t)$ 位于 xz 平面且 $\mathbf{C}_1(0)=(0,0,0)$ ，可以得出唯一非零的 $u_{i,j},v_{i,j}$ 和 $h_{i,j}$ 是

$$u_{0,1}=\alpha_{0}$$

$$u_{1,1}=2\left(\alpha_{1}-\alpha_{0}\right)$$

$$u_{2,1}=\alpha_{2}+\alpha_{0}-2\alpha_{1}$$

$$v_{1,1}=2\beta_{1}$$

$$v_{2,1}=\beta_{2}-2\beta_{1}$$

$$h_{0,0}=1$$

$$h_{1,0}=2\left(\gamma_{1}-1\right)$$

$$h_{2,0}=2\left(1-\gamma_{1}\right)\tag{8.17}$$

使用方程 (8.16) 和 (8.17) 以及幂基形式
我们得到

$$\mathbf{S}_{2}^{w}(s,t)=\begin{bmatrix}1 & s & s^{2}\end{bmatrix}\left[\mathbf{a}_{i,j}^{w}\right]\begin{bmatrix}1\\ t\end{bmatrix}\tag{8.18}$$

$$\mathbf{S}_{2}(s,t)=\left(u(s,t),v(s,t)\right)$$

其中

$$u(s,t)=\frac{U(s,t)}{H(s,t)}\,v(s,t)=\frac{V(s,t)}{H(s,t)}\tag{8.19}$$

和

$$U(s,t)=u_{0,1}t+u_{1,1}st+u_{2,1}s^{2}t$$

$$V(s,t)=v_{1,1}st+v_{2,1}s^{2}t$$

$$H(s,t)=h_{0,0}+h_{1,0}s+h_{2,0}s^{2}\tag{8.20}$$

将方程 (8.19) 和 (8.20) 代入方程 (8.13)，我们得到

$$\mathbf{S}(s,t)=\left(x(s,t),y(s,t),z(s,t)\right)\tag{8.21}$$

$$x(s,t)=\frac{X(s,t)}{W(s,t)}\,y(s,t)=\frac{Y(s,t)}{W(s,t)}\,z(s,t)=\frac{Z(s,t)}{W(s,t)}\tag{8.22}$$

和 $X(s,t)=4R^{2}U(s,t)H(s,t)$

$$=4R^{2}\left(u_{0,1}t+u_{1,1}st+u_{2,1}s^{2}t\right)\left(h_{0,0}+h_{1,0}s+h_{2,0}s^{2}\right)$$

$$=4R^{2}\left[u_{0,1}h_{0,0}t+\left(u_{1,1}h_{0,0}+u_{0,1}h_{1,0}\right)st\right.$$

$$+\left(u_{2,1}h_{0,0}+u_{1,1}h_{1,0}+u_{0,1}h_{2,0}\right)s^{2}t$$

$$\left.+\left(u_{2,1}h_{1,0}+u_{1,1}h_{2,0}\right)s^{3}t+u_{2,1}h_{2,0}s^{4}t\right]\tag{8.23}$$

$$Y(s,t)=4R^{2}V(s,t)H(s,t)$$

$$=4R^{2}\left(v_{1,1}st+v_{2,1}s^{2}t\right)\left(h_{0,0}+h_{1,0}s+h_{2,0}s^{2}\right)$$

$$=4R^{2}[v_{1,1}h_{0,0}st+\left(v_{2,1}h_{0,0}+v_{1,1}h_{1,0}\right)s^{2}t$$

$$\left.+\left(v_{1,1}h_{2,0}+v_{2,1}h_{1,0}\right)s^{3}t+v_{2,1}h_{2,0}s^{4}t\right]\tag{8.24}$$

$$Z(s,t)=2R\left[U(s,t)^{2}+V(s,t)^{2}\right]$$

$$=2R\left[u_{0,1}^{2}t^{2}+2u_{0,1}u_{1,1}st^{2}+\left(u_{1,1}^{2}+2u_{0,1}u_{2,1}+v_{1,1}^{2}\right)s^{2}t^{2}\right.$$

$$\left.+2\left(u_{1,1}u_{2,1}+v_{1,1}v_{2,1}\right)s^{3}t^{2}+\left(u_{2,1}^{2}+v_{2,1}^{2}\right)s^{4}t^{2}\right]\tag{8.25}$$

$$W(s,t)=U(s,t)^{2}+V(s,t)^{2}+4R^{2}H(s,t)^{2}$$

$$=4R^{2}h_{0,0}^{2}+8R^{2}h_{0,0}h_{1,0}s+4R^{2}\left(h_{1,0}^{2}+2h_{0,0}h_{2,0}\right)s^{2}+u_{0,1}^{2}t^{2}$$

$$+8R^{2}h_{1,0}h_{2,0}s^{3}+2u_{0,1}u_{1,1}st^{2}+4R^{2}h_{2,0}^{2}s^{4}$$

$$+\left(u_{1,1}^{2}+2u_{0,1}u_{2,1}+v_{1,1}^{2}\right)s^{2}t^{2}$$

$$+2\left(u_{1,1}u_{2,1}+v_{1,1}v_{2,1}\right)s^{3}t^{2}+\left(u_{2,1}^{2}+v_{2,1}^{2}\right)s^{4}t^{2}\tag{8.26}$$

记作

$$\mathbf{S}^w(s,t) = \left[s^i\right]\left[\mathbf{b}^w_{i,j}\right]\left[t^j\right]^T \; 0 \le i \le 4 \; 0 \le j \le 2 \tag{8.27}$$

$\mathbf{S}^w(s,t)$ 的幂基表示，并设 $\mathbf{b}^w_{i,j} = (x_{i,j}, y_{i,j}, z_{i,j}, w_{i,j})$ 。$\mathbf{b}^w_{i,j}$ 的大多数坐标为零。唯一的非零元素是

$$\begin{aligned}
x_{0,1} &= 4R^2 u_{0,1} h_{0,0} \\
x_{1,1} &= 4R^2 \left(u_{1,1} h_{0,0} + u_{0,1} h_{1,0}\right) \\
x_{2,1} &= 4R^2 \left(u_{2,1} h_{0,0} + u_{1,1} h_{1,0} + u_{0,1} h_{2,0}\right) \\
x_{3,1} &= 4R^2 \left(u_{2,1} h_{1,0} + u_{1,1} h_{2,0}\right) \\
x_{4,1} &= 4R^2 u_{2,1} h_{2,0}
\end{aligned} \tag{8.28}$$

$$\begin{aligned}
y_{1,1} &= 4R^2 v_{1,1} h_{0,0} \\
y_{2,1} &= 4R^2 \left(v_{2,1} h_{0,0} + v_{1,1} h_{1,0}\right) \\
y_{3,1} &= 4R^2 \left(v_{1,1} h_{2,0} + v_{2,1} h_{1,0}\right) \\
y_{4,1} &= 4R^2 v_{2,1} h_{2,0}
\end{aligned} \tag{8.29}$$

$$\begin{aligned}
z_{0,2} &= 2R u_{0,1}^2 \\
z_{1,2} &= 4R u_{0,1} u_{1,1} \\
z_{2,2} &= 2R \left(u_{1,1}^2 + 2u_{0,1} u_{2,1} + v_{1,1}^2\right) \\
z_{3,2} &= 4R \left(u_{1,1} u_{2,1} + v_{1,1} v_{2,1}\right) \\
z_{4,2} &= 2R \left(u_{2,1}^2 + v_{2,1}^2\right)
\end{aligned} \tag{8.30}$$

$$\begin{aligned}
w_{0,0} &= 4R^2 h_{0,0}^2 \\
w_{1,0} &= 8R^2 h_{0,0} h_{1,0} \\
w_{2,0} &= 4R^2 \left(h_{1,0}^2 + 2h_{0,0} h_{2,0}\right) \\
w_{3,0} &= 8R^2 h_{1,0} h_{2,0} \\
w_{4,0} &= 4R^2 h_{2,0}^2 \\
w_{0,2} &= u_{0,1}^2 \\
w_{1,2} &= 2u_{0,1} u_{1,1} \\
w_{2,2} &= u_{1,1}^2 + 2u_{0,1} u_{2,1} + v_{1,1}^2 \\
w_{3,2} &= 2\left(u_{1,1} u_{2,1} + v_{1,1} v_{2,1}\right) \\
w_{4,2} &= u_{2,1}^2 + v_{2,1}^2
\end{aligned} \tag{8.31}$$

方程 (8.27)-(8.31) 给出了填角表面 $\mathbf{S}^w(s,t)$ 的幂基形式。通过应用方程 (6.100) 得到 Bézier (NURBS) 表示，即

$$\left[\mathbf{P}^w_{i,j}\right] = M_4^{-1}\left[\mathbf{b}^w_{i,j}\right]\left(M_2^{-1}\right)^T$$

算法 A6.2 用于计算 M_4^{-1} 和 M_2^{-1} 。

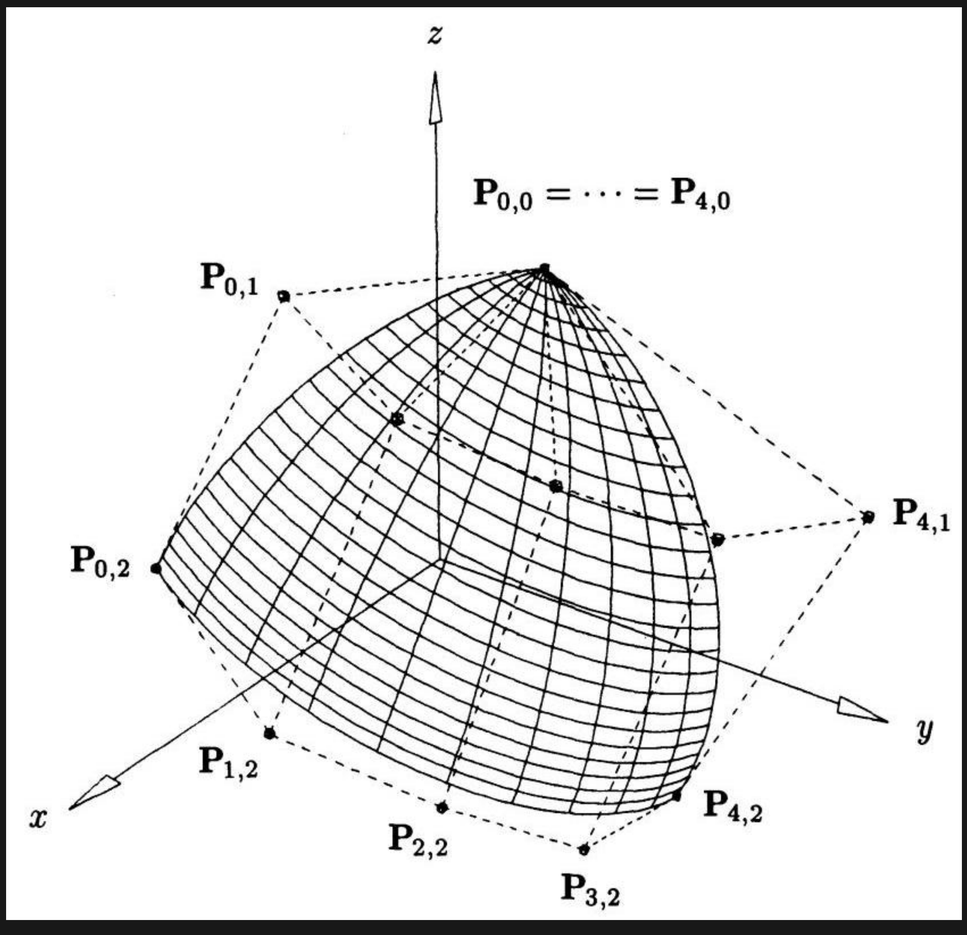


图 8.19。一个角落填角表面 - 一个任意的球面补丁，其边界是球体的大圆。

现在我们总结完整的算法。算法的输入是三个边界弧 $\mathbf{C}_1, \mathbf{C}_2$ 和 $\mathbf{C}_3$ ，它们在空间中任意定位和定向，但在其端点处相连。输出是一个 (4×2) 度的有理 Bézier 填角补丁。

算法 A8.2

MakeCornerFilletSurf (C1, C2, C3, Pwij)

{ /* 创建 NURBS 角落填角表面 */

/* 输入: C1, C2, C3 */

/* 输出: Pwij */

1. 计算三个边界弧的公共半径和中心点。

2. 使用平移、旋转以及可能的曲线反转 (见第 6 章)，将边界弧定位和定向，如图 8.18 所示。请注意，实际上只有 $\mathbf{C}_2$ 需要被定位和定向，因为只有它是计算三个控制点 $\mathbf{P}^w_{i,1} = (\alpha_i, \beta_i, \gamma_i)$. 所必需的。

3. 按照上述描述计算 $(\alpha_i, \beta_i, \gamma_i)$ 。

4. 将 $\mathbf{S}^w_2(s,t)$ 从贝塞尔形式转换为幂基形式 (方程 [8.17])。

5. 计算圆角补丁 $\mathbf{S}^w = \mathbf{S}^w_1 \circ \mathbf{S}^w_2$ 的幂基形式 (方程 [8.27] - [8.31])。

6. 将 $\mathbf{S}^w(s,t)$ 转换为有理贝塞尔形式 (方程 [8.32] 和算法 A6.2)。

7. 将 $\mathbf{S}(s,t)$ 旋转/平移回三维空间中的正确位置 (步骤 2 的逆变换)。

}

第九章　曲线和曲面拟合

1　引言

在第七章和第八章中，我们展示了如何构建常见且相对简单的曲线和曲面的 NURBS 表示，例如圆、圆锥曲线、圆柱体、旋转曲面等。这些实体只需少量数据项即可指定，例如中心点、高度、半径、旋转轴等。此外，这些少量数据项唯一地指定了几何实体。在本章中，我们进入自由形状 (或雕刻) 曲线和曲面的领域。我们研究拟合，即构建适合相当任意几何数据集 (如点和导数向量) 的 NURBS 曲线和曲面。我们区分两种类型的拟合: 插值和近似。在插值中，我们构建一条精确满足给定数据的曲线或曲面，例如，曲线通过给定的点，并在规定的点处具有给定的导数。图 9.1 显示了一条插值五个点及其端点的第一导数向量的曲线。在近似中，我们构建的曲线和曲面不一定精确满足给定数据，而只是大致满足。在某些应用中——例如通过使用坐标测量设备或数字化平板生成点数据，或通过行进方法计算表面/表面交点——可以生成大量点，这些点可能包含测量或计算噪声。在这种情况下，曲线或曲面捕捉数据的“形状”是重要的，但不需要“摇摆”通过每一个点。在近似中，通常希望对曲线或曲面与给定数据的偏差指定一个最大界限，并指定某些约束条件，即需要精确满足的数据。图 9.2 显示了一条近似一组 $m+1$ 点的曲线。指定了最大偏差界限 E ，而垂直距离 e_i 是通过将 $\mathbf{Q}_i$ 投影到曲线上获得的近似误差。每个点 $\mathbf{Q}_i$ 的 e_i 小于 E 。端点 $\mathbf{Q}_0$ 和 $\mathbf{Q}_m$ 被指定为约束，结果是 $e_0 = e_m = 0$ 。

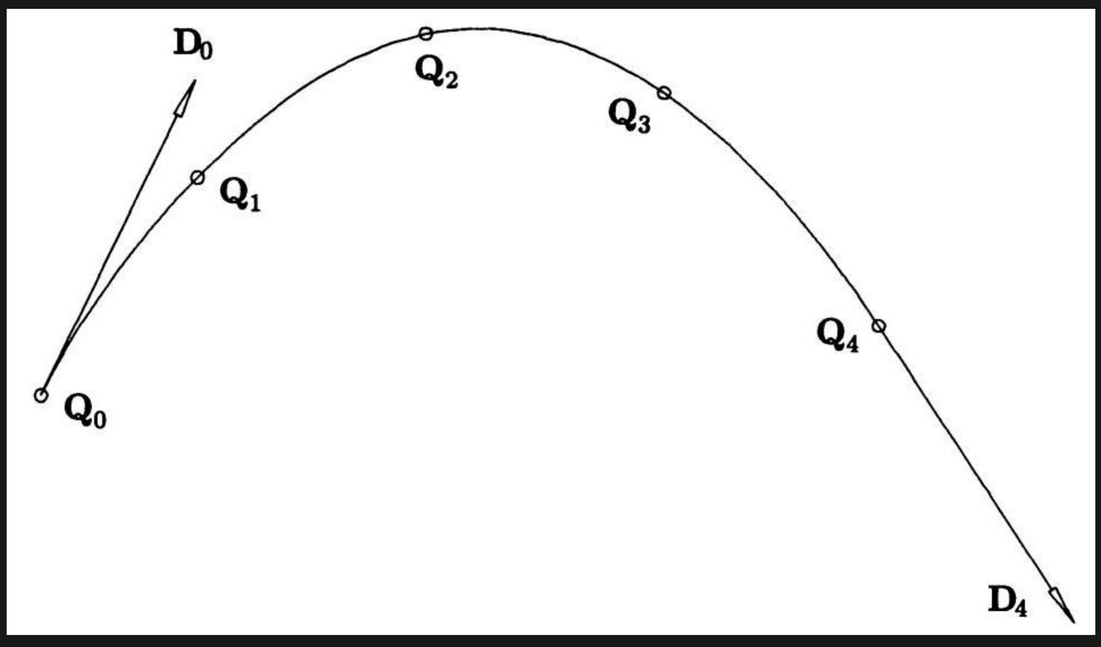


图 9.1. 一条插值五个点和两个端导数的曲线。

拟合中有许多微妙之处，实际上已有数百篇论文讨论这一主题。许多技术是启发式的，通常没有唯一或明确的“正确”答案。一个根本问题是，给定的数据从未指定唯一解。存在无限多条 NURBS 曲线可以插值图 9.1 中的七个数据项，或近似图 9.2 中的 $m+1$ 项。实施拟合算法的开发者常常被使用软件的设计师告知: “但这不是我想要的曲线!”而回复通常是: “好吧，这在数学上是正确的；它满足你给我的数据!”

拟合问题的输入通常由几何数据组成，例如点和导数。输出是 NURBS 曲线或曲面，即控制点、节点和权重。此外，必须输入度 p (或曲面的 (p, q))，或者算法必须选择一个合适的度。如果希望曲线具有 C^r 连续性，则所选的度 p 必须满足

$$p \geq r+1 \tag{9.1}$$

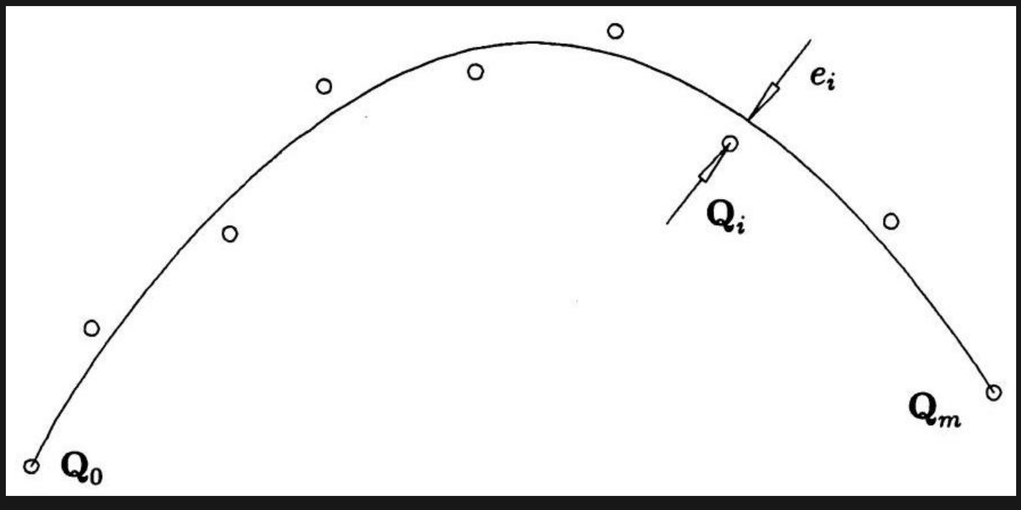


图 9.2. 一条近似 $m+1$ 点的曲线；该曲线被约束通过端点 $\mathbf{Q}_0$ 和 $\mathbf{Q}_m$ 。

(假设没有重数为 >1 的内部节点)。假设没有其他要求，选择 $p=r+1$ 通常足以进行插值。对于逼近，选择 $p>r+1$ 可能会产生更好的结果。例如，假设我们想要一个 C^1 连续曲线来逼近表示三维空间中扭曲曲线的点数据。使用三次曲线可能会产生比二次曲线更少的控制点，尽管二次曲线能提供所需的连续性。关于在拟合过程中设置权重的文献非常少。通常，所有权重都简单地设置为 1。实际上，对于插值，可能没有理由去做其他选择 (一个例外是使用分段圆弧拟合时 [Hosc92b])。对于逼近，允许权重为任意值应该会产生具有更少控制点的曲线和曲面；我们提出了一种这样的算法。最后，有许多选择节点的方法，其中大多数是启发式的。

大多数拟合算法可以分为两类: 全局算法或局部算法。全局算法通过建立和求解方程组或优化问题来进行。如果给定的数据仅包含点和导数，并且控制点是唯一的未知数 (度数、节点和权重已被预先选择)，那么系统是线性的，因此容易求解。如果给定了更复杂的数据，例如曲率，或者节点和/或权重也是系统未知数，那么得到的系统就是非线性的。从理论上讲，任何一个输入数据项的扰动都可以改变整个曲线或表面的形状；然而，变化的幅度随着与受影响数据项的距离增加而减小。局部算法在本质上更具几何特征，逐段构建曲线或表面，仅使用每一步的局部数据。数据项的扰动仅在局部改变曲线或表面。这些算法通常比全局方法计算成本更低。它们也能更好地处理尖点、直线段和其他局部数据异常。然而，在段边界实现期望的连续性水平更为棘手，局部方法通常导致多个内部节点。

在后续章节中，我们将介绍多种全局和局部的曲线和表面插值与逼近方法。这些方法因其经过验证的实用性而被选中，并且它们代表了广泛的技术交叉。我们强调，本章不涉及交互式曲线和表面设计，尽管这里涵盖的许多技术适用于该主题。在 NURBS 框架中的交互式设计通常采用以下一种 (或两种) 方法:

- 给定一个 NURBS 曲线或表面，通过修改权重和/或控制点交互式地改变其形状；这些技术在第 11 章中进行了讨论；
- 交互式构建 NURBS 曲线或曲面。局部拟合技术特别适合此任务。可以单独构建和塑造任何类型的多项式或有理段 (例如，Hermite，Bézier)。除了局部控制外，一些方法还提供全局控制，例如全局张力参数。无论采用何种方法，如果结果是分段多项式或有理曲线或曲面，通常可以在最后一步转换为 NURBS 形式。

2　全局插值

在本节中，我们提出了将曲线插值到点和导数数据的方法，以及将曲面插值到点数据的方法。

2.1　针对点数据的全局曲线插值

假设我们给定一组点 $\{\mathbf{Q}_k\}, k=0,\ldots,n$ ，并希望用一个 p 次非有理 B 样条曲线来插值这些点。如果我们为每个 $\mathbf{Q}_k$ 分配一个参数值 $\bar{u}_k$ ，并选择一个合适的节点向量 $U=\{u_0,\ldots,u_m\}$ ，我们可以建立 $(n+1)\times(n+1)$ 个线性方程组。

$$\mathbf{Q}_k = \mathbf{C}(\bar{u}_k) = \sum_{i=0}^{n} N_{i,p}(\bar{u}_k)\,\mathbf{P}_i \tag{9.2}$$

控制点 $\mathbf{P}_i$ 是 $n+1$ 个未知数。设 r 为 $\mathbf{Q}_k$ 中的坐标数 (通常为 2、3 或 4)。请注意，这种方法与 r 无关；方程 (9.2) 有一个系数矩阵，具有 r 个右侧，并且相应地，对于 $\mathbf{P}_i$ 的 r 个坐标有 r 个解集。

选择 $\bar{u}_k$ 和 U 的问题仍然存在，它们的选择会影响曲线的形状和参数化。在本节中，我们假设参数位于范围 $u\in[0,1]$ 内。选择 $\bar{u}_k$ 的三种常见方法是:

- 等距分布:

$$\bar{u}_0 = 0 \; \bar{u}_n = 1$$

$$\bar{u}_k = \frac{k}{n} \quad k=1,\ldots,n-1 \tag{9.3}$$

这种方法不推荐使用，因为当数据不均匀分布时，它可能会产生不规则的形状 (例如环)。

- 弦长: 设 d 为总弦长

$$d = \sum_{k=1}^{n} |\mathbf{Q}_k - \mathbf{Q}_{k-1}| \tag{9.4}$$

然后

$$\bar{u}_0 = 0 \; \bar{u}_n = 1$$

$$\bar{u}_k = \bar{u}_{k-1} + \frac{|\mathbf{Q}_k - \mathbf{Q}_{k-1}|}{d} \; k = 1, \ldots, n-1 \tag{9.5}$$

这是最广泛使用的方法，通常是足够的。它还为曲线提供了“良好”的参数化，意味着它近似于均匀参数化。

- 向心法: 设

$$d = \sum_{k=1}^{n} \sqrt{|\mathbf{Q}_k - \mathbf{Q}_{k-1}|}$$

然后 $\bar{u}_0 = 0 \; \bar{u}_n = 1$

$$\bar{u}_k = \bar{u}_{k-1} + \frac{\sqrt{|\mathbf{Q}_k - \mathbf{Q}_{k-1}|}}{d} \; k = 1, \ldots, n-1 \tag{9.6}$$

这是一种较新的方法 (见 [Lee89])，在数据发生急剧转弯时，它比弦长法提供更好的结果。

节点可以均匀分布，即，

$$u_0 = \cdots = u_p = 0 \; u_{m-p} = \cdots = u_m = 1$$

$$u_{j+p} = \frac{j}{n-p+1} \; j = 1, \ldots, n-p \tag{9.7}$$

然而，这种方法不推荐使用；如果与方程 (9.5) 或 (9.6) 一起使用，可能会导致奇异的方程组 (方程 [9.2])。我们推荐以下平均技术

$$u_0 = \cdots = u_p = 0 \; u_{m-p} = \cdots = u_m = 1$$

$$u_{j+p} = \frac{1}{p} \sum_{i=j}^{j+p-1} \bar{u}_i \; j = 1, \ldots, n-p \tag{9.8}$$

使用这种方法，节点反映了 $\bar{u}_k$ 的分布。此外，使用方程 (9.8) 结合方程 (9.5) 或 (9.6) 来计算 $\bar{u}_k$ 会导致一个系统 (方程 [9.2])，该系统是完全正的并且带状，半带宽小于 p (见 [DeBo78])，即，如果 $|i-k| \geq p$ 。因此，可以通过高斯消元法无需主元进行求解。

图 9.3 显示了控制点、参数和插值七个点的三次曲线的节点向量。参数通过弦长法选择，节点通过平均参数获得 (公式 [9.8])。在图 9.4 中，展示了不同参数化的比较。图 9.5 显示了使用更“分散”的数据点进行的相同比较。在这两种情况下，三次曲线都通过七个点，使用均匀参数和均匀节点 (实线曲线和顶部节点向量 - 参见公式 [9.3] 和 [9.7])；弦长参数和通过平均获得的节点 (虚线曲线和中间节点向量 - 参见公式 [9.5] 和 [9.8])；以及通过平均获得的向心参数和节点 (点划线曲线和底部节点向量 - 参见公式 [9.6] 和 [9.8])。在图 9.5 中，请注意弦长和向心参数化曲线如何适应点间距的变化。

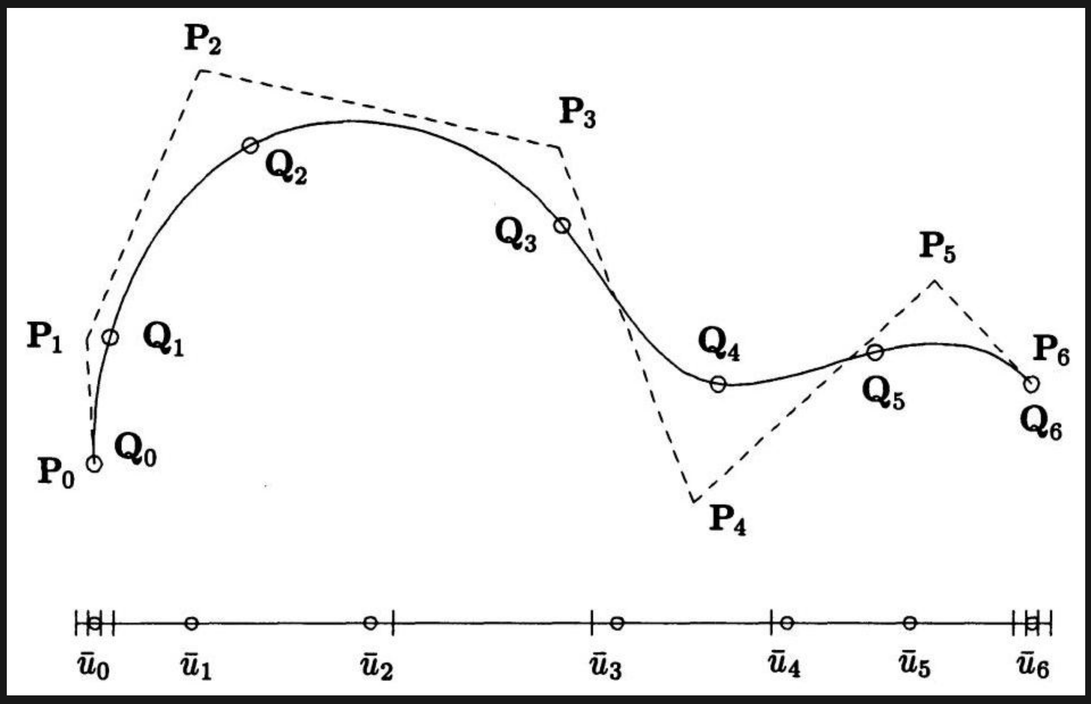


图 9.3. 使用弦长参数化和通过平均参数获得的节点向量的曲线插值示例。

节点向量 - 参见公式 [9.5] 和 [9.8])；以及通过平均获得的向心参数和节点 (点划线曲线和底部节点向量 - 参见公式 [9.6] 和 [9.8])。在图 9.5 中，请注意弦长和向心参数化曲线如何适应点间距的变化。

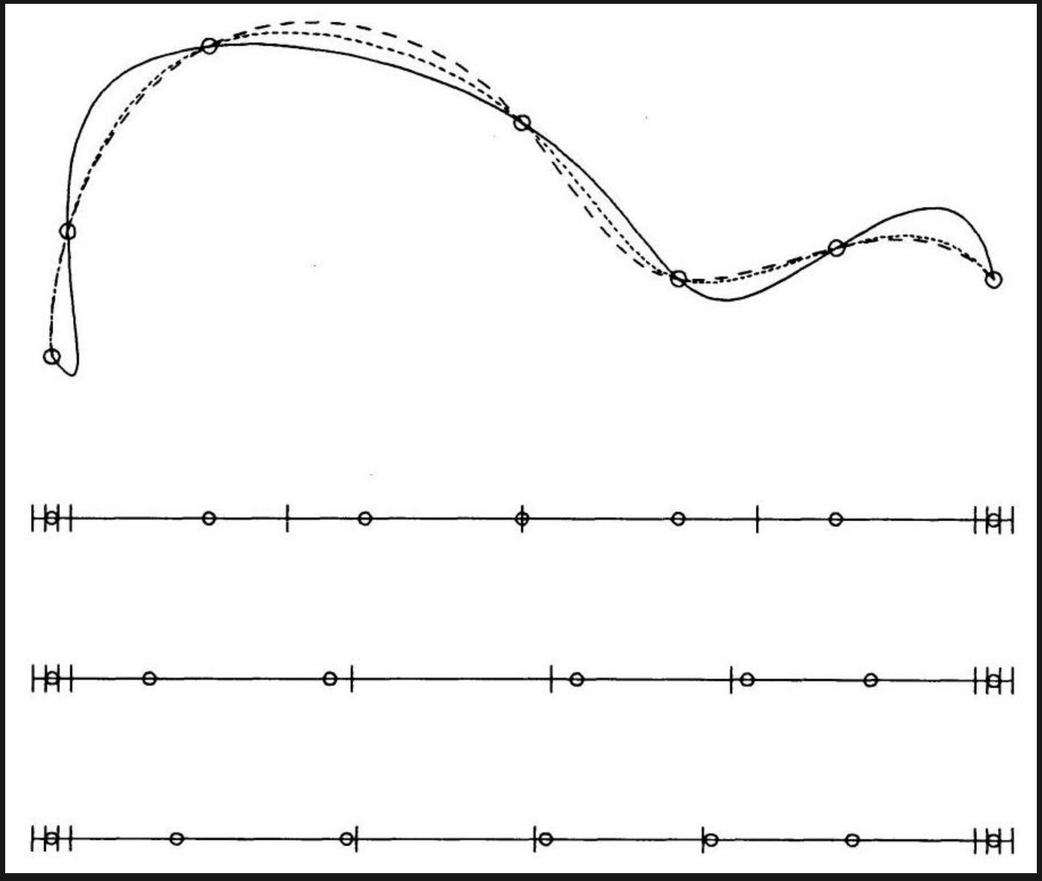


图 9.4. 使用不同参数化和节点向量的曲线插值示例。

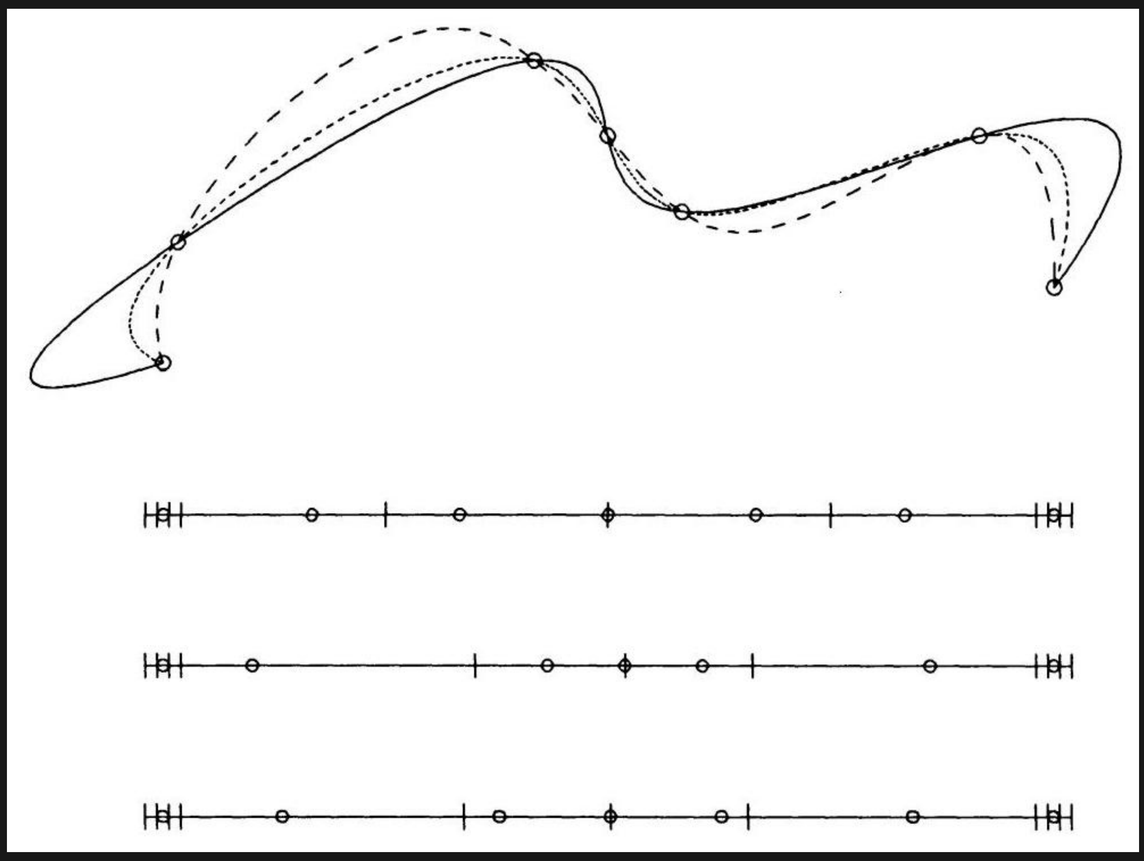


图 9.5. 使用不同参数化和节点向量的曲线插值示例。

图 9.6 说明了不同阶数的插值：实线、虚线和点划线曲线的阶数分别为 2、3 和 4。图 9.7 显示了全局插值器无法处理共线点集的情况。

一旦计算出 $\bar{u}_k$ 和节点，系统的 $(n+1)\times(n+1)$ 系数矩阵 (公式 [9.2]) 通过在每个 $\bar{u}_k, k=0,\ldots,n$ 处评估非零基函数来建立；使用算法 A2.1 和 A2.2。

示例

Ex9.1 设 $\{\mathbf{Q}_k\}=\{(0,0),(3,4),(-1,4),(-4,0),(-4,-3)\}$，并假设我们希望用三次曲线插值 $\mathbf{Q}_k$。我们使用方程 (9.5) 和 (9.8) 来计算 $\bar{u}_k$ 和 u_j，然后建立线性方程组，方程 (9.2)。各个弦长为

$$|\mathbf{Q}_1-\mathbf{Q}_0|=5\ |\mathbf{Q}_2-\mathbf{Q}_1|=4\ |\mathbf{Q}_3-\mathbf{Q}_2|=5\ |\mathbf{Q}_4-\mathbf{Q}_3|=3$$

总弦长为 $d=17$。因此

$$\bar{u}_0=0\ \bar{u}_1=\frac{5}{17}\ \bar{u}_2=\frac{9}{17}\ \bar{u}_3=\frac{14}{17}\ \bar{u}_4=1$$

使用方程 (9.8)

$$u_4=\frac{1}{3}\left(\frac{5}{17}+\frac{9}{17}+\frac{14}{17}\right)=\frac{28}{51}$$

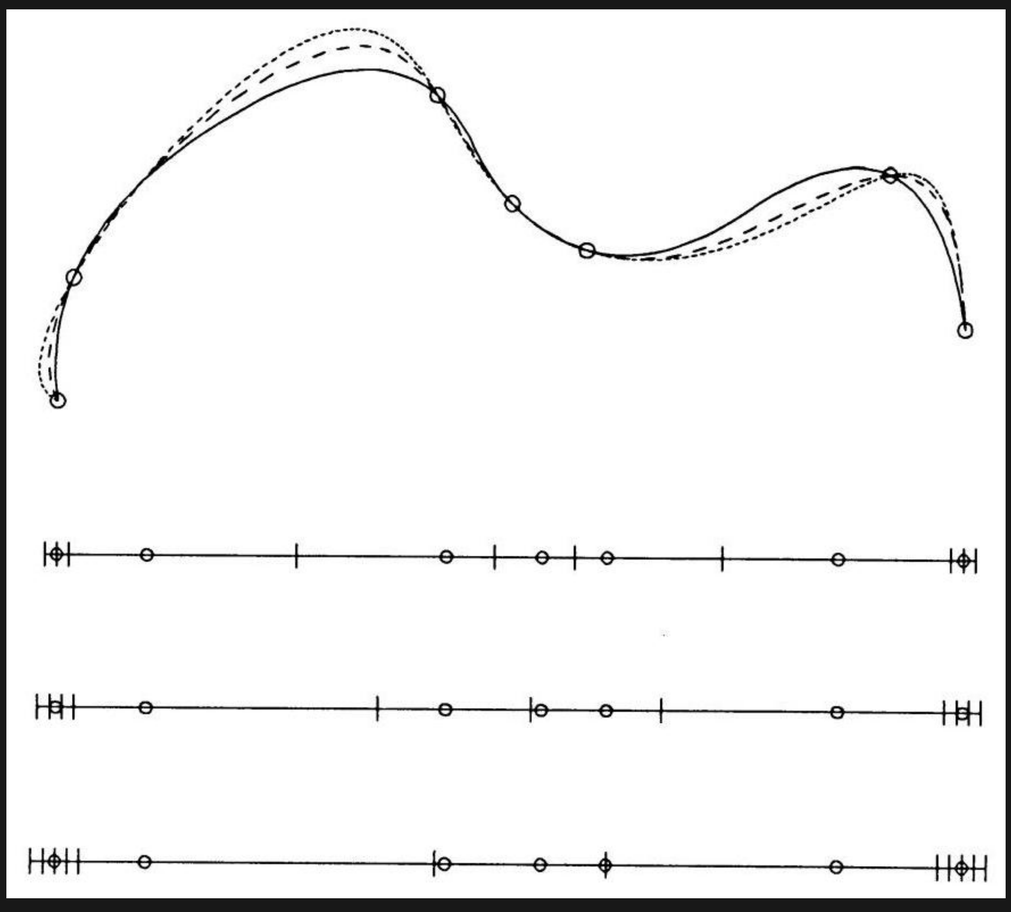


图 9.6. 使用弦长参数化和通过平均获得的节点进行不同度数的曲线插值。

因此

$$U=\left\{0,0,0,0,\frac{28}{51},1,1,1,1\right\}$$

线性方程组为

$$\begin{bmatrix}1&0&0&0&0\\N_{0,3}\left(\frac{5}{17}\right)&N_{1,3}\left(\frac{5}{17}\right)&N_{2,3}\left(\frac{5}{17}\right)&N_{3,3}\left(\frac{5}{17}\right)&0\\N_{0,3}\left(\frac{9}{17}\right)&N_{1,3}\left(\frac{9}{17}\right)&N_{2,3}\left(\frac{9}{17}\right)&N_{3,3}\left(\frac{9}{17}\right)&0\\0&N_{1,3}\left(\frac{14}{17}\right)&N_{2,3}\left(\frac{14}{17}\right)&N_{3,3}\left(\frac{14}{17}\right)&N_{4,3}\left(\frac{14}{17}\right)\\0&0&0&0&0\end{bmatrix}\begin{bmatrix}\mathbf{P}_0\\\mathbf{P}_1\\\mathbf{P}_2\\\mathbf{P}_3\\\mathbf{P}_4\end{bmatrix}$$

$$= \begin{bmatrix} \mathbf{Q}_0 \\ \mathbf{Q}_1 \\ \mathbf{Q}_2 \\ \mathbf{Q}_3 \\ \mathbf{Q}_4 \end{bmatrix}$$

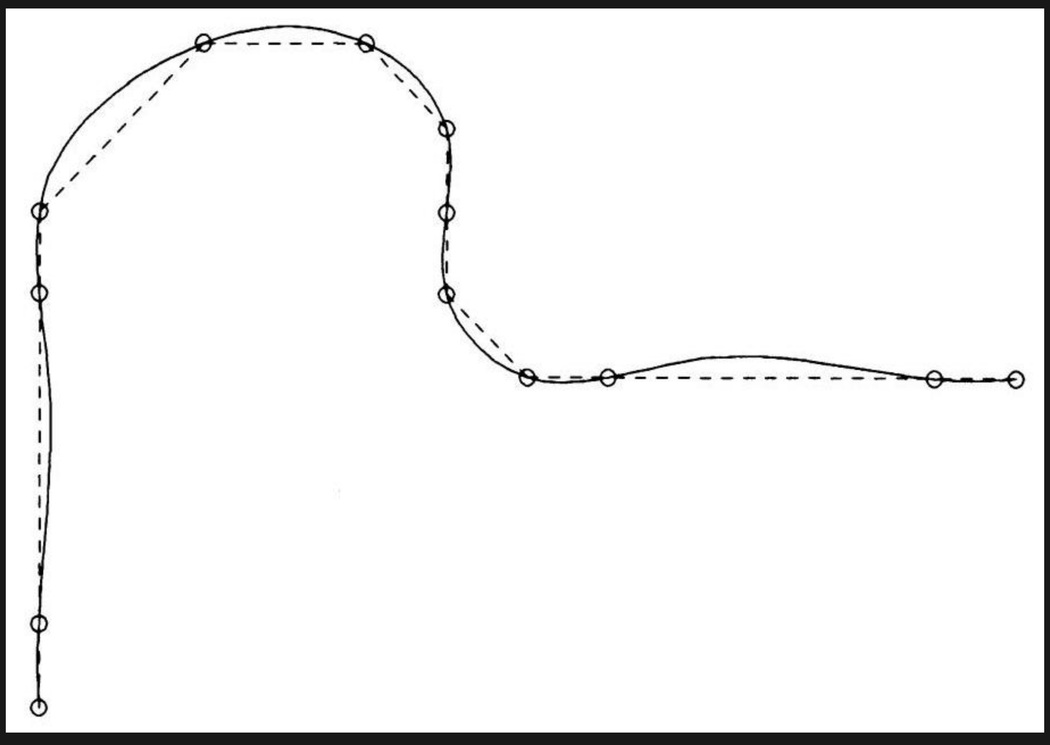


图 9.7. 对包含共线点的数据进行全局三次曲线插值 (见虚线)。

现在我们总结全局插值算法。我们假设有两个实用程序来解决线性方程组:

- LUDecomposition(A, q, sbw) 用于将 $q \times q$ 系数矩阵分解为下三角和上三角组件，半带宽为 sbw；为简单起见，我们假设 A 是一个 $q \times q$ 方阵，但应使用一个只存储非零带的实用程序；
- ForwardBackward(A, q, sbw, rhs, sol) 用于执行前向/后向替代 (见 [Pres88])；rhs [] 是系统的右侧 ($\mathbf{Q}_k$ 的坐标)，而 sol [] 是解向量 ($\mathbf{P}_i$ 的坐标)。

该算法还利用了第 2 章中的 FindSpan( ) 和 BasisFuns( )。

ALGORITHM A9.1

`GlobalCurveInterp` (n, Q, r, p, m, U, P)

```
{ /* Global interpolation through n + 1 points */
  /* Input: n, Q. r, p */
  /* Output: m, U, P */
m = n+p+1;
  Compute the uk; /* Eq. (9.5) or (9.6) */
  Compute the knot vector U ; /* Eq. (9.8) */
  Initialize array A to zero;
for (i = 0; i<=n; i++)
    { /* Set up coefficient matrix */
    span = FindSpan(n, p, uk[i], U);
    BasisFuns(span, uk[i], p, U, A[i][span-p]); /* Get ith row */
    }
```

LUDecomposition $(A, n+1, p-1)$;

```
for (i=0; i<r; i++) /*r is the number of coordinates */
  {
  for (j = 0; j <= n; j++)  rhs [j] = ith coordinate of Q [j] ;
  ForwardBackward(A, n+1, p-1, rhs, sol);
  for (j = 0; j <= n; j++)  ith coordinate of P [j] = sol [j] ;
  }
}
```

2.2 指定端导数的全局曲线插值

指定导数向量作为输入数据并不罕见。在这种情况下，求解过程类似于算法 A9.1:

1. 计算与点 $\mathbf{Q}_k$ 对应的参数 $\bar{u}_k$ ；
2. 计算节点向量 U ；
3. 评估基函数以建立线性方程组，以控制点作为未知数；
4. 求解该系统。

只有步骤 2 和 3 与前一种情况略有不同。每个导数产生一个额外的节点和控制点，因此产生一个额外的线性方程。使用第 2 章和第 3 章的导数公式来建立额外的方程，例如，方程 (2.7)、(2.10) 和 (3.3)-(3.10)。我们现在用一阶导数的示例来说明这一点。

设 $\{\mathbf{Q}_k\}, k = 0, \ldots, n$ 为点，并假设 $\mathbf{D}_0$ 和 $\mathbf{D}_n$ 分别是曲线起点和终点的一阶导数向量。我们希望用一个 p 次曲线来插值这些数据。

$$\mathbf{C}(u) = \sum_{i=0}^{n+2} N_{i,p}(u)\,\mathbf{P}_i$$

如前所述，使用方程 (9.5) 或方程 (9.6) 计算 $\bar{u}_k, k = 0, \ldots, n$ 。然后设置 $m = n + p + 3$ 并通过以下方式获得 $m + 1$ 节点:

$$u_0 = \cdots = u_p = 0 \; u_{m-p} = \cdots = u_m = 1$$

$$u_{j+p+1} = \frac{1}{p} \sum_{i=j}^{j+p-1} \bar{u}_i \; j = 0, \ldots, n-p+1 \tag{9.9}$$

注意，方程 (9.9) 类似于方程 (9.8)，只是我们通过将 j 从 0 运行到 $n - p + 1$ 来获得两个额外的节点。如前所述，方程 (9.2) 产生 $n + 1$ 个方程。

$$\mathbf{Q}_k = \mathbf{C}(\bar{u}_k) = \sum_{i=0}^{n+2} N_{i,p}(\bar{u}_k)\,\mathbf{P}_i \tag{9.10}$$

方程 (3.7) 产生两个额外的方程，即

$$-\mathbf{P}_0 + \mathbf{P}_1 = \frac{u_{p+1}}{p}\mathbf{D}_0 \tag{9.11}$$

$$-\mathbf{P}_{n+1} + \mathbf{P}_{n+2} = \frac{1 - u_{m-p-1}}{p}\mathbf{D}_n \tag{9.12}$$

将方程 (9.11) 和 (9.12) 作为第二个和倒数第二个方程插入方程 (9.10)，得到一个 $(n + 3) \times (n + 3)$ 带状线性系统。图 9.8 显示了具有指定端切线的三次曲线插值。导数的小、中、大幅度分别产生实线、虚线和点线。图 9.9 说明了对相同数据点的插值，但具有不同的端切线方向和导数幅度。

2.3 三次样条曲线插值

之前的方法 (方程 [9.9]-[9.12]) 适用于任何阶数 $p > 1$ 。如果 $p = 3$ ，则存在一种更高效的算法，产生传统的 C^2 三次样条。$\bar{u}_k$ 的计算与之前相同。节点是

$$u_0 = \cdots = u_3 = 0 \; u_{n+3} = \cdots = u_{n+6} = 1$$

$$u_{j+3} = \bar{u}_j \; j = 1, \ldots, n-1 \tag{9.13}$$

换句话说，$\mathbf{Q}_k$ 的插值发生在节点处。前两个和最后两个方程分别为

$$\mathbf{P}_0 = \mathbf{Q}_0$$

$$-\mathbf{P}_0 + \mathbf{P}_1 = \frac{u_4}{3}\mathbf{D}_0 \tag{9.14}$$

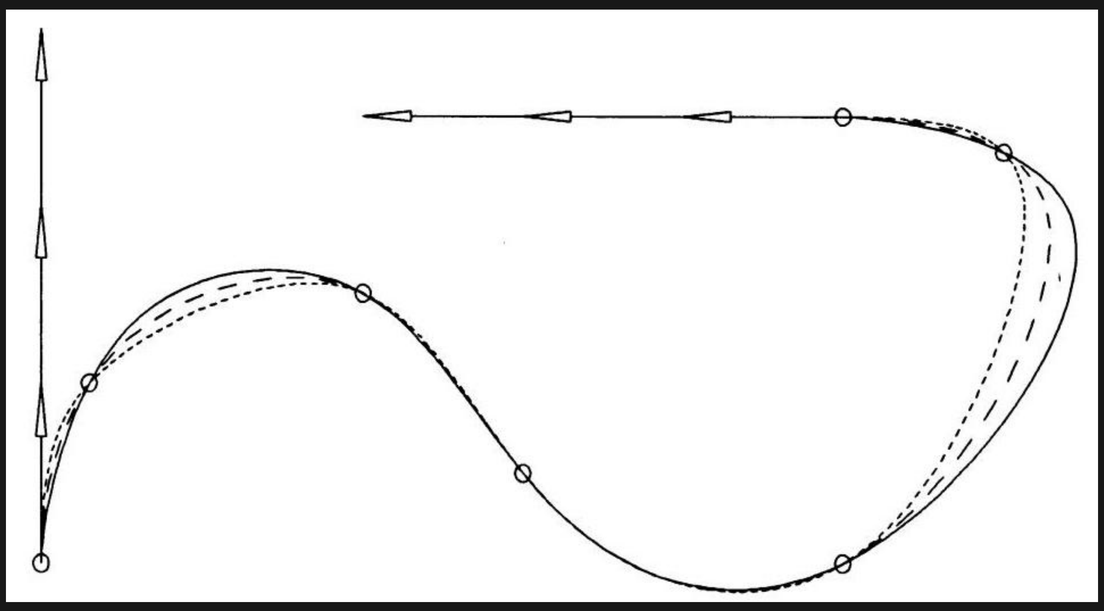


图 9.8. 具有不同端导数幅度的全局三次曲线插值。

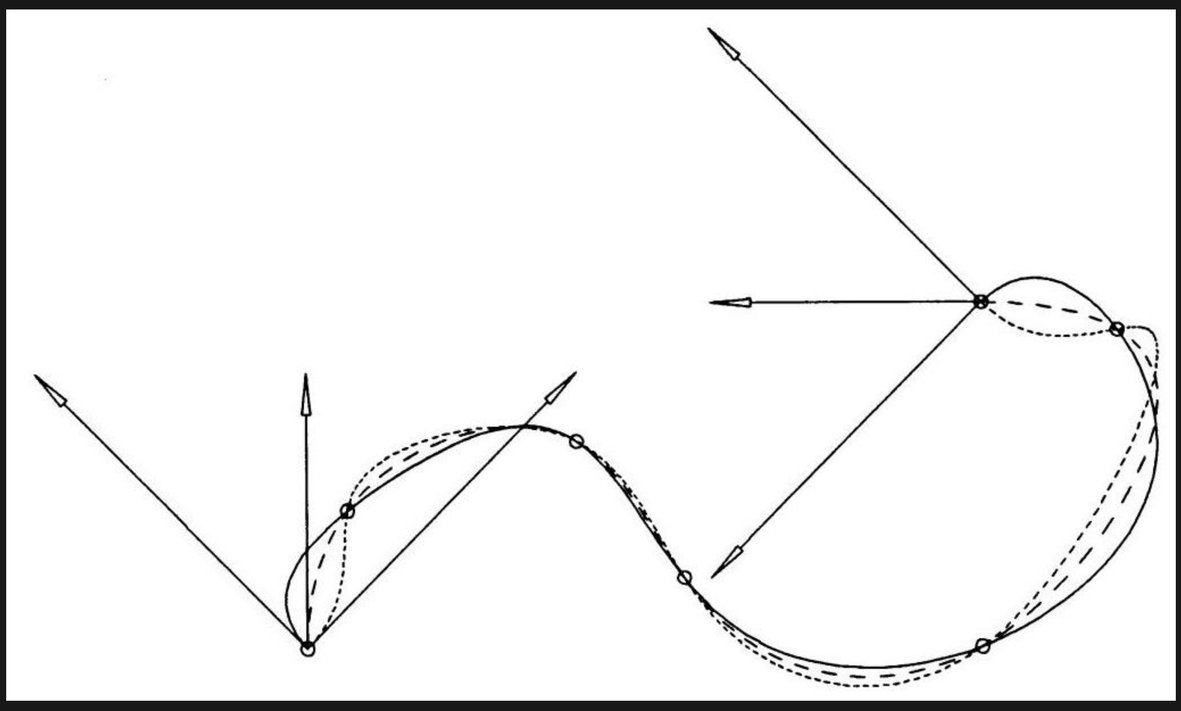


图 9.9. 具有不同端导数方向的全局三次曲线插值。

和

$$-\mathbf{P}_{n+1}+\mathbf{P}_{n+2}=\frac{1-u_{n+2}}{3}\mathbf{D}_n$$

$$\mathbf{P}_{n+2}=\mathbf{Q}_n \tag{9.15}$$

这些可以直接求解。回想一下，在一个内部节点只有三个非零的三次基函数，其余的 $n-1$ 方程具有以下形式

$$\mathbf{Q}_k=\mathbf{C}\left(\bar{u}_k\right)=N_{k,3}\left(\bar{u}_k\right)\mathbf{P}_k+N_{k+1,3}\left(\bar{u}_k\right)\mathbf{P}_{k+1}+N_{k+2,3}\left(\bar{u}_k\right)\mathbf{P}_{k+2} \tag{9.16}$$

对于 $k=1,\ldots,n-1$ 。设定

$$a_k=N_{k,3}\left(\bar{u}_k\right)\ b_k=N_{k+1,3}\left(\bar{u}_k\right)\ c_k=N_{k+2,3}\left(\bar{u}_k\right)$$

产生三对角系统

$$\begin{bmatrix}\mathbf{R}_2\\ \mathbf{R}_3\\ \vdots\\ \mathbf{R}_{n-1}\\ \mathbf{R}_n\end{bmatrix}=\begin{bmatrix}\mathbf{Q}_1-a_1\mathbf{P}_1\\ \mathbf{Q}_2\\ \vdots\\ \mathbf{Q}_{n-2}\\ \mathbf{Q}_{n-1}-c_{n-1}\mathbf{P}_{n+1}\end{bmatrix}$$

$$=\begin{bmatrix}b_1 & c_1 & 0 & \cdots & 0 & 0 & 0\\ a_2 & b_2 & c_2 & \cdots & 0 & 0 & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots\\ 0 & 0 & 0 & \cdots & a_{n-2} & b_{n-2} & c_{n-2}\\ 0 & 0 & 0 & \cdots & 0 & a_{n-1} & b_{n-1}\end{bmatrix}\begin{bmatrix}\mathbf{P}_2\\ \mathbf{P}_3\\ \vdots\\ \mathbf{P}_{n-1}\\ \mathbf{P}_n\end{bmatrix} \tag{9.17}$$

该系统可以通过一个简单的算法来解决，该算法实时计算方程 (9.17) 中矩阵的行。它需要两个长度为 $n+1$ 的局部数组 R[] 和 dd[]，以及一个长度为 4 的数组 abc[]，该数组存储 $a_k, b_k, c_k, 0$ 。它假设 $\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_{n+1}, \mathbf{P}_{n+2}$ 已经计算并加载到数组 $\mathbf{P}$ 中。该算法可以适应处理具有任意数量坐标的点。

算法 A9.2

```
SolveTridiagonal(n, Q, U, P)
    { /* Solve tridiagonal system for C2 cubic spline */
            /* Input: n, Q, U, P[0], P[1], P[n+1], P[n+2] */
            /* Output: P */
    for (i=3; i<n; i++) R[i] = Q[i-1];
    BasisFuns(4, U[4],3, U, abc);
    den = abc[1];
    P[2] = (Q[1]-abc[0]*P[1])/den;
    for (i=3; i<n; i++)
          {
          dd[i] = abc[2]/den;
          BasisFuns(i+2, U[i+2],3, U, abc);
          den = abc[1]-abc[0]*dd[i];
          P[i] = (R[i] - abc[0] * P[i-1]) /den;
        }
    dd[n] = abc[2]/den;
    BasisFuns(n+2, U[n+2],3, U, abc);
    den = abc[1] - abc[0] * dd[n] ;
    P[n] = (Q[n-1] - abc[2] * P[n+1] - abc[0] * P[n-1]) /den;
    for (i=n-1; i>=2; i-) P[i] = P[i]-dd[i+1]*P[i+1];
```

}

图 9.10 显示了使用该三次样条算法插值的数据。

通常，仅指定导数向量的切向方向而不指定大小。我们用 $\mathbf{T}_0$ 和 $\mathbf{T}_n$ 表示单位长度的切向量。在这种情况下，必须估计大小 α_0 和 α_n 。设定

$$\mathbf{D}_0 = \alpha_0 \mathbf{T}_0 \; \mathbf{D}_n = \alpha_n \mathbf{T}_n \tag{9.18}$$

并按之前的方式进行。一个合理的选择是将 α_0 和 α_n 设置为总弦长 d ，如方程 (9.4) 中计算的那样。在交互环境中，α_0 和 α_n 可以用作额外的形状控制。

2.4 指定一阶导数的全局曲线插值

现在假设在每个点上给定一阶导数 $\mathbf{D}_k$ ， $\mathbf{Q}_k, k = 0, \ldots, n$ 。有 $2(n+1)$ 个数据项和同样数量的未知控制点。

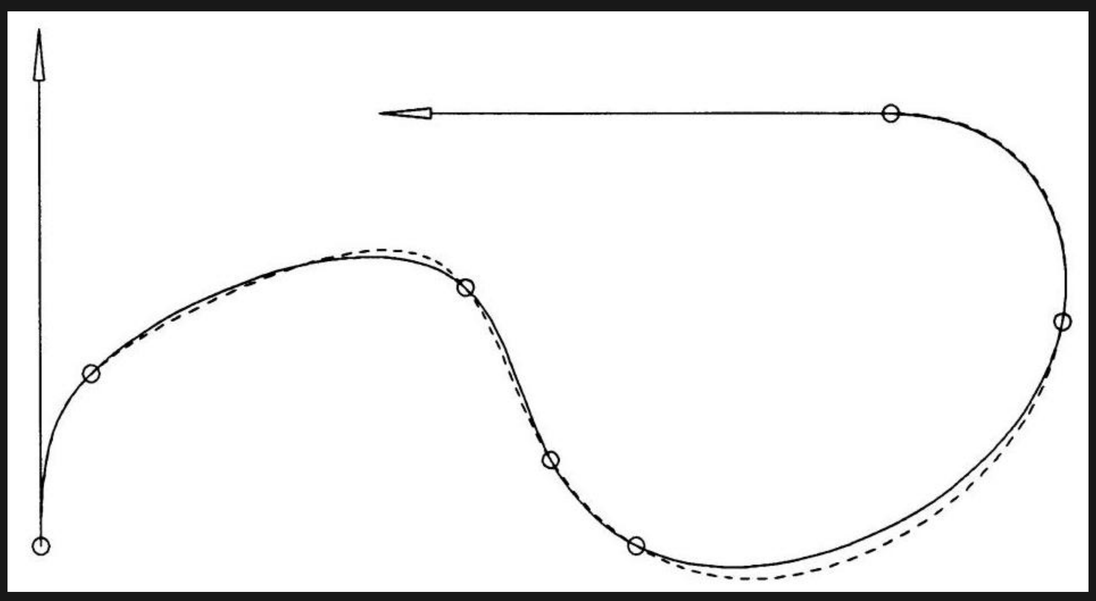


图 9.10。三次样条插值 (实线) 与通过使用弦长参数化和通过平均计算的节点获得的常规三次插值 (虚线) 进行比较。

表达 $\mathbf{Q}_k$ 的 $n+1$ 方程为

$$\mathbf{Q}_k = \mathbf{C}(\bar{u}_k) = \sum_{i=0}^{2n+1} N_{i,p}(\bar{u}_k)\,\mathbf{P}_i \tag{9.19}$$

并且从方程 (2.10) 和 (3.3) 中，表达 $\mathbf{D}_k$ 的 $n+1$ 方程为

$$\mathbf{D}_k = \mathbf{C}'(\bar{u}_k) = \sum_{i=0}^{2n+1} N'_{i,p}(\bar{u}_k)\,\mathbf{P}_i \tag{9.20}$$

参数 $\bar{u}_k$ 是通过公式 (9.5) 或公式 (9.6) 计算得出的。节点的数量是

$$2(n+1) + p + 1 \tag{9.21}$$

应该选择以反映 $\bar{u}_k$ 的分布。对于 $p = 2$ 和 $p = 3$ ，令人满意的选择是

$$U = \left\{0, 0, 0, \frac{\bar{u}_1}{2}, \bar{u}_1, \frac{\bar{u}_1 + \bar{u}_2}{2}, \bar{u}_2, \ldots, \bar{u}_{n-1}, \frac{\bar{u}_{n-1} + 1}{2}, 1, 1, 1\right\} \tag{9.22}$$

和 $U = \left\{0, 0, 0, 0, \frac{\bar{u}_1}{2}, \frac{2\bar{u}_1 + \bar{u}_2}{3}, \frac{\bar{u}_1 + 2\bar{u}_2}{3}, \ldots, \frac{\bar{u}_{n-2} + 2\bar{u}_{n-1}}{3}\right.$ ，

$$\left.\frac{\bar{u}_{n-1} + 1}{2}, 1, 1, 1, 1\right\} \tag{9.23}$$

分别。方程 (9.19) 和 (9.20) 以交替的方式合并，以产生一个 $2(n+1) \times 2(n+1)$ 带状线性系统。例如，设 $p = 3$ 。将方程 (9.23) 的第一个和最后一个内部节点表示为

$$u_4 = \frac{\bar{u}_1}{2} \; u_{m-p-1} = \frac{\bar{u}_{n-1} + 1}{2}$$

方程组为

$$\mathbf{P}_0$$

$$= \mathbf{Q}_0$$

$$-\mathbf{P}_0 + \mathbf{P}_1$$

$$= \frac{u_4}{3}\mathbf{D}_0$$

$$N_{1,3}(\bar{u}_1)\,\mathbf{P}_1 + \cdots + N_{4,3}(\bar{u}_1)\,\mathbf{P}_4$$

$$= \mathbf{Q}_1$$

$$N'_{1,3}(\bar{u}_1)\,\mathbf{P}_1 + \cdots + N'_{4,3}(\bar{u}_1)\,\mathbf{P}_4$$

$$= \mathbf{D}_1$$

$$\vdots$$

$$N_{2n-3,3}(\bar{u}_{n-1})\,\mathbf{P}_{2n-3} + \cdots + N_{2n,3}(\bar{u}_{n-1})\,\mathbf{P}_{2n} \; = \mathbf{Q}_{n-1}$$

$$N'_{2n-3,3}(\bar{u}_{n-1})\,\mathbf{P}_{2n-3} + \cdots + N'_{2n,3}(\bar{u}_{n-1})\,\mathbf{P}_{2n} \; = \mathbf{D}_{n-1}$$

$$-\mathbf{P}_{2n} + \mathbf{P}_{2n+1} = \frac{1 - u_{m-p-1}}{3}\mathbf{D}_n$$

$$\mathbf{P}_{2n+1} = \mathbf{Q}_n \tag{9.24}$$

使用算法 A2.3 (DersBasisFuns ( )) 来计算 $N_{i,3}(\bar{u}_k)$ 和 $N'_{i,3}(\bar{u}_k)$ 。

如果给定单位长度切向量 $\mathbf{T}_k$ 而不是 $\mathbf{D}_k$ ，则必须估计幅度 α_k 。设置所有 $\alpha_k = d$ ，其中 d 是总弦长，是一个合理的选择。图 9.11 显示了对点和导数的二次 (实线) 和三次 (虚线) 曲线插值。图 9.12 说明了通过在数据点使用不同导数幅度获得的不同曲线插值之间的比较。实线曲线应用 $\alpha_k = d$ (中间切线)；

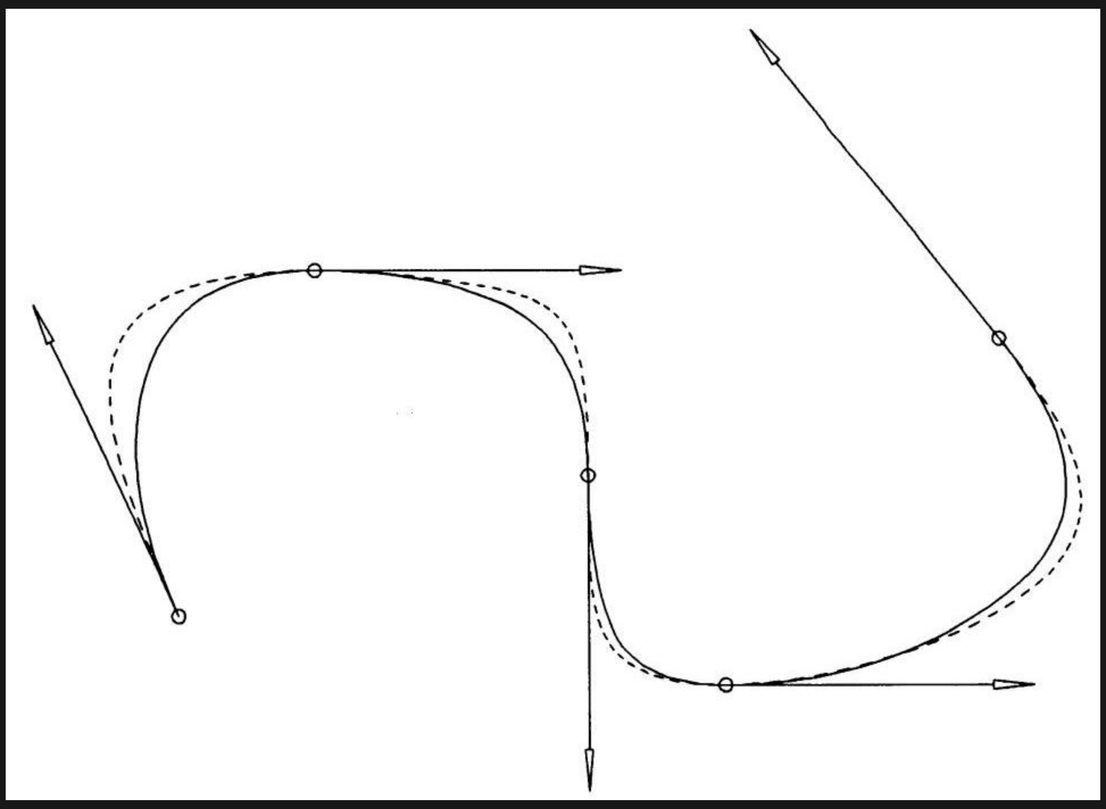


图 9.11. 在每个数据点指定导数的二次 (实线) 和三次 (虚线) 曲线插值。

376 曲线和曲面拟合，虚线曲线具有 $\alpha_k = 3/2d$ (大切线)；点线曲线是通过 $\alpha_k = 1/2d$ (小切线) 获得的。图 9.13 显示了对指定点和一阶导数的二次曲线插值的控制点。

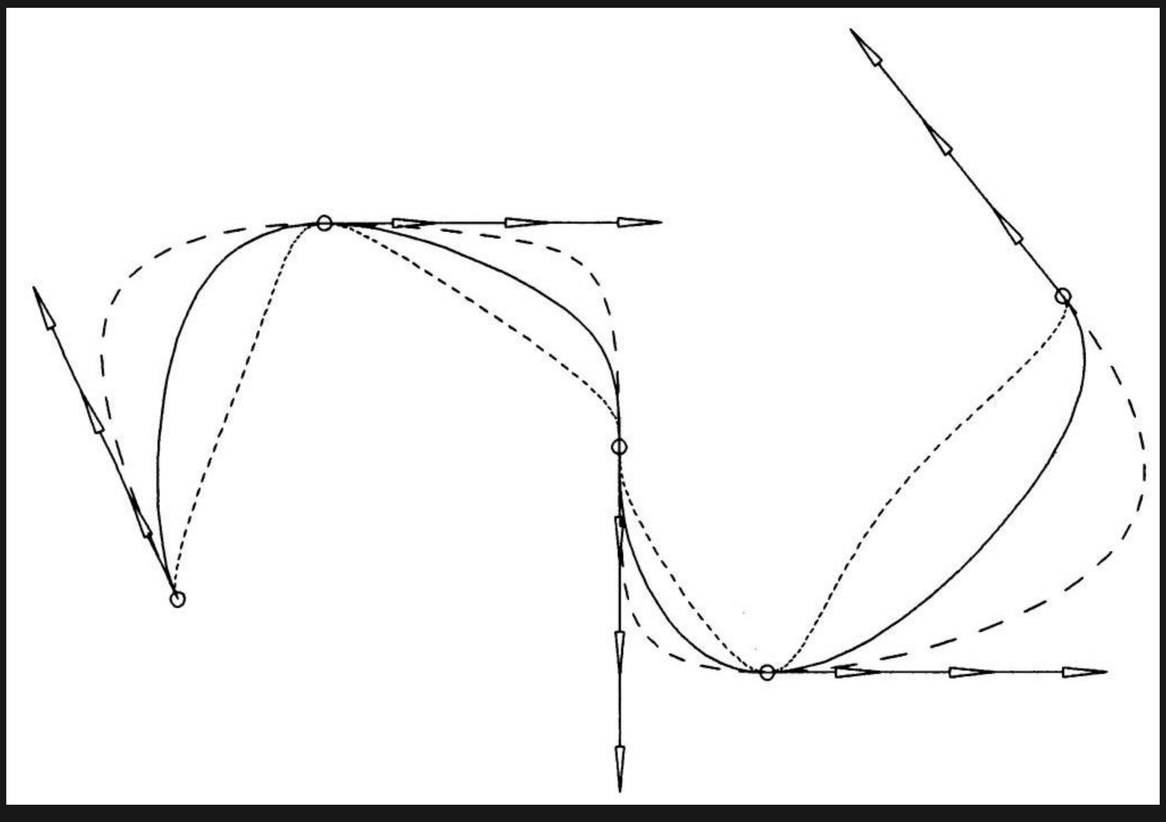


图 9.12. 在每个点具有不同切向导数的三次曲线插值。

2.5 全局曲面插值

我们现在转向曲面的全局插值。我们给定一组 $(n+1)\times(m+1)$ 数据点 $\{\mathbf{Q}_{k,\ell}\}, k=0,\ldots,n$ 和 $\ell=0,\ldots,m$，我们希望构造一个非有理的 (p, q) 次 B 样条曲面来插值这些点，即

$$\mathbf{Q}_{k,\ell} = \mathbf{S}(\bar{u}_k, \bar{v}_\ell) = \sum_{i=0}^{n}\sum_{j=0}^{m} N_{i,p}(\bar{u}_k) N_{j,q}(\bar{v}_\ell)\mathbf{P}_{i,j} \tag{9.25}$$

再次，首要任务是计算 $(\bar{u}_k, \bar{v}_\ell)$ 和节点向量 U 及 V 的合理值。我们展示如何计算 $\bar{u}_k$；$\bar{v}_\ell$ 是类似的。一个常见的方法是使用公式 (9.5) 或公式 (9.6) 来计算每个 ℓ 的参数 $\bar{u}_0^\ell, \ldots, \bar{u}_n^\ell$，然后通过对所有 $\bar{u}_k^\ell, \ell=0,\ldots,m$ 进行平均来获得每个 $\bar{u}_k$，即

$$\bar{u}_k = \frac{1}{m+1}\sum_{\ell=0}^{m} \bar{u}_k^\ell \quad k=0,\ldots,n$$

其中对于每个固定的 $\ell, \bar{u}_k^\ell, k=0,\ldots,n$，是通过公式 (9.5) 或公式 (9.6) 计算的。以下是使用公式 (9.5) 的高效算法。它同时计算 $\bar{u}_k$ 和 $\bar{v}_\ell$。它需要一个长度为 $\max(n+1, m+1)$ 的局部数组 cds [] 来存储弦长

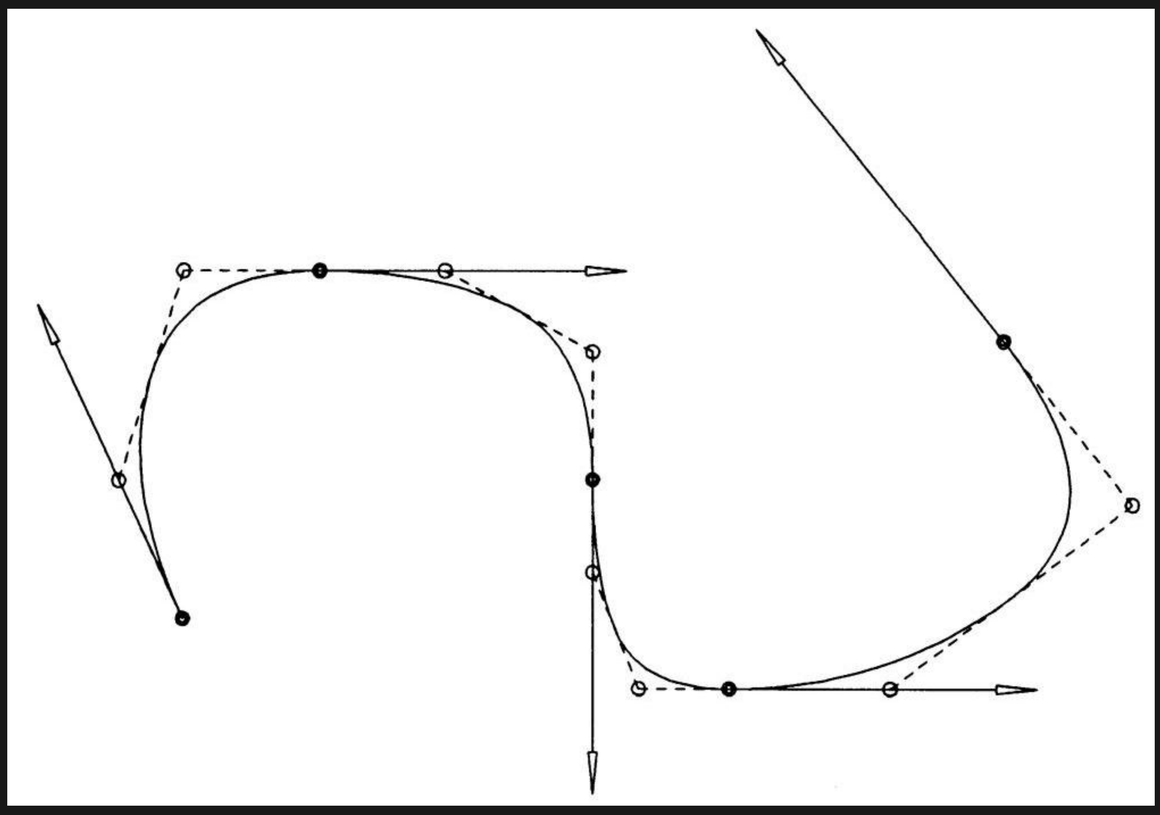


图 9.13. 二次 B 样条曲线插值器的控制点，插值于具有指定导数的数据。

距离 $|\mathbf{Q}_{k,\ell}-\mathbf{Q}_{k-1,\ell}|, k=1,\ldots,n$ (在 v 方向上也是如此)。函数 Distance3D( ) 计算两个三维点之间的距离。该算法处理当一行的总弦长等于零的情况，这是构造三面体表面的必要条件。

算法 A9.3

```
SurfMeshParams(n, m, Q, uk, vl)
        { /* Compute parameters for */
                /* global surface interpolation */
                /* Input: n, m, Q */
                /* Output: uk, vl */
          /* First get the uk */
      num = m+1; /* number of nondegenerate rows */
      uk[0] = 0.0 ; ux[n] = 1.0 ;
      for (k = 1; k < n; k + +) uk [k] = 0.0 ;
      for (l=0; l<=m; l++)
            {
            total = 0.0; /* total chord length of row */
            for (k=1; k<=n; k++)
                {
                cds[k] = Distance3D(Q[k][l], Q[k-1][l]);
                total = total + cds[k];
                }
            if (total == 0.0) num = num-1;
          else
            {
            d = 0.0 ;
            for (k=1; k<n; k++)
                {
                d = d + cds[k];
                uk [k] = uk [k] + d/ total;
              }
            }
      }
if (num == 0) return(error);
for (k = 1; k < n; k + +) uk [k] /num ;
        /* Now do the same for vl */
}
```

一旦计算出 $(\bar{u}_k,\bar{v}_\ell)$ ，节点向量 U 和 V 可以通过公式 (9.8) 获得。

现在进行控制点的计算。显然，方程 (9.25) 表示 $(n+1)\times(m+1)$ 中的线性方程，未知数为 $\mathbf{P}_{i,j}$ 。然而，由于 $\mathbf{S}(u,v)$ 是一个张量积曲面，$\mathbf{P}_{i,j}$ 可以更简单高效地通过一系列曲线插值获得。对于固定的 ℓ ，将方程 (9.25) 写为

$$\mathbf{Q}_{k,\ell}=\sum_{i=0}^{n}N_{i,p}(\bar{u}_k)\left(\sum_{j=0}^{m}N_{j,q}(\bar{v}_\ell)\mathbf{P}_{i,j}\right)=\sum_{i=0}^{n}N_{i,p}(\bar{u}_k)\mathbf{R}_{i,\ell} \tag{9.26}$$

其中

$$\mathbf{R}_{i,\ell}=\sum_{j=0}^{m}N_{j,q}(\bar{v}_\ell)\mathbf{P}_{i,j} \tag{9.27}$$

注意，方程 (9.26) 只是通过点 $\mathbf{Q}_{k,\ell}, k=0,\ldots,n$ 的曲线插值。$\mathbf{R}_{i,\ell}$ 是在固定 $v=\bar{v}_\ell$ 的 $\mathbf{S}(u,v)$ 上的等参数曲线的控制点。现在固定 i 并让 ℓ 变化，方程 (9.27) 是通过点 $\mathbf{R}_{i,0},\ldots,\mathbf{R}_{i,m}$ 的曲线插值，以 $\mathbf{P}_{i,0},\ldots,\mathbf{P}_{i,m}$ 作为计算得到的控制点。因此，获得所有 $\mathbf{P}_{i,j}$ 的算法是 (见图 9.14a-9.14d):

1. 使用 U 和 $\bar{u}_k$ ，进行 $m+1$ 曲线插值通过 $\mathbf{Q}_{0,\ell},\ldots,\mathbf{Q}_{n,\ell}$ (对于 $\ell=0,\ldots,m$)；这产生了 $\mathbf{R}_{i,\ell}$ (图 9.14b)；
2. 使用 V 和 $\bar{v}_\ell$ ，进行 $n+1$ 曲线插值通过 $\mathbf{R}_{i,0},\ldots,\mathbf{R}_{i,m}$ (对于 $i=0,\ldots,n$)；这产生了 $\mathbf{P}_{i,j}$ (图 9.14c)。

图 9.14d 描绘了结果曲面。显然，该算法是对称的；相同的曲面可以通过以下方式获得:

1. 通过 $\mathbf{Q}_{k,0},\ldots,\mathbf{Q}_{k,m}$ 进行 $n+1$ 曲线插值以获得 $\mathbf{R}_{k,j}$ (等参数曲线的控制点 $\mathbf{S}(\bar{u}_k,v)$)；
2. 然后通过 $\mathbf{R}_{0,j},\ldots,\mathbf{R}_{n,j}$ 进行 $m+1$ 曲线插值，以获得 $\mathbf{P}_{i,j}$ 。u -方向插值；(d) 显示控制点的表面插值。

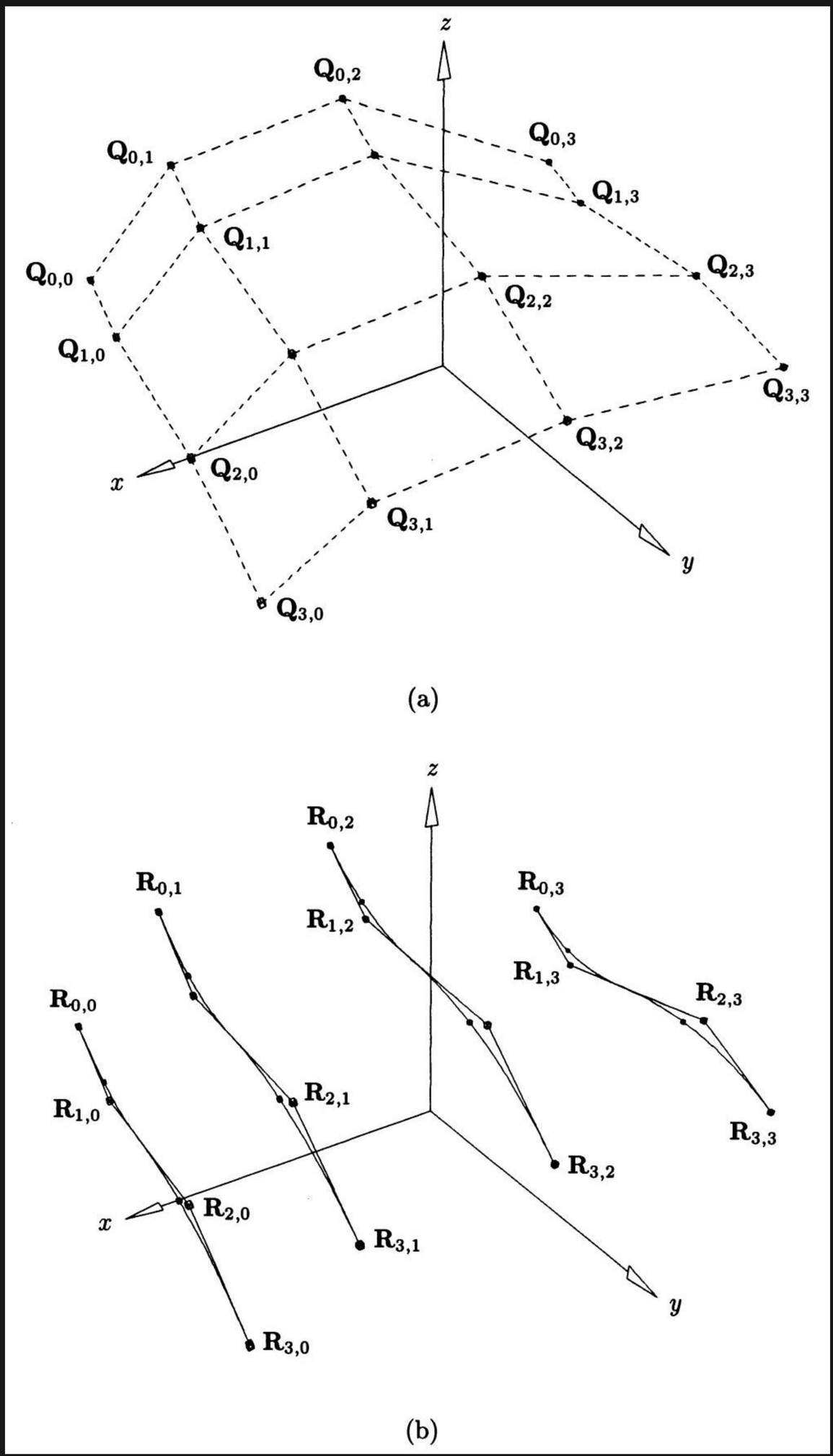


图 9.14. 全局表面插值。(a) 数据点；(b) 插值 u -方向数据点；(c) 通过控制点在 v 方向进行插值。

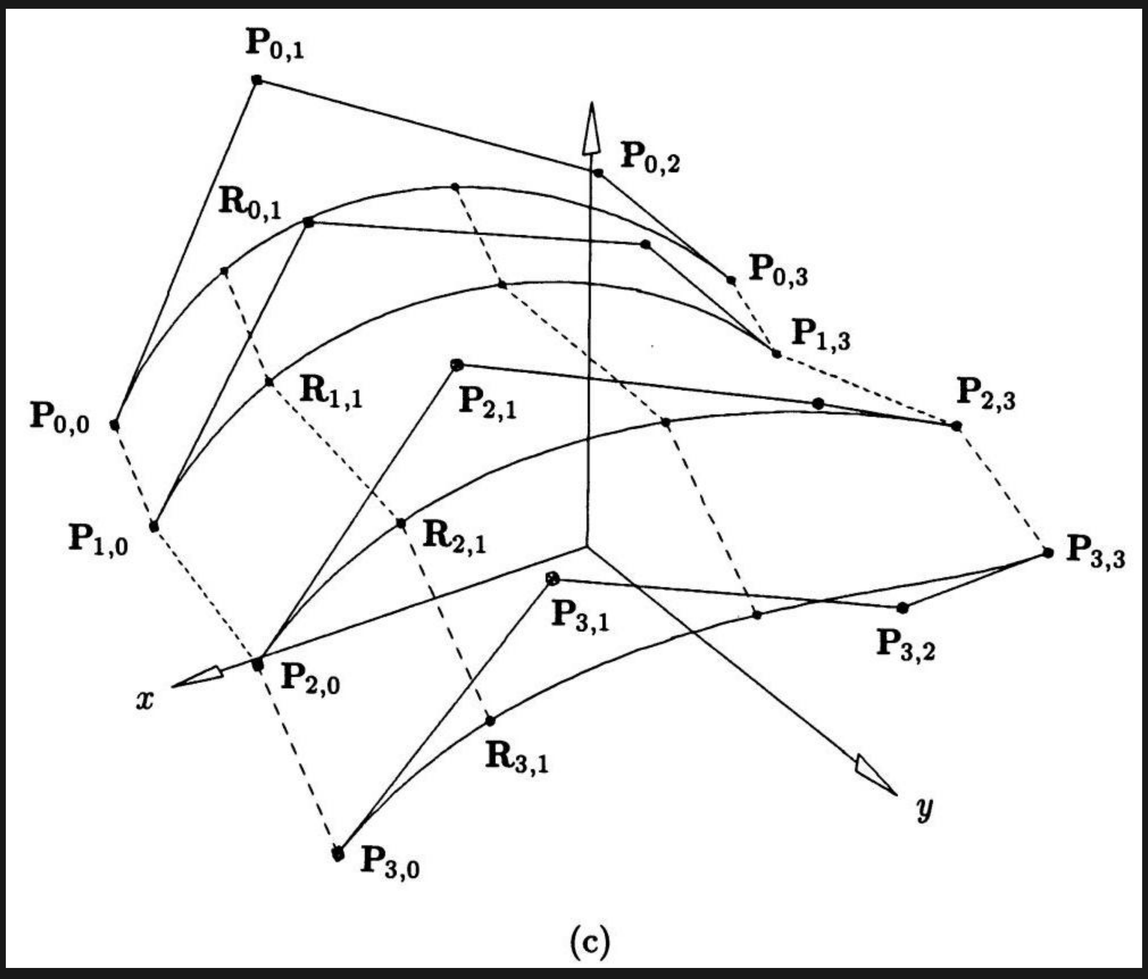


图 9.14. (续)。

总结来说，表面插值的过程如下:

ALGORITHM A9.4

```
GlobalSurfInterp(n, m, Q, p, q, U, V, P)
    { /* Global surface interpolation */
        /* Input: n, m, Q, p, q */
        /* Output: U, V, P */
    SurfMeshParams (n, m, Q, uk, vl) ; /* get parameters */
    Compute U using Eq. (9.8);
    Compute V using Eq. (9.8);
    for (l=0; l<=m; l++)
        {
        Do curve interpolation through Q[0][l], ..., Q[n][l] ;
        This yields R[0][l], ..., R[n][l] ;
        }
    for (i = 0; i<=n; i++)
        {
        Do curve interpolation through R[i][0], ..., R[i][m] ;
        This yields P[i][0], ..., P[i][m] ;
        }
    }
```

图 9.15-9.18 显示了几个表面插值的例子。数据集在图 9.15 中展示。图 9.16a – 9.16c 中展示了具有不同参数化和节点向量的表面。度数为 (2,3)，参数化和节点向量是均匀的 (图 9.16a)；弦长，通过平均获得的节点 (图 9.16b)；以及向心，通过平均获得的节点 (图 9.16c)。在图 9.17 中描绘了具有不同度数的表面插值。每个图的度数为: 图 9.17a，度数 (2,3)；图 9.17b，度数 (3,2)；以及图 9.17c ，度数 (3,3)。在图 9.18 中演示了无法处理共面的数据。

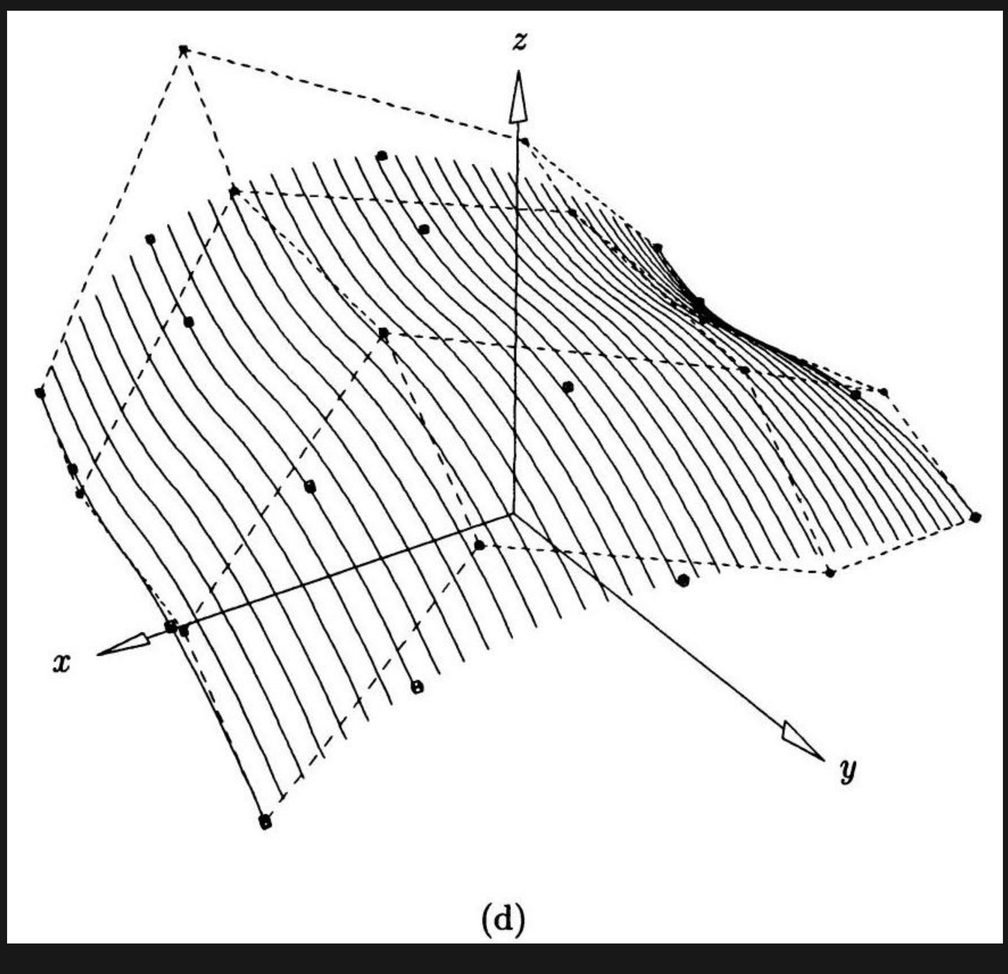


图 9.14. (续)。

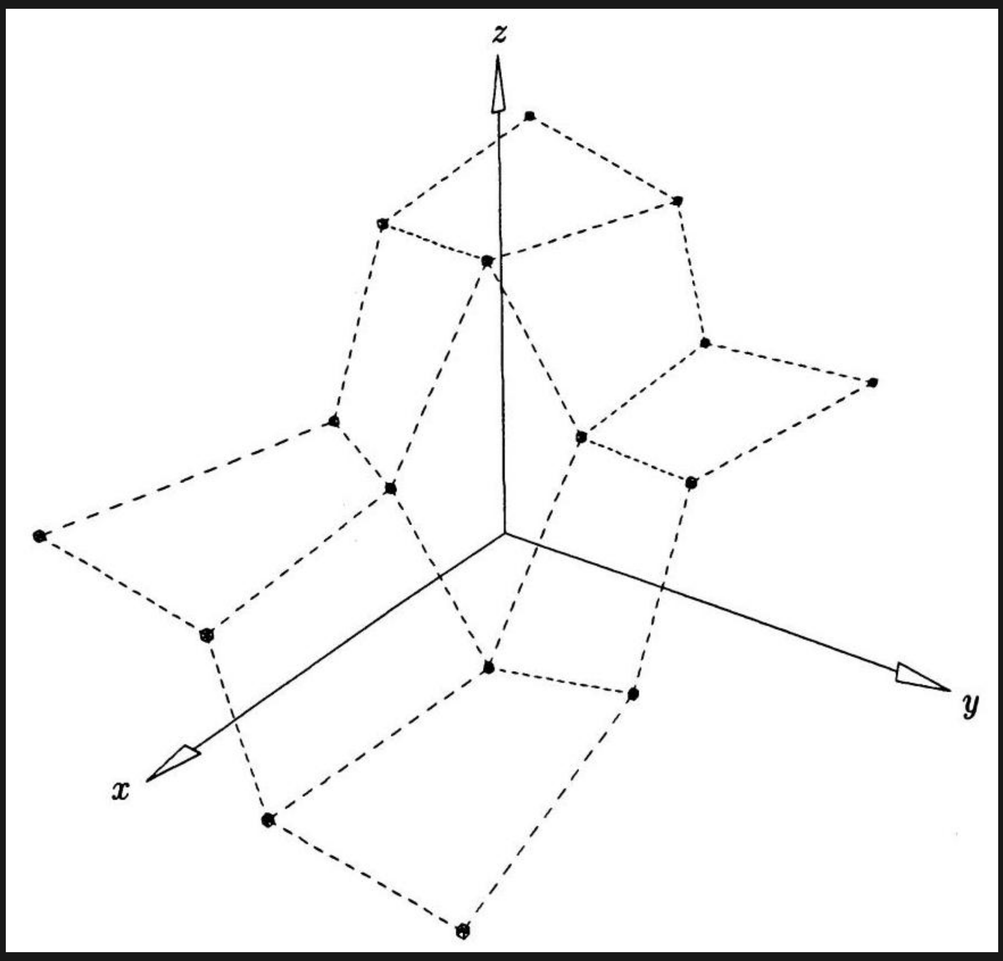


图 9.15. 用于测试表面插值的数据点。

导数约束也可以纳入全局表面插值。从概念上讲，第 2 章和第 3 章的导数公式可以用来为每个指定的导数约束定义一个额外的线性方程。不幸的是，如果每一行或每一列的数据约束数量不相同，使用两方向的曲线插值来求解未知的表面控制点将变得更加困难，如前所述。然而，通过巧妙地使用曲线节点插入和表面节点移除，可以调整算法 A9.4 以处理个别数据点的偏导数约束。最后，局部表面插值方法 (见第 9.3 节) 非常适合处理导数约束，因为它们在每个点上都需要导数。

3　局部插值

在本节中，我们介绍曲线和表面对点和切线数据的局部插值。我们从一些初步概念开始。

3.1　局部曲线插值初步

设定 $\{\mathbf{Q}_k\}, k = 0, \ldots, n$ 。局部曲线插值是指构造 n 多项式或有理曲线段 $\mathbf{C}_i(u), i = 0, \ldots, n-1$ 的一种方法，使得 $\mathbf{Q}_i$ 和 $\mathbf{Q}_{i+1}$ 是 $\mathbf{C}_i(u)$ 的端点。相邻的段以某种规定的连续性水平连接，构造过程通常从左到右逐段进行。任何产生的方程仅与少数相邻段局部相关。在 NURBS 框架下，我们使用多项式或有理 Bézier 曲线构造这些段，然后通过选择合适的节点向量获得 NURBS 曲线。

现在让 $\bar{u}_i$ 表示 $\mathbf{C}_i(u)$ 的起始参数和 $\mathbf{C}_{i-1}(u)$ 的结束参数。如果它们的切线方向在 $\bar{u}_i$ 处重合，即 $\mathbf{C}_i'(\bar{u}_i)$ 和 $\mathbf{C}_{i-1}'(\bar{u}_i)$ 指向相同方向，则 $\mathbf{C}_i(u)$ 和 $\mathbf{C}_{i-1}(u)$ 在 $\bar{u}_i$ 处具有 G^1 连续性 (对于几何而言为 G)。然而，它们的大小可能不同。G^1 连续性意味着曲线在视觉上是连续的 (光滑的)，但在参数化上可能存在不连续性。关于 G^n 连续性的更深入研究，读者可以参考 Barsky 和 DeRose [Bars89,90]。局部拟合算法旨在提供某种程度的 G 或 C 连续性；在本章中，我们仅介绍提供 G^1 或 C^1 连续性的算法。产生更高连续性的局部方法并不广泛使用。尽管 G^2 和 C^2 在三次曲线中是可能的，但如果要保持合理的灵活性，则必须使用更高的次数。

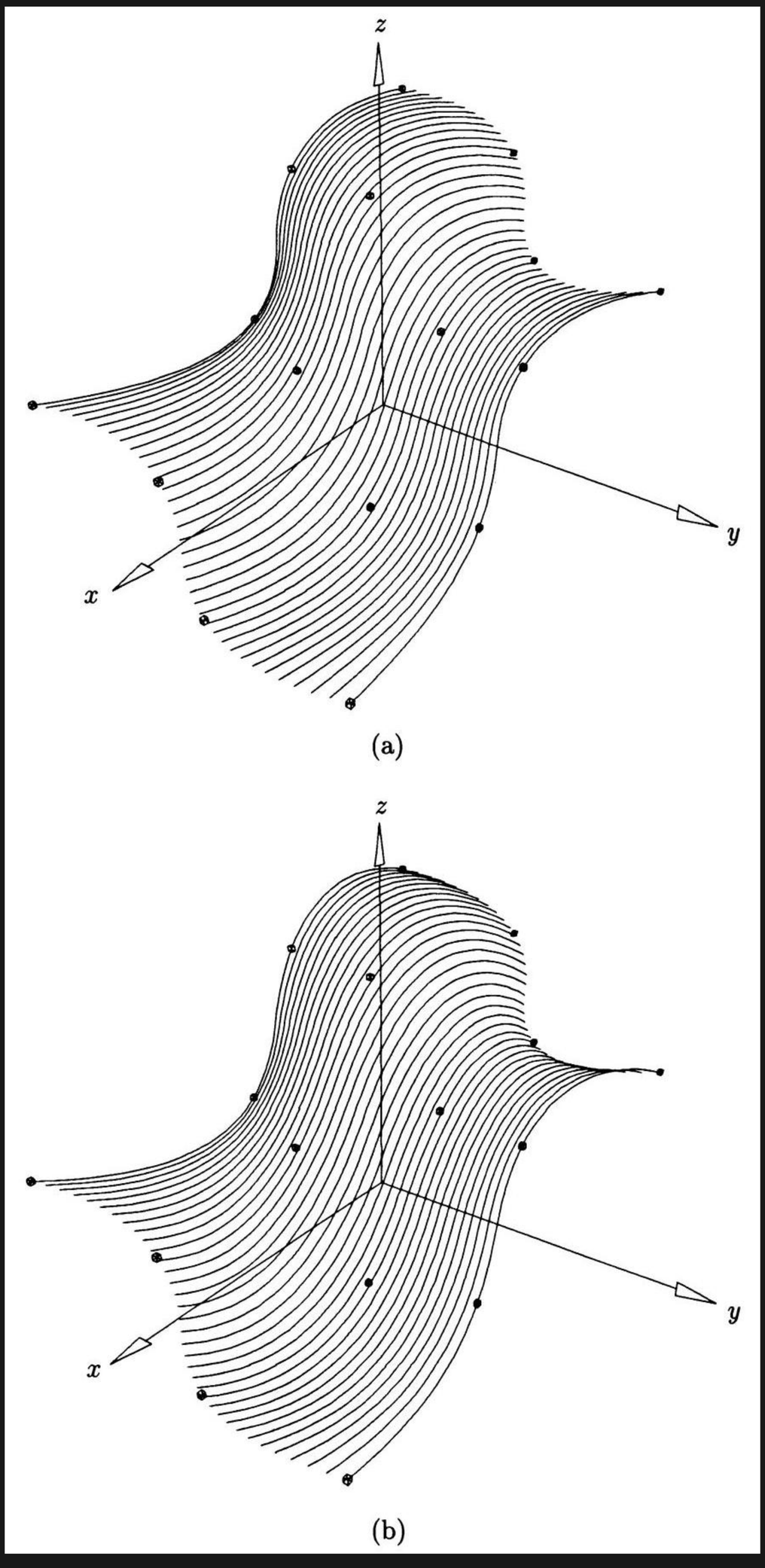


图 9.16. 通过使用不同的参数化和节点向量获得的度数为 (2,3) 的曲面插值。

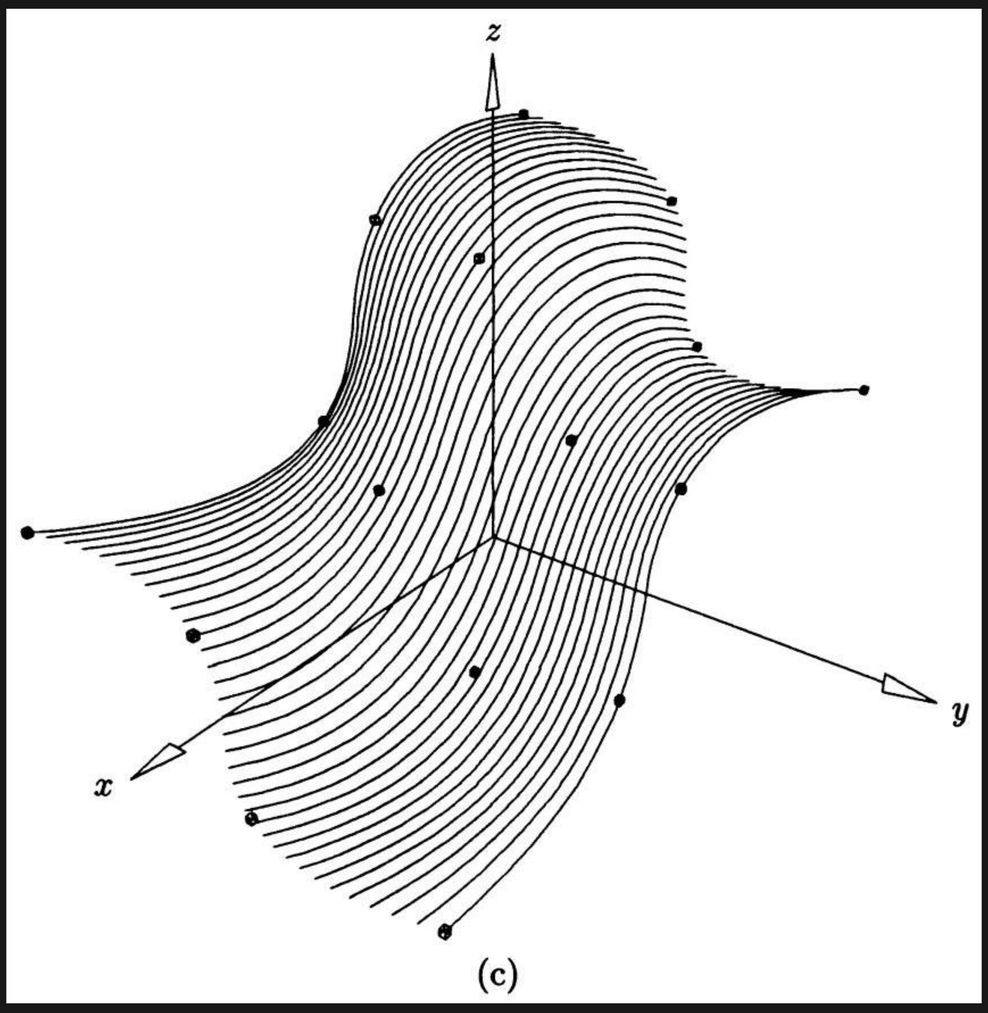


图 9.16.(续。)

获取 Bézier 段 $\mathbf{C}_i(u)$ 需要计算内部 Bézier 控制点，对于二次曲线需要一个点，对于三次曲线需要两个点。这些控制点位于与曲线在 $\mathbf{Q}_k$ 处相切的直线上；因此，我们需要在每个 $\mathbf{Q}_k$ 处的切向量 $\mathbf{T}_k$ 。在某些情况下，它们可以与 $\mathbf{Q}_k$ 一起输入；例如，当计算两个表面之间的交点时，它们很容易获得。然而，如果没有输入，则必须作为拟合算法的一部分进行计算。有多种方法可供选择；Boehm 等人 [Boeh84] 对各种方法进行了综述。让

$$\Delta\bar{u}_k = \bar{u}_k - \bar{u}_{k-1}\ \mathbf{q}_k = \mathbf{Q}_k - \mathbf{Q}_{k-1}\ \mathbf{d}_k = \frac{\mathbf{q}_k}{\Delta\bar{u}_k}$$

所有方法有两种形式之一

$$\mathbf{D}_k = (1-\alpha_k)\,\mathbf{d}_k + \alpha_k\mathbf{d}_{k+1} \tag{9.28}$$

或

$$\mathbf{T}_k = \frac{\mathbf{V}_k}{|\mathbf{V}_k|}\ \mathbf{V}_k = (1-\alpha_k)\,\mathbf{q}_k + \alpha_k\mathbf{q}_{k+1} \tag{9.29}$$

(见图 9.19)。注意，方程 (9.28) 假设 $\bar{u}_k$ 的值已经被指定。向量 $\mathbf{D}_k$ 可以被视为导数的估计值。

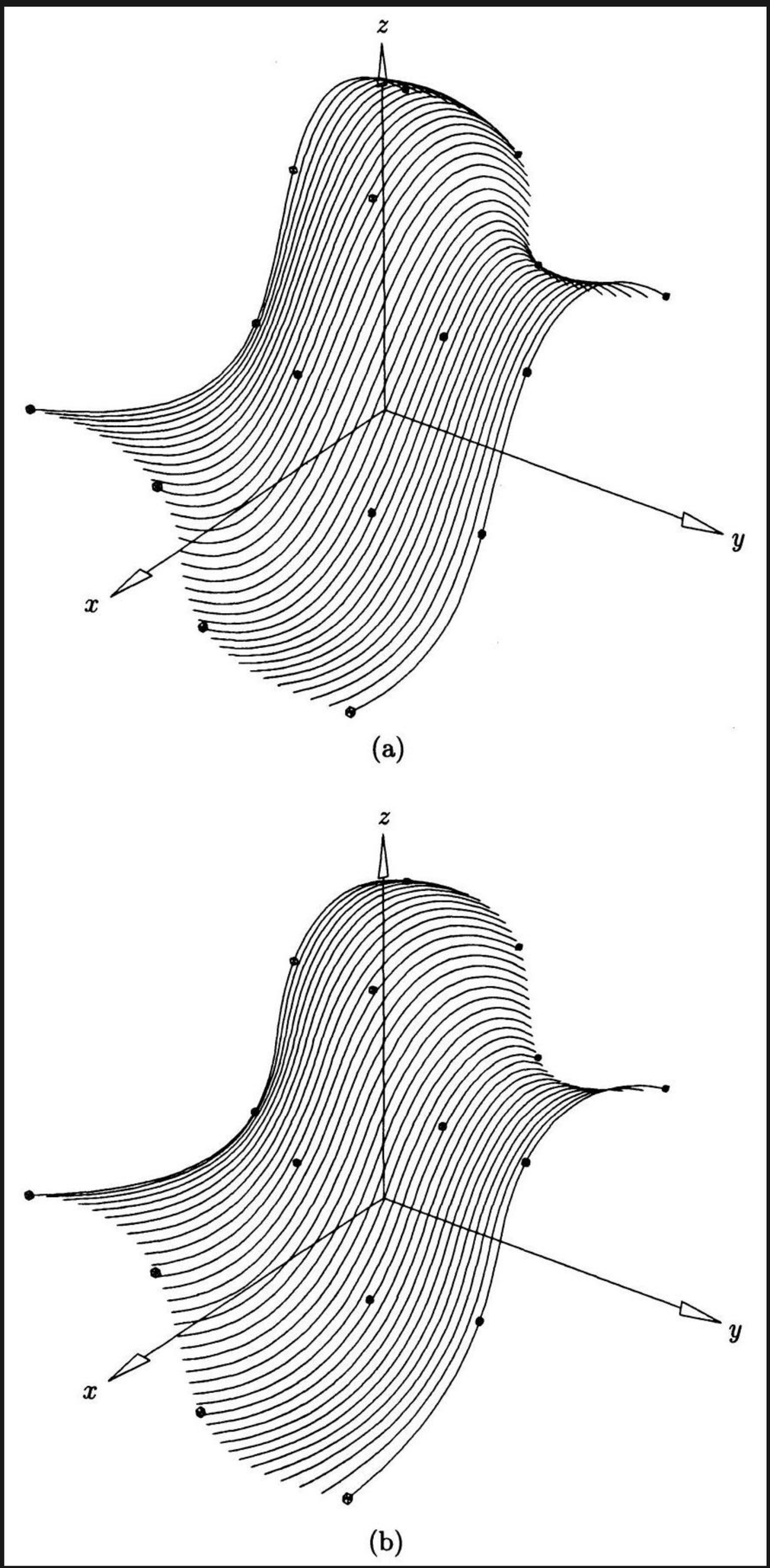


图 9.17. 使用不同次数的表面插值；参数化是弦长，节点向量通过平均获得。

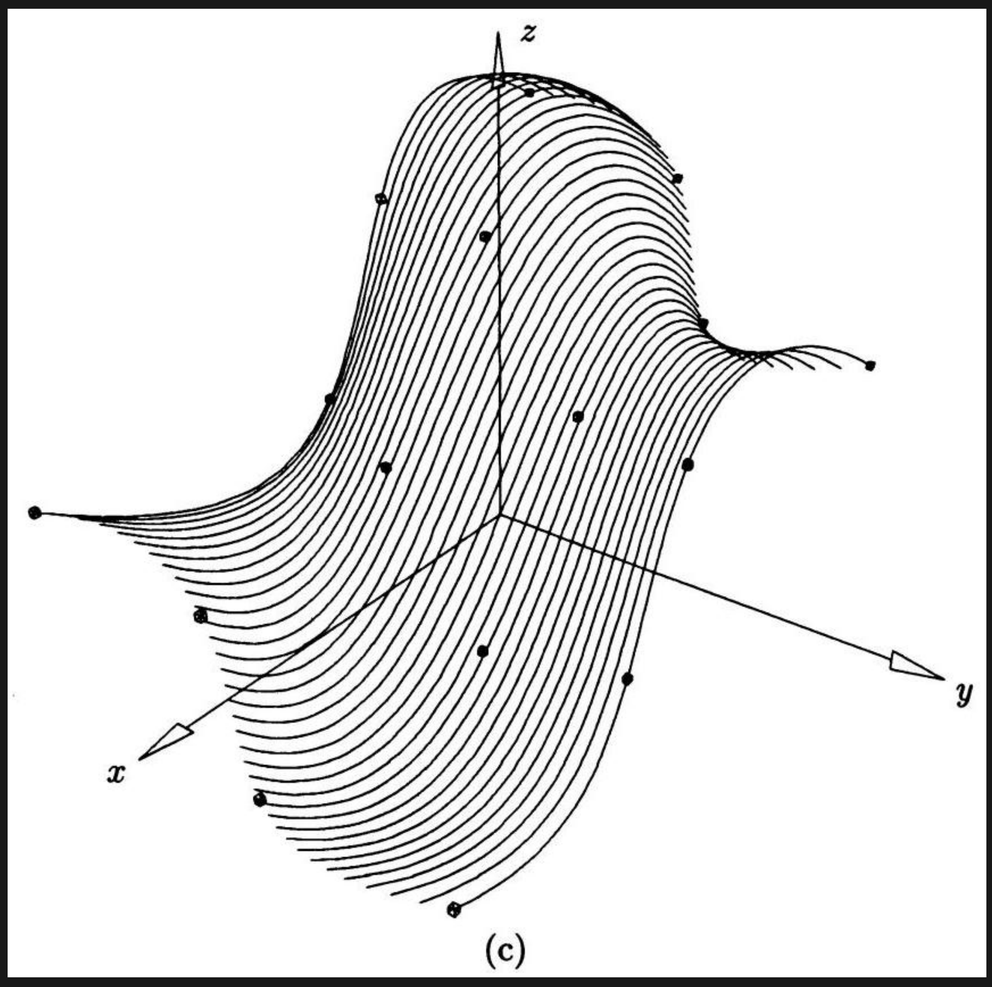


图 9.17. (续)

方程 (9.29) 不使用参数 $\bar{u}_k$，因此得到的向量应仅视为切向方向。幅度和参数的分配必须相互结合进行；它们不是独立的。还要注意，我们严格使用符号 $\mathbf{T}$ 来表示单位长度的切向量。方程 (9.28) 和 (9.29) 是线性插值。各种方案在计算插值参数 α_k 的方式上有所不同；它通常依赖于三个或五个邻近点。例如，贝塞尔方法 [DeBo78] 是一种三点法，使用

$$\alpha_k = \frac{\Delta\bar{u}_k}{\Delta\bar{u}_k + \Delta\bar{u}_{k+1}} \quad k = 1, \ldots, n-1 \tag{9.30}$$

结合方程 (9.28)。设定

$$\alpha_k = \frac{|\mathbf{q}_{k-1} \times \mathbf{q}_k|}{|\mathbf{q}_{k-1} \times \mathbf{q}_k| + |\mathbf{q}_{k+1} \times \mathbf{q}_{k+2}|} \quad k = 2, \ldots, n-2 \tag{9.31}$$

与方程 (9.29) 一起得出一种五点法来获得 $\mathbf{T}_k$ [Akim70; Renn82; Pieg87d]。它的优点在于三个共线点 $\mathbf{Q}_{k-1}, \mathbf{Q}_k$ 、$\mathbf{Q}_{k+1}$ 产生一个与线段平行的 $\mathbf{T}_k$ 。如果 $\mathbf{Q}_{k-2}, \mathbf{Q}_{k-1}, \mathbf{Q}_k$ 共线且 $\mathbf{Q}_k, \mathbf{Q}_{k+1}, \mathbf{Q}_{k+2}$ 共线，则方程 (9.31) 的分母消失。这意味着在 $\mathbf{Q}_k$ 处要么是一个拐角，要么是从 $\mathbf{Q}_{k-2}$ 到 $\mathbf{Q}_{k+2}$ 的一条直线段。在这些情况下，α_k 可以通过多种方式定义；我们选择

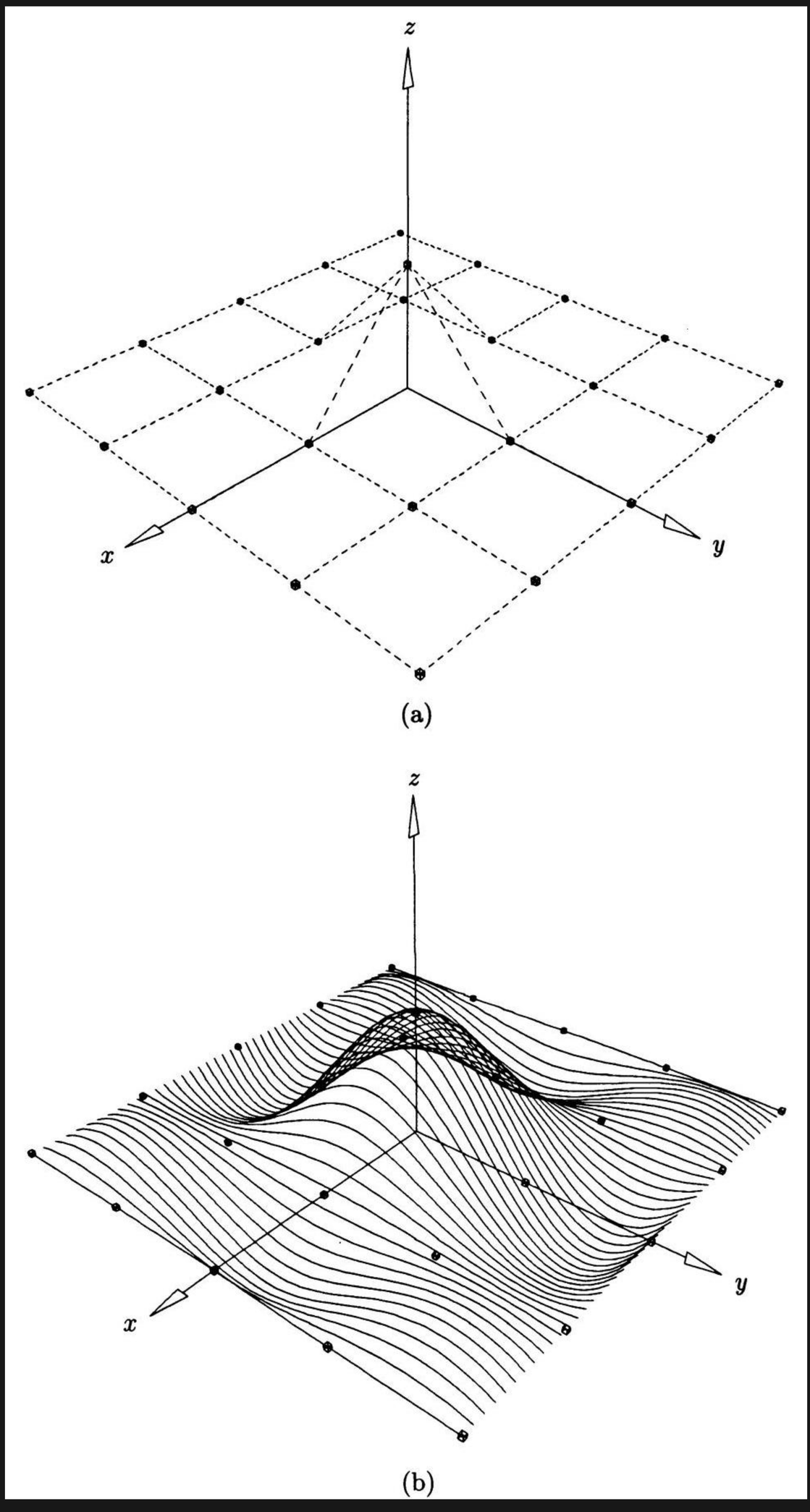


图 9.18. 全局表面插值示例。(a) 包含共面的数据点；(b) 插值表面。

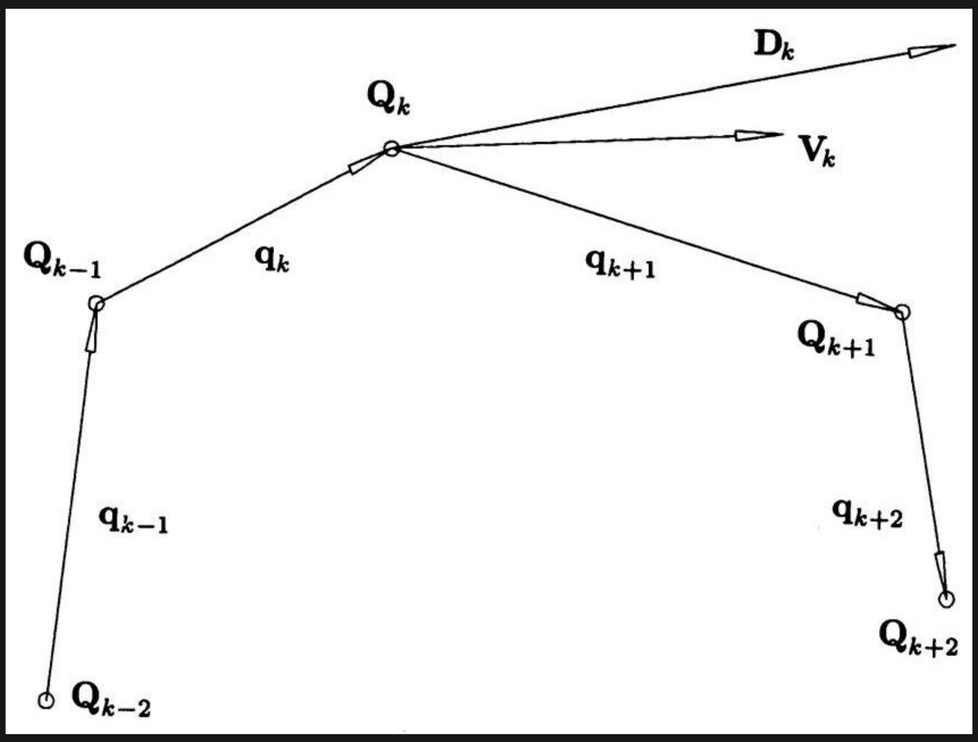


图 9.19. 局部曲线插值的切线 $(\mathbf{V}_k)$ 和导数 $(\mathbf{D}_k)$ 向量的计算。

- $\alpha_k = 1$，这意味着 $\mathbf{V}_k = \mathbf{q}_{k+1}$；如果数据暗示，则在 $\mathbf{Q}_k$ 处产生一个拐角；
- $\alpha_k = 1/2$，这意味着 $\mathbf{V}_k = 1/2\,(\mathbf{q}_k + \mathbf{q}_{k+1})$；这个选择在暗示有拐角的情况下使拐角平滑。

基于这些选择，局部曲线插值例程可以接受一个输入标志，指示是否保留拐角。所有方法在端点处都需要特殊处理。对于三点方案，我们可以设置

$$\mathbf{D}_0 = 2\mathrm{d}_1 - \mathbf{D}_1 \; \mathbf{D}_n = 2\mathrm{d}_n - \mathbf{D}_{n-1} \tag{9.32}$$

对于五点方案，我们可以使用

$$\mathbf{q}_0 = 2\mathbf{q}_1 - \mathbf{q}_2 \; \mathbf{q}_{-1} = 2\mathbf{q}_0 - \mathbf{q}_1$$

$$\mathbf{q}_{n+1} = 2\mathbf{q}_n - \mathbf{q}_{n-1} \; \mathbf{q}_{n+2} = 2\mathbf{q}_{n+1} - \mathbf{q}_n \tag{9.33}$$

以替代方程 (9.31) 和 (9.29)，以获得 $\mathbf{T}_0, \mathbf{T}_1$ 和 $\mathbf{T}_{n-1}, \mathbf{T}_n$。我们现在准备介绍几种局部插值方法。

3.2 局部抛物线曲线插值

我们从一个简单的二维非理性二次方案开始。设 $\{\mathbf{Q}_k\}, k = 0, \ldots, n$ 为 xy 平面中的点。使用某种切线估计方案来计算相应的 $\{\mathbf{T}_k\}$，例如，方程 (9.29)、(9.31) 和 (9.33)。设 L_k 表示由 $(\mathbf{Q}_k, \mathbf{T}_k)$ 定义的有向线，$\mathbf{R}_k$ 为 L_{k-1} 和 L_k 的交点。暂时假设交点存在，

并且

$$\gamma_{k-1} > 0 \; \gamma_k < 0 \tag{9.34}$$

其中

$$\mathbf{R}_k = \mathbf{Q}_{k-1} + \gamma_{k-1}\mathbf{T}_{k-1} \; \mathbf{R}_k = \mathbf{Q}_k + \gamma_k \mathbf{T}_k$$

(我们稍后会去掉这个限制)。然后我们选择作为控制点

$$\mathbf{Q}_0, \mathbf{R}_1, \mathbf{Q}_1, \mathbf{R}_2, \ldots, \mathbf{R}_n, \mathbf{Q}_n \tag{9.35}$$

设 $\bar{u}_0 = 0$ 和 $\bar{u}_n = 1$。如果 $\{\bar{u}_i\}, i = 1, \ldots, n-1$ 是满足 $\bar{u}_{i-1} < \bar{u}_i < \bar{u}_{i+1}$ 的任意数列，则方程 (9.35) 的控制点以及节点向量

$$U = \{0, 0, 0, \bar{u}_1, \bar{u}_1, \bar{u}_2, \bar{u}_2, \ldots, \bar{u}_{n-1}, \bar{u}_{n-1}, 1, 1, 1\} \tag{9.36}$$

定义了一个非理性、G^1 连续的二次 B 样条曲线，插值 $\{\mathbf{Q}_k\}$ (见图 9.20)。内部节点的选择不会影响曲线的形状，仅影响参数化。可以选择节点，使得得到的曲线是 C^1 连续的；在这样做时，我们可以去掉控制点 $\mathbf{Q}_1, \ldots, \mathbf{Q}_{n-1}$ 和每个内部节点的一个出现。我们简单地在 $\mathbf{Q}_{k-1}$ 处使起始和结束导数相等 (使用方程 [3.7] 转换到区间 $[\bar{u}_{k-2}, \bar{u}_{k-1}]$ 和 $[\bar{u}_{k-1}, \bar{u}_k]$))，并得到公式

$$\bar{u}_0 = 0 \; \bar{u}_1 = 1$$

$$\bar{u}_k = \bar{u}_{k-1} + (\bar{u}_{k-1} - \bar{u}_{k-2}) \frac{|\mathbf{R}_k - \mathbf{Q}_{k-1}|}{|\mathbf{Q}_{k-1} - \mathbf{R}_{k-1}|} \; k = 2, \ldots, n \tag{9.37}$$

插值 $\{\mathbf{Q}_k\}$ 的 C^1 连续曲线由控制点定义

$$\mathbf{Q}_0, \mathbf{R}_1, \mathbf{R}_2, \ldots, \mathbf{R}_{n-1}, \mathbf{R}_n, \mathbf{Q}_n \tag{9.38}$$

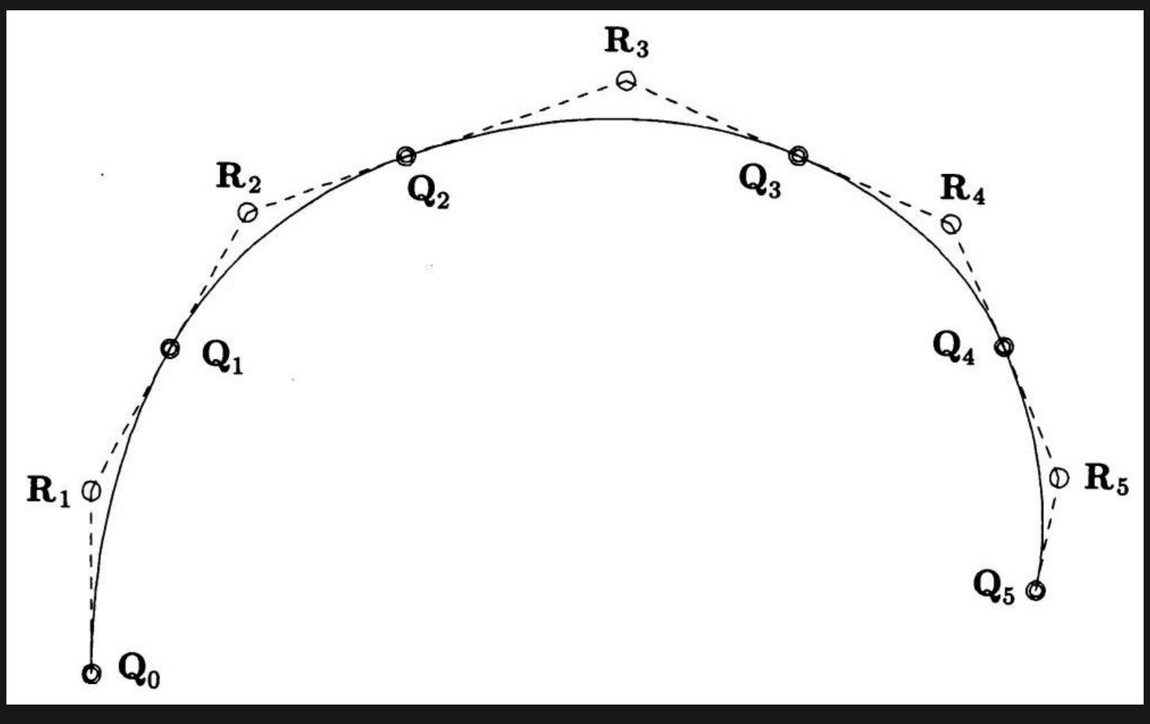


图 9.20. G^1 连续二次曲线插值。

和节点

$$U = \left\{0, 0, 0, \frac{\bar{u}_1}{\bar{u}_n}, \frac{\bar{u}_2}{\bar{u}_n}, \ldots, \frac{\bar{u}_{n-2}}{\bar{u}_n}, \frac{\bar{u}_{n-1}}{\bar{u}_n}, 1, 1, 1\right\} \tag{9.39}$$

为了实现 C^1 的连续性和控制点数量的减少，必须付出代价，即在参数化方面；图 9.21a 和 9.21b 显示了这一点。这些图中的曲线在等间距的参数值处标记。在图 9.21a 中，节点是使用公式 (9.37) 计算的，而在图 9.21b 中则使用了弦长参数化 (公式 [9.5])。请注意 $\mathbf{Q}_k$ 的间距。

现在我们去掉公式 (9.34) 中给出的限制，并详细说明在计算 $\mathbf{R}_k$ 时出现的两个特殊情况:

- $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 是平行的，因此 $\mathbf{R}_k$ 不能通过交点计算；这可能表示共线段、拐点或曲线中的 180° 转弯；
- $\mathbf{R}_k$ 可以计算，但 γ_{k-1} 和 γ_k 不满足公式 (9.34)；这表明要么是拐点，要么是超过 180° 的转弯。

如果 $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 都平行于弦 $\mathbf{Q_{k-1}Q_k}$ ，则适用共线段情况。我们通过设置简单地处理此情况

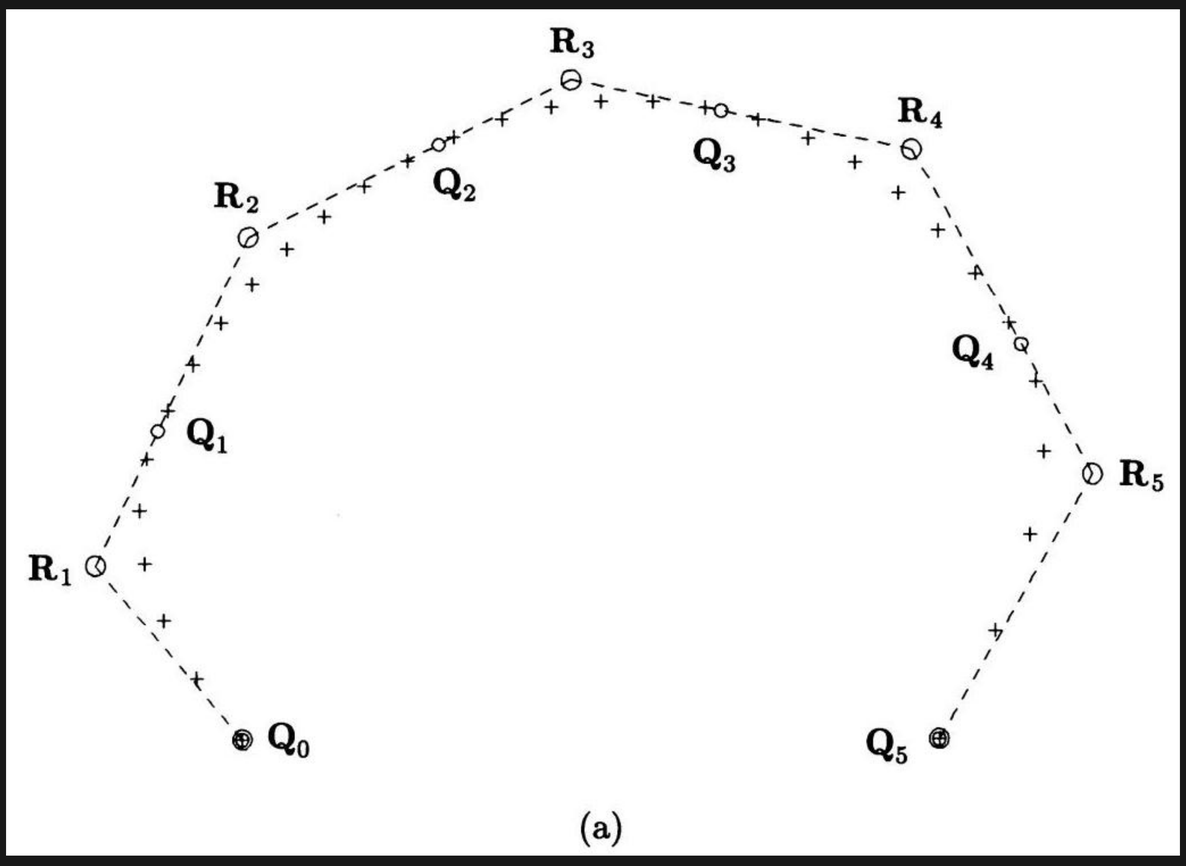


图 9.21. 图 9.20 中曲线插值的参数化。(a) C^1 参数化 (公式 [9.39]-[9.41])；(b) 使用弦长参数的 G^1 参数化。

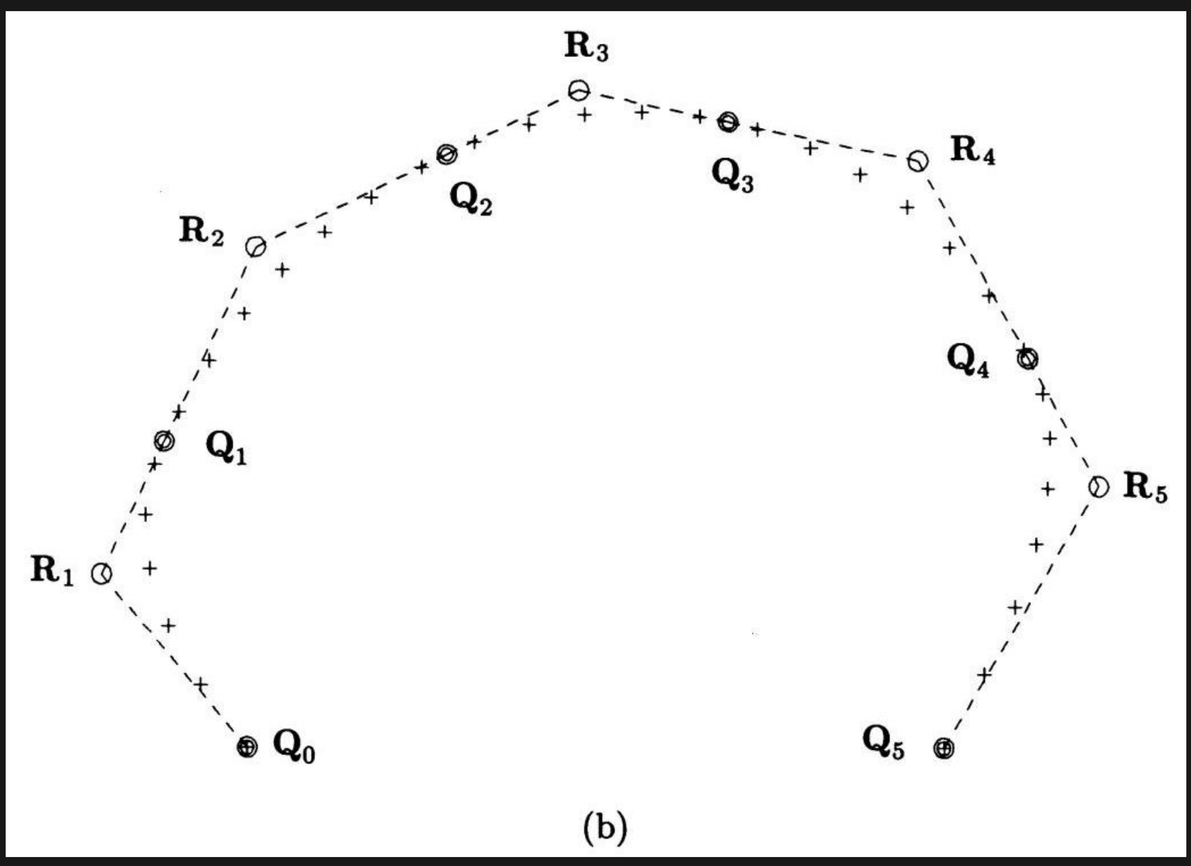


图 9.21. (续)

$$\mathbf{R}_k = \frac{1}{2}(\mathbf{Q}_{k-1} + \mathbf{Q}_k) \tag{9.40}$$

所有其他特殊情况通过在 $\mathbf{Q_{k-1}}$ 和 $\mathbf{Q}_k$ 之间创建两个抛物线段而不是一个来处理。因此，我们必须计算三个额外的点，$\mathbf{R}'_k$, $\mathbf{Q}'_k$ 和 $\mathbf{R}'_{k+1}$ (见图 9.22a 和 9.22b)。设置

$$\mathbf{R}'_k = \mathbf{Q}_{k-1} + \gamma_k \mathbf{T}_{k-1} \; \mathbf{R}'_{k+1} = \mathbf{Q}_k - \gamma_{k+1}\mathbf{T}_k \tag{9.41}$$

和

$$\mathbf{Q}'_k = \frac{\gamma_k \mathbf{R}'_{k+1} + \gamma_{k+1}\mathbf{R}'_k}{\gamma_k + \gamma_{k+1}} \tag{9.42}$$

仍需确定 γ_k 和 γ_{k+1} 的合理选择。如果 $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 是平行的 (但不与 $\mathbf{Q}_{k-1}\mathbf{Q}_k$ 平行)，则设置

$$\gamma_k = \gamma_{k+1} = \frac{1}{2}|\mathbf{Q}_{k-1}\mathbf{Q}_k| \tag{9.43}$$

这一点在图 9.22a 中得到了说明，针对 180° 转弯的情况。现在考虑 $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 不平行的情况，但方程 (9.34) 不成立 (见图 9.22b)。直观上，γ_k 和 γ_{k+1} 应该依赖于弦 $\mathbf{Q}_{k-1}\mathbf{Q}_k$ 和切线 $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 所夹的角度 θ_{k-1} 和 θ_k ，分别 $(0 \le \theta_{k-1}, \theta_k \le 90°)$ 。图 9.22c 显示了由等腰三角形定义的抛物线。对于这个图，可以很容易推导出

$$\gamma_k = \gamma_{k+1} = \frac{1}{4}\frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{\cos\theta}$$

这暗示着设置

$$\gamma_k = \frac{1}{4}\frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{\alpha\cos\theta_k + (1-\alpha)\cos\theta_{k-1}}$$

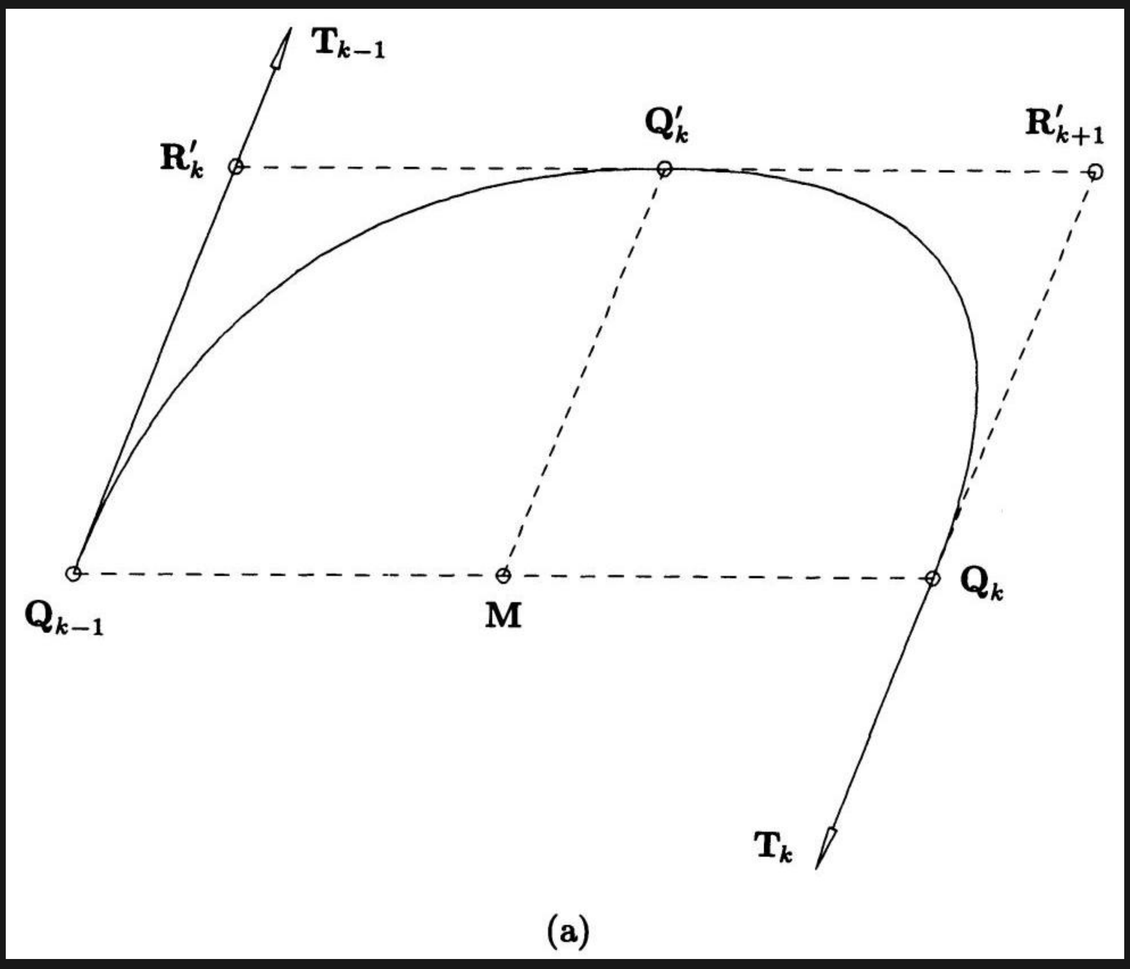


图 9.22. 计算在邻近数据点处切线的分段抛物线弧。(a) 平行的端切线；(b) 切线 $\mathbf{T}_{k-1}$ 和 $\mathbf{T}_k$ 所暗示的转折点；(c) 由等腰控制三角形定义的抛物线。

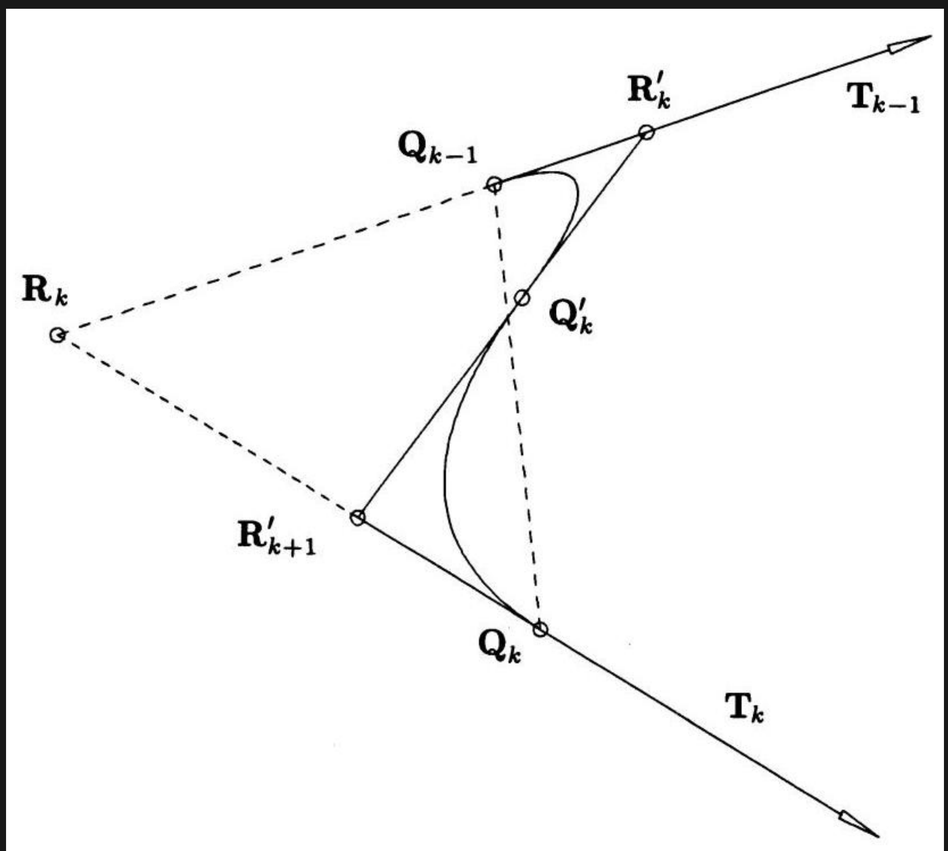
$$\gamma_{k+1} = \frac{1}{4} \frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{\alpha \cos\theta_{k-1} + (1-\alpha)\cos\theta_k} \tag{9.44}$$

其中 α 是介于 0 和 1 之间的某个常数。基于实验观察，我们选择 $\alpha = 2/3$ 。图 9.22 b 的两个曲线段是使用方程 (9.41)、(9.42) 和 (9.44) 得到的。

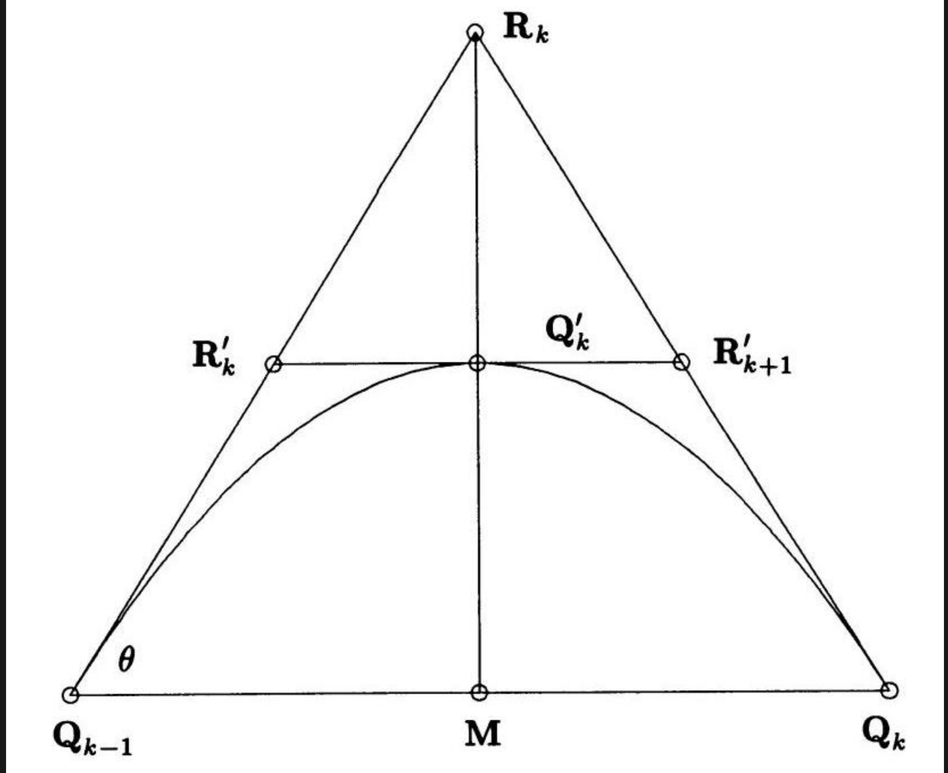
图 9.23-9.25 显示了几个示例。在图 9.23a 中，切线是通过贝塞尔方法 [DeBo78] 计算的，而在图 9.23b 中使用了 Akima 方法 [Akim70]。注意图 9.23b 中如何保持共线性。在图 9.24a 中选择了“无拐角”选项，导致每个段之间产生了 (可能不希望的) 平滑连接。图 9.24b 显示了保持拐角的情况，使用相同的算法，仅更改了标志设置。图 9.25 显示了使用该插值方法获得的鞋底轮廓。

3.3 局部有理二次曲线插值

前述方法可以修改为生成有理二次曲线。所有在 $\mathbf{Q}_k$ 的权重设置为 1；在 $\mathbf{R}_k$ 的权重可以自由选择。例如，一些应用可能希望段更“圆形”而不是抛物线形。更具体地，对于 $k = 1, \ldots, n$ ，定义了一个圆锥弧



(b)



(c)

图 9.22. (续。)

由 $\mathbf{Q}_{k-1}, \mathbf{R}_k, \mathbf{Q}_k$ 定义，权重 w_k 在 $\mathbf{R}_k$ 处。可以按如下方式设置 w_k：

1. 如果 $\mathbf{Q}_{k-1}, \mathbf{R}_k$ 和 $\mathbf{Q}_k$ 共线，则设置 $w_k = 1$；
2. 如果三角形是等腰三角形 ($|\mathbf{Q}_k \mathbf{R}_k| = |\mathbf{Q}_k \mathbf{R}_k|$)，则根据方程 (7.33) 设置 w_k (一个精确的圆弧)；

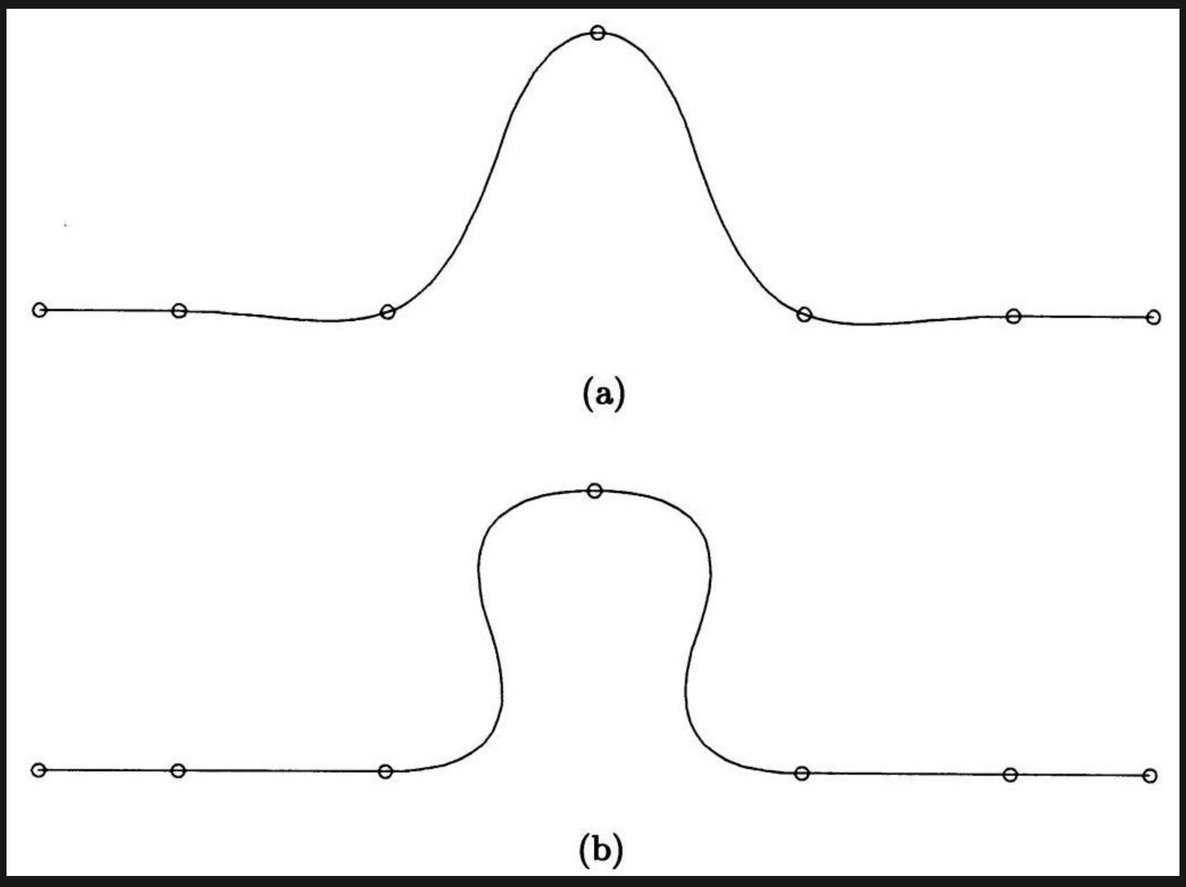


图 9.23. 局部二次曲线插值。(a) 通过贝塞尔方法计算的切线；(b) 通过 Akima 方法计算的切线。

3. 如果三角形不是等腰的，则按如下方式设置 w_k (见图 9.26):

a. 让 $\mathbf{M} = 1/2\,(\mathbf{Q}_{k-1} + \mathbf{Q}_k)$;

b. 设定 $\mathbf{S}_1$ 为 $\mathbf{MR}_k$ 的交点，与角的平分线 $\angle \mathbf{R}_k\mathbf{Q}_{k-1}\mathbf{Q}_k$ 。

c. 设定 $\mathbf{S}_2$ 为 $\mathbf{MR}_k$ 的交点，与角的平分线 $\angle \mathbf{Q}_{k-1}\mathbf{Q}_k\mathbf{R}_k$ 。

d. 设定 $\mathbf{S} = 1/2\,(\mathbf{S}_1 + \mathbf{S}_2)$;

e. 以 $\mathbf{S}$ 作为肩点，使用方程 (7.30) 和 (7.31) 计算 w_k 。

在计算 $\mathbf{R}_k$ 的特殊情况下的识别正如所详细描述的；然而，计算 $\mathbf{R}'_k$ 和 $\mathbf{R}'_{k+1}$ 略有不同。再次参考图 9.22c ，但假设等腰三角形定义了一个圆弧，由方程 (7.31)-(7.33) 可得

$$\gamma_k = \frac{1}{2}\frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{1+\cos\theta}$$

因此，我们设定

$$\gamma_k = \frac{1}{2}\frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{1+\alpha\cos\theta_k + (1-\alpha)\cos\theta_{k-1}}$$

$$\gamma_{k+1} = \frac{1}{2}\frac{|\mathbf{Q}_{k-1}\mathbf{Q}_k|}{1+\alpha\cos\theta_{k-1} + (1-\alpha)\cos\theta_k} \tag{9.45}$$

其中 α 是与方程 (9.44) 中相同的常数。

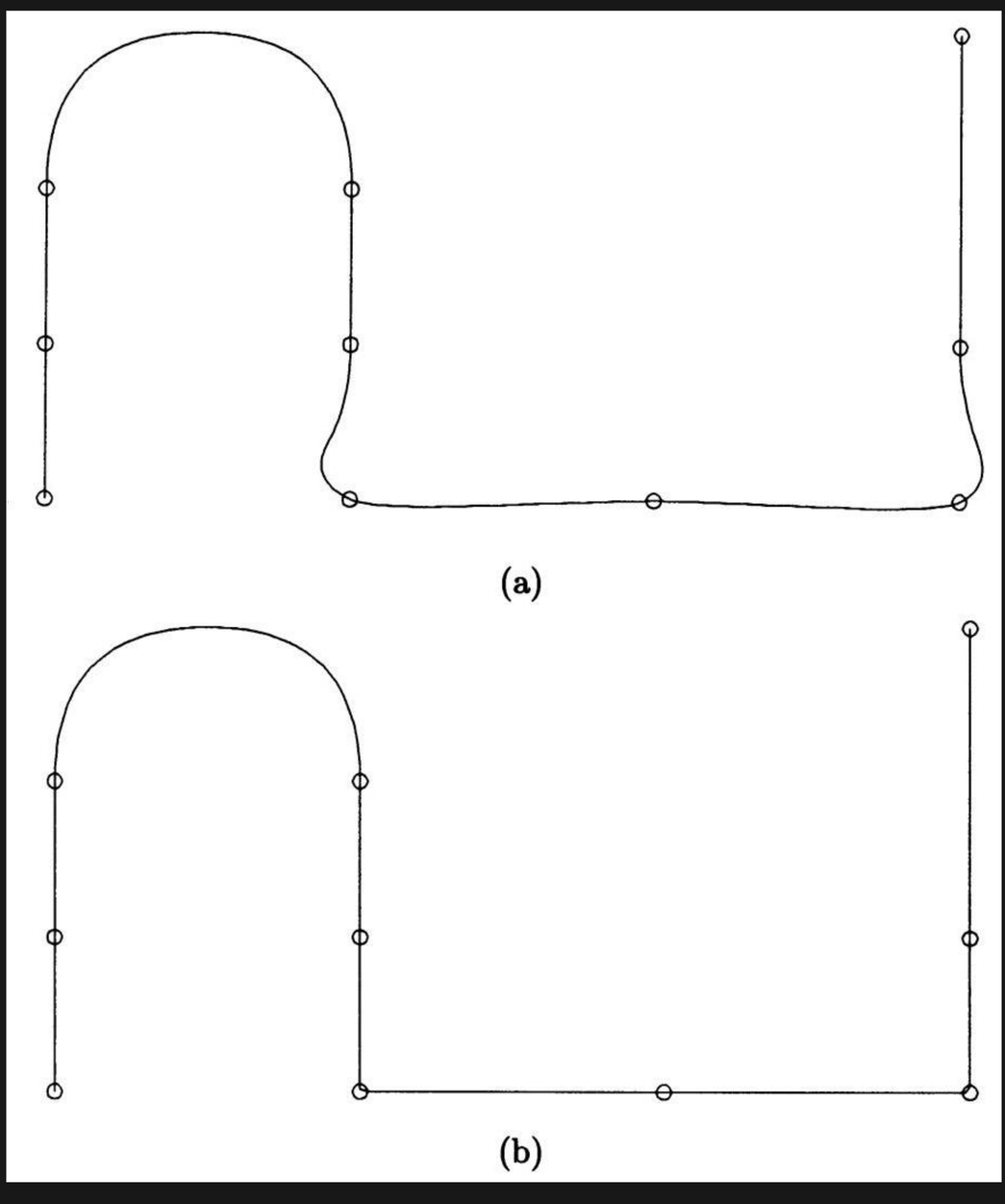


图 9.24. 局部二次曲线插值。(a) 不允许有角；(b) 保持角。

图 9.27 显示了有理二次曲线插值的一个例子。注意，从四个点精确重现了圆形，并且通过设置角标志保留了两个角。

3.4 局部三次曲线插值

三次曲线能够轻松处理三维数据和拐点，而无需特殊处理。我们首先推导出关于三次贝塞尔曲线的一般事实，这本身就是一个有趣的主题。设 $\mathbf{P}_0$ 和 $\mathbf{P}_3$ 为两个端点，$\mathbf{T}_0$ 和 $\mathbf{T}_3$ 为对应的单位长度切线方向。可以构造一条三次贝塞尔曲线 $\mathbf{C}(u), u \in [0,1]$ ，其端点和切线方向满足

$$\alpha = |\mathbf{C}'(0)| = \left|\mathbf{C}'\left(\frac{1}{2}\right)\right| = |\mathbf{C}'(1)| \tag{9.46}$$

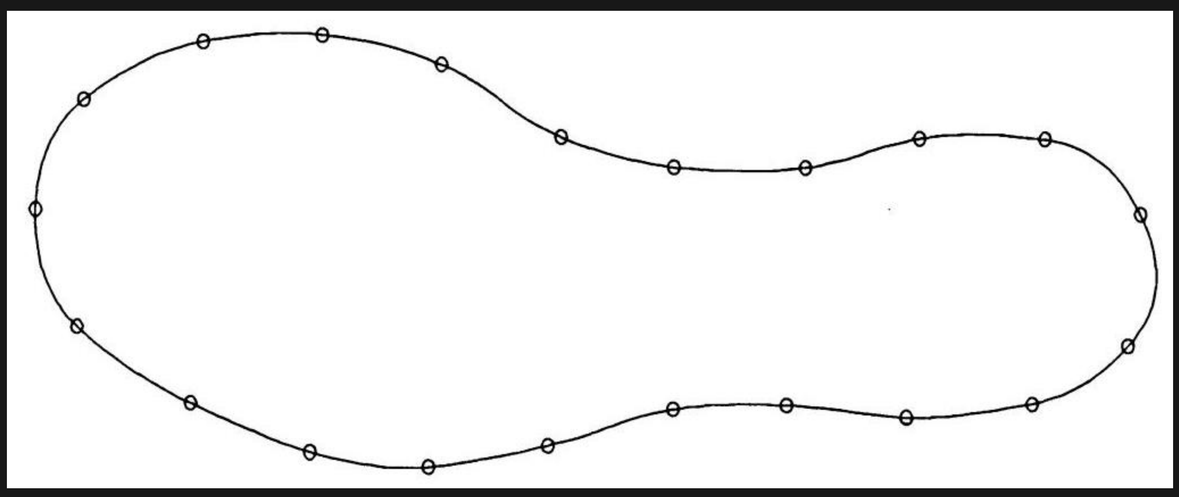


图 9.25. 一段分段 G^1 抛物线曲线插值。

(见图 9.28)，即速度在曲线的起点、中点和终点处是相等的。方程 (9.46) 和 (1.10) 暗示

$$\mathbf{P}_1 = \mathbf{P}_0 + \frac{1}{3}\alpha\mathbf{T}_0 \; \mathbf{P}_2 = \mathbf{P}_3 - \frac{1}{3}\alpha\mathbf{T}_3 \tag{9.47}$$

我们在 $u = 1/2$ 处应用第一章的 deCasteljau 算法，并回忆该算法中使用的符号 (见图 1.17)，我们写出 $\mathbf{P}_0^3 = \mathbf{C}(1/2)$ 。通过反推 deCasteljau 算法，可以推导出

$$\mathbf{P}_1^2 - \mathbf{P}_0^3 = \frac{1}{8}(\mathbf{P}_3 + \mathbf{P}_2 - \mathbf{P}_1 - \mathbf{P}_0) \tag{9.48}$$

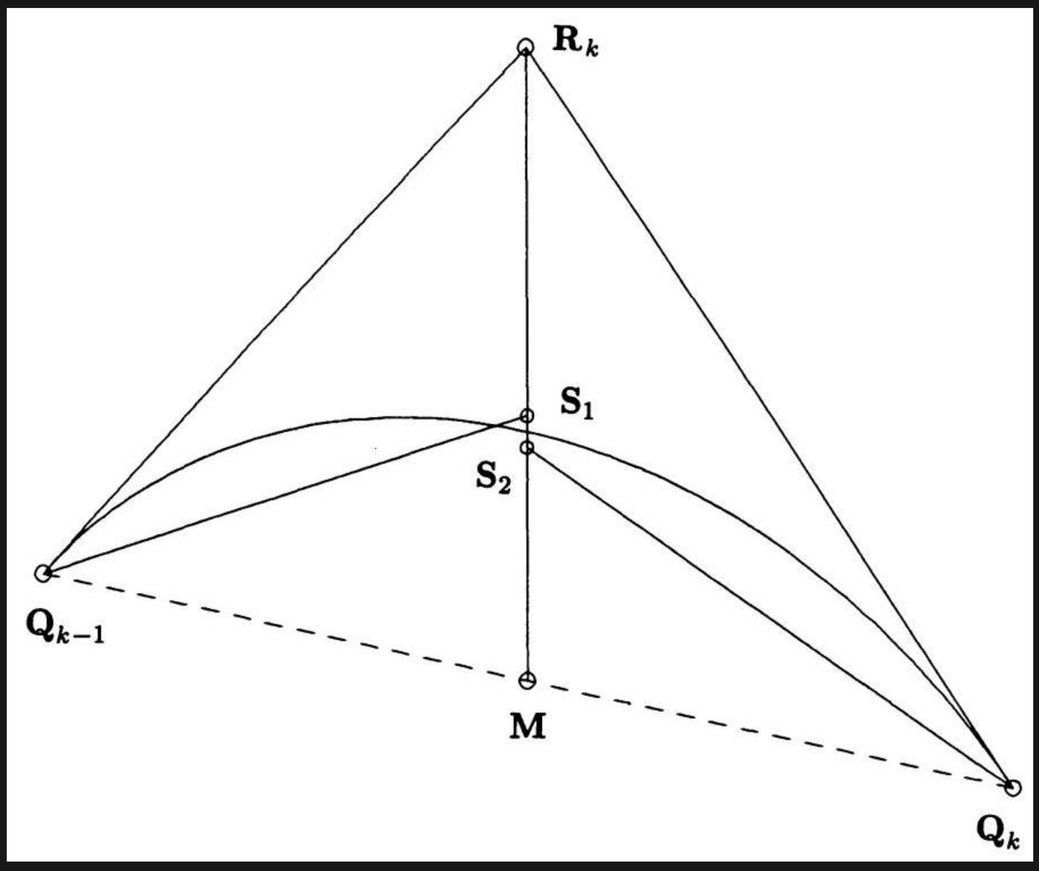


图 9.26. 为有理二次插值选择权重 (肩点)。

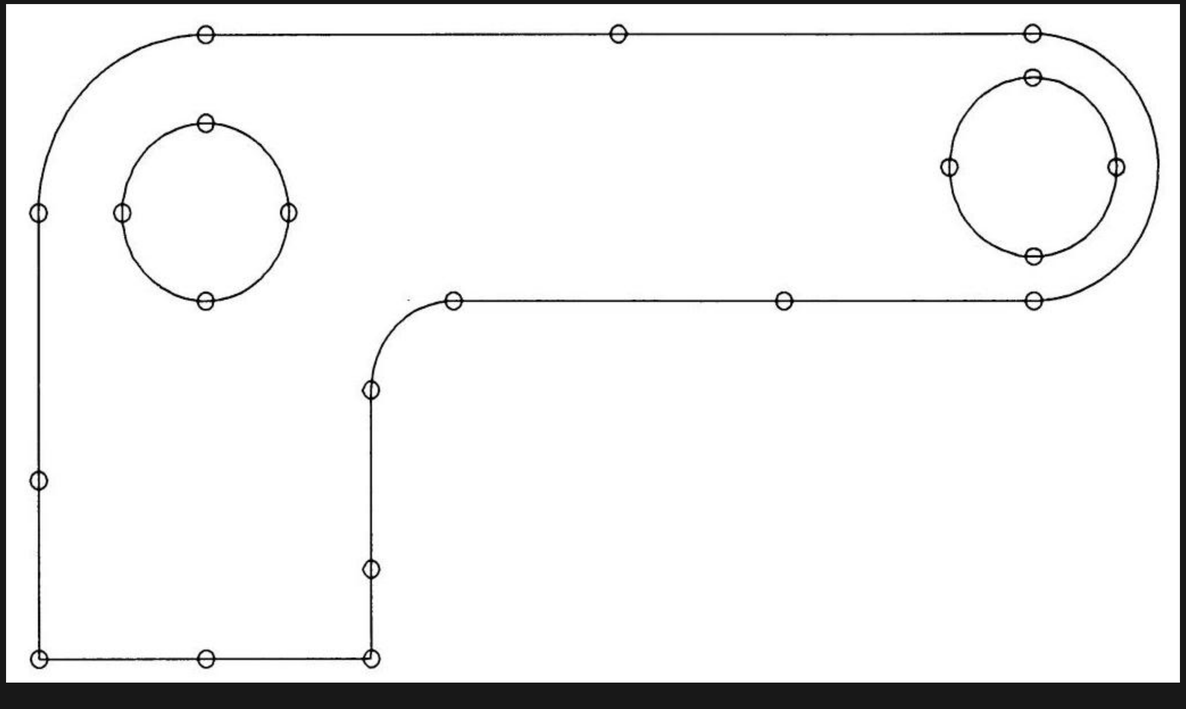


图 9.27. 有理二次曲线插值；圆形被精确再现，角落保持不变。

应用方程 (1.10) 并考虑参数范围的二分法，它

得出

$$\mathbf{C}'\left(\frac{1}{2}\right) = 6\left(\mathbf{P}_1^2 - \mathbf{P}_0^3\right) \tag{9.49}$$

将速度设为 α ，将方程 (9.48) 代入方程 (9.49)，并使用方程 (9.47)，我们得到

$$\frac{8}{6}\alpha = |\mathbf{P}_3 + \mathbf{P}_2 - \mathbf{P}_1 - \mathbf{P}_0|$$

$$= \left|\mathbf{P}_3 + \left(\mathbf{P}_3 - \frac{1}{3}\alpha\mathbf{T}_3\right) - \left(\mathbf{P}_0 + \frac{1}{3}\alpha\mathbf{T}_0\right) - \mathbf{P}_0\right|$$

这导致

$$16\alpha^2 = \alpha^2|\mathbf{T}_0 + \mathbf{T}_3|^2 - 12\alpha\left(\mathbf{P}_3 - \mathbf{P}_0\right)\cdot\left(\mathbf{T}_0 + \mathbf{T}_3\right) + 36|\mathbf{P}_3 - \mathbf{P}_0|^2$$

最终

$$a\alpha^2 + b\alpha + c = 0 \tag{9.50}$$

在哪里

$$a = 16 - |\mathbf{T}_0 + \mathbf{T}_3|^2\ b = 12\left(\mathbf{P}_3 - \mathbf{P}_0\right)\cdot\left(\mathbf{T}_0 + \mathbf{T}_3\right)\ c = -36|\mathbf{P}_3 - \mathbf{P}_0|^2$$

方程 (9.50) 有两个实数解 α ，一个是正数，一个是负数。将正数解代入方程 (9.47) 得到所需的 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 。

现在我们回到立方插值问题。设 $\{\mathbf{Q}_k\}, k = 0, \ldots, n$ 为一组三维数据点。如果没有给出切向量，则计算它们 (方程 [9.29]、[9.31] 和 [9.33])。我们必须构造一个立方 Bézier 曲线。

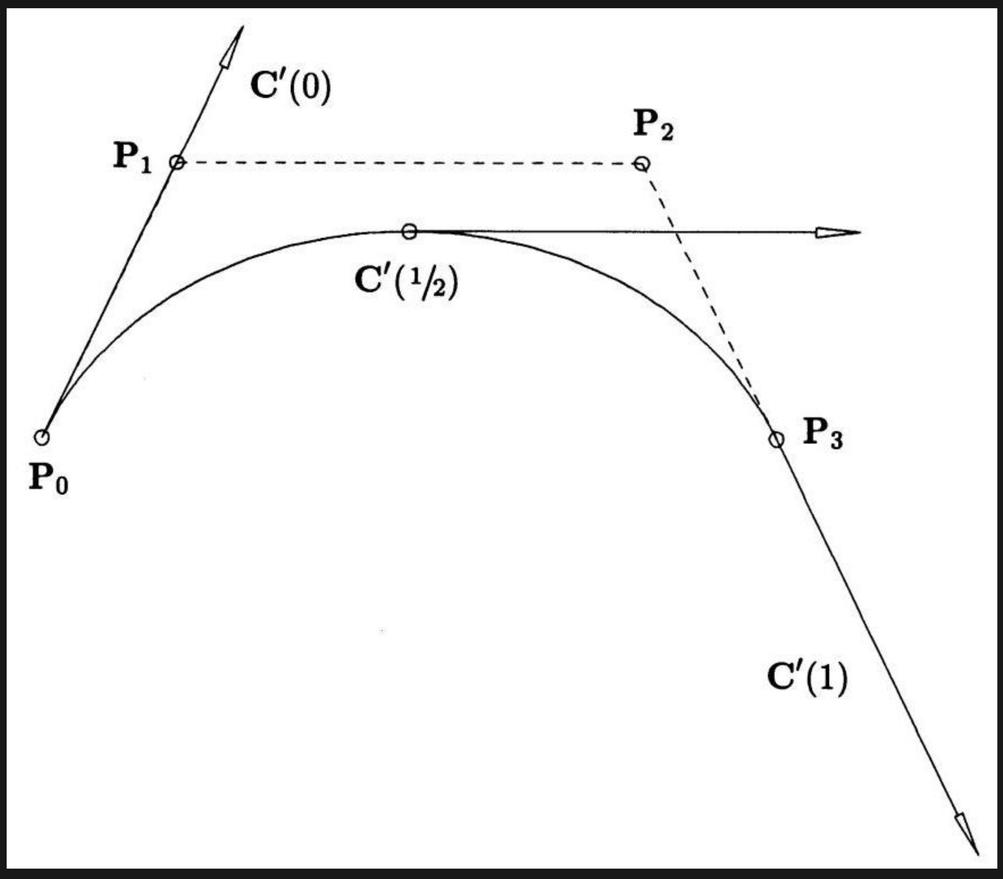


图 9.28。一个在参数 $0, 1/2$ 和 1 处具有相等切向量大小的立方 Bézier 曲线。

曲线段 $\mathbf{C}_k(u)$ 在每对 $\mathbf{Q}_k, \mathbf{Q}_{k+1}$ 之间。用 Bézier 控制点表示为

$$\mathbf{P}_{k,0} = \mathbf{Q}_k\ \mathbf{P}_{k,1}\ \mathbf{P}_{k,2}\ \mathbf{P}_{k,3} = \mathbf{Q}_{k+1} \tag{9.51}$$

我们必须确定 $\mathbf{P}_{k,1}$ 和 $\mathbf{P}_{k,2}$ 在 $\mathbf{T}_k$ 和 $\mathbf{T}_{k+1}$ 上的合适位置。可以获得一个 C^1 连续立方体，并实现对均匀参数化的良好近似。该方法源于 Renner [Renn82]。真正的均匀参数化意味着在整个参数范围内的恒定速度。我们构造一条在每个 $\mathbf{Q}_k$ 及每个 Bézier 段的中点处速度相等的曲线。设定 $\bar{u}_0 = 0$ 。现在，对于 $k = 0, \ldots, n-1$ ，$\bar{u}_{k+1}$ 和两个内控制点 $\mathbf{C}_k(u)$ 的计算如下:

1. 使用公式 (9.50) 计算 α ，使用公式 (9.47) 计算 $\mathbf{P}_{k,1}$ 和 $\mathbf{P}_{k,2}$ ；

2. 设定

$$\bar{u}_{k+1} = \bar{u}_k + 3\,|\mathbf{P}_{k,1} - \mathbf{P}_{k,0}| \tag{9.52}$$

该算法产生 n Bézier 段，每个段在其参数范围的端点和中点处的速度均为 1，$[\bar{u}_k, \bar{u}_{k+1}]$ 。因此，一个 C^1 连续的三次 B 样条曲线通过控制点定义，以插值 $\mathbf{Q}_k$ (见图 9.29)。

$$\mathbf{Q}_0, \mathbf{P}_{0,1}, \mathbf{P}_{0,2}, \mathbf{P}_{1,1}, \mathbf{P}_{1,2}, \ldots, \mathbf{P}_{n-2,2}, \mathbf{P}_{n-1,1}, \mathbf{P}_{n-1,2}, \mathbf{Q}_n \tag{9.53}$$

和节点

$$U = \left\{0, 0, 0, 0, \frac{\bar{u}_1}{\bar{u}_n}, \frac{\bar{u}_1}{\bar{u}_n}, \frac{\bar{u}_2}{\bar{u}_n}, \frac{\bar{u}_2}{\bar{u}_n}, \ldots, \frac{\bar{u}_{n-1}}{\bar{u}_n}, \frac{\bar{u}_{n-1}}{\bar{u}_n}, 1, 1, 1, 1\right\} \tag{9.54}$$

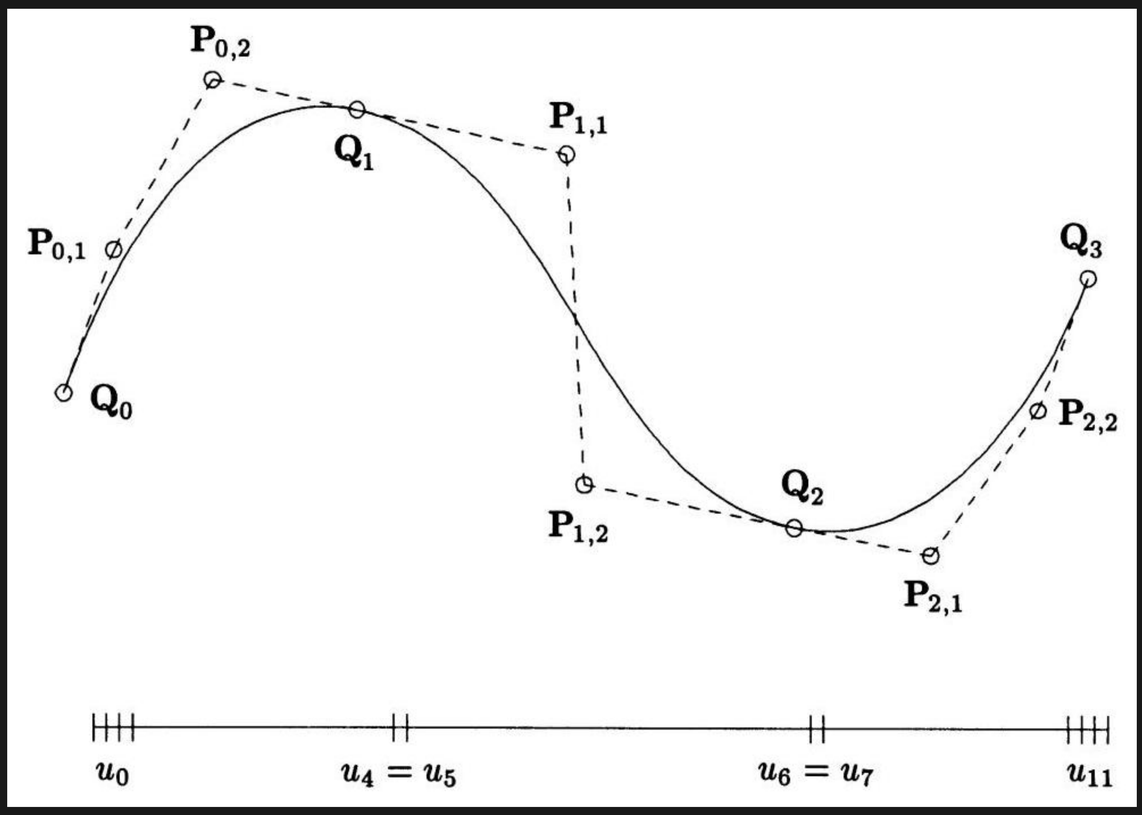


图 9.29. C^1 局部三次曲线插值。

图 9.30 显示了使用该方法进行三次曲线插值的一个示例。曲线在对应于等间隔参数值的点上标记。图 9.31 是另一个使用与图 9.7 相同数据集的插值示例。注意共线点是如何通过直线三次段拟合的。

3.5 局部双三次曲面插值

也可以实现一个 $C^{(1,1)}$ 连续的双三次局部曲面插值方案 (C^1 在 u 和 v 方向上均连续)。设 $\{\mathbf{Q}_{k,\ell}\}, k = 0, \ldots, n$ 和 $\ell = 0, \ldots, m$ 为一组数据点，设 $\{(\bar{u}_k, \bar{v}_\ell)\}$ 为通过弦长平均计算的对应参数对 (如算法 A9.3 所示)。以下方法生成满足条件的双三次曲面 $\mathbf{S}(u, v)$ 。

$$\mathbf{S}(\bar{u}_k, \bar{v}_\ell) = \sum_{i=0}^{2n+1} \sum_{j=0}^{2m+1} N_{i,3}(\bar{u}_k) N_{j,3}(\bar{v}_\ell) \mathbf{P}_{i,j} \tag{9.55}$$

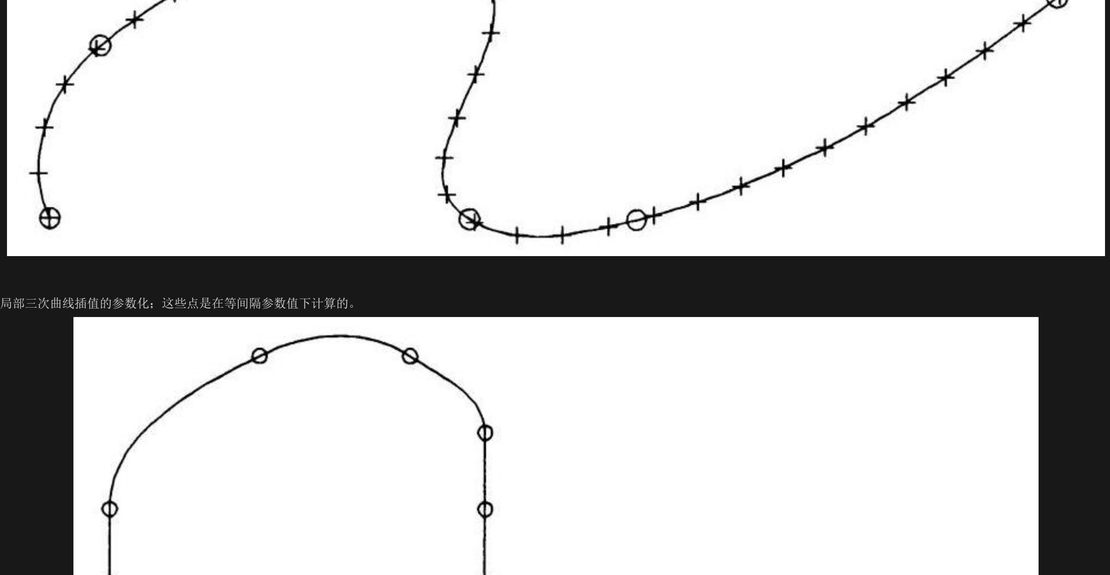


图 9.30. 局部三次曲线插值的参数化；这些点是在等间隔参数值下计算的。

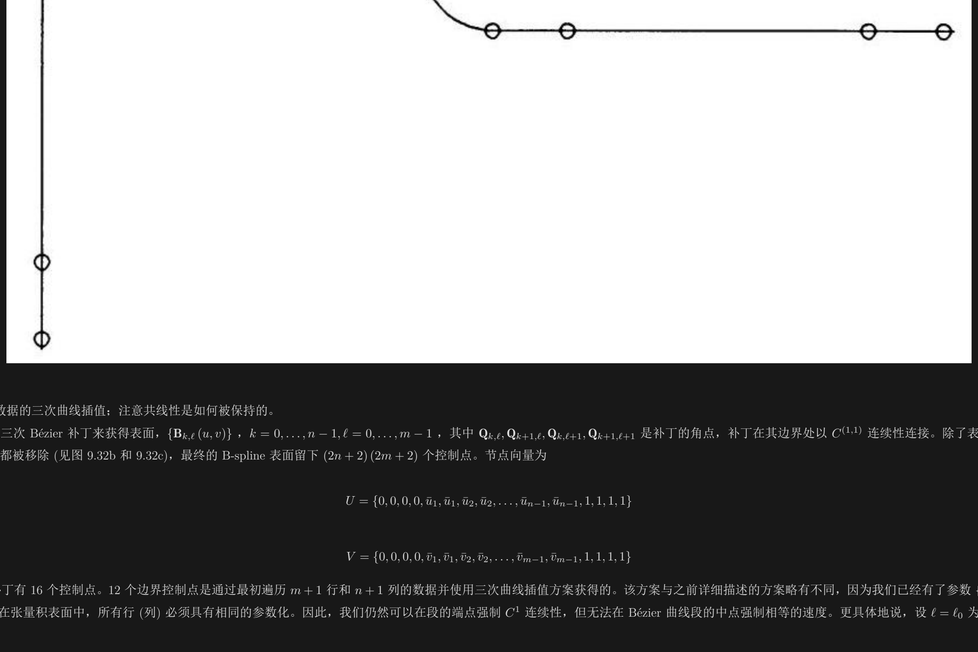


图 9.31. 对图 9.7 中数据的三次曲线插值；注意共线性是如何被保持的。

我们通过构造 nm 双三次 Bézier 补丁来获得表面，$\{\mathbf{B}_{k,\ell}(u, v)\}$ ，$k = 0, \ldots, n-1, \ell = 0, \ldots, m-1$ ，其中 $\mathbf{Q}_{k,\ell}, \mathbf{Q}_{k+1,\ell}, \mathbf{Q}_{k,\ell+1}, \mathbf{Q}_{k+1,\ell+1}$ 是补丁的角点，补丁在其边界处以 $C^{(1,1)}$ 连续性连接。除了表面边界外，所有包含原始 $\{\mathbf{Q}_{k,\ell}\}$ 的控制点的行和列都被移除 (见图 9.32b 和 9.32c)，最终的 B-spline 表面留下 $(2n+2)(2m+2)$ 个控制点。节点向量为

$$U = \{0, 0, 0, 0, \bar{u}_1, \bar{u}_1, \bar{u}_2, \bar{u}_2, \ldots, \bar{u}_{n-1}, \bar{u}_{n-1}, 1, 1, 1, 1\}$$

$$V = \{0, 0, 0, 0, \bar{v}_1, \bar{v}_1, \bar{v}_2, \bar{v}_2, \ldots, \bar{v}_{m-1}, \bar{v}_{m-1}, 1, 1, 1, 1\} \tag{9.56}$$

一个双三次 Bézier 补丁有 16 个控制点。12 个边界控制点是通过最初遍历 $m+1$ 行和 $n+1$ 列的数据并使用三次曲线插值方案获得的。该方案与之前详细描述的方案略有不同，因为我们已经有了参数 $\{(\bar{u}_k, \bar{v}_\ell)\}$ 。请注意，这些参数必须提前计算，因为在张量积表面中，所有行 (列) 必须具有相同的参数化。因此，我们仍然可以在段的端点强制 C^1 连续性，但无法在 Bézier 曲线段的中点强制相等的速度。更具体地说，设 $\ell = \ell_0$ 为固定，并考虑插值这些点的三次曲线

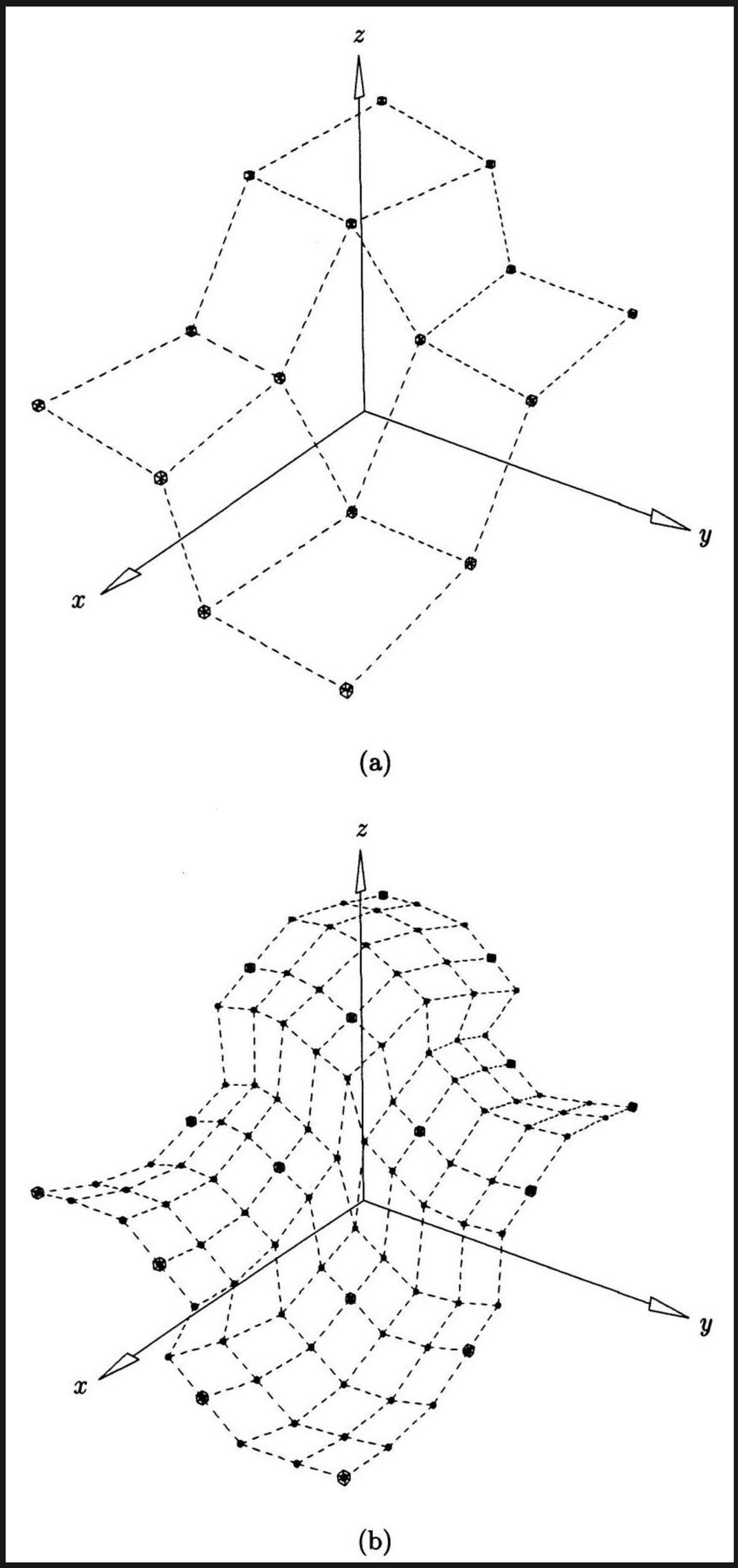
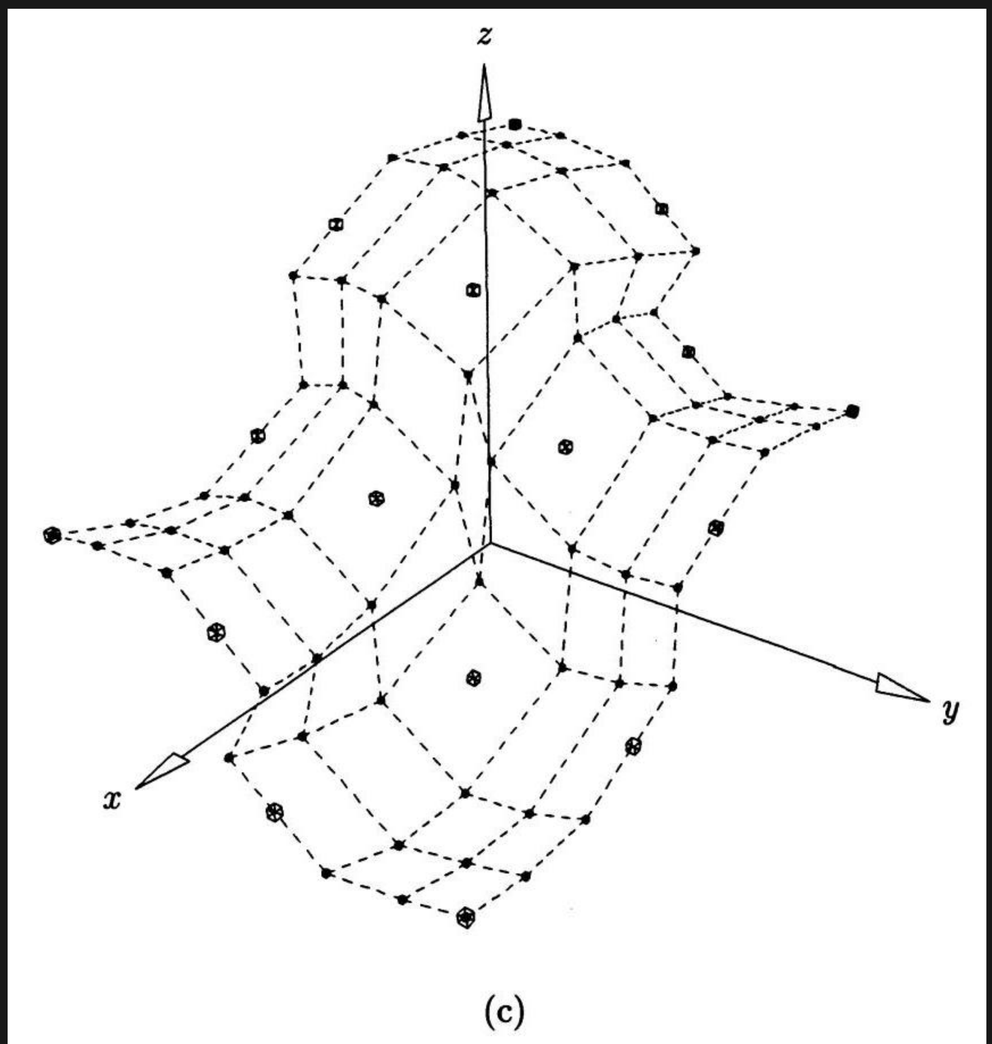


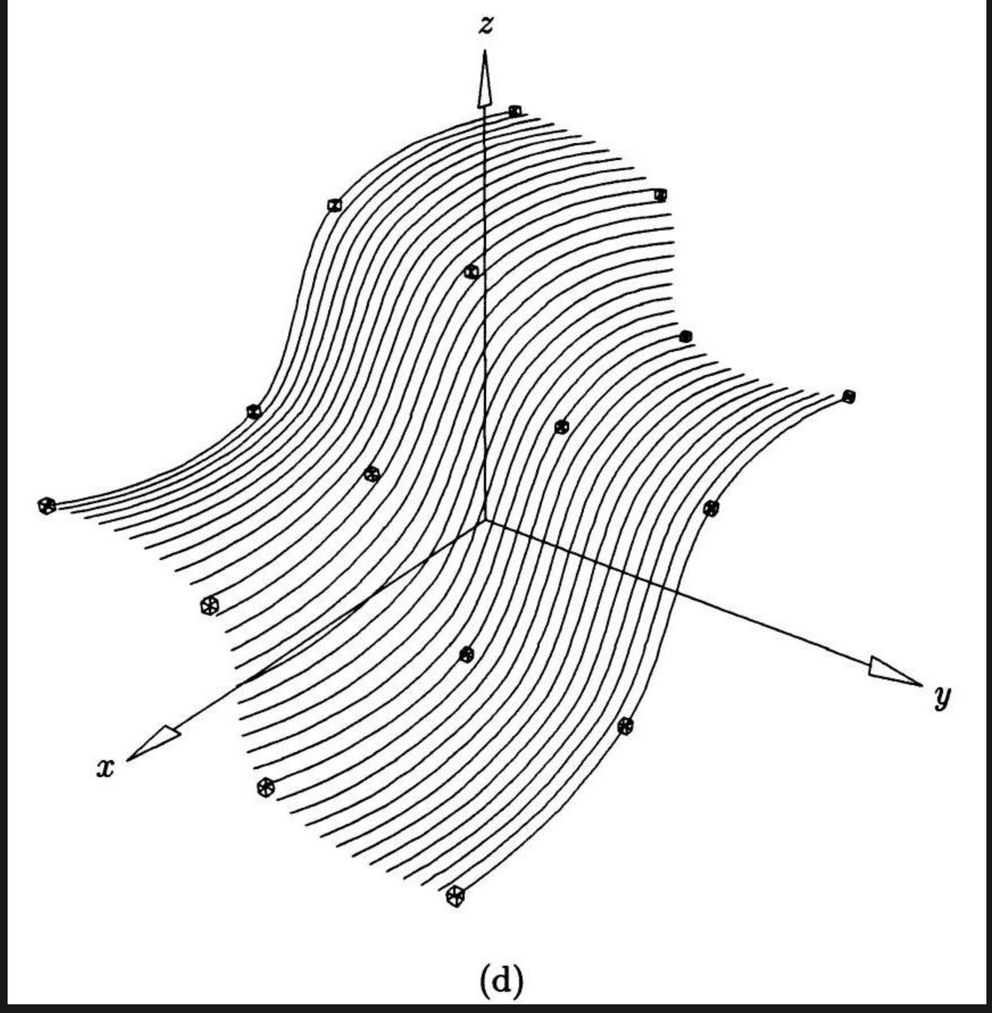
图 9.32. $C^{(1,1)}$ 局部双三次表面插值。(a) 数据集；(b) 插值的 Bézier 网格。

402 曲线和表面拟合 $\mathbf{Q}_{0,\ell_0},\ldots,\mathbf{Q}_{n,\ell_0}$ 。设 r_{ℓ_0} 表示第 ℓ_0 行的总弦长。在每个点 $\mathbf{Q}_{k,\ell_0}$ 处，计算 $\mathbf{T}^u_{k,\ell_0}$ (在 u 方向上的单位切线)，如前所述 (方程 [9.29]、[9.31] 和 [9.33])。然后，通过以下方式计算该行的内部 Bézier 点

$$\mathbf{P}^{k,\ell_0}_{1,0} = \mathbf{Q}_{k,\ell_0} + a\mathbf{T}^u_{k,\ell_0}\ \mathbf{P}^{k,\ell_0}_{2,0} = \mathbf{Q}_{k+1,\ell_0} - a\mathbf{T}^u_{k+1,\ell_0} \tag{9.57}$$



(c)



(d)

图 9.32.(续)(c) 一个 B-spline 网 (大立方体标记数据点，小立方体是 B-spline 控制点)；(d) 表面插值函数。

在哪里

$$a = \frac{r_{\ell_0}(\bar{u}_{k+1} - \bar{u}_k)}{3} = \frac{r_{\ell_0}\Delta\bar{u}_{k+1}}{3}$$

结果曲线是 C^1 连续的，导数的大小在所有 $\mathbf{Q}_{k,\ell_0}$ 处等于 r_{ℓ_0}。同样的技术应用于 $n+1$ 数据列。

现在需要计算每个 Bézier 补丁的四个内部控制点。这需要在每个 $\mathbf{Q}_{k,\ell}$ 处对混合偏导数 $\mathbf{D}^{uv}_{k,\ell}$ 的估计，以及方程 (3.24)(和其他三个角的类似公式)。我们基于三点贝塞尔方法 (方程 [9.28]、[9.30] 和 [9.32]) 推导出 $\mathbf{D}^{uv}_{k,\ell}$ 的公式。设 r_ℓ 和 s_k 表示第 ℓ 行 (第 k 列) 的总弦长。那么

$$\mathbf{D}^u_{k,\ell} = r_\ell \mathbf{T}^u_{k,\ell} \; \mathbf{D}^v_{k,\ell} = s_k \mathbf{T}^v_{k,\ell} \tag{9.58}$$

然后设置

$$\mathbf{d}^{vu}_{k,\ell} = (1-\alpha_k)\frac{\mathbf{D}^v_{k,\ell} - \mathbf{D}^v_{k-1,\ell}}{\Delta\bar{u}_k} + \alpha_k \frac{\mathbf{D}^v_{k+1,\ell} - \mathbf{D}^v_{k,\ell}}{\Delta\bar{u}_{k+1}}$$

和

$$\mathbf{d}^{uv}_{k,\ell} = (1-\beta_\ell)\frac{\mathbf{D}^u_{k,\ell} - \mathbf{D}^u_{k,\ell-1}}{\Delta\bar{v}_\ell} + \beta_\ell \frac{\mathbf{D}^u_{k,\ell+1} - \mathbf{D}^u_{k,\ell}}{\Delta\bar{v}_{\ell+1}}$$

与

$$\alpha_k = \frac{\Delta\bar{u}_k}{\Delta\bar{u}_k + \Delta\bar{u}_{k+1}} \quad \beta_\ell = \frac{\Delta\bar{v}_\ell}{\Delta\bar{v}_\ell + \Delta\bar{v}_{\ell+1}}$$

最后

$$\mathbf{D}_{k,\ell}^{uv} = \frac{\alpha_k \mathbf{d}_{k,\ell}^{uv} + \beta_\ell \mathbf{d}_{k,\ell}^{vu}}{\alpha_k + \beta_\ell} \tag{9.59}$$

在边界上必须使用适当的结束公式 (方程 [9.32])。现在使用方程 (9.59) 和 (3.24) 计算 (k, ℓ) 补丁的四个内部控制点。

$$\mathbf{P}_{1,1}^{k,\ell} = \gamma \mathbf{D}_{k,\ell}^{uv} + \mathbf{P}_{0,1}^{k,\ell} + \mathbf{P}_{1,0}^{k,\ell} - \mathbf{P}_{0,0}^{k,\ell}$$

$$\mathbf{P}_{2,1}^{k,\ell} = -\gamma \mathbf{D}_{k+1,\ell}^{uv} + \mathbf{P}_{3,1}^{k,\ell} - \mathbf{P}_{3,0}^{k,\ell} + \mathbf{P}_{2,0}^{k,\ell}$$

$$\mathbf{P}_{1,2}^{k,\ell} = -\gamma \mathbf{D}_{k,\ell+1}^{uv} + \mathbf{P}_{1,3}^{k,\ell} - \mathbf{P}_{0,3}^{k,\ell} + \mathbf{P}_{0,2}^{k,\ell}$$

$$\mathbf{P}_{2,2}^{k,\ell} = \gamma \mathbf{D}_{k+1,\ell+1}^{uv} + \mathbf{P}_{2,3}^{k,\ell} + \mathbf{P}_{3,2}^{k,\ell} - \mathbf{P}_{3,3}^{k,\ell}$$

在哪里

$$\gamma = \frac{\Delta\bar{u}_{k+1}\Delta\bar{v}_{\ell+1}}{9}$$

现在我们用一个算法进行总结。为了提高效率，事件的顺序与此处详细描述的略有不同。该算法需要局部数组: $td\,[n+1]\,[m+1]\,[3]$ ，包含 $\mathbf{T}_{k,\ell}^{u}, \mathbf{T}_{k,\ell}^{v}$ 和 $\mathbf{D}_{k,\ell}^{uv}$ ； ub $[n+1]$ 和 $vb\,[m+1]$ ，包含参数值；以及 $r\,[m+1]$ 和 $s\,[n+1]$ ，分别存储行和列的总弦长。

ALGORITHM A9.5

```
LocalSurfInterp(n, m, Q, U, V, P)
    { /* Local surface interpolation */
    { /* through (n + 1) (m + 1) points */
            /* Input: n, m, Q */
            /* Output: U, V, P */
        total = 0.0 ; /* get ub[], r [] and u direction tangents * /
        for (k=0; k<=n; k++) ub[k] = 0.0;
        for (l=0; l<=m; l++)
            {
            Compute and load T^u_{0,ℓ} into td[0][1][0];
            r[1] = 0.0 ;
            for (k=1; k<=n; k++)
                {
                Compute and load T^u_{k,ℓ} into td[k] [1] [0]
                        /* Eqs. (9.31) and (9.33) */
                d = |Q_{k,ℓ} − Q_{k−1,ℓ}| ;
                ub[k] = ub[k] + d ;
                r[l] = r[l]+d;
                }
            total = total + r[l];
            }
        for (k = 1; k < n; k + +) ub[k] = ub[k − 1] + ub[k] / total;
        ub[n] = 1.0;
        total = 0.0 ; /* get vb[], s [] and v direction tangents * /
        for (l=0; l<=m; l++) vb[l] = 0.0;
        for (k=0; k<=n; k++)
            {
            Compute and load T^v_{k,0} into td [k] [0] [1] ;
            s[k] = 0.0 ;
            for (l=1; l<=m; l++)
                {
                Compute and load T^v_{k,ℓ} into td[k] [1] [1]
                        /* Eqs. (9.31) and (9.33) */
                d = |Q_{k,ℓ} − Q_{k,ℓ−1}| ;
                vb[l] = vb[l]+d;
                s[k] = s[k] + d;
                }
            total = total + s[k];
            }
    for (l = 1; l < m; l + +) vb[l] = vb[l − 1] + vb[l] / total;
    vb[m] = 1.0;
    Load the U knot vector;
    Load the V knot vector;
    Compute all Bézier control points along each row and
            column of data points.
    for (k = 0; k <= n; k + +)
        for (l=0; l<=m; l++)
            {
            Compute the D^{uv}_{k,ℓ} by Eq. (9.59) and load into td[k] [1] [2] .
            }
    for (k = 0; k < n; k + +)
        for (l=0; l<m; l++)
            {
            Compute the four inner control points of the(k, l)th
                Bezier patch and load them into P .
            }
    Load the NURBS control points by discarding Bézier points
            along inner rows and columns. /* Figure 9.32c */
    }
```

图 9.32 显示了表面构造过程。在图 9.32a 中，数据点用小立方体标记。图 9.32b 说明了 Bézier 补丁的控制点。沿内行和内列消除 Bézier 点会产生图 9.32c 所示的 NURBS 表面控制网。表面在图 9.32d 中显示。

该插值方案可以轻松生成闭合的 $C^{(1,1)}$ 连续表面。图 9.33a 和 9.33 b 显示了一个表面插值的示例，其中数据指示在一个方向上是闭合的表面。在图 9.34a 和 9.34 b 中，表面在两个方向上都是闭合的 (类似于环形插值)。该方法的一个优良特性是，当数据点指示平面性时，会生成平坦 (平面) 补丁。图 9.35a 和 9.35b 显示了这一点的一个示例。

当数据不均匀分布时，获得令人满意的表面比获得曲线更为困难。图 9.36a 和 9.36 b 显示了对非常不均匀分布数据的插值。尽管表面并不特别令人满意，但使用前一节的全局方法得到的结果并没有太大不同。

4　全局逼近

逼近比插值更为困难。在插值中，控制点的数量由所选的次数和数据项的数量自动确定，节点放置也很简单，并且没有需要检查的曲线或表面误差。在逼近中，曲线/表面误差界限 E 与要拟合的数据一起输入。通常事先并不知道需要多少个控制点才能获得所需的精度 E ，并且 406 曲线和表面拟合

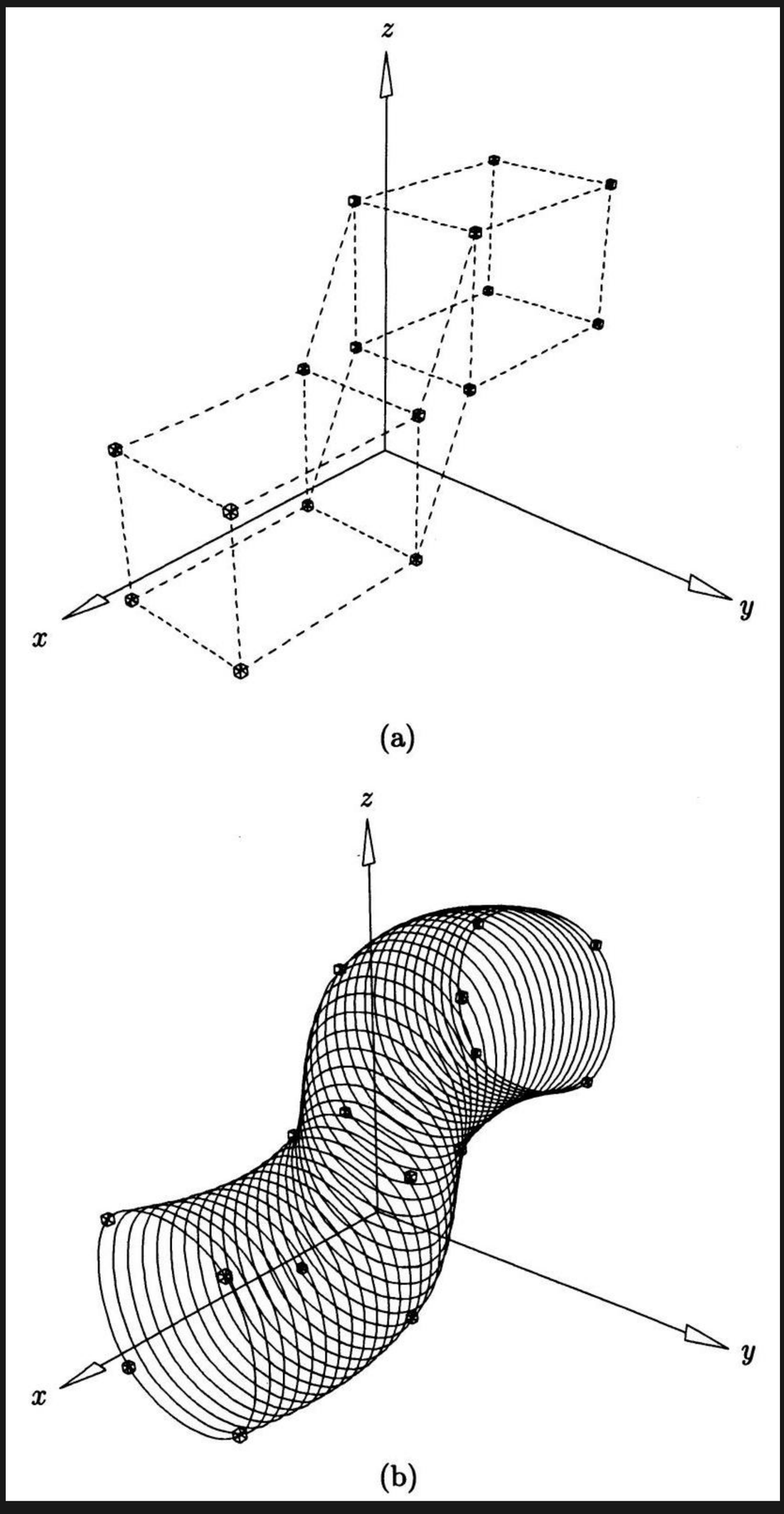


图 9.33. 使用局部双三次曲面插值的表面插值示例。(a) 数据点; (b) 插值表面。

因此，近似方法通常是迭代的。粗略来说，全局方法有两种进行方式:

1a. 从最小或少量控制点开始；

b. 使用全局拟合方法将近似曲线或表面拟合到数据上；

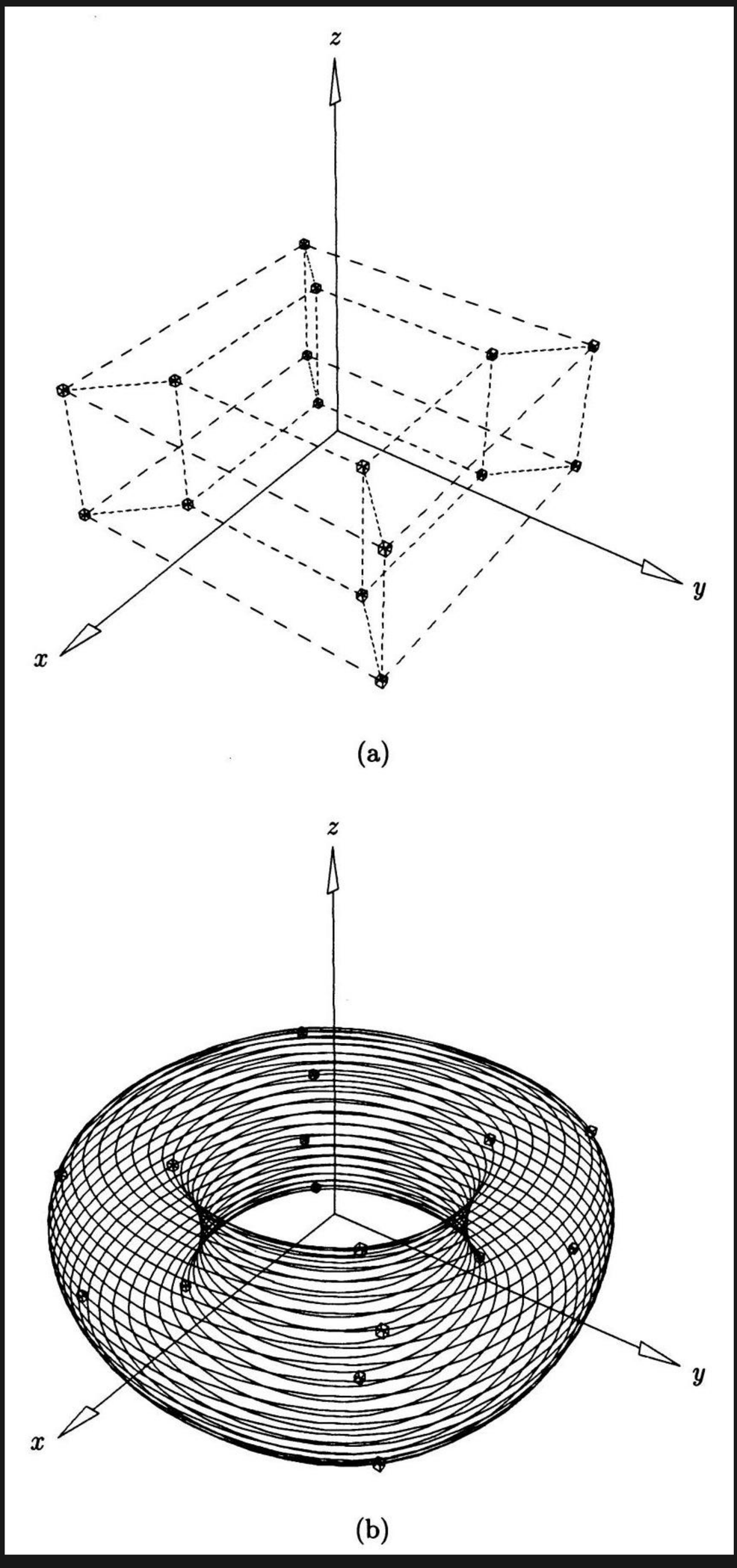


图 9.34. 使用局部双三次曲面插值的表面插值示例。(a) 数据点; (b) 插值表面。

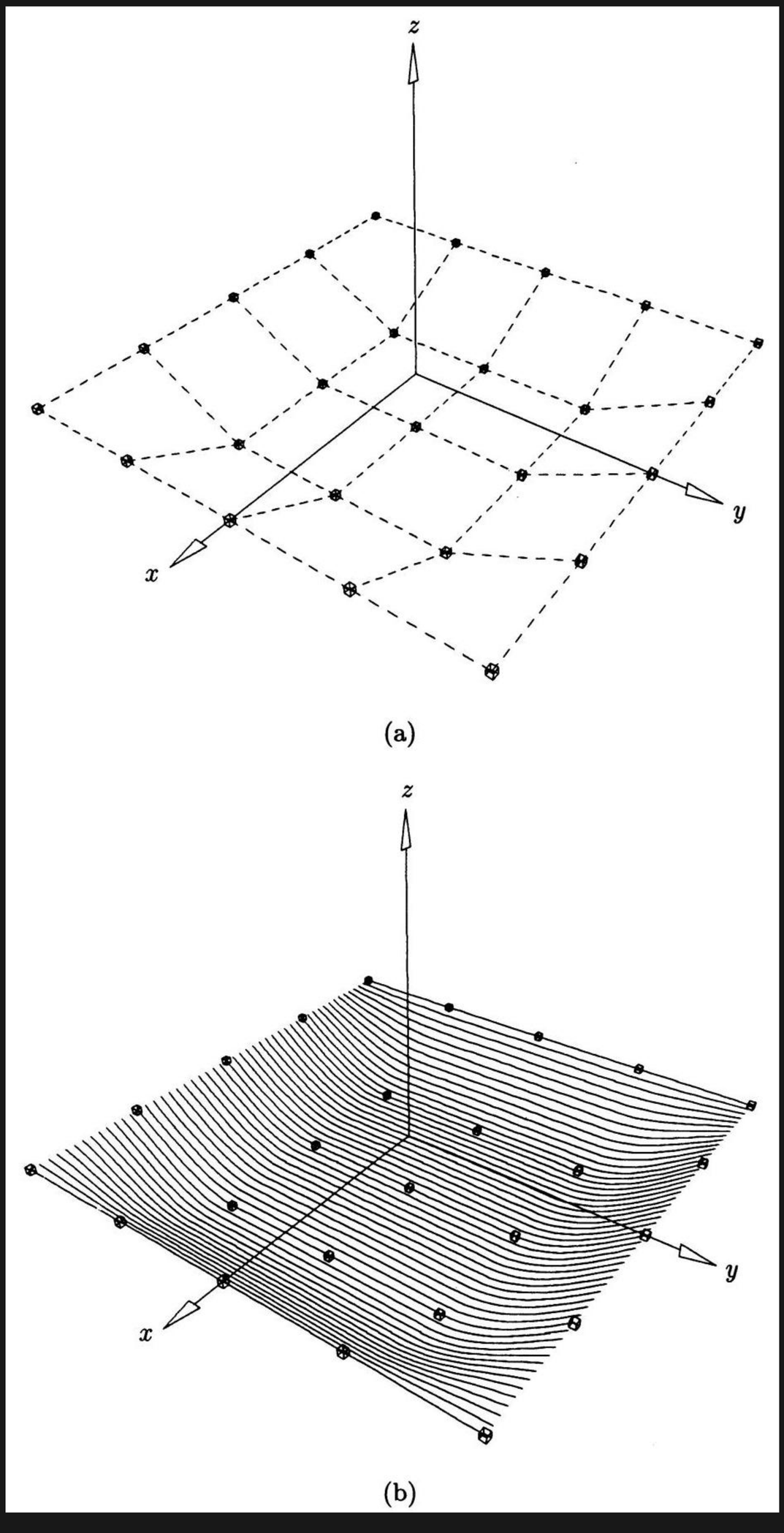
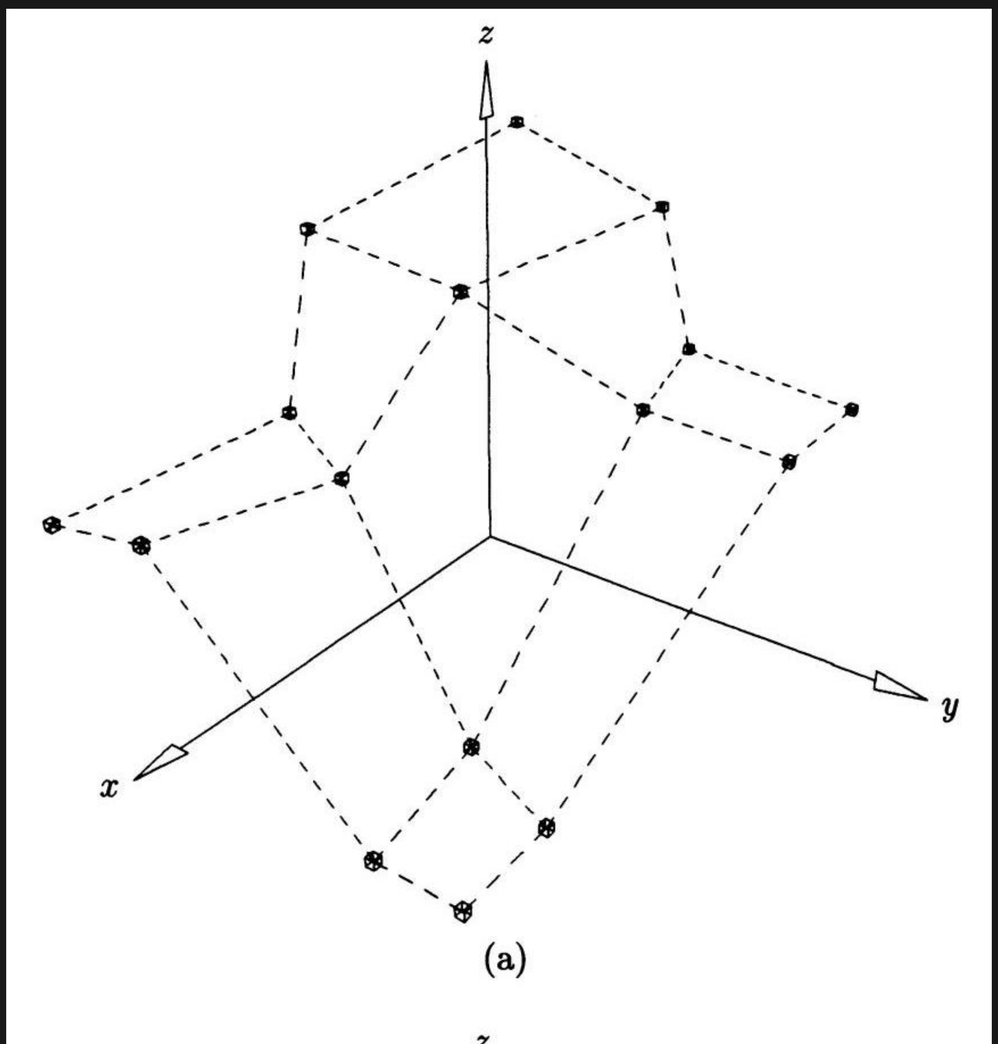
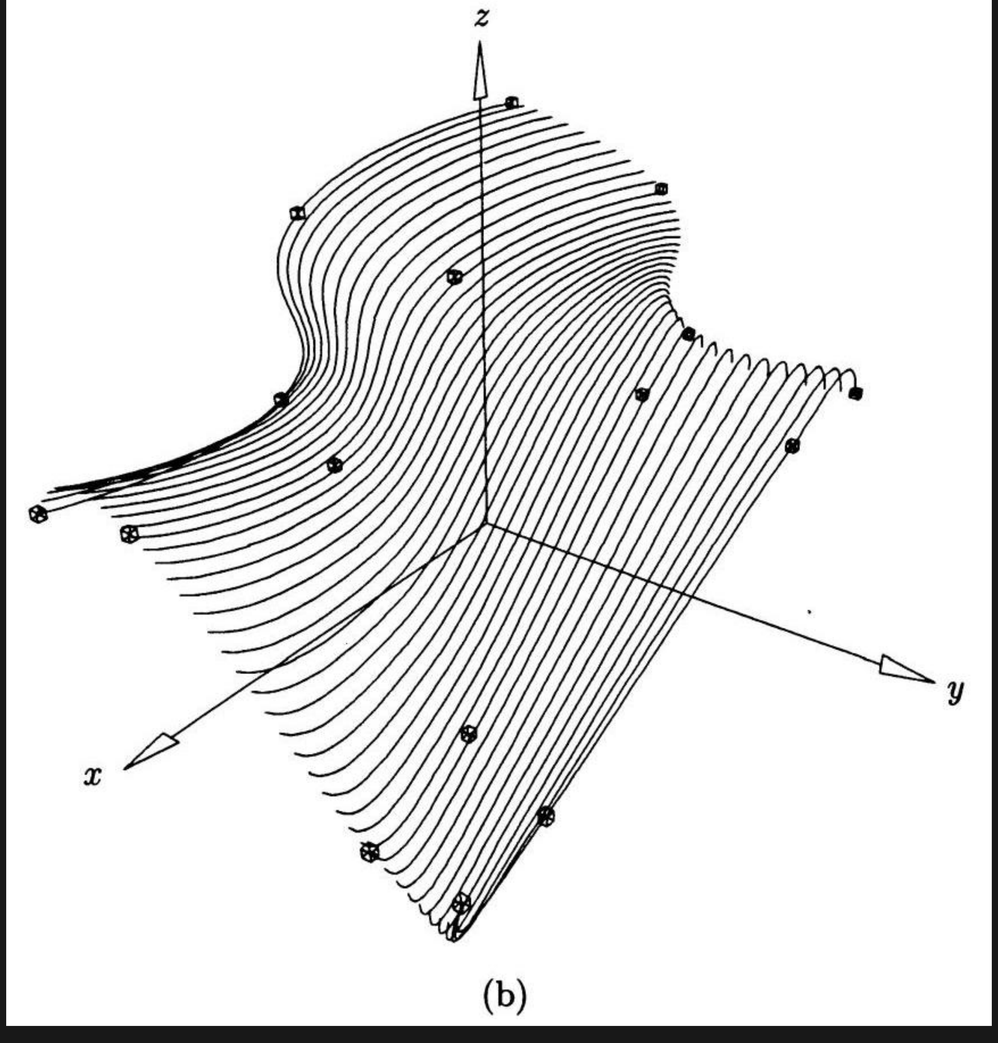


图 9.35. 使用局部双三次曲面插值的表面插值示例。(a) 数据点; (b) 插值表面。



(a)



(b)

图 9.36. 使用局部双三次曲面插值的表面插值示例。(a) 数据点; (b) 插值表面。

c. 检查曲线/表面与数据的偏差；

d. 如果偏差处处小于 E ，则返回；否则增加控制点的数量并返回到步骤 1(b)。

2a. 从最大或“多个”控制点开始，足够使得第一次近似满足 E ；

b. 使用全局方法将曲线/表面拟合到数据上；

c. 检查偏差 E 是否在所有地方都满足；

d. 如果 E 不再满足，返回之前的曲线或表面，否则减少控制点的数量并返回到步骤 2(b)。

我们在第 9.4.4 节中回到这两种方法，特别是我们给出第二种类型的示例。显然，近似的成本也高于插值。需要多次拟合，并且偏差检查可能会很昂贵。

这两种方法的一个核心要素是，给定固定数量的控制点，例如 n ，我们将一个近似曲线 (表面) 拟合到数据上。有很多方法可以做到这一点。例如，可以建立一个非线性优化问题，其中控制点、参数 $(\bar{u}_k)$ 、节点，甚至权重都是未知数。需要最小化的目标函数必须以某种方式衡量误差，例如，最小二乘法或最大偏差 (例如，见 [Laur93])。在第 9.4.1-9.4.3 节中，我们介绍了曲线和表面拟合方法，其中固定数量的控制点是唯一的未知数，并使用线性最小二乘技术进行求解。

4.1 最小二乘曲线逼近

为了避免非线性问题，我们将权重设置为 1，并预计算参数和节点的值。然后我们建立并求解未知控制点的 (唯一) 线性最小二乘问题。我们从对点数据的简单曲线拟合开始。假设 $p \geq 1, n \geq p$ 和 $\mathbf{Q}_0, \ldots, \mathbf{Q}_m \, (m > n)$ 已知。我们寻求一个 p 次非有理曲线

$$\mathbf{C}(u) = \sum_{i=0}^{n} N_{i,p}(u) \, \mathbf{P}_i \; u \in [0,1] \tag{9.61}$$

满足以下条件:

- $\mathbf{Q}_0 = \mathbf{C}(0)$ 和 $\mathbf{Q}_m = \mathbf{C}(1)$ ；

- 剩余的 $\mathbf{Q}_k$ 在最小二乘意义下被近似，即。

$$\sum_{k=1}^{m-1}\left|\mathbf{Q}_k-\mathbf{C}\left(\bar{u}_k\right)\right|^2 \tag{9.62}$$

是相对于 $n+1$ 变量的最小值，$\mathbf{P}_i$；$\{\bar{u}_k\}$ 是预计算的参数值。

我们强调，结果曲线通常并不精确地通过 $\mathbf{Q}_k$，而且 $\mathbf{C}\left(\bar{u}_k\right)$ 不是 $\mathbf{C}\left(u\right)$ 上离 $\mathbf{Q}_k$ 最近的点。设

$$\mathbf{R}_k=\mathbf{Q}_k-N_{0,p}\left(\bar{u}_k\right)\mathbf{Q}_0-N_{n,p}\left(\bar{u}_k\right)\mathbf{Q}_m\ k=1,\ldots,m-1 \tag{9.63}$$

然后设定

$$\begin{aligned}f&=\sum_{k=1}^{m-1}\left|\mathbf{Q}_k-\mathbf{C}\left(\bar{u}_k\right)\right|^2=\sum_{k=1}^{m-1}\left|\mathbf{R}_k-\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right|^2\\&=\sum_{k=1}^{m-1}\left(\mathbf{R}_k-\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right)\cdot\left(\mathbf{R}_k-\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right)\\&=\sum_{k=1}^{m-1}\left[\mathbf{R}_k\cdot\mathbf{R}_k-2\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\left(\mathbf{R}_k\cdot\mathbf{P}_i\right)\right.\\&\left.+\left(\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right)\cdot\left(\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right)\right]\end{aligned}$$

f 是 $n-1$ 变量的标量值函数，$\mathbf{P}_1,\ldots,\mathbf{P}_{n-1}$。现在我们应用线性最小二乘拟合的标准技术 [DeBo78: Vand83; Lanc86]；为了最小化 f，我们将 f 关于 $n-1$ 点的导数，$\mathbf{P}_\ell$，设为零。ℓ 阶导数是

$$\frac{\partial f}{\partial\mathbf{P}_\ell}=\sum_{k=1}^{m-1}\left(-2N_{\ell,p}\left(\bar{u}_k\right)\mathbf{R}_k+2N_{\ell,p}\left(\bar{u}_k\right)\sum_{i=1}^{n-1}N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i\right)$$

这意味着

$$-\sum_{k=1}^{m-1}N_{\ell,p}\left(\bar{u}_k\right)\mathbf{R}_k+\sum_{k=1}^{m-1}\sum_{i=1}^{n-1}N_{\ell,p}\left(\bar{u}_k\right)N_{i,p}\left(\bar{u}_k\right)\mathbf{P}_i=0$$

由此可得

$$\sum_{i=1}^{n-1}\left(\sum_{k=1}^{m-1}N_{\ell,p}\left(\bar{u}_k\right)N_{i,p}\left(\bar{u}_k\right)\right)\mathbf{P}_i=\sum_{k=1}^{m-1}N_{\ell,p}\left(\bar{u}_k\right)\mathbf{R}_k \tag{9.64}$$

方程 (9.64) 是一个关于未知数 $\mathbf{P}_1,\ldots,\mathbf{P}_{n-1}$ 的线性方程。令 $\ell=1,\ldots,n-1$ 得到 $n-1$ 个方程的系统，其中有 $n-1$ 个未知数

$$\left(N^TN\right)\mathbf{P}=\mathbf{R} \tag{9.65}$$

其中 N 是标量的 $(m-1)\times(n-1)$ 矩阵

$$N=\begin{bmatrix}N_{1,p}\left(\bar{u}_1\right)&\cdots&N_{n-1,p}\left(\bar{u}_1\right)\\\vdots&\ddots&\vdots\\N_{1,p}\left(\bar{u}_{m-1}\right)&\cdots&N_{n-1,p}\left(\bar{u}_{m-1}\right)\end{bmatrix} \tag{9.66}$$

$\mathbf{R}$ 是 $n-1$ 点的向量

$$\mathbf{R}=\begin{bmatrix}N_{1,p}\left(\bar{u}_1\right)\mathbf{R}_1+\cdots+N_{1,p}\left(\bar{u}_{m-1}\right)\mathbf{R}_{m-1}\\\vdots\\N_{n-1,p}\left(\bar{u}_1\right)\mathbf{R}_1+\cdots+N_{n-1,p}\left(\bar{u}_{m-1}\right)\mathbf{R}_{m-1}\end{bmatrix} \tag{9.67}$$

和

$$\mathbf{P}=\begin{bmatrix}\mathbf{P}_1\\\vdots\\\mathbf{P}_{n-1}\end{bmatrix}$$

请注意，方程 (9.65) 是一个系数矩阵，具有三个右侧和三组未知数 (x，y，z 坐标)。为了建立方程 (9.66) 和 (9.67)，需要一个节点向量 $U=\{u_0,\ldots,u_r\}$ 和参数 $\{\bar{u}_k\}$。可以使用方程 (9.5) 计算 $\{\bar{u}_k\}$。节点的放置应反映 $\{\bar{u}_k\}$ 的分布。如果 d 是一个正实数，则用 $i=\text{int}\left(d\right)$ 表示最大的整数，使得 $i\le d$。我们需要总共 $n+p+2$ 个节点；其中有 $n-p$ 个内部节点，以及 $n-p+1$ 个内部节点跨度。让

$$d=\frac{m+1}{n-p+1} \tag{9.68}$$

然后通过以下方式定义内部节点

$$i=\text{int}\left(jd\right)\ \alpha=jd-i$$

$$u_{p+j}=\left(1-\alpha\right)\bar{u}_{i-1}+\alpha\bar{u}_i\ j=1,\ldots,n-p \tag{9.69}$$

方程 (9.69) 保证每个节点区间至少包含一个 $\bar{u}_k$，在此条件下，deBoor [DeBo78] 表明方程 (9.65) 中的矩阵 $\left(N^TN\right)$ 是正定且良好条件的。它可以通过高斯消元法解决，无需主元选择。此外，$\left(N^TN\right)$ 的半带宽小于 $p+1$ (如果 $N_{i,j}$ 是第 i 行、第 j 列的元素，则 $N_{i,j}=0$ 如果 $|i-j|>p$)。最后，关于实现，注意矩阵 N (方程 [9.66]) 可能非常大，并且大多数元素为零；在某些应用中，$m=1000$ 和 $n=100$ 并不少见。因此，实现者可能希望开发一种存储方案，仅存储 N 的非零元素。一旦计算出 $\left(N^TN\right)$ 和 $\mathbf{R}$ (方程 [9.67])，可以释放 N。类似地，仅应存储 $\left(N^TN\right)$ 的非零带。

图 9.37a – 9.37e 显示了简单最小二乘曲线拟合的示例。在图 9.37a 中指定了七个控制点。图 9.37b 显示了九个控制点的拟合，而图 9.37c 描绘了十一个控制点的拟合。注意，从距离的角度来看，随着控制点数量的增加，近似效果有所改善。然而，当控制点的数量接近数据点的数量时，如果数据存在噪声或不必要的波动，可能会出现不理想的形状。

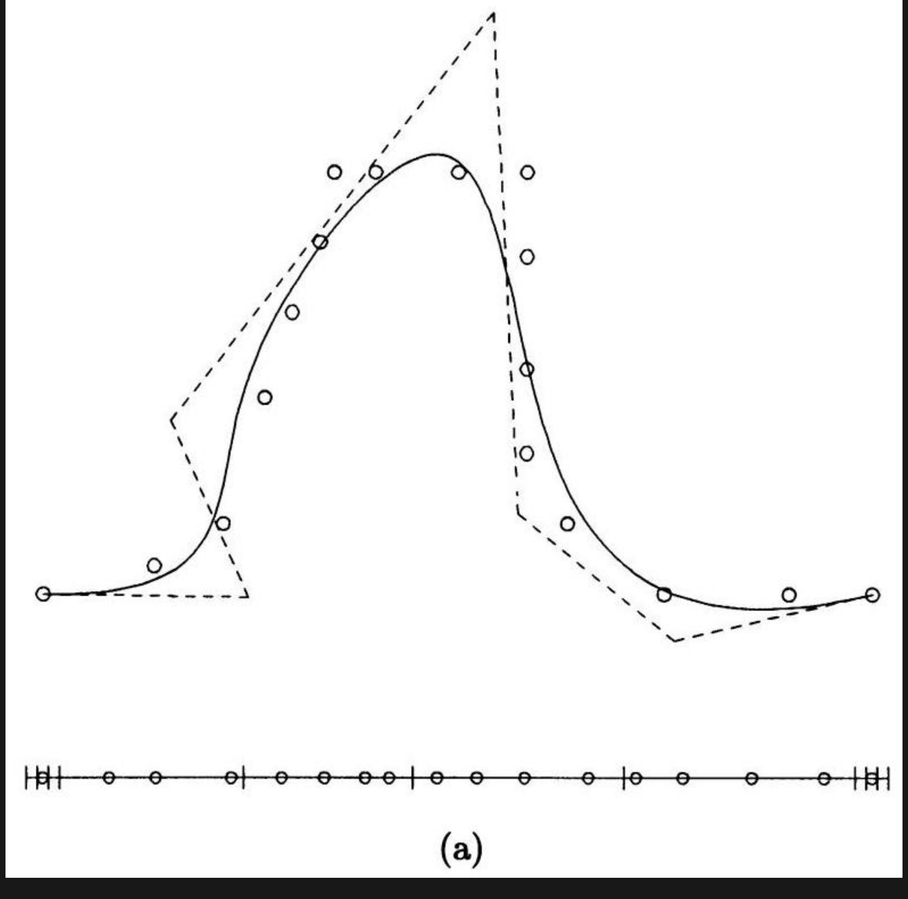


图 9.37. 全局曲线拟合。(a) 七个控制点；(b) 九个控制点；(c) 十一个控制点。

4.2 加权和约束最小二乘曲线拟合

现在我们提出一种非常通用的加权约束最小二乘曲线拟合方法。这是对 Smith 等人算法的 NURBS 适应 [Smit74]。设 $\{\mathbf{Q}_i\},i=0,\ldots,r$ 为要近似的点。可选地，可以指定任意 $\mathbf{Q}_i$ 处的第一导数 $\mathbf{D}_i$。设 $\left\{\mathbf{D}_{i(j)}\right\},j=0,\ldots,s$ 为导数集合；$-1\le s\le r$，其中 $s=-1$ 表示未指定导数。此外，任何 $\mathbf{Q}_i$ 或 $\mathbf{D}_{i(j)}$ 都可以被约束 (精确插值)。我们将 $\{\mathbf{Q}_i\}$ 和 $\left\{\mathbf{D}_{i(j)}\right\}$ 分为无约束和有约束集合。用 $\mathbf{Q}^u_{i(0)},\ldots,\mathbf{Q}^u_{i(r_u)}$ 和 $\mathbf{D}^u_{i(0)},\ldots,\mathbf{D}^u_{i(s_u)}$ 表示无约束项，用 $\mathbf{Q}^c_{i(0)},\ldots,\mathbf{Q}^c_{i(r_c)}$ 和

$\mathbf{D}^c_{i(0)},\ldots,\mathbf{D}^c_{i(s_c)}$ 表示有约束项 (s_u, s_c ，或 r_c 等于-1 表示没有与此索引对应的数据。注意 $r = r_u + r_c + 1$ 和 $s = s_u + s_c + 1$ 。我们还允许为每个无约束项分配一个正权重，因此我们有 $w^q_{i(0)},\ldots,w^q_{i(r_u)}$ 和 $w^d_{i(0)},\ldots,w^d_{i(s_u)}$ $\left(w^q_i, w^d_i > 0\right)$ 。这些权重允许对每个数据项相对于其邻居的“紧密度”施加额外影响。$w^q_i, w^d_i = 1$ 是默认值。增加权重会增加对该项的近似紧密度，而减少权重则会放松对该项的近似。请注意，这些权重与 NURBS 意义上的权重无关。

现在设 $m_u = r_u + s_u + 1$ 和 $m_c = r_c + s_c + 1$ 。我们希望在最小二乘意义上近似无约束数据，并插值有约束数据

$$m_c < n\ m_c + n < m_u + 1 \tag{9.70}$$

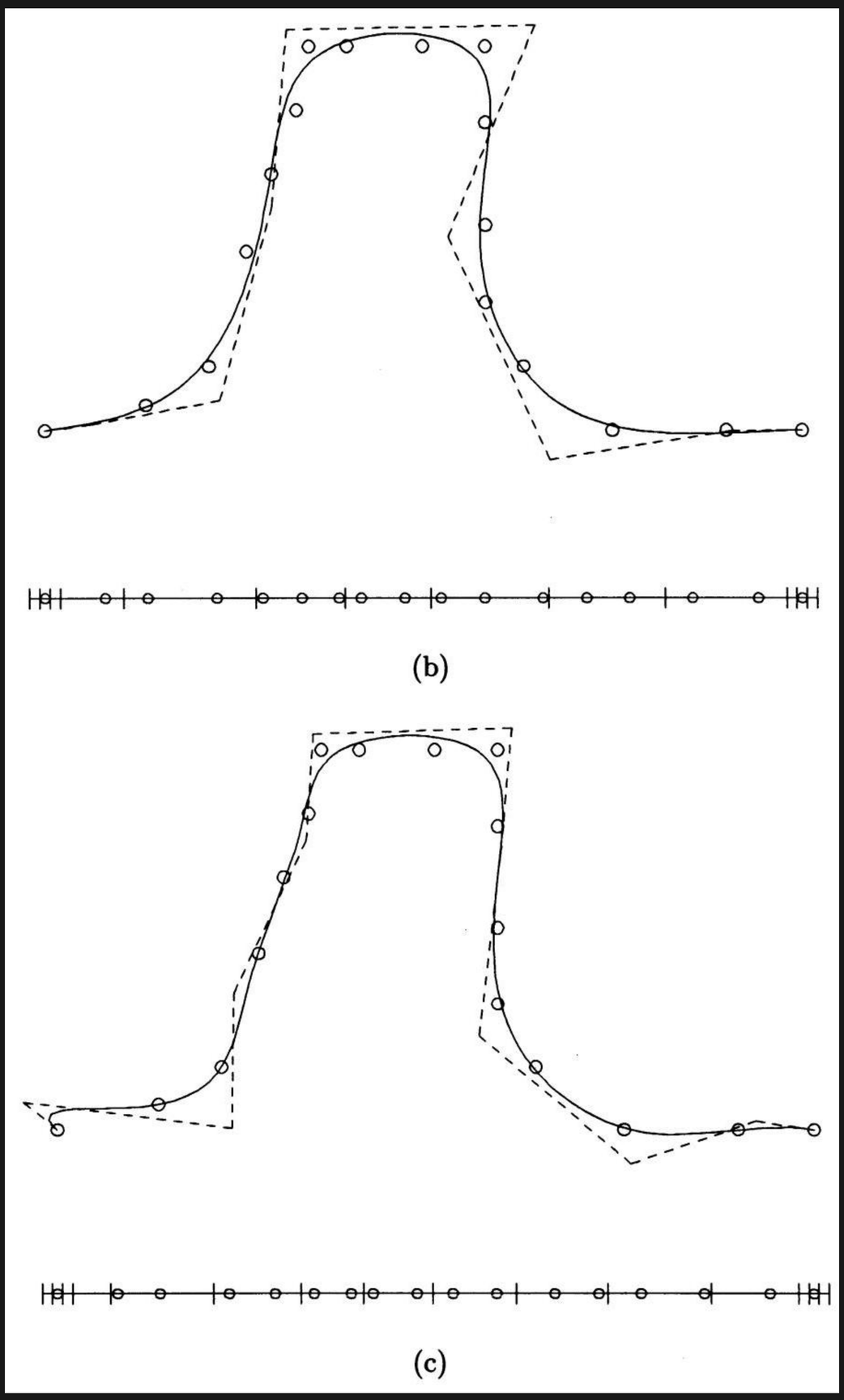


图 9.37。(续。)

现在设:

$\mathbf{S}_k, k = 0,\ldots,m_u$ ，是第 k 个不受约束的数据项 (一个 $\mathbf{Q}^u$ 或一个 $\mathbf{D}^u$);

$\mathbf{T}_k, k = 0,\ldots,m_c$ ，是第 k 个受约束的数据项 (一个 $\mathbf{Q}^c$ 或一个 $\mathbf{D}^c$);

$w_k, k = 0,\ldots,m_u$ ，是第 k 个权重 (一个 w^q 或一个 w^d).

定义向量/矩阵:

$\mathbf{S} = [\mathbf{S}_k]$ (一个包含 $m_u + 1$ 个元素的向量);

$\mathbf{T} = [\mathbf{T}_k]$ (一个包含 $m_c + 1$ 个元素的向量);

$W = [w_k]$ (一个 $(m_u + 1) \times (m_u + 1)$ 对角矩阵，主对角线上为 w_k);

$\mathbf{P} = [\mathbf{P}_k]$ (包含 $n+1$ 个未知控制点的向量);

$N = [ND_{i,p}(\bar{u}_k)]$ ，其中 $ND_{i,p}(\bar{u}_k)$ 是第 i 个基函数或其一阶导数，在 $\bar{u}_k$ 处求值，这是一个与不受约束数据项对应的参数值；这是一个 $(m_u + 1) \times (n + 1)$ 标量矩阵;

$M = [MD_{i,p}(\bar{u}_k)]$ ，其中 $MD_{i,p}(\bar{u}_k)$ 是第 i 个基函数或其一阶导数，在 $\bar{u}_k$ 处求值，这是一个与受约束数据项对应的参数值；这是一个 $(m_c + 1) \times (n + 1)$ 标量矩阵.

$\bar{u}_k$ 可以使用公式 (9.5) 计算，节点 u_j 根据公式 (9.68) 和公式 (9.69) 计算 (在公式 [9.68] 中将 m 替换为 r).

确定曲线基本上是一个约束最小化问题，涉及 $n+1$ 个未知数 ($\mathbf{P}_i$) 和 $m_c + 1$ 个约束。标准的解决方法是使用拉格朗日乘数 (见 [Kapl52])。其思想是引入 $m_c + 1$ 个额外的未知数 λ_k (拉格朗日乘数)，从而获得一个分区 $(n + m_c + 2) \times (n + m_c + 2)$ 方程组。从这个方程组中，首先为 $m_c + 1\lambda_k$ 得到一个解，随后为 $n + 1\mathbf{P}_i$ 得到解。设

$$\mathbf{A} = [\lambda_k]$$

为拉格朗日乘数的向量；注意每个 λ_k 是一个与 $\{\mathbf{Q}_i\}$ 和 $\{\mathbf{D}_{i(j)}\}$ 具有相同维度的向量。

无约束方程为

$$N\mathbf{P} = \mathbf{S}$$

而约束方程为

$$M\mathbf{P} = \mathbf{T}$$

无约束方程中的误差为 $\mathbf{S} - N\mathbf{P}$ ，我们希望在 $M\mathbf{P} = \mathbf{T}$ 的约束下最小化该误差的加权平方和。因此，使用拉格朗日乘数，我们希望最小化表达式

$$\left(\mathbf{S}^T - \mathbf{P}^T N^T\right) W (\mathbf{S} - N\mathbf{P}) + \mathbf{A}^T (M\mathbf{P} - \mathbf{T})$$

关于未知数 $\mathbf{A}$ 和 $\mathbf{P}$ 。对其进行微分并设其等于零，得到

$$-2\left(\mathbf{S}^T W N - \mathbf{P}^T N^T W N\right) + \mathbf{A}^T M = 0$$

$$M\mathbf{P} - \mathbf{T} = 0$$

转置并从 $\mathbf{A}$ 中提取 2，得到

$$N^T W N \mathbf{P} + M^T \mathbf{A} = N^T W \mathbf{S} \tag{9.71}$$

$$M\mathbf{P} = \mathbf{T} \tag{9.72}$$

或以分区矩阵形式表示

$$\begin{bmatrix} N^T W N & M^T \\ M & 0 \end{bmatrix} \begin{bmatrix} \mathbf{P} \\ \mathbf{A} \end{bmatrix} = \begin{bmatrix} N^T W \mathbf{S} \\ \mathbf{T} \end{bmatrix} \tag{9.73}$$

我们必须解方程 (9.73) 以求得 $\mathbf{A}$ 和 $\mathbf{P}$ 。如果 $N^T W N$ 和 $M\left(N^T W N\right)^{-1} M^T$ 都是可逆的，则存在唯一解。这可以通过解方程 (9.71) 来看出，得到 $\mathbf{P}$ ，即

$$\mathbf{P} = \left(N^T W N\right)^{-1} N^T W \mathbf{S} - \left(N^T W N\right)^{-1} M^T \mathbf{A} \tag{9.74}$$

乘以 M 得到

$$M\mathbf{P} = M\left(N^T W N\right)^{-1} N^T W \mathbf{S} - M\left(N^T W N\right)^{-1} M^T \mathbf{A}$$

最后，从方程 (9.72) 中我们发现

$$M\left(N^T W N\right)^{-1} M^T \mathbf{A} = M\left(N^T W N\right)^{-1} N^T W \mathbf{S} - \mathbf{T} \tag{9.75}$$

或

$$\mathbf{A} = \left(M\left(N^T W N\right)^{-1} M^T\right)^{-1} \left(M\left(N^T W N\right)^{-1} N^T W \mathbf{S} - \mathbf{T}\right) \tag{9.76}$$

因此，我们首先求解 $\mathbf{A}$ ，然后通过将 $\mathbf{A}$ 代入方程 (9.74) 来求解 $\mathbf{P}$ 。

算法 A9.6 实现了加权和约束最小二乘曲线拟合。输入为:

Q [r + 1] : 一个包含待拟合点的数组 (包括约束和非约束点)；

r: Q [] 的上标；

Wq [r + 1] : Wq [i] > 0 表示 $\mathbf{Q}_i$ 是非约束的，而 Wq [i] 是权重 w_i^q; Wq [i] < 0 表示 $\mathbf{Q}_i$ 是要约束的；

D [s + 1] : 一个包含导数的数组；s $= -1$ 表示未指定导数；

s: 在 D [] 中有 $s + 1$ 个导数；

I [s+1]: 一个整数数组；I [j] 给出与 D [j] 中导数对应的点在 Q[] 中的索引 (即 D [j] $= D_{i(j)}$)；

Wd [$s + 1$] : Wd [j] > 0 表示 D [j] 是非约束的，而 Wd [j] 是权重 $w_{i(j)}^d$; Wd[j] < 0 表示 D [j] 是要约束的；

n: 一个具有 $n + 1$ 个控制点的拟合；

p: 与度数 p 曲线的拟合。

输出节点 U 和控制点 P 。该算法使用 LU 分解 ( ) 和前向后向 ( ) 来反转矩阵 $\left(N^T W N\right)$ 并求解拉格朗日乘子 A。由于算法构造 N 的方式，$\left(N^T W N\right)$ 的半带宽小于 $p + 1$ 。一般来说，$\mathbf{A}$ 可以具有全带宽，但通常是一个小矩阵。使用第 2 章中的适当算法来确定节点跨度并计算基函数及其导数。以下局部数组是必需的:

ub [$r + 1$] : 参数；

N[mu+1][n+1], M[mc+1][n+1], S[mu+1], T[mc+1], A[mc+1]: 与之前定义的矩阵对应的数组 (除了 S[]，它是 $W\mathbf{S}$)；

W [mu + 1] : 权重；由于矩阵 W 是对角的，仅需要长度为 $m_u + 1$ 的向量；

funs [2] [p+1]: 包含基函数及其指定的导数 (见算法 A2.2 和 A2.3)。

此外，还需要几个局部数组用于临时存储在方程 (9.71)-(9.76) 中计算的矩阵。

ALGORITHM A9.6

```
WCLeastSquaresCurve(Q, r, Wq, D, s, I, Wd, n, p, U, P)
    { /* Weighted & constrained least squares curve fit */
        /* Input: Q, r, Wq, D, s, I, Wd, n, p */
        /* Output: U, P */
    ru = -1; rc = -1;
    for (i=0; i<=r; i++)
        if (Wq[i] > 0.0) ru = ru+1 ;
            else  rc = rc + 1 ;
    su = -1; sc = -1;
    for (j=0; j<=s; j++)
        if (Wd[j] > 0.0) su = su+1 ;
            else sc = sc+1;
    mu = ru + su + 1;  mc = rc + sc + 1 ;
    if (mc >= n || mc+n >= mu+1) return(error);
    Compute and load parameters ū_k into ub[] (Eq.[9.5]);
    Compute and load the knots into U[] (Eqs.[9.68],[9.69]);
                    /* Now set up arrays N, W, S, T, M* /
    j = 0; /* current index into I[] */
    mu2 = 0; mc2 = 0; /* counters up to mu and mc */
    for (i = 0; i<=r; i++)
        {
        span = FindSpan(n, p, ub[i], U);
        dflag = 0 ;
        if (j <= s)
            if (i == I[j]) dflag = 1 ;
    if ( dflag  == 0)  BasisFuns ( span , ub[i] , p, U, funs ) ;
      else DersBasisFuns(span, ub[i], p,1, U, funs);
    if (Wq[i] > 0.0 )
      { /* Unconstrained point */
      W[mu2] = Wq[i];
      Load the mu2th row of N[][] from funs[0][];
      S[mu2] = W[mu2]*Q[i];
      mu2 = mu2+1;
      }
      else
      { /* Constrained point */
      Load the mc2th row of M[][] from funs[0][];
      T[mc2] = Q[i];
      mc2 = mc2+1;
      }
    if (dflag == 1)
      { /* Derivative at this point */
      if (Wd[j] > 0.0 )
        { /* Unconstrained derivative */
        W[mu2] = Wd[j];
        Load the mu2th row of N[][] from funs[1][];
        S[mu2] = W[mu2] * D[j] ;
        mu2 = mu2+1;
        }
        else
        { /* Constrained derivative */
        Load the mc2th row of M[][] from funs[1][];
        T[mc2] = D[j];
        mc2 = mc2+1;
        }
      j = j+1;
      }
  } /* End of for-loop i=0,..., r */
Compute the matrices N^T W N and N^T W S ;
LUDecomposition (N^T W N, n + 1, p) ;
if (mc < 0 )
  { /* No constraints */
  Use ForwardBackward( ) to solve for the control points P[] .
        Eq. (9.71) reduces to (N^T W N) P = N^T W S .
```

return;
}
Compute the inverse $\left(N^TWN\right)^{-1}$, using ForwardBackward( ).
Do matrix operations to get: $M\left(N^TWN\right)^{-1}M^T$ and
$M\left(N^TWN\right)^{-1}\left(N^TW\mathbf{S}\right)-\mathbf{T}$;
Solve Eq.(9.75) for the Lagrange multipliers, load into A[];
然后 $\mathbf{P}=\left(N^TWN\right)^{-1}\left(\left(N^TW\mathbf{S}\right)-M^T\mathbf{A}\right)$ ，方程 (9.74)；
}

图 9.38-9.43 显示了加权和约束曲线拟合的几个示例。图 9.38 显示了一个无约束的拟合，所有权重均设置为 1。在图 9.39 中，权重 w_4^q, w_5^q, w_6^q 和 w_7^q 各增加到 100，从而导致曲线向数据点 $\mathbf{Q}_4,\ldots,\mathbf{Q}_7$ 拉动。请注意，尽管图形具有误导性，但曲线并未通过端点。图 9.40 显示了一个约束拟合，其中点 (用方块标记) 以及导数 (箭头向量) 作为约束。曲线通过方块标记的点，并在这些点上与指定的向量相切。在图 9.41 中引入了两个额外的权重 $w_3^q=100$ 和 $w_{15}^q=100$ 。图 9.42 显示了与图 9.40 相同的曲线，只是使用了两个新的内部导数。类似地，图 9.43 与图 9.42 相同，只是添加了权重 $w_4^q=50$ 和 $w_{16}^q=20$ 。

4.3 最小二乘曲面逼近

现在我们转向具有固定数量控制点的曲面逼近。设 $\{\mathbf{Q}_{k,\ell}\}, k=0,\ldots,r$ 和 $\ell=0,\ldots,s$ 为要被逼近的 $(r+1)\times(s+1)$ 点集，采用 a(p, q) 次非理性曲面，具有 $(n+1)\times(m+1)$ 个控制点。尽管可以建立并解决一般的最小二乘曲面拟合问题，无论是线性还是非线性，且有无权重和约束，但这项任务比曲线的情况复杂得多。我们在这里提出了一种曲面逼近方案，该方案基于我们的最小二乘曲线方案。

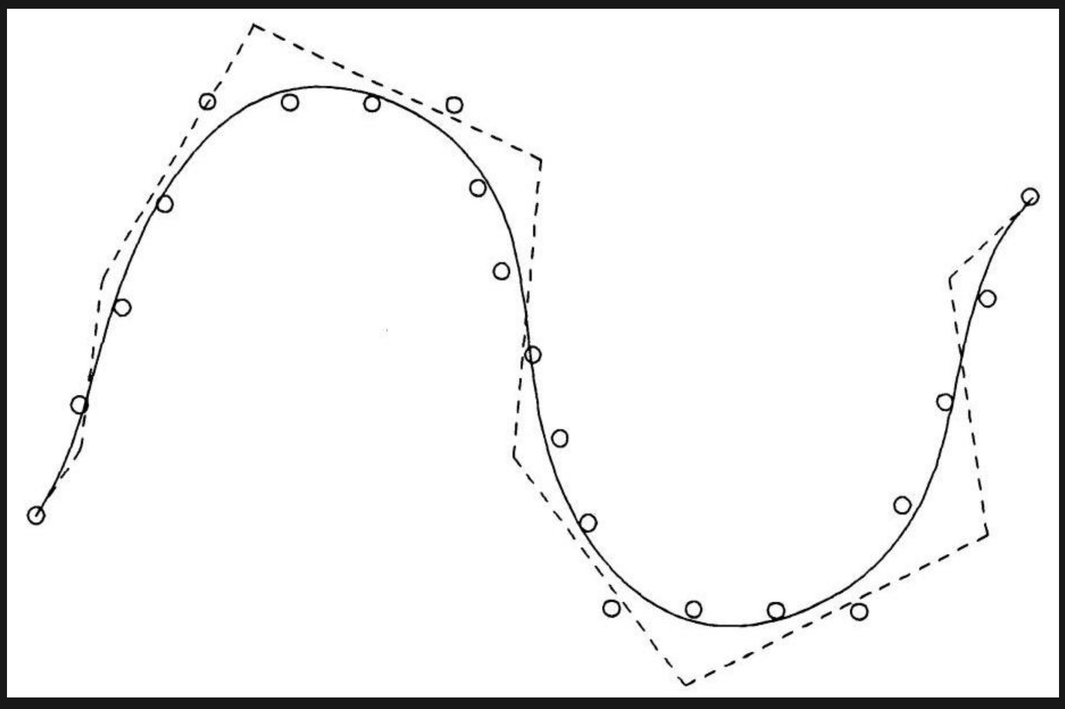


图 9.38。所有权重均等于一的无约束拟合；注意曲线并未通过端点。

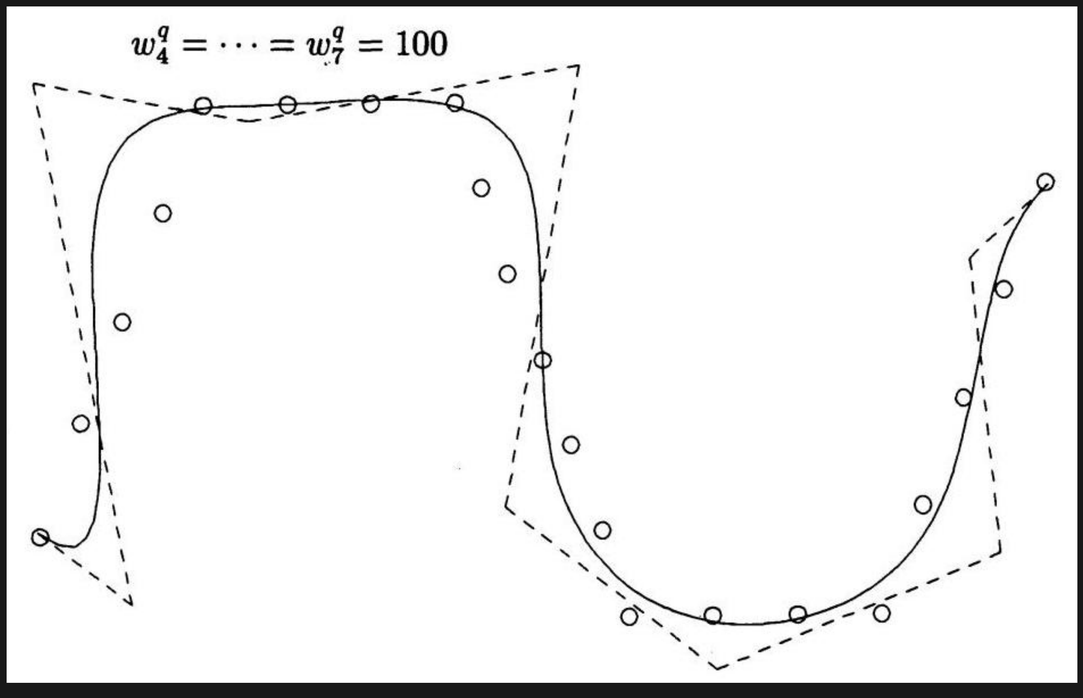


图 9.39. 不受约束的加权拟合 $w_4^q=w_5^q=w_6^q=w_7^q=100$ ；曲线不通过端点。

非常简单，并且对于大多数应用来说相当足够。与全局表面插值相同，我们只需在一个方向上拟合数据中的曲线，然后在另一个方向上通过得到的控制点拟合曲线。

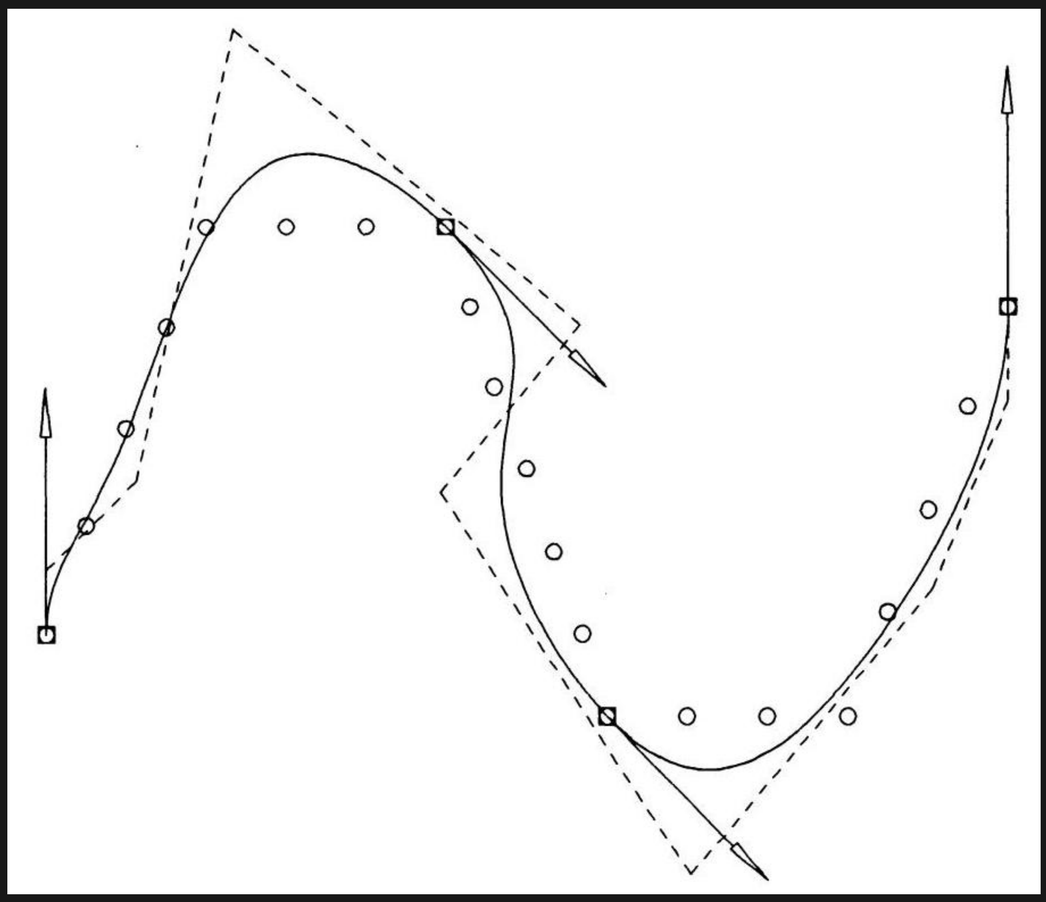


图 9.40. 受约束的曲线拟合；点约束用方框标记，切线约束用箭头表示，所有权重均为一。

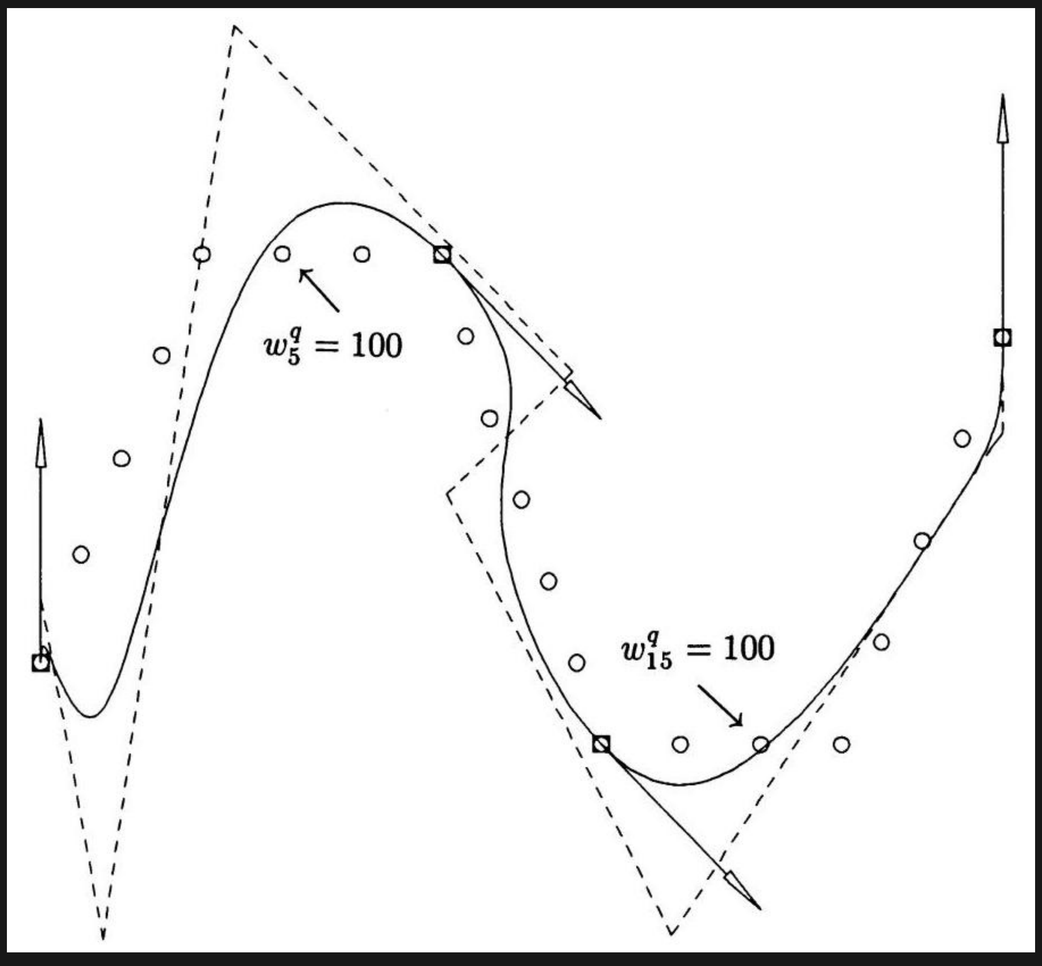


图 9.41. 来自图 9.40 的曲线，权重为 $w_5^q = w_{15}^q = 100$ 。

算法 A9.7 精确插值角点 $\mathbf{Q}_{0,0}$, $\mathbf{Q}_{r,0}$, $\mathbf{Q}_{0,s}$ 和 $\mathbf{Q}_{r,s}$ ，并近似剩余的 $\{\mathbf{Q}_{k,l}\}$ 。它使用由方程 (9.63) 和 (9.65)-(9.67) 给出的重复最小二乘曲线拟合。它拟合 $s+1$ 行数据。

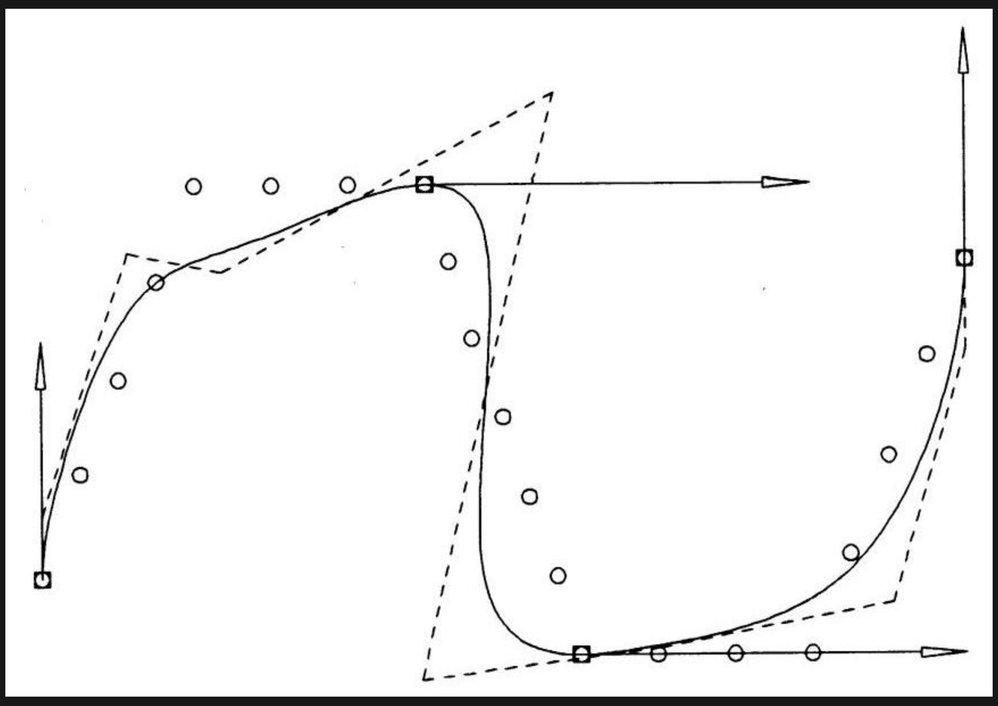


图 9.42. 来自图 9.40 的曲线，但使用两个新的切线约束。

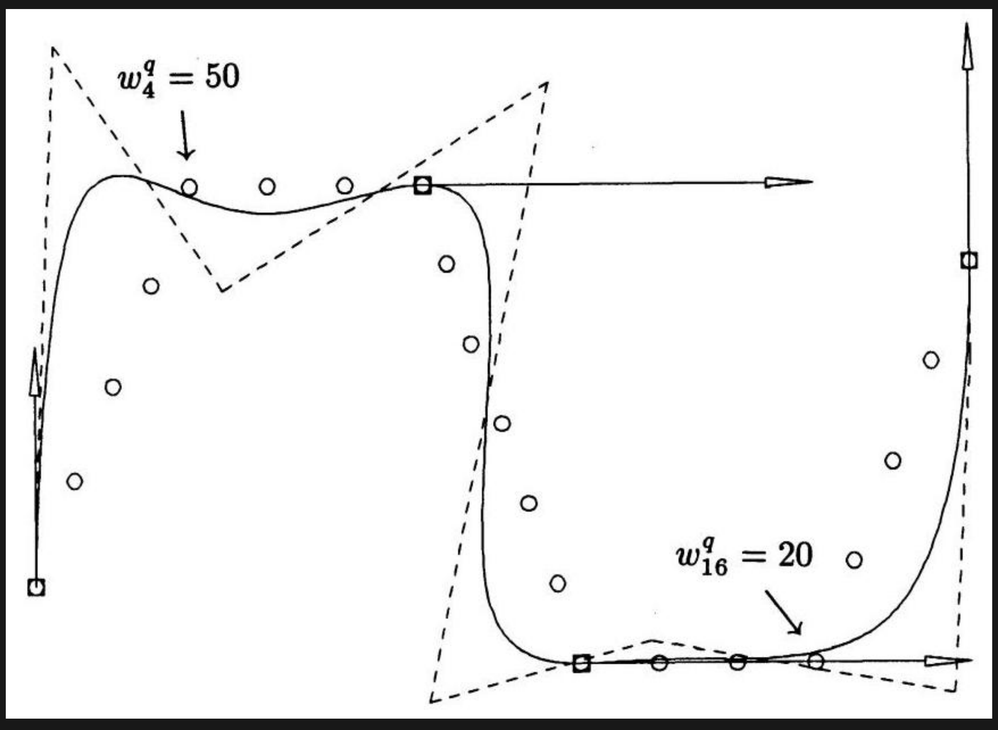


图 9.43. 来自图 9.42 的曲线，但权重为 $w_4^q = 50$ 和 $w_{16}^q = 20$ 。

首先，将得到的控制点存储在局部数组 Temp[n+1][s+1] 中。然后，它在这些控制点上拟合，以生成 $(n+1) \times (m+1)$ 表面控制点。注意，矩阵 N 和 $N^T N$ 只需在每个方向上计算一次，而 $N^T N$ 的 LU 分解也只需在每个方向上进行一次。我们需要局部数组 Nu[r-1][n-1] 和 NTNu[n-1][n-1] 用于 u 方向的拟合，以及 Nv [s − 1] [m − 1] 和 NTNv [m − 1] [m − 1] 用于 v 方向的拟合。方程 (9.68) 的向量 **R** 存储在 Ru[n-1] 和 Rv[m-1] 中。

```
ALGORITHM A9.7
  GlobalSurfApproxFixednm(r, s, Q, p, q, n, m, U, V, P)
      { /* Global surface approx with fixed num of ctrl pts */
        /* Input: r, s, Q, p, q, n, m */
        /* Output: U, V, P */
      SurfMeshParams(r, s, Q, ub, vb);
      Compute knots U by Eqs. (9.68), (9.69);
      Compute knots V by Eqs. (9.68), (9.69);
      Compute Nu[][] and NTNu[][] using Eq.(9.66);
      LUDecomposition(NTNu, n-1, p);
      for (j=0; j<=s; j++)
        { /* u direction fits */
        Temp [0] [j] = Q0,j; Temp [n] [j] = Qr,j;
        Compute and load Ru[] (Eqs.[9.63],[9.67]);
        Call ForwardBackward( ) to get the control points
              Temp[1][j],..., Temp[n-1][j];
        }
      Compute Nv[][] and NTNv[][] using Eq.(9.66);
      LUDecomposition(NTNv, m-1, q);
  for (i = 0 ; i<=n ; i++)
    { /* v direction fits */
    P [i] [0] = Temp [i] [0] ; P [i] [m] = Temp [i] [s] ;
    Compute and load Rv[] (Eqs.[9.63],[9.67]);
    Call ForwardBackward( ) to get the control points
          P [i] [1] , . . . , P [i] [m − 1] ;
    }
}
```

图 9.44-9.47 说明了表面拟合。图 9.44 显示了通过直线连接的原始数据。图 9.45a 描绘了使用 (5×6) 控制网的度数 (2,3) 最小二乘表面拟合。近似误差显示在图 9.45b 中。图 9.46a 显示了使用 (8×8) 网的近似。其近似质量要好得多，如图 9.46b 所示。为了比较，图 9.47 显示了对图 9.44 中数据的度数 (2,3) 全局插值 (使用弦长参数)。

当然，可以先拟合 $r+1$ 列，然后再拟合结果 $m+1$ 控制点的行。一般来说，结果并不相同。我们不知道有什么标准可以提前决定哪种方法会为特定数据集提供“最佳”拟合。显然，这种技术可以很容易地扩展以适应边界上的点和导数约束以及加权。内部约束则更为复杂。假设 $\mathbf{Q}_{k_0,\ell_0}, 0 < k_0 < r$ 和 $0 < \ell_0 < s$ 需要被约束。该点在 u 方向上受到约束。

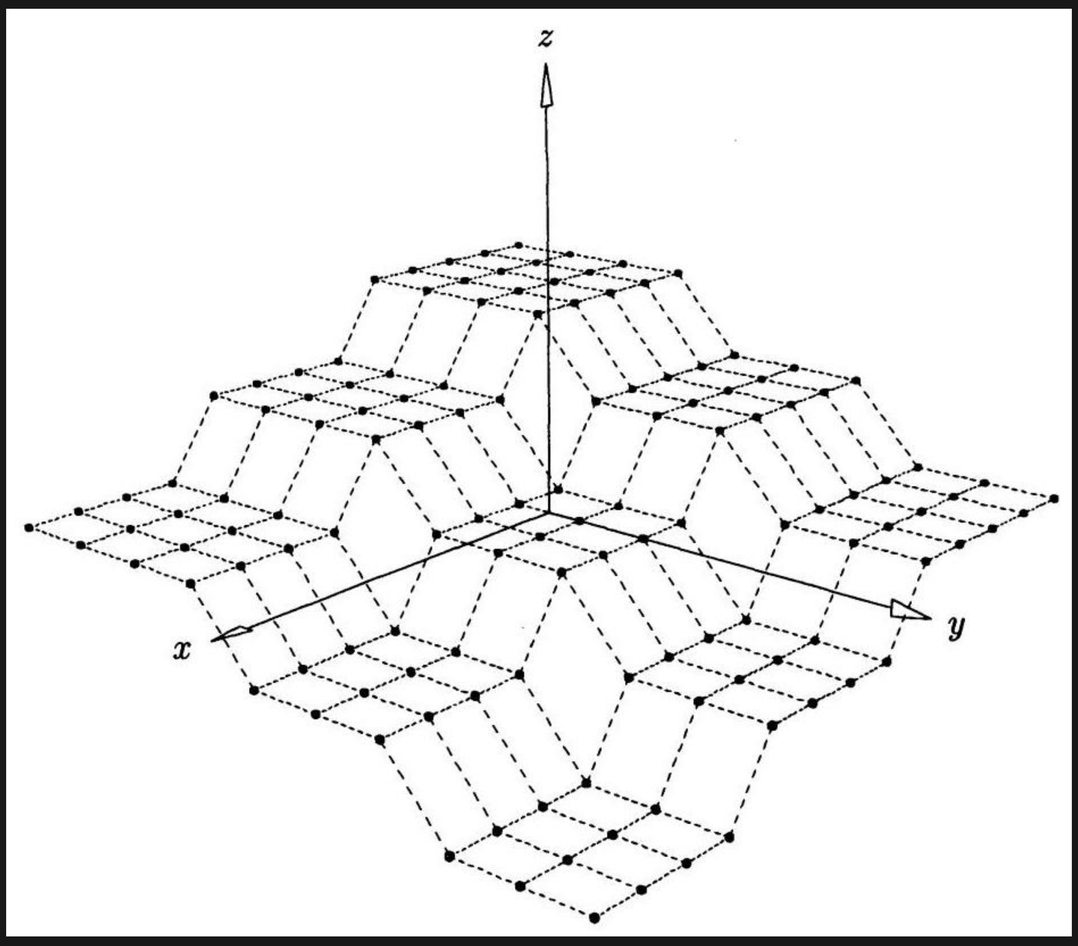


图 9.44. 表面近似的数据集。

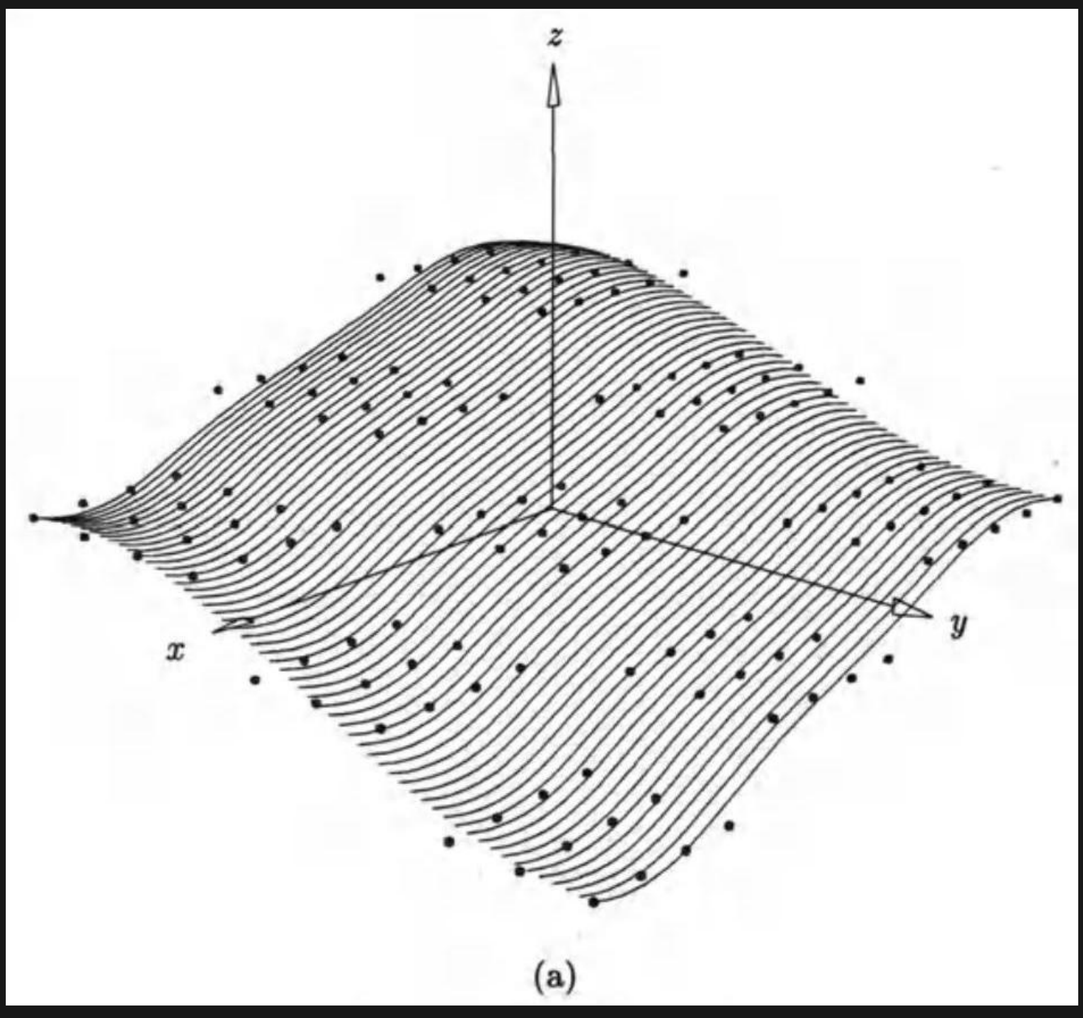


图 9.45. 使用 (5×6) 控制网的度数 (2,3) 表面的最小二乘拟合。(a) 表面近似；(b) 误差表面 (在数据点处测量的误差)。

曲线拟合，如算法 A9.6 所示。然后，如果 $\bar{u}_{k_0} \in [u_i, u_{i+1})$ ，则点 $\text{Temp}[i-p][\ell_0], \ldots, \text{Temp}[i][\ell_0]$ 必须在 v 方向上受到约束。

4.4 在指定精度内的逼近

我们现在回到将数据近似到某个用户指定的误差范围内的问题， E 。正如本节开始所述，迭代方法要么从少量控制点开始，拟合、检查偏差，并在必要时添加控制点 (类型 1)；要么从大量控制点开始，拟合、检查偏差，并在可能的情况下丢弃控制点 (类型 2)。前面提到的基于最小二乘法的方法适用于拟合步骤。偏差检查步骤通常测量最大距离，既可以表示为

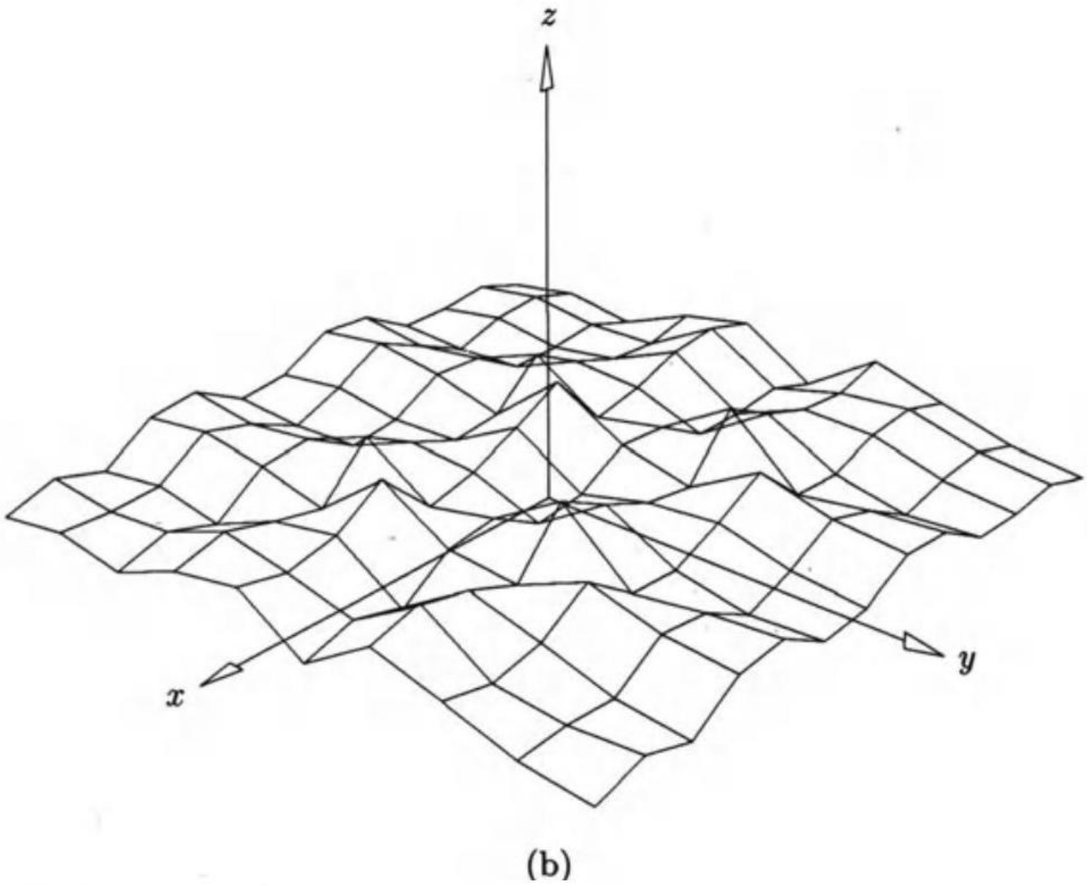
$$\max_{0 \le k \le m} |\mathbf{Q}_k - \mathbf{C}(\bar{u}_k)| \quad \text{or} \quad \max_{\substack{0 \le k \le r \\ 0 \le \ell \le s}} |\mathbf{Q}_{k,\ell} - \mathbf{S}(\bar{u}_k, \bar{v}_\ell)| \tag{9.77}$$

或表示为

$$\max_{0 \le k \le m} \left(\min_{0 \le u \le 1} |\mathbf{Q}_k - \mathbf{C}(u)| \right) \quad \text{or} \quad \max_{\substack{0 \le k \le r \\ 0 \le \ell \le s}} \left(\min_{\substack{0 \le u \le 1 \\ 0 \le v \le 1}} |\mathbf{Q}_{k,\ell} - \mathbf{S}(u,v)| \right) \tag{9.78}$$

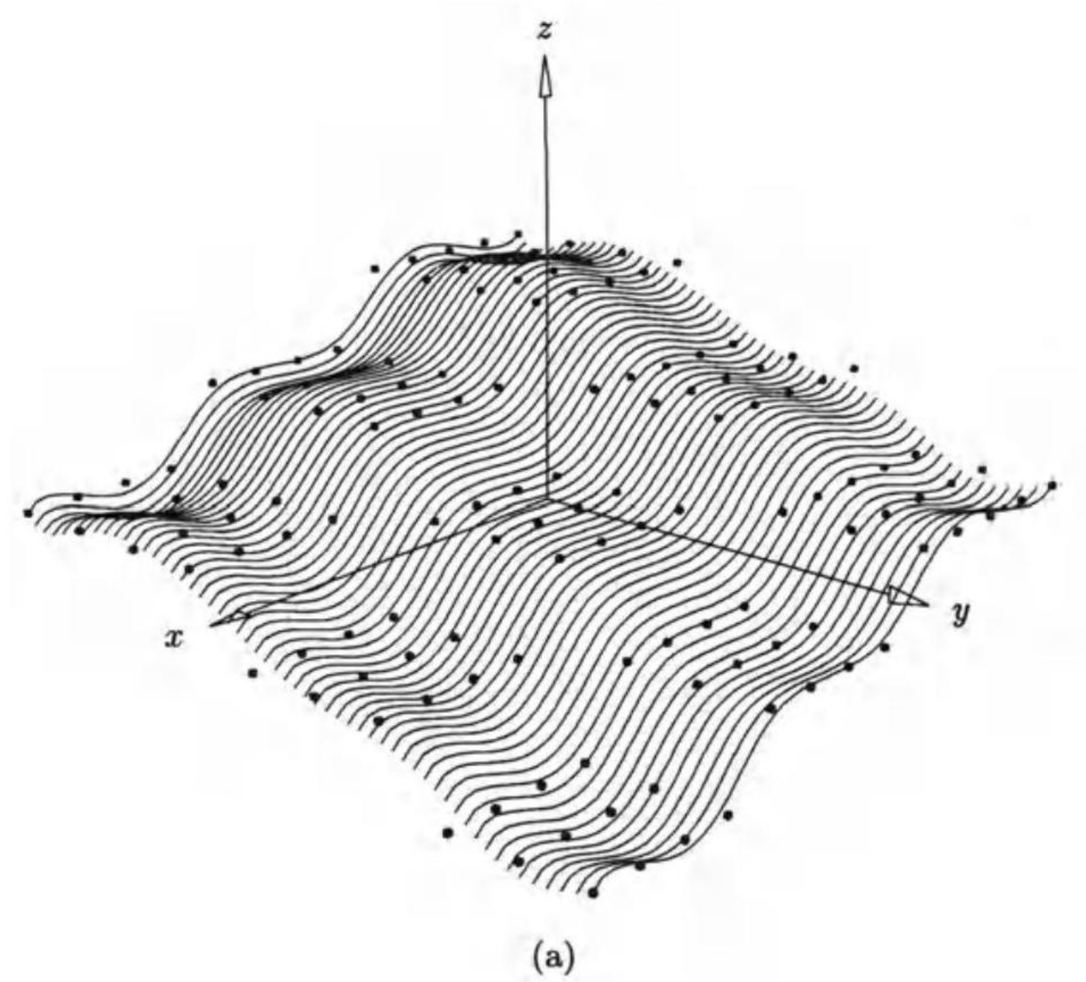
426 曲线和曲面拟合我们将方程 (9.78) 称为最大范数偏差。它使用方程 (6.4)-(6.9) 来计算每个 $\mathbf{Q}_k$ ($\mathbf{Q}_{k,\ell}$) 到 $\mathbf{C}(u)$ ($\mathbf{S}(u,v)$) 的最小距离。$\bar{u}_k$ ($\bar{u}_k, \bar{v}_\ell$) 作为牛顿迭代的良好起始点。显然，应用方程 (9.78) 的成本高于方程 (9.77)。然而，这通常是用户希望测量的内容，并且通常会导致具有更少控制点的曲线和曲面，因为

$$\min_{0 \le u \le 1} |\mathbf{Q}_k - \mathbf{C}(u)| \le |\mathbf{Q}_k - \mathbf{C}(\bar{u}_k)| \tag{9.79}$$



(b)

Figure 9.45. (*Continued.*)



(a)

图 9.46. 使用 (8×8) 控制网的二次 (2,3) 曲面拟合的最小二乘曲面。(a) 曲面近似；(b) 误差曲面 (在数据点处测量的误差)。

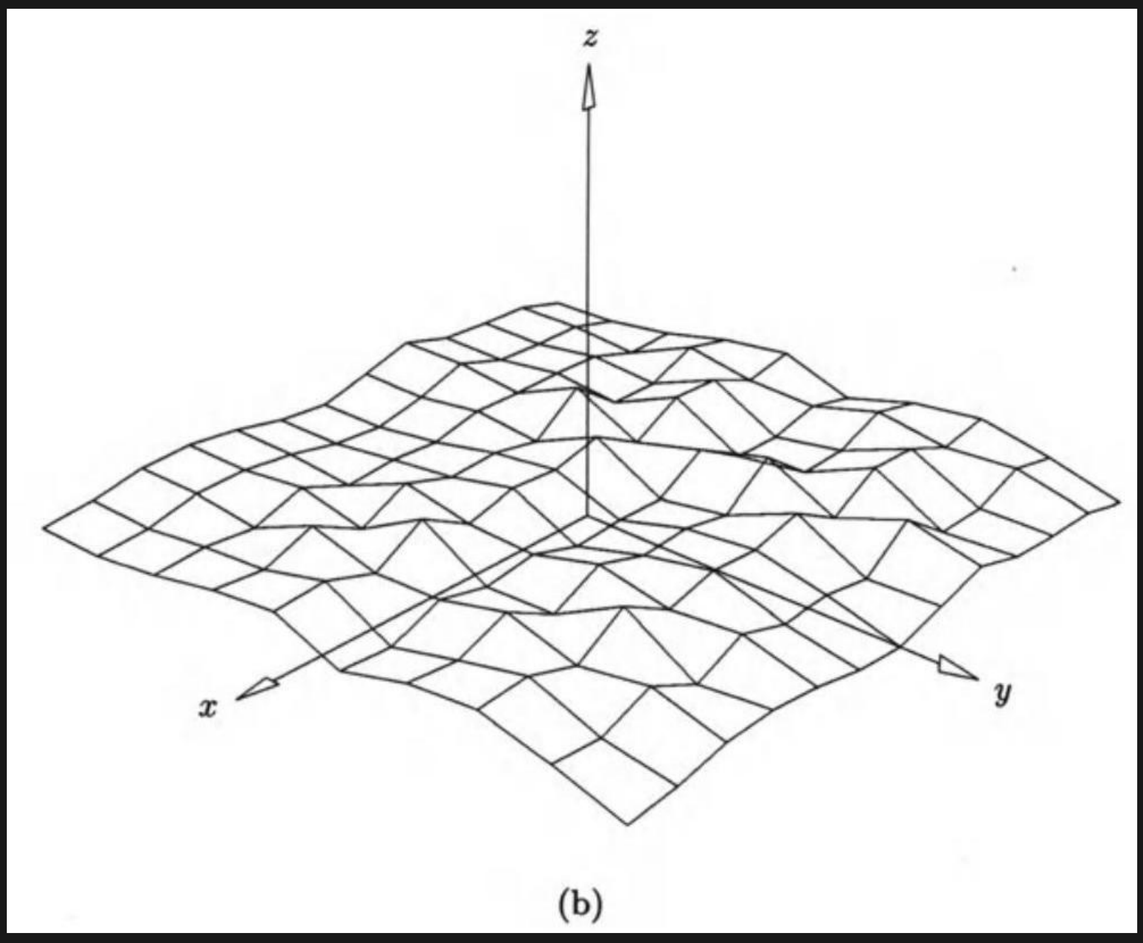


图 9.46. (续。)

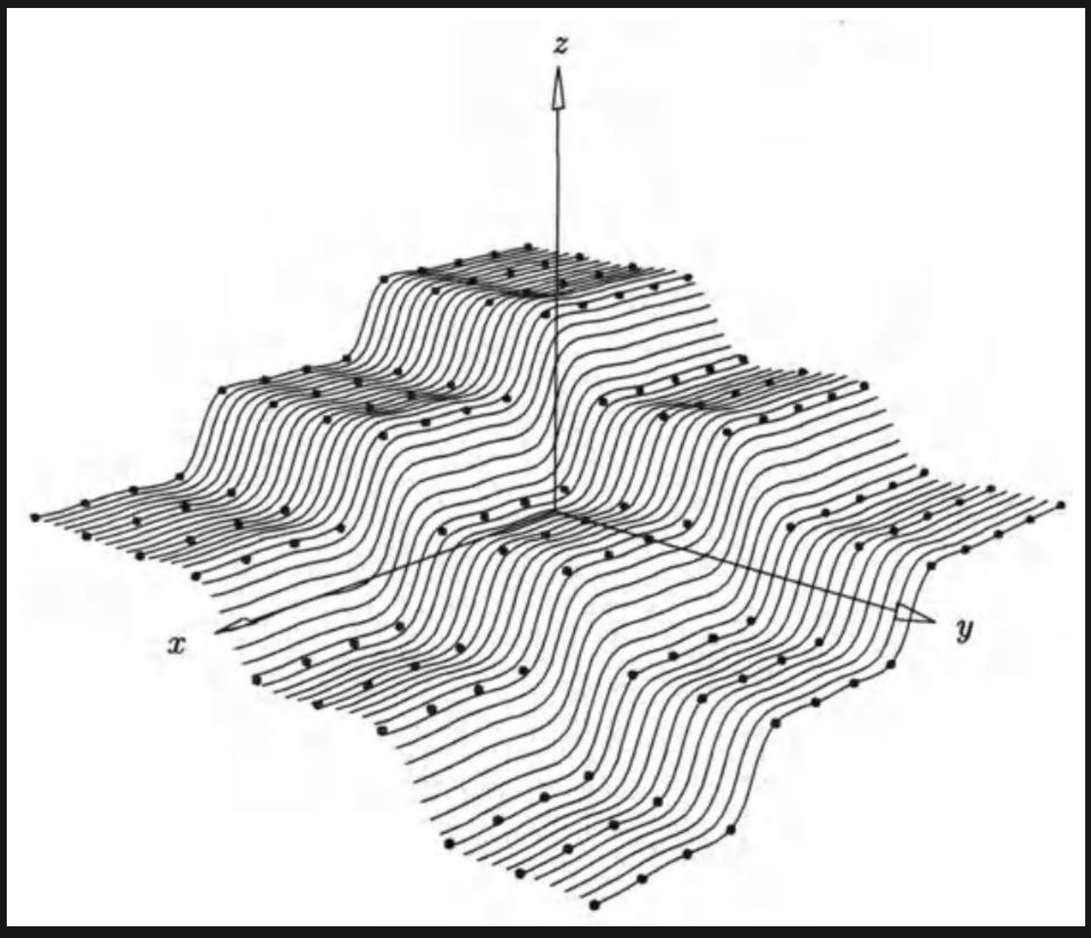


图 9.47. 显示在图 9.44 中的表面插值点。

我们强调，类型 1 和类型 2 的方法都可能无法收敛，这种情况必须在软件实现中处理。

一种类型 1 曲线逼近方法可以按如下方式进行。首先从 $p+1$ 控制点 (最小值) 开始。使用公式 (9.63) 和 (9.65) (9.67) 拟合曲线，或者如果需要内部约束，则使用算法 A9.6。在每次拟合后，使用公式 (9.78) 检查曲线偏差是否小于 E 。为每个节点区间维护记录，指示其是否已收敛；如果公式 (9.78) 对所有 k 满足 $\bar{u}_k \in [u_j, u_{j+1})$ ，则区间 $[u_j, u_{j+1})$ 已收敛。在每次拟合和随后的偏差检查后，在每个未收敛区间的中点添加一个节点，从而增加控制点。该算法必须处理在添加节点时区间可能变为空的情况 (即，该区间不包含任何 $\bar{u}_k$)，从而在拟合步骤中产生奇异的方程组。

一种类型 1 表面逼近方法可以以类似的方式实现，并提高效率。在偏差检查期间，确定具有最大偏差点数的行 ℓ_0 和列 k_0 ，这些偏差大于 E 。然后提取等参数曲线 $\mathbf{C}_{\bar{u}_{k_0}}(v)$ 和 $\mathbf{C}_{\bar{v}_{\ell_0}}(u)$ ，并独立地将它们逼近到 E 之内。这将产生新的 (增加的) 节点向量 U 和 V ，用于下一个表面拟合。使用这种方法，通常只需要进行几次表面拟合和随后的偏差检查。

我们现在开发一种类型 2 曲线近似方法，该方法结合了最小二乘拟合与第 5 章的节点移除技术。首先，我们需要更多的节点移除工具。设 u_r 为一个 p 次非理性曲线的内部节点 $\mathbf{C}(u), u_r \neq u_{r+1}$ ，并用 s 表示 u_r 的重数，即对于 $j = 1, \ldots, s$ 有 $u_{r-s+j} = u_r$ 。设 $\widehat{\mathbf{C}}(u)$ 为通过移除一个 u_r 的出现而得到的曲线。回想一下，新控制点 $\mathbf{P}_i^1$ 和 $\mathbf{P}_j^1$ 分别通过公式 (5.28) 从左侧和右侧计算得出。移除后，$\widehat{\mathbf{C}}(u)$ 与 $\mathbf{C}(u)$ 不同；Tiller [Till92] 推导了两种情况的误差界限：

- 假设 $(p+s) \mod 2 = 0$; 设定 $k = (p+s)/2$ 和

$$B_r = \left|\mathbf{P}_{r-k} - \alpha_{r-k}\mathbf{P}_{r-k+1}^1 - (1-\alpha_{r-k})\,\mathbf{P}_{r-k-1}^1\right| \tag{9.80}$$

其中

$$\alpha_{r-k} = \frac{u_r - u_{r-k}}{u_{r-k+p+1} - u_{r-k}}$$

$$\text{Then} \quad \left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| \leq N_{r-k,p}(u)\, B_r \quad \text{for } u \in [0,1] \tag{9.81}$$

- 假设 $(p+s) \mod 2 = 1$; 设定 $k = (p+s+1)/2$ 和

$$B_r = \left|\mathbf{P}_{r-k}^1 - \mathbf{P}_{r-k+1}^1\right| \tag{9.82}$$

然后 $\left|\mathbf{C}(u) - \widehat{\mathbf{C}}(u)\right| \leq (1-\alpha_{r-k+1})\, N_{r-k+1,p}(u)\, B_r$

$$\text{for } u \in [0,1] \tag{9.83}$$

与

$$\alpha_{r-k+1} = \frac{u_r - u_{r-k+1}}{u_{r-k+p+2} - u_{r-k+1}}$$

B_r 只是节点移除算法 A5.8 中计算的距离。算法 A9.8 是算法 A5.8 的修改版，仅计算 B_r 。

ALGORITHM A9.8

```
GetRemovalBndCurve(n, p, U, P, u, r, s, Br)
    { /* Get knot removal error bound (nonrational) */
            /* Input: n, p, U, P, u, r, s */
            /* Output: Br */
    ord = p+1;
    last = r-s;
    first = r-p;
    off = first-1; /* difference in index between temp and P */
    temp[0] = P[off]; temp[last+1-off] = P[last+1];
    i = first; j=last;
    ii = 1; j j = last-off;
    while (j − i > 0)
        { /* Compute new control points for one removal step */
        alfi = (u-U[i])/(U[i+ord]-U[i]);
        alfj = (u-U[j])/(U[j+ord]-U[j]);
        temp[ii] = (P[i]-(1.0-alfi)*temp[ii-1])/alfi;
        temp[jj] = (P[j]-alfj*temp[jj+1])/(1.0-alfj);
        i = i+1; ii = ii+1;
        j = j-1; j j = jj-1;
        } /* End of while-loop */
    if (j − i < 0) /∗ now get bound ∗/
        { /* Eq. (9.82) */
        Br = Distance3D(temp[ii-1], temp[jj+1]);
      }
        else
        { /* Eq. (9.80) */
        alfi = (u-U[i])/(U[i+ord]-U[i]);
      Br = Distance3D(P[i], alfi*temp[ii+1]+(1.0-alfi)
                                                    *temp[ii-1]);
      }
    }
```

现在设 $\{\mathbf{Q}_k\}$ 为一组点，$\{\bar{u}_k\}$ 为相关的参数集。用 $\{e_k\}$ 表示与这些点相关的一组误差，并设 E 为最大误差界限。给定一条曲线 $\mathbf{C}(u)$ ，其性质是每个 $\mathbf{Q}_k$ 相对于 $\mathbf{C}(u)$ 的最大范数偏差小于或等于 e_k ，算法 A9.9 在保持所有 k 的 $e_k \le E$ 的同时，尽可能多地从 $\mathbf{C}(u)$ 中移除 (大致上) 节点。它还更新 $\{e_k\}$ ，即，它累积误差。输出是新曲线，由 $\widehat{n}, \widehat{U}$ 和 $\widehat{\mathbf{P}}_i$ 表示。该算法维护两个列表，节点移除误差 B_r ，针对每个不同的内部节点；以及对于每个基函数 $N_{i,p}(u)$ ，表示在该函数域内的参数 $\bar{u}_k$ 的索引范围。显然，当移除一个节点时，这些列表必须更新；然而，影响是局部的。只有少数相邻节点的界限 B_r 发生变化，只有少数相邻基函数 (及其域) 发生变化。假设我们移除 u_r ，在移除之前其重数为 s 。移除后，基函数减少一个，在新的编号方案中，只有基函数的 $\bar{u}_k$ 索引范围需要重新计算。

$$N_{i,p}(u) \;\; i = r-p-1, \ldots, r-s \tag{9.84}$$

B_r 的值对于所有旧跨度索引位于范围内的节点发生变化。

$$\max\{r-p, p+1\}, \ldots, \min\{r+p-s+1, n\} \tag{9.85}$$

示例

Ex9.3 见图 9.48: 让立方曲线由 $\mathbf{P}_0, \ldots, \mathbf{P}_{11}$ 和 $U = \{0,0,0,0,1,2,3,4,4,5,6,7,8,8,8,8\}$.*Supposeweremoveoneoremoveor* 定义，结点 $u_8 = 4\,(r = 8, s = 2)$ 的发生。根据公式 (9.85)，必须为结点 2,3,4,5,6 计算新的 B_r 值。根据公式 (9.84)，必须为 $N_{4,3}, N_{5,3}$ 和 $N_{6,3}$ 计算新的 $\bar{u}_k$ 索引范围。

Ex9.4 见图 9.49: 让立方曲线由 $\mathbf{P}_0, \ldots, \mathbf{P}_{10}$ 和 $U = \{0,0,0,0,1,2,3,4,5,6,7,8,8,8,8\}$ 定义。假设我们移除结点 $u_7 = 4\ (r = 7, s = 1)$ 。根据公式 (9.85)，必须为结点 1,2,3,5,6,7(所有剩余的内部结点) 计算新的 B_r 值。根据公式 (9.84)，必须为 $\bar{u}_k$ 和 $N_{3,3}, N_{4,3}, N_{5,3}$ 计算新的 $N_{6,3}$ 索引范围。

算法 A9.9

```
RemoveKnotsBoundCurve(n, p, U, P, ub, ek, E, nh, Uh, Ph)
    { /* Remove knots from curve, bounded */
            /* Input: n, p, U, P, ub, ek, E */
            /* Output: ek, nh, Uh, Ph */
      Inf = 0; /* Big number */
      Get the values Br for all distinct interior knots (A9.8).
    For each basis function, get range of parameter indices.
    while (1)
        {
        Find knot with the smallest Br bound. Set r and s .
                if (Br == Inf) break; /* Finished */
                Using Eqs. (9.81), (9.83), and Algorithm A2.4 to compute
                      NewError [k] , form temp [k] = ek [k] + NewError [k] at
                      all ub[k] values falling within the relevant domain.
                If knot is removable /* All temp[k] <= E */
                  {
                  Update ek[] : ek[k] = temp[k] for relevant range.
                  Call routine similar to A5.8 to remove knot
                        (remove without tolerance check).
                  If no more internal knots, break. /* Finished */
                  Using Eq. (9.84), compute new index ranges for
                        affected basis functions.
                  Using Eq. (9.85), compute new error bounds for the
                        relevant knots.
                  }
                  else
                  {
                  Set this Br to Inf.
                  }
                }
              }
```

Figure 9.48. Removal of the knot $u_8 = 4$ one time.

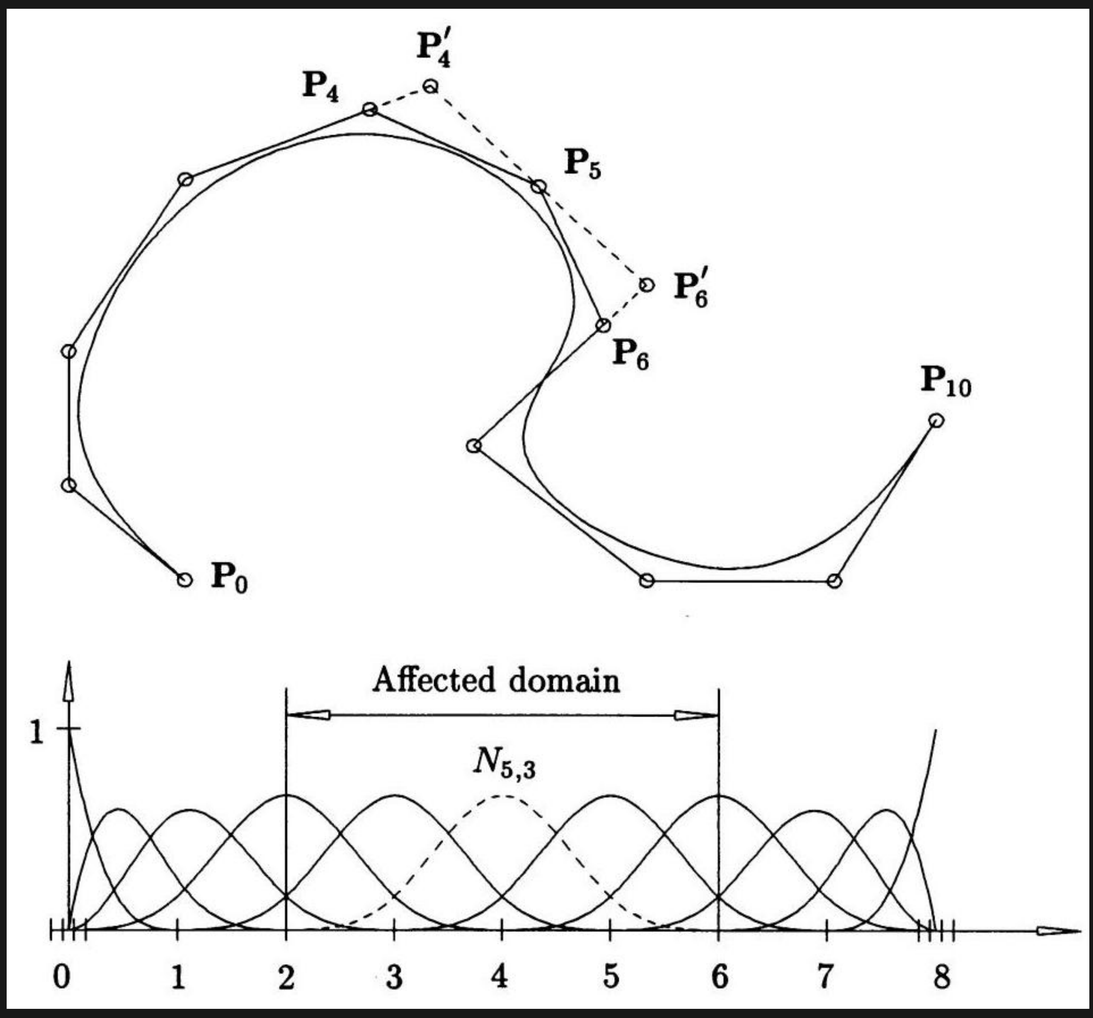


图 9.49. 移除结点 $u_7 = 4$ 一次。

我们现在可以组合一个类型 2 曲线逼近方法。

算法 A9.10

```
GlobalCurveApproxErrBnd(m,Q,P,E,n,U,P)
{ /* 全局曲线逼近到界限 E 内 */
/* 输入:m, Q, p, E */
/* 输出:n, U, P */
```

计算 $\bar{u}_k$ 并加载到 ub[] 中 (公式 [9.5])。

将 U 和 P 设置为插值 Q_k 的 1 次曲线: $U = \{0, 0, \bar{u}_1, \ldots, \bar{u}_{m-1}, 1, 1\}$ 和 $\mathbf{P}_k = \mathbf{Q}_k$ 。

初始化 e_k : ck [0] , . . . , ck [m] = 0 。

```
n = m;
for (deg=1; deg<=p; deg++)
{
RemoveKnotsBoundCurve(n, p, U, P, ub, ek, E, nh, Uh, Ph);
if (deg == p) break;
```

设 U 为通过将 Uh 的度从 deg 提升到 deg+1(将所有重数增加 1) 而获得的节点向量。重置 n(方程 [5.32])。

使用 **n** 、ub、度 deg+1 和节点向量 U 对 $\mathbf{Q}_k$ 进行最小二乘曲线拟合；这将产生新的控制点 P。

```
    Using Eqs. (6.4),(6.5), project all Qk to current curve
            to get Rk = C(ūk) . Then update ek[] and ub[]:
            ek = |Qk, Rk| and ūk = uk.
    } /* End of for-loop */
if (n == nh) return (n, U, P) ;
Set U = Uh and n = nh , and fit one final time with degree p
        and knots U.
Project the Qk and update ek[] and ub[].
RemoveKnotsBoundCurve(n, p, U, P, ub, ek, E, nh, Uh, Ph);
return(nh, Uh, Ph);
}
```

从线性曲线开始并逐步提升到度 p 的优势有三:

- 数据中固有的几何特征 (例如，尖点和曲率的不连续性) 往往在适当的阶段被捕捉；如果已知数据是平滑的，则算法 A9.10 可以从度 2 开始 (在进入 for 循环之前进行二次插值)；
- 逐步演变的曲线往往会“稳定”到自然参数化；
- 在全局拟合大量点时的一般规则是，度数越高，节点越多 (插值是极限)，波动越严重；算法 9.10 在增加度数的同时减少节点跨度，从而倾向于减少最终曲线的波动。

图 9.50a 显示了使用算法 A9.10 的立方曲线拟合示例。容差为 $E = 7/100$ 。图 9.50 b 显示了该曲线的参数化，以及控制多边形。注意双节点和单节点的混合，正如交替接触和不接触的多边形边所显示的那样。图 9.51 显示了另一个立方曲线逼近的示例，具有 $E = 8/100$ 。

在算法 A9.10 的节点移除步骤中，最大范数误差界限 E 是有保证的。最小二乘拟合所产生的误差保证小于或等于之前 (移除节点曲线) 的误差，从最小二乘的意义上讲。在实践中，它似乎在最大范数意义上总是更好。无论如何，最大范数误差可以在点投影步骤中轻松监控，当 e_k 和 $\bar{u}_k$ 被更新时。

通过使用算法 A9.6 进行拟合，并将相关的 e_k 初始化为 ∞ 而不更新它 (或其 $\bar{u}_k$)，可以轻松地将约束纳入算法中。Lyche [Lych87,88] 使用了类似的节点移除技术用于曲线和曲面逼近以及数据减少。

可以开发一种类似的类型 2 曲面逼近方法。设

$$\mathbf{S}(u, v) = \sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(u) N_{j,q}(v) \mathbf{P}_{i,j} \quad (u, v) \in [0, 1] \times [0, 1]$$

为一个 (p, q) 次 B 样条曲面。假设 u_r 是一个重数为 s 的 u 节点，

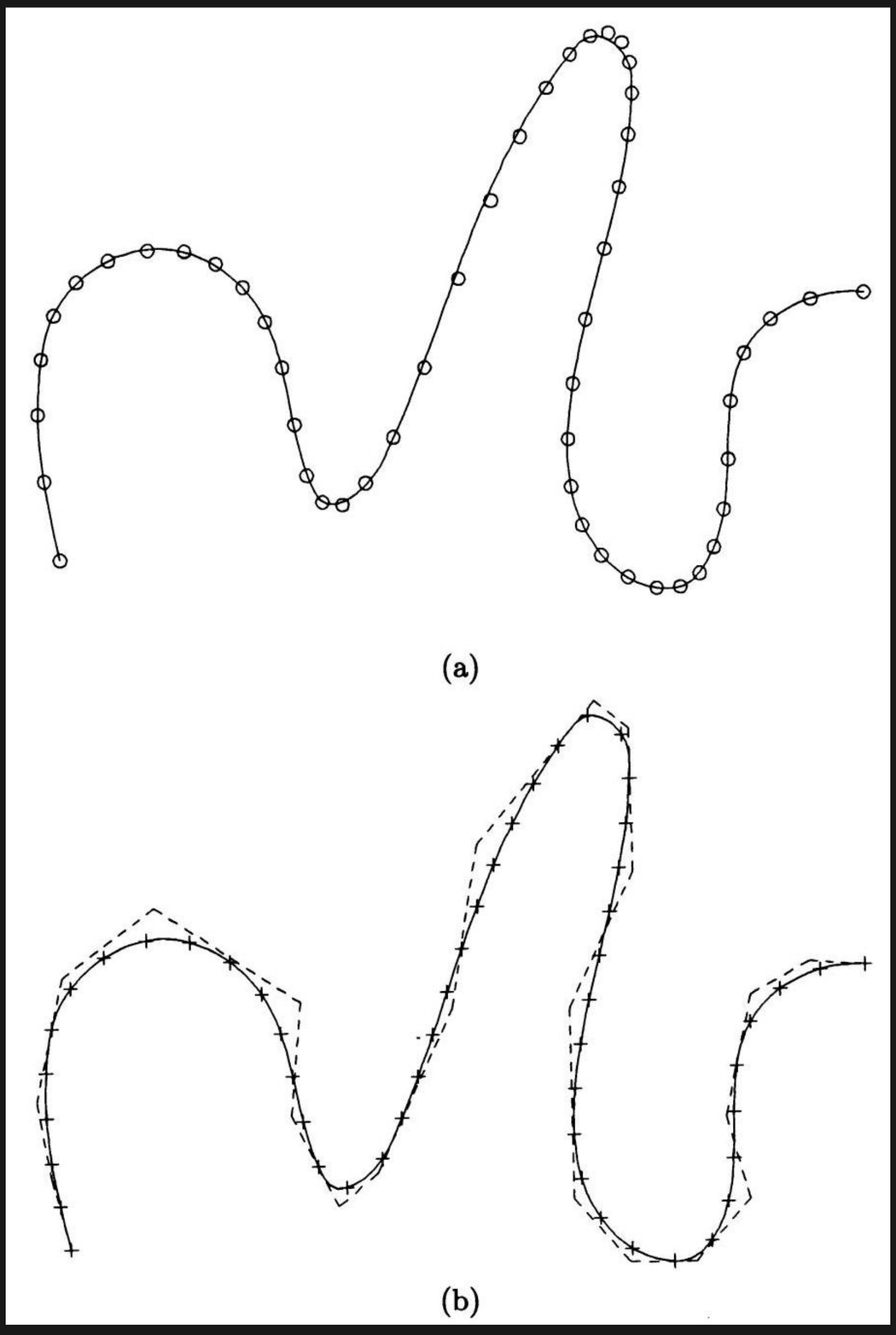


图 9.50. 使用算法 A9.10 的立方曲线拟合。

并且让 $\widehat{\mathbf{S}}(u,v)$ 表示通过移除一个 u_r 的出现而获得的曲面。考虑到 $\mathbf{S}(u,v)$ 的张量积结构，并使用 Tiller [Till92] 的技术，可以证明:

$$\text{- if } (p+s) \bmod 2 = 0 \text{ , set } k = (p+s)/2 \text{ and}$$

$$B_r^j = \left|\mathbf{P}_{r-k,j} - \alpha_{r-k}\mathbf{P}^1_{r-k+1,j} - (1-\alpha_{r-k})\,\mathbf{P}^1_{r-k-1,j}\right|$$

$$j = 0, \ldots, m \tag{9.86}$$

其中

$$\alpha_{r-k} = \frac{u_r - u_{r-k}}{u_{r-k+p+1} - u_{r-k}}$$

$$\text{Then} \quad \left|\mathbf{S}(u,v) - \widehat{\mathbf{S}}(u,v)\right| \leq N_{r-k,p}(u) \sum_{j=0}^{m} N_{j,q}(v)\, B_r^j$$

$$\text{for } (u,v) \in [0,1] \times [0,1] \tag{9.87}$$

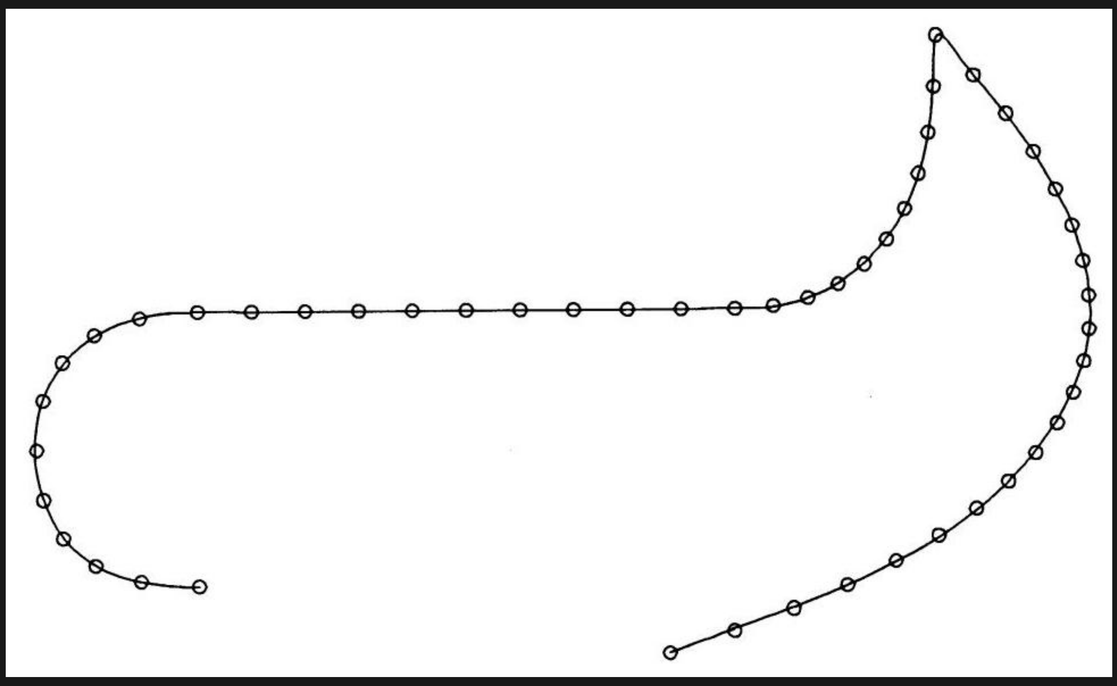


图 9.51. 使用算法 A9.10 和 $E = 8/100$ 的立方曲线拟合。

- 如果 $(p+s) \mod 2 = 1$ ，设置 $k = (p+s+1)/2$ 和

$$B_r^j = \left| \mathbf{P}_{r-k,j}^1 - \mathbf{P}_{r-k+1,j}^1 \right| \quad j = 0, \ldots, m \tag{9.88}$$

$$\text{Then } \left| \mathbf{S}(u,v) - \widehat{\mathbf{S}}(u,v) \right| \leq (1 - \alpha_{r-k+1}) N_{r-k+1,p}(u) \sum_{j=0}^{m} N_{j,q}(v) B_r^j$$

$$\text{for } (u,v) \in [0,1] \times [0,1] \tag{9.89}$$

与

$$\alpha_{r-k+1} = \frac{u_r - u_{r-k+1}}{u_{r-k+p+2} - u_{r-k+1}}$$

类似的方程适用于 v 方向的结去除。利用这些方程，可以为类似于算法 A9.8 和 A9.9 的表面开发结去除策略。结合基于最小二乘的表面拟合算法 A9.7，提供了一种将一组点 $\{\mathbf{Q}_{k,\ell}\}$ 逼近到 E 容差内的方法。

图 9.52a – 9.52e 显示了几个表面逼近的例子。待逼近的数据集显示在图 9.52a 中。图 9.52b 显示了一个度数为 (2,2) 的表面，这是通过使用 $E = 0$ 获得的插值面。将 E 增加到 1/2 会产生图 9.52c 中的表面。进一步将 E 增加到 1 会产生图 9.52 d 中的表面。最后，使用 $E = 1$ 的度数为 (3,3) 的逼近显示在图 9.52e 中。436 曲线和表面拟合

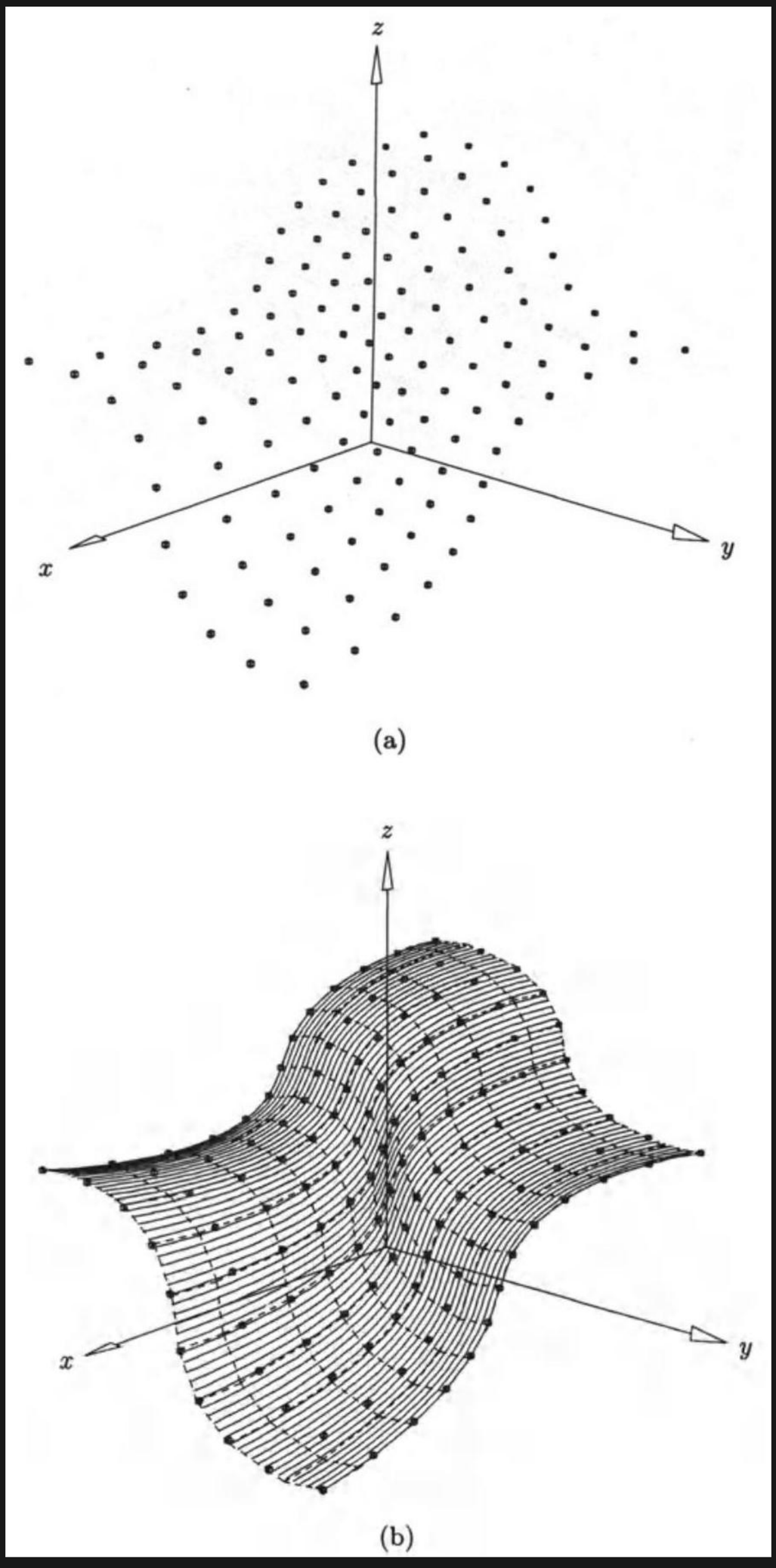
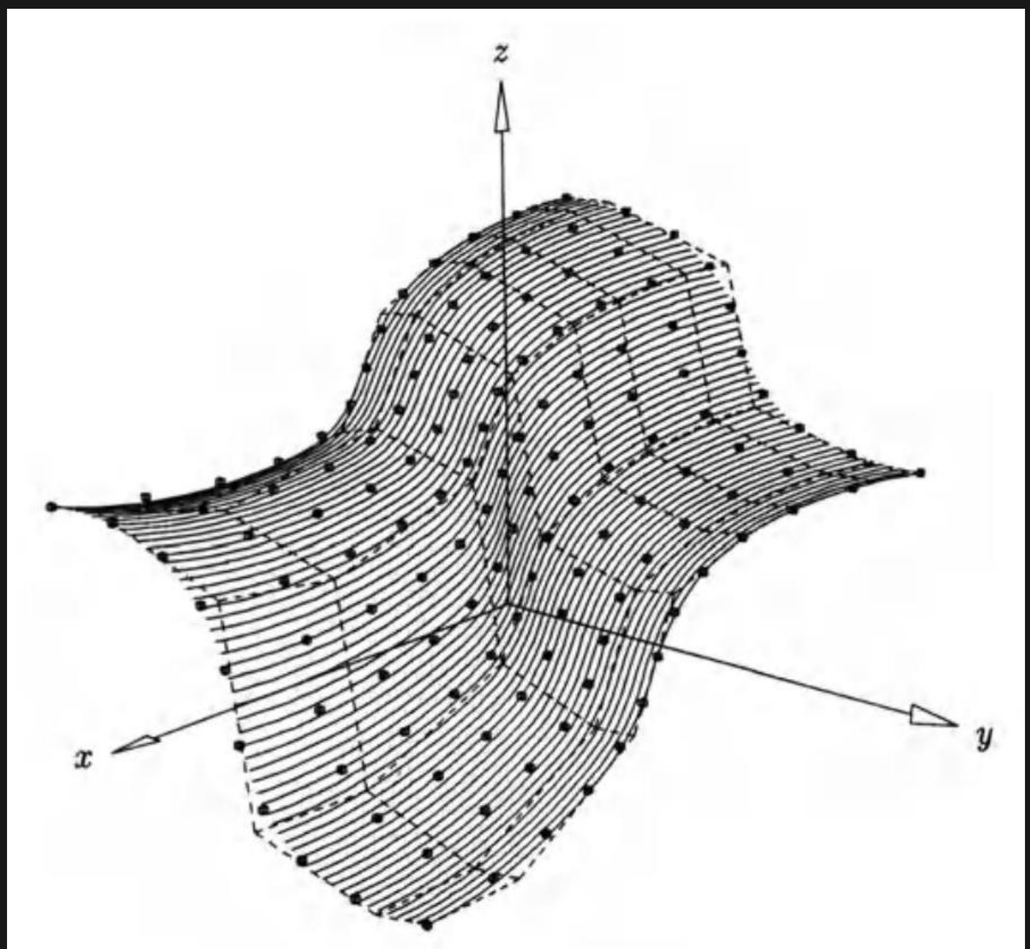
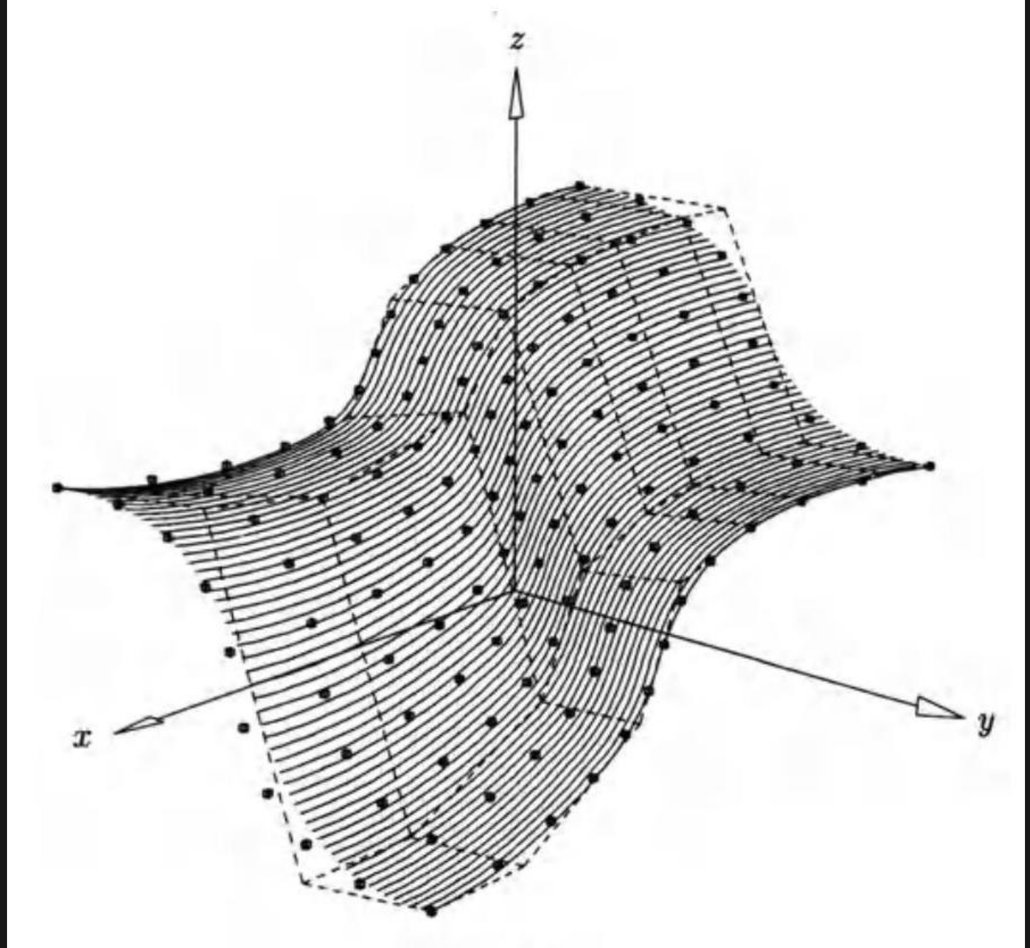


图 9.52. 使用算法 A9.10 扩展到表面的各种表面拟合示例。



(c)



(d)

图 9.52.（续。）

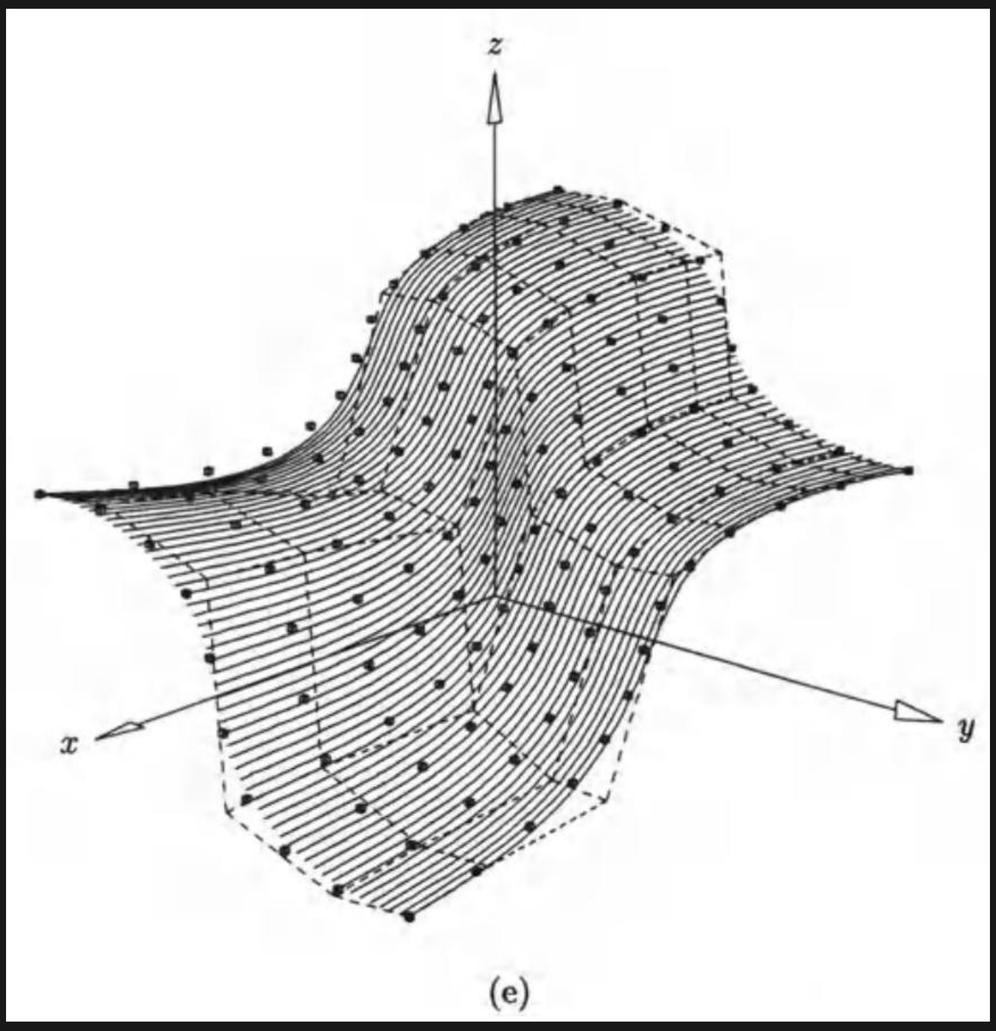


图 9.52. (续。)

算法 A9.10 和相应的表面算法实际上是数据简化算法，即它们在大数据集上效果最佳。它们假设在结去除阶段 (算法 A9.9) 可以去除大量结。如果情况并非如此，最小二乘逼近步骤可能会失败，原因可能是度数提升需要的控制点超过数据点，或者因为多个结使得方程组奇异。一个简单的解决方法是用更高阶的曲线重新开始该过程。

5 局部逼近

设定 $\mathbf{Q}_0, \ldots, \mathbf{Q}_m$ 和 E 。在这里，我们给出两个局部曲线近似的例子：第一个生成一个有理二次 B-spline 曲线，第二个生成一个非有理三次 B-spline 曲线。在这两种情况下，曲线都是通过构造而连续的 G^1 ；它们可以被设定为 C^1 连续，但对于二次情况并不推荐这样做，因为这可能导致非常糟糕的参数化。二次曲线拟合平面数据，而三次曲线拟合任意三维数据。这些算法是对 [Pieg87b; Chou92] 中提出的方法的变体。

一个好的近似算法试图生成尽可能少的段。实现这一目标的一种方法是设定 $k_s = 0$ ，然后找到最大的 $k_e \leq m$ ，使得 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 可以在 E 容差内通过一个 p 次段进行拟合。一旦找到，设定 $k_s = k_e$ 并重复该过程以找到最大的 $k_e \leq m$ ，使得 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 可以通过一个 p 次段进行拟合。这一过程持续进行，直到 $k_s = m$ 。由于在近似中 m 通常相当大，因此二分搜索比线性搜索快得多。在实际操作中，可能希望限制可以通过一个段进行近似的数据点数量。我们用 $K_{\max}$ 表示这个界限，即，二分搜索算法被修改以保证

$$k_e - k_s \leq K_{\max}$$

始终成立。这是通过在每次搜索下一个段的开始时将 k_e 初始化为 $k_s + K_{\max}$ 来实现的。设定 $K_{\max} < m$ 有两个优点:

- 对于固定的容差 E ，较小的 $K_{\max}$ 产生更准确的拟合；
- 如果 m 很大而 E 非常小 (这通常是情况)，那么每个段近似的点的平均数量相对于 m 是小的；在这种情况下，如果 $K_{\max}$ 设置得远小于 m ，算法将更有效。

我们在第 9.5.1 节和第 9.5.2 节中分别开发了 FitWithConic( ) 和 FitWithCubic( ) 例程。这些例程试图将点 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 拟合到容差 E 内，分别使用圆锥或立方段。如果拟合成功，调用 (搜索) 例程要么接受该段，要么增加 k_e 并再次调用拟合例程以尝试找到更大的段 (具体取决于搜索策略)。如果拟合不成功，调用例程将减少 k_e 并再次调用拟合例程。

对于二次和三次情况，节点向量在接受段时构建。在这两种情况下都使用双内部节点，产生 G^1 二次曲线和 C^1 三次曲线。与段边界对应的累积弦长用于二次曲线的节点值；使用公式 (9.37) 计算三次曲线的节点。

5.1 局部有理二次曲线逼近

现在我们开发例程 FitWithConic( )。我们假设点 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 位于唯一的平面中，并且给定了端切线 $\mathbf{T}_s$ 和 $\mathbf{T}_e$ 。

考虑图 9.53。暂时假设

$$\mathbf{R} = \mathbf{Q}_{k_s} + \gamma_s \mathbf{T}_s \, \mathbf{R} = \mathbf{Q}_{k_e} + \gamma_e \mathbf{T}_e \tag{9.90}$$

具有

$$\gamma_s > 0 \; \gamma_e < 0 \tag{9.91}$$

然后确定一个近似的圆锥，分三步进行:

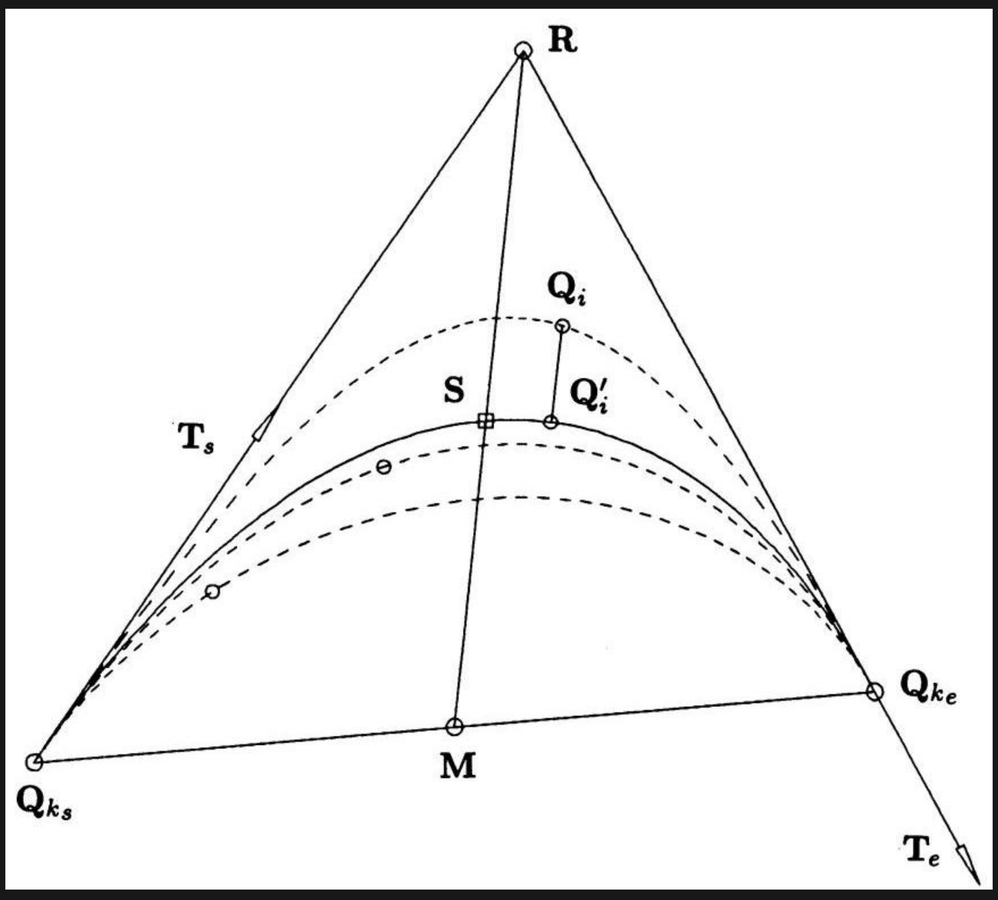


图 9.53. 圆锥曲线拟合的曲线拟合和散点检查。

1. 使用 $\mathbf{Q}_{k_s}$ 、 $\mathbf{R}$ 和 $\mathbf{Q}_{k_e}$ 作为控制多边形，按照算法 A7.2 对每个点 $\mathbf{Q}_i, i = k_s + 1, \ldots, k_e - 1$ 进行插值。这将产生中间权重 w_i 和相应的肩点坐标 $s_i = w_i / (1 + w_i)$ 。

2. 通过对 s_i 进行平均计算一个近似圆锥，即。

$$s = \frac{1}{k_e - k_s - 1} \sum_{i=k_s+1}^{k_e-1} s_i \tag{9.92}$$

根据公式 (7.30)，近似圆锥的肩点为 $\mathbf{S} = (1 - s)\,\mathbf{M} + s\mathbf{R}$ ，根据公式 (7.31)，中间权重为 $w = s/(1 - s)$ ；

3. 将每个 $\mathbf{Q}_i$ 投影到近似圆锥上，以检查它们是否都在容差范围内。

有三种特殊情况需要不同处理:

- $k_e - k_s = 1$ ，即没有内部点；算法在 $\mathbf{Q}_{k_s}$ 和 $\mathbf{Q}_{k_e}$ 之间计算一个插值线段，正如第 9.3.3 节所描述的那样；
- $\mathbf{T}_s$ 和 $\mathbf{T}_e$ 是平行的；如果它们与弦 $\mathbf{Q}_{k_s}\mathbf{Q}_{k_e}$ 平行，并且点 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 共线，则返回的圆锥段实际上是一个线段；否则不进行拟合；
- 计算了 $\mathbf{R}$ ，但公式 (9.91) 不成立；不进行拟合。

FitWithConic( ) 返回值 1 表示拟合成功，返回值 0 表示未进行拟合。在成功拟合的情况下，还返回加权中间控制点 $\mathbf{R}^w$ 。

ALGORITHM A9.11

```
FitWithConic(ks, ke, Q, Ts, Te, E, Rw)
  { /* Fit to tolerance E with conic segment */
      /* Input: ks, ke, Q, Ts, Te, E */
      /* Output: Rw */
  if (ke-ks == 1)
    { /* No interior points to interpolate */
    Fit an interpolating segment as in Section 9.3.3.
    return(1);
    }
  i = Intersect3DLines(Q[ks], Ts, Q[ke], Te, alf1, alf2, R);
  if (i != 0 )
    { /* No intersection */
    if (Q_ks, ..., Q_ke not collinear) return (0);
        else
            {
            Rw = (Q [ks] + Q [ke]) /2.0;
            return(1);
            }
    }
  if (alf1 <= 0.0 || alf2 >= 0.0) return(0);
  s = 0.0; v = Q[ke]-Q[ks];
  for (i=ks+1; i<=ke-1; i++)
      { /* Get conic interpolating each interior point */
    V1 = Q[i] - R;
    j = Intersect3DLines(Q[ks], V, R, V1, alf1, alf2, dummy);
    if (j != 0 || alf1 <= 0.0 || alf1 >= 1.0 || alf2 <= 0.0)
        return(0);
    Compute the weight wi. /* Algorithm A7.2 */
    s = s + wi/(1.0+wi);
    }
  s = s/ (ke − ks − 1) ; w = s/ (1.0 − s) ;
  if (w < WMIN || w > WMAX) /* system bounds on weights */
    return(0);
  Create Bezier segment with Q [ks] , R, Q [ke] and w .
  for (i=ks+1; i<=ke-1; i++)
    {
    Project Q[i] onto the Bezier segment.
    if (distance > E) return(0);
    }
  Rw = w ∗ R ;
  return(1);
 }
```

图 9.54a-9.54e 显示了局部二次曲线拟合的示例。对数螺旋

$$r = \frac{1}{2} e^{\theta/5}$$

被采样并进行了近似。图 9.54a − 9.54 d 的输入参数为:

图 9.54a $E = 1/100,\ K_{\max} = 50$

图 9.54 b $E = 1/1000,\ K_{\max} = 50$

图 9.54c $E = 1/10,\ K_{\max} = 6$

图 9.54 d $E = 1/10,\ K_{\max} = 10$

图 9.54e 显示了图 9.54b 中曲线的参数化。

5.2 局部非有理三次曲线逼近

现在我们开发 FitWithCubic( )。设 $\mathbf{P}_0 = \mathbf{Q}_{k_s}$ 和 $\mathbf{P}_3 = \mathbf{Q}_{k_e}$ 表示三次 Bézier 段的起点和终点，$\mathbf{T}_s$ 和 $\mathbf{T}_e$ 表示起始和结束单位切线 (见图 9.55)。我们必须确定两个内部控制点 $\mathbf{P}_1$ 和 $\mathbf{P}_2$ 。设定

$$\mathbf{P}_1 = \mathbf{P}_0 + \alpha\mathbf{T}_s \, \mathbf{P}_2 = \mathbf{P}_3 + \beta\mathbf{T}_e$$

我们必须确定 α 和 β 的值，使得由 $\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3$ 定义的三次 Bézier 曲线在所有 $\mathbf{Q}_k, k_s < k < k_e$ 的 E 容差范围内。如果 $k_e - k_s = 1$ ，那么理论上，任何 α 和 β 都可以；然而，它们应该被选择为使得导数的大小与邻近的大小自然融合。使用公式 (9.29) 和练习 9.2，对于 $\mathbf{Q}_{k_s}$ 和 $\mathbf{Q}_{k_e}$ 的导数 $\mathbf{D}_{k_s}$ 和 $\mathbf{D}_{k_e}$ 的合理估计为

$$\mathbf{D}_{k_s} = \mathbf{V}_{k_s} \text{ if } k_s = 0$$

$$\mathbf{D}_j = \frac{c_{j+1} - c_j}{c_j - c_{j-1}}\mathbf{V}_j \; j \neq \{0, m\} \; j = k_s \text{ or } j = k_e$$

$$\mathbf{D}_{k_e} = \mathbf{V}_{k_e} \text{ if } k_e = m \tag{9.93}$$

其中 $\{c_j\}$ 是累积弦长。然后设定

$$\alpha = \frac{|\mathbf{D}_{k_s}|}{3} \; \beta = -\frac{|\mathbf{D}_{k_e}|}{3} \tag{9.94}$$

现在假设 $k_e - k_s > 1$ 。我们的算法由三个步骤组成:

1. 对于每个 $k, k_s < k < k_e$ ，插值一个三次 Bézier 段到数据 $\mathbf{P}_0, \mathbf{T}_s, \mathbf{Q}_k, \mathbf{P}_3, \mathbf{T}_e$ ，从而获得一个 α_k 和一个 β_k ；Bézier 段 $\mathbf{C_k}(u)$ 定义在归一化区间 $u \in [0, 1]$ 上；

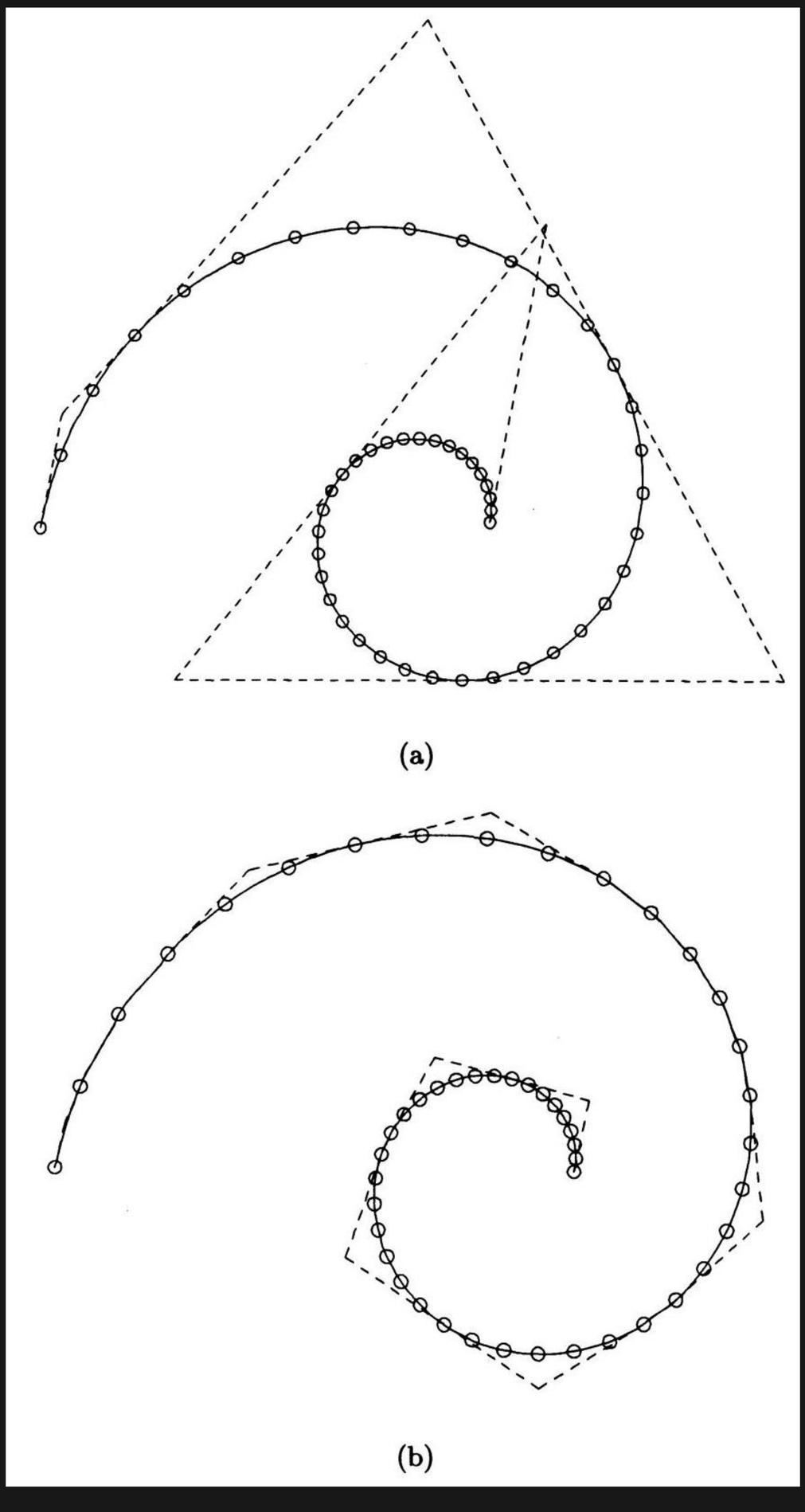
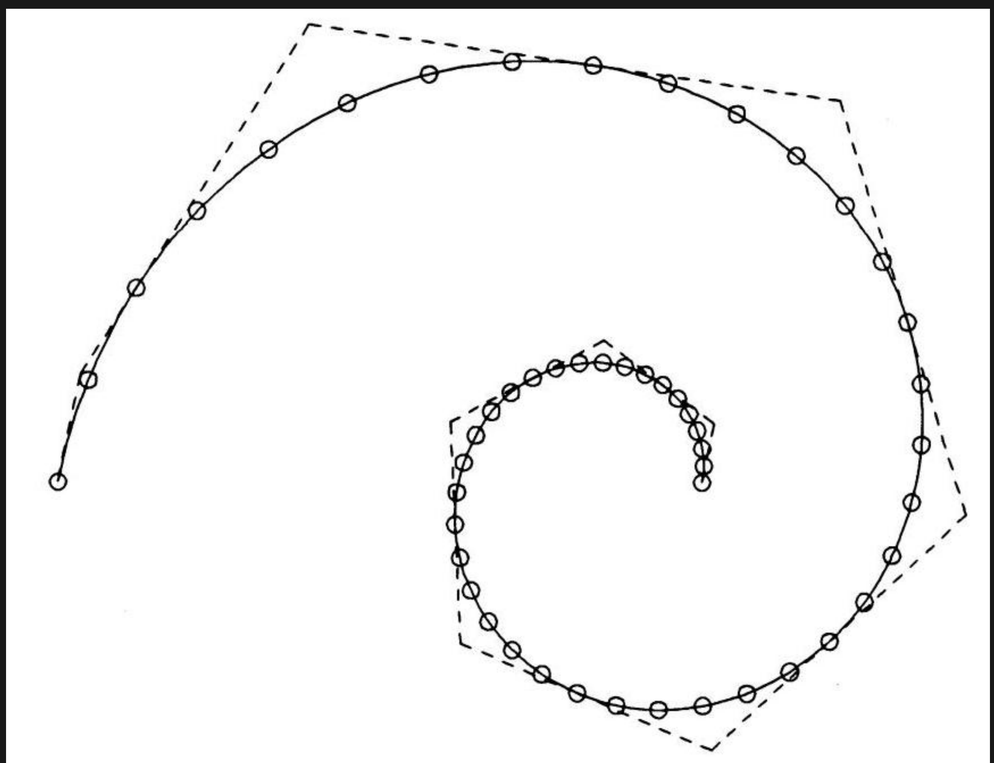
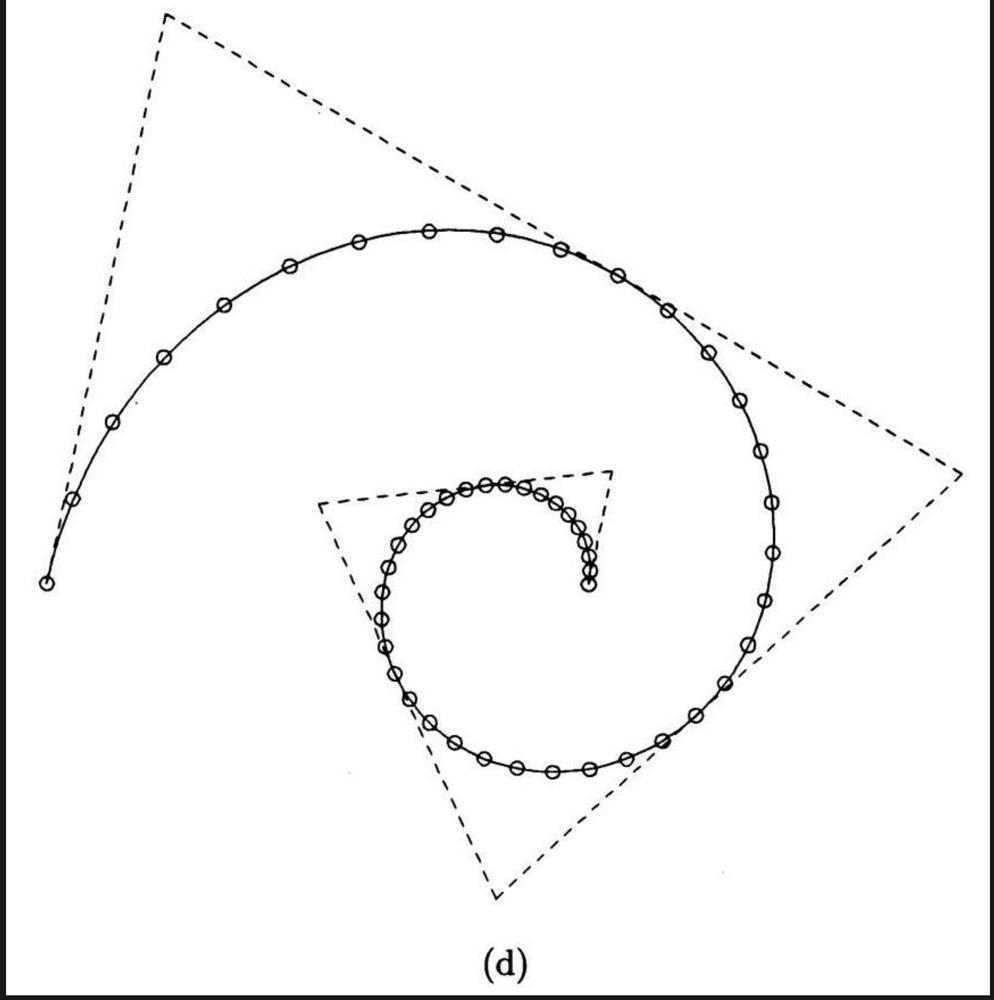


图 9.54. 局部二次曲线拟合示例。

2. 对 α_k s 和 β_k s 进行平均以获得 α 和 β ；这些定义了候选的 Bézier 段，$\mathbf{C}(u)$ ；
3. 检查所有 $\mathbf{Q}_k, k_s < k < k_e$ 是否在 E 距离 $\mathbf{C}(u)$ 之内。



(c)



(d)

图 9.54. (续。)

设 $\mathbf{P} = \mathbf{Q}_6$ 为要插值的点，并用 $B_{i,3}$ 表示 $u \in [0,1]$ 上的三次 Bernstein 多项式。我们写 $\mathbf{P}$ 为

$$\mathbf{P} = B_{0,3}\mathbf{P}_0 + B_{1,3}\mathbf{P}_1 + B_{2,3}\mathbf{P}_2 + B_{3,3}\mathbf{P}_3$$

$$= \mathbf{P}_0\left(B_{0,3} + B_{1,3}\right) + \alpha\mathbf{T}_e B_{1,3} + \beta\mathbf{T}_e B_{2,3} + \mathbf{P}_3\left(B_{2,3} + B_{3,3}\right) \tag{9.95}$$

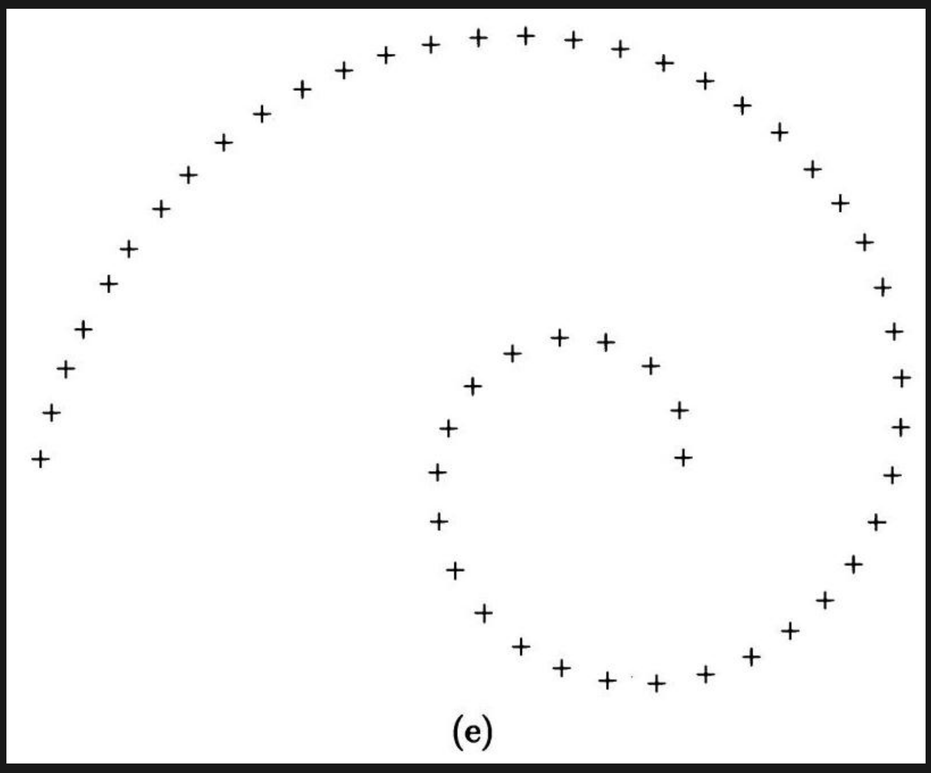


图 9.54. (续。)

最后

$$\mathbf{P} = \mathbf{P}_c + a\mathbf{T}_s + b\mathbf{T}_e \tag{9.96}$$

其中点

$$\mathbf{P}_c = \mathbf{P}_0 (B_{0,3} + B_{1,3}) + \mathbf{P}_3 (B_{2,3} + B_{3,3}) \tag{9.97}$$

位于直线 $\mathbf{P}_0\mathbf{P}_3$ 上，并

$$a = \alpha B_{1,3}\, b = \beta B_{2,3} \tag{9.98}$$

因此

$$\alpha = \frac{a}{B_{1,3}}\, \beta = \frac{b}{B_{2,3}} \tag{9.99}$$

考虑图 9.55。方程 (9.96) 表示我们通过从 $\mathbf{P}_c$ 开始，沿着与 $\mathbf{T}_s$ 平行的方向移动一定距离 (到 $\mathbf{P}_d$)，然后从那里继续朝着 $\mathbf{T}_e$ 的相反方向移动另一定距离，来获得 $\mathbf{P}$ 。由于 $\mathbf{T}_s$ 和 $\mathbf{T}_e$ 的长度为单位，因此可以得出

$$a = |\mathbf{P}_c\mathbf{P}_d|\ b = -|\mathbf{P}_d\mathbf{P}| \tag{9.100}$$

因此，我们必须从方程 (9.100) 计算 $\mathbf{P}_c$ 和 $\mathbf{P}_d$ ，然后从方程 (9.100) 计算 a 和 b ，最后从方程 (9.99) 计算 α 和 β 。

有两种情况需要考虑，平面内和不平面内。首先假设向量 $\mathbf{T}_s, \mathbf{T}_e$ 和 $\mathbf{P}_0\mathbf{P}_3$ 不共面。事实证明，存在一条唯一的三次贝塞尔曲线通过 $\mathbf{P}$ 、$\mathbf{P}_c$ ，并且 $\mathbf{P}_d$ 可以按如下方式构造 (见图 9.55):

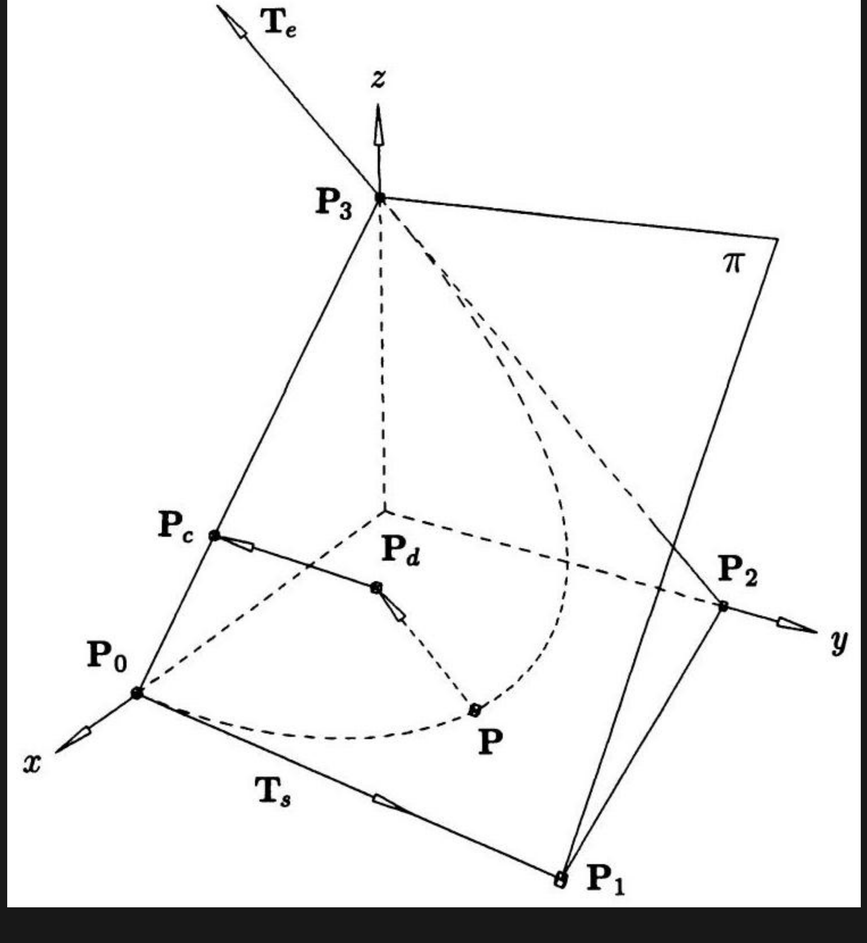


图 9.55。由端点、端切线和曲线上的一点指定的唯一三次三维 (扭曲) 贝塞尔曲线。

1. 设 π 为由 $\mathbf{P}_0, \mathbf{P}_3$ 和 $\mathbf{T}_s$ 定义的平面；
2. 将 π 与通过 $\mathbf{P}$ 并平行于 $\mathbf{T}_e$ 的直线相交 - 这将产生 $\mathbf{P}_d$ ；
3. 通过将直线 $\mathbf{P}_0\mathbf{P}_3$ 与通过 $\mathbf{P}_d$ 并平行于 $\mathbf{T}_s$ 的直线相交来获得 $\mathbf{P}_c$ 。

方程 (9.100) 现在得出 a 和 b 。为了通过方程 (9.99) 计算 α 和 β ，我们需要唯一的参数值 $\widehat{u}$ ，该值使得曲线通过点 P。从方程 (9.97) 和 $B_{i,3}$ 的和为 1 的事实，我们得到

$$\mathbf{P}_c = \gamma\mathbf{P}_0 + (1-\gamma)\,\mathbf{P}_3 \tag{9.101}$$

其中

$$\gamma = \frac{|\mathbf{P}_c\mathbf{P}_3|}{|\mathbf{P}_0\mathbf{P}_3|} = B_{0,3} + B_{1,3} \tag{9.102}$$

γ 被计算为定向距离的商，即 $\gamma \in [0,1]$ 当且仅当 $\mathbf{P}_c$ 位于 $\mathbf{P}_0$ 和 $\mathbf{P}_3$ 之间时。多项式 $B_{0,3} + B_{1,3}$ 在 u 中是三次的 (读者应绘制此函数)，对于任何给定的 $\gamma \in [0,1]$ ，方程 (9.102) 有一个唯一的解位于 $[0,1]$ 内。如果 $\gamma \notin [0,1]$ ，则拟合被中止 (返回“无拟合”给调用例程)。三次方程 (9.102) 可以通过牛顿迭代法求解。获得良好起始值的一种方法是列出一些与 0 到 1 之间的 γ 值对应的 $\widehat{u}$ 值。然后，给定任何 γ ，在表中快速搜索可以得到一个起始 $\widehat{u}$ 。最后，将 $\widehat{u}$ 代入方程 (9.99) 得到 α 和 β 。

如果 $\mathbf{T}_s$, $\mathbf{T}_e$ 和 $\mathbf{P}_0\mathbf{P}_3$ 共面，则可以存在多条通过 $\mathbf{P}$ 的立方曲线，而之前的几何构造 $\mathbf{P}_c$ 和 $\mathbf{P}_d$ 失效。有几个特殊情况使得问题复杂化，因此我们推导出一组方程，总是能得到 α 和 β 。我们首先将一个参数 $\widehat{u}$ 指派给 $\mathbf{P}$ 。使用弦长参数化可以获得一个良好的估计，即。

$$\widehat{u}_{k_s} = 0$$

$$\widehat{u}_k = \widehat{u}_{k-1} + \frac{c_k - c_{k-1}}{c_{k_e} - c_{k_s}} \tag{9.103}$$

$\widehat{u}_k$ 实际上是 $\mathbf{Q}_k$ 在 $\mathbf{Q}_{k_s}, \ldots, \mathbf{Q}_{k_e}$ 上的标准化弦长参数。现在如果 $\mathbf{P} = \mathbf{Q}_k, k_s < k < k_e$ ，那么 $\widehat{u} = \widehat{u}_k$ 。

接下来我们计算在 $\mathbf{P}$ 处的切线 $\mathbf{T}_p$ (方程 [9.29] 和 [9.31])。设

$$s = 1 - \widehat{u}\ t = \widehat{u}$$

根据定义

$$\mathbf{P} = s^3\mathbf{P}_0 + 3s^2t\mathbf{P}_1 + 3st^2\mathbf{P}_2 + t^3\mathbf{P}_3 \tag{9.104}$$

并回忆 deCasteljau 算法 (方程 [1.12]、算法 A1.5 和图 1.17)，可以推导出

$$\mathbf{P}_0^2 = s^2\mathbf{P}_0 + 2st\mathbf{P}_1 + t^2\mathbf{P}_2$$

$$\mathbf{P}_1^2 = s^2\mathbf{P}_1 + 2st\mathbf{P}_2 + t^2\mathbf{P}_3 \tag{9.105}$$

其中 $\mathbf{P}_0^2$ 和 $\mathbf{P}_1^2$ 位于由 $\mathbf{P}$ 和 $\mathbf{T}_p$ 定义的直线上 (图 9.56)。因此

$$0 = \mathbf{T}_p \times \left(\mathbf{P}_1^2 - \mathbf{P}_0^2\right)$$

$$= \mathbf{T}_p \times \left(t^2\mathbf{P}_3 + t\left(2s - t\right)\mathbf{P}_2 + s\left(s - 2t\right)\mathbf{P}_1 - s^2\mathbf{P}_0\right) \tag{9.106}$$

将 $\mathbf{P}_1 = \mathbf{P}_0 + \alpha\mathbf{T}_s$ 和 $\mathbf{P}_2 = \mathbf{P}_3 + \beta\mathbf{T}_e$ 代入方程 (9.104) 和 (9.106) 得到

$$\left(3s^2t\mathbf{T}_s\right)\alpha + \left(3st^2\mathbf{T}_e\right)\beta = \mathbf{P} - \left(s^3 + 3s^2t\right)\mathbf{P}_0 - \left(t^3 + 3st^2\right)\mathbf{P}_3 \tag{9.107}$$

$$s\left(s - 2t\right)\left(\mathbf{T}_p \times \mathbf{T}_s\right)\alpha + t\left(2s - t\right)\left(\mathbf{T}_p \times \mathbf{T}_e\right)\beta = 2st\left(\mathbf{T}_p \times \left(\mathbf{P}_0 - \mathbf{P}_3\right)\right) \tag{9.108}$$

方程 (9.107) 涉及点约束 $\mathbf{P}$ ，而方程 (9.108) 涉及切线约束 $\mathbf{T}_p$ 。它们各自对两个未知数贡献三个方程，总共六个方程 (或者在数据为 xy 平面时为四个)。一些特殊情况是:

- 如果 $\mathbf{T}_s \parallel \mathbf{T}_e$ ，则方程 (9.107) 仅产生一个方程；方程 (9.108) 提供必要的附加方程；

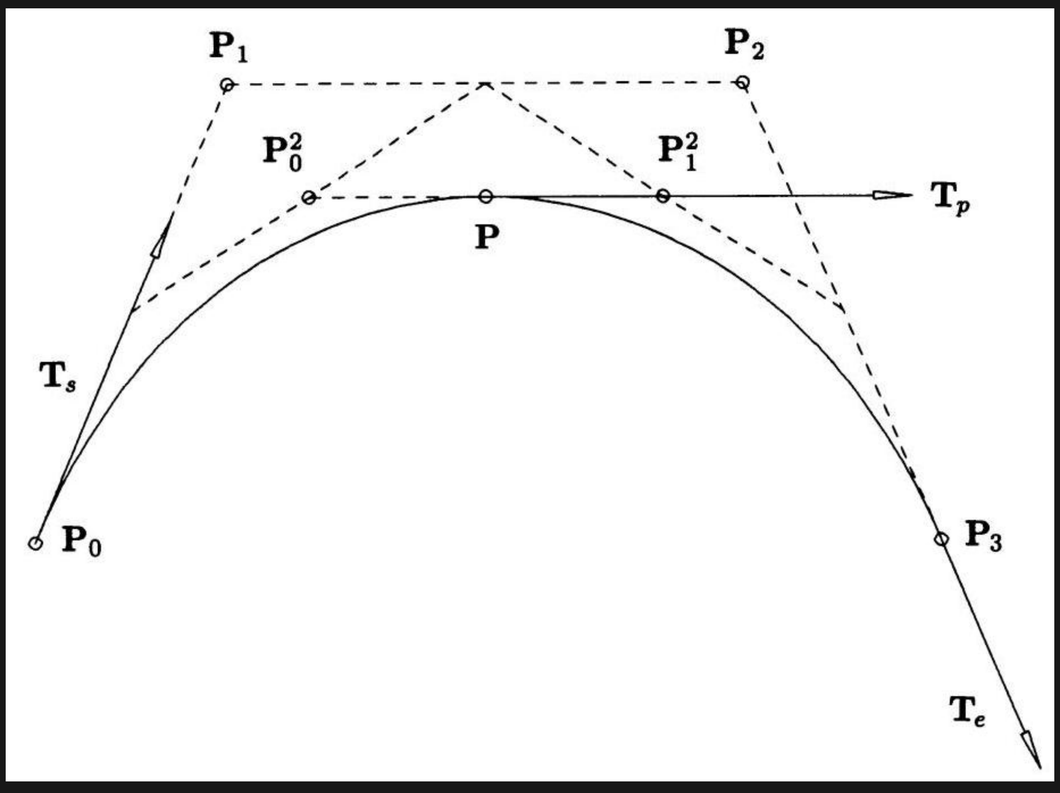


图 9.56. 由端点、端切线、曲线上的一点及该点的切线方向指定的平面立方 Bézier 曲线。

- 如果 $\mathbf{P} = \left(s^3 + 3s^2t\right)\mathbf{P}_0 + \left(t^3 + 3st^2\right)\mathbf{P}_3$ ，则方程 (9.107) 是一个齐次系统；在这种情况下，$\alpha = \beta = 0$ 可能是唯一的解，或者可能存在无限多个非平凡解。假设我们已经排除了直线情况，我们有 $\mathbf{T}_p \times (\mathbf{P}_0 - \mathbf{P}_3) \neq 0$ ，因此方程 (9.108) 提供了必要的非齐次方程。

一种简单的解决方程 (9.107) 和 (9.108) 的方法，避免了对特殊情况的担忧，是将六个方程组合在一起并通过最小二乘法求解。求解后，必须检查 $\alpha > 0$ 和 $\beta < 0$ 是否成立；如果不成立，则拟合被中止，控制权返回给调用程序。一旦方程被求解，α_k s 和 β_k s 仅需取平均，即

$$\alpha = \frac{1}{k_e - k_s - 1}\sum_{i=k_s+1}^{k_e-1}\alpha_i \quad \beta = \frac{1}{k_e - k_s - 1}\sum_{i=k_s+1}^{k_e-1}\beta_i \tag{9.109}$$

候选立方 Bézier 曲线 $\mathbf{C}(u)$ 由 $\mathbf{P}_0, \mathbf{P}_1 = \mathbf{P}_0 + \alpha\mathbf{T}_s$ 、$\mathbf{P}_2 = \mathbf{P}_3 + \beta\mathbf{T}_e$ 和 $\mathbf{P}_3$ 定义。

现在我们必须检查每个 $\mathbf{Q}_k$ 是否在曲线 $\mathbf{C}(u)$ $(k_s < k < k_e)$ 的 E 距离内。这需要牛顿迭代，之前计算的 $\widehat{u}$ 是良好的起始值。回想一下 $\mathbf{Q}_k = \mathbf{C}_k(\widehat{u}_k)$ 。设 $\Delta\alpha_k = \alpha_k - \alpha$ 和 $\Delta\beta_k = \beta_k - \beta$ 。然后从方程 (9.95) 我们得到

$$\left|\mathbf{Q}_k - \mathbf{C}\left(\widehat{u}_k\right)\right| = \left|\mathbf{C}_k\left(\widehat{u}_k\right) - \mathbf{C}\left(\widehat{u}_k\right)\right|$$

$$= \left|\Delta\alpha_k B_{1,3}\left(\widehat{u}_k\right)\mathbf{T}_s - \Delta\beta_k B_{2,3}\left(\widehat{u}_k\right)\mathbf{T}_e\right| \tag{9.110}$$

方程 (9.110) 可以用作快速接受测试: 如果它小于或等于 E ，则不需要牛顿迭代。如果需要迭代，则第 6.1 节中给出的技术是合适的。

FitWithCubic( ) 的例程如下。它返回值 1 表示拟合成功，返回值 0 表示没有拟合。我们注意到，在非共面的情况下，当 $\mathbf{T}_s$, $\mathbf{T}_e$ 和 $\mathbf{P}_0\mathbf{P}_3$ 接近共面时，$\mathbf{P}_c$ 和 $\mathbf{P}_d$ 的构造是不稳定的。因此，这个测试应该在相对宽松的容差下进行。该算法使用局部数组 uh[]、alfak[]、betak[] 来存储 $\widehat{u}_k, \alpha_k$ 和 β_k 。

ALGORITHM A9.12

```
FitWithCubic(ks, ke, Q, Ts, Te, E, P1, P2)
    { /* Fit to tolerance E with cubic segment */
      /* Input: ks, ke, Q, Ts, Te, E */
      /* Output: P1, P2 */
    if (ke-ks == 1)
      { /* No interior points to interpolate */
      Compute α and β by Eqs. (9.93),(9.94).
      Set P1 = Q_ks + α*Ts
      and P2 = Q_ke + β*Te
      return(1);
      }
    dk = ke-ks;
    line = Collinear(dk+1, Q[ks]);
    if (line == 1)
      { /* Collinear points case; create straight line */
      P1 = (2.0*Q[ks]+Q[ke])/3.0;
      P2 = (Q[ks]+2.0*Q[ke])/3.0;
      return(1);
      }
    for (k=1; k<dk; k++)
```

```
            {
            Get plane π defined by Q[ks], Q[ke] , Ts.
            if (Line(Q[k+ks], Te) lies in π )
               { /* Coplanar case */
               Compute ûk by Eq. (9.103) and load into uh[k].
               Set up Eqs. (9.107) and (9.108) and solve for αk and βk
                     (by least squares).
               if (αk > 0 && βk < 0)
                  {
                  alfak[k] = αk;  beta k[k] = βk;
                  }
                  else
                     return(0);
               }
               else
               { /* Noncoplanar case */
               Pd = Intersection of π with Line (Q[k + ks], Te) ;
               Pc = Intersection of Line(Q[ks], Q[ke]-Q[ks]) with
                           Line(Pd, Ts);
               gamma = Distance3D(Pc, Q[ke])/Distance3D(Q[ks], Q[ke]);
               if ( gamma  < 0.0 ||  gamma  > 1.0)  return(0);
               Use Newton iteration to solve Eq. (9.102) for uh[k].
               if (uh[k] < 0.0 | uh[k] > 1.0) return(0);
                     else
                     {
                     a = Distance3D (Pc, Pd) ; b = − Distance3D (Pd, Q[k + ks]) ;
                     Evaluate Bernstein polynomials and use Eq. (9.99)
                           to get alfak [k] and beta k[k] .
                  }
            }
      } /* End of for-loop: k = 1, . . . , dk − 1 ∗ /
            /* Step 2: average the αk s and βk s∗ /
alpha = beta = 0.0 ;
for ( k = 1 ; k < dk ; k + + )
         { alpha = alpha+alfak[k]; beta = beta+betak[k]; }
alpha = alpha/(dk-1); beta = beta/(dk-1);
P1 = Q[ks]+alpha*Ts; P2 = Q[ke]+beta*Te;
            /* Step 3: check deviations */
for (k=1; k<dk; k++)
      {
      u = uh[k] ;
      if (Eq. [9.110] less than E) continue;
            else
            { /* Must do Newton iterations. u is start value */
            Project Q[k + ks] to curve to get error ek.
            if (ek > E) break;
            }
      }
if (k == dk)  return (1) ;  /* segment within tolerance ∗ /
         else return(0); /* not within tolerance */
}
```

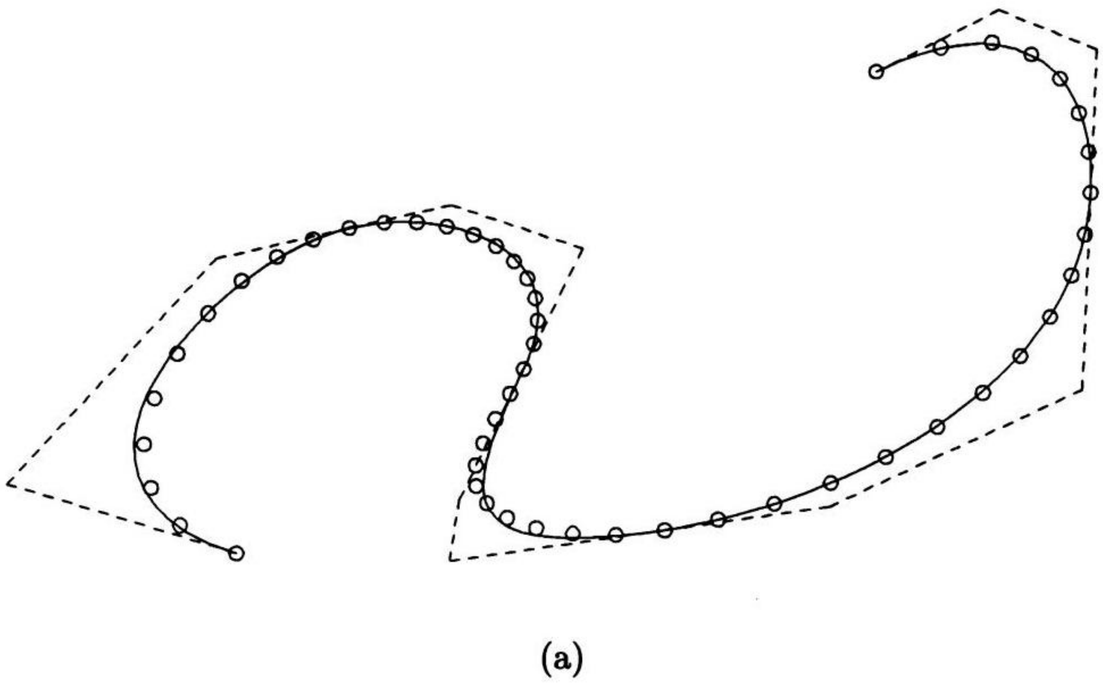
图 9.57 和 9.58 分别展示了平面和非平面拟合的示例。图 9.57 的输入数据为

图 9.57a $E = 1/10,\ K_{\max} = 11$

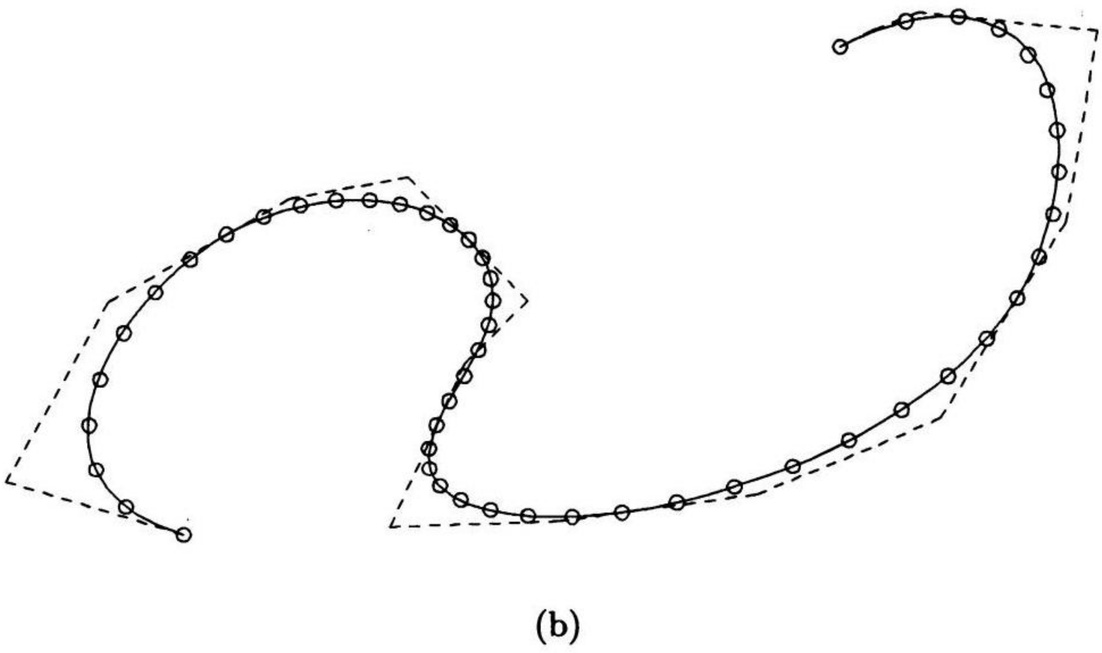
图 9.57 b $E = 1/10,\ K_{\max} = 8$

图 9.57c $E = 1/100,\ K_{\max} = 11$

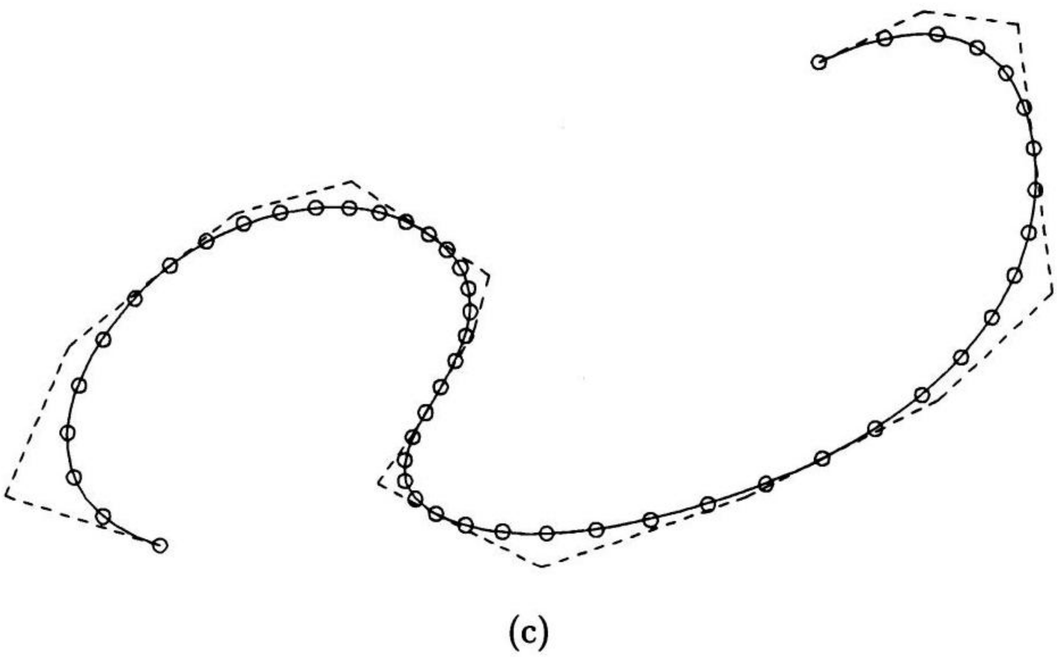
注意从图 9.57a 到图 9.57b 近似质量的变化。在相同的容差下，允许每个段最多八个点而不是十一点显著改善了近似效果。图 [latex0] 的输入数据为



(a)



(b)



(c)

图 9.57. 对平面数据的局部三次近似。

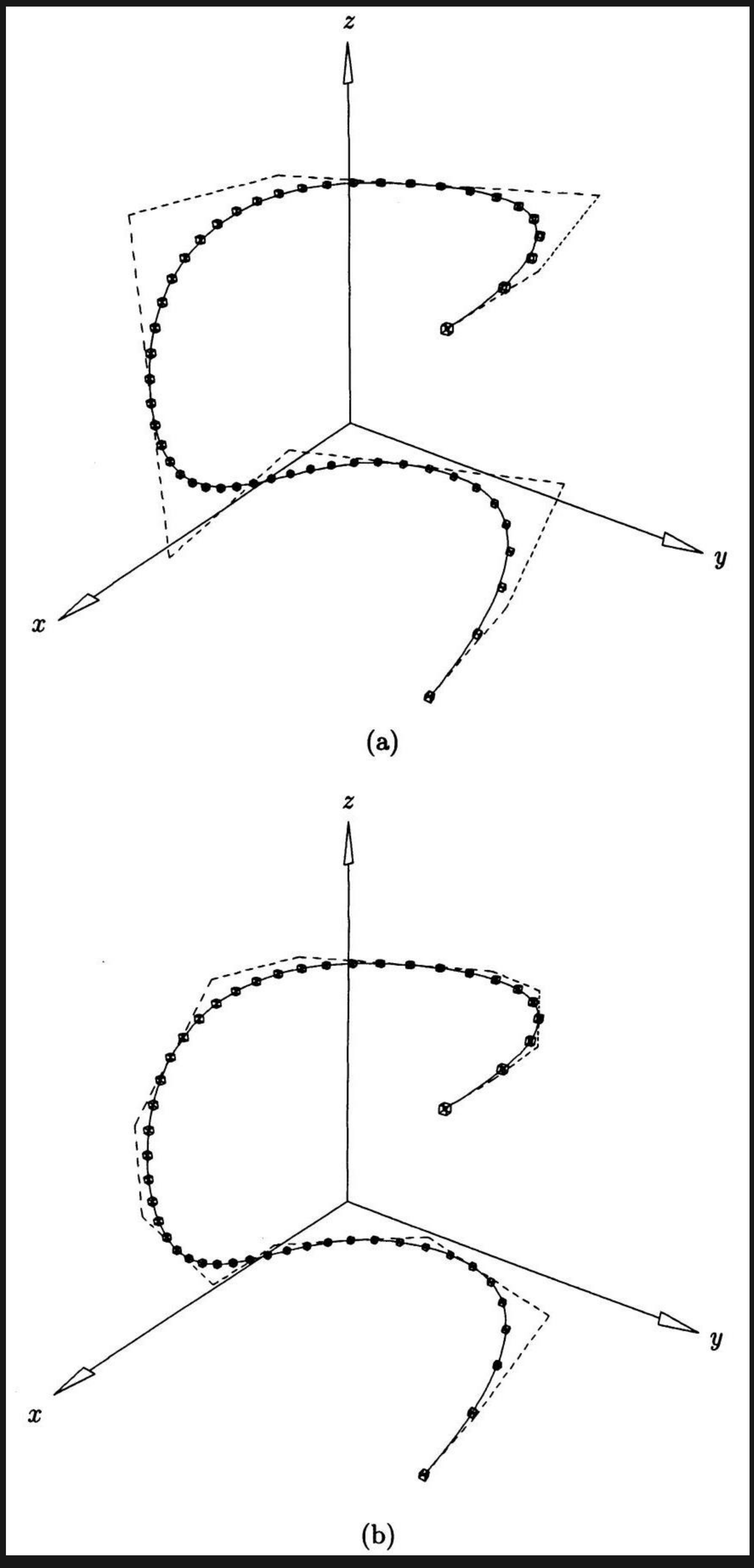


图 9.58. 对包含平面段的三维数据的局部三次近似。

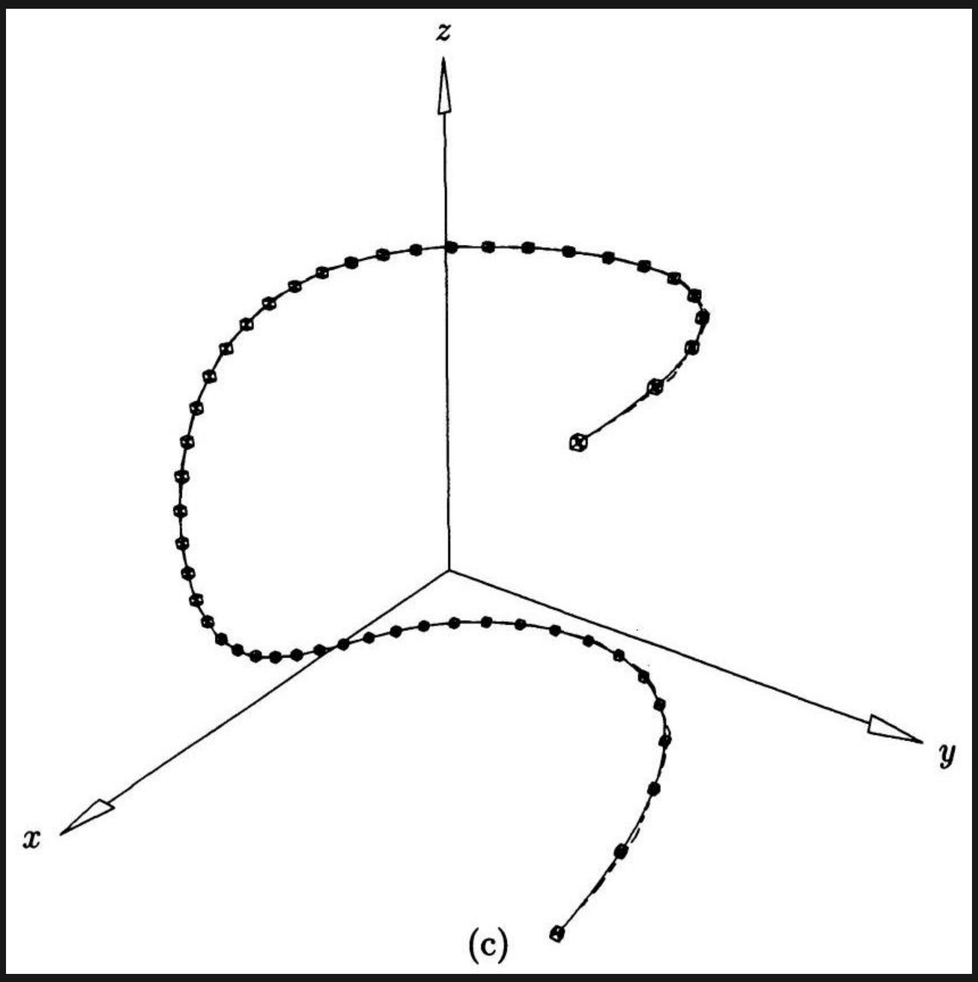


图 9.58. (续。)

图 9.58a – 9.58c 的输入数据为

图 9.58a $E = 1/10,\ K_{\max} = 20$

图 9.58 b $E = 1/100,\ K_{\max} = 20$

图 9.58c $E = 0,\ K_{\max} = 20$

零容忍导致了一个插值曲线。给定 51 个点，人们可能期望有 50 个线段。然而，由于可以始终构造一个立方体通过一个内部点，因此只需要 25 个线段。

练习

9.1. 你想用一个二次曲面 ($p = q = 2$) 对点 $\mathbf{Q}_{k,\ell}, k, \ell = 0, 1, 2, 3$ 进行插值。不要使用重复的曲线插值，而是直接使用公式 (9.25) 来建立一个包含 16 个未知数 $\mathbf{P}_{i,j}$ 的 16 个线性方程的系统。

9.2. 假设 α_k 是使用公式 (9.31) 计算的，考虑公式 (9.29) 的未归一化 $\mathbf{V}_k$ 。说服自己，如果 $\Delta\bar{u}_k$ 是使用公式 (9.5) 计算的 (归一化相对弦长)，则量 $\mathbf{V}_k/\Delta\bar{u}_k$ 是导数的良好估计。计算所有的 $\{\mathbf{V}_k\}, k = 0, \ldots, 4$ ，对于 $\{\mathbf{Q}_k\} = \{(0,0), (0,20), (20,40)$ ，$(30,30), (30,20)\}$ 。它们的大小是多少？根据公式 (9.5) 计算总弦长和 $\Delta\bar{u}_k$ 。量 $\mathbf{V}_k/\Delta\bar{u}_k$ 看起来像是导数的合理估计吗？练习 453

9.3. 绘制如图 9.48 和 9.49 所示的图形，并确定在两个二次情况下必须重新计算哪些 B_r 值和参数索引范围: $U = \{0, 0, 0, 1, 2$ ，$3, 4, 5, 6, 6, 6\}$ ，具有 $r = 5$ 和 $s = 1$ ；以及 $U = \{0, 0, 0, 1, 2, 3, 3, 4, 5, 6, 6, 6\}$ ，具有 $r = 6$ 和 $s = 2$ 。

第十章　高级表面构造技术

1 引言

在本章中，我们将讨论几种高级表面构造技术，即摆动表面、皮肤表面、扫掠表面、戈登表面和库恩斯表面。粗略来说，本章的思路是取一条或两条曲线，或一组曲线，创建一个插值这些曲线的 NURBS 表面；即，给定的曲线是在 NURBS 表面上的等参数曲线。请注意，这与第 9 章中介绍的对离散点和导数数据的插值在根本上是不同的。在某些构造中，读者可能不会明显意识到所创建的表面确实满足所需的约束。我们将其留作读者的练习，以通过数学证明或软件实现来使自己信服。

2 摆动表面

摆动表面是旋转曲面的推广。设

$$\mathbf{P}(u)=\sum_{i=0}^{n}R_{i,p}(u)\,\mathbf{P}_i \tag{10.1}$$

是定义在 xz 平面上的轮廓曲线，并设

$$\mathbf{T}(v)=\sum_{j=0}^{m}R_{j,q}(v)\,\mathbf{T}_j \tag{10.2}$$

是定义在 xy 平面上的轨迹曲线 (见图 10.1)。将 $\mathbf{P}(u)$ 和 $\mathbf{T}(v)$ 的非零坐标函数分别记为 $P_x(u),P_z(u),T_x(v)$ 和 $T_y(v)$，我们通过 [Wood87] 定义摆动表面

$$\mathbf{S}(u,v)=(\alpha P_x(u)T_x(v),\alpha P_x(u)T_y(v),P_z(u)) \tag{10.3}$$

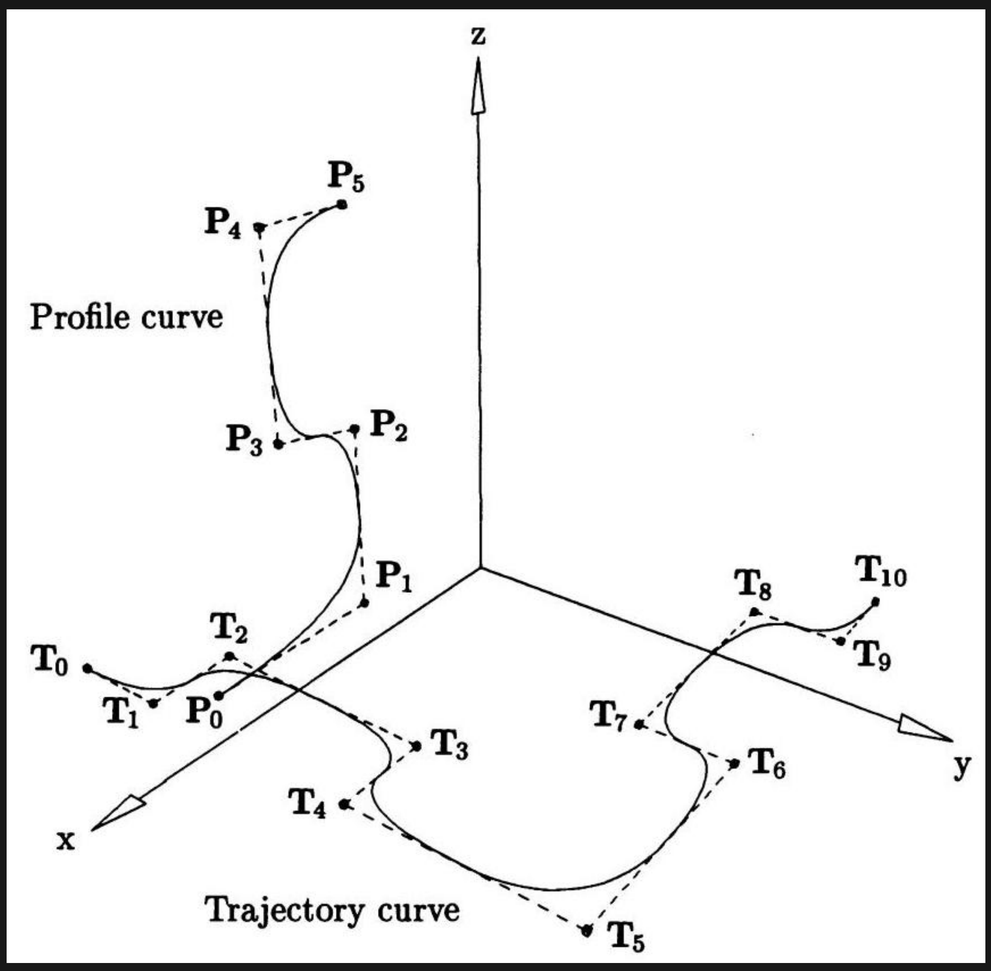


图 10.1. 用于定义摆动表面的轮廓曲线和轨迹曲线。

从几何上讲，$\mathbf{S}(u,v)$ 是通过围绕 z 轴摆动 $\mathbf{P}(u)$ 并根据 $\mathbf{T}(v).\alpha$ (一个任意缩放因子) 同时进行缩放而获得的。固定 u 会产生形状为 $\mathbf{T}(v)$ 的曲线，但在 x 和 y 方向上进行了缩放。固定 $v=v_0$，等参曲线 $\mathbf{C}_{v_0}(u)$ 是通过将 $\mathbf{P}(u)$ 旋转到包含向量 $(T_x(v_0),T_y(v_0),0)$ 的平面中，并用因子 $\alpha\,|\mathbf{T}(v_0)|$ 缩放旋转曲线的 x 和 y 坐标而获得的。z 坐标保持不变。根据 NURBS 的变换不变性特性 (P4.24)，可以得到 $\mathbf{S}(u,v)$ 的 NURBS 表示如下:

$$\mathbf{S}(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{m}R_{i,p;j,q}(u,v)\,\mathbf{Q}_{i,j} \tag{10.4}$$

其中

$$\mathbf{Q}_{i,j}=(\alpha P_{i,x}T_{j,x},\alpha P_{i,x}T_{j,y},P_{i,z}) \tag{10.5}$$

和

$$w_{i,j}=w_iw_j \tag{10.6}$$

U 和 V 的节点向量用于 $\mathbf{S}(u,v)$，它们定义了 $\mathbf{P}(u)$ 和 $\mathbf{T}(v)$。图 10.2a – 10.2c 显示了摆动的示例。图 10.2a 显示了由图 10.1 中的曲线定义的摆动表面的控制点，使用了 $\alpha=0.12$。图 10.2b 中的表面是非理性的，而图 10.2c 中的表面是通过应用权重获得的。

$$w_1^P=w_4^P=5\;w_1^T=w_4^T=w_6^T=w_9^T=5$$

其中上标 P 和 T 分别指代轮廓曲线和轨迹曲线。

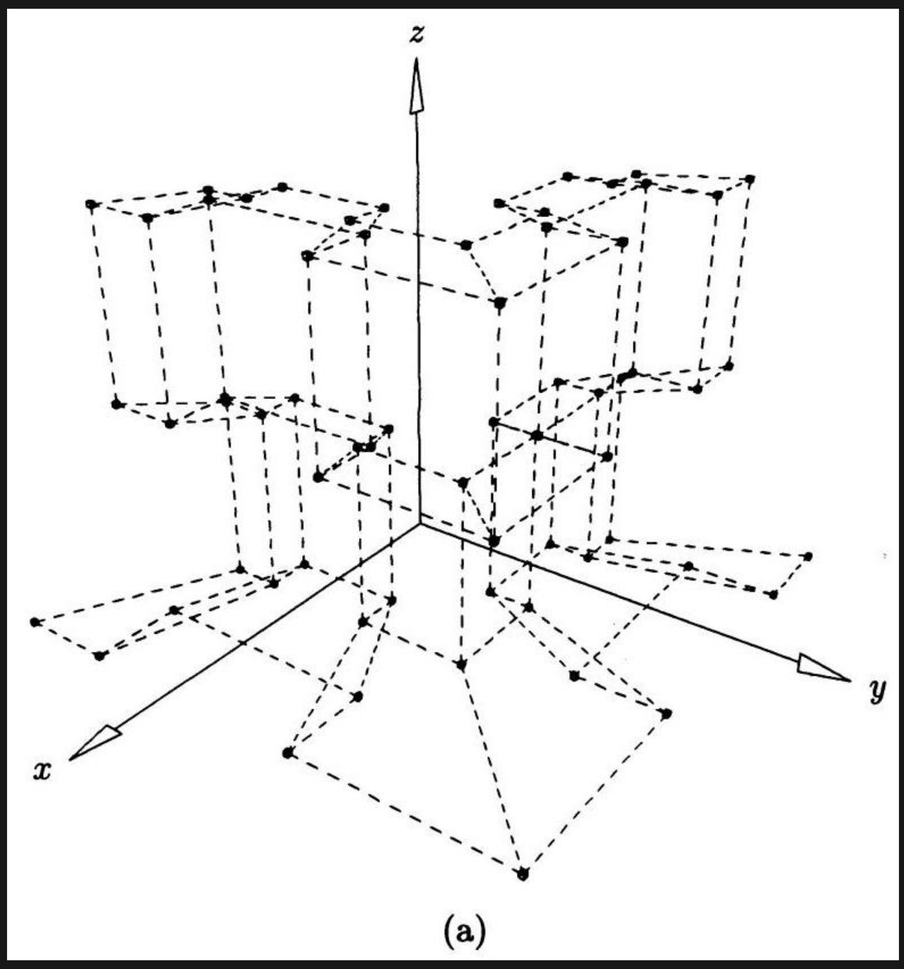


图 10.2. 摆动表面。(a) 控制网；(b) 未应用权重的摆动表面；(c) 应用了一些权重的摆动表面。

注意，$\mathbf{P}(u)$ 和 $\mathbf{T}(v)$ 可以是开放或封闭的；相应地，$\mathbf{S}(u,v)$ 在 u 和 v 方向上也可以是开放或封闭的。选择 $\alpha=1$ 和 $\mathbf{T}(v)$ 为以原点为中心、半径为 1 的圆，得到第 8 章中给出的旋转曲面。

3　皮肤表面

设 $\{\mathbf{C}_k(u)\}, k=0,\ldots,K$ 为一组曲线。我们称 $\mathbf{C}_k(u)$ 为截面曲线。在实践中，它们通常是要构建的表面的 u 方向的平面截面。然而，它们也可以是三维的。此外，我们在本节中不假设平面性。表皮化是将截面曲线融合在一起以形成表面的过程。融合方向是 v 方向，有时称为纵向方向。尽管可以在截面曲线之间使用近似，但表皮化方法通常通过 $\mathbf{C}_k(u)$ 进行插值，结果是 $\mathbf{C}_k(u)$ 在生成的表皮曲面上是等参数曲线。

表皮化实际上是一个较新的术语，指的是早在计算机出现之前就存在的放样技术。它在造船、汽车和航空工业中被广泛使用。最早的计算机化系统之一是 CONSURF，由 A. Ball 在英国航空公司开发 [Ball74, 75, 77]。该系统基于有理三次曲线。

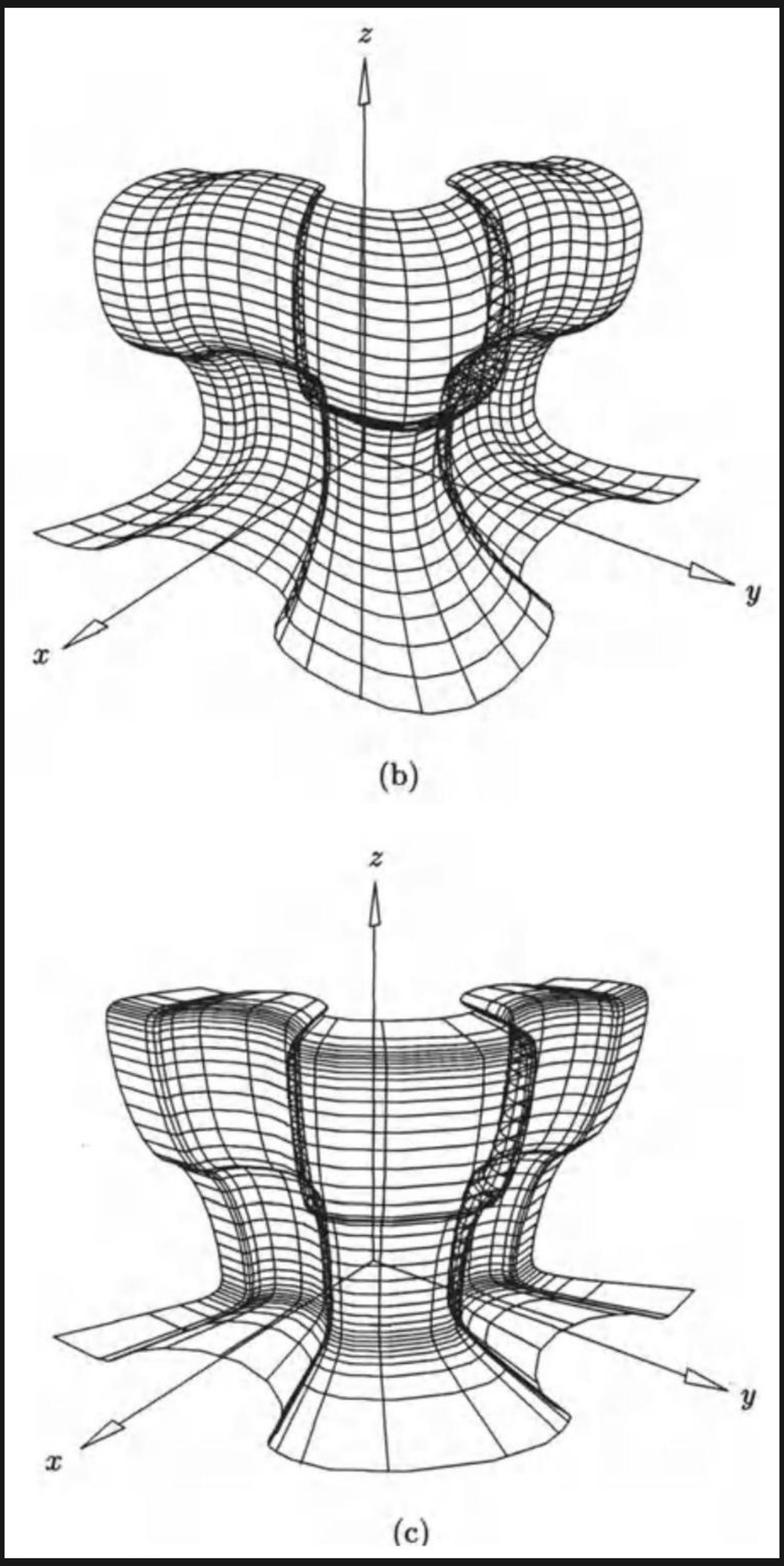


图 10.2.(续)

曲线。Filip 和 T. Ball [Fili89] 描述了一种基于任意参数截面曲线的放样程序方法。关于使用 B 样条进行表皮化的详细描述见参考文献 [Till83; Wood87, 88; Hohm91]。

基于 B-splines，我们定义皮肤化如下。设

$$\mathbf{C}_k^w(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{P}_{i,k}^w \; k = 0, \ldots, K \tag{10.7}$$

为有理或非有理的截面曲线。我们假设所有 $\mathbf{C}_k^w(u)$ 都定义在相同的节点向量 U 上，并具有共同的度数 p。如有必要，可以将曲线带到共同的 p 和 U，如第 8.4 节所述 (实际上，规则曲面是皮肤化曲面的特例)。然后，在 v 方向选择一个度数 q，并计算参数 $\{\bar{v}_k\}, k = 0, \ldots, K$ 和节点向量 V。这些随后用于在截面曲线的控制点之间进行 $n+1$ 曲线插值，从而得到皮肤化曲面的控制点 $\mathbf{Q}_{i,j}^w$。因此，$\mathbf{Q}_{i,j}^w$ 是通过 $\mathbf{P}_{i,0}^w, \ldots, \mathbf{P}_{i,K}^w$ 的插值曲线的 j 控制点。请注意，如果即使一个 $\mathbf{C}_k^w(u)$ 是有理的，那么通过 $\mathbf{P}_{i,k}^w$ 的 v 方向插值将在四维空间中进行；否则，将插值三维点 $\mathbf{P}_{i,k}$。

图 10.3a – 10.3 d 显示了皮肤化过程。原始截面曲线在图 10.3a 中显示，并定义在节点向量上。

$$\mathbf{C}_0(u): \; \{0, 0, 1, 1\} \; (p = 1)$$

$$\mathbf{C}_1(u): \; \left\{0, 0, 0, \frac{3}{10}, \frac{1}{2}, \frac{7}{10}, 1, 1, 1\right\} \; (p = 2)$$

$$\mathbf{C}_2(u): \left\{0, 0, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, 1, 1\right\} \; (p = 1)$$

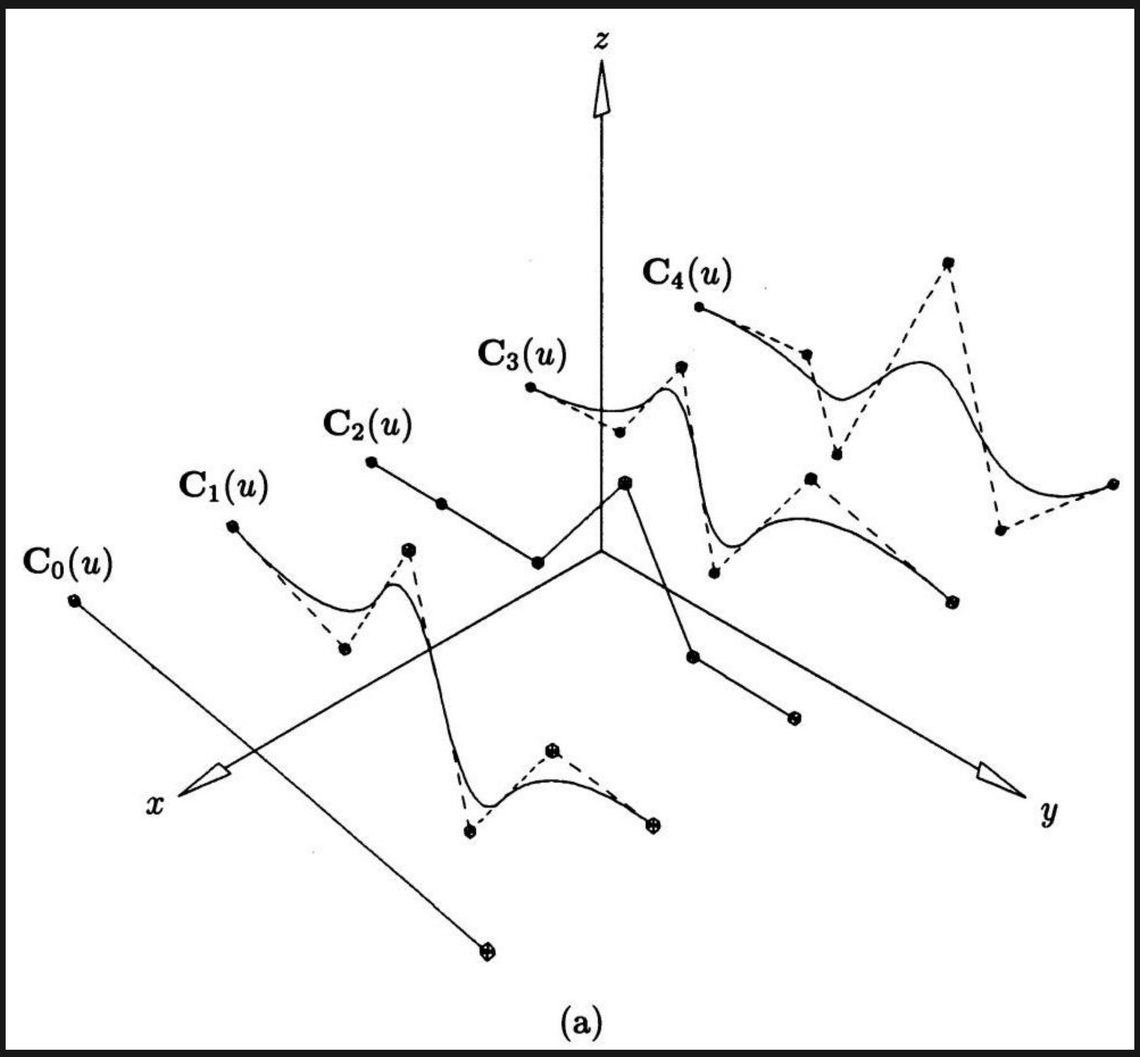
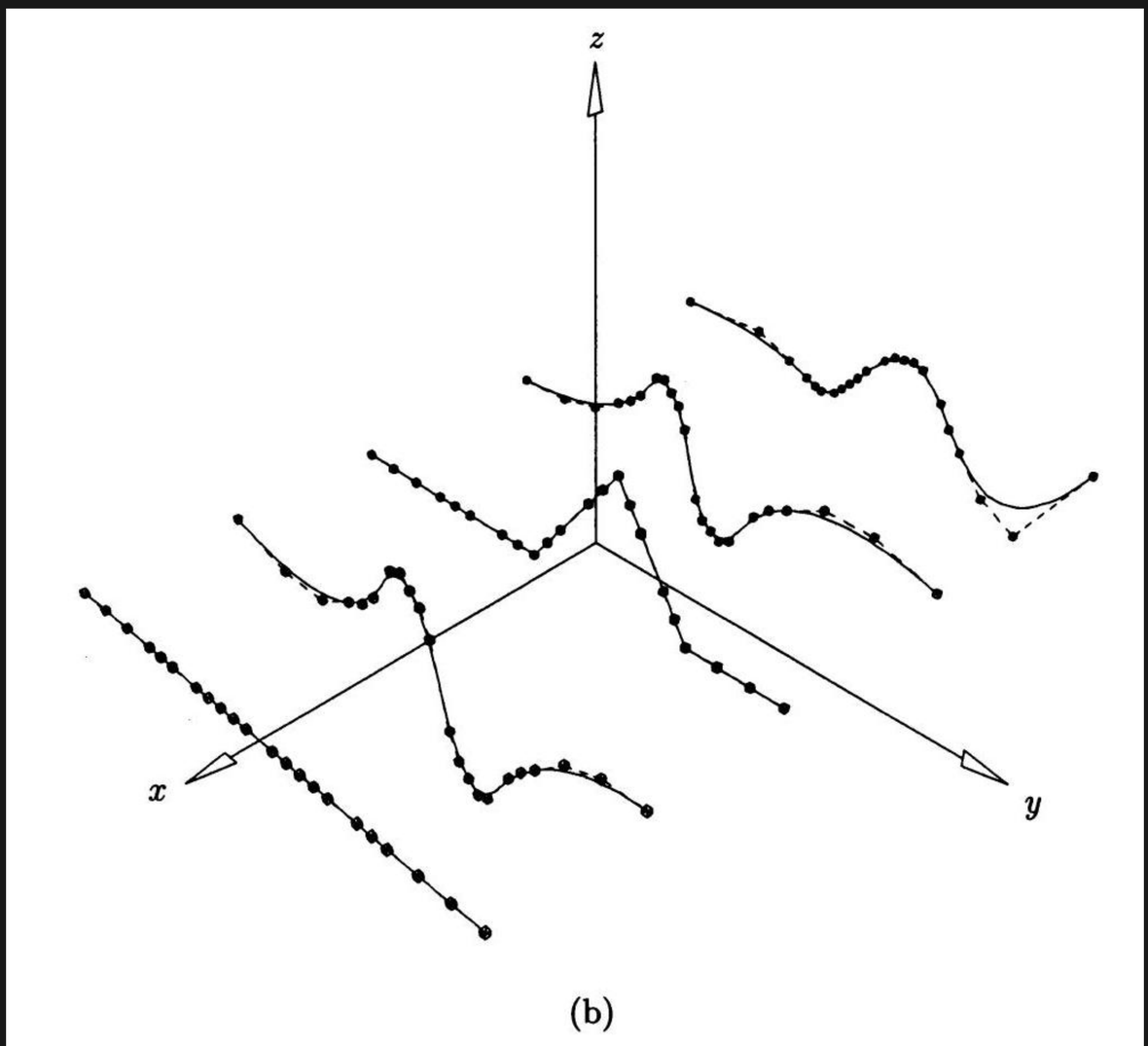


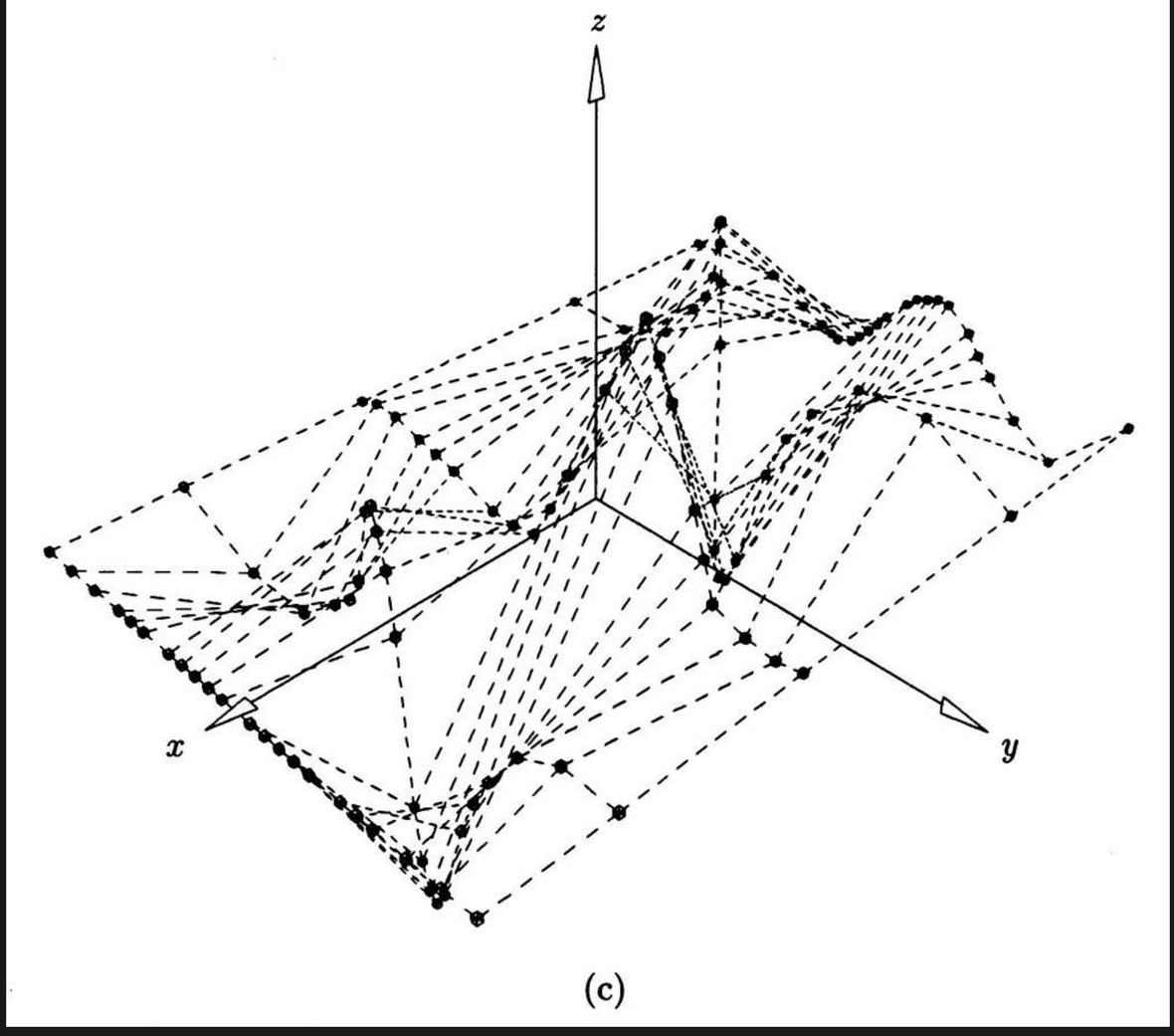
图 10.3. 表面皮肤化的过程。(a) 截面曲线；(b) 兼容的截面曲线；(c) 控制点；(d) 皮肤化曲面。

$$\mathbf{C}_3(u): \left\{0,0,0,\frac{3}{10},\frac{1}{2},\frac{7}{10},1,1,1\right\} \quad (p=2)$$

$$\mathbf{C}_4(u): \left\{0,0,0,0,\frac{3}{10},\frac{7}{10},1,1,1,1\right\} \quad (p=3)$$



(b)



(c)

图 10.3.(续。)

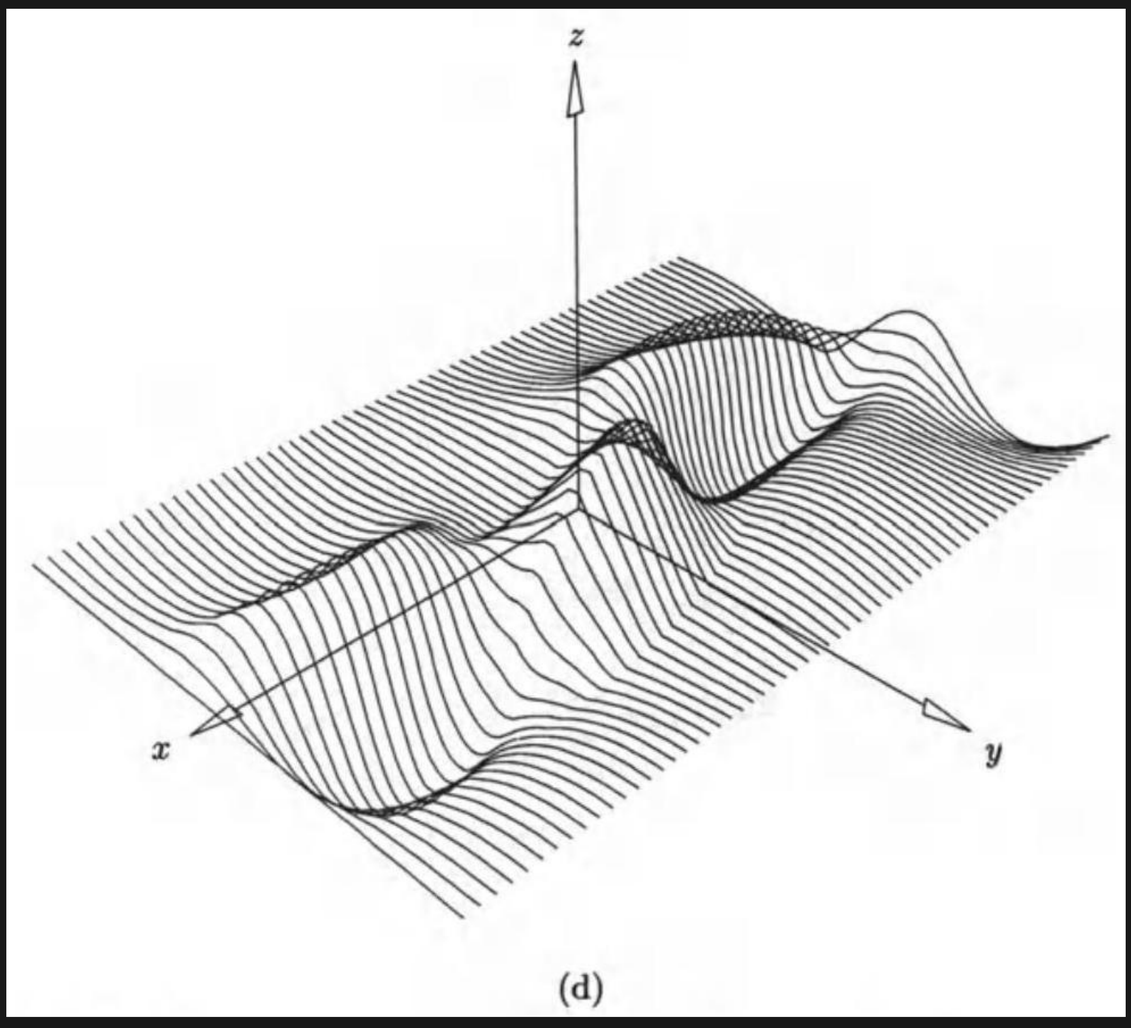


图 10.3.(续。)

在提升度数和节点细化后，公共节点向量为

$$\left\{0,0,0,0,\frac{1}{5},\frac{1}{5},\frac{1}{5},\frac{3}{10},\frac{3}{10},\frac{2}{5},\frac{2}{5},\frac{2}{5},\frac{1}{2},\frac{1}{2},\right.$$

$$\left.\frac{3}{5},\frac{3}{5},\frac{3}{5},\frac{7}{10},\frac{7}{10},\frac{4}{5},\frac{4}{5},\frac{4}{5},1,1,1,1\right\}$$

这将 u 方向上的控制点数量增加到 22。兼容的曲线如图 10.3b 所示。请注意新控制点的显著数量，这是灵活性所付出的代价。图 10.3c 显示了图 10.3d 中描绘的皮肤表面的控制网。

尽管公式简单，但实现一个稳健且可用的皮肤能力相当困难；而且交互式设计皮肤表面通常是一个乏味且迭代的过程。表面形状难以控制；它依赖于截面曲线的数量、形状和位置 (这些通常由交互设计师控制)，以及用于 v 方向插值的方法 (通常嵌入在软件中，设计师无法控制)。设计过程通常从少量截面曲线开始，然后通过迭代进行皮肤处理，进行平面切割以获得额外的截面曲线，修改新曲线以获得更理想的形状，并重新进行皮肤处理。最终，皮肤表面可能会出现不必要的自交和扭曲。然而，尽管存在这些困难，皮肤处理仍然是一种强大且广泛使用的表面设计技术。

现在我们将注意力转向 v 方向的插值。我们暂时假设所有截面曲线都是非有理的；在有理截面上进行皮肤处理会出现一系列问题，我们将在本节末尾回到这些问题。必须确定三个项目: 度数 q 、参数 $\{\bar{v}_k\}$ 和节点 $V=\{v_i\}$ 。度数 q 是任意的，唯一的限制是 $q \leq K$ 。节点 $\bar{v}_k$ 通过平均计算，例如

$$\bar{v}_0=0 \; \bar{v}_K=1$$

$$\bar{v}_k=\bar{v}_{k-1}+\frac{1}{n+1}\sum_{i=0}^{n}\frac{|\mathbf{P}_{i,k}-\mathbf{P}_{i,k-1}|}{d_i} \; k=1,\ldots,K-1 \tag{10.8}$$

其中 d_i 表示 $\mathbf{P}_{i,0},\ldots,\mathbf{P}_{i,K}$ 的总弦长。然后使用公式 (9.8) 计算节点 $\{v_i\}$ 。图 10.3d 的皮肤表面是使用这种插值方法构建的。

另一种常见的方法是使用所谓的脊曲线 (也称为路径曲线)，即

$$\mathbf{C}_s(v)=\sum_{i=0}^{n_c}R_{i,p_c}(v)\,\mathbf{R}_i \tag{10.9}$$

$\mathbf{C}_s(v)$ 可以是任意有理或无理的、平面或非平面曲线。设计师使用脊曲线在空间中定位和定向截面曲线。图 10.4 显示了脊曲线和五条平面截面曲线在三维空间中的定位。图 10.5 中展示了一种可能的定位方式。截面曲线在脊参数处定位

$$\hat{v}_k=\{0,0.25,0.5,0.78,1\}$$

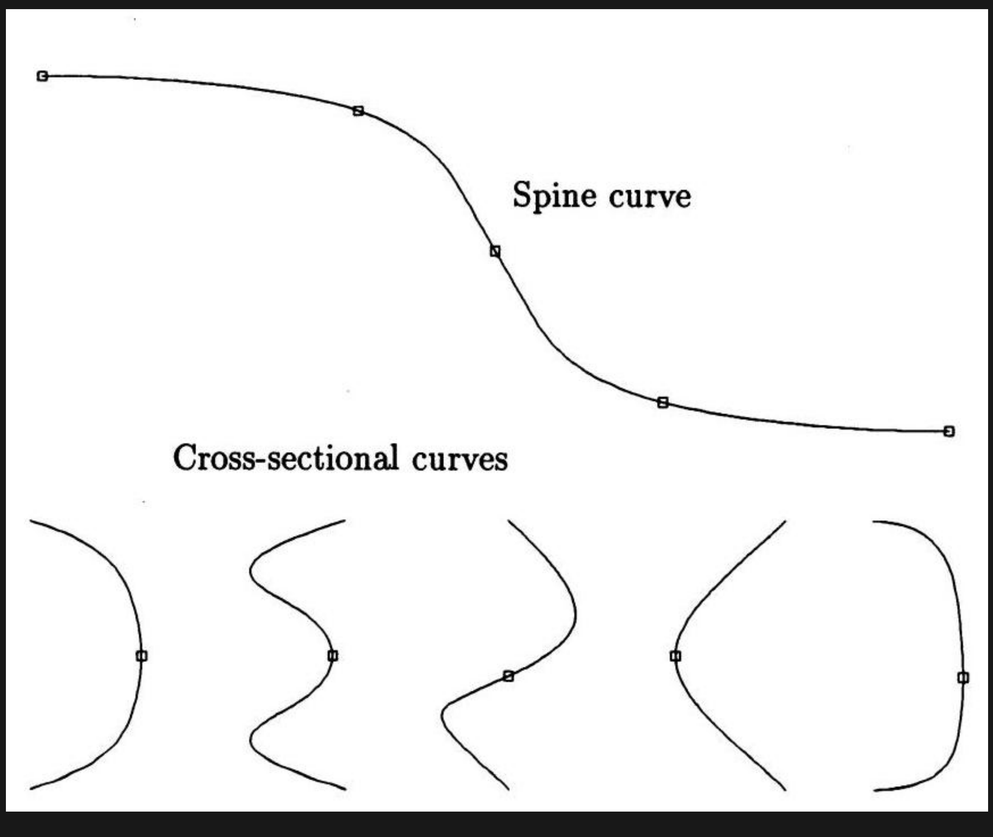


图 10.4. 基于脊的皮肤的脊曲线和横截面曲线。

并在截面曲线参数处

$$\widehat{u}_k = \left\{\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right\}$$

每条截面曲线的方向使其平面与脊曲线与截面平面交点处的切向量垂直。脊曲线还可以通过提供额外的约束来帮助 v 方向的形状控制，以用于 v 方向的插值。例如，可以使用脊切向方向: 设 $\{\bar{v}_k\}, k = 0, \ldots, K$ 为截面曲线定位时 $\mathbf{C}_s(v)$ 上点的 v 值；对于平面截面，这些是与脊与截面平面交点对应的 v 值。这些值通常是已知的，可以输入到皮肤处理程序中。请注意，通常情况下，如果使用公式 (10.8) 计算 $\bar{v}_k$ ，则 $\bar{v}_k \neq \bar{v}_k$ 。

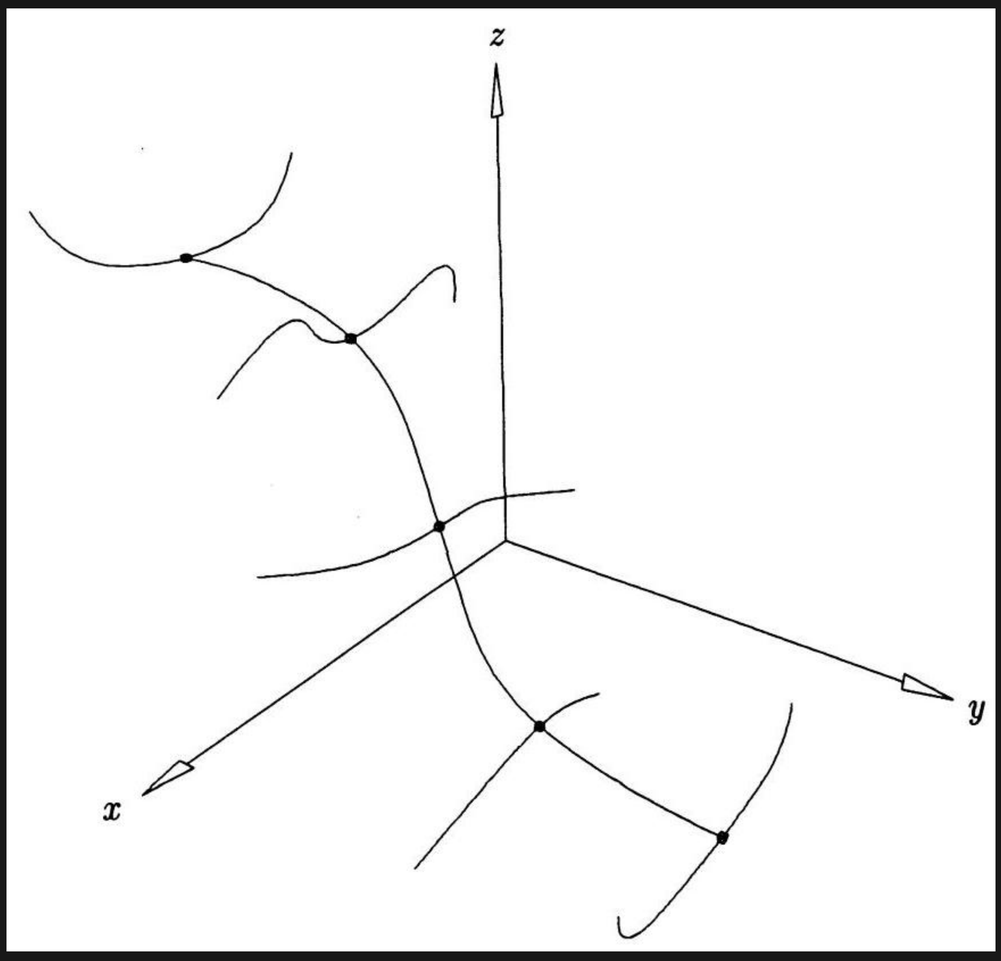


图 10.5. 沿脊曲线定位的截面曲线。

当然，可以设置 $\bar{v}_k = \widehat{v}_k$ ，但如果 $\mathbf{C}_s(v)$ 参数化不良，这可能不是一个好主意。现在让

$$\mathbf{P}_k = \mathbf{C}_s(\widehat{v}_k) \;\; \mathbf{D}_k = \mathbf{C}_s'(\widehat{v}_k) \;\; k = 0, \ldots, K$$

然后对于每个 i 和 k 我们计算一个导数向量 $\mathbf{D}_{i,k}$ 。对于每个 i 这产生了 $2(K+1)$ 插值约束 $\mathbf{P}_{i,0}, \ldots, \mathbf{P}_{i,K}$ 和 $\mathbf{D}_{i,0}, \ldots, \mathbf{D}_{i,K}$ 。通过让

$$d_k = |\mathbf{P}_k - \mathbf{P}_{k-1}| \;\; k = 1, \ldots, K$$

(见图 10.6)。然后对于 $k = 1, \ldots, K-1$ ，设置

$$\mathbf{D}_{i,k} = \frac{|\mathbf{P}_{i,k+1} - \mathbf{P}_{i,k}| + |\mathbf{P}_{i,k} - \mathbf{P}_{i,k-1}|}{d_{k+1} + d_k}\mathbf{D}_k \tag{10.10}$$

在两端

$$\mathbf{D}_{i,0} = \frac{|\mathbf{P}_{i,1} - \mathbf{P}_{i,0}|}{d_1}\mathbf{D}_0$$

$$\mathbf{D}_{i,K} = \frac{|\mathbf{P}_{i,K} - \mathbf{P}_{i,K-1}|}{d_K}\mathbf{D}_K \tag{10.11}$$

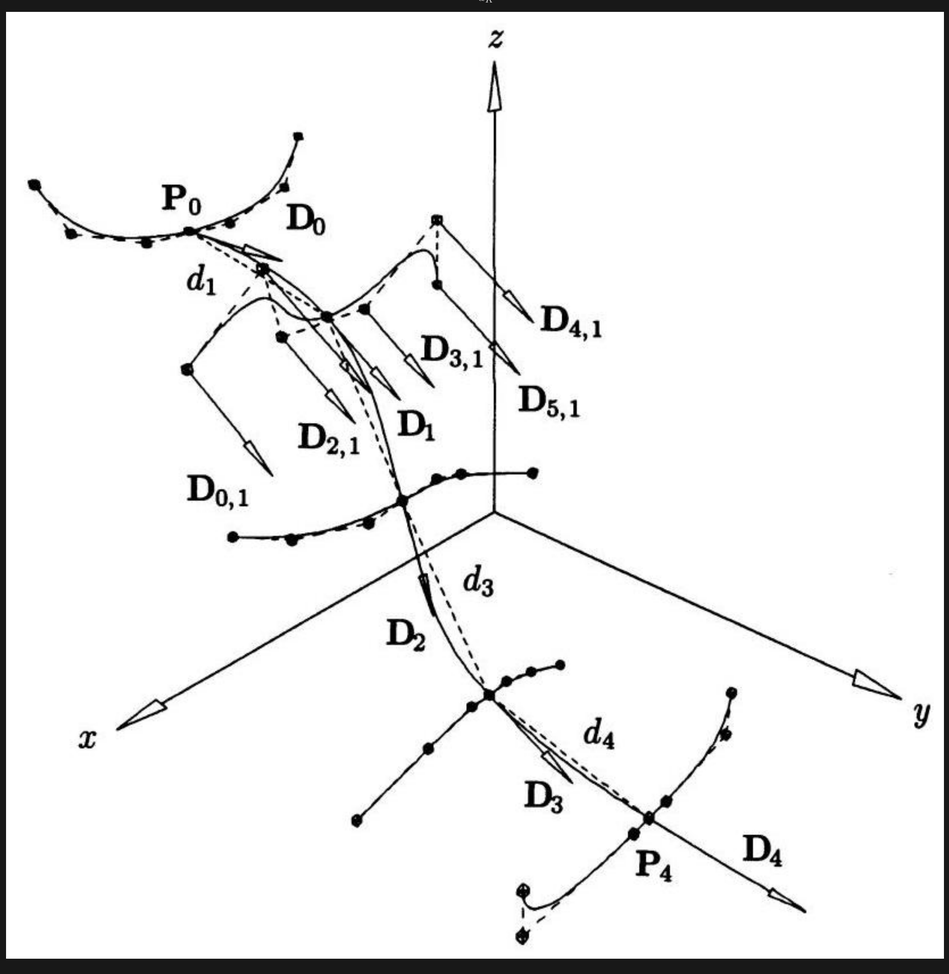


图 10.6. 脊椎蒙皮的导数计算。

因此，$\mathbf{D}_{i,k}$ 与 $\mathbf{D}_k$ 平行，但其大小调整以反映相对距离。方程 (10.10) 和 (10.11) 确保

$$\mathbf{S}_v(u, \bar{v}_k) \parallel \mathbf{D}_k \;\; \text{for all } u \text{ and } k = 0, \ldots, K \tag{10.12}$$

即，关于 v 的所有偏导数在 k 的同参数曲线上与 $\mathbf{D}_k$ 平行。这是因为这些偏导数是 $\mathbf{D}_{i,k}$ 的线性组合。让 $m=2K+1$，我们现在可以为每个固定的 i 设置 $m+1$ 个线性方程（见方程 [9.19] 和 [9.20]），即

$$\mathbf{P}_{i,k}=\sum_{j=0}^{m}N_{j,q}\left(\bar{v}_k\right)\mathbf{Q}_{i,j}\ \mathbf{D}_{i,k}=\sum_{j=0}^{m}N'_{j,q}\left(\bar{v}_k\right)\mathbf{Q}_{i,j} \tag{10.13}$$

节点向量 V 必须包含 $m+q+2$ 个节点；方程 (9.22) 和 (9.23) 为情况 $q=2$ 和 $q=3$ 提供了合适的选择。

图 10.7a 显示了由图 10.5 中的数据定义的蒙皮表面的控制点，使用了如图 10.6 所示的指定切线的三次插值；该表面在图 10.7b 中描绘。相同的数据在图 10.8a 和 10.8b 中进行了插值，没有指定导数。注意表面形状的显著差异（图 10.7b 和 10.8b）。截面曲线的位置对表面形状有明确的影响，就像曲线和表面插值中数据点的位置一样；图 10.9a 和 10.9 b 显示了两个例子。在图 10.9a 中，脊和截面曲线参数是

$$\widehat{v}_k=\{0,0.35,0.5,0.62,1\}\ \widehat{u}_k=\left\{\frac{1}{2},\frac{1}{2},\frac{1}{2},\frac{1}{2},\frac{1}{2}\right\}$$

而在图 10.9b 中它们是

$$\widehat{v}_k=\{0,0.25,0.5,0.78,1\}\ \widehat{u}_k=\left\{\frac{1}{2},\frac{3}{10},\frac{1}{2},\frac{9}{10},\frac{1}{2}\right\}$$

现在我们考虑一些截面曲线为有理的情况（方程 [10.7]）。现在必须在齐次空间中进行 v 插值。类似于方程 (10.8)，$\bar{v}_k$ 通过平均四维距离计算，即

$$\bar{v}_0=0\ \bar{v}_K=1$$

$$\bar{v}_k=\bar{v}_{k-1}+\frac{1}{n+1}\sum_{i=0}^{n}\frac{\left|\mathbf{P}^w_{i,k}-\mathbf{P}^w_{i,k-1}\right|}{d^w_i}\ k=1,\ldots,K-1 \tag{10.14}$$

其中 d^w_i 表示 $\mathbf{P}^w_{i,0},\ldots,\mathbf{P}^w_{i,K}$ 的总弦长。节点 $\{v_i\}$ 再次使用方程 (9.8) 计算。通过 $\mathbf{P}^w_{i,k}$ 的插值得到蒙皮表面的控制点 $\mathbf{Q}^w_{i,j},i=0,\ldots,n,j=0,\ldots,K$ 。等参数曲线 $\mathbf{C}^w_{\bar{v}_k}\left(u\right)=\mathbf{S}^w\left(u,\bar{v}_k\right)$ 是原始截面曲线 $\mathbf{C}^w_k\left(u\right)$ 。尽管设计师仍然可以使用脊曲线来定位和定向

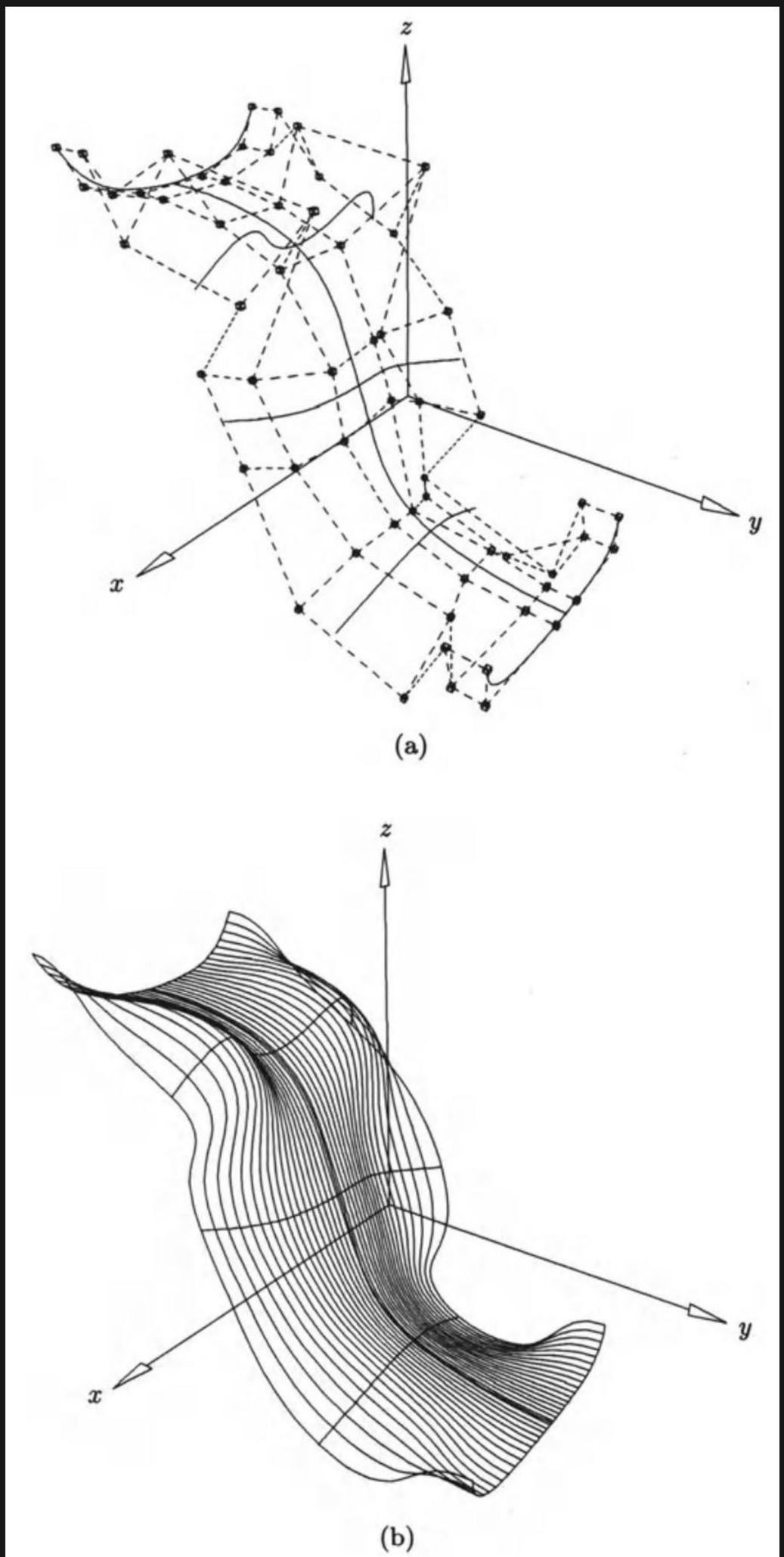


图 10.7. 脊蒙皮与脊控制插值。(a) 控制网；(b) 蒙皮表面。

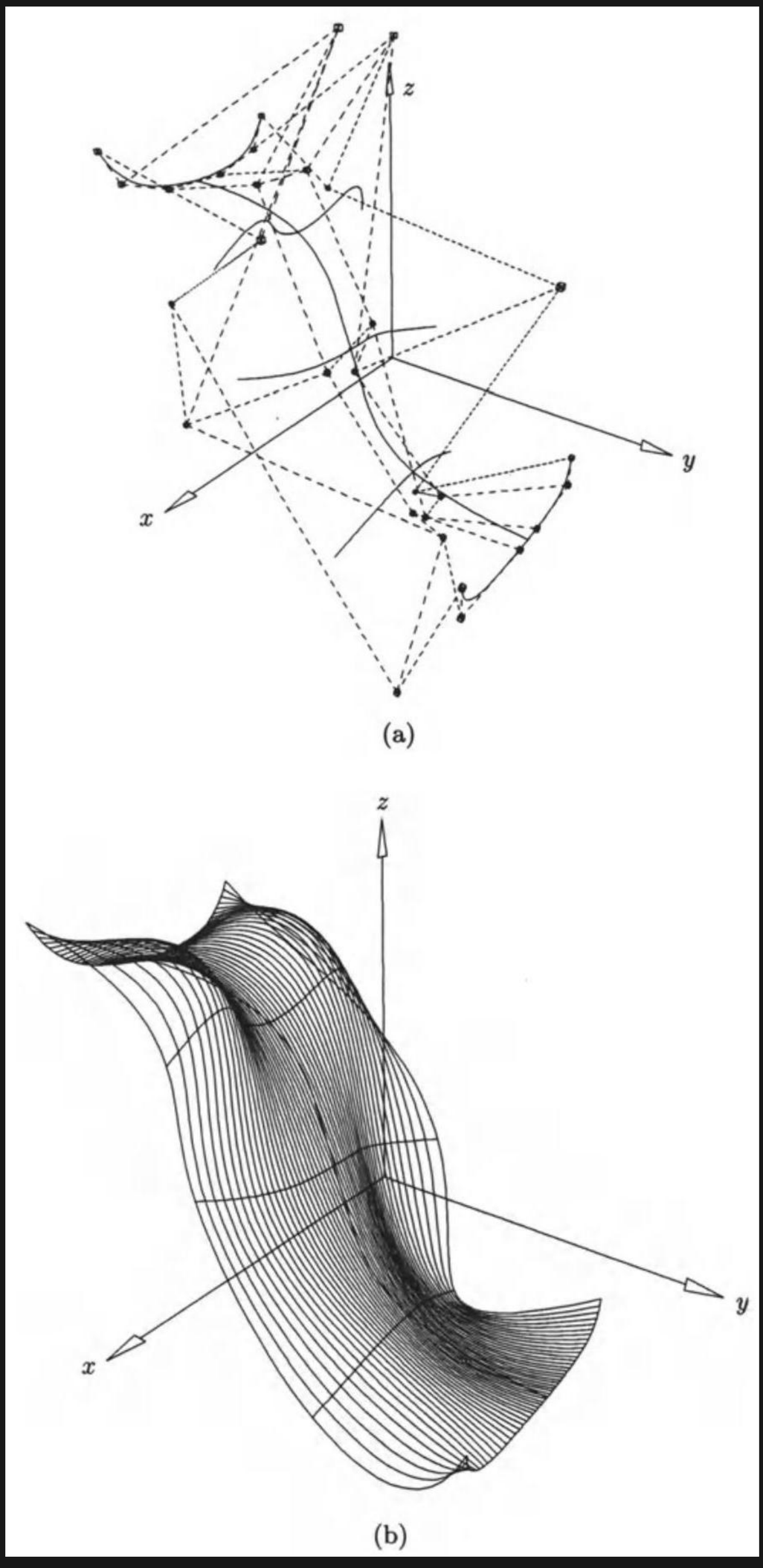
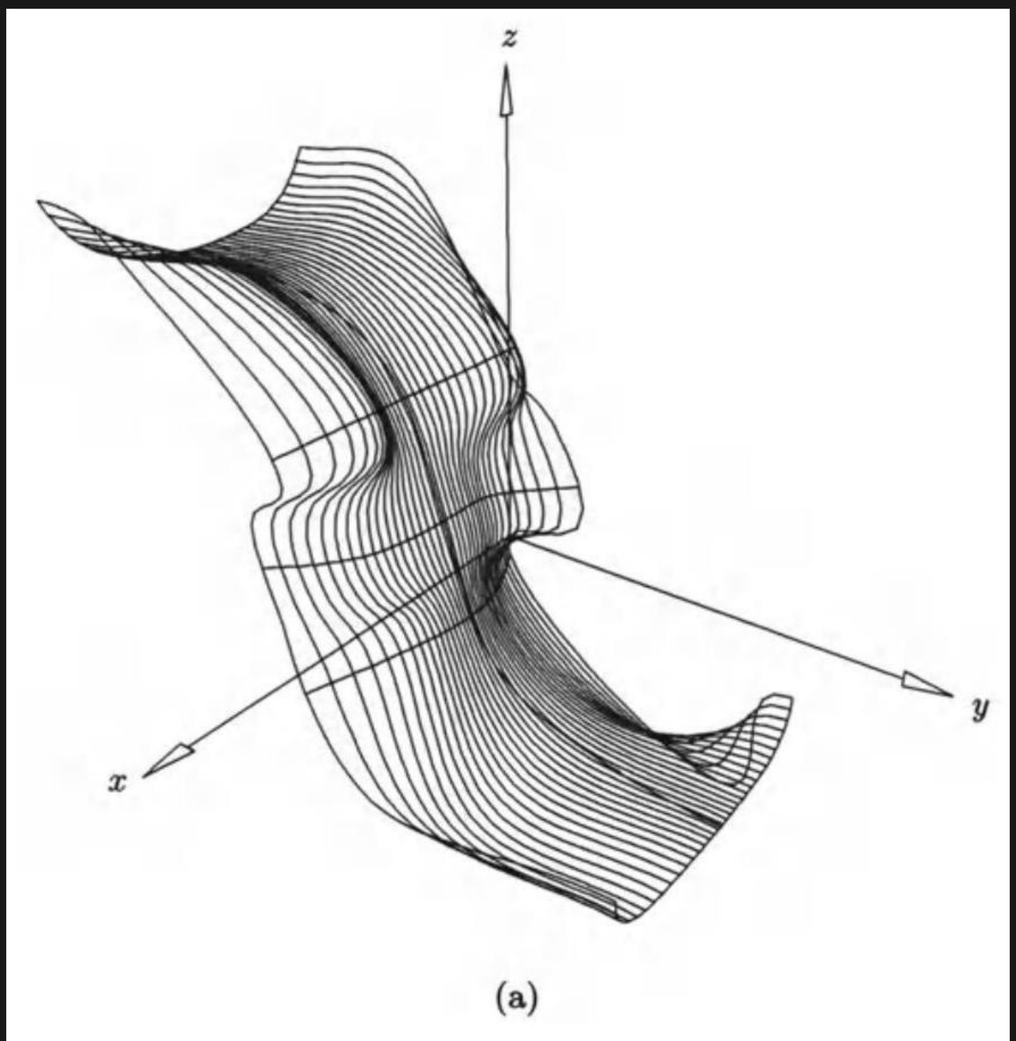


图 10.8. 脊蒙皮没有脊插值约束。(a) 控制网；(b) 蒙皮表面。

在 468 高级表面构造技术部分的曲线中，没有明显的方法可以施加 (线性) 导数约束，以确保 Eq. (10.12) 对所有 u 和 k 都满足。



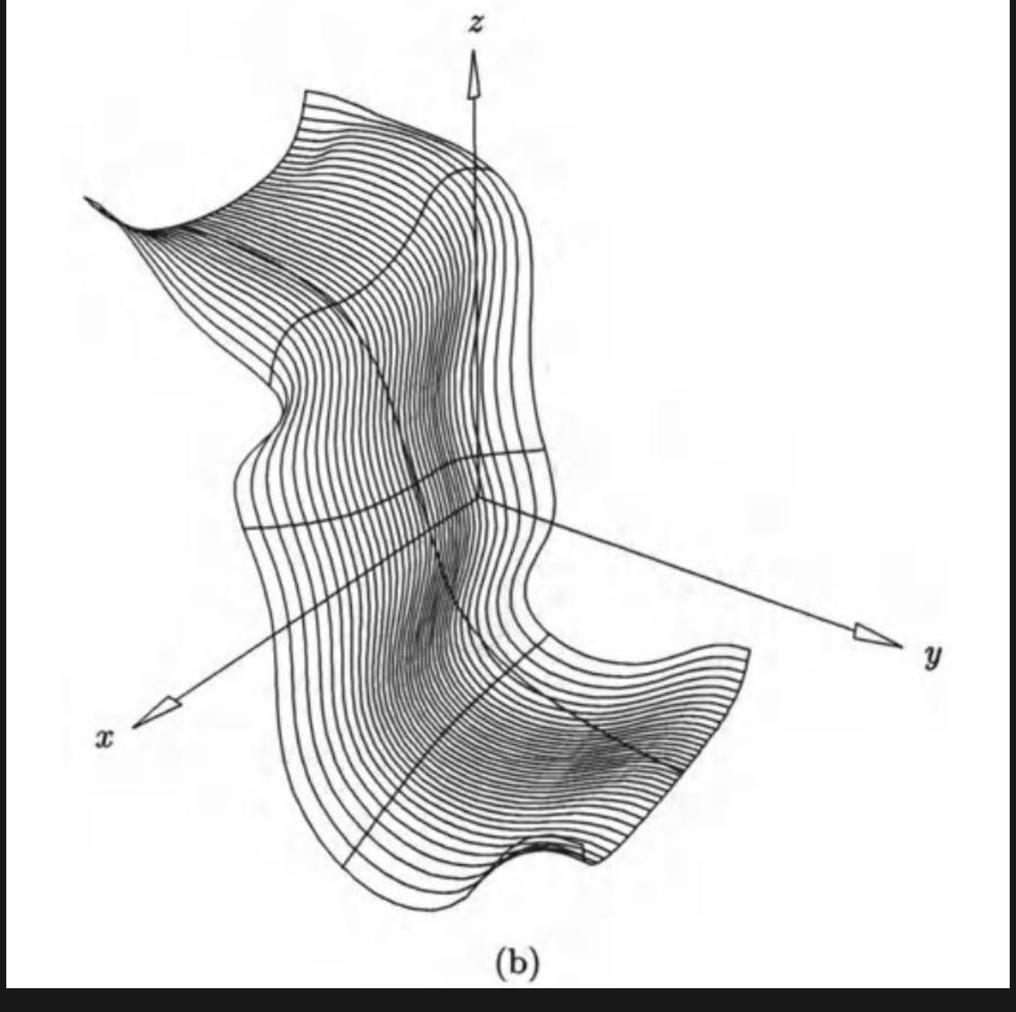


图 10.9. 脊椎皮肤处理。(a) 在不同的脊椎参数下定位的截面曲线；(b) 在不同的截面曲线参数下定位的截面曲线。

通常，通过有理截面曲线进行的皮肤处理能够产生令人满意的结果，但读者应当意识到存在几个潜在的危险，其中一些似乎没有令人满意的实际解决方案。这些问题的产生都源于表面构造发生在均匀空间中，而相应的三维和四维几何形状可能截然不同。例如，两个三维曲线可以是光滑的、形状相似且彼此接近，而它们的四维对应物可能有尖点、形状完全不同且相距甚远。更具体地说，可能出现的一些问题包括：

- v 方向的表面参数化可能相当差。举一个简单的例子，取两条截面曲线，将其中一条的所有控制点乘以一个大权重。这并不会改变三维曲线的几何形状。在两条截面之间的简单规则 (皮肤) 表面在 v 方向上的参数化较差。可以通过使用投影控制点 $\mathbf{P}_{i,k}$ 来改善三维表面参数化，以计算 $\bar{v}_k$ (如 Eq. [10.8] 所示)，但这可能会导致不稳定的四维 v 方向插值，从而在四维和三维空间中产生不理想的几何形状；
- 对于固定的 i ，表面权重 $w_{i,0}, \ldots, w_{i,K}$ 是通过插值截面控制点的权重 $\mathbf{P}^w_{i,0}, \ldots, \mathbf{P}^w_{i,K}$ 得出的。显然，插值可能会产生负值或零值 $w_{i,k}$ ，即使所有截面权重都是正值 (例如，见下文图 10.10b 的解释)。这可能会导致问题，例如在未设计为处理此类权重的系统中崩溃；
- 表面可能并不像截面曲线所指示的那样光滑。例如，设一个截面曲线为示例 Ex7.2 中给出的完整圆。正如该示例所示，圆是 C^1 连续的，尽管其均匀对应物仅在双重节点处是 C^0 连续的。假设所有其他截面曲线都是 C^1 ，人们期望得到的皮肤表面 $\mathbf{S}(u,v)$ 在 u 方向上是 C^1 连续的。但 $\mathbf{S}^w(u,v)$ 在 u 方向上仅是 C^0 连续的，因为它是通过混合 C^0 曲线形成的；一般而言，任意 u 方向的等参数曲线在 $\mathbf{S}^w(u,v)$ 上仅是 C^0 连续的，它们在三维空间中的投影可能仅是 C^0 连续的。图 10.10a 和 10.10b 展示了一个例子。这四条有理截面曲线在三维空间中都是 C^1 连续的。它们是二次曲线，在 $u = 1/2$ 处定义了一个双重节点。相应的截面曲线的权重是

$$\mathbf{C}_0 : \{1, 2, 1, 1, 1\}$$

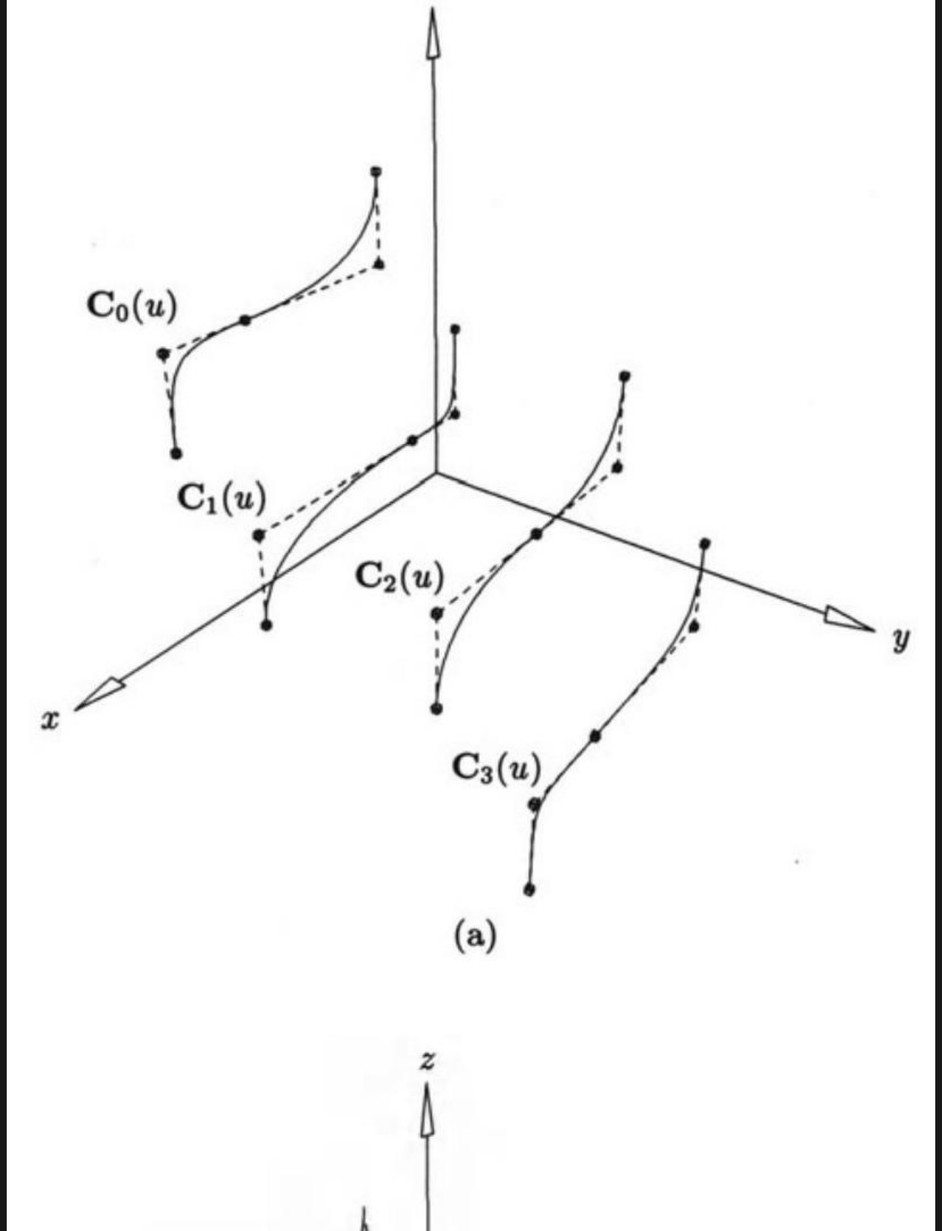
$$\mathbf{C}_1 : \{1, 1, 1, 3, 1\}$$

$$\mathbf{C}_2 : \{1, 1, 1, 1, 1\}$$

$$\mathbf{C}_3 : \{1, 4, 1, 2, 1\}$$

$\mathbf{C}_0, \mathbf{C}_1$ ，并且 $\mathbf{C}_3$ 在四维空间中仅是 C^0 连续的。在 v 方向的插值后，表面权重为:

$$w_{i,j} = \begin{bmatrix} 1 & 2 & 1 & 1 & 1 \\ 1 & 0.94 & 1 & 4.43 & 1 \\ 1 & 0.71 & 1 & -0.75 & 1 \\ 1 & 4 & 1 & 2 & 1 \end{bmatrix}$$



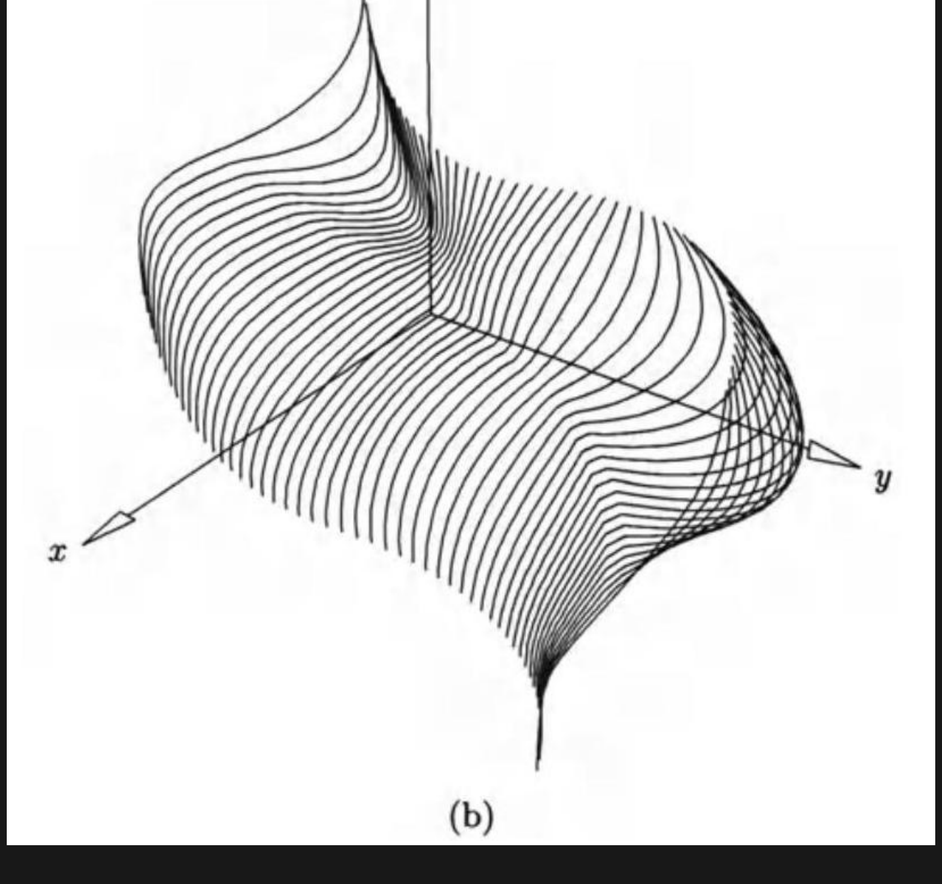


图 10.10。有理皮肤处理。(a) C^1 连续的截面曲线；(b) 皮肤表面。

(注意 $\mathbf{P}_{3,2}$ 处的负权重)。图 10.10 b 中显示的表面在 u 方向上仅表现出 C^0 连续性。根据作者的知识，解决这一困境的唯一方案涉及使用更高阶的截面曲线。例如，一个完整的圆可以表示为一个具有正权重且没有内部节点的五阶 NURBS 曲线 (示例 E7.10)，从而消除了这个问题。关于这个问题的几种解决方案，所有这些方案都有效地提高了皮肤表面的阶数，在 [Hohm91] 中进行了讨论。一个通用的解决方案是使用所谓的平滑函数。假设 $\mathbf{C}_k^w(u)$ 是一个在节点向量 U 上的 C^r 连续截面曲线，其投影 $\mathbf{C}_k(u)$ 是 C^{r+s} 连续的，$s > 0$ 。设 $f(u)$ 是在 U 上的 B-spline 标量值函数，使得

$$\widehat{\mathbf{C}}_k^w(u) = f(u)\,\mathbf{C}_k^w(u) \tag{10.15}$$

是 C^{r+s} 连续的。$f(u)$ 被称为平滑函数。对于固定的 u_0 ，$f(u_0)$ 只是将 $\mathbf{C}_k^w(u_0)$ 在由 $\mathbf{C}_k(u)$ 定义的四维锥体中缩放，即，对于所有 u ，都有 $\widehat{\mathbf{C}}_k(u) = \mathbf{C}_k(u)$ 。显然，如果所有 $\mathbf{C}_k^w(u)$ 都可以被平滑处理，那么通过相应的 $\widehat{\mathbf{C}}_k^w(u)$ 进行皮肤处理将产生一个 C^{r+s} 连续的表面，该表面插值三维截面曲线。然而，我们不知道确定 $f(u)$ 的通用算法。从左侧和右侧对方程 (10.15) 进行微分会产生一组线性方程，这些方程表达了节点处所需的连续性约束。然而，其他考虑因素是:

- 如果 $\mathbf{C}_k^w(u)$ 是闭合的，则 $\widehat{\mathbf{C}}_k^w(u)$ 应该是闭合的；
- $\widehat{\mathbf{C}}_k^w(u)$ 的所有权重应为正；
- $\widehat{\mathbf{C}}_k^w(u)$ 的权重不应剧烈变化或变得过于庞大。

一个度数 $r+s$ 函数 $f(u)$ 足以满足节点处的连续性约束，但其他约束通常迫使 $f(u)$ 具有更高的度数。最后，一旦确定了 $f(u)$，它必须与 $\mathbf{C}_k^w(u)$ 相乘。B 样条的乘法并不是一项简单的任务。可以开发一个类似于第 5 章的度数提升算法 (算法 A5.9)，即，

1. 将 $f(u)$ 和 $\mathbf{C}_k^w(u)$ 分解为贝塞尔段；
2. 段的乘法；
3. 将 $\widehat{\mathbf{C}}_k^w(u)$ 重新组合为 NURBS 曲线 (去除节点)。

4 扫掠表面

接下来，我们讨论沿任意轨迹曲线扫掠截面曲线的主题。用 $\mathbf{T}(v)$ 表示轨迹，用 $\mathbf{C}(u)$ 表示截面曲线。扫掠曲面的通用形式为

$$\mathbf{S}(u,v)=\mathbf{T}(v)+M(v)\,\mathbf{C}(u) \tag{10.16}$$

其中 $M(v)$ 是一个 3×3 矩阵，结合了 $\mathbf{C}(u)$ 的旋转和非均匀缩放，作为 v.$\mathbf{T}(v)$ 的函数，而 $\mathbf{C}(u)$ 可以是任意的: 非理性或理性，平面或非平面，开放或闭合。

一般来说，方程 (10.16) 可能会产生自交、退化和不连续的无意义和不希望的曲面。此外，在许多情况下，$\mathbf{S}(u,v)$ 并不能精确表示为 NURBS 曲面。在实践中，大多数扫掠曲面是两种特定类型之一:

1. 对于所有 v，$M(v)$ 是单位矩阵，即，对于每个 v,$\mathbf{C}(u)$ 仅由 $\mathbf{T}(v)$ 平移；
2. $M(v)$ 不是单位矩阵。

情况 1 由以下方程描述

$$\mathbf{S}(u,v)=\mathbf{T}(v)+\mathbf{C}(u) \tag{10.17}$$

该表面具有精确的 NURBS 表示。我们称这种表面为广义平移扫掠。第 8.3 节中的一般圆柱是一个特例，其中 $\mathbf{T}(v)$ 是一条直线。$\mathbf{S}(u,v)$ 的构造如下。设

$$\mathbf{T}(v)=\frac{\sum_{j=0}^{m}N_{j,q}(v)\,w_j^T\mathbf{T}_j}{\sum_{j=0}^{m}N_{j,q}(v)\,w_j^T}\quad V=\{v_0,\ldots,v_s\}$$

和

$$\mathbf{C}(u)=\frac{\sum_{i=0}^{n}N_{i,p}(u)\,w_i^C\mathbf{Q}_i}{\sum_{i=0}^{n}N_{i,p}(u)\,w_i^C}\quad U=\{u_0,\ldots,u_r\}$$

然后

$$\mathbf{S}(u,v)=\frac{\sum_{i=0}^{n}\sum_{j=0}^{m}N_{i,p}(u)\,N_{j,q}(v)\,w_{i,j}\mathbf{P}_{i,j}}{\sum_{i=0}^{n}\sum_{j=0}^{m}N_{i,p}(u)\,N_{j,q}(v)\,w_{i,j}}$$

在节点 U 和 V 上定义，并具有控制点

$$\mathbf{P}_{i,j}=\mathbf{T}_j+\mathbf{Q}_i\ i=0,\ldots,n\ j=0,\ldots,m \tag{10.18}$$

和权重

$$w_{i,j}=w_i^Cw_j^T\ i=0,\ldots,n\ j=0,\ldots,m \tag{10.19}$$

图 10.11a 和 10.11b 显示了平移扫掠的一个例子。

现在考虑情况 2。设 $\{\mathbf{O},\mathbf{X},\mathbf{Y},\mathbf{Z}\}$ 表示我们的全局坐标系统。为简化问题，我们引入一个局部正交坐标系统 $\{\mathbf{o}(v),\mathbf{x}(v),\mathbf{y}(v),\mathbf{z}(v)\}$，它沿着 $\mathbf{T}(v)$ 移动。对于每个 $v, v_0\le v\le v_s$，设

$$\mathbf{o}(v)=\mathbf{T}(v)\ \ \mathbf{x}(v)=\frac{\mathbf{T}'(v)}{|\mathbf{T}'(v)|} \tag{10.20}$$

此外，设 $\mathbf{B}(v)$ 为一个向量值函数，满足所有 v 的 $\mathbf{B}(v)\cdot\mathbf{x}(v)=0$，并设

$$\mathbf{z}(v)=\frac{\mathbf{B}(v)}{|\mathbf{B}(v)|} \tag{10.21}$$

确定该函数是扫掠过程中的一个关键且非平凡的部分。

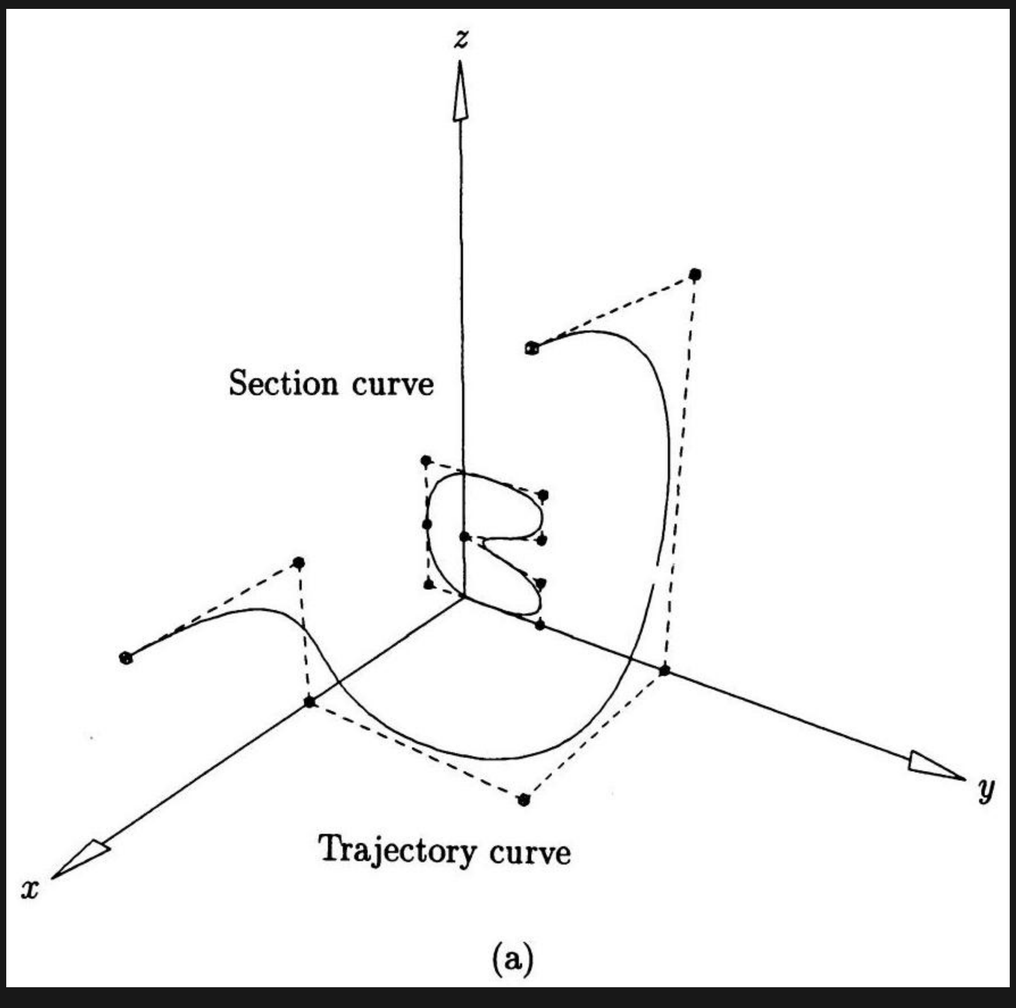


图 10.11. 平移扫掠。(a) 轨迹和截面曲线；(b) 扫掠表面。

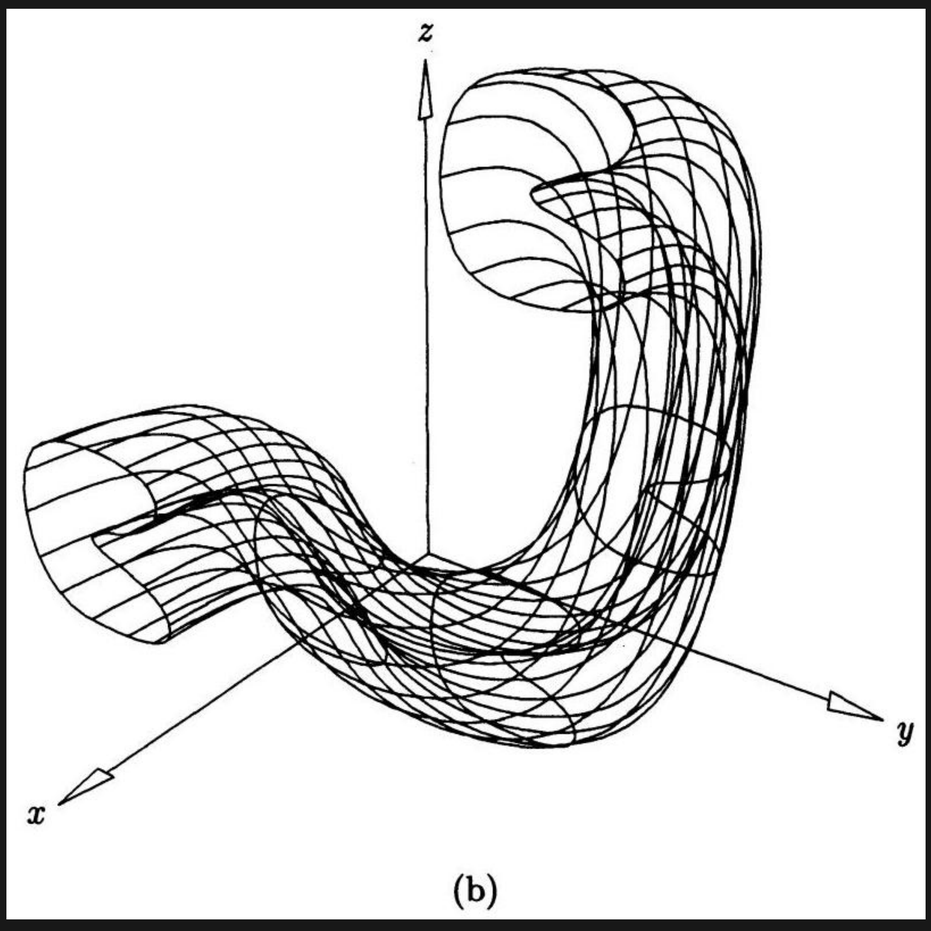


图 10.11. (续)

过程。我们将在本节末尾回到如何确定 $\mathbf{B}(v)$ 的问题；现在，我们简单地假设它是给定的。$\mathbf{y}(v)$ 随后被定义为

$$\mathbf{y}(v)=\mathbf{z}(v)\times\mathbf{x}(v) \tag{10.22}$$

为了纳入缩放，我们引入一个三维向量函数

$$\mathbf{s}(v)=(s_x(v),s_y(v),s_z(v)) \tag{10.23}$$

现在，扫过的表面被定义为

$$\mathbf{S}(u,v)=\mathbf{T}(v)+A(v)\,S(v)\,\mathbf{C}(u) \tag{10.24}$$

其中 $S(v)$ 是一个 3×3 对角矩阵，其对角元素为 $(s_x(v),s_y(v),s_z(v))$，而 $A(v)$ 是将 $\{\mathbf{O},\mathbf{X},\mathbf{Y},\mathbf{Z}\}$ 转换为 $\{\mathbf{o}(v)$、$\mathbf{x}(v),\mathbf{y}(v),\mathbf{z}(v)\}$ 的一般变换矩阵。在固定 v 值下的 $\mathbf{S}(u,v)$ 上的等参数曲线是 $\mathbf{C}(u)$ 的实例，通过 $A(v)\,S(v)$ 进行变换，并由 $\mathbf{T}(v)$ 进行平移。如果 $\mathbf{C}(u)$ 通过全局原点 $u=\bar{u}$，那么 $\mathbf{T}(v)$ 就是等参数曲线 $\mathbf{S}(u=\bar{u},v)$。

$\mathbf{S}(u,v)$ 可能无法以 NURBS 形式表示。显然，如果 $A(v)$ 和 $S(v)$ 可以精确表示为有理 B 样条函数，那么 $\mathbf{S}(u,v)$ 也可以如此。$S(v)$ 通常没有问题。如果应用缩放，通常会使用线性函数或更高阶 B 样条近似一个关于弧长参数线性的函数。然而，$A(v)$ 通常无法以 NURBS 形式表示，即使在简单情况下 $\mathbf{z}(v)=\mathbf{Z}$ 对于所有 $v_0\le v\le v_s$。因此，我们必须构造一个 NURBS 近似来表示方程 (10.24)。我们将这个近似表示为 $\widehat{\mathbf{S}}(u,v)$。我们提出三种方法:

- 第一种方法是使用公式 (10.24) 来评估一个精确位于 $\mathbf{S}(u,v)$ 上的 $(n\times m)$ 点网格，然后使用第 9 章的插值或近似技术来构造一个非理性的 B-spline 近似 $\widehat{\mathbf{S}}(u,v)$。这种方法的一个缺点是，所有的 u 或 v 等参数曲线在 $\widehat{\mathbf{S}}(u,v)$ 上都不精确；
- 另一种方法由 Bloomenthal 和 Riesenfeld [Blom91] 以及 Coquillart [Coqu87a] 提出；他们的方法涉及对轨迹的三维偏移曲线进行近似。如果 $\mathbf{C}(u)$ 经过原点，他们的算法将 $\mathbf{T}(v)$ 插值为等参数曲线；然而，除了在 $v=v_0$ 和 $v=v_s$ 处，$\mathbf{C}(u)$ 的实例并未被插值。Bloomenthal 和 Riesenfeld 为他们的方法提供了基于偏移过程误差界限的误差界限；
- 第三种方法，我们在这里更详细地介绍，是使用皮肤化。我们在 $\mathbf{T}(v)$ 上变换并放置 $\mathbf{C}(u)$ 的 $K+1$ 实例，然后在这些截面曲线之间进行皮肤化。通过增加 K 来提高生成表面的准确性 $\widehat{\mathbf{S}}(u,v)$。每个 $\mathbf{C}(u)$ 的实例被插值为 $\widehat{\mathbf{S}}(u,v)$ 上的等参数曲线。对于 v - 方向拟合，有两种策略。第一种利用 $\mathbf{T}(v)$ 的 B-spline 结构。轨迹的度数 q 和节点向量 V 被用于拟合，因此它们被继承到 $\widehat{\mathbf{S}}(u,v)$ 中。此外，拟合的参数 $\bar{v}_k$ 必须来自 $\mathbf{T}(v)$；如果 $\mathbf{T}(v)$ 是有理的，则其权重必须在进行 v -方向拟合之前考虑。如果 $\mathbf{C}(u)$ 通过全局原点，并且希望 $\mathbf{T}(v)$ 在 $\widehat{\mathbf{S}}(u,v)$ 中是等参数曲线，则需要使用此方法，但在任何情况下都可以使用该方法。另一种方法除了定位截面曲线外不使用 $\mathbf{T}(v)$；即 $\widehat{\mathbf{S}}(u,v)$ 的 v -度数被输入，参数和节点以某种方式计算，并且不需要考虑 $\mathbf{T}(v)$ 的权重。

对应于 v 的两种方法，存在两种扫掠算法。在这两种情况下，扫掠表面 $\widehat{\mathbf{S}}(u,v)$ 继承了其 u 次 p 和节点向量 U 来自 $\mathbf{C}(u)$。T 和 $\mathbf{C}$ 分别是轨迹和截面曲线，Bv 和 sv 是由方程 (10.21) 和 (10.23) 给出的输入函数。在算法 A10.1 中，不要求 v 次 q；数字 K 应视为最小值，因为根据定义 $\mathbf{T}(v)$ 的节点数量，可能需要更多的截面曲线实例。

ALGORITHM A10.1

```
SweepSurface1(T, C, Bv, Sv, K, V, Pw)
  { /* Swept surface. Trajectory interpolated. */
    /* Input: T, C, Bv, sv, K */
      /* Output: V, Pw */
  q = degree of T(v) .
ktv = number of knots of T(v) .
nsect = K + 1 ;
if (ktv <= nsect+q)
    { /* Must refine T(v) 's knot vector */
    m = nsect+q-ktv+1;
    Insert m more knots into T(v) 's knot vector. Locations
      are not critical, recursively insert at the midpoint of
      the longest span will do. New control points do not
      have to be computed, as we only require a refined
      set of knots.
    The resulting knot vector V is inherited by Ŝ(u,v) .
  }
    else
    { /* T(v)'s knot vector will do */
    V = T(v) 's knot vector.
    if (ktv > nsect+q+1) /* Must increase number of */
      nsect = ktv-q-1; /* instances of C(u) */
  }
v̄0 = T(v) 's minimum parameter value.
v̄nsect −1 = T(v)'s maximum parameter value.
for (k = 1; k < nsect − 1; k++) /* Compute parameters by */
    v̄k = (vk+1 + ··· + vk+q)/q; /* averaging knots */
for (k = 0; k < nsect; k++)
    { /* Transform and position section control points */
    Let Qi and wi be the control points and weights
      of C(u), i = 0, ..., n .
    Scale the control points Qi by sv.
    Compute {o, x, y, z} (v̄k) from Eqs. (10.20)-(10.22).
    Compute the transformation matrix A(v̄k) transforming the
      global system into {o, x, y, z} (v̄k) .
    Apply A(v̄k) to the scaled Qi .
    Reapply the weights wi , and denote the resulting
      weighted control points by Q^w_{k,i} .
    if (T(v) is rational )
      { /* w(v) is the weight function of T(v) */
          /* (4th coordinate) */
      Q^w_{k,i} = w(v̄k) Q^w_{k,i};
```

```
        }
    }
for (i = 0; i<=n; i++)
    {
    Interpolate across Q^w_{0,i}, ..., Q^w_{nsect-1,i} to obtain
      P^w_{0,i}, ..., P^w_{nsect-1,i}.
    }
}
ALGORITHM A10.2
    SweepSurface2(T, C, Bv, sv, q, K, V, Pw)
        { /* Swept surface. Trajectory not interpolated. */
            /* Input: T, C, Bv, sv, q, K */
            /* Output: V, Pw */
        Determine values v_0, ..., v_K at which to place the
            instances of C(u). A reasonable choice is to select the
            v_k so that the T(v_k) are approximately evenly
            spaced.
        for (k = 0; k <= K; k++)
            { /* Transform and position section control points */
          Let Q_i and w_i be the control points and weights
              of C(u), i = 0, ..., n.
            Scale the control points Q_i by sv.
            Compute {o, x, y, z}(v_k) from Eqs. (10.20)-(10.22).
            Compute the transformation matrix A(v_k) transforming the
              global system into {o, x, y, z}(v_k).
            Apply A(v_k) to the scaled Q_i.
          Reapply the weights w_i, and denote the resulting
              weighted control points by Q^w_{k,i}.
          }
        Determine v̄_0, ..., v̄_K, the v-parameters for the
            v-directional interpolations (Eqs.[10.8] or [10.14]).
        Given q and the v̄_k, compute the knot vector V by averaging
            (Eq. [9.8]).
        for (i=0; i<=n; i++)
            {
          Interpolate across Q^w_{0,i}, ..., Q^w_{K,i} to obtain P^w_{0,i}, ..., P^w_{K,i}.
          }
        }
```

图 10.12a 显示了轨迹和截面曲线，以及一个起始向量 $\mathbf{B}(v_0)$。截面曲线并未通过全局原点。图 10.12b 和 10.12c 分别展示了算法 A10.2 和 A10.1 的扫掠结果。由于截面曲线未通过原点，因此轨迹不包含在表面内，即使在图 10.12c 中也是如此。还注意到图 10.12c 中截面曲线放置的数量更多。图 10.13a-10.13c 显示了截面曲线通过原点的扫掠示例 (图 10.13a)。未使用轨迹控制 (算法 A10.2)。图 10.13b 未使用缩放，而图 10.13c 使用了由控制点定义的分段线性 B 样条缩放函数。

$$\left\{\left(\frac{3}{10},\frac{3}{10},1\right),\left(\frac{3}{2},\frac{3}{2},\frac{1}{2}\right),\left(\frac{3}{10},\frac{3}{10},1\right),(1,1,2)\right\}$$

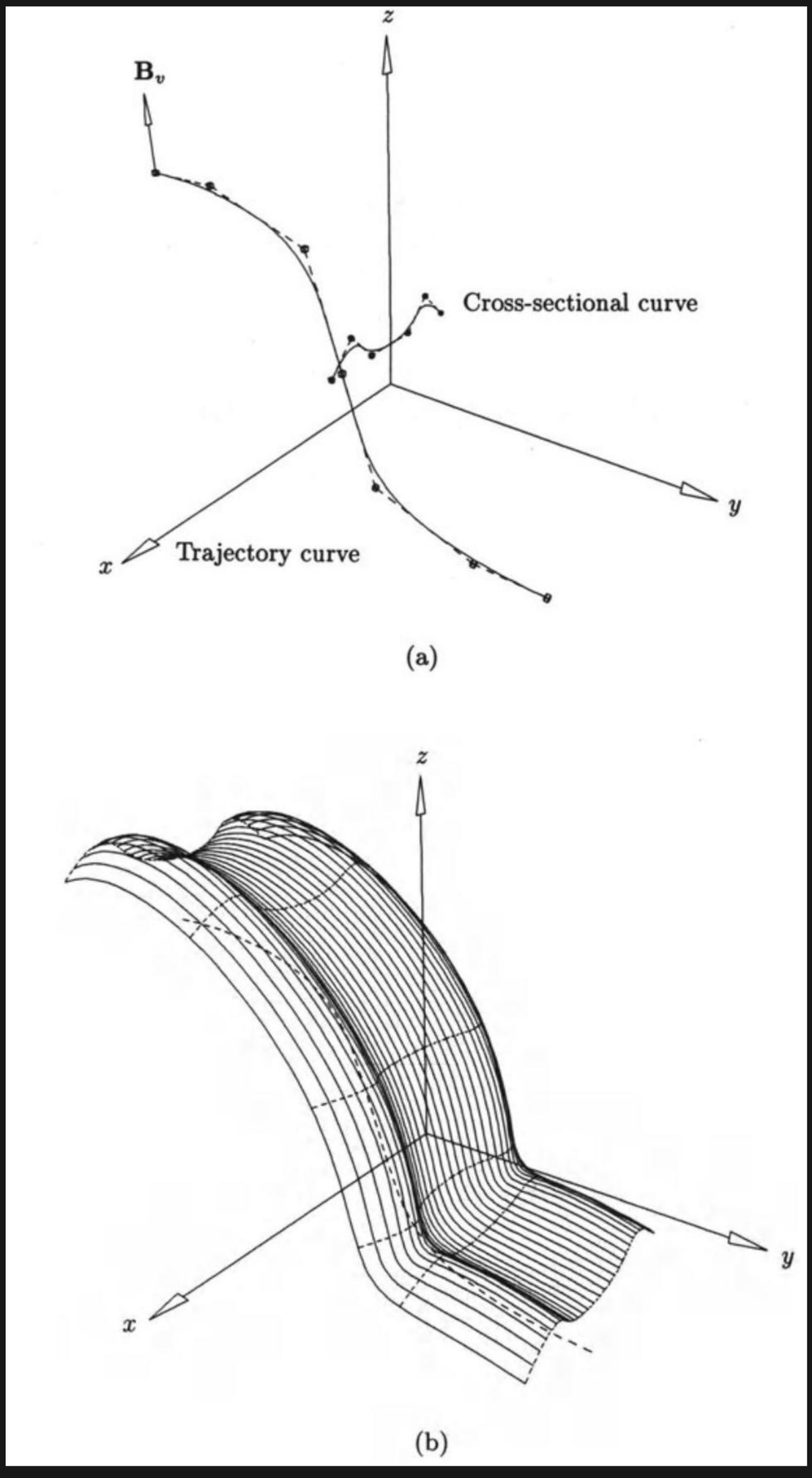


图 10.12. 扫掠示例。(a) 二维轨迹和未通过全局原点的截面曲线；(b) 无轨迹控制插值的扫掠表面；(c) 通过轨迹控制插值得到的扫掠表面。

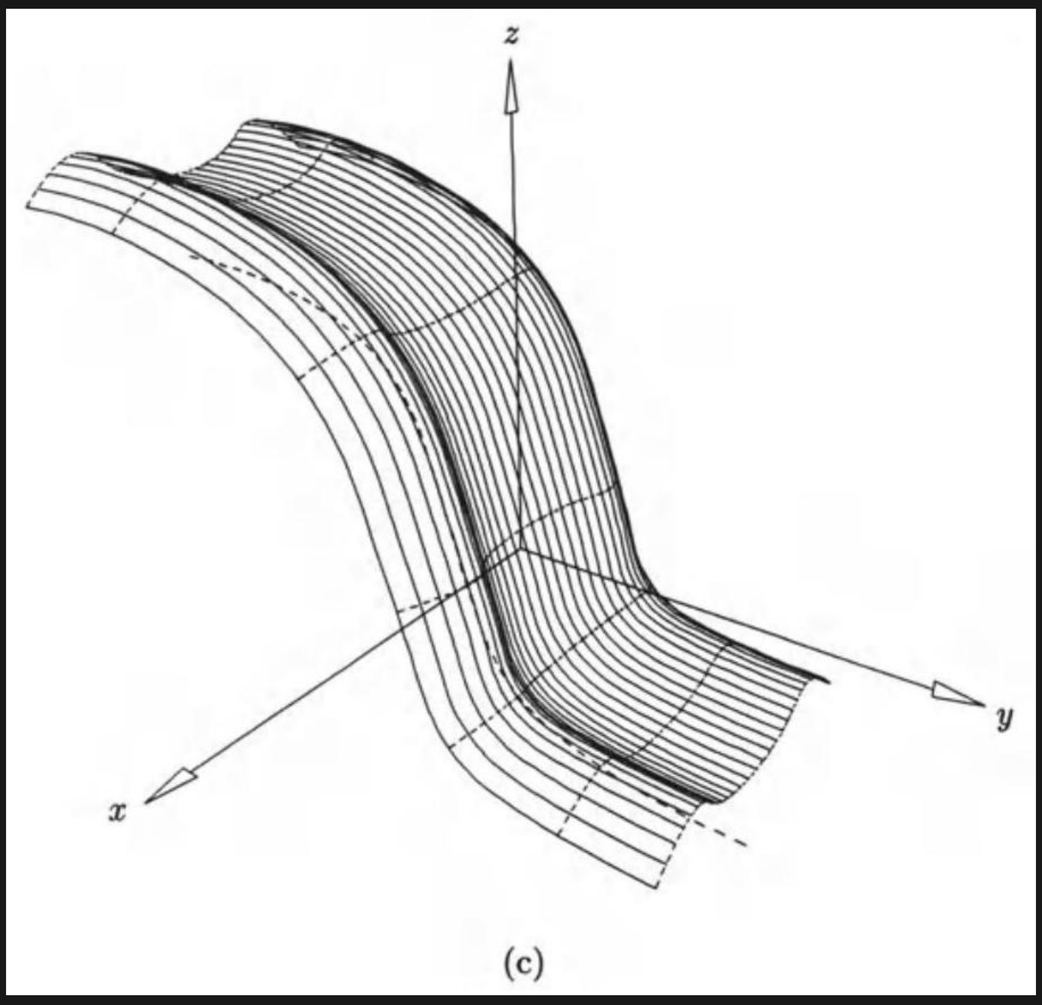


图 10.12. (续)

和节点

$$V = \left\{0, 0, \frac{3}{10}, \frac{7}{10}, 1, 1\right\}$$

前两个例子使用了平面轨迹曲线。图 10.14a 显示了一个三维 (扭曲的) 轨迹。图 10.14b 中未保持轨迹 (算法 A10.2)，而图 10.14c 中的轨迹是该表面的等曲线 (算法 A10.1)。在这两个例子中，所有截面实例都与常量向量 $(3/5, 3/5, 1)$ 进行了缩放。

截面曲线的位置和大小，以及轨迹的形状 (曲率) 对表面形状有显著影响。图 10.15a 和 10.15 b 显示了两个例子。图 10.15a 中的轨迹没有插值，而在图 10.15b 中则进行了插值。两个表面都表现出所谓的偏移现象，即它们具有自交和不必要的振荡。

现在我们回到确定合适的方向函数 $\mathbf{B}(v)$ 的问题。如果 $\mathbf{T}(v)$ 是平面曲线，解决方案很简单: 令 $\mathbf{B}(v)$ 为一个常量函数，其值是垂直于 $\mathbf{T}(v)$ 平面的向量。对于任意轨迹，情况则更为复杂。一个常见的解决方案是使用所谓的 Frenet 框架 [DoCa76; Klok86; Bron92; Silt92]。假设 $\mathbf{T}(v)$ 是二次可微的。通过以下方式定义 $\mathbf{B}(v)$:

$$\mathbf{B}(v) = \frac{\mathbf{T}'(v) \times \mathbf{T}''(v)}{|\mathbf{T}'(v) \times \mathbf{T}''(v)|} \tag{10.25}$$

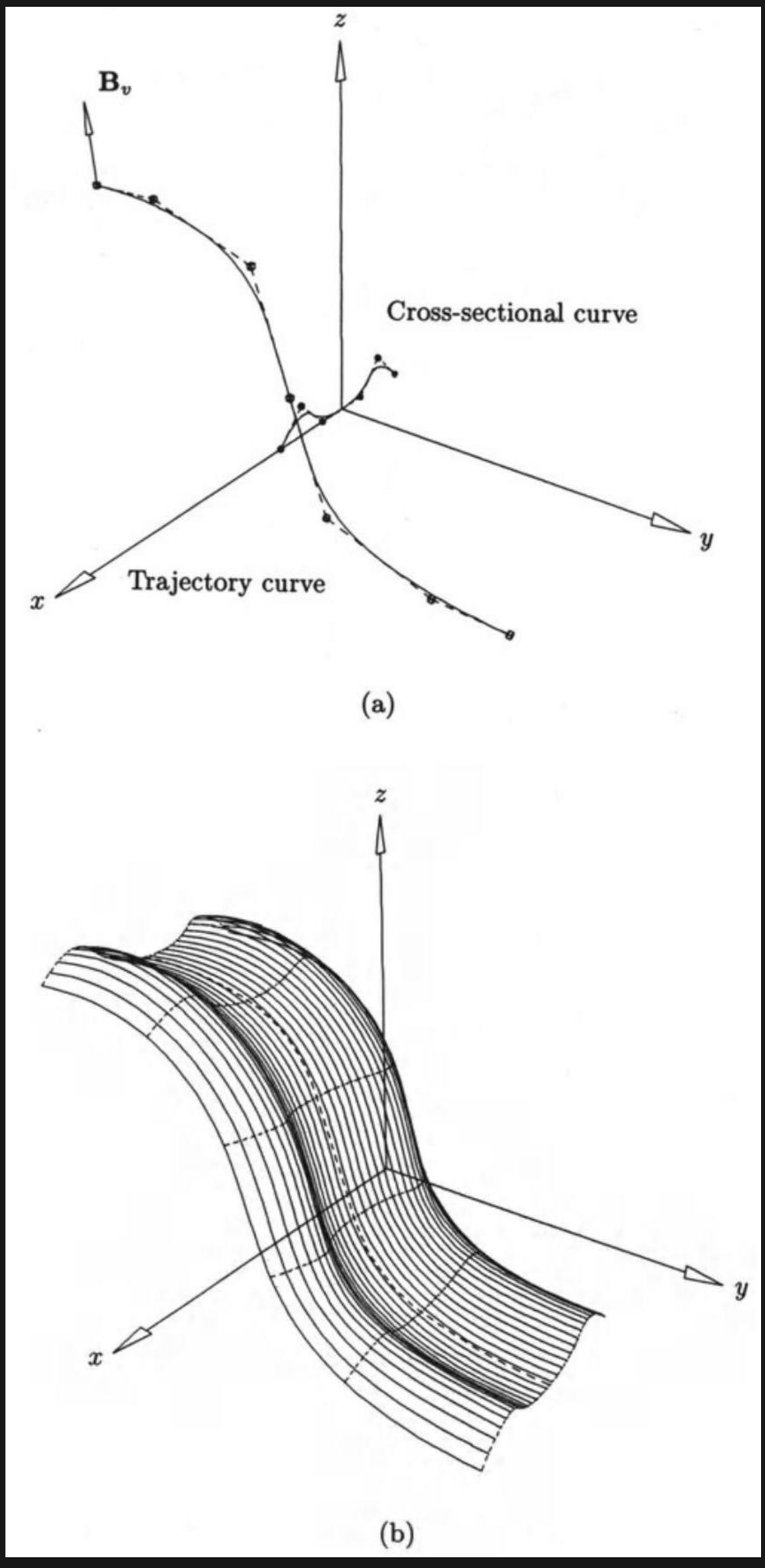


图 10.13. 扫描示例。(a) 通过全局原点的二维轨迹和截面曲线；(b) 无缩放的扫掠表面；(c) 具有缩放截面曲线的扫掠表面。

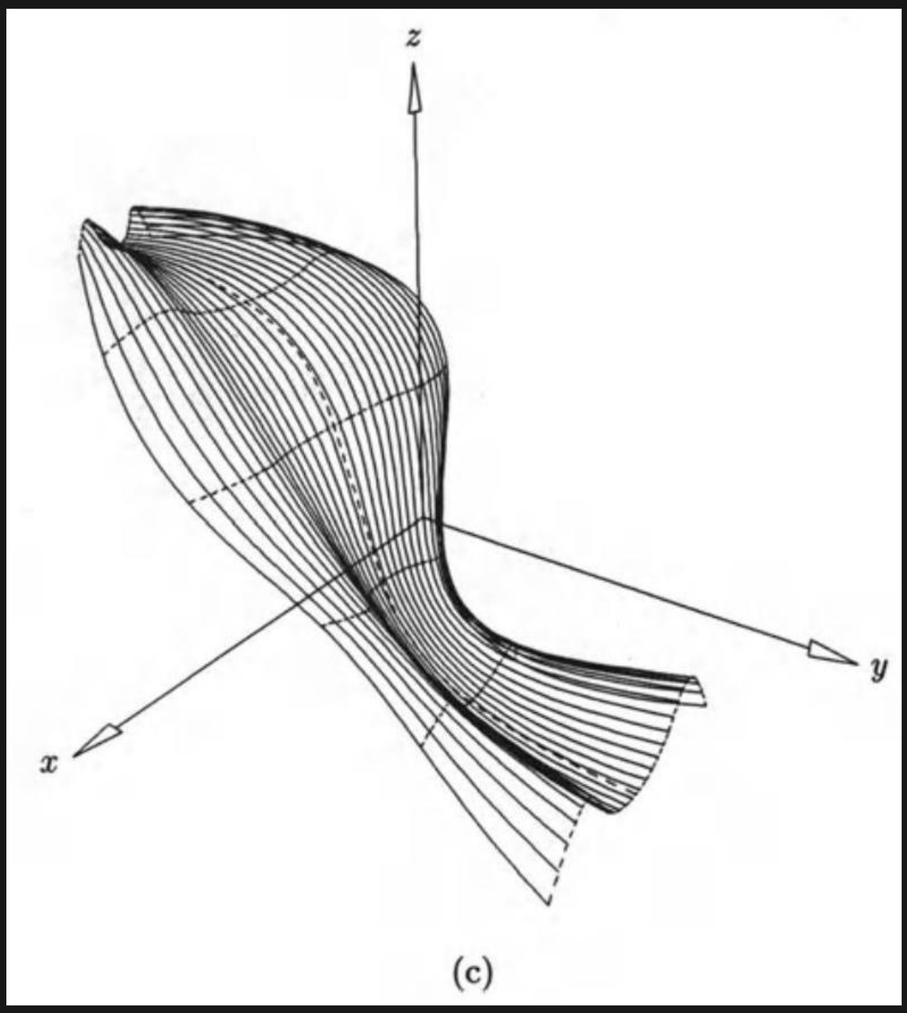


图 10.13. (续)

和 $\mathbf{N}(v)$ 通过

$$\mathbf{N}(v) = \mathbf{B}(v) \times \mathbf{T}'(v) \tag{10.26}$$

$\mathbf{T}'(v)$, $\mathbf{N}(v)$ 和 $\mathbf{B}(v)$ 在所有 v 上是相互正交的，当归一化后，它们在 $\mathbf{T}(v)$ 上形成一个称为 Frenet 框架的移动局部坐标系统。然后可以在公式 (10.21) 和算法 A10.1 和 A10.2 中使用 $\mathbf{B}(v)$ (或其 NURBS 近似)。这种方法有几个缺点:

- $\mathbf{B}(v)$ 在线段或拐点处 (或更一般地，在 $\mathbf{T}''(v) \times \mathbf{T}'(v) = 0$ 的地方) 并未由公式 (10.25) 和 (10.26) 定义；
- $\mathbf{B}(v)$ 在拐点处会突然翻转到相反方向；
- 对于三维轨迹，由 $\mathbf{B}(v)$ 给出的向量可能会围绕 $\mathbf{T}(v)$ 过度旋转，从而导致生成的扫掠表面出现不必要的扭曲。图 10.16 显示了这一现象；注意 Frenet 框架在拐点处的翻转。

前两个问题可以相对容易地解决 [Klok86; Bron92; Silt92]，但 $\mathbf{B}(v)$ 的旋转完全依赖于 $\mathbf{T}(v)$ 的形状，因此在不改变 $\mathbf{T}(v)$ 的形状的情况下无法避免。

Siltanen 和 Woodward [Silt92] 提出了一个优雅的方法，称为投影法线方法，能够避免这三个问题。设 $v_0, \ldots, v_m$ 为参数值的递增序列，其中 v_0 和 v_m 分别为 $\mathbf{T}(v)$ 域中的最小值和最大值。在每个 v_i 上，我们计算一个向量 $\mathbf{B}_i$，

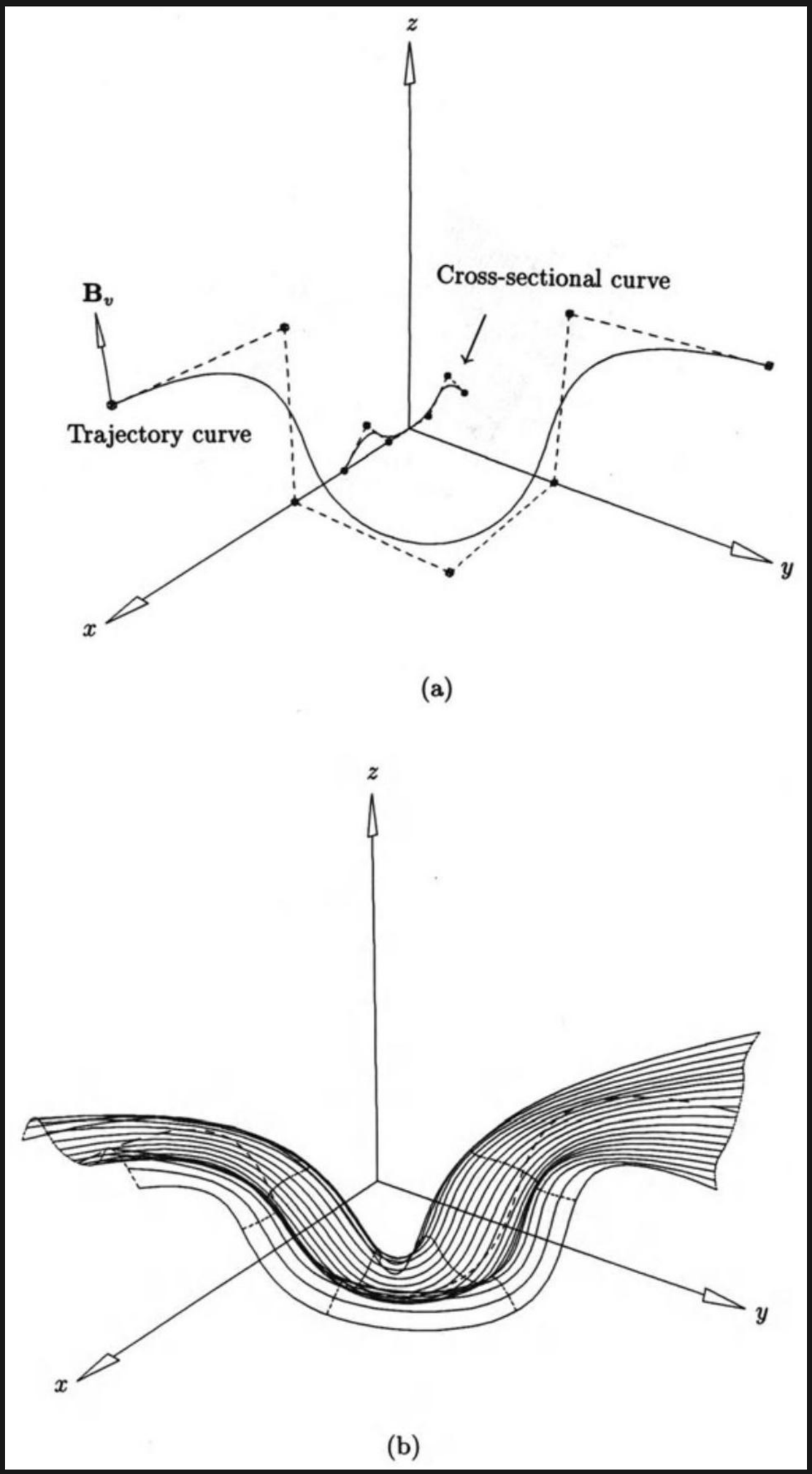


图 10.14. 带有缩放截面曲线的扫掠示例。(a) 三维轨迹和截面曲线；(b) 轨迹未插值；(c) 轨迹已插值。

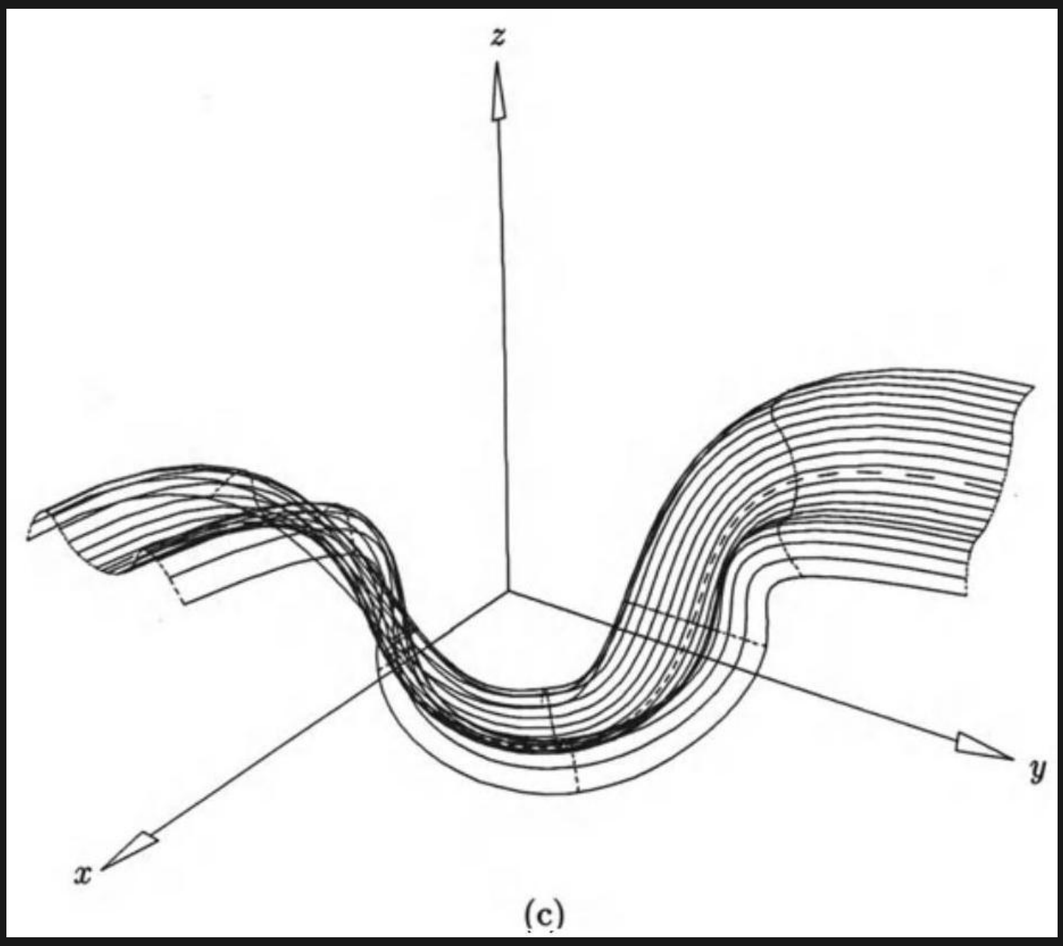


图 10.14.(续)

然后使用插值或近似来获得函数 $\mathbf{B}(v)$ 。我们使用 Siltanen 和 Woodward 的方法来计算 $\mathbf{B}_i$ 。将 $\mathbf{B}_0$ 设置为与 $\mathbf{T}'(v_0)$ 正交的任意单位长度向量。然后对于 $i=1,\ldots,m$ ，计算

$$\mathbf{T}_i = \frac{\mathbf{T}'(v_i)}{|\mathbf{T}'(v_i)|}$$

并设置

$$\mathbf{b}_i = \mathbf{B}_{i-1} - (\mathbf{B}_{i-1} \cdot \mathbf{T}_i)\,\mathbf{T}_i$$

$$\mathbf{B}_i = \frac{\mathbf{b}_i}{|\mathbf{b}_i|} \tag{10.27}$$

用语言表达，$\mathbf{B}_i$ 是通过将 $\mathbf{B}_{i-1}$ 投影到由 $\mathbf{T}_i$ 定义的平面上获得的。必须小心避免 $\mathbf{T}_i \parallel \mathbf{B}_{i-1}$ 的点。注意，如果 $\mathbf{T}(v)$ 是一个封闭曲线，则一般计算出的 $\mathbf{B}_m$ 不等于 $\mathbf{B}_0$ (这是所需的)。解决方案是设置 $\mathbf{B}_m = \overline{\mathbf{B}}_m = \mathbf{B}_0$ ，使用前述技术的反向计算 $\overline{\mathbf{B}}_{m-1},\ldots,\overline{\mathbf{B}}_1$ ，然后设置

$$\mathbf{B}_i = \frac{1}{2}\left(\mathbf{B}_i + \overline{\mathbf{B}}_i\right)\ i=1,\ldots,m-1$$

图 10.12 到 10.15 的扫掠表面使用投影法线方法来计算 $\mathbf{B}_i$ 向量。图 10.17a 和 10.17 b 显示了如何

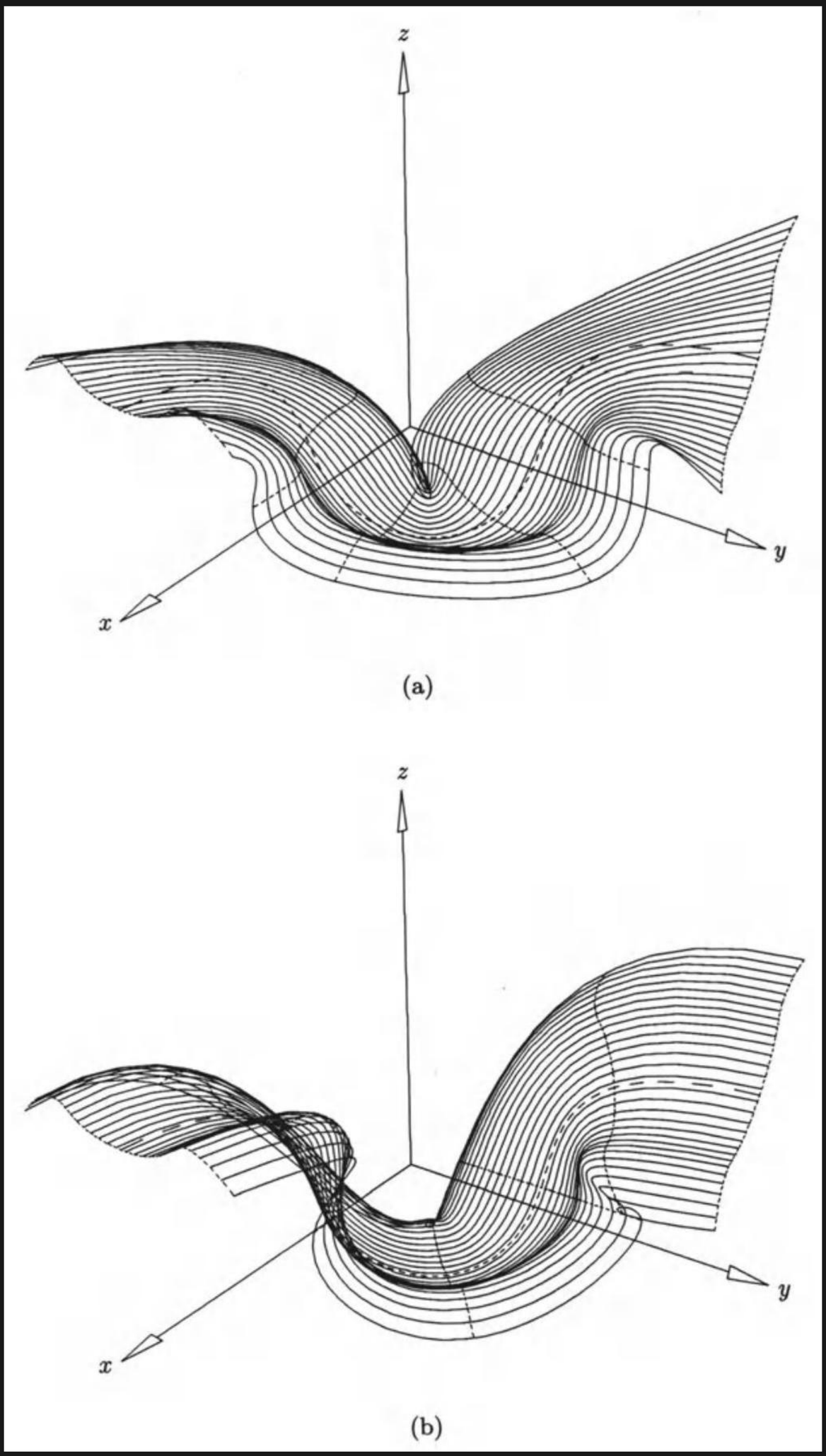


图 10.15。扫掠示例。(a) 轨迹未插值；(b) 轨迹已插值。

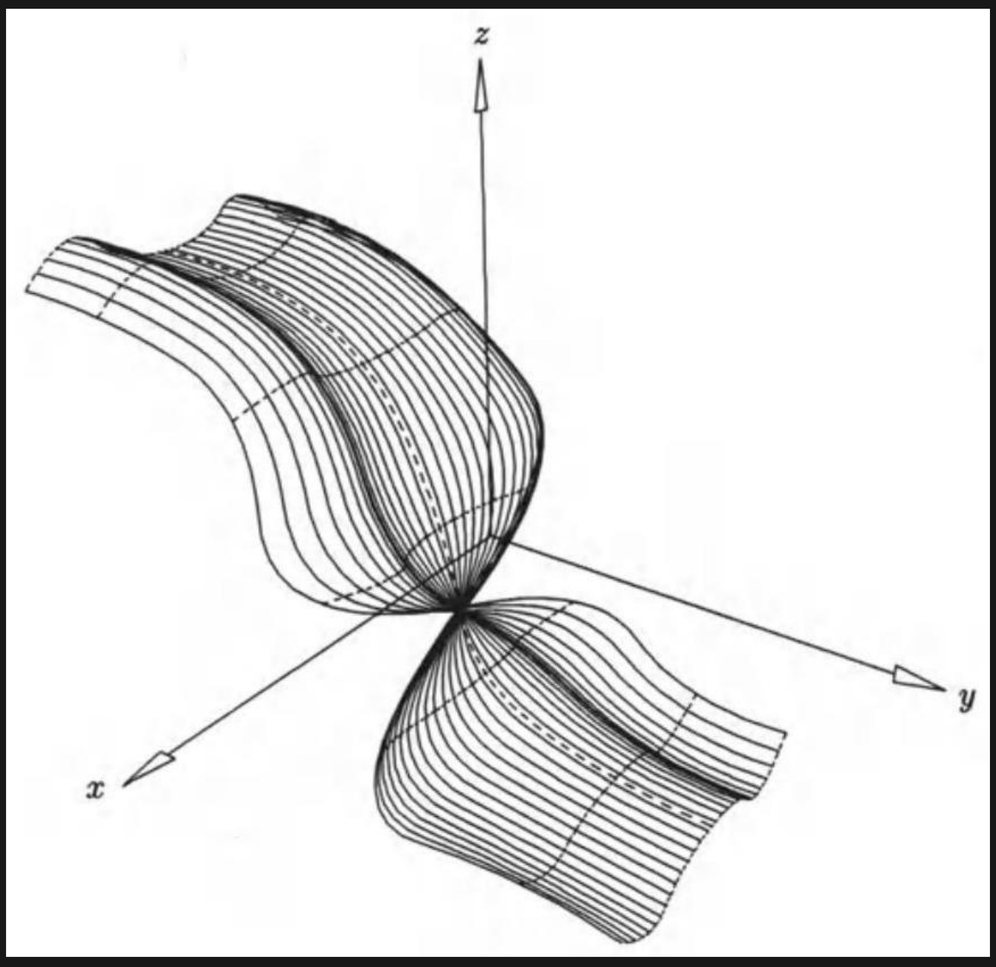


图 10.16。扫掠示例：截面曲线由 Frenet 框架放置。

$\mathbf{B}(v)$ 向量 (投影法线方法) 与 Frenet 框架相比发生变化。在图 10.17a 中，轨迹是平面的，而在图 10.17b 中则是扭曲的。在这两种情况下，移动的 $\mathbf{B}(v)$ 用实心箭头表示，Frenet 框架轴用虚线绘制。

最后，Klok [Klok86] 和 Guggenheimer [Gugg89] 提出了另一种方法，通过初始条件的微分方程的解来获得最小化旋转的 $\mathbf{B}(v)$ 。有关扫掠表面的更多阅读可以在 [Choi90; Akma92] 中找到。

5 双向曲线网络的插值

设

$$\mathbf{C}_k(u)=\sum_{i=0}^{n}N_{i,p}(u)\,\mathbf{P}_{k,i}\ k=0,\ldots,r\ u\in[0,1]$$

$$\mathbf{C}_\ell(v)=\sum_{j=0}^{m}N_{j,q}(v)\,\mathbf{P}_{\ell,j}\ \ell=0,\ldots,s\ v\in[0,1] \tag{10.28}$$

为满足兼容性条件的两组非有理 B 样条曲线 (见图 10.18):

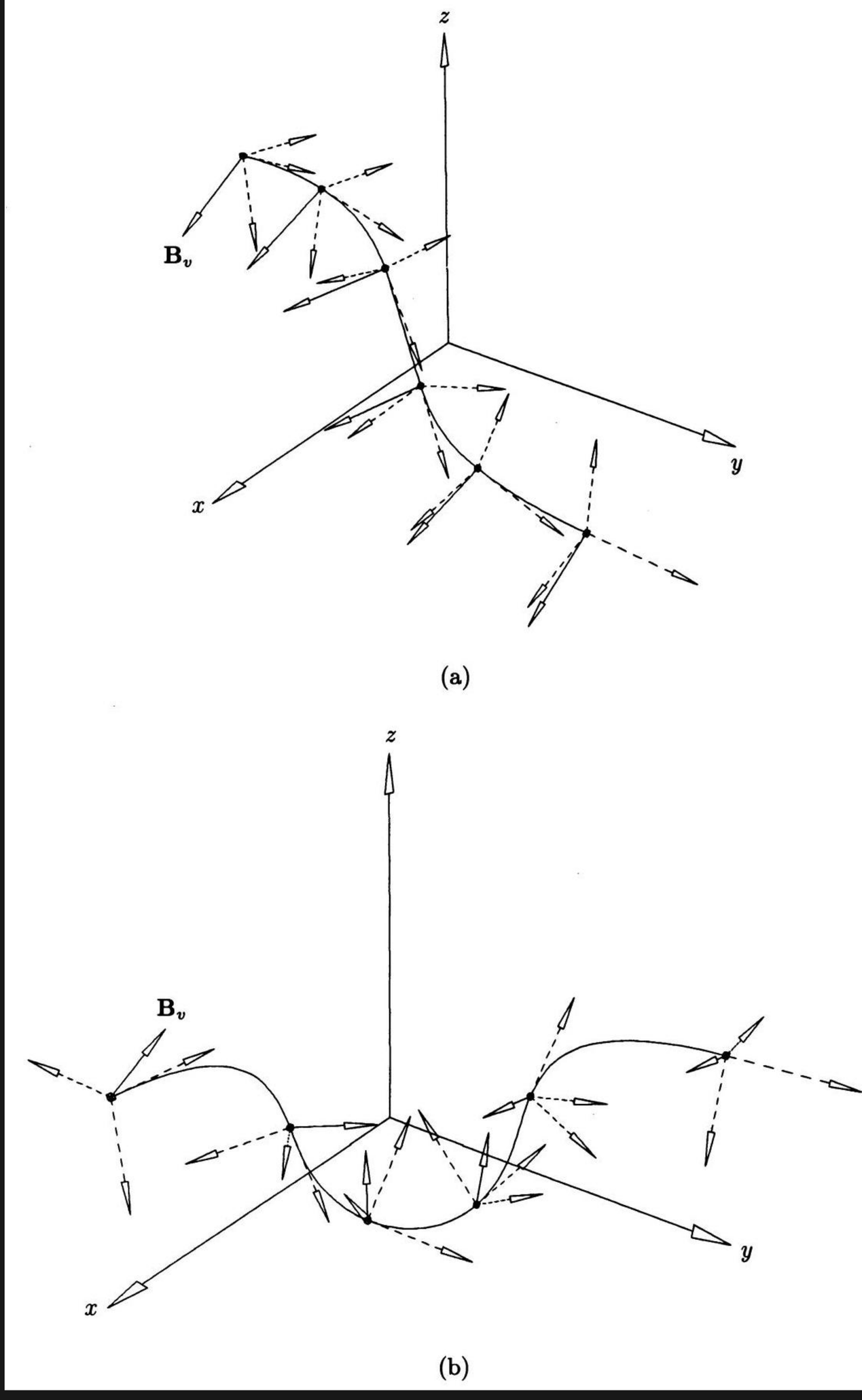


图 10.17. 沿轨迹的 $\mathbf{B}(v)$ 和 Frenet 框架的变化。(a) 平面轨迹；(b) 三维轨迹。

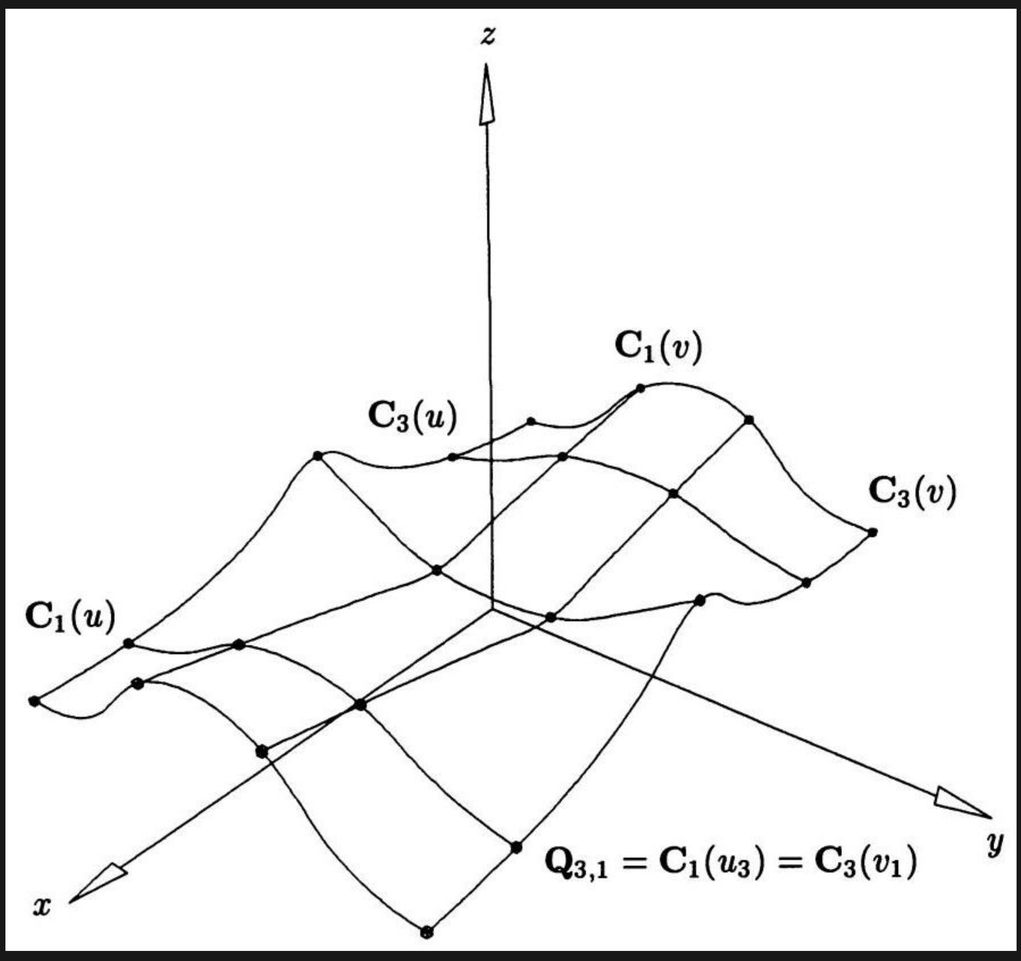


图 10.18. 兼容的曲线网络。

- 作为独立集合，它们在 B 样条意义上是兼容的，即所有的 $\mathbf{C}_k(u)$ 都在一个公共节点向量 U^C 上定义，所有的 $\mathbf{C}_\ell(v)$ 都在一个公共节点向量 V^C 上定义；
- 存在参数 $0 = u_0 < u_1 < \cdots < u_{s-1} < u_s = 1$ 和 $0 = v_0 < v_1 < \cdots < v_{r-1} < v_r = 1$ 使得

$$\mathbf{Q}_{\ell,k} = \mathbf{C}_k(u_\ell) = \mathbf{C}_\ell(v_k) \quad k = 0, \ldots, r \; \ell = 0, \ldots, s \tag{10.29}$$

我们希望构造一个 NURBS 曲面 $\mathbf{S}(u,v)$，它插值这两组曲线，即

$$\mathbf{S}(u_\ell, v) = \mathbf{C}_\ell(v) \quad \ell = 0, \ldots, s$$

$$\mathbf{S}(u, v_k) = \mathbf{C}_k(u) \quad k = 0, \ldots, r \tag{10.30}$$

请注意，我们已将曲线限制为非有理的。从理论上讲，后面给出的构造可以在有理曲线的齐次空间中进行；但通常这样做并不实用，三维结果可能会不可预测。如果涉及有理曲线，我们建议使用第 9 章的约束近似技术，以在必要的公差范围内获得有理曲线的非有理近似。有关有理曲线的更多细节，请参见 Lin 和 Hewitt [Lin94]。

戈登在 1960 年代末于通用汽车公司开发了满足方程 (10.30) 的表面技术；因此这些技术以他的名字命名，称为戈登曲面 [Gord69,71,93]。特别是，他展示了如果 $\{\phi_\ell(u)\}_{\ell=0}^{s}$ 和 $\{\psi_k(v)\}_{k=0}^{r}$ 是满足

$$\phi_\ell(u_i) = \begin{cases} 0 & \text{if } \ell \neq i \\ 1 & \text{if } \ell = i \end{cases}$$

$$\psi_k(v_i) = \begin{cases} 0 & \text{if } k \neq i \\ 1 & \text{if } k = i \end{cases} \tag{10.31}$$

然后由以下给出的表面

$$\mathbf{S}(u,v) = \sum_{\ell=0}^{s} \mathbf{C}_\ell(v)\,\phi_\ell(u) + \sum_{k=0}^{r} \mathbf{C}_k(u)\,\psi_k(v) - \sum_{\ell=0}^{s}\sum_{k=0}^{r} \mathbf{Q}_{\ell,k}\phi_\ell(u)\,\psi_k(v)$$

$$= \mathbf{L}_1(u,v) + \mathbf{L}_2(u,v) - \mathbf{T}(u,v) \tag{10.32}$$

满足方程 (10.30) 的插值条件。因此，所需的表面由三个更简单的表面组成，即两个升高表面，$\mathbf{L}_1(u,v)$ 和 $\mathbf{L}_2(u,v)$，以及张量积，$\mathbf{T}(u,v)$。戈登对各种混合函数集进行了实验，包括简单的阶跃函数、拉格朗日多项式和三次样条。

我们的任务是生成 $\mathbf{S}(u,v)$ 的 NURBS 表示。显然，如果 $\mathbf{L}_1(u,v)$、$\mathbf{L}_2(u,v)$ 和 $\mathbf{T}(u,v)$ 在 B 样条意义上都是兼容的 (定义在相同的节点向量上)，那么可以通过对它们的控制点应用相应的操作进行加法和减法，即 $\mathbf{S}(u,v)$ 的控制点 $\mathbf{P}_{i,j}$ 通过以下方式计算

$$\mathbf{P}_{i,j} = \mathbf{P}_{i,j}^{L_1} + \mathbf{P}_{i,j}^{L_2} - \mathbf{P}_{i,j}^{T} \tag{10.33}$$

其中 $\mathbf{P}_{i,j}^{L_1}, \mathbf{P}_{i,j}^{L_2}$ 和 $\mathbf{P}_{i,j}^{T}$ 分别是 $\mathbf{L}_1(u,v), \mathbf{L}_2(u,v)$ 和 $\mathbf{T}(u,v)$ 的控制点。此外，获得 $\mathbf{L}_1(u,v), \mathbf{L}_2(u,v)$ 和 $\mathbf{T}(u,v)$ 的 NURBS 表示是很简单的；前两个是皮肤表面，后者是通过插值点 $\mathbf{Q}_{\ell,k}$ 获得的 (见第 9.3 节)。皮肤和点插值过程的参数和节点的选择由给定的参数 $\{u_\ell\}$ 和 $\{v_k\}$ 以及 $\{\mathbf{C}_k(u)\}$ 和 $\{\mathbf{C}_\ell(v)\}$ 的节点 U^C 和 V^C 决定。我们用一个简单的例子进行说明。

示例

Ex10. 1 定义图 10.18 中曲线网络的数据是

$$\mathbf{C}_k(u) : n = 5, p = 3, U^C = \left\{0,0,0,0,\frac{3}{10},\frac{7}{10},1,1,1,1\right\}$$

$$\mathbf{C}_\ell(v) : m = 4, q = 2, V^C = \left\{0,0,0,\frac{2}{5},\frac{3}{5},1,1,1\right\}$$

图 10.19a 和 10.19 b 显示了 u 方向升高操作的控制点和生成的表面。节点向量为

$$U^{L_1} = \{0,0,0,0,1,1,1,1\}$$

$$V^{L_1} = \left\{0,0,0,\frac{2}{5},\frac{3}{5},1,1,1\right\}$$

v 方向的升高在图 10.20a 和 10.20b 中进行了说明。节点向量为

$$U^{L_2} = \left\{0,0,0,0,\frac{3}{10},\frac{7}{10},1,1,1,1\right\}$$

$$V^{L_2} = \{0,0,0,0.321,0.647,1,1,1\}$$

张量积曲面如图 10.21a 和 10.21b 所示。节点是

$$U^{T} = \{0,0,0,0,1,1,1,1\}$$

$$V^{T} = \{0,0,0,0.32,0.648,1,1,1\}$$

注意节点向量 V^{L_2} 和 V^T 之间的细微差别。这是因为表面蒙皮和表面插值以不同的方式计算节点向量 (但对各自的操作有重要意义)。前者使用曲线控制点，而后者利用曲线网络的交点。然而，彼此非常接近的节点是多余且不受欢迎的。一个解决方案是将所有先前计算的节点向量传递给蒙皮和插值例程，允许它们使用各自的算法计算新节点，将这些新节点与现有节点进行比较，并在差异较小的情况下用现有节点替换新节点。在这个例子中，不需要提升度，因为放样和插值使用曲线的度。然而，需要进行节点细化，如图 10.22a-10.22c 所示。戈登曲面构造如图 10.23a 所示，曲面在图 10.23b 中进行了说明。最终节点是

$$U = \left\{0,0,0,0,\frac{3}{10},\frac{7}{10},1,1,1,1\right\}$$

$$V = \{0,0,0,0.32,0.321,0.4,0.6,0.647,0.648,1,1,1\}$$

如前所述，输入曲线的度、用于放样的度以及用于构造 $\mathbf{T}(u,v)$ 的度都可以不同。最终的度 (p, q) 为 $\mathbf{S}(u,v)$ 仅是这些度的最大值。在进行节点细化之前，$\mathbf{L}_1(u,v)$ 、$\mathbf{L}_2(u,v)$ 和 $\mathbf{T}(u,v)$ 的度必须提升到 (p, q)。以下是完整算法的草图。Ck[] 和 Cl[] 是输入曲线的数组，ul[] 和 vk[] 是曲线交点的参数。要进行双向曲线网络的插值 491

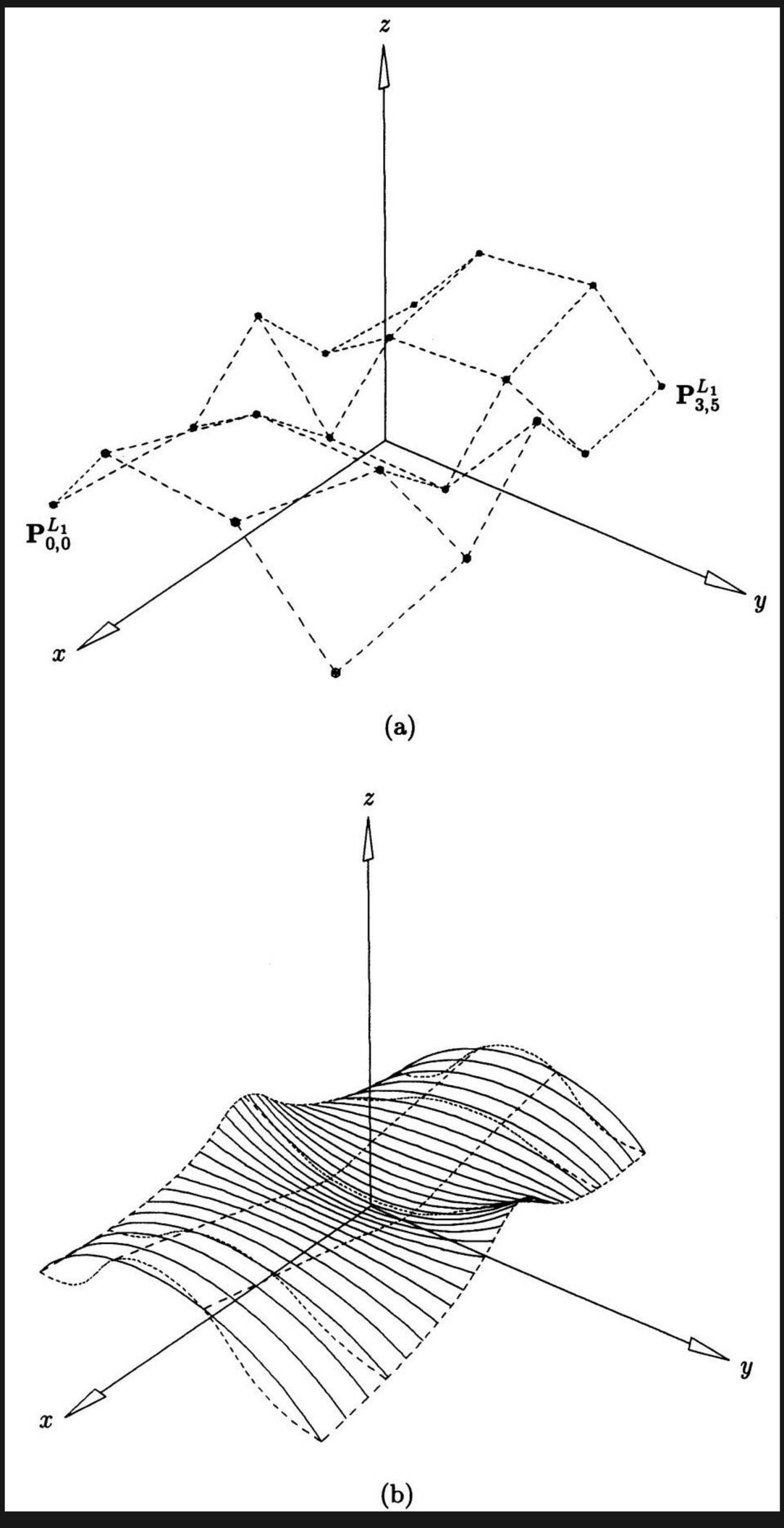


图 10.19. u -方向放样曲面。(a) 控制网；(b) 表面插值 v 曲线。

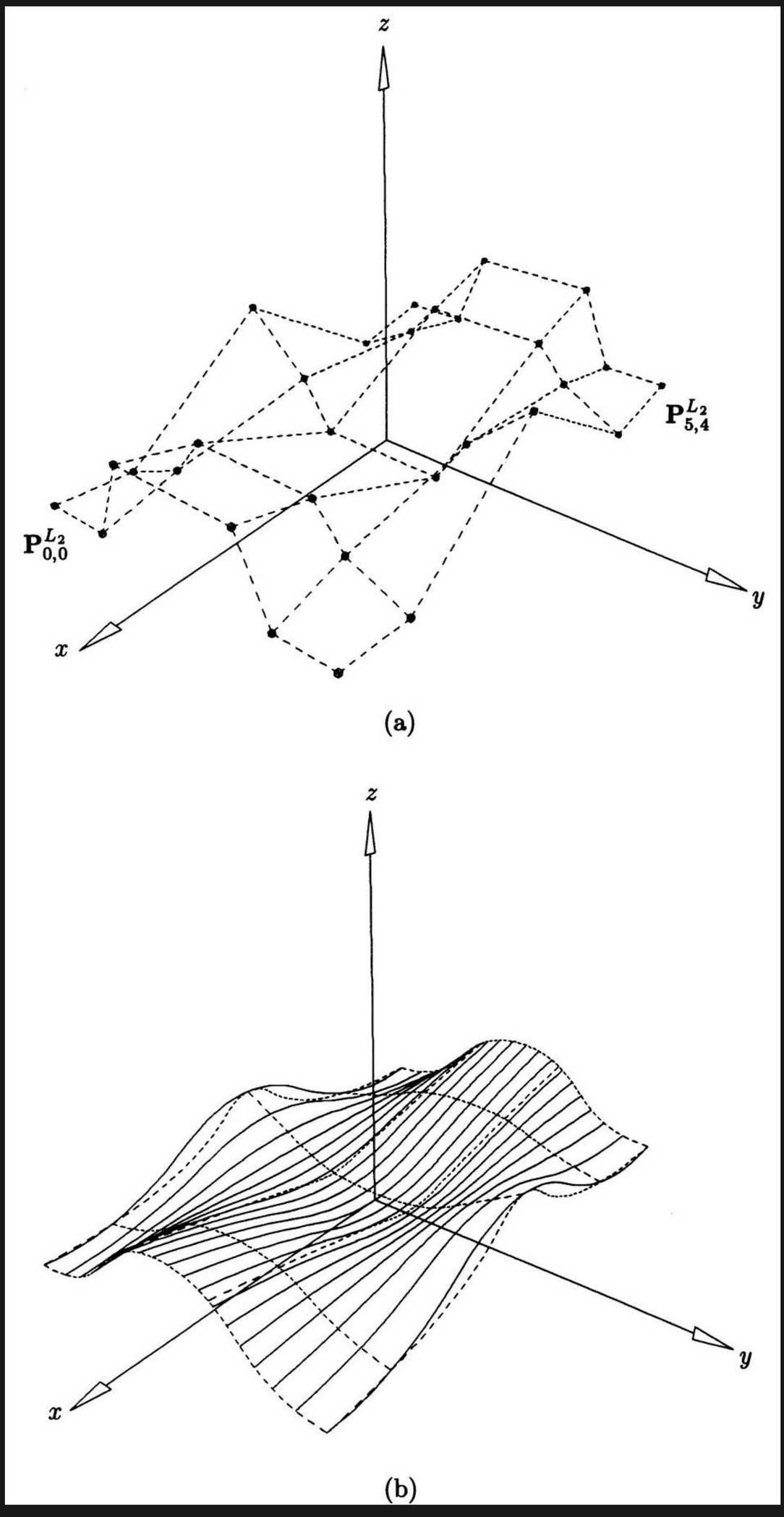


图 10.20. v -双向抬升表面。(a) 控制网；(b) 表面插值 u 曲线。

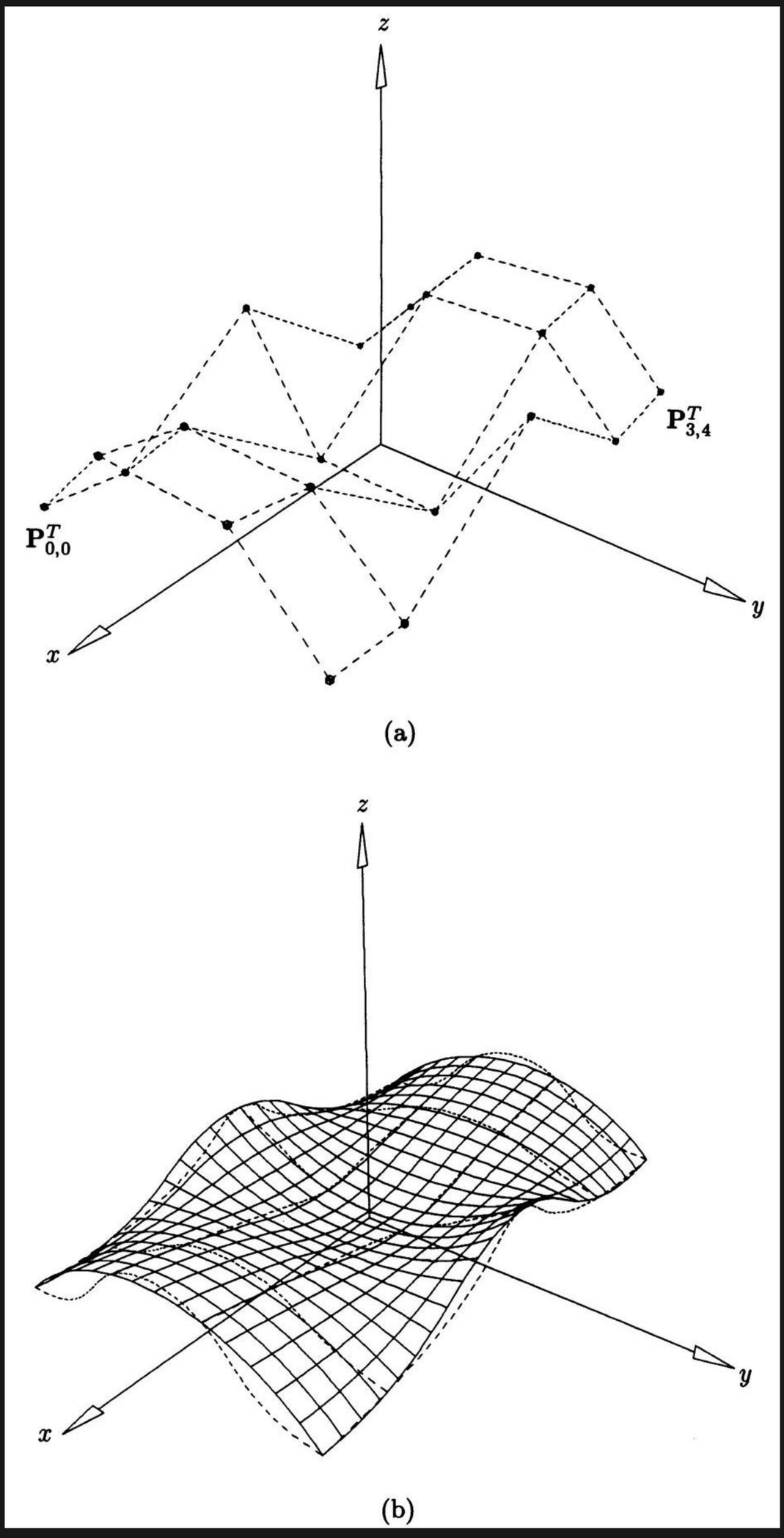


图 10.21. 张量积表面。(a) 控制网；(b) 表面插值交点。

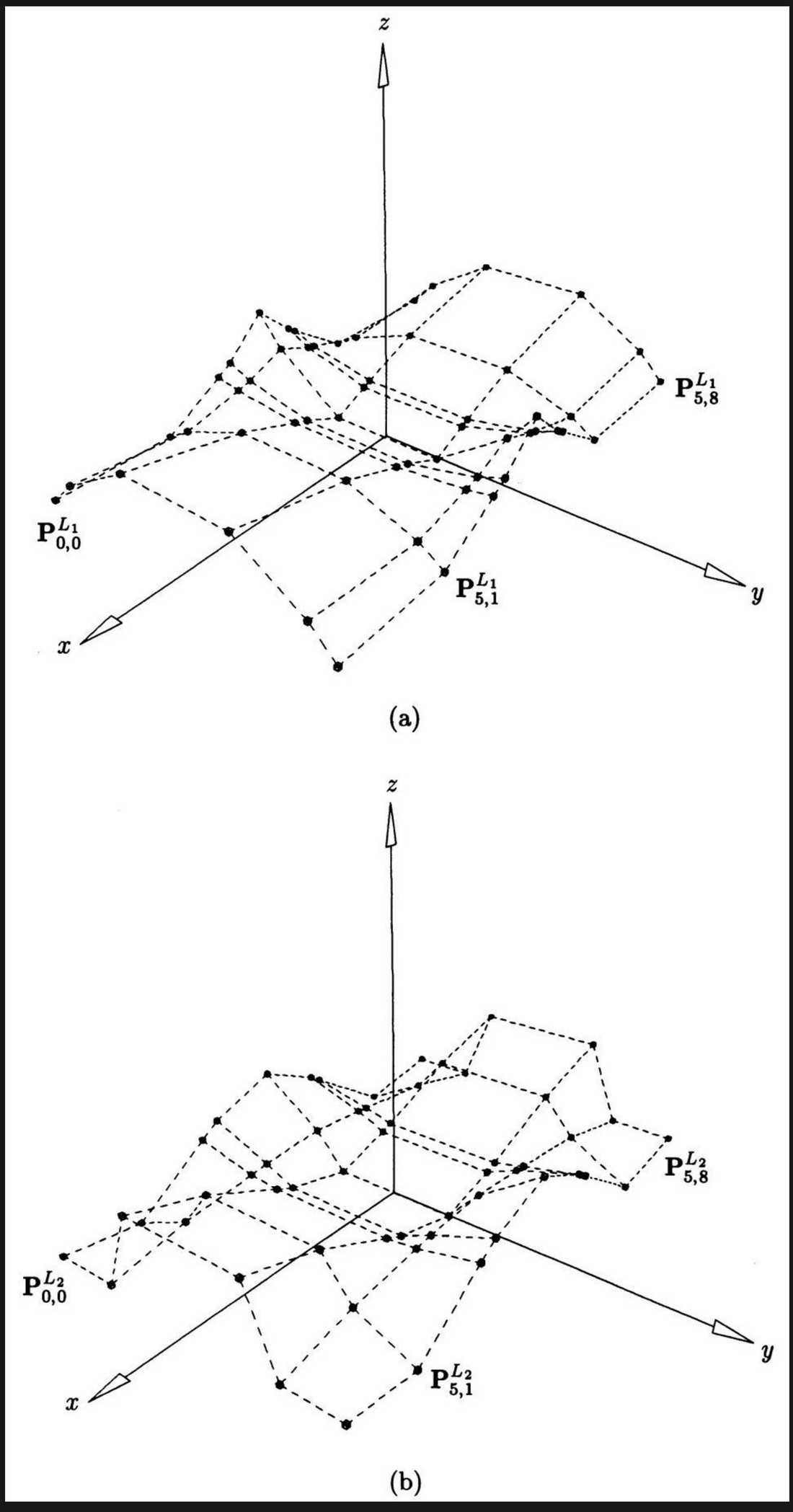


图 10.22. 节点细化以构造戈登表面。(a) 在 u 方向上细化的规则表面；(b) 在 v 方向上细化的规则表面。

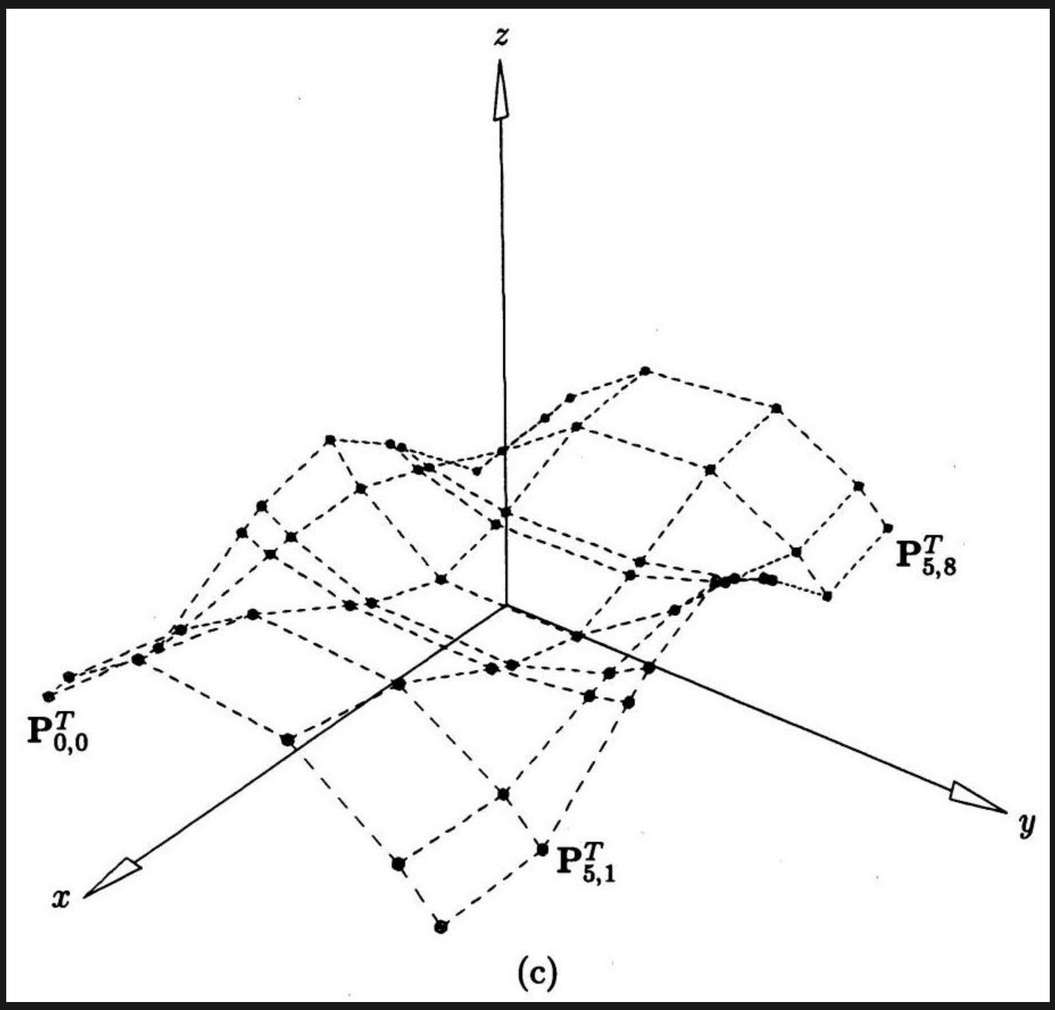


图 10.22. (续)(c) 在两个方向上细化的张量积。

完全一般，抬升和插值所用的度数 p1, q1, pt , qt 也作为输入。n, m, p, q, U, V, Pij 定义输出的戈登表面。

算法 A10.3

GordonSurface(Ck, Cl, ul, vk, r, s, pl, ql, pt, qt, n, m, p, q, U, V, Pij)

{ /* 创建戈登表面。*/

/* 输入: Ck, Cl, ul, vk, r, s, pl, ql, pt, qt */

/* 输出: n, m, p, q, U, V, Pij */

pc = Ck 曲线的度数。

qc = Cl 曲线的度数。

选择合适的 U1 和 loft 以获得 $L1(u,v)$ (度数 (pl, qc))。选择合适的 $V2$ 和 loft 以获得 $L2(u,v)$ (度数 (pc, ql))。选择合适的 UT 和 VT，并插值以获得 T(u, v) (度数 (pt, qt))。$p = \max(pc, pl, pt)$ 。q = max(qc, ql, qt)。根据需要将度数提升 $L1(u,v)$, $L2(u,v)$ 和 $T(u,v)$ 到 (p, q)。

U = Merge (U1, U2, UT) .

V = 合并 (V1, V2, VT)。

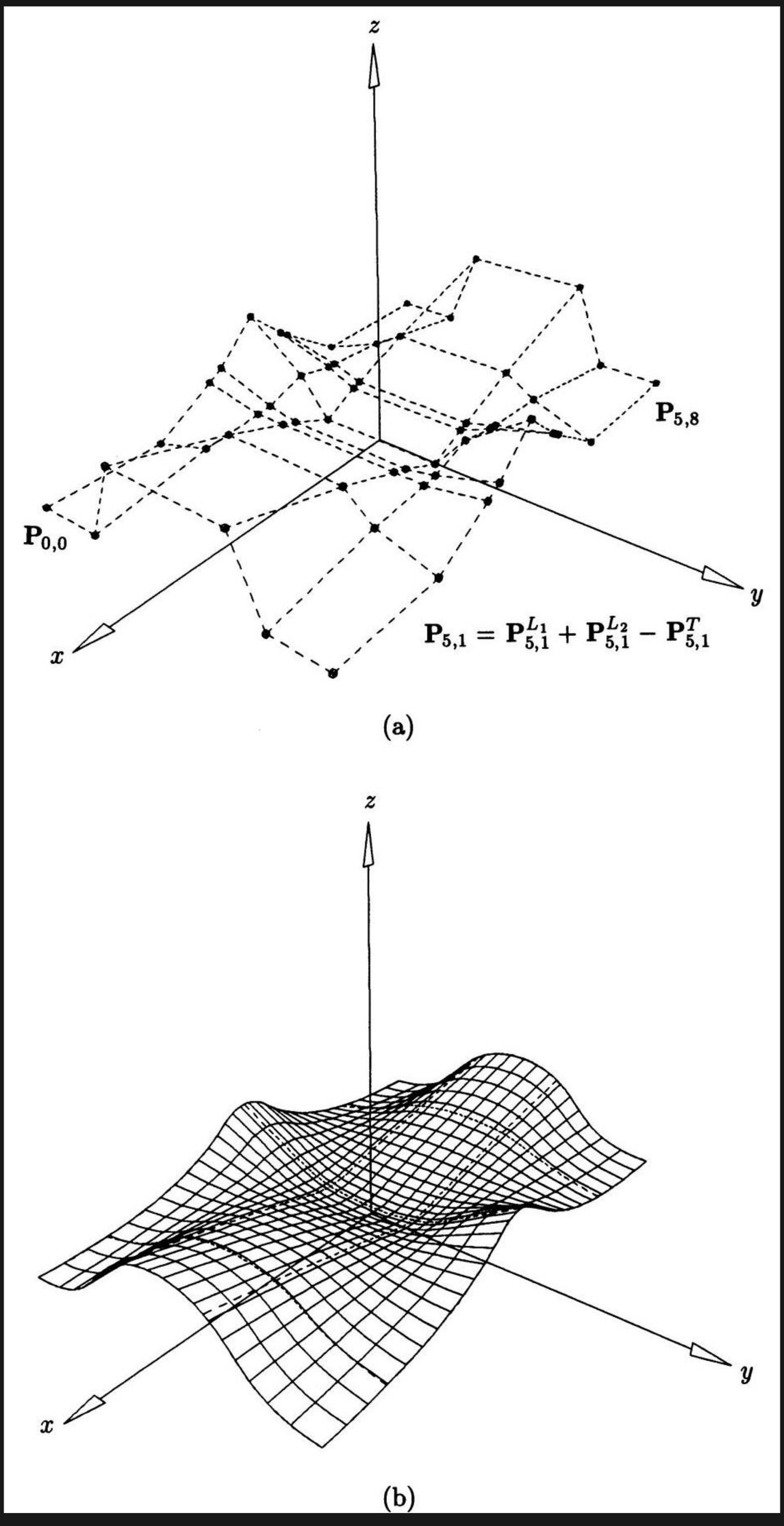


图 10.23. 一个戈登曲面。(a) 控制网；(b) 插值 u - 和 v - 方向曲面的表面。

Refine knot vectors of $L1\left(u,v\right)$, $L2\left(u,v\right)$ and $T\left(u,v\right)$
as necessary.
n = (number of u-knots)-p-2.
m = (number of v-knots)-q-2.
Compute surface control points Pij by Eq. (10.33).
}

6 库恩斯表面

现在假设我们有四条曲线

$$\mathbf{C}_k\left(u\right) = \sum_{i=0}^{n} N_{i,p}\left(u\right)\mathbf{P}_{k,i}\ k = 0,1\ u \in [0,1]$$

$$\mathbf{C}_\ell\left(v\right) = \sum_{j=0}^{m} N_{j,q}\left(v\right)\mathbf{P}_{\ell,j}\ \ell = 0,1\ v \in [0,1] \tag{10.34}$$

并且我们希望构造一个以这四条曲线为边界的曲面。此外，我们假设这些曲线满足兼容性条件

- 作为独立集合，它们在 B-spline 意义上是兼容的，即，两个 $\mathbf{C}_k\left(u\right)$ 定义在一个公共节点向量上，U ，而两个 $\mathbf{C}_\ell\left(v\right)$ 定义在一个公共节点向量上，V ；

-

$$\mathbf{S}_{0,0} = \mathbf{C}_{k=0}(u=0) = \mathbf{C}_{\ell=0}(v=0)$$

$$\mathbf{S}_{1,0} = \mathbf{C}_{k=0}(u=1) = \mathbf{C}_{\ell=1}(v=0)$$

$$\mathbf{S}_{0,1} = \mathbf{C}_{k=1}(u=0) = \mathbf{C}_{\ell=0}(v=1)$$

$$\mathbf{S}_{1,1} = \mathbf{C}_{k=1}(u=1) = \mathbf{C}_{\ell=1}(v=1) \tag{10.35}$$

(见图 10.24)。在本节中，我们不限制曲线为非有理的。这里给出的构造通常能产生令人满意的曲面，即使在齐次空间中进行。然而，这会使算法变得复杂。例如，方程 (10.35) 必须在齐次空间中成立。这要求角点权重兼容，这可能需要对曲线进行有理重新参数化 (第 6.4 节)。

双线性混合的 Coons 补丁定义为

$$\mathbf{S}(u,v) = \mathbf{R}_1(u,v) + \mathbf{R}_2(u,v) - \mathbf{T}(u,v) \tag{10.36}$$

其中 $\mathbf{R}_1(u,v)$ 和 $\mathbf{R}_2(u,v)$ 是 $\mathbf{C}_{k=0}(u)$ 和 $\mathbf{C}_{k=1}(u)$ 之间的规则曲面，以及 $\mathbf{C}_{\ell=0}(v)$ 和 $\mathbf{C}_{\ell=1}(v)$ 之间的规则曲面，而 $\mathbf{T}(u,v)$ 是双线性张量积-

uct 表面

$$\mathbf{T}(u,v) = \begin{bmatrix} 1 & u \end{bmatrix} \begin{bmatrix} \mathbf{S}_{0,0} & \mathbf{S}_{0,1} \\ \mathbf{S}_{1,0} & \mathbf{S}_{1,1} \end{bmatrix} \begin{bmatrix} 1 \\ v \end{bmatrix}$$

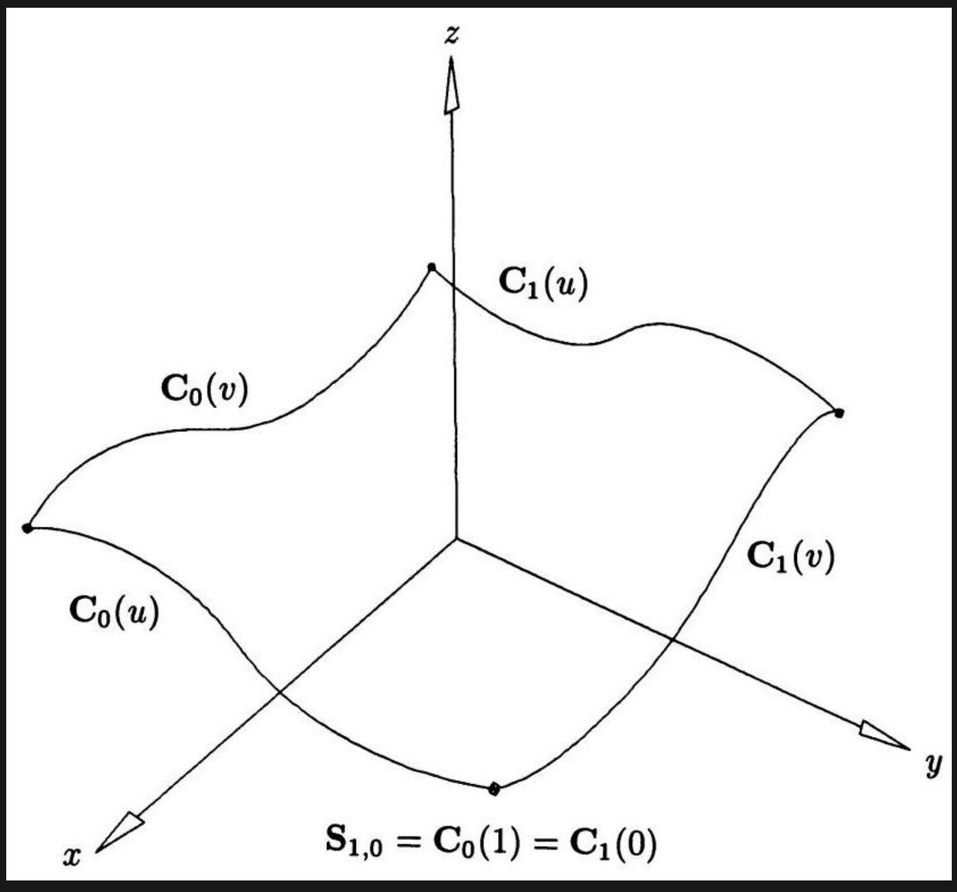
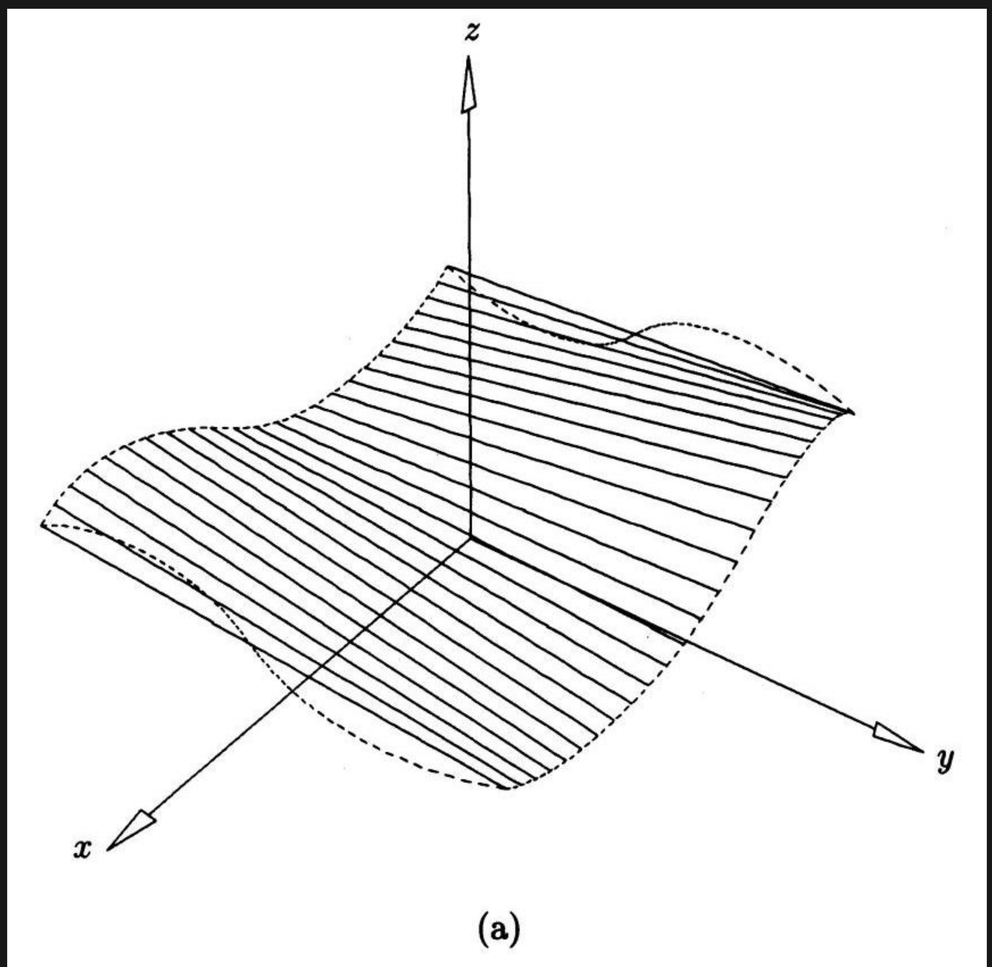


图 10.24. 定义双线性混合 Coons 补丁的兼容边界曲线。

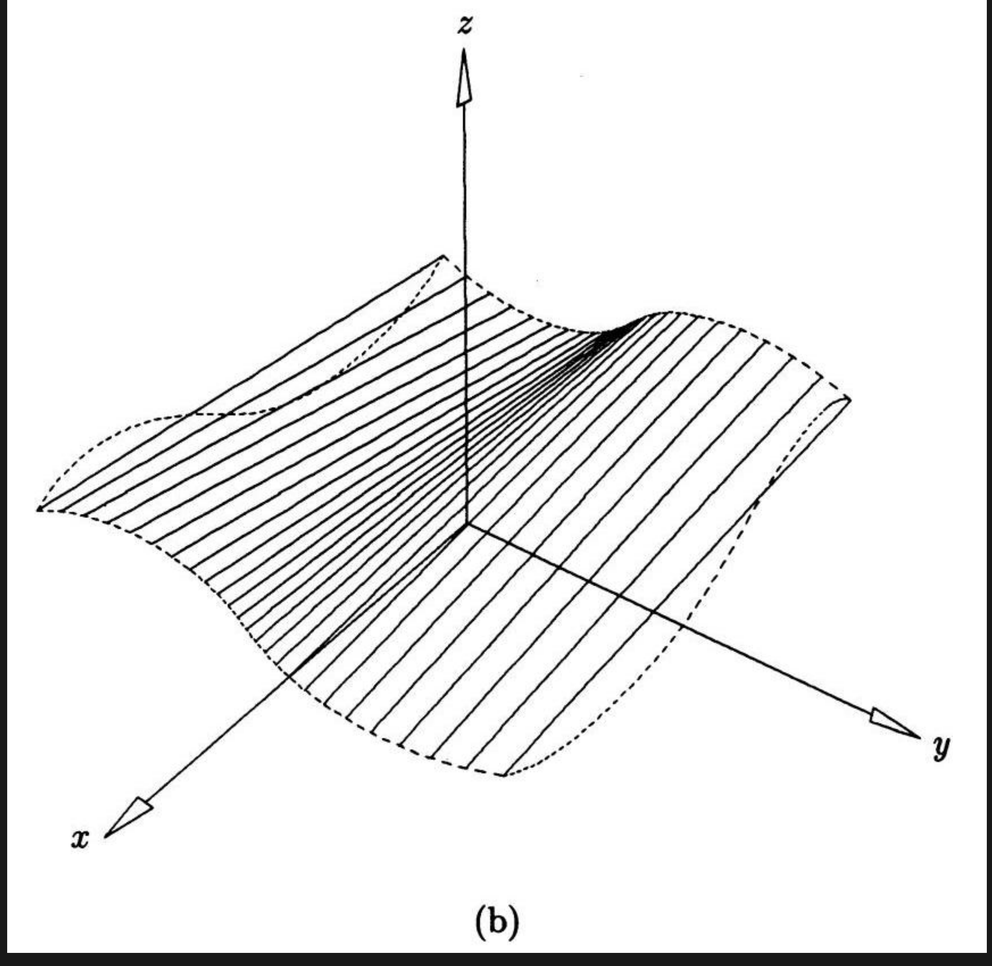
显然，这是戈登曲面的一个特例，因此可以使用公式 (10.33) 和算法 A10.3(经过简化并修改以处理有理曲线) 来获得由公式 (10.36) 给出的 Coons 曲面的 NURBS 表示。在这种情况下，输入参数 p1, q1, pt 和 qt 设置为 1。图 10.25a – 10.25 d 显示了由图 10.24 中的曲线定义的双线性混合 Coons 补丁的构造。图 10.25a 显示了在 u 方向上的一条规则曲面，图 10.25 b 则说明了 v 方向的规则曲面。图 10.25c 显示了由四个角点定义的双线性曲面。双线性混合 Coons 补丁在图 10.25d 中描绘。重要的是要注意，双线性混合 Coons 补丁通常不是双线性曲面。

Coons 在 1960 年代中期开发了他的补丁 [Coon67]。尽管我们将双线性混合 Coons 补丁呈现为戈登曲面的一个特例，但我们指出 Coons 的工作早于并影响了戈登曲面的发展。Coons 对在双向曲线网络中插值位置和切线信息感兴趣，戈登后来也有类似的兴趣。Coons 在每组四条曲线内构造了一个补丁，因此他的插值方法是局部的。戈登的方法则表示对曲线网络的全局插值。

如果两个双线性混合的 Coons 补丁共享一个公共边界曲线，它们通常只在该边界上具有 C^0 连续性；也就是说，组合表面将在公共边界上产生一个折痕。为了获得这些补丁的光滑、C^n 连续 ($n > 0$) 连接，Coons 开发了使用跨边界导数场的高阶补丁。为了说明这一点，我们展示了双立方混合的 Coons 补丁。设 $\mathbf{C}_k(u)$ 和 $\mathbf{C}_\ell(v)$ 为四个边界曲线，如公式 (10.34) 所示，满足给定的两个兼容性条件。我们还假设有四个跨边界导数场 (见图 10.26)。



(a)



(b)

图 10.25. 双线性混合的 Coons 补丁。(a) 在 u 方向的规则曲面；(b) 在 v 方向的规则曲面；(c) 一个双线性表面；(d) 一个双线性混合的 Coons 补丁。

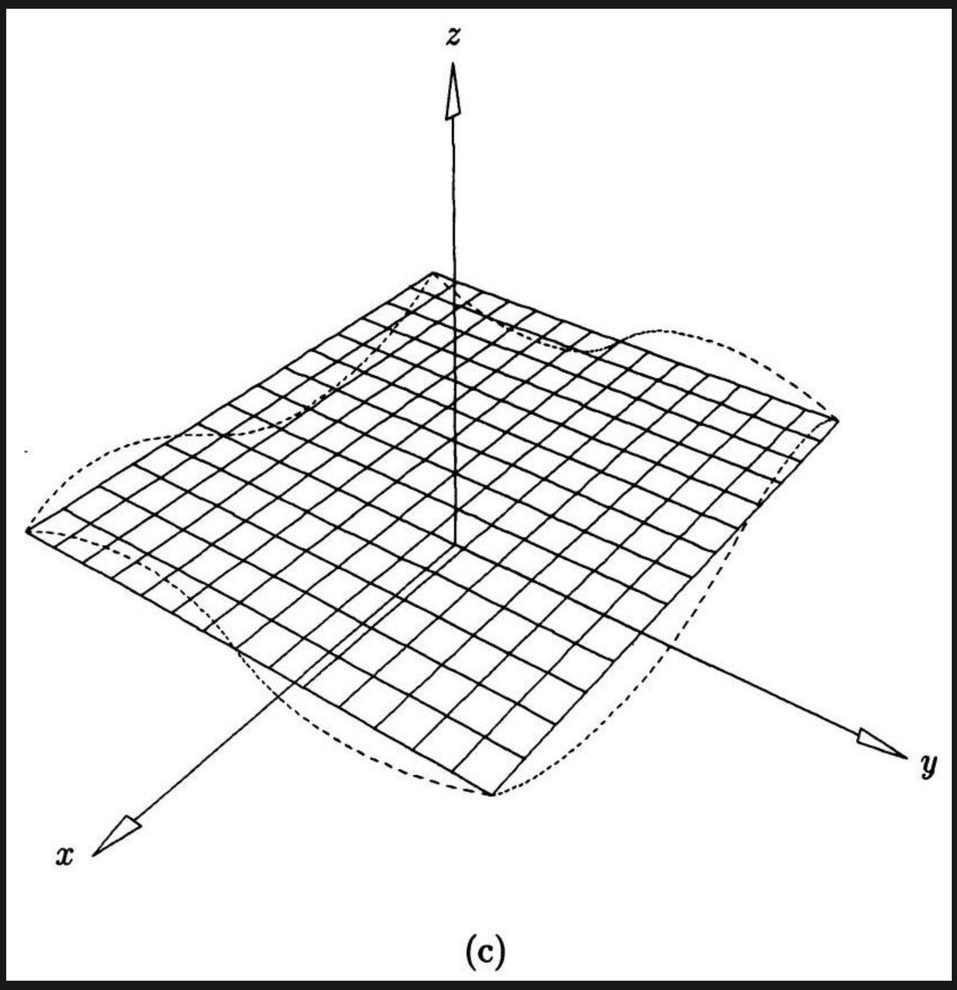


图 10.25. (续)

曲线如公式 (10.34) 所示，满足给定的两个兼容性条件。我们还假设有四个跨边界导数场 (见图 10.26)。

$$\mathbf{D}_k(u) = \sum_{i=0}^{n} N_{i,p}(u)\,\mathbf{Q}_{k,i} \; k = 0, 1 \; u \in [0,1]$$

$$\mathbf{D}_\ell(v) = \sum_{j=0}^{m} N_{j,q}(v)\,\mathbf{Q}_{\ell,j} \; \ell = 0, 1 \; v \in [0,1] \tag{10.37}$$

我们希望构造一个以 $\mathbf{C}_k(u)$ 和 $\mathbf{C}_\ell(v)$ 作为边界，并且以 $\mathbf{D}_k(u)$ 和 $\mathbf{D}_\ell(v)$ 作为沿边界的第一偏导数的表面，

也就是说

$$\mathbf{D}_{k=0}(u) = \mathbf{S}_v(u,0) \;\; \mathbf{D}_{k=1}(u) = \mathbf{S}_v(u,1)$$

$$\mathbf{D}_{\ell=0}(v) = \mathbf{S}_u(0,v) \;\; \mathbf{D}_{\ell=1}(v) = \mathbf{S}_u(1,v)$$

兼容性条件

- $\mathbf{C}_k(u)$ 和 $\mathbf{D}_k(u)$ 在 B-spline 意义上是兼容的，$\mathbf{C}_\ell(v)$ 和 $\mathbf{D}_\ell(v)$ 也是；

-

$$\mathbf{T}_{0,0} = \frac{d\mathbf{D}_{k=0}(u=0)}{du} = \frac{d\mathbf{D}_{\ell=0}(v=0)}{dv}$$

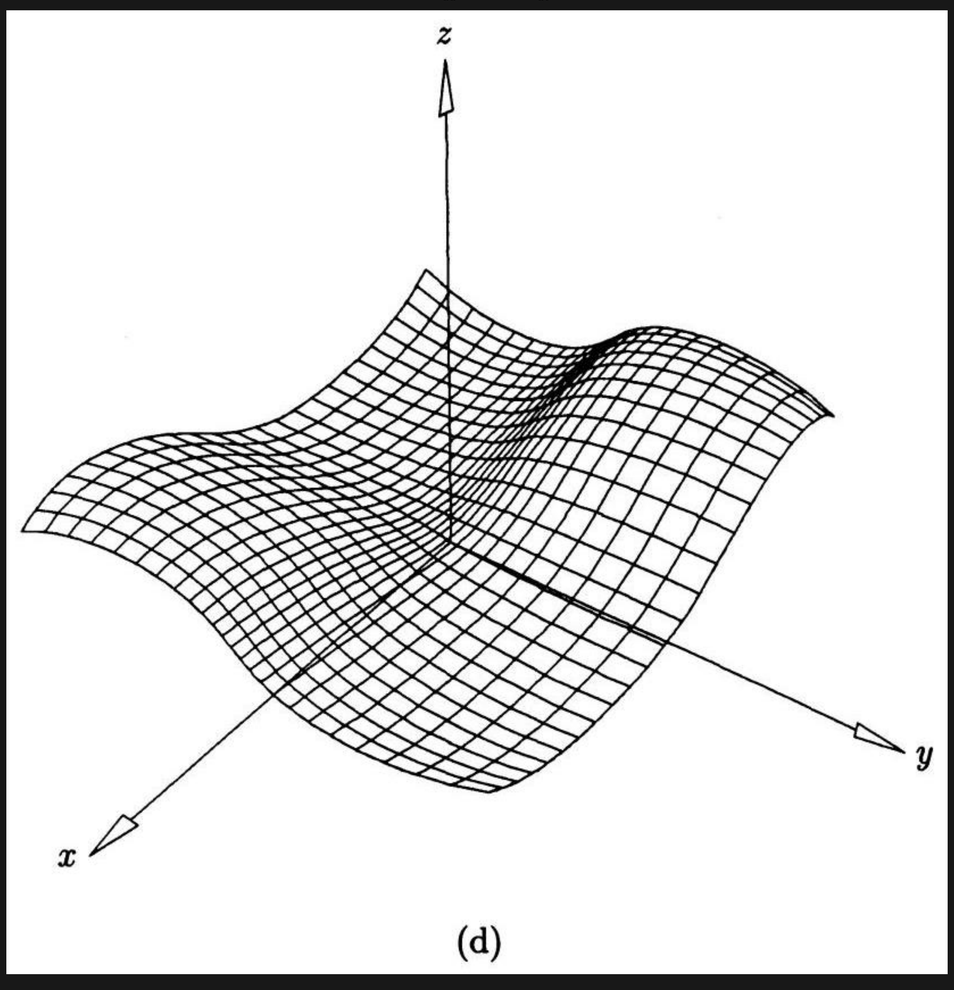


图 10.25. (续)

$$\mathbf{T}_{1,0} = \frac{d\mathbf{D}_{k=0}(u=1)}{du} = \frac{d\mathbf{D}_{\ell=1}(v=0)}{dv}$$

$$\mathbf{T}_{0,1} = \frac{d\mathbf{D}_{k=1}(u=0)}{du} = \frac{d\mathbf{D}_{\ell=0}(v=1)}{dv}$$

$$\mathbf{T}_{1,1} = \frac{d\mathbf{D}_{k=1}(u=1)}{du} = \frac{d\mathbf{D}_{\ell=1}(v=1)}{dv} \tag{10.38}$$

必须成立。$\mathbf{T}_{k,\ell}$ 是四个扭转向量。

双立方混合的 Coons 补丁定义为

$$\mathbf{S}(u,v) = \mathbf{S}_1(u,v) + \mathbf{S}_2(u,v) - \mathbf{T}(u,v) \tag{10.39}$$

其中 $\mathbf{S}_1(u,v)$ 是数据的三次混合 $\mathbf{C}_k(u), \mathbf{D}_k(u), k=0,1; \mathbf{S}_2(u,v)$ 是数据的三次混合 $\mathbf{C}_\ell(v), \mathbf{D}_\ell(v), \ell=0,1$ ； 而 $\mathbf{T}(u,v)$ 是一个双立方张量积表面 (由 16 个向量系数定义)。Coons 使用三次 Hermite 多项式来混合边界数据并形成 $\mathbf{T}(u,v)$ 。由于我们的目标是获得一个 NURBS 表面，我们使用三次 Bézier 方法进行混合并形成 $\mathbf{T}(u,v)$.$\mathbf{S}_1(u,v)$ ，它是在 v 方向上的三次 Bézier。其四行控制点的计算为 $(i=0,\ldots,n)$ 。

$$\mathbf{P}_{i,0}^{S_1} = \mathbf{P}_{k=0,i}$$

$$\mathbf{P}_{i,1}^{S_1} = \mathbf{P}_{k=0,i} + \frac{1}{3}\mathbf{Q}_{k=0,i}$$

$$\mathbf{P}_{i,2}^{S_1} = \mathbf{P}_{k=1,i} - \frac{1}{3}\mathbf{Q}_{k=1,i}$$

$$\mathbf{P}_{i,3}^{S_1} = \mathbf{P}_{k=1,i} \tag{10.40}$$

请注意，这两行内部行是使用 Bézier 导数公式 (方程 [1.10]) 推导而来的。$\mathbf{S}_2(u,v)$ 的控制点以类似方式计算，即。

$$\mathbf{P}_{0,j}^{S_2} = \mathbf{P}_{\ell=0,j}$$

$$\mathbf{P}_{1,j}^{S_2} = \mathbf{P}_{\ell=0,j} + \frac{1}{3}\mathbf{Q}_{\ell=0,j}$$

$$\mathbf{P}_{2,j}^{S_2} = \mathbf{P}_{\ell=1,j} - \frac{1}{3}\mathbf{Q}_{\ell=1,j}$$

$$\mathbf{P}_{3,j}^{S_2} = \mathbf{P}_{\ell=1,j} \tag{10.41}$$

$\mathbf{T}(u,v)$ 是一个双立方 Bézier 表面。方程 (1.10) 和 (3.24) 用于推导 16 个控制点，$\mathbf{P}_{i,j}^T, i,j=0,1,2,3$ 。作为一个例子，我们展示如何

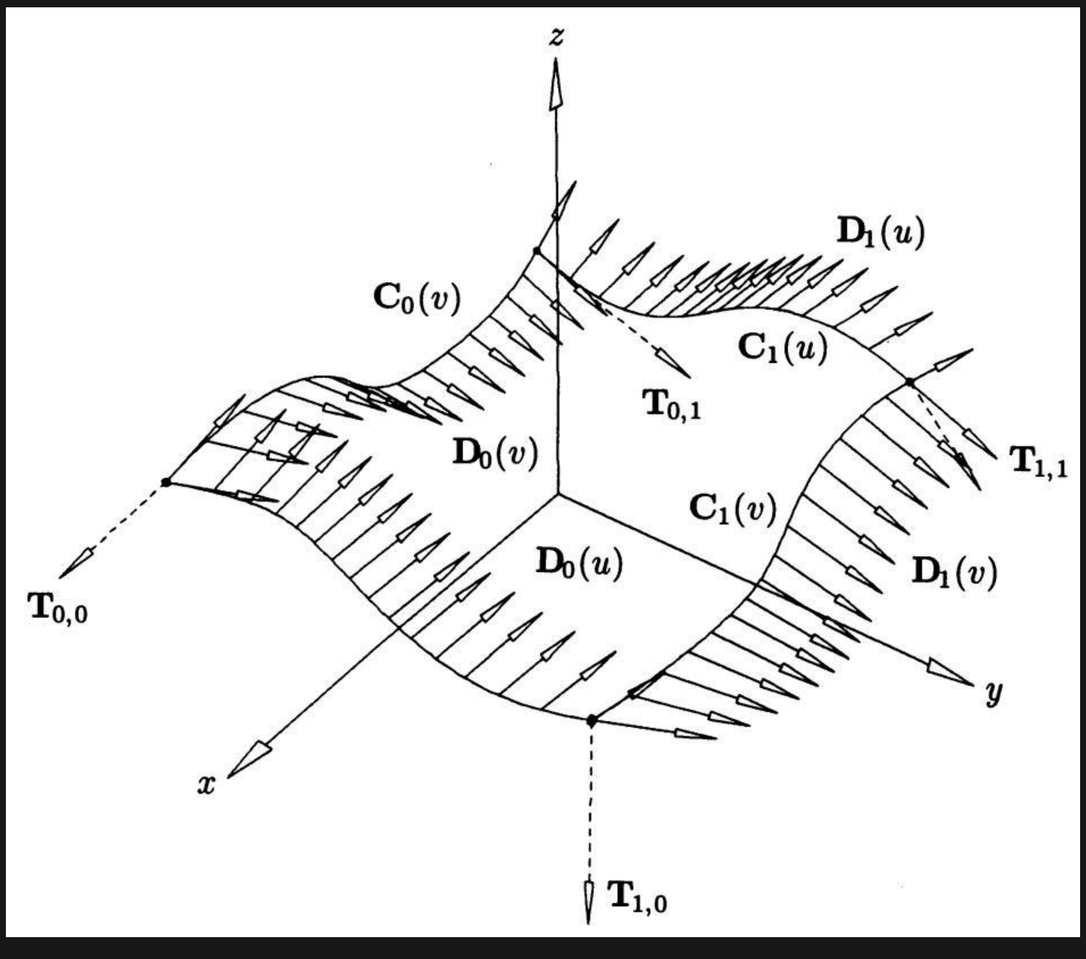


图 10.26. 兼容的边界曲线以及跨边界的切向量函数和扭曲。

计算与角落 $\mathbf{S}_{0,0}$ 相邻的四个控制点 $\mathbf{P}_{i,j}^T, i,j=0,1$ 。其他十二个控制点是类似的。

$$\mathbf{P}_{0,0}^T = \mathbf{S}_{0,0} = \mathbf{C}_{k=0}(u=0) = \mathbf{C}_{\ell=0}(v=0)$$

$$\mathbf{P}_{1,0}^T = \frac{1}{3}\mathbf{D}_{\ell=0}(v=0) + \mathbf{P}_{0,0}^T$$

$$\mathbf{P}_{0,1}^T = \frac{1}{3}\mathbf{D}_{k=0}(u=0) + \mathbf{P}_{0,0}^T$$

$$\mathbf{P}_{1,1}^T = \frac{1}{9}\mathbf{T}_{0,0} + \mathbf{P}_{1,0}^T + \mathbf{P}_{0,1}^T - \mathbf{P}_{0,0}^T \tag{10.42}$$

然后，表面 $\mathbf{S}_1(u,v), \mathbf{S}_2(u,v)$ 和 $\mathbf{T}(u,v)$ 必须通过提升次数和节点细化来实现兼容性。随后，Coons 表面的控制点 $\mathbf{P}_{i,j}$ 通过公式计算得出。

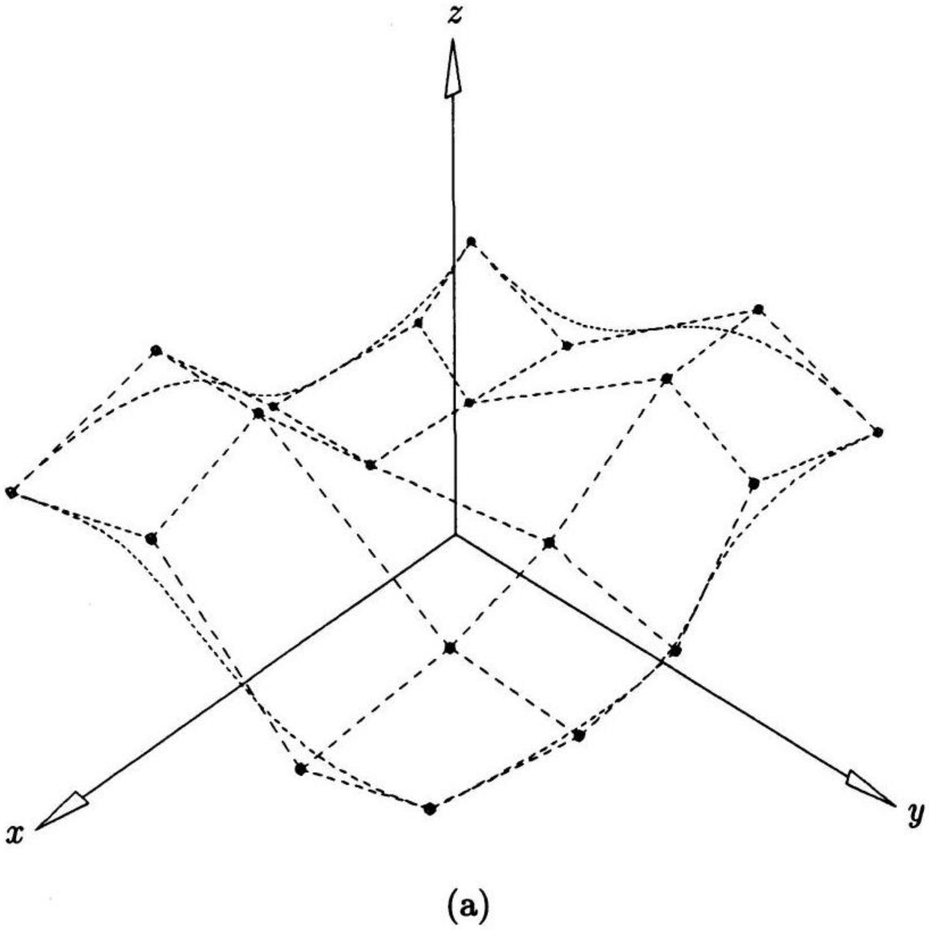
$$\mathbf{P}_{i,j} = \mathbf{P}_{i,j}^{S_1} + \mathbf{P}_{i,j}^{S_2} - \mathbf{P}_{i,j}^T \tag{10.43}$$

图 10.27-10.30 显示了构建双三次混合 Coons 补丁的过程。图 10.27a 和 10.27b 显示了一个 u 方向的三次混合表面，$\mathbf{S}_2(u,v)$ ，以及定义控制网。v 方向的三次混合在图 10.28a 和 10.28 b ($\mathbf{S}_1(u,v)$) 中进行了说明。图 10.29a 和 10.29 b 描绘了一个双三次张量积表面。最后，在进行适当的次数提升和节点细化后，图 10.30a 和 10.30b 展示了 Coons 补丁。

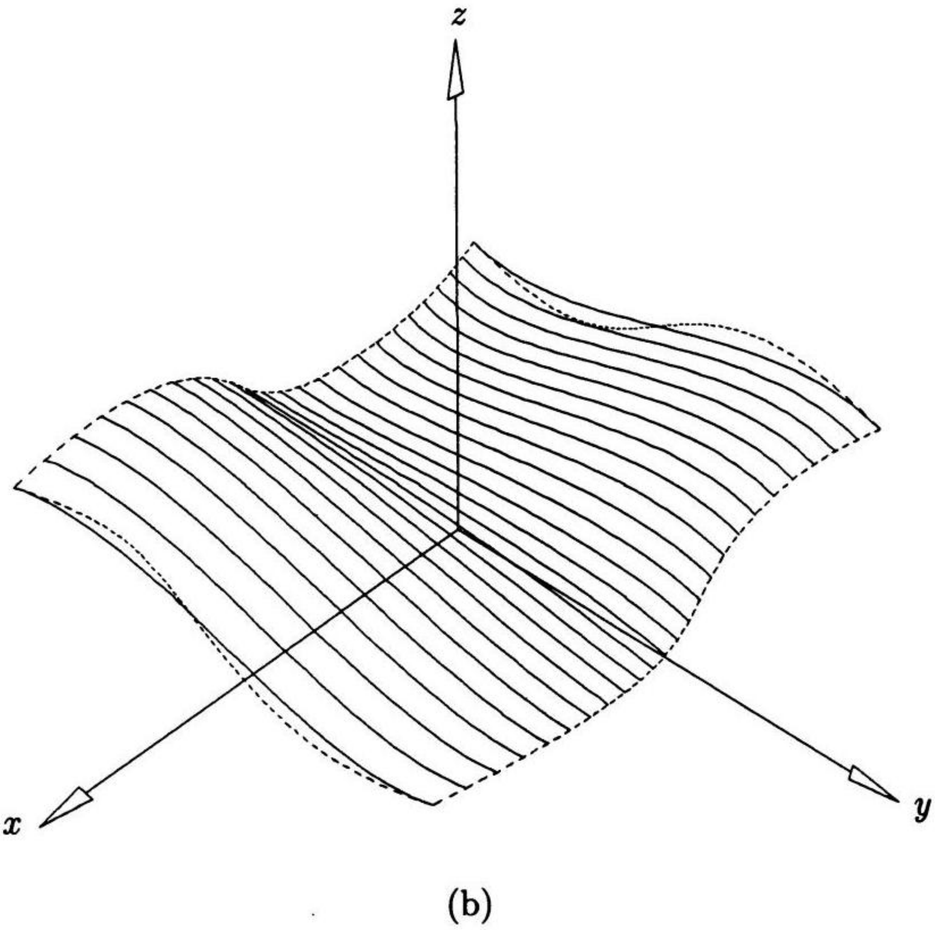
在算法 A10.4 中，Ck[]、Cl[] 和 Dk[]、Dl[] 分别是边界曲线和跨边界导数的数组。n,m,p,q,U,V ，Pij 定义了输出的双三次混合 Coons 表面。

ALGORITHM A10.4

```
BicubicBlendCoons(Ck, Cl, Dk, Dl, n, m, p, q, U, V, Pij)
  { /* Create bicubically blended Coons surface. *
      /* Input: Ck, Cl, Dk, Dl */
      /* Output: n, m, p, q, U, V, Pij */
    Compute the ctrl pts of S1(u,v) (Eq.10.40).
    Compute the ctrl pts of S2(u,v) (Eq.10.41).
    Compute the ctrl pts of T(u,v) (Eq.10.42).
    pcd = degree of the Ck curves and Dk derivatives.
    qcd = degree of the Cl curves and Dl derivatives.
    p = max(3, pcd);
    q = max(3, qcd);
    Degree elevate S1(u,v), S2(u,v) and T(u,v) to(p, q)
        if necessary.
    U = u− knot vector of S1(u,v) .
    V = v− knot vector of S2(u,v) .
```



(a)



(b)

图 10.27. u 方向的三次混合表面。(a) 控制网；(b) 表面 (在 u 方向的 Bézier 和在 v 方向的 B-spline)。

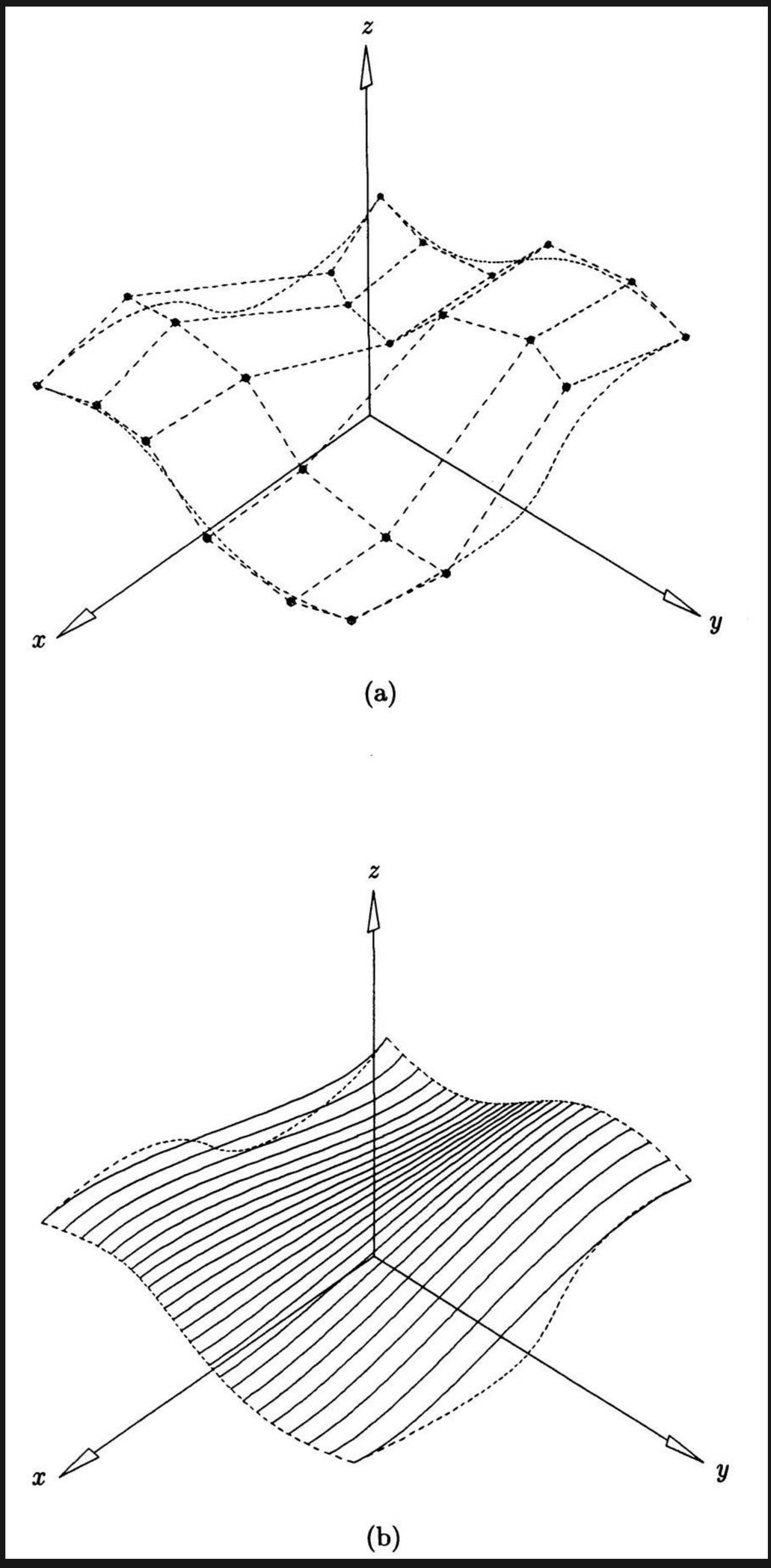
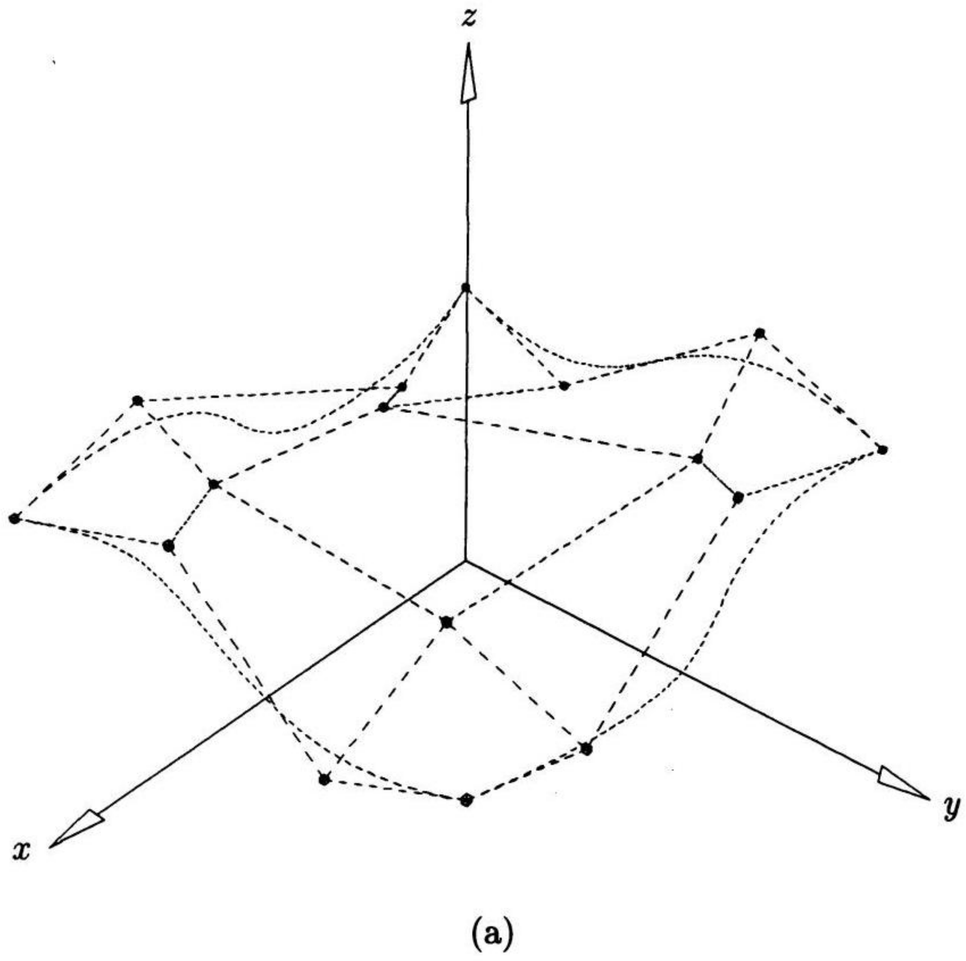
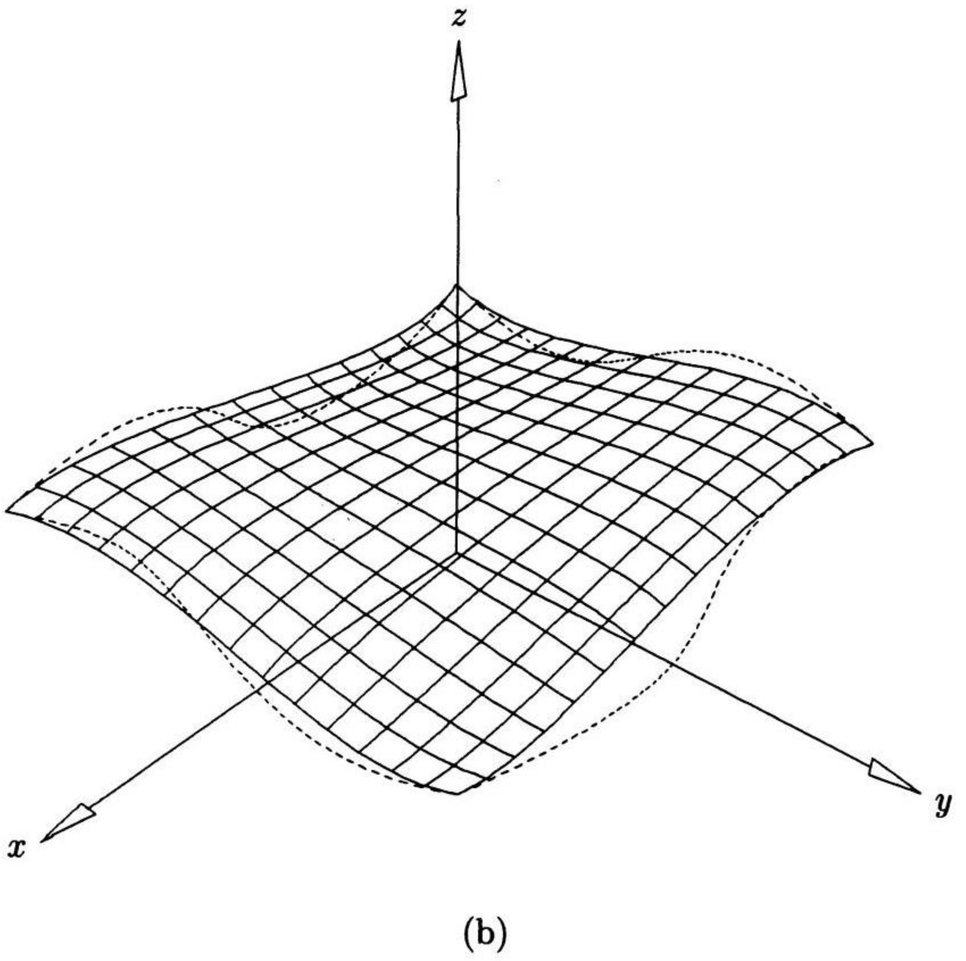


图 10.28. v 方向的三次混合表面。(a) 控制网；(b) 表面 (在 u 方向的 B-spline 和在 v 方向的 Bézier)。



(a)



(b)

图 10.29. 双三次张量积表面。(a) 控制网；(b) 双三次 Bézier 表面。

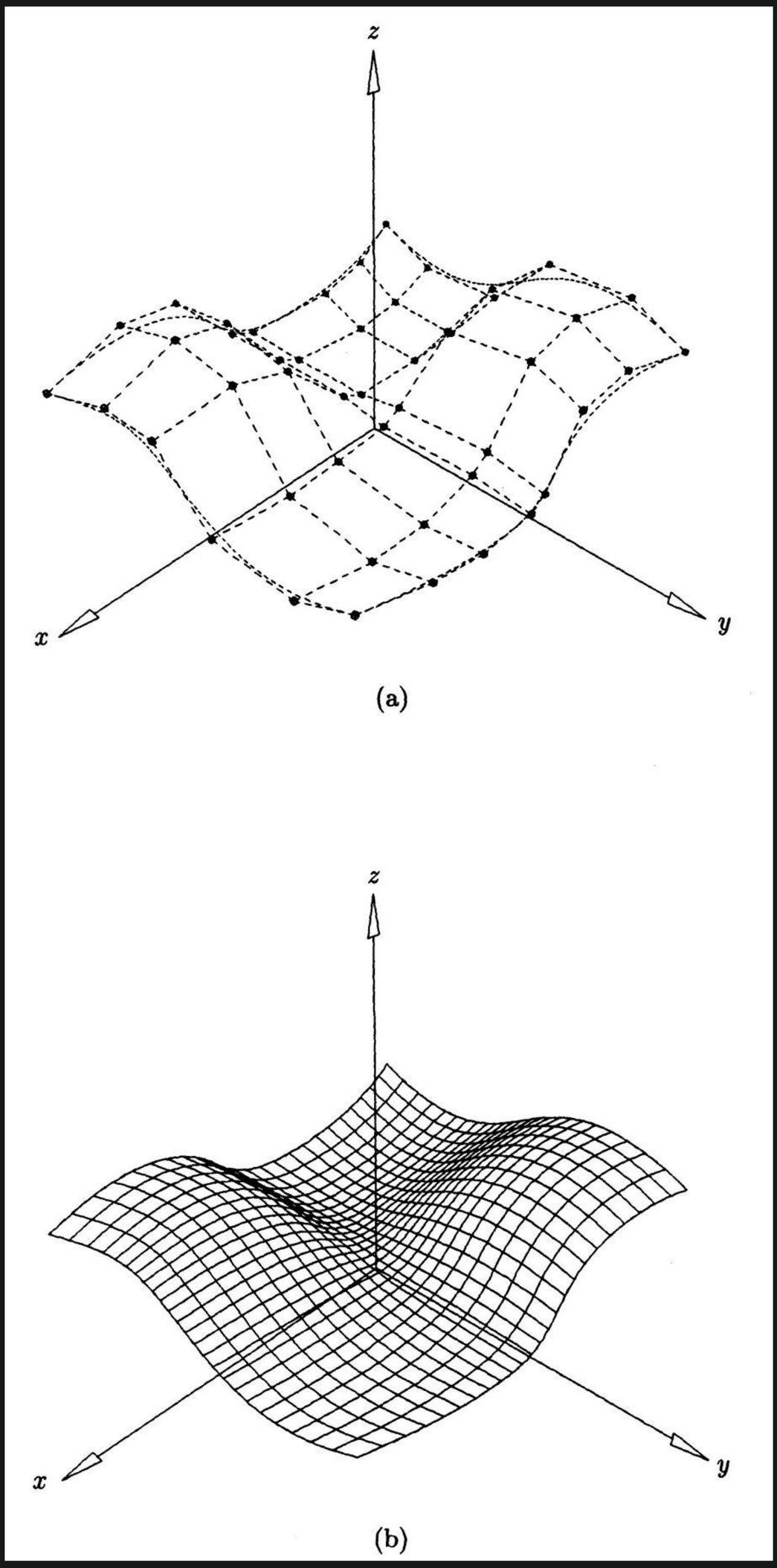


图 10.30. 双三次混合 Coons 表面。(a) 次数提升和节点细化后的控制网；(b) Coons 表面。

Knot refine $\mathbf{S}_1(u,v)$, $\mathbf{S}_2(u,v)$ and $\mathbf{T}(u,v)$ to

U and V if necessary.

n = (number of u-knots)-p-2.

m = (number of v-knots)-q-2.

Compute surface control points $\mathbf{P}_{i,j}$ by Eq. (10.43).

}

在实践中，构建一个双立方混合的 Coons 补丁并不容易。首先，跨边界的导数场很少可用；即使可用，导数的大小也可能不合适。此外，人们几乎无法期望扭转向量兼容性 (方程 [10.38]) 成立，也就是说，通常情况下

$$\frac{d\mathbf{D}_{k=0}(u=0)}{du} \neq \frac{d\mathbf{D}_{\ell=0}(v=0)}{dv}$$

其他三个角落也是如此。这个补丁最常见的用途是试图在表面拼接中平滑地填补一个三边或四边的孔，符合 G^1 的意义。边界数据是从相邻表面提取的。然而，在这种情况下，导数的大小通常没有意义，扭转向量不兼容，因此补丁与其相邻表面之间的 G^1 连续性是不可能的。如果应用可以容忍跨越边界的切向方向的小不连续性，则可以指定一个角度容差，并采取以下两种方法之一：

- 使用第 9 章的迭代逼近技术来获得扭转兼容的非理性导数场，并且在指定的角度容差内逼近原始导数场的向量方向。如果任何一个边界曲线是理性的，那么问题将变得更加复杂，因为扭转向量和导数场必须以某种方式嵌入齐次空间，以获得四维兼容性，同时仍保持三维向量方向上的角度容差。方程 (4.7) 和 (4.26) 在这个过程中是有用的；
- 让

$$\mathbf{T}^k_{0,0} = \frac{d\mathbf{D}_{k=0}(u=0)}{du}\ \mathbf{T}^\ell_{0,0} = \frac{d\mathbf{D}_{\ell=0}(v=0)}{dv}$$

$\mathbf{T}_{0,0}^{k}$ 和 $\mathbf{T}_{0,0}^{\ell}$ 是从以下控制点计算得出的: $\mathbf{D}_{k=0}(u)$ 和 $\mathbf{D}_{\ell=0}(v)$: $\mathbf{Q}_{k=0,0}, \mathbf{Q}_{k=0,1}$ 以及 $\mathbf{Q}_{\ell=0,0}, \mathbf{Q}_{\ell=0,1}$ 。 设置

$$\mathbf{T}_{0,0}=\frac{1}{2}\left(\mathbf{T}_{0,0}^{k}+\mathbf{T}_{0,0}^{\ell}\right)$$

在保持 $\mathbf{Q}_{k=0,0}$ 和 $\mathbf{Q}_{\ell=0,0}$ 不变的情况下，可以修改 $\mathbf{Q}_{k=0,1}$ 和 $\mathbf{Q}_{\ell=0,1}$ 以产生共同的扭曲 $\mathbf{T}_{0,0}$ 。在修改 $\mathbf{Q}_{k=0,1}$ 和 $\mathbf{Q}_{\ell=0,1}$ 之前应用的结插入可以用来限制角度误差并限制其影响范围。如果任何曲线或导数场是有理的，过程会更加复杂。

对于对 Coons 补丁的更深入和更广泛的研究，读者应参考 Barnhill 的文章 [Barn93] 及其中引用的文献。

第十一章　形状修改工具

1　引言

本章的目的是开发工具，使设计师能够以自然和直观的方式对现有的 NURBS 曲线或曲面进行交互式的局部修改。NURBS 曲线或曲面由其控制点、权重和节点定义；修改其中任何一个都会改变曲线或曲面的形状。通常，设计师不希望处理这些概念；他们更愿意直接指定适用于曲线或曲面的约束和形状。在本章中，我们开发的工具将这些约束和形状规范转换为控制点位置或权重值的修改。具体而言，第 11.2 节和第 11.3 节基于指定点约束 (即曲线或曲面要局部变化，以便通过给定点)；第 11.5 节处理多个点和导数约束，允许控制切线和曲率；第 11.4 节的方法允许设计师指定曲线或曲面在选定区域内应呈现的“形状”。这些几何直观的输入在第 11.2、11.4 和 11.5 节中转换为控制点重新定位，在第 11.3 节中转换为权重修改。控制点和权重对形状的影响在几何上是清晰和直观的。这在之前的图形中已经定性地展示过，例如图 3.7、3.23、4.4、4.6、4.11 和 4.12；本章量化了这种影响。尽管节点位置也会影响形状，但我们不知道这种影响的几何直观或数学简单的解释，因此我们不提供移动节点位置的技术。

本章的方法是局部的，因为 B-spline 的局部支持特性。然而，可以通过节点细化将要改变的曲线或表面区域的范围缩小，或通过节点移除将其范围扩大。交互性要求快速解决方案，这反过来意味着要么是简单的方程，要么是小的线性系统，或者是一个多阶段的解决方案，其中关键的交互阶段是快速的。第 11.2 节和第 11.3 节的方法简单且非常快速。第 11.4 节的形状算子需要将节点细化和节点移除作为前处理和后处理步骤。第 11.5 节中的技术需要将 (通常较小的) 矩阵反转作为前处理步骤。我们详细发展本章的数学，但仅提供可能的用户交互和算法组织的概要。用户界面和图形交互超出了本书的范围，而算法组织可以依赖于这些以及应用领域。

在“形状控制与修改”这一广泛类别下，许多有趣的话题被排除在本章之外——必须在某处划定界限。有关“修整”或“平滑”的文献正在不断增加。对于这些概念的定义并没有普遍的共识，但它们通常涉及几何连续性、拐点、凸性和曲率等属性。该领域的许多解决技术导致约束非线性最小化问题。因此，在交互速度下解决这些问题是困难的。我们建议读者参考 Su 和 Liu [Su89] 以及 Hoschek 和 Lasser [Hosc93] 的书籍及其众多参考文献。Welch 和 Witkin [Welc92] 描述了一种表面设计方法，其中他们最小化表面上第一和第二基本形式的加权和的积分。这些形式测量表面的拉伸和弯曲 [DoCa76]。设计师不仅可以指定点和导数，还可以指定超限约束 (例如，嵌入曲线)。他们的方法导致具有线性约束的二次目标函数的约束最小化。

一种称为自由形状变形 (FFD) 的技术可以用于粗略造型、变形和雕刻曲线和表面的集合 (例如，可以形成一个封闭的物体)。几何体被认为嵌入在一个三维塑料环境中，该环境由三变量张量积贝塞尔或 B 样条函数定义 (将 (u, v, w) 参数映射到 (x, y, z) 空间)。通过移动控制点来变形周围的塑料 $\mathbf{P}_{i,j,k}$，物体几何体继承这种变形。有关 FFD 的进一步阅读可以在 [Sede86; Coqu90, 91; Hsu92] 中找到。

B-spline 曲面 (曲线) 可以叠加形成一个新的曲面 (曲线)。换句话说，如果一组曲面 $\mathbf{S}_1, \ldots, \mathbf{S}_n$ 都在相同的 u 和 v 节点向量上定义，则新曲面由相同的节点向量和控制点定义。

$$\mathbf{P}_{i,j} = \alpha_1 \mathbf{P}_{i,j}^1 + \cdots + \alpha_n \mathbf{P}_{i,j}^n$$

其中 $\{\alpha_k\}$ 是标量，$\left\{\mathbf{P}_{i,j}^k\right\}$ 是定义 $\mathbf{S}_k$ 的控制点。Gordon 和 Coons 曲面的 NURBS 构造就是这种情况的例子 (见方程 [10.33] 和 [10.43])。这表明可以使用“加法”或“修正”曲面或曲面补丁。可以使用小而简单的部分，然后将它们叠加形成一个新曲面或修改现有曲面的某个区域。边界条件易于控制。几位作者已经使用这些技术进行交互式曲面设计 (见 [Fors88; Mats92; Welc92])。

2　控制点重新定位

设 $\mathbf{C}(u) = \sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i$ 为一个有理或非有理的 B-spline 曲线。我们想要重新定位一个任意控制点 $\mathbf{P}_k, 0 \le k \le n$。用 $\widehat{\mathbf{P}}_k$ 表示新的、平移后的控制点，用 $\mathbf{V} = \widehat{\mathbf{P}}_k - \mathbf{P}_k$ 表示平移向量。则新曲线 $\widehat{\mathbf{C}}(u)$ 由下式给出:

$$\begin{aligned}\widehat{\mathbf{C}}(u) &= R_{0,p}(u)\,\mathbf{P}_0 + \cdots + R_{k,p}(u)\,(\mathbf{P}_k + \mathbf{V}) + \cdots + R_{n,p}(u)\,\mathbf{P}_n \\ &= \mathbf{C}(u) + R_{k,p}(u)\,\mathbf{V} \end{aligned} \tag{11.1}$$

方程 (11.1) 表达了对所有曲线点 $\mathbf{C}(u)$ 的功能性平移，其中 $u \in [u_k, u_{k+p+1})$；该区间外的所有曲线点不受影响。最大平移发生在函数 $R_{k,p}(u)$ 的最大值处。图 11.1 说明了这一点。

现在假设 $\bar{u}$ 是一个固定的参数值，具有 $\bar{u} \in [u_k, u_{k+p+1})$，并且 $\mathbf{P} = \mathbf{C}(\bar{u})$。如果我们想通过重新定位 $\mathbf{P}_k$ 在方向 $\mathbf{V}$ 上移动 $\mathbf{P}$ 的距离 d，那么我们必须确定 α，使得

$$\widehat{\mathbf{P}}_k = \mathbf{P}_k + \alpha\mathbf{V} \tag{11.2}$$

影响所需的平移。设 $\widehat{\mathbf{P}} = \widehat{\mathbf{C}}(\bar{u})$。显然

$$\left|\widehat{\mathbf{P}} - \mathbf{P}\right| = d = \alpha\,|\mathbf{V}|\,R_{k,p}(\bar{u})$$

这将产生

$$\alpha = \frac{d}{|\mathbf{V}|\,R_{k,p}(\bar{u})} \tag{11.3}$$

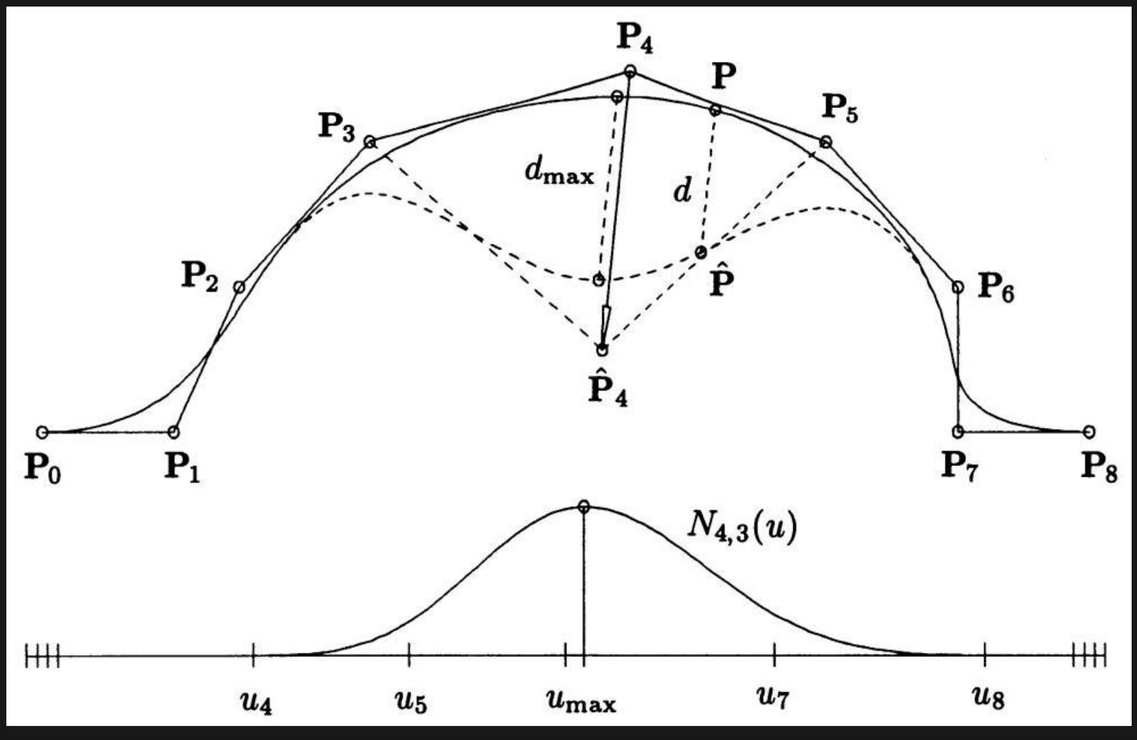


图 11.1。通过重新定位 $\mathbf{P}_4$ 移动 $\mathbf{P} = \mathbf{C}(\bar{u} = 3/5)$；相应的节点是 $t_4 = 0.52$，曲线的度数为三。

从用户界面的角度来看，控制点重新定位有两种广泛的方法，区别在于用户是选择控制多边形还是曲线。在第一种情况下，用户可能会指定一个向量方向和距离，然后选择控制多边形上的一个点。如果选择的点是一个现有的控制点，系统会简单地平移它并重新绘制曲线。如果不是，那么系统通过逆节点插入将该点变为控制点 (见第 5 章，方程 [5.19]-[5.21])，然后平移新的控制点并重新绘制曲线。在第二种情况下，设计师选择一个曲线点 $\mathbf{P}$ (系统由此计算 $\bar{u}$)，然后选择要移动到的点 $\widehat{\mathbf{P}}$ (系统由此计算 d 和 $\mathbf{V}$)。系统需要选择一个控制点 $\mathbf{P}_k$ 进行平移，以产生 $\mathbf{P}$ 所需的移动。一般来说，最多有 $p+1$ 个候选者，但通常希望选择 $\mathbf{P}_k$，使得 $R_{k,p}(u)$ 成为其最大值最接近 $\bar{u}$ 的基函数。一个通常相当接近 (但不一定重合于) 任意基函数最大值的参数值 $R_{i,p}(u)$ 被称为节点 t_i，其定义为 (见 [Ries73])。

$$t_i = \frac{1}{p}\sum_{j=1}^{p} u_{i+j} \quad i = 0, \ldots, n \tag{11.4}$$

注意，t_i 是位于 $R_{i,p}(u)$ 非零域内的节点的平均值。对于交互式控制点重新定位，我们建议基于节点值选择 k，而不是实际计算 $R_{i,p}(u)$ 的最大值；也就是说，选择 k 使得

$$|\bar{u} - t_k| = \min_i |\bar{u} - t_i| \tag{11.5}$$

当然，这些节点可以作为起始值，通过牛顿迭代法来寻找最大值。

图 11.2 和 11.3 显示了两个控制点重新定位的情况，其中索引 k 是基于 $\bar{u}$ 与节点的接近度选择的。注意，图 11.2 是令人愉悦的，而图 11.3 是不对称且不令人愉悦的。原因在于，在图 11.2$\bar{u}$ 中非常接近节点 t_k (并且接近 $R_{k,p}(u)$ 的最大值)，因此曲线的平移在 $\mathbf{C}(\bar{u})$ 周围是相当对称的，并且在那里大致达到最大值。在图 11.3 中，$\bar{u}$ 位于 t_k 和 t_{k+1} 之间的中点 (但选择了 t_k)，因此最大平移发生在 $\mathbf{C}(\bar{u})$ 的左侧。对此困境有两种解决方案:

- 插入一个节点，以便将一个节点放置在 $\bar{u}$；
- 重新定位 $\mathbf{P}_k$ 和 $\mathbf{P}_{k+1}$。

让我们插入节点 $\widehat{u}$，这迫使 t_{k+1} (新的 $(k+1)$ 阶节点) 具有值 $\bar{u}$。从方程 (11.4) 可以清楚地看出 $u_{k+1} < \widehat{u} < u_{k+p+1}$，因此

$$\bar{u} = t_{k+1} = \frac{1}{p}\left(\widehat{u} + \sum_{j=2}^{p} u_{k+j}\right)$$

和

$$\widehat{u} = p\bar{u} - \sum_{j=2}^{p} u_{k+j} \tag{11.6}$$

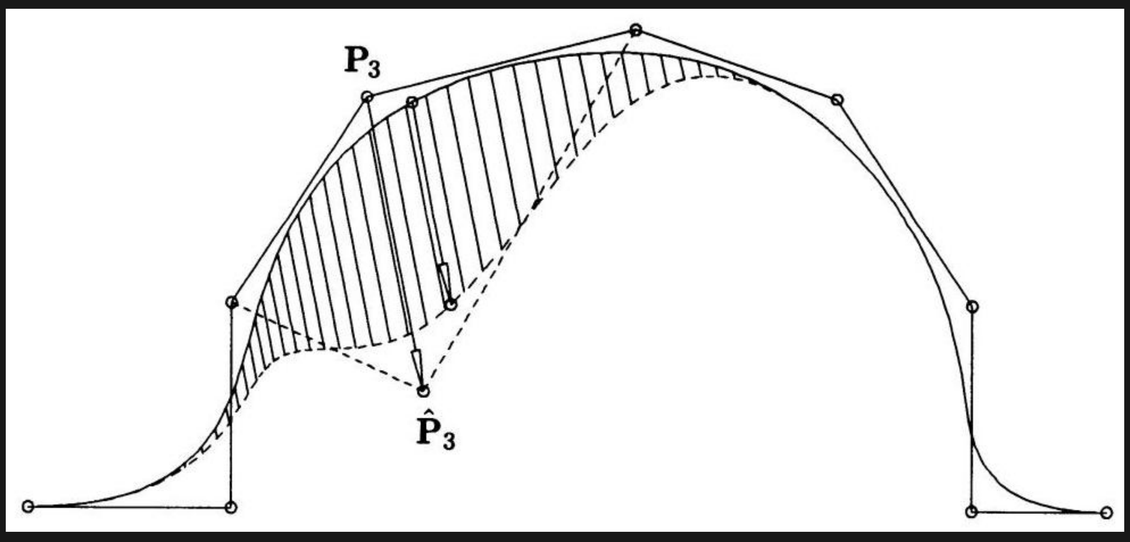


图 11.2。通过重新定位 $\mathbf{P}_3$ 移动 $\mathbf{P}=\mathbf{C}(\bar{u}=0.37)$ ； 相应的节点是 $t_3=0.35$ ， 曲线的度数为三。注意曲线一段的平行平移。

图 11.4 显示了图 11.3 的曲线。将来自方程 (11.6) 的节点 $\widehat{u}$ 插入，以强制 $\bar{u}$ 成为第 $(k+1)$ 个节点。然后将得到的 $\mathbf{Q}_{k+1}$ 移动以实现所需的曲线修改。请注意对称性和变化域的减小。我们提醒读者，$\widehat{u}$ 可能已经是一个节点，插入它可能是不必要的。例如，设 $\mathbf{C}(u)$ 为图 11.5 中的二次曲线。节点为 $t_i=\{0,1/8,3/8,5/8,7/8,1\}$ 。如果设计者选择了点，使得 $\bar{u}=1/2$ ，那么根据方程 (11.6) 有 $\widehat{u}=1/2$ 。如果插入 $\widehat{u}=1/2$ ，则 $\bar{u}$ 成为节点 t_3 ，如果移动 $\mathbf{Q}_3(=\mathbf{C}(1/2))$ ，则得到的曲线在 $u=1/2$ 处有一个尖点，如图 11.5 所示。

假设我们现在想要沿着 $\mathbf{V}$ 移动 $\mathbf{P}=\mathbf{C}(\bar{u})$ 一个距离 d ，通过在 $\mathbf{V}$ 方向上平移 $\mathbf{P}_k$ 和 $\mathbf{P}_{k+1}$ (假设 $\bar{u}\in[u_{k+1},u_{k+p+1})$)。设定

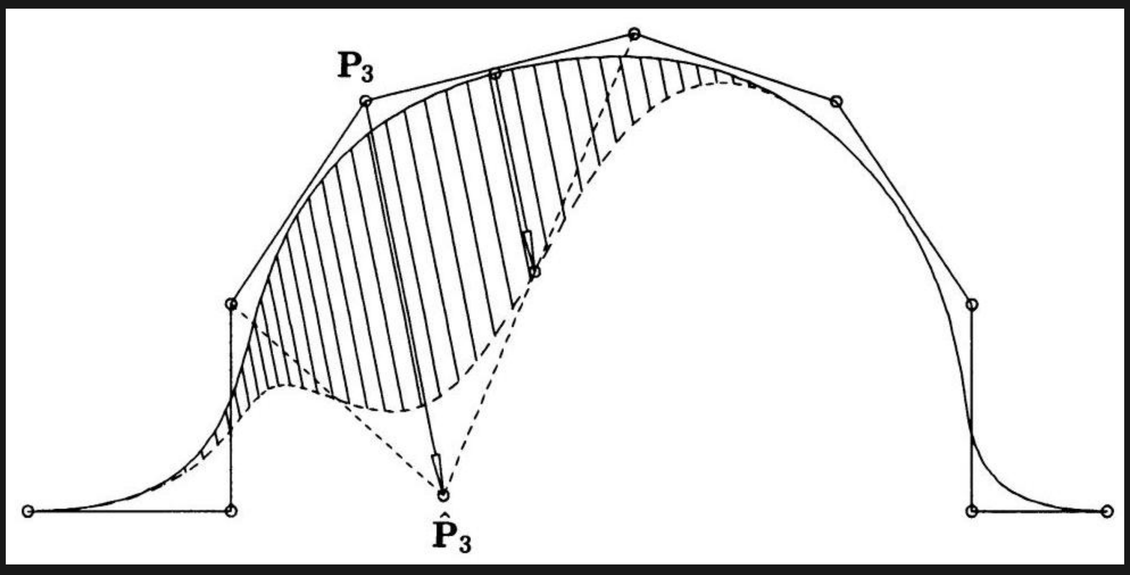


图 11.3。通过重新定位 $\mathbf{P}_3.\bar{u}=0.43=1/2\,(t_3+t_4)$ 移动 $\mathbf{P}=\mathbf{C}(\bar{u})$ ； 对应的节点是 $t_3=0.35$ ， 曲线的度数为三。

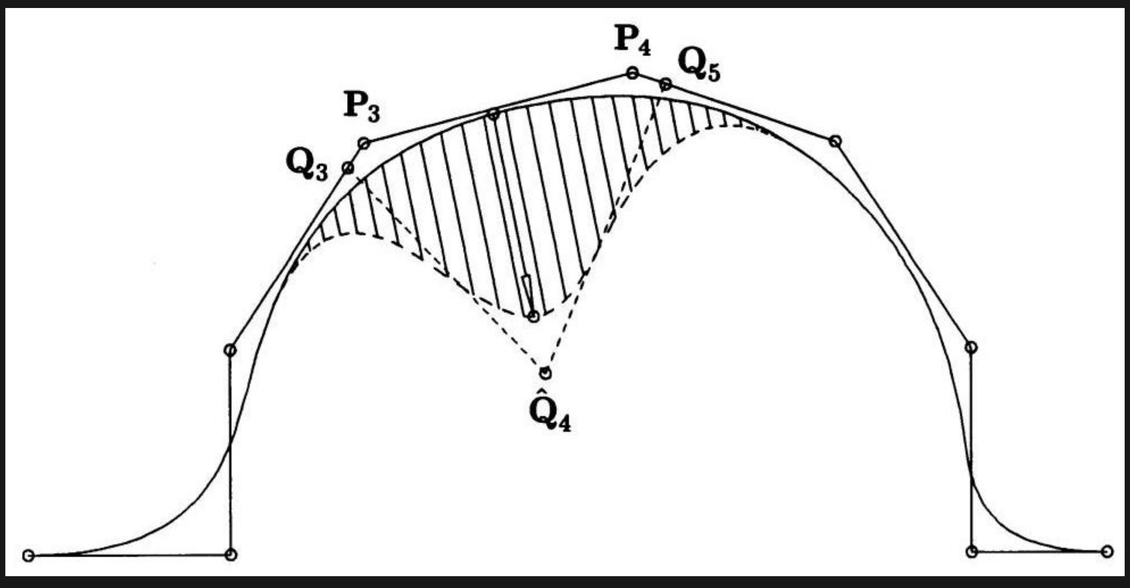


图 11.4。通过节点插入获得的新控制点的重新定位；曲线的度数为三。

$0\le\gamma\le1$ ，我们设定

$$\widehat{\mathbf{P}}_k=\mathbf{P}_k+(1-\gamma)\,\alpha\mathbf{V}$$

$$\widehat{\mathbf{P}}_{k+1}=\mathbf{P}_{k+1}+\gamma\alpha\mathbf{V} \tag{11.7}$$

参数 γ 允许我们在一个线性尺度上变化 $\mathbf{P}_k$ 和 $\mathbf{P}_{k+1}$ 之间的相对移动。设计者可以设定 γ ， 或者一个好的默认值是

$$\gamma=\frac{\bar{u}-t_k}{t_{k+1}-t_k} \tag{11.8}$$

其中 $t_k\le\bar{u}\le t_{k+1}$ 。根据平移不变性 (P4.9)，我们有

$$\left|\widehat{\mathbf{P}}-\mathbf{P}\right|=d=\left|(1-\gamma)\,\alpha\mathbf{V}R_{k,p}(\bar{u})+\gamma\alpha\mathbf{V}R_{k+1,p}(\bar{u})\right|$$

这意味着

$$\alpha=\frac{d}{|\mathbf{V}|\left[(1-\gamma)\,R_{k,p}(\bar{u})+\gamma R_{k+1,p}(\bar{u})\right]} \tag{11.9}$$

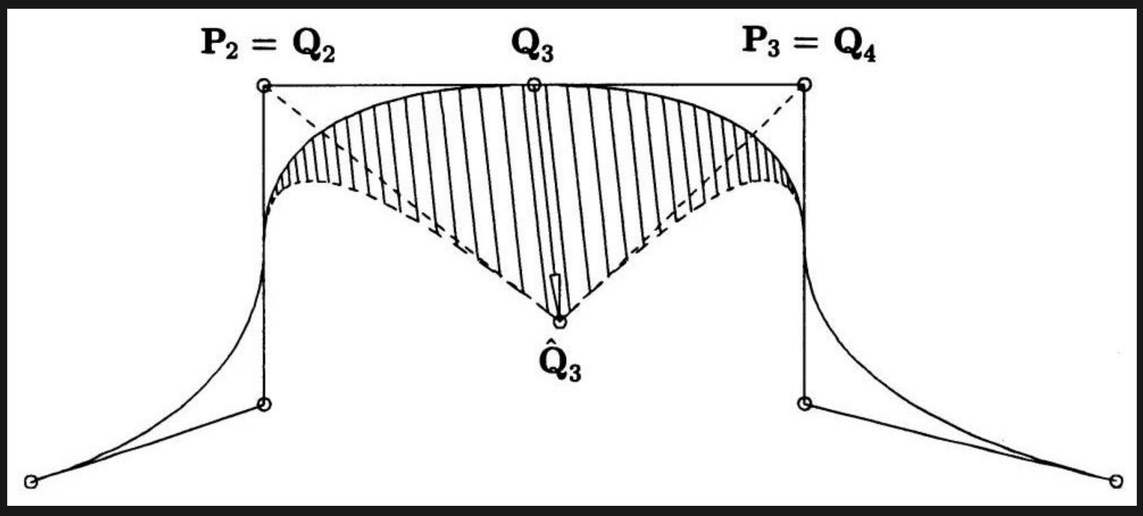


图 11.5. 通过节点插入获得的新控制点的重新定位；曲线的度数为二。

图 11.6a 显示了图 11.3 的曲线。$\mathbf{P}_3$ 和 $\mathbf{P}_4$ 都被移动了，使用了 $\gamma = 0.47$ (由公式 [11.8] 计算的默认值)。显然，这种方法的缺点是变化的范围更广，即 $[u_k, u_{k+p+2})$ 。在图 11.6b 中，$\mathbf{P}_3$ 和 $\mathbf{P}_4$ 被移动，使用了 $\gamma = 4/5$ 。

使用这些工具，可以轻松实现一个用户友好、互动式的图形界面，用于控制点的重新定位。注意，当 $\mathbf{P}_k$ 沿相同的向量方向反复平移时，公式 (11.3) 中的值 $R_{k,p}(\bar{u})$ 不会改变 (因此 α 也不会改变)。此外，只有曲线中 $[u_k, u_{k+p+1})$ 的图像部分需要在每次重新定位时进行图形更新。图 11.7 显示了一个控制点的增量重新定位。指定了一个增量 Δd ，它固定了一个 $\Delta\alpha$ 。相关的控制点然后被平移一个增量 $\Delta\alpha\mathbf{V}$ ，并且在每次激活指定的图形输入时，相关的曲线段被重新绘制。

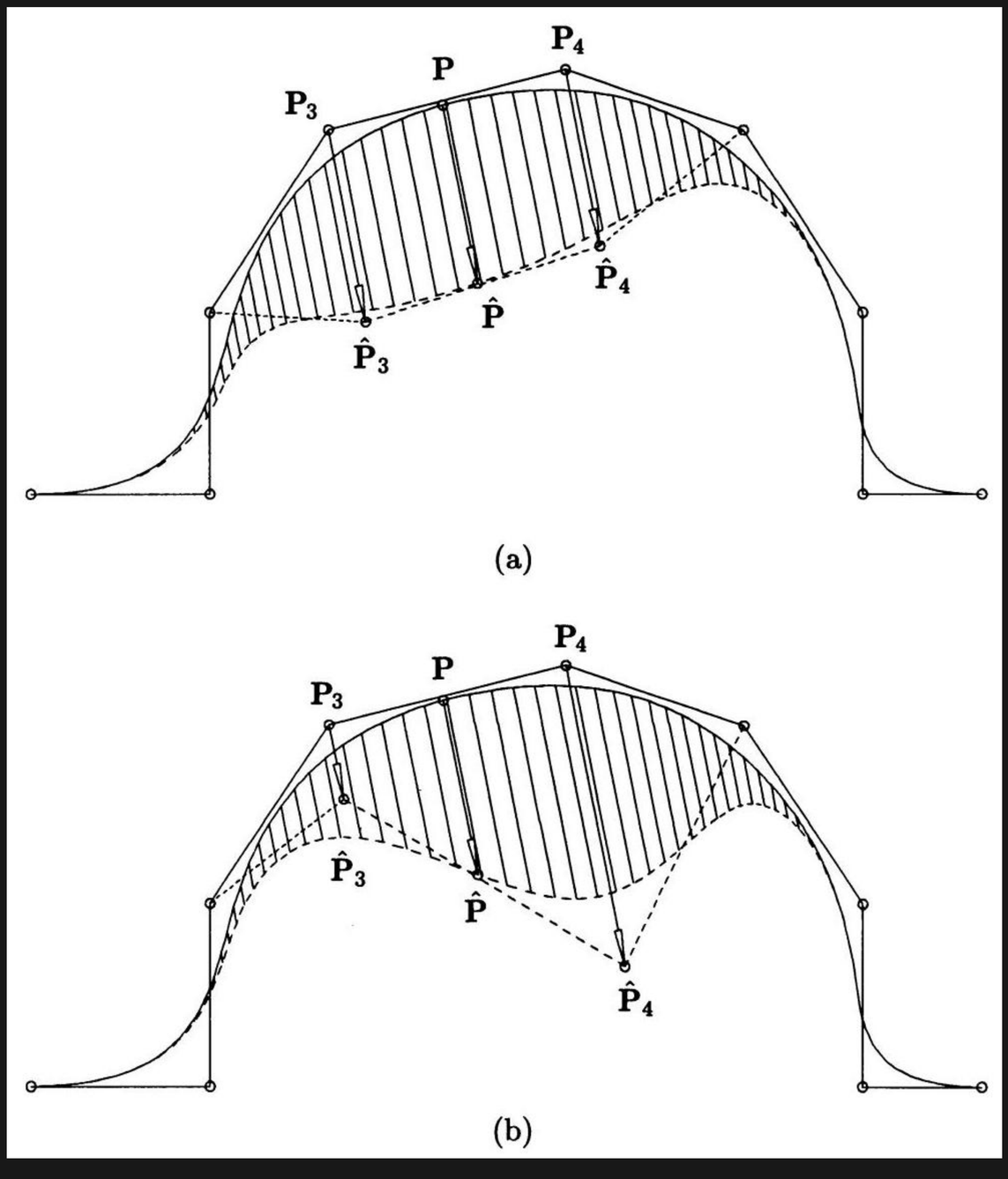


图 11.6. 同时重新定位 $\mathbf{P}_3$ 和 $\mathbf{P}_4$；$\bar{u} = 0.43$ 。(a) $\gamma = 0.47$ (默认)；(b) $\gamma = 4/5$ 。

最后，设计师可以通过选择两个额外的曲线点 $\mathbf{Q}_1$ 和 $\mathbf{Q}_2$ 来轻松限制曲线应变化的域，这两个点确定参数 $\bar{u}_1$ 和 $\bar{u}_2$ ，满足 $\bar{u}_1 < \bar{u} < \bar{u}_2$ 。如果 $\bar{u}_1$ 和 $\bar{u}_2$ 还不是节点，则每个节点插入一次。然后，如果必要的话，在区间 $[\bar{u}_1, \bar{u})$ 和 $[\bar{u}, \bar{u}_2)$ 上插入额外的节点，以确保存在一个合适的控制点 $\mathbf{P}_k$ ，使得 $\bar{u}$ 接近节点 t_k 、$\bar{u}_1 \le u_k$ 和 $u_{k+p+1} \le \bar{u}_2$ 。节点不应以大于一个的重数插入，因为这可能导致在移动控制点时出现不必要的不连续性。

这些技术可以轻松扩展到曲面。为完整性起见，我们陈述相关公式。设

$$\mathbf{S}(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} R_{i,p;j,q}(u,v)\,\mathbf{P}_{i,j}$$

为 NURBS 曲面，并假设 $\mathbf{P} = \mathbf{S}(\bar{u},\bar{v})$ 沿向量 $\mathbf{V}$ 移动，距离 d ，通过重新定位控制点 $\mathbf{P}_{k,\ell}$ 。那么

$$\hat{\mathbf{P}}_{k,\ell} = \mathbf{P}_{k,\ell} + \alpha\mathbf{V} \tag{11.10}$$

与

$$\alpha = \frac{d}{|\mathbf{V}|\,R_{k,\ell}(\bar{u},\bar{v})} \tag{11.11}$$

要么设计师选择 k 和 ℓ ，要么系统根据 $\bar{u}$ 和 $\bar{v}$ 与 u 和 v 节点 s_i 和 t_j 的接近程度来选择它们，这些节点由

$$s_i = \frac{1}{p}\sum_{r=1}^{p} u_{i+r}\ i = 0,\ldots,n$$

$$t_j = \frac{1}{q}\sum_{r=1}^{q} v_{j+r}\ j = 0,\ldots,m \tag{11.12}$$

使用类似于公式 (11.6) 的公式，可以计算并插入一个 u 或 v 节点，或两者，以减少变化的域，并将 $\bar{u}$ 和/或 $\bar{v}$ 移动到节点位置。如果 $0 \le \delta, \gamma \le 1$ ，那么 $\mathbf{P}_{k,\ell}, \mathbf{P}_{k+1,\ell}, \mathbf{P}_{k,\ell+1}$ 和 $\mathbf{P}_{k+1,\ell+1}$ 可以同时沿着 $\mathbf{V}$ 的方向移动。

$$\hat{\mathbf{P}}_{k,\ell} = \mathbf{P}_{k,\ell} + (1-\delta)(1-\gamma)\,\alpha\mathbf{V}$$

$$\widehat{\mathbf{P}}_{k+1,\ell} = \mathbf{P}_{k+1,\ell} + \delta\,(1-\gamma)\,\alpha\mathbf{V}$$

$$\widehat{\mathbf{P}}_{k,\ell+1} = \mathbf{P}_{k,\ell+1} + (1-\delta)\,\gamma\alpha\mathbf{V}$$

$$\widehat{\mathbf{P}}_{k+1,\ell+1} = \mathbf{P}_{k+1,\ell+1} + \delta\gamma\alpha\mathbf{V} \tag{11.13}$$

(11.14)

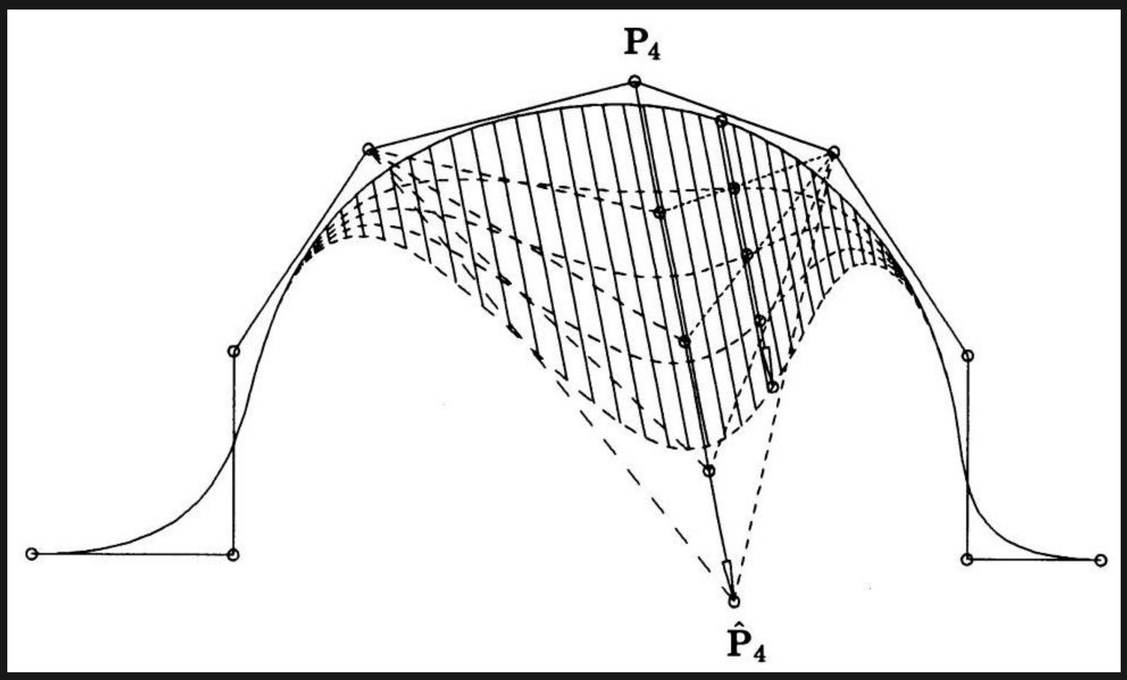


图 11.7。通过重新定位 $\mathbf{P}_4$; $\bar{u} = 3/5$ 以相等的增量拉动曲线。

$$\alpha = \frac{d}{|\mathbf{V}|\,[\,(1-\delta)(1-\gamma)R_{k,\ell}(\bar{u},\bar{v}) + \cdots + \delta\gamma R_{k+1,\ell+1}(\bar{u},\bar{v})\,]}$$

图 11.8a 和 11.8b 显示了一个控制点的转换，图 11.9a 和 11.9b 则说明了四个相邻控制点的同时移动。

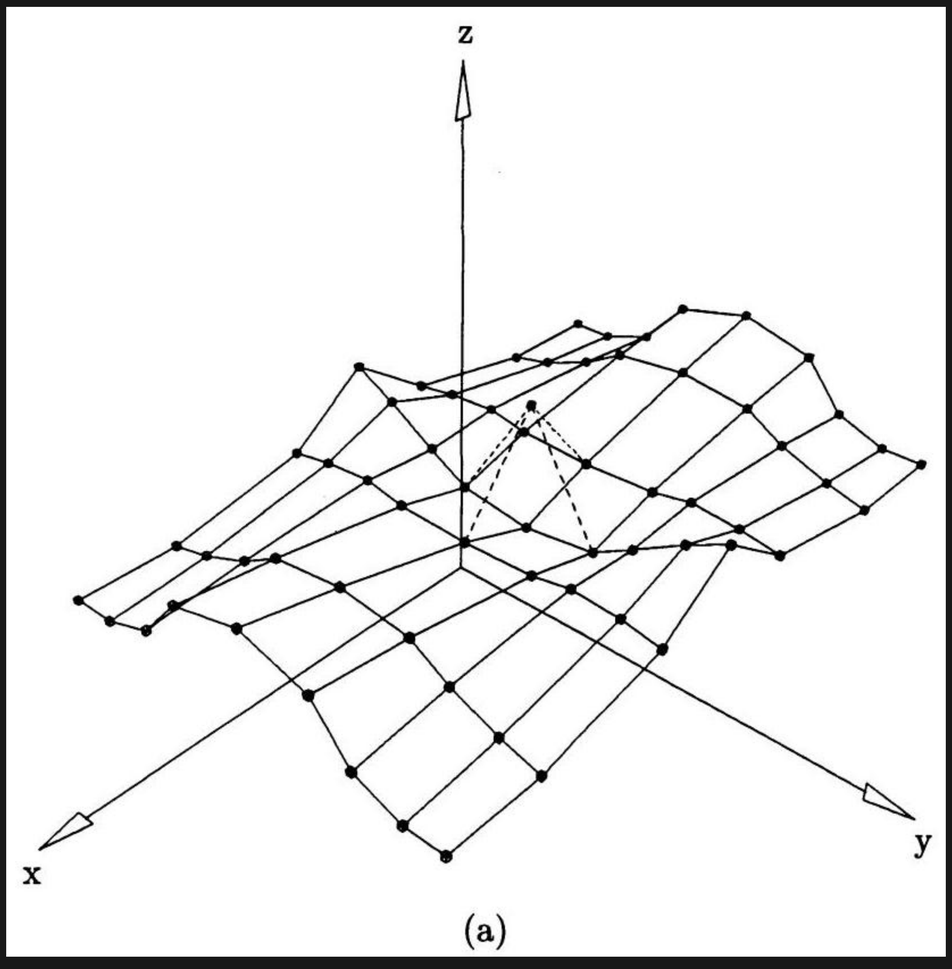


图 11.8. 重新定位 $\mathbf{P}_{4,3}$ ，与 $\bar{u} = \bar{v} = 0.55$ 。(a) 控制网；(b) 一个表面，其局部补丁沿控制点移动的方向平行移动。

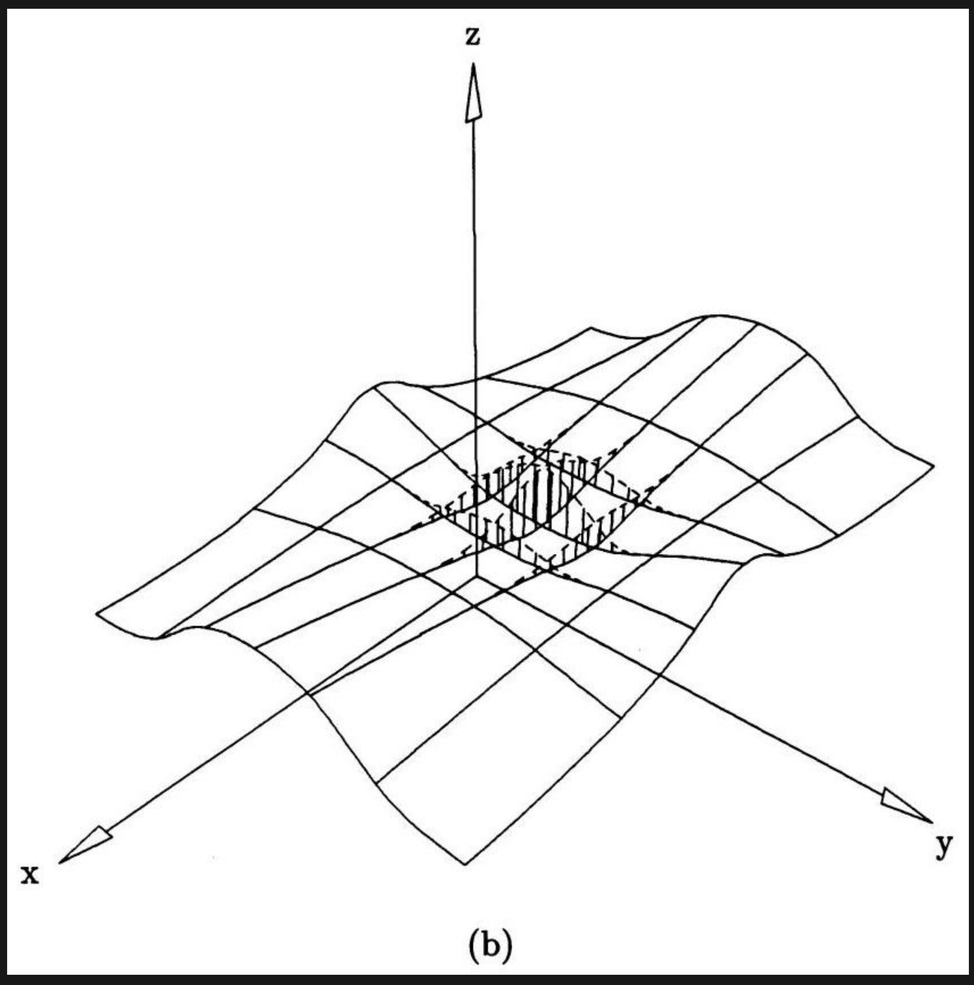


图 11.8. (续)。

类似于曲线情况，节点插入可以用来限制表面变化的矩形域，而逆节点插入可以用来在表面控制网的边缘指定位置获取额外的控制点。点 $\mathbf{Q}_u$ 在 u 方向上，由公式插入。

$$\widehat{u} = u_{k+1} + \alpha\left(u_{k+p+1} - u_{k+1}\right) \tag{11.15}$$

与

$$\alpha = \frac{w_{k,\ell}\,|\mathbf{Q}_u - \mathbf{P}_{k,\ell}|}{w_{k,\ell}\,|\mathbf{Q}_u - \mathbf{P}_{k,\ell}| + w_{k+1,\ell}\,|\mathbf{P}_{k+1,\ell} - \mathbf{Q}_u|}$$

(见 [Pieg89d])。同样，$\mathbf{Q}_v$ 由以下公式插入。

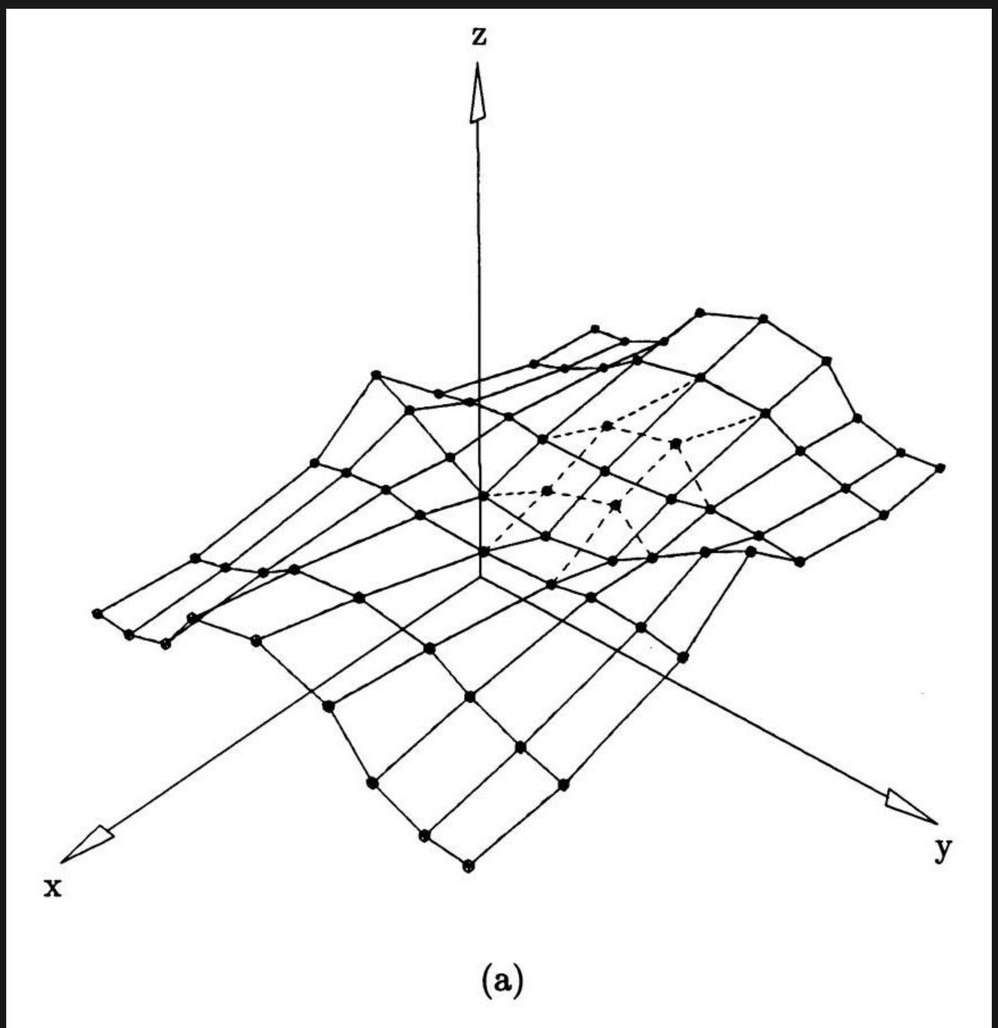
$$\widehat{v} = v_{\ell+1} + \beta\left(v_{\ell+q+1} - v_{\ell+1}\right) \tag{11.16}$$

与

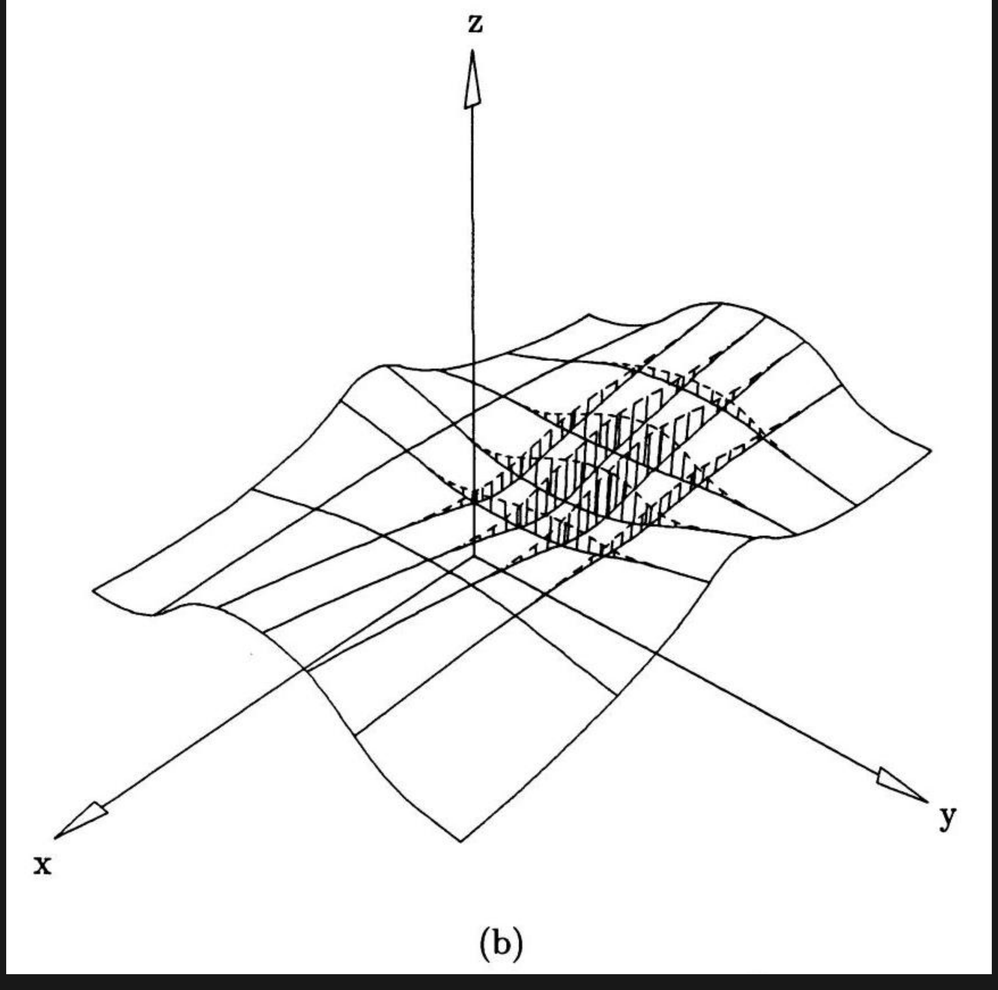
$$\beta = \frac{w_{k,\ell}\,|\mathbf{Q}_v - \mathbf{P}_{k,\ell}|}{w_{k,\ell}\,|\mathbf{Q}_v - \mathbf{P}_{k,\ell}| + w_{k,\ell+1}\,|\mathbf{P}_{k,\ell+1} - \mathbf{Q}_v|}$$

3 权重修改

在本节中，我们研究修改权重的效果，特别是对曲线的一个或两个相邻权重，以及对表面的一个权重。



(a)



(b)

图 11.9. 重新定位四个表面控制点 $\mathbf{P}_{4,3}, \mathbf{P}_{5,3}, \mathbf{P}_{4,4}$ 和 $\mathbf{P}_{5,4}$，与 $\bar{u} = \bar{v} = 3/5$。(a) 控制网；(b) 一个表面，其局部补丁沿控制点移动的方向平行移动。

3.1 单曲线权重的修改

让

$$\mathbf{C}(u) = \sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i \tag{11.17}$$

设为 NURBS 曲线，且 $\bar{u}$ 为满足 $\bar{u} \in [u_k, u_{k+p+1})$ 的固定参数值。我们在第 4 章中指出，修改权重 w_k 的值的效果是将点 $\mathbf{C}(\bar{u})$ 沿着由 $\mathbf{P}_k$ 和原始点 $\mathbf{C}(\bar{u})$ 定义的直线移动 (见图 4.2、4.4 和 4.6)。因此，虽然控制点的重新定位对曲线有平移效果，但权重的修改具有透视效果，其中受影响域内的所有曲线点沿着与对应于被修改权重的控制点相交的直线被拉动 (或推送)。在本节中，我们证明这一陈述，并建立权重的精确几何意义。

权重 w_k 可以被视为方程 (11.17) 中的一个变量，因此我们可以对 $\mathbf{C}(u)$ 关于 w_k 进行微分。使用恒等式

$$R_{k,p}(u) = \frac{N_{k,p}(u)\,w_k}{\sum_{r=0}^{n} N_{r,p}(u)\,w_r}$$

以及微分的商法则，我们得到

$$\frac{\partial}{\partial w_k}\mathbf{C}(u; w_0, \ldots, w_n) = \frac{\partial}{\partial w_k}\sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i$$

$$= \sum_{i \neq k} \frac{-N_{i,p}(u)\,w_i N_{k,p}(u)}{\left(\sum_{r=0}^{n} N_{r,p}(u)\,w_r\right)^2}\mathbf{P}_i$$

$$+\frac{N_{k,p}(u)\left(\sum_{r=0}^{n}N_{r,p}(u)w_r\right)-N_{k,p}(u)N_{k,p}(u)w_k}{\left(\sum_{r=0}^{n}N_{r,p}(u)w_r\right)^2}\mathbf{P}_k$$

$$=\sum_{i\neq k}\frac{-N_{k,p}(u)}{\sum_{r=0}^{n}N_{r,p}(u)w_r}R_{i,p}(u)\mathbf{P}_i$$

$$+\frac{R_{k,p}}{w_k}\mathbf{P}_k-\frac{N_{k,p}(u)R_{k,p}(u)}{\sum_{r=0}^{n}N_{r,p}(u)w_r}\mathbf{P}_k$$

$$=\frac{R_{k,p}(u)}{w_k}\left(\mathbf{P}_k-\sum_{r=0}^{n}R_{r,p}(u)\mathbf{P}_r\right)$$

$$=\frac{R_{k,p}(u)}{w_k}\left(\mathbf{P}_k-\mathbf{C}(u)\right)=\alpha(k,u)\left(\mathbf{P}_k-\mathbf{C}(u)\right) \tag{11.18}$$

因此，关于 w_k 的 $\mathbf{C}(u)$ 的一阶导数是一个指向从 $\mathbf{C}(u)$ 到 $\mathbf{P}_k$ 方向的向量。我们现在通过归纳法证明所有阶的导数具有这种形式。设 $\alpha^{(1)}=\alpha(k,u)$，并假设 n 阶导数具有形式 $\alpha^{(n)}\left(\mathbf{P}_k-\mathbf{C}(u)\right)$。那么

$$\frac{\partial^{n+1}\mathbf{C}(u)}{\partial w_k^{n+1}}=\frac{\partial\left[\alpha^{(n)}\left(\mathbf{P}_k-\mathbf{C}(u)\right)\right]}{\partial w_k}$$

$$=\alpha^{(n)}\left(-\alpha^{(1)}\mathbf{P}_k+\alpha^{(1)}\mathbf{C}(u)\right)+\frac{\partial\alpha^{(n)}}{\partial w_k}\left(\mathbf{P}_k-\mathbf{C}(u)\right)$$

$$=\left(\frac{\partial\alpha^{(n)}}{\partial w_k}-\alpha^{(1)}\alpha^{(n)}\right)\left(\mathbf{P}_k-\mathbf{C}(u)\right)=\alpha^{(n+1)}\left(\mathbf{P}_k-\mathbf{C}(u)\right)$$

其中

$$\alpha^{(n+1)}=\frac{\partial\alpha^{(n)}}{\partial w_k}-\alpha^{(1)}\alpha^{(n)}$$

这完成了证明。

容易推导出一个递归公式来计算 $\alpha^{(n+1)}$，从而获得高阶导数的公式

$$\frac{\partial^{n+1}\mathbf{C}(u)}{\partial w_k^{n+1}}=\alpha^{(n+1)}\left(\mathbf{P}_k-\mathbf{C}(u)\right)\ n=1,2,\ldots,\infty \tag{11.19}$$

具有

$$\alpha^{(n+1)}=(-1)(n+1)\alpha^{(n)}\alpha^{(1)} \tag{11.20}$$

方程 (11.18)-(11.20) 证明了任意曲线点 $\mathbf{C}(u)$ 随着 w_k 值的增加而沿直线朝向 $\mathbf{P}_k$ 移动。

现在我们采取一种更几何的方法。从方程 (11.17) 的曲线开始，让 w_k 变化，$0\leq w_k<\infty$。将结果曲线的家族记为 $\mathbf{C}(u;w_k)$ (见图 11.10)。假设 $\bar{u}\in[u_k,u_{k+p+1})$，并定义点

$$\mathbf{R}=\mathbf{C}(\bar{u};w_k=0)$$

$$\mathbf{M}=\mathbf{C}(\bar{u};w_k=1)$$

$$\mathbf{P}=\mathbf{C}(\bar{u};w_k\neq 0,1) \tag{11.21}$$

注意到 $\mathbf{R}$ 和 $\mathbf{M}$ 是特定曲线上的点，但 $\mathbf{P}$ 可以位于 $\mathbf{R}$ 和 $\mathbf{P}_k$ 之间的任何位置，因为 $w_k\geq 0$。现在 $\mathbf{M}$ 和 $\mathbf{P}$ 可以表示为

$$\mathbf{M}=(1-s)\mathbf{R}+s\mathbf{P}_k$$

$$\mathbf{P}=(1-t)\mathbf{R}+t\mathbf{P}_k \tag{11.22}$$

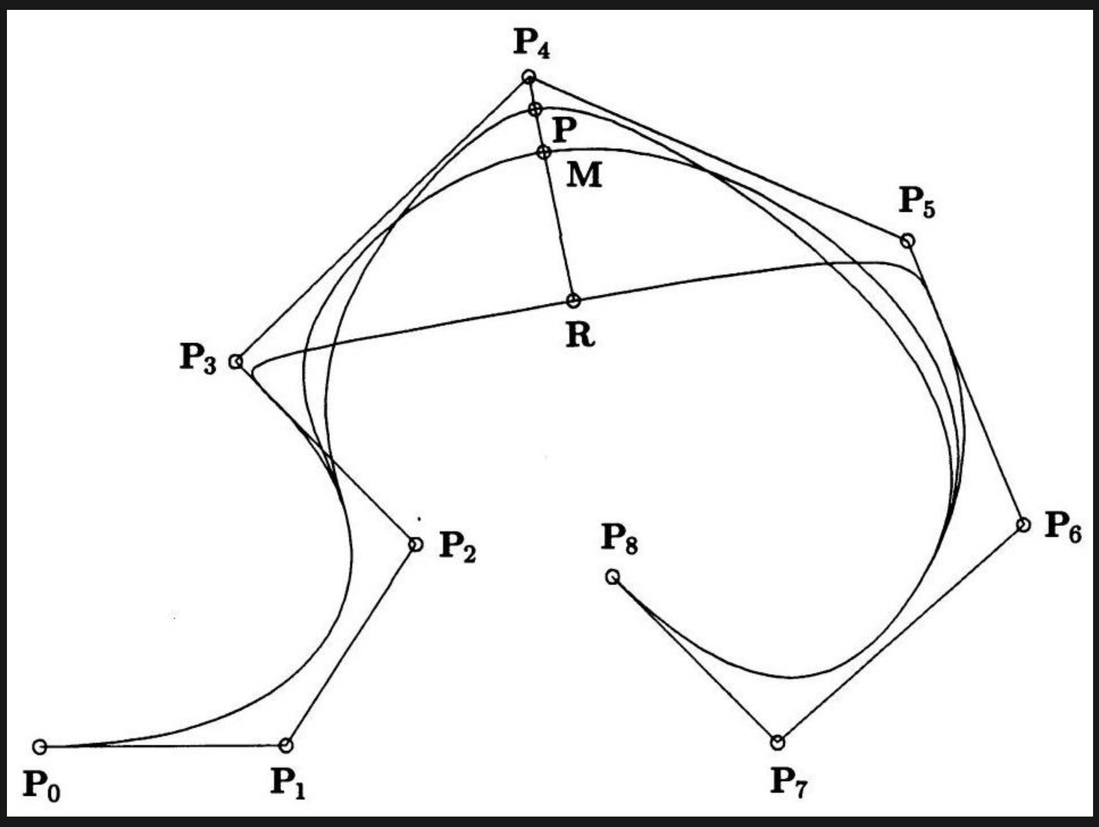


图 11.10. 权重的几何意义。

其中

$$s=\frac{N_{k,p}(\bar{u})}{\sum_{i\neq k}N_{i,p}(\bar{u})w_i+N_{k,p}(\bar{u})}$$

$$t=\frac{N_{k,p}(\bar{u})w_k}{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}=R_{k,p}(\bar{u}) \tag{11.23}$$

为了推导这些方程，注意到

$$\mathbf{R}=\frac{\sum_{i\neq k}N_{i,p}(\bar{u})w_i\mathbf{P}_i}{\sum_{i\neq k}N_{i,p}(\bar{u})w_i}\ w_k=0$$

因此我们有

$$\mathbf{P}=\sum_{i=0}^{n}R_{i,p}(\bar{u})\mathbf{P}_i=\sum_{i\neq k}R_{i,p}(\bar{u})\mathbf{P}_i+R_{k,p}(\bar{u})\mathbf{P}_k$$

$$=\frac{\sum_{i\neq k}N_{i,p}(\bar{u})w_i\mathbf{P}_i}{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}+R_{k,p}(\bar{u})\mathbf{P}_k$$

$$=\frac{\sum_{i\neq k}N_{i,p}(\bar{u})w_i}{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}\mathbf{R}+R_{k,p}(\bar{u})\mathbf{P}_k$$

$$=\left(\frac{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}-\frac{N_{k,p}(\bar{u})w_k}{\sum_{i=0}^{n}N_{i,p}(\bar{u})w_i}\right)\mathbf{R}+R_{k,p}(\bar{u})\mathbf{P}_k$$

$$= (1 - R_{k,p}(\bar{u}))\,\mathbf{R} + R_{k,p}(\bar{u})\,\mathbf{P}_k = (1-t)\,\mathbf{R} + t\mathbf{P}_k$$

其中 $t = R_{k,p}(\bar{u})$ 。最后，设定 $w_k = 1$ 得到

$$\mathbf{M} = \frac{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i \mathbf{P}_i}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})} + \frac{N_{k,p}(\bar{u})}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}\mathbf{P}_k$$

$$= \frac{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}\mathbf{R} + \frac{N_{k,p}(\bar{u})}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}\mathbf{P}_k$$

$$= \frac{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u}) - N_{k,p}(\bar{u})}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}\mathbf{R} + \frac{N_{k,p}(\bar{u})}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}\mathbf{P}_k$$

$$= (1-s)\,\mathbf{R} + s\mathbf{P}_k$$

其中

$$s = \frac{N_{k,p}(\bar{u})}{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})}$$

从方程 (11.22) 和 (11.23) 我们得到

$$w_k = \frac{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i}{N_{k,p}(\bar{u})} \Big/ \frac{\sum\limits_{i\neq k} N_{i,p}(\bar{u})\, w_i}{N_{k,p}(\bar{u})\, w_k}$$

$$= \frac{1-s}{s} \Big/ \frac{1-t}{t} = \frac{\mathbf{P}_k\mathbf{M}}{\mathbf{RM}} \Big/ \frac{\mathbf{P}_k\mathbf{P}}{\mathbf{RP}} \tag{11.24}$$

方程 (11.24) 表示 w_k 是四个点 $\mathbf{P}_k, \mathbf{R}, \mathbf{M}$ 和 $\mathbf{P}$ 的交比，记作 $(\mathbf{P}_k, \mathbf{R}; \mathbf{M}, \mathbf{P})$ (见 [Coxe74; Boeh94])。交比是射影几何中的一个基本量。关于 NURBS 曲线的形状修改，我们提到方程 (11.21)-(11.24) 的以下结果:

- 当 w_k 增加 (减少) 时，t 增加 (减少)，因此曲线被拉向 (推离) $\mathbf{P}_k$ 。
- $\mathbf{P}$ 沿着通过 $\mathbf{P}_k$ 的直线段移动；因此，修改 w_k 会以可预测的方式改变曲线；
- 当 $\mathbf{P}$ 接近 $\mathbf{P}_k$，t 时，w_k 趋近于无穷大。

现在假设点 $\mathbf{P} = \mathbf{C}(\bar{u})$ 要被拉 (推) 向 (远离) $\mathbf{P}_k$ 到一个新的位置 $\widehat{\mathbf{P}}$ ，通过修改 w_k 。设定 $d = \mathbf{P}\widehat{\mathbf{P}} = \left|\mathbf{P} - \widehat{\mathbf{P}}\right|$ 。所需的新权重 $\widehat{w}_k$ 的推导如下:

$$\widehat{w}_k = \frac{\mathbf{P}_k\mathbf{M} \cdot (\mathbf{RP} \pm d)}{\mathbf{RM} \cdot (\mathbf{P}_k\mathbf{P} \mp d)}$$

$$= \frac{\mathbf{P}_k\mathbf{M} \cdot \mathbf{RP}}{\mathbf{RM} \cdot \mathbf{P}_k\mathbf{P} \mp d \cdot \mathbf{RM}} \pm \frac{d \cdot \mathbf{P}_k\mathbf{M}}{\mathbf{RM} \cdot (\mathbf{P}_k\mathbf{P} \mp d)}$$

使用

$$w_k = \frac{\mathbf{P}_k\mathbf{M} \cdot \mathbf{RP}}{\mathbf{RM} \cdot \mathbf{P}_k\mathbf{P}}$$

我们得到

$$\widehat{w}_k = \frac{w_k \cdot \mathbf{P}_k\mathbf{P}}{\mathbf{P}_k\mathbf{P} \mp d} \pm \frac{d \cdot \mathbf{P}_k\mathbf{M}}{\mathbf{RM} \cdot (\mathbf{P}_k\mathbf{P} \mp d)}$$

$$= w_k \left(\frac{\mathbf{P}_k\mathbf{P}}{\mathbf{P}_k\mathbf{P} \mp d} \pm \frac{d \cdot \mathbf{P}_k\mathbf{P}}{\mathbf{RP} \cdot (\mathbf{P}_k\mathbf{P} \mp d)} \right)$$

$$= w_k \left(\frac{\mathbf{P}_k\mathbf{P} \cdot (\mathbf{RP} \pm d)}{\mathbf{RP} \cdot (\mathbf{P}_k\mathbf{P} \mp d)} \right) = w_k \left(\frac{(1-t)\,(\mathbf{RP} \pm d)}{t \cdot (\mathbf{P}_k\mathbf{P} \mp d)} \right)$$

$$= w_k \left(\frac{(1-t) \cdot \mathbf{RP} \mp td}{t \cdot \mathbf{P}_k\mathbf{P} \mp td} \pm \frac{d}{t \cdot (\mathbf{P}_k\mathbf{P} \mp d)} \right)$$

并且从方程 (11.24) 中我们得到

$$\widehat{w}_k = w_k \left(1 \pm \frac{d}{R_{k,p}(\bar{u})\,(\mathbf{P}_k\mathbf{P} \mp d)} \right) \tag{11.25}$$

当 $\pm$ 或 $\mp$ 出现时，拉操作时取上符号，推操作时取下符号 [Pieg89c]。

为了实现一个用户界面以修改一个曲线权重，设计者在曲线上选择一个点 $\mathbf{P}$ 和控制多边形上的一个点。系统计算参数 $\bar{u}$ 使得 $\mathbf{P} = \mathbf{C}(\bar{u})$ 。如果在控制多边形上选择的点还不是控制点，系统通过使用逆节点插入将其变为控制点。将控制点表示为 $\mathbf{P}_k$ 。然后系统通过 $\mathbf{P}$ 和 $\mathbf{P}_k$ 画一条线。设计者在这条线上选择点 $\widehat{\mathbf{P}}$ ，允许系统计算 d ，最后从方程 (11.25) 中得到 $\widehat{w}_k$ 。读者应注意到

- 两个用户选择确定 $\bar{u}$ 和 k 。可能 (但不太可能) 出现 $\bar{u} \notin [u_k, u_{k+p+1})$ ：系统应捕捉到这一点并将其报告给用户作为无效选择；
- 在公式 (11.25) 中，值 $R_{k,p}(\bar{u})$ 和 $\mathbf{P}_k\mathbf{P}$ 随着 w_k 的每次修改而变化，即使 k 和 $\bar{u}$ 保持不变；因此，如果希望进行交互式增量或连续的权重修改，最好保存原始的 $R_{k,p}(\bar{u})$, w_k 和 $\mathbf{P}_k\mathbf{P}$ 值，并在每一步中使用它们来计算新的 d 和 $\widehat{w}_k$ 值；
- 如果我们固定权重变化 Δw_k ，并将距离 $d(u) = |\mathbf{C}(u; w_k) - \mathbf{C}(u; \widehat{w}_k)|$ 视为 u 的函数，则根据公式 (11.25)，可以得出 $d(u)$ 在 $R_{k,p}(u)$ 的最大值处达到其最大值 (图 11.11)；
- 从实际的角度来看，权重修改只能/应该用于对曲线进行小幅度的变化。对于拉动操作，$\mathbf{P}_k$ 显然是可能拉动长度的内置限制。这个限制可能非常严格，特别是在通过逆结点插入添加 $\mathbf{P}_k$ 的情况下。此外，即使是适度的推拉操作通常也需要对权重进行 substantial 的修改；而广泛变化的权重可能会导致非常糟糕的参数化，这往往会在后续造成意想不到的问题。图 11.11 和 11.12 显示了这样的例子。因此，在严肃的软件实现中，限制可允许的权重值范围可能是明智的；

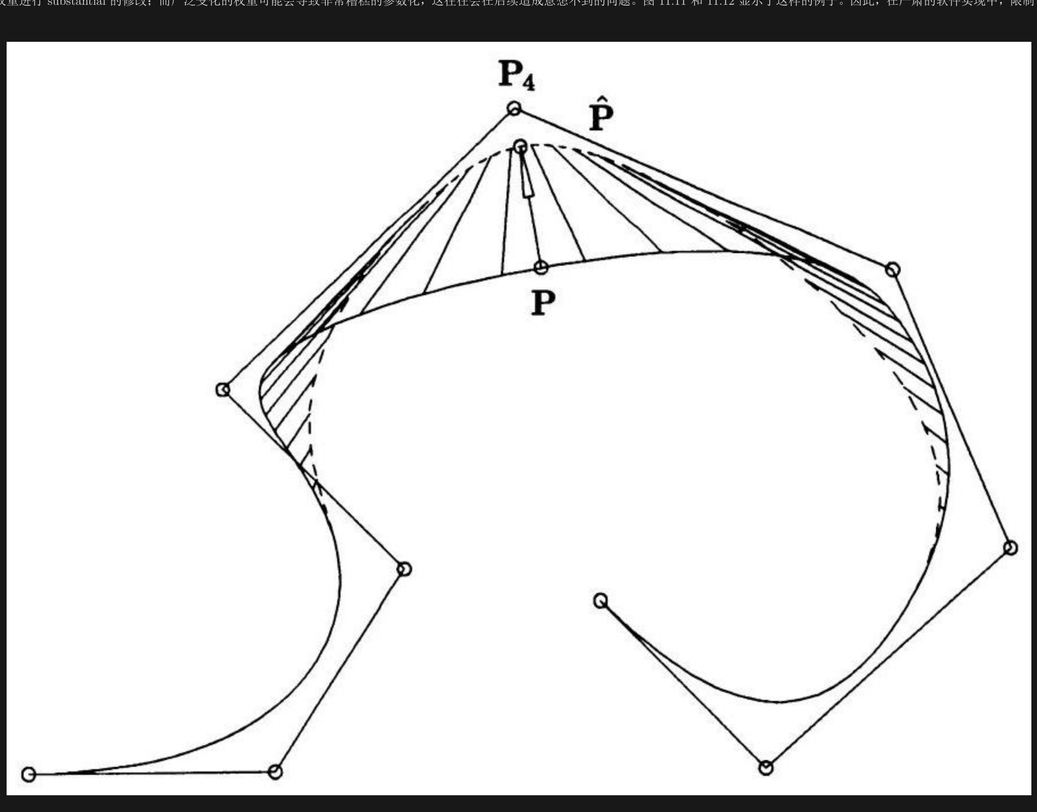


图 11.11. 通过改变权重 $w_4 = 0.2$ ，将曲线在 $t_4 = u = 0.52$ 处拉动 $d = 0.075$ 的距离朝向 $\mathbf{P}_4$ ；计算出的新权重为 $\widehat{w}_4 = 2.45$ 。注意由于权重变化而产生的曲线段的透视变换。

- 与控制点重新定位的情况一样，结点插入可以在这里用于限制曲线变化的范围，使其位于两个用户指定的曲线点之间。

图 11.13 显示了增量拉动的一个例子。注意权重的指数变化。

3.2 两个相邻曲线权重的修改

现在我们考虑同时修改两个邻近曲线的权重，w_k 和 w_{k+1} 。这样的修改会将曲线拉向 (或推离) 控制多边形的边 $\mathbf{P}_k\mathbf{P}_{k+1}$ 。固定 $\bar{u} \in [u_{k+1}, u_{k+p+1})$ ，设 $\mathbf{P} = \mathbf{C}(\bar{u}; w_k, w_{k+1})$ 并定义

$$\mathbf{R} = \mathbf{C}(\bar{u}; w_k = w_{k+1} = 0) \tag{11.26}$$

(见图 11.14)。然后 $\mathbf{P}$ 可以表示为

$$\mathbf{P} = (1 - a_k - a_{k+1})\,\mathbf{R} + a_k\mathbf{P}_k + a_{k+1}\mathbf{P}_{k+1} \tag{11.27}$$

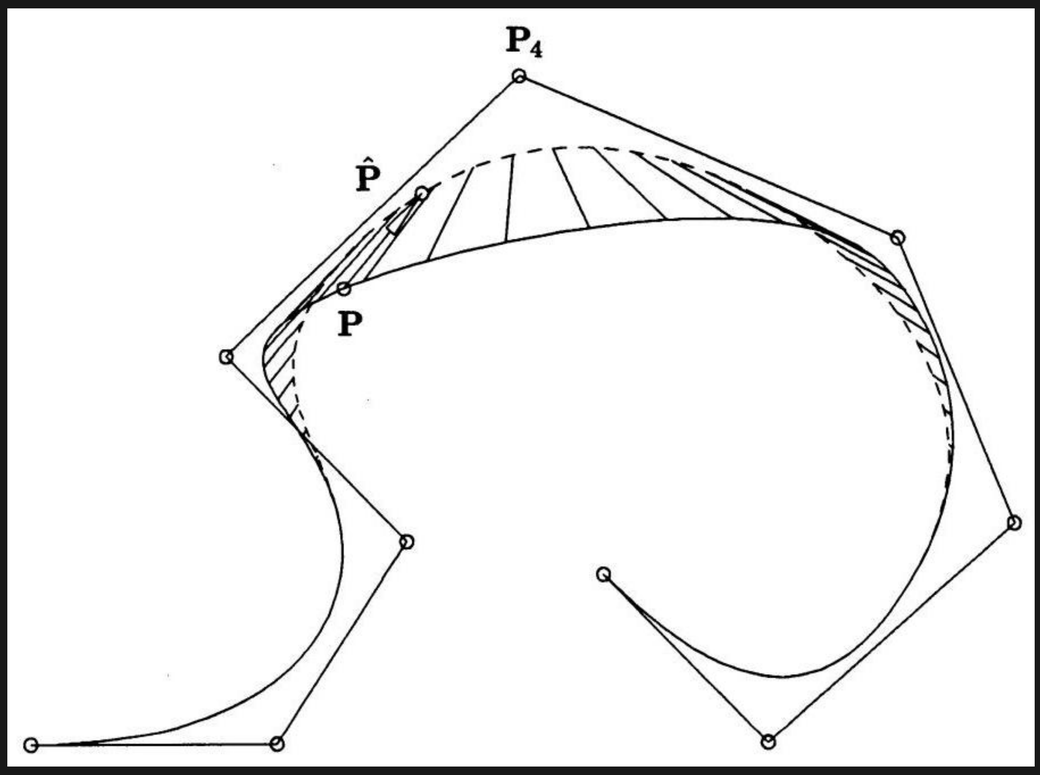


图 11.12。通过改变权重 $w_4 = 0.2$ ，将曲线在 $u = 0.46$ 处拉向 $\mathbf{P}_4$ 的距离 $d = 0.075$ ；计算得到的新权重为 $\hat{w}_4 = 0.97$ 。

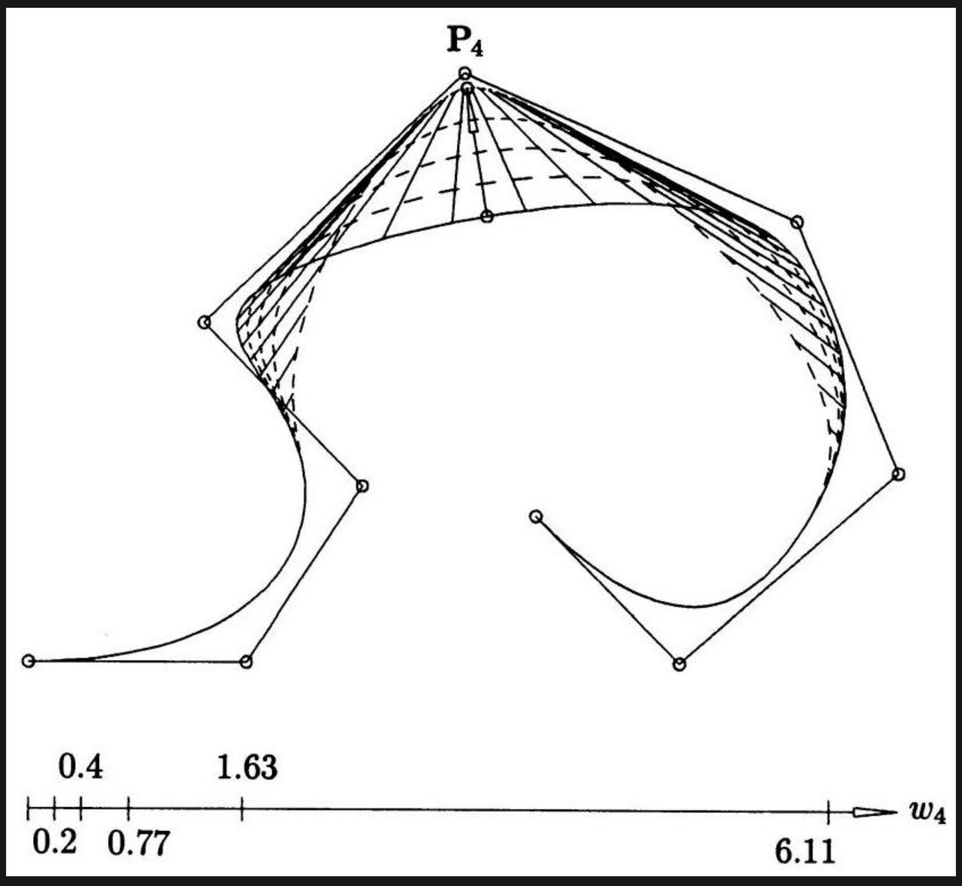


图 11.13。通过改变权重 $w_4 = 0.2$ ，以相等增量 $\Delta = 0.022$ 将曲线在 $t_4 = u = 0.52$ 处拉向 $\mathbf{P}_4$ ；计算得到的新权重为 $\hat{w}_4 = \{0.4, 0.77, 1.63, 6.11\}$ 。

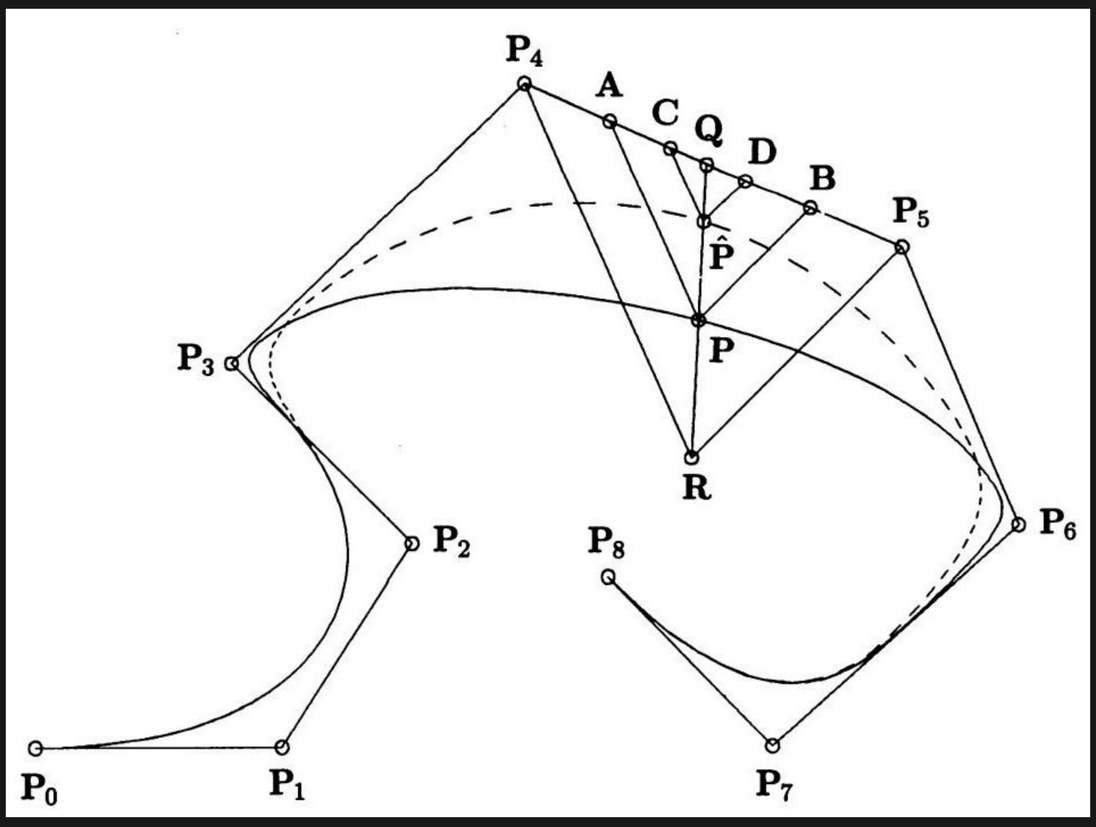


图 11.14。通过同时改变 w_4 和 w_5 ，同时向 $\mathbf{P}_4$ 和 $\mathbf{P}_5$ 拉动。

与

$$a_k = \frac{N_{k,p}(\bar{u})\, w_k}{\sum_{i=0}^{n} N_{i,p}(\bar{u})\, w_i} \quad a_{k+1} = \frac{N_{k+1,p}(\bar{u})\, w_{k+1}}{\sum_{i=0}^{n} N_{i,p}(\bar{u})\, w_i} \tag{11.28}$$

方程 (11.27) 表示 $\mathbf{P}$ 位于重心坐标系 $(\mathbf{RP}_k\mathbf{P}_{k+1})$ 的平面上 (不熟悉重心坐标的读者应参考 Boehm 和 Prautzsch [Boch94] 或 Farin [Fari93])。现在假设 $\mathbf{P}$ 移动到一个新位置 $\widehat{\mathbf{P}}$ 。我们必须确定新的权重 $\widehat{w}_k$ 和 $\widehat{w}_{k+1}$ ，以导致所需的移动。根据方程 (11.27)，$\widehat{\mathbf{P}}$ 可以表示为

$$\widehat{\mathbf{P}} = (1 - \widehat{a}_k - \widehat{a}_{k+1})\,\mathbf{R} + \widehat{a}_k\mathbf{P}_k + \widehat{a}_{k+1}\mathbf{P}_{k+1} \tag{11.29}$$

与

$$\widehat{a}_k = \frac{N_{k,p}(\bar{u})\,\widehat{w}_k}{d}\ \widehat{a}_{k+1} = \frac{N_{k+1,p}(\bar{u})\,\widehat{w}_{k+1}}{d}$$

where

$$d = \sum_{i \neq k,k+1} N_{i,p}(\bar{u})\, w_i + N_{k,p}(\bar{u})\,\widehat{w}_k + N_{k+1,p}(\bar{u})\,\widehat{w}_{k+1}$$

如果 $\widehat{w}_k$ 和 $\widehat{w}_{k+1}$ 表达为

$$\widehat{w}_k = \beta_k w_k\ \widehat{w}_{k+1} = \beta_{k+1} w_{k+1} \tag{11.30}$$

那么经过一些操作可以得到

$$\beta_k = \frac{1 - a_k - a_{k+1}}{a_k} \Big/ \frac{1 - \widehat{a}_k - \widehat{a}_{k+1}}{\widehat{a}_k}$$

$$\beta_{k+1} = \frac{1 - a_k - a_{k+1}}{a_{k+1}} \Big/ \frac{1 - \widehat{a}_k - \widehat{a}_{k+1}}{\widehat{a}_{k+1}} \tag{11.31}$$

方程 (11.27)-(11.31) 导致了一种简单的几何方法来求解未知数 β_k 和 β_{k+1} ，从而从 w_k 和 w_{k+1} 中获得 $\widehat{w}_k$ 和 $\widehat{w}_{k+1}$ 。由于 $a_k, a_{k+1}, \widehat{a}_k$ 和 $\widehat{a}_{k+1}$ 是相对于 $\mathbf{R}, \mathbf{P}_k$ 和 $\mathbf{P}_{k+1}$ 的重心坐标，它们可以通过 (见图 11.14) 表示。

$$a_k = \frac{|\mathbf{B} - \mathbf{P}_{k+1}|}{|\mathbf{P}_{k+1} - \mathbf{P}_k|}\ a_{k+1} = \frac{|\mathbf{A} - \mathbf{P}_k|}{|\mathbf{P}_{k+1} - \mathbf{P}_k|}$$

$$\widehat{a}_k = \frac{|\mathbf{D} - \mathbf{P}_{k+1}|}{|\mathbf{P}_{k+1} - \mathbf{P}_k|}\ \widehat{a}_{k+1} = \frac{|\mathbf{C} - \mathbf{P}_k|}{|\mathbf{P}_{k+1} - \mathbf{P}_k|} \tag{11.32}$$

其中 $\mathbf{A}, \mathbf{B}, \mathbf{C}$ 和 $\mathbf{D}$ 是沿 $\mathbf{P}_k\mathbf{P}_{k+1}$ 的点，使得 $\mathbf{PA}$ 和 $\widehat{\mathbf{P}}\mathbf{C}$ 与 $\mathbf{RP}_k$ 平行，而 $\mathbf{PB}$ 和 $\widehat{\mathbf{P}}\mathbf{D}$ 与 $\mathbf{RP}_{k+1}$ 平行。为了使 $a_k, \bar{u}$ 不为零，必须在 $[u_k, u_{k+p+1})$ 内；同样，对于不为零的 $a_{k+1}\bar{u}$ ，必须在 $[u_{k+1}, u_{k+p+2})$ 内。如果选择 $\mathbf{P}$ 使得 $\bar{u}$ 在这些区间的交集内，则 a_k 和 a_{k+1} 都不为零，因此 $\widehat{\mathbf{P}}$ 可以放置在三角形 $\mathbf{RP}_k\mathbf{P}_{k+1}$ 内的任何位置。然而，如果 $\bar{u} \in [u_k, u_{k+1})$ ，则 $a_{k+1} = 0$，方程 (11.27) 退化为

$$\mathbf{P} = (1 - a_k)\,\mathbf{R} + a_k\mathbf{P}_k \tag{11.33}$$

即，同时修改退化为单一权重修改。

图 11.15 和 11.16 显示了同时的推拉。注意，在图 11.15 中曲线变化是令人愉悦且对称的，而在图 11.16 中则不是。这是因为在图 11.15 中，运动沿着由 $\mathbf{R}$ 和初始 $\mathbf{P} = \mathbf{C}(\bar{u}; w_k)$ 定义的线进行。在这种情况下，我们有

$$\frac{\widehat{a}_k}{a_k} = \frac{\widehat{a}_{k+1}}{a_{k+1}}$$

这意味着 $\beta_k = \beta_{k+1}$ 和

$$\frac{w_k}{w_{k+1}} = \frac{\widehat{w}_k}{\widehat{w}_{k+1}}$$

也就是说，权重的比率被保留。

同时权重修改的用户界面可能如下所示。设计师选择一个控制多边形边，$\mathbf{P}_k\mathbf{P}_{k+1}$ 。系统计算相应的节点，t_k 和 t_{k+1} ，并设置 $\bar{u} = 1/2\,(t_k + t_{k+1})$ 。然后系统计算 $\mathbf{R}, \mathbf{P} = \mathbf{C}(\bar{u})$ 和 $\mathbf{Q}$ ($\mathbf{RP}$ 与 $\mathbf{P}_k\mathbf{P}_{k+1}$ 的交点；见

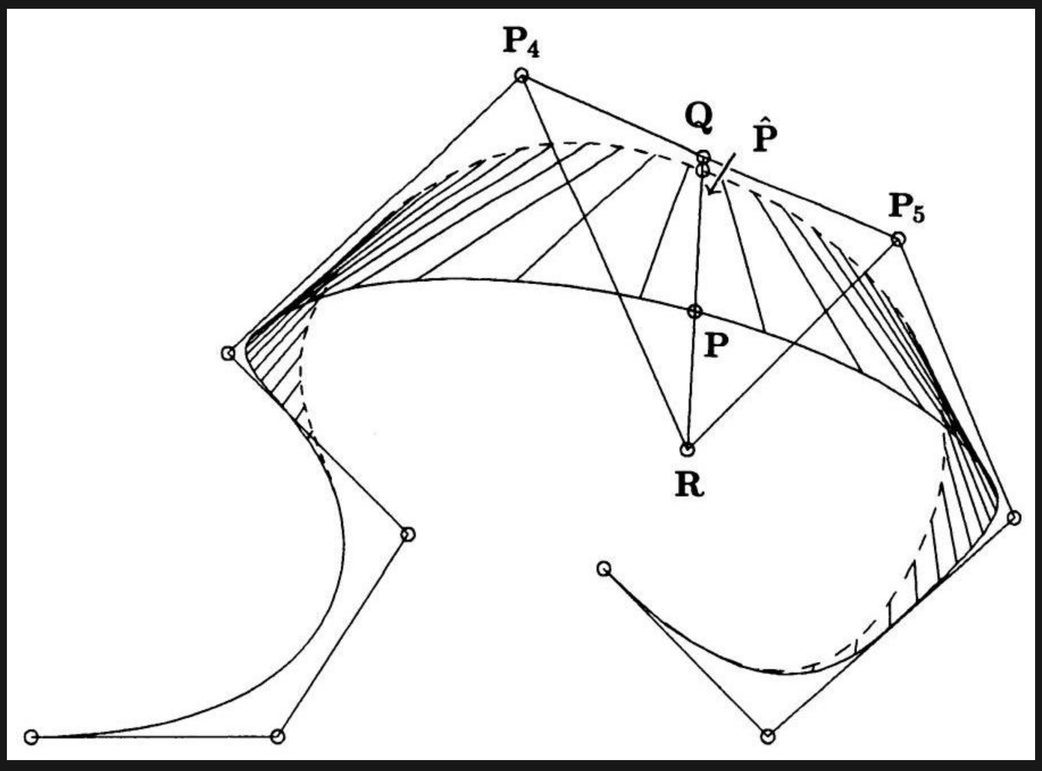


图 11.15。在 $u = 1/2\,(t_4 + t_5) = 0.608$ 处以距离 $d = 0.088$ 沿着 **RQ** 方向进行同时拉动，通过改变 $w_4 = w_5 = 0.05$ ；新的权重为 $\widehat{w}_4 = \widehat{w}_5 = 1.23$ 。曲线的变化既不是透视的也不是平行的。

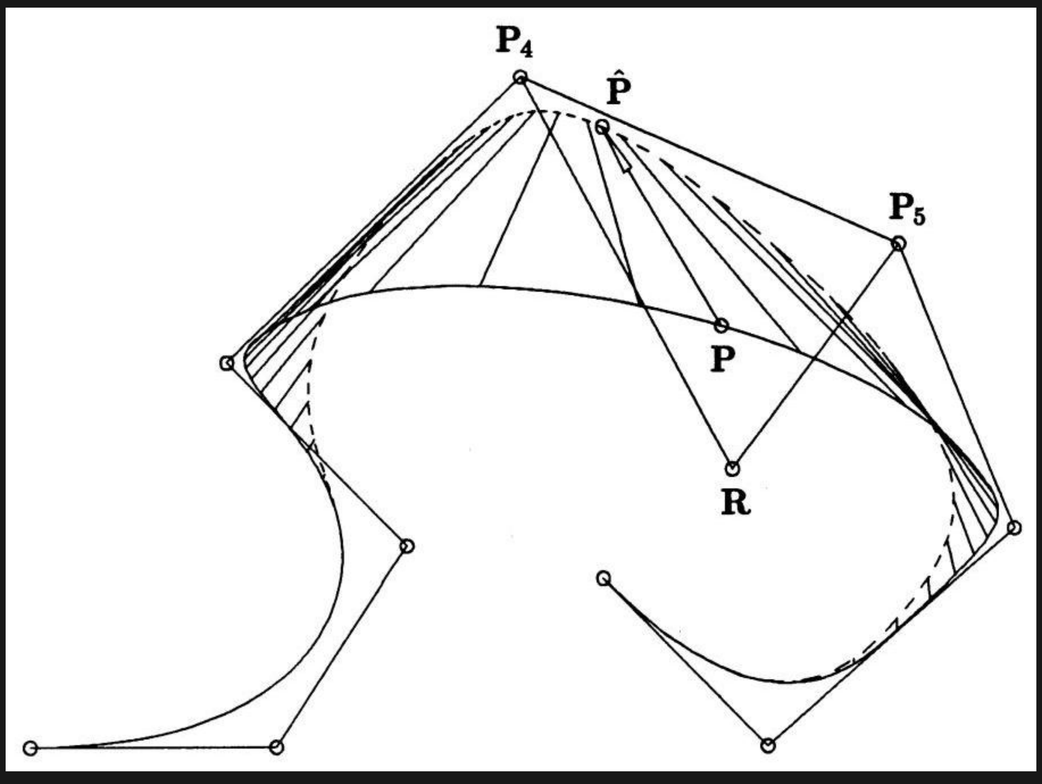


图 11.16。在 $u = 0.612$ 处以距离 $d = 0.088$ 沿着与 RQ 不同的方向进行同时拉动，通过改变 $w_4 = w_5 = 0.05$ ；新的权重为 $\widehat{w}_4 = 1.89$ 和 $\widehat{w}_5 = 0.48$ 。

图 11.14。这些组件，包括线段 **RQ** ，被绘制在屏幕上。设计师现在可以交互式地重新定位 **P** 沿着线 **RQ** ，同时系统计算新的权重并重新绘制受影响的曲线段。当然，设计师应该能够覆盖默认值 $\bar{u}$ 和默认线段 **RQ** 。

另一种情况是允许设计师选择曲线点 **P** (从而 $\bar{u}$)，然后系统通过找到两个节点 t_k 和 t_{k+1} 来选择多边形边 $\mathbf{P}_k\mathbf{P}_{k+1}$ ，使得 $t_k < \bar{u} < t_{k+1}$ 。然而，如果 $\bar{u}$ 等于或接近一个节点，这将是有问题的，因为如果任意选择一个多边形边，推拉效果是非对称的。在这种情况下，可以插入一个节点 $\widehat{u}$ ，将 $\bar{u}$ 放置在两个节点之间的中间位置。具体来说

$$\bar{u} = \frac{1}{2}\left(\hat{t}_k + \hat{t}_{k+1}\right)$$

$$= \frac{1}{2p}\left(u_{k+1} + \cdots + u_{k+p-1} + \widehat{u} + u_{k+2} + \cdots + u_{k+p} + \widehat{u}\right)$$

$$= \frac{1}{2p}\left(u_{k+1} + u_{k+p} + 2\widehat{u} + 2\sum_{j=k+2}^{k+p-1} u_j\right)$$

这意味着

$$\widehat{u} = p\bar{u} - \frac{1}{2}\left(u_{k+1} + u_{k+p}\right) - \sum_{j=2}^{p-1} u_{k+j}$$

然而，我们提醒读者，$\widehat{u}$ 可能已经是一个结；此外，生成的新多边形腿 $\mathbf{P}_k\mathbf{P}_{k+1}$ 可能非常接近 $\mathbf{P} = \mathbf{C}\,(\bar{u})$ ，在这种情况下，可用的拉伸长度非常有限。

最后，请注意我们之前假设修改权重的两个控制点是相邻的。这是因为这样可以产生令人满意的结果，并且通常是所期望的。然而，方程 (11.27)-(11.32) 的推导并没有利用这一假设。事实上，如果选择 $\mathbf{P}_r$ 和 $\mathbf{P}_s$ ，那么唯一的要求是 $\bar{u} \in [u_r, u_{r+p+1})$ 和 $\bar{u} \in [u_s, u_{s+p+1})$ ，以便重心坐标 a_r 和 a_s 不会消失。有关此情况的更多细节，请参见 [Pieg89c]。

3.3 单表面权重的修改

修改单个表面权重 $w_{k,\ell}$ 类似于曲线情况。假设 $\bar{u} \in [u_k, u_{k+p+1})$ 和 $\bar{v} \in [v_\ell, v_{\ell+q+1})$ ，并定义

$$\mathbf{R} = \mathbf{S}\,(\bar{u}, \bar{v}; w_{k,\ell} = 0)$$

$$\mathbf{M} = \mathbf{S}\,(\bar{u}, \bar{v}; w_{k,\ell} = 1)$$

$$\mathbf{P} = \mathbf{S}\,(\bar{u}, \bar{v}; w_{k,\ell} \neq 0, 1) \tag{11.35}$$

然后 **M** 和 **P** 可以表示为

$$\mathbf{M} = (1 - s)\,\mathbf{R} + s\mathbf{P}_{k,\ell}$$

$$\mathbf{P} = (1 - t)\,\mathbf{R} + t\mathbf{P}_{k,\ell} \tag{11.36}$$

其中

$$s = \frac{N_{k,p}\,(\bar{u})\,N_{\ell,q}\,(\bar{v})}{\sum\limits_{i,j \neq k,\ell} N_{i,p}\,(\bar{u})\,N_{j,q}\,(\bar{v})\,w_{i,j} + N_{k,p}\,(\bar{u})\,N_{\ell,q}\,(\bar{v})}$$

$$t = \frac{N_{k,p}(\bar{u}) N_{\ell,q}(\bar{v}) w_{k,\ell}}{\sum_{i=0}^{n} \sum_{j=0}^{m} N_{i,p}(\bar{u}) N_{j,q}(\bar{v}) w_{i,j}} = R_{k,\ell}(\bar{u}, \bar{v}) \tag{11.37}$$

$(i, j \neq k, \ell$ means exclude the (k, ℓ) th term. $)$ 使用方程 (11.36) 和 (11.37)，可以得到交叉比

$$w_{k,\ell} = \frac{1-s}{s} \Big/ \frac{1-t}{t} = \frac{\mathbf{P}_{k,\ell}\mathbf{M}}{\mathbf{RM}} \Big/ \frac{\mathbf{P}_{k,\ell}\mathbf{P}}{\mathbf{RP}} \tag{11.38}$$

方程 (11.35)-(11.38) 意味着

- 当 $w_{k,\ell}$ 增加 (减少) 时，t 增加 (减少)，因此表面被拉向 (推离) $\mathbf{P}_{k,\ell}$ ；
- $\mathbf{P}$ 沿着通过 $\mathbf{P}_{k,\ell}$ 的直线段移动；因此，对 $w_{k,\ell}$ 的修改以可预测的方式改变表面；
- 当 $\mathbf{P}$ 接近时，$\mathbf{P}_{k,\ell}$，t 接近 1，而 $w_{k,\ell}$ 趋向于无穷大。

最后，如果 $\mathbf{P}$ 要移动到新位置 $\widehat{\mathbf{P}}$ 和 $d = \left|\mathbf{P} - \widehat{\mathbf{P}}\right|$ ，则所需的新权重值 $\widehat{w}_{k,\ell}$ 为

$$\widehat{w}_{k,\ell} = w_{k,\ell}\left(1 \pm \frac{d}{R_{k,\ell}(\bar{u}, \bar{v})\left(\mathbf{P}_{k,\ell}\mathbf{P} \mp d\right)}\right) \tag{11.39}$$

当出现 $\pm$ 或 $\mp$ 时，拉动操作的情况下取上面的符号，而推送操作的情况下取下面的符号 [Pieg89d]。

单个表面权重的修改用户界面可以以类似于曲线的方式实现。设计师可以选择一个现有的控制点或一个或两个位于控制网边缘的点；如果是两个，则它们必须位于控制网的一个四边形上，一个在 u 方向的边上，另一个在 v 方向的边上。如果没有选择现有的控制点，则使用逆节点插入来获得 $\mathbf{P}_{k,\ell}$ 。从那里，过程与曲线修改相同。与之前提到的五个关于曲线权重修改的备注类似的声明也适用于表面权重修改。图 11.17 说明了表面推送。

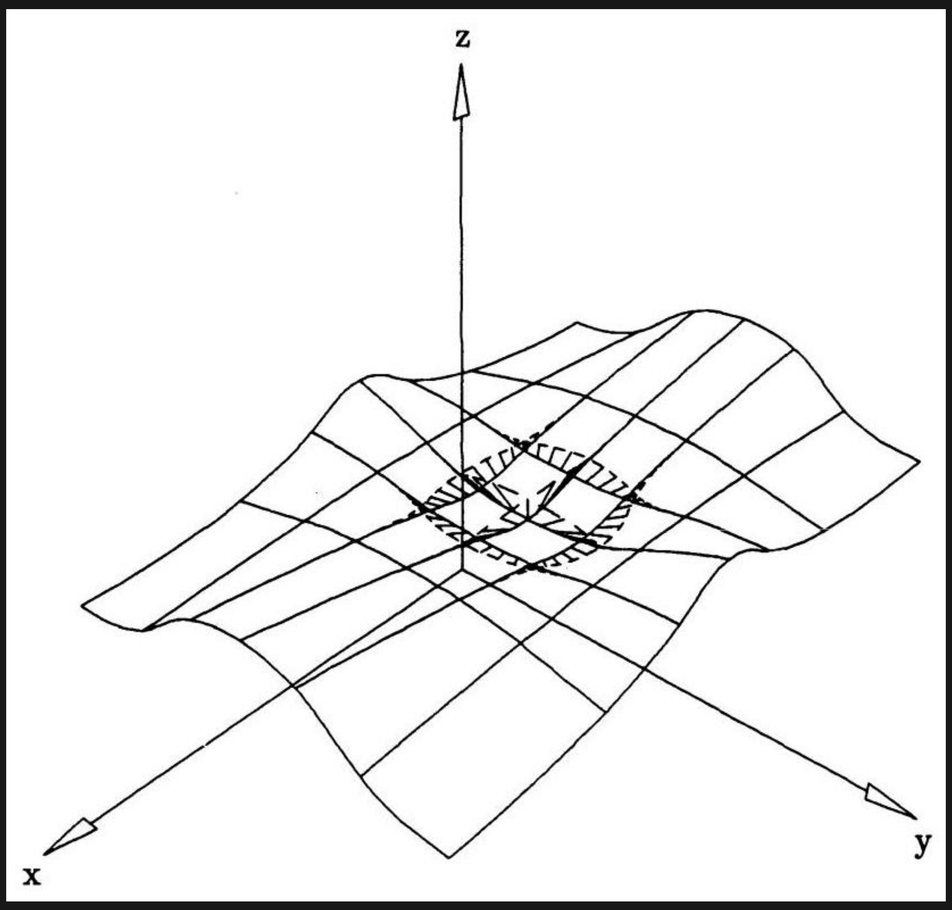


图 11.17. 在 $u = v = 0.55$ 处向 $\mathbf{P}_{4,3}$ 推送表面，距离 $d = 1$ ；注意表面形状的透视变化。

4 形状操作符

在此定义的形状操作符是用于粗略雕刻曲线和表面的工具。它们的操作如下:

1. 指定一个局部曲线段或表面区域，在其中进行形状修改；然后使用节点细化在局部引入更多控制点；
2. 根据各种参数和/或函数重新定位对指定段或区域影响局部的控制点；
3. 对局部段或区域应用在指定容差内的节点移除，以减少定义控制点的数量。

我们在这里介绍三种操作符: 扭曲、展平和弯曲。文献中可以找到其他类似的操作符，例如，拉伸、扭转和渐变 [Barr83; Cobb84]。我们强调我们的算法完全基于几何；它们不考虑材料的物理特性或弹性和塑性理论。

在介绍操作符之前，我们提到一些与节点细化和移除步骤相关的问题。细化的方法和程度会影响应用形状操作符的结果。因此，尽管设计者应该免于细节，但他必须能够控制细化的程度。假设 $[u_s, u_e]$ 是一个感兴趣的曲线段。如果 u_s 和 u_e 还不是节点，则将其插入。将 n 额外节点递归插入到 $[u_s, u_e]$ 中最长节点跨度的中点是一个好的方法。这往往会生成大致均匀间隔的节点，并且希望 (但不一定) 控制点也会大致均匀间隔。现在，如果 u_s 和 u_e 是定义最终细化区间限制的节点 (注意 e 在细化过程中会变化)，那么可以在不修改 $[u_s, u_e]$ 外部曲线的情况下重新定位以下控制点。

$$\mathbf{P}_s, \ldots, \mathbf{P}_{e-p-1} \tag{11.40}$$

其中 p 是曲线的度数。在形状操作之后，节点移除通常会去除许多插入的节点。如果曲线不应在 $[u_s, u_e]$ 外部被修改，则只有以下节点是移除的候选者。

$$u_i \; i = s + \frac{p}{2}, \ldots, e - \frac{p}{2} - 1 \;\text{ if } p \text{ is even}$$

$$u_i \; i = s + \frac{p+1}{2}, \ldots, e - \frac{p+1}{2} \;\text{ if } p \text{ is odd} \tag{11.41}$$

4.1 变形

扭曲是一种工具，可以用来相对任意地变形曲线的局部段或表面上的区域。首先考虑曲线扭曲，它是通过基于公式的控制点重新定位来实现的。

$$\widehat{\mathbf{P}}_i = \mathbf{P}_i + fd\mathbf{W} \tag{11.42}$$

一般来说，f 是一个函数，d 是一个常数，而 $\mathbf{W}$ 可以是一个函数或一个常数。f 控制变形形状，d 是控制点移动的上限 (因此也是变形距离的上限)，而 $\mathbf{W}$ 给出了变形的方向。设计师通常通过选择曲线点 $\mathbf{C}(u_s)$ 和 $\mathbf{C}(u_e)$ 来确定变形的程度。曲线变形限制在这个区间内。一个有用的变形函数为

$$f(t) = \frac{R_{3,3}(t)}{R_{\max}} \tag{11.43}$$

其中 $R_{\max} = R_{3,3}(1/2)$ 和 $R_{3,3}(t)$ 是有理立方 B 样条基函数

$$R_{3,3}(t) = \frac{N_{3,3}(t) w_3}{\sum_{i=0}^{6} N_{i,3}(t) w_i} \tag{11.44}$$

由节点向量定义

$$T = \left\{0, 0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1, 1, 1\right\} \tag{11.45}$$

和权重

$$w_i = \{1, 1, 1, w_3, 1, 1, 1\}$$

图 11.18 显示了 $f(t)$ 对于 w_3 的几个不同值。显然，这种形式的函数适合于拉出一个隆起或在曲线上推入一个凹陷。现在假设 w_3 是固定的，设计师指定一个最大变形距离 d 。变形方向 $\mathbf{W}$ 仍需指定。合理的选择包括:

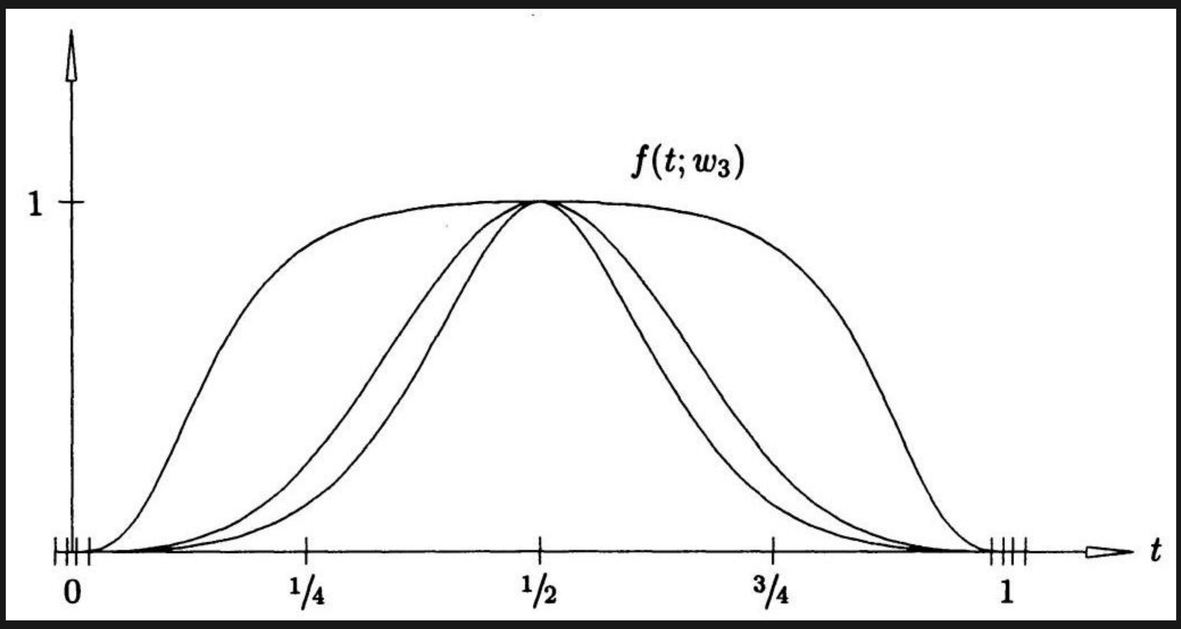


图 11.18. 使用有理基函数 $R_{3,3}(t;w_3)$ 和 $w_3=\{1/5,1,30\}$ 的变形函数。

- 由以下给出的变量方向

$$\mathbf{W}(t)=\pm\mathbf{N}(u(t)) \tag{11.46}$$

其中 $\mathbf{N}(u)$ 是 $\mathbf{C}(u)$ 和 $u(t)=(u_e-u_s)t+u_s$ 的单位法向量；- 由以下给出的恒定方向

$$\mathbf{W}=\pm\mathbf{N}\left(\frac{1}{2}(u_s+u_e)\right) \tag{11.47}$$

区间 $[u_s,u_e]$ 现在被细化，产生控制点 $\mathbf{P}_i, i=s,\ldots,e-p-1$ (方程 [11.40])，这些控制点可以根据方程 (11.42) 进行重新定位。为了应用该方程，我们必须为每个 $\mathbf{P}_i, i=s,\ldots,e-p-1$ 指定一个参数 t_i 。设 δ 表示从 $\mathbf{P}_{s-1}$ 到 $\mathbf{P}_{e-p}$ 的控制多边形沿线的总累积弦长。用 t_i 表示从 $\mathbf{P}_{s-1}$ 到 $\mathbf{P}_i$ 的累积弦长，经过 δ 归一化。注意到 $0\le t_i\le 1$ 对于 $s\le i\le e-p-1$ 。然后 $\mathbf{P}_i$ 被重新定位为

$$\widehat{\mathbf{P}}_i=\mathbf{P}_i+f(t_i)\,d\mathbf{W}(t_i)\;\; i=s,\ldots,e-p-1 \tag{11.48}$$

最后，按照之前讨论的内容，对区间应用节点移除。

图 11.19a-11.19d 显示了曲线扭曲的步骤。在图 11.19a 中，选择了曲线段 $[\mathbf{C}(u_s),\mathbf{C}(u_e)]$ 、(恒定) 方向 $\mathbf{W}$ 和距离 d 。图 11.19b 显示了节点细化后的曲线。局部控制点被重新定位，如图 11.19c 所示。图 11.19d 描绘了应用节点移除后的最终曲线。

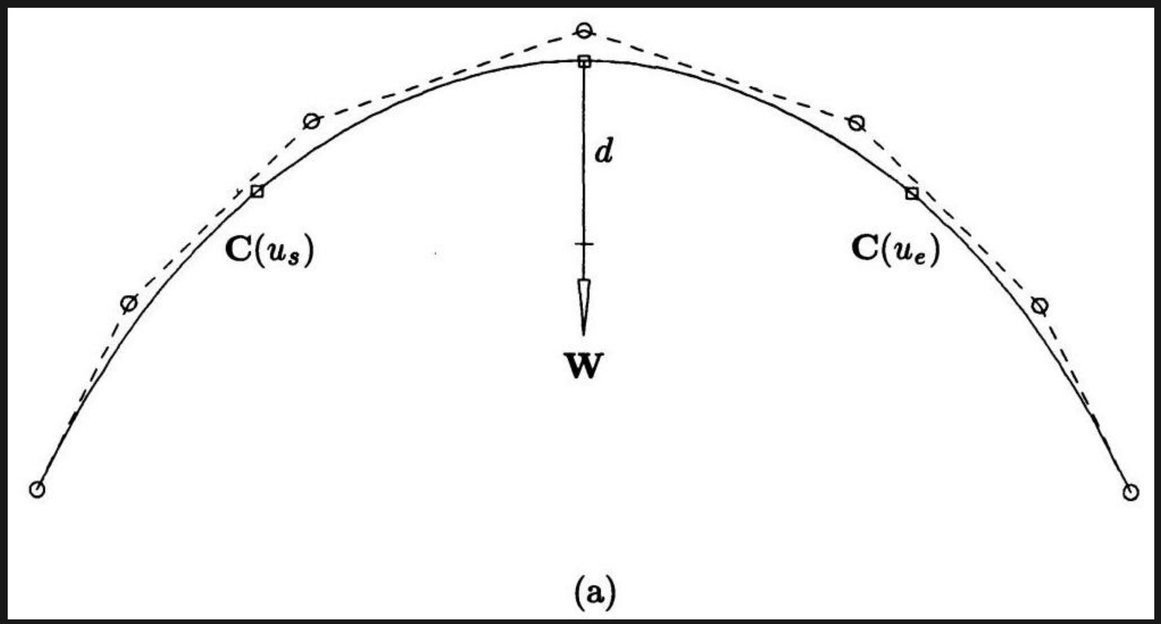


图 11.19. 曲线扭曲。(a) 原始曲线，跨度为 $[\mathbf{C}(u_s),\mathbf{C}(u_e)]$ ，将在方向 $\mathbf{W}$ 上扭曲距离 d ；(b) 细化后的曲线跨度；(c) 重新定位的控制点；(d) 移除的节点。

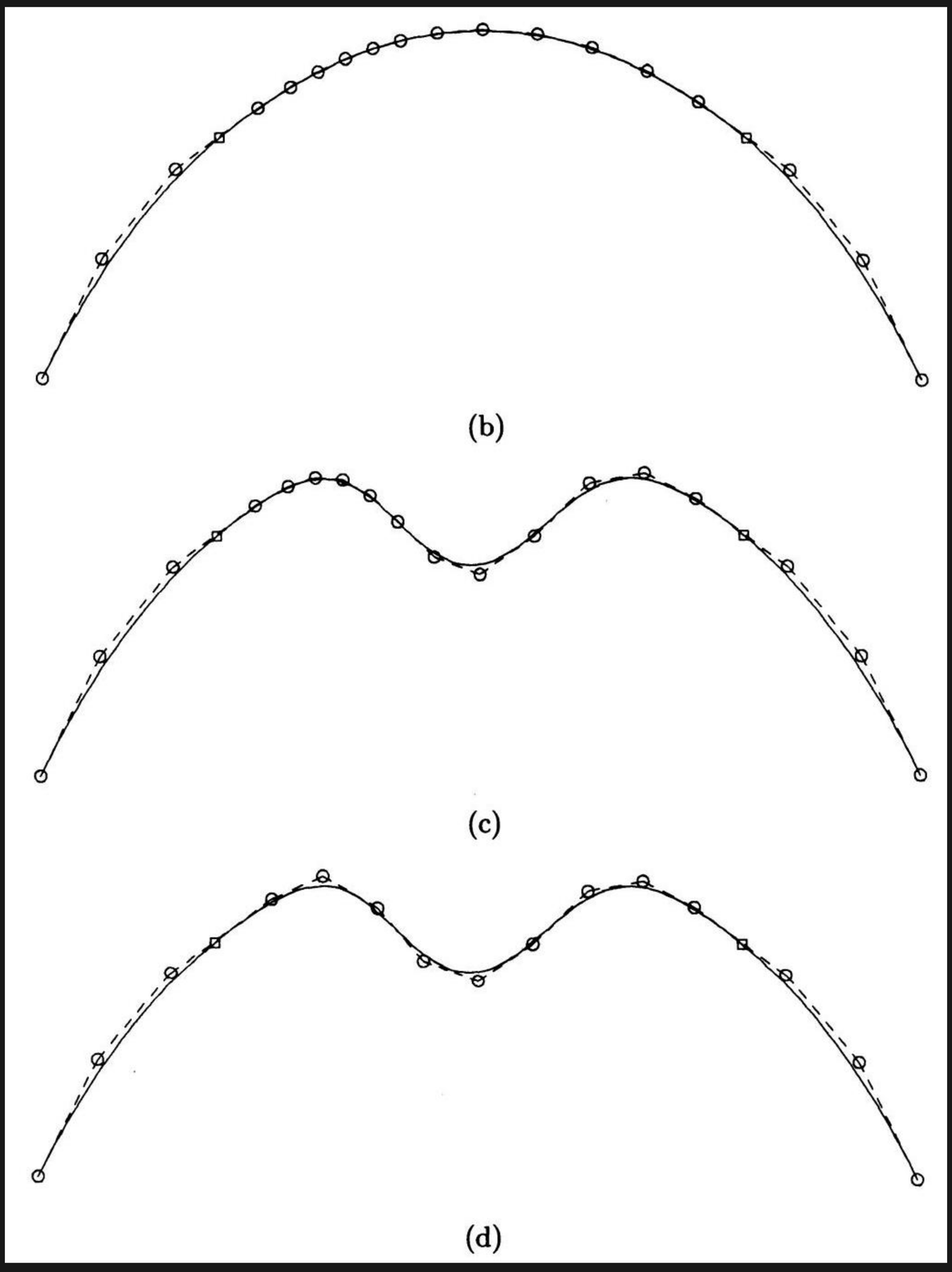


图 11.19.（续。）

改变定义参数的效果在图 11.20a-11.20c 中得以展示。在图 11.20a 中，使用不同的权重来“平方”扭曲区域。图 11.20b 显示了在保持权重和距离不变的情况下，改变扭曲方向的效果。最后，在图 11.20c 中，应用不同的距离来控制扭曲的幅度。

我们通过以下几点总结曲线扭曲:

- 显然，几乎任何形状和幅度的扭曲都是可能的；只需适当地定义函数 $f(t)$ 和 $\mathbf{W}(t)$ 。图 11.21a 和 11.21b 显示了两个进一步的例子；

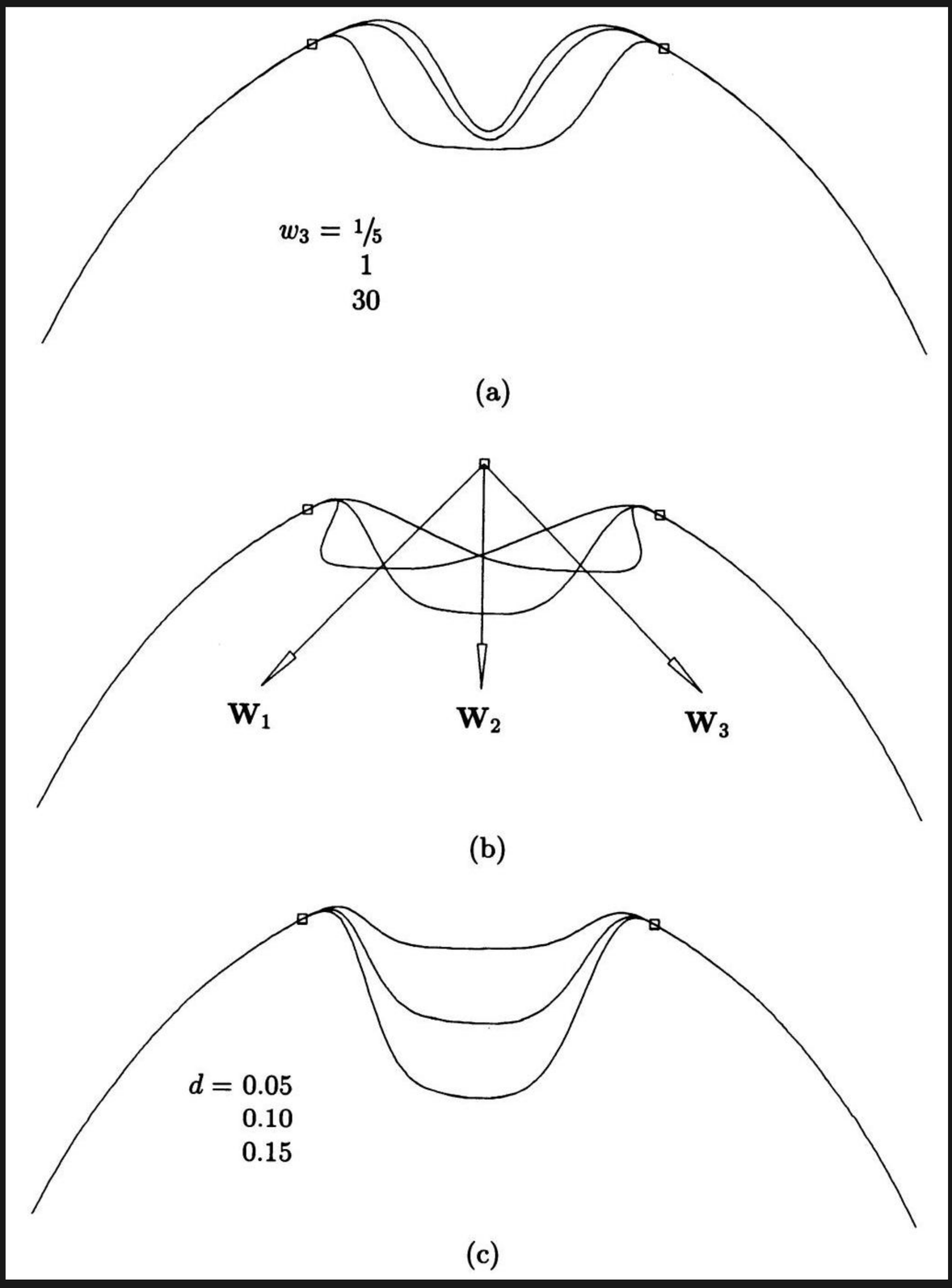


图 11.20。曲线扭曲 (a) 使用不同的权重 $w_3 = \{1/5, 1, 30\}$；(b) 使用不同的方向 $w_3 = 20$；(c) 使用不同的距离 $d = \{0.05, 0.1, 0.15\}$。

- 对于交互式扭曲，设计者可能希望选择一个曲线点作为扭曲的中心，然后在滑动尺度上修改幅度 d、权重 w_3 和扭曲段的范围。该段选择尽可能对称于扭曲中心，沿控制多边形的距离进行测量。交互完成后进行节点移除；节点细化必须作为预处理步骤进行（可以在整个曲线上，或者在用户指定的“最大”段上），或者细化必须在段长度扩展时进行更新（扩展）；
- 而不是将 t_i 计算为归一化距离，可以使用由方程 (11.4) 计算的 B 样条节点；在这种情况下，函数 f 和 $\mathbf{W}$ 必须在区间 $[u_s, u_e]$ 上进行参数化。

表面变形类似于曲线变形。我们区分两种类型，区域变形和表面曲线变形。在区域变形中，我们对非零表面积的区域应用变形函数；对于表面曲线变形，我们允许单变量变形函数沿着位于表面上的曲线滑动。

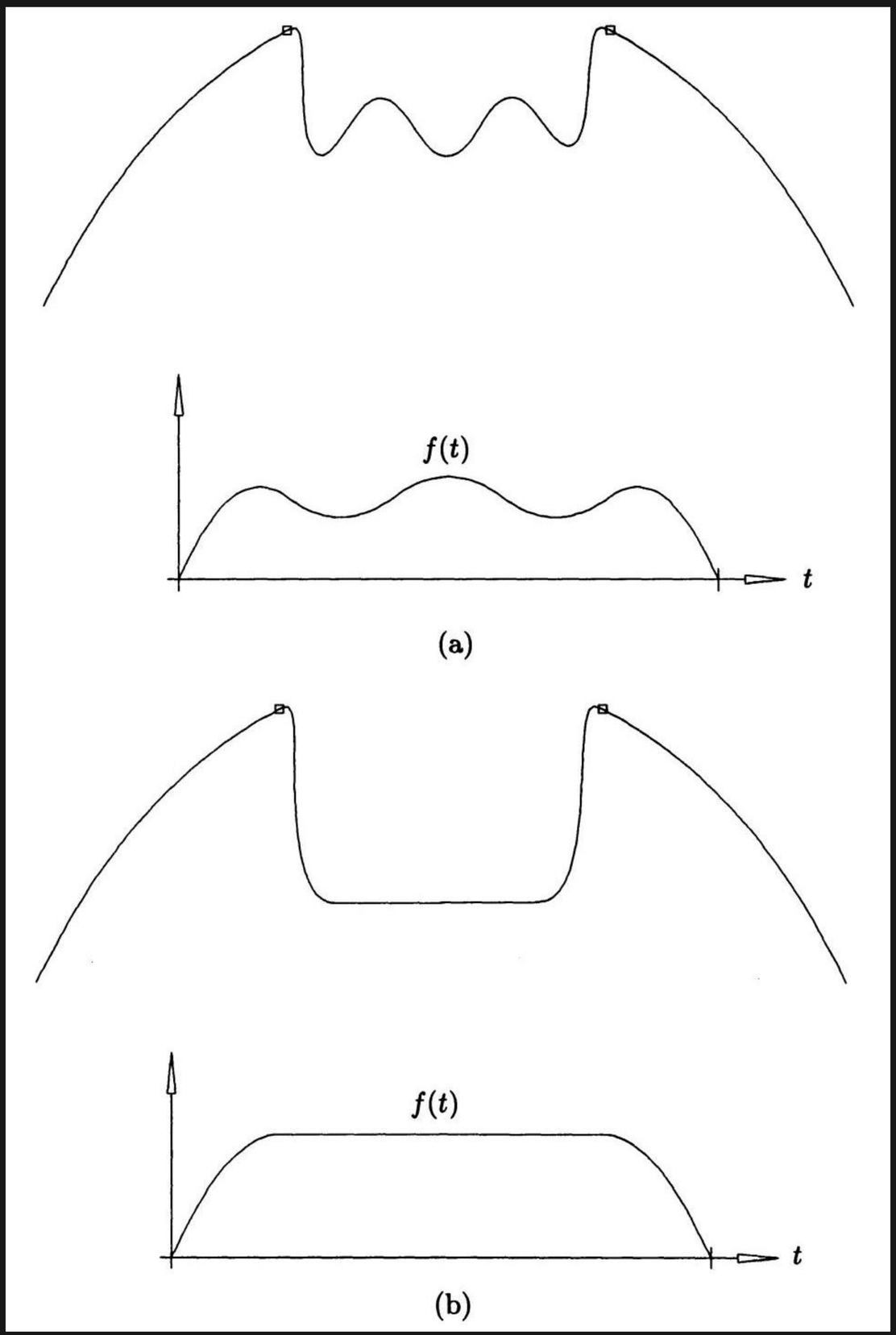


图 11.21. 具有不同形状函数的曲线变形。

设 $\mathcal{R}$ 为表面参数空间中的一个闭合区域，$\mathbf{S}(u,v)$ 。示例见图 11.22a 和 11.23a。区域变形是通过重新定位控制点产生的，基于公式

$$\widehat{\mathbf{P}}_{i,j} = \mathbf{P}_{i,j} + f d\mathbf{W} \tag{11.49}$$

类似于曲线变形，节点细化应用于区域 $[u_\alpha, u_\beta] \times [v_\gamma, v_\delta]$ (在图 11.22a 和 11.23a 中用虚线表示)，然后使用公式 (11.49)。节点移除在适当的区域应用 (见公式 [11.41])，作为后处理步骤。每个控制点 $\mathbf{P}_{i,j}, i = \alpha, \ldots, \beta - p - 1, j = \gamma, \ldots, \delta - q - 1$ 必须与一对参数 (s_i, t_j) 相关联。这些参数用于确定哪些 $\mathbf{P}_{i,j}$ 需要重新定位，并评估公式 (11.49) 中的函数 f 和 $\mathbf{W}$ 。类似于曲线变形，我们使用归一化累积弦长。特别地，我们对点集 $\mathbf{P}_{i,j}, i = \alpha - 1, \ldots, \beta - p$ 和 $j = \gamma - 1, \ldots, \delta - q$ 应用算法 A9.3。只有当 (u_i, v_j) 位于变形区域 $\mathcal{R}$ 中时，$\mathbf{P}_{i,j}$ 才会通过公式 (11.49) 重新定位，其中

$$u_i = (u_\beta - u_\alpha)\, s_i + u_\alpha \; v_j = (v_\delta - v_\gamma)\, t_j + v_\gamma$$

f 可以是定义在 $[0,1] \times [0,1]$ 上的任何标量值函数。与曲线的情况一样，B 样条基函数适用于拉动和推送。例如，归一化的双变量、双立方、有理 B 样条基函数适用于矩形区域。

$$f(s,t) = \frac{R_{3,3;3,3}(s,t)}{R_{\max}} \tag{11.50}$$

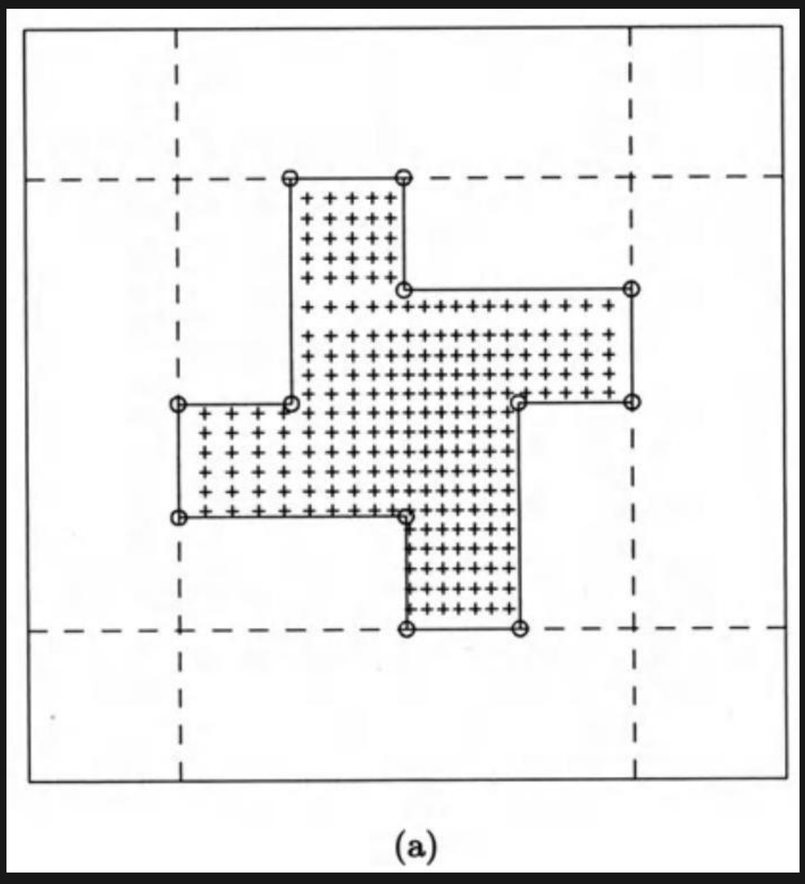


图 11.22. 表面区域变形。(a) 一个具有边界框和局部控制点的区域；(b) 控制点重新定位后的表面变形。

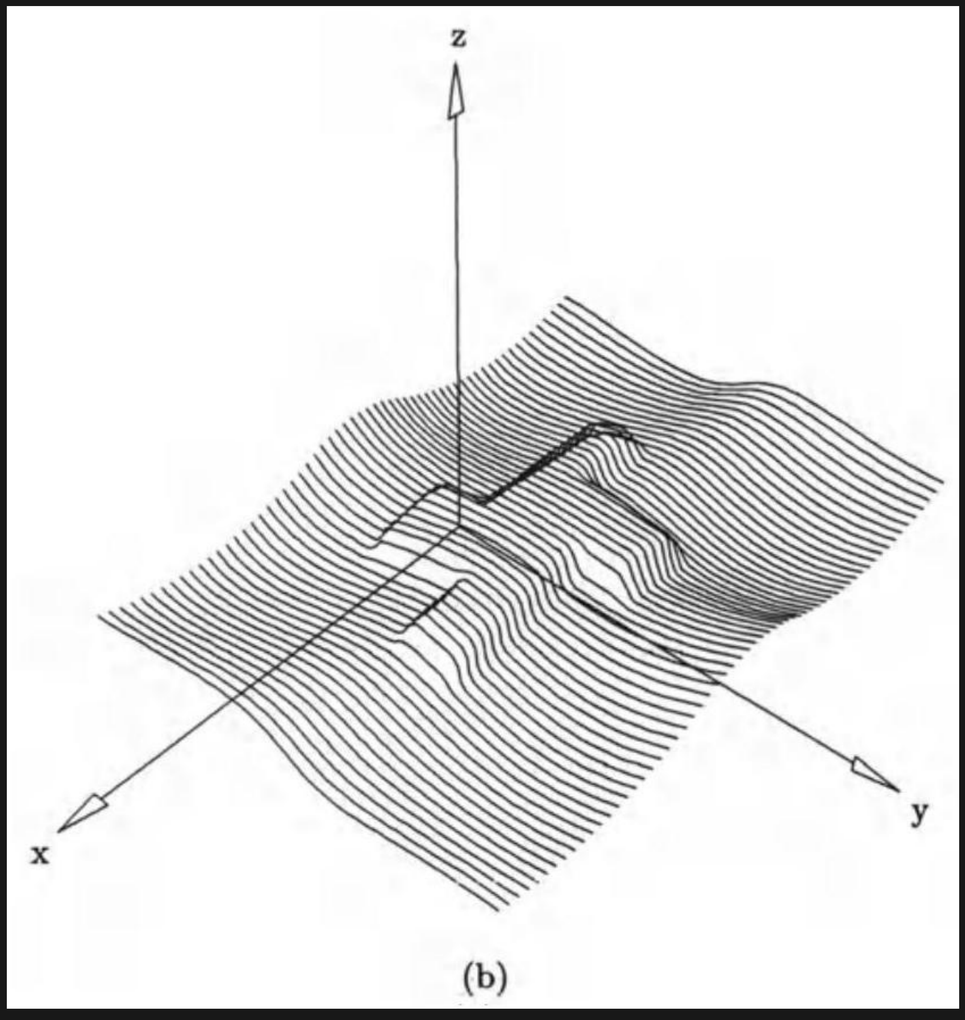


图 11.22.(续)

定义在节点向量上

$$S = \left\{0, 0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1, 1, 1\right\}$$

和

$$T = \left\{0, 0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1, 1, 1\right\}$$

所有权重均设置为 1，除了 $w_{3,3}$ ，该值可自由选择。作为另一个例子，如果所需的变形区域表现出圆形对称性，则可以使用旋转的一元 B 样条基函数。

在任意域上定义变形函数是困难的。图 11.22 和 11.23 中的区域变形是使用简单函数创建的。

$$f(s,t) = \begin{cases} 1 & \text{if } (u(s), v(t)) \in \mathcal{R} \\ 0 & \text{otherwise} \end{cases} \tag{11.51}$$

其中

$$u(s) = (u_\beta - u_\alpha)\, s + u_\alpha \; v(t) = (v_\delta - v_\gamma)\, t + v_\gamma$$

该变形函数像一个模具冲头，即，每个控制点 $(u(s_i), v(t_j)) \in \mathcal{R}$ 在 **W** 方向上移动了距离 d 。向量 **W** 可以是每个点的表面法线，或任何其他用户定义的向量。对于图 11.22 和 11.23，使用了光线投射、点在多边形内的例程来确定点在多边形区域内的包含关系。注意在图 11.23 中，该例程处理自相交的多边形。

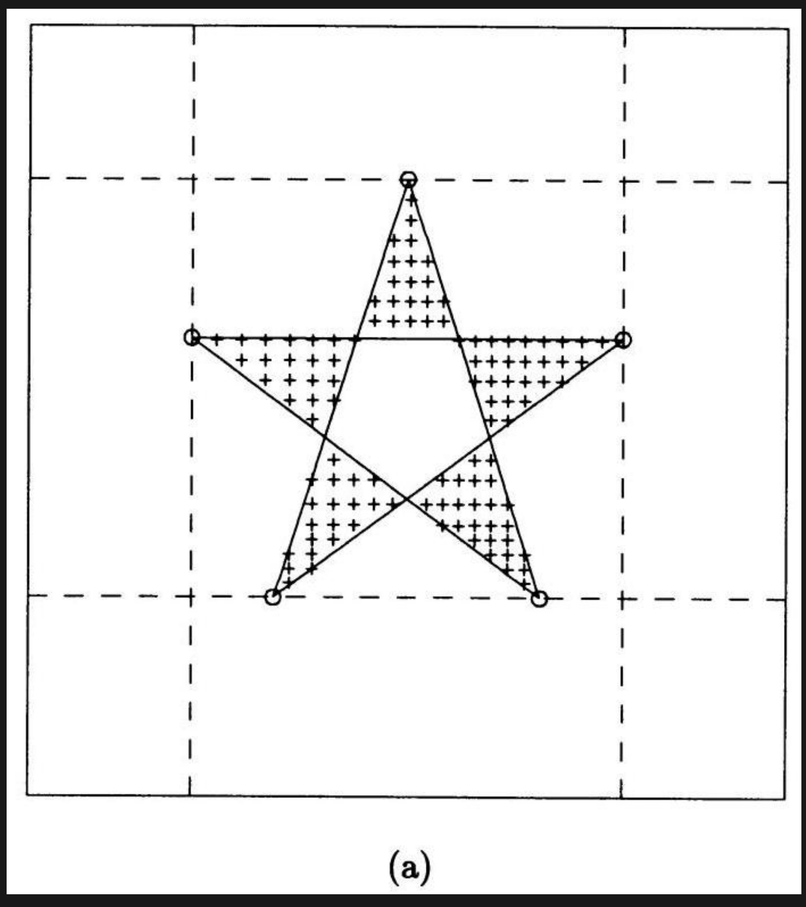


图 11.23. 表面区域变形。(a) 一个具有边界框和局部控制点的区域；(b) 控制点重新定位后的表面变形。

现在我们描述表面曲线变形。假设

$$\mathbf{C}(\lambda) = (u(\lambda), v(\lambda)) \tag{11.52}$$

是位于 $\mathbf{S}(u,v)$ 参数空间中的一条曲线，而 r 是沿着每个 λ 在 $\mathbf{C}(\lambda)$ 法线方向上测量到 $\mathbf{C}(\lambda)$ 两侧的距离。通过重新定位所有那些其参数 $(u(s_i), v(t_j))$ 在 $\mathbf{C}(\lambda)$ (重力场) 距离 r 范围内的表面控制点 $\mathbf{P}_{i,j}$ 来应用表面曲线变形。相关的 $\mathbf{P}_{i,j}$ 根据公式移动。

$$\widehat{\mathbf{P}}_{i,j} = \mathbf{P}_{i,j} + f(d_{i,j})\, d\mathbf{W}(\lambda_{i,j}) \tag{11.53}$$

在这里 $d_{i,j}$ 是 $(u(s_i), v(t_j))$ 到 $\mathbf{C}(\lambda)$ 的距离，而 $\lambda_{i,j}$ 对应于 $(u(s_i), v(t_j))$ 在 $\mathbf{C}(\lambda)$ 上的投影。扭曲方向的合理定义是

$$\mathbf{W}(\lambda) = \mathbf{N}(\mathbf{C}(\lambda)) \tag{11.54}$$

在这里 $\mathbf{N}(u,v)$ 是表面法向量。

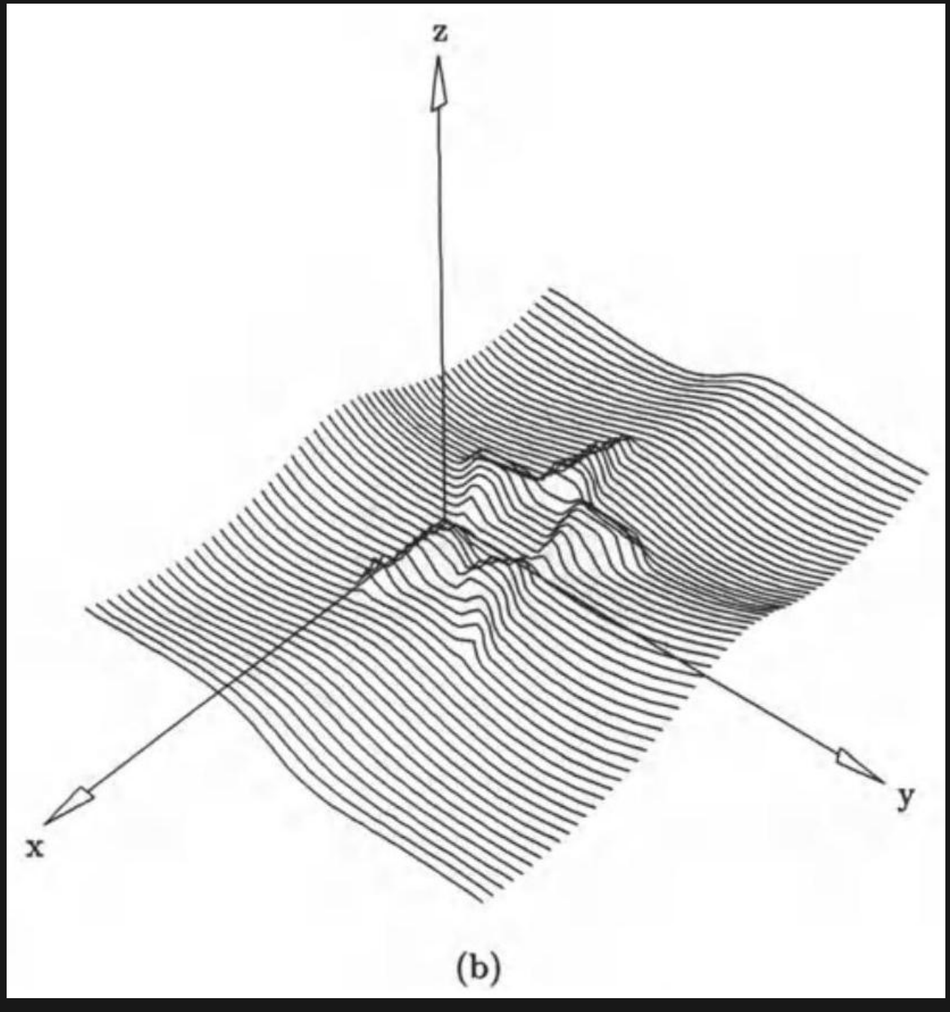


图 11.23. (续)

再次，f 的定义可能很微妙。我们使用了这个函数

$$f(d_{i,j}) = \begin{cases} 1 & \text{if } d_{i,j} \le r \\ 0 & \text{otherwise} \end{cases} \tag{11.55}$$

应用方程 (11.53) 的关键步骤是计算 $d_{i,j}$。如果 $\mathbf{C}(\lambda)$ 是一条折线，这个过程会简化。图 11.24 显示了折线扭曲的一个例子。图 11.24a 描述了由五条折线构成的缩写 NURBS。图 11.24b 显示了控制点的边界框 (用 + 标记)，这些控制点位于五个字母的重力场内。最终的折线扭曲表面在图 11.24c 中进行了说明。

4.2 扁平化

扁平化操作符允许设计师轻松地将直线段引入曲线和将平面区域引入表面。没有插入多个节点，因此曲线或表面的弯曲部分与平坦部分之间的过渡具有原始几何形状固有的平滑性。

首先考虑曲线扁平化，如图 11.25 和 11.26 所示：给定一条曲线 $\mathbf{C}(u)$ 和一条线 $\mathbf{L}$。曲线扁平化由三个步骤组成：

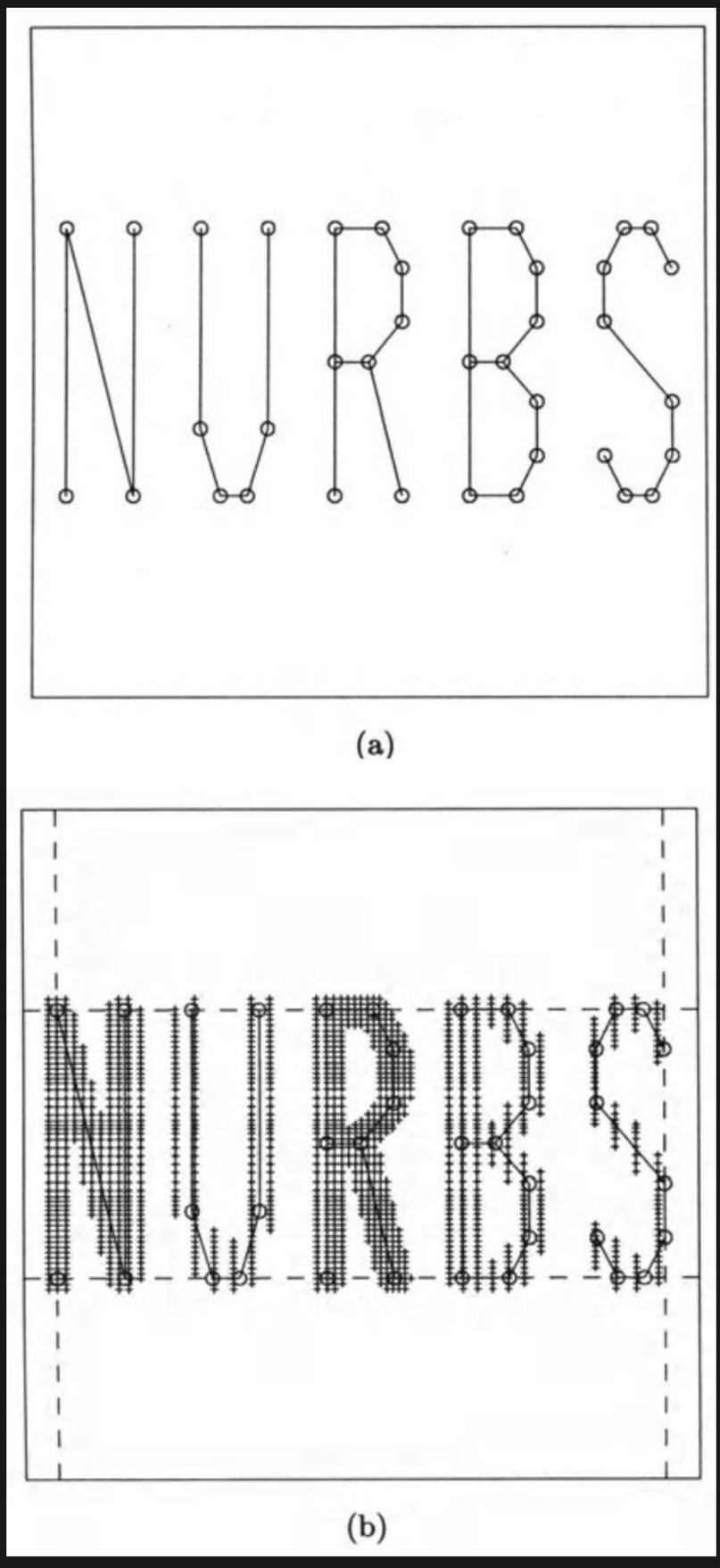


图 11.24. 表面折线扭曲。(a) 表示缩写 NURBS 的折线；(b) 局部于折线重力场的控制点；(c) 控制点重新定位后的表面折线扭曲。

1. 精炼曲线的节点以获得更多控制点；
2. 将一些指定的控制点 $\mathbf{P}_i$ 投影到该直线上；用 $\widehat{\mathbf{P}}_i$ 表示 $\mathbf{P}_i$ 的投影，将 $\mathbf{P}_i$ 替换为 $\widehat{\mathbf{P}}_i$；
3. 移除不必要的节点。

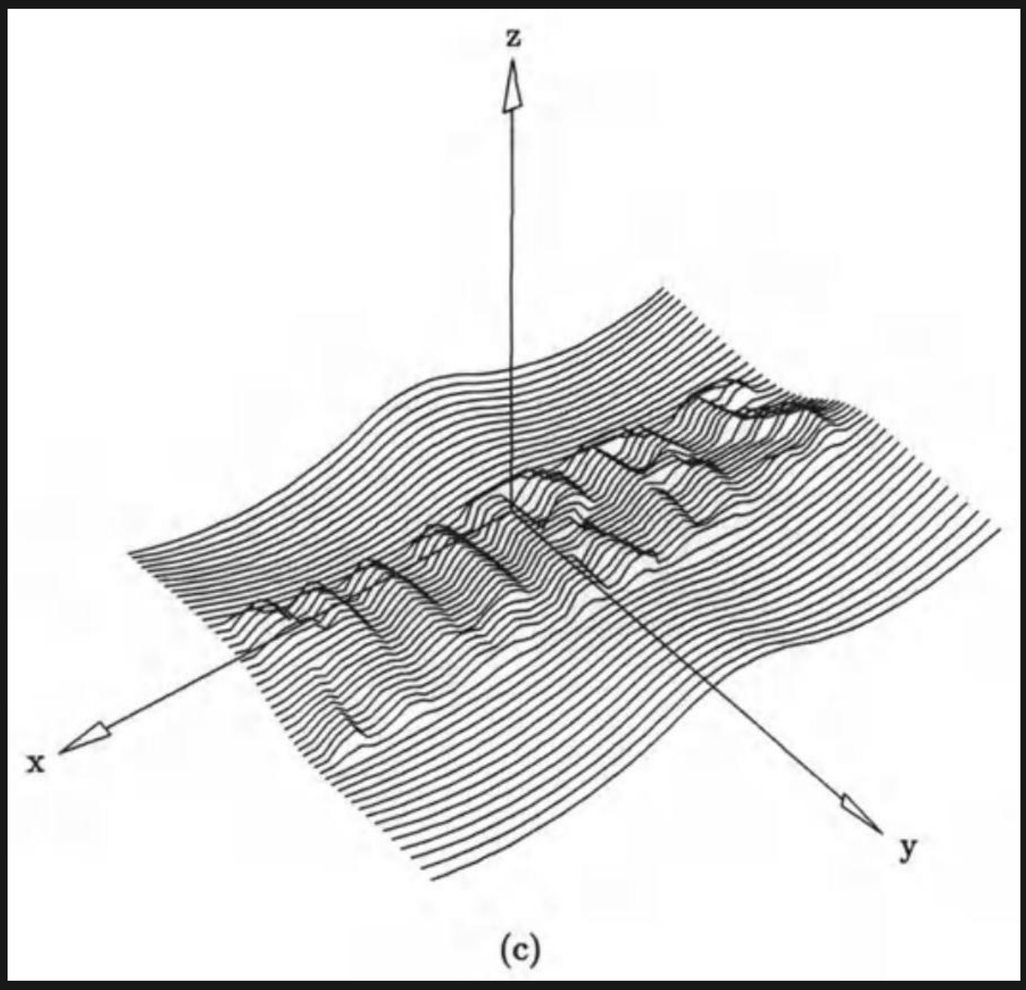


图 11.24. (续。)

除了 $\mathbf{L}$ 和 $\mathbf{C}(u)$，以下项目可以由设计者指定或由系统默认 (图 11.25a):

- 一个向量 $\mathbf{W}$，定义投影线，即 $\widehat{\mathbf{P}}_i = \mathbf{P}_i + \alpha\mathbf{W}$($\alpha$ 可以是负值或正值；该向量可以默认与 $\mathbf{L}$ 垂直；
- 参数 u_s 和 u_e：只有当控制点 $\mathbf{P}_i$ 局部于区间 $[u_s, u_e]$ 时，才会进行投影，即 $i = s, \ldots, e-p-1$；
- 线 $\mathbf{L}$ 上的两个点 $\mathbf{Q}_1$ 和 $\mathbf{Q}_2$；只有当控制点 $\mathbf{P}_i$ 的投影 $\widehat{\mathbf{P}}_i$ 位于线段 $\mathbf{Q}_1\mathbf{Q}_2$ 内时，才会用其投影 $\widehat{\mathbf{P}}_i$ 替换控制点 $\mathbf{P}_i$。

请注意，最后两个项目是规则，分别指示要投影哪些 $\mathbf{P}_i$ 和接受哪些 $\widehat{\mathbf{P}}_i$。对于图 11.25a 中所示的情况，将 $\mathbf{Q}_1 = \mathbf{C}(u_s)$ 和 $\mathbf{Q}_2 = \mathbf{C}(u_e)$ 设置为 $\mathbf{L}$ 与 $\mathbf{C}(u)$ 相交的点是一个合理的系统默认值。如果曲线是 xy 平面并与 $\mathbf{L}$ 相交，如图 11.25a 所示，另一个直观的规则是仅投影位于 $\mathbf{L}$ 一侧的那些 $\mathbf{P}_i$，如图 11.25b 所示。如果仅投影与 $[u_s, u_e]$ 相关的控制点，当 $\mathbf{L}$ 与曲线相交时，可能会出现意外结果 (图 11.25c)。如果曲线不与 $\mathbf{L}$ 相交，则投影所有与 $[u_s, u_e]$ 相关的控制点会使线段 $[\mathbf{C}(u_s), \mathbf{C}(u_e)]$ 平坦化，而曲线的其余部分不受影响。这在图 11.26a 和 11.26b 中得到了说明。

最后，我们提醒注意，只有当投影结果至少为 $p+1$ 个连续点时，才能获得平坦的线段，$\widehat{\mathbf{P}}_i$，其中 p 是曲线的度数。如果

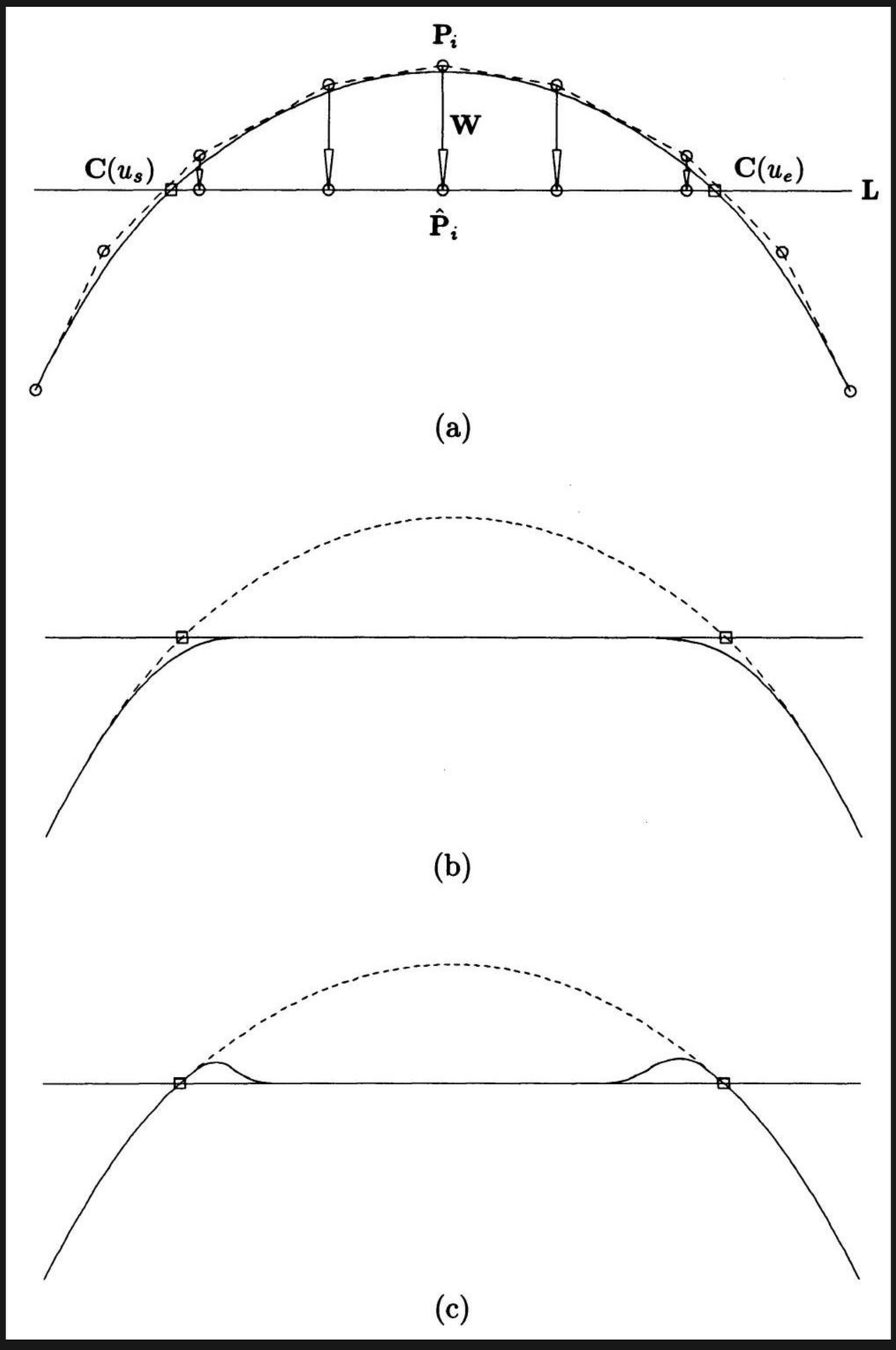


图 11.25. 曲线平坦化。(a) 原始曲线，其跨度为 $[\mathbf{C}(u_s), \mathbf{C}(u_e)]$，将在 $\mathbf{W}$ 的方向上平坦化，直到与曲线相交的线 $\mathbf{L}$；(b) 通过投影位于 $\mathbf{L}$ 一侧的所有控制点获得的平坦曲线；(c) 通过投影与跨度 $[u_s, u_e]$ 相关的控制点获得的平坦曲线。

如果未能满足该条件，则需要更多的节点细化。如果指定了 u_s 和 u_e，则仅在区间 $[u_i, u_j]$ 内需要细化，其中 u_i 是满足 $u_i < u_s$ 的最大节点，而 u_j 是满足 $u_e < u_j$ 的最小节点。

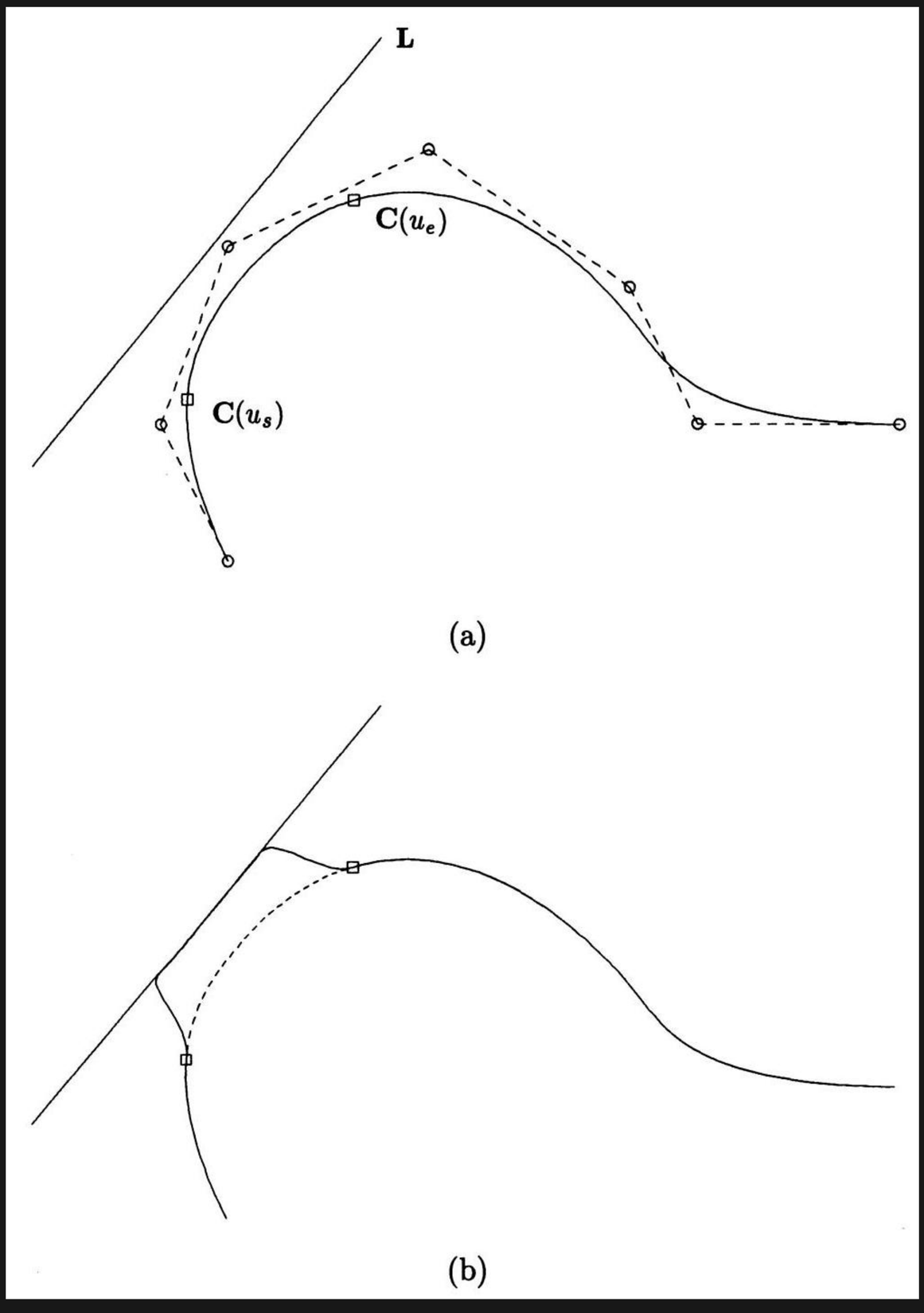


图 11.26. 曲线扁平化。(a) 原始曲线，其跨度为 $[\mathbf{C}(u_s), \mathbf{C}(u_e)]$ ，在与不交叉曲线的直线 $\mathbf{L}$ 垂直的方向上进行扁平化；(b) 通过投影控制点获得的扁平化曲线，控制点局部于跨度 $[u_s, u_e]$ 。

通过将控制点 $\mathbf{P}_{i,j}$ 投影到平面 π 上来扁平化一个表面区域。$\mathbf{P}_{i,j}$ 被它们的投影 $\widehat{\mathbf{P}}_{i,j}$ 替代。除了 π ，还可以指定 (或默认) 以下内容:

- 一个向量 $\mathbf{W}$ 定义投影线，即 $\widehat{\mathbf{P}}_{i,j} = \mathbf{P}_{i,j} + \alpha\mathbf{W}(\alpha$ 可以是负值或正值；该向量可以默认设置为与 π 垂直；
- 一个区域 $\mathcal{R}_{uv}$ ，在 uv 空间中由边界曲线定义，只有当其影响局限于区域 $\mathcal{R}_{uv}$ 时，$\mathbf{C}_{uv}(w) = (u(w), v(w)); \mathbf{P}_{i,j}$ 才会被投影。如果 $[u_\alpha, u_\beta] \times [v_\gamma, v_\delta]$ 是包含 $\mathcal{R}_{uv}$ 的最小最大矩形，则应对区间 $[u_i, u_j]$ 和 $[v_k, v_\ell]$ 应用节点细化，其中 u_i 和 v_k 是满足 $u_i < u_\alpha$ 和 $v_k < v_\gamma$ 的最大节点，而 u_j 和 v_ℓ 是满足 $u_\beta < u_j$ 和 $v_\delta < v_\ell$ 的最小节点。
- 一个区域，$\mathcal{R}_\pi$ ，在平面上 π ，由边界曲线 $\mathbf{C}_\pi(w)$ 定义：控制点 $\mathbf{P}_{i,j}$ 仅在 $\widehat{\mathbf{P}}_{i,j}$ 位于区域 $\mathcal{R}_\pi$ 内时被其投影 $\widehat{\mathbf{P}}_{i,j}$ 替代。

最常见的是，$\mathbf{C}_{uv}(w)$ 和 $\mathbf{C}_\pi(w)$ 是简单曲线，例如多边形或圆，对于这些曲线，确定点是否包含在相应区域内是简单且高效的。如果平面 π 与表面相交，则设计者可能希望指定仅在 π 一侧的那些 $\mathbf{P}_{i,j}$ 被投影，而不是要求一个区域 $\mathcal{R}_{uv}$ 。Cobb [Cobb84] 建议使用额外的平面来限制要投影的控制网区域。最后，投影必须产生至少 $(p+1) \times (q+1)\,\widehat{\mathbf{P}}_{i,j}$ 的连接网格，以便在表面上获得一个平坦区域。图 11.27a 和 11.27b 展示了表面展平的一个例子。

4.3 弯曲

几位作者研究了曲线的圆形弯曲和表面的圆柱形弯曲，例如，参见文献 [Barr83; Four83; Cobb84]。这些作者基于三角学发展了将点映射到圆上的方程。在这些映射下，直线段映射到一个圆上，而一般曲线段则以圆形方式弯曲。Cobb [Cobb84] 将这种技术与节点细化相结合，应用于控制点的映射，从而弯曲 B 样条曲线和表面。

我们采取一种更一般的几何方法来进行弯曲。类似于其他形状操作，弯曲分为三个步骤:

1. 细化要弯曲的曲线或表面区域；
2. 通过重新定位控制点来弯曲区域；
3. 移除不必要的节点。

只有步骤 2 需要详细说明。首先考虑曲线 (见图 11.28)。我们将曲线 $\mathbf{C}(u)$ 在 $\mathbf{C}(u_s)$ 和 $\mathbf{C}(u_e)$ 之间的区域围绕弯曲曲线 $\mathbf{B}(w)$ 进行弯曲，通过将相关控制点 $\mathbf{P}_i$ 映射到 $\mathbf{B}(w)$ 的“方向”。假设 u_s 和 u_e 是节点，仅映射 $\mathbf{P}_s, \ldots, \mathbf{P}_{e-p-1}$ 。将 $\mathbf{P}_i$ 的图像表示为 $\widehat{\mathbf{P}}_i$ 。我们需要一个额外的点 $\mathbf{O}$ ，称为弯曲中心。点 $\mathbf{R}_i$ 是通过将每个线段 $\mathbf{OP}_i$ 与 $\mathbf{B}(w)$ 相交得到的。最后，固定一个交叉比 λ ，每个 $\widehat{\mathbf{P}}_i$ 的位置由 $\lambda, \mathbf{O}, \mathbf{R}_i$ 和 $\mathbf{P}_i$ 确定，如下所示。类似于公式 (11.22)，我们有

$$\mathbf{R}_i = (1 - s_i)\,\mathbf{O} + s_i\mathbf{P}_i$$

$$\widehat{\mathbf{P}}_i = (1 - t_i)\,\mathbf{O} + t_i\mathbf{P}_i \tag{11.56}$$

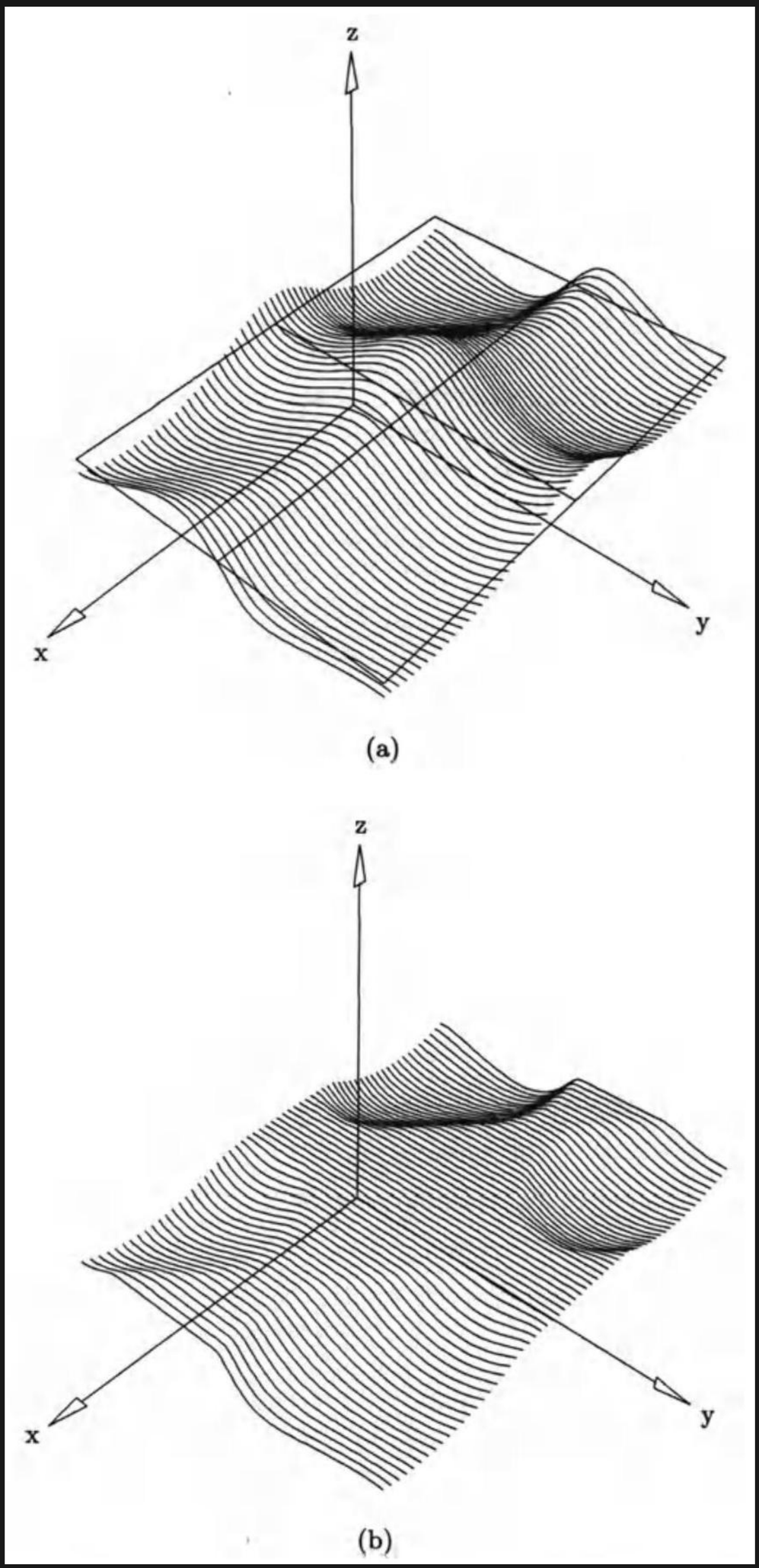


图 11.27. 表面展平。(a) 一个表面和一个平面；(b) 通过投影所有位于平面一侧的控制点获得的展平表面。

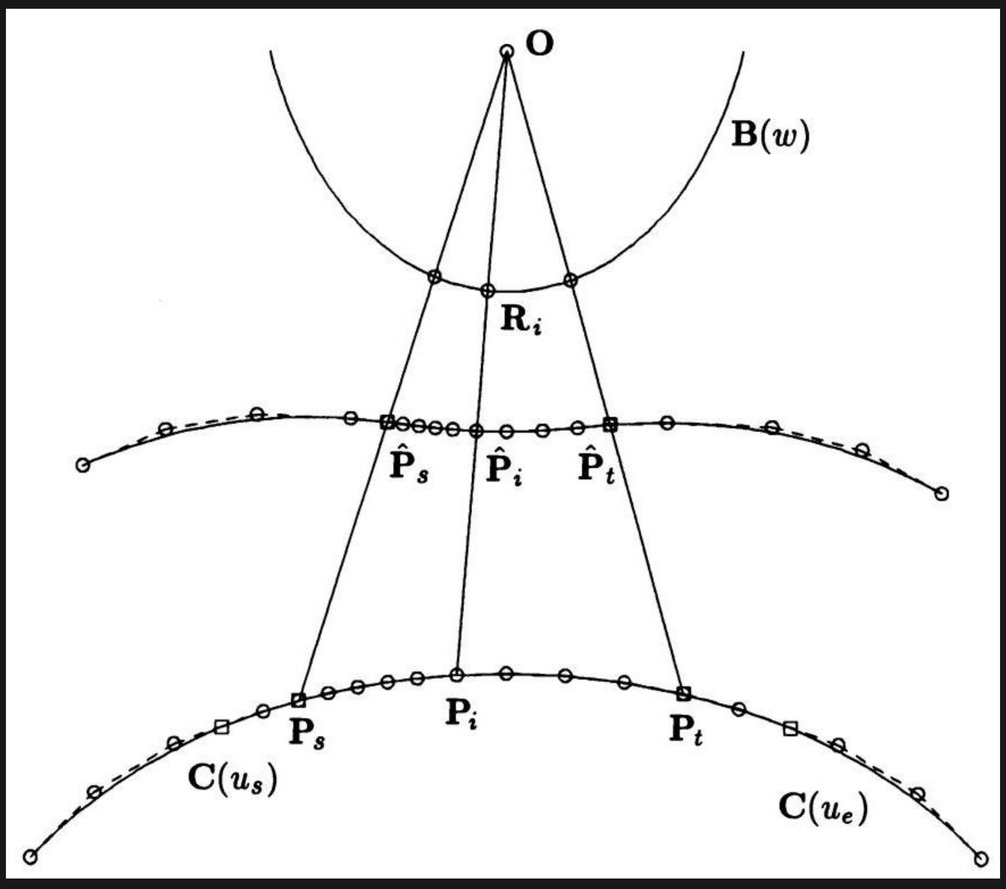


图 11.28. 曲线弯曲。

由此可得

$$s_i = \frac{\mathbf{OR}_i}{\mathbf{OP}_i} \tag{11.57}$$

最后

$$\lambda = \frac{t_i\,(1-s_i)}{s_i\,(1-t_i)}$$

产生

$$t_i = \frac{\lambda s_i}{1+(\lambda-1)\,s_i} \tag{11.58}$$

将 t_i 代入公式 (11.56) 产生 $\widehat{\mathbf{P}}_i$ 。

总结来说，$u_s, u_e, \mathbf{B}(w), \mathbf{O}$ 和 λ 被给出，弯曲算法依次计算: $\mathbf{R}_i$ (线/曲线交点)、s_i, t_i 和 $\widehat{\mathbf{P}}_i$ 。交叉比 λ 可以在 $-\infty$ 到 ∞ 的范围内进行交互式修改。注意 $\lambda = 1$ 意味着 $t_i = s_i$ ，因此 $\widehat{\mathbf{P}}_i = \mathbf{R}_i$ ；即 $\lambda = 1$ 将所有 $\mathbf{P}_i$ 映射到 $\mathbf{B}(w)$ ，因此 $\mathbf{C}(u)$ 在 $\mathbf{C}(u_s)$ 和 $\mathbf{C}(u_e)$ 之间的区域假设了 $\mathbf{B}(w)$ 的 (近似) 形状。对于 $\lambda = 0$ ，所有 $\widehat{\mathbf{P}}_i = \mathbf{O}$ ；以及 $\widehat{\mathbf{P}}_i \to \mathbf{P}_i$ 作为 $\lambda \to \infty$ 。

设 $t = e-p-1$ 。在计算 $\widehat{\mathbf{P}}_s, \ldots, \widehat{\mathbf{P}}_t$ 后，控制点 $\mathbf{P}_0, \ldots, \mathbf{P}_{s-1}$ 和 $\mathbf{P}_{t+1}, \ldots, \mathbf{P}_n$ 必须旋转并平移到位置，以实现 $[u_{\min}, u_s]$ 和 $[u_e, u_{\max}]$ 上未改变的曲线段与 $[u_s, u_e]$ 上的弯曲段之间的自然过渡。设 θ 表示弯曲映射旋转向量 $\mathbf{P}_t - \mathbf{P}_{t-1}$ 的角度。新的 $\widehat{\mathbf{P}}_{t+1}, \ldots, \widehat{\mathbf{P}}_n$ 通过围绕点 $\mathbf{P}_t$ 旋转 $\mathbf{P}_{t+1}, \ldots, \mathbf{P}_n$ 一个角度 θ ，然后通过向量 $\widehat{\mathbf{P}}_t - \mathbf{P}_t$ 进行平移而获得。类似的旋转 (由 $\mathbf{P}_{s+1} - \mathbf{P}_s$) 的旋转定义) 和平移必须应用于 $\mathbf{P}_0, \ldots, \mathbf{P}_{s-1}$ 。

图 11.29-11.31 显示了圆形和抛物线弯曲的示例。在图 11.29 中，变化 λ 的效果得到了说明。弯曲区域关于通过弯曲中心 $\mathbf{O}$ 的线 $\mathbf{L}$ 是对称的，并且与 $\mathbf{C}(u)$ 垂直。注意，当 λ 接近 1 时，弯曲区域逐渐呈现出圆形的形状。图 11.30 显示了将中心 O 放置在圆内的效果。$\mathbf{O}_1$ 是圆心，$\mathbf{O}_2$ 位于线 $\mathbf{L}$ 上，而 $\mathbf{O}_3$ 是一个任意点。图 11.31 说明了使用抛物线弯曲曲线来消除曲线中的两个不必要的拐点。

最后，我们指出，为了实现额外的形状控制，交叉比可以是一个函数 $\lambda(u)$ ，定义在 $u \in [u_s, u_e]$ 上。然后计算 $\mathbf{P}_i, \lambda_i = \lambda(t_i)$ ，其中 t_i 是第 i 个节点 (与方程 [11.58] 的 t_i 无关)。

现在我们转向表面弯曲。曲线弯曲技术向表面的自然扩展产生了非常一般和强大的表面修改能力。我们从表面弯曲的一般定义开始，但随后将详细讨论限制在一个简单但有用的特例上。

一个表面 $\mathbf{S}(u,v)$ 围绕一个弯曲表面 $\mathbf{B}(s,t)$ 弯曲，朝向一个中心点 $\mathbf{O}$ 或一个中心曲线 $\mathbf{O}(w)$ 。设弯曲区域 $\mathcal{R}$ 在 uv 空间中被最小最大矩形 $\mathcal{M} = [u_\alpha, u_\beta] \times [v_\gamma, v_\delta]$ 所限制。在对控制点的子集 $\mathbf{P}_{i,j}, i = \alpha, \ldots, \beta-p-1, j = \gamma, \ldots, \delta-q-1$ 进行节点细化后，其对 $\mathcal{R}$ 的影响是局部的，映射到新位置 $\widehat{\mathbf{P}}_{i,j}$ ，使用交叉比 λ ；也可以使用一个函数 $\lambda(u,v)$ 。如果使用一个点 $\mathbf{O}$ ，则通过线段 $\mathbf{OP}_{i,j}$ 与表面 $\mathbf{B}(s,t)$ 的交点获得点 $\mathbf{R}_{i,j}$ 。如果给定一条曲线 $\mathbf{O}(w)$ ，则必须计算参数 $w_{i,j}$ ，并通过将 $\mathbf{O}(w_{i,j})\,\mathbf{P}_{i,j}$ 与 $\mathbf{B}(s,t)$ 相交来获得 $\mathbf{R}_{i,j}$ 。然后计算 $\widehat{\mathbf{P}}_{i,j}$ 。

$$s_{i,j} = \frac{\mathbf{OR}_{i,j}}{\mathbf{OP}_{i,j}} \quad \text{or} \quad s_{i,j} = \frac{\mathbf{O}(w_{i,j})\,\mathbf{R}_{i,j}}{\mathbf{O}(w_{i,j})\,\mathbf{P}_{i,j}} \tag{11.59}$$

$$t_{i,j} = \frac{\lambda s_{i,j}}{1+(\lambda-1)\,s_{i,j}} \tag{11.60}$$

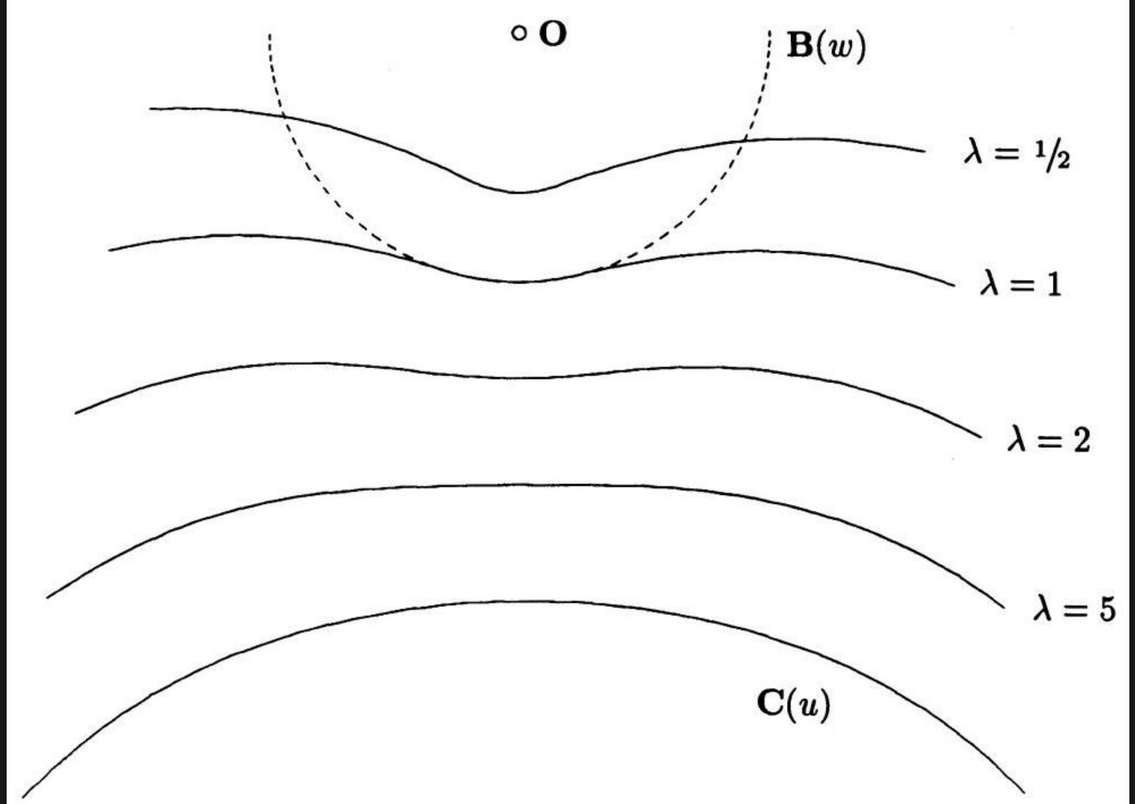


图 11.29。使用圆弧作为弯曲曲线的曲线弯曲示例 $\mathbf{B}(w)$ ；交叉比为 $\lambda = \{1/2, 1, 2, 5\}$ 。

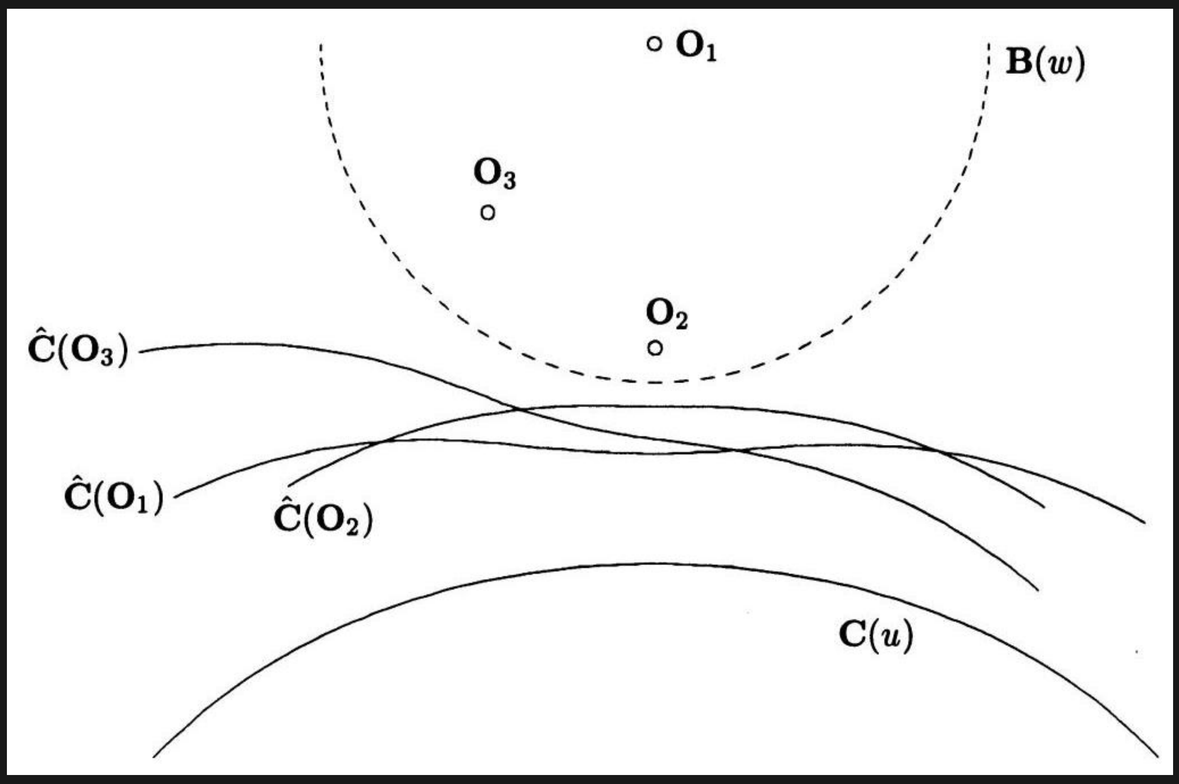


图 11.30。通过使用不同的弯曲中心位置获得的曲线弯曲示例。

最后

$$\widehat{\mathbf{P}}_{i,j} = (1 - t_{i,j})\,\mathbf{O} + t_{i,j}\mathbf{P}_{i,j} \text{ or } \widehat{\mathbf{P}}_{i,j} = (1 - t_{i,j})\,\mathbf{O}\,(w_{i,j}) + t_{i,j}\mathbf{P}_{i,j} \tag{11.61}$$

随后，未映射的控制点必须旋转和平移到确保弯曲区域平滑自然过渡到未弯曲区域的位置。节点移除作为最后一步应用。例如，通过让 $\mathbf{B}(s,t)$ 为一个球体，中心为 $\mathbf{O}$，可以获得一个球形弯曲。如果 $\mathbf{B}(s,t)$ 是一个圆柱体，而 $\mathbf{O}(w)$ 是其轴，则会产生一个圆柱形弯曲。

在其完全一般性下，表面弯曲显然是复杂的。$\mathbf{R}_{i,j}$ 的计算需要线/面交点，获得合适的参数 $w_{i,j}$ 可能很困难，并且如何旋转和移动未映射的控制点并不明显。我们在此详细说明关于表面等值线全长的通用圆柱形弯曲的特殊情况；即，弯曲区域具有 $[u_\alpha, u_\beta] \times [v_{\min}, v_{\max}]$ 或 $[u_{\min}, u_{\max}] \times [v_\gamma, v_\delta]$ 的形式，而 $\mathbf{B}(s,t)$ 是通过沿着（可能是弯曲的）轴线扫过一个圆而获得的通用圆柱体。

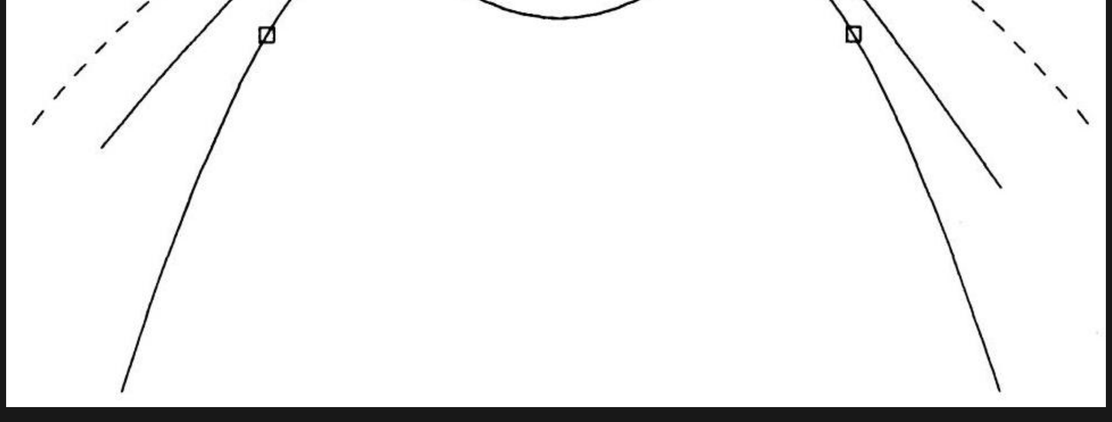


图 11.31. 用于消除拐点的抛物线弯曲。

$\mathbf{O}(w)$。我们展示了在固定 $u = u_c$ 下关于等值线的弯曲；固定 $v = v_c$ 的情况是类似的。假设 u_c, u_α 和 u_β 满足 $u_\alpha < u_c < u_\beta$。令 $\mathbf{C}_{u_c}(v)$ 表示在 $\mathbf{S}(u,v)$ 上的等值线，位于 $u = u_c$（见图 11.32）。轴 $\mathbf{O}(w)$ 可以通过两种方式定义。它可以被定义为沿着表面法线的 $\mathbf{C}_{u_c}(v)$ 的偏移，即。

$$\mathbf{O}(v) = \mathbf{C}_{u_c}(v) + d\mathbf{N}(u_c, v) \tag{11.62}$$

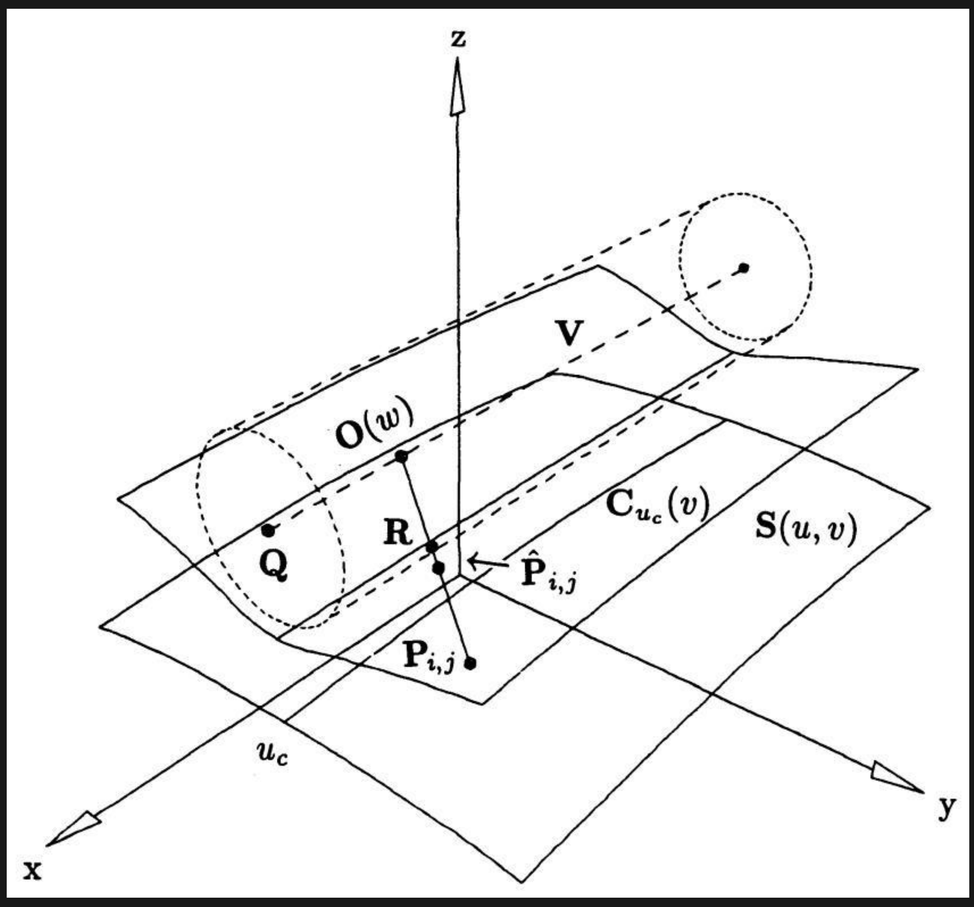


图 11.32. 使用圆柱体的表面弯曲。

在这里，$\mathbf{N}(u_c, v)$ 是位于 (u_c, v) 的单位长度表面法向量，d 是一个常量偏移距离。我们注意到，通常情况下，$\mathbf{O}(v)$ 的形状可能与 $\mathbf{C}_{u_c}(v)$ 的形状截然不同，并且 $\mathbf{O}(v)$ 不能精确表示为 NURBS 曲线 (例如，见 [Till84; Coqu87b; Hosc88])。然而，我们在这里假设 $\mathbf{O}(v)$ 行为良好，并且在形状上与 $\mathbf{C}_{u_c}(v)$ 相似；对于表面弯曲，这是一个合理的假设。此外，我们只需要计算 $\mathbf{O}(v)$ 上的选定点 (使用公式 [11.62])；我们不需要 $\mathbf{O}(v)$ 的显式 NURBS 表示。最后，我们注意到在这种情况下，参数 w 对应于 v 。

定义 $\mathbf{O}(w)$ 的第二种方法是让它成为位于 $\mathbf{C}_{u_c}(v)$ 定义的平面中的一条直线。在这种情况下，w 并不对应于 v 。通过计算一个最小二乘平面 (由 $\mathbf{C}_{u_c}(v)$ 的控制点定义)，该方法甚至可以用于非平面 $\mathbf{C}_{u_c}(v)$ 。

通过对 $m+1$ 控制点的行应用曲线弯曲方法来弯曲 $\mathbf{S}(u,v)$ ；即，对于 $j=0,\ldots,m$ ，执行以下操作:

1. 对于 $i=s,\ldots,e-p-1$:

1a. 计算 $w_{i,j}$ ，使得 $\mathbf{O}(w_{i,j})$ 是 $\mathbf{O}(w)$ 上离 $\mathbf{P}_{i,j}$ 最近的点；如果

$\mathbf{O}(w)=\mathbf{Q}+w\mathbf{V}$ 是一条直线，则 $w_{i,j}$ 由公式给出

$$w_{i,j}=\frac{\mathbf{V}\cdot(\mathbf{P}_{i,j}-\mathbf{Q})}{|\mathbf{V}|^2} \tag{11.63}$$

如果 $\mathbf{O}(w)$ 是 $\mathbf{C}_{u_c}(v)$ 的偏移量，则可以使用第 (6.1) 节的点到曲线投影技术；第 j 个 v 节点 t_j 是牛顿迭代的良好起始点；

1b. 然后设置

$$\mathbf{R}_{i,j}=\mathbf{O}(w_{i,j})+r\frac{\mathbf{P}_{i,j}-\mathbf{O}(w_{i,j})}{|\mathbf{P}_{i,j}-\mathbf{O}(w_{i,j})|} \tag{11.64}$$

其中 r 是一般圆柱体 $\mathbf{B}(s,t)$ 的半径；

1c. 使用公式 (11.59)-(11.61) 计算 $\hat{\mathbf{P}}_{i,j}$ ；

2. 通过类似于曲线弯曲的过程，将 $\mathbf{P}_{0,j},\ldots,\mathbf{P}_{s-1,j}$ 和 $\mathbf{P}_{e-p,j},\ldots,\mathbf{P}_{n,j}$ 旋转并平移到新位置。

图 11.33 和 11.34 显示了圆柱面弯曲的示例。在图 11.33 中，平面表面使用 $\lambda=\{1/2,3/2,-3\}$ 进行弯曲。注意，正的 λ 会将表面弯曲向圆柱内，而负的则会将其弯曲远离圆柱。图 11.34 显示了使用沿等参数曲线 $\mathbf{C}_{u_c}(v)$ 接触表面的圆柱对同一平面表面的圆柱弯曲。同样，λ 的符号允许表面向两个方向弯曲。重要的是要注意，尽管表面是平面的，但它的度数不是 (1×1) ；图 11.34 中的表面具有度数 (3×2) 。一般来说，表面必须至少具有度数 (2×2) ，以避免在控制点重新定位后出现不连续性。554 形状修改工具

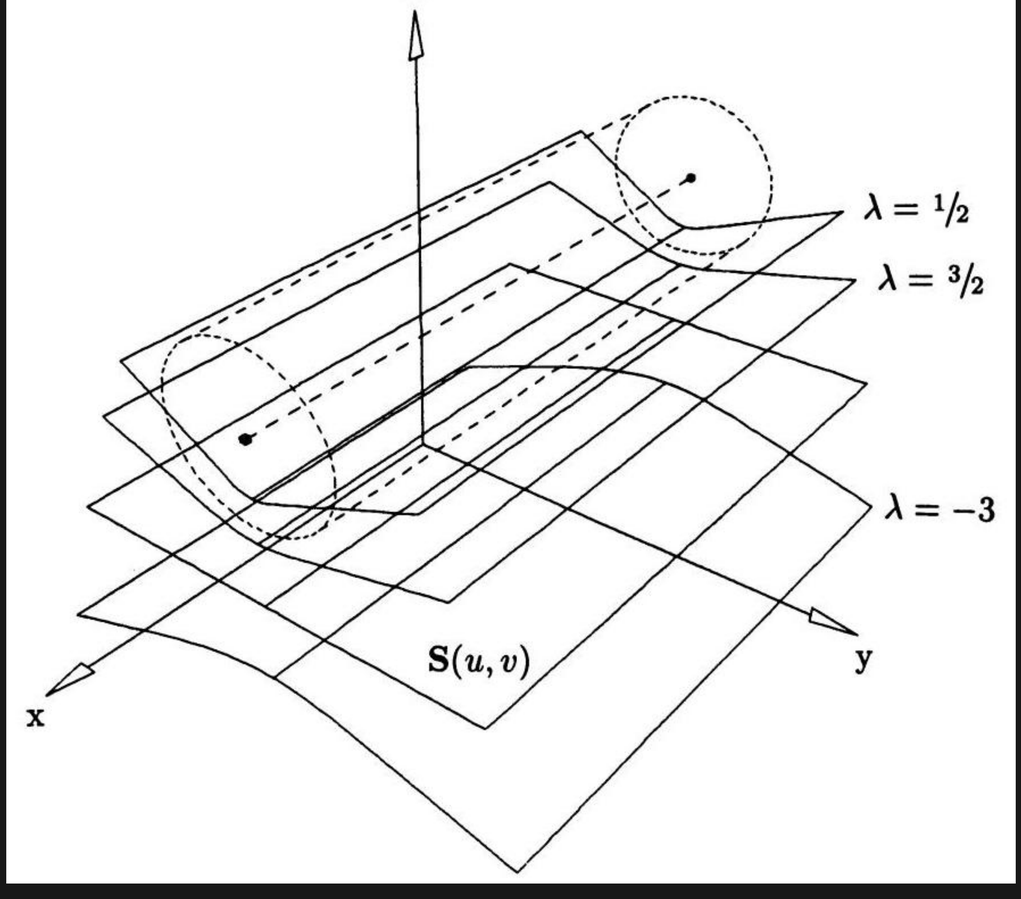


图 11.33. 使用不接触圆柱的表面弯曲示例，$\lambda=\{1/2,3/2,-3\}$ 。

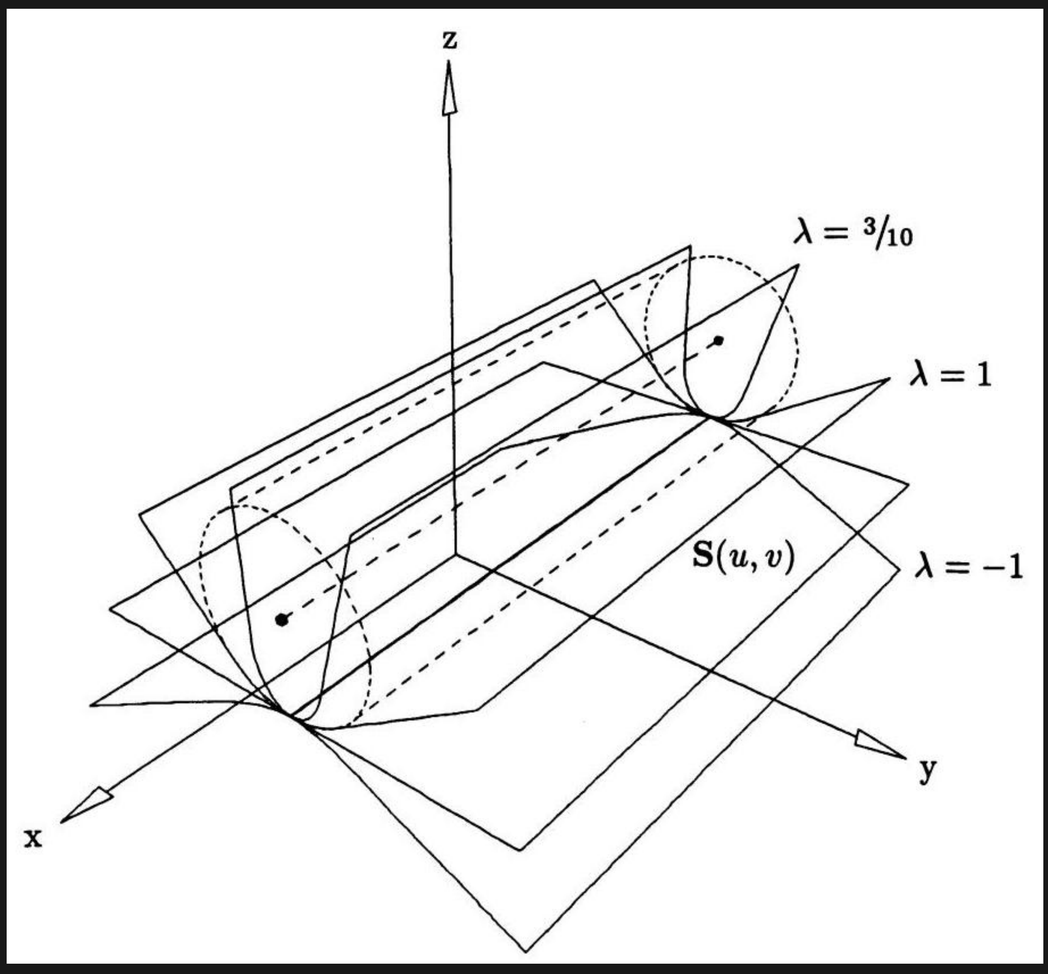


图 11.34. 使用接触圆柱的表面弯曲示例，$\lambda = \{3/10, 1, -1\}$ 。

5 基于约束的曲线和表面造型

接下来，我们提出了一种强制曲线或曲面在选定参数值处假定指定导数的方法。该方法允许任意阶数的导数，包括零阶导数，因此也包括点约束。通过约束一阶和二阶导数，可以控制曲线/曲面的切线和曲率。这些约束通过以最小化所有控制点的组合移动的方式重新定位控制点来满足。通过两步过程解决完全或欠定的线性方程组。这个两步解决方案允许基于约束的修改以交互速度实现。关于这个主题的更多材料可以在 [Fowl92, 93] 中找到。

5.1 基于约束的曲线修改

设 $\mathbf{C}(u) = \sum_{i=0}^{n} R_{i,p}(u)\,\mathbf{P}_i$ 为一条 NURBS 曲线。假设我们给定参数值 $\{u_r\}, r = 1, \ldots, R$ 和一组导数约束，$\left\{\Delta\mathbf{D}_r^{(k)}\right\}$.$\Delta\mathbf{D}_r^{(k)}$ 表示在 $u = u_r$. k 处 k 阶导数的变化可以取任何非负值，但通常为 $k = 0, 1, 2$ ，因为这些值控制位置、切线和曲率的直观几何属性。注意，我们以导数变化的形式指定约束，而不是以目标值的形式。

现在固定 k 和 r ，并用 $\mathbf{D}_r^{(k)}$ 表示在 $u = u_r$ 处 $\mathbf{C}(u)$ 的 k 阶导数。然后

$$\mathbf{D}_r^{(k)} = \sum_{i=0}^{n} R_{i,p}^{(k)}(u_r)\,\mathbf{P}_i \tag{11.65}$$

其中 $R_{i,p}^{(k)}$ 表示基函数的 k 阶导数。如果曲线是非理性的，则使用算法 A2.3 计算基函数的导数。否则，使用在第 4.3 节中获得理性曲线导数所使用的相同技术计算 $R_{i,p}^{(k)}$ ，即

$$R_{i,p}(u) = \frac{w(u)\,R_{i,p}(u)}{w(u)} = \frac{A_i(u)}{w(u)}$$

其中

$$w(u) = \sum_{j=0}^{n} w_j N_{j,p}(u)$$

使用莱布尼茨法则对 $A_i(u)$ 进行微分得到

$$R_{i,p}^{(k)}(u) = \frac{A_i^{(k)}(u) - \sum_{j=1}^{k} \binom{k}{j} w^{(j)}(u)\,R_{i,p}^{(k-j)}(u)}{w(u)} \tag{11.66}$$

其中

$$A_i^{(k)}(u) = w_i N_{i,p}^{(k)}(u)$$

在 $u = u_r$ 处应用约束 $\Delta\mathbf{D}_r^{(k)}$ ，我们得到新的 k 阶导数

$$\sum_{i=0}^{n} R_{i,p}^{(k)}(u_r)\,(\mathbf{P}_i + \Delta\mathbf{P}_i) = \mathbf{D}_r^{(k)} + \Delta\mathbf{D}_r^{(k)} \tag{11.67}$$

因此

$$\sum_{i=0}^{n} R_{i,p}^{(k)}(u_r)\,\Delta\mathbf{P}_i = \Delta\mathbf{D}_r^{(k)} \tag{11.68}$$

方程 (11.68) 是关于 $n+1$ 个未知数 $\{\Delta\mathbf{P}_i\}$ 的一个线性方程 (实际上是 $p+1$ 个未知数，因为 B 样条的局部支持性质)。我们注意到 $\Delta\mathbf{D}_r^{(k)}$ 可以是零向量，实际上这就是指定某个点或导数不发生变化的方式。如果 M 是约束的总数，$M \geq R$ ，那么将方程 (11.68) 应用到每个约束上会产生一个关于 $n+1$ 个未知数的 M 个线性方程的系统，即

$$B\left[\Delta\mathbf{P}_i\right] = \left[\Delta\mathbf{D}_r^{(k)}\right] \tag{11.69}$$

在我们展示如何求解方程 (11.69) 之前，以下几点说明是重要的:

- 方程 (11.69) 假设约束有某种顺序；顺序可以是任意的，但一种自然的方法是先按 r 排序，然后按 k 排序。也就是说，假设 $\Delta\mathbf{D}_r^{(k)}$ 的顺序为 $\Delta\mathbf{D}_1, \ldots, \Delta\mathbf{D}_M$ ，并写成 $\Delta\mathbf{D}_i = \Delta\mathbf{D}_{r_i}^{(k_i)}$ 那么 $i < j$ 意味着 $r_i \leq r_j$ ，并且，或者 $r_i < r_j$ 或 $k_i < k_j$ ；
- 由于 B 样条的局部支持性质以及约束通常仅在少数参数位置应用的事实，许多 B 的列可能为零；也就是说，未知数 $\Delta\mathbf{P}_i$ 的数量少于 $n+1$ 。一个高效的算法应该建立方程 (11.69) 的系统，仅包含那些发生变化的 $\Delta\mathbf{P}_i$ ；用 N 表示实际的未知数 $\Delta\mathbf{P}_i$ 的数量；
- 我们需要对方程 (11.69) 的精确解；然而，系统可能是欠定、完全确定或超定的，并且它可能没有精确解。对于后面提出的解法，我们需要一个欠定 ($M < N$) 或完全确定 ($M = N$) 的系统，其中 B 的所有行都是线性独立的。当指定的约束条件超过受影响控制点的度 p 或数量 N 时，就会出现超定系统 ($M > N$) 。显然，每个节点区间上不能指定超过 $p+1$ 的约束条件。如有必要，可以使用节点插入和/或度提升来确保一组约束产生一个欠定或完全确定的系统。如果 $M \leq N$ ，那么 B 的行的线性依赖通常不是问题 - 参见 [Scho66; Good81; Fowl93] 获取数学细节 - 但一个稳健的算法应该在解过程的第一步检查这一点；
- 通常设计者不需要指定施加约束的参数 $\{u_r\}$ ；理论上这些也可以是未知数，但这会导致非线性方程。因此，像往常一样，系统应该根据设计者选择的曲线点来选择参数。考虑到这些说明，我们现在将方程 (11.69) 写为

$$B\left[\Delta\mathbf{P}_{\ell_i}\right] = \left[\Delta\mathbf{D}_j\right]\ \ell = 1, \ldots, N\ j = 1, \ldots, M \tag{11.70}$$

或者为了简洁

$$B\Delta\mathbf{P} = \Delta\mathbf{D} \tag{11.71}$$

目前，假设 $M < N$ 。方程 (11.70) 存在无穷多解。我们希望找到最小长度解，即最小化所有控制点的组合移动的解 (最小化 N 维向量 $\Delta\mathbf{P}$ 的长度)。根据线性代数，我们可以将 N 向量 $\Delta\mathbf{P}$ 表示为两个相互正交的分量，一个在 B 的行空间中，另一个在 B 的零空间中，即

$$\Delta\mathbf{P} = B^T\lambda + \mathbf{z} \tag{11.72}$$

其中 λ 是某个 M 向量 $B\mathbf{z} = 0$ 和 $\left(\lambda^T B\right) \cdot \mathbf{z} = 0$ 。因此可以得出

$$|\Delta\mathbf{P}|^2 = \left(B^T\lambda + \mathbf{z}\right)^T \cdot \left(B^T\lambda + \mathbf{z}\right)$$

$$= \left(B^T\lambda\right)^T \cdot \left(B^T\lambda\right) + \mathbf{z}^T \cdot \mathbf{z}$$

$$= \left|B^T\lambda\right|^2 + |\mathbf{z}|^2 \tag{11.73}$$

从方程 (11.71) 和 (11.72) 中我们得到

$$\Delta\mathbf{D} = B\Delta\mathbf{P}$$

$$= B\left(B^T\lambda + \mathbf{z}\right)$$

$$= BB^T\lambda + B\mathbf{z}$$

$$= BB^T\lambda \tag{11.74}$$

因此

$$\lambda = \left(BB^T\right)^{-1}\Delta\mathbf{D} \tag{11.75}$$

由于 $\mathbf{z}$ 的分量与系统无关，但它们对 $\Delta\mathbf{P}$ 的长度有贡献 (根据方程 [11.73])，我们通过设定 $\mathbf{z} = 0$ 来获得方程 (11.70) 的最小长度解。将方程 (11.75) 代入 (11.72) 得到

$$\Delta\mathbf{P} = A\Delta\mathbf{D} \text{ where } A = B^T\left(BB^T\right)^{-1} \tag{11.76}$$

如果 BB^T 的行 B 线性无关，则矩阵 BB^T 是非奇异的。最后，如果 $M = N$ ，则方程 (11.70) 有唯一解

$$\Delta\mathbf{P} = A\Delta\mathbf{D} \text{ where } A = B^{-1} \tag{11.77}$$

我们指出，也可以 (并且非常有用) 限制某些控制点不发生变化。这在数学上简单地通过将 B 的相应列设置为零来实现。在算法上，这意味着相应的 $\Delta\mathbf{P}_{i_\ell}$ 不出现在方程 (11.70) 中，并且 N 减少了 1。通常，一个约束影响 $p+1$ 个控制点，并且在任何跨度中可以指定最多 $p+1$ 个约束。如果一个控制点被限制不移动，那么这将减少 p 在任何受此控制点影响的跨度中可以指定的约束数量。我们在下面给出这些例子的说明。

方程 (11.76)(或方程 [11.77]) 表示解决过程中的第一步。在进行第二步之前，我们总结了将我们带到这一点的事件序列:

1. 设计师在曲线上选择点，产生由系统确定的参数 $\{u_\ell\}$ ，在这些参数上施加约束；他同时指明在每个参数上施加何种类型的约束，但在此阶段不需要实际指定约束；
2. 系统建立方程 (11.70)；如果它是超定的，系统要么通知设计师以便他可以修改选择，要么通过节点插入和/或提升次数的方式添加控制点，使得结果方程组是欠定或完全确定的；
3. 然后通过方程 (11.76) 或 (11.77) 计算矩阵 A 。

设计师现在可以交互式地指定约束 $[\Delta\mathbf{D}_j]$ 。每次约束发生变化时，矩阵 A 必须与新的向量 $[\Delta\mathbf{D}_j]$ 相乘，以产生控制点位置的变化 $[\Delta\mathbf{P}_{i_\ell}]$ 。相关的控制点随后被更新为

$$\widehat{\mathbf{P}}_{i_\ell} = \mathbf{P}_{i_\ell} + \Delta\mathbf{P}_{i_\ell}\ \ell = 1, \ldots, N \tag{11.78}$$

示例

Ex11.1 图 11.35a 显示了由 $\mathbf{P}_0, \ldots, \mathbf{P}_7$ 定义的立方曲线，以及 $U = \{0, 0, 0, 0, 1/5, 2/5, 3/5, 4/5, 1, 1, 1, 1\}$ 。我们在 $u = 1/2$ 应用一个点约束

$$\Delta\mathbf{D}_1 = \Delta\mathbf{D}_1^{(0)} = (0.04, -0.08, 0)$$

然后

$$B\Delta\mathbf{P} = [\Delta\mathbf{D}_1]$$

其中 B 是 1×4 矩阵

$$B = \left[R_{2,3}\left(\frac{1}{2}\right), R_{3,3}\left(\frac{1}{2}\right), R_{4,3}\left(\frac{1}{2}\right), R_{5,3}\left(\frac{1}{2}\right)\right]$$

而 $\Delta\mathbf{P}$ 是具有以下分量的 4×1 列向量

$$\Delta\mathbf{P}_2, \Delta\mathbf{P}_3, \Delta\mathbf{P}_4, \Delta\mathbf{P}_5$$

因此，BB^T 是一个 1×1 矩阵，而 $A = B^T\left(BB^T\right)^{-1}$ 是一个 4×1 矩阵

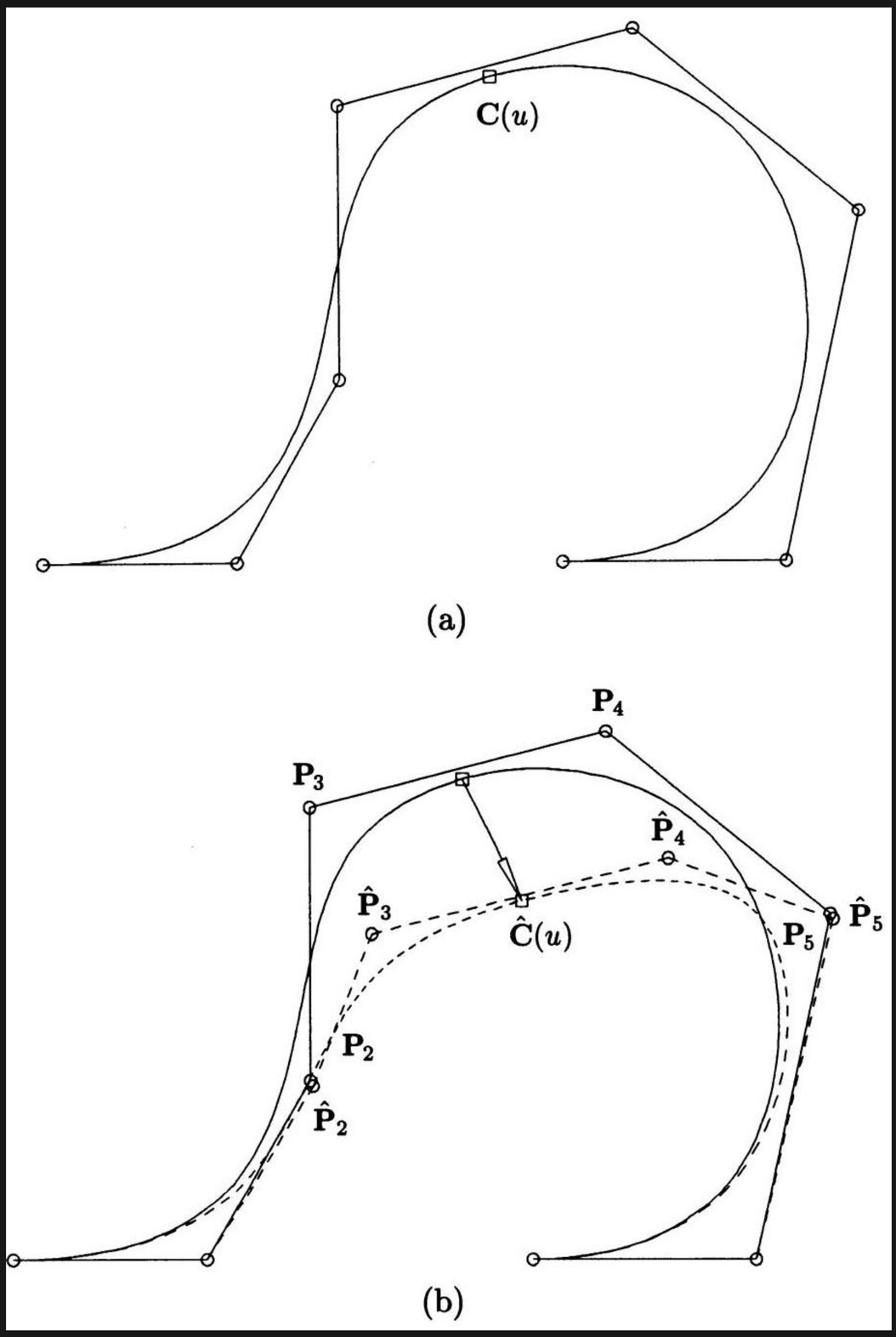


图 11.35. 基于约束的曲线修改使用位置约束。(a) 原始立方曲线及要移动的点；(b) 所有四个局部控制点均可移动；(c) 仅允许 $\mathbf{P}_3$ 和 $\mathbf{P}_4$ 移动；(d) 仅允许 $\mathbf{P}_3$ 移动。

具有以下分量

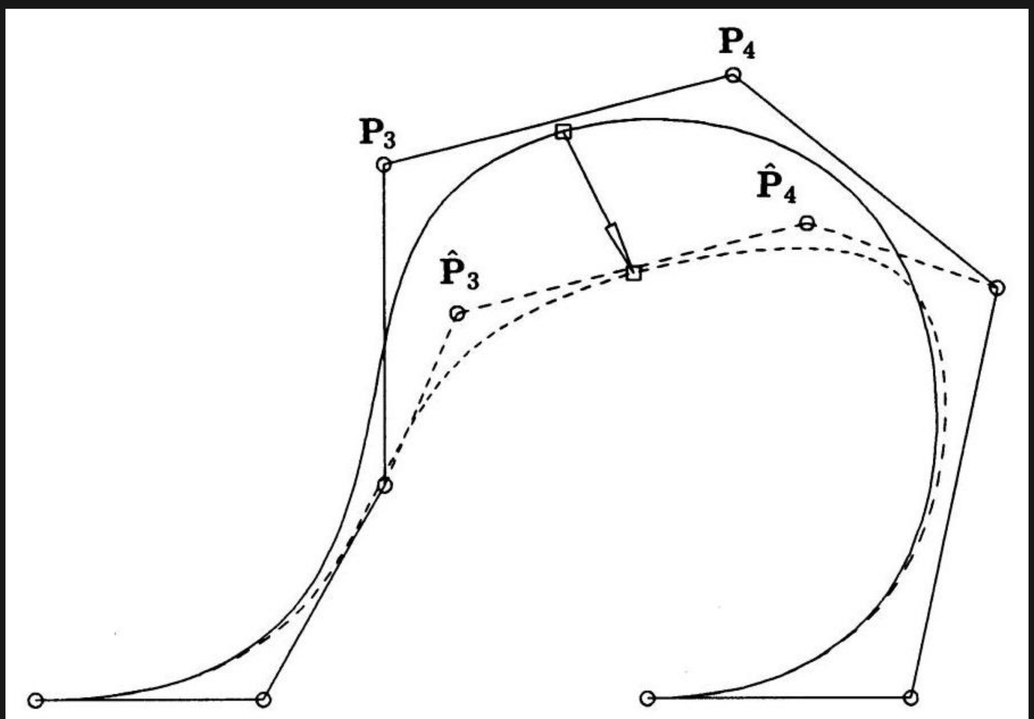
$$a_0, a_1, a_2, a_3$$

因此

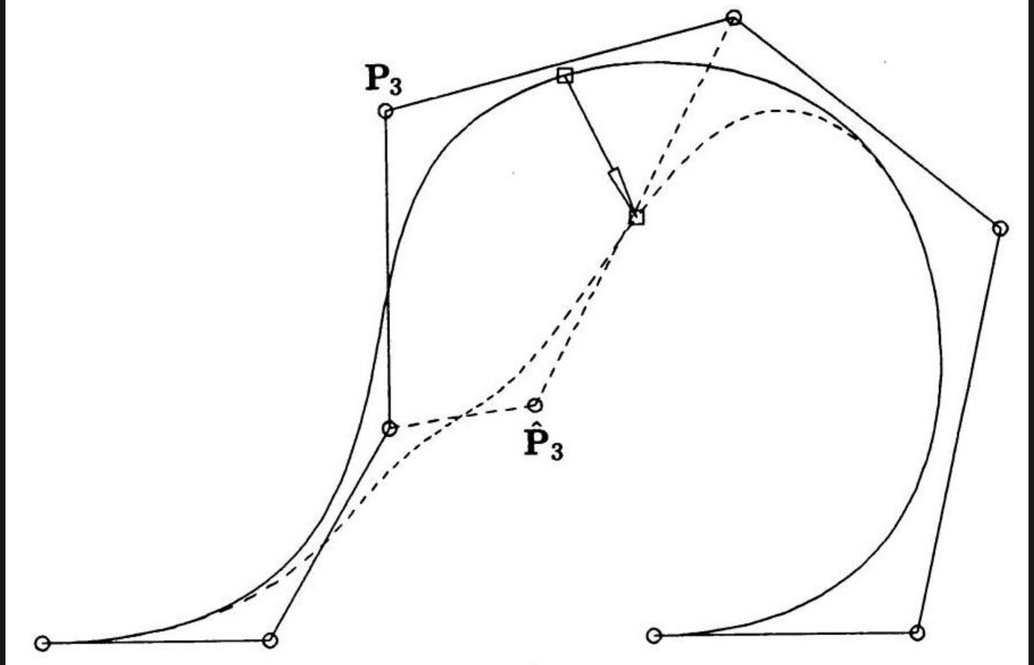
$$\Delta\mathbf{P}_i = a_{i-2}\Delta\mathbf{D}_1 \; i = 2, 3, 4, 5$$

应用 $\Delta\mathbf{P}_i$ 的结果如图 11.35b 所示。注意到 $\mathbf{P}_3$ 和 $\mathbf{P}_4$ 移动得最多，因为这些控制点具有最大的

图 11.35. (续)



(c)



(d)

35 (*Continued*)

在 $u = 1/2$ 处对曲线的影响。我们可以限制 $\mathbf{P}_2$ 和 $\mathbf{P}_5$ 的移动。然后 B 是 1×2 矩阵

$$B = \left[R_{3,3}\left(\frac{1}{2}\right), R_{4,3}\left(\frac{1}{2}\right) \right]$$

并且 $\Delta\mathbf{P}$ 的分量是 $\Delta\mathbf{P}_3$ ，而 $\Delta\mathbf{P}_4.A$ 的分量是 a_0 和 a_1 (与上面的 a_0, a_1 不同)，并且

$$\Delta\mathbf{P}_i = a_{i-3}\Delta\mathbf{D}_1 \; i = 3, 4$$

图 11.35c 显示了仅允许 $\mathbf{P}_3$ 和 $\mathbf{P}_4$ 变化的结果，而图 11.35d 则说明了仅允许 $\mathbf{P}_3$ 移动的情况。

这个例子清楚地表明，对于给定的一组点和导数约束，通常在指定多少个以及哪些控制点应该变化时存在灵活性，而这个选择会影响相应曲线变化的范围和外观 (例如，对称性)。然而，对于设计师而言，处理控制点可能不如对曲线施加纯点和导数约束直观。一个良好的用户界面应该同时提供一组合理的默认值和方便的工具，以便更复杂的设计师限制控制点的移动。最后，请注意，任何控制点的限制必须纳入方程 (11.70) 和解决过程的第一阶段。

在图 11.36a-11.36c 中，展示了点和导数约束。图 11.36a 仅显示了导数约束的应用，同时允许 $\mathbf{P}_3, \mathbf{P}_4$ 和 $\mathbf{P}_5$ 移动。在 $u = 0.556$ 处，第一导数向量的大小增加，导致局部张力效应。如果仅允许一个控制点移动，如图 11.36b 所示，观察结果非常有趣。曲线采用了新的导数，但由于缺乏位置约束，它遭受了位置位移。在图 11.36c 中，通过约束曲线在 $u = 0.556$ 处不移动，并假设通过减小旧导数的大小获得的新导数，解决了这个问题。允许两个控制点移动，以获得一个不被过度确定的方程组。

最后，我们提到 Fowler 和 Bartels [Fowl93] 提供了一种不同 (但等效) 的方法来表示方程 (11.76) 和 (11.77) 中的矩阵 A 。这种方法在数学上稍微复杂一些，但在某些情况下更有效。

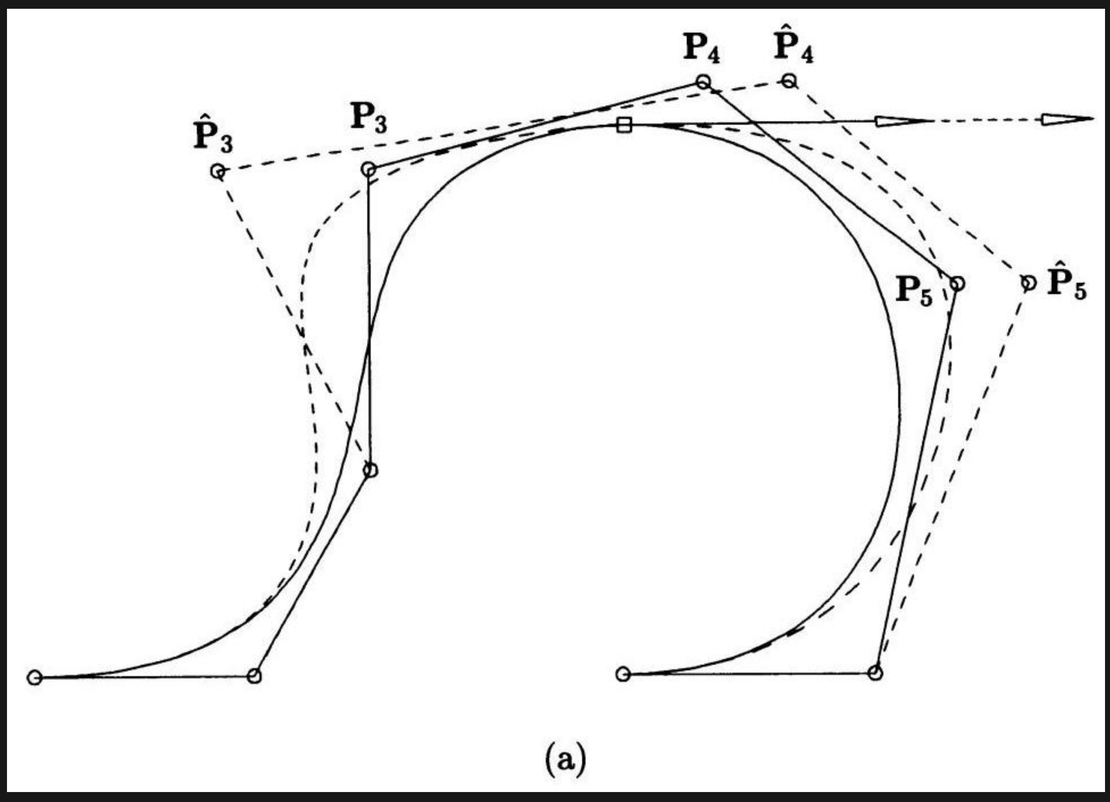


图 11.36. 基于约束的曲线修改，使用位置和导数约束。(a) 应用导数约束，并允许 $\mathbf{P}_3, \mathbf{P}_4$ 和 $\mathbf{P}_5$ 移动；(b) 应用导数约束，仅允许 $\mathbf{P}_4$ 移动；(c) 应用导数和位置约束，并允许 $\mathbf{P}_3$ 和 $\mathbf{P}_4$ 移动。

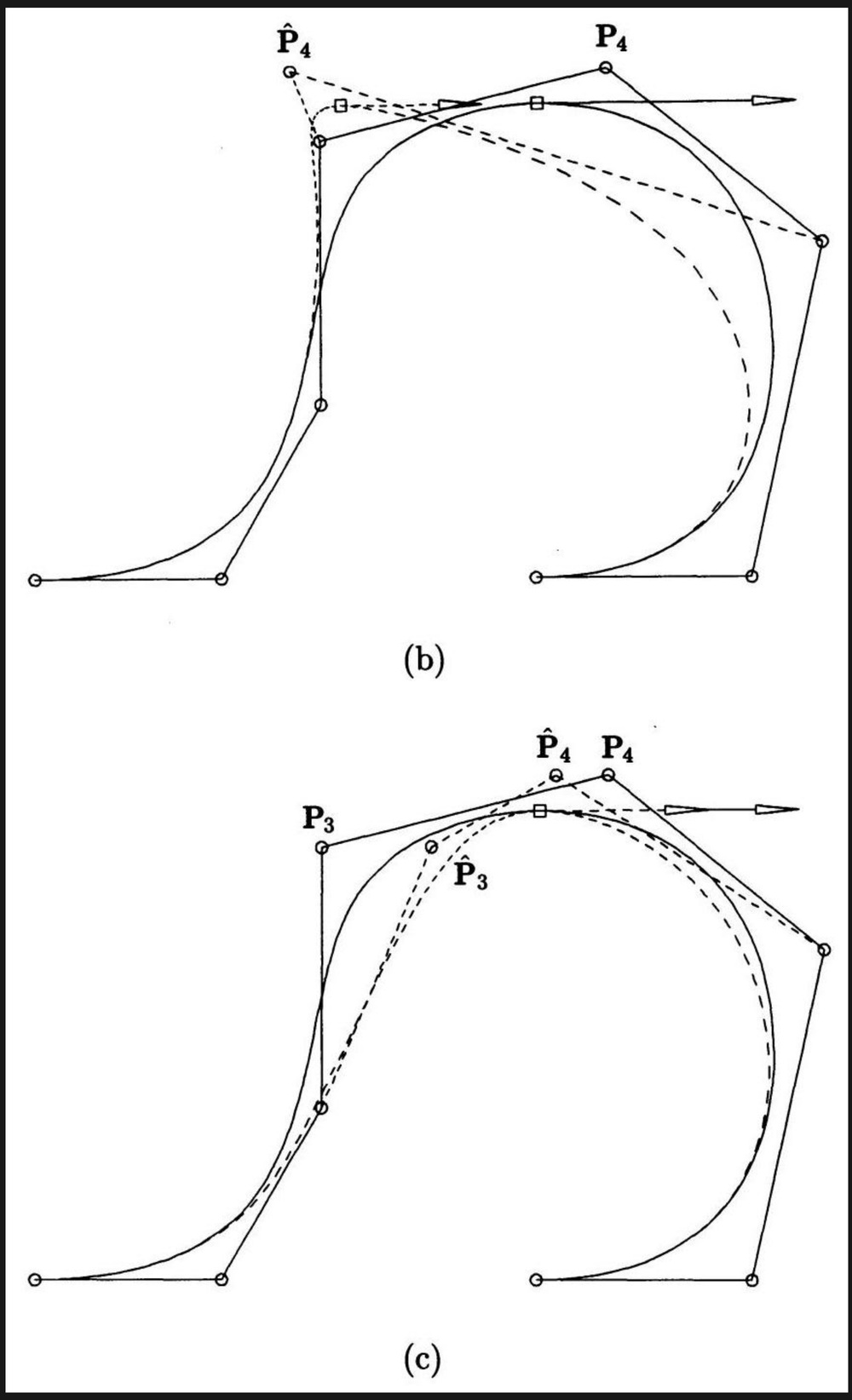


图 11.36. (续)

5.2 基于约束的表面修改

设 $\mathbf{S}(u,v)=\sum_{i=0}^{n}\sum_{j=0}^{m}R_{i,p;j,q}(u,v)\,\mathbf{P}_{i,j}$ 为一个 (p,q) 次 NURBS 曲面。在本节中，我们选择将 $\mathbf{S}(u,v)$ 写成以下形式

$$\mathbf{S}(u,v)=\sum_{s=0}^{(n+1)(m+1)-1}R_s(u,v)\,\mathbf{P}_s \tag{11.79}$$

其中

$$s=j(n+1)+i \tag{11.80}$$

我们称 s 为对应于 (i, j) 的绝对地址。注意到

$$\mathbf{P}_s=\mathbf{P}_{i,j}\;R_s(u,v)=R_{i,p;j,q}(u,v) \tag{11.81}$$

其中

$$i=s \bmod n+1\;j=\frac{s}{n+1} \tag{11.82}$$

如果 $\mathbf{S}(u,v)$ 是非有理的，则 $R_s(u,v)=N_{i,p}(u)\,N_{j,q}(v)$，其中 i 和 j 由公式 (11.82) 定义。现在假设我们给定了一组点和偏导数约束 $\left\{\Delta\mathbf{D}_r^{(k,\ell)}\right\},k,\ell\geq 0$，这些约束在参数位置 $\{(u,v)_r\},r=1,\ldots,R$ 处指定。那么对于每个 r 我们有

$$\sum_{s=0}^{+1)(m+1)-1}R_s^{(k,\ell)}\left((u,v)_r\right)\Delta\mathbf{P}_s=\Delta\mathbf{D}_r^{(k,\ell)} \tag{11.83}$$

偏导数 $R_s^{(k,\ell)}\left((u,v)_r\right)$ 可以以与第 4.5 节中描述的 $\mathbf{S}^{(k,\ell)}(u,v)$ 完全相同的方式计算。然后，正如曲线的情况一样，我们可以建立 $M\times N$ 线性方程组

$$B\left[\Delta\mathbf{P}_{s_\alpha}\right]=\left[\Delta\mathbf{D}_r^{(k,\ell)}\right]\;\alpha=1,\ldots,N \tag{11.84}$$

M 是约束的数量 $(M\geq R)$，而 N 是可以自由移动的控制点的数量。一般来说，一个约束涉及 $(p+1)\times(q+1)$ 个控制点，但一个控制点可以通过不将其包含在公式 (11.84) 中而保持固定。方程 (11.84) 可以是欠定的、完全确定的或超定的。如果是超定的，则可以使用节点插入和/或次数提升来获得欠定或完全确定的系统。

方程 (11.84) 的求解方式与曲线的情况完全相同，即

$$\left[\Delta\mathbf{P}_{s_\alpha}\right]=A\left[\Delta\mathbf{D}_r^{(k,\ell)}\right] \tag{11.85}$$

其中

$$A=B^T\left(BB^T\right)^{-1} \tag{11.86}$$

(或 $A=B^{-1}$ 如果方程 [11.84] 是完全确定的)。这产生了控制点的最小组合移动。最后一步是

$$\widehat{\mathbf{P}}_{s_\alpha}=\mathbf{P}_{s_\alpha}+\Delta\mathbf{P}_{s_\alpha}\;\alpha=1,\ldots,N \tag{11.87}$$

尽管基于曲线和曲面的约束修改的数学几乎是相同的，但设置矩阵 A 所需的软件对于曲面来说更为复杂。算法 A11.1 说明了一种可能的解决方案。除了定义曲面的数据外，还输入一个结构列表 (CList)。该列表包含 R 个节点；每个节点包含一对参数 $(u,v)_r$ 和在该点应用的约束类型列表 (实际约束不被输入)。输出包括:

M : 约束的数量 (以及 **A** 的列数)；

N : 变化的控制点数量 (以及 **A** 的行数)；

A[] [] : 矩阵 A ；

CpInd[] : 变化的控制点的绝对地址 s_α 。

此外，返回以下整数值之一:

0 : 无错误；

1 : 系统是过定的；

2 : 矩阵是奇异的 (B 的行不是线性独立的)。

为了构建 A 和 CpInd，使用了三个局部二维数组 B[M][Nmax]、FreeCp[p+1][q+1] 和 Map[n+1][m+1]，以及一个四维数组 Funs 和两个新例程 FreeCtrlPts(u, v, FreeCp) 和 RatDersBasisFuns(u, v, Funs)。B 包含方程 (11.84) 的矩阵 B 。如果 $\mathbf{P}_{i,j}$ 不是由方程 (11.84) 定义的系统的一部分，则 Map[i] [j] $=-1$ ；否则，Map[i] [j] $=\alpha\geq 0$ 是 CpInd 的索引，而 CpInd [α] $=s$ 是对应于 (i, j) 的绝对地址。对于给定的 (u, v)，FreeCtrlPts( ) 返回布尔数组 FreeCp，指示在 (u, v) 处影响表面的可能 $(p+1)\times(q+1)$ 控制点中哪些是可以移动的。这个例程可以以多种有意义的方式实现，因此我们在这里不再详细说明。例程 RatDersBasisFuns(u, v, Funs) 返回基函数及其偏导数。Funs [k] [1] [i] [j] 是 (i, j)th 基函数的 (k,ℓ) 次偏导数。

ALGORITHM A11.1

```
ConstBasedSurfMod1(n, m, p, q, U, V, Pw, CList, R, M, N, A, CpInd)
    { /* Build the A matrix for surface constraints */
        /* Input: n, m, p, q, U, V, Pw, CList, R */
        /* Output: M, N, A, CpInd */
  Loop through the list CList and count constraints
        to obtain M.
  Nmax = Max ((n+1)*(m+1), R*(p+1)*(q+1)); /* Max possible N */
  for (i=0; i<M; i++)
        for (j = 0; j < Nmax; j++) B[i][j] = 0.0 ;
  for (i = 0; i<=n; i++)
        for (j = 0; j <= m; j++) Map[i][j] = -1 ;
  brow = 0; N = 0; / * N is computed in this loop */
  for (r=1; r<=R; r++)
        {
        Extract (u, v) value from the r th list node.
        FreeCtrlPts(u, v, FreeCp);
        uspan = FindSpan(n, p, u, U);
        vspan = FindSpan(m, q, v, V);
        for (i = 0; i<=p; i++)
        for (j=0; j<=q; j++)
            if (FreeCp[i][j])
                { /* This ctrl pt is free to move */
                iup = i+uspan-p; jvq = j+vspan-q;
                if (Map[iup][jvq] == -1)
                    { /* Ctrl pt not yet in Eq. system */
                    Map[iup][jvq] = N;
                    CpInd[N] = jvq*(n+1)+iup;
                    N = N + 1 ;
                   }
                }
    /* Compute required functions */
    RatDersBasisFuns(u, v, Funs);
    Loop through each type of constraint for this(u, v)and do :
        {
        Set k and ℓ indicating which derivative.
        for (i = 0; i<=p; i++)
            for (j=0; j<=q; j++)
                {
                alf = Map[i+uspan-p][j+vspan-q];
                if (alf >= 0) B[brow][alf] = Funs[k][l][i][j];
                }
        brow = brow+1;
        } /* End of loop through each type of constraint */
  } /* End of for-loop: r = 1,...,R */
  if (M > N) return (1); /* system overdetermined */
  if (M == N)
        if(B singular)return(2);
            else
            {
            A = B^{-1}; return (0);
            }
  if (BB^T singular) return(2);
        else
        {
        A = B^T(BB^T)^{-1}; return(0);
       }
  }
```

图 11.37-11.39 显示了应用表面约束的示例。在图 11.37a 和 11.37 b 中，在 $u=v=1/2$ 处施加了一个点约束，允许所有影响该点表面的控制点移动。图 11.37a 显示了新的控制网，图 11.37b 则说明了表面形状的变化。请注意，局部域周围的控制点移动非常有限。图 11.38a 和 11.38 b 显示了如果仅允许 $\mathbf{P}_{1,2},\mathbf{P}_{2,2}$ 和 $\mathbf{P}_{4,2}$ 在形状中移动会发生什么。最后，在图 11.39a 和 11.39 b 中施加了偏导数和位置约束。在图 11.39a 中，偏导数的大小增加，导致了张力效应，而在图 11.39b 中，偏导数围绕点 $\mathbf{S}(1/2,1/2)$ 旋转，导致表面形状扭曲。

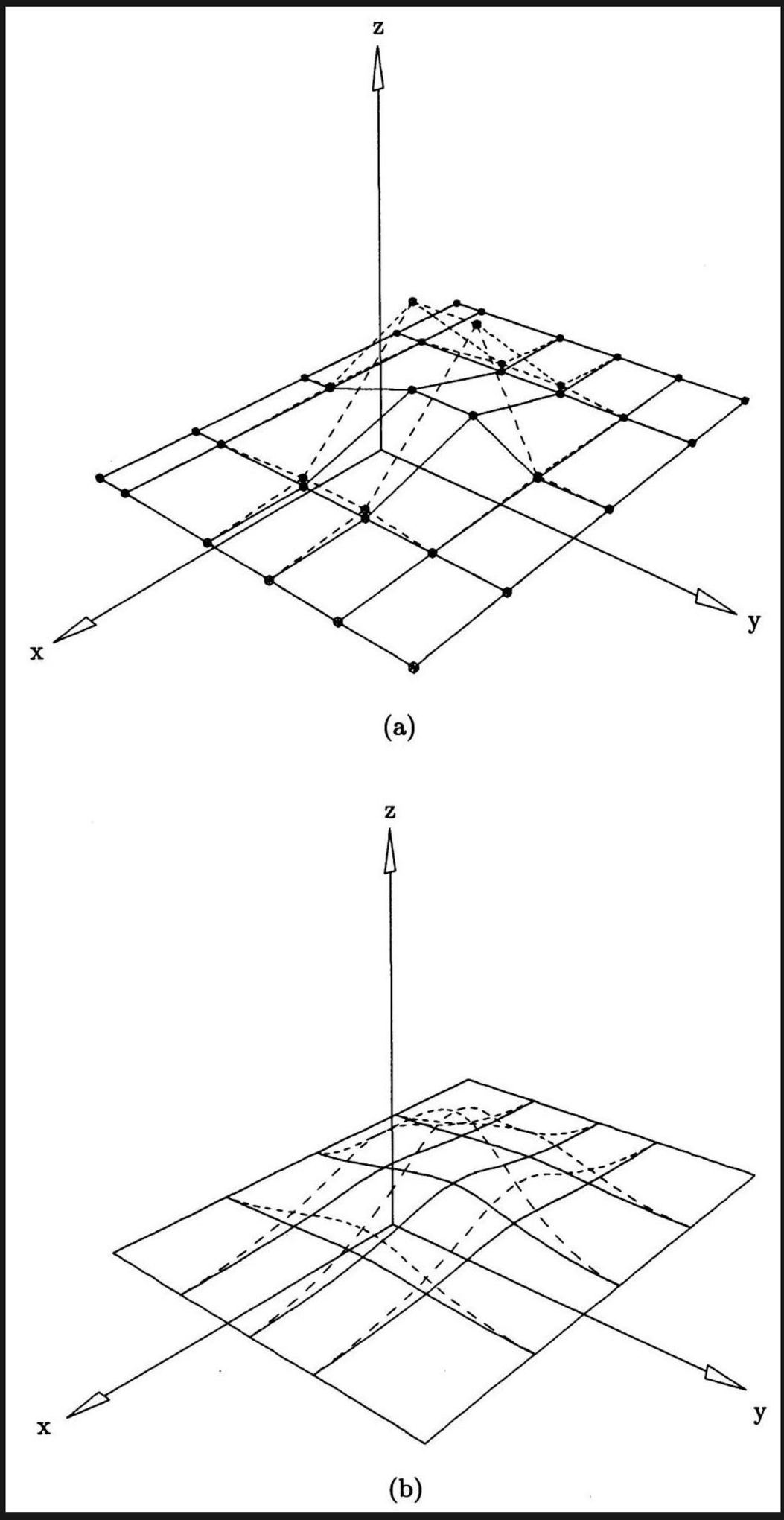


图 11.37. 基于约束的表面修改，带有位置约束。(a) 显示所有移动的局部控制点的表面网；(b) 形状发生变化的表面。

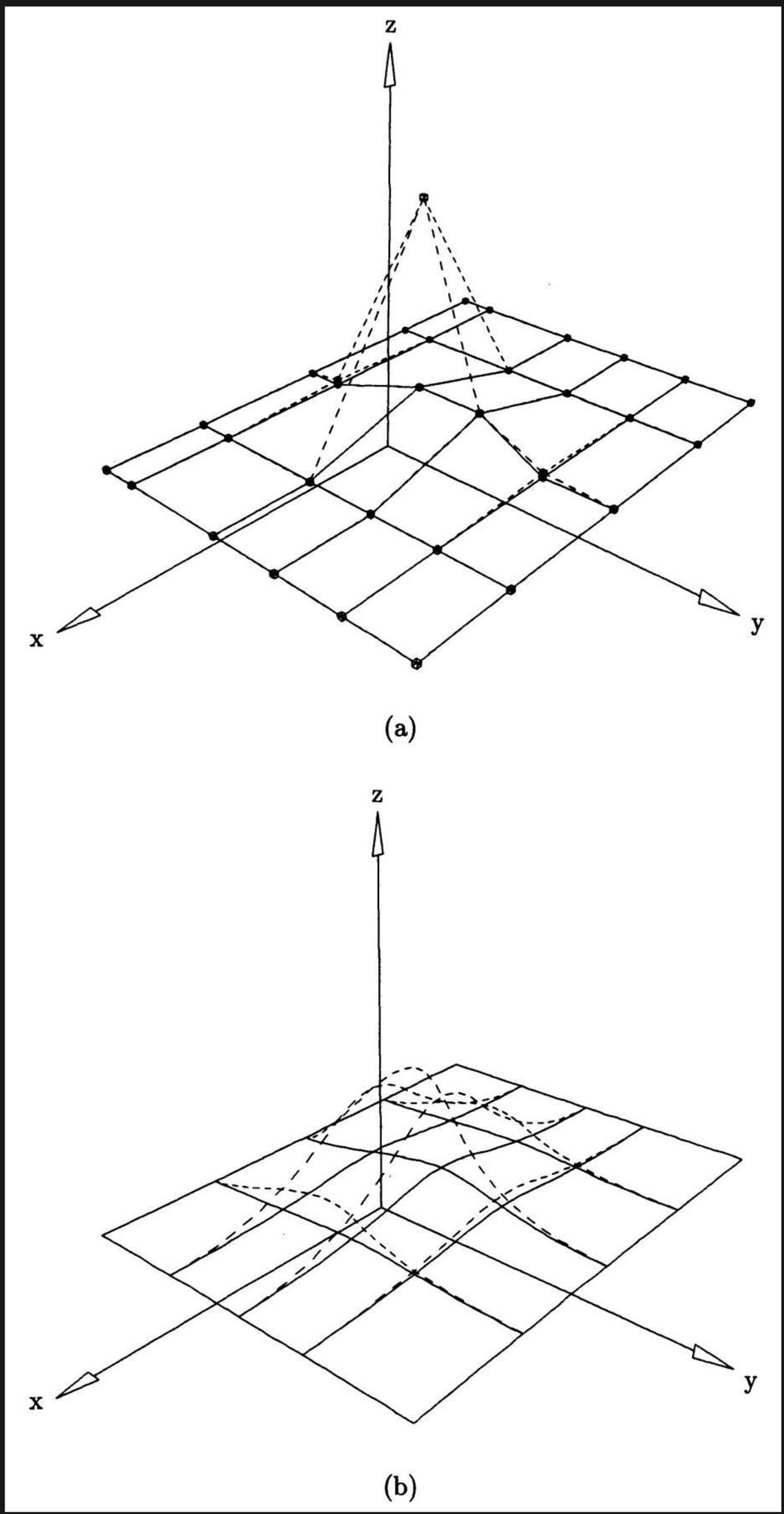


图 11.38. 基于约束的表面修改，带有位置约束。(a) 显示仅允许 $\mathbf{P}_{1,2}, \mathbf{P}_{2,2}$ 和 $\mathbf{P}_{4,2}$ 移动的表面网；(b) 形状发生变化的表面。

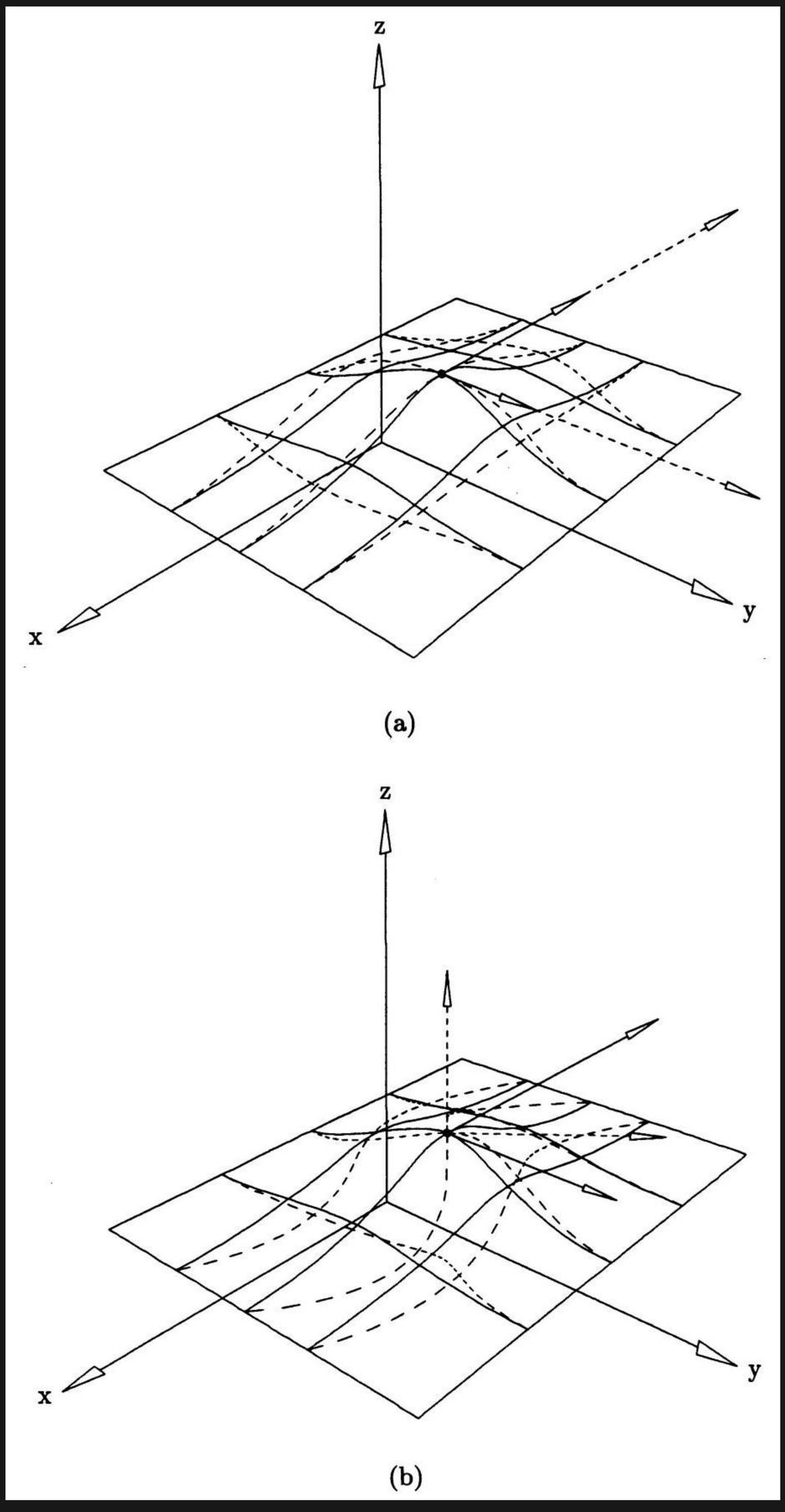


图 11.39. 基于约束的表面修改，带有位置和导数约束。(a) 偏导数大小的增加引发局部张力效应；(b) 偏导数方向的变化引发局部扭曲效应。

最后，我们指出，Fowler [Fowl92] 提出了一个替代 (等效) 解决方案，用于求解 $\Delta \mathbf{P}_{i,j}$ ，该方案利用了 $\mathbf{S}(u,v)$ 的张量积结构。它的通用性不如这种方法，但在某些情况下更为高效。

第十二章　标准与数据交换

1　引言

本章的中心主题是本书中所实践的 NURBS 几何与其他表示曲线和表面的方法之间的关系，以及在两个系统之间交换几何数据的几个重要标准格式。我们在整本书中使用了一种特定形式的节点向量；第 12.2 节讨论了另一种类型的节点向量及两种类型之间的转换。第 12.3 节描述了 NURBS 曲线和表面在三个标准 (IGES、STEP 和 PHIGS) 中的定义。我们指出，我们的 NURBS 与这些标准的 NURBS 兼容 (唯一的例外是 IGES 和 PHIGS 允许不连续的曲线和表面)。最后，第 12.4 节提供了一些关于在 NURBS 系统与其他系统之间交换几何数据的一般指导。

2　结点向量

节点向量可以分为两组: 固定和非固定；在每组中又有均匀和非均匀节点向量。以下是一些示例:
固定，均匀:

$$\{0,0,0,0,1,2,3,4,4,4,4\}$$

$$\{-0.5,-0.5,-0.5,1,2.5,4,4,4\}$$

固定，非均匀:

$$\{0,0,0,2,3,6,7,7,7\}$$

$$\{0,0,0,0,1,2,2,3,4,5,5,5,5\}$$

非固定，均匀:

$$U_1=\{-3,-2,-1,0,1,2,3,4,5\}$$

$$U_2=\{0,1,2,3,4,5,6,7,8,9,10,11\}$$

非固定，非均匀:

$$\{0,0,1,2,3,4\}$$

$$\{-2,-1,0,4,5,6,7\}$$

因此，夹紧/未夹紧指的是第一个和最后一个节点值是否以等于度数加一的重数重复，而均匀/非均匀则指节点间距。要实现均匀且未夹紧，所有节点跨度必须长度相等。要实现均匀且夹紧，仅“内部”节点跨度必须长度相等。读者可能已经注意到，我们现在称之为夹紧、均匀和非均匀的内容，正是我们迄今为止称之为非周期性、均匀和非均匀的内容。在对未夹紧节点向量进行简要研究之后，我们将为这种术语在节点流中间的变化提供一个微弱的借口。由于本书中假设使用夹紧节点向量，因此在此无需进一步阐述。然而，在继续研究未夹紧节点向量之前，我们提供以下与夹紧节点向量相关的提醒:

- 如果我们给定一个具有 $m+1$ 个节点和端点重数为 $p+1$ 的夹紧节点向量，那么我们需要 $n+1\,(=m-p)$ 个控制点来定义一个 p 次曲线；也就是说，节点向量决定了曲线的度数、控制点的数量以及有效的参数范围；
- 从几何上讲，夹紧曲线可以是开放的或闭合的；如果一条曲线是闭合的，那么它是否以 C^k 连续性闭合取决于第一个 k 和最后一个 k 个内部节点跨度，以及第一个 $k+1$ 和最后一个 $k+1$ 个权重和控制点。

现在考虑未夹紧的均匀节点向量 U_1 。使用 Cox-deBoor 算法，方程 (2.5)，我们可以计算 $8-p$ 度 p 基函数，适用于 $p=0,\ldots,7$ 。度为 2 和 3 的基函数，以及一条二次曲线和一条三次曲线，分别显示在图 12.1a 和 12.1b 中。节点位置标记在曲线上。注意，这些曲线在端点处并未夹紧，即它们的起始点和终点与控制点不重合。此外，在 $u\in[u_i,u_{i+1})$ 处评估曲线需要函数 $N_{i-p,p}(u),\ldots,N_{i,p}(u)$ ；因此，这些曲线仅在参数范围 $u\in[u_p,u_{m-p}]$ 上定义，这与夹紧曲线的情况相同。因此，图 12.1a 的二次曲线在 $u\in[-1,3]$ 上定义，而图 12.1b 的三次曲线在 $u\in[0,2]$ 上定义。图 12.2 显示了如何使用未夹紧的均匀节点向量 U_2 定义一条闭合的三次曲线。由于 $m=11$ 和 $p=3$ ，我们需要八个控制点 $(n=7)$ 。我们在末尾环绕 $p\,(=3)$ 控制点；即 $\mathbf{P}_5=\mathbf{P}_0,\mathbf{P}_6=\mathbf{P}_1$ 和 $\mathbf{P}_7=\mathbf{P}_2$ 。有效的参数范围是 $u\in[3,8]$ 。这导致曲线闭合，并具有 C^2 连续性。

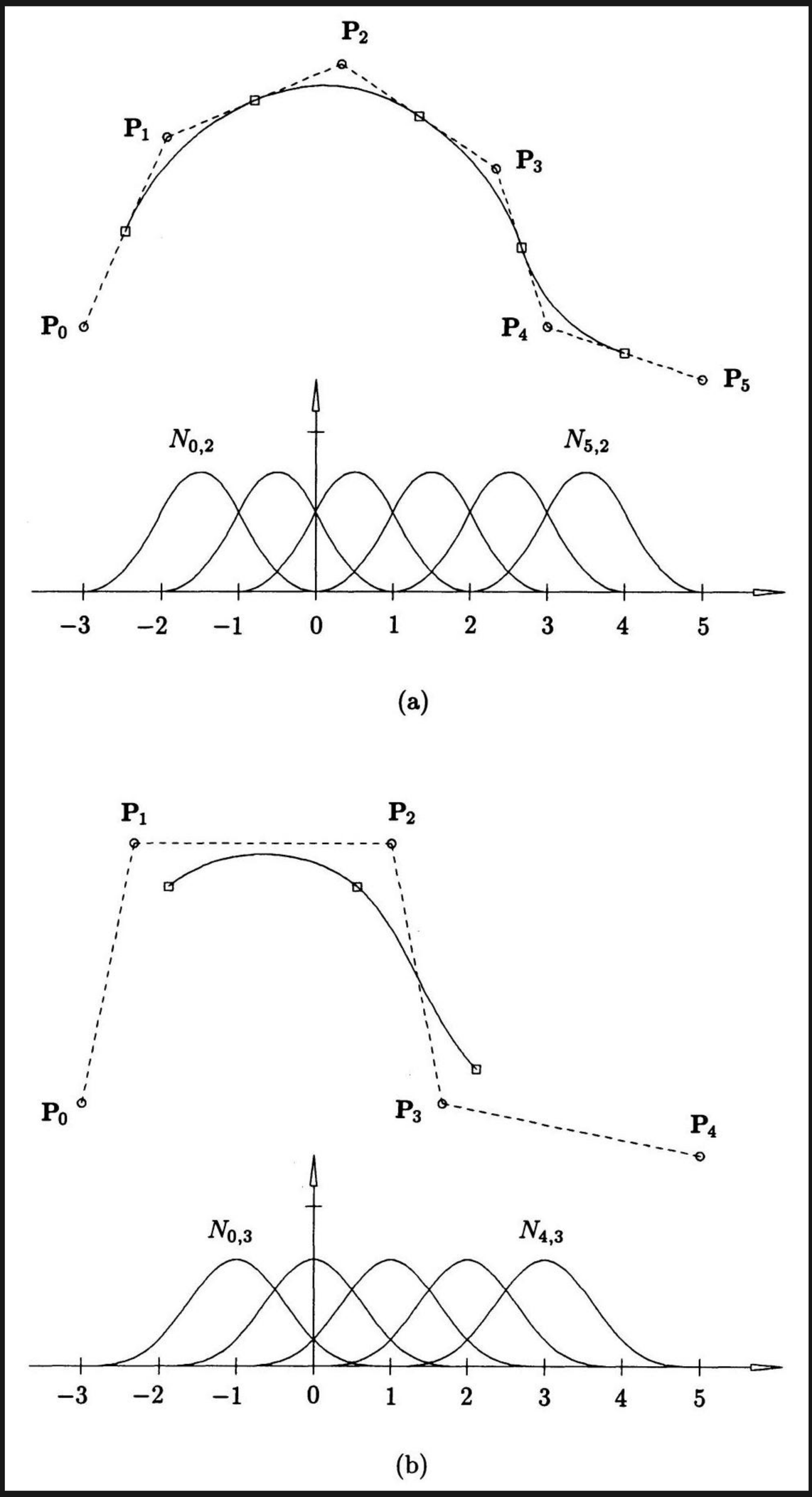


图 12.1. 未夹紧的 NURBS 曲线。(a) 二次曲线；(b) 三次曲线。

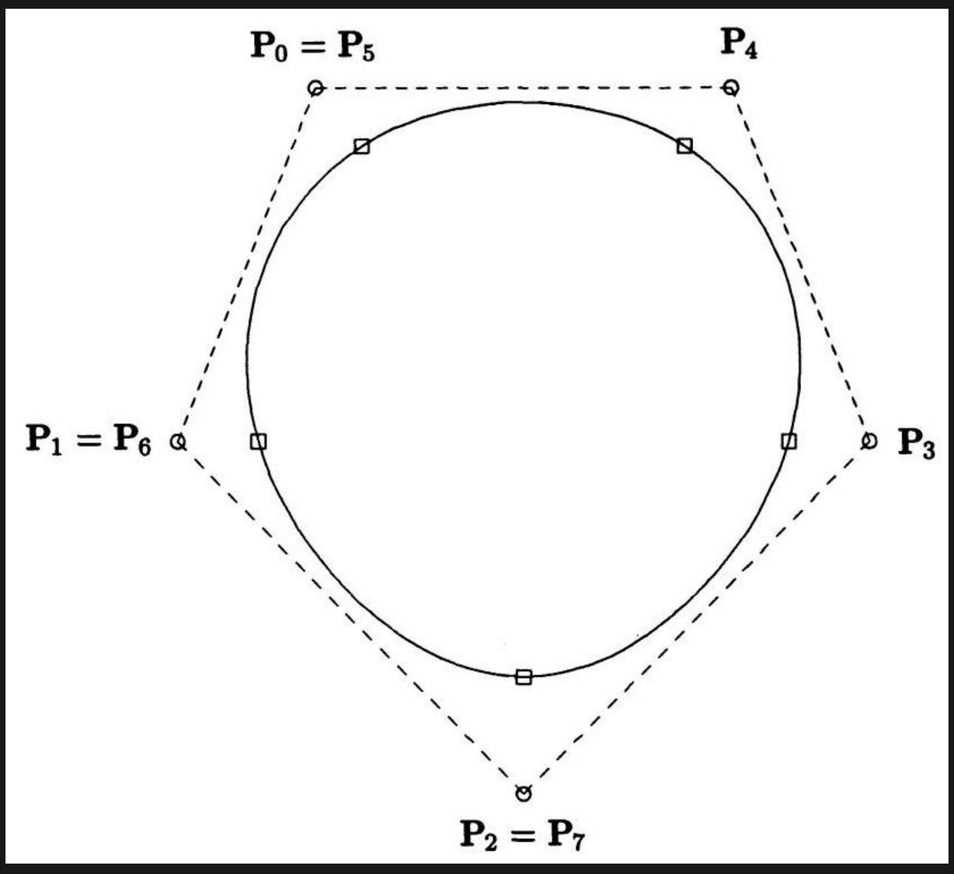


图 12.2. 闭合的 C^2 连续未夹紧三次曲线。

之前所述的关于未固定的均匀曲线的所有内容同样适用于未固定的非均匀曲线，唯一的例外是闭合曲线的起始/结束处的 C^{p-1} 连续性。由于连续性依赖于节点值以及控制点位置，闭合的非均匀曲线的 C^{p-1} 连续性并不一定意味着控制点的精确包裹。图 12.3a 展示了在固定的均匀节点向量上定义的闭合立方曲线。

$$U = \{3, 3, 3, 3, 4, 5, 6, 7, 8, 8, 8, 8\}$$

该曲线在起始/结束点处是 C^2 连续的。图 12.3b 和 12.3c 显示了 (精确地) 相同曲线的未固定版本；图 12.3b 中的曲线使用了节点。

$$U = \{0, 0.5, 2.3, 3, 4, 5, 6, 7, 8, 8.7, 10.5, 11\}$$

图 12.3c 中的曲线使用了节点。

$$U = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$$

在这里重要的是指出，从算法的角度来看，固定曲线和未固定曲线之间几乎没有区别。只需稍作修改，本书前五章中给出的所有基本算法也可以用于未固定曲线。

在 CAD/CAM 的早期阶段 (1980 年代初之前)，使用非均匀节点向量、夹紧或不夹紧的工作非常少。尽管在这个领域的术语一直令人困惑，早期的 CAD/CAM 工作者通常将 B 样条称为非周期性或周期性；非周期性与夹紧同义 (如本书所述)，这些用于定义开放曲线和曲面 (最初是均匀的，后来是非均匀的)。周期性在数学上等同于不夹紧、均匀，这些用于构造 C^{p-1} 连续封闭曲线和曲面。参考图 12.1a 和 12.1b，可以看到不夹紧、均匀的基函数都是彼此的平移，因此它们常被称为周期性。对于周期性 B 样条，不需要存储节点向量，不需要 Cox-deBoor 算法，也不需要控制点的环绕来获得 C^{p-1} 连续封闭曲线。一个简单的矩阵形式的 (单个) 基函数，加上一个特殊的控制点索引方案，就足以评估周期性 B 样条曲线 (详见 [Mort85] 的讨论)。尽管更高效，这种对周期性 B 样条的特殊处理几乎在实践中消失，可能是因为软件开发和维护所需的时间已经变得比 CPU 时间昂贵得多。夹紧和不夹紧是较新的术语，可能会逐渐增多使用，仅仅因为它们更直观和准确。尽管在文献中这些术语出现得相对不频繁，但它们在 CAD/CAM 社区似乎相当普遍。我们在本书中使用非周期性这个术语，仅仅是因为我们尚未打破这个习惯。

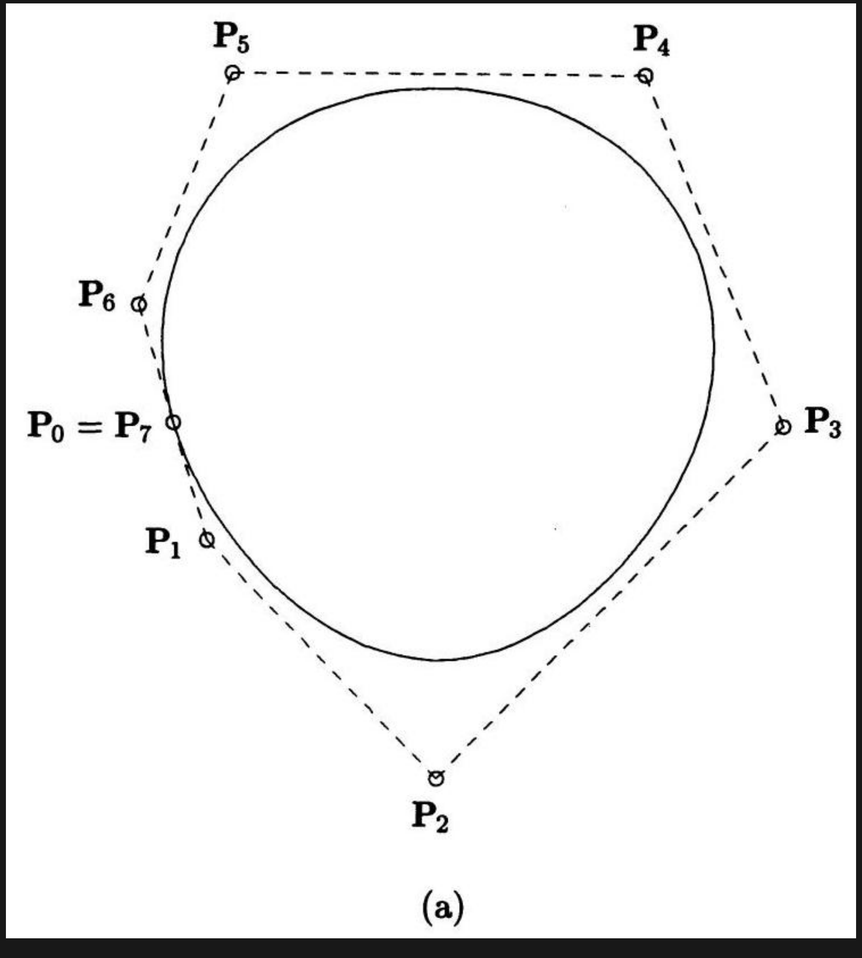


图 12.3. 使用不同节点向量解锁闭合曲线的示例。(a) 闭合夹紧曲线；(b) 无环绕解锁；(c) 带环绕解锁。

目前许多系统仅使用夹紧 B 样条。然而，IGES 和 STEP 都支持无夹紧 B 样条的交换。双向的精确转换都是可能的。夹紧曲线无非是将节点 u_p 和 u_{m-p} 插入，直到它们的重数达到 p，然后丢弃位于夹紧区域之外的节点和控制点。节点插入公式 (公式 (5.15)) 同样适用于无夹紧曲线，只要不对范围在 0 到 m 之外的节点进行索引 (在 u_p 和 u_{m-p} 夹紧时不会发生这种情况)。我们建议 (但留作练习) 读者编写一个例程，以提取任意参数 u_1 和 u_2 之间的子曲线，其中 $u_1 < u_2$。这样的例程不仅处理夹紧，还有许多其他用途，包括我们在下一节中提到的一个用途。

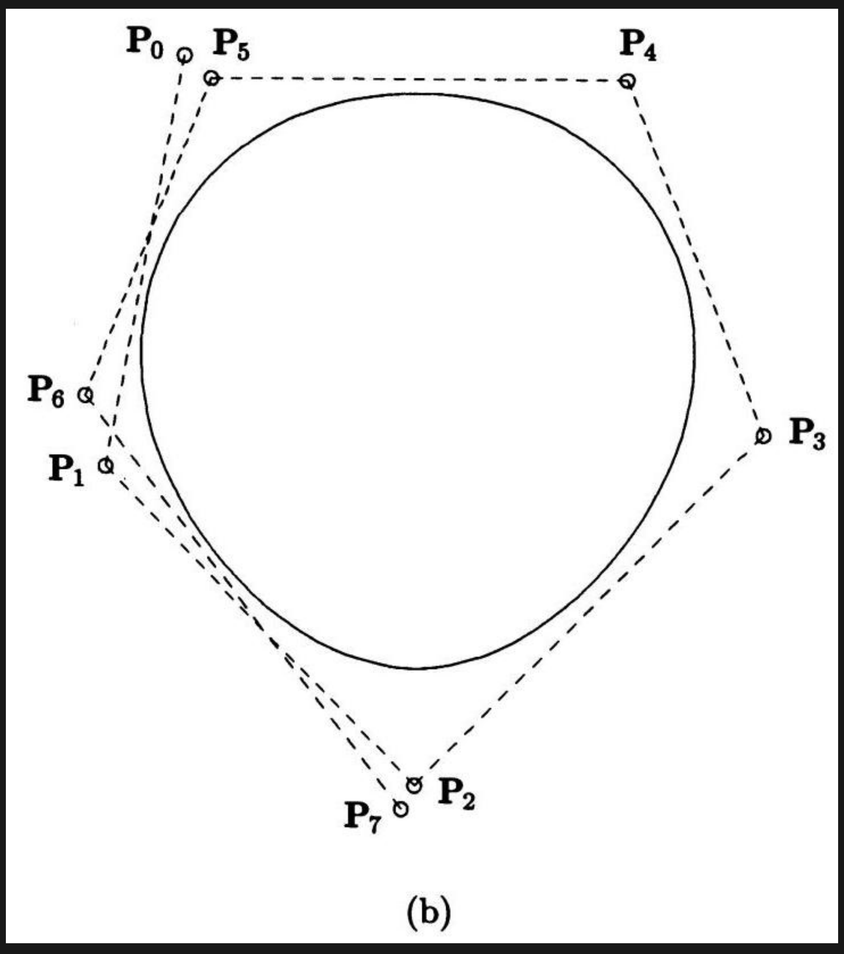


图 12.3.(续)

解锁本质上是节点移除。尽管解锁很少被要求 (是的，确实有仅处理无夹紧 B 样条的系统)，我们在此呈现算法 A12.1，该算法计算每端的 $p-1$ 新控制点和每端的 p 新节点，所有操作均在原地进行。我们提醒读者，结果并不是唯一的 (见图 12.3b 和 12.3c)。选择 $2p$ 新端节点是相对自由的，而新控制点的位置依赖于节点。我们选择新节点如下:

$$u_{p-i-1} = u_{p-i} - (u_{n-i+1} - u_{n-i}) \;\; i = 0, \ldots, p-1 \tag{12.1}$$

和

$$u_{n+i+2} = u_{n+i+1} + (u_{p+i+1} - u_{p+i}) \;\; i = 0, \ldots, p-1 \tag{12.2}$$

这种节点选择会导致控制点的环绕，如图 12.2 和 12.3c 所示，前提是夹紧曲线以 C^{p-1} 连续性闭合。算法为:

(c)

Figure 12.3. (Continued.)

```
## ALGORITHM A12.1
UnclampCurve(n, p, U, Pw)
{ /* Unclamp a clamped curve */
    /* Input: n, p, U, Pw */
    /* Output: U, Pw */
for (i=0; i<=p-2; i++) /* Unclamp at left end */
    {
    U[p-i-1] = U[p-i] - (U[n-i+1] - U[n-i]);
    k = p - 1 ;
    for (j=i; j>=0; j--)
        {
        alfa = (U[p]-U[k])/(U[p+j+1]-U[k]);
        Pw[j] = (Pw[j] - alfa*Pw[j+1]) / (1.0 - alfa);
        k = k - 1 ;
        }
    }
U[0] = U[1] - (U[n-p+2]-U[n-p+1]); /* Set first knot */
for (i=0; i<=p-2; i++) /* Unclamp at right end */
    {
    U[n+i+2] = U[n+i+1] + (U[p+i+1] - U[p+i]);
    for (j=i; j>=0; j--)
        {
alfa = (U[n+1]-U[n-j])/(U[n-j+i+2]-U[n-j]);
Pw[n-j] = (Pw[n-j] - (1.0 - alfa)*Pw[n-j-1])/alfa;
    }
  }
U[n+p+1] = U[n+p] + (U[2*p]-U[2*p-1]); /* Set last knot */
}
```

图 12.4a-12.4d 显示了对一个 4 次曲线的左端进行解夹的步骤。图 12.4b-12.4d 分别对应于 i 第 $i = 0, 1, 2$ 步的完成。

$$U = \left\{0, 0, 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{2}, \frac{1}{2}, \frac{3}{4}, \frac{3}{4}, 1, 1, 1\right\}$$

(见示例 Ex7.2)。九个控制点已被标记。图 12.5b 说明了通过算法 A12.1 解夹后的圆。注意，控制多边形并没有环绕，因为没有控制点重合。这是因为圆在齐次空间中仅是 C^0 连续的。解夹后的节点向量为:

$$U = \left\{-\frac{1}{4}, -\frac{1}{4}, 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{2}, \frac{1}{2}, \frac{3}{4}, \frac{3}{4}, 1, \frac{5}{4}, \frac{5}{4}\right\}$$

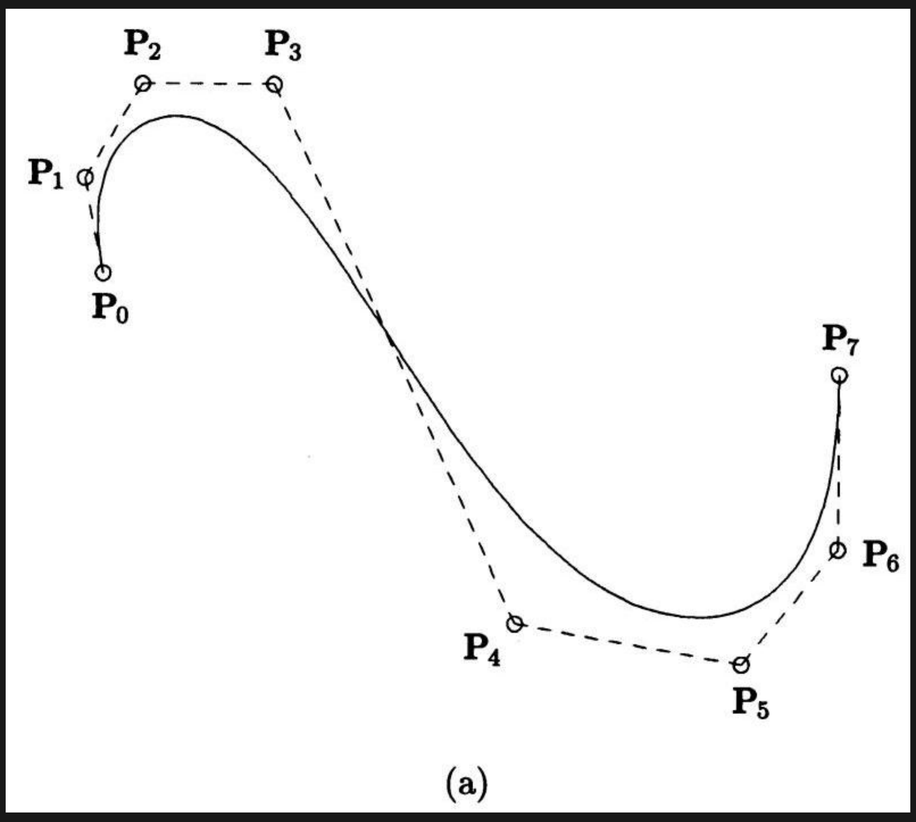


图 12.4。对四次曲线左端进行解夹的过程。(a) 原始夹紧曲线；(b) 计算的 $i = 0; \rightarrow \mathbf{P}_0^0$；(c) 计算的 $i = 1; \rightarrow \mathbf{P}_0^1$ 和 $\mathbf{P}_1^1$；(d) 计算的 $i = 2; \rightarrow \mathbf{P}_0^2, \mathbf{P}_1^2$ 和 $\mathbf{P}_2^2$。

显然，将这些结果扩展到曲面是直接的。通过将算法 A12.1 的计算应用于控制点的 $m+1$ 行 ($n+1$ columns)，可以在 $u\,(v)$ 方向上对曲面进行解锁。图 12.6a-12.6d 显示了曲面解锁的示例。图 12.6a 显示了原始的度数为 (3,2) 的曲面。在图 12.6b 中展示了在 u 方向上的解锁，而在图 12.6c 中则展示了在 v 方向上的解锁。图 12.6d 显示了在两个方向上的解锁。

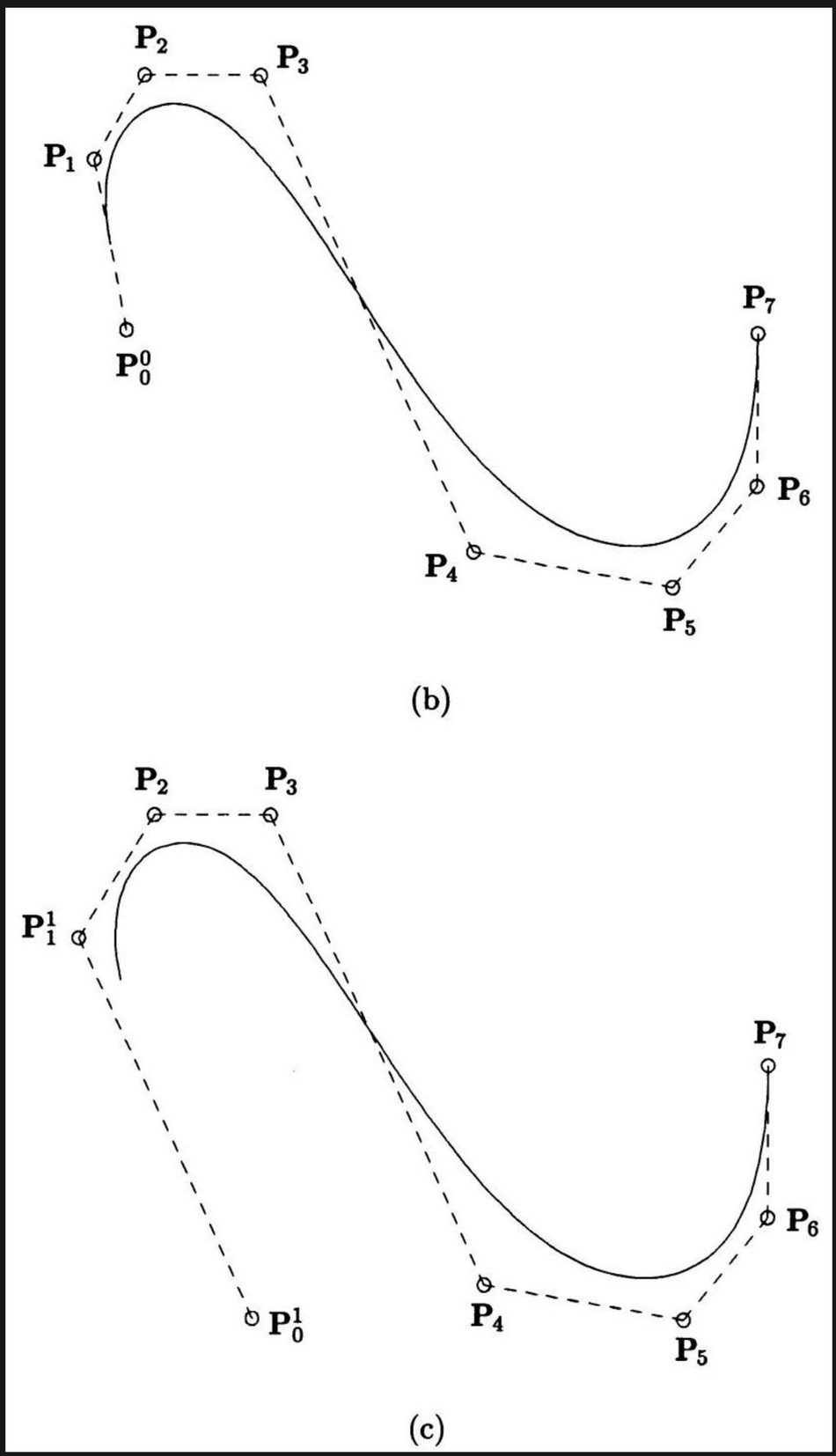


图 12.4.(续)

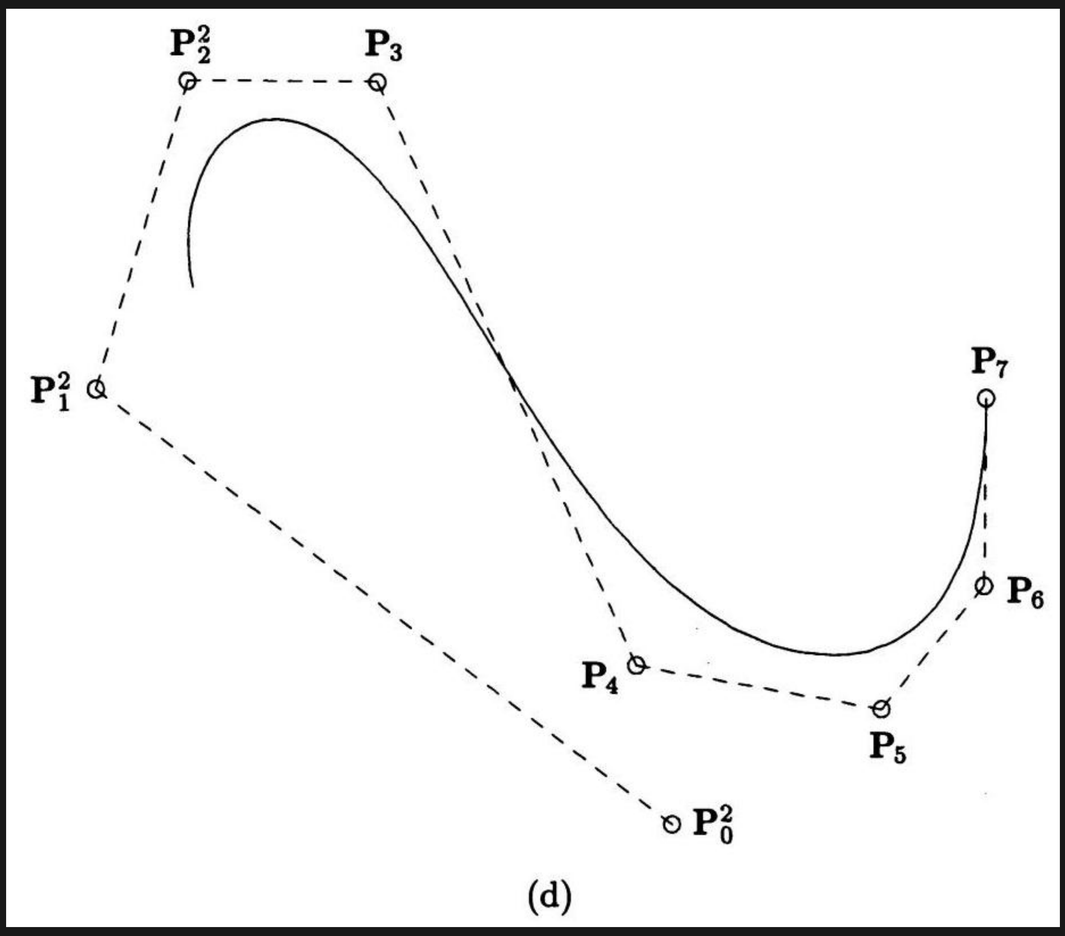


图 12.4.(续)

3 标准中的 NURBS

NURBS 不仅是 CAD/CAM/CAE 行业的事实标准，而且还被纳入了多个国际和美国国家标准:

- IGES: 初始图形交换规范；
- STEP: 产品模型数据交换标准；
- PHIGS: 程序员的层次交互图形系统。

本节的主题是这些标准中 NURBS 的定义。目的是概念上传达每个标准中 NURBS 的定义。为此，我们采用非正式的风格，避免使用每个标准的具体术语。读者应查阅相关文件，以便对这些标准进行更严格的研究 [PHIG92; IGE93; STEP94]。有关进一步阅读，请参见 Bloor 和 Owen [Blor91] 以及 Vergeest [Verg91] 关于 STEP 的文献，以及 Howard [Howa91a] 和 Howard 等 [Howa91b] 关于 PHIGS 的文献。任何设计新 NURBS 系统的人都应确保其符合与 NURBS 相关的这些标准。

3.1 IGES

IGES(美国国家标准) 是当今 CAD/CAM/CAE 系统中最广泛使用的产品数据交换格式。IGES 于 1980 年代初开发，规定了用于交换图形和几何数据的格式，支持各种应用，如制图、电路设计、有限元和管道设计。支持曲线、表面和三维固体。NURBS 曲线在 IGES 中的定义如下:

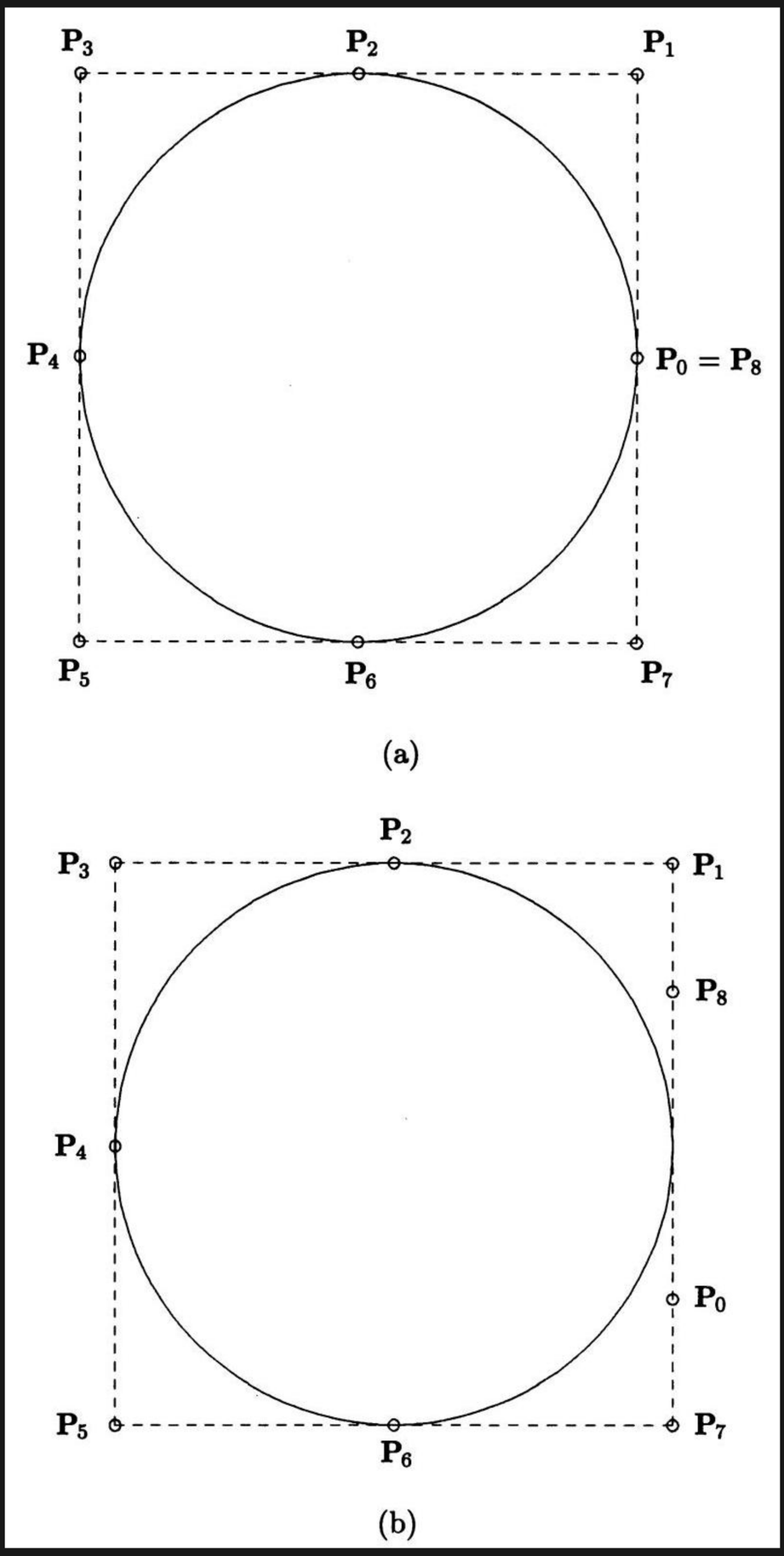


图 12.5. 解锁九点 NURBS 圆。(a) 原始曲线；(b) 解锁曲线。

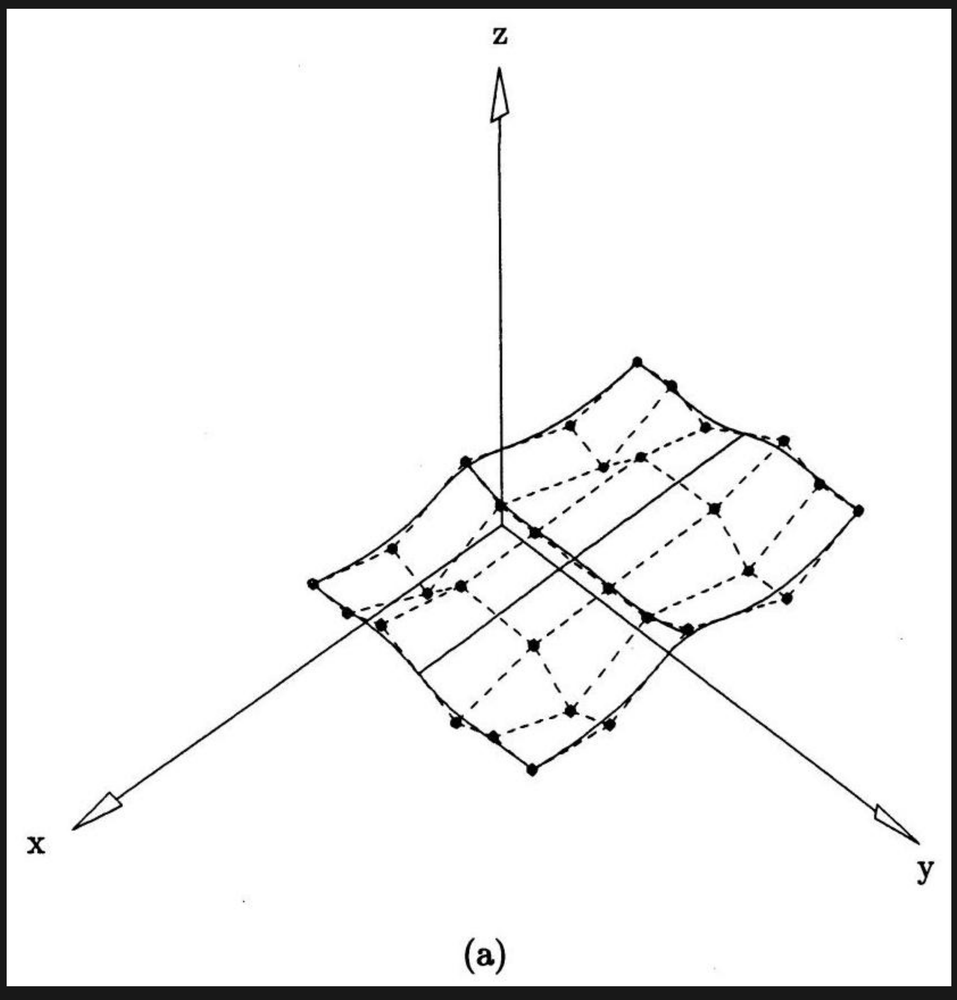


图 12.6. 解锁一个度数为 (3,2) 的表面。(a) 原始表面；(b) 在 u 方向解锁；(c) 在 v 方向解锁；(d) 在两个方向解锁。

- 度数，p ，和控制点数量，$n+1$ ；
- 欧几里得控制点，$\mathbf{P}_i$ ，和权重，w_i ；
- 一个包含 $m+1=n+p+2$ 个节点的节点向量，U ；
- 起始和结束参数值，s_0 和 s_1 ；
- 其他非必要但有用的信息: 曲线是平面还是非平面、开放还是封闭、真正有理 (w_i 不全相等) 还是非有理、周期性还是非周期性；此外，曲线可以标记为特殊类型，例如直线、圆弧或圆锥弧。

注意，控制点和权重在 IGES 中是分开的项目；没有均匀控制点的概念，$\mathbf{P}_i^w$ 。仅允许正权重。“周期性”和“非周期性”这两个术语在 IGES 手册中没有定义，因此它们是没有意义的。它们通常被解释为分别表示未夹紧和夹紧，如第 12.2 节所定义。对节点的唯一约束是 $m=n+p+1$ 和 $u_{i-1}\le u_i$ 对于 $i=1,\dots,m$ 。因此，几乎任何情况都是允许的，包括重数大于 p 的内部节点。参数 s_0 和 s_1 是定义的一个重要部分，因为它们定义了曲线的起点和终点。唯一的限制是 $u_0\le s_0<s_1\le u_m$ 。也就是说，预期的曲线可以是由节点 $u_0,\dots,u_m$ 定义的较大曲线的一个适当子曲线。一个仅允许夹紧曲线且没有 s_0 和 s_1 值的系统可以通过应用子曲线提取来适应 IGES NURBS 曲线 (包括未夹紧曲线)，这只是通过在 s_0 和 s_1 处插入节点来实现，使用算法 A5.1。

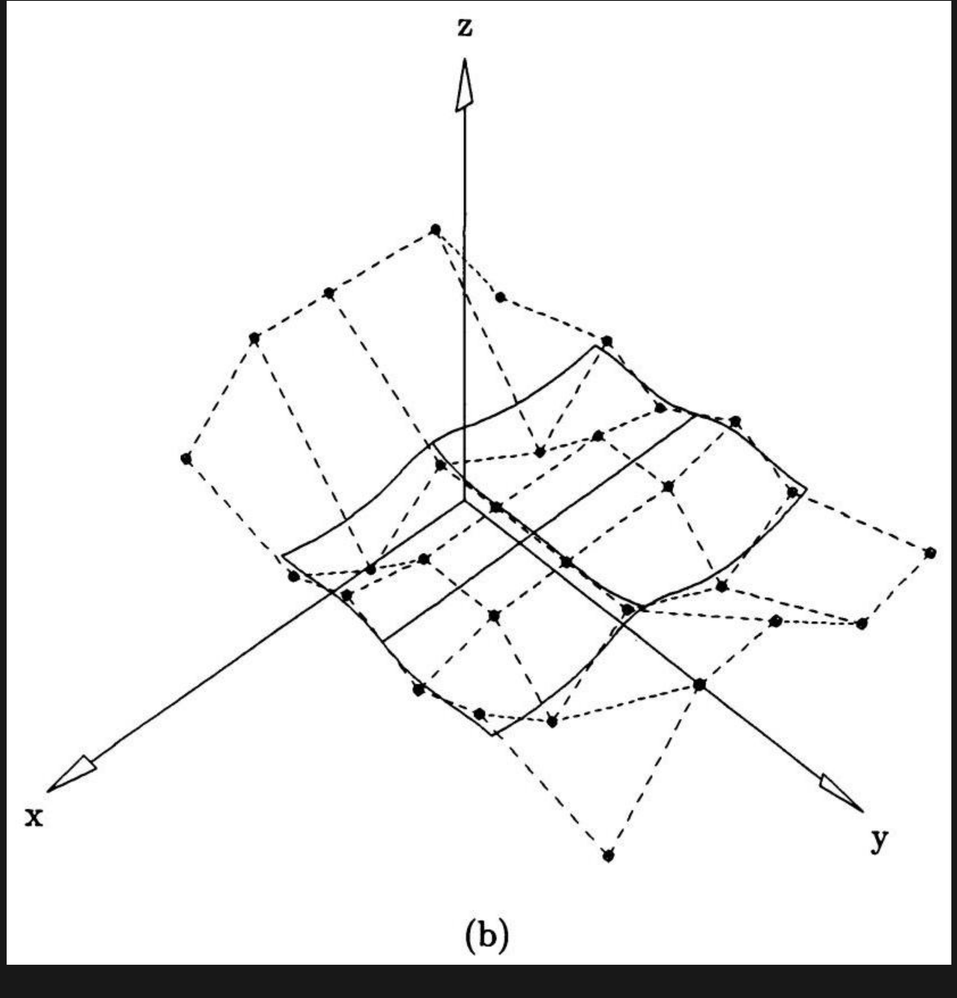
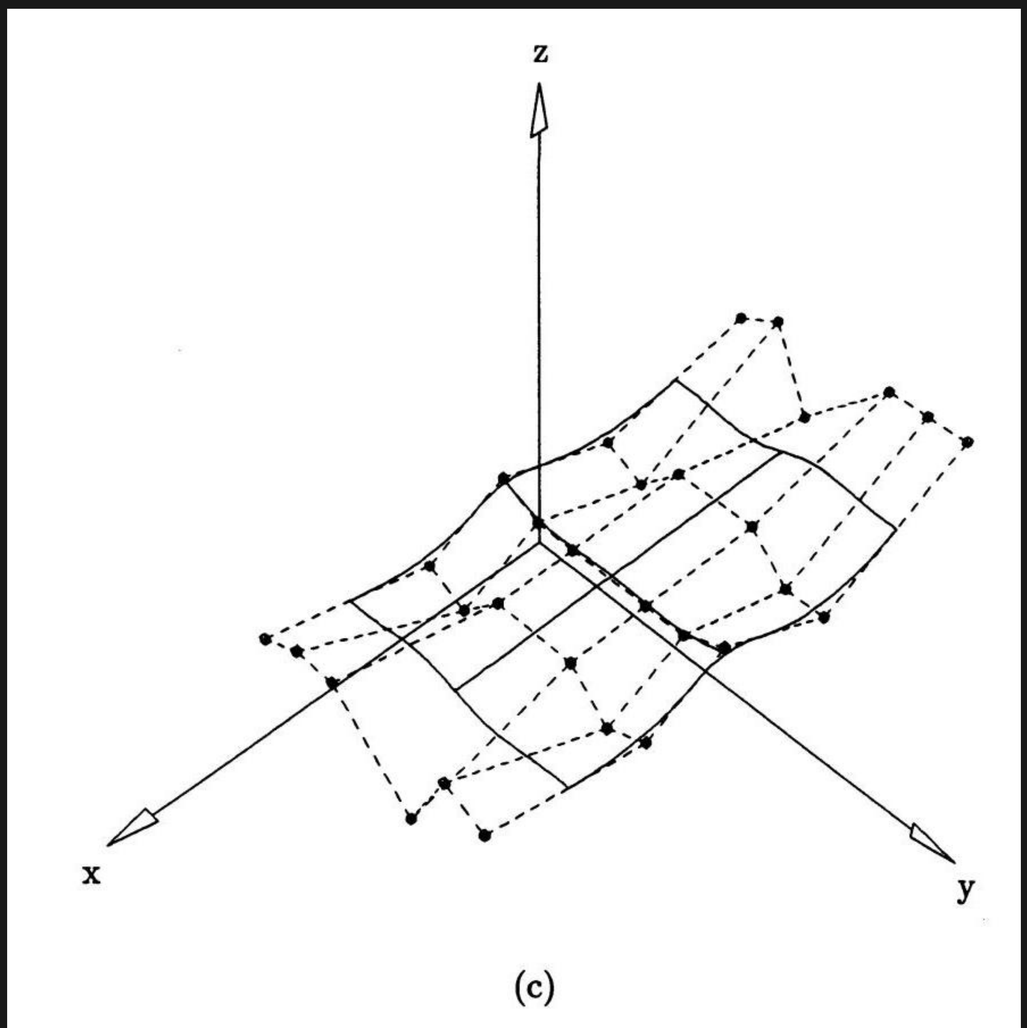


图 12.6.(续)

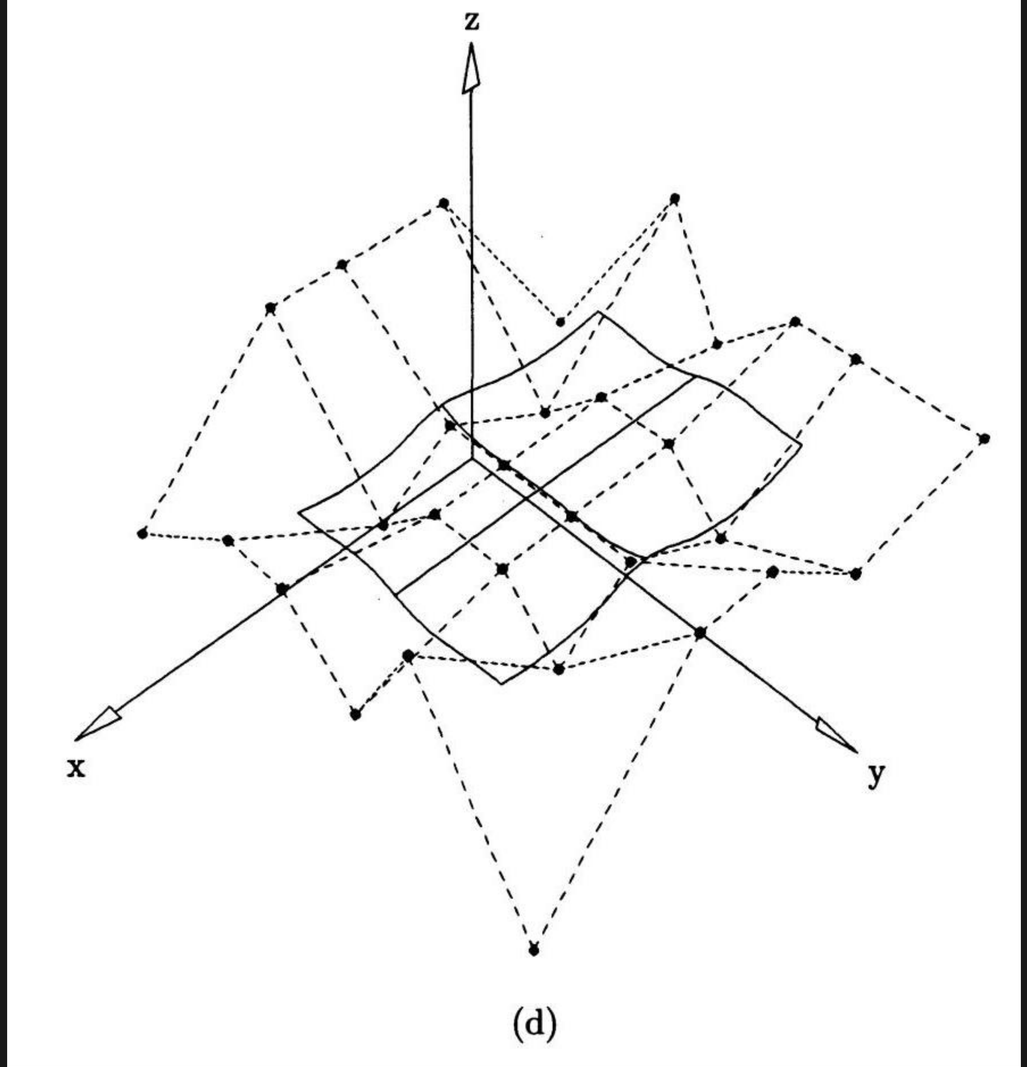
IGES NURBS 曲面是类比定义的。控制点和权重是分开的；权重必须为正。对 u 和 v 节点的唯一限制是 $r=n+p+1, s=m+q+1, u_{i-1}\le u_i$ 对于 $i=1,\dots,r$ 和 $v_{j-1}\le v_j$ 对于 $j=1,\dots,s$ ，其中 p 和 $n+1$ ，以及 q 和 $m+1$ 分别是 u 和 v 方向上的度数和控制点数量。参数 s_0,s_1 和 t_0,t_1 定义了预期的曲面，该曲面可以是由给定节点、权重和控制点定义的曲面的适当子曲面。如果需要，使用节点插入来提取和限制预期的曲面。有信息标志指示曲面在每个方向上是开放还是封闭，以及是周期性 (?) 还是非周期性 (?)，以及曲面是否真正是有理的 (所有 $w_{i,j}$ 相等) 或非有理的。IGES NURBS 曲面可以标记为九种特殊类型之一: 平面、圆柱、锥体、球体、环面、旋转曲面、一般柱面、规则曲面或一般二次曲面。

3.2 STEP

STEP 是一种新兴的国际标准，用于产品模型数据的交换。它代表了一项重大的国际努力，其复杂性、广度和深度远超 IGES。NURBS 曲线和曲面在所谓的第 42 部分中定义，该部分涉及产品模型的基本几何标准和数据交换及拓扑。实际上，由于 NURBS 表示法的通用性、数值稳定性和广泛的工业接受度，它被选为通过 STEP 交换一般样条和单跨度多项式及有理曲线和曲面的唯一方法。第 42 部分包括曲线、曲面和三维实体。NURBS 曲线在 STEP 中的指定方式为:



(c)



(d)

图 12.6. (续)

- 度数，p 和控制点的数量，$n+1$；
- 欧几里得控制点，$\mathbf{P}_i$ 和权重，w_i (没有 $\mathbf{P}_i^w$ 的概念)；仅允许正权重；
- 一个节点向量，U ，包含 $m+1=n+p+2$ 个节点，并满足 $u_{i-1} \leq u_i$ 对于 $i=1,\ldots,m$ ；STEP 限制节点的重数 - 第一个和最后一个节点值的重数最多为 $p+1$ ，所有内部节点值的最大重数为 p ；
- 其他信息，例如节点向量的类型、曲线是开放的还是封闭的、是否自相交，以及如果适用的类型标签 (折线、圆、圆锥)。

“修剪”曲线的概念通过参数 s_0 和 $s_1, u_0 \leq s_0 < s_1 \leq u_m$ 存在于 STEP 中，但处于更高的层次；它并未嵌入曲线的定义中。STEP 根据节点向量的形式区分四种类型的 NURBS 曲线: 均匀、准均匀、Bézier 和非均匀。如果 $u_i - u_{i-1} = d$ 对于 $i=1,\ldots,m$ 和某个正常数 d ，则节点向量是均匀的。这对应于第 12.2 节术语中的未夹紧和均匀。准均匀对应于夹紧和均匀，即 $u_0 = \cdots = u_p, u_{m-p} = \cdots = u_m$ ，并且 $u_i - u_{i-1} = d$ 对于 $i=p+1,\ldots,m-p$ 和某个正常数 d 。Bézier 意味着不同的节点值均匀分布，并且第一个和最后一个具有重数 $p+1$ ，所有内部节点具有重数 p ，即曲线是分段 Bézier 的。任何不属于这三类的节点向量称为非均匀。

STEP NURBS 表面以类似方式定义。控制点和权重是分开的，权重必须为正。只有当其 u 和 v 节点均为各自类型时，表面才是均匀、准均匀或 (分段)Bézier；否则它是非均匀的。存在自交标志，针对 u 和 v 方向的开放/封闭标志，以及 (如适用) 指示特殊类型的标签，如平面、圆柱、规则表面等。

3.3 PHIGS

尽管 IGES 和 STEP 是数据交换标准，但 PHIGS 是一个国际标准，指定了一个与设备无关的交互式图形编程接口。PHIGS 于 1980 年代中期出现，NURBS 于 1992 年作为 PHIGS PLUS 扩展的一部分被纳入该标准 [Howa91b]。支持 NURBS 的 PHIGS 实现允许应用程序员直接传递 NURBS 曲线和曲面以进行显示。NURBS 曲线在 PHIGS 中的定义如下:

- 阶数 (= 度 + 1)、控制点数量和节点数量；
- 欧几里得控制点 $\mathbf{P}_i$ ，如果曲线是非有理的，则使用四维齐次控制点 $\mathbf{P}_i^w$ ；一个标志指示有理性。权重必须为正；
- 节点：仅允许使用夹紧节点向量，可以是均匀或非均匀的。节点的重数没有限制；
- 修剪参数 s_0 和 s_1 ，定义预期曲线的实际起始和结束。

NURBS 曲面的定义类似，只是没有修剪参数。允许更一般的修剪循环，由曲面参数域中的 NURBS 曲线组成。

4 NURBS 系统的数据交换

在本节中，我们假设两个曲线和曲面系统 A 和 B，并研究它们之间的数据交换。我们假设系统 A 是基于本书设计和实现的，即它使用任意阶数的夹紧 NURBS 曲线和曲面。为了实用性和与大多数现有系统的兼容性，我们还要求所有权重为正，并且内部节点的重数不超过阶数。目前，我们对系统 B 中几何体的表示方式没有任何假设。

在两个以不同方式表示几何的系统之间交换曲线或曲面时，有三种可能的情况:

- 可以进行数学上精确的转换，这意味着浮点数舍入是唯一的误差。两种系统中的几何在几何和参数上都是等价的；一个例子是第 6.6 节中给出的 B-spline 和分段幂基形式之间的转换；
- 转换在几何上是精确的，但参数化发生了变化；一个例子是圆，在系统 B 中通过三角函数进行参数化。注意，如果圆用于在系统 A 和 B 中形成某些类型的曲面 (例如规则曲面、皮肤曲面、戈登曲面和库恩斯曲面)，则得到的曲面几何可能会不同；
- 在数学上，两种系统之间不存在精确的转换；在这种情况下，最佳的做法是对某个指定的容差进行近似。例如，如果系统 B 仅允许三次曲线，则必须对来自系统 A 的高阶曲线进行近似。

出于我们的目的，我们区分两种类型的几何:

- 解析几何的简单曲线和曲面，如直线、圆、圆锥曲线、平面、圆柱或圆锥、球体和环面；- 其他一切。假设系统 B 可以表示解析类型，则在系统 A 和 B 之间的这些交换在两个方向上都是几何精确的。考虑从 B 到 A 的方向。大多数系统以有理形式 (例如，有理幂基或贝塞尔) 表示解析类型，或以易于推导几何特征 (如中心、轴、半径、顶点等) 的形式表示。第 6.6 节展示了如何将分段有理幂基或贝塞尔形式转换为 NURBS；从其他有理形式转换为 NURBS 也是类似的。从几何特征构建解析类型的 NURBS 表示是第 7 章和第 8 章的主题。将解析类型从系统 A 传递到 B 更为困难。问题在于曲线或曲面的 NURBS 表示并不是唯一的。例如，有无限多种组合的度数、节点、控制点和权重可以生成以原点为中心的完整单位圆。因此，确定 NURBS 曲线是否为解析类型并计算其几何特征并不简单。第 7.6 节提出了一种方法来确定二次有理贝塞尔曲线的几何特征。然而，可能需要进行度数降低和节点插入，以将 NURBS 曲线转换为分段二次有理形式 (如果可能的话)。此外，只有当 NURBS 曲线在每个贝塞尔段上产生相同的几何特征 (在公差范围内) 时，它才是圆锥曲线。我们强调，我们仅对系统 A 中构成 NURBS 曲线或曲面的内容做出假设；我们对特定解析类型如何构建为 NURBS 不做任何假设。实际上，这种几何形状可能在另一个系统中以 NURBS 形式创建，但使用与本书中所呈现的节点、权重和控制点的不同组合，然后传递到系统 A。为 NURBS 几何提供类型标签，如 IGES 和 STEP 中允许的 (见第 12.3 节)，在这方面提供了一些帮助；但这并不能消除问题。

现在考虑更复杂的几何形状，从一般的规则和旋转表面到偏移、混合、倒角以及一般自由形状的曲线和表面。系统 B 可能使用三种方法来表示这样的几何形状:

- 形式为 $f(x,y)=0$ 或 $f(x,y,z)=0$ 的隐式方程；
- 程序定义；
- 多项式或有理样条的多种形式之一。

理论上，可以使用被称为隐式化的技术将 NURBS 精确转换为分段隐式形式 (例如，参见 [deMo84; Sede84; Hoff89] 及其中的参考文献)。然而，这在计算上并不实用；从隐式形式精确转换为 NURBS 形式通常是不可能的，因此需要近似。在实践中，隐式方程很少用于表示复杂几何，因此我们不再深入探讨这一主题。

如果一条曲线或表面不是通过显式系数或函数直接定义，而是通过其他基曲线或表面间接定义，并结合一个程序或公式从基几何上的点计算出目标几何上的点，则我们称其为程序定义。

偏移曲线和表面是程序定义几何的例子。偏移表面 $\mathbf{O}(u,v)$ 由以下方式指定:

$$\mathbf{O}(u,v)=\mathbf{S}(u,v)+d\mathbf{N}(u,v) \tag{12.3}$$

其中 $\mathbf{S}(u,v)$ 是基表面，$d\neq 0$ 是一个常数标量，而 $\mathbf{N}(u,v)$ 是 $\mathbf{S}(u,v)$ 在 (u, v) 处的单位长度法向量。偏移曲线和表面在数学上是复杂的，通常很难精确表示为 NURBS 形式 (参见 [Till84; Faro86; Coqu87b; Hosc88])。混合和倒角表面有时是通过程序定义的 [Choi91]，而 Filip 和 Ball [Fili89] 描述了一种皮肤化的程序方法。这些表面很少能够在没有近似的情况下传递到 NURBS(或任何其他) 系统中。

大多数 CAD/CAM/CAE 系统使用分段多项式或有理曲线和曲面来表示复杂几何。我们统称这些为样条曲线。样条曲线有许多不同类型；但无论它有什么华丽的名称，样条曲线 (曲面) 仅由若干段 (补丁) 组成，这些段以某种形式的连续性拼接在一起。为了简洁起见，我们在本节的其余部分将讨论限制于曲线，但类似的陈述和算法也适用于曲面。为了确定特定类型的样条曲线是否具有等效的 NURBS 表示 (反之亦然)，必须考虑三件事:

- 度数；
- 有理性；
- 连续性。

度数和有理性是简单的。度数可以精确提高，但不能降低；而多项式是有理函数，但反之则不成立。由于我们的系统 A 允许任意度数的有理曲线，因此在将几何从系统 B 引入系统 A 时在这方面没有问题。然而，如果系统 B 限制度数或仅允许非有理几何，那么在从系统 A 转到系统 B 时，必须对相应的曲线进行近似。

连续性更为复杂。设 $\mathbf{C}(u)$ 为由区间 $[u_{i-1},u_i]$ 上的线段 $\mathbf{C}_i(u)$, $i=1,\ldots,m$.$\mathbf{C}_i(u)$ 组成的样条曲线，其中 $u_0<u_1<\cdots<u_{m-1}<u_m$ 。我们称 $\{u_i\}$ 为断点；如果 $\mathbf{C}(u)$ 处于 B-spline 形式，则 $\{u_i\}$ 为不同的节点值。每对线段 $\mathbf{C}_i(u)$ 和 $\mathbf{C}_{i+1}(u)$ 以某种类型和顺序的连续性连接，而数据交换中的基本问题始终归结为接收系统是否能够表示与适当连续性连接的线段。我们所说的连续性类型是指参数连续性 (C) 与几何连续性 (G)；有关几何连续性和各种构造以具有 G 连续性的样条的更多细节，请参见第 9.3.1 节以及 [Bars89, 90; Hosc93] 和其中的参考文献。我们提醒读者:

- G^1 在 u_i 处的连续性意味着一阶导数向量 $\mathbf{C}_i'(u_i)$ 和 $\mathbf{C}_{i+1}'(u_i)$ 指向相同的方向，但可以具有不同的大小；
- 对于 B-spline，节点的重数表示基函数的连续性，这通常是曲线在均匀条件下的参数连续性；

space;

- 曲线段的重新参数化会改变其与邻近段的参数连续性，但不会改变其几何连续性。

由于系统 A 允许任意的节点间距和重数，它能够精确地、几何地和参数化地表示几乎所有至少是 C^0 连续的多项式和有理样条曲线。然而，从 A 转到 B 可能需要对段进行近似或重新参数化 (这不会改变整体曲线的几何形状)。例如，如果系统 B 使用有理 B 样条但不允许多个内部节点，则必须对具有双内部节点的 C^{p-2} 连续 B 样条曲线进行近似。如果系统 B 允许多个节点但要求不同的节点 (断点) 均匀分布，则系统 A 的所有曲线可以在几何上精确地传递到系统 B；然而，这需要重新参数化，并且在此过程中参数连续性会丧失。算法为:

1. 插入所有内部节点，直到它们的重数达到 p ，从而获得分段贝塞尔曲线；这将产生系统 B 中曲线的控制点；
2. 设

$$d=\frac{u_m-u_0}{m}$$

并定义系统 B 的节点向量为

$$U=\{\underbrace{u_0,\ldots,u_0}_{p+1},\underbrace{u_1,\ldots,u_1}_{p},\ldots,\underbrace{u_{m-1},\ldots,u_{m-1}}_{p},\underbrace{u_m,\ldots,u_m}_{p+1}\}$$

其中

$$u_i=u_0+i\cdot d\; i=1,\ldots,m-1$$

3. 应用节点移除 (至零容忍)，但所有断点的重数必须至少为一，以确保它们均匀分布。

注意，如果使用此算法将曲线从 A 传递到 B 然后再返回到 A ，则原始曲线和返回的曲线在 A 中在参数上和节点数量上是不同的。还要注意，尽管曲线在几何上是等价的，但如果它们在系统 A 中用于构造规则曲面 (例如)，则生成的曲面可能在几何上并不等价。

无论系统 B 中使用何种类型的样条，连续性问题由两个部分组成:

- 表示是否允许不均匀间隔的断点；
- 可能的连续性类型和阶数是什么？

我们指出，即使在没有全局断点概念的系统中，第一部分的问题也可以得到回答。例如，许多几何连续样条是通过局部方法构造的，使用每个段的局部参数 $t,t\in[0,1]$ 。这样的系统可能允许全局参数化，在这种情况下，全局断点 u_i 是可用的，并且很可能允许非均匀间距。在没有全局参数化概念的系统中，隐含了均匀间隔的全局断点。如前所述，重新参数化可以用于将非均匀间隔的断点转换为均匀间隔的断点；然而，在此过程中，参数连续性会发生变化。相反，重新参数化将均匀间隔的断点转换为非均匀间隔的断点在将 (几何上精确地) 连续样条转换为连续 B 样条 G^k 时是有用的 (例如，这在第 9.3.2 节，方程 [9.37]-[9.39] 中进行了)。关于第二部分，必须确定允许的最小连续性阶数。例如，基于五次 G^2 连续样条的系统可以表示所有五次 G^k 连续样条，对于 $k\geq 2$ ；然而，具有 $k<2$ 的五次 G^k 连续样条必须被近似。作为另一个例子，仅允许单个内部节点的 B 样条系统的曲线是 C^{p-1} 连续的。因此，所有连续性阶数都是可能的，但其度数必须至少比所需的最小连续性高一个 (即，最小连续性阶数可能取决于度数)。

我们现在总结系统 A 和 B 之间样条曲线的交换。所有至少是 C^0 连续的样条曲线都可以精确 (几何和参数上) 地从系统 B 传递到 A。对于 G 连续曲线，多个节点是必要的，但如果需要，可以使用重新参数化来实现 C 连续性并降低内部节点的重数。在某些类型的表面构造 (例如，规则表面) 中使用重新参数化曲线可以改变表面几何。一个典型的转换算法首先计算每个段的幂基或贝塞尔系数，然后应用第 6.6 节的技术来构建 NURBS 表示。

从系统 A 到系统 B 的转换可能更为复杂。算法 A12.2 给出了确定这种转换应如何进行的逻辑思维过程。将 p 次 NURBS 曲线表示为 $\mathbf{C}$ ，并假设系统 B 可以表示 k 次的样条，其中 $p_{\min}\leq k\leq p_{\max}$ 。

ALGORITHM A12.2

```
if (C is rational and System B only represents polynomials)
    {
    Approximate C ;
    return;
    }
if (p > p_max)
    {
    Approximate C ;
    return;
    }
if (p < p_min)
    {
    DegreeElevateCurve (C) ;  /* to degree p_min* /
    return;
    }
Let m be the minimum order continuity allowed in System B
    for a degree p curve.
Determine the minimum order, k , of continuity at any
    breakpoint of C .
if (k < m)
    {
```

Approximate C ;
return;
}
Extract the segments of **C** (knot insertion), and convert them
to the segment coefficients required by System B
(reparameterization may be necessary).
If applicable, load the global breakpoints into System B.

确定最小连续性值 m 和 k 需要详细说明。存在 C 连续性和 G 连续性，对于有理曲线，均匀空间中的连续性与欧几里得空间中的连续性是不同的。值 m 和 k 必须指代系统 B 的样条所基于的连续性类型。例如，假设系统 B 使用有理 G^2 连续样条。该样条基于欧几里得空间中的切线和曲率连续性，因此值 k 必须基于系统 A 的 NURBS 曲线在欧几里得空间中的 G 连续性。注意，示例 Ex7.2 中的二次完整圆在系统 B 中可以精确表示为 G^2 样条，尽管作为 NURBS 曲线，它在欧几里得空间中仅是 C^1 连续的，并且在均匀空间中甚至存在尖点。

我们以这样的观察结束本章:NURBS 形式不仅是一种强大而灵活的曲线和曲面设计方法，而且显然是今天最适合用于复杂曲线和曲面数据交换的表示方式。

第十三章　B-样条编程概念

1　引言

在前面的十二章中，我们介绍了使用 NURBS 建模的不同方面。这些章节基本上涵盖了两件事:(1) 基础数学，和 (2) 说明如何将工具转化为计算机代码的算法。尽管每章中呈现的伪代码相当详细，包含了将其转化为可工作例程所需的所有信息，但从算法草图到有用系统的过程仍然很漫长。如何设计一个有用的系统是一个非常困难的问题 (与许多研究人员认为这是一个次要的“实现问题”相反)。一个 NURBS 系统必须满足许多要求，才能在复杂的表面或实体建模器中发挥作用。在本章中，我们介绍了一些编程概念，我们相信这些概念有助于良好的编程实践。我们充分意识到编程风格和约定的多样性。因此，我们并不试图呈现最佳或最终的解决方案，因为根本不存在这样的解决方案。

在撰写本书的过程中，我们开发了一个全面的库 Nlib V2.0，采用 ANSI C 实现。该库包含实现书中讨论的每个工具所需的众多例程。在本章中，我们介绍该库的理念、架构和组织。在开发 Nlib V2.0 时，我们要求我们的例程遵循以下一般标准 [见 Stra92]:

- 工具性: 应使用可用工具来构建新的例程；
- 可移植性: 将软件移植到不同的软件和硬件平台应该是容易的；
- 可重用性: 程序应以能够重用代码部分的方式编写；
- 可测试性: 代码应保持一致和简单，以便测试和调试变得容易；
- 可靠性: 错误应以一致和优雅的方式处理；
- 可增强性: 代码必须易于理解，以便可以在不费力的情况下添加新功能；
- 可修复性: 定位错误应该相对容易；
- 一致性: 编程约定在整个库中应保持一致；
- 可读性: 程序必须易于阅读和理解。

我们认为这些标准对于最小化通常所称的“软件维护噩梦”是必要的。软件行业中的一种常见问题是“软件垃圾化”，即当一名员工离开公司时，他的代码被丢弃，因为没有人理解它，因此维护变得不可能。

除了这些一般标准外，我们还遵循了一些具体标准:

- 编程风格: 我们希望代码看起来与书中的数学非常相似，以便读者在跟随各个步骤时不会遇到太大困难；此外，我们遵循了 [Baly94] 中建议的“用户友好”编程风格；
- 可用性: 由于 NURBS 在许多应用领域非常流行，因此非专家应能够轻松使用该库构建各种类型的高级应用；
- 数值效率:NURBS 被选入许多标准的原因之一是其优良的数值特性；NURBS 评估与我们精心编写的其他数值代码混合在一起，以避免破坏整体数值效率；
- 基于对象的编程: 编程曲线和表面需要处理大量数据；为了避免携带这些数据并增强代码的理解，我们将各种实体分组为对象。

关于基于对象的编程 (OBP): 在处理 OBP 时，许多人自动想到诸如 C++ 之类的语言。我们没有用 C++ 实现 Nlib V2.0，仅仅是因为普通的 C 就足够好了，允许 C 和 C++ 程序员使用库例程。

在接下来的章节中，我们将讨论以下问题:(1) 数据类型和可移植性，(2) 数据结构，(3) 内存分配，(4) 错误处理，(5) 工具例程，(6) 算术运算，(7) 示例程序，(8) 一些额外的结构，以及 (9) 系统结构。这些问题往往比 NURBS 的数学更复杂。数学基本上是对或错；而软件则是一个没有边界的大灰色区域。

2　数据类型与可移植性

在本节中，我们将处理以下可移植性问题:

- 数据类型可移植性，例如，一个系统将整数表示为 32 位数字，而另一个系统仅使用 16 位；
- 操作可移植性，例如，算术运算在不同机器上产生不同的结果；
- 领域可移植性，例如，一个系统是一个主要在二维空间中执行操作的绘图系统，而另一个系统是一个需要在三维空间中进行计算的设计系统。

我们通过三种方式来处理这些问题:(1) 使用预定义的数据类型，避免显式使用编译器定义的数据类型，(2) 引入特殊的算术函数，处理所有二维、三维或四维领域 (有理或无理，二维或三维) 中的所有算术运算，以及 (3) 定义无量纲的对象。要解决可移植性问题，只需在其中一个包含文件中更改几行代码，或修复一些算术例程，或重新定义一些对象。其余系统可以包含数百万行代码，保持不变。

在 Nlib V2.0 中，我们使用了几种预定义的数据类型。一些示例包括:

```
typedef int INDEX;
typedef short FLAG;
typedef int INTEGER;
typedef double REAL;
typedef char * STRING;
typedef short DEGREE;
typedef double PARAMETER;
```

INDEX 数据类型用于声明作为不同维度数组的最高索引的变量。例如，如果最高索引从不超过 32,767(16 位)，那么当前定义是足够的。然而，如果需要更大维度的数组，则必须使用该定义。

```
typedef long INDEX;
```

对于 FLAG 数据类型 (例如，用于在 switch 语句中“标记”或选择不同的情况) 和 INTEGER 数据类型也适用类似的论点。如果系统仅使用 8 位整数，则唯一需要的更改是将 INTEGER 定义为 short 而不是 int。REAL 数据类型默认定义为 double，如果单精度满足要求或系统的其余部分使用单精度，则可以更改为 float。STRING 数据类型的引入是出于便利和概念原因。

DEGREE 和 PARAMETER 数据类型的解释与 INDEX 数据类型类似，即曲线或表面的度数远低于用作变量的整数。因此，短整数是相当足够的。将 DEGREE 定义为新数据类型也表明它与普通整数不同；它属于共同定义曲线或表面的参数。PARAMETER 类型也是如此，可以是 double 或 float，具体取决于应用程序或当前系统的浮点计算。

浮点计算中最困难的任务之一是处理数值不精确性。几乎不可能不使用容差来处理点重合或测量两个浮点数的相等性。在 Nlib V2.0 中，我们使用编译器定义的以下常量:

```
#define BIGD DBL_MAX
#define SMAD DBL_MIN
#define BIGI INT_MAX
#define SMAI INT_MIN
#define DEPS DBL_EPSILON
```

BIGD 和 SMAD 分别是最大的和最小的双精度数。ANSI C 标准规定它们必须在 float.h 中定义。它还要求 BIGD 至少为 10^{37} 或更大，SMAD 至少为 10^{-37} 或更小。双精度 epsilon DEPS 的要求至少为 10^{-9} 或更小。

整数变量的限制存储在 limits.h 中。一些典型的范围是 int 的 -32,768 到 32,767，long 的 -2,147,483,648 到 2,147,483,647。无符号长整型的范围可达 4,294,967,295。使用这些常量可以确保计算的一致性，无论使用何种硬件平台。

通过对对象进行特殊算术函数操作，确保了操作和域的可移植性。后续章节中给出了示例。

3　数据结构

如第 13.1 节所述，我们打算使用基于对象的概念，并遵循前几章的数学符号和概念。作为一个例子，在曲线评估例程中，我们写道

```
FLAG N_evncur( CURVE *cur, PARAMETER u, POINT *C )
```

或者，给定一个曲线和一个参数，计算在给定参数值下的曲线上的一个点。这种编程方式的好处在于用户无需了解不同实体 (曲线、参数和点) 在内部是如何表示的。它们被视为由更小的组件构建的对象。要定义一条曲线 (或者我们可以说，定义一个曲线对象)，我们需要存储这些实体:

- n : 控制点的最高索引；
- $\mathbf{P}_0^w, \ldots, \mathbf{P}_n^w$: 控制点；
- p : 次数；
- m : 节点的最高索引；
- $u_0, \ldots, u_m$: 节点。

尽管这些数据可以直接传递给每个 NURBS 例程，但对于程序员来说，这相当不方便且不自然，因为他们在不同的抽象层次上思考。例如，在处理用于交互设计的曲线时，他们在不考虑控制点和节点数量的情况下与曲线进行工作。或者，例如，在处理曲线与曲线的交点时，如果需要检查边界框的重叠，他们可能会将思维降到控制多边形的层次。出于这些以及许多其他类似原因，Nlib V2.0 将每个几何实体视为由更简单的组件构建的对象。

在设计一个大型结构时，首先从基本数据类型创建简单的构建块。曲线对象的基本元素之一是控制点。它可以是有理的或非有理的，二维或三维，即它需要包含两个、三个或四个坐标。此外，它在 B 样条计算中作为控制点和在简单几何操作中使用的欧几里得点 (例如计算距离、边界框等) 发挥双重作用。第三个要求是能够将此实体称为 $P[i]$ ，就像书中提到的其数学抽象一样。为了简化问题并满足基于对象的概念，我们将控制点与“常规”欧几里得点分开，就像我们在书中所做的那样，用 $\mathbf{P}$ 表示欧几里得点，用 $\mathbf{P}^w$ 表示可以加权的控制点。因此，点的定义为 (见图 13.1):

```
typedef struct point
{
    REAL x,
         y,
         z;
} POINT;
typedef struct cpoint
{
    REAL x,
         y,
         z,
         w;
} CPOINT;
```

P :

max width=

max width=

Pw:		y	**Z**	

图 13.1. 点和控制点数据结构。

这些定义的一个吸引人的特点是它们使编程相当简洁。为此付出的代价是内存的浪费，例如，如果曲线是一个二维非有理曲线，那么用于存储控制点的内存的一半是浪费的。虽然这在过去确实是一个问题，但代码的理解和维护已变得比速度和内存更为重要。然而，有一件事可以做，以限制算术操作仅限于二维或三维，以防曲线是非有理或平面的。第三个和/或第四个分量可以设置为一个特殊值。*NlibV2.0* 使用如下:

```
#define _NOZ +BIGD
#define NOW -BIGD
```

如果第三个分量设置为 *NOZ* (没有 z)，那么对 z 组件不执行任何操作，类似地，在 x NOW 的情况下，忽略 w 组件。

在定义了点和控制点之后，现在让我们构建更复杂的对象。下一个抽象层次包含控制多边形和节点向量对象，定义如下 (见图 13.2 和 13.3):

```
typedef struct cpolygon
{
    INDEX n;
    CPOINT *Pw;
} CPOLYGON;
typedef struct knotvector
{
    INDEX m;
    REAL *U;
} KNOTVECTOR;
pol:
```

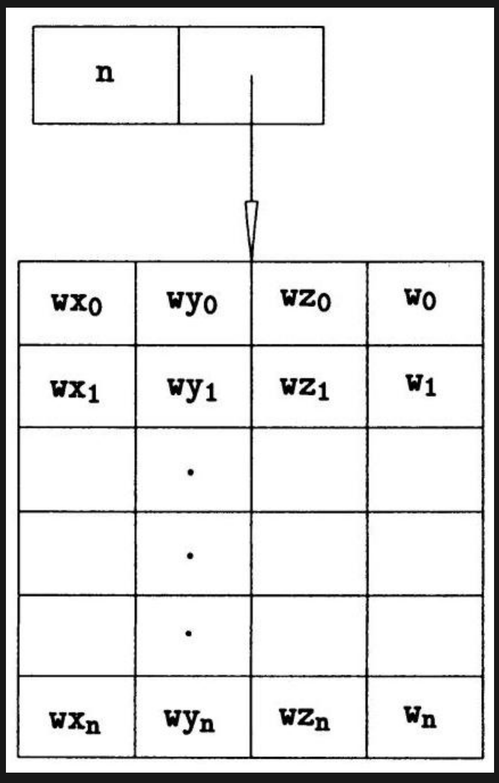


图 13.2. 控制多边形数据结构。

换句话说，控制多边形对象只是一个结构，包含最高索引和指向控制点数组的指针。节点向量也有一个最高索引字段和指向节点数组的指针。这些结构包含了最低限度的数据。可以根据特殊需要扩展它们，例如，控制多边形结构可以存储关于边界框的信息，或者节点向量结构可以存储一个标志，以指示它是均匀的还是非均匀的。现在，给定一个控制多边形、一个节点向量和一个指定的度数，可以从这些组成部分构建一条曲线，如下所示 (见图 13.4):

```
typedef struct curve
{
    CPOLYGON *pol;
    DEGREE p;
    KNOTVECTOR *knt;
} CURVE;
```

也就是说，一个曲线对象由一个多边形指针、一个度数和一个节点向量指针组成。这个定义的美在于，各个组成部分可以很容易地提取以进行进一步处理。例如，如果需要计算曲线的包围盒，在该例程中会提取多边形子对象并传递给处理包围盒计算的另一个例程。同样，如果需要查找一个参数所在的节点区间，节点向量对象会从曲线中分离出来并传递给一个例程。假设当前例程接收到一个曲线指针，编程相对简单。... CPOLYGON *pol; KNOTVECTOR *knt;

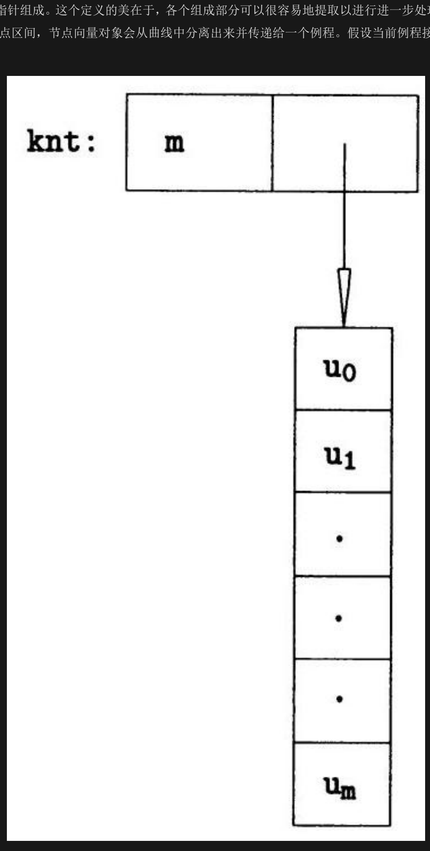


图 13.3. 节点向量数据结构。

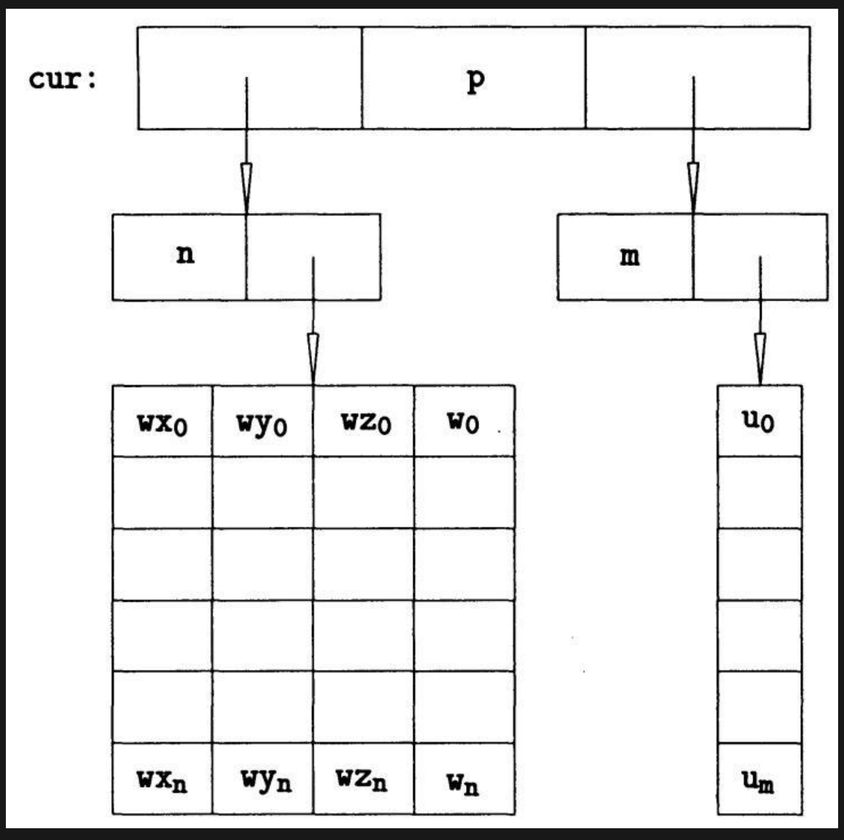


图 13.4. 曲线数据结构。

```
    ...
pol = cur->pol;
knt = cur->knt;
    ...
N_polbox(pol, ...);
N_findsp(knt,...);
```

换句话说，简单的指针赋值允许程序降到更低的抽象层次，执行必要的操作，并将结果返回到调用例程的层次。

表面对象的定义非常相似。它需要一个控制网、两个度数和两个节点向量。控制网定义为 (见图 13.5):

```
typedef struct cnet
{
    INDEX n,
          m;
    CPOINT **Pw;
} CNET;
```

它包含两个方向上的最高索引和一个指向指针的指针，即一个指向指针数组的指针，该数组指向每列控制点的第一个元素。当写作 $P[i][j]$ 时，$P[i]$ 指的是指针数组中指向第 i 列开头的第 i 个指针。索引 [j]

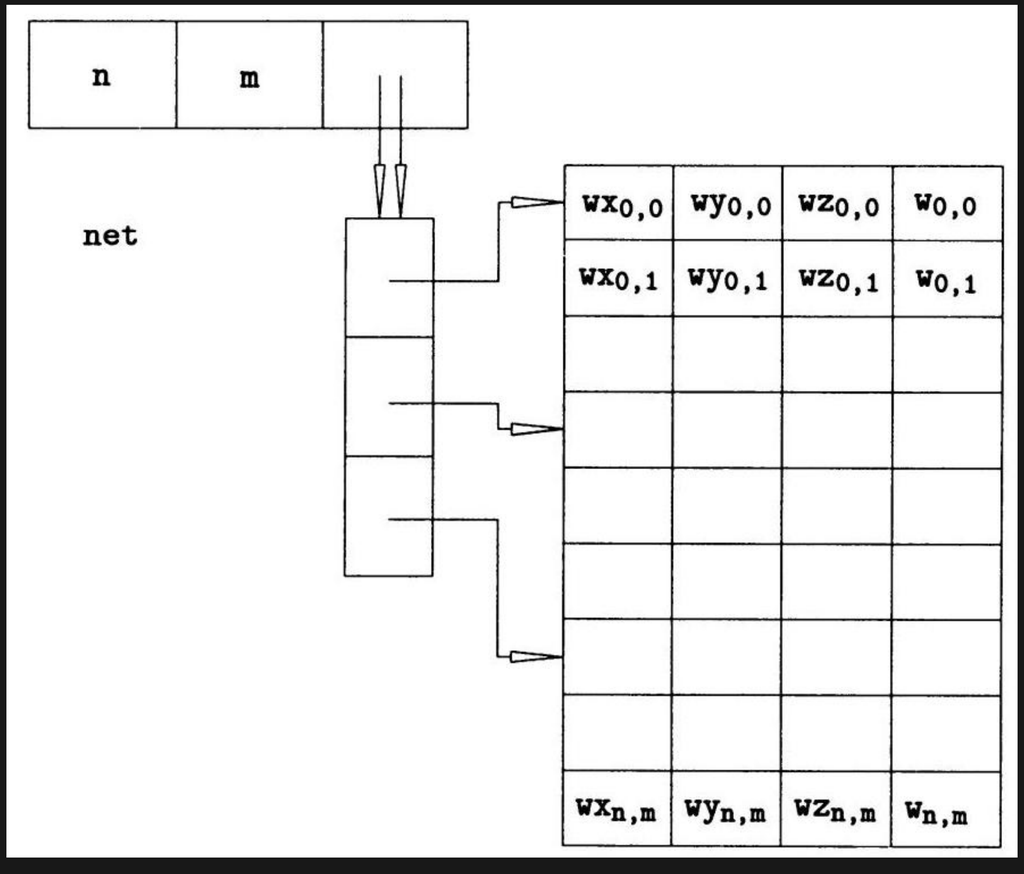


图 13.5. 控制网数据结构。

是该列中的第 j 个偏移量。将控制网、度数和两个节点向量结合在一起，得到一个定义为 (见图 13.6) 的表面:

```
typedef struct surface
{
    CNET *net;
    DEGREE p,
           q;
    KNOTVECTOR *knu,
               *krv;
} SURFACE;
```

再次，控制网和节点向量被轻松提取以执行各种操作。如果需要对同一曲线或表面执行多个操作，这些定义变得相当自然且非常经济；只需传递指向不同结构的指针即可。

4　内存分配

C 语言最强大的工具之一是其动态分配和释放内存的能力。然而，这种能力是有代价的: 不当的内存分配、使用或释放可能导致致命错误。因此，Nlib V2.0 对内存分配和释放有非常严格的规则。最重要的一条是，任何例程都不允许分配和释放内存，除非是专门编写的内存处理程序。

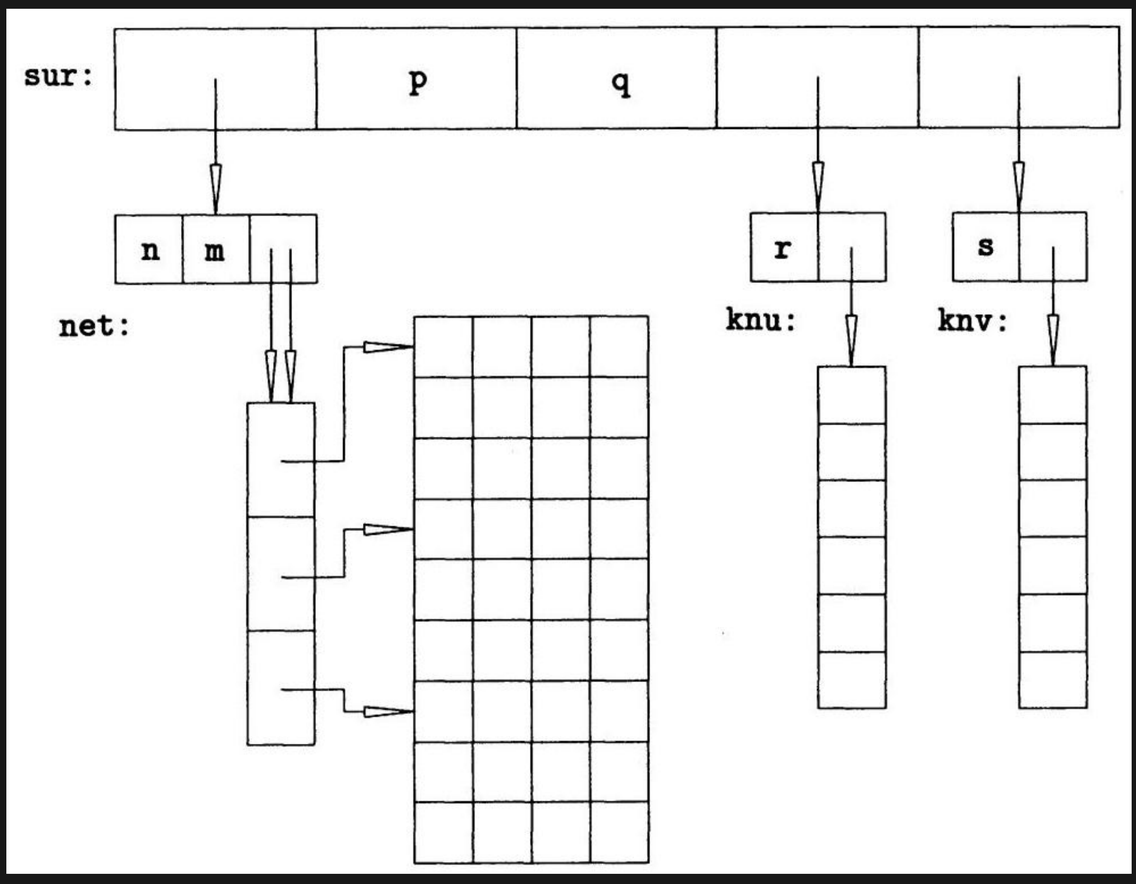


图 13.6. 表面数据结构。

在例程中分配的所有内存且未返回给调用例程的内存必须在返回之前被释放。这个事实，加上适当的错误处理，可能导致相当混乱的代码，正如这个代码段所示。

```
x = (double *) malloc ( n1*sizeof(double) );
if ( x == NULL ) return ( error ) ;
y = (double *) malloc ( n2*sizeof(double) );
if ( y == NULL )
{
    free( x );
    return ( error ) ;
}
z = (double *) malloc ( n3*sizeof(double) );
if ( z == NULL )
{
    free( x );
    free( y );
    return ( error ) ;
}
```

如果需要多个分配，这个过程可能会迅速变得相当复杂。可以很容易看出，$n+1$ 内存分配需要 $n\,(n+1)\,/2$ 行代码来处理释放，以应对可能的错误，例如，要求 21 个数组的代码包含 210 行释放语句，仅用于处理动态内存分配可能失败的情况。

为了使事情更易于管理，Nlib V2.0 采用了一个三步过程来处理内存分配和释放:

- 在进入新程序时，初始化一个空的内存栈；
- 如果需要内存，则调用特殊例程来分配所需的内存，并在适当的栈上保存内存指针 (见图 13.7)；
- 离开程序时，遍历内存栈，并释放由保存的指针指向的内存。

正如后面部分的编程示例所示，这种类型的内存管理相当简洁，隐藏了所有不必要的细节，并确保所有分配的内存都被正确释放。现在让我们更详细地检查这三个步骤。

每个 NURBS 例程的结构使用这个简单的示例进行了概述:

```
...
FLAG error = 0 ;
    INDEX n, m;
    CPOINT **Pw;
```

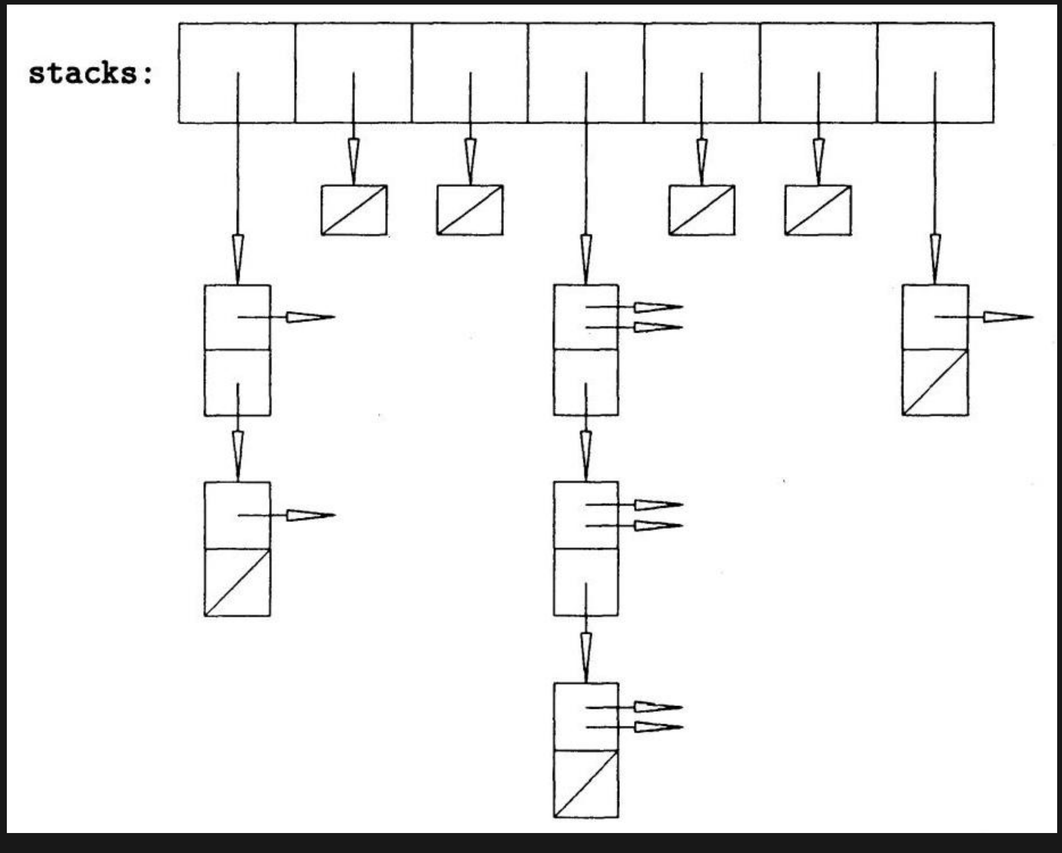


图 13.7. 内存堆栈。

```
        STACKS S;
        ...
```

```
        N_inurbs(&S);
            ...
        Pw = S_getc2d(n, m,&S);
            if ( Pw == NULL ) { error = 1; goto EXIT; }
            ...
        Pw[i][j] = ...;
            ...
        EXIT:
N_enurbs(&S);
return ( error ) ;
```

每个例程都有自己的内存堆栈，在 nurbs.h 中定义如下 (见图 13.7):

```
typedef struct stacks
{
    I1DNODE *i1d;
    I2DNODE *i2d;
    R1DNODE *r1d;
    R2DNODE *r2d;
    ...
    KNTNODE *knt;
} STACKS;
```

换句话说，它是一个指向链表节点的指针结构，这些节点存储不同类型的内存指针——一维或二维的 INTEGER 和 REAL 数组、POINT 数组等。当进入一个例程时，为该结构分配内存，并且指针由 N_inurbs( ) 例程设置为 NULL。为了分配内存，程序 (1) 声明适当的内存指针 (在前面的示例中为 **Pw)，并 (2) 调用内存例程 (S_getc2d( ))。内存例程然后分配所需的内存并将指针保存在堆栈上。作为一个例子，让我们看看 S_getc2d( ) 如何为二维控制点数组分配内存 (该例程中使用的错误处理在第 3.5 节中讨论)。

```
#include "nurbs.h"
static STRING rname = "S_GETC2D";
CPOINT **S_getc2d( INDEX n, INDEX m, STACKS *S )
{
    INDEX k, l;
    CPOINT **Pw, *Qw;
内存分配 605
C1DNODE *c1d;
C2DNODE *c2d;
/* Allocate memory */
Pw = (CPOINT **) malloc ( (n+1)*sizeof(CPOINT *) );
if ( Pw == NULL )
{
      E_seterr(MEM_ERR, rname);
      return NULL;
}
Qw = (CPOINT *) malloc ( (n+1)*(m+1)*sizeof(CPOINT));
if ( Qw == NULL )
{
    E_seterr(MEM_ERR, rname);
    free( Pw );
    return NULL;
}
/* Make pointer assignments */
l = 0 ;
for ( k = 0 ; k <= n ; k + + )
{
    Pw [k] = & Qw [l] ;
    l = l+m+1;
}
/* Put pointers on memory stacks * /
c1d = (C1DNODE *) malloc ( sizeof(C1DNODE) );
if ( c1d == NULL )
{
    E_seterr(MEM_ERR, rname);
    free( Pw ); free( Qw );
    return NULL;
}
c1d->ptr = Qw;
c1d->next = S->c1d;
S->c1d = c1d;
c2d = (C2DNODE *) malloc ( sizeof(C2DNODE) );
if ( c2d == NULL )
{
    E_seterr(MEM_ERR, rname);
    free( Pw );
606 B-spline 编程概念
    return NULL;
}
c2d->ptr = Pw;
c2d->next = S->c2d;
S− > c2 d = c2 d ;
/* Exit */
return Pw ;
}
```

节点 C1DNODE 和 C2DNODE 在 geometry.h 中定义为

```
typedef struct c1dnode
{
    CPOINT *ptr;
    struct c1dnode *next;
} C1DNODE;
typedef struct c2dnode
{
    CPOINT **ptr;
    struct c2dnode *next;
} C2DNODE;
```

其他类型的内存指针也使用类似的节点。

一旦内存被分配，它可以在程序中根据需要使用。最后，所有分配的内存由 N_enurbs( ) 例程释放，该例程遍历不同的内存堆栈，释放分配的内存，并销毁链表。其结构如下:

```
#include "nurbs.h"
VOID N_enurbs( STACKS *S )
{
    I1DNODE *i1p;
    ...
    KNTNODE *krp;
    /* Free integer arrays */
    while ( S->i1d != NULL )
    {
        i1p = S->i1d;
        S->i1d = i1p->next;
        free( i1p->ptr );
        free( i1p );
    }
...
/* Free knot vectors */
while ( S->knt ! = NULL )
{
    knp = S− > knt ;
    S->knt = knp->next;
    free( knp->ptr );
    free( knp );
}
```

}

这种内存管理方式提供了一种相对简洁的方法来处理内存分配和释放这一困难任务。它使程序员免于处理令人分心的细节，并记住释放所有动态分配的内存。在目前的形式下，该方法不允许在运行程序的中间进行释放。如果这在特定应用中造成问题，可以编写简单的释放例程来释放内存并从内存栈中消除节点。

关于 N_inurbs( ) 和 N_enurbs( ) 例程的说明: 在之前的讨论中，这些例程初始化和终止内存栈。然而，这些例程也可以用于初始化和反初始化程序员希望在整个 NURBS 例程中使用的其他全局参数。这是每个 NURBS 例程都以 N_inurbs( ) 开始并以 N_enurbs( ) 结束的众多原因之一，即使不需要动态内存分配。这个概念类似于为车载电话预先布线的汽车。司机可能从来不想要电话，但如果他需要，安装就变成了一个五分钟 (干净的) 工作。Nlib V2.0 中的 NURBS 例程为动态内存分配和初始化“预先布线”，因此添加新功能变得简单，并且不需要改变程序的整体结构。

5　错误控制

错误处理是设计系统中最关键的问题之一。每个例程都可能出现错误，而调用例程必须以合理的方式处理这些错误。基本上有两种类型的错误处理:

- 直接控制: 在发生错误时打印消息；如果错误是致命的，则终止该过程；
- 间接控制: 每个例程向调用例程返回一个错误标志，该标志决定如何处理错误以及如何将此信息传递给更高层的例程。

在像编译器这样的系统中，直接控制是合适的，但在 NURBS 系统中则不可接受。例如，一个几何例程可以返回一个错误标志，指示给定的线条无法相交，因为表示线条的输入数据包含损坏的信息 (例如，零长度的线)。由于这个错误可能可以从调用例程中纠正，因此可以成功完成该过程，完全不可接受的是终止整个会话并打印错误信息。

NURBS 例程需要在后台处理错误，通过将错误标志传回每个调用例程来实现。这可以通过多种不同的方式完成。一种复杂的方法是使用错误堆栈。如果一个例程发现了错误，它会被放入堆栈中，并将控制权返回给调用例程。该例程要么处理错误，要么将自己的错误放入堆栈中，并将控制权返回给更高层的例程。在每个更高层次上，调用例程会接收到一个错误堆栈 (见图 13.8)，显示错误历史记录，在每个层次上，例程可以纠正错误 (如果可能)，或者将堆栈 (其自己的错误在顶部) 传递到更高一层。最终，高于 NURBS 库的例程将处理该错误。让我们考虑一个简单的例子。第 9 章讨论的数据简化算法使用以下简单的调用层次结构:

Data reduction;
　Interpolate with given knot vector;
　Remove knots;
　Compute degree elevated knot vector;
　Least-squares approximation with given knot vector;
　　LU decomposition;

如果 LU 分解在最深层次失败，则无法执行最小二乘近似。数据简化例程可以选择重新启动整个过程，使用更高阶的插值曲线 (这正是实际实现所做的)，而不是终止整个过程。

尽管错误堆叠是一种复杂的技术，但它需要一些编程来响应每个级别的不同错误。在 Nlib V2.0 中没有采用这种技术，主要有两个原因:(1) 每个 NURBS 例程只调用少数其他例程，即调用栈相对较小，和 (2) 调用例程

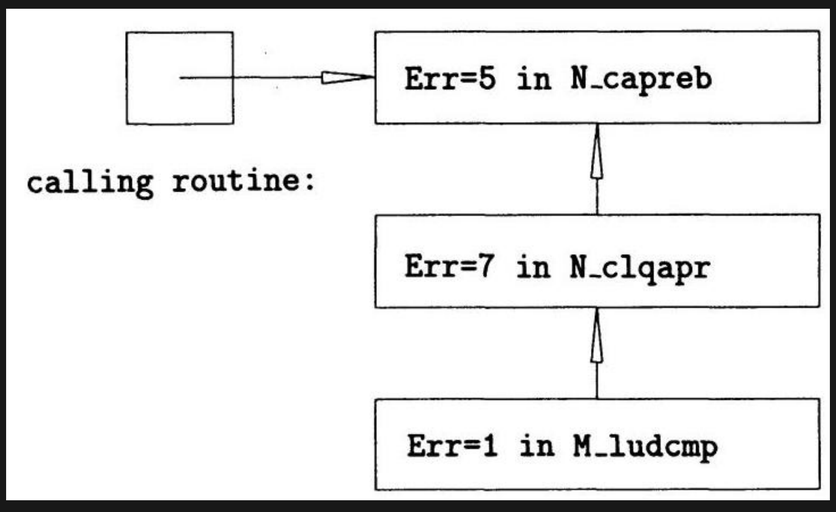


图 13.8. 错误堆栈。

检查大多数错误，因此只有少数错误仍需在较低级别的例程中被检测到。因此，对于 Nlib V2.0 来说，传递一个错误标志似乎是合理的，并使每个例程负责处理此错误。错误标志是一个简单的结构，在 datastr.h 中定义如下:

typedef struct enode

$$
\begin{cases}
u_{0} = 0, \\
u_{1} = 0, \\
u_{2} = 0, \\
u_{3} = 0, \\
u_{4} = 0, \\
u_{5} = 0, \\
u_{6} = 0, \\
u_{7} = 0, \\
u_{8} = 0, \\
u_{9} = 0, \\
u_{10} = 0, \\
u_{11} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{14} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{14} = 0, \\
u_{12} = 0, \\
u_{13} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{24} = 0, \\
u_{34} = 0, \\
u_{34} = 0, \\
u_{34

也就是说，该结构包含一个错误编号和检测到错误的例程名称。最常见的错误被编号并在 datastr.h 中定义。一些示例包括:

```
#define CUR_ERR 1
  #define SUR_ERR 2
#define DEG_ERR 3
#define KNT_ERR 4
  #define WEI_ERR 5
      ...
```

#define MEM_ERR 15

```
...
```

其中 CUR_ERR 是曲线定义错误；KNT_ERR 是节点向量错误，例如递减节点；MEM_ERR 是内存分配错误，等等；为了方便，可以创建一个错误文件，其中详细解释所有这些错误。使用这些错误编号，需要一个全局错误变量。Nlib V2.0 在 globals.h 中定义并初始化错误变量 NERR，如下所示:

```
extern volatile ENODE NERR = {0, " " } ;
```

一旦检测到错误，NERR 通过错误例程 E_seterr( ) 接收一个新值:

```
#include "nurbs.h"
extern volatile ENODE NERR;
VOID E_seterr( INTEGER err, STRING name )
{
  NERR.eno = err;
  NERR.fna = name;
}
```

Nlib V2.0 具有一套错误例程，用于检查所有可能的错误并将 NERR 设置为适当的值。我们在这里提供两个示例：第一个检查节点是否是非递减的，而第二个执行完整的曲线检查，即检查曲线定义的一致性、权重和节点向量。

```
#include "nurbs.h"
FLAG E_kntdef( KNOTVECTOR *knt, STRING rname )
{
  FLAG error = 0 ;
  INDEX i, m;
  REAL *U;
  /* Convert to local notation */
  m = knt->m;
  U = knt− > U ;
  /* Check knots */
  for ( i = 0; i < m; i++ )
  {
      if ( U[i] > U[i+1] )
      {
        E_seterr(KNT_ERR, rname);
        error = 1 ;
        break;
      }
  }
  /* Exit */
  return ( error ) ;
}
```

换句话说，如果未发现错误，例程返回 0 ，否则返回 1 并将 NERR 设置为适当的值。

在许多应用中，曲线数据是从文件中读取的，例如从 IGES 文件。在调用任何 NURBS 例程之前，确保文件包含正确的信息是明智的，即执行完整的曲线 (表面) 检查是有用的 (也是必要的)。这个检查的一个额外积极特征是，当曲线和表面被传递给 NURBS 例程时，不再需要进一步的曲线 (表面) 错误检查。在考虑完整曲线检查例程之前，还讨论了两个错误例程:(1) 检查曲线定义的一致性，以及 (2) 检查权重。首先考虑曲线定义检查例程。

```
#include "nurbs.h"
FLAG E_curdef ( CURVE *cur, STRING rname )
{
  FLAG olddim, oldrat, newdim, newrat, error = 0;
  INDEX i, n, m;
错误控制 611
    DEGREE p;
    CPOINT *Pw;
    /* Convert to local notation */
  n = cur->pol->n;
  Pw = cur->pol->Pw;
  p = cur->p;
    m = cur->knt->m;
  /* Check definition */
    if ((n + p + 1)! = m)
    {
      E_seterr(CUR_ERR, rname);
      error = 1;
      goto EXIT;
  }
  /* Check for consistency */
    if ( Pw[0].z == NOZ ) olddim = 2; else olddim = 3;
    if ( Pw[0].w == NOW ) oldrat = 0; else oldrat = 1;
    for ( i=1; i<=n; i++ )
    {
      if ( Pw[i].z == NOZ ) newdim = 2; else newdim = 3;
      if ( newdim != olddim )
      {
        E_seterr(CUR_ERR, rname);
        error = 1;
        break;
      }
      if ( Pw[i].w == NOW ) newrat = 0; else newrat = 1;
      if ( newrat != oldrat )
      {
        E_seterr(CUR_ERR, rname);
        error = 1;
        break;
      }
  }
  /* Exit */
  EXIT:
  return ( error ) ;
}
```

612 B-spline Programming Concepts

The weight check routine is:

```
#include "nurbs.h"
FLAG E_curwei( CURVE *cur, STRING rname )
{
  FLAG error = 0 ;
  INDEX i, n;
  CPOINT *Pw;
  /* Convert to local notation */
  n = cur->pol->n;
  Pw = cur->pol->Pw;
  /* Check if curve is rational */
  if ( Pw[0] . w == _NOW ) return ( 0 ) ;
  /* Now check the weights */
  for ( i = 0; i <=n; i++ )
  {
    if ( Pw[i].w <= 0.0 )
    {
        E_seterr(WEI_ERR, rname);
        error = 1;
```

```
                    break;
                }
            }
            /* Exit */
            return ( error ) ;
        }
```

As a practical matter, application programmers might want to set a limit to the largest permissible weight and check for it in this routine. Large weights create very bad parametrizations, which in turn can result in unacceptable surfaces constructed from curves.

The input checking routine is:

```
#include "nurbs.h"
    FLAG E_curinp( CURVE *cur, STRING rname )
{
实用例程 613
标记错误;
KNOTVECTOR *knt;
/* 转换为本地表示法 */
knt = cur->knt;
/* 检查曲线定义 */
error = E_curdef(cur, rname);
if ( error > 0 ) return ( 1 ) ;
/* 检查曲线权重 */
error = E_curwei(cur, rname);
if ( error > 0 ) return ( 1 ) ;
/* 检查节点向量 */
error = E_kntdef(knt, rname);
if ( error > 0 ) return ( 1 ) ;
/* 退出 */
return(0);
}
```

每个 NURBS 例程原则上可以单独处理错误检查。然而，使用错误例程使编程变得更加简洁。程序员可以专注于手头的任务，例如曲线细化，而诸如错误检查和内存分配等杂项任务则在概念层面上处理。

6 实用程序

实用例程帮助程序员创建特殊结构或将一种类型的输入转换为另一种类型。这类实用程序的种类繁多，但必须在某处划定界限。在 Nlib V2.0 中，存在几十个支持特殊结构构造的例程。

创建曲线结构可以通过多种不同方式完成，具体取决于在什么级别上可用什么类型的信息。一个简单的例子是:

```
#include "nurbs.h"
VOID U_makcu1( CURVE *cur, CPOLYGON *pol, DEGREE p,
    KNOTVECTOR *knt )
{
614 B-spline 编程概念
cur->pol = pol;
cur->p = p;
cur->knt = knt;
}
```

这可能是最简单的工具类型，负责将现有结构的指针分配给曲线结构的元素。多边形和节点向量也可以通过工具例程创建。一个简单的例子是:

```
#include "nurbs.h"
FLAG U_makp12( CPOLYGON *pol, REAL *wx, REAL *wy, REAL *wz,
                    REAL *w, INDEX n, STACKS *S )
{
    INDEX i;
    CPOINT *Pw;
    /* Allocate memory for point array */
    Pw = S_getc1d(n, S);
    if ( Pw == NULL ) return (1);
    /* Fill in point array */
    for ( i = 0; i <=n; i++ )
    {
      Pw[i].x = wx[i];
      Pw[i].y = wy[i];
        if ( wz != NULL )
          Pw[i].z = wz[i];
        else
          Pw[i].z = _NOZ;
        if ( w != NULL )
          Pw[i].w = w[i];
        else
          Pw[i].w = _NOW;
    }
    /* Make pointer assignments */
    pol − > n = n ;
    pol->Pw = Pw;
    /* Exit */
    return (0);
}
```

该例程不仅创建了一个多边形结构 (在调用例程中分配内存以存储该结构)，还将传统的 (wx, wy, wz, w) 类型输入转换为基于对象的点输入。同时，它还负责为非有理或平面曲线正确设置 z 和 w 组件。我们展示了一个类似的例程，它从最小的片段创建曲线:

```
#include "nurbs.h"
FLAG U_makcu3( CURVE *cur, REAL *wx, REAL *wy, REAL *wz,
                    REAL *w, INDEX n, DEGREE p, REAL *U, INDEX m,
                    STACKS *S )
{
    CPOINT *Pw;
    CPOLYGON *pol;
    KNOTVECTOR *knt;
    /* Allocate memory */
    Pw = S − getcpa (wx, wy, wz, w, n, S) ;
    if ( Pw == NULL ) return (1);
    pol = S_getpol(S);
    if ( pol == NULL ) return (1);
    knt = S_getknt(S);
    if ( knt == NULL ) return (1);
    /* Make pointer assignments */
    pol − > n = n ;
pol->Pw = Pw;
knt->m = m;
knt->U = U;
cur->pol = pol;
cur->p = p;
cur->knt = knt;
/* 退出 */
return (0);
}
```

注意内存例程 S_getcpa( )、S_getpol( ) 和 S_getknt( ) 是如何与这个工具集成得如此良好的。没有进行显式的分配和释放，但通过全局错误标志 NERR 进行了适当的错误处理。这个例程也是一个很好的示例，说明了如何在更高级的例程中处理错误。如果在之前的三个内存例程中的任何一个检测到错误，NERR 将被设置并返回控制。U_makcu3 通过将控制返回到更高一级来处理错误。由于内存分配在内存例程和工具例程中都是问题，因此不需要对全局 NERR 进行新的设置。

作为最后一个示例，让我们考虑一个表面工具例程，该例程根据控制点和节点向量信息创建一个表面。

```
#include "nurbs.h"
FLAG U_maksu2( SURFACE *sur, CPOINT **Pw, INDEX n, INDEX m,
DEGREE P, DEGREE q, REAL *U, REAL *V, INDEX r,
INDEX S, STACKS *S )
{
CNET *net;
```

```
KNOTVECTOR *knu, *knv;
/* 为控制网分配内存 */
/* 和节点向量结构 * */
net = S_getnet(S);
if ( net == NULL ) return ( 1 ) ;
knu = S_getknt(S);
if ( knu == NULL ) return ( 1 ) ;
knv = S_getknt(S);
if ( knv == NULL ) return ( 1 ) ;
/* 进行指针赋值 */
net->n = n;
net->m = m;
net->Pw = Pw;
knu->m = r;
knu->U = U;
knv->m = s;
knv->U = V;
sur->net = net;
sur->p = p;
sur->q = q;
sur->knu = knu;
sur->knv = knv;
/* 退出 */
return (0);
}
```

7 算术例程

对 NURBS 曲线和曲面的操作，例如评估或节点插入，需要对控制点进行算术运算。由于控制点被视为对象，因此需要特殊的算术例程来执行诸如凸组合 (角落切割)、求和或投影到欧几里得空间等任务。除了对对象进行算术运算外，这些例程还有助于理解程序，并使得将 NURBS 例程移植到不同环境成为可能。例如，假设一个基于有理曲线和曲面的三维设计系统将被用作仅采用非有理曲线的二维制图系统。由于控制点对象是无维的，因此只需更改其定义，并修改少量算术例程。系统的其余部分，可能包含数百万行代码，保持不变。

我们展示了在 Nlib V2.0 中使用的一些算术例程的示例。第一个示例是初始化例程。计算曲线或表面上的一个点需要求和，这要求将和初始化为零。

```
#include "nurbs.h"
VOID A_initcp( CPOINT Aw, CPOINT *Bw )
{
        Bw->x = Aw.x;
Bw->y = Aw.y;
        Bw->z = Aw.z;
        Bw->w = Aw.w;
}
```

该例程妥善处理不同类型的控制点，无论是有理的还是非有理的，二维的还是三维的，通过 Aw 的第三和第四坐标。

下一个示例是一个更新给定控制点的例程，该控制点再次用于计算混合控制点的和。

```
#include "nurbs.h"
VOID A_updcpt( REAL alpha, CPOINT Aw, CPOINT *Bw )
{
    Bw->x = Bw->x + alpha*Aw.x;
    Bw->y = Bw->y + alpha*Aw.y;
    if ( Aw.z != _NOZ )
        Bw->z = Bw->z + alpha*Aw.z;
    else
        Bw->z = _NOZ;
    if ( Aw.w != _NOW )
        Bw->w = Bw->w + alpha*Aw.w;
    else
        Bw->w = _NOW;
}
```

作为最后一个示例，我们考虑投影例程，该例程将有理点映射到欧几里得空间。

```
        #include "nurbs.h"
        VOID A_euclid( CPOINT Pw, POINT *P )
        {
          if ( Pw.w !== _NOW )
          {
            P->x = Pw.x/Pw.w;
            P->y = Pw.y/Pw.w;
            if ( Pw.z != _NOZ )
              P->z = Pw.z/Pw.w;
            else
              P− > z = 0.0 ;
          }
          else
          {
            P− > x = Pw.x ;
            P− > y = Pw.y ;
            if ( Pw.z != _NOZ )
              P− > z = Pw · z ;
          else
              P− > z = 0.0 ;
          }
    }
```

These routines are used in the next section to illustrate two NURBS routines, curve and surface evaluations.

8 示例程序

在本节中，我们展示两个完整的程序，一个用于曲线评估，另一个用于表面评估。这些例程展示了 NURBS 程序的结构，以及不同的实用例程如何与更高级别的 NURBS 例程集成。首先考虑曲线评估器。

```
#include "nurbs.h"
static STRING rname = "N_EVNCUR";
extern CPOINT CZERO;
FLAG N_evncur( CURVE *cur, PARAMETER u, POINT *C )
{
    FLAG error = 0 ;
    INDEX i, j, k;
    DEGREE p;
    REAL *N;
    KNOTVECTOR *knt;
    CPOINT *Pw, Cw;
        STACKS S;
        /* Start NURBS environment */
        N_inurbs(&S);
        /* Get local notation */
        Pw = cur->pol->Pw;
        p = cur->p;
        knt = cur->knt;
        /* Check parameter */
        error = E_parval(knt, u, rname);
        if ( error == 1 ) goto EXIT;
        /* Compute non-vanishing B-splines */
        N = S_ getr1d (p,& S) ;
        if ( N == NULL ) { error = 1; goto EXIT; }
        error = N_allbas(knt, p, u, LEFT, N,&j);
        if( error == 1 ) goto EXIT;
        /* Compute the point on the curve */
        A_initcp(CZERO, &Cw);
        for ( i = 0; i <= p; i++ )
```

```
        {
            k = j − p + i ;
            A_updcpt(N[i], Pw[k],&Cw);
        }
        A_euclid(Cw, C);
        /* End NURBS and exit */
        EXIT:
        N_enurbs(&S);
        return(error);
    }
```

As mentioned earlier, each NURBS routine starts with N_inurbs( ), which initializes the memory stacks to NULL. After initialization, local notations are normally introduced to help follow the computational details. The notation Pw [k] is visually more pleasing than cur− > pol− > Pw [k] and much closer to the customary notation, $\mathbf{P}_k^w$. The next step in almost every NURBS routine is to check the incoming parameters for possible error. E_parval( ) checks if the given parameter u is outside of the range $[u_0, u_m]$. The rest of the code deals with the 曲线点的实际计算。为此，首先计算非消失基函数，然后分配适量的内存以存储它们 (由于没有度的限制，内存必须动态分配)。然后使用前一节讨论的算术例程来计算和

$$\mathbf{C}^w(u) = \sum_{k=j-p}^{j} N_{k,p}(u)\,\mathbf{P}_k^w$$

并确定该点在欧几里得空间中的位置；注意索引变换以使用适当的基函数。最后，N_enurbs( ) 释放所有分配的内存，并通过返回零 (无错误) 或一 (错误保存在 NERR 中) 将控制权返回给调用例程。
这个下一个测试程序展示了计算曲线上的一个点是多么简单。

```
#include ”nurbs.h”
#include ”globals.h”
REAL wx [6] = {0.2, 0.8, 0.4, 1.2, 1.2, 0.8} ;
REAL wy[6] = {0.2, 0.4, 0.4, 0.8, 0.4, 0.2} ;
REAL wz [6] = {0.2, 0.4, 0.2, 0.4, 0.4, 0.2} ;
REAL w[6] = { 1.0,2.0,1.0,2.0,2.0,1.0 };
REAL U[10] = {0.0, 0.0, 0.0, 0.0, 0.3, 0.7, 1.0, 1.0, 1.0,
                                1.0 };
main( )
{
    FLAG error;
    CURVE CUR;
    PARAMETER u;
    POINT C;
    STACKS S;
    /* Start processing */
    N_inurbs(&S);
    /* Create the curve */
    U_makcu3(&cur, wx, wy, wz, w,5,3, U,9,&S);
    /* Compute a point on the curve */
    printf( ”Enter parameter u = ” ); scanf( ”%lf”, &u );
    error = N_evncur(&cur, u, &C);
    if ( error > 0 )
    {
        printf( ”error = %d %s”, NERR.eno, NERR.fna );
        goto EXIT;
    }
    printf( ”C = %lf %lf %lf”, C.x, C.y, C.z );
    /* End NURBS session */
    EXIT:
    N_enurbs(&S);
}
```

许多实用程序例程允许程序员以多种不同方式定义曲线，例如，读取曲线文件，从曲线草图接口接收 (wx, wy, wz, w) 坐标，等等。
表面程序的结构与曲线程序非常相似。这个下一个表面评估器展示了一个例子。#include ”nurbs.h” static STRING rname = ”N_EVNSUR”;

```
extern CPOINT CZERO;
FLAG N_evnsur( SURFACE *sur, PARAMETER u, PARAMETER v,
POINT *S )
{
FLAG error = 0 ;
INDEX i, j, k, l, ju, jv;
DEGREE p, q ;
REAL *NU, *NV;
KNOTVECTOR *knu, *knv;
CPOINT **Pw, *Tw, Sw;
STACKS ST;
/* 开始 NURBS 环境 */
N_inurbs(&ST);
/* 获取局部符号 */
Pw = sur->net->Pw;
p = sur->p;
q = sur − > q ;
knu = sur->knu;
knv = sur− > knv ;
/* 检查参数 */
error = E_parval(knu, u, rname);
if ( error == 1 ) goto EXIT;
error = E_parval (knv, v, rname);
        if ( error == 1 ) goto EXIT;
        /* Compute nonvanishing B-splines */
        NU = S_getr1d(p,&ST);
        if ( NU == NULL ) { error = 1; goto EXIT; }
        NV = S_getr1d(q,&ST);
        if ( NV == NULL ) { error = 1; goto EXIT; }
        error = N_allbas(knu, p, u, LEFT, NU,&ju);
        if ( error == 1 ) goto EXIT;
        error = N_allbas(knv, q, v, LEFT, NV,&jv);
        if ( error == 1 ) goto EXIT;
        /* Compute the point on the surface * /
        Tw = S_getc1d(p,&ST);
        if ( Tw == NULL ) { error = 1; goto EXIT; }
        for ( i=0; i<=p; i++ )
        {
            A_initcp(CZERO,&Tw[i]);
            k = ju-p+i;
            for ( j=0; j<=q; j++ )
            {
                l = jv-q+j;
                A_updcpt(NV[j], Pw[k][l],&Tw[i]);
            }
        }
        A_initcp(CZERO, &Sw);
        for ( i=0; i<=p; i++ )
        {
            A_updcpt(NU[i], Tw[i],&Sw);
        }
        A_euclid(Sw, S);
        /* End NURBS and exit */
        EXIT:
        N_enurbs(&ST);
        return(error);
    }
```

The next test program shows another way to create a surface and to evaluate a point on it. The routine U_inpsur( ) reads in a data file, named SURFACE.DAT, containing the control polygon and knot vector information. U_inisur( ) is a routine that initializes the surface object to the NULL object to let U_inpsur( ) know that memory allocation is required.

```
#include "nurbs.h"
#include "globals.h"
main( )
{
    FLAG error;
    SURFACE SUR;
    PARAMETER u, v;
    POINT S;
    STACKS ST;
    /* Start processing */
    N_inurbs(&ST);
    /* Read in surface data */
    U_inisur(&sur);
    error = U_inpsur(&sur,"SURFACE.DAT",&ST);
    if ( error > 0 )
    {
        printf( "error = %d %s", NERR.eno, NERR.fna );
        goto EXIT;
    }
    /* Compute surface point */
    printf("Enter parameters (u, v) = " ); scanf("%lf%lf",&u,&v);
    error = N_evnsur(&sur, u, v,&S);
    if ( error > 0 )
    {
        printf( "error = %d %s", NERR.eno, NERR.fna );
      goto EXIT;
    }
    printf( "S = %lf %lf %lf", S.x, S.y, S.z );
    /* End NURBS session */
    EXIT:
    N_enurbs(&ST);
}
```

9 额外结构

为了使 NURBS 库能够用于多种目的，需要一些额外的结构 (对象)。这些结构主要用于将 NURBS 形式转换为其他形式，重新参数化 NURBS 实体等。

对 NURBS 曲线和曲面对象的简单修改会产生标量值的一元和二元 B 样条函数。唯一需要更改的是用定义为曲线值和曲面值的控制多边形和控制网对象进行替换:

```
typedef struct cvalue
{
    INDEX n;
    REAL *fu;
} CVALUE;
typedef struct svalue
{
    INDEX n,
          m;
    REAL **fuv;
} SVALUE;
```

可以编写特殊例程，例如函数评估和节点插入，处理主要用于函数组合的函数 (见第 6.4 节)。

向量值多项式在数据交换和形式转换中发挥着重要作用。Nlib V2.0 具有曲线和曲面多项式实体，以能够从 NURBS 形式转换为分段多项式形式，并反向转换。这些定义为:

```
typedef struct cpoly
{
    CVECTOR *CVE ;
    DEGREE p;
    INTERVAL I;
} CPOLY;
typedef struct spoly
{
    SVECTOR *SVE ;
    DEGREE p,
           q;
    RECTANGLE R;
} SPOLY;
```

其中曲线向量 CVECTOR 和曲面向量 SVECTOR 实体的定义与控制多边形和控制网对象相同。区间和矩形对象则简单地

```
typedef struct interval
{
    PARAMETER ul,
              ur;
} INTERVAL;
typedef struct rectangle
{
    PARAMETER ul,
              ur,
              vb,
              vt,
} RECTANGLE;
```

也就是说，多项式由其向量系数以及多项式所限制的一维和二维区间/矩形定义。

Bézier 曲线和曲面在任何基于 NURBS 的系统中发挥着重要作用，并不是因为它们是 NURBS 的特例，而是因为许多操作可以在单个 Bézier 组件上非常高效地执行。因此，引入了两个特殊结构，一个用于曲线，一个用于曲面。

```
typedef struct bcurve
{
    CPOLYGON *pol;
    INTERVAL I;
} BCURVE;
typedef struct bsurface
{
    CNET *net;
    RECTANGLE R;
} BSURFACE;
```

因此，控制多边形/网和参数区间/矩形唯一地定义了 Bézier 曲线/曲面。

最后，我们介绍一些主要用于简单几何计算和某些数学例程的附加对象。Nlib V2.0 拥有广泛的几何和数学库，用于支持各种 NURBS 操作。首先，我们考虑一些几何对象。

```
typedef struct vector
{
    REAL x,
         y,
         z;
} VECTOR;
typedef struct line
{
    POINT P;
    VECTOR V;
    FLAG bounded;
} LINE;
```



```
typedef struct plane
{
    POINT P;
    VECTOR N;
} PLANE;
typedef struct circle
```

```
{
    POINT C;
    REAL r;
    VECTOR N;
} CIRCLE;
```

尽管 POINT 和 VECTOR 结构包含相同的结构元素，但它们在概念上是不同的，因此需要分别引入。最后一个例子是表示实矩阵的矩阵对象。

```
typedef struct rmatrix
{
    INDEX n,
          m;
    REAL **RM;
    MATRIXTYPE mtp;
    INDEX bandwidth;
} RMATRIX;
```

其中 MATRIXTYPE 定义如下:

```
typedef enum
{
    full,
    lowerleft,
    banded
} MATRIXTYPE;
```

对于整数 (IMATRIX)、点 (PMATRIX) 和控制点 (CMATRIX) 矩阵，提供了类似的结构。行向量和列向量分别被视为 $(1\times\ m)$ 和 $(n\times 1)$ 矩阵。没有引入特殊的向量结构。

10 系统结构

Nlib V2.0 具有以下例程作为复杂结构的一部分:

- NURBS 例程 (N_*.c)；
- Bézier 例程 (B_*.c)；
- 几何例程 (G_ * .c) ；
- 数学例程 (M_ * .c) ；
- 实用程序例程 (U_ * .c) ;
- 内存分配例程 (S_ * .c) ;
- 错误处理例程 (E_ * .c) ;
- 算术例程 (A_ * .c) .

这些例程必须集成到一个复杂系统中，以满足前面提到的所有要求。各模块之间的依赖关系如图 13.9 所示。不同的层代表各个模块，箭头表示它们对下层例程的依赖关系，即它们调用这些例程。在中心有三个模块: 内存分配、错误处理和算术例程。虽然这些模块在某种程度上是相互依赖的，例如，错误例程可能需要一些内存，而内存例程必须报告错误，但它们之间的依赖关系并不像上层模块那样显著。随着层级的提升，例程变得依赖于所有下层程序。例如，数学例程需要实用程序来创建矩阵；它们需要内存、错误检查和特殊算术来执行矩阵运算，其中矩阵的元素是控制点。在顶部是 NURBS 程序，它们调用系统中的其他所有程序。例如，几何例程很少需要调用现有的 NURBS 程序。如果确实发生这种情况，可以编写一个特殊的低层例程，以保持各层之间的单向对应关系。如果系统的某些部分要用于除 NURBS 几何以外的其他目的，这种不同层次的隔离变得十分重要。例如，数学和几何库应该是独立的库，以便它们可以用于简单的几何和数学计算，而不必担心更大的 Bézier 和 NURBS 库。

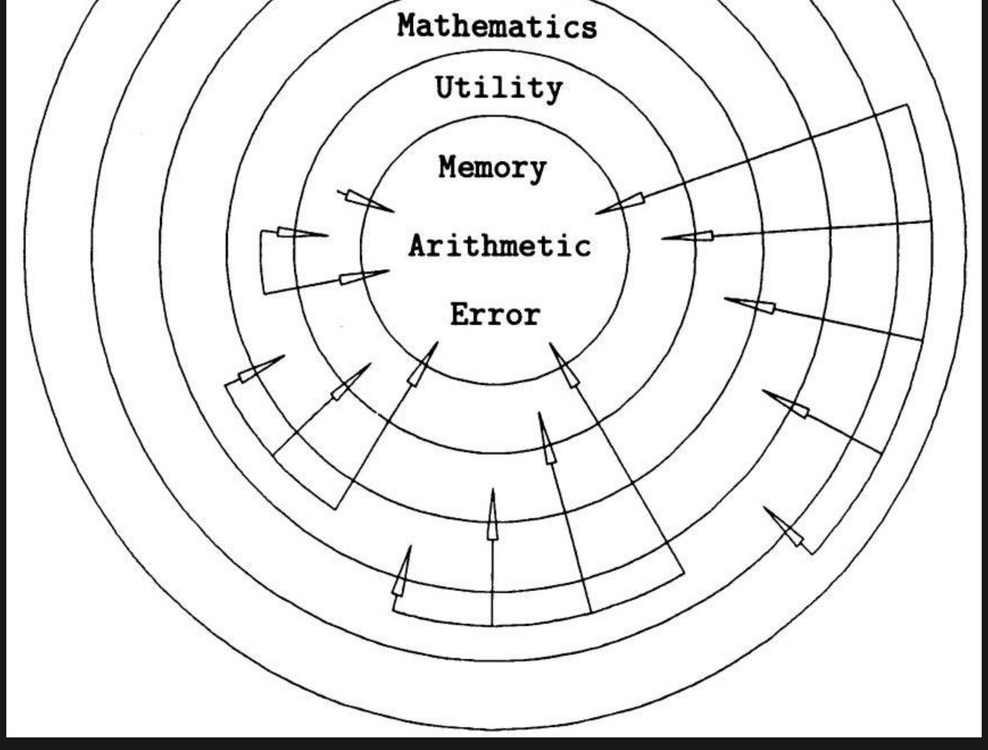


图 13.9. Nlib V2.0 的系统结构。

我们通过重申对系统设计的看法来结束这一部分: 这非常困难，它不是唯一的，并且随着计算机技术的进步而不断变化。然而，设计和实现一个系统是启发性的，并且在获取广泛而深入的 NURBS 技术知识方面至关重要。正如 Herbert B. Voelcker 所说 [Pieg93]

“进行理论研究和实验系统构建都很重要。它们是协同的，单纯追求其中任何一个都可能导致无效的理论或古怪、不透明的系统。”

本书旨在提供 NURBS 的理论和实现。让读者判断这一目标是否成功实现。

第十四章　参考文献

[Ahuj68] Ahuja, D.V., 和 Coons, S.A., 建筑和展示的几何，IBM 系统杂志，第 3-4 期，188-205 页，1968 年。

[Akim70] Akima, H., 基于局部程序的新插值和光滑曲线拟合方法，ACM 杂志，第 17 卷，589-602 页，1970 年。

[Akma92] Akman, V., 和 Arslan, A., 在拓扑图画书中使用所有图形元素进行扫掠，计算机与图形，第 16 卷，第 3 期，273-281 页，1992 年。

[Ball74] Ball, A., CONSURF. 第 1 部分: 锥形放样平铺的介绍，CAD，第 6 卷，第 4 期，243-249 页，1974 年。

[Ball75] Ball, A., CONSURF. 第 2 部分: 算法描述，CAD，第 7 卷，第 4 期，237-242 页，1975 年。

[Ball77] Ball, A., CONSURF. 第 3 部分: 程序的使用，CAD，第 9 卷，第 1 期，页码 9-12，1977 年。

[Baly94] Ballay, H. 和 Storn, R., 一种检查 C 语言编码规范的工具，C/C++ 用户期刊，第 12 卷，第 27 期，页码 41-50，1994 年。

[Barn93] Barnhill, R., Coons' 补丁和凸组合，见 [Pieg93]。

[Barr83] Barr, A.H., 固体原始体的全局和局部变形，SIGGRAPH 83 教程笔记，底特律，1983 年。

[Bars89] Barsky, B.A. 和 DeRose, T.D., 参数曲线的几何连续性: 三种等价表征，IEEE 计算机图形与应用，第 9 卷，第 6 期，页码 60-68，1989 年。

[Bars90] Barsky, B.A. 和 DeRose, T.D., 参数曲线的几何连续性: 几何连续样条的构造，IEEE 计算机图形与应用，第 10 卷，第 1 期，页码 60-68，1990 年。

[Bart87] Bartels, R.H., Beatty, J.C. 和 Barsky, B.A., 计算机图形学和几何建模中样条的介绍，加利福尼亚州圣马特奥: 摩根 • 考夫曼，1987 年。

[Beac91] Beach, R.C., 计算机辅助设计的曲线和曲面的介绍，纽约: 范诺斯特兰德 • 雷恩霍尔德，1991 年。

[Bern12] Bernstein, S.N.，基于概率计算的维尔斯特拉斯定理证明，Commun. Soc. Math. Khrakow，第 12 卷，第 2 期，页码 $1-2, 1912$ 。

[Bezi72] Bézier, P.E.，《数值控制: 数学与应用》，纽约: 约翰 • 威利，1972 年。

[Bezi86] Bézier, P.E.，《UNISURF CAD 系统的数学基础》，伦敦: 巴特沃斯，1986 年。

[Blom91] Bloomenthal, M. 和 Riesenfeld, R.F.，通过张量积 NURBS 近似扫掠曲面，计算机视觉与图形中的曲线与表面 II，SPIE 会议论文集 (光学仪器工程师学会)，第 1610 卷，页码 132-144，1991 年。

[Blor91] Bloor, M. 和 Owen, J.，CAD/CAM 产品数据交换: 下一步，CAD，第 23 卷，第 4 期，页码 237-243，1991 年。

[Boeh80] Boehm, W.，在 B 样条曲线中插入新节点，CAD，第 12 卷，第 4 期，页码 199-201，1980 年。

[Boeh84] Boehm, W.，Farin, G. 和 Kahmann, J.，CAGD 中的曲线和表面方法综述，计算机辅助几何设计，第 1 卷，第 1 期，页码 1-60，1984 年。

[Boeh85a] Boehm, W. 和 Prautzsch, H.，插入算法，CAD，第 17 卷，第 2 期，页码 58-59，1985 年。

[Boeh85b] Boehm, W., 关于结点插入算法的效率, 计算机辅助几何设计, 第 2 卷，第 1-3 期，页码 141-143, 1985 年.

[Boeh91] Boehm, W., 和 Hansford, D., 二次曲面上的 Bézier 片段, 收录于《曲线和曲面设计中的 NURBS》，Farin, G., 编辑，费城: SIAM, 页码 1-14, 1991 年.

[Boeh94] Boehm, W., 和 Prautzsch, H., 几何设计的几何概念, Wellesley, MA: A.K. Peters, 1994 年.

[Bron92] Bronsvoort, W., 和 Waarts, J., 一种将广义圆柱的表面转换为 B 样条表面的方法, 计算与图形, 第 16 卷，第 2 期，页码 175-178, 1992 年.

[Butt76] Butterfield, K.R., B 样条基的所有导数的计算, 数学应用研究所期刊, 第 17 卷，页码 15-25, 1976 年.

[Chan81] Chang, G., 和 Wu, J., Bézier 技术的数学基础, CAD, 第 13 卷，第 3 期，页码 133-136, 1981 年.

[Choi90] Choi, B.K., 和 Lee, C., 通过坐标变换和混合进行扫掠曲面建模, CAD, 第 22 卷，第 2 期，页码 87-96, 1990 年.

[Choi91] Choi, B.K., CAD/CAM 的表面建模, 纽约: Elsevier, 1991 年.

[Chou92] Chou, J., 和 Piegl, L., 使用三次有理 B-spline 进行数据缩减, IEEE 计算机图形与应用, 第 12 卷, 第 3 期, 第 60-68 页, 1992 年.

[Chou95] Chou, J.J., 高阶 Bézier 圆, CAD, 即将出版, 1995 年.

[Cobb84] Cobb, E.S., " 使用 B-spline 表示法设计雕塑表面," 博士学位论文, 犹他大学, 1984 年.

[Cohe80] Cohen, E., Lyche, T., 和 Riesenfeld, R.F., 离散 B-spline 和细分技术在计算机辅助几何设计和计算机图形学中的应用, 计算机图形与图像处理, 第 14 卷, 第 87-111 页, 1980 年.

[Cohe85] Cohen, E., Lyche, T., 和 Schumaker, L.L., 提高样条曲线次数的算法, ACM TOG, 第 4 卷, 第 3 期, 第 171-181 页, 1985 年.

[Coon67] Coons, S.A., 用于计算机辅助设计空间形状的表面, MAC-TR-41, 麻省理工学院, 1967 年 6 月.

[Coqu87a] Coquillart, S., 一种基于控制点的扫掠技术, IEEE 计算机图形与应用, 第 7 卷, 第 11 期, 第 36-45 页, 1987 年.

[Coqu87b] Coquillart, S., 计算 B-spline 曲线的偏移, CAD, 第 19 卷, 第 6 期, 第 305-309 页, 1987 年.

[Coqu90] Coquillart, S., 扩展自由形变: 一种用于三维几何建模的雕刻工具, Comput. Graph., 第 24 卷, 第 4 期, 页 187-193, 1990 年.

[Coqu91] Coquillart, S., 和 Jancene, P., 动画自由形变: 一种交互式动画技术, Comput. Graph., 第 25 卷, 第 4 期, 页 23-26, 1991 年.

[Cox72] Cox, M.G., B 样条的数值评估, Jour. Inst. Math. Ap-plic., 第 10 卷, 页 134-149, 1972 年.

[Coxe67] Coxeter, H.S.M., 和 Greitzer, S.L., 几何学再探, 华盛顿特区: 美国数学协会, 1967 年.

[Coxe74] Coxeter, H.S.M., 投影几何, 加拿大多伦多: 多伦多大学出版社, 1974 年.

[Coxe80] Coxeter, H.S.M., 几何学导论, 纽约: 约翰 • 威利出版社, 1980 年.

[Curr47] Curry, H.B., 和 Schoenberg, I.J., 关于样条分布及其极限: 波利亚分布函数, 摘要 380t, 美国数学学会公报, 第 53 卷, 页 109, 1947 年.

[Dani89] Daniel, M., 和 Daubisse, J.C., 在幂基中使用贝塞尔曲线和曲面的数值问题, 计算辅助几何设计, 第 6 卷, 页 121-128, 1989 年.

[Dann85] Dannenberg, L., 和 Nowacki, H., 使用多项式基进行表面表示的近似转换, 计算机辅助几何设计, 第 2 卷, 第 123-132 页, 1985 年.

[DeBo72] De Boor, C., 关于使用 B-spline 进行计算, 近似理论杂志, 第 6 卷, 第 50-62 页, 1972 年.

[DeBo78] De Boor, C., B-spline 实用指南, 纽约: Springer-Verlag, 1978 年.

[DeBo87] De Boor, C., 切角总是有效, 计算机辅助几何设计, 第 4 卷, 第 1-2 期, 第 125-131 页, 1987 年.

[DeBo93] De Boor, C., B(asic)-spline 基础, 收录于 [Pieg93].

[deCa86] de Casteljau, P., 形状数学与计算机辅助设计, 伦敦: Kogan Page, 1986 年.

[deCa93] de Casteljau, P., 用于曲线和曲面建模的极坐标形式，应用于雪铁龙, 收录于 [Pieg93].

[deMo84] de Montaudouin, Y., 和 Tiller, W., 计算机辅助几何设计中的 Cayley 方法, 计算机辅助几何设计, 第 1 卷, 第 4 期, 第 309-326 页, 1984 年.

[DoCa76] Do Carmo, M.P., 曲线和曲面的微分几何, Englewood Cliffs, NJ: Prentice-Hall, 1976 年.

[Eck93] Eck, M., Bézier 曲线的度数降低, 计算机辅助几何设计, 第 10 卷, 第 237-251 页, 1993 年.

[Fari83] Farin, G.E., 有理 Bézier 曲线的算法, CAD, 第 15 卷, 第 2 期, 第 73-77 页, 1983 年.

[Fari89] Farin, G.E., 有理曲线和曲面, 收录于《计算机辅助几何设计中的数学方面》, Lyche, T. 和 Schumaker, L.L. 主编, 纽约: 学术出版社, 1989 年.

[Fari93] Farin, G.E., 计算机辅助几何设计的曲线与曲面 - 实用指南, 第 3 版, 波士顿: 学术出版社, 1993 年.

[Faro86] Farouki, R., 非退化偏移曲面的逼近, 计算机辅助几何设计, 第 3 卷, 第 15-43 页, 1986 年.

[Faro87] Farouki, R.T., 和 Rajan, V.T., 关于伯恩斯坦形式多项式的数值条件, 计算机辅助几何设计, 第 4 卷, 第 191-216 页, 1987 年.

[Faro88] Farouki, R.T., 和 Rajan, V.T., 伯恩斯坦形式多项式的算法, 计算机辅助几何设计, 第 5 卷, 第 1-26 页, 1988 年.

[Faux81] Faux, I.D., 和 Pratt, M.J., 设计与制造的计算几何, 英国奇切斯特: Ellis Horwood Ltd., 1981 年.

[Ferg66] Ferguson, J.C., 在特定参数曲线类中表征的形状, 报告 3122-31, TRW 公司, 加利福尼亚州雷东多海滩, 1966 年.

[Ferg67] Ferguson, J.C., 以特殊类别的参数曲线为特征的形式 - II，报告 3122-3-237，TRW 公司，加州雷东多海滩，1967 年。

[Ferg69] Ferguson, J.C. 和 Miller, K.L., 在一类三次代数曲线中对形状的表征，报告 5322-3-5，TRW 公司，加州雷东多海滩，1969 年。

[Ferg93] Ferguson, J.C., 自由形状曲线和超曲面的 F 方法定义，见 [Pieg93]。

[Fili89] Filip, D. 和 Ball, T., 程序化表示拾升表面，IEEE 计算机图形与应用，第 9 卷，第 6 期，页 27-33，1989 年。

[Fole90] Foley, J., van Dam, A., Feiner, S. 和 Hughes, J., 计算机图形学: 原理与实践，马萨诸塞州雷丁: 阿迪森-韦斯利，1990 年。

[Forr68] Forrest, A.R.，“计算机辅助设计的曲线与表面，”博士论文，剑桥大学，英国剑桥，1968 年。

[Forr70] Forrest, A.R., 非理性扭曲立方曲线的形状分类，基于 Bézier 多边形，CAD 组文件编号 52，剑桥大学，英国剑桥，1970 年。

[Forr72] Forrest, A.R., 通过 Bézier 多项式进行交互式插值和逼近，计算机期刊，第 15 卷，第 1 期，页 71-79，1972 年。更正和更新版本见 *CAD* ，第 22 卷，第 9 期，页 527-537，1990 年。

[Forr80] Forrest, A.R., 扭曲立方曲线: 一种计算机辅助几何设计方法, CAD, 第 12 卷，第 2 期，页 165-172, 1980.

[Fors88] Forsey, D., 和 Bartels, R., 层次 B 样条细化, Comput. Graph., 第 22 卷，第 4 期，页 205-212, 1988.

[Four83] Fournier, A., 和 Wesley, M., 弯曲多面体对象, CAD, 第 15 卷，第 2 期，页 79-87, 1983.

[Fowl92] Fowler, B., 张量积曲面的几何操作, Comput. Graph. 特刊, 交互式 3D 图形研讨会, 页 101-108, 1992.

[Fowl93] Fowler, B., 和 Bartels, R., 基于约束的曲线操作, IEEE Comput. Graph. and Appl., 第 13 卷，第 5 期，页 43-49, 1993.

[Fuhr92] Fuhr, R.D., 和 Kallay, M., 单调线性有理样条插值, Comput. Aid. Geom. Des., 第 9 卷，第 4 期，页 313-319, 1992.

[Geis90] Geise, G., 和 Langbecker, U., 具有四条圆锥边界曲线的有限二次段, Comput. Aid. Geom. Des., 第 7 卷，页 141-150, 1990.

[Good81] Goodman, T., 通过 Hermite-Birkhoff 样条进行 Hermite-Birkhoff 插值, Proc. Roy. Soc. Edinburgh, 第 88 卷 (A)，第 3/4 部分，页 195-201, 1981.

[Gord69] Gordon, W., 通过曲线网络进行样条混合表面插值, 数学与力学杂志, 第 18 卷，第 10 期，第 931-952 页, 1969 年.

[Gord71] Gordon, W., 二元和多元插值与逼近的混合函数方法, SIAM 数值分析杂志, 第 8 卷，第 158-177 页, 1971 年.

[Gord74a] Gordon, W.J., 和 Riesenfeld, R.F., 用于计算机辅助设计自由形状曲线和表面的伯恩斯坦-贝塞尔方法, 计算机协会杂志, 第 21 卷，第 2 期，第 293-310 页, 1974 年.

[Gord74b] Gordon, W.J., 和 Riesenfeld, R.F., B 样条曲线和表面, 收录于《计算机辅助几何设计》，Barnhill, R.E., 和 Riesenfeld, R.F., 编辑, 纽约: 学术出版社, 1974 年.

[Gord93] Gordon, W., 通过混合函数方法定义雕刻表面, 收录于 [Pieg93].

[Gugg89] Guggenheimer, H., 沿轨迹计算框架, 计算机辅助几何设计, 第 6 卷，第 77-78 页, 1989 年.

[Hoff89] Hoffmann, C.M., 几何与实体建模, 加州圣马特奥: 摩根考夫曼, 1989 年.

[Hohm91] Hohmeyer, M. 和 Barsky, B., 皮肤化有理 B 样条曲线以构建插值表面, 计算机视觉、图形与图像处理: 图形模型与图像处理, 第 53 卷，第 6 期，第 511-521 页, 1991 年.

[Hosc88] Hoschek, J., 偏移曲线的样条逼近, Comput. Aid. Geom. Des., 第 5 卷，第 33-40 页, 1988.

[Hosc92a] Hoschek, J., 二次曲面上的贝塞尔曲线和曲面片, 收录于《计算机辅助几何设计中的数学方法 II》，Lyche, T. 和 Schumaker, L. 编辑, 纽约: 学术出版社, 第 331-342 页, 1992.

[Hosc92b] Hoschek, J., 圆形样条, CAD, 第 24 卷，第 11 期，第 611-618 页, 1992.

[Hosc93] Hoschek, J. 和 Lasser, D., 计算机辅助几何设计基础, Wellesley, MA: A.K. Peters, Ltd., 1993.

[Howa91a] Howard, T., 评估 PHIGS 在 CAD 和一般图形应用中的表现, CAD, 第 23 卷，第 4 期，第 244-251 页, 1991.

[Howa91b] Howard, T.L.J., Hewitt, W.T., Hubbold, R.J., 和 Wyrwas, K.M., PHIGS 和 PHIGS PLUS 的实用介绍, Reading, MA: Addison-Wesley, 1991.

[Hsu92] Hsu, W., Hughes, J., 和 Kaufman, H., 自由形状变形的直接操作, Comput. Graph., 第 26 卷，第 2 期，第 177-184 页, 1992.

[IGE93] 初始图形交换规范 (IGES) 第 5.2 版, ANSI Y14.26M, 可从美国产品数据协会 (US PRO) 获取, Fairfax, VA, USA, 1993.

[Ilyi84] Ilyin, V.A., 和 Poznyak, E.G., 《解析几何》，莫斯科:Mir Publishers，1984 年。

[Kapl52] Kaplan, W.，《高级微积分》，马萨诸塞州雷丁:Addison Wesley，1952 年。

[Klok86] Klok, F., 两个移动坐标框架用于沿 3D 轨迹的扫掠，计算机辅助几何设计，卷 3，第 217-229 页，1986 年。

[Knut73] Knuth, D.E.，《计算机程序设计的艺术》。第 1 卷，基础算法，马萨诸塞州雷丁:Addison-Wesley，1973 年。

[Lach88] Lachance, M.A., 参数曲面的切比雪夫经济化，计算机辅助几何设计，卷 5，第 195-208 页，1988 年。

[Lanc86] Lancaster, P., 和 Salkauskas, K.，《曲线和曲面拟合》，纽约: 学术出版社，1986 年。

[Lane80] Lane, J.M., 和 Riesenfeld, R.F., 一种用于计算机生成和显示分段多项式曲面的理论发展，IEEE 模式分析与机器智能杂志，卷 PAMI-2，第 1 期，第 35-46 页，1980 年。

[Lane83] Lane, J.M., 和 Riesenfeld, R.F., B 样条逼近的变差减小性质的几何证明，逼近理论杂志，卷 37，第 1-4 页，1983 年。

[Laur93] Laurent-Gengoux, P., 和 Mekhilef, M., NURBS 表示的优化，计算机辅助设计，卷 25，第 11 期，第 699-710 页，1993 年。

[Lawr72] Lawrence, J.D., 特殊平面曲线目录, 纽约: Dover, 1972.

[Lee83] Lee, E.T.Y., B-spline 入门, 波音文档, 1983.

[Lee87] Lee, E.T.Y., 用于圆锥曲线的有理二次 Bézier 表示, 收录于《几何建模: 算法与新趋势》, Farin, G.E. 编辑, 费城: SIAM, 第 3-19 页, 1987.

[Lee89] Lee, E.T.Y., 在参数曲线插值中选择节点, CAD, 第 21 卷, 第 363-370 页, 1989.

[Lee91] Lee, E.T.Y. 和 Lucian, M.L., 有理 B 样条的 Möbius 重新参数化, 计算机辅助几何设计, 第 8 卷, 第 213-215 页, 1991.

[Limi44] Liming, R.A., 实用解析几何及其在航空器中的应用, 纽约: Macmillan, 1944.

[Limi79] Liming, R.A., 计算机图形学数学, Fallbrook, CA: Aero Publishers Inc., 1979.

[Lin94] Lin, F. 和 Hewitt, W., 将 Coons-Gordon 曲面表示为 NURBS, CAD, 第 26 卷, 第 2 期, 第 145-155 页, 1994.

[Lore86] Lorentz, G.G., Berntein 多项式, 纽约: Chelsea Publishing Co., 1986.

[Lych85] Lyche, T., Cohen, E. 和 Morken, K., 张量积样条的节点线细化算法, 计算机辅助几何设计, 第 2 卷, 第 1-3 期, 第 133-139 页, 1985.

[Lych87] Lyche, T., 和 Morken, K., 参数 B-spline 曲线和曲面的节点去除, Comput. Aid. Geom. Des., 第 4 卷, 第 217-230 页, 1987.

[Lych88] Lyche, T., 和 Morken, K., 一种用于样条的数据减少策略及其在函数和数据近似中的应用, IMA Jour. Num. Anal., 第 8 卷, 第 185-208 页, 1988.

[Mats92] Matsuki, N., 一种用于 B-spline 曲面的交互式形状修改方法, 收录于计算机集成制造中的人类因素, Olling, G., 和 Kimura, F., 编辑, 阿姆斯特丹: Elsevier Science Publishers B.V. (North-Holland) (IFIP), 第 385-397 页, 1992.

[Mort85] Mortenson, M.E., 几何建模, 纽约: John Wiley, 1985.

[Patt85] Patterson, R.R., 伯恩斯坦-贝济耶曲线参数的投影变换, ACM TOG, 第 4 卷, 第 4 期, 第 276-290 页, 1985.

[Pieg86] Piegl, L., 计算机辅助设计中有理贝济耶方案的几何研究, Comput. in Industry, 第 7 卷, 第 401-410 页, 1986.

[Pieg87a] Piegl, L., 和 Tiller, W., 使用有理 B-splines 的曲线和曲面构造, CAD, 第 19 卷, 第 9 期, 第 485-498 页, 1987.

[Pieg87b] Piegl, L., 一种使用圆锥曲线平滑散乱数据的技术, Comput. in Industry, 第 9 卷, 第 223-237 页, 1987.

[Pieg87c] Piegl, L., 关于在计算机辅助几何设计中使用无限控制点, Comput. Aid. Geom. Des., 第 4 卷, 第 155-166 页, 1987 年.

[Pieg87d] Piegl, L., 通过有理 Bézier 曲线进行交互式数据插值, IEEE Comput. Graph. and Appl., 第 7 卷, 第 4 期, 第 45-58 页, 1987 年.

[Pieg88a] Piegl, L., 使用具有无限控制点的有理 Bézier 逼近形式的 Hermite 和 Coons 类插值, CAD, 第 20 卷, 第 1 期, 第 2-10 页, 1988 年.

[Pieg88b] Piegl, L., Coons 类型的补丁, Comput. and Graph., 第 12 卷, 第 2 期, 第 221-228 页, 1988 年.

[Pieg89a] Piegl, L., 计算机辅助几何设计中的关键发展, CAD, 第 21 卷, 第 5 期, 第 262-273 页, 1989 年.

[Pieg89b] Piegl, L., 和 Tiller, W., 有理 B 样条圆的动物园, IEEE Comput. Graph. and Appl., 第 9 卷, 第 5 期, 第 48-56 页, 1989 年.

[Pieg89c] Piegl, L., 修改有理 B 样条的形状. 第 1 部分: 曲线, CAD, 第 21 卷, 第 8 期, 第 509-518 页, 1989 年.

[Pieg89d] Piegl, L., 修改有理 B 样条的形状. 第 2 部分: 表面, CAD, 第 21 卷, 第 9 期, 第 538-546 页, 1989 年.

[Pieg90] Piegl, L., 圆锥样条计算算法, 土木工程计算杂志, 第 4 卷, 第 3 期, 页 180-197, 1990 年.

[Pieg91a] Piegl, L., 关于 NURBS 的调查, IEEE 计算机图形与应用, 第 10 卷, 第 1 期, 页 55-71, 1991 年.

[Pieg91b] Piegl, L., 和 Tiller, W., B 样条曲线的存储高效分解, CSE 91-01, 南佛罗里达大学计算机科学与工程系, 坦帕, FL, 1991 年.

[Pieg93] Piegl, L., 编辑, 计算机辅助几何建模的基础发展, 伦敦: 学术出版社, 1993 年.

[Pieg94] Piegl, L., 和 Tiller, W., B 样条曲线的度提升的软件工程方法, CAD, 第 26 卷, 第 1 期, 页 17-28, 1994 年.

[Pieg95] Piegl, L., 和 Tiller, W., B 样条曲线的度降低算法, CAD, 第 27 卷, 第 2 期, 1995 年.

[Prau84] Prautzsch, H., B 样条曲线的度提升, 计算机辅助几何设计, 第 1 卷, 第 1 期, 页 193-198, 1984 年.

[Prau91] Prautzsch, H., 和 Piper, B., 提高样条曲线度的快速算法, 计算机辅助几何设计, 第 8 卷, 页 253-265, 1991 年.

[Prau92] Prautzsch, H., 和 Gallagher, T., B 样条或贝塞尔曲面是否具有几何变异减小特性?, 计算机辅助几何设计，卷 9，第 2 期，页 119-124，1992 年。

[Pres88] Press, W., Flannery, B., Teukolsky, S., 和 Vetterling, W., 数值计算方法 C ，剑桥，英国: 剑桥大学出版社，1988 年。

[PHIG92] 程序员的分层交互图形系统 (PHIGS)，ISO/IEC 9592-4: (1992)，可从美国国家标准与技术研究院 (NIST) 获取，马里兰州盖瑟斯堡。

[Rams87] Ramshaw, L., 绽放: 一种连接点的样条方法，报告 19，数字系统研究中心，加州帕洛阿尔托，1987 年。

[Renn82] Renner, G., 机械工程实践中的形状描述方法，工业计算，卷 3，页 137-142，1982 年。

[Ries73] Riesenfeld, R.F., “B 样条逼近在计算机辅助设计几何问题中的应用，” 博士论文，锡拉丘兹大学，1973 年。

[Ries81] Riesenfeld, R.F., 计算机图形学中的齐次坐标和射影平面，IEEE 计算机图形与应用，卷 1，第 1 期，页 50-55，1981 年。

[Robe65] Roberts, L.G., 齐次矩阵表示和 n 维构造的操作，技术报告 MS-1405，林肯实验室，麻省理工学院，马萨诸塞州剑桥，1965 年。

[Roge90] Rogers, D.F., 和 Adams, J.A., 计算机图形学的数学元素, 第 2 版, 纽约: McGraw-Hill, 1990.

[Salm79] Salmon, G., 圆锥曲线论, Longman, Green & Co., 第 6 版, 伦敦, 1879, 由 Dover 出版社重印, 纽约.

[Scho46] Schoenberg, I.J., 关于用解析函数逼近等距数据的问题的贡献, Quart. Appl. Math., 第 4 卷, 第 45-99 页, 1946.

[Scho66] Schoenberg, I.J., 关于 Hermite-Birkhoff 插值, 数学分析与应用杂志, 第 16 卷, 第 3 期, 第 538-543 页, 1966.

[Sede84] Sederberg, T., Anderson, D., 和 Goldman, R., 参数曲线和曲面的隐式表示, 计算机视觉、图形与图像处理, 第 28 卷, 第 72-84 页, 1984.

[Sede86] Sederberg, T., 和 Parry, S., 实体几何对象的自由形变, 计算机图形, 第 20 卷, 第 4 期, 第 151-160 页, 1986.

[Silt92] Siltanen, P., 和 Woodward, C., 3D 偏移曲线、扫掠曲面和蒙皮的法向方向方法, 欧洲图形学会议论文集 92, 第 11 卷, 第 3 期, 第 C-449-C-457 页, 1992.

[Smit71] Smith, L.B., 用固定数量的点和最大内切面积绘制椭圆、双曲线或抛物线, 计算机期刊, 第 14 卷，第 1 期，81-86 页，1971 年。

[Smit74] Smith, R., Price, J., 和 Howser, L., 使用三次样条函数的平滑算法, NASA 技术报告, TN D-7397, NASA 兰利研究中心, 亨普顿, VA, 1974 年。

[STEP94] 产品模型数据交换标准 (STEP), ISO 10303, 一系列文档。第 42 部分: 几何和拓扑表示, ISO 10303-42, 可从 ISO 秘书处获取, 国家标准与技术研究所 (NIST), 盖瑟斯堡, MD, 1994 年。

[Ston89] Stone, M.C., 和 DeRose, T.C., 参数立方曲线的几何特征, ACM TOG, 第 8 卷，第 3 期，147-163 页，1989 年。

[Stra92] Straker, D., C 风格标准与指南, 纽约: 普伦蒂斯霍尔, 1992 年。

[Stro91] Strotman, T., 私人通信, 1991 年。638 参考文献

[Su89] Su, B., 和 Liu, D., 计算几何 - 曲线与曲面建模, 波士顿: 学术出版社, 1989 年。

[Till83] Tiller, W., 用于曲线和曲面表示的有理 B 样条, IEEE 计算机图形与应用, 第 3 卷，第 6 期，61-69 页，1983 年。

[Till84] Tiller, W. 和 Hanson, E., 二维轮廓的偏移，IEEE 计算机图形与应用，卷 4，第 9 期，页 36-46，1984 年。

[Till92] Tiller, W., NURBS 曲线和曲面的去结算法，CAD，卷 24，第 8 期，页 445–453，1992 年。

[Vand83] Vandergraft, J., 数值计算导论，纽约: 学术出版社，1983 年。

[Verg91] Vergeest, J., 使用 STEP 的 CAD 表面数据交换，CAD，卷 23，第 4 期，页 269-281，1991 年。

[Vers75] Versprille, K.J., “有理 B 样条逼近形式的计算机辅助设计应用，” 博士论文，锡拉丘兹大学，1975 年。

[Wang81] Wang, C.Y., 参数立方曲线和参数 B 样条立方曲线的形状分类，CAD，卷 13，第 4 期，页 199-206，1981 年。

[Watk88] Watkins, M.A. 和 Worsey, A.J., Bézier 曲线的度数降低，CAD，卷 20，第 7 期，页 398–405，1988 年。

[Wein92] Weinstein, S.E. 和 Xu, Y., 通过逼近和插值降低 Bézier 曲线的度数，见《逼近理论》, Anastassiou, G. A. 编辑，纽约:Dekker，1992 年，页 503-512。

[Welc92] Welch, W. 和 Witkin, A., 变分表面建模，计算机图形，卷 26，第 2 期，页 157-166，1992 年。

[Wood87] Woodward, C., B-spline 表面的横截面设计，计算机与图形，卷 11，第 2 期，页 193-201，1987 年。

[Wood88] Woodward, C., 用于交互式 B-spline 表面插值的皮肤技术，计算机辅助设计，卷 20，第 8 期，页 441–451，1988 年。

[Yama88] Yamaguchi, F., 计算机辅助几何设计中的曲线和表面，纽约: 施普林格出版社，1988 年。

第十五章　索引

仿射变换, 83, 105, 118, 130
算法
Bézier 到幂基的转换
矩阵, 269
双三次混合的 Coons 曲面，
502
计算节点移除误差界限
曲线，428
基于约束的曲面修改，
564
B 样条导数的控制点
曲线，98
曲面，114
从 NURBS 转换到其他
样条形式，590
曲线
近似
全球
到点和导数，417
到点，误差有界，431
局部
误差有界的三次方程，448
误差有界的二次方程，440
全局插值
到点的三次样条，373
到点，369
分解 B 样条
曲线为分段贝塞尔
段, 173
表面分段为分段贝塞尔
补丁, 177
度提升
曲线, 206, 223
表面, 209
有理 B 样条的导数
INDEX
曲线, 127
表面, 137
查找节点跨度, 68
戈登表面, 494
结
插入
曲线, 151
表面, 155
精化
曲线, 164
表面, 167
移除曲线, 185
NURBS
圆, 308
开放圆锥弧, 317
旋转面, 346
球面上的三边补丁，359
点
和 B-spline 的导数
曲线，93，99
表面，111，115
关于 Bézier
曲线，22，24
表面，39
关于 B-spline
曲线，82
表面，103
功率基
曲线, 7
曲面, 36
关于有理 B 样条
曲线, 124, 153
曲面, 134
功率基到 Bézier 转换
矩阵, 275
算法 (续):
二次有理 Bézier 弧, 314
从曲线中移除节点, 错误
有界, 429
表面
对点的全局近似,
422
拟合参数值, 377
插值
对点的全局, 380
对点的局部, 404
扫掠表面
插值轨迹, 475
未插值轨迹, 477
解锁 B-spline 曲线, 577
value
伯恩斯坦多项式的值，20
一个 B 样条基函数的值，74
及其导数，76
值
非零 B 样条基的值
函数，70
及其导数，72
n 次伯恩斯坦
多项式，20，21
逼近，361
重心坐标，528
基函数
B 样条
计算算法，67-78
连续性，57，58
定义和示例，50-55
导数，57-63，78
性质，55-58，63-67
曲线

贝塞尔，10，15-17
幂基，6
有理贝塞尔, 27
有理 B 样条, 118
B 样条的乘积, 102, 104
表面
贝塞尔, 37
幂基, 35
有理贝塞尔, 40
有理 B 样条, 128-130
弯曲, 547
圆形, 550
圆柱形, 551
抛物线, 550
伯恩斯坦多项式, 10, 52, 104
贝塞尔
与幂基形式的比较,
24,25
曲线, 9, 382
定义, 9
次数提升, 202-205
次数降低, 214, 219-221
导数, 22
性质, 12-14
表面
定义, 37
属性, 38
混合函数 (另见基函数)
函数), 6
突变点, 47, 51, 588
B 样条
基函数 (另见基函数)
函数), 50
曲线 (另见曲线, B 样条), 81
编程 (另见软件), 593
表面 (另见表面, B 样条),
100
圆, 1, 26, 28, 32, 45, 244, 254, 279,
295,296
构造, 298-309
高阶有理贝塞尔,
326 – 330
非夹紧, 578
夹紧
曲线, 576
节点, 571
复合 B 样条曲线, 181, 192
圆锥
构造, 310-320
离心率, 281
形式
转换与类型
分类，320-325
隐式方程，282，320
标准位置，283
最大内切面积，289
320
NURBS，321
有理贝塞尔，291，321
圆锥，形式 (续):
有理幂基，288，321
形状因子 (另见形状因子)
261
肩点，294，394，439
基于约束的修改
曲线，555
表面，562
约束，361
连续性
在蒙皮中，469
B 样条
基函数，57，58
曲线，88
表面，109
闭合 B-spline 曲线, 575
分段多项式和有理
曲线, 47
的阶数, 47, 588
参数 (C) 与几何 (G),
382,588
控制网
Bézier, 37
数据结构, 600
控制点
Bézier, 10
B-spline, 81
数据结构, 597
无限 (另见无限控制
点), 295
多重, 90
有理 B 样条, 117
重新定位, 84, 108, 511, 516, 533,
555,562
最小长度解, 557
加权, 30, 120
控制多边形
贝塞尔, 11
B 样条, 82
数据结构, 598
有理 B-样条, 117
转换
Bézier 到/从幂基
多项式, 265, 272
B-样条到/从分段幂
基多项式, 180, 265-278
圆锥曲线, 320
凸包性质, 229
Bézier, 12, 38
索引 643
B-spline, 83, 105
有理 B-spline, 118, 130
Coons 曲面
双立方混合, 497
双线性混合, 496

交叉比, 524, 547
跨边界导数, 497
曲线
Bézier, 9, 82, 119
次数提升, 202-205
次数降低, 214, 219-221
B-样条
定义和性质, 81-91
导数和连续性, 88, 180
夹紧, 576
三次, 8, 12
数据结构, 596, 599
的导数, 2, 6
点和导数的评估
通过节点插入, 151
拟合
近似, 410, 413, 427, 438,
441
插值，364，370，371，373，
388,392,395
隐式形式，1
参数形式，1
分段
贝塞尔，47，48
多项式，47-49，82
有理，47，50
幂基形式，6
有理贝塞尔
定义，27
齐次形式，30
属性, 27
有理 B-样条
定义和属性, 117-120
导数, 125-127
反转, 263
通过节点的分裂/细分
插入, 151
非夹紧, 576
尖点, 8, 13
圆柱, 40
数据交换, 571, 580, 583, 586
deCasteljau 算法, 24, 32, 38, 145,
314, 396, 446
B-spline 的分解为
分段
Bézier, 169-179
多项式, 162, 168
度
提升, 188-212, 326
降维, 212-227
的导数:
伯恩斯坦多项式, 17
Bézier 曲线, 22
B-样条
基函数, 57-63
曲线, 88, 91-100
关于权重, 520
曲面, 109-115
两个函数的组合, 247
参数化
曲线, 2
曲面, 4
幂基
曲线, 6
表面，36
有理基函数，555
有理 B 样条
曲线，125-127
表面，136-138
椭圆，26，282，290，293
完整构造，320
拟合，选择
节点，365，370，371，374，389，398，
412
参数，364，365，376，386，389，
398
扁平化，542
Frenet 框架, 479
全局拟合算法, 363, 405
戈登曲面, 485
齐次
坐标, 29
表示
圆弧的, 33
NURBS 曲线的, 120
NURBS 曲面的, 132
霍纳法
曲线, 6, 9
表面, 36
双曲线, 26, 282, 290, 293
IGES, 580
参数样条, 270, 277
隐式方程
与参数形式相比, 4, 5
曲线的, 1
表面的, 2
无限控制点, 295, 298, 309, 312
拐点, 8
插值, 361
逆结点插入, 154, 512, 525
532
等参曲线，35
通过节点插入提取，161
在贝塞尔曲面上，37
在 B 样条曲面上，109
在有理 B 样条曲面上，135
$\mathbf{K}_{\text{not}}$
插入，141-161，309，326
精细化，162-179，265，533
移除，179-188，272，427，533
576
节点
夹紧开放 (非周期性)，66，572
夹紧与非夹紧，571

数据结构，598
定义，50，51
合并，162
多重性，57，58，64
非周期性 (夹紧开放)，66，572
控制多边形的划分，
147
周期性与非周期性，574
跨度，51
均匀与非均匀，66，571
最小二乘逼近，410
线，7，10
局部拟合算法，363，382，437
局部支持，49，55，118，128
升高表面 (另见皮肤)
表面), 457
循环, 8, 13
牛顿迭代, 230
节点, B 样条, 512, 516, 530
NURBS 表示
双线性曲面，333
Coons 曲面，496
椭球体，349
椭圆抛物面, 349
一般圆锥, 338
一般圆柱, 334
戈登曲面, 485
双曲抛物面, 334
平面, 334
规则曲面, 337
皮肤曲面, 457
球体, 344
旋转曲面, 340
扫描曲面, 472
摆动表面, 455
球面上的三边补丁, 351
环面, 344
偏移, 475, 479, 552, 588
顺序, 50
抛物线, 7, 11, 26, 282, 290, 293
参数误差, 221
参数形式, 1, 4
与隐式方程的比较
形式, 4, 5
的导数, 2, 4
曲线的，1
表面的，2
表面法向量，4
统一划分，16, 27, 38, 57, 118,
128
周期曲线与非周期曲线，575
透视图，30, 121
PHIGS，585
点
反演 (另见点，投影)，
229
投影
到曲线，229-232
表面，232-234
多项式，6
幂基形式，5，9，24
的导数，6
曲线的，5
表面的，35
编程，B 样条 (另见
软件)，593
索引 645
曲线和表面的投影
平行，236
视角，237
有理
B 样条
基函数 (另见基函数
)，118，534
曲线 (另见曲线，有理
B 样条)，117
曲面 (另见曲面，
有理 B 样条)，128
贝塞尔
曲线定义，27
表面定义, 40
函数, 26
重新参数化, 44, 241-263, 265,
269,271,589
与一个
线性函数, 250, 263
线性有理函数, 255
分段多项式函数, 250
分段有理函数, 251
多项式函数, 245
形状因子, 261, 293
形状不变 (另见形状
因子), 261
形状算子, 533
皮肤表面, 457
平滑函数, 471
软件, 593
算术例程, 616
数据结构, 596
数据类型, 594
错误控制, 607
示例程序, 618
内存分配, 601
系统结构, 626
实用例程, 613
球体, 2
STEP, 290, 583
表面
贝塞尔, 37, 38, 105, 130
B 样条
定义, 100
导数和连续性, 109
属性, 105-109
表面 (续):

数据结构，601
点和导数的评估
通过节点插入，161
拟合
逼近，419，432
插值，376，399
隐式形式，2
等曲线，35，37
法向量，4，36
反转，265
参数形式，2
偏导数，4
幂基形式，35
有理 Bézier，40
有理 B 样条
定义，128
导数，136-138
性质，130
通过节点的分割/细分
插入，158
张量积定义，34，50
扫掠曲面，472
摆动表面, 455
切向量反演, 235
张力, 561
张量积
B 样条基函数, 102, 104 表面, 34
曲线和表面的变换, 236, 240
扭转向量, 500 均匀参数化, 2 非夹紧圆, 578 曲线, 576 结, 571 变差减小性质, 12, 38, 87, 119, 131
分段多项式曲线的向量空间, 47, 48 函数, 63-66
扭曲, 533, 538, 542
权重
关于的微分, 520 的修改, 120, 130, 131, 261, 518, 526, 531
负的, 186, 469
有理贝塞尔, 27
有理 B 样条, 117, 120, 131